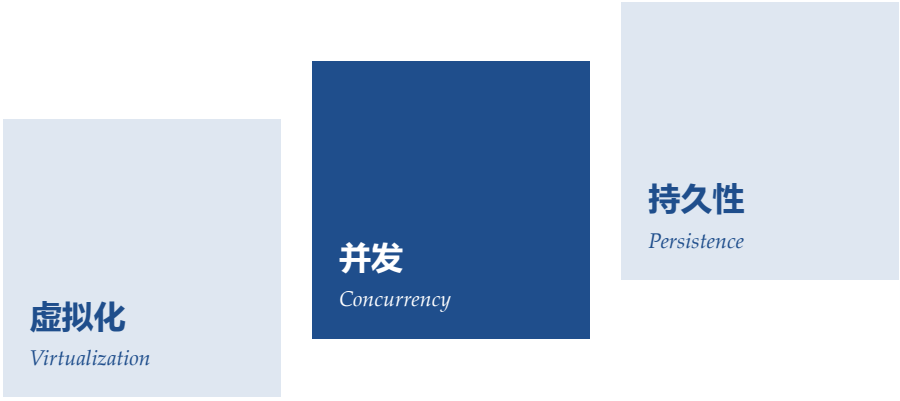
# 操作系统导论

*Operating Systems: Three Easy Pieces*




Remzi H. Arpaci-Dusseau
Andrea C. Arpaci-Dusseau

原著 · Version 1.10

献给 *Vedat S. Arpaci*——我们一生的灵感源泉

© 2008–26, ARPACI-DUSSEAU

# 前言

## 致所有读者

欢迎阅读本书！希望你读这本书时，能像我们写这本书时一样享受。本书名为《**操作系统：三个简单的部分**》（Operating Systems: Three Easy Pieces，可以在 `http://ostep.org` 上找到），书名显然是在致敬一位名叫 Richard Feynman 的人写的物理学讲义 [F96]，那是史上最出色的讲义之一。本书无疑达不到那位著名物理学家树立的高标准，但如果你想弄明白操作系统（以及更广义的系统）究竟是怎么回事，它或许也够用了。

所谓“三个简单的部分”，指的是本书围绕的三大主题：**虚拟化**（virtualization）、**并发**（concurrency）和**持久性**（persistence）。讨论这些概念时，我们最终会讲到操作系统所做的大部分重要工作；希望你在这个过程中也能得到一些乐趣。学习新东西很有趣，对吧？至少（通常）应该是这样。

每个大主题都分成若干章，大多数章先提出一个具体问题，再说明如何解决它。各章篇幅都不长，并且尽量注明各种思想真正的出处。写这本书的目标之一，就是尽量把历史脉络梳理出来，因为我们认为，这能帮学生更清楚地理解事物现在如何、过去如何、将来又会如何。看看香肠是怎么做出来的，几乎和弄明白香肠有什么用一样重要[1]。

书中有几种贯穿始终的手法，值得在这里先介绍一下。第一种是问题的**关键**（crux）。每次要解决一个问题，我们都会先说明其中最要紧的是什么；这样的**关键问题**会在正文中明确标出来，并且（但愿）能用后文介绍的技术、算法和思想来解决。

在很多地方，我们会把系统随时间推移的行为展示出来，借此说明它是怎么工作的。这些**时间线**（timeline）是理解的精髓所在：比如，如果你知道一个进程（process）遇到缺页异常（page fault）时会发生什么，你就已经走在真正理解虚拟内存（virtual memory）如何运作的路上了；如果你明白日志文件系统（journaling file system）把一个块写到磁盘时发生了什么，你就已经朝着精通存储系统迈出了第一步。

正文中还有许多**补充**（aside）和**提示**（tip），给主线内容增添一些色彩。补充通常讨论与正文相关（但也许不是必需）的内容；提示通常是一些普遍适用的经验，可以用在你构建的系统上。为了方便查阅，书末的索引列出了全部提示和补充（还有全部关键问题，crux 的复数形式 cruces 有点古怪）。

[1] 提示：用来吃！要是你吃素，那就是用来躲开。

全书还用到了一种最古老的教学方法：**对话**（dialogue），借它换个角度来呈现部分内容。对话用来引出各大主题（后面你会看到，引出的方式和桃子有关），也不时用来复习学过的内容。对话还让我们有机会用更幽默的笔调来写。至于你觉得它们有没有用、好不好笑，那就完全是另一回事了。

每一大部分的开头，我们会先介绍操作系统提供的一种**抽象**（abstraction），然后在后续各章中讨论提供这种抽象所需的机制、策略和其他支持。抽象是计算机科学方方面面的基础，所以它在操作系统中同样必不可少，也就不足为奇了。

在各章中，我们尽可能使用**真实代码**，而不是**伪代码**（pseudocode），因此几乎所有例子，你都应该能自己敲进去运行。在真实系统上运行真实代码，是学习操作系统的最好方法，所以我们鼓励你尽量这么做。我们也把代码公开了，供你阅读[2]。

书中多处穿插了一些**作业**（homework），帮你确认自己理解了所讲的内容。很多作业是对操作系统某个部分的小型**模拟**（simulation）；你应该下载这些作业，运行它们来自测。这些作业模拟器有一个特点：给它们不同的随机种子，就能生成几乎无穷无尽的题目；还可以让模拟器替你解题。这样，你就可以反复自测，直到理解得足够透彻为止。

本书最重要的补充材料是一组**项目**（project），你要自己设计、实现和测试代码，从中了解真实系统是如何工作的。所有项目（以及上面提到的代码示例）都使用 **C 编程语言** [KR88]；C 是一门简单而强大的语言，大多数操作系统都建立在它之上，因此值得把它加进你的语言工具箱。项目分两类（思路见在线附录）。第一类是**系统编程**（systems programming）项目，非常适合刚接触 C 和 UNIX、想学习如何做底层 C 编程的人。第二类基于麻省理工学院开发的一个真实的操作系统内核 xv6 [CK+08]，非常适合已经会一些 C、想亲手深入操作系统内部的学生。在威斯康星大学，我们用三种不同的方式开过这门课：全部做系统编程项目、全部做 xv6 项目，或者两者混合。

我们正在陆续公开项目说明和测试框架。详情请见我们的代码仓库[3]。如果你不是在上这门课，这些材料也能让你自己动手做这些项目，更好地掌握所学内容。遗憾的是，遇到困难时没有助教可以缠着问，但人生中并不是什么都能免费（不过书可以！）。

[2] https://github.com/remzi-arpacidusseau/ostep-code

[3] https://github.com/remzi-arpacidusseau/ostep-projects

 © 2008–26, ARPACI-DUSSEAU

## 致教师

如果你是讲师或教授，想用这本书来教课，尽管用。你可能已经注意到了，各章都可以从下面这个网页免费在线获取：

```
http://www.ostep.org
```

你也可以在 `http://lulu.com` 或 `http://amazon.com` 上购买印刷版，具体请在上面的网页上查找。

本书（目前）正确的引用方式如下：

**Operating Systems: Three Easy Pieces**
Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau
Arpaci-Dusseau Books
August, 2018 (Version 1.00) or July, 2019 (Version 1.01)
or October, 2023 (Version 1.10)
`http://www.ostep.org`

这门课的内容刚好适合用一个 15 周的学期来讲，大部分主题都能讲到相当的深度。如果要压缩到一个 10 周的学季里，大概就得把每个部分都删掉一些细节。书中还有几章讲虚拟机监视器（virtual machine monitor），我们通常会在学期中找个时间塞进去：要么放在虚拟化这一大部分的最后，要么放在临近期末，作为一段插叙来讲。

本书有一个略显不同寻常的地方：许多操作系统教材把并发放在前面讲，本书却推迟到学生理解了 CPU 和内存的虚拟化之后才讲。根据我们讲授这门课近 20 年的经验，如果学生还不明白什么是地址空间（address space）、什么是进程，以及为什么上下文切换（context switch）可能在任意时刻发生，他们就很难理解并发问题是怎么产生的，也不明白为什么要去解决它。而一旦理解了这些概念，再引入线程（thread）的概念以及线程带来的问题，就会变得相当容易，至少容易一些。

我们尽可能用黑板（或白板）来讲课。在偏重概念的课上，我们带着几个主要的思想和例子来到教室，用板书把它们讲出来。讲义也很有用，可以根据所学内容给学生出一些具体的题目。在偏重实践的课上，我们干脆把笔记本电脑接到投影仪上，直接展示真实代码；这种方式特别适合讲并发，也适合在讨论课上给学生展示与他们的项目相关的代码。我们一般不用幻灯片来讲课，但现在也为喜欢这种讲课方式的老师准备了一套幻灯片。

这些材料你想要哪一份，都可以给我们发电子邮件。我们已经和世界各地的许多人分享过这些材料，也有人反过来贡献了他们自己的材料。

最后还有一个请求：如果你使用免费的在线章节，请直接**链接**到它们，而不要另存一份副本。这样既能帮我们统计使用情况（每月有上百万章次的下载），也能确保学生拿到最新（也是最好?）的版本。

## 致学生

如果你是正在读这本书的学生，谢谢你！能提供一些材料，帮助你学习操作系统的知识，是我们的荣幸。我们俩都怀念本科时读过的一些教材（例如 Hennessy 和 Patterson 的那本计算机体系结构经典教材 [HP90]），希望这本书也能成为你美好的回忆之一。

你可能已经注意到，这本书可以免费在线获取[4]。主要原因只有一个：教材一般都太贵了。我们希望，这本书能成为新一波免费教材中的第一本，帮助那些追求学业的人，不管他们来自世界的哪个角落，也不管他们愿意花多少钱买书。就算做不到这一点，它至少也是一本免费的书，有总比没有强。

我们还希望尽可能把书中大部分内容的原始出处指给你看：那些多年来塑造了操作系统领域的伟大论文和伟大人物。思想不是凭空而来的；它们来自聪明而勤奋的人（其中包括多位图灵奖得主[5]），因此我们应该尽可能去赞颂这些思想和这些人。这样做，我们也许就能更好地理解曾经发生过的那些革命，而不是把教材写得好像这些思想一直就存在似的 [K62]。此外，这些参考文献或许还能鼓励你自己去深入钻研；阅读本领域的著名论文，无疑是最好的学习方法之一。

[4]在这里岔开说一句：我们这里说的“免费”并不意味着开源，也不意味着这本书没有通常的版权保护，它是有版权的！它的意思是，你可以下载这些章节，用来学习操作系统。为什么不像 Linux 是开源内核那样，把书也开源呢？因为我们认为，一本书应该从头到尾保持同一种口吻，而我们也下了很大功夫去营造这种口吻。读这本书时，应该有点像在和人对话，听对方给你讲解。这就是我们这样做的原因。

[5]图灵奖是计算机科学的最高奖项，就像诺贝尔奖，只不过你从来没听说过它。

 © 2008–26, ARPACI-DUSSEAU

## 致谢

这一节用来感谢帮助我们完成这本书的人。眼下最重要的是：**你的名字也可以出现在这里！**不过，你得帮忙才行。所以，请给我们发来反馈，帮这本书除除 bug。说不定你还会出名！至少，你的名字会出现在某本书里。

迄今为止帮过我们的人包括：

Aaron Gember (Colgate), Aashrith H Govindraj (USF), Abdallah Ahmed, Abhinav Mehra, Abhinay Reddy, Abhirami Senthilkumaran*, Abhishek Bhattacherjee (NITR), Adam Drescher* (WUSTL), Adam Eggum, Adam Morrison, Aditya Venkataraman, Adriana Iamnitchi and class (USF), Ahmad Jarara, Ahmed Fikri*, Aisha Atif, Ajaykrishna Raghavan, Akiel Khan, Alain Clark (github), Alex Curtis, Alex Giorev, Alex Wyler, Alex Zhao (U. Colorado at Colorado Springs), Alexander Nordin (MIT), Ali Razeen (Duke), Alistair Martin, Amila Senadheera, AmirBehzad Eslami, Anand Mundada, Andrei Bozântan, Andrew Thomas Chan, Andrew Mahler, Andrew Moss, Andrew Valencik (Saint Mary's), Angela Demke Brown (Toronto), Antonella Bernobich (UoPeople)*, Arek Bulski, Arman Jasuja, Arun Rajan (Stonybrook), Aryan Arora, Axel Solis Trompler (KIT), B. Brahmananda Reddy (Minnesota), Bala Subrahmanyam Kambala, Bart Miller (Wisconsin), Basti Ortiz (github), Ben Kushigian (U. Mass), Benita Bose, Benjamin Wilhelm (Konstanz), BigAPG (github), Bill Yang, Biswajit Mazumder (Clemson), Bo Liu (UCSD), Bobby Adashev, Bobby Jack, Björn Lindberg, Brad Spendlove (Randolph), Brandon Harshe (U. Minn), Brennan Payne, Brian Gorman, Brian Kroth, Burak Gorduk, Calder White (University of Waterloo), Caleb Sumner (Southern Adventist), Cara Lauritzen, Charlotte Kissinger, Chen Huo (Shippensburg University), Chris Simionovici (U. Toronto), Cheng Su, Chien Chi, Chien-Chung Shen (Delaware)*, chriskorosu (github), Christian Stober, Christoph Jaeger (HTWK Leipzig), C.J. Stanbridge (Memorial U. of Newfoundland), Cody Hanson, Constantinos Georgiades, Dakota Cookenmaster (Southern Adventist), Dakota Crane (U. Washington-Tacoma), Dan Soendergaard (U. Aarhus), Dan Tsafrir (Technion), Daniel J Williams (IBM)*, Daniela Ferreira Franco Moura, Danilo Bruschi (Universita Degli Studi Di Milano), Darby Asher Noam Haller, David Hanle (Grinnell), David Hartman, Dawn Flood, Deepika Muthukumar, Demir Delic, Dennis Zhou, Dheeraj Shetty (North Carolina State), Diego Oleiarz (FaMAF UNC), dominikw1 (github), Dominic White, Dorian Arnold (New Mexico), Dustin Metzler, Dustin Passofaro, Dylan Kaplan, Eduardo Stelmaszczyk, Efkan S. Goktepe, Emad Sadeghi, Emil Hessman, Emily Jacobson, Emmett Witchel (Texas), Eric Freudenthal (UTEP), Erik Hjelmås Eric Kleinberg, Eric Johansson, Erik Turk, Ernst Biersack (France), Ethan Wood, Evan Leung, Fangjun Kuang (U. Stuttgart), Feng Zhang (IBM), Finn Kuusisto*, Francesco Piccoli, Gavin Inglis (Notre Dame), Gia Hoang Tran, Giovanni Di Santi, Giovanni Lagorio (DIBRIS), Giovanni Moricca, Glenn Bruns (CSU Monterey Bay), Glen Granzow (College of Idaho), Greggory Van Dycke, Guilherme Baptista, gurugio (github), Tian Guo (WPI), Hamid Reza Ghasemi, Hao Chen, Hao Liu, Hein Meling (Stavanger), Helen Gaiser (HTWG Konstanz), Henry Abbey, Hilmar Gústafsson (Aalborg University), Holly Johnson (USF), Hrishikesh Amur, Huanchen Zhang*, Huseyin Sular, Hugo Diaz, Hyrum Carroll (Columbus State), Ilya Oblomkov, Immacolata Fasano, Itai Hass (Toronto), Jack Xu (Wisconsin), Jackson "Jake" Haenchen (Texas), Jacob Levinson (UCLA), Jaeyoung Cho (SKKU), Jagannathan Eachambadi, Jake Gillberg, Jakob Olandt, James Earley, James Perry (U. Michigan-Dearborn)*, Jan Reineke (Universität des Saarlandes), Jason MacLafferty (Southern Adventist), Jason Waterman (Vassar), Jay Lim, Jeremiah Ryan Buizon, Jerod Weinman (Grinnell), Jersey Deng (UCLA), Jhih-Cheng Luo, Jiao Dong (Rutgers), Jia-Shen Boon, Jiawen Bao, Jidong Xiao (Boise State), Jingxin Li, jmcruzGH (github), Joe Jean (NYU), Joel Kuntz (Saint Mary's), Joel Hassan (U. Helsinki), Joel Sommers (Colgate), Johannes Kinder, John Brady (Grinnell), John Komenda, John McEachen (NPS), John Shawger (Wisconsin), Jonathan Perry (MIT), Joshua Carpenter (NCSU), Josip Cavar, Jun He, Justinas Petuchovas, k4lizen, Kai Mast (Wisconsin), Karl Schultheisz, Karl Wallinger, Kartik Singhal, Katherine Dudenas, Katie Coyle (Georgia Tech),

Kaushik Kannan, Kemal Bıçakcı, Kevin Liu*, Khaled Emara, Kyle Hale (Illinois Institute of Technology), KyoungSoo Park (KAIST), Kyutae Lee*, Lanyue Lu, Laura Xu, legate (github), Lei Tian (U. Nebraska-Lincoln), Leonardo Medici (U. Milan), Leslie Schultz, Liang Yin, Lihao Wang, Looserof, Louie Lu, Luigi Finetti (FaMAF, UNC) Luna Gal (Wooster), lyazj (github), Manav Batra (IIIT-Delhi), Manu Awasthi (Samsung), Marcel van der Holst, Marco Guazzone (U. Piemonte Orientale), Marco Pedrinazzi (Universita Degli Studi Di Milano), Marius Rodi, Mart Oskamp, Martha Ferris, Martinho Macedo, Masashi Kishikawa (Sony), Matt Reichoff, Matías De Pascuale (FaMAF Universidad Nacional De Cordoba), Matthew Prinz, Matthias St. Pierre, Mattia Monga (U. Milan), Matty Williams, Megan Cutrofello, Meng Huang, Michael Machtel (Hochschule Konstanz), Michael Walfish (NYU), Michael Wu (UCLA), Mike Griepentrog, Ming Chen (Stonybrook), Mohammed Alali (Delaware), Mohamed Omran (GUST), Mondo Gao (Wisconsin), Muhammad Mobeen Movania, Muhammad Yasoob Ullah Khalid, Murat Kerim Aslan, Murugan Kandaswamy, Nadeem Shaikh, Nan Xiao, Natasha Eilbert, Natasha Stopa, Nathan Dipiazza, Nathan Sullivan, Neeraj Badlani (N.C. State), Neil Perry, Nelson Gomez, Neven Sajko, Nghia Huynh (Texas), Ngu (Nicholas) Q. Truong, Nicholas Mandal, Nick Weinandt, Nishin Shouzab, Nitin Puranik, Nizare Leonetti, Noah Jackson, Otto Sievert, ParagDixit (github), Patel Pratyush Ashesh (BITS-Pilani), Patricio Jara, Patrick Elsen, Patrizio Palmisano, Pavle Kostovic, Perry Kivolowitz, Peter Peterson (Minnesota), Phani Karan, Pieter Kockx, Po-Hao Su (Taiwan) Prabhsimrandeep Singh, Prairie Rose Goodwin, Radford Smith, Raunak Seksaria, Repon Kumar Roy, Reynaldo H. Verdejo Pinochet, Riccardo Mutschlechner, Rick Perry, Richard Campanha (Georgia Tech), Ripudaman Singh, Rita Pia Folisi, Robert Ordóñez and class (Southern Adventist), Robin Li (Cornell), Roger Wattenhofer (ETH), Rohan Das (Toronto)*, Rohan Pasalkar (Minnesota), Rohan Puri, Roman Bureacov, Ross Aiken, Ruslan Kiselev, Ryan Albrecht, Ryland Herrick, Sam Kelly, Sam Noh (UNIST), Sameer Punjal, Samer Al-Kiswany, Sandeep Ummadi (Minnesota), Santiago Marini, Sankaralingam Panneerselvam, Sarvesh Tandon (Wisconsin), Satish Chebrolu (NetApp), Satyanarayana Shanmugam*, Scott Catlin, Scott Clark, Scott Lee (UCLA), Scott Schoeller, sepehr541 (github), Seth Pollen, Sharad Punuganti, Shawn Ge, Shivaram Venkataraman, Shreevatsa R., Simon Pratt (University of Waterloo), Sirui Chen, Sivaraman Sivaraman*, Song Jiang (Wayne State), Spencer Harston (Weber State), Srinivasan Thirunarayanan*, stadtz (github), Stardustman (github), Stefan Dekanski, Stephen Bye, Stephen Schaub, Suriyhaprakhas Balaram Sankari, Sy Jin Cheah, Teri Zhao (EMC), Thanumalayan S. Pillai, thasinaz (github), Thomas Griebel, Thomas John Lesniak (Wisconsin), Thomas Scrace, Tianxia Bai, Tobi Popoola (Boise State), Tong He, Tongxin Zheng, Tony Adkins, Torin Rudeen (Princeton), Tuo Wang, Tyler Couto, Varun Vats, Veera Sivarajan, Vegard Stikbakke, Victor Chernyakin, Vikas Goel, Waciuma Wanjohi, William Royle (Grinnell), Winson Huang (github), Wojciech Rok (AGH University of Kraków), Xiang Peng, Xu Di, Yanjun Wen, Yanyan Jiang, Yifan Hao, Yuanyuan Chen, Yubin Ruan, Yudong Sun, Yue Zhuo (Texas A&M), Yufeng Zhang (UCLA), Yufui Ren, Yuxing Xiang (Peking), Zef RosnBrick, Zeyuan Hu (Texas), Zhengguang Zhou (Wisconsin), ZiHan Zheng (USTC), zino23 (github), Zuyu Zhang.

特别感谢上面标有星号的各位，他们提出的改进建议远远超出了我们的期望。

此外，衷心感谢 Joe Meehean 教授（Lynchburg）为每一章写的详细笔记；感谢 Jerod Weinman 教授（Grinnell）和他全班同学做的精彩小册子；感谢 Chien-Chung Shen 教授（Delaware）宝贵而细致的审读和意见；感谢 Adam Drescher（WUSTL）认真的审读和建议；感谢 Glen Granzow（College of Idaho）极其详尽的意见和提示；感谢 Michael Walfish（NYU）的热情和详细的改进建议；感谢 Peter Peterson（UMD）提供的许多有用的反馈和评论；感谢 Mark Kampe（Pomona）详尽的批评意见（我们真希望能把所有建议都改好！）；感谢 Youjip Won（Hanyang）把本书译成韩文（！）并提出许多深刻的建议；感谢 Terence Kelly 撰写关于内存映射（memory mapping）的补充。他们都给了我们无法估量的帮助！

 © 2008–26, ARPACI-DUSSEAU

特别感谢 Peter Reiher 教授（UCLA）以本书的风格写出了一组精彩的安全章节。我们有幸在很多年前就认识了 Peter，却万万没想到二十年后会以这种方式合作。了不起的工作！

还要深深感谢这些年来上过 537 这门课的数百名学生。尤其是 2008 年秋季学期的那个班，是他们促成了这些讲义最早的书面形式（他们受够了连本像样的教材都没有，真是一群爱施压的学生！），然后又给了足够多的称赞，让我们坚持写了下去（包括那年课程评估里一条令人捧腹的评语："ZOMG! You should totally write a textbook!"，意思是"天哪！你们绝对该写本教材！"）。

我们还要深深感谢少数几位勇敢的学生，他们选修了 xv6 项目实验课，那门课的大部分内容如今已经并入了 537 的主课。2009 年春季：Justin Cherniak、Patrick Deline、Matt Czech、Tony Gregerson、Michael Griepentrog、Tyler Harter、Ryan Kroiss、Eric Radzikowski、Wesley Reardan、Rajiv Vaidyanathan 和 Christopher Waclawik。2009 年秋季：Nick Bearson、Aaron Brown、Alex Bird、David Capel、Keith Gould、Tom Grim、Jeffrey Hugo、Brandon Johnson、John Kjell、Boyan Li、James Loethen、Will McCardell、Ryan Szaroletta、Simon Tso 和 Ben Yule。2010 年春季：Patrick Blesi、Aidan Dennis-Oehling、Paras Doshi、Jake Friedman、Benjamin Frisch、Evan Hanson、Pikkili Hemanth、Michael Jeung、Alex Langenfeld、Scott Rick、Mike Treffert、Garret Staus、Brennan Wall、Hans Werner、Soo-Young Yang 和 Carlos Griffin（差一点就算上了）。

我们的博士生虽然没有直接参与这本书，但我们对系统的了解，很多都是从他们那里学来的。他们在威斯康星大学期间，我们经常和他们交流，而真正的工作都是他们做的；听他们讲自己在做什么，我们每周都能学到新东西。下面列出的是已经毕业的博士：Nitin Agrawal、Ram Alagappan、Leo Arulraj、Lakshmi Bairavasundaram、John Bent、Nathan Burnett、Tyler Caraza-Harter、Vijay Chidambaram、Timothy Denehy、Thanh Do、Aishwarya Ganesan、Haryadi Gunawi、Jun He、Todd Jones、Jing Liu、Lanyue Lu、Joe Meehean、Yuvraj Patel、Thanu Pillai、Florentina Popovici、Vijayan Prabhakaran、Anthony Rebello、Muthian Sivathanu、Sriram Subramanian、Swami Sundararaman、Zev Weiss、Kan Wu、Suli Yang、Yiying Zhang、Yupu Zhang、Yifan Dai、Guanzhou Hu。还有下面几位博士后：Shaohua Duan、Samer Al-Kiswany、Sudarsun Kannan、Kai Mast。

当然，还有许多其他学生（本科生和硕士生）和合作者与我们合写过论文。我们也要感谢他们；想知道他们是谁，请看我们的网页或 DBLP。

我们的研究生主要由美国国家科学基金会（NSF）、美国能源部科学办公室（DOE）资助，也得到了业界的捐赠和资助。我们尤其感谢 NSF 多年来的支持，因为我们的研究塑造了本书许多章的内容。

感谢 Thomas Griebel，是他要求给这本书换个更好的封面。虽然我们没有采纳他的具体建议（一只恐龙，你敢信吗？），但要是没有他，封面上就不会有那张美丽的哈雷彗星照片。

最后，我们还要感谢 Aaron Brown。他很多年前（2009 年春季）先上了这门课，接着上了 xv6 实验课（2009 年秋季），最后又当了这门课大约两年的研究生助教（2010 年秋季到 2012 年春季）。他不知疲倦的工作大大改进了这些项目（尤其是 xv6 方面的项目），从而让威斯康星大学无数本科生和研究生的学习体验变得更好。正如 Aaron 会说的那样（以他一贯的简洁）："Thx."

## 写在最后

Yeats 有一句名言：“教育不是注满一桶水，而是点燃一把火。”他说对了，但同时也说错了[6]。你确实得“注满”一点水，而这些讲义当然就是来帮你完成这部分教育的；毕竟，你去 Google 面试时，要是人家问你一道怎么用信号量（semaphore）的刁钻题，真知道信号量是什么总归有好处，对吧?

不过，Yeats 更大的观点显然切中要害：教育真正的意义，在于让你对某样东西产生兴趣，主动去多学一些，而不只是为了在某门课上拿个好成绩，把非消化不可的东西消化掉。正如我们其中一人的父亲（Remzi 的父亲 Vedat Arpaci）常说的：“要在课堂之外学习。”

我们写这些讲义，是为了激发你对操作系统的兴趣，让你自己去读更多相关的东西，去和你的教授聊聊这个领域里正在进行的那些激动人心的研究，甚至亲自参与这些研究。这是一个了不起的领域（!），充满了令人兴奋的奇妙思想，它们深刻而重要地塑造了计算的历史。虽然我们明白，这把火不会在你们每个人心中点燃，但希望它能在许多人心中点燃，哪怕只有少数几个人也好。因为这把火一旦点燃，你就真正有能力做出一番大事了。这也正是教育真正的意义所在：走出去，研究许多新奇迷人的课题，去学习，去成长，而最重要的是，找到能在你心中点燃一把火的东西。

*Andrea* 和 *Remzi*
一对夫妻
威斯康星大学计算机科学教授
但愿也是首席点火官[7]

[6]如果他真的说过这句话的话；和许多名言一样，这句妙语的来历扑朔迷离。

[7]如果这听起来像是我们在承认自己有过纵火前科，那你大概没领会我们的意思。大概。如果这听起来很肉麻，嗯，那是因为它确实很肉麻，不过你只好原谅我们了。

## 参考文献

[CK+08] "The xv6 Operating System" by Russ Cox, Frans Kaashoek, Robert Morris, Nickolai Zeldovich. From: `http://pdos.csail.mit.edu/6.828/2008/index.html`. *xv6 was developed as a port of the original* UNIX *version 6 and represents a beautiful, clean, and simple way to understand a modern operating system.*

[F96] "Six Easy Pieces: Essentials Of Physics Explained By Its Most Brilliant Teacher" by Richard P. Feynman. Basic Books, 1996. *This book reprints the six easiest chapters of Feynman's Lectures on Physics, from 1963. If you like Physics, it is a fantastic read.*

[HP90] "Computer Architecture a Quantitative Approach" (1st ed.) by David A. Patterson and John L. Hennessy . Morgan-Kaufman, 1990. *A book that encouraged each of us at our undergraduate institutions to pursue graduate studies; we later both had the pleasure of working with Patterson, who greatly shaped the foundations of our research careers.*

[KR88] "The C Programming Language" by Brian Kernighan and Dennis Ritchie. Prentice-Hall, April 1988. *The C programming reference that everyone should have, by the people who invented the language.*

[K62] "The Structure of Scientific Revolutions" by Thomas S. Kuhn. University of Chicago Press, 1962. *A great and famous read about the fundamentals of the scientific process. Mop-up work, anomaly, crisis, and revolution. We are mostly destined to do mop-up work, alas.*

 © 2008–26, ARPACI-DUSSEAU

# 目录


致所有读者 . . . iii
致教师 . . . v
致学生 . . . vi
致谢 . . . vii
写在最后 . . . x
参考文献 . . . xi

**1 关于本书的对话** . . . 24

**2 操作系统简介** . . . 3
2.1 虚拟化 CPU . . . 5
2.2 虚拟化内存 . . . 6
2.3 并发 . . . 8
2.4 持久性 . . . 10
2.5 设计目标 . . . 11
2.6 一点历史 . . . 12
2.7 小结 . . . 15

**第一部分 虚拟化**

**3 关于虚拟化的对话** . . . 19

**4 抽象：进程** . . . 22
4.1 抽象：进程 . . . 23
4.2 进程 API . . . 24
4.3 创建进程：再多讲一点 . . . 25
4.4 进程状态 . . . 25
4.5 数据结构 . . . 27
4.6 小结 . . . 29

**5 插叙：进程 API** . . . 32
5.1 fork() 系统调用 . . . 32
5.2 wait() 系统调用 . . . 34
5.3 最后是 exec() 系统调用 . . . 35
5.4 为什么？API 的设计动机 . . . 35
5.5 进程控制与用户 . . . 38

5.6 实用工具 . . . 39
5.7 小结 . . . 40

**6 机制：受限直接执行** . . . 44
6.1 基本技术：受限直接执行 . . . 44
6.2 问题 1：受限的操作 . . . 45
6.3 问题 2：在进程之间切换 . . . 47
6.4 担心并发？ . . . 52
6.5 小结 . . . 53

**7 调度：简介** . . . 58
7.1 工作负载假设 . . . 58
7.2 调度指标 . . . 59
7.3 先进先出（FIFO） . . . 59
7.4 最短作业优先（SJF） . . . 61
7.5 最短完成时间优先（STCF） . . . 62
7.6 新的指标：响应时间 . . . 62
7.7 时间片轮转 . . . 63
7.8 结合 I/O . . . 65
7.9 不再有先知 . . . 66
7.10 小结 . . . 66

**8 调度：多级反馈队列** . . . 69
8.1 MLFQ：基本规则 . . . 69
8.2 尝试 1：如何改变优先级 . . . 71
8.3 尝试 2：优先级提升 . . . 73
8.4 尝试 3：更好的记账 . . . 74
8.5 MLFQ 的调优及其他问题 . . . 75
8.6 MLFQ：小结 . . . 76

**9 调度：比例份额** . . . 80
9.1 基本概念：彩票代表你的份额 . . . 80
9.2 彩票机制 . . . 81
9.3 实现 . . . 83
9.4 一个例子 . . . 83
9.5 怎样分配彩票？ . . . 84
9.6 步长调度 . . . 84
9.7 Linux 的完全公平调度程序（CFS） . . . 86
9.8 小结 . . . 89

**10 多处理器调度（进阶）** . . . 93
10.1 背景：多处理器架构 . . . 94
10.2 别忘了同步 . . . 95
10.3 最后一个问题：缓存亲和性 . . . 96
10.4 单队列调度 . . . 96
10.5 多队列调度 . . . 98
10.6 Linux 多处理器调度程序 . . . 100
10.7 小结 . . . 100

 © 2008–23, ARPACI-DUSSEAU

**11 关于 CPU 虚拟化的总结对话** . . . 104

**12 关于内存虚拟化的对话** . . . 106

**13 抽象：地址空间** . . . 108
13.1 早期系统 . . . 108
13.2 多道程序设计和分时 . . . 109
13.3 地址空间 . . . 109
13.4 目标 . . . 111
13.5 小结 . . . 112

**14 插叙：内存 API** . . . 116
14.1 内存类型 . . . 116
14.2 malloc() 调用 . . . 117
14.3 free() 调用 . . . 118
14.4 常见错误 . . . 118
14.5 底层操作系统支持 . . . 121
14.6 其他调用 . . . 122
14.7 小结 . . . 122

**15 机制：地址转换** . . . 125
15.1 假设 . . . 126
15.2 一个例子 . . . 126
15.3 动态（基于硬件的）重定位 . . . 128
15.4 硬件支持：总结 . . . 131
15.5 操作系统层面的问题 . . . 132
15.6 小结 . . . 133

**16 分段** . . . 138
16.1 分段：泛化的基址/界限 . . . 138
16.2 引用的是哪个段? . . . 141
16.3 栈怎么办? . . . 142
16.4 支持共享 . . . 143
16.5 细粒度与粗粒度的分段 . . . 143
16.6 操作系统支持 . . . 144
16.7 小结 . . . 146

**17 空闲空间管理** . . . 149
17.1 假设 . . . 150
17.2 底层机制 . . . 150
17.3 基本策略 . . . 156
17.4 其他方法 . . . 158
17.5 小结 . . . 160

**18 分页：简介** . . . 163
18.1 一个简单的例子和概览 . . . 163
18.2 页表存放在哪里? . . . 166
18.3 页表里究竟有什么? . . . 167
18.4 分页：不仅太大，而且太慢 . . . 168

18.5 一次内存跟踪 . . . 170
18.6 小结 . . . 172

**19 分页：更快的转换（TLB） . . . 175**
19.1 TLB 的基本算法 . . . 175
19.2 示例：访问数组 . . . 176
19.3 谁来处理 TLB 未命中? . . . 179
19.4 TLB 的内容：里面有什么? . . . 180
19.5 TLB 的问题：上下文切换 . . . 181
19.6 问题：替换策略 . . . 183
19.7 一个真实的 TLB 项 . . . 183
19.8 小结 . . . 184

**20 分页：更小的页表 . . . 189**
20.1 简单的办法：更大的页 . . . 189
20.2 混合方案：分页和分段 . . . 190
20.3 多级页表 . . . 193
20.4 反置页表 . . . 199
20.5 把页表交换到磁盘 . . . 199
20.6 小结 . . . 199

**21 超越物理内存：机制 . . . 202**
21.1 交换空间 . . . 203
21.2 存在位 . . . 204
21.3 缺页异常 . . . 205
21.4 内存满了怎么办? . . . 205
21.5 缺页异常的控制流 . . . 206
21.6 置换实际上何时发生 . . . 207
21.7 小结 . . . 208

**22 超越物理内存：策略 . . . 212**
22.1 缓存管理 . . . 212
22.2 最优置换策略 . . . 213
22.3 一个简单的策略：FIFO . . . 215
22.4 另一个简单的策略：随机 . . . 216
22.5 利用历史：LRU . . . 217
22.6 工作负载示例 . . . 219
22.7 实现基于历史的算法 . . . 220
22.8 近似 LRU . . . 222
22.9 考虑脏页 . . . 223
22.10 其他虚拟内存策略 . . . 223
22.11 抖动 . . . 223
22.12 小结 . . . 224

**23 完整的虚拟内存系统 . . . 228**
23.1 VAX/VMS 虚拟内存 . . . 228
23.2 Linux 虚拟内存系统 . . . 233
23.3 小结 . . . 241

**24 关于内存虚拟化的总结对话** . . . 243

## 第二部分 并发

**25 关于并发的对话** . . . 246

**26 并发：简介** . . . 250
26.1 为什么要用线程? . . . 251
26.2 一个例子：创建线程 . . . 252
26.3 为什么会更糟：共享数据 . . . 254
26.4 问题的核心：不受控制的调度 . . . 255
26.5 对原子性的期望 . . . 258
26.6 另一个问题：等待另一个线程 . . . 260
26.7 小结：为什么要在操作系统课上讲这些? . . . 260

**27 插叙：线程 API** . . . 264
27.1 创建线程 . . . 264
27.2 等待线程结束 . . . 266
27.3 锁 . . . 268
27.4 条件变量 . . . 269
27.5 编译和运行 . . . 271
27.6 小结 . . . 271

**28 锁** . . . 275
28.1 锁的基本思想 . . . 275
28.2 Pthread 锁 . . . 276
28.3 实现一个锁 . . . 276
28.4 评价锁 . . . 277
28.5 控制中断 . . . 277
28.6 失败的尝试：只用加载和存储 . . . 278
28.7 用测试并设置实现可用的自旋锁 . . . 279
28.8 评价自旋锁 . . . 282
28.9 比较并交换 . . . 282
28.10 链接加载和条件存储 . . . 283
28.11 获取并增加 . . . 285
28.12 自旋太多：怎么办? . . . 285
28.13 简单的办法：让出来就行，宝贝 . . . 286
28.14 使用队列：用睡眠代替自旋 . . . 287
28.15 不同的操作系统，不同的支持 . . . 290
28.16 两阶段锁 . . . 290
28.17 小结 . . . 292

**29 基于锁的并发数据结构** . . . 295
29.1 并发计数器 . . . 295
29.2 并发链表 . . . 298
29.3 并发队列 . . . 302
29.4 并发散列表 . . . 305

29.5 小结 . . . 306

**30 条件变量** . . . 310
30.1 定义和例程 . . . 311
30.2 生产者/消费者（有界缓冲区）问题 . . . 314
30.3 覆盖条件 . . . 320
30.4 小结 . . . 324

**31 信号量** . . . 327
31.1 信号量的定义 . . . 327
31.2 二值信号量（锁） . . . 328
31.3 用信号量控制顺序 . . . 329
31.4 生产者/消费者（有界缓冲区）问题 . . . 331
31.5 读写锁 . . . 335
31.6 哲学家就餐问题 . . . 336
31.7 线程节流 . . . 340
31.8 如何实现信号量 . . . 340
31.9 小结 . . . 341

**32 常见并发问题** . . . 344
32.1 有哪些类型的 bug? . . . 344
32.2 非死锁 bug . . . 345
32.3 死锁 bug . . . 348
32.4 小结 . . . 355

**33 基于事件的并发（进阶）** . . . 358
33.1 基本思想：事件循环 . . . 358
33.2 一个重要的 API：select()（或 poll()） . . . 359
33.3 使用 select() . . . 360
33.4 为什么更简单？不需要锁 . . . 360
33.5 一个问题：阻塞的系统调用 . . . 360
33.6 一个解决方案：异步 I/O . . . 362
33.7 另一个问题：状态管理 . . . 363
33.8 使用事件时仍然存在的难题 . . . 363
33.9 小结 . . . 365

**34 关于并发的总结对话** . . . 368

## 第三部分 持久性

**35 关于持久性的对话** . . . 370

**36 I/O 设备** . . . 373
36.1 系统架构 . . . 373
36.2 典型设备 . . . 374
36.3 典型协议 . . . 375
36.4 用中断降低 CPU 开销 . . . 376
36.5 用 DMA 更高效地搬运数据 . . . 377

36.6 设备交互的方法 . . . 378
36.7 融入操作系统：设备驱动程序 . . . 379
36.8 案例研究：一个简单的 IDE 磁盘驱动程序 . . . 380
36.9 历史注记 . . . 381
36.10 小结 . . . 383

**37 硬盘驱动器** . . . 385
37.1 接口 . . . 385
37.2 基本几何结构 . . . 386
37.3 一个简单的磁盘驱动器 . . . 386
37.4 I/O 时间：算一算 . . . 389
37.5 磁盘调度 . . . 391
37.6 小结 . . . 396

**38 廉价磁盘冗余阵列（RAID）** . . . 399
38.1 接口与 RAID 内部结构 . . . 400
38.2 故障模型 . . . 400
38.3 如何评估 RAID . . . 401
38.4 RAID 0 级：条带化 . . . 401
38.5 RAID 1 级：镜像 . . . 404
38.6 RAID 4 级：用奇偶校验节省空间 . . . 407
38.7 RAID 5 级：轮转奇偶校验 . . . 410
38.8 RAID 比较：小结 . . . 411
38.9 RAID 的其他有趣问题 . . . 411
38.10 小结 . . . 412

**39 插叙：文件和目录** . . . 415
39.1 文件和目录 . . . 415
39.2 文件系统接口 . . . 417
39.3 创建文件 . . . 417
39.4 读写文件 . . . 418
39.5 读写，但不按顺序 . . . 420
39.6 共享文件表条目：fork() 和 dup() . . . 422
39.7 用 fsync() 立即写入 . . . 423
39.8 重命名文件 . . . 425
39.9 获取文件信息 . . . 425
39.10 删除文件 . . . 427
39.11 创建目录 . . . 428
39.12 读取目录 . . . 428
39.13 删除目录 . . . 430
39.14 硬链接 . . . 430
39.15 符号链接 . . . 431
39.16 权限位和访问控制列表 . . . 433
39.17 创建和挂载文件系统 . . . 435
39.18 小结 . . . 436

**40 文件系统实现** . . . 440
40.1 思考的方式 . . . 440
40.2 整体组织 . . . 441
40.3 文件组织：inode . . . 443
40.4 目录组织 . . . 446
40.5 空闲空间管理 . . . 448
40.6 访问路径：读和写 . . . 448
40.7 缓存和缓冲 . . . 451
40.8 小结 . . . 453

**41 局部性和快速文件系统** . . . 456
41.1 问题：性能很差 . . . 456
41.2 FFS：解决方案是了解磁盘 . . . 457
41.3 组织结构：柱面组 . . . 457
41.4 策略：如何分配文件和目录 . . . 459
41.5 测量文件的局部性 . . . 460
41.6 大文件的例外规则 . . . 462
41.7 关于 FFS 的其他几件事 . . . 464
41.8 小结 . . . 465

**42 崩溃一致性：FSCK 和日志** . . . 468
42.1 一个详细的例子 . . . 469
42.2 方案一：文件系统检查程序 . . . 471
42.3 方案二：日志（或预写日志） . . . 473
42.4 方案三：其他方法 . . . 481
42.5 小结 . . . 482

**43 日志结构文件系统** . . . 486
43.1 顺序地写入磁盘 . . . 487
43.2 顺序而高效地写入 . . . 488
43.3 要缓冲多少? . . . 489
43.4 问题：找到 inode . . . 489
43.5 用间接层来解决：inode map . . . 490
43.6 完成方案：检查点区域 . . . 491
43.7 从磁盘读取文件：回顾 . . . 491
43.8 目录怎么办? . . . 492
43.9 新的问题：垃圾回收 . . . 492
43.10 判断块是否存活 . . . 494
43.11 策略问题：清理哪些块，何时清理? . . . 495
43.12 崩溃恢复和日志 . . . 495
43.13 小结 . . . 496

**44 基于闪存的 SSD** . . . 500
44.1 存储单个位 . . . 500
44.2 从位到 bank/平面 . . . 501
44.3 闪存的基本操作 . . . 501
44.4 闪存的性能和可靠性 . . . 503

44.5 从原始闪存到基于闪存的 SSD . . . 504
44.6 FTL 的组织方式：一种糟糕的方法 . . . 505
44.7 日志结构的 FTL . . . 505
44.8 垃圾回收 . . . 508
44.9 映射表的大小 . . . 509
44.10 磨损均衡 . . . 513
44.11 SSD 的性能和成本 . . . 514
44.12 小结 . . . 515

**45 数据完整性与保护** . . . 521
45.1 磁盘故障模式 . . . 521
45.2 处理 latent sector error . . . 523
45.3 检测损坏：校验和 . . . 523
45.4 使用校验和 . . . 526
45.5 一个新问题：misdirected write . . . 526
45.6 最后一个问题：lost write . . . 527
45.7 Scrubbing . . . 527
45.8 校验和的开销 . . . 528
45.9 小结 . . . 528

**46 关于持久性的总结对话** . . . 532

**47 关于分布式的对话** . . . 534

**48 分布式系统** . . . 535
48.1 通信基础 . . . 536
48.2 不可靠的通信层 . . . 536
48.3 可靠的通信层 . . . 537
48.4 通信抽象 . . . 541
48.5 远程过程调用（RPC） . . . 542
48.6 小结 . . . 546

**49 Sun 的网络文件系统（NFS）** . . . 549
49.1 一个基本的分布式文件系统 . . . 550
49.2 走近 NFS . . . 550
49.3 重点：简单、快速的服务器崩溃恢复 . . . 551
49.4 快速崩溃恢复的关键：无状态 . . . 551
49.5 NFSv2 协议 . . . 552
49.6 从协议到分布式文件系统 . . . 554
49.7 用幂等操作处理服务器故障 . . . 554
49.8 提高性能：客户端缓存 . . . 557
49.9 缓存一致性问题 . . . 557
49.10 评估 NFS 的缓存一致性 . . . 559
49.11 对服务器端写缓冲的影响 . . . 559
49.12 小结 . . . 561

**50 Andrew 文件系统（AFS）** . . . 564
50.1 AFS 第 1 版 . . . 564

50.2 第 1 版的问题 . . . 565
50.3 改进协议 . . . 566
50.4 AFS 第 2 版 . . . 566
50.5 缓存一致性 . . . 567
50.6 崩溃恢复 . . . 570
50.7 AFSv2 的规模与性能 . . . 571
50.8 AFS 的其他改进 . . . 572
50.9 小结 . . . 573

**51 关于分布式的总结对话** . . . 576

## 第四部分 安全

**52 关于安全的对话** . . . 580

**53 操作系统安全简介** . . . 582
53.1 引言 . . . 582
53.2 我们要保护什么? . . . 583
53.3 安全目标与安全策略 . . . 584
53.4 设计安全的系统 . . . 586
53.5 操作系统安全基础 . . . 587
53.6 小结 . . . 589

**54 身份认证** . . . 591
54.1 引言 . . . 591
54.2 给进程附加身份 . . . 592
54.3 如何认证用户? . . . 593
54.4 基于你知道什么来认证 . . . 594
54.5 基于你拥有什么来认证 . . . 597
54.6 基于你是什么来认证 . . . 599
54.7 认证非人类主体 . . . 601
54.8 小结 . . . 603

**55 访问控制** . . . 605
55.1 引言 . . . 605
55.2 访问控制问题的几个重要方面 . . . 606
55.3 用 ACL 实现访问控制 . . . 608
55.4 用能力实现访问控制 . . . 610
55.5 强制访问控制与自主访问控制 . . . 612
55.6 访问控制机制的实际问题 . . . 613
55.7 小结 . . . 616

**56 用密码学保护信息** . . . 619
56.1 引言 . . . 619
56.2 密码学 . . . 620
56.3 公钥密码学 . . . 622
56.4 密码学散列 . . . 624
56.5 破解密码学 . . . 625

 © 2008–23, ARPACI-DUSSEAU

56.6 密码学与操作系统 . . . 627
56.7 静态数据加密 . . . 628
56.8 密码学能力 . . . 630
56.9 小结 . . . 631

**57 分布式系统安全** . . . 634
57.1 引言 . . . 634
57.2 身份认证的作用 . . . 634
57.3 分布式系统中的公钥认证 . . . 636
57.4 分布式系统中的密码认证 . . . 638
57.5 SSL/TLS . . . 639
57.6 其他身份认证方法 . . . 641
57.7 一些更高层的工具 . . . 643
57.8 小结 . . . 645

## 附录

**A 关于虚拟机监视器的对话** . . . 649

**B 虚拟机监视器** . . . 650
B.1 引言 . . . 650
B.2 动机：为什么需要 VMM . . . 650
B.3 虚拟化 CPU . . . 651
B.4 虚拟化内存 . . . 654
B.5 信息鸿沟 . . . 658
B.6 小结 . . . 659

**C 关于管程的对话** . . . 661

**D 管程（已过时）** . . . 662
D.1 何必用管程? . . . 663
D.2 除了自动加锁，还有别的好处吗? . . . 663
D.3 理论与实践相遇之处 . . . 665
D.4 糟了，有竞态 . . . 665
D.5 稍微看看内部实现 . . . 666
D.6 管程的其他用法 . . . 668
D.7 用管程实现信号量 . . . 669
D.8 现实世界中的管程 . . . 670
D.9 小结 . . . 672

**E 关于实验的对话** . . . 674

**F 实验：教程** . . . 676
F.1 一个简单的 C 程序 . . . 676
F.2 编译和执行 . . . 677
F.3 有用的选项 . . . 677
F.4 链接库 . . . 678
F.5 分别编译 . . . 679

 © 2008–23, ARPACI-DUSSEAU

F.6 Makefile . . . 679
F.7 调试 . . . 682
F.8 文档 . . . 683
F.9 推荐阅读 . . . 685

**G 实验：系统项目** . . . 687
G.1 入门项目 . . . 687
G.2 U Shell . . . 687
G.3 内存分配库 . . . 688
G.4 并发入门 . . . 688
G.5 并发 Web 服务器 . . . 688
G.6 文件系统检查程序 . . . 688
G.7 文件系统碎片整理程序 . . . 688
G.8 并发文件服务器 . . . 688

**H 实验：xv6 项目** . . . 690
H.1 入门项目 . . . 690
H.2 进程和调度 . . . 690
H.3 虚拟内存入门 . . . 690
H.4 写时复制映射 . . . 691
H.5 内存映射 . . . 691
H.6 内核线程 . . . 691
H.7 高级内核线程 . . . 691
H.8 基于区段的文件系统 . . . 691
H.9 快速文件系统 . . . 691
H.10 日志文件系统 . . . 691
H.11 文件系统检查程序 . . . 692

 © 2008–23, ARPACI-DUSSEAU

# 1

# 关于本书的对话

**教授**：欢迎来读这本书！它叫 ***Operating Systems in Three Easy Pieces***（操作系统：三个简单的部分），我在这儿教你操作系统方面该懂的东西。我叫“教授”；你是谁？

**学生**：您好，教授！您大概也猜到了，我叫“学生”。我已经准备好学习了！

**教授**：很好。有什么问题吗？

**学生**：当然有！为什么叫“三个简单的部分”（*Three Easy Pieces*）？

**教授**：这个简单。嗯，你看，*Richard Feynman* 有一套很棒的物理学讲义……

**学生**：哦！就是写《别闹了，费曼先生》（*Surely You're Joking, Mr. Feynman*）的那个人，对吧？那本书太棒了！这本书也会像那本一样好笑吗？

**教授**：呃……嗯，不会。那本书确实很棒，我很高兴你读过。我希望这本书更像他的物理讲义。那套讲义里的一些基础内容被汇编成了一本书，叫 *Six Easy Pieces*（六个简单的部分）。他讲的是物理；我们要讲的是操作系统这个好题目，分成三个简单的部分。这很合适，因为操作系统的难度大约只有物理的一半。

**学生**：嗯，我挺喜欢物理的，所以这大概是好事。那是哪三个部分呢？

**教授**：就是我们要学的三个核心思想：**虚拟化**（*virtualization*）、**并发**（*concurrency*）和**持久性**（*persistence*）。学这些思想的过程中，我们会把操作系统的工作原理学个遍：它怎样决定接下来在 CPU 上运行哪个程序，在虚拟内存（*virtual memory*）系统中怎样应对内存过载，虚拟机监视器（*virtual machine monitor*）怎样工作，怎样管理磁盘上的信息，甚至还会讲一点：部分组件出了故障时，怎样让分布式系统（*distributed system*）照样工作。就是这类东西。

**学生**：说真的，我完全不知道您在说什么。

**教授**：很好！这说明你来对课堂了。

**学生**：我还有一个问题：学这些东西最好的办法是什么？

**教授**：问得好！每个人当然都得自己摸索，不过换作我会这样做：去上课，听教授讲解这些内容。然后，每周结束时读一读这些讲义，让这些思想在脑子里扎得更牢。过一段时间以后（提示：考试之前！），再读一遍讲义来巩固知识。当然，你的教授肯定会布置

一些作业和项目，所以你应该把它们做了；尤其是做项目，写真正的代码去解决真正的问题，这是把讲义中的思想付诸实践的最好办法。正如孔子所说……

**学生**：哦，我知道！“我听了就忘了，我看了就记住了，我做了就明白了。”大概是这样吧。

**教授**：（吃惊地）你怎么知道我要说什么?!

**学生**：顺着话头就猜到了。而且我是孔子的忠实读者，更是荀子的忠实读者；其实荀子才是这句话更准确的出处[1]。

**教授**：（惊呆了）嗯，我想我们一定会相处得很好！真的会很好。

**学生**：教授，如果可以的话，就再问一个问题。这些对话是干什么用的？我是说，这不就是一本书吗？为什么不直接讲内容呢？

**教授**：啊，问得好，问得好！嗯，我觉得有时候跳出叙述、思考一下是有好处的；这些对话就是这样的时刻。所以，你和我要一起努力，把这些相当复杂的思想弄明白。你准备好了吗？

**学生**：所以我们得动脑子？嗯，我准备好了。我是说，不然我还能干什么呢？除了这本书，我也没什么别的生活。

**教授**：遗憾的是，我也没有。那我们开始干活吧！

[1]据 http://www.barrypopik.com（2012 年 12 月 19 日，题为“Tell me and I forget; teach me and I may remember; involve me and I will learn”），儒家思想家荀子说过：“不闻不若闻之，闻之不若见之，见之不若知之，知之不若行之。”后来，不知为什么，这句智慧之言被安到了孔子头上。感谢 Jiao Dong（罗格斯大学）告诉我们这件事！

 © 2008–25, ARPACI-DUSSEAU

# 2

# 操作系统简介

如果你正在上本科的操作系统课，你应该已经大致知道计算机程序运行时在做什么了。如果不知道，这本书（以及对应的课程）会很难学。所以你也许该先放下这本书，或者赶紧跑去最近的书店，把必要的背景知识补上再回来（Patt 和 Patel 的书 [PP03]、Bryant 和 O'Hallaron 的书 [BOH10] 都相当不错）。

那么，程序运行时会发生什么？

嗯，运行中的程序只做一件非常简单的事：执行指令。处理器从内存中**取**（fetch）出一条指令，对它**译码**（decode），弄清楚这是哪条指令，然后**执行**（execute）它，也就是做这条指令该做的事，比如把两个数相加、访问内存、检查某个条件、跳转到某个函数，等等。执行完这条指令后，处理器接着处理下一条指令，如此往复，每秒钟数百万次（如今甚至数十亿次），直到程序最终结束[1]。

这就是我们刚刚描述的**冯·诺依曼**（Von Neumann）计算模型的基本内容[2]。听起来很简单，对吧？但在这门课里我们会了解到，程序运行时还有许多别的事情在热火朝天地进行着，主要目的就是让系统**易于使用**（easy to use）。

事实上，有一组软件专门负责这些事：让程序运行起来很容易（甚至让你仿佛能同时运行很多程序），让程序共享内存，让程序能和设备交互，还有其他诸如此类的有趣事情。这组软件叫作**操作系统**（operating system，**OS**）[3]，因为它要保证系统运转得正确、高效，而且好用。

操作系统做到这一点的主要途径，是一种我们称为**虚拟化**（virtualization）的通用技术。也就是说，操作系统拿来一个**物理**（physical）资源（比如处理器、内存或者磁盘），把它变成一个更通用、更强大、更易用的**虚拟**（virtual）形式。因此，我们有时把操作系统称为**虚拟机**（virtual machine）。

当然，为了让用户能告诉操作系统做什么，从而使用虚拟机的各种功能（比如运行程序、分配内存或访问文件），操作系统还提供了一些可供调用的接口（API）。事实上，一个典型的操作系统会向应用程序提供几百个**系统调用**（system call）。由于操作系统提供这些调用

[1]当然，现代处理器为了让程序跑得更快，在底层做了许多稀奇古怪、让人心惊的事情，比如同时执行多条指令，甚至乱序发出和完成指令！不过这些不是我们这里关心的；我们只关心大多数程序所假定的那个简单模型：指令看上去是一条接一条、井然有序地依次执行的。

[2]冯·诺依曼是计算机系统的早期先驱之一。他在博弈论和原子弹方面也做过开创性的工作，还在 NBA 打过六年球。好吧，这几件事里有一件不是真的。

[3]操作系统早期还有一个名字叫**监督程序**（supervisor），甚至叫**主控程序**（master control program）。显然，后一个名字听起来有点过于野心勃勃（详见电影《Tron》），所以谢天谢地，最后流行开来的是“操作系统”这个名字。

来运行程序、访问内存和设备，以及做其他相关的事，我们有时也说操作系统为应用程序提供了一个**标准库**（standard library）。

最后，虚拟化让许多程序得以运行（从而共享 CPU）、并发地（concurrently）访问各自的指令和数据（从而共享内存）、访问设备（从而共享磁盘等），所以操作系统有时也被称为**资源管理器**（resource manager）。CPU、内存和磁盘都是系统的**资源**（resource）；因此操作系统的职责就是**管理**（manage）这些资源，并且要做得高效、公平，或者兼顾其他许多可能的目标。为了更好地理解操作系统的职责，我们来看几个例子。

关键问题：
如何虚拟化资源

本书要回答的一个核心问题非常简单：操作系统如何虚拟化资源？这就是我们问题的关键。操作系统为什么要这样做不是主要问题，答案应该显而易见：这让系统更容易使用。因此，我们关注的是怎么做：操作系统实现了哪些机制和策略来做到虚拟化？操作系统怎样高效地做到这一点？需要哪些硬件支持？

我们会用像这样带底色的“关键问题”框，标出构建操作系统时要解决的具体问题。在讲某个主题的讲义里，你可能会看到一个或多个关键问题（cruces，没错，这才是它正确的复数形式），它们点出了要解决的问题。当然，章节里的细节会给出解决方案，或者至少给出解决方案的基本要素。

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/time.h>
4  #include <assert.h>
5  #include "common.h"
6
7  int
8  main(int argc, char *argv[])
9  {
10     if (argc != 2) {
11         fprintf(stderr, "usage: cpu <string>\n");
12         exit(1);
13     }
14     char *str = argv[1];
15     while (1) {
16         Spin(1);
17         printf("%s\n", str);
18     }
19     return 0;
20 }
```

图 2.1：**简单的例子：循环打印的代码（cpu.c）**

 © 2008–25, ARPACI-DUSSEAU

## 2.1 虚拟化 CPU

图 2.1 是我们的第一个程序。它做的事不多。实际上，它只是调用 `Spin()`，这个函数反复检查时间，运行满一秒后就返回。然后，程序打印出用户在命令行中传入的字符串，并永远这样重复下去。

假设我们把这个文件保存为 `cpu.c`，然后在只有一个处理器（我们有时也叫它 **CPU**）的系统上编译并运行它。我们会看到这样的结果：

```
prompt> gcc -o cpu cpu.c -Wall
prompt> ./cpu "A"
A
A
A
A
^C
prompt>
```

这次运行没什么意思：系统开始运行程序，程序反复检查时间，直到过了一秒。一秒过后，代码打印出用户传入的字符串（在这个例子中是字母“A”），然后继续。注意，这个程序会永远运行下去；按下“Control-c”（在基于 UNIX 的系统上，这会终止在前台运行的程序）就能让它停下来。

现在做同样的事，不过这次运行同一个程序的多个实例。这个例子稍微复杂一点，结果见图 2.2。

嗯，现在事情变得有意思一点了。尽管只有一个处理器，这四个程序却似乎都在同时运行！这个魔法是怎么变出来的[4]?

```
prompt> ./cpu A & ./cpu B & ./cpu C & ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
...
```

图 2.2：**同时运行多个程序**

[4]注意，我们是用 & 符号同时运行四个进程（process）的。在 zsh shell 中，这样做会把作业放到后台运行，于是用户可以马上发出下一条命令，在这个例子中就是再运行一个程序。如果你用的是别的 shell（例如 tcsh），做法会略有不同；详情请查阅网上的文档。

 © 2008–25, ARPACI-DUSSEAU

原来，这是操作系统在硬件的一点帮助下营造出来的**假象**（illusion）：系统仿佛拥有大量虚拟 CPU（virtual CPU）。把一个 CPU（或者少数几个 CPU）变成看似无限多个 CPU，从而让许多程序看起来同时在运行，这就是我们所说的**虚拟化 CPU**（virtualizing the CPU），它是本书第一大部分的重点。

当然，要运行程序、停止程序，或者以其他方式告诉操作系统运行哪些程序，就需要一些接口（API），让你把自己的意图传达给操作系统。整本书我们都会谈到这些 API；事实上，它们是大多数用户与操作系统打交道的主要方式。

你也许还会注意到，能同时运行多个程序，带来了各种新问题。例如，如果两个程序都想在某个时刻运行，应该让哪个运行？这个问题由操作系统的**策略**（policy）来回答；操作系统中许多不同的地方都用策略来处理这类问题，所以在学习操作系统实现的各种基本**机制**（mechanism），比如同时运行多个程序的能力时，我们也会研究这些策略。这也正是操作系统作为**资源管理器**（resource manager）的职责所在。

## 2.2 虚拟化内存

现在来看内存。现代计算机提供的**物理内存**（physical memory）模型非常简单。内存就是一个字节数组；**读**（read）内存时，必须指定一个**地址**（address），才能访问存放在那里的数据；**写**（write）或**更新**（update）内存时，还必须指定要写到这个地址的数据。

程序运行时，一直在访问内存。程序把它的所有数据结构（data structure）都放在内存中，并通过各种指令访问它们，比如加载和存储指令，或者其他要显式访问内存才能完成工作的指令。别忘了，程序的每一条指令也都在内存中；因此，每次取指令都要访问内存。

```
1  #include <unistd.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include "common.h"
5
6  int
7  main(int argc, char *argv[])
8  {
9      int *p = malloc(sizeof(int));                    // a1
10     assert(p != NULL);
11     printf("(%d) address pointed to by p: %p\n",
12            getpid(), p);                             // a2
13     *p = 0;                                          // a3
14     while (1) {
15         Spin(1);
16         *p = *p + 1;
17         printf("(%d) p: %d\n", getpid(), *p);       // a4
18     }
19     return 0;
20 }
```

图 2.3：**访问内存的程序（`mem.c`）**

```
prompt> ./mem & ./mem &
[1] 24113
[2] 24114
(24113) address pointed to by p: 0x200000
(24114) address pointed to by p: 0x200000
(24113) p: 1
(24114) p: 1
(24114) p: 2
(24113) p: 2
(24113) p: 3
(24114) p: 3
(24113) p: 4
(24114) p: 4
...
```

图 2.4：**多次运行访问内存的程序**

我们来看一个通过调用 `malloc()` 分配内存的程序（见图 2.3）。这个程序的输出如下：

```
prompt> ./mem
(2134) address pointed to by p: 0x200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```

这个程序做了几件事。首先，它分配了一些内存（a1 行）。然后，它打印出这块内存的地址（a2），再把数字零放进新分配内存的第一个位置（a3）。最后，它进入循环，每次延迟一秒，然后把 `p` 所保存的地址处的值加一。每条打印语句还会打印出运行中程序的所谓进程标识符（process identifier，PID）。每个运行中的进程都有唯一的 PID。

同样，第一次的结果没什么意思。新分配的内存位于地址 `0x200000`。程序运行时，会慢慢地更新这个值，并打印出结果。

现在，我们再次运行同一个程序的多个实例，看看会发生什么（图 2.4）。从这个例子可以看到，每个运行中的程序都在同一个地址（`0x200000`）分配了内存，但每个程序似乎都在独立地更新 `0x200000` 处的值！就好像每个运行中的程序都有自己的私有内存，而不是和其他运行中的程序共享同一块物理内存[5]。

事实上，这正是操作系统**虚拟化内存**（virtualizing memory）时发生的事情。每个进程访问的是它自己私有的**虚拟地址空间**（virtual address space，有时简称**地址空间**，address space），操作系统以某种方式把它映射到机器的物理内存上。一个运行中程序的内存引用，不会影响其他进程或操作系统本身的地址空间；在这个运行中的程序看来，整个物理内存都

[5]要让这个例子成立，你需要确保关闭了地址空间随机化（address-space randomization）；随机化其实可以很好地防御某些类型的安全漏洞。如果你想学怎样用栈溢出（stack-smashing）攻击闯进计算机系统，就更值得自己去多了解一下。我们可不是在推荐这么做……

```c
#include <stdio.h>
#include <stdlib.h>
#include "common.h"
#include "common_threads.h"

volatile int counter = 0;
int loops;

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        counter++;
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "usage: threads <value>\n");
        exit(1);
    }
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);

    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value   : %d\n", counter);
    return 0;
}
```

图 2.5：**多线程程序（`threads.c`）**

归它独享。然而现实是，物理内存是一种共享资源，由操作系统管理。这一切究竟是怎么做到的，也是本书第一部分要讨论的内容，这一部分讲的就是**虚拟化**（virtualization）。

## 2.3 并发

本书的另一个主题是**并发**（concurrency）。我们用这个概念性的术语来指一类问题：同一个程序同时（即并发地，concurrently）处理许多事情时，就会出现这类问题，而且必须解决。并发问题最早出现在操作系统内部；从上面虚拟化的例子可以看到，操作系统同时应付着许多事情，先运行一个进程，再运行另一个，如此往复。而这样做，会引出一些深刻而有趣的问题。

遗憾的是，并发问题已经不再局限于操作系统本身。事实上，现代**多线程**（multi-threaded）程序也有同样的问题。我们用一个**多线程**程序的例子来演示一下（图 2.5）。

 © 2008–25, ARPACI-DUSSEAU

虽然你现在可能还不能完全理解这个例子（后面并发部分的章节会讲很多相关内容），但基本思想很简单。主程序用 `Pthread_create()` 创建了两个**线程**（thread）[6]。你可以把线程看成一个函数，它和其他函数运行在同一个内存空间里，而且同一时刻有不止一个这样的函数在活动。在这个例子中，每个线程都从一个叫 `worker()` 的例程开始运行，这个例程只是在循环中把一个计数器递增 `loops` 次。

下面是把变量 `loops` 的输入值设为 1000 时运行这个程序的记录。`loops` 的值决定了两个工作线程（worker thread）各自在循环中递增共享计数器多少次。当 `loops` 设为 1000 时，你预计 `counter` 的最终值是多少？

```
prompt> gcc -o threads threads.c -Wall -pthread
prompt> ./threads 1000
Initial value : 0
Final value   : 2000
```

你可能已经猜到了，两个线程结束时，计数器的最终值是 2000，因为每个线程都把计数器递增了 1000 次。实际上，当 `loops` 的输入值设为 $N$ 时，我们预计程序最终输出 $2N$。但实际情况没这么简单。我们用更大的 `loops` 值运行同一个程序，看看会怎样：

```
prompt> ./threads 100000
Initial value : 0
Final value   : 143012  // 啊??
prompt> ./threads 100000
Initial value : 0
Final value   : 137298  // 这是什么鬼??
```

这次运行中，我们输入的值是 100,000，得到的最终值却不是 200,000，而是 143,012。再运行一次时，我们不仅又得到了错误的值，而且这个值和上一次还不一样。事实上，如果你用很大的 `loops` 值反复运行这个程序，你可能会发现有时甚至能得到正确的答案！为什么会这样呢？

原来，这些奇怪而反常的结果，和指令的执行方式有关：指令是一次执行一条的。遗憾的是，上面程序中的关键部分，也就是递增共享计数器的地方，需要三条指令：一条把计数器的值从内存加载到寄存器，一条把它加一，一条把它存回内存。因为这三条指令不是**原子地**（atomically）执行的（即不是一下子全部执行完），所以会发生奇怪的事情。我们将在本书第二部分详细讨论的，正是这种**并发**（concurrency）问题。

关键问题：
如何构建正确的并发程序

同一个内存空间里有许多线程并发执行，怎样才能构建出能正确工作的程序？需要操作系统提供哪些原语？硬件应该提供哪些机制？怎样用它们来解决并发问题？

[6] 实际应该调用的是小写的 `pthread_create()`；大写的版本是我们自己写的包装函数：它调用小写的版本，并检查返回码，确保调用成功。详见代码。

## 2.4 持久性

这门课的第三个主题是**持久性**（persistence）。在系统内存中，数据很容易丢失，因为 DRAM 这类设备存储数据的方式是**易失**（volatile）的；一旦断电（power loss）或者系统崩溃（system crash），内存中的所有数据都会丢失。因此，我们需要能够**持久地**（persistently）存储数据的硬件和软件；这种存储对任何系统都至关重要，因为用户非常在乎自己的数据。

硬件方面是某种**输入/输出**（input/output，**I/O**）设备；在现代系统中，**硬盘驱动器**（hard drive）是存放长期信息的常用仓库，不过**固态硬盘**（solid-state drive，**SSD**）在这个领域也在迎头赶上。

操作系统中通常负责管理磁盘的那部分软件叫作**文件系统**（file system）；用户创建的所有**文件**（file），都由它负责可靠、高效地存放到系统的磁盘上。

和操作系统为 CPU 与内存提供的抽象（abstraction）不同，操作系统并不会为每个应用程序都创建一个私有的虚拟磁盘。相反，它假定用户常常想要**共享**（share）文件中的信息。例如，写 C 程序时，你可能先用编辑器（例如 Emacs[7]）创建并编辑 C 文件（`emacs -nw main.c`）。完成后，你可能会用编译器把源代码变成可执行文件（例如 `gcc -o main main.c`）。最后，你可能会运行这个新的可执行文件（例如 `./main`）。这样，你就能看到文件是怎样在不同进程之间共享的。首先，Emacs 创建了一个文件，作为编译器的输入；编译器用这个输入文件创建出一个新的可执行文件（中间要经过很多步骤，详情请去上编译原理课）；最后，这个新的可执行文件运行了起来。于是，一个新程序就诞生了！

为了更好地理解这一点，我们来看一些代码。图 2.6 中的代码创建了一个文件（`/tmp/file`），里面包含字符串“hello world”。

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <assert.h>
4  #include <fcntl.h>
5  #include <sys/types.h>
6
7  int main(int argc, char *argv[]) {
8      int fd = open("/tmp/file",
9                    O_WRONLY|O_CREAT|O_TRUNC,
10                   S_IRWXU);
11     assert(fd > -1);
12     int rc = write(fd, "hello world\n", 12);
13     assert(rc == 12);
14     close(fd);
15     return 0;
16 }
```

图 2.6：**执行 I/O 的程序（`io.c`）**

[7]你应该用 Emacs。如果你用的是 vi，那你大概有点什么问题。如果你用的根本不是一个真正的代码编辑器，那就更糟了。

 © 2008–25, ARPACI-DUSSEAU

为了完成这项任务，程序向操作系统发出了三次调用。第一次调用 `open()`，打开并创建这个文件；第二次调用 `write()`，向文件写入一些数据；第三次调用 `close()`，只是关闭文件，表示程序不会再向它写入数据了。这些**系统调用**（system call）会转给操作系统中叫作**文件系统**（file system）的那部分，由它处理请求，并向用户返回某种错误码。

你可能想知道，操作系统为了真正写入磁盘都做了些什么。我们可以给你看，但你得先保证闭上眼睛；这个过程就是这么难看。文件系统要做不少工作：首先要弄清楚这些新数据放在磁盘上的什么位置，然后在文件系统维护的各种结构中记录下来。要做到这些，就需要向底层存储设备（storage device）发出 I/O 请求，要么读取已有的结构，要么更新（写入）它们。写过**设备驱动程序**（device driver）[8]的人都知道，让设备替你做点事情，是一个复杂而琐碎的过程。它要求你对底层设备接口及其确切语义有深入的了解。幸运的是，操作系统通过系统调用，提供了一种标准而简单的访问设备的方式。因此，操作系统有时被看作一个**标准库**（standard library）。

当然，关于怎样访问设备、文件系统怎样在设备之上持久地管理数据，还有很多细节。出于性能考虑，大多数文件系统会先把这类写操作推迟一段时间，希望把它们合成更大的批次。为了应对写入过程中系统崩溃的问题，大多数文件系统都采用了某种复杂的写入协议（protocol），比如**日志**（journaling）或**写时复制**（copy-on-write），仔细安排写入磁盘的顺序，确保写入过程中一旦出现故障，系统之后能够恢复到合理的状态。为了让各种常见操作都很高效，文件系统采用了许多不同的数据结构和访问方法（access method），从简单的列表到复杂的 B 树（B-tree）。如果你现在还看不太懂这些，很好！我们会在本书关于**持久性**（persistence）的第三部分详细讨论这些内容：先讨论一般的设备和 I/O，再详细讨论磁盘、RAID 和文件系统。

关键问题：
如何持久地存储数据
文件系统是操作系统中负责管理持久数据的部分。要正确地做到这一点，需要哪些技术？要获得高性能，需要哪些机制和策略？面对硬件和软件故障，怎样实现可靠性？

## 2.5 设计目标

现在你已经大致知道操作系统实际上做什么了：它拿来 CPU、内存或磁盘这样的物理**资源**（resource），把它们**虚拟化**（virtualize）。它处理与**并发**（concurrency）有关的棘手问题。它还**持久地**（persistently）存储文件，从而让文件长期保持安全。既然我们想构建这样一个系统，就需要心里有一些目标，帮助我们聚焦设计与实现，并在必要时做出权衡；找到恰当的一组权衡，是构建系统的关键。

最基本的目标之一，是建立一些**抽象**（abstraction），让系统方便、易用。我们在计算机科学里做的一切，都以抽象为基础。有了抽象，我们才能把一个大程序拆成许多小而易懂

[8] 设备驱动程序是操作系统中的一段代码，它知道如何与某个特定的设备打交道。后面我们会进一步讨论设备和设备驱动程序。

的部分来编写；才能用 C[9] 这样的高级语言写程序，而不必考虑汇编；才能用汇编写代码，而不必考虑逻辑门；才能用逻辑门搭出处理器，而不必过多考虑晶体管。抽象太基本了，基本到我们有时会忘了它有多重要；但在这里我们不会忘。因此，在每一部分里，我们都会讨论一些随着时间发展起来的主要抽象，给你一种思考操作系统各个部分的方式。

设计和实现操作系统的一个目标是提供高**性能**（performance）；换一种说法，我们的目标是**尽量减少操作系统的开销**（minimize the overheads）。虚拟化和让系统易于使用是非常值得的，但不能不惜任何代价；因此，我们必须设法在开销不太大的前提下，提供虚拟化和其他操作系统功能。这些开销有多种形式：额外的时间（更多的指令）和额外的空间（内存中或磁盘上）。只要可能，我们就会寻找能减少其中一种开销、最好两种都减少的方案。不过，完美并不总是能够达到的；我们要学会察觉这一点，并（在合适的时候）容忍它。

另一个目标是在应用程序之间以及操作系统和应用程序之间提供**保护**（protection）。因为我们希望让许多程序同时运行，所以要确保其中一个程序的恶意或意外的不良行为不会伤害其他程序；我们当然更不希望某个应用程序能够伤害操作系统本身（因为那会影响系统上运行的所有程序）。保护是操作系统背后一条主要原则的核心，那就是**隔离**（isolation）；把进程彼此隔离开来是实现保护的关键，因此也是操作系统必须做的许多事情的基础。

操作系统还必须不间断地运行；一旦它出故障，系统上运行的所有应用程序也都会出故障。正因为有这种依赖，操作系统常常力求提供高度的**可靠性**（reliability）。随着操作系统变得越来越复杂（有时包含数百万行代码），构建一个可靠的操作系统是相当大的挑战；事实上，这个领域正在进行的许多研究（包括我们自己的一些工作 [BS+09, SS+10]）关注的正是这个问题。

还有其他一些合理的目标：在我们这个越来越注重环保的世界里，**能效**（energy-efficiency）很重要；防范恶意应用程序的**安全性**（security）（其实是保护的延伸）至关重要，尤其是在如今这个高度联网的时代；随着操作系统运行在越来越小的设备上，**移动性**（mobility）也越来越重要。根据系统的使用方式，操作系统会有不同的目标，因此实现方式很可能至少会略有不同。不过我们会看到，本书要介绍的许多操作系统构建原则，在各种不同的设备上都很有用。

## 2.6 一点历史

在结束这篇简介之前，我们简要介绍一下操作系统的发展历史。和人类构建的任何系统一样，随着工程师们逐渐了解操作系统设计中什么才是重要的，好的想法也在操作系统中日积月累。这里我们讨论几项主要的发展。想了解更详细的内容，请看 Brinch Hansen 写的那篇出色的操作系统史 [BH00]。

[9] 你们中有人可能会反对把 C 叫作高级语言。不过别忘了，这是一门操作系统课，在这里我们只要不用一直写汇编，就已经很开心了！

### 早期操作系统：只是一些库

起初，操作系统做的事情不多。基本上，它只是一组常用函数的库；例如，“操作系统”会提供底层 I/O 处理的 API，省得系统中的每个程序员都自己去写这类代码，开发者的日子也就好过一些。

通常，在那些老式的大型机系统上，一次只运行一个程序，由一名人类操作员控制。你以为现代操作系统会做的许多事情（例如决定以什么顺序运行作业），当时都是由这名操作员来做的。如果你是个聪明的开发者，你就会对这位操作员好一点，这样他们也许会把你的作业挪到队伍前面去。

这种计算方式被称为**批处理**（batch processing）：先准备好一些作业，然后由操作员把它们成“批”地运行。那时候出于成本考虑，计算机不是交互式使用的：让一个用户坐在计算机前使用它实在太贵了，因为大部分时间它只会闲着，而机房每小时要为此花费数十万美元 [BH00]。

### 超越库：保护

后来，操作系统不再只是一个提供常用服务的简单库，而是在管理机器方面扮演了更核心的角色。其中一个重要方面，是人们意识到代表操作系统运行的代码是特殊的：它控制着设备，因此应当和普通的应用程序代码区别对待。为什么呢？嗯，想象一下，如果你允许任何应用程序读取磁盘上的任何地方，那隐私就无从谈起了，因为任何程序都可以读取任何文件。因此，把用来管理你的文件的**文件系统**（file system）实现成一个库，是没什么道理的。我们需要别的东西。

于是，人们发明了**系统调用**（system call），最先采用它的是 Atlas 计算系统 [K+61, L78]。它的思路是：不把操作系统的例程做成库、让程序一次过程调用（procedure call）就能访问，而是增加一对特殊的硬件指令和硬件状态，让进入操作系统变成一个更正式、受控的过程。

系统调用和**过程调用**（procedure call）的关键区别在于：系统调用在把控制权转移（即跳转）到操作系统的同时，还会提升**硬件特权级**（hardware privilege level）。用户应用程序运行在所谓的**用户模式**（user mode）下，这意味着硬件会限制应用程序能做的事情；例如，运行在用户模式下的应用程序通常不能向磁盘发起 I/O 请求、访问任意物理内存页，或者在网络上发送数据包（packet）。发起系统调用时（通常是通过一条叫作**陷阱**（trap）的特殊硬件指令），硬件把控制权转移到操作系统事先设置好的**陷阱处理程序**（trap handler），同时把特权级提升到**内核模式**（kernel mode）。在内核模式下，操作系统可以完全访问系统的硬件，因此可以做诸如发起 I/O 请求或者给程序分配更多内存之类的事情。操作系统处理完请求后，执行一条特殊的**从陷阱返回**（return-from-trap）指令，把控制权交还给用户程序：这条指令切换回用户模式，同时让应用程序从刚才停下的地方继续执行。

### 多道程序设计时代

操作系统真正腾飞，是在大型机之后的计算时代，也就是**小型机**（minicomputer）时代。像 Digital Equipment 公司的 PDP 系列这样的经典机器，让计算机变得便宜多了；于是，不再是一个大机构共用一台大型机，机构里的一个小团队就很可能拥有自己的计算机。

 © 2008–25, ARPACI-DUSSEAU

毫不意外，成本下降的一大影响是开发活动多了起来：更多聪明人用上了计算机，让计算机系统做出了更多有趣而漂亮的事情。

特别是，人们希望更好地利用机器资源，于是**多道程序设计**（multiprogramming）变得十分普遍。操作系统不再一次只运行一个作业，而是把多个作业装入内存，在它们之间快速切换，从而提高 CPU 利用率。这种切换尤其重要，因为 I/O 设备很慢；让程序在 I/O 处理期间占着 CPU 干等，是在浪费 CPU 时间。为什么不切换到另一个作业、让它运行一会儿呢？

为了支持多道程序设计，并在有 I/O 和中断（interrupt）的情况下让各项工作重叠进行，操作系统不得不在许多方面做出概念上的创新。**内存保护**（memory protection）之类的问题变得重要起来；我们不希望一个程序能够访问另一个程序的内存。弄清楚如何处理多道程序设计带来的**并发**（concurrency）问题也至关重要；在有中断的情况下确保操作系统行为正确，是一个很大的挑战。我们会在本书后面研究这些问题以及相关的主题。

那个时期一项重大的实际进展，是 UNIX 操作系统的诞生，这主要归功于贝尔实验室（没错，就是那家电话公司）的 Ken Thompson（和 Dennis Ritchie）。UNIX 从不同的操作系统中吸取了许多好的想法（特别是 Multics [O72]，也有一些来自 TENEX [B+72] 和 Berkeley Time-Sharing System [S68] 这样的系统），但让它们更简单、更易用。很快，这个团队就开始把装有 UNIX 源代码的磁带寄给世界各地的人，其中许多人随后参与进来，自己为系统添砖加瓦；详见下面的**补充**[10]。

### 现代

小型机之后，出现了一种更便宜、更快、面向大众的新型机器：**个人计算机**（personal computer），也就是我们今天所说的 **PC**。在苹果早期机器（例如 Apple II）和 IBM PC 的带领下，这类新机器很快成为计算领域的主导力量，因为它们的低成本让每张桌子上都能有一台机器，而不再是一个工作组共用一台小型机。

遗憾的是，对操作系统来说，PC 起初是一次大倒退，因为早期的系统忘记了（或者根本不知道）小型机时代学到的教训。例如，像 **DOS**（**微软**的 **Disk Operating System**）这样的早期操作系统，并不认为内存保护有多重要；于是，一个恶意的（或者只是写得很差的）应用程序就可以在内存中到处乱写。早期几代 **Mac OS**（v9 及更早版本）在作业调度上采用了协作方式（cooperative approach）；因此，一个不小心陷入无限循环的线程就可能霸占整个系统，迫使系统重启。这一代系统缺失的操作系统功能列出来是一份痛苦而冗长的清单，长到这里没法一一讨论。

幸运的是，在经历了几年的痛苦之后，小型机操作系统的那些老功能开始进入桌面系统。例如，Mac OS X/macOS 以 UNIX 为核心，具备这样一个成熟系统应有的全部功能。Windows 也同样吸收了计算历史上许多好的想法，特别是从 Windows NT 开始，这是微软操作系统技术的一次巨大飞跃。即使是今天的手机，运行的操作系统（例如 Linux）也更像 20 世纪 70 年代小型机上运行的系统，而不像 20 世纪 80 年代 PC 上运行的系统（谢天

[10] 我们会用补充框以及其他相关的文本框，来提醒你注意一些不太适合放进正文主线的内容。有时候，我们甚至只是用它们来讲个笑话，因为一路上找点乐子有何不可呢？没错，很多笑话都挺烂的。

补充：UNIX 的重要性

UNIX 在操作系统历史上的重要性怎么强调都不为过。UNIX 受到更早的一些系统（特别是麻省理工学院著名的 **Multics** 系统）的影响，汇集了许多绝妙的想法，造出了一个既简单又强大的系统。

最初的“贝尔实验室”UNIX 背后，有一条统一的原则：构建一些小而强大的程序，它们可以连接起来，组成更大的工作流。输入命令用的 **shell** 提供了**管道**（pipe）这样的原语，支持这种元层次的编程，于是把多个程序串起来完成更大的任务就变得很容易。例如，要找出一个文本文件中含有单词“foo”的行，再数一数这样的行有多少，你只需输入：`grep foo file.txt|wc -l`，就用 `grep` 和 `wc`（word count，统计字数）这两个程序完成了任务。

UNIX 环境对程序员和开发者都很友好，还为新的 **C 编程语言**提供了编译器。程序员能方便地写出自己的程序并分享出去，这让 UNIX 大受欢迎。作者们把副本免费送给任何索要的人（这是**开源软件**（open-source software）的一种早期形式），这大概也起了很大作用。

同样至关重要的是代码的易得性和可读性。一个用 C 写成的漂亮而小巧的内核，吸引着其他人来摆弄它，加入酷炫的新功能。例如，伯克利一个富有进取心的团队，在 **Bill Joy** 的带领下，做出了一个很棒的发行版（**Berkeley Systems Distribution**，即 **BSD**），它有一些先进的虚拟内存（virtual memory）、文件系统和网络子系统。Joy 后来与人共同创立了 **Sun Microsystems**。

遗憾的是，随着各家公司试图主张所有权、从中牟利，UNIX 的传播放慢了一些；这是律师介入之后不幸（但常见）的结果。许多公司都有自己的变体：Sun Microsystems 的 **SunOS**、IBM 的 **AIX**、HP 的 **HPUX**（又名“H-Pucks”），以及 SGI 的 **IRIX**。AT&T/贝尔实验室和这些公司之间的法律纠纷给 UNIX 蒙上了一层阴影，许多人怀疑它能否存活下来，特别是在 Windows 问世并占领了大部分个人计算机（personal computer）市场之后……

谢地）；看到操作系统发展鼎盛时期诞生的好想法进入了现代世界，真让人高兴。更好的是，这些想法还在不断发展，提供更多功能，让现代系统对用户和应用程序都更好。

## 2.7 小结

以上就是对操作系统的简介。如今的操作系统让系统相对容易使用，而你今天用到的几乎所有操作系统，都受到了本书将要讨论的那些发展的影响。

遗憾的是，由于时间所限，操作系统中有许多部分本书不会涉及。例如，操作系统中有大量**网络**（networking）代码；这方面的内容，留给你去网络课上学习。同样，**图形**（graphics）设备也特别重要；去上图形学课，拓展你在这方面的知识。最后，有些操作系统书会花大量篇幅讨论**安全**（security）；我们也会讨论，但只讨论这个意义上的安全：操作系统必须在运行的程序之间提供保护，并让用户能够保护自己的文件。你在安全课上可能会学到的那些更深层的安全问题，我们不会深入。

 © 2008–25, ARPACI-DUSSEAU

补充：后来有了 Linux

UNIX 很幸运：一位名叫 **Linus Torvalds** 的年轻芬兰黑客决定写一个自己的 UNIX。它大量借鉴了原系统背后的原则和思想，但没有借用原来的代码，从而避开了法律问题。他争取到了世界各地许多人的帮助，利用了当时已经存在的成熟 GNU 工具 [G85]，很快，**Linux** 就诞生了（现代开源软件运动也随之诞生）。

进入互联网时代后，大多数公司（例如 Google、Amazon、Facebook 等）都选择运行 Linux，因为它免费，而且可以很方便地修改以满足自己的需要；事实上，很难想象如果没有这样一个系统，这些新公司还能取得成功。当智能手机成为面向用户的主导平台时，出于许多相同的原因，Linux 在那里也站稳了脚跟（通过 Android）。Steve Jobs 则把他基于 UNIX 的 **NeXTStep** 操作环境带到了苹果，从而让 UNIX 在桌面上流行起来（虽然许多苹果技术的用户大概根本不知道这一点）。因此，UNIX 延续至今，而且比以往任何时候都更重要。如果你信计算之神的话，真该为这个美好的结局感谢它们。

不过，我们会讲到许多重要的主题，包括 CPU 和内存虚拟化的基础知识、并发，以及通过设备和文件系统实现的持久性。别担心！虽然要讲的东西很多，但大部分都相当酷，等走到这条路的尽头，你会对计算机系统真正如何工作有全新的认识。现在，开始干活吧！

 © 2008–25, ARPACI-DUSSEAU

## 参考文献

[BS+09] "Tolerating File-System Mistakes with EnvyFS" by L. Bairavasundaram, S. Sundararaman, A. Arpaci-Dusseau, R. Arpaci-Dusseau. USENIX '09, San Diego, CA, June 2009. *A fun paper about using multiple file systems at once to tolerate a mistake in any one of them.*

[BH00] "The Evolution of Operating Systems" by P. Brinch Hansen. In 'Classic Operating Systems: From Batch Processing to Distributed Systems.' Springer-Verlag, New York, 2000. *This essay provides an intro to a wonderful collection of papers about historically significant systems.*

[B+72] "TENEX, A Paged Time Sharing System for the PDP-10" by D. Bobrow, J. Burchfiel, D. Murphy, R. Tomlinson. CACM, Volume 15, Number 3, March 1972. *TENEX has much of the machinery found in modern operating systems; read more about it to see how much innovation was already in place in the early 1970's.*

[B75] "The Mythical Man-Month" by F. Brooks. Addison-Wesley, 1975. *A classic text on software engineering; well worth the read.*

[BOH10] "Computer Systems: A Programmer's Perspective" by R. Bryant and D. O'Hallaron. Addison-Wesley, 2010. *Another great intro to how computer systems work. Has a little bit of overlap with this book — so if you'd like, you can skip the last few chapters of that book, or simply read them to get a different perspective on some of the same material. After all, one good way to build up your own knowledge is to hear as many other perspectives as possible, and then develop your own opinion and thoughts on the matter. You know, by thinking!*

[G85] "The GNU Manifesto" by R. Stallman. 1985. www.gnu.org/gnu/manifesto.html. *A huge part of Linux's success was no doubt the presence of an excellent compiler,* gcc*, and other relevant pieces of open software, thanks to the GNU effort headed by Stallman. Stallman is a visionary when it comes to open source, and this manifesto lays out his thoughts as to why.*

[K+61] "One-Level Storage System" by T. Kilburn, D.B.G. Edwards, M.J. Lanigan, F.H. Sumner. IRE Transactions on Electronic Computers, April 1962. *The Atlas pioneered much of what you see in modern systems. However, this paper is not the best read. If you were to only read one, you might try the historical perspective below [L78].*

[L78] "The Manchester Mark I and Atlas: A Historical Perspective" by S. H. Lavington. Communications of the ACM, Volume 21:1, January 1978. *A nice piece of history on the early development of computer systems and the pioneering efforts of the Atlas. Of course, one could go back and read the Atlas papers themselves, but this paper provides a great overview and adds some historical perspective.*

[O72] "The Multics System: An Examination of its Structure" by Elliott Organick. MIT Press, 1972. *A great overview of Multics. So many good ideas, and yet it was an over-designed system, shooting for too much, and thus never really worked. A classic example of what Fred Brooks would call the "second-system effect" [B75].*

[PP03] "Introduction to Computing Systems: From Bits and Gates to C and Beyond" by Yale N. Patt, Sanjay J. Patel. McGraw-Hill, 2003. *One of our favorite intro to computing systems books. Starts at transistors and gets you all the way up to C; the early material is particularly great.*

[RT74] "The UNIX Time-Sharing System" by Dennis M. Ritchie, Ken Thompson. CACM, Volume 17: 7, July 1974. *A great summary of* UNIX *written as it was taking over the world of computing, by the people who wrote it.*

[S68] "SDS 940 Time-Sharing System" by Scientific Data Systems. TECHNICAL MANUAL, SDS 90 11168, August 1968. *Yes, a technical manual was the best we could find. But it is fascinating to read these old system documents, and see how much was already in place in the late 1960's. One of the minds behind the Berkeley Time-Sharing System (which eventually became the SDS system) was Butler Lampson, who later won a Turing award for his contributions in systems.*

[SS+10] "Membrane: Operating System Support for Restartable File Systems" by S. Sundararaman, S. Subramanian, A. Rajimwale, A. Arpaci-Dusseau, R. Arpaci-Dusseau, M. Swift. FAST '10, San Jose, CA, February 2010. *The great thing about writing your own class notes: you can advertise your own research. But this paper is actually pretty neat — when a file system hits a bug and crashes, Membrane auto-magically restarts it, all without applications or the rest of the system being affected.*

 © 2008–25, ARPACI-DUSSEAU

## 作业

本书的大多数章节（最终会是所有章节）末尾都有作业。做这些作业很重要，因为每份作业都能让你（读者）对本章介绍的概念有更多实际体验。

作业有两类。第一类基于**模拟**（simulation）。计算机系统的模拟只是一个简单的程序，它假装完成真实系统所做的一些有趣的工作，然后报告一些输出指标，展示系统的行为。例如，一个硬盘模拟器可以接收一系列请求，模拟它们在具有某种性能特征的硬盘上需要多长时间才能完成，然后报告这些请求的平均延迟。

模拟的妙处在于，它让你能轻松地探索系统的行为，而不必费力去运行一个真实的系统。事实上，它甚至能让你造出现实世界中不可能存在的系统（例如快得难以想象的硬盘），从而看到未来技术可能带来的影响。

当然，模拟也有缺点。就其本质而言，模拟只是对真实系统行为的近似。如果漏掉了现实行为中某个重要的方面，模拟就会报告出错误的结果。因此，对模拟的结果总要抱有几分怀疑。归根结底，真正重要的是系统在现实世界中的表现。

第二类作业需要和**真实世界的代码**（real-world code）打交道。其中有些作业侧重测量，另一些只需要做一点小规模的开发和实验。这两种作业都只是对一个更大世界的小小探索，而这个世界正是你应该走进去的：如何在基于 UNIX 的系统上用 C 编写系统代码。事实上，想在这个方向上更进一步，就需要做比这些作业规模更大的项目；因此，除了做作业，我们强烈建议你做一些项目，来巩固你的系统技能。一些项目见这个页面（https://github.com/remzi-arpacidusseau/ostep-projects）。

要做这些作业，你很可能需要一台基于 UNIX 的机器，运行 Linux、macOS 或类似的系统。它还应该装有 C 编译器（例如 `gcc`）和 Python。你还应该会用某种真正的代码编辑器来编辑代码。

 © 2008–25, ARPACI-DUSSEAU

# 第一部分

# 虚拟化

# 3

# 关于虚拟化的对话

**教授**：那么，我们就从操作系统三个部分中的第一部分讲起：**虚拟化**（*virtualization*）。

**学生**：可是什么是虚拟化呢，尊敬的教授？

**教授**：想象一下，我们有一个桃子。

**学生**：一个桃子？（难以置信）

**教授**：对，一个桃子。我们把它叫作**物理**（*physical*）桃子。可是有很多人都想吃这个桃子。我们希望给每个吃桃子的人都端上一个他们自己的桃子，让大家都高兴。我们把给他们的桃子叫作**虚拟**（*virtual*）桃子；我们设法用这一个物理桃子造出许多个虚拟桃子。重要的是：在这个假象里，每个吃桃子的人看上去都拥有一个物理桃子，但实际上并没有。

**学生**：所以大家在分着吃这个桃子，自己却根本不知道？

**教授**：对！正是如此。

**学生**：可是只有一个桃子啊。

**教授**：对。然后呢……？

**学生**：嗯，如果我在和别人分一个桃子，我想我会注意到的。

**教授**：啊，没错！说得好。不过吃桃子的人多了就是这样：他们大多数时候都在打盹或者忙别的，所以你可以把桃子拿走，给别人吃一会儿。这样我们就营造出了许多个虚拟桃子的假象，每人一个桃子！

**学生**：听起来像个糟糕的竞选口号。您说的是计算机，对吧，教授？

**教授**：啊，小蚱蜢，你想要一个更具体的例子。好主意！我们拿最基本的资源，也就是 *CPU* 来说。假设系统里有一个物理 *CPU*（虽然现在常常有两个、四个甚至更多）。虚拟化做的事，就是让系统上运行的应用程序觉得，这一个 *CPU* 是许多个虚拟 *CPU*（*virtual CPU*）。这样，虽然每个应用程序都以为自己有一个 *CPU* 可用，实际上却只有一个。于是操作系统营造出了一个美丽的假象：它把 *CPU* 虚拟化了。

**学生**：哇！听起来像魔法一样。再多讲讲！这是怎么做到的？

**教授**：到时候就知道了，年轻的学生，别急。看来你已经准备好开始了。

**学生**：准备好了！嗯，算是吧。我得承认，我有点担心您又要开始讲桃子了。

**教授**：别太担心；我其实根本不喜欢桃子。好了，我们开始吧……

# 4

# 抽象：进程

本章讨论操作系统提供给用户的最基本的抽象（abstraction）之一：**进程**（process）。非正式地说，进程的定义非常简单：它就是一个**运行中的程序**（running program）[V+65, BH70]。程序本身是没有生命的东西：它只是躺在磁盘上，是一堆指令（也许还有一些静态数据），等着被调动起来。正是操作系统拿起这些字节，让它们运行起来，把程序变成有用的东西。

其实，人们常常想同时运行不止一个程序；比如，想想你的台式机或笔记本电脑，你可能想同时运行网页浏览器、邮件程序、游戏、音乐播放器等等。实际上，一个典型的系统看上去可能同时运行着几十甚至上百个进程。这样一来，系统用起来就很方便，因为你永远不必操心有没有空闲的 CPU；你只管运行程序就行。于是，我们的挑战是：

关键问题：
如何提供有许多 CPU 的假象？
尽管只有少数几个物理 CPU 可用，操作系统怎样才能提供 CPU 几乎取之不尽的假象？

操作系统通过**虚拟化**（virtualizing）CPU 来营造这个假象。它运行一个进程，然后停下它、运行另一个进程，如此往复，从而让人觉得存在许多虚拟 CPU（virtual CPU），而实际上只有一个（或少数几个）物理 CPU。这项基本技术叫作 CPU **分时共享**（time sharing），它让用户想运行多少个并发进程（concurrent process）就运行多少个；可能的代价是性能，因为 CPU 一旦要共享，每个进程都会运行得慢一些。

要实现 CPU 的虚拟化，而且要实现得好，操作系统既需要一些底层的机制，也需要一些高层的智能。我们把这些底层的东西称为**机制**（mechanism）；机制是实现某项所需功能的底层方法或协议（protocol）。例如，后面我们会学习如何实现**上下文切换**（context switch），它让操作系统能够在给定的 CPU 上停止运行一个程序、开始运行另一个程序；所有现代操作系统都采用这种分时机制。

操作系统的一部分智能建立在这些机制之上，表现为**策略**（policy）。策略是操作系统中用来做出某种决定的算法。例如，有许多程序都可以在 CPU 上运行，操作系统应该运行哪一个？操作系统中的**调度策略**（scheduling policy）会做出这个决定，做决定时很可能会用到历史信息（例如，在过去一分钟里哪个程序运行得更多？）、工作负载（workload）知

提示：使用分时（和空分）共享

**分时共享**（time sharing）是操作系统共享资源的一项基本技术。让一个实体使用资源一小会儿，再让另一个实体使用一小会儿，如此往复，这个资源（例如 CPU 或网络链路）就能被许多实体共享。与分时共享相对的是**空分共享**（space sharing），即把资源（在空间上）划分给想要使用它的各方。例如，磁盘空间天然就是一种空分共享（space sharing）的资源；一个块一旦分配给某个文件，通常在用户删除原文件之前，不会再分配给别的文件。

识（例如，运行的是哪些类型的程序?）以及性能指标（例如，系统优化的是交互性能还是吞吐量（throughput)?)。

## 4.1 抽象：进程

操作系统为运行中的程序提供的抽象，我们称之为**进程**（process）。前面说过，进程就是一个运行中的程序；在任何时刻，要概括一个进程，只要把它在执行过程中访问或影响到的系统各部分清点一遍就行。

因此，要理解进程由什么构成，就得理解它的**机器状态**（machine state）：程序在运行时能读取或更新的东西。在任何时刻，机器的哪些部分对这个程序的执行是重要的?

构成进程的机器状态（machine state）中，一个显而易见的组成部分是它的*内存*。指令在内存中；运行中的程序读写的数据也在内存中。因此，进程能够寻址的内存（称为它的**地址空间**，address space）是进程的一部分。

进程的机器状态（machine state）还包括*寄存器*；许多指令会显式地读取或更新寄存器，因此寄存器显然对进程的执行很重要。

注意，有一些特别的寄存器也是机器状态（machine state）的一部分。例如，**程序计数器**（program counter，**PC**，有时也叫**指令指针**，instruction pointer，**IP**）告诉我们程序接下来要执行哪条指令；类似地，**栈指针**（stack pointer）和相关的**帧指针**（frame pointer）用来管理存放函数参数、局部变量和返回地址的栈。

最后，程序也经常访问持久存储（persistent storage）设备。这类 *I/O* 信息可能包括进程当前打开的文件列表。

提示：策略与机制分离

在许多操作系统中，一种常见的设计范式是把高层的策略与底层的机制分离开来[L+75]。可以认为，机制回答的是系统*怎么做*的问题；例如，操作系统怎样执行上下文切换? 策略则回答*选哪个*的问题；例如，操作系统现在应该运行哪个进程? 把两者分开，就能轻松地改变策略，而不必重新考虑机制。这是**模块化**（modularity）的一种形式，而模块化是软件设计的一条通用原则。

## 4.2 进程 API

尽管我们要到后面的章节才讨论真正的进程 API（process API），这里先大致介绍一下任何操作系统的接口都必须包含哪些内容。每个现代操作系统都以某种形式提供这些 API。

- **创建**（create）：操作系统必须提供某种创建新进程的方法。当你在 shell 中输入一条命令，或者双击一个应用程序图标时，就会调用操作系统来创建一个新进程，运行你指定的程序。
- **销毁**（destroy）：既然有创建进程的接口，系统也会提供强行销毁进程的接口。当然，许多进程运行完后会自行退出；但如果它们不退出，用户可能想杀掉它们，因此用来终止失控进程的接口非常有用。
- **等待**（wait）：有时，等待一个进程停止运行是有用的；因此通常会提供某种等待接口。
- **其他控制**（miscellaneous control）：除了杀掉或等待进程，有时还可以对进程做其他控制。例如，大多数操作系统都提供某种方法来暂停进程，让它停止运行一段时间，之后再恢复它，让它继续运行。
- **状态**（status）：通常还有一些接口，用来获取进程的状态信息，比如它已经运行了多长时间，或者它处于什么状态。

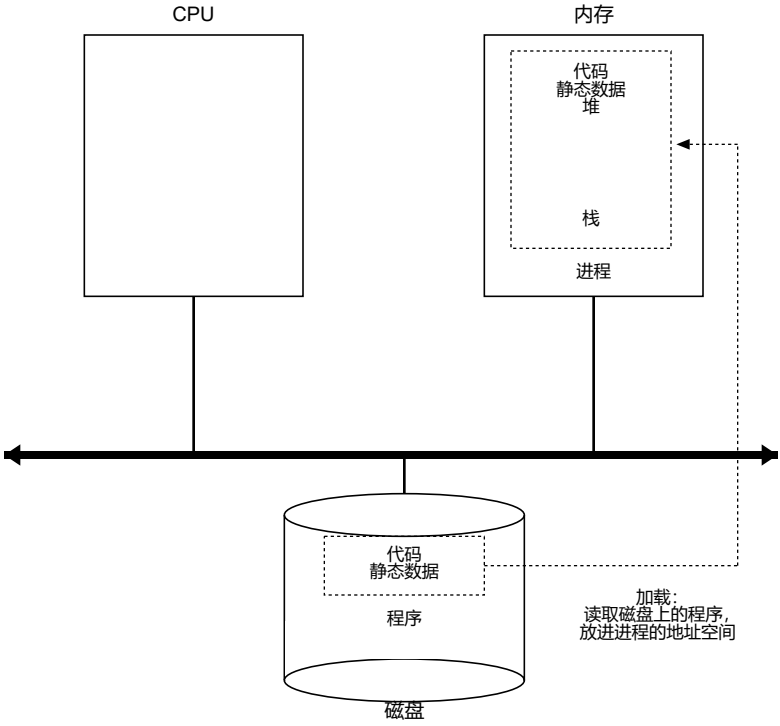



图 4.1：**加载：从程序到进程**

## 4.3 创建进程：再多讲一点

有一个谜团我们应该稍微揭开一下：程序是怎样变成进程的？具体来说，操作系统怎样把一个程序运行起来？进程创建（process creation）实际上是怎么工作的？

要运行一个程序，操作系统首先要做的，是把它的代码和所有静态数据（例如已初始化的变量）**加载**（load）到内存中，放进进程的地址空间。程序最初以某种**可执行格式**（executable format）存放在**磁盘**（disk）上（在一些现代系统中，是**基于闪存的 SSD**，flash-based SSD）；因此，把程序和静态数据加载到内存的过程，需要操作系统从磁盘读出这些字节，放到内存中的某个地方（如图 4.1 所示）。

在早期的（或简单的）操作系统中，加载过程是**立即**（eagerly）完成的，也就是在运行程序之前一次性全部加载；现代操作系统则**惰性地**（lazily）执行这个过程，也就是只在程序执行期间需要某段代码或数据时，才加载那一部分。要真正理解代码和数据的惰性加载是如何工作的，你得更多地了解**分页**（paging）和**交换**（swapping）的机制，这些主题我们会在以后讨论内存虚拟化时讲到。现在只要记住：在运行任何东西之前，操作系统显然都得做一些工作，把程序的重要部分从磁盘弄到内存中。

代码和静态数据加载到内存之后，在运行进程之前，操作系统还需要做几件事。必须为程序的**运行时栈**（run-time stack，或者直接叫**栈**，stack）分配一些内存。你应该已经知道，C 程序用栈来存放局部变量、函数参数和返回地址；操作系统分配这块内存并交给进程。操作系统很可能还会用参数来初始化栈；具体来说，它会填好 `main()` 函数的参数，也就是 `argc` 和 `argv` 数组。

操作系统还可能为程序的**堆**（heap）分配一些内存。在 C 程序中，堆用来存放显式申请的动态分配数据；程序通过调用 `malloc()` 申请这类空间，通过调用 `free()` 显式释放。链表（linked list）、散列表（hash table）、树以及其他有趣的数据结构（data structure）都需要用到堆。堆一开始会很小；程序运行时，会通过 `malloc()` 库 API 申请更多内存，这时操作系统可能会参与进来，为进程分配更多内存，以满足这些调用。

操作系统还会做一些其他的初始化工作，特别是与输入/输出（I/O）有关的工作。例如，在 UNIX 系统中，每个进程默认都打开了三个**文件描述符**（file descriptor），分别对应标准输入（standard input）、标准输出（standard output）和标准错误（standard error）；有了这些描述符，程序就能方便地从终端读取输入，并把输出打印到屏幕上。我们会在本书关于**持久性**（persistence）的第三部分进一步学习 I/O、文件描述符之类的内容。

把代码和静态数据加载到内存，创建并初始化栈，再做一些与 I/O 设置有关的工作，至此，操作系统（终于）为程序的执行搭好了舞台。于是它还剩最后一项任务：从入口点，也就是 `main()`，开始运行程序。操作系统跳转到 `main()` 例程（借助下一章要讨论的一种专门机制），把 CPU 的控制权交给新创建的进程，程序就此开始执行。

## 4.4 进程状态

现在我们已经大致知道了进程是什么（尽管我们还会继续细化这个概念），也大致知道了它是怎样创建的，下面来谈谈某一时刻进程可能处于哪些**状态**（state）。“进程处于某种状

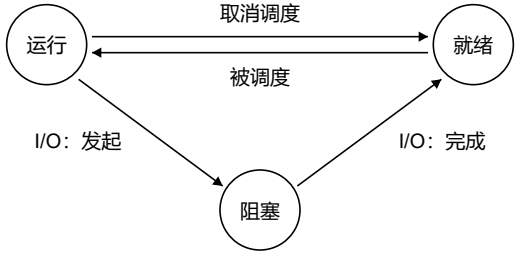



图 4.2：**进程：状态转换**

态”这个概念，出现在早期的计算机系统中 [DV66, V+65]。简单来说，进程可以处于以下三种状态之一：

- **运行**（running）：在运行状态下，进程正在处理器上运行。这意味着它正在执行指令。
- **就绪**（ready）：在就绪状态下，进程已经准备好运行，但出于某种原因，操作系统选择此刻不运行它。
- **阻塞**（blocked）：在阻塞状态下，进程执行了某种操作，在另外某个事件发生之前都无法运行。一个常见的例子是：进程向磁盘发起 I/O 请求时，就会被阻塞，于是其他进程可以使用处理器。

把这些状态画成图，就是图 4.2。从图中可以看到，进程可以在操作系统的安排下，在就绪和运行状态之间来回移动。从就绪变为运行，意味着进程**被调度**（scheduled）了；从运行变为就绪，意味着进程被**取消调度**（descheduled）了。进程一旦被阻塞，例如因为发起了一次 I/O 操作，操作系统就会让它保持阻塞，直到某个事件发生（例如 I/O 完成）；这时，进程又回到就绪状态（如果操作系统这样决定，还可能马上再次进入运行状态）。

举个例子，看看两个进程可能怎样经历其中一些状态。首先，假设有两个进程在运行，它们都只使用 CPU（不做 I/O）。在这种情况下，每个进程的状态轨迹可能是这样的（图 4.3）。

在下一个例子中，第一个进程运行一段时间后发起了一次 I/O。这时，该进程被阻塞，让另一个进程有机会运行。图 4.4 展示了这个场景的轨迹。

更具体地说，进程$_0$发起一次 I/O，并在等待它完成时被阻塞；例如，进程从磁盘读取数据或者等待网络数据包（packet）时，就会被阻塞。操作系统发现进程$_0$没有使用 CPU，就

| 时间 | 进程$_0$ | 进程$_1$ | 说明 |
|---|---|---|---|
| 1 | 运行 | 就绪 | |
| 2 | 运行 | 就绪 | |
| 3 | 运行 | 就绪 | |
| 4 | 运行 | 就绪 | 进程$_0$运行结束 |
| 5 | – | 运行 | |
| 6 | – | 运行 | |
| 7 | – | 运行 | |
| 8 | – | 运行 | 进程$_1$运行结束 |

图 4.3：**跟踪进程状态：只使用 CPU**

| 时间 | 进程$_0$ | 进程$_1$ | 说明 |
|---|---|---|---|
| 1 | 运行 | 就绪 | |
| 2 | 运行 | 就绪 | |
| 3 | 运行 | 就绪 | 进程$_0$发起 I/O |
| 4 | 阻塞 | 运行 | 进程$_0$被阻塞， |
| 5 | 阻塞 | 运行 | 于是进程$_1$运行 |
| 6 | 阻塞 | 运行 | |
| 7 | 就绪 | 运行 | I/O 完成 |
| 8 | 就绪 | 运行 | 进程$_1$运行结束 |
| 9 | 运行 | – | |
| 10 | 运行 | – | 进程$_0$运行结束 |

图 4.4：**跟踪进程状态：CPU 和 I/O**

开始运行进程$_1$。进程$_1$运行期间，I/O 完成了，进程$_0$重新回到就绪状态。最后，进程$_1$结束，进程$_0$运行，随后也结束了。

注意，即使在这个简单的例子中，操作系统也要做出许多决定。首先，在进程$_0$发起 I/O 时，系统必须决定运行进程$_1$；这样做让 CPU 保持忙碌，从而提高了资源利用率（resource utilization）。其次，在进程$_0$的 I/O 完成时，系统决定不切换回进程$_0$；这是不是一个好决定还不清楚。你怎么看？这类决定由操作系统的**调度程序**（scheduler）做出，我们将在几章之后讨论这个主题。

## 4.5 数据结构

操作系统也是一个程序，和任何程序一样，它也有一些关键的数据结构，用来记录各种相关信息。例如，为了记录每个进程的状态，操作系统很可能会为所有就绪的进程维护某种**进程列表**（process list），并用一些额外的信息记录当前正在运行的是哪个进程。操作系统还必须以某种方式记录被阻塞的进程；当一个 I/O 事件完成时，操作系统应该确保唤醒正确的进程，让它再次准备好运行。

图 4.5 展示了 xv6 内核中操作系统需要为每个进程记录哪些信息 [CK+08]。在 Linux、Mac OS X 或 Windows 这样的“真正的”操作系统中，也有类似的进程结构（process structure）；去查一查，看看它们要复杂多少。

从图中可以看到操作系统为进程记录的几项重要信息。对于一个已停止的进程，**寄存器上下文**（register context）保存着它的寄存器内容。进程停止时，它的寄存器会被保存到这块内存中；通过恢复这些寄存器（也就是把它们的值放回真正的物理寄存器），操作系统就能继续运行这个进程。我们会在以后的章节中进一步学习这种叫作**上下文切换**（context switch）的技术。

从图中还可以看到，除了运行、就绪和阻塞之外，进程还可能处于其他一些状态。有时，系统会有一个**初始**（initial）状态，进程在创建过程中处于这个状态。此外，进程还可能处于一个**最终**（final）状态：它已经退出，但还没有被清理（在基于 UNIX 的系统中，这称为

 © 2008–23, ARPACI-DUSSEAU

```
// xv6 为停止并在之后重新启动一个进程，
// 而保存和恢复的寄存器
struct context {
  int eip;
  int esp;
  int ebx;
  int ecx;
  int edx;
  int esi;
  int edi;
  int ebp;
};

// 进程可能处于的各种状态
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };

// xv6 为每个进程记录的信息，
// 包括它的寄存器上下文和状态
struct proc {
  char *mem;                  // 进程内存的起始地址
  uint sz;                    // 进程内存的大小
  char *kstack;               // 这个进程的
                              // 内核栈底
  enum proc_state state;      // 进程状态
  int pid;                    // 进程 ID
  struct proc *parent;        // 父进程
  void *chan;                 // 非零时，表示在 chan 上睡眠
  int killed;                 // 非零时，表示已被杀掉
  struct file *ofile[NOFILE]; // 打开的文件
  struct inode *cwd;          // 当前目录
  struct context context;     // 切换到这里来运行进程
  struct trapframe *tf;       // 当前中断的
                              // 陷阱帧
};
```

图 4.5：**xv6 的 proc 结构**

**僵尸**（zombie）状态[1]）。这个最终状态很有用，因为它让其他进程，通常是创建这个进程的**父进程**（parent process），能够检查该进程的返回码，看看刚结束的进程是否执行成功（在基于 UNIX 的系统中，程序成功完成任务时通常返回零，否则返回非零值）。完成后，父进程会做最后一次调用（例如 `wait()`），等待子进程（child process）结束，同时告诉操作系统，可以清理与这个已经消亡的进程有关的数据结构了。

[1]没错，僵尸（zombie）状态。和真正的僵尸一样，这些僵尸相对容易杀死。不过，通常推荐使用的是不同的方法。

补充：数据结构——进程列表

操作系统里满是各种重要的**数据结构**（data structure），我们会在这些讲义中讨论它们。**进程列表**（process list，也叫**任务列表**，task list）是第一个这样的结构。它是比较简单的数据结构之一，但任何能够同时运行多个程序的操作系统，肯定都会有类似这样的结构，来记录系统中所有正在运行的程序。有时，人们把保存某个进程信息的单个结构称为**进程控制块**（Process Control Block，**PCB**），这只是一种花哨的说法，指的是一个包含每个进程信息的 C 结构体（有时也叫**进程描述符**，process descriptor）。

## 4.6 小结

我们介绍了操作系统最基本的抽象：进程。简单地说，它就是一个运行中的程序。有了这个概念上的认识，接下来我们就要进入具体细节：实现进程所需的底层机制，以及聪明地调度进程所需的高层策略。通过把机制和策略结合起来，我们将逐步理解操作系统是怎样虚拟化 CPU 的。

补充：进程的关键术语

- **进程**（process）是操作系统对运行中的程序的主要抽象。在任何时刻，进程都可以用它的状态来描述：它的**地址空间**（address space）中内存的内容、CPU 寄存器的内容（包括**程序计数器**（program counter）和**栈指针**（stack pointer）等），以及有关 I/O 的信息（例如可以读写的已打开文件）。
- **进程 API**（process API）由程序可以发出的、与进程有关的调用组成。通常包括创建、销毁以及其他有用的调用。
- 进程会处于几种**进程状态**（process state）之一，包括运行、就绪和阻塞。不同的事件会使进程从一种状态转换到另一种状态，例如被调度（scheduled）或被取消调度（descheduled），或者等待 I/O 完成。
- **进程列表**（process list）包含系统中所有进程的信息。每个条目存放在一个有时叫作**进程控制块**（PCB）的结构里，它其实就是一个包含某个特定进程信息的结构体。

## 参考文献

[BH70] "The Nucleus of a Multiprogramming System" by Per Brinch Hansen. Communications of the ACM, Volume 13:4, April 1970. *This paper introduces one of the first* **microkernels** *in operating systems history, called Nucleus. The idea of smaller, more minimal systems is a theme that rears its head repeatedly in OS history; it all began with Brinch Hansen's work described herein.*

[CK+08] "The xv6 Operating System" by Russ Cox, Frans Kaashoek, Robert Morris, Nickolai Zeldovich. From: https://github.com/mit-pdos/xv6-public. *The coolest real and little OS in the world. Download and play with it to learn more about the details of how operating systems actually work. We have been using an older version (2012-01-30-1-g1c41342) and hence some examples in the book may not match the latest in the source.*

[DV66] "Programming Semantics for Multiprogrammed Computations" by Jack B. Dennis, Earl C. Van Horn. Communications of the ACM, Volume 9, Number 3, March 1966. *This paper defined many of the early terms and concepts around building multiprogrammed systems.*

[L+75] "Policy/mechanism separation in Hydra" by R. Levin, E. Cohen, W. Corwin, F. Pollack, W. Wulf. SOSP '75, Austin, Texas, November 1975. *An early paper about how to structure operating systems in a research OS known as Hydra. While Hydra never became a mainstream OS, some of its ideas influenced OS designers.*

[V+65] "Structure of the Multics Supervisor" by V.A. Vyssotsky, F. J. Corbato, R. M. Graham. Fall Joint Computer Conference, 1965. *An early paper on Multics, which described many of the basic ideas and terms that we find in modern systems. Some of the vision behind computing as a utility are finally being realized in modern cloud systems.*

 © 2008–23, ARPACI-DUSSEAU

### 作业（模拟）

程序 process-run.py 能让你看到：进程运行时，不管是在使用 CPU（例如执行一条加法指令），还是在做 I/O（例如向磁盘发送请求并等待它完成），进程状态是怎样变化的。详见 README。

### 问题

1. 用以下选项运行 process-run.py：-l 5:100,5:100。CPU 利用率（CPU utilization，例如 CPU 处于使用状态的时间百分比）应该是多少？你是怎么知道的？用 -c 和 -p 选项看看你是否正确。
2. 现在用这些选项运行：./process-run.py -l 4:100,1:0。这些选项指定了一个有 4 条指令（全部使用 CPU）的进程，以及一个只发起一次 I/O 并等待它完成的进程。两个进程全部完成需要多长时间？用 -c 和 -p 看看你是否正确。
3. 交换进程的顺序：-l 1:0,4:100。现在会发生什么？交换顺序有影响吗？为什么？（和往常一样，用 -c 和 -p 看看你是否正确。）
4. 现在我们来研究其他一些选项。一个重要的选项是 -S，它决定了进程发起 I/O 时系统如何反应。当这个选项设为 SWITCH_ON_END 时，系统在一个进程做 I/O 期间不会切换到另一个进程，而是等到这个进程完全结束。运行下面两个进程（-l 1:0,4:100 -c -S SWITCH_ON_END），一个做 I/O，另一个做 CPU 计算，会发生什么？
5. 现在运行同样的进程，但把切换行为设为：每当一个进程在等待（WAITING）I/O 时，就切换到另一个进程（-l 1:0,4:100 -c -S SWITCH_ON_IO）。现在会发生什么？用 -c 和 -p 确认你是否正确。
6. 另一个重要的行为是 I/O 完成时该做什么。使用 -I IO_RUN_LATER 时，I/O 完成后，发起它的进程不一定马上运行，当时正在运行的进程会继续运行。运行下面这组进程会发生什么？

   ```
   ./process-run.py -l 3:0,5:100,5:100,5:100 -S SWITCH_ON_IO
   -c -p -I IO_RUN_LATER
   ```

   系统资源得到有效利用了吗？
7. 现在运行同样的进程，但设置 -I IO_RUN_IMMEDIATE，这会立即运行发起 I/O 的那个进程。这种行为有什么不同？为什么让刚完成 I/O 的进程马上再次运行可能是个好主意？
8. 现在用一些随机生成的进程运行，选项为 -l 3:50,3:50，再分别加上 -s 1、-s 2 或 -s 3。看看你能否预测轨迹会是什么样子。分别使用 -I IO_RUN_IMMEDIATE 和 -I IO_RUN_LATER 时，会发生什么？分别使用 -S SWITCH_ON_IO 和 -S SWITCH_ON_END 时，又会发生什么？

# 5

# 插叙：进程 API

补充：插叙

插叙这类章节讲的是系统中更偏实践的内容，尤其关注操作系统 API 以及怎么使用它们。如果你不喜欢实践性的东西，可以跳过这些插叙。不过你应该喜欢实践性的东西，因为，嗯，它们在现实生活中通常很有用；比如，公司一般不会因为你那些不实用的本事而雇用你。

在这个插叙中，我们讨论 UNIX 系统中的进程创建（process creation）。UNIX 用一对系统调用（system call）`fork()` 和 `exec()` 来创建新进程（process），这是最耐人寻味的进程创建方式之一。第三个例程是 `wait()`，进程可以用它等待自己创建的进程执行完毕。下面我们结合几个简单的例子，更详细地介绍这些接口。于是，我们的问题是：

关键问题：如何创建和控制进程

操作系统应该为创建和控制进程提供哪些接口？这些接口应该怎样设计，才能做到功能强大、容易使用、性能又高？

## 5.1 `fork()` 系统调用

`fork()` 系统调用用来创建新进程 [C63]。不过要事先提醒你：它肯定是你调用过的最奇怪的例程[1]。具体来说，假设你有一个正在运行的程序，它的代码如图 5.1 所示；仔细看看这段代码，最好自己敲一遍并运行一下！

运行这个程序（名为 p1.c）时，你会看到如下输出：

```
prompt> ./p1
hello (pid:29146)
parent of 29147 (pid:29146)
child (pid:29147)
prompt>
```

[1]好吧，我们承认这一点并没有十足的把握；谁知道你在没人看着的时候都调用了些什么例程呢？但无论你调用例程的习惯有多特别，`fork()` 都够古怪的。

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main(int argc, char *argv[]) {
6      printf("hello (pid:%d)\n", (int) getpid());
7      int rc = fork();
8      if (rc < 0) {
9          // fork 失败
10         fprintf(stderr, "fork failed\n");
11         exit(1);
12     } else if (rc == 0) {
13         // 子进程（新进程）
14         printf("child (pid:%d)\n", (int) getpid());
15     } else {
16         // 父进程走这条路径（main）
17         printf("parent of %d (pid:%d)\n",
18                rc, (int) getpid());
19     }
20     return 0;
21 }
```

图 5.1：**调用 `fork()`（p1.c）**

我们来更详细地看看 p1.c 中发生了什么。进程刚开始运行时，会打印一条 hello 消息，消息里包含它的**进程标识符**（process identifier），也叫 **PID**。这个进程的 PID 是 29146；在 UNIX 系统中，如果想对某个进程做点什么，比如让它停止运行，就要用 PID 来指定这个进程。到目前为止一切都好。

接下来是有意思的部分。进程调用了 `fork()` 系统调用，操作系统就是靠它来创建新进程的。奇怪之处在于：新创建的进程（几乎）是调用进程的一份精确副本。也就是说，在操作系统看来，现在好像有两个 p1 程序在运行，而且两者都即将从 `fork()` 系统调用返回。新创建的进程叫作**子进程**（child），与之相对，创建它的进程叫作**父进程**（parent）。子进程并不像你可能预想的那样从 `main()` 开始运行（注意，“hello”消息只打印了一次）；它就像自己调用了 `fork()` 一样，直接从那里开始了自己的生命。

你可能已经注意到：子进程并不是一模一样的副本。具体来说，虽然它现在有了自己的地址空间（address space）副本（即自己的私有内存）、自己的寄存器、自己的 PC 等等，但它从 `fork()` 返回给调用者的值是不同的。父进程收到的是新创建的子进程的 PID，而子进程收到的返回值是零。这种区别很有用，因为这样一来，就很容易写出分别处理这两种情况的代码（如上所示）。

你可能还注意到：（p1.c 的）输出并不是**确定的**（deterministic）。子进程创建之后，系统中就有了两个我们关心的活动进程：父进程和子进程。假设我们在单 CPU 的系统上运行（为简单起见），那么这时父进程和子进程都有可能先运行。在上面的例子中，是父进程先运行，所以它先打印出了消息。在其他情况下，也可能反过来，如下面这次输出所示：

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main(int argc, char *argv[]) {
7      printf("hello (pid:%d)\n", (int) getpid());
8      int rc = fork();
9      if (rc < 0) {          // fork 失败，退出
10         fprintf(stderr, "fork failed\n");
11         exit(1);
12     } else if (rc == 0) { // 子进程（新进程）
13         printf("child (pid:%d)\n", (int) getpid());
14     } else {               // 父进程走这条路径
15         int rc_wait = wait(NULL);
16         printf("parent of %d (rc_wait:%d) (pid:%d)\n",
17                rc, rc_wait, (int) getpid());
18     }
19     return 0;
20 }
```

图 5.2：**调用 `fork()` 和 `wait()`（p2.c）**

```
prompt> ./p1
hello (pid:29146)
child (pid:29147)
parent of 29147 (pid:29146)
prompt>
```

CPU **调度程序**（scheduler）决定某一时刻运行哪个进程，我们很快就会详细讨论这个话题。由于调度程序很复杂，我们通常无法有把握地断定它会怎么选择，也就无法断定哪个进程会先运行。这种**不确定性**（non-determinism）会带来一些有趣的问题，在**多线程程序**（multi-threaded programs）中尤其如此；因此，等到本书第二部分研究**并发**（concurrency）时，我们会看到多得多的不确定性。

## 5.2 `wait()` 系统调用

到目前为止，我们还没做多少事：只是创建了一个子进程，让它打印一条消息然后退出。有时候，让父进程等子进程把手头的事做完，其实相当有用。这项任务由 `wait()` 系统调用（或者它更完整的兄弟 `waitpid()`）完成；详见图 5.2。

在这个例子（p2.c）中，父进程调用 `wait()` 来阻塞（即推迟自己的执行），直到子进程执行完毕。子进程结束后，父进程不再阻塞，`wait()` 在父进程中返回。

在上面的代码中加入 `wait()` 调用，输出就变成确定的了。你能看出为什么吗？来，想一想。

（等你想一想……好了）

既然你已经想过了，下面是输出：

```
prompt> ./p2
hello (pid:29266)
child (pid:29267)
parent of 29267 (rc_wait:29267) (pid:29266)
prompt>
```

有了这段代码，我们就知道子进程总是先打印。为什么能知道呢？嗯，它可能像之前那样直接先运行，于是在父进程之前打印。不过，如果碰巧是父进程先运行，它会立即调用 `wait()`；这个系统调用要等子进程运行完并退出后才会返回[2]。因此，即使父进程先运行，它也会礼貌地等子进程运行结束，然后 `wait()` 返回，父进程再打印自己的消息。

## 5.3 最后是 `exec()` 系统调用

进程创建 API 中最后一个、也很重要的部分是 `exec()` 系统调用[3]。如果你想运行一个和调用程序不同的程序，这个系统调用就很有用。比如，在 p2.c 中调用 `fork()`，只有在你想继续运行同一个程序的副本时才有用。然而，你常常想运行另一个程序；`exec()` 做的正是这件事（图 5.3）。

在这个例子中，子进程调用 `execvp()` 来运行程序 wc，也就是统计字数的程序。实际上，它对源文件 p3.c 运行 wc，于是告诉我们这个文件里有多少行、多少个单词、多少个字节：

```
prompt> ./p3
hello (pid:29383)
child (pid:29384)
      29     107    1030 p3.c
parent of 29384 (rc_wait:29384) (pid:29383)
prompt>
```

`fork()` 系统调用很奇怪；它的同伙 `exec()` 也不怎么正常。它的做法是：给定一个可执行程序的名字（例如 wc）和一些参数（例如 p3.c），从这个可执行程序中**加载**（load）代码（和静态数据），用它覆盖当前的代码段（和当前的静态数据）；程序的堆、栈和内存空间的其他部分都会被重新初始化。然后操作系统直接运行这个程序，把所有参数作为该进程的 argv 传进去。因此，它并没有创建新进程，而是把当前正在运行的程序（原来是 p3）变成了另一个正在运行的程序（wc）。子进程执行 `exec()` 之后，就几乎像 p3.c 从来没有运行过一样；`exec()` 调用一旦成功，就永远不会返回。

## 5.4 为什么？API 的设计动机

当然，你可能会有一个大大的疑问：创建新进程本该是件简单的事，为什么要设计出这么古怪的接口？原来，`fork()` 和 `exec()` 分开，对于构建 UNIX shell 至关重要，因为它

[2]也有少数情况下，`wait()` 会在子进程退出之前返回；和往常一样，详情请读手册页（manual page）。另外，对于本书中任何绝对化、不加限定的说法，比如“子进程总是先打印”或者“UNIX 是世界上最好的东西，比冰淇淋还好”，都要多加小心。

[3]在 Linux 上，`exec()` 有六个变体：`execl()`、`execlp()`、`execle()`、`execv()`、`execvp()` 和 `execvpe()`。详情请读手册页。

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6
7  int main(int argc, char *argv[]) {
8      printf("hello (pid:%d)\n", (int) getpid());
9      int rc = fork();
10     if (rc < 0) {          // fork 失败，退出
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     } else if (rc == 0) { // 子进程（新进程）
14         printf("child (pid:%d)\n", (int) getpid());
15         char *myargs[3];
16         myargs[0] = strdup("wc");   // 程序："wc"
17         myargs[1] = strdup("p3.c"); // 参数：输入文件
18         myargs[2] = NULL;           // 标记数组结束
19         execvp(myargs[0], myargs);  // 运行 wc 统计字数
20         printf("this shouldn't print out");
21     } else {               // 父进程走这条路径
22         int rc_wait = wait(NULL);
23         printf("parent of %d (rc_wait:%d) (pid:%d)\n",
24                rc, rc_wait, (int) getpid());
25     }
26     return 0;
27 }
```

图 5.3：**调用 `fork()`、`wait()` 和 `exec()`（p3.c）**

让 shell 可以在调用 `fork()` 之后、调用 `exec()` 之前运行一段代码；这段代码可以修改即将运行的程序的环境，从而让各种有意思的功能都能轻松实现。

shell 只是一个用户程序[4]。它向你显示一个**提示符**（prompt），然后等你输入。你输入一条命令（即一个可执行程序的名字，外加若干参数）；在大多数情况下，shell 接着会找出这个可执行程序在文件系统（file system）中的位置，调用 `fork()` 创建一个新的子进程来

提示：做对（Lampson 定律）

正如 Lampson 在他广受好评的《Hints for Computer Systems Design》[L83] 中所说："**做对**（get it right）。抽象（abstraction）和简洁都不能代替做对。"有时候，你就是得做正确的事；而一旦做对了，它会比其他选择好得多。设计进程创建 API 的方法有很多；然而，`fork()` 和 `exec()` 的组合既简单又极其强大。在这一点上，UNIX 的设计者就是做对了。因为 Lampson 常常"做对"，所以我们用他的名字来命名这条定律。

[4]shell 有很多种，比如 tcsh、bash 和 zsh。你应该挑一个，读读它的手册页，深入了解它；所有 UNIX 专家都是这么做的。

运行这条命令，调用 exec() 的某个变体来运行命令，然后调用 wait() 等待命令完成。子进程完成后，shell 从 wait() 返回，再次打印出提示符，准备接收你的下一条命令。

fork() 和 exec() 的分离，让 shell 可以相当轻松地做一大堆有用的事。例如：

```
prompt> wc p3.c > newfile.txt
```

在上面的例子中，程序 wc 的输出被**重定向**（redirected）到输出文件 newfile.txt 中（大于号表示这种重定向）。shell 完成这项任务的方式相当简单：子进程创建之后、调用 exec() 之前，shell（准确地说，是在子进程中执行的代码）关闭**标准输出**（standard output），然后打开文件 newfile.txt。这样一来，即将运行的程序 wc 的所有输出都会被发送到这个文件，而不是屏幕上。之所以能做到这一点，是因为打开的文件描述符（file descriptor）在 exec() 调用前后保持打开 [SR05]。

图 5.4（第 38 页）展示了一个正是这样做的程序。这种重定向之所以行得通，是因为它对操作系统怎样管理文件描述符做了一个假设。具体来说，UNIX 系统从零开始查找空闲的文件描述符。在这个例子中，STDOUT_FILENO 会是第一个可用的描述符，因此在调用 open() 时被分配出去。之后子进程往标准输出文件描述符写的东西（比如用 printf() 这样的例程写），就会被透明地送进新打开的文件，而不是屏幕。

运行 p4.c 程序的输出如下：

```
prompt> ./p4
prompt> cat p4.output
      32     109     846 p4.c
prompt>
```

关于这个输出，你会注意到（至少）两个有意思的细节。第一，运行 p4 时，看起来好像什么都没发生；shell 只是打印出命令提示符，马上就准备好接收你的下一条命令了。但事实并非如此：程序 p4 确实调用了 fork() 来创建新的子进程，然后通过调用 execvp() 运行了 wc 程序。屏幕上之所以看不到任何输出，是因为输出已经被重定向到文件 p4.output 中了。第二，用 cat 查看这个输出文件，可以看到运行 wc 本该得到的输出都在里面。很酷，对吧？

UNIX 的管道（pipe）也是用类似的方式实现的，不过用的是 pipe() 系统调用。这时，一个进程的输出连到内核中的一个**管道**（pipe），也就是一个队列（queue），另一个进程的输入也连到同一个管道；这样，一个进程的输出就能无缝地成为下一个进程的输入，从而可以把很长、很有用的一串命令连在一起。举个简单的例子：在文件中查找某个单词，再数一数这个单词出现了多少次；有了管道以及 grep 和 wc 这两个工具，这件事很简单：只要在命令提示符下输入 grep -o foo file | wc -l，然后惊叹于结果就行了。

最后，虽然我们只是从较高的层面勾勒了进程 API（process API），但关于这些调用，还有大量细节值得学习和消化；比如，等到本书第三部分讨论文件系统时，我们会进一步了解文件描述符。现在只需知道：fork()/exec() 组合是创建和操作进程的一种强大方式。

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    int rc = fork();
    if (rc < 0) {
        // fork 失败
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        // 子进程：把标准输出重定向到文件
        close(STDOUT_FILENO);
        open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC,
             S_IRWXU);
        // 现在 exec "wc"……
        char *myargs[3];
        myargs[0] = strdup("wc");   // 程序：wc
        myargs[1] = strdup("p4.c"); // 参数：要统计的文件
        myargs[2] = NULL;           // 标记数组结束
        execvp(myargs[0], myargs);  // 运行 wc 统计字数
    } else {
        // 父进程走这条路径（main）
        int rc_wait = wait(NULL);
    }
    return 0;
}
```

图 5.4：**以上全部再加上重定向（p4.c）**

## 5.5 进程控制与用户

除了 `fork()`、`exec()` 和 `wait()`，UNIX 系统中还有许多其他与进程交互的接口。例如，`kill()` 系统调用用来向进程发送**信号**（signals），信号可以让进程暂停、终止，或者做其他有用的事。为了方便，大多数 UNIX shell 都把某些组合键配置成向当前正在运行的进程发送特定的信号；例如，control-c 向进程发送 SIGINT（中断，interrupt）信号，通常会终止它；control-z 发送 SIGTSTP（停止，stop）信号，让进程在执行中途暂停（之后可以用命令让它恢复运行，比如很多 shell 都有的内置命令 fg）。

整个信号子系统提供了一套丰富的基础设施，用来把外部事件传递给进程，包括在单个进程中接收和处理这些信号的方式，以及向单个进程乃至整个**进程组**（process groups）发送信号的方式。要使用这种通信方式，进程应该用 `signal()` 系统调用来“捕获”各种信号；这样，某个信号传给进程时，进程就会暂停正常执行，转而运行一段特定的代码来响应它。想进一步了解信号及其诸多复杂细节，请读别的资料 [SR05]。

补充：RTFM——去读手册页

在本书中，提到某个系统调用或库调用时，我们常常会让你去读**手册页**（manual pages，简称 **man pages**）。手册页是 UNIX 系统上最初的文档形式；要知道，它们出现的时候，所谓的**万维网**（web）还不存在呢。

花些时间读手册页，是系统程序员成长过程中的关键一步；这些页面里藏着大量有用的小知识。特别值得一读的，是你所用的 shell（比如 **tcsh** 或 **bash**）的手册页，当然还有你的程序用到的所有系统调用的手册页（看看有哪些返回值和错误情况）。

最后，读手册页还能帮你免去一些尴尬。当你向同事请教 `fork()` 的某个复杂细节时，他们可能只会回一句："RTFM。"这是同事在委婉地催你去读手册页（Read The **Man pages**）。RTFM 里的 F 只是给这句话增添了一点色彩……

这自然引出一个问题：谁能向进程发送信号，谁不能？一般来说，我们所用的系统可能同时有很多人在用；如果其中某个人能随意发送 SIGINT 这样的信号去中断某个进程，很可能就此把它终止，系统的易用性和安全性就会受到损害。因此，现代系统对**用户**（user）这一概念有很明确的定义。用户先输入密码以建立凭证（credential），然后登录，获得对系统资源的访问权限。之后，用户可以启动一个或多个进程，并完全控制它们（暂停它们、杀掉它们等）。用户一般只能控制自己的进程；操作系统的职责是把资源（比如 CPU、内存和磁盘）分配给每个用户及其进程，以实现系统的整体目标。

## 5.6 实用工具

还有许多命令行工具也很有用。例如，用 ps 命令可以看到哪些进程正在运行；读一读手册页，了解可以传给 ps 的一些有用的选项。top 工具也很有用，它会显示系统中的进程，以及它们各自占用了多少 CPU 和其他资源。有趣的是，很多时候你运行 top，它都会声称自己是头号资源大户；看来它多少有点自恋。kill 命令可以向进程发送任意信号，稍微好用一点的 killall 也可以。使用这些命令一定要小心；如果你不小心杀掉了窗口管理器，你面前的这台计算机可能就变得很难用了。

补充：超级用户（root）

系统通常需要一个能**管理**（administer）系统的用户，这个用户不像大多数用户那样受到限制。这样的用户应该能杀掉任意进程，比如某个正以某种方式滥用系统的进程，哪怕它不是这个用户启动的。这样的用户还应该能运行一些强大的命令，比如 shutdown（不出所料，它会关闭系统）。在基于 UNIX 的系统中，这些特殊能力属于**超级用户**（superuser，有时也叫 **root**）。大多数用户不能杀掉其他用户的进程，但超级用户可以。当 root 很像当蜘蛛侠：能力越大，责任越大 [QI15]。因此，为了提高**安全性**（security），也为了避免代价高昂的错误，通常最好以普通用户的身份工作；如果确实需要以 root 身份工作，务必小心行事，因为计算世界里所有的破坏力现在都握在你手里了。

 © 2008–25, ARPACI-DUSSEAU

最后，还有许多不同种类的 CPU 监视工具，能让你快速了解系统的负载情况；例如，我们总是在 Macintosh 的工具栏上运行着 **MenuMeters**（Raging Menace 公司的软件），这样随时都能看到 CPU 的使用情况。一般来说，对正在发生的事了解得越多越好。

## 5.7 小结

我们介绍了一些与 UNIX 进程创建有关的 API：`fork()`、`exec()` 和 `wait()`。不过，我们只是触及了皮毛。想了解更多细节，当然要读 Stevens 和 Rago 的书 [SR05]，尤其是其中关于进程控制（process control）、进程关系和信号的几章；那里面的智慧值得好好汲取。

虽然我们对 UNIX 进程 API（process API）依然热情不减，但也应该指出，并不是所有人都这么看好它。例如，来自微软、波士顿大学和瑞士苏黎世联邦理工学院（ETH）的系统研究人员最近发表了一篇论文，详细阐述了 `fork()` 的一些问题，并提倡使用其他更简单的进程创建 API，比如 **`spawn()`** [B+19]。读一读这篇论文以及它引用的相关工作，了解这种不同的视角。虽然一般来说信任这本书是好事，但也要记住，作者是有自己观点的；这些观点未必（总是）像你以为的那样得到广泛认同。

补充：进程 API 关键术语

- 每个进程都有一个名字；在大多数系统中，这个名字是一个数字，叫作**进程 ID**（process ID，**PID**）。
- 在 UNIX 系统中，**`fork()`** 系统调用用来创建新进程。创建者叫作**父进程**（parent process）；新创建的进程叫作**子进程**（child process）。就像现实生活中有时会发生的那样 [J16]，子进程和父进程几乎一模一样。
- **`wait()`** 系统调用让父进程可以等待子进程执行完毕。
- **`exec()`** 系列系统调用让子进程摆脱与父进程的相似，去执行一个全新的程序。
- UNIX **shell** 通常用 `fork()`、`wait()` 和 `exec()` 来启动用户命令；`fork` 和 `exec` 分开，才让**输入/输出重定向**（input/output redirection）、**管道**（pipe）等很酷的功能成为可能，而且完全不需要修改所运行的程序。
- 进程控制（process control）以**信号**（signal）的形式提供，信号可以让作业停止、继续，甚至终止。
- 某个人能控制哪些进程，由**用户**（user）这一概念来界定；操作系统允许多个用户进入系统，并确保每个用户只能控制自己的进程。
- **超级用户**（superuser）可以控制所有进程，其实还能做很多别的事；出于安全考虑，应该尽量少用、谨慎使用这个角色。

## 参考文献

[B+19] "A fork() in the road" by Andrew Baumann, Jonathan Appavoo, Orran Krieger, Timothy Roscoe. HotOS '19, Bertinoro, Italy. *A fun paper full of `fork()`ing rage. Read it to get an opposing viewpoint on the UNIX process API. Presented at the always lively HotOS workshop, where systems researchers go to present extreme opinions in the hopes of pushing the community in new directions.*

[C63] "A Multiprocessor System Design" by Melvin E. Conway. AFIPS '63 Fall Joint Computer Conference, New York, USA 1963. *An early paper on how to design multiprocessing systems; may be the first place the term `fork()` was used in the discussion of spawning new processes.*

[DV66] "Programming Semantics for Multiprogrammed Computations" by Jack B. Dennis and Earl C. Van Horn. Communications of the ACM, Volume 9, Number 3, March 1966. *A classic paper that outlines the basics of multiprogrammed computer systems. Undoubtedly had great influence on Project MAC, Multics, and eventually UNIX.*

[J16] "They could be twins!" by Phoebe Jackson-Edwards. The Daily Mail. March 1, 2016.. *This hard-hitting piece of journalism shows a bunch of weirdly similar child/parent photos and is frankly kind of mesmerizing. Go ahead, waste two minutes of your life and check it out. But don't forget to come back here! This, in a microcosm, is the danger of surfing the web.*

[L83] "Hints for Computer Systems Design" by Butler Lampson. ACM Operating Systems Review, Volume 15:5, October 1983. *Lampson's famous hints on how to design computer systems. You should read it at some point in your life, and probably at many points in your life.*

[QI15] "With Great Power Comes Great Responsibility" by The Quote Investigator. Available: https://quoteinvestigator.com/2015/07/23/great-power. *The quote investigator concludes that the earliest mention of this concept is 1793, in a collection of decrees made at the French National Convention. The specific quote: "Ils doivent envisager qu'une grande responsabilité est la suite inséparable d'un grand pouvoir", which roughly translates to "They must consider that great responsibility follows inseparably from great power." Only in 1962 did the following words appear in Spider-Man: "…with great power there must also come–great responsibility!" So it looks like the French Revolution gets credit for this one, not Stan Lee. Sorry, Stan.*

[SR05] "Advanced Programming in the UNIX Environment" by W. Richard Stevens, Stephen A. Rago. Addison-Wesley, 2005. *All nuances and subtleties of using UNIX APIs are found herein. Buy this book! Read it! And most importantly,* ***live it.***

## 作业（模拟）

本次模拟作业围绕 `fork.py` 展开，这是一个简单的进程创建模拟器，可以展示进程之间如何在一棵“家族”树中相互关联。关于如何运行这个模拟器，请阅读相应的 README。

## 问题

1. 运行 `./fork.py -s 10`，看看执行了哪些操作。你能预测每一步的进程树（process tree）是什么样子吗？用 `-c` 选项检查你的答案。试试不同的随机种子（`-s`），或者加入更多操作（`-a`），熟悉一下这个模拟器。
2. 模拟器提供的一个可调参数是 fork_percentage，用 `-f` 选项设置。它越高，下一个操作越可能是 fork；越低，下一个操作越可能是 exit。用大量操作（例如 `-a 100`）运行模拟器，把 fork_percentage 从 0.1 调到 0.9。随着这个比例的变化，你认为最终的进程树会是什么样子？用 `-c` 检查你的答案。
3. 现在用 `-t` 选项切换输出方式（例如运行 `./fork.py -t`）。给定一组进程树，你能说出执行了哪些操作吗？
4. 值得注意的一件事，是子进程退出时会发生什么：它自己的子进程在进程树中会怎样？为了研究这个问题，我们来构造一个具体的例子：

   ```
   ./fork.py -A a+b,b+c,c+d,c+e,c-
   ```

   在这个例子中，进程“a”创建“b”，“b”又创建“c”，“c”再创建“d”和“e”。然而，随后“c”退出了。你认为退出之后进程树应该是什么样子？如果使用 `-R` 选项呢？自己去了解一下孤儿进程（orphan process）会发生什么，补充一些背景知识。
5. 最后一个要探索的选项是 `-F`，它会跳过中间步骤，只要求你填出最终的进程树。运行 `./fork.py -F`，看看你能否通过观察生成的一系列操作写出最终的树。用不同的随机种子多试几次。
6. 最后，把 `-t` 和 `-F` 一起使用。这会显示最终的进程树，然后让你填出执行过的操作。通过观察这棵树，你能确定到底执行了哪些操作吗？哪些情况下能确定？哪些情况下不能？试试不同的随机种子，深入研究这个问题。

补充：编程作业

编程作业是一些小练习：你要编写代码，在真实的机器上运行，从而积累一些使用基本操作系统 API 的经验。毕竟，你（很可能）是个计算机科学家，所以应该喜欢写代码，对吧？如果不喜欢，你总还可以去搞计算机理论，不过那可相当难。当然，要真正成为专家，你得花不少时间在机器上折腾；事实上，要找一切借口写点代码，看看它是怎么工作的。花些时间吧，你知道自己能成为一位大师。

## 作业（编程）

在这个作业中，你要熟悉一下刚刚读到的进程管理（process management）API。别担心，它比听起来还要有趣！一般来说，你越能抽出时间写代码，收获就越大，那为什么不现在就开始呢？

## 问题

1. 编写一个调用 `fork()` 的程序。在调用 `fork()` 之前，让主进程（main process）访问一个变量（例如 x），并把它设为某个值（例如 100）。子进程中这个变量的值是多少？当子进程和父进程都修改 x 的值时，这个变量会怎样？
2. 编写一个程序，它先（用 `open()` 系统调用）打开一个文件，然后调用 `fork()` 创建新进程。子进程和父进程都能访问 `open()` 返回的文件描述符吗？当它们并发地（concurrently）写这个文件，也就是同时写入时，会发生什么？
3. 再用 `fork()` 编写一个程序。子进程应该打印“hello”，父进程应该打印“goodbye”。你应该设法确保子进程总是先打印；你能不在父进程中调用 **`wait()`** 就做到这一点吗？
4. 编写一个程序，调用 `fork()`，然后调用某种形式的 `exec()` 来运行程序 `/bin/ls`。看看你能否试遍 `exec()` 的所有变体，包括（在 Linux 上的）`execl()`、`execle()`、`execlp()`、`execv()`、`execvp()` 和 `execvpe()`。你认为同一个基本调用为什么会有这么多变体？
5. 现在编写一个程序，在父进程中用 `wait()` 等待子进程结束。`wait()` 返回什么？如果在子进程中使用 `wait()` 会发生什么？
6. 对上一个程序稍作修改，这次用 `waitpid()` 代替 `wait()`。什么时候 `waitpid()` 会有用？
7. 编写一个程序，创建一个子进程，然后在子进程中关闭标准输出。如果子进程关闭描述符之后调用 `printf()` 打印一些输出，会发生什么？
8. 编写一个程序，创建两个子进程，并用 `pipe()` 系统调用把一个子进程的标准输出连接到另一个子进程的标准输入（standard input）。

 © 2008–25, ARPACI-DUSSEAU

# 6

# 机制：受限直接执行

为了虚拟化（virtualize）CPU，操作系统需要设法让许多看起来同时运行的作业共享一个物理 CPU。基本思路很简单：让一个进程（process）运行一小会儿，再让另一个进程运行一小会儿，如此轮流下去。这样**分时共享**（time sharing）CPU，就实现了虚拟化。

不过，构建这样的虚拟化机制有几个难点。第一个是性能：怎样实现虚拟化，才不会给系统增加过多的开销？第二个是控制：怎样才能在高效运行进程的同时，保持对 CPU 的控制？控制对操作系统尤其重要，因为它负责管理资源；没有控制，进程就可能一直运行下去、占据整台机器，或者访问它不该访问的信息。因此，在保持控制的同时获得高性能，是构建操作系统的核心挑战之一。

关键问题：
如何在保持控制的同时高效地虚拟化 CPU
操作系统必须高效地虚拟化 CPU，同时保持对系统的控制。为此，既需要硬件支持，也需要操作系统支持。为了有效地完成工作，操作系统常常会审慎地借助一点硬件支持。

## 6.1 基本技术：受限直接执行

不出所料，为了让程序跑得像人们期望的那样快，操作系统的开发者想出了一种技术，我们把它叫作**受限直接执行**（limited direct execution）。这个想法中“直接执行”的部分很简单：直接在 CPU 上运行程序就行了。因此，当操作系统想启动一个程序时，它会在进程列表（process list）中为这个程序创建一个进程条目，为它分配一些内存，（从磁盘）把程序代码加载到内存中，找到程序的入口点（即 `main()` 例程或类似的东西），跳转过去，开始运行用户的代码。图 6.1 展示了这个基本的直接执行协议（暂时还没有任何限制），它用普通的调用和返回跳转到程序的 `main()`，之后再回到内核。

听起来很简单，不是吗？但在虚拟化 CPU 的过程中，这种方法会带来几个问题。第一个问题很简单：如果直接运行程序，操作系统怎样在高效运行程序的同时，确保程序不做我们不希望它做的事？第二个问题：进程运行时，操作系统怎样让它停下来、切换到另一个进程，从而实现虚拟化 CPU 所需的**分时共享**（time sharing）？

在下面回答这些问题的过程中，我们会更清楚地了解虚拟化 CPU 需要什么。在设计这些技术时，我们还会看到名字里“受限”这一部分从何而来；如果对运行的程序不加限制，

| 操作系统 | 程序 |
| --- | --- |
| 在进程列表中创建条目 | |
| 为程序分配内存 | |
| 把程序加载到内存中 | |
| 用 argc/argv 设置栈 | |
| 清除寄存器 | |
| 执行**调用** main() | |
| | 运行 main() |
| | 从 main 执行**返回** |
| 释放进程的内存 | |
| 把进程从进程列表中删除 | |

图 6.1：**直接执行协议（无限制）**

操作系统就什么都控制不了，因而只能算“一个库”——对一个有抱负的操作系统来说，这可真是件可悲的事！

## 6.2 问题 1：受限的操作

直接执行有一个明显的好处：快。程序直接在硬件 CPU 上运行，因此执行速度和人们预期的一样快。但在 CPU 上直接运行会带来一个问题：如果进程想执行某种受限的操作，比如向磁盘发出 I/O 请求，或者获取更多的系统资源（比如 CPU 或内存），该怎么办？

关键问题：如何执行受限的操作

进程必须能够执行 I/O 和其他一些受限的操作，但又不能让进程完全控制整个系统。操作系统和硬件怎样协作才能做到这一点？

补充：为什么系统调用看起来像过程调用

你可能会好奇，为什么调用 `open()` 或 `read()` 这样的系统调用（system call），看起来和 C 语言中普通的过程调用（procedure call）一模一样；也就是说，既然看起来就像过程调用，系统怎么知道它是系统调用，并做好该做的一切？原因很简单：它就是一个过程调用，只不过在这个过程调用里藏着著名的陷阱（trap）指令。具体来说，当你调用（比如说）`open()` 时，你执行的是一个进入 C 库的过程调用。无论是 `open()` 还是库提供的其他任何系统调用，库都会遵循与内核商定的调用约定，把 `open()` 的参数放到约定好的位置（比如栈上，或者特定的寄存器里），把系统调用号（system-call number）也放到约定好的位置（同样是栈上或寄存器里），然后执行前面说的陷阱（trap）指令。陷阱（trap）指令之后，库中的代码会取出返回值，再把控制权交还给发出系统调用的程序。所以，C 库中负责发出系统调用的那部分代码是用汇编手写的：它们必须严格遵守约定，才能正确处理参数和返回值，并执行与具体硬件相关的陷阱（trap）指令。现在你知道了，为什么你自己不必写汇编代码来陷入操作系统：已经有人替你写好了。

 © 2008–23, ARPACI-DUSSEAU

提示：使用受保护的控制权转移
硬件通过提供不同的执行模式来帮助操作系统。在**用户模式**（user mode）下，应用程序不能完全访问硬件资源。在**内核模式**（kernel mode）下，操作系统可以访问机器的全部资源。硬件还提供了专门的指令：**陷入**（trap）内核的指令、**从陷阱返回**（return-from-trap）用户模式程序的指令，以及让操作系统告诉硬件**陷阱表**（trap table）在内存中什么位置的指令。

一种办法是干脆让任何进程在 I/O 和其他相关操作上为所欲为。但这样一来，我们想要的许多系统就都无法构建了。例如，如果我们想让文件系统（file system）在允许访问文件之前先检查权限，就不能让任何用户进程直接向磁盘发出 I/O；否则，进程完全可以读写整个磁盘，所有保护也就形同虚设了。

因此，我们采用的方法是引入一种新的处理器模式，叫作**用户模式**（user mode）；在用户模式下运行的代码，能做的事情受到限制。例如，在用户模式下运行时，进程不能发出 I/O 请求；如果它这样做，处理器就会触发异常，然后操作系统多半会杀掉这个进程。

与用户模式相对的是**内核模式**（kernel mode），操作系统（即内核）就运行在这种模式下。在内核模式下，运行的代码可以为所欲为，包括执行特权操作（privileged operation），比如发出 I/O 请求和执行各种受限的指令。

不过，我们还剩下一个难题：用户进程想执行某种特权操作（比如从磁盘读数据）时，该怎么办？为此，几乎所有现代硬件都允许用户程序执行**系统调用**（system call）。系统调用最早出现在 Atlas [K+61, L78] 这类古老的机器上，它让内核可以小心地把某些关键功能开放给用户程序，比如访问文件系统、创建和销毁进程、与其他进程通信，以及分配更多内存。大多数操作系统提供几百个调用（详见 POSIX 标准 [P10]）；早期的 UNIX 系统则只开放了一个更精简的子集，大约二十个调用。

要执行系统调用，程序必须执行一条特殊的**陷阱**（trap）指令。这条指令会跳转到内核，同时把特权级别提升到内核模式；一旦进入内核，系统就可以执行任何需要的特权操作（如果允许的话），替调用进程把事情办完。完成后，操作系统执行一条特殊的**从陷阱返回**（return-from-trap）指令；你可能已经猜到，它会返回到发起调用的用户程序，同时把特权级别降回用户模式。

硬件在执行陷阱（trap）时需要格外小心，因为它必须保存足够多的调用者寄存器，以便操作系统发出从陷阱返回（return-from-trap）指令时能正确返回。例如，在 x86 上，处理器会把程序计数器（program counter）、标志位和另外几个寄存器压入该进程的**内核栈**（kernel stack，每个进程各有一个）；从陷阱返回（return-from-trap）时，再把这些值从栈中弹出，恢复执行用户模式程序（详见 Intel 的系统手册 [I11]）。其他硬件系统使用的约定不同，但基本概念在各个平台上都差不多。

上面的讨论漏掉了一个重要细节：陷阱（trap）怎么知道在操作系统内部运行哪段代码？显然，调用进程不能像过程调用那样指定要跳转到的地址；否则，程序就能跳到内核里的任

何地方，这显然是个**非常糟糕的主意**[1]。因此，内核必须严格控制陷阱（trap）发生时执行哪些代码。

内核的做法是在启动时设置**陷阱表**（trap table）。机器启动时处于特权（内核）模式，因此可以根据需要自由地配置机器硬件。操作系统最早做的事情之一，就是告诉硬件在发生某些异常事件时运行哪段代码。例如，发生硬盘中断（interrupt）、发生键盘中断，或者程序发出系统调用时，应该运行哪段代码？操作系统通常用某种特殊指令，把这些**陷阱处理程序**（trap handlers）的位置告诉硬件。硬件得到通知后，会记住这些处理程序的位置，直到机器下次重启；这样，在发生系统调用和其他异常事件时，硬件就知道该怎么做（即跳转到哪段代码）。

为了指定具体是哪个系统调用，通常会给每个系统调用分配一个**系统调用号**（system-call number）。因此，用户代码负责把所需的系统调用号（system-call number）放到寄存器中，或者栈上的指定位置；操作系统在陷阱处理程序（trap handler）中处理系统调用时，会检查这个号码，确认它有效，如果有效，就执行相应的代码。这种间接层（level of indirection）起到了一种**保护**（protection）作用：用户代码不能指定确切的跳转地址，而必须通过号码来请求某项特定的服务。

最后再说一句：能够执行“告诉硬件陷阱表（trap table）在哪里”的指令，是一种非常强大的能力。所以你可能已经猜到，它也是一种**特权**（privileged）操作。如果你试图在用户模式下执行这条指令，硬件不会允许，而且你大概能猜到会发生什么（提示：再见了，违规的程序）。思考题：如果你能安装自己的陷阱表（trap table），你能对系统干出哪些可怕的事？你能接管整台机器吗？

图 6.2 中的时间线（时间自上而下递增）总结了这个协议（protocol）。我们假设每个进程都有一个内核栈；进入和退出内核时，（由硬件）把寄存器（包括通用寄存器和程序计数器）保存到这个栈上，再从这里恢复。

受限直接执行（**LDE**）协议分为两个阶段。在第一阶段（启动时），内核初始化陷阱表（trap table），CPU 记住陷阱表的位置供以后使用。内核通过一条特权指令（privileged instruction）完成这项工作（所有特权指令都用粗体标出）。在第二阶段（运行进程时），内核先做好一些准备（比如在进程列表中分配一个节点、分配内存），然后用一条从陷阱返回（return-from-trap）指令开始执行进程；这会把 CPU 切换到用户模式，并开始运行进程。进程想发出系统调用时，就会陷入操作系统；操作系统处理完后，再次通过从陷阱返回（return-from-trap）把控制权交还给进程。之后进程完成自己的工作，从 `main()` 返回；这通常会返回到一段存根代码中，由它正确地退出程序（比如调用 `exit()` 系统调用，这会陷入操作系统）。这时，操作系统完成清理工作，整个过程就结束了。

## 6.3 问题 2：在进程之间切换

直接执行的下一个问题是如何在进程之间切换。在进程之间切换应该很简单，对吧？操作系统只要决定停下一个进程、启动另一个进程就行了。这有什么大不了的？但实际上这有

[1]想象一下，跳进一段访问文件的代码，而且恰好跳过了权限检查；事实上，有了这种能力，狡猾的程序员很可能就能让内核运行任意的代码序列 [S07]。总之，尽量避免这类非常糟糕的主意。

| **操作系统 @ 启动**<br>**（内核模式）** | **硬件** | |
|---|---|---|
| **初始化陷阱表** | | |
| | 记住以下处理程序的地址……<br>系统调用处理程序 | |
| **操作系统 @ 运行**<br>**（内核模式）** | **硬件** | **程序**<br>**（用户模式）** |
| 在进程列表中创建条目<br>为程序分配内存<br>把程序加载到内存中<br>用 argv 设置用户栈<br>在内核栈中填入寄存器/PC<br>**从陷阱返回** | | |
| | （从内核栈）恢复寄存器<br>切换到用户模式<br>跳转到 main | |
| | | 运行 main()<br>…… |
| | | 调用系统调用<br>**陷入**操作系统 |
| | （向内核栈）保存寄存器<br>切换到内核模式<br>跳转到陷阱处理程序 | |
| 处理陷阱<br>完成系统调用的工作<br>**从陷阱返回** | | |
| | （从内核栈）恢复寄存器<br>切换到用户模式<br>跳转到陷阱之后的 PC | |
| | | ……<br>从 main 返回<br>**陷入**（通过 `exit()`） |
| 释放进程的内存<br>把进程从进程列表中删除 | | |

图 6.2：**受限直接执行协议**

点棘手：具体来说，如果一个进程正在 CPU 上运行，那么按照定义，操作系统就没有在运行。如果操作系统没有在运行，它又怎么能做任何事呢？（提示：它做不到）这听起来几乎像个哲学问题，但它是个实实在在的问题：如果操作系统没有在 CPU 上运行，它显然无法采取任何行动。问题的关键也就在这里。

关键问题：如何重新获得 CPU 的控制权
操作系统怎样才能**重新获得** CPU 的**控制权**（regain control），以便在进程之间切换？

 © 2008–23, ARPACI-DUSSEAU

提示：在安全的系统中，要提防用户的输入

为了在系统调用期间保护操作系统，我们费了很大力气：加入了硬件陷阱（trap）机制，并确保对操作系统的所有调用都经过它。尽管如此，要实现一个**安全的**（secure）操作系统，还有许多别的方面需要考虑。其中之一是处理系统调用边界上的参数；操作系统必须检查用户传入的内容，确保参数指定得正确，否则就拒绝这次调用。

例如，对于 `write()` 系统调用，用户要指定一个缓冲区（buffer）的地址，作为写入的数据来源。如果用户（无论是无意还是恶意）传入了一个“错误的”地址（比如位于地址空间（address space）中内核那一部分的地址），操作系统必须发现这一点并拒绝这次调用。否则，用户就有可能读取内核的全部内存；而内核的（虚拟）内存通常还包含系统的全部物理内存（physical memory），所以这个小小的疏忽，就能让程序读取系统中任何其他进程的内存。

总之，安全的系统必须对用户输入抱有高度的怀疑。不这样做，无疑会导致软件很容易被攻破，让人绝望地觉得世界是个不安全的可怕地方，还会让过于轻信的操作系统开发者丢掉饭碗。

### 协作方式：等待系统调用

过去有些系统采用的是所谓**协作**（cooperative）方式，例如早期版本的 Macintosh 操作系统 [M11]，以及老式的 Xerox Alto 系统 [A79]。在这种方式下，操作系统相信系统中的进程会表现得合乎情理。操作系统假定，运行时间太长的进程会定期让出（yield）CPU，好让操作系统决定去运行别的任务。

于是你可能会问：在这个理想世界里，友好的进程怎么让出 CPU 呢？其实，大多数进程会相当频繁地发出**系统调用**（system call），比如打开文件然后读取它、向另一台机器发送消息，或者创建新进程，CPU 的控制权也就随之交给了操作系统。这类系统通常还会提供一个显式的 **yield** 系统调用，它什么都不做，只是把控制权交给操作系统，好让操作系统运行其他进程。

应用程序做了非法的事情时，也会把控制权交给操作系统。例如，如果应用程序除以零，或者试图访问它不该访问的内存，就会产生一个**陷阱**（trap），陷入操作系统。这时操作系统又重新控制了 CPU，并且多半会终止这个违规的进程。

因此，在协作式调度系统中，操作系统要等系统调用或某种非法操作发生，才能重新获得 CPU 的控制权。你可能也在想：这种被动的方法是不是不太理想？比如，如果某个进程陷入了死循环（无论它是恶意的，还是满是 bug），再也不发出系统调用，会怎么样？那时操作系统又能怎么办？

### 非协作方式：操作系统接管控制权

其实，如果进程拒绝发出系统调用（或者出了错），不把控制权交还给操作系统，那么没有硬件的额外帮助，操作系统几乎什么也做不了。事实上，在协作方式（cooperative approach）下，进程陷入死循环时，你唯一的办法就是求助于计算机系统中解决一切问题

 © 2008–23, ARPACI-DUSSEAU

的古老方案：**重启机器**。于是，我们又遇到了“获得 CPU 控制权”这个总目标下的一个子问题。

关键问题：如何在没有协作的情况下获得控制权
即使进程不配合，操作系统怎样才能获得 CPU 的控制权？操作系统能做些什么，确保失控的进程不会接管整台机器？

答案其实很简单，许多构建计算机系统的人多年前就发现了：**时钟中断**（timer interrupt）[M+63]。可以对定时器设备编程，让它每隔若干毫秒产生一次中断；中断产生时，当前运行的进程会被暂停，操作系统中预先配置好的**中断处理程序**（interrupt handler）开始运行。此时，操作系统重新获得了 CPU 的控制权，于是可以随心所欲：停止当前进程，启动另一个进程。

和前面讨论系统调用时一样，操作系统必须告诉硬件，发生时钟中断时运行哪段代码；因此，操作系统在启动时就做了这件事。另外，同样是在启动过程中，操作系统必须启动定时器，这当然是一项特权操作。定时器一旦启动，操作系统就可以放心，因为控制权最终总会回到它手里，于是它可以自由地运行用户程序。定时器也可以关闭，这同样是一项特权操作；等我们更深入地了解并发（concurrency）之后，再讨论这一点。

注意，发生中断时，硬件也要承担一些责任，尤其是要为中断发生时正在运行的程序保存足够多的状态，以便随后的从陷阱返回（return-from-trap）指令能够正确地恢复这个程序的运行。这和显式的系统调用陷入内核时硬件的做法非常相似：硬件把各种寄存器保存下来（比如保存到内核栈上），从陷阱返回（return-from-trap）指令就能很容易地恢复它们。

## 保存和恢复上下文

既然操作系统已经重新获得了控制权，无论是通过系统调用协作地获得，还是通过时钟中断更强硬地夺回，接下来都要做一个决定：是继续运行当前正在运行的进程，还是切换到另一个进程。这个决定由操作系统中一个叫作**调度程序**（scheduler）的部分做出；接下来的几章会详细讨论调度策略（scheduling policy）。

如果决定切换，操作系统就会执行一段底层代码，我们称之为**上下文切换**（context switch）。上下文切换在概念上很简单：操作系统只要为当前正在执行的进程保存几个寄存器的值（比如保存到它的内核栈上），再为即将执行的进程恢复几个寄存器的值（从它的内核栈中恢复）。这样，最终执行从陷阱返回（return-from-trap）指令时，系统就不会回到原来那个进程，而是去恢复执行另一个进程。

提示：处理应用程序的不当行为
操作系统常常要处理行为不端的进程：它们或者出于故意（恶意），或者出于意外（bug），想做不该做的事。在现代系统中，操作系统处理这类不当行为的办法，就是直接终止违规者。一次出局！也许有些残酷，但当你试图非法访问内存或执行非法指令时，操作系统还能怎么办呢？

提示：用时钟中断重新获得控制权
加入**时钟中断**（timer interrupt）之后，即使进程不配合，操作系统也能重新在 CPU 上运行。因此，要帮操作系统保持对机器的控制，这项硬件特性至关重要。

提示：重启很有用
前面提到，在协作式抢占（cooperative preemption）下，死循环（以及类似行为）唯一的解决办法就是重启机器。你可能会对这种拙劣的办法嗤之以鼻，但研究人员已经证明，重启（或者更一般地说，让某个软件从头再来）在构建健壮的系统时可能是一个极其有用的工具 [C+04]。
具体来说，重启之所以有用，是因为它把软件带回到一个已知的、很可能经过了更多测试的状态。重启还会回收过时的或泄漏的资源（比如内存），这些资源用别的办法可能很难处理。最后，重启很容易自动化。出于上述种种原因，在大规模集群的互联网服务中，系统管理软件定期**重启**（reboot）一批机器、让它们复位，从而获得上面列出的好处，并不是什么稀罕事。
所以，下次你重启机器时，你可不只是在用某种拙劣的办法应付了事，而是在用一种久经考验的方法改善计算机系统的行为。干得好！

为了保存当前正在运行的进程的上下文，操作系统会执行一些底层汇编代码，保存当前进程的通用寄存器、PC 和内核栈指针，然后恢复即将执行的进程的这些寄存器和 PC，并切换到它的内核栈。因为切换了栈，内核进入切换代码时还处在被中断进程的上下文中，返回时却已经处在即将执行的那个进程的上下文中了。当操作系统最终执行从陷阱返回（return-from-trap）指令时，即将执行的进程就成了当前正在运行的进程。上下文切换就此完成。

图 6.3 展示了整个过程的时间线。在这个例子中，进程 A 正在运行，然后被时钟中断打断。硬件把 A 的寄存器保存到 A 的内核栈上，然后进入内核，切换到内核模式。在时钟中断处理程序（timer interrupt handler）中，操作系统决定从运行进程 A 切换到运行进程 B。这时，它调用 `switch()` 例程，这个例程小心地把当前的寄存器值保存到 A 的进程结构（process structure）中，从 B 的进程结构条目中恢复进程 B 的寄存器，然后**切换上下文**（switches contexts）：具体做法是修改栈指针（stack pointer），改用 B 的内核栈（而不是 A 的）。最后，操作系统从陷阱返回（return-from-trap），恢复 B 的寄存器，并开始运行 B。

注意，在这个协议中发生了两种寄存器的保存和恢复。第一种发生在时钟中断时；这时，正在运行的进程的*用户寄存器*由*硬件*隐式地保存，用的是这个进程的内核栈。第二种发生在操作系统决定从 A 切换到 B 时；这时，*内核寄存器*由*软件*（即操作系统）显式地保存，不过这次是保存到内存中这个进程的进程结构（process structure）里。后一个动作让系统从“好像刚从 A 陷入内核”变成“好像刚从 B 陷入内核”。

 © 2008–23, ARPACI-DUSSEAU

| 操作系统 @ 启动（内核模式） | 硬件 |
|---|---|
| **初始化陷阱表** | |
| | 记住以下处理程序的地址……<br>系统调用处理程序<br>定时器处理程序 |
| **启动中断定时器** | |
| | 启动定时器<br>X 毫秒后中断 CPU |

| 操作系统 @ 运行（内核模式） | 硬件 | 程序（用户模式） |
|---|---|---|
| | | 进程 A<br>…… |
| | **时钟中断**<br>保存 regs(A) → k-stack(A)<br>切换到内核模式<br>跳转到陷阱处理程序 | |
| 处理陷阱<br>调用 `switch()` 例程<br>保存 regs(A) → proc_t(A)<br>恢复 regs(B) ← proc_t(B)<br>切换到 k-stack(B)<br>**从陷阱返回（进入 B）** | | |
| | 恢复 regs(B) ← k-stack(B)<br>切换到用户模式<br>跳转到 B 的 PC | |
| | | 进程 B<br>…… |

图 6.3：**受限直接执行协议（时钟中断）**

为了让你更清楚地了解这种切换是怎么实现的，图 6.4 给出了 xv6 的上下文切换代码。看看你能不能看懂它（要看懂，你得懂一点 x86，还得懂一点 xv6）。上下文结构 `old` 和 `new` 分别位于旧进程和新进程的进程结构（process structure）中。

## 6.4 担心并发?

你们中一些专心又爱思考的读者，现在可能在想：“嗯……如果在系统调用期间发生了时钟中断，会怎么样?”或者“如果正在处理一个中断时又来了另一个中断，会怎么样？内核处理起来不会很难吗?”问得好——看来你们还是很有希望的！

答案是肯定的，操作系统确实需要考虑在处理中断或陷阱（trap）期间又发生另一个中断时会怎样。事实上，这正是本书整个第二部分讨论的主题，也就是**并发**（concurrency）；我们会把详细的讨论推迟到那时。

 © 2008–23, ARPACI-DUSSEAU

```
1  # void swtch(struct context *old, struct context *new);
2  #
3  # 把当前的寄存器上下文保存到 old 中，
4  # 然后从 new 中加载寄存器上下文。
5  .globl swtch
6  swtch:
7    # 保存旧的寄存器
8    movl 4(%esp), %eax  # 把 old 指针放进 eax
9    popl 0(%eax)        # 保存旧的 IP
10   movl %esp, 4(%eax)  # 以及栈指针
11   movl %ebx, 8(%eax)  # 以及其他寄存器
12   movl %ecx, 12(%eax)
13   movl %edx, 16(%eax)
14   movl %esi, 20(%eax)
15   movl %edi, 24(%eax)
16   movl %ebp, 28(%eax)
17
18   # 加载新的寄存器
19   movl 4(%esp), %eax  # 把 new 指针放进 eax
20   movl 28(%eax), %ebp # 恢复其他寄存器
21   movl 24(%eax), %edi
22   movl 20(%eax), %esi
23   movl 16(%eax), %edx
24   movl 12(%eax), %ecx
25   movl 8(%eax), %ebx
26   movl 4(%eax), %esp  # 栈在这里切换
27   pushl 0(%eax)       # 把返回地址放到位
28   ret                 # 最后返回到新的上下文中
```

图 6.4：**xv6 的上下文切换代码**

为了吊吊你的胃口，我们先简单勾勒一下操作系统处理这些棘手情况的一些基本思路。操作系统可以做的一件简单的事，是在处理中断期间**禁用中断**（disable interrupts）；这样可以确保在处理一个中断时，不会有其他中断被送到 CPU。当然，操作系统这样做时必须小心；中断禁用得太久，可能导致中断丢失，而这（用技术术语来说）很糟糕。

操作系统还开发了许多精巧的**加锁**（locking）方案，来保护对内部数据结构（data structure）的并发访问。这样，内核中就可以同时进行多项活动，这在多处理器（multiprocessor）上尤其有用。不过，我们会在本书关于并发的下一部分中看到，这类加锁可能很复杂，还会导致各种有趣又难以发现的 bug。

## 6.5 小结

我们介绍了实现 CPU 虚拟化的一些关键底层机制，这组技术统称为**受限直接执行**（limited direct execution）。基本思路很简单：直接在 CPU 上运行你想运行的程序，但事先要设置好硬件，限制进程在没有操作系统协助的情况下能做的事。

补充：上下文切换要花多长时间

你自然会问：上下文切换这样的操作要花多长时间？系统调用呢？对此好奇的读者可以用一个叫作 **lmbench** [MS96] 的工具，它测量的正是这些东西，还有其他几项可能相关的性能指标。

随着时间的推移，测量结果改善了很多，大致和处理器性能的提升同步。例如，1996 年在 200 MHz 的 P6 CPU 上运行 Linux 1.3.37 时，一次系统调用大约需要 4 微秒，一次上下文切换大约需要 6 微秒 [MS96]。现代系统的表现几乎好了一个数量级，在 2 GHz 或 3 GHz 处理器的系统上，结果都在 1 微秒以下。

需要指出的是，并非所有的操作系统操作都和 CPU 性能同步提升。正如 Ousterhout 所观察到的，许多操作系统操作是内存密集型的，而这些年来内存带宽（bandwidth）的提升远不如处理器速度那么显著 [O90]。因此，视你的工作负载（workload）而定，买最新最好的处理器，未必能像你希望的那样让操作系统变快。

现实生活中也有类似的做法。例如，有孩子的人，或者至少听说过孩子的人，大概都知道给房间做**婴儿防护**（baby proofing）是怎么回事：把装着危险物品的柜子锁上，把电源插座盖住。房间这样准备好之后，你就可以放心地让孩子随意走动，因为你知道房间里最危险的地方都已经被限制住了。

与此类似，操作系统也给 CPU“做了婴儿防护（baby proofing）”：先（在启动时）设置好陷阱处理程序（trap handler）、启动中断定时器（interrupt timer），然后只在受限的模式下运行进程。这样，操作系统就可以相当放心：进程能够高效地运行，只有在执行特权操作，或者独占 CPU 太久、需要被换下时，才需要操作系统介入。

至此，我们有了虚拟化 CPU 的基本机制。但还有一个重大问题没有回答：在某一时刻应该运行哪个进程？回答这个问题正是调度程序的任务，也是我们接下来要研究的主题。

 © 2008–23, ARPACI-DUSSEAU

补充：CPU 虚拟化关键术语（机制）

- CPU 应该至少支持两种执行模式：受限的**用户模式**（user mode）和特权的（不受限的）**内核模式**（kernel mode）。
- 典型的用户应用程序在用户模式下运行，通过**系统调用**（system call）**陷入**（trap）内核，请求操作系统提供服务。
- 陷阱（trap）指令会小心地保存寄存器状态，把硬件状态切换到内核模式，然后跳转到操作系统中预先指定的目的地：**陷阱表**（trap table）。
- 操作系统处理完系统调用后，通过另一条特殊的**从陷阱返回**（return-from-trap）指令返回到用户程序；这条指令会降低特权级别，并把控制权交还给跳入操作系统的那条陷阱（trap）指令之后的指令。
- 陷阱表（trap table）必须由操作系统在启动时设置好，操作系统还要确保用户程序无法轻易修改它。这些都是**受限直接执行**（limited direct execution）协议的一部分，这个协议既能高效地运行程序，又不会让操作系统失去控制。
- 程序一旦开始运行，操作系统就必须借助硬件机制，确保用户程序不会永远运行下去，这个机制就是**时钟中断**（timer interrupt）。这种方法是 CPU 调度的一种**非协作**（non-cooperative）方式。
- 有时，操作系统在时钟中断或系统调用期间，可能想从运行当前进程切换到运行另一个进程，实现这种切换的底层技术叫作**上下文切换**（context switch）。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[A79] "Alto User's Handbook" by Xerox. Xerox Palo Alto Research Center, September 1979. Available: http://history-computer.com/Library/AltoUsersHandbook.pdf. *An amazing system, way ahead of its time. Became famous because Steve Jobs visited, took notes, and built Lisa and eventually Mac.*

[C+04] "Microreboot – A Technique for Cheap Recovery" by G. Candea, S. Kawamoto, Y. Fujiki, G. Friedman, A. Fox. OSDI '04, San Francisco, CA, December 2004. *An excellent paper pointing out how far one can go with reboot in building more robust systems.*

[I11] "Intel 64 and IA-32 Architectures Software Developer's Manual" by Volume 3A and 3B: System Programming Guide. Intel Corporation, January 2011. *This is just a boring manual, but sometimes those are useful.*

[K+61] "One-Level Storage System" by T. Kilburn, D.B.G. Edwards, M.J. Lanigan, F.H. Sumner. IRE Transactions on Electronic Computers, April 1962. *The Atlas pioneered much of what you see in modern systems. However, this paper is not the best one to read. If you were to only read one, you might try the historical perspective below [L78].*

[L78] "The Manchester Mark I and Atlas: A Historical Perspective" by S. H. Lavington. Communications of the ACM, 21:1, January 1978. *A history of the early development of computers and the pioneering efforts of Atlas.*

[M+63] "A Time-Sharing Debugging System for a Small Computer" by J. McCarthy, S. Boilen, E. Fredkin, J. C. R. Licklider. AFIPS '63 (Spring), May, 1963, New York, USA. *An early paper about time-sharing that refers to using a timer interrupt; the quote that discusses it: "The basic task of the channel 17 clock routine is to decide whether to remove the current user from core and if so to decide which user program to swap in as he goes out."*

[MS96] "lmbench: Portable tools for performance analysis" by Larry McVoy and Carl Staelin. USENIX Annual Technical Conference, January 1996. *A fun paper about how to measure a number of different things about your OS and its performance. Download lmbench and give it a try.*

[M11] "Mac OS 9" by Apple Computer, Inc.. January 2011. Available at the following URL: http://en.wikipedia.org/wiki/Mac_OS_9. *You can probably even find an OS 9 emulator out there if you want to; check it out, it's a fun little Mac!*

[O90] "Why Aren't Operating Systems Getting Faster as Fast as Hardware?" by J. Ousterhout. USENIX Summer Conference, June 1990. *A classic paper on the nature of operating system performance.*

[P10] "The Single UNIX Specification, Version 3" by The Open Group, May 2010. Available: http://www.unix.org/version3/. *This is hard and painful to read, so probably avoid it if you can. Like, unless someone is paying you to read it. Or, you're just so curious you can't help it!*

[S07] "The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)" by Hovav Shacham. CCS '07, October 2007. *One of those awesome, mind-blowing ideas that you'll see in research from time to time. The author shows that if you can jump into code arbitrarily, you can essentially stitch together any code sequence you like (given a large code base); read the paper for the details. The technique makes it even harder to defend against malicious attacks, alas.*

## 作业（测量）

补充：测量作业

测量作业是一些小练习：你要编写代码，在真实的机器上运行，测量操作系统或硬件性能的某个方面。这类作业的目的，是让你对真实的操作系统有一点亲手操作的经验。

在这个作业中，你要测量系统调用和上下文切换的开销。测量系统调用的开销相对容易。例如，你可以反复调用一个简单的系统调用（比如读取 0 字节），并测量所花的时间；用总时间除以迭代次数，就能估算出一次系统调用的开销。

有一件事必须考虑：计时器的精度和准确度。常用的计时器是 `gettimeofday()`；详情请读手册页（manual page）。你会在手册页中看到，`gettimeofday()` 返回自 1970 年以来的时间，以微秒为单位；但这并不意味着这个计时器能精确到微秒。连续调用 `gettimeofday()` 并测量结果，了解这个计时器到底有多精确；这会告诉你，空系统调用（null system call）测试要迭代多少次，才能得到可靠的测量结果。如果 `gettimeofday()` 对你来说不够精确，可以研究一下 x86 机器上的 `rdtsc` 指令。

测量上下文切换的开销要稍微棘手一些。lmbench 基准测试的做法是：在单个 CPU 上运行两个进程，并在它们之间建立两个 UNIX 管道（pipe）；管道只是 UNIX 系统中进程之间相互通信的众多方式之一。第一个进程向第一个管道写入数据，然后等待从第二个管道读取数据；操作系统看到第一个进程在等待从第二个管道读数据，就把它置于阻塞状态，并切换到另一个进程；另一个进程从第一个管道读取数据，然后向第二个管道写入数据。当第二个进程再次试图从第一个管道读数据时，它也会阻塞，于是这种来回通信的循环就持续下去。lmbench 反复测量这种通信的开销，就能较好地估算出上下文切换的开销。你可以用管道，或者 UNIX 套接字（socket）之类的其他通信机制，尝试实现类似的测量。

在多 CPU 系统上测量上下文切换的开销会遇到一个困难：你必须确保进行上下文切换的进程位于同一个处理器上。幸运的是，大多数操作系统都提供了把进程绑定到某个处理器上的调用；例如，在 Linux 上，你要找的就是 `sched_setaffinity()` 调用。确保两个进程位于同一个处理器上，你测量的就是操作系统在同一个 CPU 上停下一个进程、恢复另一个进程的开销。

# 7

# 调度：简介

到现在为止，运行进程（process）的底层**机制**（mechanism），例如上下文切换（context switch），应该已经清楚了；如果还不清楚，请回到前一两章，把这些机制的工作原理再读一遍。不过，我们还没有弄明白操作系统调度程序（scheduler）所采用的上层**策略**（policy）。下面就来做这件事：我们会介绍一系列**调度策略**（scheduling policy，有时也叫**调度规则**，discipline），它们是许多聪明又勤奋的人多年来陆续提出的。

事实上，调度的起源比计算机系统还早；最早的一些方法来自运筹管理领域，后来才用到计算机上。这一点不足为奇：流水线以及人类的许多其他活动也都需要调度，其中有许多相同的考量，包括对效率近乎执着的追求。于是，我们的问题是：

关键问题：如何制定调度策略
思考调度策略，该从怎样的基本框架入手？有哪些关键的假设？哪些指标是重要的？最早的计算机系统采用过哪些基本方法？

## 7.1 工作负载假设

在探讨各种可能的策略之前，我们先对系统中运行的进程做一些简化假设，这些进程有时统称为**工作负载**（workload）。确定工作负载是制定策略的关键一环；对工作负载了解得越多，策略就能调校得越精细。

我们这里对工作负载所做的假设大多并不现实，不过这（暂时）没关系，因为我们会随着讨论的深入逐步放宽这些假设，最终得到一个可以称为……（戏剧性停顿）……**全面运转的调度规则**[1]的东西。

对于系统中运行的进程（有时也叫作业），我们做以下假设：

1. 每个作业的运行时间相同。
2. 所有作业同时到达。
3. 作业一旦开始，就一直运行到完成。
4. 所有作业都只使用 CPU（即不执行 I/O）。
5. 每个作业的运行时间是已知的。

[1]请用说“一个全面运转的死星”（A fully-operational Death Star）的语气读出来。

我们说过其中许多假设都不现实；不过，正如奥威尔的《动物庄园》[O45] 里有些动物比其他动物更平等，本章中也有些假设比其他假设更不现实。尤其是“每个作业的运行时间已知”这一条，也许会让你觉得别扭：这等于说调度程序无所不知；这样（大概）固然很好，但短期内恐怕不会实现。

## 7.2 调度指标

除了对工作负载做出假设，我们还需要一样东西来比较不同的调度策略：**调度指标**（scheduling metric）。指标就是我们用来衡量某样东西的标准，而在调度中，有好几种指标都说得通。

不过眼下，我们还是让生活简单一点，只用一个指标：**周转时间**（turnaround time）。作业的周转时间定义为作业完成的时刻减去作业到达系统的时刻。更形式化地说，周转时间 $T_{turnaround}$ 为：

$$T_{turnaround} = T_{completion} - T_{arrival} \tag{7.1}$$

由于我们假设所有作业同时到达，所以目前 $T_{arrival} = 0$，因而 $T_{turnaround} = T_{completion}$。随着我们放宽前面的假设，这一点会有所改变。

你应该注意到，周转时间是一个**性能**（performance）指标，它是本章关注的重点。另一个值得关注的指标是**公平性**（fairness），可以用 **Jain 公平性指数**（Jain's Fairness Index）[J91] 等方式来衡量。在调度中，性能和公平往往互相矛盾；例如，调度程序可能优化了性能，代价却是让几个作业无法运行，从而降低了公平性。这个难题告诉我们，生活并不总是完美的。

## 7.3 先进先出（FIFO）

我们能实现的最基本的算法叫作**先进先出**（First In, First Out，**FIFO**）调度，有时也叫**先来先服务**（First Come, First Served，**FCFS**）。

FIFO 有不少优点：它显然很简单，因而容易实现。而且，在我们的假设下，它的效果相当不错。

我们一起来看一个简单的例子。假设系统中大约同时（$T_{arrival} = 0$）到达了三个作业：A、B 和 C。由于 FIFO 总得让某个作业排在最前面，我们假设虽然它们同时到达，但 A 比 B 早到了一丁点，B 又比 C 早到了一丁点。再假设每个作业都运行 10 秒。这些作业的**平均周转时间**（average turnaround time）是多少？

从图 7.1 可以看出，A 在 10 秒时完成，B 在 20 秒时完成，C 在 30 秒时完成。因此，这三个作业的平均周转时间就是 $\frac{10+20+30}{3} = 20$。计算周转时间就这么简单。

现在我们放宽其中一个假设。具体来说，放宽假设 1，不再假设每个作业的运行时间相同。这时 FIFO 表现如何？你能构造出什么样的工作负载，让 FIFO 表现很差？

（继续读之前先想一想……接着想……想到了吗?!）

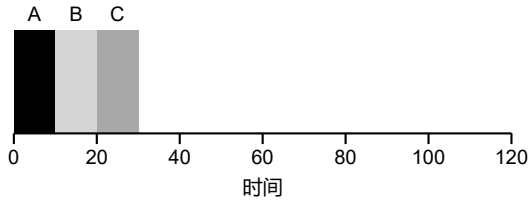



图 7.1：**FIFO 的简单例子**

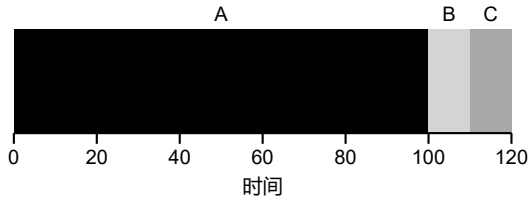



图 7.2：**FIFO 为什么没那么好**

想必你已经想出来了，不过以防万一，我们还是举个例子，看看长度不同的作业会给 FIFO 调度带来什么麻烦。具体来说，仍然假设有三个作业（A、B 和 C），但这次 A 运行 100 秒，而 B 和 C 各运行 10 秒。

从图 7.2 可以看到，作业 A 先完整地运行了 100 秒，B 和 C 才有机会运行。因此，系统的平均周转时间很高：足足 110 秒（$\frac{100+110+120}{3} = 110$），令人痛苦。

这个问题通常称为**护航效应**（convoy effect）[B+79]：一些耗时相对较短的资源使用者，排在了一个重量级资源使用者后面。这种调度场景也许会让你想起超市里只有一条结账队伍的情形，想起看到排在你前面的人推着三车满满的货物、还掏出了支票簿时的心情：这得等上好一阵子了[2]。

那么该怎么办？面对作业运行时间长短不一的新现实，我们怎样才能设计出更好的算法？先想一想，再往下读。

提示：SJF 原则

最短作业优先（Shortest Job First）体现了一条通用的调度原则：只要系统在乎每位顾客（在本书里就是作业）感受到的周转时间，就可以用它。想想你排过的任何一条队：如果相关的店家在乎顾客满意度，他们多半已经考虑到了 SJF。例如，超市通常设有“十件以下”通道，以免只买几样东西的顾客排在一家人后面干等，而那家人正为某场即将到来的核冬天囤货。

[2]这种情况下的建议做法：要么赶紧换一条队，要么深深地、慢慢地、放松地吸一口气。没错，吸气，呼气。没事的，别担心。

 © 2008–23, ARPACI-DUSSEAU

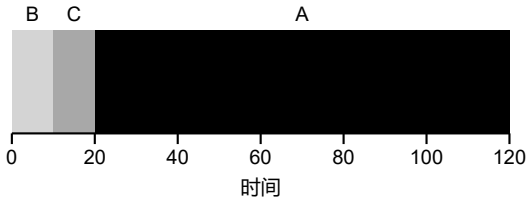



图 7.3：**SJF 的简单例子**

## 7.4 最短作业优先（SJF）

其实，一个非常简单的方法就能解决这个问题。这个想法是从运筹学 [C54, PV56] 借来的，后来用到了计算机系统的作业调度上。这种新的调度规则（scheduling discipline）叫作**最短作业优先**（Shortest Job First，SJF），名字应该很好记，因为它把策略说得相当完整：先运行最短的作业，再运行次短的，依此类推。

我们还用上面的例子，但改用 SJF 作为调度策略。图 7.3 显示了运行 A、B、C 的结果。看了这张图，你应该就明白为什么 SJF 在平均周转时间上表现得好得多了。只是把 B 和 C 放在 A 之前运行，SJF 就把平均周转时间从 110 秒降到了 50 秒（$\frac{10+20+120}{3} = 50$），改善了一倍多。

事实上，在所有作业同时到达的假设下，我们可以证明 SJF 确实是一种**最优**（optimal）调度算法。不过，你上的是系统课，不是理论课或运筹学课；这里不许证明。

于是，我们借助 SJF 找到了一种不错的调度方法，但我们的假设仍然相当不现实。我们再放宽一个。具体来说，可以针对假设 2，改为假设作业可以在任何时刻到达，而不是同时到达。这会带来什么问题?

（再停下来想一想……你在想吗？加油，你能行的）

我们还是用一个例子来说明问题。这次假设 A 在 $t = 0$ 时到达，需要运行 100 秒；而 B 和 C 在 $t = 10$ 时到达，各需要运行 10 秒。如果用纯粹的 SJF，就会得到图 7.4 所示的调度。

从图中可以看出，虽然 B 和 C 在 A 之后不久就到达了，它们仍然不得不等到 A 完成，因此遭遇了同样的护航问题。这三个作业的平均周转时间是 103.33 秒（$\frac{100+(110-10)+(120-10)}{3}$）。调度程序能做些什么呢?

补充：抢占式调度程序

在过去的批处理（batch processing）时代，人们开发了一些**非抢占式**（non-preemptive）调度程序；这类系统会把每个作业都运行到完成，然后才考虑是否运行新的作业。几乎所有现代调度程序都是**抢占式**（preemptive）的，它们很乐意暂停一个进程的运行，转而运行另一个进程。这意味着调度程序用到了我们之前学过的机制；具体来说，调度程序可以执行**上下文切换**（context switch），暂时停下一个正在运行的进程，恢复（或启动）另一个进程。

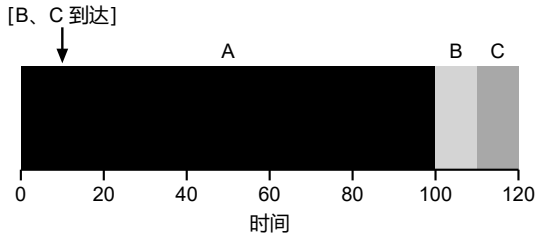



图 7.4：**B 和 C 晚到时的 SJF**

## 7.5 最短完成时间优先（STCF）

为了解决这个问题，我们需要放宽假设 3（作业必须运行到完成），那就放宽吧。我们还需要调度程序本身提供一些机制。根据前面对时钟中断（timer interrupt）和上下文切换的讨论，你可能已经猜到，B 和 C 到达时，调度程序当然可以有别的做法：它可以**抢占**（preempt）作业 A，决定运行另一个作业，以后再接着运行 A。按我们的定义，SJF 是一种**非抢占式**（non-preemptive）调度程序，因此会遇到上面描述的问题。

幸运的是，有一种调度程序恰好能做到这一点：给 SJF 加上抢占，这就是**最短完成时间优先**（Shortest Time-to-Completion First，**STCF**）调度程序，也叫**抢占式最短作业优先**（Preemptive Shortest Job First，**PSJF**）调度程序 [CK68]。每当有新作业进入系统，STCF 调度程序就判断剩下的作业（包括新作业）中谁的剩余时间最少，然后调度那个作业。因此，在我们的例子中，STCF 会抢占 A，把 B 和 C 运行到完成；只有它们都完成之后，才会调度 A 剩下的时间。图 7.5 展示了一个例子。

结果，平均周转时间大为改善：$\frac{(120-0)+(20-10)+(30-10)}{3} = 50$ 秒。和前面一样，在新的假设下，可以证明 STCF 是最优的；既然所有作业同时到达时 SJF 是最优的，你大概也能凭直觉看出 STCF 为什么是最优的。

## 7.6 新的指标：响应时间

因此，如果我们知道作业的长度，作业只使用 CPU，而且唯一的指标是周转时间，那么 STCF 就是一种很好的策略。事实上，对早期的一些批处理计算系统来说，这类调度算法也

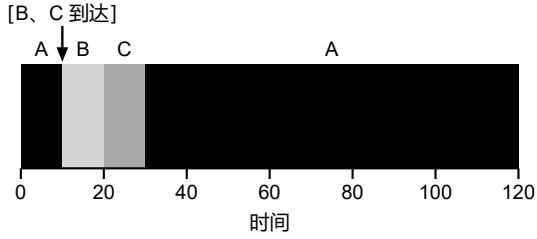



图 7.5：**STCF 的简单例子**

 © 2008–23, ARPACI-DUSSEAU

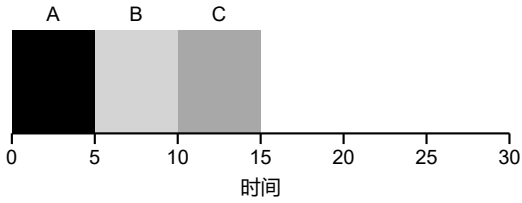



图 7.6：**又是 SJF（对响应时间不利）**

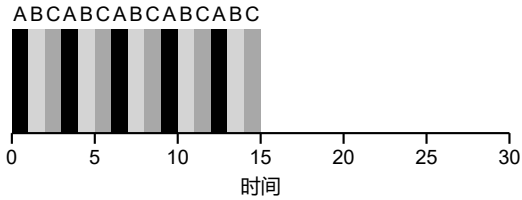



图 7.7：**时间片轮转（对响应时间有利）**

说得通。然而，分时（time sharing）机器的出现改变了这一切。这时用户坐在终端前操作，还要求系统有良好的交互性能。于是，一种新的指标诞生了：**响应时间**（response time）。

我们把响应时间定义为从作业到达系统到它第一次被调度的时间[3]。更形式化地说：

$$T_{response} = T_{firstrun} - T_{arrival} \tag{7.2}$$

例如，如果采用图 7.5 中的调度（A 在时刻 0 到达，B 和 C 在时刻 10 到达），各个作业的响应时间如下：作业 A 为 0，B 为 0，C 为 10（平均：3.33）。

你可能已经想到，STCF 及类似的调度规则（scheduling discipline）对响应时间并不怎么友好。例如，如果三个作业同时到达，第三个作业必须等前两个作业完整地运行完，才能被调度一次。这种方法虽然对周转时间很有利，对响应时间和交互性（interactivity）却相当糟糕。试想一下，你坐在终端前打字，却因为别的作业排在你前面，要等 10 秒才能看到系统的响应：可不怎么愉快。

于是，我们又面临一个问题：如何构建一个对响应时间敏感的调度程序？

## 7.7 时间片轮转

为了解决这个问题，我们引入一种新的调度算法，传统上称为**时间片轮转**（Round-Robin，**RR**）调度 [K64]。基本思路很简单：RR 不把作业运行到完成，而是让一个作业运行一个**时间片**（time slice，有时也叫**调度量子**，scheduling quantum），然后切换到运行队列（run queue）中的下一个作业。它反复这样做，直到所有作业都完成。因此，RR 有时也叫**时间分片**（time-slicing）。注意，时间片的长度必须是时钟中断周期的整数倍；因此，

[3]有些人的定义略有不同，例如还要算上作业产生某种“响应”之前的时间；我们的定义是其中最理想的情况，相当于假设作业一被调度就立即产生响应。

提示：平摊可以降低开销

某个操作有固定开销时，系统中常用一种通用技术：**平摊**（amortization）。少承担几次这笔开销（即少执行几次这个操作），系统的总开销就降下来了。例如，如果时间片设为 10 毫秒，而上下文切换的开销是 1 毫秒，那么大约 10% 的时间花在了上下文切换上，因而被浪费掉了。如果想平摊（amortize）这笔开销，可以加大时间片，比如设为 100 毫秒。这样一来，花在上下文切换上的时间不到 1%，时间分片（time-slicing）的开销就被**平摊**（amortize）掉了。

如果时钟每 10 毫秒中断（interrupt）一次，时间片就可以是 10 毫秒、20 毫秒或者其他任何 10 毫秒的倍数。

为了更详细地理解 RR，我们来看一个例子。假设三个作业 A、B、C 同时到达系统，而且都要运行 5 秒。SJF 调度程序会把每个作业都运行到完成，再运行下一个（图 7.6）。相比之下，时间片为 1 秒的 RR 会快速地在这几个作业之间轮转（图 7.7）。

RR 的平均响应时间是 $\frac{0+1+2}{3} = 1$；而 SJF 的平均响应时间是 $\frac{0+5+10}{3} = 5$。

可以看到，时间片的长度对 RR 至关重要。时间片越短，RR 在响应时间这一指标上的表现就越好。然而，时间片太短也有问题：上下文切换的开销会一下子成为左右整体性能的主要因素。因此，确定时间片长度对系统设计者来说是一种权衡：时间片要足够长，以便平摊（amortize）切换的开销；又不能太长，以免系统失去响应能力。

注意，上下文切换的开销并不只来自操作系统保存和恢复几个寄存器的操作。程序运行时，会在 CPU 缓存（cache）、TLB、分支预测器以及其他片上硬件中积累大量状态。切换到另一个作业时，这些状态会被清掉，换上与新作业相关的状态，这可能带来相当明显的性能开销 [MB91]。

因此，如果响应时间是我们唯一的指标，采用合理时间片的 RR 就是一个出色的调度程序。可是我们的老朋友周转时间呢？再来看上面的例子。A、B、C 的运行时间都是 5 秒，它们同时到达，调度程序是时间片为 1 秒（已经算长了）的 RR。从上图可以看出，A 在 13 秒时完成，B 在 14 秒时完成，C 在 15 秒时完成，平均为 14。相当糟糕！

所以毫不奇怪，如果以周转时间为指标，RR 确实是最差的策略之一。从直觉上讲，这也说得通：RR 每个作业只运行一小会儿就转到下一个，把每个作业的完成时间都拖到了尽可能晚。由于周转时间只关心作业何时完成，RR 几乎是最差的，在很多情况下甚至比简单的 FIFO 还差。

更一般地说，任何**公平**（fair）的策略（例如 RR），也就是在很短的时间尺度上把 CPU 平均分给各个活动进程的策略，在周转时间这类指标上都会表现不佳。这确实是一种固有的**权衡**（trade-off）：如果你愿意不公平，就可以先把较短的作业运行完，代价是响应时间；如果你看重公平，响应时间就会降低，代价是周转时间。这类权衡在系统中很常见；鱼与熊掌不可兼得[4]。

[4]这句英文俗语（You can't have your cake and eat it too）常常让人糊涂，因为它本该是“You can't *keep* your cake and eat it too”（蛋糕不能既留着又吃掉，这不是很明显吗？）。令人惊讶的是，维基百科上竟然有一页专门讲这句俗语；更令人惊讶的是，那一页读起来还挺有意思 [W15]。用意大利语说，就是不能 *Avere la botte piena e la moglie ubriaca*（既要酒桶满满，又要老婆喝醉）。

提示：重叠可以提高利用率
只要有可能，就让操作**重叠**（overlap）起来，以最大限度地提高系统的利用率（utilization）。重叠在许多不同的领域都很有用，比如执行磁盘 I/O 或向远程机器发送消息的时候；在这两种情况下，先启动操作，再切换去做其他工作，都是好主意，可以提高系统的整体利用率和效率。

我们已经设计了两类调度程序。第一类（SJF、STCF）优化周转时间，但对响应时间不利。第二类（RR）优化响应时间，但对周转时间不利。我们还有两个假设需要放宽：假设 4（作业不执行 I/O）和假设 5（每个作业的运行时间已知）。接下来就来处理这两个假设。

## 7.8 结合 I/O

首先，我们放宽假设 4：当然，所有程序都会执行 I/O。试想一个不接受任何输入的程序：它每次都会产生同样的输出。再试想一个没有输出的程序：它就像那句老话里在森林里倒下、却没人看见的树；它运行了也无关紧要。

作业发起 I/O 请求时，调度程序显然要做一个决定，因为当前正在运行的作业在 I/O 期间不会使用 CPU；它被**阻塞**（blocked），等待 I/O 完成。如果 I/O 是发给硬盘驱动器（hard disk drive）的，进程可能会被阻塞几毫秒甚至更久，具体取决于驱动器当前的 I/O 负载。因此，调度程序这时大概应该在 CPU 上调度另一个作业。

I/O 完成时，调度程序也要做一个决定。这时会产生一个中断，操作系统开始运行，把发起 I/O 的进程从阻塞状态移回就绪（ready）状态。当然，它甚至可以决定在那时就运行这个作业。操作系统应该如何对待每个作业呢？

为了更好地理解这个问题，假设有两个作业 A 和 B，每个都需要 50 毫秒的 CPU 时间。不过，两者有一个明显的区别：A 运行 10 毫秒后就发起一次 I/O 请求（这里假设每次 I/O 耗时 10 毫秒），而 B 只是使用 CPU 50 毫秒，不执行 I/O。调度程序先运行 A，再运行 B（图 7.8）。

假设我们要构建一个 STCF 调度程序。A 被拆成了 5 个 10 毫秒的子作业，而 B 只是一次 50 毫秒的 CPU 需求，这样的调度程序该怎样考虑这一点呢？显然，只运行完一个作业再运行另一个，而不考虑如何把 I/O 纳入进来，是不太合理的。

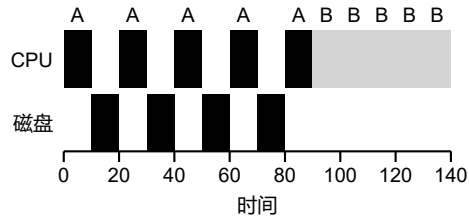



图 7.8：**资源利用不佳**

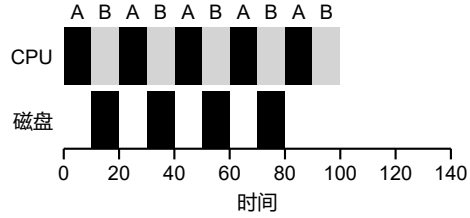



图 7.9：**重叠让资源得到更好的利用**

一种常见的做法是把 A 的每个 10 毫秒子作业都当作一个独立的作业。这样，系统一开始要做的选择是：调度一个 10 毫秒的 A，还是一个 50 毫秒的 B。用 STCF，选择很明确：选较短的那个，在这里就是 A。然后，A 的第一个子作业完成后，就只剩下 B，于是 B 开始运行。接着，A 提交了一个新的子作业，它抢占 B 并运行 10 毫秒。这样做就实现了**重叠**（overlap）：一个进程在使用 CPU 的同时，另一个进程在等待 I/O 完成；系统因此得到了更好的利用（见图 7.9）。

由此可见调度程序如何把 I/O 纳入考虑：把每个 CPU 突发（CPU burst）都当作一个作业，调度程序就能确保“交互式”的进程频繁地运行。在这些交互式作业执行 I/O 时，其他 CPU 密集型作业得以运行，从而更好地利用了处理器。

## 7.9 不再有先知

有了处理 I/O 的基本方法，我们来看最后一个假设：调度程序知道每个作业的长度。我们之前说过，这大概是我们能做出的最糟糕的假设。事实上，在通用操作系统中（比如我们关心的这些），操作系统通常对每个作业的长度知之甚少。那么，在没有这种先验（a priori）知识的情况下，我们如何构建一种表现得像 SJF/STCF 的方法呢？此外，我们如何把 RR 调度程序中的一些思想融合进来，让响应时间也相当好呢？

## 7.10 小结

我们介绍了调度背后的基本思想，并提出了两类方法。第一类运行剩余时间最短的作业，从而优化周转时间；第二类在所有作业之间交替运行，从而优化响应时间。一类的长处恰恰是另一类的短处，唉，这是系统中常见的固有权衡。我们还看到了如何把 I/O 纳入考虑，但仍然没有解决操作系统根本无法预知未来这个问题。很快我们就会看到如何克服这个问题：构建一个用最近的过去来预测未来的调度程序。这种调度程序叫作**多级反馈队列**（multi-level feedback queue），它是下一章的主题。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[B+79] “The Convoy Phenomenon” by M. Blasgen, J. Gray, M. Mitoma, T. Price. ACM Operating Systems Review, 13:2, April 1979. *Perhaps the first reference to convoys, which occurs in databases as well as the OS.*

[C54] “Priority Assignment in Waiting Line Problems” by A. Cobham. Journal of Operations Research, 2:70, pages 70–76, 1954. *The pioneering paper on using an SJF approach in scheduling the repair of machines.*

[K64] “Analysis of a Time-Shared Processor” by Leonard Kleinrock. Naval Research Logistics Quarterly, 11:1, pages 59–73, March 1964. *May be the first reference to the round-robin scheduling algorithm; certainly one of the first analyses of said approach to scheduling a time-shared system.*

[CK68] “Computer Scheduling Methods and their Countermeasures” by Edward G. Coffman and Leonard Kleinrock. AFIPS ’68 (Spring), April 1968. *An excellent early introduction to and analysis of a number of basic scheduling disciplines.*

[J91] “The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling” by R. Jain. Interscience, New York, April 1991. *The standard text on computer systems measurement. A great reference for your library, for sure.*

[O45] “Animal Farm” by George Orwell. Secker and Warburg (London), 1945. *A great but depressing allegorical book about power and its corruptions. Some say it is a critique of Stalin and the pre-WWII Stalin era in the U.S.S.R; we say it’s a critique of pigs.*

[PV56] “Machine Repair as a Priority Waiting-Line Problem” by Thomas E. Phipps Jr., W. R. Van Voorhis. Operations Research, 4:1, pages 76–86, February 1956. *Follow-on work that generalizes the SJF approach to machine repair from Cobham’s original work; also postulates the utility of an STCF approach in such an environment. Specifically, “There are certain types of repair work, … involving much dismantling and covering the floor with nuts and bolts, which certainly should not be interrupted once undertaken; in other cases it would be inadvisable to continue work on a long job if one or more short ones became available (p.81).”*

[MB91] “The effect of context switches on cache performance” by Jeffrey C. Mogul, Anita Borg. ASPLOS, 1991. *A nice study on how cache performance can be affected by context switching; less of an issue in today’s systems where processors issue billions of instructions per second but context-switches still happen in the millisecond time range.*

[W15] “You can’t have your cake and eat it” by Authors: Unknown.. Wikipedia (as of December 2015). http://en.wikipedia.org/wiki/You_can’t_have_your_cake_and_eat_it. *The best part of this page is reading all the similar idioms from other languages. In Tamil, you can’t “have both the moustache and drink the soup.”*

### 作业（模拟）

程序 `scheduler.py` 可以让你看到，不同的调度程序在响应时间、周转时间和总等待时间等调度指标（scheduling metric）下表现如何。详情请见 README。

### 问题

1. 用 SJF 和 FIFO 调度程序运行三个长度为 200 的作业，计算响应时间和周转时间。
2. 现在做同样的事，但作业长度不同：100、200 和 300。
3. 再做同样的事，并且还要用时间片为 1 的 RR 调度程序算一遍。
4. 对于什么样的工作负载，SJF 的周转时间和 FIFO 相同?
5. 对于什么样的工作负载和量子长度（quantum length），SJF 的响应时间和 RR 相同?
6. 随着作业长度增加，SJF 的响应时间会怎样变化？你能用模拟器展示这个趋势吗?
7. 随着量子长度（quantum length）增加，RR 的响应时间会怎样变化? 给定 $N$ 个作业，你能写出一个求最坏情况响应时间的公式吗?

 © 2008–23, ARPACI-DUSSEAU

# 8

# 调度：多级反馈队列

本章要设计的，是调度领域最著名的方法之一：**多级反馈队列**（Multi-level Feedback Queue，MLFQ）。多级反馈队列（MLFQ）调度程序（scheduler）最早由 Corbato 等人于 1962 年 [C+62] 在一个叫兼容分时系统（Compatible Time-Sharing System，CTSS）的系统中提出；这项工作，连同后来在 Multics 上的工作，让 ACM 把它的最高荣誉**图灵奖**（Turing Award）授予了 Corbato。此后多年，这种调度程序不断改进，演变成今天一些现代系统里的实现。

MLFQ 要解决的根本问题有两方面。首先，它希望优化周转时间（turnaround time）。上一章讲过，这要靠先运行较短的作业来实现；可惜操作系统一般并不知道一个作业要运行多久，而 SJF（或 STCF）这类算法恰恰需要这种信息。其次，MLFQ 希望让系统对交互式用户（即坐在那里盯着屏幕、等进程（process）完成的用户）显得反应灵敏，从而把响应时间（response time）降到最低；遗憾的是，像时间片轮转（Round-Robin）这样的算法虽然降低了响应时间，周转时间却很糟糕。于是问题来了：既然我们通常对进程一无所知，怎样才能构建一个达成这些目标的调度程序？调度程序如何在系统运行时了解它所运行的作业有什么特点，从而做出更好的调度决策？

关键问题：
没有完备的信息，如何调度？
在没有作业长度这种先验知识的情况下，我们如何设计一个调度程序，既能让交互式作业的响应时间最短，又能让周转时间最短？

## 8.1 MLFQ：基本规则

为了构建这样一个调度程序，本章将介绍多级反馈队列（Multi-level Feedback Queue）背后的基本算法。现有的 MLFQ 实现在细节上各不相同 [E95]，但大多数方法都差不多。

在我们的讨论中，MLFQ 有许多不同的**队列**（queue），每个队列有不同的**优先级**（priority level）。在任何时刻，一个就绪（ready）的作业都只在一个队列中。MLFQ 用优先级来决定某一时刻该运行哪个作业：优先级更高的作业（即位于更高队列中的作业）会被选中运行。

提示：从历史中学习
多级反馈队列（Multi-level Feedback Queue）是一个很好的例子，说明系统可以从过去学习，以此预测未来。这类方法在操作系统中很常见（在计算机科学的许多其他地方也是，包括硬件的分支预测器和缓存（cache）算法）。当作业的行为有阶段性、因而可以预测时，这类方法就能奏效；当然，使用这类技术必须小心，因为它们很容易出错，让系统做出比毫无信息时还糟糕的决策。

当然，同一个队列中可能有不止一个作业，因而它们的优先级相同。这种情况下，我们就在这些作业之间使用时间片轮转（Round-Robin）调度。

于是，我们得到了 MLFQ 的前两条基本规则：

- **规则 1：**如果 A 的优先级 > B 的优先级，运行 A（不运行 B）。
- **规则 2：**如果 A 的优先级 = B 的优先级，A 和 B 按 RR 方式运行。

因此，MLFQ 调度的关键在于调度程序如何设置优先级。MLFQ 并不给每个作业一个固定的优先级，而是根据观察到的行为来改变作业的优先级。例如，如果一个作业在等待键盘输入时反复让出（yield）CPU，MLFQ 就会让它保持高优先级，因为交互式进程大概就是这样的。反之，如果一个作业长时间密集地使用 CPU，MLFQ 就会降低它的优先级。这样，MLFQ 会在进程运行时设法了解它们，从而用作业的历史来预测它未来的行为。

如果把某一时刻各个队列的样子画出来，可能会看到类似下面的情形（第 70 页的图 8.1）。图中，两个作业（A 和 B）处于最高优先级，作业 C 处于中间，作业 D 处于最低优先级。按照我们目前对 MLFQ 工作方式的了解，调度程序只会在 A 和 B 之间轮流分配时间片（time slice），因为它们是系统中优先级最高的作业；可怜的作业 C 和 D 永远都没机会运行，真是岂有此理！

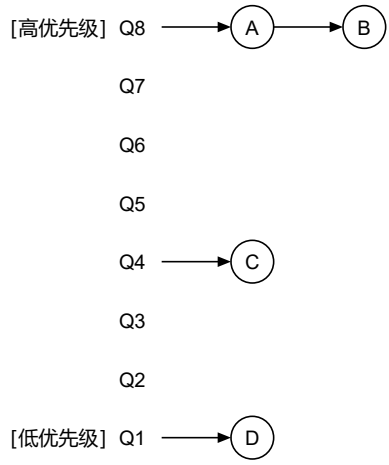



图 8.1：**MLFQ 示例**

当然，只给出某些队列的一张静态快照（snapshot），并不能真正让你明白 MLFQ 是如何工作的。我们需要了解的是作业的优先级如何随时间变化。而这恰恰就是我们接下来要做的（只有头一回读本书的人才会觉得意外）。

## 8.2 尝试 1：如何改变优先级

现在我们必须决定，在一个作业的整个生命周期中，MLFQ 如何改变它的优先级，从而改变它所在的队列。这时要记住我们面对的工作负载（workload）：其中既有交互式作业，它们运行时间短，而且可能频繁让出 CPU；也有一些运行时间较长的“CPU 密集型”作业，它们需要大量 CPU 时间，但响应时间并不重要。

为此要引入一个新概念：作业的**配额**（allotment）。配额（allotment）是作业在某个优先级上能用的时间，用完之后调度程序就会降低它的优先级。为简单起见，我们先假设配额（allotment）就等于一个时间片。

下面是我们第一次尝试的优先级调整算法：

- **规则 3：**作业进入系统时，被放在最高优先级，即最上面的队列。
- **规则 4a：**如果作业在运行时用完了它的配额（allotment），它的优先级就会降低（即下移一个队列）。
- **规则 4b：**如果作业在配额（allotment）用完之前就让出了 CPU（例如执行 I/O 操作），它就留在同一优先级，即它的配额被重置。

### 例 1：单个长时间运行的作业

我们来看几个例子。先看系统中只有一个长时间运行的作业时会发生什么。时间片为 10 毫秒，配额（allotment）设为等于时间片。图 8.2 显示了在有三个队列的调度程序中，这个作业随时间如何变化。

从例子中可以看到，作业以最高优先级（Q2）进入。运行一个 10 毫秒的时间片后，调度程序把它的优先级降低一级，于是作业到了 Q1。在 Q1 运行一个时间片后，作业最终被降到系统中的最低优先级（Q0），并一直留在那里。很简单，对吧？

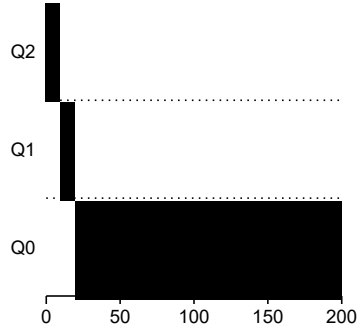



图 8.2：**长时间运行的作业随时间的变化**

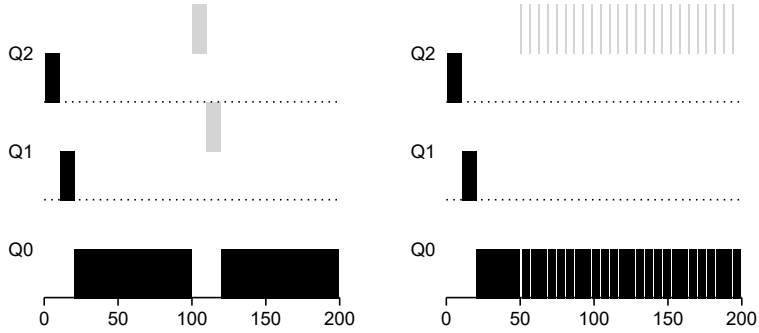



图 8.3：**来了一个交互式作业：两个例子**

### 例 2：来了一个短作业

现在来看一个更复杂的例子，希望借此看到 MLFQ 如何近似 SJF。这个例子中有两个作业：A 是一个长时间运行的 CPU 密集型作业，B 是一个运行时间很短的交互式作业。假设 A 已经运行了一段时间，然后 B 到达。会发生什么？MLFQ 会对 B 近似 SJF 吗？

第 72 页的图 8.3（左）画出了这个场景的结果。作业 A（黑色）在优先级最低的队列中运行（所有长时间运行的 CPU 密集型作业都会这样）。B（灰色）在 $T = 100$ 时到达，因此被放进最高的队列；它的运行时间很短（只有 20 毫秒），还没降到最底层队列，用两个时间片就运行完了。随后 A 继续以低优先级运行。

从这个例子中，你应该能理解这个算法的一个主要目标：由于算法并不知道一个作业是短是长，它会先假设这个作业很短，给它高优先级。如果确实是短作业，它很快就能运行完；如果不是，它就会慢慢沿着队列下移，很快表明自己是一个长时间运行、更像批处理（batch processing）的进程。MLFQ 就是这样近似 SJF 的。

### 例 3：I/O 怎么办?

现在来看一个涉及 I/O 的例子。正如上面的规则 4b 所说，如果一个进程在用完配额（allotment）之前就让出了处理器，我们就让它保持在同一优先级。这条规则的用意很简单：例如，如果一个交互式作业在做大量 I/O（比如等待用户的键盘或鼠标输入），它就会在配额（allotment）用完之前让出 CPU；这种情况下，我们不想惩罚这个作业，因此只是让它保持在同一层。

图 8.3（右）展示了这条规则如何起作用：交互式作业 B（灰色）每次只用 1 毫秒 CPU 就去执行 I/O，它和长时间运行的批处理作业 A（黑色）竞争 CPU。由于 B 不断地释放 CPU，MLFQ 让 B 保持在最高优先级；如果 B 是交互式作业，MLFQ 就进一步实现了让交互式作业快速运行的目标。

 © 2008–23, ARPACI-DUSSEAU

提示：调度必须能抵御攻击
你可能觉得调度策略（scheduling policy）和**安全**（security）无关，无论是操作系统内部的调度（如这里讨论的），还是更广泛场景中的调度（例如分布式存储系统处理 I/O 请求时 [Y+18]）；但在越来越多的情况下，调度恰恰是一个安全问题。想想现代数据中心，世界各地的用户在那里共享 CPU、内存、网络和存储系统；如果在策略的设计和执行上不加小心，单个用户就可能损害其他用户，为自己谋利。因此，调度策略是系统安全的重要组成部分，必须精心设计。

### 当前 MLFQ 的问题

这样，我们就有了一个基本的 MLFQ。它似乎做得相当不错：在长时间运行的作业之间公平地分配 CPU，又让短作业或 I/O 密集型的交互式作业快速运行。遗憾的是，到目前为止我们设计的方法有严重缺陷。你能想到有哪些吗？
（这里请停下来，尽可能狡猾地想一想）

首先，存在**饥饿**（starvation）问题：如果系统中有“太多”交互式作业，它们加起来就会占用全部 CPU 时间，于是长时间运行的作业永远得不到 CPU 时间（它们**饿死**（starve）了）。即使在这种情况下，我们也希望这些作业能有所进展。

其次，聪明的用户可以改写他们的程序，**钻调度程序的空子**（game the scheduler）。钻调度程序的空子（game the scheduler），一般是指要点花招，骗调度程序把超出你应得份额的资源分给你。我们描述的算法容易受到下面这种攻击：在配额（allotment）用完之前，发起一次 I/O 操作（例如访问某个文件），从而让出 CPU；这样你就能留在同一个队列，获得更高比例的 CPU 时间。如果做得好，比如先用掉配额（allotment）的 99% 再让出 CPU，一个作业几乎可以独占 CPU。

最后，一个程序的行为可能随时间改变；原本 CPU 密集的程序可能转入一个交互阶段。按照我们目前的方法，这样的作业就运气不佳了，得不到系统中其他交互式作业那样的待遇。

## 8.3 尝试 2：优先级提升

我们来试着改变规则，看看能否避免饥饿问题。怎样才能保证 CPU 密集型作业有所进展（哪怕进展不大）呢？

这里的简单想法是周期性地**提升**（boost）系统中所有作业的优先级。实现的办法有很多，我们就做一件简单的事：把它们全都扔进最上面的队列；于是有了一条新规则：

- **规则 5：**经过一段时间 $S$ 后，把系统中所有作业都移到最上面的队列。

这条新规则一下子解决了两个问题。首先，进程一定不会饿死：作业待在最上面的队列中，就会以轮转的方式和其他高优先级作业共享 CPU，因而最终会得到服务。其次，如果一个 CPU 密集型作业变成了交互式的，那么它一旦得到优先级提升（priority boost），调度程序就会正确地对待它。

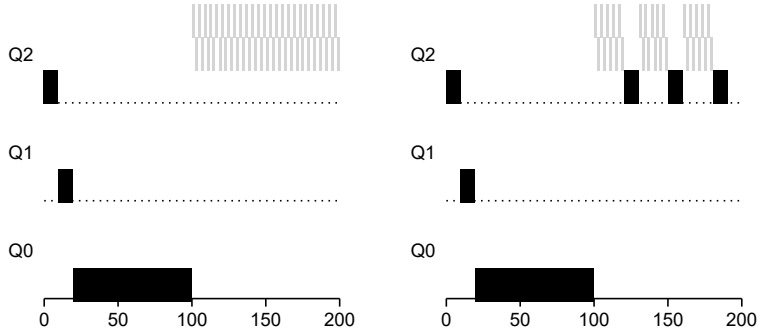



图 8.4：**没有（左）和有（右）优先级提升**

我们来看一个例子。这里只看一个长时间运行的作业在和两个运行时间很短的交互式作业竞争 CPU 时的表现。图 8.4（第 74 页）给出了两张图。左图没有优先级提升（priority boost），因此两个短作业到达后，长时间运行的作业就饿死了；右图每 100 毫秒提升一次优先级（这个值大概太小了，只是为了举例），这样至少保证了长时间运行的作业会有所进展：它每 100 毫秒被提升到最高优先级，从而能周期性地运行。

当然，引入时间段 $S$ 就带来一个显而易见的问题：$S$ 应该设成多少？备受尊敬的系统研究者 John Ousterhout [O11] 过去常把系统中这类值称为 **voo-doo constants**，因为要把它们设对，似乎需要某种黑魔法。遗憾的是，$S$ 就有这种味道。如果设得太高，长时间运行的作业可能会饿死；设得太低，交互式作业又可能得不到合理的 CPU 份额。因此，合适的值往往由系统管理员来找；如今则越来越多地交给基于机器学习的自动方法 [A+17]。

## 8.4 尝试 3：更好的记账

现在我们还有一个问题要解决：如何防止有人钻我们调度程序的空子？你可能已经猜到，真正的罪魁祸首是规则 4a 和 4b，它们允许作业在配额（allotment）到期之前让出 CPU，从而保住自己的优先级。那么我们该怎么办呢？

这里的解决办法是对 MLFQ 每一层的 CPU 时间做更好的**记账**（accounting）。调度程序不应在进程执行 I/O 时忘掉它在某一层已经用掉了多少配额（allotment），而应记录下来；进程一旦用完配额（allotment），就被降到下一个优先级的队列。它是一次长时间用完配额

提示：避免 voo-doo constants（Ousterhout 定律）

只要有可能，避免 voo-doo constants 都是个好主意。遗憾的是，正如上面的例子所示，这往往很难做到。可以试着让系统自己学习出一个好的值，但这同样不简单。常见的结果是弄出一个写满默认参数值的配置文件，系统运转不太正常时，经验丰富的管理员可以去调整这些值。可以想象，这些值通常没人去改，于是我们只能指望默认值在实际使用中效果不错。这条提示由我们当年的操作系统教授 John Ousterhout 提供，因此我们把它称为 **Ousterhout 定律**（Ousterhout's Law）。

 © 2008–23, ARPACI-DUSSEAU

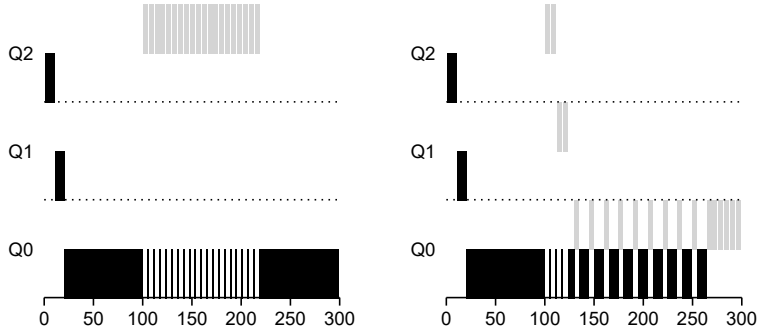



图 8.5：**没有（左）和有（右）防作弊措施**

（allotment），还是分成许多次小段用完，都不应有区别。于是，我们把规则 4a 和 4b 改写成下面这一条规则：

- **规则 4：**一旦作业用完了它在某一层的时间配额（allotment），无论它中间让出了多少次 CPU，它的优先级都会降低（即下移一个队列）。

我们来看一个例子。图 8.5 显示了一个工作负载试图钻调度程序空子时的情况：左边用的是旧的规则 4a 和 4b，右边用的是新的防作弊规则 4。如果没有任何防作弊保护，进程就可以在配额（allotment）用完之前发起一次 I/O，从而留在同一优先级，独占 CPU 时间。右图有了更好的记账（accounting），无论进程的 I/O 行为如何，它都会慢慢地沿着队列下移，因而无法获得不公平的 CPU 份额。

## 8.5 MLFQ 的调优及其他问题

MLFQ 调度还会引出其他一些问题。一个大问题是这样的调度程序该如何**参数化**（parameterize）。例如，应该有多少个队列？每个队列的时间片应该多大？配额（allotment）呢？为了避免饥饿并顾及行为的变化，应该多久提升一次优先级？这些问题都没有简单的答案，只有对工作负载积累一些经验，再对调度程序做相应的调优，才能达到令人满意的平衡。

例如，大多数 MLFQ 变体允许不同队列使用不同长度的时间片。高优先级的队列通常分到较短的时间片；毕竟这些队列里是交互式作业，在它们之间快速切换（例如 10 毫秒或更短）是合理的。相比之下，低优先级的队列里是长时间运行的 CPU 密集型作业，因此较长的时间片（例如几百毫秒）效果很好。图 8.6 展示了一个例子：两个作业在最高的队列中运行 20 毫秒（时间片为 10 毫秒），在中间的队列中运行 40 毫秒（时间片为 20 毫秒），在最低的队列中时间片为 40 毫秒。

Solaris 的 MLFQ 实现，即分时（Time-Sharing，TS）调度类，配置起来特别方便。它提供了一组表，精确规定进程的优先级在整个生命周期中如何变化、每个时间片有多长、多久提升一次作业的优先级 [AD00]；管理员可以随意摆弄这些表，让调度程序表现出不同的行为。这些表的默认配置是 60 个队列，时间片长度从 20 毫秒（最高优先级）缓慢增加到几百毫秒（最低优先级），优先级大约每 1 秒提升一次。

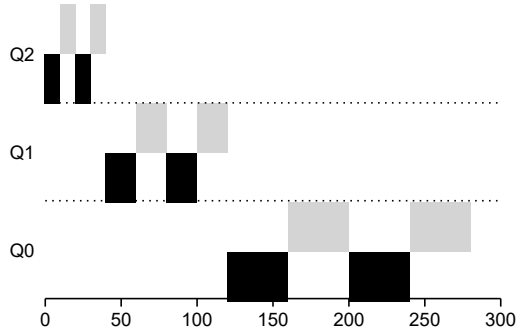



图 8.6：**优先级越低，时间片越长**

其他 MLFQ 调度程序不使用表，也不使用本章描述的这些具体规则，而是用数学公式来调整优先级。例如，FreeBSD 4.3 版的调度程序根据进程用掉了多少 CPU，用一个公式计算作业当前的优先级 [LM+89]；此外，CPU 使用量会随时间衰减，从而用一种不同于本章的办法实现所需的优先级提升（priority boost）。Epema 的论文对这类 **decay-usage** 算法及其性质做了很好的综述 [E95]。

最后，许多调度程序还有其他一些特性，你也可能碰到。例如，有些调度程序把最高的几个优先级留给操作系统的工作；因此，普通的用户作业永远无法获得系统中最高的那几级优先级。有些系统还允许用户提供一些**建议**（advice）来帮助设置优先级；例如，使用命令行工具 `nice`，就可以（在一定程度上）提高或降低作业的优先级，从而增加或减少它在任一时刻运行的机会。详情请见手册页（manual page）。

## 8.6 MLFQ：小结

我们介绍了一种叫作多级反馈队列（MLFQ）的调度方法。希望你现在能明白它为什么叫这个名字：它有多级队列，并利用反馈来确定某个作业的优先级。历史是它的向导：留意作业随时间的行为，并据此对待它们。

本章各处陆续改进的 MLFQ 规则，在这里汇总一下，方便查看：

- **规则 1：** 如果 A 的优先级 > B 的优先级，运行 A（不运行 B）。
- **规则 2：** 如果 A 的优先级 = B 的优先级，A 和 B 用所在队列的时间片以轮转方式运行。
- **规则 3：** 作业进入系统时，被放在最高优先级，即最上面的队列。

提示：尽量利用建议

操作系统很少清楚什么对系统中的每个进程最好，因此提供一些接口，让用户或管理员给操作系统一些**提示**（hint），往往很有用。我们常把这类提示称为**建议**（advice），因为操作系统不必照办，只是可以把它纳入考虑，以便做出更好的决策。这类提示在操作系统的许多部分都很有用，包括调度程序（例如 `nice`）、内存管理器（例如 `madvise`），以及文件系统（file system，例如 informed prefetching and caching [P+95]）。

- **规则 4：**一旦作业用完了它在某一层的时间配额（allotment），无论它中间让出了多少次 CPU，它的优先级都会降低（即下移一个队列）。
- **规则 5：**经过一段时间 $S$ 后，把系统中所有作业都移到最上面的队列。

MLFQ 有意思的地方在于：它不要求预先知道作业的性质，而是观察作业的执行情况，据此确定它的优先级。这样，它做到了两全其美：对运行时间很短的交互式作业，它能提供出色的整体性能（和 SJF/STCF 相近）；对长时间运行的 CPU 密集型工作负载，它又是公平的，能让它们有所进展。正因如此，许多系统，包括 BSD UNIX 的衍生版本 [LM+89, B86]、Solaris [M06]，以及 Windows NT 和之后的 Windows 操作系统 [CS97]，都用某种形式的 MLFQ 作为基础调度程序。

## 参考文献

[A+17] "Automatic Database Management System Tuning Through Large-scale Machine Learning" by Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, Bohan Zhang. SIGMOD '17. *This isn't about the application of machine learning to CPU scheduling in the OS, but rather a cool early example of automatically tuning parameters of a database via ML techniques. Worth a read, if you like ML… which, alas, everyone seems to these days.*

[AD00] "Multilevel Feedback Queue Scheduling in Solaris" by Andrea Arpaci-Dusseau. Available: http://www.ostep.org/Citations/notes-solaris.pdf. *A great short set of notes by one of the authors on the details of the Solaris scheduler. OK, we are probably biased in this description, but the notes are pretty darn good.*

[B86] "The Design of the UNIX Operating System" by M.J. Bach. Prentice-Hall, 1986. *One of the classic old books on how a real UNIX operating system is built; a definite must-read for kernel hackers.*

[C+62] "An Experimental Time-Sharing System" by F. J. Corbato, M. M. Daggett, R. C. Daley. IFIPS 1962. *A bit hard to read, but the source of many of the first ideas in multi-level feedback scheduling. Much of this later went into Multics, which one could argue was the most influential operating system of all time.*

[CS97] "Inside Windows NT" by Helen Custer and David A. Solomon. Microsoft Press, 1997. *The NT book, if you want to learn about something other than UNIX. Of course, why would you? OK, we're kidding; you might actually work for Microsoft some day you know.*

[E95] "An Analysis of Decay-Usage Scheduling in Multiprocessors" by D.H.J. Epema. SIGMETRICS '95. *A nice paper on the state of the art of scheduling back in the mid 1990s, including a good overview of the basic approach behind decay-usage schedulers.*

[LM+89] "The Design and Implementation of the 4.3BSD UNIX Operating System" by S.J. Leffler, M.K. McKusick, M.J. Karels, J.S. Quarterman. Addison-Wesley, 1989. *Another OS classic, written by four of the main people behind BSD. The later versions of this book, while more up to date, don't quite match the beauty of this one.*

[M06] "Solaris Internals: Solaris 10 and OpenSolaris Kernel Architecture" by Richard McDougall. Prentice-Hall, 2006. *A good book about Solaris and how it works.*

[O11] "John Ousterhout's Home Page" by John Ousterhout. www.stanford.edu/~ouster/. *The home page of the famous Professor Ousterhout. The two co-authors of this book had the pleasure of taking graduate operating systems from Ousterhout while in graduate school; indeed, this is where the two co-authors got to know each other, eventually leading to marriage, kids, and even this book. Thus, you really can blame Ousterhout for this entire mess you're in.*

[P+95] "Informed Prefetching and Caching" by R.H. Patterson, G.A. Gibson, E. Ginting, D. Stodolsky, J. Zelenka. SOSP '95, Copper Mountain, Colorado, October 1995. *A fun paper about some very cool ideas in file systems, including how applications can give the OS advice about what files it is accessing and how it plans to access them.*

[Y+18] "Principled Schedulability Analysis for Distributed Storage Systems using Thread Architecture Models" by Suli Yang, Jing Liu, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. OSDI '18, San Diego, California. *A recent work of our group that demonstrates the difficulty of scheduling I/O requests within modern distributed storage systems such as Hive/HDFS, Cassandra, MongoDB, and Riak. Without care, a single user might be able to monopolize system resources.*

### 作业（模拟）

程序 `mlfq.py` 可以让你看到本章介绍的 MLFQ 调度程序是如何工作的。详情请见 README。

### 问题

1. 随机生成几个只有两个作业、两个队列的问题来运行；计算每个问题的 MLFQ 执行轨迹。限制每个作业的长度并关闭 I/O，会让你轻松一些。
2. 要重现本章中的每个例子，应该怎样运行调度程序?
3. 应该怎样配置调度程序的参数，才能让它的行为和时间片轮转（Round-Robin）调度程序完全一样?
4. 构造一个包含两个作业的工作负载和一组调度程序参数，让其中一个作业利用旧的规则 4a 和 4b（用 `-S` 选项打开）钻调度程序的空子（game the scheduler），在某段时间内获得 99% 的 CPU。
5. 假设一个系统最高队列的量子长度（quantum length）为 10 毫秒，为了保证一个长时间运行（且可能饿死）的作业至少获得 5% 的 CPU，你需要多久把作业提升回最高优先级一次（用 `-B` 选项)?
6. 调度中有一个问题：刚完成 I/O 的作业应该加到队列的哪一端；`-I` 选项可以改变调度模拟器在这方面的行为。试一些工作负载，看看能否观察到这个选项的效果。

 © 2008–23, ARPACI-DUSSEAU

# 9

# 调度：比例份额

本章介绍另一种调度程序（scheduler），叫作**比例份额**（proportional-share）调度程序，有时也叫**公平份额**（fair-share）调度程序。比例份额（proportional-share）调度基于一个简单的概念：调度程序不去优化周转时间（turnaround time）或响应时间（response time），而是设法保证每个作业获得一定比例的 CPU 时间。

比例份额（proportional-share）调度有一个很好的早期例子，叫作**彩票调度**（lottery scheduling），出自 Waldspurger 和 Weihl 的研究 [WW94]；不过这个想法肯定出现得更早 [KL88]。基本思想很简单：每隔一段时间，就抽一次奖，决定下一个该运行哪个进程（process）；应该更常运行的进程，就给它更多中奖的机会。很简单，对吧？下面来看细节！不过先看我们的关键问题：

关键问题：如何按比例分享 CPU
怎样设计调度程序，才能按比例分享 CPU？实现这一点的关键机制是什么？这些机制效果如何？

## 9.1 基本概念：彩票代表你的份额

彩票调度（lottery scheduling）的背后，是一个非常基本的概念：**彩票**（tickets）。彩票（ticket）用来表示一个进程（或者用户，或者别的什么）应得的某种资源的份额。一个进程持有的彩票（ticket）占总数的百分比，就代表它在这种系统资源上的份额。

来看一个例子。假设有两个进程 A 和 B，A 有 75 张彩票（ticket），B 只有 25 张。那么我们希望 A 得到 75% 的 CPU，B 得到剩下的 25%。

彩票调度（lottery scheduling）以概率的方式（而不是确定的方式）实现这一目标：每隔一段时间，比如每个时间片（time slice），抽一次奖。抽奖很简单：调度程序必须知道彩票（ticket）的总数，在我们的例子中是 100 张。然后，调度程序选出一张中奖彩票（ticket），也就是 0 到 99 之间的一个数[1]。假设 A 持有 0 到 74 号彩票（ticket），B 持有 75 到 99 号，那么中奖彩票就直接决定了是 A 运行还是 B 运行。随后，调度程序载入中奖进程的状态并运行它。

[1]计算机科学家总是从 0 开始计数。这在非计算机人士看来太奇怪了，以至于有些名人觉得有必要专门写文章，解释我们为什么这么做 [D82]。

提示：利用随机性

彩票调度（lottery scheduling）最美妙的地方之一，就是它对**随机性**（randomness）的运用。需要做决定时，采用这种随机化的方法往往既稳健又简单。

和更传统的决策方法相比，随机方法至少有三个优点。第一，随机方法常常能避开一些奇怪的边界情况，而更传统的算法可能难以应付这些情况。例如 LRU 置换策略（replacement policy），后面讲虚拟内存（virtual memory）的一章会更详细地研究它。LRU 通常是一种不错的置换算法，但在某些循环顺序工作负载（workload）下，它的性能会落到最坏情况。随机方法就没有这样的最坏情况。

第二，随机方法很轻量，只需记录很少的状态来跟踪各个候选项。在传统的公平份额（fair-share）调度算法中，要记录每个进程已经得到了多少 CPU，就得对每个进程单独记账（accounting），而且每运行完一个进程都要更新。用随机的办法，每个进程只需要最少量的状态，比如它持有多少张彩票（ticket）。

最后，随机方法可以非常快。只要生成随机数够快，做决定也就够快，因此在许多需要速度的地方都可以使用随机方法。当然，对速度的要求越高，所用的随机数就越趋向于伪随机。

下面是一个彩票调度程序（lottery scheduler）抽出的一串中奖彩票（ticket）：

```
63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43  0 49 12
```

由此得到的调度结果如下：

```
A     A  A     A  A  A  A  A  A     A     A  A  A  A  A  A
   B        B                    B     B
```

从这个例子可以看出，彩票调度（lottery scheduling）利用了随机性，因此只在概率意义上满足预期的比例，并不保证一定满足。在上面的例子中，B 在 20 个时间片里只运行了 4 个（20%），而不是期望的 25%。不过，这两个作业竞争的时间越长，它们就越有可能达到期望的比例。

## 9.2 彩票机制

彩票调度（lottery scheduling）还提供了几种操纵彩票（ticket）的机制，用法各不相同，有时很管用。其中一种是 **ticket currency** 的概念。有了 ticket currency，持有一组彩

提示：用彩票表示份额

在彩票调度（lottery scheduling）以及步长调度（stride scheduling）的设计中，最强大（也最基本）的机制之一就是**彩票**（ticket）。在这些例子里，彩票（ticket）用来表示进程在 CPU 上的份额，但它的用途要广泛得多。例如，在较新的关于 hypervisor 虚拟内存管理的研究中，Waldspurger 展示了如何用彩票（ticket）来表示客户操作系统在内存上的份额 [W02]。所以，如果你需要一种机制来表示所有权的比例，这个概念也许正是……（卖个关子）……彩票（ticket）。

 © 2008–26, ARPACI-DUSSEAU

```
// counter: 用来记录是否已经找到中奖者
int counter = 0;

// winner: 调用某个随机数生成器，
//         得到一个 >= 0 且 <= (totaltickets - 1) 的值
int winner = getrandom(0, totaltickets);

// current: 用它来遍历作业列表
node_t *current = head;
while (current) {
    counter = counter + current->tickets;
    if (counter > winner)
        break; // 找到中奖者了
    current = current->next;
}
// 'current' 就是中奖者：调度它……
```

图 9.1：**彩票调度的决策代码**

票（ticket）的用户可以用自己喜欢的任意货币单位，把彩票分配给自己的各个作业；系统随后会自动把这种货币换算成正确的全局值。

例如，假设用户 A 和 B 各分到 100 张彩票（ticket）。用户 A 运行两个作业 A1 和 A2，并以 A 的货币各给它们 500 张彩票（ticket），总共 1000 张。用户 B 只运行一个作业，给了它 10 张彩票（ticket），总共 10 张。系统把 A1 和 A2 各自以 A 的货币计的 500 张，换算成全局货币的 50 张；同样，B1 的 10 张彩票（ticket）被换算成 100 张。然后，在全局货币的全部 200 张彩票（ticket）上抽奖，决定运行哪个作业。

```
User A -> 500 (A's currency) to A1 -> 50 (global currency)
       -> 500 (A's currency) to A2 -> 50 (global currency)
User B -> 10 (B's currency) to B1 -> 100 (global currency)
```

另一种有用的机制是 **ticket transfer**。借助 ticket transfer，一个进程可以把自己的彩票（ticket）暂时交给另一个进程。这种能力在客户端/服务器（client/server）的场景中尤其有用：客户端进程向服务器发送消息，请服务器代它做一些工作。为了加快工作，客户端可以把彩票（ticket）交给服务器，从而在服务器处理客户端请求时，尽量提高服务器的性能。处理完以后，服务器再把彩票（ticket）交还给客户端，一切恢复原样。

最后，**ticket inflation** 有时也是一种有用的技术。借助 ticket inflation，进程可以临时提高或降低自己持有的彩票（ticket）数。当然，在进程之间互不信任的竞争场景中，这样做没什么意义：一个贪心（greedy）的进程可以给自己发大量彩票（ticket），从而霸占整台机器。ticket inflation 适用于进程之间互相信任的环境；在这种情况下，如果某个进程知道自己需要更多 CPU 时间，就可以提高自己的彩票（ticket）数，借此向系统表明这种需要，而且完全不必和其他进程通信。

## 9.3 实现

彩票调度（lottery scheduling）最令人惊叹的一点，大概就是它实现起来非常简单。你只需要一个好的随机数生成器来选出中奖彩票（ticket），一个数据结构（data structure，比如一个列表）来记录系统中的进程，以及彩票的总数。

假设我们把进程放在一个列表里。下面是一个例子，其中有三个进程 A、B 和 C，各自持有一定数量的彩票（ticket）。




要做出调度决定，我们先得从彩票（ticket）总数（400）中选出一个随机数（也就是中奖号码）[2]。假设我们选中的是 300。接下来，只需遍历这个列表，用一个简单的计数器帮我们找出中奖者（图 9.1，第 82 页）。

这段代码遍历进程列表（process list），把每个进程的彩票（ticket）数累加到 `counter` 上，直到它的值超过 `winner`。一旦超过，当前的列表元素就是中奖者。以中奖彩票（ticket）是 300 为例，过程如下。首先，`counter` 加上 A 的彩票（ticket），变成 100；因为 100 小于 300，循环继续。接着，`counter` 加上 B 的彩票（ticket），变成 150，仍然小于 300，于是循环再次继续。最后，`counter` 变成 400（显然大于 300），于是跳出循环，此时 `current` 指向 C（中奖者）。

为了让这个过程尽可能高效，一般最好把列表按彩票（ticket）数从多到少排好序。排序不影响算法的正确性；不过总体来说，它能让循环走的步数最少，尤其是当少数几个进程持有大部分彩票（ticket）时。

## 9.4 一个例子

为了让彩票调度（lottery scheduling）的动态行为更容易理解，我们来简单研究一下两个互相竞争的作业的完成时间：它们的彩票（ticket）数相同（都是 100），运行时间也相同（都是 $R$，我们会改变它的值）。

在这个场景中，我们希望两个作业大致同时完成；但由于彩票调度（lottery scheduling）的随机性，有时一个作业会比另一个先完成。为了量化这种差异，我们定义一个简单的**公平性指标**（fairness metric）$F$，它就是第一个作业的完成时间除以第二个作业的完成时间。例如，如果 $R = 10$，第一个作业在时刻 10 完成（第二个作业在时刻 20 完成），那么 $F = \frac{10}{20} = 0.5$。两个作业几乎同时完成时，$F$ 会非常接近 1。我们的目标是：完全公平的调度程序应该做到 $F = 1$。

图 9.2（第 84 页）画出了两个作业的长度（$R$）从 1 变化到 1000 时，30 次试验的平均公平性（fairness），结果由本章末尾提供的模拟器生成。从图中可以看出，作业长度不太长

[2] 出人意料的是，正如 Björn Lindberg 指出的，要正确地做到这一点并不容易；详见 http://stackoverflow.com/questions/2509679/how-to-generate-a-random-number-from-within-a-range。

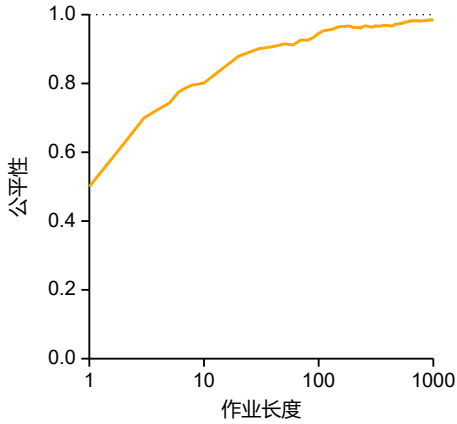



图 9.2：**彩票调度的公平性研究**

时，平均公平性可能相当低。只有当作业运行了相当多的时间片之后，彩票调度程序（lottery scheduler）才会接近期望的公平结果。

## 9.5 怎样分配彩票？

关于彩票调度（lottery scheduling），还有一个问题我们没有讨论：怎样给作业分配彩票（ticket）？这个问题很棘手，因为系统的行为显然在很大程度上取决于彩票（ticket）如何分配。一种办法是假定用户最清楚该怎么分；在这种情况下，每个用户分到一定数量的彩票（ticket），然后可以按自己的意愿把彩票分给自己运行的任何作业。然而，这个方案其实算不上方案：它根本没有告诉你该怎么做。因此，给定一组作业，“彩票（ticket）分配问题”仍然没有答案。

## 9.6 步长调度

你可能还会想：为什么要用随机性呢？正如前面看到的，随机性虽然让我们得到了一个简单（而且大致正确）的调度程序，但它偶尔无法给出完全准确的比例，在较短的时间尺度上尤其如此。为此，Waldspurger 发明了**步长调度**（stride scheduling），这是一种确定性的公平份额（fair-share）调度程序 [W95]。

步长调度（stride scheduling）也很简单。系统中的每个作业都有一个**步长**（stride），它和作业持有的彩票（ticket）数成反比。在前面的例子中，作业 A、B、C 分别持有 100、50 和 250 张彩票（ticket）；我们可以用某个大数分别除以每个进程分到的彩票（ticket）数，算出它们各自的步长（stride）。例如，用 10,000 分别除以这几个彩票（ticket）数，就得到 A、B、C 的步长（stride）：100、200 和 40。我们把这个值叫作每个进程的步长（stride）；每当一个进程运行一次，我们就把它的一个计数器（叫作它的 **pass** 值）加上它的步长（stride），以此记录它的总体进度。

然后，调度程序用步长（stride）和 pass 值来决定下一个该运行哪个进程。基本思路很简单：任何时候，都挑选目前 pass 值最小的进程来运行；运行完一个进程后，就把它的 pass 计数器加上它的步长（stride）。Waldspurger 给出了一个伪代码（pseudocode）实现 [W95]：

```
curr = remove_min(queue);    // 挑出 pass 值最小的客户
schedule(curr);              // 运行一个量子
curr->pass += curr->stride; // 用步长更新 pass 值
insert(queue, curr);         // 把 curr 放回队列
```

在我们的例子中，一开始有 A、B、C 三个进程，步长（stride）分别是 100、200 和 40，pass 值都从 0 开始。因此，最初任何一个进程都可以运行，因为它们的 pass 值同样最低。假设我们选了 A（随便选的；pass 值同样最低的进程都可以选）。A 运行；用完这个时间片后，我们把它的 pass 值更新为 100。接着运行 B，它的 pass 值随后变成 200。最后运行 C，它的 pass 值增加到 40。这时，算法会挑出 pass 值最低的进程，也就是 C，运行它，并把它的 pass 值更新为 80（你应该还记得，C 的步长（stride）是 40）。然后 C 会再运行一次（它的 pass 值仍然最低），pass 值升到 120。现在 A 会运行，pass 值更新为 200（和 B 的相等了）。接下来 C 又会运行两次，pass 值先后更新为 160 和 200。此时，所有的 pass 值又都相等了，这个过程会不断重复，永无止境。图 9.3（第 85 页）跟踪了调度程序随时间推移的行为。

从图中可以看出，C 运行了五次，A 运行了两次，B 只运行了一次，恰好和它们的彩票（ticket）数 250、100、50 成比例。彩票调度（lottery scheduling）是随着时间推移、按概率达到这些比例的；步长调度（stride scheduling）则在每个调度周期结束时精确地达到这些比例。

所以你可能会想：既然步长调度（stride scheduling）这么精确，为什么还要用彩票调度（lottery scheduling）呢？原来，彩票调度（lottery scheduling）有一个步长调度（stride scheduling）所没有的好性质：没有全局状态。想象一下，在上面的步长调度（stride scheduling）例子进行到一半时，有一个新作业加入进来；它的 pass 值该设成多少？设成 0 吗？如果这样，它就会霸占 CPU。而用彩票调度（lottery scheduling）时，每个进程都没有全局状态；我们只需加入新进程以及它持有的彩票（ticket），更新那唯一一个记录彩票

| Pass(A)<br>（步长 = 100） | Pass(B)<br>（步长 = 200） | Pass(C)<br>（步长 = 40） | 谁运行? |
|---|---|---|---|
| 0 | 0 | 0 | A |
| 100 | 0 | 0 | B |
| 100 | 200 | 0 | C |
| 100 | 200 | 40 | C |
| 100 | 200 | 80 | C |
| 100 | 200 | 120 | A |
| 200 | 200 | 120 | C |
| 200 | 200 | 160 | C |
| 200 | 200 | 200 | … |

图 9.3：**步长调度：跟踪过程**

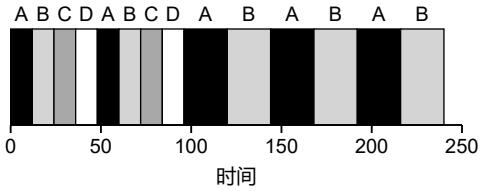



图 9.4：**CFS 的简单示例**

总数的全局变量，然后照常进行即可。这样一来，彩票调度（lottery scheduling）就能更容易、更合理地接纳新进程。

## 9.7 Linux 的完全公平调度程序（CFS）

尽管已经有了上面这些关于公平份额（fair-share）调度的早期工作，Linux 的一种（较早的）方法却另辟蹊径，实现了类似的目标。这个调度程序名叫**完全公平调度程序**（Completely Fair Scheduler，**CFS**）[J09]，它实现了公平份额（fair-share）调度，而且做得非常高效、可扩展。

为了达到高效的目标，CFS 力求把花在调度决策上的时间降到最少，靠的既是它本身的设计，也是它巧妙选用的、非常适合这项任务的数据结构。最近的研究表明，调度程序的效率出人意料地重要；具体来说，Kanev 等人在一项针对 Google 数据中心的研究中指出，即使经过激进的优化，调度仍然用掉了整个数据中心大约 5% 的 CPU 时间 [K+15]。因此，尽可能降低这项开销，是现代调度程序架构的一个关键目标。

### 基本操作

大多数调度程序都基于固定时间片的概念，而 CFS 的做法有些不同。它的目标很简单：把一个 CPU 公平、均匀地分给所有相互竞争的进程。它靠一种简单的计数技术做到这一点，这种技术叫作**虚拟运行时间**（virtual runtime，**`vruntime`**）。

每个进程运行时，都会累积 `vruntime`。在最基本的情况下，每个进程的 `vruntime` 以相同的速度增长，与物理（真实）时间成正比。做调度决策时，CFS 会挑选 `vruntime` 最低的进程作为下一个运行的进程。

这就引出了一个问题：调度程序怎么知道何时该停下当前运行的进程，转而运行下一个呢？这里的矛盾很明显：如果 CFS 切换得太频繁，公平性会提高（即使在极短的时间窗口内，CFS 也能保证每个进程得到自己那份 CPU），但代价是性能，因为上下文切换（context switch）太多；如果 CFS 切换得不那么频繁，性能就会提高（上下文切换减少），但代价是短期内的公平性。

CFS 通过各种控制参数来处理这种矛盾。第一个是 **`sched_latency`**。CFS 用这个值来决定一个进程运行多久之后再考虑切换（实际上就是动态地决定它的时间片）。`sched_latency` 的典型值是 48（毫秒）；CFS 用这个值除以 CPU 上正在运行的进程数（$n$），得到每个进程的时间片，从而确保在这段时间内，CFS 是完全公平的。

例如，如果有 $n = 4$ 个进程在运行，CFS 就用 `sched_latency` 除以 $n$，得到每个进程 12 ms 的时间片。然后 CFS 调度第一个作业并运行它，直到它用完 12 ms 的（虚拟）运行时间，再检查是否有 `vruntime` 更低的作业可以改为运行。在这个例子里确实有，于是 CFS 会切换到另外三个作业之一，依此类推。图 9.4（第 86 页）给出了一个例子：四个作业（A、B、C、D）按这种方式各运行了两个时间片；然后其中两个（C、D）完成，只剩下两个作业，它们随后以时间片轮转（Round-Robin）的方式各运行 24 ms。

但是，如果运行的进程“太多”了怎么办？那样不就会导致时间片太小，进而上下文切换太多吗？好问题！答案是肯定的。

为了解决这个问题，CFS 增加了另一个参数 **`min_granularity`**，它通常设成 6 ms 左右的值。CFS 绝不会把一个进程的时间片设得比这个值还小，以确保不会在调度开销上花费太多时间。

例如，如果有十个进程在运行，按最初的算法，要用 `sched_latency` 除以十来确定时间片，结果是 4.8 ms。但因为有 `min_granularity`，CFS 会把每个进程的时间片改设为 6 ms。虽然在 48 ms 的目标调度延迟（`sched_latency`）内，CFS 不会（完全）做到完美公平，但会很接近，同时还能保持很高的 CPU 效率。

注意，CFS 使用周期性的时钟中断（timer interrupt），这意味着它只能在固定的时间间隔上做决策。这个中断（interrupt）触发得很频繁（比如每 1 ms 一次），让 CFS 有机会醒来，判断当前作业是否已经运行到头。如果某个作业的时间片不是时钟中断间隔的整数倍，也没关系；CFS 会精确地记录 `vruntime`，这意味着从长远来看，它终究会逼近理想的 CPU 分享。

### 权重（nice 值）

CFS 还允许控制进程的优先级（priority），让用户或管理员可以给某些进程分配更多的 CPU 份额。它不是靠彩票（ticket）做到这一点的，而是借助 UNIX 一种经典的机制：进程的 **nice** 级别。一个进程的 nice 参数可以设为 –20 到 +19 之间的任意值，默认是 0。nice 值为正表示优先级较低，为负表示优先级较高；唉，人要是太 nice（友善），就得不到那么多（调度上的）关注了。

CFS 把每个进程的 nice 值映射到一个权重，如下所示：

```
static const int prio_to_weight[40] = {
  /* -20 */ 88761, 71755, 56483, 46273, 36291,
  /* -15 */ 29154, 23254, 18705, 14949, 11916,
  /* -10 */  9548,  7620,  6100,  4904,  3906,
  /*  -5 */  3121,  2501,  1991,  1586,  1277,
  /*   0 */  1024,   820,   655,   526,   423,
  /*   5 */   335,   272,   215,   172,   137,
  /*  10 */   110,    87,    70,    56,    45,
  /*  15 */    36,    29,    23,    18,    15,
};
```

有了这些权重，我们就能（像之前一样）算出每个进程实际的时间片，不过这次考虑了它们之间的优先级差异。假设有 $n$ 个进程，计算公式如下：

$$\text{time_slice}_k = \frac{\text{weight}_k}{\sum_{i=0}^{n-1} \text{weight}_i} \cdot \text{sched_latency} \tag{9.1}$$

我们举个例子，看看这个公式是怎么起作用的。假设有两个作业 A 和 B。A 是我们最宝贵的作业，所以给它分配 –5 的 nice 值，让它的优先级更高；B 嘛，因为我们讨厌它[3]，所以只有默认的优先级（nice 值等于 0）。这意味着（查上表）$\text{weight}_A$ 是 3121，而 $\text{weight}_B$ 是 1024。如果你再计算每个作业的时间片，就会发现 A 的时间片大约是 `sched_latency` 的 $\frac{3}{4}$（因此是 36 ms），B 的大约是 $\frac{1}{4}$（因此是 12 ms）。

除了推广时间片的计算方法，CFS 计算 `vruntime` 的方式也要相应调整。下面是新的公式：它取进程 $i$ 累计的实际运行时间（$\text{runtime}_i$），再按进程权重的反比缩放，也就是用默认权重 1024（$\text{weight}_0$）除以它的权重 $\text{weight}_i$。在前面这个例子中，A 的 `vruntime` 累积速度是 B 的三分之一。

$$\text{vruntime}_i = \text{vruntime}_i + \frac{\text{weight}_0}{\text{weight}_i} \cdot \text{runtime}_i \tag{9.2}$$

上面这张权重表的构造有一个巧妙之处：只要 nice 值的差保持不变，这张表就能保持 CPU 的分配比例不变。例如，如果进程 A 的 nice 值改成 5（而不是 –5），进程 B 的 nice 值改成 10（而不是 0），CFS 调度它们的方式会和之前完全一样。自己算一算，看看为什么。

### 使用红黑树

如上所述，CFS 的一个重点是效率。对调度程序来说，效率有很多方面，其中一个就是这么简单：当调度程序需要找到下一个要运行的作业时，应该尽可能快地找到。列表这样简单的数据结构无法扩展：现代系统里有时有上千个进程，因此每隔几毫秒就把一个长列表搜索一遍是一种浪费。

CFS 的解决办法是把进程放在一棵**红黑树**（red-black tree）中 [B72]。红黑树是众多平衡树中的一种。简单的二叉树遇到最坏的插入顺序时，性能可能退化得和列表一样；平衡树则多做一点额外的工作来保持较小的深度，从而保证操作的时间是对数级的（而不是线性的）。

CFS 并不把所有进程都放进这个结构，只有正在运行（或可运行）的进程才放在里面。如果一个进程进入睡眠（比如在等待一次 I/O 完成，或者等待一个网络包到达），它就会被移出这棵树，在别处另行记录。

来看一个例子，让这一点更清楚。假设有十个作业，它们的 `vruntime` 值分别是：1、5、9、10、14、18、17、21、22 和 24。如果把这些作业放在一个有序列表里，找出下一个要运行的作业很简单：直接取出第一个元素就行。但是，把那个作业（按顺序）放回列表时，我们就得扫描列表，找到正确的插入位置，这是一个 $O(n)$ 的操作。任何查找的效率也都很低，平均同样需要线性时间。

把同样的值放进红黑树，大多数操作都会更高效，如图 9.5（第 89 页）所示。进程在树中按 `vruntime` 排序，大多数操作（比如插入和删除）的时间都是对数级的，即 $O(\log n)$。$n$ 达到上千时，对数级的效率明显比线性的高。

[3] 是的，是的，我们这里是故意用错语法的（原文“because we hates it”），请不要来提交错误修正。为什么这么写？嗯，只不过是极其轻微地致敬一下《指环王》以及我们最喜欢的反英雄咕噜，没什么好激动的。

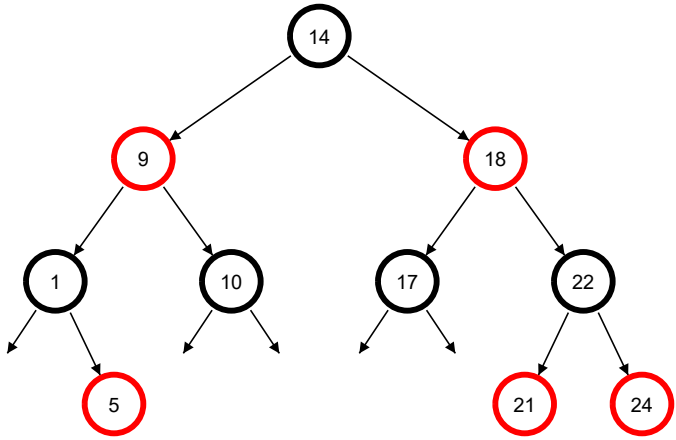



图 9.5：**CFS 红黑树**

### 处理 I/O 和睡眠的进程

总是挑选 `vruntime` 最低的进程来运行，遇到睡眠了很长时间的作业就会出问题。想象有两个进程 A 和 B，其中一个（A）一直在运行，另一个（B）睡眠了很长时间（比如 10 秒）。当 B 醒来时，它的 `vruntime` 会比 A 的落后 10 秒；因此（如果不小心的话），B 会在接下来的 10 秒里霸占 CPU 来追赶 A，实际上让 A 陷入了饥饿（starvation）。

CFS 处理这种情况的办法是，在作业醒来时修改它的 `vruntime`。具体来说，CFS 把这个作业的 `vruntime` 设为树中能找到的最小值（别忘了，树里只有正在运行的作业）[B+18]。CFS 由此避免了饥饿，但也付出了代价：经常短暂睡眠的作业永远得不到它们应得的公平份额（fair share） [AC97]。

### CFS 的其他有趣之处

CFS 还有许多其他特性，多到在本书的这个位置无法一一讨论。它有大量用来提高缓存（cache）性能的启发式方法，有高效处理多个 CPU 的策略（本书后面会讨论），还能把一大组进程当作一个单位来调度（而不是把每个进程当作一个独立的实体），此外还有许多其他有趣的特性。想了解更多，请阅读最近的研究，可以从 Bouron 的论文 [B+18] 读起。

## 9.8 小结

我们介绍了比例份额（proportional-share）调度的概念，并简要讨论了三种方法：彩票调度（lottery scheduling）、步长调度（stride scheduling）和 CFS。彩票调度（lottery scheduling）巧妙地利用随机性来实现比例份额（proportional-share）；步长调度（stride scheduling）则以确定性的方式做到这一点。CFS 是本章讨论的唯一一个“真正的”调度程序，它有点像带动态时间片的加权时间片轮转（Round-Robin），但在设计上注重可扩展性

提示：在合适的时候使用高效的数据结构

很多时候，用列表就够了。很多时候，又不够。知道什么时候该用哪种数据结构，是优秀工程能力的一个标志。在这里讨论的例子里，早期调度程序使用的简单列表在现代系统上效果很差，在数据中心里负载很重的服务器上尤其如此。这类系统有成千上万个活跃进程；每隔几毫秒就在每个核心上搜索一遍长列表，来找出下一个要运行的作业，会浪费宝贵的 CPU 周期。我们需要更好的结构，CFS 用一个出色的红黑树实现给出了答案。更一般地说，在为你正在构建的系统选择数据结构时，要仔细考虑它的访问模式和使用频率；理解了这些，你就能为手头的任务实现合适的结构。

（scalability），在高负载下也表现良好；在很长一段时间里，它都是使用最广泛的公平份额（fair-share）调度程序。

没有哪个调度程序是万能的，公平份额（fair-share）调度程序自然也分到了它“公平的一份”问题。一个问题是，这些方法和 I/O 配合得不太好 [AC97]；如上所述，偶尔执行 I/O 的作业可能得不到它应得的那份 CPU。另一个问题是，它们没有解决彩票（ticket）或优先级该如何分配这个难题：你怎么知道该给浏览器分配多少张彩票，或者该把文本编辑器的 nice 值设成多少？其他通用的调度程序（比如前面讨论过的 MLFQ）会自动处理这些问题，因此可能更容易部署。

好消息是，在许多领域里，这些问题并不是主要矛盾，比例份额（proportional-share）调度程序在那里能发挥很大的作用。例如，在**虚拟化**（virtualized）的数据中心（或**云**，cloud）中，你可能想把四分之一的 CPU 周期分给 Windows 虚拟机（virtual machine），其余的分给你的 Linux 主系统，这时比例分享就既简单又有效。这个思想也可以推广到其他资源上；详见 Waldspurger 的论文 [W02]。

最后，Linux 的世界总在变化。从 Linux 6.6 起，默认调度程序换成了 Earliest Eligible Virtual Deadline First（EEVDF）调度程序，这是另一种公平份额（fair-share）方法。想了解更多，就去读读关于 EEVDF 的资料吧 [Z23, SA95]。你还以为学完了呢！

 © 2008–26, ARPACI-DUSSEAU

## 参考文献

[AC97] "Extending Proportional-Share Scheduling to a Network of Workstations" by Andrea C. Arpaci-Dusseau and David E. Culler. PDPTA'97, June 1997. *A paper by one of the authors on how to extend proportional-share scheduling to work better in a clustered environment.*

[B+18] "The Battle of the Schedulers: FreeBSD ULE vs. Linux CFS" by J. Bouron, S. Chevalley, B. Lepers, W. Zwaenepoel, R. Gouicem, J. Lawall, G. Muller, J. Sopena. USENIX ATC '18, July 2018, Boston, Massachusetts. *A recent, detailed work comparing Linux CFS and the FreeBSD schedulers. An excellent overview of each scheduler is also provided. The result of the comparison: inconclusive. In some cases CFS was better, and in others, ULE (the BSD scheduler), was. Sometimes in life there are no easy answers.*

[B72] "Symmetric binary B-Trees: Data Structure And Maintenance Algorithms" by Rudolf Bayer. Acta Informatica, Volume 1, Number 4, December 1972. *A cool balanced tree introduced before you were born (most likely). One of many balanced trees out there; study your algorithms book for more alternatives!*

[D82] "Why Numbering Should Start At Zero" by Edsger Dijkstra, August 1982. Available: http://www.cs.utexas.edu/users/EWD/ewd08xx/EWD831.PDF. *A short note from E. Dijkstra, one of the pioneers of computer science. We'll be hearing much more on this guy in the section on Concurrency. In the meanwhile, enjoy this note, which includes this motivating quote: "One of my colleagues — not a computing scientist — accused a number of younger computing scientists of 'pedantry' because they started numbering at zero." The note explains why doing so is logical.*

[K+15] "Profiling A Warehouse-scale Computer" by S. Kanev, P. Ranganathan, J. P. Darago, K. Hazelwood, T. Moseley, G. Wei, D. Brooks. ISCA '15, June, 2015, Portland, Oregon. *A fascinating study of where the cycles go in modern data centers, which are increasingly where most of computing happens. Almost 20% of CPU time is spent in the operating system, 5% in the scheduler alone!*

[J09] "Inside The Linux 2.6 Completely Fair Scheduler" by M. Tim Jones. December 15, 2009. https://pages.cs.wisc.edu/%7Eremzi/OSTEP/Citations/inside-cfs.pdf. *This is a simple overview of CFS from its earlier days. CFS was created by Ingo Molnar in a short burst of creativity which led to a 100K kernel patch developed in 62 hours.*

[KL88] "A Fair Share Scheduler" by J. Kay and P. Lauder. CACM, Volume 31 Issue 1, January 1988. *An early reference to a fair-share scheduler.*

[SA95] "Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation" by Ion Stoica and Hussein M. Abdel-Wahab. Technical report, CS Department, Old Dominion University, TR-95-22. *The technical report that introduced EEVDF. Ion Stoica has since gone on to having great impact in systems, including with Apache Spark, the founding of Databricks, and much more.*

[WW94] "Lottery Scheduling: Flexible Proportional-Share Resource Management" by Carl A. Waldspurger and William E. Weihl. OSDI '94, November 1994. *The landmark paper on lottery scheduling that got the systems community re-energized about scheduling, fair sharing, and the power of simple randomized algorithms.*

[W95] "Lottery and Stride Scheduling: Flexible Proportional-Share Resource Management" by Carl A. Waldspurger. Ph.D. Thesis, MIT, 1995. *The award-winning thesis of Waldspurger's that outlines lottery and stride scheduling. If you're thinking of writing a Ph.D. dissertation at some point, you should always have a good example around, to give you something to strive for: this is such a good one.*

[W02] "Memory Resource Management in VMware ESX Server" by Carl A. Waldspurger. OSDI '02, Boston, Massachusetts. *The paper to read about memory management in VMMs (a.k.a., hypervisors). In addition to being relatively easy to read, the paper contains numerous cool ideas about this new type of VMM-level memory management.*

[Z23] "The EEVDF Scheduler" by Peter Zijlstra. Please go to this URL for details on EEVDF: https://docs.kernel.org/scheduler/sched-eevdf.html. *Linux is always a'changing!*

 © 2008–26, ARPACI-DUSSEAU

### 作业（模拟）

程序 `lottery.py` 可以让你观察彩票调度程序（lottery scheduler）是如何工作的。详情请见 README。

### 问题

1. 计算 3 个作业、随机种子分别为 1、2、3 时的模拟结果。
2. 现在用两个特定的作业运行：两个作业的长度都是 10，但一个（作业 0）只有 1 张彩票（ticket），另一个（作业 1）有 100 张（例如 `-l 10:1,10:100`）。彩票（ticket）数这么不均衡时，会发生什么？作业 0 会在作业 1 完成之前运行吗？多久运行一次？总的来说，这样的彩票（ticket）不均衡会对彩票调度（lottery scheduling）的行为产生什么影响？
3. 运行两个作业，长度都是 100，彩票（ticket）数也都是 100（`-l 100:100,100:100`）。这时调度程序有多不公平？用一些不同的随机种子运行，求出（概率意义上的）答案；用一个作业比另一个作业早完成多少来衡量不公平程度。
4. 随着量子大小（quantum size，`-q`）变大，你对上一题的答案会如何变化？
5. 你能画出本章中那张图的一个版本吗？还有什么值得探索的？换成步长调度程序（stride scheduler），这张图会是什么样子？

# 10

# 多处理器调度（进阶）

本章介绍**多处理器调度**（multiprocessor scheduling）的基础知识。这个话题偏难，最好等你比较扎实地学过并发（concurrency，也就是本书的第二个“简单部分”）之后再来读。

**多处理器**（multiprocessor）系统多年来只存在于计算领域的高端，如今却越来越常见，已经进入台式机、笔记本电脑，甚至移动设备。这种普及源于**多核**（multicore）处理器的兴起：同一块芯片上集成了多个 CPU 核心。这类芯片之所以流行，是因为计算机架构师发现，要让单个 CPU 快很多，又不消耗（远远）过多的电能，实在太难。于是，现在我们每个人手上都有好几个 CPU 可用，这是件好事，对吧？

当然，CPU 多了，难题也随之而来。其中最主要的一个是：典型的应用程序（比如你写的某个 C 程序）只会用到一个 CPU；加再多 CPU，这个应用程序也不会跑得更快。要解决这个问题，你得重写应用程序，让它能**并行**（parallel）运行，比如使用**线程**（threads），这在本书第二部分会详细讨论。多线程（multi-threaded）应用程序可以把工作分散到多个 CPU 上，因此给它更多 CPU 资源，它就能跑得更快。

除了应用程序，操作系统也面临一个新问题（这不奇怪！）：**多处理器调度**（multiprocessor scheduling）。到目前为止，我们已经讨论了单处理器调度背后的许多原则；怎样把这些思想推广到多个 CPU 上？我们又必须克服哪些新问题？于是问题来了：

关键问题：如何在多个 CPU 上调度作业

操作系统应该如何在多个 CPU 上调度作业？会出现哪些新问题？原来那些老技术还管用吗，还是需要新的思路？

补充：进阶章节

进阶章节要用到本书很大一部分内容才能真正读懂，但从逻辑上讲，它们又该排在这些前置内容之前。例如，本章讲多处理器调度（multiprocessor scheduling），如果你先读过中间关于并发的部分，理解起来会容易得多；但按内容归类，它属于本书讲虚拟化（virtualization）的部分，具体来说是讲 CPU 调度的那部分。因此，这类章节最好不按顺序读；就本章而言，建议在读完本书第二部分之后再读。

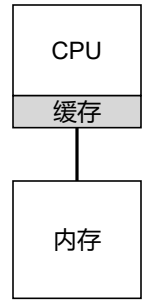



图 10.1：**带缓存的单个 CPU**

## 10.1 背景：多处理器架构

要理解多处理器调度（multiprocessor scheduling）面临的新问题，就得先理解单 CPU 硬件和多 CPU 硬件之间一个全新的根本区别。这个区别的核心在于硬件**缓存**（cache）的使用方式（如图 10.1），以及数据究竟如何在多个处理器之间共享。下面我们从较高的层次进一步讨论这个问题。更多细节请参阅其他资料 [CSG99]，尤其是高年级本科或研究生的计算机体系结构课程。

在只有一个 CPU 的系统中，**硬件缓存**（hardware cache）组成一个层次结构，通常能帮助处理器更快地运行程序。缓存是一些小而快的存储器，（一般来说）保存着系统主存中常用数据的副本。相比之下，主存保存着全部数据，但访问这块更大的存储器要慢一些。把经常访问的数据放在缓存里，系统就能让又大又慢的存储器看起来像是快的。

举个例子：假设一个程序发出一条显式的加载指令，从内存中取一个值，而系统很简单：只有一个 CPU，带一个小缓存，外加一个大的主存。程序第一次发出这条加载指令时，数据在主存里，因此要花很长时间才能取到（可能要几十纳秒，甚至几百纳秒）。处理器料到这个数据可能还会用到，于是把取来的数据复制一份放进 CPU 缓存。如果程序之后再次读取同一个数据，CPU 会先在缓存里找；如果找到了，取数据就快得多（比如只要几纳秒），程序也就跑得更快。

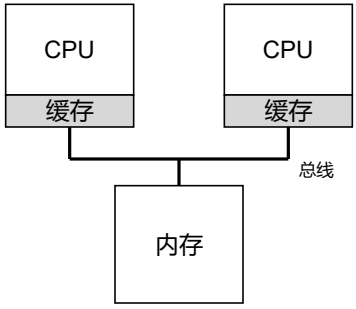



图 10.2：**两个带缓存的 CPU 共享内存**

 © 2008–25, ARPACI-DUSSEAU

因此，缓存建立在**局部性**（locality）这个概念之上。局部性有两种：**时间局部性**（temporal locality）和**空间局部性**（spatial locality）。时间局部性的思想是：一个数据被访问过之后，很可能在不久的将来再次被访问；想象一下循环里被一遍又一遍访问的变量，甚至指令本身。空间局部性的思想是：如果程序访问了地址 $x$ 处的数据，那么它很可能也会访问 $x$ 附近的数据；想想程序顺序遍历一个数组，或者指令一条接一条地执行。由于许多程序都具有这两种局部性，硬件系统就能比较准确地猜出该把哪些数据放进缓存，因而效果很好。

接下来是棘手的部分：如果一个系统里有多个处理器，共享同一个主存（如图 10.2 所示），会怎么样？

原来，有多个 CPU 时，缓存要复杂得多。例如，假设运行在 CPU 1 上的程序读取了地址 $A$ 处的一个数据（值为 $D$）；由于 CPU 1 的缓存里没有这个数据，系统就从主存中取出它，得到值 $D$。然后程序修改了地址 $A$ 处的值，只是把新值 $D'$ 写进了自己的缓存；把数据一路写到主存很慢，所以系统（通常）会晚些再做。接着，假设操作系统决定停止运行这个程序，把它挪到 CPU 2 上。程序再次读取地址 $A$ 处的值；CPU 2 的缓存里没有这个数据，于是系统从主存中取，得到的是旧值 $D$，而不是正确的值 $D'$。糟糕！

这个一般性问题叫作**缓存一致性**（cache coherence）问题，解决它涉及许多微妙之处，相关的研究文献非常多 [SHW11]。这里我们略过所有细节，只讲几个要点；想了解更多，就去上一门（或者三门）计算机体系结构课吧。

基本的解决方案由硬件提供：硬件会监视内存访问，基本上能保证一切按“正确”的方式进行，并维持只有一个共享内存（shared memory）的视图。在基于总线（bus）的系统中（如上所述），实现这一点的一种方法是使用一种叫作**总线窥探**（bus snooping）的老技术 [G83]：每个缓存都监听把自己连到主存的那条总线，留意内存的更新。某个 CPU 一旦看到自己缓存着的数据被更新，就会让自己的副本**失效**（invalidate，即把它从自己的缓存中删除），或者**更新**（update）它（即把新值也放进自己的缓存）。前面暗示过，回写缓存（write-back cache）会让这件事更复杂（因为写到主存的操作要过一阵才能看见），但你可以想象这个基本方案是如何工作的。

## 10.2 别忘了同步

既然缓存已经做了这么多工作来提供一致性，那么程序（或操作系统本身）访问共享数据时，还需要操心什么吗？遗憾的是，答案是肯定的，本书讲并发的第二部分对此有非常详细的说明。这里我们不深入细节，只简单勾勒（或回顾）一下其中的一些基本思想（假定你已经熟悉并发）。

多个 CPU 访问（尤其是更新）共享的数据项或数据结构（data structure）时，很可能需要使用锁（lock）这类互斥（mutual exclusion）原语来保证正确性（其他方法，比如构建**无锁**（lock-free）数据结构，很复杂，只在少数情况下使用；详见并发部分讲死锁（deadlock）的那一章）。例如，假设有一个共享队列（queue），多个 CPU 正在并发访问它。如果没有锁，即使有底层的一致性协议（coherence protocol），并发地（concurrently）往队列里添加或删除元素也不会按预期工作；需要用锁来原子地（atomically）把数据结构更新到新状态。

具体来说，设想图 10.3 这段从共享链表（linked list）中删除元素的代码。假设两个 CPU 上的线程同时进入这个例程。如果线程 1 执行了第一行，它的 `tmp` 变量里就保存了 `head` 的

```
1  typedef struct __Node_t {
2      int              value;
3      struct __Node_t *next;
4  } Node_t;
5
6  int List_Pop() {
7      Node_t *tmp = head;          // 记住原来的头节点
8      int value   = head->value;   // .....以及它的值
9      head        = head->next;    // 前进到下一个节点
10     free(tmp);                   // 释放原来的头节点
11     return value;                // 返回头节点的值
12 }
```

图 10.3：**简单的链表删除代码**

当前值；如果线程 2 接着也执行了第一行，它自己私有的 `tmp` 变量里也会保存同一个 `head` 值（`tmp` 是在栈上分配的，因此每个线程都有自己私有的一份）。于是，两个线程不是各自从链表头部删除一个元素，而是都试图删除同一个头部元素，从而导致各种问题（比如在第 10 行试图把头部元素释放两次，还可能把同一个数据值返回两次）。

当然，解决办法是**加锁**（locking），让这类例程变得正确。在这个例子中，分配一个简单的互斥锁（mutex，例如 `pthread_mutex_t m;`），然后在例程开头加上 `lock(&m)`、在末尾加上 `unlock(&m)`，就能解决问题，确保代码按预期执行。遗憾的是，我们会看到，这种做法并非没有问题，尤其是在性能方面。具体来说，随着 CPU 数量增加，访问一个同步的共享数据结构会变得相当慢。

## 10.3 最后一个问题：缓存亲和性

构建多处理器调度程序（multiprocessor scheduler）时，还会出现最后一个与缓存有关的问题，叫作**缓存亲和性**（cache affinity）[TTG95]。这个概念很简单：一个进程（process）在某个 CPU 上运行时，会在这个 CPU 的缓存和 TLB 中积累相当多的状态。下次这个进程运行时，让它在同一个 CPU 上运行往往是有利的，因为如果它的部分状态已经在那个 CPU 的缓存里了，它就会跑得更快。反之，如果每次都让进程在不同的 CPU 上运行，进程的性能就会变差，因为它每次运行都得重新加载状态（注意，多亏了硬件的缓存一致性协议（cache coherence protocol），它在不同的 CPU 上也能正确运行）。因此，多处理器调度程序（multiprocessor scheduler）在做调度决策时应该考虑缓存亲和性（cache affinity），只要有可能，就尽量让进程留在同一个 CPU 上。

## 10.4 单队列调度

有了这些背景知识，下面我们讨论如何为多处理器系统构建调度程序（scheduler）。最基本的方法是直接沿用单处理器调度的框架，把所有需要调度的作业放进一个队列；我们把这种方法叫作**单队列多处理器调度**（single-queue multiprocessor scheduling），简称 **SQMS**。这种方法的优点是简单：现成的策略本来就能挑出下一个最该运行的作业，把它改

造成能在多个 CPU 上工作的版本（比如有两个 CPU 时，就挑出两个最该运行的作业）并不费事。

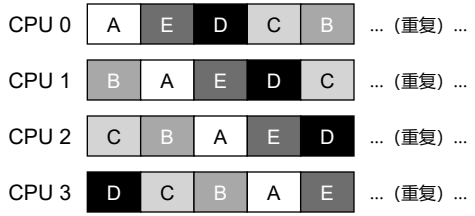
然而，SQMS 有明显的缺点。第一个问题是缺乏**可扩展性**（scalability）。为了确保调度程序在多个 CPU 上正确工作，开发者会像前面说的那样，在代码里加入某种形式的**锁**（locking）。锁能确保 SQMS 的代码访问那个唯一的队列时（比如找下一个要运行的作业），得到正确的结果。

遗憾的是，锁会大大降低性能，尤其是在系统中 CPU 数量增多时 [A90]。这把唯一的锁上的争用（contention）越激烈，系统花在锁开销上的时间就越多，花在本该做的工作上的时间就越少（注：要是哪天能在这里放上真实的测量数据就好了）。

SQMS 的第二个主要问题是缓存亲和性（cache affinity）。例如，假设有五个作业要运行（$A$、$B$、$C$、$D$、$E$），有四个处理器。那么调度队列（scheduling queue）是这样的：

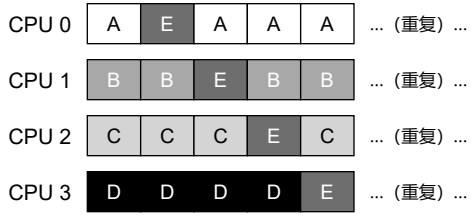



假设每个作业运行一个时间片（time slice）后换下一个作业，那么随着时间推移，各 CPU 上的调度可能是这样的：




由于每个 CPU 只是从全局共享的队列里挑下一个要运行的作业，每个作业最终都在各个 CPU 之间来回跳，这恰恰和缓存亲和性（cache affinity）的要求背道而驰。

为了解决这个问题，大多数 SQMS 调度程序都包含某种亲和性（affinity）机制，让进程更有可能继续在同一个 CPU 上运行。具体来说，可以为某些作业保持亲和性（affinity），而挪动另一些作业来平衡负载。例如，设想同样这五个作业按下面这样调度：




在这种安排下，作业 $A$ 到 $D$ 都不在处理器之间移动，只有作业 $E$ 在 CPU 之间**迁移**（migrating），从而为大多数作业保持了亲和性（affinity）。下一轮你可以决定迁移另一个作业，这样也能实现某种亲和性（affinity）上的公平。不过，实现这样的方案可能会很复杂。

由此可见，SQMS 方法有优点也有缺点。如果已经有一个单 CPU 调度程序，按定义它只有一个队列，那么实现 SQMS 很简单。但是，它的扩展性不好（因为有同步开销），也不太容易保持缓存亲和性（cache affinity）。

## 10.5 多队列调度

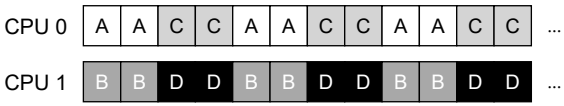
由于单队列调度程序（single-queue scheduler）存在这些问题，有些系统选择使用多个队列，比如每个 CPU 一个。我们把这种方法叫作**多队列多处理器调度**（multi-queue multiprocessor scheduling，**MQMS**）。

在 MQMS 中，基本的调度框架由多个调度队列组成。每个队列可能遵循某种特定的调度规则（scheduling discipline），比如时间片轮转（Round-Robin），当然也可以使用任何算法。作业进入系统时，会根据某种启发式规则（比如随机挑选，或者挑一个作业比其他队列少的队列）被放进某一个调度队列，而且只放进一个。此后它基本上独立调度，从而避免了单队列（single-queue）方法中信息共享和同步（synchronization）带来的问题。

例如，假设系统只有两个 CPU（标记为 CPU 0 和 CPU 1），有若干作业进入系统：比如 *A*、*B*、*C* 和 *D*。由于现在每个 CPU 都有一个调度队列，操作系统必须决定把每个作业放进哪个队列。它可能会这样放：

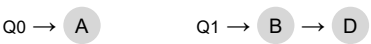



根据队列的调度策略（scheduling policy），现在每个 CPU 在决定运行什么时，都有两个作业可选。例如，采用**时间片轮转**（round robin）时，系统可能产生这样的调度：

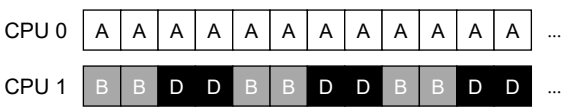



MQMS 相对于 SQMS 有一个明显的优势：它天生就应该更具可扩展性。随着 CPU 数量增加，队列的数量也随之增加，因此锁争用（lock contention）和缓存争用（cache contention）不应成为核心问题。此外，MQMS 本身就提供了缓存亲和性（cache affinity）：作业一直留在同一个 CPU 上，可以重复利用缓存里的内容。

不过，如果你一直在留心，可能已经发现了一个新问题，它是基于多队列（multi-queue）的方法所固有的：**负载不均**（load imbalance）。假设设置和上面一样（四个作业、两个 CPU），但随后其中一个作业（比如 *C*）结束了。现在调度队列变成了这样：

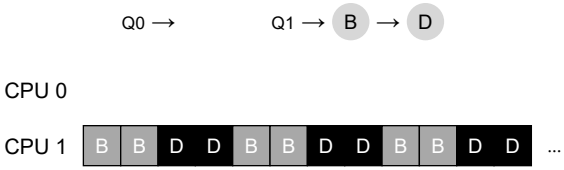



如果接着在系统的每个队列上运行时间片轮转（Round-Robin）策略，就会得到这样的调度：




 © 2008–25, ARPACI-DUSSEAU

从图中可以看到，*A* 得到的 CPU 时间是 *B* 和 *D* 的两倍，这不是我们想要的结果。更糟的是，假设 *A* 和 *C* 都结束了，系统中只剩下作业 *B* 和 *D*。两个调度队列以及由此得到的时间线会是这样：



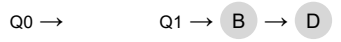

太糟糕了，CPU 0 闲着！（此处插入悲壮而阴森的音乐）于是我们的 CPU 使用时间线看起来相当凄惨。

那么，可怜的多队列多处理器调度程序（multi-queue multiprocessor scheduler）该怎么办？我们怎样才能克服负载不均（load imbalance）这个隐蔽的问题，打败……霸天虎[1]的邪恶势力？我们怎样才能不再问这些和这本原本很精彩的书几乎毫不相干的问题？

关键问题：如何应对负载不均
多队列多处理器调度程序（multi-queue multiprocessor scheduler）应该如何应对负载不均（load imbalance），以便更好地实现它的调度目标？

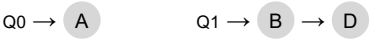
这个问题显而易见的答案是把作业挪来挪去，这种技术我们（又一次）称为**迁移**（migration）。把作业从一个 CPU 迁移到另一个 CPU，就能真正做到负载均衡（load balance）。

我们来看几个例子，把问题讲清楚一些。同样，一个 CPU 闲着，另一个上面有一些作业。



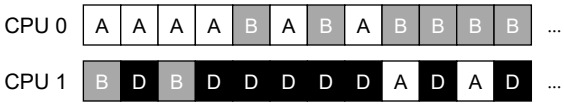

在这种情况下，该怎么迁移很好理解：操作系统只要把 *B* 或 *D* 中的一个挪到 CPU 0 上就行了。只迁移这一次就实现了负载均衡（load balance），皆大欢喜。

更棘手的情况出现在前面那个例子里：*A* 独自留在 CPU 0 上，*B* 和 *D* 在 CPU 1 上交替运行。




在这种情况下，迁移一次解决不了问题。你会怎么做？可惜，答案是持续地迁移一个或多个作业。一种可能的办法是不断切换作业，如下面的时间线所示。图中，一开始 *A* 独自在 CPU 0 上，*B* 和 *D* 在 CPU 1 上交替运行。几个时间片之后，*B* 被挪到 CPU 0 上和 *A* 竞争，而 *D* 则在 CPU 1 上独享几个时间片。这样负载就均衡了：

[1]鲜为人知的事实：塞伯坦星球就是被糟糕的 CPU 调度决策毁掉的。就让这成为本书第一次也是最后一次提到变形金刚吧，我们为此真诚地道歉。




当然，可能的迁移模式还有很多。但现在到了棘手的部分：系统该如何决定迁移？

一种基本的方法是**工作窃取**（work stealing）[FLR98]：作业较少的（源）队列时不时看一眼另一个（目标）队列有多满；如果目标队列（明显）比源队列满，源队列就从目标队列“窃取”一个或多个作业，以平衡负载。

当然，这种方法有一种天然的矛盾：查看其他队列太频繁，开销就高，也难以扩展，而可扩展恰恰是当初实现多队列（multi-queue）调度的全部目的！反过来，查看得太少，又可能出现严重的负载不均（load imbalance）。和系统策略设计中常见的情况一样，找到合适的阈值仍然是一门玄妙的手艺。

## 10.6 Linux 多处理器调度程序

有意思的是，在 Linux 社区里，构建多处理器调度程序（multiprocessor scheduler）一直没有形成统一的方案。随着时间推移，先后出现了三种不同的调度程序：O(1) 调度程序、完全公平调度程序（Completely Fair Scheduler，CFS）和 BF 调度程序（BFS）[2]。Meehean 的博士论文很好地概述了这几种调度程序的优缺点 [M11]；这里我们只总结几点基本情况。

O(1) 和 CFS 都使用多个队列，而 BFS 使用单个队列，可见两种方法都能成功。当然，它们在其他细节上还有许多差别。例如，O(1) 是基于优先级（priority）的调度程序（类似前面讲过的 MLFQ），它随时间调整进程的优先级，并调度优先级最高的进程，以实现各种调度目标，尤其注重交互性（interactivity）。CFS 则是一种确定性的比例份额（proportional-share）方法，更像前面讲过的步长调度（stride scheduling）。BFS 是三者中唯一的单队列（single-queue）方法，它也是比例份额（proportional-share）的，但基于一种更复杂的方案，叫作 Earliest Eligible Virtual Deadline First（EEVDF）[SA96]。这些现代算法请自己去读一读；现在你应该能看懂它们是怎么工作的了！

## 10.7 小结

本章介绍了多处理器调度（multiprocessor scheduling）的各种方法。单队列方法（SQMS）实现简单，负载均衡得好，但天生难以扩展到很多处理器，也难以保持缓存亲和性（cache affinity）。多队列方法（MQMS）扩展性更好，也能很好地处理缓存亲和性（cache affinity），但有负载不均（load imbalance）的问题，而且更复杂。无论哪种方法都没有简单的答案：构建通用的调度程序仍是一项艰巨的任务，因为代码上的小改动就可能导致行为上的巨大差异。除非你确切知道自己在做什么，或者至少有人为此付你一大笔钱，否则别去碰这件事。

[2] BF 代表什么，请自己去查；先提醒一句，胆小的人慎查。

## 参考文献

[A90] “The Performance of Spin Lock Alternatives for Shared-Memory Multiprocessors” by Thomas E. Anderson. IEEE TPDS Volume 1:1, January 1990. *A classic paper on how different locking alternatives do and don't scale. By Tom Anderson, very well known researcher in both systems and networking. And author of a very fine OS textbook, we must say.*

[B+10] “An Analysis of Linux Scalability to Many Cores Abstract” by Silas Boyd-Wickizer, Austin T. Clements, Yandong Mao, Aleksey Pesterev, M. Frans Kaashoek, Robert Morris, Nickolai Zeldovich. OSDI '10, Vancouver, Canada, October 2010. *A terrific modern paper on the difficulties of scaling Linux to many cores.*

[CSG99] “Parallel Computer Architecture: A Hardware/Software Approach” by David E. Culler, Jaswinder Pal Singh, and Anoop Gupta. Morgan Kaufmann, 1999. *A treasure filled with details about parallel machines and algorithms. As Mark Hill humorously observes on the jacket, the book contains more information than most research papers.*

[FLR98] “The Implementation of the Cilk-5 Multithreaded Language” by Matteo Frigo, Charles E. Leiserson, Keith Randall. PLDI '98, Montreal, Canada, June 1998. *Cilk is a lightweight language and runtime for writing parallel programs, and an excellent example of the work-stealing paradigm.*

[G83] “Using Cache Memory To Reduce Processor-Memory Traffic” by James R. Goodman. ISCA '83, Stockholm, Sweden, June 1983. *The pioneering paper on how to use bus snooping, i.e., paying attention to requests you see on the bus, to build a cache coherence protocol. Goodman's research over many years at Wisconsin is full of cleverness, this being but one example.*

[M11] “Towards Transparent CPU Scheduling” by Joseph T. Meehean. Doctoral Dissertation at University of Wisconsin—Madison, 2011. *A dissertation that covers a lot of the details of how modern Linux multiprocessor scheduling works. Pretty awesome! But, as co-advisors of Joe's, we may be a bit biased here.*

[SHW11] “A Primer on Memory Consistency and Cache Coherence” by Daniel J. Sorin, Mark D. Hill, and David A. Wood. Synthesis Lectures in Computer Architecture. Morgan and Claypool Publishers, May 2011. *A definitive overview of memory consistency and multiprocessor caching. Required reading for anyone who likes to know way too much about a given topic.*

[SA96] “Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation” by Ion Stoica and Hussein Abdel-Wahab. Technical Report TR-95-22, Old Dominion University, 1996. *A tech report on this cool scheduling idea, from Ion Stoica, now a professor at U.C. Berkeley and world expert in networking, distributed systems, and many other things.*

[TTG95] “Evaluating the Performance of Cache-Affinity Scheduling in Shared-Memory Multiprocessors” by Josep Torrellas, Andrew Tucker, Anoop Gupta. Journal of Parallel and Distributed Computing, Volume 24:2, February 1995. *This is not the first paper on the topic, but it has citations to earlier work, and is a more readable and practical paper than some of the earlier queuing-based analysis papers.*

 © 2008–25, ARPACI-DUSSEAU

## 作业（模拟）

在这个作业中，我们将用 `multi.py` 模拟一个多处理器 CPU 调度程序，并了解它的一些细节。关于这个模拟器及其选项的更多信息，请阅读相关的 README。

## 问题

1. 首先，我们来熟悉这个模拟器，后面要用它研究怎样构建高效的多处理器调度程序（multiprocessor scheduler）。第一个模拟只运行一个作业，它的运行时间为 30，工作集（working set）大小为 200。在一个模拟的 CPU 上运行这个作业（这里叫作作业“a”），命令如下：

   ```
   ./multi.py -n 1 -L a:30:200
   ```

   它需要多长时间才能完成？打开 `-c` 选项可以看到最终答案，打开 `-t` 选项可以逐个时钟滴答地跟踪这个作业，看清调度程序是怎样安排它的。

2. 现在增大缓存，让这个作业的工作集（working set，大小为 200）能放进缓存（默认大小为 100）；例如，运行：

   ```
   ./multi.py -n 1 -L a:30:200 -M 300
   ```

   你能预测出作业放进缓存之后会运行多快吗？（提示：记住预热速率这个关键参数，它由 `-r` 选项设置。）打开求解选项（`-c`）运行，检查你的答案。

3. `multi.py` 有一个很酷的地方：你可以用不同的跟踪选项看到更多细节。运行和上面相同的模拟，但这次打开剩余时间跟踪（`-T`）。这个选项会显示每个时间步调度到 CPU 上的作业，以及每个时钟滴答之后这个作业还剩多少运行时间。你注意到第二列是怎样递减的吗？

4. 现在再加一项跟踪，用 `-C` 选项显示每个 CPU 缓存对每个作业的状态。对每个作业，每个缓存要么显示一个空格（缓存对这个作业是冷的），要么显示一个“w”（缓存对这个作业是热的）。在这个简单的例子里，缓存在什么时候对作业“a”变热？把预热时间参数（`-w`）改成比默认值小或大的值，会发生什么？

5. 到现在，你应该已经很清楚模拟器在单个 CPU 上运行单个作业时是怎么工作的了。不过，这不是讲多处理器 CPU 调度的一章吗？没错！那我们开始处理多个作业吧。具体来说，在一个双 CPU 系统上运行下面三个作业，即输入：

   ```
   ./multi.py -n 2 -L a:100:100,b:100:50,c:100:50
   ```

   在集中式的时间片轮转（Round-Robin）调度程序下，你能预测出这需要多长时间吗？用 `-c` 看看你对不对，然后用 `-t` 深入看看逐步的细节，再用 `-C` 看看这些作业的缓存是否有效地预热了。你注意到了什么？

6. 现在我们施加一些显式的控制，来研究本章讲到的**缓存亲和性**（cache affinity）。为此，你需要用到 `-A` 选项。这个选项可以限制调度程序能把某个作业放在哪些 CPU 上。这

里，我们用它把作业“b”和“c”放在 CPU 1 上，而把“a”限制在 CPU 0 上。要实现这个魔法，输入：

```
./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 \
    -A a:0,b:1,c:1
```

别忘了打开各种跟踪选项，看看到底发生了什么！你能预测这个版本会运行多快吗？它为什么表现更好？把“a”“b”“c”以其他组合放到两个处理器上，会运行得更快还是更慢？

7. 带缓存的多处理器有一个有意思的地方：与在单个处理器上运行作业相比，使用多个 CPU（及其缓存）时，作业有机会获得比预期更好的加速。具体来说，在 $N$ 个 CPU 上运行时，有时可以获得超过 $N$ 倍的加速，这种情况叫作**超线性加速**（super-linear speedup）。为了验证这一点，用下面的作业描述和一个小缓存创建三个作业：

   ```
   -L a:100:100,b:100:100,c:100:100
   ```

   分别在有 1、2、3 个 CPU 的系统上运行（`-n 1`、`-n 2`、`-n 3`）。然后再做一遍，但把每个 CPU 的缓存大小改为 100。随着 CPU 数量增加，你注意到性能有什么变化？用 `-c` 验证你的猜测，再用其他跟踪选项深入研究。

8. 模拟器另一个值得研究的方面是按 CPU 调度的选项，即 `-p` 选项。再次用两个 CPU 运行，并使用这样的三作业配置（`-L a:100:100,b:100:50,c:100:50`）。和上面你手动设置的亲和性（affinity）限制相比，这个选项表现如何？把“窥视间隔”（`-P`）改小或改大，性能会怎样变化？随着 CPU 数量增加，这种按 CPU 调度的方法效果如何？

9. 最后，你可以随意生成随机的工作负载（workload），看看能否预测出它们在不同处理器数量、缓存大小和调度选项下的性能。这样做下去，你很快就会成为**多处理器调度大师**，这可是件相当了不起的事。祝你好运！

# 11

# 关于 CPU 虚拟化的总结对话

**教授**：那么，同学，你学到什么了吗？

**学生**：嗯，教授，这问题好像有点诱导啊。我觉得您就是想让我说“学到了”。

**教授**：没错。不过我是真心在问。拜托，给教授留点面子，好不好？

**学生**：好吧，好吧。我觉得我确实学到了几样东西。首先，我稍微了解了操作系统是怎样虚拟化（*virtualize*）CPU 的。要弄懂这件事，我得理解一大堆重要的**机制**（*mechanism*）：陷阱（*trap*）和陷阱处理程序（*trap handler*）、时钟中断（*timer interrupt*），还有在进程（*process*）之间切换时，操作系统和硬件必须怎样小心地保存和恢复状态。

**教授**：很好，很好！

**学生**：不过，这些交互看起来还是有点复杂；我怎样才能学得更深入呢？

**教授**：嗯，这是个好问题。我觉得没有什么能代替亲手去做；光是读这些东西，并不能让你真正体会到其中的门道。去做课程项目吧，我敢打赌，做到最后，你差不多就全明白了。

**学生**：听起来不错。我还能跟您说点什么？

**教授**：嗯，在你努力弄懂操作系统基本机制的过程中，有没有体会到一点操作系统的设计哲学？

**学生**：嗯……我想有吧。操作系统好像相当多疑。它想确保自己始终掌控着机器。它希望程序尽可能高效地运行（这也是**受限直接执行**（*limited direct execution*）背后的全部道理），但一旦遇到出错的或者恶意的进程，它也希望能说一句：“啊哈！别急，朋友。”多疑主宰一切，也确实让操作系统一直掌控着机器。也许正因为如此，我们才把操作系统看作资源管理器（*resource manager*）。

**教授**：确实如此，听起来你开始把这些东西串起来了！不错。

**学生**：谢谢。

**教授**：那么，构建在这些机制之上的策略（*policy*）呢？有什么有意思的收获吗？

**学生**：那方面当然也有一些收获。也许有点显而易见，不过显而易见也可以是好事。比如把短作业挪到队列（*queue*）前面这个思路。自从有一次我在商店买口香糖，排在我

前面的那个人信用卡怎么都刷不了，我就知道这是个好主意了。我跟您说，他可绝对不是什么短作业。

**教授**：这么说那个可怜的家伙，好像有点不厚道。还有别的吗？

**学生**：嗯，还有就是，原来可以造出一个聪明的调度程序（*scheduler*），既像 *SJF* 又像 *RR*——*MLFQ* 真挺巧妙的。要构建一个真正的调度程序，看起来挺难的。

**教授**：确实很难。所以直到今天，用哪个调度程序仍然有争议；比如看看 *Linux* 里 *CFS*、*BFS* 和 *O(1)* 调度程序之间的论战就知道了。还有，不，我不会把 *BFS* 的全名说出来的。

**学生**：我也不会让您说的！这些关于策略的论战好像会永远争下去；真的存在正确答案吗？

**教授**：可能没有。毕竟，就连我们自己定的指标都是互相矛盾的：如果你的调度程序周转时间（*turnaround time*）好，响应时间（*response time*）就差，反过来也一样。正如 *Lampson* 所说，也许目标不是找到最好的方案，而是避免灾难。

**学生**：这有点让人泄气。

**教授**：好的工程就是这样。不过它也可以很鼓舞人心！其实就看你从什么角度去看了。我个人认为，务实是件好事，而务实的人明白，并不是所有问题都有干净又简单的解决办法。还有什么让你感兴趣的吗？

**学生**：我真的很喜欢“钻调度程序的空子（*game the scheduler*）”这个想法；下次我在亚马逊的 *EC2* 服务上跑作业的时候，也许可以研究研究。说不定我能从别的毫无防备的（更重要的是，不懂操作系统的）客户那里偷一些 *CPU* 周期！

**教授**：看来我可能造出了一个怪物！你知道，我可不想被人叫作弗兰肯斯坦教授。

**学生**：可这不正是您的目的吗？让我们对某样东西感兴趣，兴趣大到会自己去钻研？点燃火种之类的？

**教授**：我想是吧。不过我没想到这真管用！

 © 2008–23, ARPACI-DUSSEAU

# 12

# 关于内存虚拟化的对话

**学生**：那么，虚拟化（*virtualization*）讲完了吗？

**教授**：没有！

**学生**：嘿，没必要这么激动吧；我只是问个问题而已。学生本来就该提问的，对吧？

**教授**：嗯，教授们确实总这么说，但他们真正的意思是：**如果**是好问题，**而且**你确实动脑子想过，那就问吧。

**学生**：唉，这话可真让我泄气。

**教授**：目的达到了。不管怎样，虚拟化还远远没讲完！你刚刚看到的只是怎样虚拟化*CPU*，真正的大怪物还躲在衣柜里：内存。虚拟化内存很复杂，需要我们弄懂硬件和操作系统之间更多错综复杂的交互细节。

**学生**：听起来很酷。为什么这么难？

**教授**：嗯，细节很多，你得在脑子里把它们理清楚，才能真正建立起关于整个过程的心智模型。我们会从简单的开始，先讲像基址（*base*）/界限（*bounds*）这样非常基本的技术，再慢慢增加复杂度，应对新的挑战，其中包括*TLB*和多级页表（*multi-level page table*）这些有趣的话题。最后，我们就能讲清楚一个功能完整的现代虚拟内存（*virtual memory*）管理器是怎样工作的。

**学生**：太好了！对于被这么多信息淹没、又总是睡眠不足的可怜学生，您有什么建议吗？

**教授**：睡眠不足好办：多睡觉（少开派对）。至于理解虚拟内存，先从这一点开始：**用户程序产生的每一个地址都是虚拟地址**。操作系统只是给每个进程（*process*）营造了一个假象，具体来说，就是让它以为自己拥有一大块私有的内存；在一些硬件的帮助下，操作系统会把这些假装的虚拟地址（*virtual address*）转换成真实的物理地址（*physical address*），从而找到所需的信息。

**学生**：好，我想我能记住……（自言自语）用户程序的每个地址都是虚拟的，用户程序的每个地址都是虚拟的，每个……

**教授**：你在嘀咕什么呢？

**学生**：哦，没什么……（尴尬的停顿）……话说回来，操作系统为什么又要提供这种假象来着？

**教授**：主要是为了**易用性**（*ease of use*）：操作系统会让每个程序都觉得自己有一个很大的连续**地址空间**（*address space*），可以把代码和数据放进去；因此，作为程序员，你永远不用操心诸如“这个变量该放在哪儿?”之类的事情，因为程序的虚拟地址空间（*virtual address space*）很大，有的是地方放这些东西。如果你得操心怎样把所有代码和数据塞进一块又小又挤的内存里，程序员的日子就要难过得多。

**学生**：还有别的原因吗？

**教授**：嗯，**隔离**（*isolation*）和**保护**（*protection*）也是大事。我们可不希望一个出错的程序能读取，或者更糟，能覆盖别的程序的内存，对吧？

**学生**：大概不希望吧。除非那个程序是你讨厌的人写的。

**教授**：嗯……我觉得下学期你的课表里可能得加一门道德伦理课了。也许操作系统课没有把正确的道理传达给你。

**学生**：也许应该加。不过别忘了，教我们“对付出错的进程，操作系统的正确做法就是杀掉这个违规进程”的，可不是我！

 © 2008–23, ARPACI-DUSSEAU

# 13

# 抽象：地址空间

早年间，构建计算机系统很容易。你问为什么？因为用户期望不高。正是那些该死的用户，带着他们对“易用”“高性能”“可靠”等等的期望，才真正惹出了这么多头疼的事。下次你遇到这样的计算机用户，记得为他们惹出的所有麻烦向他们道声谢。

## 13.1 早期系统

从内存的角度看，早期的机器并没有为用户提供多少抽象（abstraction）。基本上，机器的物理内存（physical memory）看起来就像图 13.1（第 108 页）那样。

操作系统是一组例程（其实就是一个库），放在内存里，本例中从物理地址（physical address）0 开始；内存里还有一个正在运行的程序，也就是一个进程（process），它直接放在物理内存中（本例中从物理地址 64k 开始），用掉其余所有内存。这里几乎没有什么假象，用户对操作系统也没什么期望。那时候操作系统开发者的日子可真轻松，不是吗？

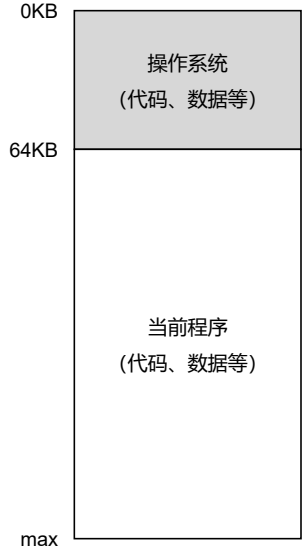



图 13.1：**操作系统：早期**

## 13.2 多道程序设计和分时

过了一段时间，由于机器很贵，人们开始更有效地共享机器。于是**多道程序设计**（multiprogramming）时代诞生了 [DV66]：在某一时刻，有多个进程准备运行，操作系统在它们之间切换，比如一个进程决定执行 I/O 时就切换。这样做提高了 CPU 的有效**利用率**（utilization）。在那个年代，每台机器都要花费几十万甚至上百万美元（你还觉得你的 Mac 贵！），所以这种**效率**（efficiency）上的提升尤为重要。

然而，没过多久，人们开始对机器提出更多要求，**分时**（time sharing）时代随之诞生 [S59, L60, M62, M83]。具体来说，许多人意识到了批处理（batch processing）计算的局限，程序员自己尤其深受其苦 [CV65]：他们受够了漫长（因而低效）的“编程—调试”循环。**交互性**（interactivity）变得重要起来，因为可能有许多用户同时使用一台机器，每个人都在等待（或者说盼望）自己当前正在执行的任务能及时给出响应。

实现分时的一种方法是：让一个进程运行一小会儿，让它完全访问所有内存（图 13.1），然后停下它，把它的全部状态保存到某种磁盘上（包括所有物理内存），再加载另一个进程的状态，让它运行一会儿，这样就实现了某种粗糙的机器共享 [M+63]。

遗憾的是，这种方法有一个大问题：它实在太慢了，内存越来越大之后尤其如此。保存和恢复寄存器级的状态（PC、通用寄存器等）相对较快，但把内存的全部内容保存到磁盘上，性能就差得惨不忍睹。因此，我们更愿意做的是，在进程之间切换时把它们留在内存里，让操作系统能高效地实现分时，如图 13.2（第 110 页）所示。

在这张图中有 A、B、C 三个进程，每个进程都从 512 KB 的物理内存中分到一小块。假设只有一个 CPU，操作系统选择运行其中一个进程，比如 A，其他进程（B 和 C）则在就绪队列（ready queue）里等待运行。

随着分时越来越流行，你大概能猜到，人们对操作系统提出了新的要求。尤其是，多个程序同时驻留在内存中，**保护**（protection）就成了一个重要问题：你可不希望一个进程能读取，或者更糟，能写入其他进程的内存。

## 13.3 地址空间

然而，我们必须把那些烦人的用户放在心上，而要做到这一点，操作系统就得为物理内存创建一个**易于使用**（easy to use）的抽象。我们把这个抽象叫作**地址空间**（address space），它是运行中的程序（running program）眼中系统内存的样子。理解操作系统对内存的这一基本抽象，是理解内存怎样虚拟化（virtualize）的关键。

进程的地址空间包含运行中的程序的全部内存状态。例如，程序的**代码**（code，即指令）必须放在内存中的某个地方，因此它们在地址空间里。程序运行时，用**栈**（stack）来记录自己在函数调用链中所处的位置，分配局部变量，以及向例程传递参数和返回值。最后，**堆**（heap）用于动态分配、由用户管理的内存，比如在 C 中调用 `malloc()`，或者在 C++、Java 这类面向对象语言中调用 `new` 所得到的内存。当然，地址空间里还有其他东西（例如静态初始化的变量），不过现在我们只假设有这三个部分：代码、栈和堆。

 © 2008–23, ARPACI-DUSSEAU




图 13.2：**三个进程共享内存**

在图 13.3（第 111 页）的例子中，我们有一个很小的地址空间，只有 16 KB[1]。程序代码位于地址空间的顶部（本例中从 0 开始，挤在地址空间的第一个 1K 里）。代码是静态的（因此很容易放进内存），所以我们可以把它放在地址空间的顶部，并且知道程序运行时它不会再需要更多空间。

接下来是地址空间里两个在程序运行时可能增长（和收缩）的区域：堆和栈。这样放，是因为二者都要能增长；把它们放在地址空间的两端，它们只要朝相反的方向增长就行了。因此，堆紧接在代码之后开始（在 1 KB 处），向下增长（比如用户通过 `malloc()` 申请更多内存时）；栈从 16 KB 处开始，向上增长（比如用户进行过程调用（procedure call）时）。不过，栈和堆的这种放法只是一种约定；只要你愿意，也可以用别的方式安排地址空间。我们后面会看到，多个**线程**（threads）共存于一个地址空间中时，就再也没有像这样划分地址空间的好办法了，唉。

当然，我们描述地址空间时，描述的是操作系统为正在运行的程序所提供的**抽象**（abstraction）。程序其实并不在物理地址 0 到 16 KB 的内存里；它被加载到某个（或某些）任意的物理地址上。看看图 13.2 中的进程 A、B 和 C，你就能看出每个进程是怎样被加载到内存中不同地址上的。于是问题来了：

[1]我们会经常使用这样的小例子，因为（a）画一个 32 位的地址空间很麻烦，而且（b）算起来更难。我们喜欢简单的算术。

 © 2008–23, ARPACI-DUSSEAU

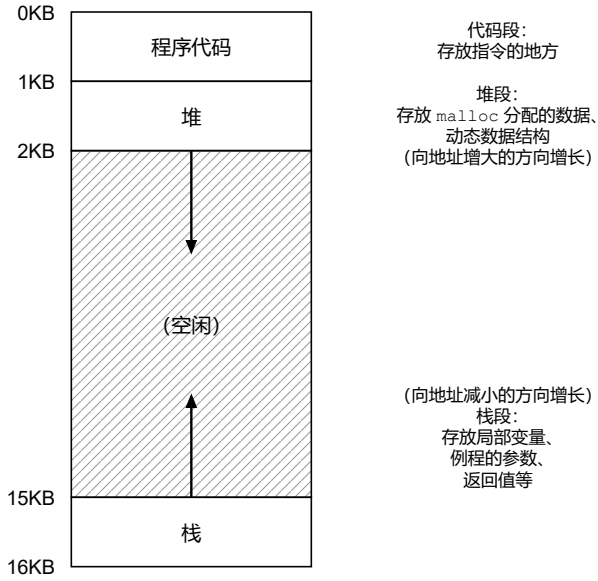



图 13.3：**一个地址空间示例**

关键问题：如何虚拟化内存
多个正在运行的进程共享同一块物理内存，操作系统怎样在它之上，为每个进程构建出这样一个私有的、可能很大的地址空间抽象？

操作系统这样做，我们就说它在**虚拟化内存**（virtualizing memory），因为运行中的程序以为自己被加载到了某个特定的地址（比如 0），并拥有一个可能非常大（比如 32 位或 64 位）的地址空间；而实际情况却大不相同。

例如，图 13.2 中的进程 A 试图在地址 0（我们称之为**虚拟地址**，virtual address）执行一次加载时，操作系统必须借助一些硬件支持，设法确保这次加载实际上不是去物理地址 0，而是去物理地址 320 KB（A 被加载到内存中的位置）。这正是内存虚拟化的关键，而世界上每一台现代计算机系统都建立在内存虚拟化之上。

## 13.4 目标

这就引出了操作系统在这一部分的任务：虚拟化内存。不过，操作系统不只要虚拟化内存，还要做得漂亮。为了确保操作系统做到这一点，我们需要一些目标来指引方向。这些目标我们以前见过（想想引言那一章），以后还会再见到，但它们确实值得反复强调。

[2] “透明”的这种用法有时会让人困惑；有些学生以为“透明”意味着把一切都公开，就像政府应该做到的那样。而这里的意思恰恰相反：操作系统提供的假象不应该被应用程序察觉到。因此，按通常的用法，一个透明的系统指的是一个难以被察觉的系统，而不是一个按照《信息自由法》的规定回应查询请求的系统。

提示：隔离原则
隔离是构建可靠系统的一条关键原则。如果两个实体彼此适当隔离，就意味着其中一个出故障不会影响另一个。操作系统努力让各个进程彼此隔离，以此防止一个进程危害另一个。有了内存隔离（memory isolation），操作系统还能进一步确保运行中的程序无法影响底层操作系统的运行。有些现代操作系统把隔离做得更彻底，把操作系统的各个部分也彼此隔开。因此，这样的**微内核**（microkernel）[BH70, R+89, S+03] 可能比典型的宏内核（monolithic kernel）设计更可靠。

虚拟内存（VM）系统的一个主要目标是**透明**（transparency）[2]。操作系统实现虚拟内存的方式应该让运行中的程序察觉不到。因此，程序不应该意识到内存被虚拟化了；相反，程序的表现就好像它拥有自己私有的物理内存一样。在幕后，操作系统（和硬件）完成所有工作，在许多不同的作业之间复用内存，从而实现这个假象。

虚拟内存的另一个目标是**效率**（efficiency）。操作系统应该努力让虚拟化尽可能**高效**，无论是在时间上（即不让程序运行得慢很多），还是在空间上（即不为支持虚拟化所需的数据结构（data structure）占用太多内存）。要在时间上做到高效，操作系统就得依赖硬件支持，包括 TLB 这样的硬件特性（我们会在适当的时候学到它们）。

最后，虚拟内存的第三个目标是**保护**（protection）。操作系统应该**保护**各个进程不受彼此影响，也保护操作系统本身不受进程影响。进程执行加载、存储或取指令时，不应该能以任何方式访问或影响任何其他进程或操作系统本身的内存内容（也就是说，不能访问或影响其地址空间之外的任何东西）。因此，保护（protection）让我们能在进程之间实现**隔离**（isolation）这一特性；每个进程都应该运行在自己与世隔绝的茧里，不受其他出错甚至恶意进程的侵害。

在接下来的几章里，我们将重点探讨虚拟化内存所需的基本**机制**（mechanisms），包括硬件和操作系统的支持。我们还会研究操作系统中会遇到的一些比较重要的**策略**（policies），包括如何管理空闲空间（free space），以及空间不足时把哪些页（page）踢出内存。在这个过程中，我们会一步步帮你弄清现代虚拟内存系统实际上是怎样工作的[3]。

## 13.5 小结

我们已经介绍了操作系统的一个主要子系统：虚拟内存。虚拟内存系统负责给每个运行中的程序营造一种假象，让它以为自己拥有一个大的、稀疏的、私有的地址空间；每个虚拟地址空间（virtual address space）包含程序的全部指令和数据，程序可以通过虚拟地址引用它们。操作系统在硬件的大力帮助下，会把每一次这样的虚拟内存引用转换成物理地址，再交给物理内存，以取出或更新所需的信息。操作系统会同时为许多进程提供这项服务，确保程序之间互相保护，也保护操作系统本身。整个方法需要大量的机制，还需要一些关键的策略才能运转；我们将自底向上，先描述关键的机制。那么，我们开始吧！

[3]或者，我们会说服你退掉这门课。不过先别急；只要你能撑过虚拟内存这一部分，你很可能就能一路撑到最后！

 © 2008–23, ARPACI-DUSSEAU

补充：你看到的每个地址都是虚拟的

写过打印指针的 C 程序吗？你看到的那个值（某个很大的数，通常以十六进制打印）就是一个**虚拟地址**（virtual address）。想知道你程序的代码放在哪里吗？你也可以把它打印出来，没错，只要能打印出来，它也是一个虚拟地址。事实上，作为用户级程序的程序员，你能看到的任何地址都是虚拟地址。只有操作系统凭着虚拟化内存的那些巧妙技术，才知道这些指令和数据究竟位于机器物理内存的什么地方。所以千万别忘了：如果你在程序里打印一个地址，它是一个虚拟地址，是内存布局的一种假象；只有操作系统（和硬件）才知道真相。

下面这个小程序（`va.c`）打印出 `main()` 例程的位置（代码所在的地方）、`malloc()` 返回的堆分配值，以及栈上一个整数的位置：

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main(int argc, char *argv[]) {
4   printf("location of code : %p\n", main);
5   printf("location of heap : %p\n", malloc(100e6));
6   int x = 3;
7   printf("location of stack: %p\n", &x);
8   return x;
9 }
```

在 64 位的 Mac 上运行，得到如下输出：

```
location of code : 0x1095afe50
location of heap : 0x1096008c0
location of stack: 0x7fff691aea64
```

从中可以看出，代码位于地址空间的最前面，接着是堆，而栈则在这个巨大虚拟空间的另一端。所有这些地址都是虚拟的，操作系统和硬件会把它们转换成真实的物理位置，以便取出相应的值。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[BH70] "The Nucleus of a Multiprogramming System" by Per Brinch Hansen. Communications of the ACM, 13:4, April 1970. *The first paper to suggest that the OS, or kernel, should be a minimal and flexible substrate for building customized operating systems; this theme is revisited throughout OS research history.*

[CV65] "Introduction and Overview of the Multics System" by F. J. Corbato, V. A. Vyssotsky. Fall Joint Computer Conference, 1965. *A great early Multics paper. Here is the great quote about time sharing: "The impetus for time-sharing first arose from professional programmers because of their constant frustration in debugging programs at batch processing installations. Thus, the original goal was to time-share computers to allow simultaneous access by several persons while giving to each of them the illusion of having the whole machine at his disposal."*

[DV66] "Programming Semantics for Multiprogrammed Computations" by Jack B. Dennis, Earl C. Van Horn. Communications of the ACM, Volume 9, Number 3, March 1966. *An early paper (but not the first) on multiprogramming.*

[L60] "Man-Computer Symbiosis" by J. C. R. Licklider. IRE Transactions on Human Factors in Electronics, HFE-1:1, March 1960. *A funky paper about how computers and people are going to enter into a symbiotic age; clearly well ahead of its time but a fascinating read nonetheless.*

[M62] "Time-Sharing Computer Systems" by J. McCarthy. Management and the Computer of the Future, MIT Press, Cambridge, MA, 1962. *Probably McCarthy's earliest recorded paper on time sharing. In another paper [M83], he claims to have been thinking of the idea since 1957. McCarthy left the systems area and went on to become a giant in Artificial Intelligence at Stanford, including the creation of the LISP programming language. See McCarthy's home page for more info:* http://www-formal.stanford.edu/jmc/

[M+63] "A Time-Sharing Debugging System for a Small Computer" by J. McCarthy, S. Boilen, E. Fredkin, J. C. R. Licklider. AFIPS '63 (Spring), New York, NY, May 1963. *A great early example of a system that swapped program memory to the "drum" when the program wasn't running, and then back into "core" memory when it was about to be run.*

[M83] "Reminiscences on the History of Time Sharing" by John McCarthy. 1983. Available: http://www-formal.stanford.edu/jmc/history/timesharing/timesharing.html. *A terrific historical note on where the idea of time-sharing might have come from including some doubts towards those who cite Strachey's work [S59] as the pioneering work in this area.*

[NS07] "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation" by N. Nethercote, J. Seward. PLDI 2007, San Diego, California, June 2007. *Valgrind is a lifesaver of a program for those who use unsafe languages like C. Read this paper to learn about its very cool binary instrumentation techniques – it's really quite impressive.*

[R+89] "Mach: A System Software kernel" by R. Rashid, D. Julin, D. Orr, R. Sanzi, R. Baron, A. Forin, D. Golub, M. Jones. COMPCON '89, February 1989. *Although not the first project on microkernels per se, the Mach project at CMU was well-known and influential; it still lives today deep in the bowels of Mac OS X.*

[S59] "Time Sharing in Large Fast Computers" by C. Strachey. Proceedings of the International Conference on Information Processing, UNESCO, June 1959. *One of the earliest references on time sharing.*

[S+03] "Improving the Reliability of Commodity Operating Systems" by M. M. Swift, B. N. Bershad, H. M. Levy. SOSP '03. *The first paper to show how microkernel-like thinking can improve operating system reliability.*

 © 2008–23, ARPACI-DUSSEAU

## 作业（编程）

在这个作业中，我们只是学习几个有用的工具，用来查看基于 Linux 的系统上的虚拟内存使用情况。这只是简单提示一下能做些什么；要真正成为专家，你得自己深入钻研（一如既往！）。

## 问题

1. 你首先应该了解的 Linux 工具是非常简单的 `free`。先输入 `man free`，把它的整个手册页（manual page）读一遍；它很短，别担心！
2. 现在运行 `free`，也许可以加上一些有用的参数（例如 `-m`，以 MB 为单位显示内存总量）。你的系统有多少内存？有多少是空闲的？这些数字和你的直觉相符吗？
3. 接下来，编写一个使用一定数量内存的小程序，叫作 `memory-user.c`。这个程序应该接受一个命令行参数：它要使用的内存大小（以 MB 为单位）。运行时，它应该分配一个数组，并不停地遍历这个数组，访问其中的每一项。程序应该无限期地这样做，或者也可以运行一段在命令行中指定的时间。
4. 现在，在运行 `memory-user` 程序的同时（在同一台机器的另一个终端窗口里）运行 `free` 工具。程序运行时，内存使用总量有什么变化？杀掉 `memory-user` 程序之后呢？这些数字和你的预期相符吗？用不同的内存使用量试一试。使用非常大的内存时会发生什么？
5. 我们再试一个工具，叫作 `pmap`。花点时间，仔细读一读 `pmap` 的手册页。
6. 要使用 `pmap`，你得知道你感兴趣的进程的**进程 ID**（process ID）。因此，先运行 `ps auxw`，查看所有进程的列表；然后挑一个有意思的，比如浏览器。这里也可以用你的 `memory-user` 程序（事实上，为了方便，你甚至可以让这个程序调用 `getpid()`，把自己的 PID 打印出来）。
7. 现在对其中一些进程运行 `pmap`，使用各种选项（比如 `-X`）来查看进程的许多细节。你看到了什么？与我们对代码/栈/堆的简单设想相比，一个现代的地址空间由多少个不同的部分组成？
8. 最后，对使用不同内存量的 `memory-user` 程序运行 `pmap`。你看到了什么？`pmap` 的输出和你的预期相符吗？

 © 2008–23, ARPACI-DUSSEAU

# 14

# 插叙：内存 API

在这个插叙中，我们讨论 UNIX 系统中的内存分配接口。这些接口非常简单，所以本章也短小精悍[1]。我们要解决的主要问题是：

> 关键问题：如何分配和管理内存
> 在 UNIX/C 程序中，理解如何分配和管理内存，是构建健壮、可靠软件的关键。常用的接口有哪些？应该避免哪些错误？

## 14.1 内存类型

C 程序运行时会分配两种内存。第一种叫作**栈**（stack）内存，它的分配和释放由编译器替你（程序员）隐式管理，所以有时也叫**自动**（automatic）内存。

在 C 语言里，要在栈上分配内存很容易。例如，假设你要在函数 `func()` 中为一个整数 `x` 分配一些空间，只需这样写：

```
void func() {
  int x; // 在栈上声明一个整数
  ...
}
```

剩下的事由编译器完成：你调用 `func()` 时，它会确保在栈上腾出空间；你从函数返回时，它会替你释放这块内存。因此，如果你希望某些信息在函数调用结束后依然存在，最好别把它放在栈上。

正是因为需要长期存在的内存，才有了第二种内存：**堆**（heap）内存。它的所有分配和释放都由你（程序员）显式处理。这无疑是一份沉重的责任！当然，这也是许多 bug 的根源。不过只要你足够细心，就能正确使用这些接口，不会遇到太多麻烦。下面这个例子在堆上分配了一个整数：

```
void func() {
  int *x = (int *) malloc(sizeof(int));
  ...
}
```

[1]说真的，我们希望每一章都是这样！只不过我们觉得这一章更短、更切中要害。

这段小代码有几点值得说明。首先，你可能注意到了，这一行里既有栈分配，也有堆分配：编译器看到你声明了指针（int *x），就知道要为一个整数指针腾出空间；随后，程序调用 malloc()，在堆上为一个整数申请空间；这个例程返回该整数的地址（成功时如此，失败时返回 NULL），地址再存放到栈上，供程序使用。

堆内存要显式管理，用法也更多样，因此无论对用户还是对系统，它都带来了更多挑战。本章接下来就重点讨论堆内存。

## 14.2 malloc() 调用

**malloc()** 调用非常简单：你传给它一个大小，请它在堆上分配这么多空间；它要么成功，返回指向新分配空间的指针，要么失败，返回 NULL[2]。

查看手册页（manual page）就能知道使用 malloc 需要做些什么；在命令行输入 man malloc，你会看到：

```
#include <stdlib.h>
...
void *malloc(size_t size);
```

从这些信息可以看出，要使用 malloc，你只需包含头文件 stdlib.h。实际上，你甚至都不需要这样做，因为所有 C 程序默认都会链接 C 库，而 C 库里就有 malloc() 的代码；加上这个头文件，只是让编译器能检查你调用 malloc() 的方式是否正确（例如，传入的参数个数和类型是否正确）。

malloc() 只有一个参数，类型是 size_t，表示你需要多少字节。不过，大多数程序员不会在这里直接写数字（比如 10）；实际上，这算不上好的写法。通常的做法是借助各种例程和宏。例如，要为一个双精度浮点数分配空间，只需这样写：

```
double *d = (double *) malloc(sizeof(double));
```

哇，好多 double 啊！这次调用 malloc() 时用了 sizeof() 运算符来申请大小合适的空间。在 C 中，sizeof() 通常被看作编译时运算符，也就是说，实际大小在编译时就已经知道，传给 malloc() 的参数会直接换成一个数字（这里是 8，即 double 的大小）。正因如此，应该把 sizeof() 看作运算符，而不是函数调用（函数调用发生在运行时）。

你也可以把变量名（而不只是类型）传给 sizeof()，但在某些情况下可能得不到想要的结果，所以要小心。例如，看看下面这段代码：

> **提示：拿不准时，就试一试**
> 如果你拿不准正在用的某个例程或运算符到底如何工作，最好的办法就是亲手试一试，确认它的行为符合预期。读手册页或其他文档固然有用，但真正重要的是它实际怎样工作。写点代码测一测！这无疑是确保代码按你期望运行的最好办法。事实上，为了核实我们关于 sizeof() 的说法确实没错，我们就是这么做的！

[2]注意，C 中的 NULL 其实没什么特别的，通常只是一个表示零的宏，例如 #define NULL 0，有时是 #define NULL (void *)0。

```
int *x = malloc(10 * sizeof(int));
printf("%d\n", sizeof(x));
```

第一行为一个包含 10 个整数的数组申请了空间，这没什么问题。但在下一行使用 `sizeof()` 时，它返回的是一个很小的值，比如 4（32 位机器）或 8（64 位机器）。这是因为在这里，`sizeof()` 以为我们只是在问一个整数指针有多大，而不是我们动态分配了多少内存。不过，有时 `sizeof()` 的结果确实和你预想的一样：

```
int x[10];
printf("%d\n", sizeof(x));
```

这时，编译器有足够的静态信息，知道分配了 40 个字节。

另一个需要小心的地方是字符串。为字符串申请空间时，请使用这样的惯用写法：`malloc(strlen(s) + 1)`，它用函数 `strlen()` 获取字符串的长度，再加上 1，为字符串结束符留出空间。在这里使用 `sizeof()` 可能会带来麻烦。

你可能还注意到，`malloc()` 返回的是 `void` 类型的指针。这只是 C 语言传回地址的一种方式：把地址交给程序员，由程序员决定怎么用。程序员还会再帮一把，用上所谓的**强制类型转换**（cast）；在上面的例子中，程序员把 `malloc()` 的返回类型转换成了指向 `double` 的指针。强制类型转换其实没起什么实际作用，只是告诉编译器和其他可能在读你代码的程序员：“没错，我知道自己在干什么。”对 `malloc()` 的结果做强制类型转换，只是让人放心一点；程序的正确性并不需要它。

## 14.3 `free()` 调用

其实，分配内存是整件事里容易的部分；什么时候释放、怎么释放，甚至要不要释放，才是难点。要释放不再使用的堆内存，程序员只需调用 **`free()`**：

```
int *x = malloc(10 * sizeof(int));
...
free(x);
```

这个例程只接受一个参数，即 `malloc()` 返回的指针。你可能注意到了，用户并不传入所分配区域的大小，这个大小必须由内存分配库自己记录。

## 14.4 常见错误

使用 `malloc()` 和 `free()` 时，有不少常见错误。下面这些，是我们讲授本科操作系统课程时一再见到的。这些例子全都能编译、运行，编译器一声不吭；要写出正确的 C 程序，编译通过是必要条件，但远远不够，这一点你以后会体会到（往往是吃了苦头之后）。

事实上，正确管理内存一直是个大难题，以至于许多较新的语言都支持**自动内存管理**（automatic memory management）。在这类语言中，分配内存时你会调用类似 `malloc()` 的东西（通常是 **new** 或类似的操作，用来分配一个新对象），但从来不需要调用什么来释放空间；**垃圾回收器**（garbage collector）会运行起来，找出哪些内存你已经不再引用，然后替你释放。

提示：编译通过或能运行 ≠ 程序正确
程序编译通过了（!），甚至正确运行了一次或很多次，并不说明它就是正确的。可能是许多因素凑在一起，让你以为它能正常工作，可后来某些情况一变，它就不行了。学生常见的反应是说（或者喊）“可它以前是好的啊!”，然后怪罪编译器、操作系统、硬件，甚至（恕我们直言）教授。但问题通常就出在你以为它会在的地方——你的代码里。先动手调试，再去怪别的部件吧。

### 忘记分配内存

许多例程都要求你在调用之前先分配好内存。例如，例程 `strcpy(dst, src)` 把字符串从源指针复制到目标指针。可是一不小心，你就可能这样写：

```
char *src = "hello";
char *dst;          // 糟糕！未分配
strcpy(dst, src); // 段错误，程序挂掉
```

运行这段代码，很可能会导致**段错误**（segmentation fault）[3]，这是个花哨的说法，意思是：**你这个愚蠢的程序员，内存用错了，我很生气。**

这里正确的代码应该是这样的：

```
char *src = "hello";
char *dst = (char *) malloc(strlen(src) + 1);
strcpy(dst, src); // 正常工作
```

或者，你也可以使用 `strdup()`，让生活更轻松。更多信息请阅读 `strdup` 的手册页。

### 没有分配足够的内存

与此相关的一个错误是没有分配足够的内存，有时也叫**缓冲区溢出**（buffer overflow）。在上面的例子中，常见的错误是给目标缓冲区（buffer）留的空间差一点才够。

```
char *src = "hello";
char *dst = (char *) malloc(strlen(src)); // 太小了!
strcpy(dst, src); // 正常工作
```

奇怪的是，这个程序常常看起来运行正常，这取决于 malloc 的实现方式以及其他许多细节。有时，字符串复制会越过所分配空间的末尾多写一个字节，但这可能并无害处，也许只是覆盖了一个不再使用的变量。有时，这类溢出的危害极大；事实上，系统中许多安全漏洞正是由此而来 [W06]。还有些时候，malloc 库本来就多分配了一点空间，所以你的程序其实并没有乱写别的变量的值，运行得好好的。另外一些时候，程序确实会出错崩溃。于是我们又学到了宝贵的一课：程序正确运行过一次，并不意味着它就是正确的。

[3]这个名字听起来很玄乎，但你很快就会知道，为什么这种非法内存访问叫作段错误；如果这都不能激励你读下去，还有什么能呢?

### 忘记初始化分配的内存

这种错误是：你正确地调用了 `malloc()`，却忘了往新分配的内存里填值。千万别这样！如果真忘了，程序迟早会遇到**未初始化读**（uninitialized read），也就是从堆中读到一些值未知的数据。谁知道里面是什么呢？运气好的话，读到的值（比如零）还能让程序照常工作；运气不好，读到的就是随机而有害的东西。

### 忘记释放内存

另一个常见错误叫作**内存泄漏**（memory leak），发生在你忘记释放内存的时候。对于长时间运行的应用程序或系统（比如操作系统本身），这是个大问题：内存一点点泄漏，最终会把内存耗尽，到那时就只能重启了。因此，一般来说，一块内存用完以后，你应该确保把它释放掉。注意，用带垃圾回收（garbage collection）的语言在这里也帮不上忙：只要你还持有某块内存的引用，任何垃圾回收器都不会释放它；所以即使在更现代的语言中，内存泄漏依然是个问题。

有些情况下，不调用 `free()` 似乎也说得过去。例如，你的程序生命周期很短，很快就会退出；这时进程（process）一结束，操作系统就会清理它分配的所有页（page），因此严格来说并不会发生内存泄漏。这样做确实“行得通”（参见第 121 页的补充），但这多半是个不该养成的坏习惯，所以选择这种做法时要慎重。从长远来看，作为程序员，你的目标之一是养成好习惯；其中一个习惯，就是清楚自己是怎样管理内存的，并且（在 C 这样的语言中）释放自己分配的内存块。就算不释放也能侥幸过关，最好也还是养成习惯，把你显式分配的每一个字节都释放掉。

### 在用完之前就释放内存

有时程序还没用完内存就把它释放了；这种错误叫作**悬空指针**（dangling pointer），你也猜得到，这同样不是好事。之后再使用它，可能会让程序崩溃，或者覆盖有效的内存（例如，你调用了 `free()`，随后又调用 `malloc()` 分配别的东西，而这次分配重新用上了那块被错误释放的内存）。

### 重复释放内存

程序有时还会把同一块内存释放不止一次，这叫作**双重释放**（double free）。这样做的结果是未定义的。可以想象，内存分配库可能会被搞糊涂，做出各种奇怪的事情；常见的结果是崩溃。

### 错误地调用 `free()`

我们要讨论的最后一个问题是错误地调用 `free()`。毕竟，`free()` 只希望你传给它之前从 `malloc()` 得到的某个指针。如果传入别的值，就可能（而且确实会）出坏事。因此，这种**无效释放**（invalid free）很危险，当然也应该避免。

补充：为什么进程退出后就不会泄漏内存

编写短期运行的程序时，你可能会用 `malloc()` 分配一些空间。程序运行完、马上要结束了：退出前有必要调用一大堆 `free()` 吗？不这样做似乎不对，但实际上不会有任何内存真正“丢失”。原因很简单：系统中其实有两个层次的内存管理。

第一层由操作系统负责：进程运行时，操作系统把内存分给它；进程退出（或以其他方式终止）时再收回来。第二层在每个进程内部，例如调用 `malloc()` 和 `free()` 时在堆内进行的管理。即使你没有调用 `free()`（从而泄漏了堆中的内存），程序运行结束时，操作系统也会回收这个进程的全部内存（包括代码页、栈页，以及这里涉及的堆页）。不管你地址空间（address space）中的堆是什么状态，进程结束时操作系统都会收回所有这些页，所以即使你没释放内存，也不会有内存丢失。

因此，对于短期运行的程序，泄漏内存通常不会造成什么实际问题（尽管这算不上好的写法）。但如果你写的是长时间运行的服务器，比如永不退出的 Web 服务器或数据库管理系统（database management system），泄漏的内存就是大得多的问题，最终会在应用程序耗尽内存时导致崩溃。当然，有一个程序里的内存泄漏问题还要严重得多，那就是操作系统本身。这再次说明：写内核代码的人，干的是最难的活……

### 小结

如你所见，误用内存的方式有很多。正因为内存错误太常见，人们围绕它发展出了一整套工具生态，帮你找出代码中的这类问题。去看看 **purify** [HJ92] 和 **valgrind** [SN05] 吧，两者都很擅长帮你找到内存相关问题的根源。一旦用惯了这些强大的工具，你会纳闷以前没有它们是怎么过来的。

## 14.5 底层操作系统支持

你可能注意到了，我们讨论 `malloc()` 和 `free()` 时一直没提系统调用（system call）。原因很简单：它们不是系统调用，而是库调用（library call）。malloc 库在你的虚拟地址空间（virtual address space）内管理空间，而它本身建立在一些系统调用之上，这些系统调用会进入操作系统，申请更多内存，或者把一些内存还给系统。

其中一个系统调用叫作 `brk`，用来改变程序 **break**（即 program break，常译作“程序间断点”）的位置，也就是堆末尾的位置。它接受一个参数（新 break 的地址），根据新 break 比当前 break 大还是小，相应地增大或缩小堆。另一个调用 `sbrk` 传入的是增量，其他方面用途类似。

注意，永远不要直接调用 `brk` 或 `sbrk`。它们是给内存分配库用的；你要是自己去用，很可能会出（大）乱子。还是老老实实用 `malloc()` 和 `free()` 吧。

最后，你还可以用 `mmap()` 调用从操作系统获取内存。传入合适的参数，`mmap()` 就能在你的程序中创建一个**匿名**（anonymous）内存区域。这个区域不与任何特定文件关联，而是与**交换空间**（swap space）关联；交换空间我们稍后讲虚拟内存（virtual memory）时会详细讨论。这块内存同样可以当作堆来使用和管理。更多细节请阅读 `mmap()` 的手册页。

 © 2008–25, ARPACI-DUSSEAU

## 14.6 其他调用

内存分配库还支持其他一些调用。例如，`calloc()` 分配内存后还会在返回前把它清零；这样可以避免一类错误：你以为内存已经清零，于是忘了自己初始化（参见前面讲“未初始化读（uninitialized read）”的那一段）。如果你已经为某样东西（比如数组）分配了空间，后来又要往里添加内容，例程 `realloc()` 也很有用：它会创建一块更大的新内存区域，把旧区域的内容复制过去，并返回指向新区域的指针。

## 14.7 小结

本章介绍了一些与内存分配有关的 API。和往常一样，我们只讲了基础知识，更多细节可以看其他资料，比如《C 程序设计语言》[KR88] 和 Stevens 的《UNIX 环境高级编程》[SR05]（第 7 章）。如果想看一篇很酷的现代论文，了解如何自动检测并纠正其中许多问题，请参阅 Novark 等人的论文 [N+07]；这篇论文还很好地总结了常见问题，并提出了一些发现和修复这些问题的巧妙思路。

## 参考文献

[HJ92] "Purify: Fast Detection of Memory Leaks and Access Errors" by R. Hastings, B. Joyce. USENIX Winter '92. *The paper behind the cool Purify tool, now a commercial product.*

[KR88] "The C Programming Language" by Brian Kernighan, Dennis Ritchie. Prentice-Hall 1988. *The C book, by the developers of C. Read it once, do some programming, then read it again, and then keep it near your desk or wherever you program.*

[N+07] "Exterminator: Automatically Correcting Memory Errors with High Probability" by G. Novark, E. D. Berger, B. G. Zorn. PLDI 2007, San Diego, California. *A cool paper on finding and correcting memory errors automatically, and a great overview of many common errors in C and C++ programs. An extended version of this paper is available CACM (Volume 51, Issue 12, December 2008).*

[SN05] "Using Valgrind to Detect Undefined Value Errors with Bit-precision" by J. Seward, N. Nethercote. USENIX '05. *How to use valgrind to find certain types of errors.*

[SR05] "Advanced Programming in the UNIX Environment" by W. Richard Stevens, Stephen A. Rago. Addison-Wesley, 2005. *We've said it before, we'll say it again: read this book many times and use it as a reference whenever you are in doubt. The authors are always surprised at how each time they read something in this book, they learn something new, even after many years of C programming.*

[W06] "Survey on Buffer Overflow Attacks and Countermeasures" by T. Werthman. Available: www.nds.rub.de/lehre/seminar/SS06/Werthmann_BufferOverflow.pdf. *A nice survey of buffer overflows and some of the security problems they cause. Refers to many of the famous exploits.*

## 作业（编程）

这个作业会让你熟悉一下内存分配。首先，你要写几个有 bug 的程序（好玩!）。然后，用一些工具帮你找出自己埋下的 bug。接着，你会意识到这些工具有多棒，以后就会一直用它们，从而让自己更快乐、更高效。这些工具是调试器（例如 gdb）和名为 valgrind 的内存错误检测器 [SN05]。

## 问题

1. 首先，编写一个名为 null.c 的简单程序，它创建一个整数指针，把它设为 NULL，然后尝试对它解引用。把它编译成名为 null 的可执行文件。运行这个程序时会发生什么？
2. 接下来，编译这个程序时加上符号信息（使用 -g 选项）。这样可执行文件里会包含更多信息，调试器就能拿到变量名之类更有用的信息。输入 gdb ./null，在调试器中运行这个程序，等 gdb 运行起来后，再输入 run。gdb 显示了什么？
3. 最后，对这个程序使用 valgrind 工具。我们会用 valgrind 中的 memcheck 来分析发生了什么。输入以下命令运行：valgrind --leak-check=yes ./null。运行后发生了什么？你能看懂这个工具的输出吗？
4. 编写一个简单的程序，用 malloc() 分配内存，但在退出前忘记释放。这个程序运行时会发生什么？你能用 gdb 找出它的问题吗？用 valgrind（同样带上 --leak-check=yes 选项）呢？
5. 编写一个程序，用 malloc 创建一个名为 data、大小为 100 的整数数组；然后把 data[100] 设为零。运行这个程序时会发生什么？用 valgrind 运行这个程序时会发生什么？这个程序正确吗？
6. 编写一个程序，分配一个整数数组（同上），把它释放掉，然后尝试打印数组中某个元素的值。程序能运行吗？对它使用 valgrind 时会发生什么？
7. 现在给 free 传一个奇怪的值（例如，指向你上面分配的数组中间的指针）。会发生什么？你需要借助工具来发现这类问题吗？
8. 试试其他一些内存分配接口。例如，实现一个类似向量（vector）的简单数据结构（data structure），以及用 realloc() 管理这个向量的相关例程。用一个数组存储向量的元素；用户向向量添加一项时，用 realloc() 为它分配更多空间。这样的向量性能如何？和链表（linked list）相比怎么样？用 valgrind 帮你找 bug。
9. 多花些时间，读一读 gdb 和 valgrind 的用法。熟悉自己的工具至关重要；花点时间，让自己成为 UNIX 和 C 环境下的调试高手。

# 15

# 机制：地址转换

在实现 CPU 虚拟化（virtualization）时，我们重点讨论了一种通用机制：**受限直接执行**（limited direct execution，**LDE**）。LDE 的思路很简单：大部分时候让程序直接在硬件上运行；但在某些关键时刻，比如进程（process）发起系统调用（system call）或者发生时钟中断（timer interrupt）时，要设法让操作系统介入，确保事情按“正确”的方式进行。这样，操作系统只需一点硬件支持，就能尽量不去打扰正在运行的程序，实现高效的虚拟化；同时，它又在这些关键时刻**介入**（interposing），从而保持对硬件的控制。高效和控制，是所有现代操作系统的两个主要目标。

虚拟化内存时，我们会采用类似的策略：既要提供所需的虚拟化，又要兼顾高效和控制。要高效，就得借助硬件支持。这种支持起初很简陋（比如只有几个寄存器），但会逐渐变得相当复杂，比如后面会讲到的 TLB、页表（page table）支持等。要控制，操作系统就得确保任何应用程序都只能访问自己的内存；为了让应用程序之间互不干扰、让操作系统不受应用程序破坏，这里同样需要硬件帮忙。最后，我们还希望虚拟内存（virtual memory）系统在*灵活性*上多做一点：程序应该能按自己想要的任何方式使用地址空间（address space），从而让系统更便于编程。由此，我们得到了更具体的关键问题：

关键问题：
如何高效、灵活地虚拟化内存
如何构建高效的内存虚拟化？如何提供应用程序需要的灵活性？如何控制应用程序能访问哪些内存位置，确保它的内存访问受到恰当的限制？如何高效地做到这一切？

我们要用的技术可以看作对受限直接执行（limited direct execution）这一通用方法的补充，叫作**基于硬件的地址转换**（hardware-based address translation），简称**地址转换**（address translation）。有了地址转换，硬件会转换每一次内存访问（例如取指令、加载或存储），把指令给出的**虚拟**（virtual）地址换成所需信息实际所在的**物理**（physical）地址。因此，每次内存引用时，硬件都会做一次地址转换，把应用程序的内存引用重定向（redirect）到它们在内存中的实际位置。

当然，光靠硬件还不能虚拟化内存，硬件只是提供了高效实现虚拟化的底层机制。操作系统必须在关键时刻介入，把硬件设置好，让转换正确进行；因此，操作系统必须**管理内存**

（manage memory）：记录哪些位置空闲、哪些在用，并审慎地干预，以保持对内存使用的控制。

和以前一样，这一切工作的目标都是营造一个美丽的**假象**（illusion）：程序拥有自己私有的内存，里面放着它自己的代码和数据。而在这个虚拟的现实背后，是丑陋的物理真相：CPU（一个或多个）在不同程序之间来回切换，许多程序实际上同时共享着内存。通过虚拟化，操作系统（在硬件的帮助下）把丑陋的机器现实变成了一个有用、强大又好用的抽象（abstraction）。

## 15.1 假设

我们对内存虚拟化的最初尝试会非常简单，简单得近乎可笑。笑吧，尽管笑；等你费力去弄懂 TLB、多级页表（multi-level page table）和其他各种技术奇迹的来龙去脉时，很快就轮到操作系统笑你了。不喜欢被操作系统嘲笑？那恐怕由不得你，操作系统就是这个脾气。

具体来说，我们暂时假设用户的地址空间必须连续地放在物理内存（physical memory）中。为了简单，我们还假设地址空间不会太大，确切地说，它小于物理内存的大小。最后，我们还假设每个地址空间的大小完全相同。这些假设听起来不现实也没关系，我们会一步步放宽它们，最终实现符合实际的内存虚拟化。

## 15.2 一个例子

为了更好地理解实现地址转换要做什么、为什么需要这样的机制，我们来看一个简单的例子。假设某个进程的地址空间如图 15.1 所示。我们要分析的是一小段代码：它从内存中加载一个值，加上 3，再把结果存回内存。这段代码用 C 语言写出来，大概是这样的：

```
void func() {
    int x = 3000; // 谢谢你，Perry。
    x = x + 3;    // 我们关注的这行代码
    ...
```

编译器会把这行代码翻译成汇编，大致如下（x86 汇编）。在 Linux 上可以用 `objdump`，在 Mac 上可以用 `otool` 来反汇编（disassemble）：

```
128: movl 0x0(%ebx), %eax  ;把 0+ebx 处的值加载到 eax
132: addl $0x03, %eax      ;eax 寄存器加 3
135: movl %eax, 0x0(%ebx)  ;把 eax 存回内存
```

这段代码比较直观。它假设 `x` 的地址已经存放在寄存器 `ebx` 中，然后用 `movl` 指令（意为“longword” move，即长字传送）把该地址上的值加载到通用寄存器 `eax`。下一条指令把 `eax` 加 3，最后一条指令把 `eax` 中的值存回内存中原来的位置。

从图 15.1（第 127 页）可以看到代码和数据在进程地址空间中的布局：这三条指令位于地址 128，在靠近顶部的代码段中；变量 `x` 的值位于地址 15 KB，在靠近底部的栈（stack）中。图中栈上 `x` 所在的位置标出了它的初始值 3000。

从进程的角度看，这几条指令运行时会发生以下内存访问：

- 从地址 128 取指令

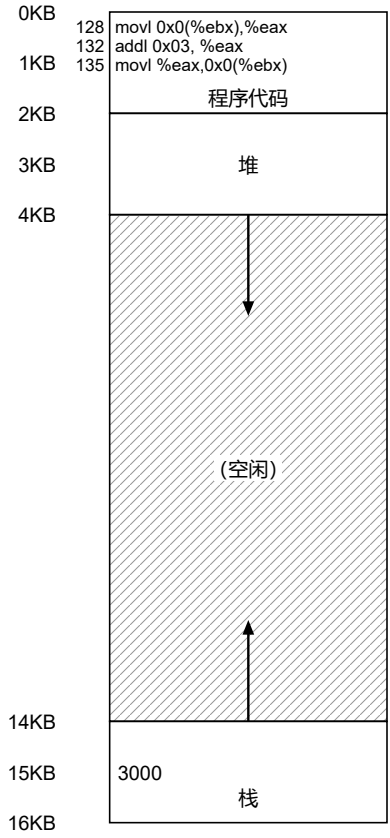



图 15.1：**一个进程及其地址空间**

- 执行这条指令（从地址 15 KB 加载）
- 从地址 132 取指令
- 执行这条指令（不访问内存）
- 从地址 135 取指令
- 执行这条指令（存储到地址 15 KB）

提示：介入很强大

介入（interposition）是一种通用且强大的技术，在计算机系统中常常能收到很好的效果。在虚拟化内存时，硬件会介入每一次内存访问，把进程发出的每个虚拟地址（virtual address）转换成所需信息实际存放的物理地址（physical address）。不过，介入这种技术的适用范围远不止于此；事实上，几乎在任何定义良好的接口上都可以介入，用来增加新功能，或者改进系统的其他方面。这类做法的一个常见好处是**透明**（transparency）：介入通常不需要改动客户端的接口，因此客户端本身也无需任何修改。

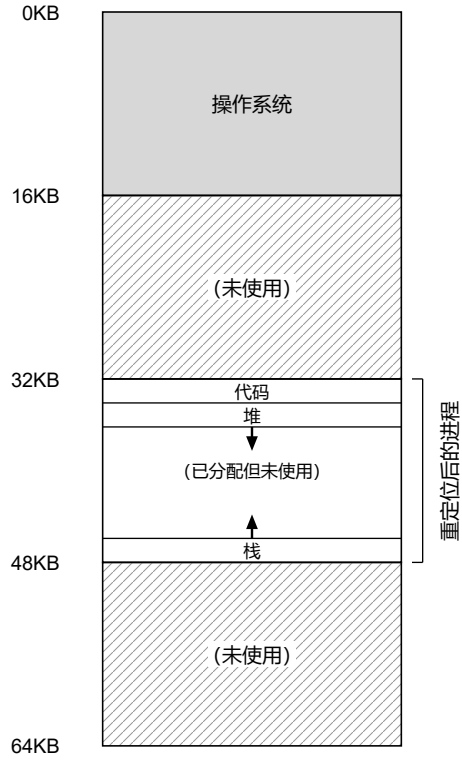



图 15.2：**只有一个重定位进程的物理内存**

从程序的角度看，它的**地址空间**（address space）从地址 0 开始，最大到 16 KB；它产生的所有内存引用都应该落在这个范围内。然而，为了虚拟化内存，操作系统希望把这个进程放到物理内存的其他地方，不一定从地址 0 开始。于是问题来了：怎样才能在内存中**重定位**（relocate）这个进程，同时又对进程**透明**（transparent）？地址空间实际位于另外某个物理地址，怎样才能营造出虚拟地址空间（virtual address space）从 0 开始的假象？

图 15.2 展示了这个进程的地址空间放进内存后，物理内存可能的样子。可以看到，操作系统把物理内存的第一个槽位留给了自己，并把上面例子中的进程重定位到了从物理地址 32 KB 开始的槽位。另外两个槽位（16 KB–32 KB 和 48 KB–64 KB）是空闲的。

## 15.3 动态（基于硬件的）重定位

为了对基于硬件的地址转换（hardware-based address translation）有个初步了解，我们先讨论它最早的形式。20 世纪 50 年代末，最早的一批分时（time sharing）机器引入了一个简单的思想，叫作**基址加界限**（base and bounds）；这种技术也叫**动态重定位**（dynamic relocation），下文两种说法会交替使用 [SS74]。

 © 2008–23, ARPACI-DUSSEAU

补充：基于软件的重定位

早期还没有硬件支持的时候，有些系统纯粹靠软件方法，实现了一种比较粗糙的重定位。其基本技术叫作**静态重定位**（static relocation）：由一个叫**加载器**（loader）的软件拿到即将运行的可执行文件，把其中的地址改写为物理内存中期望的偏移位置。

例如，假设有一条指令从地址 1000 加载数据到寄存器（例如 `movl 1000, %eax`），而程序的地址空间是从地址 3000 开始加载的（而不是程序以为的 0），那么加载器就会改写这条指令，给每个地址都加上偏移量（offset）3000（例如改成 `movl 4000, %eax`）。这样就实现了对进程地址空间的简单静态重定位（static relocation）。

然而，静态重定位（static relocation）有不少问题。首先也是最重要的一点：它不提供保护。进程可能产生错误的地址，从而非法访问其他进程甚至操作系统的内存；一般来说，要实现真正的保护，很可能离不开硬件支持 [WL+93]。另一个缺点是，地址空间一旦放好，之后就很难再重定位到别的位置 [M65]。

具体来说，每个 CPU 内需要两个硬件寄存器：一个叫**基址**（base）寄存器，另一个叫**界限**（bounds）寄存器（有时也叫**限制**（limit）寄存器）。有了这对基址和界限寄存器（base and bounds registers），我们就能把地址空间放到物理内存中任何想放的位置，同时确保进程只能访问它自己的地址空间。

在这种方案下，每个程序在编写和编译时，都假定自己是从地址零开始加载的。但程序开始运行时，操作系统会决定把它加载到物理内存的什么位置，并把基址寄存器（base register）设为这个地址。在上面的例子中，操作系统决定把进程加载到物理地址 32 KB，于是就把基址寄存器设为这个值。

进程运行起来以后，有意思的事情就开始了。此时，进程产生的任何内存引用，都会被处理器按以下方式**转换**（translated）：

**物理地址 = 虚拟地址 + 基址**

进程产生的每个内存引用都是一个**虚拟地址**（virtual address）；硬件把基址寄存器的内容加到这个地址上，得到的就是可以发给内存系统的**物理地址**（physical address）。

为了更好地理解这一点，我们来跟踪一条指令执行时的情况。具体来说，就看前面那段指令序列中的这一条：

```
128: movl 0x0(%ebx), %eax
```

程序计数器（PC）现在是 128。硬件需要取这条指令时，先把这个值加上基址寄存器的值 32 KB（32768），得到物理地址 32896，然后从这个物理地址取出指令。接着，处理器开始执行这条指令。执行过程中，进程会从虚拟地址 15 KB 加载数据，处理器拿到这个地址后，同样加上基址寄存器的值（32 KB），得到最终的物理地址 47 KB，从而取到所需的内容。

把虚拟地址转换为物理地址，正是我们所说的**地址转换**（address translation）技术：硬件拿到进程以为自己正在引用的虚拟地址，把它转换为数据实际所在的物理地址。由于这种

提示：基于硬件的动态重定位
有了动态重定位（dynamic relocation），一点点硬件就能起到很大作用。具体来说，**基址**（base）寄存器用来把（程序产生的）虚拟地址转换为物理地址，**界限**（bounds）或**限制**（limit）寄存器则确保这些地址不超出地址空间的范围。两者合在一起，就提供了一种简单而高效的内存虚拟化。

地址重定位发生在运行时，而且即使进程已经开始运行，我们也可以移动它的地址空间，所以这种技术通常称为**动态重定位**（dynamic relocation）[M65]。

你现在可能会问：那个界限寄存器（bounds register，也叫限制寄存器）去哪了？这不是基址（base）加界限的方法吗？没错，确实是。你可能已经猜到，界限寄存器是用来帮助实现保护的。具体来说，处理器会先检查内存引用是否在界限之内，以确保它合法；在上面的简单例子中，界限寄存器始终设为 16 KB。如果进程产生的虚拟地址大于（或等于）界限，或者是负数，CPU 就会触发异常（exception），这个进程多半会被终止。所以，界限的作用就是确保进程产生的所有地址都合法，都在进程的“界限”之内——这你大概也猜到了。

需要指出的是，基址和界限寄存器（base and bounds registers）是位于芯片上的硬件结构（每个 CPU 一对）。人们有时把处理器中负责地址转换的这部分称为**内存管理单元**（memory management unit，MMU）；随着我们开发出更复杂的内存管理技术，还会往 MMU 里加入更多电路。

顺便说一下，界限寄存器有两种定义方式。一种（如上所述）是保存地址空间的大小，硬件先用它检查虚拟地址，再加上基址（base）。另一种是保存地址空间末尾的*物理地址*（physical address），硬件先加上基址（base），再检查地址是否在界限之内。两种方式在逻辑上是等价的；为简单起见，我们通常假设采用前一种。

### 地址转换示例

为了更详细地了解基址加界限的地址转换，我们来看一个例子。假设有一个进程，它的地址空间大小为 4 KB（没错，小得不现实），被加载到了物理地址 16 KB 处。下面是几次地址转换的结果：

| 虚拟地址 | | 物理地址 |
|---|---|---|
| 0 | → | 16 KB |
| 1 KB | → | 17 KB |
| 3000 | → | 19384 |
| 4400 | → | 故障（越界） |

从这个例子可以看出，只要把基址（base）加到虚拟地址上，就能得到相应的物理地址；虚拟地址完全可以看作地址空间内的*偏移量*（offset）。只有当虚拟地址“太大”或为负数时，才会出现故障（fault），并触发异常。

## 15.4 硬件支持：总结

现在我们来总结一下需要硬件提供哪些支持（另见第 131 页的图 15.3）。首先，正如 CPU 虚拟化一章讨论过的，我们需要两种不同的 CPU 模式。操作系统运行在**特权模式**（privileged mode，也称**内核模式**，kernel mode）下，可以访问整台机器；应用程序运行在**用户模式**（user mode）下，能做的事情受到限制。CPU 当前处于哪种模式，可以用一个位来表示，这个位也许就存放在某种**处理器状态字**（processor status word）中；在某些特殊情况下，例如发生系统调用或其他某种异常、中断（interrupt）时，CPU 会切换模式。

硬件还必须提供**基址和界限寄存器**（base and bounds registers）本身；因此，每个 CPU 都多了一对寄存器，它们属于 CPU 的**内存管理单元**（MMU）。用户程序运行时，硬件会把基址（base）加到用户程序产生的虚拟地址上，以此转换每个地址。硬件还必须能检查地址是否有效，这要靠界限寄存器和 CPU 内部的一些电路来完成。

硬件还应该提供专门的指令来修改基址和界限寄存器（base and bounds registers），让操作系统在运行不同进程时更改它们。这些指令属于**特权**（privileged）指令，只有在内核（或特权）模式（kernel/privileged mode）下才能修改这些寄存器。试想一下，如果用户进程能在运行时随意修改基址寄存器，它能造成多大的破坏[1]。想想看！然后马上把这种阴暗的念头从脑子里赶出去，噩梦就是这种可怕的东西做成的。

最后，用户程序试图非法访问内存（使用“越界”地址）时，CPU 必须能够产生**异常**（exception）。这时，CPU 应该停止执行用户程序，转而让操作系统的“越界”**异常处理程序**（exception handler）运行。操作系统的处理程序随后决定如何应对，在这种情况下多半会终止该进程。同样，如果用户程序试图修改（特权的）基址和界限寄存器（base and bounds

> 补充：数据结构——空闲链表
> 操作系统必须记录物理内存中哪些部分没有被使用，这样才能给进程分配内存。能完成这项任务的数据结构（data structure）当然有很多种；最简单的一种（我们这里就假设用它）是**空闲链表**（free list），它就是一个链表（linked list），记录物理内存中当前没有被使用的各段范围。

| **硬件要求** | **说明** |
|---|---|
| 特权模式 | 防止用户模式的进程执行特权操作 |
| 基址/界限寄存器 | 每个 *CPU* 需要一对寄存器，用于支持地址转换和界限检查 |
| 能够转换虚拟地址并检查是否在界限内 | 负责转换和检查界限的电路；这里的实现非常简单 |
| 用于更新基址/界限的特权指令 | 操作系统必须能在用户程序运行前设置这些值 |
| 用于注册异常处理程序的特权指令 | 操作系统必须能告诉硬件发生异常时运行哪段代码 |
| 能够触发异常 | 进程试图执行特权指令或访问越界内存时 |

图 15.3：**动态重定位：硬件要求**

[1] wreak 后面除了接 havoc（破坏），还能接别的词吗？[W17]

| 操作系统要求 | 说明 |
| --- | --- |
| 内存管理 | 需要为新进程分配内存；<br>回收已终止进程的内存；<br>一般通过**空闲链表**管理内存 |
| 基址/界限管理 | 必须在上下文切换时正确设置基址/界限 |
| 异常处理 | 发生异常时要运行的代码；<br>通常会终止出错的进程 |

图 15.4：**动态重定位：操作系统的职责**

registers），CPU 也应该触发异常，并运行“试图在用户模式下执行特权操作（privileged operation）”的处理程序。CPU 还必须提供某种方法，让操作系统告诉它这些处理程序在哪里，因此还需要再加几条特权指令（privileged instruction）。

## 15.5 操作系统层面的问题

硬件为支持动态重定位（dynamic relocation）提供了新功能，操作系统也随之面临一些必须处理的新问题；硬件支持和操作系统管理结合起来，就实现了一个简单的虚拟内存。具体来说，要实现我们这个基址加界限版本的虚拟内存，操作系统必须在几个关键时刻介入。

第一，创建进程时，操作系统必须采取行动，在内存中为进程的地址空间找到位置。好在根据我们的假设，每个地址空间都（a）小于物理内存，并且（b）大小相同，所以这件事对操作系统来说很容易：它可以把物理内存看成一个槽位数组，记录每个槽位是空闲还是在用。创建新进程时，操作系统要搜索一个数据结构（通常称为**空闲链表**（free list）），为新的地址空间找到位置，然后把这个位置标记为已用。如果地址空间大小可变，情况就会复杂得多，不过这个问题我们留到后面的章节再讨论。

我们来看一个例子。从图 15.2（第 128 页）可以看到，操作系统把物理内存的第一个槽位留给了自己，并把上面例子中的进程重定位到了从物理地址 32 KB 开始的槽位。另外两个槽位（16 KB–32 KB 和 48 KB–64 KB）是空闲的，因此空闲链表中应该有这两项。

第二，进程终止时（无论是正常退出，还是因行为不端被强行杀掉），操作系统必须做一些工作，回收它的全部内存，供其他进程或操作系统使用。进程终止后，操作系统会把它的内存放回空闲链表，并视需要清理相关的数据结构。

第三，发生上下文切换（context switch）时，操作系统还必须多做几步。毕竟，每个 CPU 上只有一对基址和界限寄存器（base and bounds registers），而每个程序被加载到内存中不同的物理地址，所以每个运行中的程序（running program）对应的寄存器值都不一样。因此，操作系统在进程之间切换时，必须保存和恢复基址和界限寄存器（base and bounds registers）。具体来说，操作系统决定停止运行某个进程时，必须把基址和界限寄存器（base and bounds registers）的值保存到内存中，放在每个进程各有一份的某种结构里，比如**进程结构**（process structure）或**进程控制块**（process control block，PCB）。同样，操作系统恢复运行某个进程（或第一次运行它）时，必须把 CPU 上的基址和界限寄存器（base and bounds registers）设为这个进程对应的值。

 © 2008–23, ARPACI-DUSSEAU

| **操作系统 @ 启动（内核模式）** | **硬件** | **（还没有程序）** |
|---|---|---|
| **初始化陷阱表** | | |
| | 记住以下处理程序的地址……<br>系统调用处理程序<br>定时器处理程序<br>非法内存访问处理程序<br>非法指令处理程序 | |
| **启动中断定时器** | | |
| | 启动定时器；X 毫秒后中断 | |
| **初始化进程表** | | |
| **初始化空闲链表** | | |

图 15.5：**受限直接执行（动态重定位）@ 启动**

需要指出的是，进程停止（即没有在运行）时，操作系统可以相当容易地把它的地址空间从内存中的一个位置移到另一个位置。具体做法是：操作系统先暂停调度这个进程；然后把地址空间从当前位置复制到新位置；最后更新保存在进程结构（process structure）中的基址寄存器，让它指向新位置。等进程恢复运行时，它的（新）基址寄存器也随之恢复，进程接着运行，完全不知道自己的指令和数据已经搬到了内存中一个全新的位置。

第四，操作系统必须提供**异常处理程序**（exception handler），也就是前面说的那些需要调用的函数；操作系统在启动时（通过特权指令）安装这些处理程序。例如，如果某个进程试图访问界限之外的内存，CPU 就会触发异常，操作系统必须做好准备，在这种异常发生时采取行动。操作系统通常的反应是毫不留情：它多半会终止这个出错的进程。操作系统应当严密保护它所运行的机器，因此，哪个进程要是试图访问不该访问的内存、执行不该执行的指令，操作系统绝不会对它客气。再见了，不守规矩的进程，很高兴认识你。

图 15.5 和图 15.6（第 134 页）用时间线的形式展示了硬件和操作系统之间的大部分交互。第一张图展示操作系统启动时怎样让机器做好准备；第二张图展示的是进程 A 开始运行后发生的事情，请注意，它的内存转换完全由硬件处理，操作系统没有介入。某个时刻（第二张图的中间），发生了时钟中断，操作系统切换到进程 B，而 B 执行了一次“错误的加载”（访问了一个非法的内存地址）；这时操作系统必须介入，终止该进程并做好清理：释放 B 的内存，并从进程表（process table）中删除它的条目。从图中可以看出，我们仍然遵循**受限直接执行**（limited direct execution）的基本思路。大多数情况下，操作系统只需把硬件设置好，然后让进程直接在 CPU 上运行；只有在进程行为不端时，操作系统才需要介入。

## 15.6 小结

本章借助虚拟内存中的一种特定机制，也就是**地址转换**（address translation），扩展了受限直接执行（limited direct execution）的概念。有了地址转换，操作系统就能控制进程的每一次内存访问，确保访问不超出地址空间的界限。这项技术之所以高效，关键在于硬件支持：硬件为每次访问快速完成转换，把虚拟地址变成物理地址。这一切对被重定位

 © 2008–23, ARPACI-DUSSEAU

| **操作系统 @ 运行（内核模式）** | **硬件** | **程序（用户模式）** |
|---|---|---|
| **启动进程 A：** | | |
| 在进程表中分配条目 | | |
| 为进程分配内存 | | |
| 设置基址/界限寄存器 | | |
| **从陷阱返回**（进入 A） | | |
| | 恢复 A 的寄存器 | |
| | 切换到**用户模式** | |
| | 跳转到 A 的（初始）PC | |
| | | **进程 A 运行** |
| | | 取指令 |
| | 转换虚拟地址 | |
| | 执行取指 | |
| | | 执行指令 |
| | 如果是显式的加载/存储： | |
| | 确保地址合法 | |
| | 转换虚拟地址 | |
| | 执行加载/存储 | |
| | | （A 运行……） |
| | **时钟中断** | |
| | 切换到**内核模式** | |
| | 跳转到处理程序 | |
| **处理定时器** | | |
| 决定：停止 A，运行 B | | |
| 调用 `switch()` 例程 | | |
| 保存 A 的寄存器 | | |
| 到 A 的进程结构 | | |
| （包括基址/界限） | | |
| 恢复 B 的寄存器 | | |
| 从 B 的进程结构 | | |
| （包括基址/界限） | | |
| **从陷阱返回**（进入 B） | | |
| | 恢复 B 的寄存器 | |
| | 切换到**用户模式** | |
| | 跳转到 B 的 PC | |
| | | **进程 B 运行** |
| | | 执行错误的加载 |
| | 加载越界； | |
| | 切换到**内核模式** | |
| | 跳转到陷阱处理程序 | |
| **处理陷阱** | | |
| 决定终止进程 B | | |
| 释放 B 的内存 | | |
| 释放 B 在进程表中的条目 | | |

图 15.6：**受限直接执行（动态重定位）@ 运行时**

 © 2008–23, ARPACI-DUSSEAU

（relocated）的进程都是透明的：进程完全不知道自己的内存引用正在被转换，于是形成了一个美妙的假象。

我们还介绍了一种具体的虚拟化方式，叫作基址加界限，也叫动态重定位（dynamic relocation）。基址加界限的虚拟化相当高效，因为只需多一点硬件逻辑，把基址寄存器加到虚拟地址上，再检查进程产生的地址是否在界限之内就行了。基址加界限还提供了保护（protection）：操作系统和硬件联手，确保任何进程都无法产生超出自身地址空间的内存引用。保护（protection）无疑是操作系统最重要的目标之一；没有保护，操作系统就无法控制机器：如果进程可以随意覆盖内存，它们很容易干出覆盖陷阱表（trap table）、进而接管整个系统这类坏事。

遗憾的是，这种简单的动态重定位（dynamic relocation）技术确实存在效率问题。例如，从图 15.2（第 128 页）可以看到，被重定位（relocated）的进程占用了从 32 KB 到 48 KB 的物理内存；但由于进程的栈和堆（heap）都不大，两者之间的空间全都白白浪费了。这种浪费通常称为**内部碎片**（internal fragmentation），因为已分配单元内部的空间没有被充分利用（即形成了碎片），造成了浪费。按我们目前的做法，即使物理内存足够容纳更多进程，我们也只能把地址空间放进固定大小的槽位里，所以可能产生内部碎片[2]。因此，我们需要更精巧的机制，以便更好地利用物理内存、避免内部碎片。我们首先要尝试的，是对基址加界限稍作推广，也就是**分段**（segmentation），下一章就来讨论它。

[2]另一种办法是在地址空间里放一个固定大小的栈，紧挨在代码区下方，再在栈下方放一个可以增长的堆。但这样会限制灵活性，让递归和深层嵌套的函数调用变得困难，所以我们希望避免这种做法。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[M65] "On Dynamic Program Relocation" by W.C. McGee. IBM Systems Journal, Volume 4:3, 1965, pages 184–199. *This paper is a nice summary of early work on dynamic relocation, as well as some basics on static relocation.*

[P90] "Relocating loader for MS-DOS .EXE executable files" by Kenneth D. A. Pillay. Microprocessors & Microsystems archive, Volume 14:7 (September 1990). *An example of a relocating loader for MS-DOS. Not the first one, but just a relatively modern example of how such a system works.*

[SS74] "The Protection of Information in Computer Systems" by J. Saltzer and M. Schroeder. CACM, July 1974. *From this paper: "The concepts of base-and-bound register and hardware-interpreted descriptors appeared, apparently independently, between 1957 and 1959 on three projects with diverse goals. At M.I.T., McCarthy suggested the base-and-bound idea as part of the memory protection system necessary to make time-sharing feasible. IBM independently developed the base-and-bound register as a mechanism to permit reliable multiprogramming of the Stretch (7030) computer system. At Burroughs, R. Barton suggested that hardware-interpreted descriptors would provide direct support for the naming scope rules of higher level languages in the B5000 computer system." We found this quote on Mark Smotherman's cool history pages [S04]; see them for more information.*

[S04] "System Call Support" by Mark Smotherman. May 2004. people.cs.clemson.edu/~mark/syscall.html. *A neat history of system call support. Smotherman has also collected some early history on items like interrupts and other fun aspects of computing history. See his web pages for more details.*

[WL+93] "Efficient Software-based Fault Isolation" by Robert Wahbe, Steven Lucco, Thomas E. Anderson, Susan L. Graham. SOSP '93. *A terrific paper about how you can use compiler support to bound memory references from a program, without hardware support. The paper sparked renewed interest in software techniques for isolation of memory references.*

[W17] Answer to footnote: "Is there anything other than havoc that can be wreaked?" by Waciuma Wanjohi. October 2017. *Amazingly, this enterprising reader found the answer via google's Ngram viewing tool (available at the following URL: http://books.google.com/ngrams). The answer, thanks to Mr. Wanjohi: "It's only since about 1970 that 'wreak havoc' has been more popular than 'wreak vengeance'. In the 1800s, the word wreak was almost always followed by 'his/their vengeance'." Apparently, when you wreak, you are up to no good, but at least wreakers have some options now.*

## 作业（模拟）

程序 `relocation.py` 可以让你观察：在带有基址和界限寄存器（base and bounds registers）的系统中，地址转换是如何进行的。详情请见 README。

## 问题

1. 分别用种子 1、2、3 运行，算出进程产生的每个虚拟地址是在界限内还是越界。如果在界限内，算出转换后的地址。
2. 用这组选项运行：`-s 0 -n 10`。界限寄存器`-l` 要设成多少，才能保证产生的所有虚拟地址都在界限内?
3. 用这组选项运行：`-s 1 -n 10 -l 100`。基址（base）最大可以设成多少，才能让地址空间仍然整个放得进物理内存?
4. 把上面的一些问题再做一遍，但使用更大的地址空间和物理内存。
5. 随机生成的虚拟地址中，有多大比例是有效的？这个比例随界限寄存器的值如何变化？用不同的随机种子运行，界限值从 0 一直取到地址空间的最大值，画出一张图。

 © 2008–23, ARPACI-DUSSEAU

# 16

# 分段

到目前为止，我们一直把每个进程（process）的整个地址空间（address space）都放进内存。有了基址和界限寄存器（base and bounds registers），操作系统可以很容易地把进程重定位（relocate）到物理内存（physical memory）的不同位置。不过，你可能已经注意到我们这些地址空间有个有意思的地方：在栈（stack）和堆（heap）之间，正中间有一大块“空闲”空间。

从图 16.1 可以想见，虽然栈和堆之间的空间并没有被进程使用，但我们把整个地址空间重定位到物理内存中某处时，这块空间照样占着物理内存；因此，用一对基址和界限寄存器（base and bounds registers）来虚拟化（virtualize）内存，这种简单的做法很浪费。而且，整个地址空间放不进内存时，程序就很难运行；所以，基址加界限（base and bounds）并不像我们希望的那样灵活。于是：

关键问题：如何支持大地址空间

栈和堆之间可能有大量空闲空间（free space），这样的大地址空间该如何支持？注意，在我们的例子中，地址空间都很小（只是假想的），浪费看起来还不算严重。但想象一下 32 位的地址空间（4 GB 大小）：一个典型的程序只会用到几 MB 内存，却仍然要求整个地址空间都驻留在内存中。

## 16.1 分段：泛化的基址/界限

为了解决这个问题，人们提出了一个想法，叫作**分段**（segmentation）。这个想法相当古老，至少可以追溯到 20 世纪 60 年代初 [H61, G62]。思路很简单：与其在 MMU 中只放一对基址和界限寄存器（base and bounds registers），为什么不给地址空间的每个逻辑**段**（segment）都配一对呢？段就是地址空间中一段特定长度的连续区域。在我们一直使用的典型地址空间里，有三个逻辑上不同的段：代码、栈和堆。有了分段，操作系统就可以把这三个段分别放到物理内存的不同位置，从而避免未使用的虚拟地址空间（virtual address space）占满物理内存。

我们来看一个例子。假设我们要把图 16.1 中的地址空间放进物理内存。每个段都有一对基址和界限寄存器（base and bounds registers），所以可以把每个段独立地放进物理内

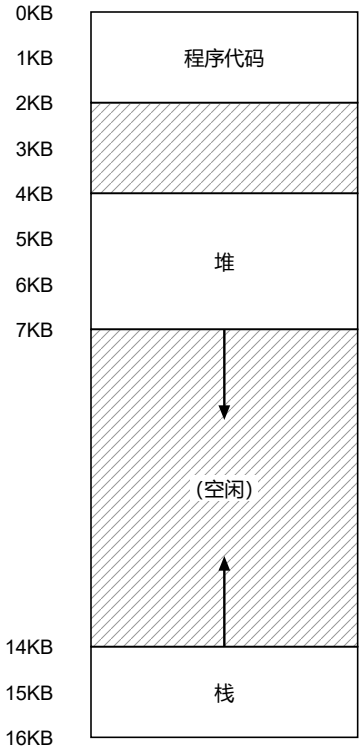



图 16.1：**地址空间（再看一次）**

存。例如，图 16.2（第 140 页）展示了一个 64 KB 的物理内存，里面放着这三个段，另有 16 KB 留给操作系统。

从图中可以看到，物理内存中只为用到的内存分配了空间。因此，即使地址空间很大、其中有大量未使用的部分（我们有时把这种地址空间称为**稀疏地址空间**，sparse address space），也能容纳下来。

MMU 中支持分段所需的硬件结构正如你所料：在这个例子里，就是三对基址和界限寄存器（base and bounds registers）。下面的图 16.3 给出了上例中的寄存器值；每个界限寄存器（bounds register）保存的是一个段的大小。

从图中可以看到，代码段（code segment）放在物理地址（physical address）32 KB 处，大小为 2 KB；堆段（heap segment）放在 34 KB 处，大小为 3 KB。这里的大小寄存器（size register）和前面介绍的界限寄存器完全一样：它告诉硬件这个段中到底有多少字节是有效的（硬件由此就能判断程序何时越界，访问了非法地址）。

我们用图 16.1 中的地址空间做一次地址转换（address translation）。假设程序引用了虚拟地址（virtual address）100，从图 16.1（第 139 页）可以直观地看到，它位于代码段中。引用发生时（比如取指令时），硬件会把基址（base）加上这个段内的偏移量（offset，这里是 100），得到所需的物理地址：100 + 32 KB，即 32868。然后它会检查这个地址是否

 © 2008–25, ARPACI-DUSSEAU

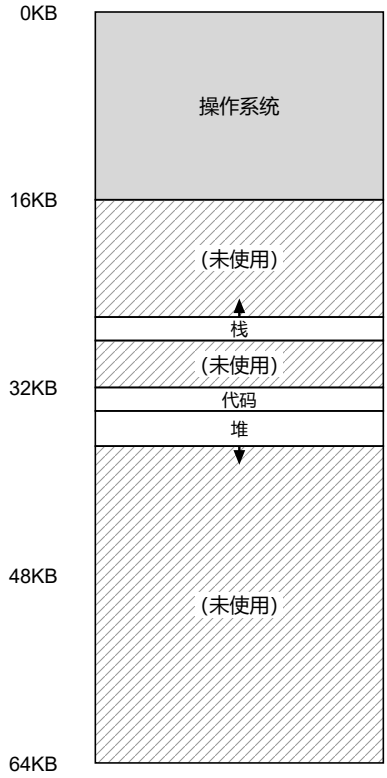



图 16.2：**把段放进物理内存**

在界限（bounds）之内（100 小于 2 KB），确认在界限内之后，就向物理内存地址 32868 发出引用。

| 段 | 基址 | 大小 |
|---|---|---|
| 代码 | 32K | 2K |
| 堆 | 34K | 3K |
| 栈 | 28K | 2K |

图 16.3：**段寄存器的值**

补充：段错误

**段错误**（segmentation fault）也叫段违规（segmentation violation），这个术语源自分段机器上的非法内存访问。好笑的是，这个术语一直沿用至今，就连完全不支持分段的机器上也在用。不过，要是你搞不清自己的代码为什么老是出错，那就不那么好笑了。

 © 2008–25, ARPACI-DUSSEAU

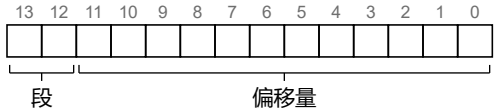
现在来看堆中的一个地址：虚拟地址 4200（同样参见图 16.1）。如果我们直接把虚拟地址 4200 加到堆的基址（base，34 KB）上，得到的物理地址是 39016，这并不是正确的物理地址。我们首先要做的，是提取出堆内的*偏移量*（offset），也就是这个地址引用的是这个段中的哪个（或哪些）字节。因为堆从虚拟地址 4 KB（4096）开始，所以 4200 的偏移量实际上是 4200 减去 4096，即 104。然后我们把这个偏移量加到基址寄存器（base register）中的物理地址上，得到想要的结果：34920。

如果我们试图引用一个非法地址（即 7 KB 或更大的虚拟地址），也就是超出堆末尾的地址，会怎样？你可以想象会发生什么：硬件检测到地址越界，陷入（trap）操作系统，很可能导致这个出错的进程被终止。现在你知道这个让所有 C 程序员都闻之色变的著名术语是怎么来的了：**段违规**（segmentation violation）或**段错误**（segmentation fault）。

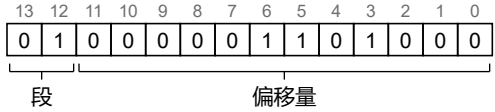
## 16.2 引用的是哪个段?

硬件在转换时会用到段寄存器（segment register）。它怎么知道一个地址引用的是哪个段，以及这个地址在段内的偏移量呢?

一种常见的方法有时称为**显式**（explicit）方法：根据虚拟地址的最高几位，把地址空间划分成段；VAX/VMS 系统就使用了这种技术 [LL82]。在上面的例子中，我们有三个段，因此需要两位来完成这项任务。如果用 14 位虚拟地址的最高两位来选择段，虚拟地址就是这个样子：




在我们的例子中，如果最高两位是 00，硬件就知道这个虚拟地址在代码段中，于是用代码段的基址（base）和界限把地址重定位到正确的物理位置。如果最高两位是 01，硬件就知道地址在堆中，于是使用堆的基址（base）和界限。我们把上面那个堆中的虚拟地址拿来转换一下，确保你理解了这一点。虚拟地址 4200 的二进制形式如下：




从图中可以看到，最高两位（01）告诉硬件我们引用的是哪个*段*（segment）。低 12 位是段内的*偏移量*（offset）：0000 0110 1000，即十六进制的 0x068，或十进制的 104。因此，硬件只需取前两位来决定使用哪个段寄存器（segment register），再把后面 12 位当作段内偏移量。把基址寄存器加上偏移量，硬件就得到了最终的物理地址。注意，有了偏移量，界限检查也变得简单：只需检查偏移量是否小于界限；如果不小于，这个地址就是非法的。因此，如果基址（base）和界限是数组（每个段一项），硬件为了得到所需的物理地址，所做的事情大致如下：

```
// 取 14 位虚拟地址的最高 2 位
Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
// 再取偏移量
Offset  = VirtualAddress & OFFSET_MASK
if (Offset >= Bounds[Segment])
    RaiseException(PROTECTION_FAULT)
else
    PhysAddr = Base[Segment] + Offset
    Register = AccessMemory(PhysAddr)
```

在我们这个贯穿始终的例子中，可以把上面的常量填上具体的值：`SEG_MASK` 设为 `0x3000`，`SEG_SHIFT` 设为 `12`，`OFFSET_MASK` 设为 `0xFFF`。

你可能还注意到，用最高两位选段，而我们只有三个段（代码、堆、栈），这样地址空间中就有一个段闲置了。为了充分利用虚拟地址空间，避免出现闲置的段，有些系统把代码和堆放在同一个段里，这样只需一位就能选出用哪个段 [LL82]。

用最高的若干位来选段，还有一个问题：它限制了虚拟地址空间的使用。具体来说，每个段的大小都有上限，在我们的例子中是 4 KB（用最高两位选段，意味着 16 KB 的地址空间被切成四块，在这个例子里每块 4 KB）。如果正在运行的程序想把某个段，比如堆或栈，扩大到超过这个上限，那就没办法了。

硬件还有其他办法来判断某个地址属于哪个段。在**隐式**（implicit）方法中，硬件根据地址是怎样产生的来确定段。比如，如果地址是由程序计数器（program counter）产生的（也就是取指令），那么这个地址就在代码段中；如果地址基于栈指针（stack pointer）或基址指针（base pointer），那它一定在栈段（stack segment）中；其他地址则都在堆中。

## 16.3 栈怎么办?

到目前为止，我们漏掉了地址空间中一个重要的组成部分：栈。在上面的图中，栈被重定位到了物理地址 28 KB，但它有一个关键的不同：它是反向增长的（即朝着低地址方向增长）。在物理内存中，它从 28 KB[1] 处“开始”，往回增长到 26 KB，对应虚拟地址 16 KB 到 14 KB；它的地址转换必须换一种方式进行。

首先需要一点额外的硬件支持。除了基址（base）和界限值，硬件还需要知道段朝哪个方向增长（比如用一个位来表示：段朝正方向增长时设为 1，朝负方向增长时设为 0）。硬件需要记录的信息更新后如图 16.4 所示：

硬件知道段可以朝负方向增长之后，转换这类虚拟地址的方式也要稍作改变。我们拿一个栈中的虚拟地址为例，看看它是怎么转换的。

在这个例子中，假设我们要访问虚拟地址 15 KB，它应该映射到物理地址 27 KB。这个虚拟地址的二进制形式是 11 1100 0000 0000（十六进制 0x3C00）。硬件用最高两位（11）指定段，剩下的偏移量是 3 KB。为了得到正确的负偏移量，我们必须用 3 KB 减去段的最大大

[1] 为简单起见，我们说栈从 28 KB 处“开始”，但这个值其实是紧挨在反向增长区域下方的那个字节；第一个有效字节实际上是 28 KB 减 1。相比之下，正向增长的区域从段中第一个字节的地址开始。我们这样处理，是因为这样计算物理地址最简单：物理地址就是基址（base）加上负偏移量。

| 段 | 基址 | 大小（最大 4K） | 正向增长? |
|---|---|---|---|
| 代码$_{00}$ | 32K | 2K | 1 |
| 堆$_{01}$ | 34K | 3K | 1 |
| 栈$_{11}$ | 28K | 2K | 0 |

图 16.4：**段寄存器（支持负向增长）**

小：在这个例子里，段最大可以是 4 KB，因此正确的负偏移量是 3 KB 减 4 KB，即 −1 KB。只要把这个负偏移量加到基址（base，28 KB）上，就得到了正确的物理地址：27 KB。至于界限检查，只要确保负偏移量的绝对值小于或等于段的当前大小（这里是 2 KB）即可。

## 16.4 支持共享

随着对分段的支持日益完善，系统设计者很快意识到，只要再加一点硬件支持，就能获得新的效率提升。具体来说，为了节省内存，有时在地址空间之间**共享**（share）某些内存段会很有用。尤其是**代码共享**（code sharing），它很常见，如今的系统里也还在用。

要支持共享，硬件还得再加一点东西，也就是**保护位**（protection bits）。基本的做法是给每个段加几个位，表明程序能否读写这个段，也许还表明能否执行段内的代码。把代码段设为只读后，同一份代码就可以在多个进程之间共享，而不用担心破坏隔离性（isolation）；每个进程仍然以为自己在访问私有的内存，而操作系统在暗中共享着这块进程无法修改的内存，于是假象得以维持。

图 16.5 举例展示了硬件（和操作系统）额外记录的信息。可以看到，代码段被设为可读、可执行，因此内存中同一个物理段（physical segment）可以映射到多个虚拟地址空间中。

有了保护位（protection bit），前面描述的硬件算法也得跟着改。除了检查虚拟地址是否在界限之内，硬件还必须检查某次访问是否被允许。如果用户进程试图写只读段，或者从不可执行的段中执行代码，硬件就应该触发异常（exception），让操作系统来处理这个出错的进程。

## 16.5 细粒度与粗粒度的分段

到目前为止，我们的例子大多针对只有几个段的系统；这种分段可以看作**粗粒度**（coarse-grained）的，因为它把地址空间切成相对较大、较粗的几块。不过，一些早期系统（例如 Multics [CV65, DD68]）更加灵活，允许地址空间由大量较小的段组成，这称为**细粒度**（fine-grained）分段。

| 段 | 基址 | 大小（最大 4K） | 正向增长? | 保护 |
|---|---|---|---|---|
| 代码$_{00}$ | 32K | 2K | 1 | 读-执行 |
| 堆$_{01}$ | 34K | 3K | 1 | 读-写 |
| 栈$_{11}$ | 28K | 2K | 0 | 读-写 |

图 16.5：**段寄存器的值（带保护位）**

 © 2008–25, ARPACI-DUSSEAU

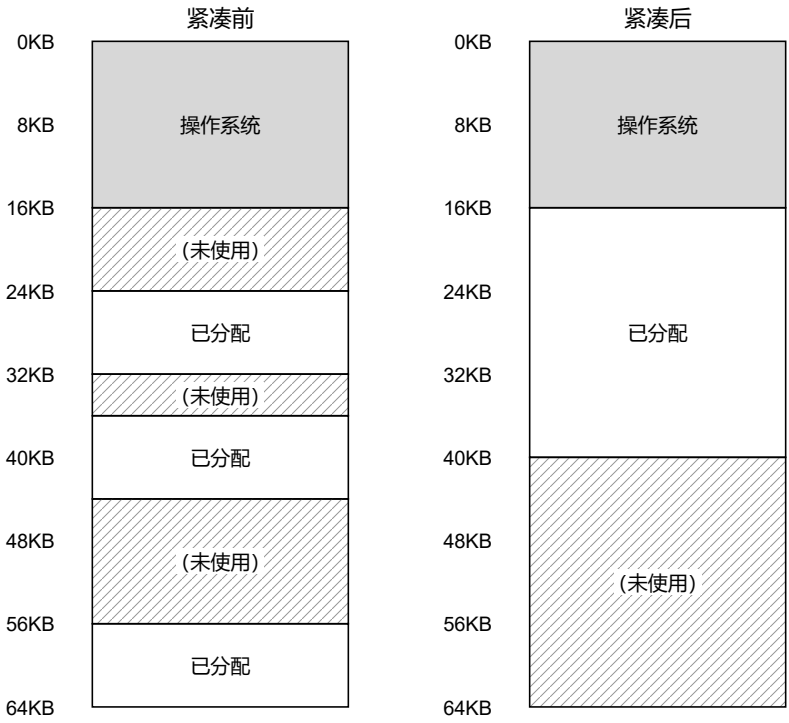



图 16.6：**紧凑前与紧凑后的内存**

要支持很多段，硬件还得再多做一些：在内存中存放某种**段表**（segment table）。这类段表（segment table）通常支持创建非常多的段，从而让系统能比我们前面讨论的更灵活地使用段。例如，像 Burroughs B5000 这样的早期机器支持成千上万个段，并期望编译器把代码和数据切分成独立的段，再由操作系统和硬件来支持 [RK68]。当时的想法是，有了细粒度的段，操作系统就能更清楚地了解哪些段在使用、哪些没有，从而更有效地利用主存。

## 16.6 操作系统支持

现在你应该对分段的工作原理有了基本的了解。系统运行时，地址空间的各个部分被重定位到物理内存中；与整个地址空间只用一对基址/界限寄存器（base/bounds register）的简单方法相比，这样能节省大量物理内存。具体来说，栈和堆之间所有未使用的空间都不必在物理内存中分配，于是我们能在物理内存中放下更多地址空间，并为每个进程支持大而稀疏的虚拟地址空间。

然而，分段给操作系统带来了一些新问题。第一个是老问题了：上下文切换（context switch）时操作系统该做什么？现在你应该能猜到：必须保存和恢复段寄存器（segment register）。显然，每个进程都有自己的虚拟地址空间，操作系统必须在让进程重新运行之前，把这些寄存器设置正确。

提示：如果有 1000 种解决方案，那就没有一种真正好的
已经有这么多不同的算法试图把外部碎片（external fragmentation）降到最低，这说明了一个更深层的事实：这个问题不存在一种“最好”的解法。因此，我们只能退而求其次，选一个合理的方案，并希望它足够好。唯一真正的解决办法（我们在接下来的章节中会看到）是彻底绕开这个问题：永远不以可变大小的块来分配内存。

第二个问题是段增长（或者缩小）时操作系统如何配合。比如，程序可能调用 `malloc()` 来分配一个对象。有时，现有的堆就能满足这个请求，于是 `malloc()` 会为这个对象找到空闲空间，并把指向它的指针返回给调用者。但有时，堆段（heap segment）本身需要增长。这时，内存分配库会发起系统调用（system call）来扩大堆，比如传统 UNIX 中的 `sbrk()` 系统调用。然后，操作系统（通常）会提供更多空间，把段大小寄存器（segment size register）更新为新的（更大的）大小，并通知分配库操作成功；分配库就可以为新对象分配空间，并成功返回给调用它的程序。注意，如果已经没有更多物理内存可用，或者操作系统认为调用进程占用的内存已经太多了，操作系统也可能拒绝这个请求。

最后一个、也许是最重要的问题，是管理物理内存中的空闲空间。创建新的地址空间时，操作系统必须能在物理内存中为它的各个段找到空间。之前我们假设每个地址空间大小都相同，因此可以把物理内存看作一组槽位，每个进程放进其中一个。现在，每个进程有好几个段，而且每个段的大小可能都不一样。

由此产生的普遍问题是：物理内存很快就会布满一个个由空闲空间形成的小“空洞（hole）”，导致很难为新的段分配空间，也很难扩大已有的段。我们把这个问题称为**外部碎片**（external fragmentation）[R69]；参见图 16.6（左）。

在这个例子中，来了一个进程，想分配一个 20 KB 的段。例子中一共有 24 KB 空闲，但它们并不是一整块连续的空间（而是三块不连续的区域）。因此，操作系统无法满足这个 20 KB 的请求。请求扩大某个段时也可能出现类似的问题：如果段后面紧接着的那些物理空间不可用，即使物理内存的其他地方还有空闲字节，操作系统也只能拒绝这个请求。

解决这个问题的一种方法是重新排列现有的段，把物理内存**紧凑**（compact）起来。例如，操作系统可以停下正在运行的进程，把它们的数据复制到一块连续的内存区域中，修改它们的段寄存器（segment register）值，让它们指向新的物理位置，从而腾出一大块空闲内存。这样一来，新的分配请求就能成功了。不过，紧凑（compaction）的代价很高，因为复制段是内存密集型操作，通常要占用相当多的处理器时间；紧凑（compaction）后的物理内存见图 16.6（右）。讽刺的是，紧凑（compaction）还会让扩大已有段的请求更难满足，可能因此需要进一步重新排列。

一种更简单的办法也许是使用空闲链表（free list）管理算法，尽量保留大块可供分配的内存。人们用过的方法真有好几百种，包括经典算法，如**最佳适应**（best-fit，维护一个空闲空间链表，把能满足请求且大小最接近的那块返回给请求者）、**最坏适应**（worst-fit）、**首次适应**（first-fit），以及更复杂的方案，如**伙伴算法**（buddy algorithm）[K68]。如果你想进一步了解这类算法，Wilson 等人写的一篇出色的综述是个很好的起点 [W+95]；你也可以

 © 2008–25, ARPACI-DUSSEAU

等到后面的章节，我们会介绍其中一些基础内容。不过遗憾的是，无论算法多聪明，外部碎片依然存在；好的算法只是尽量减少它。

## 16.7 小结

分段解决了不少问题，也有助于我们更有效地虚拟化内存。分段不只是实现了动态重定位（dynamic relocation），还能更好地支持稀疏地址空间（sparse address space），因为它避免了地址空间各逻辑段（logical segment）之间可能存在的大量内存浪费。它也很快，因为分段所需的运算很简单，非常适合用硬件实现；地址转换的开销极小。此外还有一个附带的好处：代码共享（code sharing）。如果把代码放在单独的段中，这个段就有可能被多个正在运行的程序共享。

然而，我们也看到了，在内存中分配大小可变的段会带来一些问题，我们希望能克服它们。第一个问题就是前面讨论过的外部碎片。由于段的大小可变，空闲内存会被切成大小不一的碎块，因此满足内存分配请求可能很困难。我们可以尝试使用聪明的算法 [W+95]，或者定期紧凑（compact）内存，但这个问题是根本性的，很难避免。

第二个、也许更重要的问题是，分段还是不够灵活，无法支持我们完全通用的稀疏地址空间（sparse address space）。例如，如果一个很大、但只有零星部分被用到的堆全在一个逻辑段（logical segment）里，那么整个堆仍然必须驻留在内存中才能被访问。换句话说，如果地址空间的使用方式与底层分段的设计初衷不完全吻合，分段的效果就不太好。因此，我们需要找到一些新的解决方案。准备好了吗？

## 参考文献

[CV65] "Introduction and Overview of the Multics System" by F. J. Corbato, V. A. Vyssotsky. Fall Joint Computer Conference, 1965. *One of five papers presented on Multics at the Fall Joint Computer Conference; oh to be a fly on the wall in that room that day!*

[DD68] "Virtual Memory, Processes, and Sharing in Multics" by Robert C. Daley and Jack B. Dennis. Communications of the ACM, Volume 11:5, May 1968. *An early paper on how to perform dynamic linking in Multics, which was way ahead of its time. Dynamic linking finally found its way back into systems about 20 years later, as the large X-windows libraries demanded it. Some say that these large X11 libraries were MIT's revenge for removing support for dynamic linking in early versions of UNIX!*

[G62] "Fact Segmentation" by M. N. Greenfield. Proceedings of the SJCC, Volume 21, May 1962. *Another early paper on segmentation; so early that it has no references to other work.*

[H61] "Program Organization and Record Keeping for Dynamic Storage" by A. W. Holt. Communications of the ACM, Volume 4:10, October 1961. *An incredibly early and difficult to read paper about segmentation and some of its uses.*

[I09] "Intel 64 and IA-32 Architectures Software Developer's Manuals" by Intel. 2009. Available: http://www.intel.com/products/processor/manuals. *Try reading about segmentation in here (Chapter 3 in Volume 3a); it'll hurt your head, at least a little bit.*

[K68] "The Art of Computer Programming: Volume I" by Donald Knuth. Addison-Wesley, 1968. *Knuth is famous not only for his early books on the Art of Computer Programming but for his typesetting system TeX which is still a powerhouse typesetting tool used by professionals today, and indeed to typeset this very book. His tomes on algorithms are a great early reference to many of the algorithms that underlie computing systems today.*

[L83] "Hints for Computer Systems Design" by Butler Lampson. ACM Operating Systems Review, 15:5, October 1983. *A treasure-trove of sage advice on how to build systems. Hard to read in one sitting; take it in a little at a time, like a fine wine, or a reference manual.*

[LL82] "Virtual Memory Management in the VAX/VMS Operating System" by Henry M. Levy, Peter H. Lipman. IEEE Computer, Volume 15:3, March 1982. *A classic memory management system, with lots of common sense in its design. We'll study it in more detail in a later chapter.*

[RK68] "Dynamic Storage Allocation Systems" by B. Randell and C.J. Kuehner. Communications of the ACM, Volume 11:5, May 1968. *A nice overview of the differences between paging and segmentation, with some historical discussion of various machines.*

[R69] "A note on storage fragmentation and program segmentation" by Brian Randell. Communications of the ACM, Volume 12:7, July 1969. *One of the earliest papers to discuss fragmentation.*

[W+95] "Dynamic Storage Allocation: A Survey and Critical Review" by Paul R. Wilson, Mark S. Johnstone, Michael Neely, David Boles. International Workshop on Memory Management, Scotland, UK, September 1995. *A great survey paper on memory allocators.*

### 作业（模拟）

这个程序可以让你看到，在分段系统中地址转换是如何进行的。详情请见 README。

### 问题

1. 首先，我们用一个很小的地址空间来转换几个地址。下面是一组简单的参数，配上几个不同的随机种子；你能转换这些地址吗？
   - `segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 0`
   - `segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 1`
   - `segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 2`
2. 现在，看看我们是否理解了用上一题的参数构造的这个小地址空间。段 0 中最高的合法虚拟地址是多少？段 1 中最低的合法虚拟地址呢？在整个地址空间中，最低和最高的非法地址分别是多少？最后，运行 `segmentation.py` 时该怎样使用 `-A` 选项，来检验你的答案对不对？
3. 假设在 128 字节的物理内存中有一个 16 字节的小地址空间。对于指定的地址流，你要怎样设置基址（base）和界限，才能让模拟器产生下面的转换结果：有效、有效、违规……违规、有效、有效？假设参数如下：

```
segmentation.py -a 16 -p 128
    -A 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15
    --b0 ? --l0 ? --b1 ? --l1 ?
```

4. 假设我们想生成这样一个问题：随机生成的虚拟地址中大约 90% 是有效的，不会造成段违规（segmentation violation）。应该如何配置模拟器？哪些参数对得到这个结果比较重要？
5. 你能让模拟器运行后没有一个虚拟地址是有效的吗？怎么做？

 © 2008–25, ARPACI-DUSSEAU

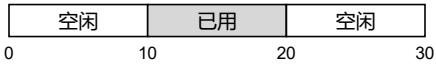
# 17

# 空闲空间管理

本章暂时把内存虚拟化（virtualization）的讨论放一放，岔开来谈谈所有内存管理系统都要面对的一个基本问题：无论是管理进程（process）堆（heap）中各页（page）的 malloc 库，还是管理进程地址空间（address space）各个部分的操作系统本身，都绕不开它。具体来说，我们要讨论**空闲空间管理**（free-space management）的相关问题。

我们把问题说得更具体一些。管理空闲空间（free space）当然可以很容易，我们在讨论**分页**（paging）概念时就会看到。如果你管理的空间被划分成固定大小的单元，事情就很简单：只要用一个列表记下这些固定大小的单元；客户端请求一个单元时，返回列表的第一项即可。

如果你管理的空闲空间由大小可变的单元组成，空闲空间管理（free-space management）就变得更困难（也更有意思）了。这种情况出现在用户级的内存分配库中（比如 `malloc()` 和 `free()`），也出现在用**分段**（segmentation）实现虚拟内存（virtual memory）时管理物理内存（physical memory）的操作系统中。无论哪种情况，这里的问题都叫作**外部碎片**（external fragmentation）：空闲空间被切成大小不一的小块，变得支离破碎；即使空闲空间的总量超过了请求的大小，后续的请求也可能因为找不到一块能满足它的连续空间而失败。




上图展示了这个问题的一个例子。这里可用的空闲空间一共有 20 字节；不巧的是，它被分成了两块，每块 10 字节。结果，即使有 20 字节空闲，一个 15 字节的请求也会失败。由此，我们引出了本章要解决的问题。

关键问题：如何管理空闲空间

要满足大小可变的请求，应该如何管理空闲空间？可以用什么策略来尽量减少碎片？各种方法的时间和空间开销如何？

## 17.1 假设

本章的讨论主要围绕用户级内存分配库中的分配器（allocator）展开，这些分配器有着悠久的历史。我们借鉴了 Wilson 的出色综述 [W+95]，但也鼓励感兴趣的读者去读这篇综述本身，了解更多细节[1]。

我们假设有一个基本的接口，就像 `malloc()` 和 `free()` 提供的那样。具体来说，`void *malloc(size_t size)` 接受一个参数 `size`，即应用程序请求的字节数；它返回一个指针，指向一块这么大（或更大）的区域。这个指针没有特定类型，用 C 的行话来说就是 void 指针（void pointer）。与之配套的例程 `void free(void *ptr)` 接受一个指针，释放对应的内存块（chunk of memory）。注意这个接口隐含的意思：用户释放空间时，不会告诉库这块空间有多大；因此，库必须在只拿到一个指针的情况下，就能算出这块内存有多大。本章稍后会讨论怎么做到这一点。

这个库所管理的空间，历史上称为堆；用来管理堆中空闲空间的通用数据结构（data structure），是某种**空闲链表**（free list）。这个结构记录了所管理的内存区域中所有的空闲块（free chunk）。当然，这个数据结构不一定真的是链表（linked list），只要是某种能记录空闲空间的数据结构就行。

我们还假设，这里主要关心的是前面说的**外部碎片**（external fragmentation）。分配器当然也可能有**内部碎片**（internal fragmentation）的问题：如果分配器分出去的内存块（chunk of memory）比请求的大，那么块中多出来的、没有被请求（因而没有用上）的空间就算作内部碎片（因为浪费发生在已分配单元的内部），这是空间浪费的又一个例子。不过，为简单起见，也因为外部碎片是两种碎片中更有意思的一种，我们主要关注外部碎片。

我们还假设，内存一旦分给客户端，就不能再被重定位（relocate）到内存中的其他位置。例如，如果程序调用 `malloc()`，拿到了指向堆中某块空间的指针，那么这块内存区域实际上就“归”这个程序所有了（库不能移动它），直到程序通过相应的 `free()` 调用把它还回来。因此，也就没法对空闲空间做**紧凑**（compaction），而紧凑本来有助于对抗碎片[2]。不过，操作系统在实现**分段**（segmentation）时，可以用紧凑（compaction）来处理碎片（见分段那一章的讨论）。

最后，我们假设分配器管理的是一块连续的字节区域。有时，分配器可以要求这块区域增长；比如，用户级的内存分配库在空间用完时，可能会通过 `sbrk` 这样的系统调用（system call）请内核扩大堆。不过，为简单起见，我们只假设这块区域在整个生命周期内大小固定不变。

## 17.2 底层机制

在深入策略细节之前，我们先介绍大多数分配器都会用到的一些通用机制。首先，我们讨论分割（splitting）与合并（coalescing）的基础知识，几乎所有分配器都会用到这两种

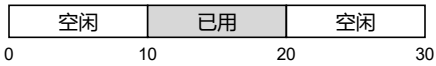
[1] 它将近 80 页长，所以你得真的很感兴趣才行！

[2] 一旦把指向某块内存的指针交给 C 程序，一般就很难找出指向这块区域的所有引用（指针），它们可能存放在其他变量中，甚至在执行过程中的某一时刻存放在寄存器里。在强类型、带垃圾回收（garbage collection）的语言中，情况可能并非如此，因此这些语言可以用紧凑（compaction）这种技术来对抗碎片。

常见技术。其次，我们介绍如何快速、相对轻松地记录已分配区域的大小。最后，我们讨论如何在空闲空间内部构建一个简单的链表，来记录哪些空间是空闲的、哪些不是。

### 分割与合并

空闲链表包含一组元素，描述了堆中剩余的空闲空间。假设有下面这个 30 字节的堆：




这个堆的空闲链表上有两个元素。一个描述第一个 10 字节的空闲段（free segment），即字节 0–9；另一个描述第二个空闲段，即字节 20–29：




如上所述，任何大于 10 字节的请求都会失败（返回 NULL）；根本没有一块这么大的连续内存可用。正好 10 字节的请求，用任意一个空闲块都能轻松满足。但如果请求的大小小于 10 字节，会怎么样呢?

假设我们只请求一个字节的内存。这时，分配器会执行一个称为**分割**（splitting）的操作：它会找到一块能满足请求的空闲内存，把它一分为二。第一块返回给调用者，第二块留在链表上。因此，在上面的例子中，如果请求 1 字节，而分配器决定用链表上两个元素中的第二个来满足请求，那么 `malloc()` 会返回 20（这 1 字节已分配区域的地址），链表最后会变成这样：




从图中可以看到，链表基本没变；唯一的变化是这块空闲区域现在从 21 而不是 20 开始，长度也只剩 9 了[3]。因此，当请求比某个空闲块小时，分配器通常就会用到分割。

许多分配器中还有一个与之相辅相成的机制，叫作空闲空间的**合并**（coalescing）。我们再回到上面的例子（空闲 10 字节、已用 10 字节、再空闲 10 字节）。

对于这个（很小的）堆，如果应用程序调用 `free(10)`，把堆中间的空间还回来，会发生什么？如果我们不假思索地直接把这块空闲空间加回链表，链表可能会变成这样：




注意问题所在：虽然整个堆现在都是空闲的，但它看起来被分成了三块，每块 10 字节。因此，如果用户请求 20 字节，简单地遍历链表找不到这么大的空闲块，只能返回失败。

为了避免这个问题，分配器会在释放内存块（chunk of memory）时合并空闲空间。思路很简单：把一块空闲内存还回去时，仔细查看要归还的块（chunk）以及附近空闲块的地址；如果新释放的空间正好紧挨着一个（或者像这个例子中那样，两个）现有的空闲块，就把它们合并成一个更大的空闲块。因此，有了合并，最后的链表应该是这样的：

[3]这里的讨论假设没有头部（header）。这个假设不现实，但我们暂时用它来简化问题。




事实上，这正是还没做任何分配时，堆的链表最初的样子。有了合并，分配器能更好地确保应用程序有大块的空闲空间可用。

### 记录已分配区域的大小

你可能已经注意到，`free(void *ptr)` 这个接口不接受大小参数；这就默认了 malloc 库只凭一个指针，就能迅速确定被释放的内存区域有多大，进而把这块空间放回空闲链表。

为了做到这一点，大多数分配器会在**头部**（header）中存放一点额外的信息。头部保存在内存中，通常就在分出去的那块内存前面。我们再来看一个例子（图 17.1）。在这个例子中，我们看的是一块大小为 20 字节的已分配内存块（chunk），由 `ptr` 指向；假设用户调用了 `malloc()`，并把结果存进了 `ptr`，例如 `ptr = malloc(20);`。

头部至少包含已分配区域的大小（这里是 20）；它还可能包含用来加快释放的额外指针、用来做额外完整性（integrity）检查的幻数（magic number），以及其他信息。我们假设头部很简单，只包含区域的大小和一个幻数（magic number），如下所示：

```
typedef struct {
    int size;
    int magic;
} header_t;
```

上面的例子如图 17.2 所示。当用户调用 `free(ptr)` 时，库会用简单的指针运算算出头部从哪里开始：

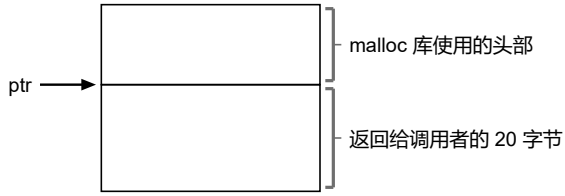



图 17.1：**已分配区域及其头部**

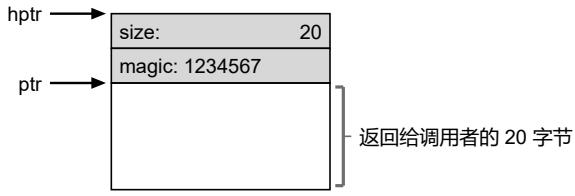



图 17.2：**头部的具体内容**

```
void free(void *ptr) {
    header_t *hptr = (header_t *) ptr - 1;
    ...
```

拿到指向头部的指针后，库可以很容易地检查幻数（magic number）是否与预期值相符，作为一次健全性检查（`assert(hptr->magic == 1234567)`），并通过简单的计算（即把头部的大小加上区域的大小）算出新释放区域的总大小。注意上一句中这个小而关键的细节：空闲区域的大小是头部的大小加上分配给用户的空间大小。因此，当用户请求 $N$ 字节内存时，库搜索的并不是大小为 $N$ 的空闲块，而是大小为 $N$ 加上头部大小的空闲块。

### 嵌入空闲链表

到目前为止，我们一直把这个简单的空闲链表当作一个概念上的东西；它只是一个描述堆中空闲内存块（chunk of memory）的链表。但我们如何在空闲空间本身内部构建这样一个链表呢？

如果是一般的链表，要分配新节点时，直接调用 `malloc()` 申请节点所需的空间就行了。可惜，在内存分配库内部，你不能这么做！你得在空闲空间本身内部构建这个链表。如果这听起来有点怪，不用担心；它确实有点怪，但还没怪到你做不到的地步！

假设我们要管理一块 4096 字节的内存（即堆的大小是 4 KB）。要把它当作空闲链表来管理，我们首先得初始化这个链表；最开始，链表应该只有一项，大小为 4096 减去头部的大小。下面是链表节点的定义：

```
typedef struct __node_t {
    int              size;
    struct __node_t *next;
} node_t;
```

现在我们来看一段代码，它初始化堆，并把空闲链表的第一个元素放进这块空间。我们假设堆建在通过系统调用 `mmap()` 获得的一块空闲空间中；这不是构建堆的唯一方法，但在这个例子中很合适。代码如下：

```
// mmap() 返回指向一块空闲空间的指针
node_t *head = mmap(NULL, 4096, PROT_READ|PROT_WRITE,
                    MAP_ANON|MAP_PRIVATE, -1, 0);
head->size   = 4096 - sizeof(node_t);
head->next   = NULL;
```

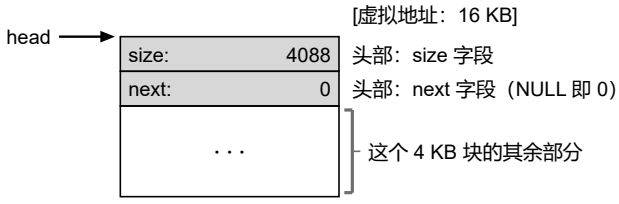



图 17.3：**只有一个空闲块的堆**

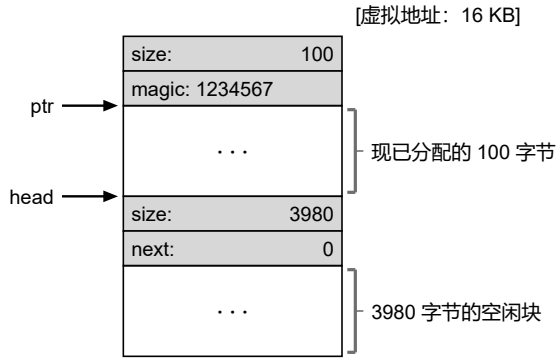



图 17.4：**分配一次之后的堆**

运行这段代码后，链表里只有一项，大小为 4088。没错，这是个很小的堆，但用来举例正合适。`head` 指针保存着这个范围的起始地址；我们假设它是 16 KB，其实任何虚拟地址（virtual address）都可以。画出来，堆就是图 17.3 的样子。

现在，假设有人请求一块内存，比如 100 字节。为了满足这个请求，库首先要找一个大到足以容纳这个请求的块（chunk）；因为只有一个大小为 4088 的空闲块，所以就选它。然后，库把这个块（chunk）**分割**（splitting）成两块：一块大小足以容纳请求以及上面说的头部，另一块是剩下的空闲块。假设头部占 8 字节，其中一个整数表示大小，一个整数表示幻数（magic number），那么堆中的空间现在就像图 17.4 那样。

因此，收到 100 字节的请求后，库从现有的那个空闲块中分配了 108 字节，返回一个指向它的指针（图中标为 `ptr`），把头部信息紧挨着存放在已分配空间前面，供以后 `free()` 时使用，并把链表中那个唯一的空闲节点缩小到 3980 字节（4088 减 108）。

现在我们来看有三块已分配区域时的堆，每块 100 字节，加上头部是 108 字节。图 17.5 展示了这个堆。

从图中可以看到，堆的前 324 字节现在已经分配出去，因此这段空间里有三个头部，以及调用程序正在使用的三个 100 字节区域。空闲链表依旧平淡无奇：只有一个节点（由 `head` 指向），不过经过三次分割，它现在只剩 3764 字节了。但当调用程序通过 `free()` 归还一些内存时，会发生什么呢？

在这个例子中，应用程序调用 `free(16500)`，归还了中间那块已分配的内存。16500 这个值是这样算出来的：内存区域的起始地址 16384，加上前一块的 108 字节，再加上这一块的 8 字节头部。这个值在前面的图中用指针 `sptr` 表示。

库立刻算出这块空闲区域的大小，然后把这个空闲块加回空闲链表。假设我们把它插在空闲链表的开头，现在的空间就像图 17.6 这样。

现在，链表的开头是一个 100 字节的小空闲块，由链表头指向；后面是一个 3764 字节的大空闲块。我们的链表终于不止一个元素了！没错，空闲空间产生了碎片，这很不幸，但很常见。

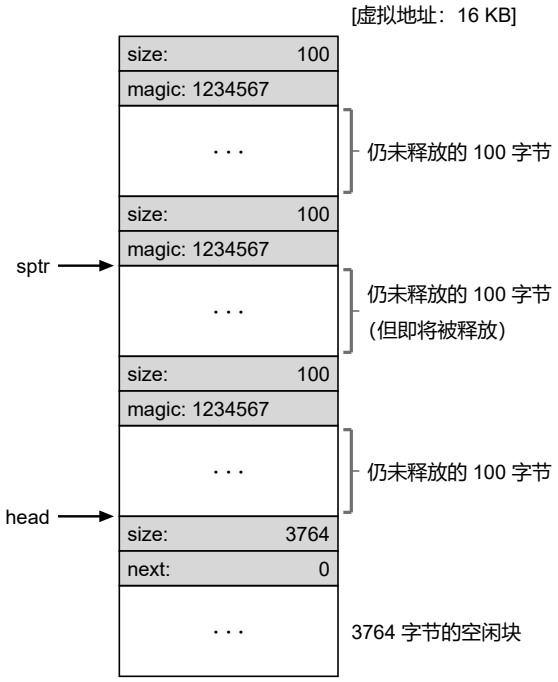



图 17.5：**有三块已分配时的空闲空间**

最后一个例子：假设现在最后两块正在使用的内存也被释放了。如果不合并，最后就会产生碎片（图 17.7）。

从图中可以看到，现在一团糟！为什么？很简单，我们忘了**合并**（coalescing）链表。所有内存其实都是空闲的，却被切成了许多块，看起来像是碎片化（fragmented）的内存，实际上并不是。解决方法很简单：遍历链表，把相邻的块（chunk）**合并**（coalescing）起来；完成后，堆就又完整了。

### 扩大堆

最后还要讨论许多分配库中都有的一个机制：如果堆的空间用完了，该怎么办？最简单的做法就是直接失败。有时这是唯一的选择，因此返回 NULL 也是一种体面的做法。别难过！你尽力了，虽败犹荣。

大多数传统的分配器一开始只有一个很小的堆，空间用完后再向操作系统申请更多内存。通常，这意味着它们会发起某种系统调用来扩大堆，例如大多数 UNIX 系统中的 `sbrk`，再从扩出来的空间里分配新的块（chunk）。为了处理 `sbrk` 请求，操作系统会找到空闲的物理页（physical page），把它们映射到请求进程的地址空间中，然后返回新的堆末尾地址；到这时，就有了一个更大的堆，请求也就能顺利满足了。

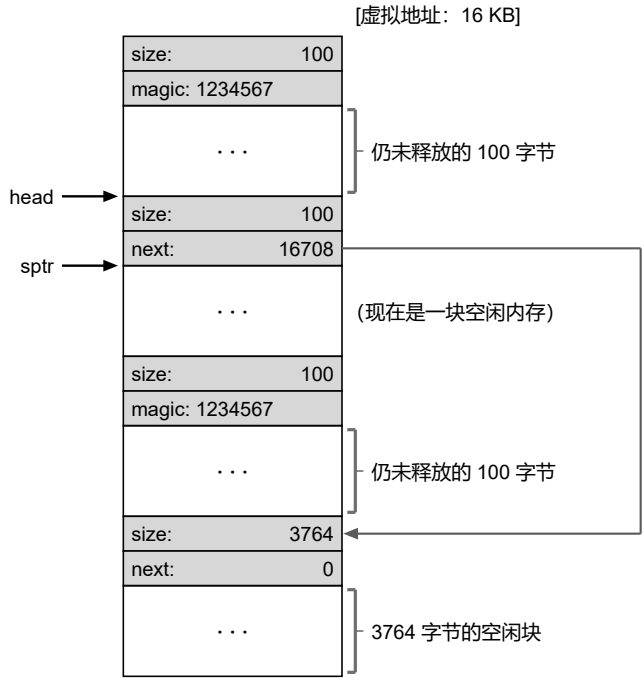



图 17.6：**有两块已分配时的空闲空间**

## 17.3 基本策略

既然手头已经有了一些机制，我们来看看管理空闲空间的一些基本策略。这些方法大多相当简单，你自己也能想出来；不妨在往下读之前先试一试，看看能不能想出所有这些方案（甚至想出一些新的！）。

理想的分配器既要快，又要让碎片最少。遗憾的是，由于分配和释放请求的序列可以是任意的（毕竟它们由程序员决定），任何一种策略在遇到不利的输入时都可能表现得很糟。因此，我们不会介绍一种“最好”的方法，而是介绍一些基本方法，并讨论它们的优缺点。

### 最佳适应

**最佳适应**（best fit）策略很简单：首先，搜索空闲链表，找出所有大小等于或大于请求大小的空闲内存块（chunk of memory）。然后，返回这些候选块（chunk）中最小的那一块；这就是所谓的最佳适应块（best-fit chunk），也可以叫最小适应块。遍历一次空闲链表，就足以找到要返回的那一块。

最佳适应（best fit）背后的直觉很简单：返回一个大小接近用户请求的块（chunk），以此来减少空间浪费。不过，这是有代价的：简单的实现要穷尽搜索才能找到合适的空闲块，性能损失很大。

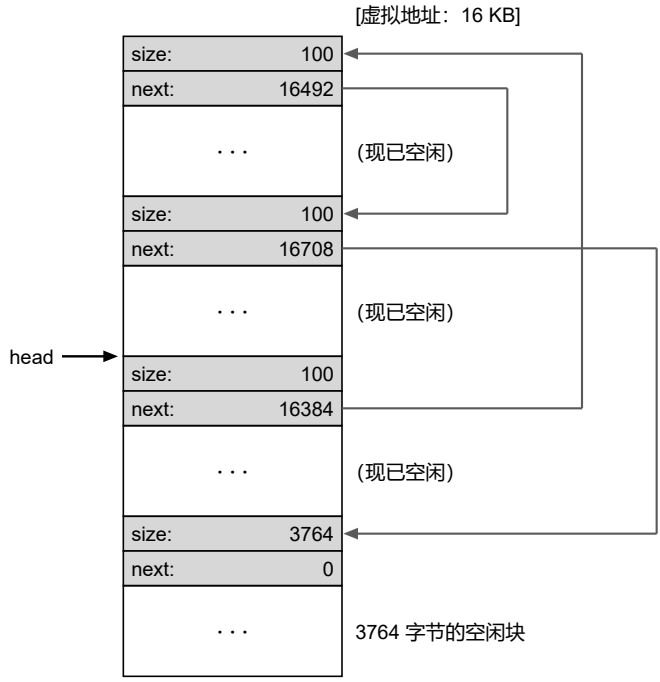



图 17.7：**未合并的空闲链表**

## 最坏适应

**最坏适应**（worst fit）方法与最佳适应（best fit）相反：找到最大的块（chunk），从中分出请求的大小，把剩下的（大）块留在空闲链表上。这样，最坏适应（worst fit）试图留下大的空闲块，而不是像最佳适应（best fit）那样留下许多小块（chunk）。不过同样，它需要完整地搜索一遍空闲空间，因此代价可能很高。更糟的是，大多数研究表明它的表现很差，会导致过多的碎片，开销也依然很高。

## 首次适应

**首次适应**（first fit）方法直接找到第一个足够大的块（chunk），从中分出请求的大小返回给用户。和前面一样，剩下的空闲空间留着供后续请求使用。

首次适应（first fit）的优点是快，不需要穷尽搜索所有空闲空间；但它有时会让空闲链表的开头堆满小对象。因此，分配器如何管理空闲链表的顺序就成了一个问题。一种方法是采用**按地址排序**（address-based ordering）：让链表按空闲空间的地址排好序，合并就更容易，碎片也往往会减少。

### 下次适应

**下次适应**（next fit）算法不是每次都从链表开头开始首次适应（first fit）的搜索，而是多维护一个指针，指向上一次查找到的链表位置。这样做的目的是让搜索更均匀地分布在整个链表上，避免链表开头被切得零零碎碎。这种方法的性能和首次适应（first fit）非常接近，因为同样避免了穷尽搜索。

### 例子

下面是上述几种策略的几个例子。设想一个空闲链表上有三个元素，大小分别是 10、30 和 20。这里我们忽略头部和其他细节，只关注各种策略如何运作：




假设有一个大小为 15 的分配请求。最佳适应（best fit）方法会搜索整个链表，发现 20 最合适，因为它是能容纳这个请求的最小空闲空间。得到的空闲链表是：




就像这个例子一样，最佳适应（best fit）方法常常会留下一个小的空闲块。最坏适应（worst fit）方法与之类似，但找的是最大的块（chunk），在这个例子中是 30。得到的链表是：




在这个例子中，首次适应（first fit）策略的结果和最坏适应（worst fit）一样：它选中的也是第一个能满足请求的空闲块。区别在于搜索开销：最佳适应（best fit）和最坏适应（worst fit）都要查看整个链表；首次适应（first fit）找到一个合适的空闲块就停下，因此减少了搜索开销。

这些例子只触及了分配策略的皮毛。要更深入地理解，还需要结合真实的工作负载（workload），并考虑合并这类更复杂的分配器行为，做更细致的分析。你说，这是不是很适合留作作业？

## 17.4 其他方法

除了上面介绍的基本方法，人们还提出了大量技术和算法，从各方面改进内存分配。我们在这里列出其中几种供你参考，让你的思路不止于最佳适应（best fit）分配。

### 分离空闲链表

有一种由来已久的有趣方法，是使用**分离空闲链表**（segregated lists）。基本思路很简单：如果某个应用程序有一种（或几种）经常请求的大小，就单独维护一个链表，专门管理这种大小的对象；其他所有请求则转发给一个更通用的内存分配器（memory allocator）。

补充：伟大的工程师真的很伟大

Jeff Bonwick 不仅写了本章提到的 slab 分配器（slab allocator），还领导开发了一个了不起的文件系统 ZFS。像他这样的工程师，是硅谷的核心。几乎每一个伟大的产品或技术背后，都有一个人（或一小群人），他们的天赋、能力和投入都远超常人。正如 Facebook 的 Mark Zuckerberg 所说：“在自己岗位上出类拔萃的人，并不只是比相当不错的人好一点点。他们要好上 100 倍。”这就是为什么直到今天，一两个人还能创办一家永远改变世界面貌的公司（想想 Google、Apple 或 Facebook）。努力工作，你也可能成为这样的“100 倍”人才！如果做不到，那就想办法和这样的人共事；你一天学到的，会比大多数人一个月学到的还多。

这种方法的好处显而易见。专门拿出一块内存来满足某一特定大小的请求，碎片问题就小得多；而且，当请求的大小正好合适时，分配和释放请求都能处理得很快，因为不需要在链表里做复杂的搜索。

就像任何好主意一样，这种方法也给系统带来了新的复杂性。例如，应该拿出多少内存给专门服务特定大小请求的内存池，又该留多少给通用内存池？有一个分配器相当漂亮地解决了这个问题，那就是超级工程师 Jeff Bonwick 为 Solaris 内核设计的 **slab 分配器**（slab allocator）[B94]。

具体来说，内核启动时，这个分配器会为那些可能被频繁请求的内核对象，比如锁（lock）、文件系统（file system）的 inode 等，分配一些**对象缓存**（object caches）；这样，每个对象缓存都是某个特定大小的分离空闲链表（segregated list），能快速地处理内存分配和释放请求。当某个缓存（cache）的空闲空间不足时，它会向一个更通用的内存分配器请求若干内存 **slab**，请求的总量是页大小（page size）和相应对象大小的倍数。反过来，当某个 slab 中所有对象的引用计数（reference count）都降为零时，通用分配器就可以从专用分配器那里收回这些 slab，这通常发生在虚拟内存系统需要更多内存的时候。

slab 分配器（slab allocator）还比大多数分离空闲链表（segregated list）方法更进一步：它让链表上的空闲对象保持预先初始化的状态。Bonwick 指出，数据结构的初始化和销毁代价很高 [B94]；slab 分配器（slab allocator）让链表中被释放的对象保持已初始化状态，免去了对每个对象反复初始化和销毁，明显降低了开销。

### 伙伴分配

由于合并对分配器至关重要，有些方法就是围绕着让合并变简单来设计的。一个很好的例子是二**进制伙伴分配器**（binary buddy allocator）[K65]。

在这样的系统中，空闲内存首先在概念上被看作一整块大小为 $2^N$ 的空间。当有内存请求时，搜索空闲空间的过程会递归地把空闲空间一分为二，直到找到一块大到足以容纳请求、再分一次就会太小的块（block）为止。这时，就把这个块返回给用户。下面这个例子展示了为一个 7 KB 的请求搜索时，一块 64 KB 的空闲空间是怎样逐步划分的（图 17.8，第 160 页）。

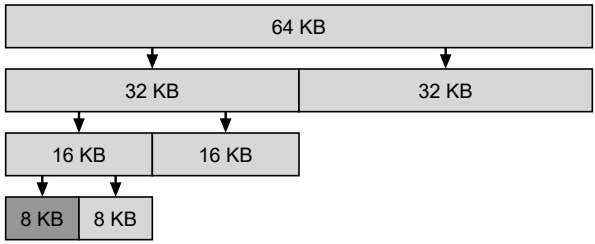



图 17.8：**伙伴系统管理的堆示例**

在这个例子中，最左边的 8 KB 块被分配出去（图中深灰色），返回给用户；注意，这种方案可能会有**内部碎片**（internal fragmentation），因为你只能分出大小为 2 的幂的块。

伙伴分配（buddy allocation）的妙处体现在块被释放的时候。把这个 8 KB 的块还给空闲链表时，分配器会检查它的“伙伴”8 KB 块是否空闲；如果是，就把两块合并成一个 16 KB 的块。接着检查这个 16 KB 块的伙伴是否空闲；如果是，再把这两块合并。这种递归合并沿着树一直向上进行，要么恢复出整个空闲空间，要么在发现某个伙伴正在使用时停下来。

伙伴分配（buddy allocation）之所以效果这么好，是因为确定某个块的伙伴很简单。你可能会问，怎么确定？想想上面空闲空间中各个块的地址。只要仔细想一想，你就会发现每一对伙伴的地址只差一位；是哪一位，取决于它在伙伴树（buddy tree）中的层级。这样，你就对二进制伙伴分配（binary buddy allocation）方案的工作原理有了基本的了解。想了解更多细节，照例请参阅 Wilson 的综述 [W+95]。

### 其他想法

上面介绍的许多方法都有一个大问题：缺乏**可扩展性**（scaling）。具体来说，搜索链表可能相当慢。因此，高级的分配器会使用更复杂的数据结构来降低这些开销，牺牲简单性来换取性能。例子包括平衡二叉树、伸展树或 partially-ordered tree [W+95]。

现代系统通常有多个处理器，运行多线程（multi-threaded）的工作负载；本书的并发（concurrency）部分会详细介绍这些内容。因此，人们花了大量精力让分配器在多处理器（multiprocessor）系统上运行良好，这并不奇怪。Berger 等人 [B+00] 和 Evans [E06] 给出了两个很好的例子，细节请去看他们的论文。

这只是人们这些年来关于内存分配器的成千上万种想法中的两个；好奇的话可以自己去读。再不济，也读读 glibc 分配器的工作原理 [S15]，感受一下真实世界的样子。

## 17.5 小结

本章讨论了内存分配器最基本的形式。这样的分配器无处不在：它们被链接进你写的每个 C 程序，也存在于为自身数据结构管理内存的底层操作系统中。和许多系统一样，构建这样的系统需要做许多权衡；你对分配器面对的具体工作负载了解得越多，就越能有针对性地调优，让它在这种负载下工作得更好。在现代计算机系统中，构建一个快速、省空间、可扩展、能在各种工作负载下都表现良好的分配器，仍是一个持续的挑战。

## 参考文献

[B+00] "Hoard: A Scalable Memory Allocator for Multithreaded Applications" by Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, Paul R. Wilson. ASPLOS-IX, November 2000. *Berger and company's excellent allocator for multiprocessor systems. Beyond just being a fun paper, also used in practice!*

[B94] "The Slab Allocator: An Object-Caching Kernel Memory Allocator" by Jeff Bonwick. USENIX '94. *A cool paper about how to build an allocator for an operating system kernel, and a great example of how to specialize for particular common object sizes.*

[E06] "A Scalable Concurrent malloc(3) Implementation for FreeBSD" by Jason Evans. April, 2006. http://people.freebsd.org/~jasone/jemalloc/bsdcan2006/jemalloc.pdf. *A detailed look at how to build a real modern allocator for use in multiprocessors. The "jemalloc" allocator is in widespread use today, within FreeBSD, NetBSD, Mozilla Firefox, and within Facebook.*

[K65] "A Fast Storage Allocator" by Kenneth C. Knowlton. Communications of the ACM, Volume 8:10, October 1965. *The common reference for buddy allocation. Random strange fact: Knuth gives credit for the idea not to Knowlton but to Harry Markowitz, a Nobel-prize winning economist. Another strange fact: Knuth communicates all of his emails via a secretary; he doesn't send email himself, rather he tells his secretary what email to send and then the secretary does the work of emailing. Last Knuth fact: he created TeX, the tool used to typeset this book. It is an amazing piece of software*[4].

[S15] "Understanding glibc malloc" by Sploitfun. February, 2015. sploitfun.wordpress.com/2015/02/10/understanding-glibc-malloc/. *A deep dive into how glibc malloc works. Amazingly detailed and a very cool read.*

[W+95] "Dynamic Storage Allocation: A Survey and Critical Review" by Paul R. Wilson, Mark S. Johnstone, Michael Neely, David Boles. International Workshop on Memory Management, Scotland, UK, September 1995. *An excellent and far-reaching survey of many facets of memory allocation. Far too much detail to go into in this tiny chapter!*

[4] Actually we use LaTeX, which is based on Lamport's additions to TeX, but close enough.

## 作业（模拟）

程序 `malloc.py` 可以让你探索本章描述的那种简单空闲空间分配器（free-space allocator）的行为。基本用法详见 README。

## 问题

1. 首先，用选项 `-n 10 -H 0 -p BEST -s 0` 运行，生成一些随机的分配和释放操作。你能预测 alloc()/free() 会返回什么吗？你能猜出每次请求之后空闲链表的状态吗？随着时间推移，你注意到空闲链表有什么变化？
2. 使用最坏适应（worst fit，`-p WORST`）策略搜索空闲链表时，结果有什么不同？哪些东西变了？
3. 改用首次适应（first fit，`-p FIRST`）呢？这时什么变快了？
4. 对于上面这些问题，链表的排序方式会影响某些策略找到空闲位置所需的时间。使用不同的空闲链表排序方式（`-l ADDRSORT`、`-l SIZESORT+`、`-l SIZESORT-`），看看这些策略和链表排序方式是如何相互作用的。
5. 空闲链表的合并可能相当重要。增加随机分配的次数（比如增加到 `-n 1000`）。随着时间推移，较大的分配请求会怎么样？分别在不合并和合并的情况下运行（即不带和带 `-C` 选项）。你看到结果有什么不同？每种情况下空闲链表随时间变得有多大？这时链表的排序方式有影响吗？
6. 把已分配比例 `-P` 调到 50 以上会怎么样？接近 100 时分配情况会怎样？接近 0 时呢？
7. 你可以发出什么样的特定请求，来产生高度碎片化（fragmented）的空闲空间？使用 `-A` 选项创建碎片化（fragmented）的空闲链表，看看不同的策略和选项如何改变空闲链表的组织方式。

 © 2008–23, ARPACI-DUSSEAU

# 18

# 分页：简介

有一种说法是，操作系统解决几乎任何空间管理问题时，都会采取两种方法之一。第一种是把空间切成大小可变的块，就像我们在虚拟内存（virtual memory）的**分段**（segmentation）中看到的那样。遗憾的是，这种方法本身就存在一些难题。具体来说，把空间划分成大小不同的块之后，空间本身会变得**碎片化**（fragmented），分配也会随着时间推移越来越困难。

所以，也许值得考虑第二种方法：把空间切成大小固定的块。在虚拟内存中，这种思想叫作**分页**（paging），它可以追溯到一个早期的重要系统 Atlas [KE+62, L78]。我们不再把进程（process）的地址空间（address space）划分成代码、堆（heap）、栈（stack）等若干个大小可变的逻辑段（logical segment），而是把它划分成大小固定的单元，每个单元叫作一**页**（page）。相应地，我们把物理内存（physical memory）看作由大小固定的槽组成的数组，这些槽叫作**页帧**（page frame）；每个页帧可以容纳虚拟内存的一个页。我们面临的挑战是：

关键问题：如何用页来虚拟化内存
如何用页来虚拟化（virtualize）内存，从而避免分段的那些问题？基本技术有哪些？如何让这些技术运行良好，同时把空间和时间开销降到最低？

## 18.1 一个简单的例子和概览

为了说得更清楚，我们来看一个简单的例子。图 18.1（第 164 页）展示了一个很小的地址空间，总共只有 64 字节，分成 0、1、2、3 四个虚拟页（virtual page），每页 16 字节。当然，真实的地址空间要大得多，通常是 32 位，也就是 4 GB 的地址空间，甚至是 64 位[1]；本书常用很小的例子，好让内容更容易理解。

如图 18.2 所示，物理内存同样由若干个大小固定的槽组成，这里是 8 个页帧，整个物理内存只有 128 字节，同样小得离谱。从图中可以看到，虚拟地址空间（virtual address space）的各个页被放在了物理内存中的不同位置；图中还显示，操作系统自己也占用了一部分物理内存。

[1] 64 位的地址空间大得惊人，很难想象。打个比方也许有帮助：如果把 32 位地址空间看作一个网球场那么大，64 位地址空间就大约有整个欧洲那么大（!）。

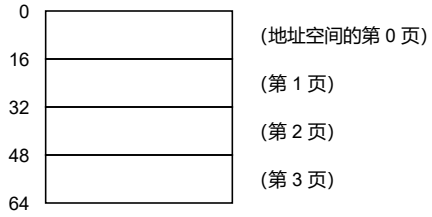



图 18.1：**一个简单的 64 字节地址空间**




图 18.2：**64 字节的地址空间放在 128 字节的物理内存中**

我们将会看到，与之前的方法相比，分页有许多优点。最重要的改进可能是*灵活性*：有了完善的分页机制，无论进程怎样使用地址空间，系统都能有效地支持地址空间这一抽象（abstraction）；例如，我们不必对堆和栈的增长方向以及它们的用法做任何假设。

另一个优点是分页让空闲空间管理（free-space management）变得很简单。例如，当操作系统想把这个 64 字节的小地址空间放进只有 8 个页帧的物理内存时，只需找到四个空闲页（free page）即可。操作系统也许为此维护了一个**空闲链表**（free list），记录所有空闲页，从中取出前四个就行了。在这个例子中，操作系统把地址空间的虚拟页 0 放在了物理帧（physical frame）3，虚拟页 1 放在物理帧 7，页 2 放在帧（frame）5，页 3 放在帧 2。页帧 1、4、6 目前是空闲的。

为了记录地址空间的每个虚拟页放在物理内存的什么位置，操作系统通常会为每个进程维护一个数据结构（data structure），叫作**页表**（page table）。页表的主要作用是保存地址空间中每个虚拟页的**地址转换**（address translation），让我们知道每个页位于物理内存的哪个位置。对于我们这个简单的例子（第 164 页的图 18.2），页表有以下四项：虚拟页 0 → 物理帧 3、VP 1 → PF 7、VP 2 → PF 5 和 VP 3 → PF 2。

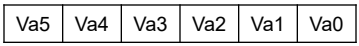
要记住，页表是每个进程各有一份的。我们讨论的页表结构大多如此，后面会提到一个例外：**反置页表**（inverted page table）。如果上面的例子中还有另一个进程要运行，操作系统就得为它管理另一张页表，因为它的虚拟页显然会映射到不同的物理页（physical page），除非有页面共享。

现在，我们已经可以动手做一个地址转换的例子了。假设那个只有 64 字节地址空间的小进程正在执行一次内存访问：

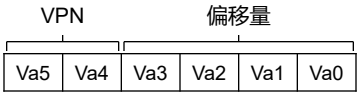
```
movl <virtual address>, %eax
```

具体来说，我们只关注从地址 `<virtual address>` 把数据显式加载到寄存器 `eax` 的过程（因此忽略在此之前必然发生过的取指令）。

为了**转换**（translate）进程产生的这个虚拟地址（virtual address），我们首先要把它拆成两部分：**虚拟页号**（virtual page number，VPN）和页内的**偏移量**（offset）。在这个例子中，进程的虚拟地址空间是 64 字节，所以虚拟地址一共需要 6 位（$2^6 = 64$）。因此，虚拟地址可以看成下面这样：




图中 Va5 是虚拟地址的最高位，Va0 是最低位。我们知道页大小（page size）是 16 字节，因此可以把虚拟地址进一步划分成这样：




地址空间 64 字节，页大小 16 字节，一共 4 个页；要从中选出一个，用地址的最高 2 位正好。因此，虚拟页号（VPN）是 2 位。剩下的位（这里是 4 位）告诉我们要访问页中的哪个字节，我们把它叫作偏移量。

进程产生一个虚拟地址时，操作系统和硬件必须相互配合，把它转换成有意义的物理地址（physical address）。例如，假设上面那条加载指令访问的是虚拟地址 21：

```
movl 21, %eax
```

把“21”写成二进制，得到“010101”。由此可以看出这个虚拟地址是怎样拆分成虚拟页号（VPN）和偏移量的：




所以，虚拟地址“21”位于虚拟页“01”（即 1）的第 5 个（第“0101”个）字节。有了虚拟页号，我们就可以用它来索引页表，找出虚拟页 1 位于哪个物理帧。在上面的页表中，它对应的**物理帧号**（physical frame number，**PFN**；有时也叫**物理页号**，physical page number，**PPN**）是 7（二进制 111）。于是，只需把 VPN 换成 PFN，就能完成这个虚拟地址的转换，然后向物理内存发出加载请求（图 18.3）。

注意，偏移量保持不变（也就是说，它不用转换），因为偏移量只是告诉我们要访问页内的哪个字节。最终的物理地址是 1110101（十进制 117），这正是我们想要加载数据的位置（第 164 页的图 18.2）。

有了这个基本概念，你对分页可能已经有了一些疑问，下面我们就把它们提出来（但愿也能解答）。例如，这些页表存放在哪里？页表里一般有什么，又有多大？分页会让系统变得（太）慢吗？对于这些以及其他一些耐人寻味的问题，下文至少会给出部分答案。接着往下读吧！

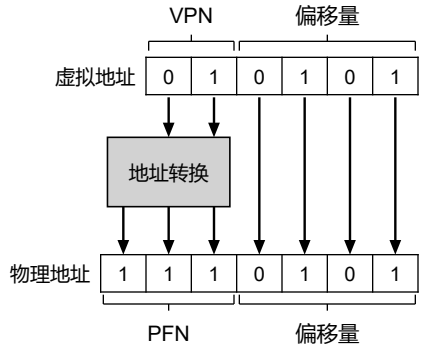



图 18.3：**地址转换过程**

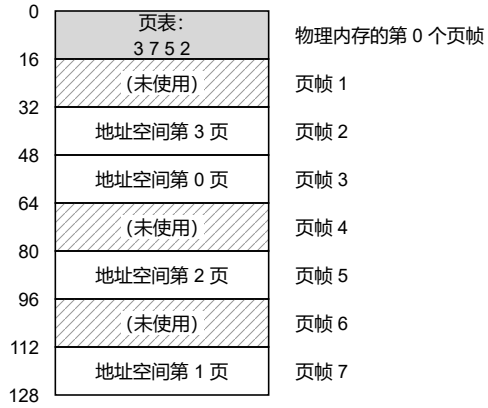



图 18.4：**示例：位于内核物理内存中的页表**

## 18.2 页表存放在哪里?

页表可能会变得非常大，比我们之前讨论过的小小的段表（segment table）或基址/界限寄存器对（base/bounds pair）大得多。例如，设想一个典型的 32 位地址空间，页大小为 4 KB。这样的虚拟地址会拆分成 20 位的 VPN 和 12 位的偏移量。回想一下，1 KB 的页需要 10 位偏移量，再加两位就是 4 KB。

补充：数据结构——页表

现代操作系统的内存管理子系统中，最重要的数据结构之一就是**页表**（page table）。一般来说，页表保存的是**虚拟地址到物理地址的转换**（virtual-to-physical address translation），让系统知道地址空间的每个页实际位于物理内存的什么地方。由于每个地址空间都需要这样的转换，系统中一般每个进程都有一张页表。页表的具体结构要么由硬件决定（较老的系统），要么可以由操作系统更灵活地管理（现代系统）。

 © 2008–25, ARPACI-DUSSEAU

20 位的 VPN 意味着操作系统要为每个进程管理 $2^{20}$ 个（大约一百万个）地址转换；假设每个**页表项**（page table entry，PTE）需要 4 个字节，用来保存物理转换以及其他有用的信息，那么每张页表就需要整整 4 MB 内存！这已经相当大了。再想象一下有 100 个进程在运行：这意味着光是这些地址转换，操作系统就需要 400 MB 内存！即使在机器动辄拥有好几 GB 内存的今天，拿出其中一大块只用来存放转换信息，似乎也有点疯狂，不是吗？至于 64 位地址空间的页表会有多大，我们连想都不敢想；那实在太可怕了，说不定会把你彻底吓跑。

正因为页表这么大，我们不会在 MMU 中设置专门的片上硬件来存放当前运行进程的页表，而是把每个进程的页表存放在内存中的某个地方。我们暂时假设页表位于操作系统管理的物理内存中；后面会看到，操作系统自己的内存有很大一部分也可以被虚拟化，因此页表可以存放在操作系统的虚拟内存中（甚至可以被交换到磁盘上），不过现在讲这些只会让人更糊涂，所以先不管它。图 18.4（第 166 页）展示了操作系统内存中的一张页表；看到里面那一小组转换了吗？

## 18.3 页表里究竟有什么？

我们来谈谈页表的组织方式。页表就是一种数据结构，用来把虚拟地址映射到物理地址，确切地说，是把虚拟页号映射到物理帧号（physical frame number）。所以，用什么数据结构都可以。最简单的形式叫作**线性页表**（linear page table），它就是一个数组。操作系统用虚拟页号（VPN）来索引这个数组，查找该索引处的页表项（PTE），从而找到所需的物理帧号（PFN）。目前我们先假设采用这种简单的线性结构；在后面的章节中，我们会用更高级的数据结构来解决分页的一些问题。

再来看每个 PTE 的内容。PTE 中有许多不同的位，都值得了解一下。常见的一种是**有效位**（valid bit），表示某个转换是否有效。例如，程序开始运行时，代码和堆位于地址空间的一端，栈位于另一端。中间所有未使用的空间都会被标记为**无效**（invalid）；如果进程试图访问这样的内存，就会陷入操作系统，而操作系统很可能会终止这个进程。因此，有效位对于支持稀疏地址空间至关重要：只需把地址空间中所有未使用的页标记为无效，就不必为这些页分配物理帧，从而节省大量内存。

PTE 中还可能有**保护位**（protection bits），表示这个页能否读取、写入或执行。同样，以这些位不允许的方式访问页，就会陷入操作系统。

还有几个位也很重要，不过暂时不细谈。**存在位**（present bit）表示这个页是在物理内存中，还是已被**换出**（swapped out）到磁盘上。等到我们研究如何把地址空间的一部分**交换**（swap）到磁盘、以支持比物理内存更大的地址空间时，会进一步了解这套机制；交换

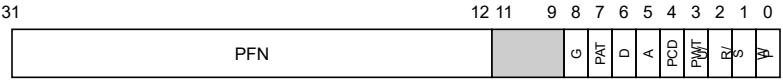



图 18.5：**一个 x86 页表项（PTE）**

补充：为什么没有有效位?

你可能会注意到，在 Intel 的例子中，没有单独的有效位和存在位（present bit），只有一个存在位（P）。如果这一位为 1（P=1），就表示这个页既存在又有效。如果为 0（P=0），则表示这个页可能不在内存中（但它是有效的），也可能是无效的。访问 P=0 的页会陷入操作系统；操作系统必须借助它自己维护的其他结构，判断这个页是有效的（那也许就应该把它换回内存），还是无效的（那就说明程序在试图非法访问内存）。这种精打细算在硬件中很常见：硬件往往只提供最基本的一组功能，操作系统在此基础上构建完整的服务。

让操作系统可以把很少使用的页移到磁盘上，从而腾出物理内存。**脏位**（dirty bit）也很常见，表示这个页自从调入内存以来是否被修改过。

**访问位**（**reference bit**，又称 **accessed bit**）有时用来跟踪一个页是否被访问过，可以帮助判断哪些页比较常用、应该留在内存中；这些信息在**页面置换**（page replacement）时至关重要，后续章节会详细研究这个话题。

图 18.5 展示了 x86 架构的一个页表项示例 [I09]。它包含一个存在位（P）；一个读/写位（R/W），决定是否允许写这个页；一个用户/超级用户位（user/supervisor bit，U/S），决定用户模式（user mode）的进程能否访问这个页；几个位（PWT、PCD、PAT 和 G），决定硬件如何缓存（cache）这些页；一个访问位（A）和一个脏位（D）；最后是页帧号（PFN）本身。

想了解 x86 分页支持的更多细节，请阅读 Intel 架构手册 [I09]。不过先提醒一句：这类手册信息量很大（要在操作系统里写代码使用这类页表，当然非读不可），但刚开始读可能比较吃力。你需要一点耐心，外加十足的求知欲。

## 18.4 分页：不仅太大，而且太慢

我们已经知道，放在内存中的页表可能太大。其实，它们还会拖慢速度。以这条简单的指令为例：

```
movl 21, %eax
```

同样，我们只看对地址 21 的显式引用，不考虑取指令。在这个例子中，我们假设由硬件替我们完成转换。要取出所需的数据，系统得先把虚拟地址 21 **转换**成正确的物理地址 117。也就是说，从地址 117 取数据之前，系统必须先从进程的页表中取出相应的页表项，完成转换，然后才能从物理内存加载数据。

为此，硬件必须知道当前运行进程的页表在哪里。我们暂时假设有一个**页表基址寄存器**（page-table base register），里面存放着页表起始位置的物理地址。为了找到所需 PTE 的位置，硬件会执行以下计算：

```
VPN      = (VirtualAddress & VPN_MASK) >> SHIFT
PTEAddr = PageTableBaseRegister + (VPN * sizeof(PTE))
```

```
1  // 从虚拟地址中提取 VPN
2  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4  // 计算页表项（PTE）的地址
5  PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7  // 取出 PTE
8  PTE = AccessMemory(PTEAddr)
9
10 // 检查进程能否访问这个页
11 if (PTE.Valid == False)
12     RaiseException(SEGMENTATION_FAULT)
13 else if (CanAccess(PTE.ProtectBits) == False)
14     RaiseException(PROTECTION_FAULT)
15 else
16     // 访问合法：计算物理地址并取数据
17     offset   = VirtualAddress & OFFSET_MASK
18     PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
19     Register = AccessMemory(PhysAddr)
```

图 18.6：**使用分页访问内存**

在我们的例子中，VPN_MASK 设为 0x30（十六进制 30，即二进制 110000），用来从完整的虚拟地址中挑出 VPN 位；SHIFT 设为 4，即偏移量的位数，这样就能把 VPN 位移到低位，得到正确的整数虚拟页号。例如，对于虚拟地址 21（010101），掩码运算把它变成 010000，移位再把它变成 01，也就是虚拟页 1，和预期一致。然后，我们用这个值作为索引，在页表基址寄存器（page-table base register）指向的 PTE 数组中查找。

知道了 PTE 的物理地址，硬件就可以从内存中把它取出来，从中提取 PFN，再把它和虚拟地址中的偏移量拼接起来，得到所需的物理地址。具体来说，可以理解为把 PFN 左移 SHIFT 位，再和偏移量按位或，就得到最终地址，如下所示：

```
offset   = VirtualAddress & OFFSET_MASK
PhysAddr = (PFN << SHIFT) | offset
```

最后，硬件就可以从内存中取出所需的数据，放进寄存器 eax。程序终于成功地从内存中加载了一个值！

总结一下，图 18.6（第 169 页）给出了每次内存引用时的初步协议（protocol）。每次内存引用（无论是取指令，还是显式的加载或存储），分页都要额外再做一次内存引用，先从页表中取出转换。这可是很大的工作量！额外的内存引用代价高昂，在这种情况下，很可能会让进程慢一倍甚至更多。

现在，你应该能看出我们必须解决的两个实实在在的问题了。如果不精心设计硬件和软件，页表既会让系统运行得太慢，又会占用太多内存。分页看起来是满足内存虚拟化需求的绝佳方案，但必须先克服这两个关键问题。

## 18.5 一次内存跟踪

在结束本章之前，我们来跟踪一个简单的内存访问例子，展示使用分页时产生的全部内存访问。我们关心的是下面这段 C 代码，放在一个名为 `array.c` 的文件中：

```
int array[1000];
...
for (i = 0; i < 1000; i++)
    array[i] = 0;
```

用下面的命令编译并运行 `array.c`：

```
prompt> gcc -o array array.c -Wall -O
prompt> ./array
```

当然，要真正弄清这段代码（它只是初始化一个数组）会产生哪些内存访问，我们还得知道（或者假设）一些其他信息。首先，我们得**反汇编**（disassemble）生成的二进制文件（在 Linux 上用 `objdump`，在 Mac 上用 `otool`），看看循环初始化数组用的是哪些汇编指令。下面是得到的汇编代码：

```
1024 movl $0x0,(%edi,%eax,4)
1028 incl %eax
1032 cmpl $0x03e8,%eax
1036 jne  1024
```

如果你懂一点 **x86**，这段代码其实很容易理解[2]。第一条指令把值 0（写作 `$0x0`）写到数组所在位置的虚拟内存地址；这个地址的计算方法是：`%edi` 的内容加上 `%eax` 乘以 4。因此，`%edi` 保存的是数组的基址（base），而 `%eax` 保存的是数组下标（`i`）；之所以乘以 4，是因为这是一个整数数组，每个元素占 4 个字节。

第二条指令把 `%eax` 中保存的数组下标加 1，第三条指令把这个寄存器的内容与十六进制值 `0x03e8`（即十进制 1000）比较。如果比较结果表明两个值还不相等（这正是 `jne` 指令要检测的），第四条指令就跳回循环的开头。

要弄清这段指令序列会产生哪些内存访问（虚拟的和物理的），还得做一些假设：代码和数组位于虚拟内存的什么位置，页表的内容是什么、放在哪里。

在这个例子中，我们假设虚拟地址空间大小为 64 KB（小得不现实），页大小为 1 KB。

现在只需要知道页表的内容和它在物理内存中的位置。假设页表是线性的（基于数组），位于物理地址 1 KB（1024）处。

至于页表的内容，这里只需关心少数几个虚拟页的映射。首先是代码所在的虚拟页。由于页大小为 1 KB，虚拟地址 1024 位于虚拟地址空间的第二个页（VPN=1，因为 VPN=0 是第一个页）。假设这个虚拟页映射到物理帧 4（VPN 1 → PFN 4）。

其次是数组本身。它的大小是 4000 字节（1000 个整数），我们假设它位于虚拟地址 40000 到 44000（不含最后一个字节）。这个十进制范围对应的虚拟页是 VPN=39 … VPN=42。因此，我们需要为这些页建立映射。在这个例子中，假设虚拟地址到物理地址的

[2]这里我们稍微作了点弊：为简单起见，假设每条指令都是 4 字节；实际上，x86 指令的长度是可变的。

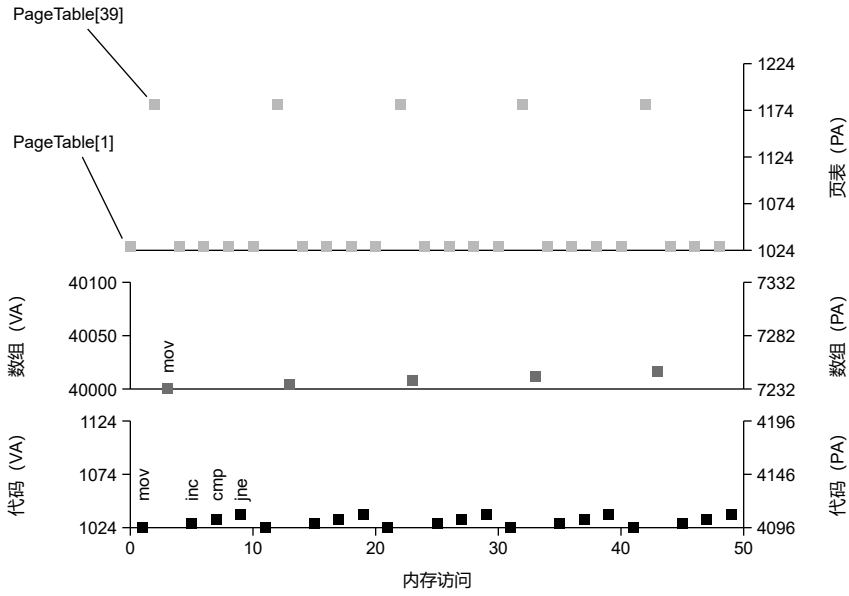



图 18.7：**一次虚拟（及物理）内存跟踪**

映射如下：（VPN 39 → PFN 7）、（VPN 40 → PFN 8）、（VPN 41 → PFN 9）、（VPN 42 → PFN 10）。

现在可以跟踪程序的内存引用了。程序运行时，每次取指令都会产生两次内存引用：一次访问页表，找到指令所在的物理帧；另一次访问指令本身，把它取到 CPU 中处理。此外，`mov` 指令还有一次显式的内存引用；这又会先增加一次页表访问，把数组的虚拟地址转换成正确的物理地址，然后才是对数组本身的访问。

图 18.7（第 171 页）描绘了前五次循环迭代的整个过程。最下面的图用黑色在 y 轴上标出指令的内存引用，左边是虚拟地址，右边是实际的物理地址；中间的图用深灰色标出数组访问，同样是左边虚拟地址、右边物理地址；最上面的图用浅灰色标出页表的内存访问，只有物理地址，因为在这个例子中页表位于物理内存中。x 轴是整个跟踪过程中循环前五次迭代的内存访问；每次循环有 10 次内存访问，包括四次取指令、一次显式的内存更新，以及五次页表访问（为前面这五次访问做地址转换）。

看看你能否理解这张图中呈现的规律。特别是，循环在前五次迭代之后继续运行时，会有什么变化？会访问哪些新的内存位置？你能想出来吗？

这只是最简单的例子（仅几行 C 代码），但你或许已能感到，理解真实应用的内存行为有多复杂。别担心：肯定会更糟，因为即将介绍的机制只会让本已复杂的系统更复杂。抱歉[3]！

[3]我们其实并不觉得抱歉。不过，我们为不觉得抱歉而感到抱歉，如果这说得通的话。

## 18.6 小结

本章介绍了**分页**（paging）的概念，它是应对内存虚拟化挑战的一种方案。与分段等之前的方法相比，分页有许多优点。首先，它不会产生外部碎片（external fragmentation），因为分页在设计上就把内存划成了大小固定的单元。其次，它很灵活，允许稀疏地使用虚拟地址空间。

然而，分页如果实现得不仔细，机器就会变慢（访问页表要多出很多次内存访问），内存也会被浪费（内存里塞满了页表，而不是有用的应用程序数据）。因此，我们得多想想，设计出既能用又好用的分页系统。好在下两章会讲怎么做。

 © 2008–25, ARPACI-DUSSEAU

## 参考文献

[KE+62] “One-level Storage System” by T. Kilburn, D.B.G. Edwards, M.J. Lanigan, F.H. Sumner. IRE Trans. EC-11, 2, 1962. Reprinted in Bell and Newell, “Computer Structures: Readings and Examples”. McGraw-Hill, New York, 1971. *The Atlas pioneered the idea of dividing memory into fixed-sized pages and in many senses was an early form of the memory-management ideas we see in modern computer systems.*

[I09] “Intel 64 and IA-32 Architectures Software Developer’s Manuals” Intel, 2009. Available: http://www.intel.com/products/processor/manuals. *In particular, pay attention to “Volume 3A: System Programming Guide Part 1” and “Volume 3B: System Programming Guide Part 2”.*

[L78] “The Manchester Mark I and Atlas: A Historical Perspective” by S. H. Lavington. Communications of the ACM, Volume 21:1, January 1978. *This paper is a great retrospective of some of the history of the development of some important computer systems. As we sometimes forget in the US, many of these new ideas came from overseas.*

 © 2008–25, ARPACI-DUSSEAU

## 作业（模拟）

在这个作业中，你将使用一个名为`paging-linear-translate.py`的简单程序，检验自己是否理解了：使用线性页表时，虚拟地址是怎样转换成物理地址的。详见README。

## 问题

1. 在做任何转换之前，我们先用模拟器研究一下，在不同参数下线性页表的大小如何变化。改变参数，算出相应的线性页表大小。下面是一些建议的输入；使用`-v`选项，可以看到有多少页表项被填充了。首先，为了理解线性页表的大小如何随地址空间的增长而变化，用下面这些选项运行：
   - `-P 1k -a 1m -p 512m -v -n 0`
   - `-P 1k -a 2m -p 512m -v -n 0`
   - `-P 1k -a 4m -p 512m -v -n 0`
2. 接下来，看看线性页表的大小如何随页大小的增长而变化。运行这些命令之前，先想想预期的趋势。随着地址空间的增长，页表大小应该如何变化？随着页大小的增长呢？为什么一般不使用很大的页？
   - `-P 1k -a 1m -p 512m -v -n 0`
   - `-P 2k -a 1m -p 512m -v -n 0`
   - `-P 4k -a 1m -p 512m -v -n 0`
3. 现在来做一些转换。从一些小例子开始，并用`-u`选项改变分配给地址空间的页数。增大每个地址空间中已分配页所占的比例，会发生什么？
   - `-P 1k -a 16k -p 32k -v -u 0`
   - `-P 1k -a 16k -p 32k -v -u 25`
   - `-P 1k -a 16k -p 32k -v -u 50`
   - `-P 1k -a 16k -p 32k -v -u 75`
   - `-P 1k -a 16k -p 32k -v -u 100`
4. 现在换换花样，试试一些不同的随机种子，以及一些不同的（有时相当疯狂的）地址空间参数。这些参数组合中，哪些是不现实的？为什么？
   - `-P 8 -a 32 -p 1024 -v -s 1`
   - `-P 8k -a 32k -p 1m -v -s 2`
   - `-P 1m -a 256m -p 512m -v -s 3`
5. 用这个程序试试其他一些问题。你能找出程序在什么情况下就不能正常工作了吗？例如，如果地址空间比物理内存大，会发生什么？

 © 2008–25, ARPACI-DUSSEAU

# 19

# 分页：更快的转换（TLB）

使用分页（paging）作为支持虚拟内存（virtual memory）的核心机制，可能会带来很高的性能开销。分页把地址空间（address space）切成很小的、大小固定的单元，也就是页（page），因此需要大量的映射信息。由于这些映射信息一般存放在物理内存（physical memory）中，从逻辑上讲，程序每产生一个虚拟地址（virtual address），分页都需要额外查一次内存。每次取指令或者显式加载、存储之前，都先去内存里取转换信息，这样做慢得令人无法接受。于是，我们的问题是：

关键问题：
如何加速地址转换
如何加速地址转换（address translation），尽量避免分页似乎必不可少的那次额外内存引用？需要什么样的硬件支持？操作系统需要怎样参与？

想让事情变快时，操作系统通常需要一些帮助。而帮助往往来自操作系统的老朋友：硬件。为了加速地址转换，我们要加入一个叫作**转换后备缓冲区**（translation-lookaside buffer，**TLB**，也常称快表）的部件 [CG68, C95]，之所以叫这个名字，是出于历史原因 [CP78]。TLB 是芯片上**内存管理单元**（memory-management unit，**MMU**）的一部分，它就是一个硬件**缓存**（cache），存放常用的虚拟地址到物理地址（physical address）的转换；所以，叫它**地址转换缓存**（address-translation cache）会更贴切。每次虚拟内存引用时，硬件都先检查 TLB，看里面有没有所需的转换；如果有，就（快速地）完成转换，无须查询存有全部转换的页表（page table）。TLB 对性能的影响极大，可以说，正是 TLB 让虚拟内存真正变得可行 [C95]。

## 19.1 TLB 的基本算法

图 19.1 粗略描述了硬件可能怎样处理虚拟地址转换，这里假设页表是一个简单的数组，即**线性页表**（linear page table），并且使用**硬件管理的 TLB**（hardware-managed TLB），即访问页表的大部分工作由硬件承担（下面会进一步解释）。

硬件遵循的算法是这样的：首先，从虚拟地址中提取虚拟页号（virtual page number，VPN），见图 19.1 的第 1 行；然后检查 TLB 中是否有这个 VPN 的转换（第 2 行）。如果有，就是一次 **TLB 命中**（TLB hit），说明 TLB 中存有这个转换。成功了！接下来，只要保护检

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB 命中
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset   = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                    // TLB 未命中
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19         RetryInstruction()
```

图 19.1：**TLB 控制流算法**

查没有失败（第 4 行），我们就可以从相应的 TLB 项中取出页帧号（PFN），把它和原虚拟地址中的偏移量（offset）拼接起来，得到所需的物理地址，然后访问内存（第 5–7 行）。

如果 CPU 在 TLB 中没有找到转换，即发生 **TLB 未命中**（TLB miss），就还有一些工作要做。在这个例子中，硬件会访问页表来查找转换（第 11–12 行）；如果进程（process）产生的虚拟内存引用有效且可以访问（第 13、15 行），就用这个转换更新 TLB（第 18 行）。这一系列操作代价高昂，主要是因为访问页表需要一次额外的内存引用（第 12 行）。最后，TLB 更新之后，硬件会重新执行这条指令；这一次，转换可以在 TLB 中找到，内存引用很快就能完成。

和所有缓存一样，TLB 也建立在这样一个前提上：在通常情况下，转换都能在缓存中找到（也就是命中）。如果命中，增加的开销就很小，因为 TLB 就在处理器核心附近，而且设计得非常快。如果未命中，就要付出分页的高昂代价：必须访问页表来查找转换，从而带来一次额外的内存引用（页表更复杂的话还会更多）。如果这种情况经常发生，程序很可能会明显变慢；与大多数 CPU 指令相比，内存访问的代价相当高，而 TLB 未命中会导致更多的内存访问。因此，我们希望尽可能避免 TLB 未命中。

## 19.2 示例：访问数组

为了把 TLB 的工作方式讲清楚，我们来跟踪一串简单的虚拟地址，看 TLB 怎样提升这串访问的性能。在这个例子中，假设内存中有一个由 10 个 4 字节整数组成的数组，起始虚拟地址是 100。再假设我们有一个很小的 8 位虚拟地址空间（virtual address space），页大小（page size）为 16 字节；这样，虚拟地址就分成 4 位的 VPN 和 4 位的偏移量：共有 16 个虚拟页（virtual page），每页 16 个字节。

图 19.2（第 177 页）展示了数组在系统的 16 个 16 字节页上的布局。可以看到，数组的第一个元素（a[0]）从 VPN=06、偏移量=04 处开始；那一页只能放下三个 4 字节整数。数组接着延伸到下一页（VPN=07），接下来的四个元素（a[3] ⋯ a[6]）都在这一页上。这个数组的最后三个元素（a[7] ⋯ a[9]）则位于地址空间的再下一页（VPN=08）。

现在考虑一个依次访问数组每个元素的简单循环，用 C 语言写出来大致是这样的：

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```

为简单起见，我们姑且假定这个循环只访问数组的内存（忽略变量 i 和 sum，以及指令本身）。访问数组的第一个元素（a[0]）时，CPU 会看到一次对虚拟地址 100 的加载。硬件从中提取出 VPN（VPN=06），用它在 TLB 中查找有效的转换。假设这是程序第一次访问这个数组，结果会是一次 TLB 未命中。

下一次访问的是 a[1]，这里有个好消息：TLB 命中！数组的第二个元素紧挨着第一个元素，和它在同一页上；访问第一个元素时已经访问过这一页，转换已经加载到 TLB 中了，所以这次能成功。访问 a[2] 同样成功（又一次命中），因为它也和 a[0]、a[1] 位于同一页。

遗憾的是，程序访问 a[3] 时，又遇到了一次 TLB 未命中。不过，接下来的几个元素（a[4] ⋯ a[6]）又都会在 TLB 中命中，因为它们都位于内存中的同一页。

最后，访问 a[7] 又引发了一次 TLB 未命中，这也是最后一次。硬件再次查询页表，弄清这个虚拟页在物理内存中的位置，并相应地更新 TLB。剩下的两次访问（a[8] 和 a[9]）都受益于这次 TLB 更新：硬件在 TLB 中查找它们的转换时，又得到两次命中。

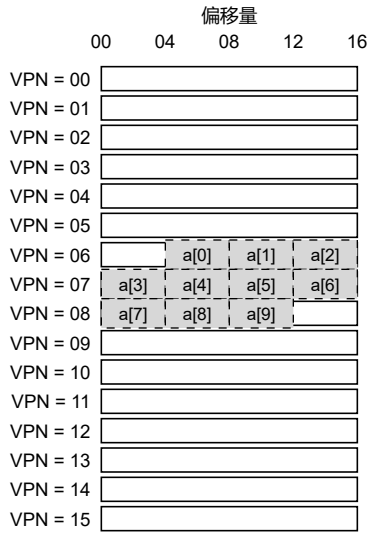



图 19.2：**示例：一个很小的地址空间中的数组**

提示：尽可能使用缓存

缓存是计算机系统中最基本的性能技术之一，人们一再用它来“让常见情况（common case）更快”[HP06]。硬件缓存背后的思想是利用指令和数据引用中的**局部性**（locality）。局部性通常有两种：**时间局部性**（temporal locality）和**空间局部性**（spatial locality）。时间局部性是指，最近访问过的指令或数据项，很可能很快会被再次访问。想想循环变量或循环中的指令：它们会在一段时间内被反复访问。空间局部性是指，如果程序访问了地址 x 处的内存，它很可能很快就会访问 x 附近的内存。想象一下顺序遍历某个数组，访问完一个元素再访问下一个。当然，这些性质取决于程序的具体特点，因此它们并不是铁律，更像是经验法则。

无论是用于指令、数据还是地址转换（比如我们的 TLB），硬件缓存利用局部性的办法都是把内存中的内容复制一份，放在容量小、速度快的片上存储器里。处理器不必非得去（缓慢的）内存里满足请求，而是可以先看看缓存中有没有就近的副本；如果有，处理器就能快速访问它（也就是只需几个 CPU 周期），避免花费访问内存所需的大量时间（许多纳秒）。

你可能会想：既然 TLB 这类缓存这么好，为什么不干脆把缓存做得更大，把所有数据都放进去呢？遗憾的是，这就会碰上物理定律这类更根本的限制。缓存要快，就必须小，因为光速之类的物理限制会起作用。大缓存注定是慢的，这就失去了缓存的意义。因此，我们只能用小而快的缓存；剩下的问题是，如何最好地利用它们来提升性能。

我们来总结一下这十次数组访问期间 TLB 的活动：**未命中**、命中、命中、**未命中**、命中、命中、命中、**未命中**、命中、命中。因此，TLB 的**命中率**（hit rate，即命中次数除以总访问次数）是 70%。我们其实希望命中率接近 100%，所以这个数字不算高，但它不是零，这可能有点出人意料。尽管这是程序第一次访问这个数组，得益于**空间局部性**（spatial locality），TLB 还是提升了性能。数组的元素紧密地排列在页中（也就是说，它们在**空间**上彼此靠近），因此只有第一次访问某一页上的元素时，才会出现 TLB 未命中。

还要注意页大小在这个例子中起的作用。哪怕页大小只是大一倍（32 字节而不是 16 字节），访问数组时的未命中次数也会更少。典型的页大小更接近 4 KB，因此这类密集的、基于数组的访问能获得极佳的 TLB 性能，每一页只会遇到一次未命中。

关于 TLB 的性能还有最后一点：如果程序在这个循环结束后不久再次访问这个数组，而 TLB 又足够大、能缓存所需的转换，我们很可能会看到更好的结果：命中、命中、命中、命中、命中、命中、命中、命中、命中、命中。在这种情况下，TLB 命中率（TLB hit rate）之所以高，是因为**时间局部性**（temporal locality），也就是内存项在**时间**上很快被再次引用。和任何缓存一样，TLB 的成功依赖于空间局部性和时间局部性，而这两者都是程序的性质。如果所关心的程序具有这样的局部性，TLB 命中率（TLB hit rate）就很可能很高；很多程序都是如此。

 © 2008–23, ARPACI-DUSSEAU

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)     // TLB 命中
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset   = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                      // TLB 未命中
11     RaiseException(TLB_MISS)
```

图 19.3：**TLB 控制流算法（由操作系统处理）**

## 19.3 谁来处理 TLB 未命中?

我们必须回答一个问题：谁来处理 TLB 未命中?可能的答案有两个：硬件，或者软件（操作系统）。早年间，硬件的指令集很复杂（有时称为 **CISC**，即复杂指令集计算机，complex-instruction set computer），而造硬件的人不太信任那些狡猾的操作系统开发者。因此，TLB 未命中完全交给硬件处理。为此，硬件必须确切地知道页表在内存中的位置（通过**页表基址寄存器**，page-table base register，图 19.1 的第 11 行就用到了它），以及页表的确切格式；发生未命中时，硬件会“遍历”页表，找到正确的页表项（page table entry）并取出所需的转换，用这个转换更新 TLB，然后重新执行指令。Intel x86 就是采用**硬件管理 TLB**（hardware-managed TLB）的“较老”架构之一：它使用固定的**多级页表**（multi-level page table，详见下一章），当前页表由 CR3 寄存器指向 [I09]。

更现代的架构（例如 MIPS R10k [H93] 或 Sun 的 SPARC v9 [WG00]，它们都是 **RISC**，即精简指令集计算机，reduced-instruction set computer）采用所谓的**软件管理的 TLB**（software-managed TLB）。发生 TLB 未命中时，硬件只是触发一个异常（图 19.3 的第 11 行），暂停当前的指令流，把特权级提升到内核模式（kernel mode），然后跳转到**陷阱处理程序**（trap handler）。你可能已经猜到，这个陷阱处理程序（trap handler）就是操作系统中专门为处理 TLB 未命中而编写的代码。这段代码运行时，会在页表中查找转换，用特殊的“特权”指令更新 TLB，然后从陷阱返回（return-from-trap）；此时，硬件会重新执行这条指令，这一次 TLB 就会命中。

我们来讨论几个重要的细节。第一，这里的从陷阱返回（return-from-trap）指令，要和之前处理系统调用（system call）时看到的从陷阱返回指令稍有不同。在系统调用的情况下，从陷阱返回（return-from-trap）应该从陷入操作系统那条指令的下一条指令处继续执行，就像从过程调用（procedure call）返回时，会回到紧跟在调用指令之后的那条指令一样。而在 TLB 未命中的情况下，从未命中处理的陷阱（trap）返回时，硬件必须从引发陷阱的那条指令处继续执行；这样重试（retry）一次，指令就会再运行一遍，这一次 TLB 就会命中。因此，根据陷阱（trap）或异常的引发方式，硬件在陷入操作系统时必须保存不同的 PC，这样到了该恢复的时候才能正确地继续执行。

补充：RISC 与 CISC

20 世纪 80 年代，计算机体系结构界发生了一场大论战。一方是 **CISC** 阵营，CISC 是**复杂指令集计算**（Complex Instruction Set Computing）的缩写；另一方是 **RISC**，即**精简指令集计算**（Reduced Instruction Set Computing）[PS81]。RISC 一方的领军人物是伯克利的 David Patterson 和斯坦福的 John Hennessy（两人也合写过几本著名的书 [HP06]），不过，后来因 RISC 方面的开创性工作而获得图灵奖（Turing Award）的是 John Cocke [CM00]。

CISC 指令集往往包含大量指令，而且每条指令的功能都相对强大。例如，你可能会看到一条字符串复制指令，它接受两个指针和一个长度，把字节从源地址复制到目标地址。CISC 背后的思想是，指令应该是高级原语，这样汇编语言本身更容易使用，代码也更紧凑。

RISC 指令集恰恰相反。RISC 背后的一个关键观察是：指令集其实是编译器的生成目标，而编译器真正想要的只是几个简单的原语，用来生成高性能的代码。因此，RISC 的支持者主张，尽可能多地把东西从硬件中去掉（尤其是微码），让剩下的部分简单、统一、快速。

早期，RISC 芯片影响巨大，因为它们明显更快 [BC91]；人们写了许多论文，还成立了几家公司（例如 MIPS 和 Sun）。然而，随着时间推移，Intel 等 CISC 厂商把许多 RISC 技术融入了处理器核心，例如在流水线前端增加几级，把复杂指令转换成微指令，再以类似 RISC 的方式处理。这些创新，再加上每块芯片上的晶体管越来越多，让 CISC 保持了竞争力。最终，这场争论平息了，如今两类处理器都能做得很快。

第二，运行 TLB 未命中处理代码时，操作系统要格外小心，不能引发无穷无尽的 TLB 未命中。解决办法有很多。例如，可以把 TLB 未命中处理程序放在物理内存中**不经映射**（unmapped）、不需要地址转换的区域；也可以在 TLB 中保留一些项给永久有效的转换，把其中一些槽位留给处理程序代码本身，这些**固定**（wired）转换总能在 TLB 中命中。

软件管理方法的主要优点是*灵活性*：操作系统可以用任何它想用的数据结构（data structure）来实现页表，而不必改动硬件。另一个优点是*简单*，这一点从 TLB 控制流就能看出来（对比图 19.3 中的第 11 行和图 19.1 中的第 11–19 行）。发生未命中时，硬件不用做太多事：只需触发一个异常，剩下的交给操作系统的 TLB 未命中处理程序即可。

## 19.4 TLB 的内容：里面有什么？

我们来更详细地看看硬件 TLB 的内容。典型的 TLB 可能有 32、64 或 128 项，并且采用所谓的**全相联**（fully associative）结构。简单来说，这意味着任何一个转换都可以放在 TLB 中的任何位置，硬件会并行搜索整个 TLB 来找到所需的转换。一个 TLB 项大概是这样的：

VPN | PFN | 其他位

注意，每一项中都同时有 VPN 和 PFN，因为一个转换可能被放在任何一个位置（用硬件术语来说，TLB 是一种**全相联缓存**，fully-associative cache）。硬件会并行搜索这些项，看有没有匹配的。

补充：TLB 的有效位 ≠ 页表的有效位

一个常见的错误是把 TLB 中的有效位（valid bit）和页表中的有效位混为一谈。在页表中，如果某个页表项（PTE）被标记为无效，就表示进程还没有分配这个页，正常工作的程序不应该访问它。访问无效页时，通常的反应是陷入操作系统，由操作系统杀死这个进程。

相比之下，TLB 的有效位只是表示某个 TLB 项中是否存有有效的转换。例如，系统启动时，TLB 项通常一开始都是无效的，因为那里还没有缓存任何地址转换。一旦启用了虚拟内存，程序开始运行并访问各自的虚拟地址空间，TLB 就会逐渐被填充，很快就满是有效项了。

TLB 的有效位在执行上下文切换（context switch）时也很有用，下面会进一步讨论。把所有 TLB 项都设为无效，系统就能确保即将运行的进程不会误用前一个进程的虚拟地址到物理地址的转换。

更有意思的是“其他位”。例如，TLB 通常有一个**有效**（valid）位，表示这一项中是否存有有效的转换。另一类常见的是**保护**（protection）位，决定可以怎样访问一个页，这和页表中一样。例如，代码页（code page）可能被标记为可读、可执行，而堆页（heap page）可能被标记为可读、可写。此外，可能还有其他几个字段，包括**地址空间标识符**（address-space identifier）、**脏位**（dirty bit）等等；更多信息见下文。

## 19.5 TLB 的问题：上下文切换

有了 TLB，在进程之间，也就是在地址空间之间切换时，就会出现新问题。具体来说，TLB 中存放的虚拟地址到物理地址的转换，只对当前运行的进程有效；这些转换对其他进程没有意义。因此，从一个进程切换到另一个进程时，硬件或操作系统（或两者一起）必须小心，确保即将运行的进程不会误用之前运行的某个进程的转换。

为了更好地理解这种情况，我们来看一个例子。一个进程 P1 在运行时，会假定 TLB 中可能缓存着对它有效的转换，也就是来自 P1 页表的转换。在这个例子中，假设 P1 的第 10 个虚拟页映射到物理帧（physical frame）100。

再假设还有另一个进程 P2，操作系统可能很快就会决定进行上下文切换，转而运行它。假设 P2 的第 10 个虚拟页映射到物理帧 170。如果两个进程的项都在 TLB 中，TLB 的内容就会是这样：

| VPN | PFN | 有效 | 保护 |
|---|---|---|---|
| 10 | 100 | 1 | rwx |
| — | — | 0 | — |
| 10 | 170 | 1 | rwx |
| — | — | 0 | — |

在上面的 TLB 中，问题显而易见：VPN 10 既可能转换成 PFN 100（P1），也可能转换成 PFN 170（P2），但硬件无法分辨哪一项属于哪个进程。因此，要让 TLB 正确、高效地支持多个进程的虚拟化（virtualization），我们还得多做些工作。于是，关键问题来了：

 © 2008–23, ARPACI-DUSSEAU

关键问题：
上下文切换时如何管理 TLB 的内容
在进程之间进行上下文切换时，TLB 中上一个进程的转换对即将运行的进程没有意义。为了解决这个问题，硬件或操作系统应该怎么做？

这个问题有许多可能的解决办法。一种方法是在上下文切换时直接**刷新**（flush）TLB，在运行下一个进程之前把它清空。在基于软件的系统中，可以用一条显式的（特权）硬件指令来完成刷新；在硬件管理 TLB 的系统中，可以在页表基址寄存器（page-table base register）改变时执行刷新；注意，操作系统在上下文切换时反正都要修改 PTBR。无论哪种情况，刷新操作都只是把所有有效位设为 0，本质上就是清空 TLB 的内容。

在每次上下文切换时刷新 TLB，就得到了一个可行的方案，因为进程永远不会意外地用到 TLB 中错误的转换。然而，这是有代价的：进程每次运行时，访问自己的数据页（data page）和代码页都必然会遇到 TLB 未命中。如果操作系统频繁地在进程之间切换，这个代价可能会很高。

为了降低这种开销，有些系统增加了硬件支持，让 TLB 可以跨上下文切换共享。具体来说，有些硬件系统在 TLB 中提供了**地址空间标识符**（address space identifier，**ASID**）字段。可以把 ASID 看作**进程标识符**（process identifier，**PID**），只是它的位数通常更少（例如 ASID 是 8 位，而 PID 是 32 位）。

如果给上面那个 TLB 示例加上 ASID，显然进程就能很方便地共享 TLB 了：只需 ASID 字段，就能区分原本完全相同的转换。下面是加上 ASID 字段后的 TLB：

| VPN | PFN | 有效 | 保护 | ASID |
|---|---|---|---|---|
| 10 | 100 | 1 | rwx | 1 |
| — | — | 0 | — | — |
| 10 | 170 | 1 | rwx | 2 |
| — | — | 0 | — | — |

因此，有了地址空间标识符（address space identifier），TLB 就能同时存放来自不同进程的转换，而不会混淆。当然，硬件还需要知道当前运行的是哪个进程，才能执行转换，所以操作系统在上下文切换时，必须把某个特权寄存器设为当前进程的 ASID。

顺便说一下，你可能还想到了另一种情况：TLB 中有两项非常相似。在下面这个例子中，两个不同进程的两项，VPN 不同，却指向同一个物理页（physical page）：

| VPN | PFN | 有效 | 保护 | ASID |
|---|---|---|---|---|
| 10 | 101 | 1 | r-x | 1 |
| — | — | 0 | — | — |
| 50 | 101 | 1 | r-x | 2 |
| — | — | 0 | — | — |

例如，两个进程共享一个页（比如代码页）时，就可能出现这种情况。在上面的例子中，进程 1 和进程 2 共享物理页 101；P1 把这个页映射到它地址空间的第 10 页，而 P2 把

 © 2008–23, ARPACI-DUSSEAU

它映射到自己地址空间的第 50 页。在二进制程序或共享库中共享代码页很有用，因为它可以减少使用中的物理页数量，从而降低内存开销。

## 19.6 问题：替换策略

和任何缓存一样，TLB 还需要考虑另一个问题：**缓存替换**（cache replacement）。具体来说，往 TLB 中放入新项时，必须**替换**（replace）掉一个旧项，于是问题来了：替换哪一个？

关键问题：如何设计 TLB 替换策略
往 TLB 中加入新项时，应该替换哪一项？当然，目标是让**未命中率**（miss rate）尽可能低，或者说让**命中率**（hit rate）尽可能高，从而提升性能。

等到研究把页交换到磁盘的问题时，我们会比较详细地讨论这类策略；这里只简单介绍几种典型的策略。一种常见的方法是淘汰（evict）**最近最少使用**（least-recently-used，**LRU**）的项。LRU 试图利用内存引用流中的局部性，它假设最近没有被使用过的项很可能就是合适的淘汰对象。另一种典型的方法是采用**随机**（random）策略，随机淘汰一个 TLB 映射。这种策略简单，而且能避免一些极端情况下的反常表现，因而很有用；例如，当程序在大小为 $n$ 的 TLB 上循环访问 $n+1$ 个页时，像 LRU 这样“合理”的策略就会表现得极不合理：LRU 每次访问都会未命中，而随机策略要好得多。

## 19.7 一个真实的 TLB 项

最后，我们简单看一个真实的 TLB。这个例子来自 MIPS R4000 [H93]，它是一个使用软件管理 TLB 的现代系统；图 19.4 展示了一个稍作简化的 MIPS TLB 项。

MIPS R4000 支持 32 位地址空间，页大小为 4 KB。因此可以预料，典型的虚拟地址会分成 20 位的 VPN 和 12 位的偏移量。然而，从图中的 TLB 项可以看到，VPN 只有 19 位；原来，用户地址只会来自地址空间的一半（另一半留给内核），因此只需要 19 位的 VPN。VPN 可以转换成最多 24 位的物理帧号（PFN），因此可以支持最多 64 GB（物理）主存的系统（$2^{24}$ 个 4 KB 的页）。

MIPS TLB 中还有其他几个有意思的位。其中有一个*全局位*（global，G），用于在进程之间全局共享的页；如果设置了全局位，ASID 就会被忽略。还有 8 位的 *ASID*，操作系统可以用它来区分不同的地址空间，这在前面讲过。给你提个问题：如果同时运行的进程超过 256（$2^8$）个，操作系统该怎么办？此外还有 3 个*一致性*（Coherence，C）位，决定硬件如何缓存这个页，这有点超出本书的范围；一个*脏位*，页被写过之后置位（后面会看到它的用途）；一个*有效位*，告诉硬件这一项中是否存有有效的转换。另外还有一个*页掩码*（page mask）字段（图中没有画出），用于支持多种页大小；后面我们会看到为什么更大的页可能有用。最后，64 位中有一些位没有使用（图中灰色部分）。

MIPS 的 TLB 通常有 32 或 64 个这样的项，其中大多数由运行中的用户进程使用。不过，也有少数几项是留给操作系统的。操作系统可以设置一个 *wired* 寄存器，告诉硬件要为

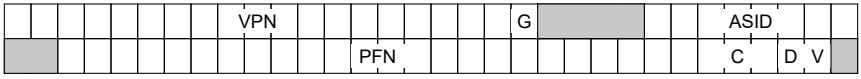



图 19.4：**一个 MIPS TLB 项**

提示：RAM 不总是 RAM（Culler 定律）

**随机存取存储器**（random-access memory，**RAM**）这个名字意味着，访问 RAM 的任何部分都一样快。一般来说，这样看待 RAM 没什么问题；但由于 TLB 这类硬件和操作系统特性，访问内存中的某个页可能代价高昂，特别是当这个页当前没有被 TLB 映射时。因此，最好时刻记住这条实现上的提示：**RAM 不总是 RAM**。有时随机访问（random access）地址空间，尤其是访问的页数超过了 TLB 的覆盖范围时，可能会造成严重的性能损失。由于我们的一位导师 David Culler 以前总说 TLB 是许多性能问题的根源，我们以他的名字命名这条定律：**Culler 定律**（Culler's Law）。

操作系统保留多少个 TLB 槽位。这些保留的映射用于操作系统在关键时刻要访问的代码和数据；在这种时刻发生 TLB 未命中会很麻烦（例如在 TLB 未命中处理程序中）。

由于 MIPS 的 TLB 是由软件管理的，就得有用来更新 TLB 的指令。MIPS 提供了四条这样的指令：TLBP，探查 TLB，看某个转换是否在其中；TLBR，把一个 TLB 项的内容读到寄存器中；TLBWI，替换指定的 TLB 项；以及 TLBWR，替换一个随机的 TLB 项。操作系统用这些指令来管理 TLB 的内容。当然，这些指令必须是**特权**指令，这一点至关重要；想象一下，如果用户进程能修改 TLB 的内容，它能做出什么事来（提示：几乎什么都能做，包括接管机器、运行它自己的恶意“操作系统”，甚至让太阳（Sun）消失）。

## 19.8 小结

我们已经看到，硬件可以帮助我们加快地址转换。有了一个小巧的专用片上 TLB 作为地址转换缓存（address-translation cache），大多数内存引用有望不必访问主存中的页表就能完成。因此，在通常情况下，程序运行起来几乎和内存根本没被虚拟化时一样快；这对操作系统来说是一项了不起的成就，当然也是现代系统能够使用分页的必要前提。

然而，TLB 并不能让所有程序都一帆风顺。具体来说，如果程序在短时间内访问的页数超过了 TLB 能容纳的页数，程序就会产生大量的 TLB 未命中，从而运行得慢很多。我们把这种现象称为超出 **TLB 覆盖范围**（TLB coverage），对某些程序来说，这可能是个大问题。一种解决办法是支持更大的页，下一章会讨论：把关键的数据结构放到程序地址空间中用大页（large page）映射的区域，就可以增大 TLB 的有效覆盖范围。**数据库管理系统**（database management system，**DBMS**）这类程序经常利用大页（large page）支持，因为它们有一些既庞大、又被随机访问的数据结构。

还有一个 TLB 问题值得一提：TLB 访问很容易成为 CPU 流水线中的瓶颈，在使用所谓**物理索引缓存**（physically-indexed cache）时尤其如此。对于这种缓存，地址转换必须在访问缓存之前完成，这可能会明显拖慢速度。由于存在这个潜在的问题，人们研究了各

 © 2008–23, ARPACI-DUSSEAU

种巧妙的方法，用虚拟地址来访问缓存，从而在缓存命中（cache hit）时省去代价高昂的转换步骤。这种**虚拟索引缓存**（virtually-indexed cache）解决了一些性能问题，但也给硬件设计带来了新问题。更多细节请参阅 Wiggins 出色的综述 [W03]。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[BC91] “Performance from Architecture: Comparing a RISC and a CISC with Similar Hardware Organization” by D. Bhandarkar and Douglas W. Clark. Communications of the ACM, September 1991. *A great and fair comparison between RISC and CISC. The bottom line: on similar hardware, RISC was about a factor of three better in performance.*

[CM00] “The evolution of RISC technology at IBM” by John Cocke, V. Markstein. IBM Journal of Research and Development, 44:1/2. *A summary of the ideas and work behind the IBM 801, which many consider the first true RISC microprocessor.*

[C95] “The Core of the Black Canyon Computer Corporation” by John Couleur. IEEE Annals of History of Computing, 17:4, 1995. *In this fascinating historical note, Couleur talks about how he invented the TLB in 1964 while working for GE, and the fortuitous collaboration that thus ensued with the Project MAC folks at MIT.*

[CG68] “Shared-access Data Processing System” by John F. Couleur, Edward L. Glaser. Patent 3412382, November 1968. *The patent that contains the idea for an associative memory to store address translations. The idea, according to Couleur, came in 1964.*

[CP78] “The architecture of the IBM System/370” by R.P. Case, A. Padegs. Communications of the ACM. 21:1, 73-96, January 1978. *Perhaps the first paper to use the term* **translation lookaside buffer**. *The name arises from the historical name for a cache, which was a* **lookaside buffer** *as called by those developing the Atlas system at the University of Manchester; a cache of address translations thus became a* **translation lookaside buffer**. *Even though the term lookaside buffer fell out of favor, TLB seems to have stuck, for whatever reason.*

[H93] “MIPS R4000 Microprocessor User’s Manual”. by Joe Heinrich. Prentice-Hall, June 1993. Available: http://cag.csail.mit.edu/raw/documents/R4400_Uman_book_Ed2.pdf *A manual, one that is surprisingly readable. Or is it?*

[HP06] “Computer Architecture: A Quantitative Approach” by John Hennessy and David Patterson. Morgan-Kaufmann, 2006. *A great book about computer architecture. We have a particular attachment to the classic first edition.*

[I09] “Intel 64 and IA-32 Architectures Software Developer’s Manuals” by Intel, 2009. Available: http://www.intel.com/products/processor/manuals. *In particular, pay attention to “Volume 3A: System Programming Guide” Part 1 and “Volume 3B: System Programming Guide Part 2”.*

[PS81] “RISC-I: A Reduced Instruction Set VLSI Computer” by D.A. Patterson and C.H. Sequin. ISCA ’81, Minneapolis, May 1981. *The paper that introduced the term RISC, and started the avalanche of research into simplifying computer chips for performance.*

[SB92] “CPU Performance Evaluation and Execution Time Prediction Using Narrow Spectrum Benchmarking” by Rafael H. Saavedra-Barrera. EECS Department, University of California, Berkeley. Technical Report No. UCB/CSD-92-684, February 1992.. *A great dissertation about how to predict execution time of applications by breaking them down into constituent pieces and knowing the cost of each piece. Probably the most interesting part that comes out of this work is the tool to measure details of the cache hierarchy (described in Chapter 5). Make sure to check out the wonderful diagrams therein.*

[W03] “A Survey on the Interaction Between Caching, Translation and Protection” by Adam Wiggins. University of New South Wales TR UNSW-CSE-TR-0321, August, 2003. *An excellent survey of how TLBs interact with other parts of the CPU pipeline, namely hardware caches.*

[WG00] “The SPARC Architecture Manual: Version 9” by David L. Weaver and Tom Germond. SPARC International, San Jose, California, September 2000. Available: www.sparc.org/standards/SPARCV9.pdf. *Another manual. I bet you were hoping for a more fun citation to end this chapter.*

## 作业（测量）

在这个作业中，你要测量 TLB 的大小以及访问 TLB 的代价。这个想法基于 Saavedra-Barrera 的工作 [SB92]，他提出了一种简单而优美的方法，只用一个非常简单的用户级程序，就能测量缓存层次结构的许多方面。详情可以读他的论文。

基本思路是访问一个大型数据结构（例如数组）中的若干个页，并为这些访问计时。例如，假设某台机器的 TLB 大小恰好是 4（这非常小，但便于讨论）。如果你写一个程序，只访问 4 个或更少的页，每次访问都应该是 TLB 命中，因此相对较快。然而，一旦在循环中反复访问 5 个或更多的页，每次访问的代价就会突然跳升到 TLB 未命中的水平。

遍历一次数组的基本代码大致如下：

```
int jump = PAGESIZE / sizeof(int);
for (i = 0; i < NUMPAGES * jump; i += jump)
    a[i] += 1;
```

这个循环会更新数组 a 每一页中的一个整数，总共更新 NUMPAGES 个页。把这样的循环反复运行并计时（比如在外面再套一层循环，重复几亿次，或者重复到足够运行几秒钟为止），就能算出每次访问（平均）需要多长时间。随着 NUMPAGES 的增加，观察代价在哪里出现跳变，就可以大致确定第一级 TLB 有多大，判断是否存在第二级 TLB（如果存在，它有多大），并大致了解 TLB 命中和未命中会如何影响性能。

图 19.5（第 187 页）显示了随着循环中访问的页数增加，每次访问的平均时间如何变化。从图中可以看到，只访问少数几个（8 个或更少）页时，平均访问时间大约是 5 纳秒。访问 16 个或更多的页时，每次访问的时间会突然跳到大约 20 纳秒。最后一次代价跳变出现在大约 1024 个页时，此时每次访问大约需要 70 纳秒。从这些数据可以得出结论：TLB 分为两级；第一级很小（大概能容纳 8 到 16 项）；第二级更大但更慢（大约能容纳 512 项）。第一级 TLB 命中和 TLB 未命中的代价总体相差很大，大约有 14 倍。TLB 性能很重要！

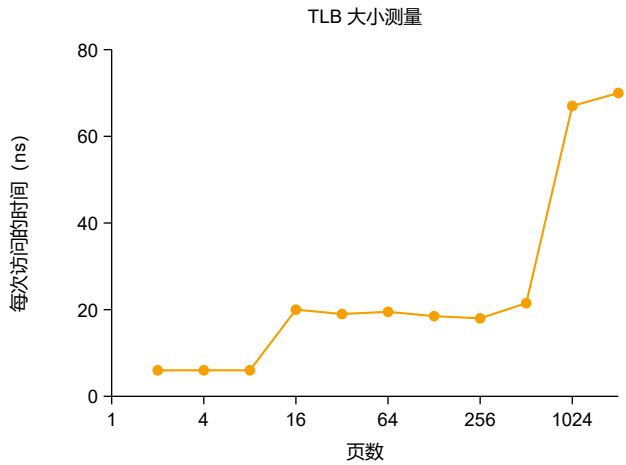



图 19.5：**测出 TLB 的大小和未命中代价**

## 问题

1. 计时需要用到计时器（例如 `gettimeofday()`）。这种计时器的精度如何？一个操作要持续多长时间，你才能精确地为它计时？（这有助于确定你需要在循环中把页访问重复多少次，才能成功计时。）
2. 编写一个名为 `tlb.c` 的程序，大致测量访问每个页的代价。程序的输入应该是：要访问的页数和试验次数。
3. 现在用你喜欢的脚本语言（bash?）写一个脚本来运行这个程序，让访问的页数从 1 逐步增加到几千，比如每次翻一倍。在不同的机器上运行这个脚本，收集一些数据。需要多少次试验才能得到可靠的测量结果？
4. 接下来，把结果画成图，样子和上面那张差不多。用个好用的工具，比如 ploticus，甚至 zplot。可视化通常能让数据容易理解得多；你觉得这是为什么？
5. 需要注意的一点是编译器优化。编译器会做各种巧妙的事情，比如某个循环递增的值如果之后再也没有被程序的其他部分用到，编译器就会把这个循环删掉。你怎样才能确保编译器不会把上面的主循环从你的 TLB 大小估算程序中删掉？
6. 另一点需要注意的是，如今大多数系统都配有多个 CPU，当然，每个 CPU 都有自己的 TLB 层次结构。要想真正得到好的测量结果，你必须让代码只在一个 CPU 上运行，而不是让调度程序（scheduler）把它在 CPU 之间来回迁移（migration）。怎样才能做到这一点？（提示：在 Google 上搜索“pinning a thread”找些线索。）如果不这样做，代码从一个 CPU 移到另一个 CPU 上，会发生什么？
7. 另一个可能出现的问题与初始化有关。如果在访问上面的数组 `a` 之前没有初始化它，第一次访问的代价会非常高，因为有按需置零（demand zeroing）这类初次访问的开销。这会影响你的代码及其计时吗？你能做些什么来抵消这些潜在的开销？

 © 2008–23, ARPACI-DUSSEAU

# 20

# 分页：更小的页表

现在我们来解决分页（paging）带来的第二个问题：页表（page table）太大，占用的内存太多。先从线性页表（linear page table）说起。你也许还记得[1]，线性页表会变得相当大。我们再假设一个 32 位（$2^{32}$ 字节）的地址空间（address space），页大小（page size）为 4 KB（$2^{12}$ 字节），每个页表项（page table entry）占 4 字节。这样，一个地址空间里大约有一百万（$2^{32} / 2^{12}$）个虚拟页（virtual page）；再乘以页表项的大小，就会发现页表足有 4 MB。还要记得：系统中通常每个进程（process）都有一张页表！如果有一百个活跃进程（这在现代系统上并不稀奇），光是页表就要占用几百 MB 内存！因此，我们需要一些技术来减轻这个沉重的负担。这类技术有很多，我们这就开始。不过，先来看看关键问题：

关键问题：如何让页表更小?
简单的基于数组的页表（通常称为线性页表）太大，在典型的系统上会占用太多内存。怎样才能让页表更小？关键思路是什么？这些新的数据结构（data structure）又会带来哪些低效之处?

## 20.1 简单的办法：更大的页

有一个简单的办法可以减小页表：用更大的页。还是以 32 位地址空间为例，但这次假设页大小为 16 KB。这样，VPN 就是 18 位，偏移量（offset）是 14 位。假设每个 PTE 的大小不变（4 字节），线性页表现在有 $2^{18}$ 个条目，每张页表总共 1 MB，缩小到原来的四分之一。不出所料，页表缩小的倍数正好等于页大小增大的倍数。

但这种方法的主要问题是，大页（large page）会导致每一页内部出现浪费，这个问题叫作**内部碎片**（internal fragmentation），因为浪费发生在分配单元的**内部**。结果，应用程序分配了一页又一页，却只用到每页的一小部分，内存很快就被这些过大的页占满。因此，大多数系统在通常情况下都使用比较小的页：4 KB（如 x86）或 8 KB（如 SPARCv9）。唉，我们的问题没法这么简单地解决。

[1]当然，你也可能不记得了；分页这东西是不是有点失控了？话虽如此，在着手解决之前，一定要先弄清楚你要解决的是什么问题；事实上，只要理解了问题，往往就能自己推导出解决办法。这里的问题应该很清楚：简单的线性（基于数组的）页表太大了。

补充：多种页大小

顺便提一下，现在许多体系结构（例如 MIPS、SPARC、x86-64）都支持多种页大小。通常用的是较小的页（page），4 KB 或 8 KB。不过，如果某个“聪明”的应用程序提出要求，也可以让地址空间中的某一部分使用一个大页（large page，例如 4 MB），这样应用程序就能把某个经常使用的（大型）数据结构放在这里，而只占用一个 TLB 项。这种大页（large page）的用法在数据库管理系统（database management system）和其他高端商业应用中很常见。不过，支持多种页大小的主要原因并不是节省页表空间，而是减轻 TLB 的压力，让程序能访问更多的地址空间，又不至于产生太多 TLB 未命中（TLB miss）。但研究人员也指出 [N+02]，使用多种页大小会让操作系统的虚拟内存（virtual memory）管理器明显变得更复杂，因此有时最省事的做法，就是给应用程序提供一个新接口，让它们直接申请大页（large page）。

## 20.2 混合方案：分页和分段

生活中，只要某件事有两种合理但不同的做法，你就应该看看能不能把两者结合起来，做到两全其美。我们把这种结合称为**混合方案**（hybrid）。比如说，既然可以把巧克力和花生酱结合成一种美妙的混合体，也就是 Reese's 花生酱杯 [M28]，何必只吃巧克力或者只吃花生酱呢?

多年前，Multics 的设计者们（特别是 Jack Dennis）在构建 Multics 虚拟内存系统时，偶然想到了这样一个主意 [M07]。具体来说，Dennis 想到把分页和分段（segmentation）结合起来，以减少页表的内存开销。仔细看看一个典型的线性页表，就能明白为什么这样可行。假设某个地址空间里，堆（heap）和栈（stack）实际用到的部分都很小。我们用一个很小的 16 KB 地址空间来举例，页大小为 1 KB（图 20.1）；这个地址空间的页表如图 20.2 所示。

这个例子假设唯一的代码页（VPN 0）映射到物理页（physical page）10，唯一的堆页（VPN 4）映射到物理页 23，地址空间另一端的两个栈页（VPN 14 和 15）分别映射到物理页 28 和 4。从图中可以看出，页表的大部分都没有用上，全是**无效**（invalid）条目。真是浪费！而这还只是一个很小的 16 KB 地址空间。想象一下 32 位地址空间的页表，里面可能浪费掉多少空间！算了，还是别想了，太可怕了。

于是有了我们的混合方案：与其为进程的整个地址空间准备一张页表，为什么不给每个逻辑段（logical segment）各准备一张呢？在这个例子中，我们就可以有三张页表，分别对应地址空间中的代码、堆和栈部分。

回想一下，在分段中，我们有一个**基址**（base）寄存器，告诉我们每个段在物理内存（physical memory）中的位置，还有一个**界限**（bound）或**限制**（limit）寄存器，告诉我们这个段的大小。在混合方案中，MMU 里仍然有这些结构；只不过这里的基址寄存器（base register）不再指向段（segment）本身，而是保存该段页表的*物理地址*（physical address）。界限寄存器（bounds register）则用来表示页表的末尾（即页表中有多少个有效页）。

我们用一个简单的例子来说明。假设有一个 32 位的虚拟地址空间（virtual address space），页大小为 4 KB，地址空间分成四个段。在这个例子中我们只用三个段：一个用于代码，一个用于堆，一个用于栈。

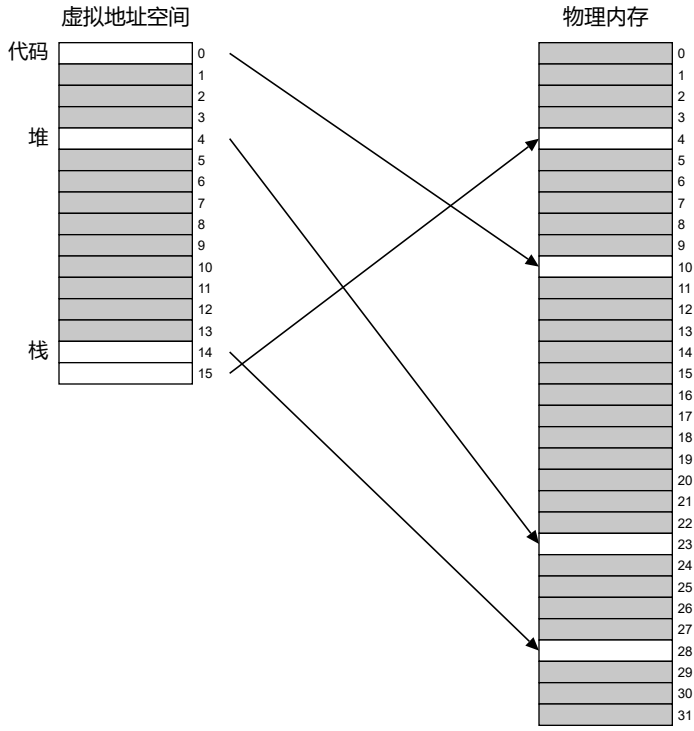
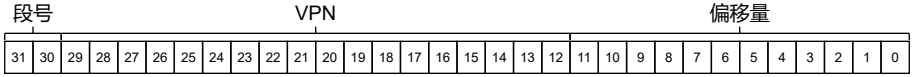



图 20.1：**页大小为 1 KB 的 16 KB 地址空间**

| PFN | 有效位 | 保护位 | 存在位 | 脏位 |
|---|---|---|---|---|
| 10 | 1 | r-x | 1 | 0 |
| - | 0 | — | - | - |
| - | 0 | — | - | - |
| - | 0 | — | - | - |
| 23 | 1 | rw- | 1 | 1 |
| - | 0 | — | - | - |
| - | 0 | — | - | - |
| - | 0 | — | - | - |
| - | 0 | — | - | - |
| - | 0 | — | - | - |
| - | 0 | — | - | - |
| - | 0 | — | - | - |
| - | 0 | — | - | - |
| - | 0 | — | - | - |
| 28 | 1 | rw- | 1 | 1 |
| 4 | 1 | rw- | 1 | 1 |

图 20.2：**16 KB 地址空间的页表**

为了确定一个地址属于哪个段，我们用地址空间的最高两位来区分。假设 00 是未使用的段，01 表示代码，10 表示堆，11 表示栈。这样，虚拟地址（virtual address）的格式如下：




因此，假设硬件中有三对基址/界限寄存器（base/bounds register），代码、堆和栈各一对。进程运行时，每个段的基址寄存器都保存着该段线性页表的物理地址；这样，系统中的每个进程现在都关联着三张页表。发生上下文切换（context switch）时，必须修改这些寄存器，让它们指向新运行进程的页表所在的位置。

发生 TLB 未命中时（假设 TLB 由硬件管理，即由硬件负责处理 TLB 未命中），硬件用表示段号（segment number）的那几位（`SN`）来决定使用哪一对基址/界限寄存器（base/bounds register）。然后，硬件取出其中的物理地址，按如下方式和 VPN 组合，得到页表项（PTE）的地址：

```
SN            = (VirtualAddress & SEG_MASK) >> SN_SHIFT
VPN           = (VirtualAddress & VPN_MASK) >> VPN_SHIFT
AddressOfPTE = Base[SN] + (VPN * sizeof(PTE))
```

这个过程应该很眼熟：它和我们之前看到的线性页表几乎一模一样。当然，唯一的区别是，这里用的是三个段基址寄存器中的一个，而不是单独的一个页表基址寄存器（page-table base register）。

我们这个混合方案的关键区别在于每个段都有一个界限寄存器，其中保存的是该段中最大有效页的值。例如，如果代码段（code segment）用了它的前三页（0、1 和 2），代码段的页表就只会分配三个条目，界限寄存器设为 3；访问超出段末尾的内存会产生异常，很可能导致进程被终止。这样一来，与线性页表相比，混合方案能节省大量内存：栈和堆之间未分配的页不再占用页表空间（以前占着空间，只是为了把它们标记为无效）。

不过你可能已经注意到，这种方法并非没有问题。第一，它仍然要求我们使用分段；正如之前讨论过的，分段并不像我们希望的那样灵活，因为它假定地址空间有某种特定的使用模式。比如，如果堆很大却用得很稀疏，页表还是会浪费很多空间。第二，这种混合方案会让外部碎片（external fragmentation）再次出现。虽然大部分内存是以页为单位管理的，但页表的大小现在可以是任意的（PTE 大小的整数倍）。因此，要在内存中为页表找到空闲空间（free space）就更复杂了。出于这些原因，人们继续寻找更好的办法来实现更小的页表。

提示：使用混合方案

当你有两个都不错、看起来又互相对立的想法时，总应该看看能不能把它们结合成一个**混合方案**（hybrid），做到两全其美。例如，杂交玉米品种公认比任何天然品种都更强健。当然，杂交并不总是好主意；看看斑驴（zeedonk，也叫 zonkey）就知道了，它是斑马和驴的杂交种。如果你不相信有这种动物，就去查一查，准备好大吃一惊吧。

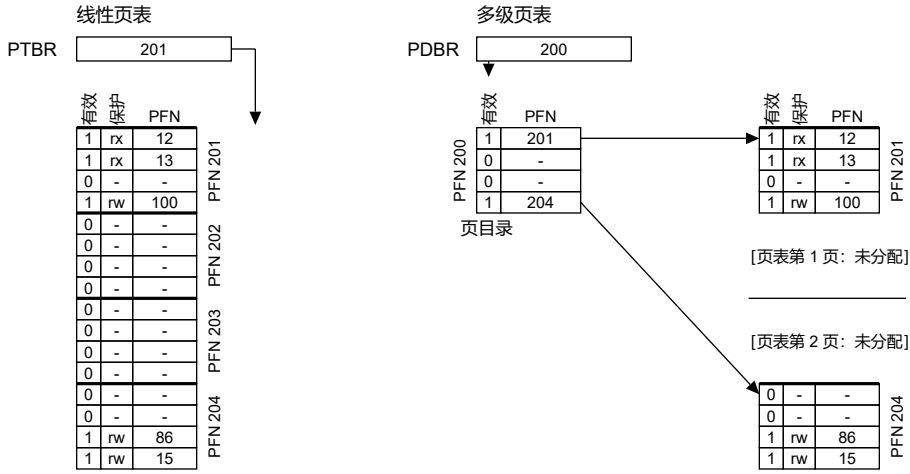



图 20.3：**线性页表（左）和多级页表（右）**

## 20.3 多级页表

另一种方法不依赖分段，但要解决的是同一个问题：怎样去掉页表中所有那些无效区域，而不是把它们全都留在内存里？我们把这种方法称为**多级页表**（multi-level page table），因为它把线性页表变成了类似树的结构。这种方法非常有效，许多现代系统都采用了它（例如 x86 [BOH10]）。下面我们详细介绍。

多级页表背后的基本思想很简单。首先，把页表按页大小切成若干块；然后，如果某一整页的页表项（PTE）都无效，就干脆不分配这一页页表。为了记录页表的每一页是否有效（如果有效，还要记录它在内存中的位置），我们使用一个新的结构，叫作**页目录**（page directory）。页目录既可以告诉你页表的某一页在哪里，也可以告诉你页表的某一整页里没有任何有效页。

图 20.3 给出了一个例子。图的左边是经典的线性页表；尽管地址空间中间的大部分区域都无效，我们仍然要为这些区域分配页表空间（即页表中间的两页）。右边是多级页表。页目录只把页表中的两页标记为有效（第一页和最后一页）；因此，只有页表的这两页驻留在内存中。由此，我们可以这样直观地理解多级页表：它只是让线性页表的一部分消失，腾出这些页帧（page frame）另作他用，并用页目录记录页表的哪些页已经分配。

在简单的两级页表中，页目录为页表的每一页提供一个条目。它由若干**页目录项**（page directory entries，**PDE**）组成。PDE（至少）包含一个**有效位**（valid bit）和一个**页帧号**（page frame number，PFN），这一点和 PTE 类似。但正如上面暗示的，这个有效位的含义稍有不同：如果 PDE 有效，就说明它（通过 PFN）指向的那一页页表中，至少有一个 PTE 的有效位被设为 1。如果 PDE 无效（即等于 0），PDE 的其余部分就是未定义的。

与我们目前见过的方法相比，多级页表有一些明显的优点。第一，也许也是最明显的一点，多级页表分配的页表空间只与你实际使用的地址空间大小成正比；因此它通常很紧凑，并且支持稀疏地址空间。

提示：理解时空权衡
构建数据结构时，总要考虑**时空权衡**（time-space trade-offs）。通常，如果你想让某个数据结构的访问更快，就得为它付出多占空间的代价。

第二，只要构造得当，页表的每一部分都正好能放进一页，这让内存管理更容易：操作系统在需要分配或增长页表时，只要拿下一个空闲页（free page）就行。相比之下，简单的线性页表本身不分页（non-paged）[2]，只是一个用 VPN 作索引的 PTE 数组；采用这种结构，整个线性页表必须连续地存放在物理内存中。对于比较大的页表（比如 4 MB）来说，要找到这么大一块连续的空闲物理内存可能相当困难。而在多级结构中，我们通过页目录增加了一个**间接层**（level of indirection），由页目录指向页表的各个部分；正是这层间接，让我们可以把页表的各页放在物理内存中任何想放的地方。

需要注意的是，多级页表是有代价的：发生 TLB 未命中时，要从内存加载两次才能从页表中拿到正确的转换信息（一次读页目录，一次读 PTE 本身），而线性页表只需加载一次。因此，多级页表是**时空权衡**（time-space trade-off）的一个小例子。我们想要更小的页表，也得到了，但这不是免费的：虽然在常见情况（common case），即 TLB 命中（TLB hit）时，性能显然完全一样，但页表变小之后，TLB 未命中的代价更高了。

另一个明显的缺点是复杂性。在 TLB 未命中时，无论由硬件还是操作系统来查找页表，查找过程都无疑比简单的线性页表更复杂。我们常常愿意增加复杂性来提高性能或降低开销；就多级页表而言，我们让页表查找变得更复杂，为的是节省宝贵的内存。

### 一个详细的多级页表示例

为了更好地理解多级页表背后的思想，我们来看一个例子。设想一个大小为 16 KB 的小地址空间，页大小为 64 字节。这样，我们就有一个 14 位的虚拟地址空间，其中 VPN 占 8 位，偏移量占 6 位。即使只用了地址空间的一小部分，线性页表也会有 $2^8$（256）个条目。图 20.4（第 195 页）展示了这样一个地址空间的例子。

在这个例子中，虚拟页 0 和 1 用于代码，虚拟页 4 和 5 用于堆，虚拟页 254 和 255 用于栈；地址空间的其余页都没有使用。

为了给这个地址空间构建两级页表，我们从完整的线性页表出发，把它按页大小切成若干块。回想一下，（在这个例子中）完整的页表有 256 个条目；假设每个 PTE 大小为 4 字节。因此，页表大小为 1 KB（256 × 4 字节）。由于页大小为 64 字节，1 KB 的页表可以分成 16 个 64 字节的页；每页可以容纳 16 个 PTE。

现在我们需要弄清楚的是，拿到一个 VPN 后，怎样先用它索引页目录，再索引页表中的某一页。记住，两者都是条目数组；因此，我们只需弄清楚怎样用 VPN 的各部分分别构造出两者的索引。

[2]这里我们做了一些假设：所有页表都完整地驻留在物理内存中（也就是说，它们没有被交换到磁盘上）；我们很快就会放宽这个假设。

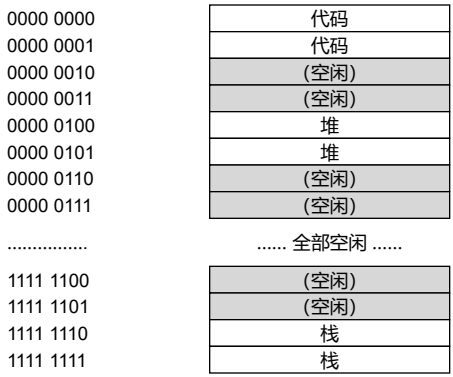
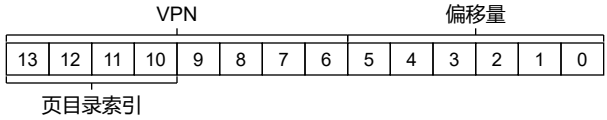



图 20.4：**页大小为 64 字节的 16 KB 地址空间**

我们先来索引页目录。这个例子中的页表很小：256 个条目，分布在 16 个页上。页目录需要为页表的每一页提供一个条目，因此它有 16 个条目。这样，我们就需要用 VPN 中的四位来索引页目录；我们使用 VPN 的最高四位，如下所示：




从 VPN 中取出**页目录索引**（page-directory index，简称 `PDIndex`）后，就可以通过一个简单的计算找到页目录项（PDE）的地址：`PDEAddr = PageDirBase + (PDIndex * sizeof(PDE))`。这样就得到了页目录中的对应条目，接下来检查它，继续完成地址转换（address translation）。

如果页目录项（page directory entry）被标记为无效，我们就知道这次访问是无效的，于是引发异常。但如果 PDE 有效，就还有更多工作要做。具体来说，我们现在必须从这个页目录项（page directory entry）所指向的那一页页表中取出页表项（PTE）。为了找到这个 PTE，我们要用 VPN 的剩余位来索引这部分页表：




然后就可以用这个**页表索引**（page-table index，简称 `PTIndex`）来索引页表本身，得到 PTE 的地址：

```
PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
```

注意，从页目录项（page directory entry）中得到的页帧号（page frame number），必须先左移到正确的位置，再和页表索引（page-table index）组合，形成 PTE 的地址。

为了检验这一切是否讲得通，我们现在往多级页表中填入一些实际的值，然后转换一个虚拟地址。先从这个例子的**页目录**（page directory）看起（图 20.5 左边）。

| 页目录 | | 页表的一页（@PFN:100） | | | 页表的一页（@PFN:101） | | |
|---|---|---|---|---|---|---|---|
| PFN | 有效? | PFN | 有效位 | 保护位 | PFN | 有效位 | 保护位 |
| 100 | 1 | 10 | 1 | r-x | — | 0 | — |
| — | 0 | 23 | 1 | r-x | — | 0 | — |
| — | 0 | — | 0 | — | — | 0 | — |
| — | 0 | — | 0 | — | — | 0 | — |
| — | 0 | 80 | 1 | rw- | — | 0 | — |
| — | 0 | 59 | 1 | rw- | — | 0 | — |
| — | 0 | — | 0 | — | — | 0 | — |
| — | 0 | — | 0 | — | — | 0 | — |
| — | 0 | — | 0 | — | — | 0 | — |
| — | 0 | — | 0 | — | — | 0 | — |
| — | 0 | — | 0 | — | — | 0 | — |
| — | 0 | — | 0 | — | — | 0 | — |
| — | 0 | — | 0 | — | — | 0 | — |
| — | 0 | — | 0 | — | — | 0 | — |
| — | 0 | — | 0 | — | 55 | 1 | rw- |
| 101 | 1 | — | 0 | — | 45 | 1 | rw- |

图 20.5：**页目录以及页表的几个部分**

从图中可以看到，每个页目录项（PDE）都描述了该地址空间页表中某一页的情况。在这个例子中，地址空间有两个有效区域（分别在开头和末尾），中间是许多无效映射。

物理页 100 是页表第 0 页所在的物理帧（physical frame），这一页共有 16 个页表项，对应地址空间中的前 16 个 VPN。这部分页表的内容见图 20.5（中间部分）。

页表的这一页包含前 16 个 VPN 的映射；在我们的例子中，代码段的 VPN 0 和 1 是有效的，堆的 4 和 5 也是。因此，表中有这几页各自的映射信息，其余条目都标记为无效。

页表的另一个有效页位于 PFN 101 中。这一页包含地址空间中最后 16 个 VPN 的映射；详见图 20.5（右边）。

在这个例子中，栈所在的 VPN 254 和 255 的映射是有效的。希望你能从这个例子中看出，多级索引（multi-level index）结构能节省多少空间。在这个例子中，我们不必为线性页表分配完整的十六页，只需分配三页：一页用于页目录，两页用于含有有效映射的页表块（page-table chunk）。对于大的（32 位或 64 位）地址空间，节省的空间显然会多得多。

提示：小心复杂性

系统设计者应当警惕给系统增加复杂性。优秀的系统构建者会用最简单的系统完成手头的任务。例如，如果磁盘空间很充裕，就不该设计一个费尽心思节省每一个字节的文件系统（file system）；同样，如果处理器很快，在操作系统中写一个清晰易懂的模块，比为手头的任务写出针对 CPU 极致优化的手工汇编代码要好。要警惕不必要的复杂性，无论它表现为过早优化（premature optimization）的代码还是别的形式；这类做法会让系统更难理解、维护和调试。安托万·德·圣埃克苏佩里（Antoine de Saint-Exupéry）有句名言："完美不是无以复加，而是无可删减。"他没写的是："谈论完美，要比真正做到完美容易得多。"

 © 2008–23, ARPACI-DUSSEAU

最后，我们用这些信息来完成一次地址转换。这里有一个地址，它指向 VPN 254 的第 0 个字节：`0x3F80`，二进制为 `11 1111 1000 0000`。

回想一下，我们用 VPN 的最高 4 位来索引页目录。因此，`1111` 会选中上面页目录的最后一项（如果从第 0 项算起，就是第 15 项）。这一项指向位于地址 `101` 的一个有效的页表页。然后，我们用 VPN 的接下来 4 位（`1110`）索引这一页页表，找到所需的 PTE。1110 是这一页的倒数第二项（第 14 项），它告诉我们，虚拟地址空间的第 254 页映射到物理页 55。把 PFN=55（即十六进制 `0x37`）和 offset=000000 拼接起来，就得到了所需的物理地址，可以把请求发给内存系统了：`PhysAddr = (PTE.PFN << SHIFT) + offset = 00 1101 1100 0000 = 0x0DC0`。

现在你应该大致知道，如何用一个指向页表各页的页目录来构建两级页表了。不过遗憾的是，事情还没完。我们接下来会看到，有时两级页表还不够！

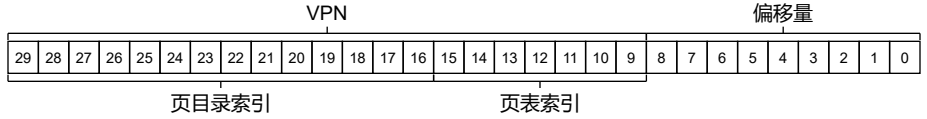
### 超过两级

到目前为止，我们的例子都假设多级页表只有两级：一个页目录，加上页表的若干部分。在某些情况下，树还可以更深（而且确实需要更深）。

我们用一个简单的例子来说明为什么更深的多级页表会有用。在这个例子中，假设有一个 30 位的虚拟地址空间，页很小（512 字节）。因此，虚拟地址中虚拟页号（virtual page number）部分占 21 位，偏移量占 9 位。

记住我们构建多级页表的目标：让页表的每一部分都能放进一页。到目前为止，我们只考虑了页表本身；可是，如果页目录变得太大怎么办？

为了确定多级页表需要多少级才能让页表的所有部分都放进一页，我们首先要算出一页能放多少个页表项。页大小为 512 字节，假设 PTE 大小为 4 字节，你应该能算出一页能放 128 个 PTE。因此，索引页表中的某一页时，我们需要用 VPN 的最低 7 位（$\log_2 128$）作为索引：




从上图你可能还会注意到，留给（很大的）页目录的位有 14 位之多。如果页目录有 $2^{14}$ 个条目（PDE 为 4 字节），它就不是只占一页，而是占 128 页，让多级页表的每一部分都能放进一页，这个目标也就落空了。

为了解决这个问题，我们给树再加一级：把页目录本身也拆分成多页，然后在它上面再加一个页目录，用来指向页目录的各页。于是，我们可以这样划分虚拟地址：




现在，索引上层页目录时，我们使用虚拟地址最高的那几位，即图中的“页目录索引 0”（page-directory index 0）；用这个索引可以从顶级页目录中取出页目录项（page directory entry）。如果它有效，就把顶级 PDE 中的物理帧号（physical frame number）和 VPN 的下

一部分，即“页目录索引 1”（page-directory index 1）组合起来，查询第二级页目录。最后，如果这一项也有效，就把页表索引（page-table index）和第二级 PDE 中的地址组合起来，得到 PTE 的地址。呼！工作量真不小。而这一切，只是为了在多级页表中查找一个条目。

### 转换过程：别忘了 TLB

我们再一次以算法的形式给出控制流（图 20.6），总结一下用两级页表做地址转换的整个过程。这张图展示了*每一次*内存引用时硬件中发生的事情（假设 TLB 由硬件管理）。

从图中可以看出，在进行任何复杂的多级页表访问之前，硬件会先检查 TLB；如果命中，就像以前一样，直接构造出物理地址，完全不用访问页表。只有在 TLB 未命中时，硬件才需要执行完整的多级查找。在这条路径上，你可以看到传统两级页表的代价：要多访问两次内存，才能查到一个有效的转换。

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB 命中
4    if (CanAccess(TlbEntry.ProtectBits) == True)
5      Offset   = VirtualAddress & OFFSET_MASK
6      PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7      Register = AccessMemory(PhysAddr)
8    else
9      RaiseException(PROTECTION_FAULT)
10 else                    // TLB 未命中
11   // 先取页目录项
12   PDIndex = (VPN & PD_MASK) >> PD_SHIFT
13   PDEAddr = PDBR + (PDIndex * sizeof(PDE))
14   PDE     = AccessMemory(PDEAddr)
15   if (PDE.Valid == False)
16     RaiseException(SEGMENTATION_FAULT)
17   else
18     // PDE 有效：再从页表中取 PTE
19     PTIndex = (VPN & PT_MASK) >> PT_SHIFT
20     PTEAddr = (PDE.PFN<<SHIFT) + (PTIndex*sizeof(PTE))
21     PTE     = AccessMemory(PTEAddr)
22     if (PTE.Valid == False)
23       RaiseException(SEGMENTATION_FAULT)
24     else if (CanAccess(PTE.ProtectBits) == False)
25       RaiseException(PROTECTION_FAULT)
26     else
27       TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
28       RetryInstruction()
```

图 20.6：**多级页表的控制流**

## 20.4 反置页表

在页表的世界里，还有一种更极端的节省空间的办法：**反置页表**（inverted page tables）。这种方法不是为系统中每个进程各维护一张页表，而是只保留一张页表，系统中的每个物理页（physical page）在其中都有一个条目。这个条目告诉我们，哪个进程在使用这一页，以及该进程的哪个虚拟页映射到这个物理页。

这样一来，要找到正确的条目，就得搜索这个数据结构。线性扫描的代价很高，因此通常会在这个基本结构之上再建一张散列表（hash table），以加快查找。PowerPC 就是这类体系结构的一个例子 [JM98]。

更一般地说，反置页表（inverted page table）说明了我们从一开始就在强调的一点：页表只不过是数据结构。你可以对数据结构做很多疯狂的事情：让它们更小或更大，更慢或更快。多级页表和反置页表（inverted page table）只是众多可能做法中的两个例子。

## 20.5 把页表交换到磁盘

最后，我们再放宽一个假设。到目前为止，我们一直假设页表驻留在内核拥有的物理内存中。然而，即使用了这么多技巧来减小页表，页表仍然可能大到无法一次全部放进内存。因此，有些系统把这些页表放在**内核虚拟内存**（kernel virtual memory）中，这样在内存有点紧张时，系统就可以把其中一部分页表**交换**（swap）到磁盘上。等我们更详细地了解了如何把页移进移出内存之后，会在后面的章节（即 VAX/VMS 的案例研究）中进一步讨论这个问题。

## 20.6 小结

我们已经看到了真实的页表是如何构建的：不一定只是线性数组，也可以是更复杂的数据结构。这类页表需要在时间和空间之间权衡：页表越大，处理 TLB 未命中就越快，反之亦然。因此，选择什么样的结构，在很大程度上取决于具体环境的约束。

在内存受限的系统中（比如许多较老的系统），小的结构比较合理；而在内存比较充足、工作负载（workload）又会频繁使用大量页的系统中，较大的页表能加快 TLB 未命中的处理，可能才是正确的选择。有了软件管理的 TLB（software-managed TLB），数据结构的整个设计空间都向操作系统创新者（提示：就是你）敞开了，任你施展。你能想出哪些新的结构？它们能解决什么问题？入睡时想想这些问题，做只有操作系统开发者才会做的大梦吧。

## 参考文献

[BOH10] "Computer Systems: A Programmer's Perspective" by Randal E. Bryant and David R. O'Hallaron. Addison-Wesley, 2010. *We have yet to find a good first reference to the multi-level page table. However, this great textbook by Bryant and O'Hallaron dives into the details of x86, which at least is an early system that used such structures. It's also just a great book to have.*

[JM98] "Virtual Memory: Issues of Implementation" by Bruce Jacob, Trevor Mudge. IEEE Computer, June 1998. *An excellent survey of a number of different systems and their approach to virtualizing memory. Plenty of details on x86, PowerPC, MIPS, and other architectures.*

[LL82] "Virtual Memory Management in the VAX/VMS Operating System" by Hank Levy, P. Lipman. IEEE Computer, Vol. 15, No. 3, March 1982. *A terrific paper about a real virtual memory manager in a classic operating system, VMS. So terrific, in fact, that we'll use it to review everything we've learned about virtual memory thus far a few chapters from now.*

[M28] "Reese's Peanut Butter Cups" by Mars Candy Corporation. Published at stores near you. *Apparently these fine confections were invented in 1928 by Harry Burnett Reese, a former dairy farmer and shipping foreman for one Milton S. Hershey. At least, that is what it says on Wikipedia. If true, Hershey and Reese probably hate each other's guts, as any two chocolate barons should.*

[N+02] "Practical, Transparent Operating System Support for Superpages" by Juan Navarro, Sitaram Iyer, Peter Druschel, Alan Cox. OSDI '02, Boston, Massachusetts, October 2002. *A nice paper showing all the details you have to get right to incorporate large pages, or **superpages**, into a modern OS. Not as easy as you might think, alas.*

[M07] "Multics: History" Available: http://www.multicians.org/history.html. *This amazing web site provides a huge amount of history on the Multics system, certainly one of the most influential systems in OS history. The quote from therein: "Jack Dennis of MIT contributed influential architectural ideas to the beginning of Multics, especially the idea of combining paging and segmentation." (from Section 1.2.1)*

### 作业（模拟）

这个有趣的小作业用来检验你是否理解多级页表的工作原理。没错，上一句用“有趣”这个词是否恰当，还有些争议。这个程序叫 `paging-multilevel-translate.py`，名字大概不出你所料；详见 README。

### 问题

1. 使用线性页表时，假设 TLB 未命中时由硬件查找页表，你只需要一个寄存器来定位页表。要定位两级页表，需要几个寄存器？三级页表呢？
2. 用模拟器分别以随机种子 0、1 和 2 做地址转换，并用 `-c` 选项检查你的答案。每次查找需要多少次内存引用？
3. 根据你对缓存（cache）工作原理的理解，你认为对页表的内存引用在缓存中会表现如何？它们会带来大量缓存命中（cache hit），从而访问很快吗？还是会带来大量未命中，从而访问很慢？

 © 2008–23, ARPACI-DUSSEAU

# 21

# 超越物理内存：机制

到目前为止，我们一直假设地址空间（address space）小得不切实际，能放进物理内存（physical memory）。实际上，我们一直假设所有运行中进程（process）的地址空间全都能放进内存。现在我们要放宽这些很强的假设，转而支持许多同时运行的大地址空间。

为此，我们需要在**存储器层次结构**（memory hierarchy）中再加一层。到目前为止，我们一直假设所有页（page）都驻留在物理内存中。然而，为了支持大地址空间，操作系统需要一个地方，把地址空间中当前不太急需的部分暂时存放起来。一般来说，这个地方的特点是容量要比内存大；因此它通常也更慢（要是它更快，我们直接把它当内存用不就行了?）。在现代系统中，这个角色通常由**硬盘驱动器**（hard disk drive）来担任。于是，在我们的存储器层次结构中，又大又慢的硬盘位于最底层，内存就在它上面一层。由此就引出了问题的关键：

关键问题：如何超越物理内存
操作系统怎样利用一个更大、更慢的设备，透明地提供巨大虚拟地址空间（virtual address space）的假象?

你可能会问：为什么要为一个进程支持单个巨大的地址空间? 答案仍然是方便和易用。有了巨大的地址空间，你就不必操心内存里有没有足够的空间放程序的数据结构（data structure）；只要自然地写程序，按需分配内存就行。这是操作系统提供的一个强大假象，让你的生活简单了很多。不用谢！与之形成对比的是使用**内存覆盖**（memory overlays）技术的老系统，这种技术要求程序员根据需要，手动把一段段代码或数据移进移出内存 [D97]。想象一下那会是什么样子：调用函数或访问某些数据之前，你得先设法把这段代码或数据放进内存；真够呛！

有了交换空间（swap space），操作系统不仅能为单个进程，还能为多个同时运行的进程提供巨大虚拟内存的假象。多道程序设计（multiprogramming，即“同时”运行多个程序，以更好地利用机器）发明以后，几乎就必须能把一部分页换出（swap out），因为早期的机器显然无法同时容纳所有进程所需的全部页。因此，多道程序设计加上对易用性的追求，让我们希望能使用比实际物理内存更多的内存。这是所有现代虚拟内存系统都会做的事；接下来我们就深入了解它。

补充：存储技术

后面我们会更深入地探讨 I/O 设备实际上是如何工作的（见 I/O 设备一章），所以请耐心一点！当然，这个较慢的设备不一定是硬盘，也可以是更现代的设备，比如基于闪存（flash）的 SSD。这些我们也会讨论。现在，只要假设我们有一个容量大、速度相对较慢的设备，可以借助它营造出一个非常大的虚拟内存（virtual memory）假象，这个虚拟内存甚至比物理内存本身还大。

## 21.1 交换空间

我们要做的第一件事，是在磁盘上预留一些空间，用来把页移来移去。在操作系统中，我们一般把这样的空间称为**交换空间**（swap space），因为我们把内存中的页换出（swap out）到这里，也从这里把页换入（swap in）内存。我们就简单地假设，操作系统能以页为单位读写交换空间。为此，操作系统需要记住某一页的**磁盘地址**（disk address）。

交换空间的大小很重要，因为它最终决定了系统在某一时刻最多能使用多少个内存页（memory page）。为简单起见，我们暂时假设它非常大。

图 21.1 是一个小例子，其中有一个 4 页的物理内存和一个 8 页的交换空间。进程 0、1、2 这三个进程正在共享物理内存；不过，这三个进程都只有一部分有效页（valid page）在内存中，其余的页都在磁盘上的交换空间里。第四个进程（进程 3）的所有页都被换出到了磁盘上，所以它显然当前没有运行。交换空间中还有一个块（block）是空闲的。哪怕只是这样一个小例子，也希望你能从中看出，系统是如何利用交换空间假装内存比实际更大的。

需要注意的是，交换空间并不是磁盘上唯一用于交换的位置。例如，假设你在运行一个程序的二进制文件（比如 `ls`，或者你自己编译的 `main` 程序）。这个二进制文件的代码页（code page）最初在磁盘上，程序运行时，它们会被加载到内存中（要么在程序开始执行时一次性全部加载，要么像现代系统那样，在需要时一次加载一页）。不过，如果系统需要为其他用途腾出物理内存，它可以放心地重新使用这些代码页占用的内存空间，因为以后随时可以从文件系统（file system）里那个二进制文件中把它们重新换入。

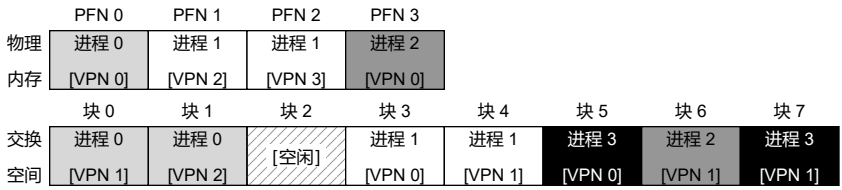



图 21.1：**物理内存和交换空间**

## 21.2 存在位

现在磁盘上已经有了一些空间，我们需要在系统的更高层添加一些机制，以便把页换出到磁盘，或从磁盘换入。为简单起见，我们假设系统使用由硬件管理的 TLB（hardware-managed TLB）。

首先回想一下引用内存时会发生什么。运行中的进程会产生虚拟内存引用（取指令或访问数据），在我们假设的这种系统中，硬件会先把它们转换为物理地址（physical address），再从内存中取出所需的数据。

记住，硬件先从虚拟地址（virtual address）中提取 VPN，然后检查 TLB 中有没有匹配项（即 **TLB 命中**，TLB hit）；如果命中，就生成相应的物理地址，并从内存中取数据。我们希望这是常见情况（common case），因为它很快（不需要额外的内存访问）。

如果 TLB 中找不到这个 VPN（即 **TLB 未命中**，TLB miss），硬件就会（借助**页表基址寄存器**，page table base register）在内存中找到页表（page table），并用 VPN 作为索引，查找这一页的**页表项**（page table entry，PTE）。如果这一页有效，并且就在物理内存中，硬件就从 PTE 中取出 PFN，把它装入 TLB，然后重试（retry）这条指令，这次就会产生 TLB 命中；到这里一切顺利。

但是，如果我们希望允许页被交换到磁盘上，就得再加一些机制。具体来说，硬件查看 PTE 时，可能会发现这一页不在物理内存中。硬件（在软件管理 TLB 的方案中则是操作系统）要借助每个页表项中的一项新信息来判断这一点，这项信息叫作**存在位**（present bit）。如果存在位（present bit）为 1，就表示这一页在物理内存中，一切都像上面说的那样进行；如果为 0，这一页就不在内存中，而是在磁盘上的某个地方。

访问不在物理内存中的页，这一行为通常称为**缺页异常**（page fault）。

发生缺页异常时，操作系统会出面：一段专门的代码，即**缺页异常处理程序**（page-fault handler），开始运行并处理这次异常。下面我们就来介绍它。

补充：交换相关的术语及其他

虚拟内存系统中的术语可能有点让人困惑，而且因机器和操作系统而异。例如，从更宽泛的意义上说，**缺页异常**（page fault，也叫缺页中断）可以指任何引发了某种故障的页表引用：既可能是我们这里讨论的这类故障，即页不存在故障，有时也可能是非法内存访问。事实上，一次访问明明完全合法（访问的页已经映射到进程的虚拟地址空间中，只是当时不在物理内存里），却被称为“故障”（fault），这本身就有点奇怪；它其实应该叫**缺页**（page miss）。不过，人们说某个程序“频繁缺页”时，通常是指它在访问自己虚拟地址空间中被操作系统换出到磁盘的那些部分。

我们猜想，这种行为之所以被称为“故障”，与操作系统中处理它的机制有关。一旦发生了不寻常的事情，也就是硬件不知道该如何处理的事情，硬件就干脆把控制权交给操作系统，指望它能把事情处理好。在这种情况下，进程想访问的页不在内存中；硬件只能做它唯一能做的事：引发异常，然后由操作系统接手。由于这和进程做了非法操作时发生的事情完全一样，把这种情况称为“故障”，也许就不足为奇了。

 © 2008–23, ARPACI-DUSSEAU

补充：为什么硬件不处理缺页异常

从 TLB 的经验我们知道，硬件设计者几乎什么事都不愿意交给操作系统去做。那他们为什么放心让操作系统来处理缺页异常呢？主要原因有几个。第一，需要访问磁盘的缺页异常很慢；即使操作系统处理一次异常要花很长时间、执行大量指令，磁盘操作本身向来要慢得多，相比之下，运行软件的额外开销微乎其微。第二，为了处理缺页异常，硬件就得懂交换空间、懂怎样向磁盘发出 I/O，还得知道许多它目前不太清楚的细节。因此，出于性能和简单性两方面的考虑，缺页异常由操作系统来处理，连搞硬件的人也会满意。

## 21.3 缺页异常

回想一下，在 TLB 未命中的处理上，系统分为两类：一类是硬件管理的 TLB（hardware-managed TLB），由硬件在页表中查找所需的转换；另一类是软件管理的 TLB（software-managed TLB），由操作系统查找。无论哪类系统，只要页不存在，都由操作系统负责处理缺页异常。这时会运行操作系统的**缺页异常处理程序**（page-fault handler，这个名字起得恰如其分），由它决定该怎么做。几乎所有系统都用软件处理缺页异常；即使 TLB 由硬件管理，硬件也会把这项重要职责交给操作系统。

如果页不存在，并且已经被交换到磁盘上，操作系统就需要把这一页换入内存，才能处理缺页异常。于是就有一个问题：操作系统怎么知道去哪里找所需的页？在许多系统中，页表是存放这类信息的天然位置。因此，PTE 中通常存放该页 PFN 等数据的那些位，操作系统可以拿来存放磁盘地址。操作系统收到某一页的缺页异常后，会查看 PTE 找到磁盘地址，然后向磁盘发出请求，把这一页取进内存。

磁盘 I/O 完成后，操作系统会更新页表，把这一页标记为存在，更新页表项（PTE）的 PFN 字段，记录新取进来的页在内存中的位置，然后重试这条指令。这次重试可能会产生 TLB 未命中，处理这次未命中时会用这个转换更新 TLB（也可以在处理缺页异常时就更新 TLB，省掉这一步）。最后，再重试一次时，就会在 TLB 中找到这个转换，于是从内存中转换后的物理地址处取出所需的数据或指令。

注意，在 I/O 进行期间，进程会处于**阻塞**（blocked）状态。因此，在处理缺页异常的同时，操作系统可以自由地运行其他就绪（ready）的进程。由于 I/O 代价高昂，让一个进程因缺页异常而做的 I/O 与另一个进程的执行**重叠**（overlap）起来，是多道程序系统最有效地利用硬件的又一种方式。

## 21.4 内存满了怎么办?

你可能已经注意到，在上面描述的过程中，我们假设有足够多的空闲内存，可以把一页从交换空间**换入**（page in）。当然，情况未必如此；内存可能已经满了（或接近满了）。因此，操作系统可能要先**换出**（page out）一页或多页，为即将调入的新页腾出空间。决定踢出、或者说**置换**（replace）哪一页的做法，称为**页面置换策略**（page-replacement policy）。

```
1 VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2 (Success, TlbEntry) = TLB_Lookup(VPN)
3 if (Success == True)   // TLB 命中
4   if (CanAccess(TlbEntry.ProtectBits) == True)
5     Offset   = VirtualAddress & OFFSET_MASK
6     PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7     Register = AccessMemory(PhysAddr)
8   else
9     RaiseException(PROTECTION_FAULT)
10 else                   // TLB 未命中
11   PTEAddr = PTBR + (VPN * sizeof(PTE))
12   PTE = AccessMemory(PTEAddr)
13   if (PTE.Valid == False)
14     RaiseException(SEGMENTATION_FAULT)
15   else
16     if (CanAccess(PTE.ProtectBits) == False)
17       RaiseException(PROTECTION_FAULT)
18     else if (PTE.Present == True)
19       // 假设 TLB 由硬件管理
20       TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
21       RetryInstruction()
22     else if (PTE.Present == False)
23       RaiseException(PAGE_FAULT)
```

图 21.2：**缺页异常控制流算法（硬件）**

实际上，人们在设计好的页面置换策略（page-replacement policy）上花了很多心思，因为一旦踢错了页，程序性能就要付出巨大代价。做出错误的决定，可能会让程序以磁盘的速度而不是内存的速度运行；以现在的技术来说，这意味着程序可能会慢 10,000 倍甚至 100,000 倍。因此，这种策略值得我们详细研究；事实上，下一章正是要做这件事。眼下只要知道有这样一种策略、它建立在本章介绍的机制之上，就够了。

## 21.5 缺页异常的控制流

有了这些知识，我们现在可以大致勾勒出内存访问的完整控制流了。换句话说，要是有人问你“程序从内存中取数据时会发生什么?”，你应该对各种可能的情况都心中有数。更多细节请看图 21.2 和图 21.3 中的控制流；前一张图展示硬件在地址转换（address translation）时做什么，后一张展示操作系统在发生缺页异常时做什么。

从图 21.2 的硬件控制流可以看出，发生 TLB 未命中时，现在有三种重要的情况需要理解。第一种情况是这一页既**存在**（present）又**有效**（valid），见第 18–21 行；这时，TLB 未命中处理程序（TLB miss handler）只需从 PTE 中取出 PFN，重试指令，这次就会 TLB 命中，然后像之前（多次）描述的那样继续执行。第二种情况（第 22–23 行）必须运行缺页异常处理程序（page-fault handler）：虽然这一页是进程可以合法访问的（毕竟它是有效的），

```
1 PFN = FindFreePhysicalPage()
2 if (PFN == -1)                // 没找到空闲页
3     PFN = EvictPage()         // 置换算法
4 DiskRead(PTE.DiskAddr, PFN)   // 睡眠（等待 I/O）
5 PTE.present = True            // 更新页表：
6 PTE.PFN     = PFN             // （存在位/地址转换）
7 RetryInstruction()            // 重试指令
```

图 21.3：**缺页异常控制流算法（软件）**

但它不在物理内存中。第三种（也是最后一种）情况是访问了一个无效页（invalid page），比如程序里有 bug（第 13–14 行）。这时，PTE 中的其他位都无关紧要了；硬件会捕获这次无效访问，然后运行操作系统的陷阱处理程序（trap handler），很可能会终止出错的进程。

从图 21.3 的软件控制流，可以看出操作系统处理缺页异常大致要做哪些事。首先，操作系统必须为即将调入的页找到一个物理帧（physical frame）；如果没有空闲的页，就只能等置换算法（replacement algorithm）运行，把一些页踢出内存，腾出来供这里使用。拿到物理帧之后，处理程序再发出 I/O 请求，从交换空间读入这一页。最后，等这个缓慢的操作完成，操作系统更新页表并重试指令。重试会导致 TLB 未命中，再重试一次就会 TLB 命中，这时硬件就能访问所需的内容了。

## 21.6 置换实际上何时发生

到目前为止，我们描述置换如何发生时，都假设操作系统会一直等到内存完全满了，才置换一页，也就是把它淘汰（evict），为其他页腾出空间。可以想象，这有点不切实际，操作系统有很多理由更主动地保留一小部分空闲内存。

为了保留少量空闲内存，大多数操作系统都会设置某种**高水位线**（high watermark，*HW*）和**低水位线**（low watermark，*LW*），帮助决定何时开始从内存中淘汰页。具体做法如下：操作系统一旦发现可用页少于 *LW* 个，就会运行一个负责释放内存的后台线程（thread）。这个线程会不断淘汰页，直到有 *HW* 个可用页为止。这个后台线程有时叫作**换页守护进程**（swap daemon 或 page daemon）[1]；它为运行中的进程和操作系统释放了一些内存，于是心满意足地去睡觉了。

一次执行多次置换，就能做一些新的性能优化。例如，许多系统会把若干页**聚集**（cluster）或**成组**（group）起来，一次性写到交换分区（swap partition），从而提高磁盘的效率 [LL82]；我们后面更详细地讨论磁盘时会看到，这种聚集能减少磁盘的寻道（seek）和旋转开销，明显提高性能。

为了配合后台换页线程，图 21.3 中的控制流需要稍作修改：算法不再直接执行置换，而只是检查有没有可用的空闲页（free page）。如果没有，就通知后台换页线程需要空闲页；后台线程释放出一些页后，会重新唤醒原来的线程，原来的线程随后就能换入所需的页，接着做自己的工作。

[1] “daemon” 这个词通常读作 “demon”，是一个古老的术语，指在后台做有用工作的线程或进程。而且（又一次！）这个术语原来也出自 Multics [CS94]。

提示：在后台做事

有工作要做时，把它放到**后台**（background）去做往往是个好主意：既能提高效率，又能把操作成组处理。操作系统经常在后台做事；例如，许多系统会先把文件写操作缓冲在内存中，然后才真正把数据写到磁盘上。这样做有很多潜在的好处：一是提高磁盘效率，因为磁盘现在可以一次收到很多写请求，从而能更好地调度它们；二是降低写操作的延迟，因为应用程序会觉得写操作很快就完成了；三是有可能减少工作量，因为这些写操作可能根本不需要写到磁盘（比如文件被删除了）；四是更好地利用**空闲时间**（idle time），因为后台工作也许可以在系统空闲时完成，从而更好地利用硬件 [G+95]。

## 21.7 小结

在这短短的一章中，我们引入了这样一个概念：可以访问比系统实际拥有的物理内存更多的内存。要做到这一点，页表结构就得更复杂一些，因为必须包含（某种）**存在位**（present bit），告诉我们这一页是否在内存中。如果不在，操作系统的**缺页异常处理程序**（page-fault handler）就会运行，处理这次**缺页异常**（page fault），从而安排把所需的页从磁盘传到内存，可能还要先置换内存中的一些页，为即将换入的页腾出空间。

请记住（这一点很重要，也很神奇！），所有这些操作对进程都是**透明**的。在进程看来，它只是在访问自己私有的、连续的虚拟内存。在幕后，页被放在物理内存中任意的（不连续的）位置，有时甚至根本不在内存中，需要从磁盘取回。虽然我们希望一般情况下内存访问都很快，但在某些情况下，完成一次访问需要多次磁盘操作；像执行一条指令这样简单的事情，在最坏的情况下也可能要花好几毫秒才能完成。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[CS94] "Take Our Word For It" by F. Corbato, R. Steinberg. www.takeourword.com/TOW146 (Page 4). *Richard Steinberg writes: "Someone has asked me the origin of the word daemon as it applies to computing. Best I can tell based on my research, the word was first used by people on your team at Project MAC using the IBM 7094 in 1963." Professor Corbato replies: "Our use of the word daemon was inspired by the Maxwell's daemon of physics and thermodynamics (my background is in physics). Maxwell's daemon was an imaginary agent which helped sort molecules of different speeds and worked tirelessly in the background. We fancifully began to use the word daemon to describe background processes which worked tirelessly to perform system chores."*

[D97] "Before Memory Was Virtual" by Peter Denning. In the Beginning: Recollections of Software Pioneers, Wiley, November 1997. *An excellent historical piece by one of the pioneers of virtual memory and working sets.*

[G+95] "Idleness is not sloth" by Richard Golding, Peter Bosch, Carl Staelin, Tim Sullivan, John Wilkes. USENIX ATC '95, New Orleans, Louisiana. *A fun and easy-to-read discussion of how idle time can be better used in systems, with lots of good examples.*

[LL82] "Virtual Memory Management in the VAX/VMS Operating System" by Hank Levy, P. Lipman. IEEE Computer, Vol. 15, No. 3, March 1982. *Not the first place where page clustering was used, but a clear and simple explanation of how such a mechanism works. We sure cite this paper a lot!*

 © 2008–23, ARPACI-DUSSEAU

## 作业（测量）

这个作业会向你介绍一个新工具 **vmstat**，以及如何用它来了解内存、CPU 和 I/O 的使用情况。在做下面的练习和问题之前，请先阅读相关的 README，并看看 `mem.c` 中的代码。

## 问题

1. 首先，在同一台机器上打开两个独立的终端连接，这样你就可以方便地在一个窗口里运行一个东西，在另一个窗口里运行另一个。
   现在，在一个窗口中运行 `vmstat 1`，它会每秒显示一次机器使用情况的统计信息。阅读手册页（manual page）、相关的 README 以及你需要的其他资料，弄懂它的输出。在下面的所有练习中，都让这个窗口一直运行 `vmstat`。
   接下来，我们运行程序 `mem.c`，但只使用很少的内存。输入 `./mem 1` 就可以做到（它只使用 1 MB 内存）。运行 `mem` 时，CPU 使用情况的统计数据有什么变化？`user` 时间这一列的数字合理吗？同时运行多个 `mem` 实例时，情况又会怎样变化？

2. 现在，我们一边运行 `mem`，一边观察一些内存统计数据。我们重点关注两列：`swpd` 和 `free`，分别表示已使用的虚拟内存量和空闲内存量。运行 `./mem 1024`（它会分配 1024 MB），观察这些值如何变化。然后终止正在运行的程序（按 control-c），再观察这些值如何变化。你注意到这些值有什么特点？特别是，程序退出时，`free` 这一列有什么变化？`mem` 退出时，空闲内存的增加量是否符合预期？

3. 接下来我们看交换相关的两列（`si` 和 `so`），它们表示有多少数据从磁盘换入、又有多少换出到磁盘。当然，要让这两列有动静，你需要用大量内存来运行 `mem`。首先，看看你的 Linux 系统上有多少空闲内存（比如输入 `cat /proc/meminfo`；输入 `man proc` 可以了解 `/proc` 文件系统的细节，以及在那里能找到哪些类型的信息）。`/proc/meminfo` 中最前面的几项之一，就是系统的内存总量。假设大约是 8 GB；那么先运行 `mem 4000`（大约 4 GB），观察换入/换出这两列。它们会出现非零值吗？然后再试 5000、6000 等。与第一轮循环相比，程序进入第二轮循环（以及之后的循环）时，这些值有什么变化？在第二轮、第三轮以及之后的循环中，（总共）有多少数据被换入和换出？（这些数字合理吗？）

4. 做和上面相同的实验，但这次观察其他统计数据，比如 CPU 利用率（utilization）和块 I/O 统计。运行 `mem` 时，它们有什么变化？

5. 现在我们来看看性能。为 `mem` 选一个能轻松放进内存的输入（比如系统内存为 8 GB 时选 4000）。第 0 轮循环（以及之后的第 1、2 轮等）要多长时间？然后选一个明显超出内存大小的值（同样假设内存为 8 GB，比如选 12000）。这时每轮循环要多长时间？带宽（bandwidth）数字相比如何？不停交换时的性能，和所有数据都能轻松放进内存时相比，差别有多大？你能画一张图，以 `mem` 使用的内存大小为 x 轴、访问这些内存的带宽为 y 轴吗？最后，在全部能放进内存和放不进内存这两种情况下，第一轮循环的性能与之后的循环相比如何？

 © 2008–23, ARPACI-DUSSEAU

6. 交换空间不是无限的。你可以用带 `-s` 选项的 `swapon` 工具查看有多少交换空间可用。如果用越来越大的值运行 `mem`，直到超出看起来可用的交换空间，会发生什么？内存分配在什么时候会失败？
7. 最后，如果你水平比较高，可以用 `swapon` 和 `swapoff` 配置系统使用不同的交换设备。详情请阅读手册页。如果你能用上不同的硬件，看看交换到传统硬盘、基于闪存的 SSD，甚至 RAID 阵列时，交换性能有什么变化。用更新的设备能把交换性能提高多少？能多接近在内存中运行的性能？

 © 2008–23, ARPACI-DUSSEAU

# 22

# 超越物理内存：策略

在虚拟内存（virtual memory）管理器里，空闲内存多的时候日子很好过。发生缺页异常（page fault）时，从空闲页链表（free-page list）里找一个空闲页，分给发生缺页的那一页就行了。嘿，操作系统，恭喜！你又成功了。

可惜，空闲内存少的时候，事情就变得有点意思了。这时，**内存压力**（memory pressure）会迫使操作系统开始把页（page）**换出**（paging out），给正在使用的页腾出地方。决定**淘汰**（evict）哪一页（或哪些页），是操作系统**置换策略**（replacement policy）的职责。从历史上看，这是早期虚拟内存系统要做的最重要的决策之一，因为当年的系统物理内存（physical memory）很少。至少，这组策略本身也挺有意思，值得多了解一些。于是，我们要解决的问题是：

关键问题：如何决定淘汰哪一页

操作系统如何决定把哪一页（或哪些页）从内存中淘汰出去？这个决定由系统的置换策略（replacement policy）做出，置换策略通常遵循一些通用原则（下面会讨论），但也会加入一些调整，以避免在极端情况下出现异常行为。

## 22.1 缓存管理

在深入讨论各种策略之前，我们先更详细地描述一下要解决的问题。既然主存保存的是系统中全部页的一个子集，那么完全可以把它看作系统中虚拟内存页的**缓存**（cache）。因此，为这个缓存选择置换策略（replacement policy）时，我们的目标是让**缓存未命中**（cache miss）的次数最少，也就是让必须从磁盘取页的次数最少。换个角度，也可以说我们的目标是让**缓存命中**（cache hit）的次数最多，也就是让被访问的页能在内存中找到的次数最多。

知道了缓存命中和未命中的次数，就能算出程序的**平均内存访问时间**（average memory access time，**AMAT**），计算机体系结构的设计者就用这个指标来衡量硬件缓存[HP06]。具体来说，有了这些值，就可以按下式算出程序的 AMAT：

$$\text{AMAT} = T_M + (P_{\text{Miss}} \cdot T_D) \tag{22.1}$$

其中 $T_M$ 是访问内存的开销，$T_D$ 是访问磁盘的开销，$P_{\text{Miss}}$ 是在缓存中找不到数据（未命中）的概率；$P_{\text{Miss}}$ 的取值在 0.0 到 1.0 之间，有时我们也用百分比形式的未命中率（miss

rate）来代替概率（例如，未命中率为 10% 就表示 $P_{\text{Miss}} = 0.10$）。注意，访问内存中数据的开销你总是要付的；但一旦未命中，你还得额外付出从磁盘取数据的开销。

举个例子，假设有一台机器，地址空间（address space）很小，只有 4 KB，页大小（page size）为 256 字节。于是，虚拟地址（virtual address）由两部分组成：4 位的 VPN（最高几位）和 8 位的偏移量（offset，最低几位）。因此，这个例子中的进程（process）总共可以访问 $2^4$ 即 16 个虚拟页（virtual page）。假设进程产生了以下内存引用，也就是以下虚拟地址：0x000、0x100、0x200、0x300、0x400、0x500、0x600、0x700、0x800、0x900。这些虚拟地址指向地址空间前十页中每一页的第一个字节（每个虚拟地址的第一个十六进制数字就是页号）。

再假设除了虚拟页 3 以外，其他每一页都已经在内存中了。于是，这串内存引用会出现以下情况：命中、命中、命中、未命中、命中、命中、命中、命中、命中、命中。我们可以算出**命中率**（hit rate，即在内存中找到的引用所占的百分比）：90%，因为 10 次引用中有 9 次在内存中找到了。因此**未命中率**（miss rate）是 10%（$P_{\text{Miss}} = 0.1$）。一般来说，$P_{\text{Hit}} + P_{\text{Miss}} = 1.0$，即命中率加未命中率（miss rate）等于 100%。

要计算 AMAT，需要知道访问内存的开销和访问磁盘的开销。假设访问内存的开销（$T_M$）约为 100 纳秒，访问磁盘的开销（$T_D$）约为 10 毫秒，那么 AMAT 为：$100\text{ns} + 0.1 \cdot 10\text{ms}$，即 100ns + 1ms，也就是 1.0001 毫秒，大约 1 毫秒。如果命中率是 99.9%（$P_{\text{Miss}} = 0.001$），结果就大不一样了：AMAT 是 10.1 微秒，大约快了 100 倍。命中率越接近 100%，AMAT 就越接近 100 纳秒。

遗憾的是，从这个例子可以看出，在现代系统中磁盘访问的开销实在太高，哪怕未命中率（miss rate）很小，也会很快成为程序整体 AMAT 的主要部分。显然，我们必须尽可能避免未命中，否则程序就只能以磁盘的速度慢慢跑。办法之一就是精心设计一个聪明的策略，这正是我们接下来要做的。

## 22.2 最优置换策略

要更好地了解某个置换策略（replacement policy）表现如何，最好拿它和可能的最佳置换策略做个比较。其实，Belady 多年前就提出了这样的**最优**（optimal）策略 [B66]（他最初称之为 MIN）。最优置换策略（replacement policy）产生的未命中总次数最少。Belady 证明，最优策略其实很简单（可惜很难实现！）：置换将来最晚才会被访问的那一页，这样产生的缓存未命中次数最少。

提示：和最优策略比较很有用

最优策略虽然没法真正拿来用，但在模拟或其他研究中，作为比较的基准却极其有用。光说你那花哨的新算法命中率达到 80%，本身没什么意义；如果同时说明最优策略的命中率是 82%（因此你的新方法已经非常接近最优），结果就更有意义，也有了参照。所以，在你做的任何研究中，知道最优结果是多少，都能让你做出更好的比较：它能告诉你还有多大的改进空间，也能告诉你什么时候可以停止改进策略了，因为它已经足够接近理想状态 [AD03]。

最优策略背后的直觉应该不难理解。你可以这样想：如果必须扔掉某一页，为什么不扔掉要过最久才会用到的那一页呢?这样做，实际上就是认定缓存中其他所有页都比这一页重要。理由很简单：你会先用到其他那些页，然后才会用到它。

我们来跟踪一个简单的例子，看看最优策略会做出哪些决定。假设程序依次访问以下虚拟页：0、1、2、0、1、3、0、3、1、2、1。图 22.1 展示了最优策略的行为，这里假设缓存能放三页。

从图中可以看到以下过程。毫不奇怪，前三次访问都是未命中，因为缓存一开始是空的；这种未命中有时称为**冷启动未命中**（cold-start miss），或**强制性未命中**（compulsory miss）。接着我们再次引用页 0 和页 1，它们都在缓存中命中。然后，我们又遇到一次未命中（访问页 3），但这次缓存已经满了，必须置换一页！问题来了：该置换哪一页？用最优策略，我们会查看当前缓存中每一页（0、1、2）将来的情况，发现 0 几乎马上就会被访问，1 稍后会被访问，而 2 要过最久才会被访问。所以最优策略很容易做出选择：淘汰页 2，于是缓存中是页 0、1、3。接下来的三次引用都命中了，但随后访问页 2 时（它早就被淘汰了），又发生了一次未命中。这时最优策略再次查看缓存中每一页（0、1、3）将来的情况，发现只要不淘汰页 1（它马上就要被访问），就没问题。示例中淘汰的是页 3，其实淘汰页 0 也同样可以。最后，访问页 1 命中，这串访问序列就结束了。

我们还可以计算这个缓存的命中率：命中 6 次、未命中 5 次，命中率等于命中次数 /（命中次数 + 未命中次数），即 6 /（6 + 5），也就是 54.5%。也可以算出不计强制性未命中（compulsory miss）时的命中率（即每一页的第一次未命中不算），结果是 85.7%。

遗憾的是，正如我们之前在设计调度策略（scheduling policy）时看到的，未来通常是无法预知的；通用操作系统不可能实现最优策略[1]。因此，在设计真正能部署的实际策略时，我们要另想办法来决定淘汰哪一页。最优策略只用作比较的基准，让我们知道离“完美”还有多远。

| 访问 | 命中/未命中? | 淘汰 | 访问后的缓存状态 |
|---|---|---|---|
| 0 | 未命中 | | 0 |
| 1 | 未命中 | | 0, 1 |
| 2 | 未命中 | | 0, 1, 2 |
| 0 | 命中 | | 0, 1, 2 |
| 1 | 命中 | | 0, 1, 2 |
| 3 | 未命中 | 2 | 0, 1, 3 |
| 0 | 命中 | | 0, 1, 3 |
| 3 | 命中 | | 0, 1, 3 |
| 1 | 命中 | | 0, 1, 3 |
| 2 | 未命中 | 3 | 0, 1, 2 |
| 1 | 命中 | | 0, 1, 2 |

图 22.1：**最优策略的跟踪过程**

[1]如果你能做到，请告诉我们！我们可以一起发财。要不然，就一起像那些“发现”了冷核聚变的科学家那样，遭到众人的嘲笑和讥讽 [FP89]。

 © 2008–25, ARPACI-DUSSEAU

补充：缓存未命中的类型

在计算机体系结构领域，设计者发现，把未命中按类型分成三类有时很有用：强制性未命中（compulsory miss）、容量未命中（capacity miss）和冲突未命中（conflict miss），有时合称 **3C**（Three C's）[H87]。**强制性未命中**（compulsory miss，也叫**冷启动未命中**，cold-start miss [EF78]）是因为缓存一开始是空的，而这是对该数据项的第一次引用；相比之下，**容量未命中**（capacity miss）是因为缓存空间用完了，必须淘汰一项才能把新的一项放进缓存。第三类未命中（**冲突未命中**，conflict miss）出现在硬件中，原因是硬件缓存对数据项能放在哪里有限制，这和所谓的**组相联**（set-associativity）有关；操作系统的页缓存（page cache）不会出现这种未命中，因为这类缓存总是**全相联**（fully-associative）的，也就是说，页可以放在内存中的任何位置，没有限制。详见 H&P [HP06]。

## 22.3 一个简单的策略：FIFO

许多早期系统不去费力逼近最优，而是采用了非常简单的置换策略（replacement policy）。例如，有些系统采用 **FIFO**（先进先出，first-in, first-out）置换：页进入系统时，只是简单地放进一个队列（queue）；需要置换时，淘汰队列尾部的那一页（即“最先进入”的页）。FIFO 有一个很大的优点：实现起来非常简单。

我们来看看 FIFO 在示例引用序列上的表现（图 22.2，第 215 页）。跟踪过程同样以页 0、1、2 的三次强制性未命中（compulsory miss）开始，然后页 0 和页 1 都命中了。接着引用页 3，未命中；用 FIFO 做置换决定很简单：选最先进来的那一页（图中的缓存状态按 FIFO 顺序排列，最先进入的页在最左边），也就是页 0。不巧的是，下一次访问的正是页 0，于是又一次未命中，又要置换（这次换掉页 1）。之后页 3 命中，但页 1 和页 2 都未命中，最后页 1 命中。

| 访问 | 命中/未命中? | 淘汰 | 访问后的缓存状态 |
|---|---|---|---|
| 0 | 未命中 | | 先入→ 0 |
| 1 | 未命中 | | 先入→ 0, 1 |
| 2 | 未命中 | | 先入→ 0, 1, 2 |
| 0 | 命中 | | 先入→ 0, 1, 2 |
| 1 | 命中 | | 先入→ 0, 1, 2 |
| 3 | 未命中 | 0 | 先入→ 1, 2, 3 |
| 0 | 未命中 | 1 | 先入→ 2, 3, 0 |
| 3 | 命中 | | 先入→ 2, 3, 0 |
| 1 | 未命中 | 2 | 先入→ 3, 0, 1 |
| 2 | 未命中 | 3 | 先入→ 0, 1, 2 |
| 1 | 命中 | | 先入→ 0, 1, 2 |

图 22.2：**FIFO 策略的跟踪过程**

补充：Belady 异常

Belady（就是提出最优策略的那位）和同事发现了一个有趣的引用序列，它的表现有点出人意料 [BNS69]。这个内存引用序列是：1、2、3、4、1、2、5、1、2、3、4、5。他们研究的置换策略（replacement policy）是 FIFO。有趣的地方在于：缓存大小从 3 页增加到 4 页时，缓存命中率会怎样变化。

一般来说，你会预期缓存变大时命中率会上升（变好）。但在这个例子里，用 FIFO 时命中率反而变差了！你可以自己算一算命中和未命中的次数。这种奇怪的现象通常称为 **Belady 异常**（Belady's Anomaly），这让他的合著者们很是郁闷。

另一些策略，比如 LRU，就不会出现这个问题。你能猜到为什么吗？原来，LRU 具有所谓的 **stack property** [M+70]。对具有这种属性的算法来说，大小为 $N+1$ 的缓存自然包含了大小为 $N$ 的缓存中的内容。因此，增大缓存时，命中率要么不变，要么提高。FIFO 和随机策略（还有其他一些策略）显然不满足 stack property，因此可能出现这种异常行为。

和最优策略相比，FIFO 明显差得多：命中率只有 36.4%，不计强制性未命中（compulsory miss）也只有 57.1%。FIFO 根本无法判断各个块（block）的重要程度：尽管页 0 已经被访问过好几次，FIFO 还是把它踢了出去，仅仅因为它是最先被调入内存的那一页。

## 22.4 另一个简单的策略：随机

另一种类似的置换策略（replacement policy）是随机（Random）策略，它在内存压力（memory pressure）下只是随机挑一页来置换。随机策略的特点和 FIFO 类似：实现简单，但在挑选要淘汰的块时，并不费心去做得聪明些。我们来看看随机策略在我们那个著名的示例引用序列上表现如何（见图 22.3）。

| 访问 | 命中/未命中? | 淘汰 | 访问后的缓存状态 |
|---|---|---|---|
| 0 | 未命中 | | 0 |
| 1 | 未命中 | | 0, 1 |
| 2 | 未命中 | | 0, 1, 2 |
| 0 | 命中 | | 0, 1, 2 |
| 1 | 命中 | | 0, 1, 2 |
| 3 | 未命中 | 0 | 1, 2, 3 |
| 0 | 未命中 | 1 | 2, 3, 0 |
| 3 | 命中 | | 2, 3, 0 |
| 1 | 未命中 | 3 | 2, 0, 1 |
| 2 | 命中 | | 2, 0, 1 |
| 1 | 命中 | | 2, 0, 1 |

图 22.3：**随机策略的跟踪过程**

 © 2008–25, ARPACI-DUSSEAU

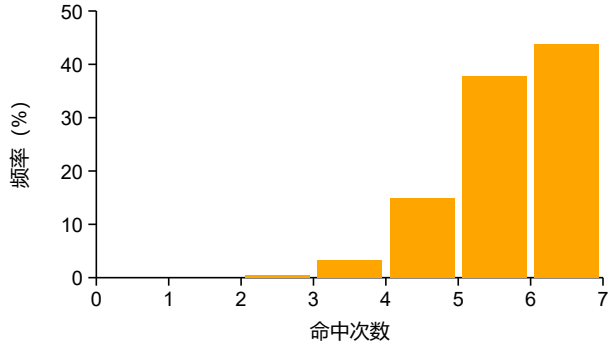



图 22.4：**随机策略在 10,000 次试验中的表现**

当然，随机策略的表现完全取决于它挑选时运气好坏。在上面的例子中，随机策略比 FIFO 稍好一点，比最优策略稍差一点。实际上，我们可以把随机策略的实验跑上几千次，看看它总体上表现如何。图 22.4 展示了随机策略在 10,000 次试验中的命中次数，每次试验使用不同的随机种子。可以看到，有时候（略多于 40% 的情况下），随机策略和最优策略一样好，在这个示例序列上命中 6 次；有时候它又差得多，只命中 2 次甚至更少。随机策略表现如何，全凭手气。

## 22.5 利用历史：LRU

遗憾的是，像 FIFO 或随机这样简单的策略，大概都有一个共同的问题：可能会把重要的页踢出去，也就是马上又要被引用的页。FIFO 踢出的是最先调入的页；如果这一页上恰好有重要的代码或数据结构（data structure），它照样会被扔掉，尽管很快又得把它调回来。因此，FIFO、随机以及类似的策略不太可能接近最优，我们需要更聪明的办法。

就像在调度策略中做过的那样，为了更好地预测未来，我们再一次借助过去，以历史为指导。例如，如果程序最近访问过某一页，那么它很可能在不久的将来再次访问这一页。

页面置换策略（page-replacement policy）能利用的一种历史信息是**频率**（frequency）：如果一页被访问了很多次，也许就不应该置换它，因为它显然有一定价值。更常用的是页的另一个属性，即访问的**近期性**（recency）：一页最近一次被访问的时间越近，也许就越有可能再次被访问。

这一类策略基于人们所说的**局部性原理**（principle of locality）[D70]，这条原理其实只是对程序及其行为的一种观察。简单地说，它指的是：程序往往会非常频繁地访问某些代码序列（例如在循环中）和数据结构（例如循环访问的数组）；因此，我们应该设法利用历史来判断哪些页是重要的，到了要淘汰的时候，把这些页留在内存中。

于是，一类基于历史的简单算法就诞生了。**最不经常使用**（Least-Frequently-Used，**LFU**）策略在必须淘汰时，置换使用频率最低的那一页。类似地，**最近最少使用**（Least-Recently-Used，**LRU**）策略置换最近最少使用的那一页。这些算法很好记：知道了名字，就确切地知道它做什么。对一个名字来说，这是再好不过的特性。

 © 2008–25, ARPACI-DUSSEAU

| 访问 | 命中/未命中? | 淘汰 | 访问后的缓存状态 |
|---|---|---|---|
| 0 | 未命中 | | LRU→ 0 |
| 1 | 未命中 | | LRU→ 0, 1 |
| 2 | 未命中 | | LRU→ 0, 1, 2 |
| 0 | 命中 | | LRU→ 1, 2, 0 |
| 1 | 命中 | | LRU→ 2, 0, 1 |
| 3 | 未命中 | 2 | LRU→ 0, 1, 3 |
| 0 | 命中 | | LRU→ 1, 3, 0 |
| 3 | 命中 | | LRU→ 1, 0, 3 |
| 1 | 命中 | | LRU→ 0, 3, 1 |
| 2 | 未命中 | 0 | LRU→ 3, 1, 2 |
| 1 | 命中 | | LRU→ 3, 2, 1 |

图 22.5：**LRU 策略的跟踪过程**

为了更好地理解 LRU，我们来看看 LRU 在示例引用序列上的表现。图 22.5（第 218 页）展示了结果。从图中可以看到 LRU 怎样利用历史信息，做得比随机或 FIFO 这样的无状态（stateless）策略更好。在这个例子中，LRU 第一次需要置换时淘汰了页 2，因为 0 和 1 都是在它之后才被访问的。接着它置换了页 0，因为 1 和 3 的访问都比它更近。在这两次置换中，LRU 基于历史做出的决定都是正确的，因此接下来的引用都命中了。所以，在我们的例子中，LRU 做到了最好，性能和最优策略一样[2]。

我们还应该指出，这些算法也有与之相反的版本：**最经常使用**（Most-Frequently-Used，**MFU**）和**最近最常使用**（Most-Recently-Used，**MRU**）。在大多数情况下（不是所有情况!），这些策略效果都不好，因为它们非但不利用大多数程序表现出的局部性，反而对它视而不见。

补充：局部性的类型

程序通常会表现出两种局部性（locality）。第一种叫**空间局部性**（spatial locality）：如果页 P 被访问了，那么它周围的页（比如 P－1 或 P＋1）很可能也会被访问。第二种叫**时间局部性**（temporal locality）：最近被访问过的页，很可能在不久的将来再次被访问。硬件系统的缓存层次结构（cache hierarchy）在很大程度上建立在“存在这些局部性”这一假设之上：硬件部署了多级指令缓存、数据缓存和地址转换缓存（address-translation cache），好让程序在存在这种局部性时跑得更快。

当然，人们常说的**局部性原理**（principle of locality），并不是所有程序都必须遵守的铁律。实际上，有些程序访问内存（或磁盘）的方式相当随机，在访问序列中几乎没有或完全没有局部性。因此，虽然设计任何缓存（无论硬件还是软件）时都应该把局部性放在心上，但它并不能保证成功。它只是一种启发式方法，在计算机系统设计中往往很管用。

[2]好吧，结果是我们凑出来的。但有时候，为了说明一个观点，凑结果是必要的。

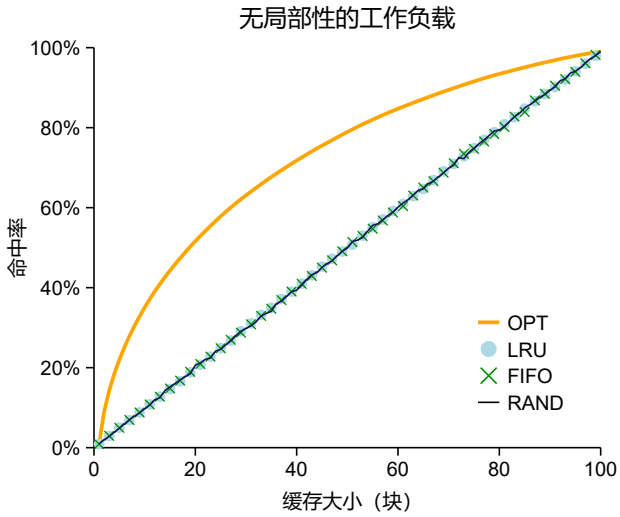



图 22.6：**无局部性的工作负载**

## 22.6 工作负载示例

我们再看几个例子，以便更好地理解其中一些策略的行为。这里我们研究更复杂的**工作负载**（workload），而不是很短的访问序列。不过，即使是这些工作负载，也做了大量简化；更好的研究应该使用真实应用程序的访问记录。

第一个工作负载没有局部性，也就是说，每次引用的都是被访问页集合中的一个随机页。在这个简单的例子中，工作负载一共涉及 100 个不同的页，每次随机选择下一个要引用的页，总计访问 10,000 次。在实验中，我们让缓存大小从很小（1 页）一直变到足以容纳所有不同的页（100 页），以观察各个策略在不同缓存大小下的表现。

图 22.6 画出了最优、LRU、随机和 FIFO 策略的实验结果。图的 y 轴是每个策略达到的命中率；x 轴是按上面所说变化的缓存大小。

从图中可以得出几个结论。第一，工作负载没有局部性时，用哪种实际可行的策略都差不多；LRU、FIFO 和随机策略的表现完全一样，命中率完全由缓存大小决定。第二，缓存大到足以容纳整个工作负载时，用哪种策略也都无所谓；只要被引用的所有块都放得进缓存，所有策略（甚至随机策略）的命中率都会趋近 100%。最后，可以看到最优策略明显好于这些实际可行的策略；要是真能偷看未来，置换的效果会好得多。

我们接下来研究的工作负载叫作“80-20”工作负载，它表现出局部性：80% 的引用落在 20% 的页上（“热”页）；其余 20% 的引用落在另外 80% 的页上（“冷”页）。这个工作负载同样一共有 100 个不同的页；因此，大部分时间引用的是“热”页，其余时间引用的是“冷”页。图 22.7（第 220 页）展示了各个策略在这个工作负载下的表现。

 © 2008–25, ARPACI-DUSSEAU

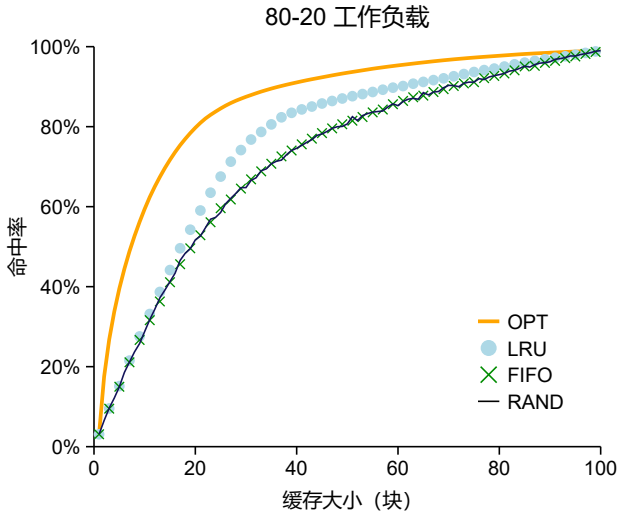



图 22.7：**80-20 工作负载**

从图中可以看出，虽然随机策略和 FIFO 的表现都还不错，但 LRU 更好，因为它更有可能留住热页（hot page）；这些页过去经常被引用，很可能在不久的将来再次被引用。最优策略又一次表现得更好，这说明 LRU 的历史信息并不完美。

你现在可能想问：LRU 相对于随机和 FIFO 的改进，真的有那么重要吗？答案和往常一样：“看情况。”如果每次未命中的代价都很高（这并不少见），那么即使命中率只提高一点点，也就是未命中率（miss rate）只降低一点点，性能也会大不一样。如果未命中的代价没那么高，那么 LRU 可能带来的好处自然也就没那么重要了。

我们再看最后一个工作负载。我们把它叫作“循环顺序”工作负载：按顺序引用 50 个页，从 0 开始，接着是 1、……，一直到页 49，然后循环回来重复这些访问，对这 50 个不同的页总共访问 10,000 次。最后这张图（图 22.8）展示了各个策略在这个工作负载下的表现。

这种工作负载在许多应用程序中都很常见（包括数据库这类重要的商业应用 [CD85]），它对 LRU 和 FIFO 来说都是最坏情况。在循环顺序工作负载下，这两种算法会踢出较老的页；可惜，由于工作负载是循环的，这些较老的页反而会比策略想留在缓存里的那些页更早被访问。实际上，对 50 个页的循环顺序工作负载来说，即使缓存能放下 49 页，命中率也是 0%。有意思的是，随机策略的表现明显更好，虽然离最优还有差距，但至少命中率不为零。看来随机策略也有一些不错的特性，其中之一就是不会在极端情况下出现怪异的行为。

## 22.7 实现基于历史的算法

可以看到，像 LRU 这样的算法通常比 FIFO 或随机这样较简单的策略做得更好，后者可能会扔掉重要的页。遗憾的是，基于历史的策略给我们带来了一个新的挑战：怎么实现它们？

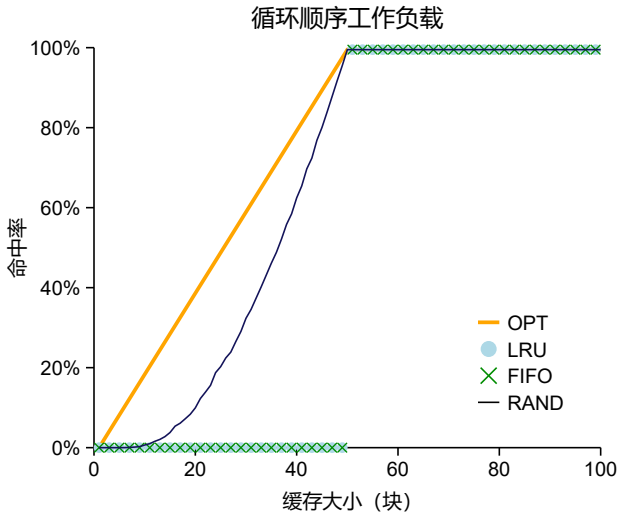



图 22.8：**循环工作负载**

以 LRU 为例。要完美地实现它，需要做很多工作。具体来说，每次访问页时（即每次内存访问，无论是取指令还是加载、存储），我们都必须更新某个数据结构，把这一页移到链表（linked list）的最前面（即 MRU 一侧）。相比之下，FIFO 只有在淘汰一页（移除最先进入的页）或者向链表中加入新页（加到最后进入的一侧）时，才会访问它的页链表。为了记录哪些页是最近最少使用（Least-Recently-Used）的、哪些是最近最常使用（Most-Recently-Used）的，系统必须在每次内存引用时都做一些记录工作。显然，如果不格外小心，这种记录工作可能会大大降低性能。

加快速度的一个办法是加一点硬件支持。例如，机器可以在每次访问页时，更新内存中的一个时间字段。这个字段可以放在每个进程的页表（page table）中，也可以干脆放在内存中某个单独的数组里，系统中每个物理页（physical page）对应一项。这样，一页被访问时，硬件就会把它的时间字段设为当前时间。然后，置换页时，操作系统只需扫描系统中所有的时间字段，就能找到最近最少使用（Least-Recently-Used）的页。

可惜，系统中的页越来越多时，仅仅为了找到绝对意义上最近最少使用（Least-Recently-Used）的那一页，就去扫描一个巨大的时间数组，代价高得让人无法接受。设想一台有 4 GB 内存的现代机器，内存被切成 4 KB 的页。这台机器有 100 万个页，因此即使以现代 CPU 的速度，找到 LRU 页也要花很长时间。这就引出了一个问题：我们真的需要找到绝对最老的那一页来置换吗？用一个近似的结果能不能凑合？

关键问题：如何实现 LRU 置换策略
既然实现完美的 LRU 代价很高，我们能否用某种方式近似它，同时仍然获得想要的效果？

 © 2008–25, ARPACI-DUSSEAU

## 22.8 近似 LRU

答案是肯定的：从计算开销的角度看，近似 LRU 更为可行，而且许多现代系统实际上也正是这么做的。这个想法需要一点硬件支持，即**访问位**（use bit，有时也叫 **reference bit**）。最早的访问位出现在第一个支持分页（paging）的系统，也就是 Atlas 单级存储（one-level store）中 [KE+62]。系统中每一页都有一个访问位，这些访问位存放在内存中的某个地方（比如可以放在每个进程的页表中，或者干脆放在某处的一个数组里）。每当一页被引用（即被读或写）时，硬件就把它的访问位设为 1。不过，硬件从不清除这个位（即把它设为 0），这是操作系统的职责。

操作系统如何利用访问位来近似 LRU 呢？方法可能有很多，**时钟算法**（clock algorithm）[C69] 就是其中一种简单的做法。想象系统中所有的页排成一个环形链表。一根**时钟指针**（clock hand）一开始指向某一页（具体是哪一页并不重要）。需要置换时，操作系统检查当前指向的页 P 的访问位是 1 还是 0。如果是 1，说明页 P 最近被使用过，因此不是置换的好人选。于是把 P 的访问位设为 0（清除），时钟指针（clock hand）前进到下一页（P + 1）。算法一直这样进行下去，直到找到一个访问位为 0 的页，这说明这一页最近没有被使用过（最坏的情况则是所有页最近都被使用过，这时我们已经把所有页搜索了一遍，清除了全部访问位）。

注意，这并不是利用访问位来近似 LRU 的唯一方法。实际上，只要定期清除访问位，再根据各页的访问位是 1 还是 0 来决定置换哪一页，任何这样的方法都可以。Corbato 的时钟算法（clock algorithm）只是早期取得了一定成功的方法之一，它有一个很好的特性：不需要反复扫描整个内存来寻找未被使用的页。

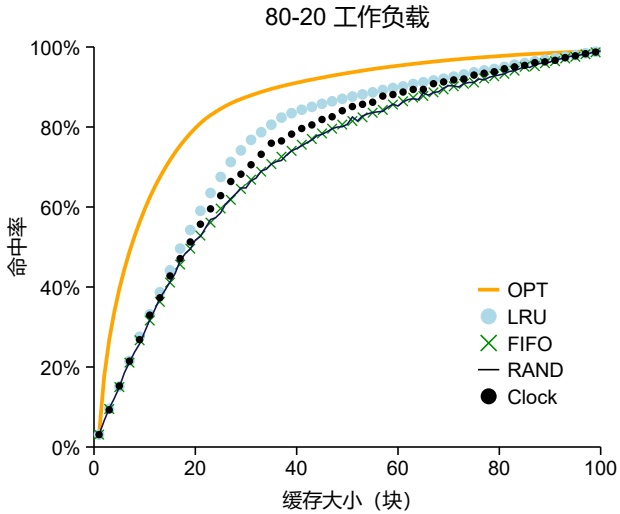



图 22.9：**80-20 工作负载（加入时钟算法）**

图 22.9 展示了时钟算法（clock algorithm）某个变体的表现。这个变体在置换时随机扫描各页：遇到访问位为 1 的页，就清除这个位（即设为 0）；找到访问位为 0 的页，就选它作为淘汰对象。可以看到，虽然它比不上完美的 LRU，但比完全不考虑历史的方法要好。

## 22.9 考虑脏页

对时钟算法（clock algorithm）常做的一个小修改（同样最早由 Corbato 提出 [C69]），是额外考虑一页在内存中是否被修改过。这样做的原因是：如果一页已经被**修改**（modified）过，因而是**脏**（dirty）的，那么要淘汰它就必须先把它写回磁盘，这代价很高。如果它没有被修改过，因而是**干净**（clean）的，淘汰它就没有任何开销；对应的物理帧（physical frame）可以直接另作他用，不需要额外的 I/O。因此，有些虚拟内存系统更倾向于淘汰干净页（clean page）而不是脏页（dirty page）。

为了支持这种行为，硬件应该提供一个**修改位**（modified bit，又称**脏位**，dirty bit）。每当一页被写入，硬件就把这个位设为 1，这样页面置换（page replacement）算法就能把它考虑进来。例如，可以修改时钟算法（clock algorithm），让它优先寻找既未被使用又干净的页来淘汰；如果找不到，再找未被使用但脏的页，依此类推。

## 22.10 其他虚拟内存策略

页面置换（page replacement）并不是虚拟内存子系统采用的唯一策略（尽管它可能是最重要的）。例如，操作系统还必须决定什么时候把一页调入内存。这个策略有时称为**页面选择**（page selection）策略，Denning 就是这么叫它的 [D70]。在这方面，操作系统有几种不同的做法可选。

对大多数页，操作系统只是采用**按需分页**（demand paging），也就是说，在页被访问时才把它调入内存，可以说是“按需”调入。当然，操作系统也可以猜测某一页马上就要被用到，于是提前把它调进来；这种行为称为**预取**（prefetching），只有在有相当把握猜中时才应该这么做。例如，有些系统会假设：如果代码页 P 被调入了内存，那么代码页 P + 1 很可能马上就会被访问，因此也应该一并调入内存。

另一个策略决定操作系统如何把页写到磁盘上。当然，可以一次只写一页；不过，许多系统会先在内存中把一批待写的页收集起来，然后一次（更高效地）写到磁盘上。这种做法通常称为写操作的**聚集**（clustering），或者干脆叫**成组**（grouping）。它之所以有效，是因为磁盘驱动器（disk drive）有这样的特性：执行一次大的写操作，比执行许多次小的写操作效率更高。

## 22.11 抖动

在结束本章之前，我们来讨论最后一个问题：如果内存根本不够用，也就是正在运行的这组进程对内存的需求超出了可用的物理内存，操作系统该怎么办？在这种情况下，系统会不停地换页，这种状况有时称为**抖动**（thrashing）[D70]。

 © 2008–25, ARPACI-DUSSEAU

一些早期的操作系统有一套相当精巧的机制，能在抖动（thrashing）发生时检测到它并加以应对。例如，对于一组进程，系统可以决定不运行其中的一部分，希望剩下那些进程的**工作集**（working set，即它们正在活跃使用的页）能够放进内存，让这些进程得以继续推进。这种方法一般称为**准入控制**（admission control），它的道理是：有时候少做一些但做好，比一下子什么都做却什么都做不好要强。这种情况我们在现实生活中也常常遇到，在现代计算机系统中同样如此（很遗憾）。

当前的一些系统对内存过载采取了更加严厉的做法。例如，某些版本的 Linux 在内存不够用时会运行**内存耗尽杀手**（out-of-memory killer，即 OOM killer）；这个守护进程（daemon）会选出一个占用大量内存的进程并杀掉它，以相当粗暴的方式减少内存占用。虽然这种方法能成功缓解内存压力（memory pressure），但也可能带来问题，比如它要是杀掉了 X 服务器，所有需要显示界面的应用程序就都没法用了。

## 22.12 小结

我们已经介绍了一些页面置换策略（page-replacement policy）以及其他策略，它们是所有现代操作系统虚拟内存子系统的一部分。现代系统会对时钟算法（clock algorithm）这类简单直接的 LRU 近似算法做一些调整；例如，**抗扫描**（scan resistance）是 ARC [MM03] 等许多现代算法的重要组成部分。抗扫描算法（scan-resistant algorithm）通常和 LRU 类似，但会尽量避开 LRU 的最坏情况，也就是我们在循环顺序工作负载中看到的那种情况。因此，页面置换（page replacement）算法仍在不断演进。

过去很多年里，置换算法（replacement algorithm）的重要性有所下降，因为内存访问时间和磁盘访问时间相差实在太大。具体来说，由于换页到磁盘的代价非常高，频繁换页的开销让人无法承受；简单地说，无论你的置换算法（replacement algorithm）多好，只要频繁置换，系统就会慢得无法忍受。因此，最好的解决办法很简单（虽然没什么技术含量）：买更多内存。

然而，近年来基于闪存（flash）的 SSD 等速度快得多的存储设备（storage device）不断推陈出新，又一次改变了这些性能比例，页面置换（page replacement）算法也随之迎来复兴。这方面的最新工作见 [SS10, W+21]。

## 参考文献

[AD03] "Run-Time Adaptation in River" by Remzi H. Arpaci-Dusseau. ACM TOCS, 21:1, February 2003. *A summary of one of the authors' dissertation work on a system named River, where he learned that comparison against the ideal is an important technique for system designers.*

[B66] "A Study of Replacement Algorithms for Virtual-Storage Computer" by Laszlo A. Belady. IBM Systems Journal 5(2): 78-101, 1966. *The paper that introduces the simple way to compute the optimal behavior of a policy (the MIN algorithm).*

[BNS69] "An Anomaly in Space-time Characteristics of Certain Programs Running in a Paging Machine" by L. A. Belady, R. A. Nelson, G. S. Shedler. Communications of the ACM, 12:6, June 1969. *Introduction of the little sequence of memory references known as Belady's Anomaly. How do Nelson and Shedler feel about this name, we wonder?*

[CD85] "An Evaluation of Buffer Management Strategies for Relational Database Systems" by Hong-Tai Chou, David J. DeWitt. VLDB '85, Stockholm, Sweden, August 1985. *A famous database paper on the different buffering strategies you should use under a number of common database access patterns. The more general lesson: if you know something about a workload, you can tailor policies to do better than the general-purpose ones usually found in the OS.*

[C69] "A Paging Experiment with the Multics System" by F.J. Corbato. Included in a Festschrift published in honor of Prof. P.M. Morse. MIT Press, Cambridge, MA, 1969. *The original (and hard to find!) reference to the clock algorithm, though not the first usage of a use bit. Thanks to H. Balakrishnan of MIT for digging up this paper for us.*

[D70] "Virtual Memory" by Peter J. Denning. Computing Surveys, Vol. 2, No. 3, September 1970. *Denning's early and famous survey on virtual memory systems.*

[EF78] "Cold-start vs. Warm-start Miss Ratios" by Malcolm C. Easton, Ronald Fagin. Communications of the ACM, 21:10, October 1978. *A good discussion of cold- vs. warm-start misses.*

[FP89] "Electrochemically Induced Nuclear Fusion of Deuterium" by Martin Fleischmann, Stanley Pons. Journal of Electroanalytical Chemistry, Volume 26, Number 2, Part 1, April, 1989. *The famous paper that would have revolutionized the world in providing an easy way to generate nearly-infinite power from jars of water with a little metal in them. Unfortunately, the results published (and widely publicized) by Pons and Fleischmann were impossible to reproduce, and thus these two well-meaning scientists were discredited (and certainly, mocked). The only guy really happy about this result was Marvin Hawkins, whose name was left off this paper even though he participated in the work, thus avoiding association with one of the biggest scientific goofs of the 20th century.*

[HP06] "Computer Architecture: A Quantitative Approach" by John Hennessy and David Patterson. Morgan-Kaufmann, 2006. *A marvelous book about computer architecture. Read it!*

[H87] "Aspects of Cache Memory and Instruction Buffer Performance" by Mark D. Hill. Ph.D. Dissertation, U.C. Berkeley, 1987. *Mark Hill, in his dissertation work, introduced the Three C's, which later gained wide popularity with its inclusion in H&P [HP06]. The quote from therein: "I have found it useful to partition misses … into three components intuitively based on the cause of the misses (page 49)."*

[KE+62] "One-level Storage System" by T. Kilburn, D.B.G. Edwards, M.J. Lanigan, F.H. Sumner. IRE Trans. EC-11:2, 1962. *Although Atlas had a use bit, it only had a very small number of pages, and thus the scanning of the use bits in large memories was not a problem the authors solved.*

[M+70] "Evaluation Techniques for Storage Hierarchies" by R. L. Mattson, J. Gecsei, D. R. Slutz, I. L. Traiger. IBM Systems Journal, Volume 9:2, 1970. *A paper that is mostly about how to simulate cache hierarchies efficiently; certainly a classic in that regard, as well for its excellent discussion of some of the properties of various replacement algorithms. Can you figure out why the stack property might be useful for simulating a lot of different-sized caches at once?*

[MM03] "ARC: A Self-Tuning, Low Overhead Replacement Cache" by Nimrod Megiddo and Dharmendra S. Modha. FAST 2003, February 2003, San Jose, California. *An excellent modern paper about replacement algorithms, which includes a new policy, ARC, that is now used in some systems. Recognized in 2014 as a "Test of Time" award winner by the storage systems community at the FAST '14 conference.*

[SS10] "FlashVM: Virtual Memory Management on Flash" by Mohit Saxena, Michael M. Swift. USENIX ATC '10, June, 2010, Boston, MA. *An early, excellent paper by our colleagues at U. Wisconsin about how to use Flash for paging. One interesting twist is how the system has to take **wearout**, an intrinsic property of Flash-based devices, into account. Read more about Flash-based SSDs later in this book if you are interested.*

[W+21] "The Storage Hierarchy is Not a Hierarchy: Optimizing Caching on Modern Storage Devices with Orthus" by Kan Wu, Zhihan Guo, Guanzhou Hu, Kaiwei Tu, Ramnatthan Alagappan, Rathijit Sen, Kwanghyun Park, Andrea Arpaci-Dusseau, Remzi Arpaci-Dusseau. FAST '21, held virtually — thanks, COVID-19. *Our own work on a caching approach on modern devices; is it directly related to page replacement? Perhaps not. But it is a fun paper.*

## 作业（模拟）

模拟器 `paging-policy.py` 可以让你试验各种页面置换策略。详情请见 README。

## 问题

1. 用以下选项生成随机地址：`-s 0 -n 10`、`-s 1 -n 10` 和 `-s 2 -n 10`。把策略依次改为 FIFO、LRU、OPT。判断这些地址序列中的每次访问是命中还是未命中。
2. 对于大小为 5 的缓存，分别为以下策略生成最坏情况的地址引用序列：FIFO、LRU 和 MRU（最坏情况的引用序列会导致尽可能多的未命中）。对于这些最坏情况的引用序列，缓存要大多少，性能才会显著提高并接近 OPT？
3. 生成一个随机的访问序列（即用 Python 写一个脚本，输出随机地址，然后把这些地址输入模拟器）。你预计不同的策略在这样的序列上会表现如何？
4. 现在生成一个具有一定局部性的访问序列。怎样才能生成这样的序列？LRU 在它上面表现如何？LRU 比 RAND 好多少？CLOCK 表现如何？如果 CLOCK 使用不同数量的时钟位，结果又如何？
5. 用 valgrind 这样的程序对一个真实应用程序插桩，生成虚拟页引用序列。例如，运行 `valgrind --tool=lackey --trace-mem=yes ls` 会输出一份近乎完整的访问记录，包含程序 `ls` 的每一次指令引用和数据引用。要让它能用于上面的模拟器，你得先把每个虚拟内存引用转换成虚拟页号（virtual page number）引用（方法是屏蔽掉偏移量，再把剩下的位右移）。对你的应用程序的访问序列来说，缓存要多大才能满足大部分请求？画一张图，展示缓存逐渐增大时它的工作集（working set）情况。

 © 2008–25, ARPACI-DUSSEAU

# 23

# 完整的虚拟内存系统

在结束对内存虚拟化（memory virtualization）的学习之前，我们来更仔细地看看完整的虚拟内存（virtual memory）系统是怎样搭建起来的。我们已经见过这类系统的关键要素，包括各种页表（page table）设计、与 TLB 的交互（有时甚至由操作系统自己处理），以及决定哪些页（page）留在内存、哪些页踢出内存的策略。然而，一个完整的虚拟内存系统还包含许多其他特性，涉及性能、功能和安全等诸多方面。于是，我们的关键问题是：

关键问题：如何构建完整的虚拟内存系统
实现一个完整的虚拟内存系统需要哪些特性？它们如何提高性能、增强安全性，或以其他方式改进系统？

我们会通过介绍两个系统来回答这个问题。第一个是最早的“现代”虚拟内存管理器之一，即 **VAX/VMS** 操作系统中的虚拟内存管理器 [LL82]，它开发于 20 世纪 70 年代到 80 年代初；这个系统里有多得惊人的技术和方法一直沿用至今，因此非常值得研究。有些思想即使已有 50 年历史，仍然值得了解；这个道理在其他大多数领域（比如物理学）的人看来再平常不过，但在技术驱动的学科（比如计算机科学）里却得专门说一说。

第二个是 **Linux**，原因应该不言而喻。Linux 应用广泛，能高效地运行在各种系统上，小到性能孱弱的手机，大到现代数据中心里可扩展性（scalability）最强的多核（multicore）系统。因此，它的虚拟内存系统必须足够灵活，才能在所有这些场景下都顺利运行。我们会逐一讨论这两个系统，看看前几章介绍的概念是如何在一个完整的内存管理器中融为一体的。

## 23.1 VAX/VMS 虚拟内存

20 世纪 70 年代末，**数字设备公司**（Digital Equipment Corporation，**DEC**）推出了 VAX-11 小型机（minicomputer）体系结构。在小型机时代，DEC 是计算机行业的巨头；可惜，一系列错误的决策，再加上 PC 的兴起，让它慢慢地（但不可避免地）走向了消亡 [C03]。这个体系结构有好几种实现，包括 VAX-11/780 和性能稍弱的 VAX-11/750。

这个系统的操作系统叫 VAX/VMS（或者干脆叫 VMS），它的主要设计者之一是 Dave Cutler，他后来主持开发了微软的 Windows NT [C93]。VMS 面临一个通用性问题：它要运行在各种各样的机器上，从非常便宜的 VAXen（没错，复数就是这么写的），到同一体系

结构家族中极其高端、强大的机器。因此，操作系统的机制和策略必须在差异如此之大的各种系统上都能工作（而且要工作得很好）。

另外，VMS 还是一个很好的例子，展示了如何用软件创新来掩盖体系结构的一些固有缺陷。操作系统常常要依靠硬件来构建高效的抽象（abstraction）和假象，但硬件设计者有时并不能把每件事都做对；我们会在 VAX 硬件中看到几个这样的例子，也会看到 VMS 操作系统如何在硬件存在这些缺陷的情况下，依然构建出一个有效、能用的系统。

### 内存管理硬件

VAX-11 为每个进程（process）提供了一个 32 位的虚拟地址空间（virtual address space），划分成 512 字节的页。因此，虚拟地址（virtual address）由 23 位的 VPN 和 9 位的偏移量（offset）组成。此外，VPN 的高两位用来区分页属于哪个段（segment）；因此，正如我们之前见过的，这个系统是分页（paging）和分段（segmentation）的混合体。

地址空间（address space）的下半部分称为“进程空间”（process space），每个进程各有一份。进程空间（process space）的前一半称为 P0，里面放着用户程序，还有一个向下增长的堆（heap）；后一半称为 P1，里面是向上增长的栈（stack）。地址空间的上半部分称为系统空间（system space，S），不过只用到了其中的一半。受保护的操作系统代码和数据就放在这里，所有进程也就以这种方式共享了操作系统。

VMS 设计者的一大顾虑是 VAX 硬件的页小得出奇（512 字节）。这个大小是出于历史原因选定的，它有一个根本性的问题：会让简单的线性页表（linear page table）变得过大。因此，VMS 设计者最早确立的目标之一，就是确保 VMS 不会让页表把内存撑爆。

这个系统用两种办法减轻页表给内存带来的压力。第一，VAX-11 把用户地址空间分成两段，每个进程的这两个区域（P0 和 P1）各有一张页表；这样，栈和堆之间未使用的那部分地址空间就不需要页表空间了。基址（base）和界限寄存器（bounds register）的用法和你预想的一样：基址寄存器（base register）保存该段页表的地址，界限寄存器保存页表的大小，也就是页表项（page table entry）的个数。

第二，操作系统把用户页表放在内核虚拟内存（kernel virtual memory）中，进一步减轻了内存压力（memory pressure）；用户页表 P0 和 P1 各一张，因此每个进程有两张。这样，分配或扩大页表时，内核从自己位于 S 段的虚拟内存中分配空间。如果内存压力（memory pressure）非常大，内核可以把这些页表的页换出（swap out）到磁盘，从而腾出物理内存（physical memory）另作他用。

补充：通用性的诅咒

操作系统常常受困于所谓的**通用性诅咒**（the curse of generality）：它们要为一大类应用程序和系统提供通用的支持。这个诅咒的根本后果是，操作系统很难对某一种具体环境支持得特别好。对 VMS 来说，这个诅咒实实在在，因为 VAX-11 体系结构有许多种不同的实现。今天也一样：人们指望 Linux 在你的手机、电视机顶盒、笔记本电脑、台式机上都跑得好，也指望它在云数据中心里运行着数千个进程的高端服务器上跑得好。

把页表放在内核虚拟内存（kernel virtual memory）中，意味着地址转换（address translation）会更加复杂。例如，要转换 P0 或 P1 中的一个虚拟地址，硬件首先要在这个进程的 P0 或 P1 页表中查找该页的页表项；而要找到这个页表项，硬件可能得先查位于物理内存中的系统页表；这次转换完成后，硬件才知道页表的那一页在哪里，最后才能得到要访问的内存地址。好在 VAX 有硬件管理的 TLB（hardware-managed TLB），能让这一切快起来：它通常（但愿如此）能绕过这种费力的查找。

### 真实的地址空间

研究 VMS 有一个很妙的地方：我们可以看到一个真实的地址空间是如何构造的（图 23.1）。到目前为止，我们一直假设地址空间很简单，只有用户代码、用户数据和用户堆（user heap）；但从图中可以看出，真实的地址空间要复杂得多。

例如，代码段（code segment）永远不会从第 0 页开始。第 0 页被标记为不可访问，以便在一定程度上支持检测**空指针**（null-pointer）访问。可见，设计地址空间时要考虑的一点是对调试的支持，这里不可访问的第 0 页就以某种形式提供了这种支持。

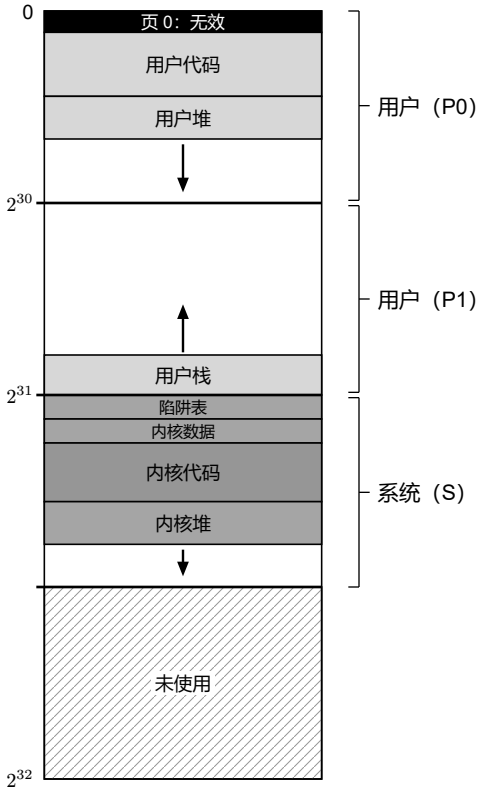



图 23.1：**VAX/VMS 的地址空间**

 © 2008–23, ARPACI-DUSSEAU

补充：为什么空指针访问会导致段错误

现在你应该很清楚，解引用空指针时到底发生了什么。进程像下面这样，产生一个值为 0 的虚拟地址：

```
int *p = NULL; // 设置 p = 0
*p = 10;       // 试图把 10 存到虚拟地址 0
```

硬件试图在 TLB 中查找这个 VPN（这里也是 0），结果 TLB 未命中（TLB miss）。于是去查页表，发现 VPN 0 对应的表项被标记为无效。这样就产生了一次无效访问，控制权转交给操作系统，操作系统很可能会终止这个进程。在 UNIX 系统上，进程会收到一个信号（signal），可以借此对这种错误做出反应；但如果没有捕获这个信号，进程就会被杀掉。

也许更重要的是，内核虚拟地址空间（kernel virtual address space），也就是内核的数据结构（data structure）和代码，是每个用户地址空间的一部分。上下文切换（context switch）时，操作系统修改 `P0` 和 `P1` 寄存器，让它们指向即将运行的那个进程的页表；但它不修改 `S` 的基址（base）和界限寄存器，因此每个用户地址空间里映射的都是“同一套”内核结构。

把内核映射到每个地址空间中，是出于好几方面的考虑。这种做法让内核轻松了不少；例如，在 `write()` 这样的系统调用（system call）中，操作系统从用户程序那里拿到一个指针时，可以很容易地把数据从这个指针处复制到自己的结构中。操作系统可以照常编写和编译，不必操心它访问的数据从哪里来。相反，如果内核完全位于物理内存中，要做把页表的页换出到磁盘这类事情就会相当困难；如果给内核单独一个地址空间，在用户应用程序和内核之间移动数据又会变得复杂而痛苦。采用这种做法（如今已被广泛使用）后，在应用程序看来，内核几乎就像一个库，只不过是受保护的库。

关于这个地址空间，最后一点和保护有关。显然，操作系统不希望用户应用程序读写操作系统的数据或代码。因此，硬件必须支持给页设置不同的保护级别。VAX 的做法是在页表的保护位（protection bit）中指明，CPU 必须处于什么特权级别才能访问某一页。这样，系统数据和代码的保护级别就比用户数据和代码更高；用户代码试图访问这类信息时，就会陷入操作系统，然后（你猜对了）这个出错的进程多半会被终止。

### 页面置换

VAX 中的页表项（PTE）包含以下各位：一个有效位（valid bit）、一个保护字段（4 位）、一个修改位（modified bit，也叫脏位，dirty bit）、一个留给操作系统使用的字段（5 位），最后是物理帧号（PFN），用来记录该页在物理内存中的位置。细心的读者可能已经注意到：没有**访问位**（reference bit）！因此，VMS 的置换算法（replacement algorithm）只能在没有硬件支持的情况下，设法判断哪些页是活跃的。

开发者还担心**内存大户**（memory hogs），也就是那些占用大量内存、让其他程序难以运行的程序。到目前为止我们看过的大多数策略，都难以防止这种霸占行为；例如，LRU 是一种*全局*策略，它不会在进程之间公平地分配内存。

补充：模拟访问位

其实，要想大致知道系统中哪些页正在使用，并不一定需要硬件访问位。早在 20 世纪 80 年代初，Babaoglu 和 Joy 就证明了可以用 VAX 上的保护位（protection bit）来模拟访问位 [BJ81]。基本思路是：如果你想知道系统中哪些页正处于活跃使用状态，就把页表中的所有页都标记为不可访问（但要把进程实际可以访问哪些页的信息保存下来，比如放在页表项的“保留给操作系统的字段”中）。进程访问某一页时，就会陷入操作系统；操作系统随后检查这一页是否确实允许访问，如果是，就把它恢复成正常的保护状态（例如只读或读写）。到了置换的时候，操作系统可以检查哪些页仍然被标记为不可访问，从而知道哪些页最近没被用过。

这种“模拟”访问位的关键，是在降低开销的同时，仍能较好地掌握页的使用情况。操作系统不能太积极地把页标记为不可访问，否则开销会太高；也不能太消极，否则所有页最后都会显示为已访问，操作系统又无从判断该淘汰（evict）哪一页了。

为了解决这两个问题，开发者提出了 **segmented FIFO** 置换策略（replacement policy）[RL81]。思路很简单：每个进程能保留在内存中的页数有一个上限，称为它的**驻留集大小**（resident set size，**RSS**）。这些页都放在一个 FIFO 链表（linked list）里；当进程超出它的 RSS 时，就淘汰“最先进入”的那一页。FIFO 显然不需要任何硬件支持，因此很容易实现。

当然，正如我们前面看到的，纯 FIFO 的表现并不算好。为了改进 FIFO 的性能，VMS 引入了两个**二次机会链表**（second-chance lists）：一个全局的*干净页空闲链表*（clean-page free list）和一个*脏页链表*（dirty-page list），页在被淘汰出内存之前，会先放到这两个链表里。当进程 P 超出它的 RSS 时，就从该进程自己的 FIFO 中移出一页；如果这一页是干净的（没被修改过），就放到干净页链表（clean-page free list）的末尾；如果是脏的（被修改过），就放到脏页链表（dirty-page list）的末尾。

如果另一个进程 Q 需要一个空闲页（free page），它就从全局干净页空闲链表（clean-page free list）中取走第一个空闲页。不过，如果在这一页被回收之前，原来的进程 P 又访问它、发生了缺页，P 就会从空闲链表（free list）或脏页链表（dirty-page list）中把它收回来，从而省掉一次代价高昂的磁盘访问。这两个全局二次机会链表（second-chance list）越大，segmented FIFO 算法的表现就越接近 LRU [RL81]。

VMS 用到的另一项优化，也有助于弥补 VMS 页太小的问题。具体来说，页这么小，交换时的磁盘 I/O 可能非常低效，因为磁盘更擅长大块传输。为了让交换 I/O 更高效，VMS 做了许多优化，其中最重要的是**聚集**（clustering）。有了聚集，VMS 会把全局脏页链表（dirty-page list）中的一大批页归到一起，一次性写到磁盘上（这些页也就变干净了）。大多数现代系统都使用聚集：页可以放在交换空间（swap space）中的任意位置，操作系统因此能把页归成组，用次数更少、规模更大的写操作写出去，从而提高性能。

### 其他巧妙的技巧

VMS 还有另外两个如今已成标配的技巧：按需置零（demand zeroing）和写时复制（copy-on-write）。我们现在来介绍这些**惰性**（lazy）优化。**按需置零**（demand zeroing）就

是 VMS（以及大多数现代系统）中的一种惰性做法。为了更好地理解它，我们来看往地址空间里添加一页的例子，比如在堆里添加一页。朴素的实现是这样的：操作系统收到往堆里添加一页的请求后，在物理内存中找一页，把它清零（出于安全考虑必须这么做，否则你就能看到别的进程用这一页时留下的内容了！），然后把它映射到你的地址空间中，也就是按需设置页表，让它指向那个物理页（physical page）。但这种朴素的实现可能代价很高，尤其是进程根本没用到这一页的时候。

有了按需置零（demand zeroing），操作系统在把页加入地址空间时只做很少的工作：在页表中放一个表项，把这一页标记为不可访问。如果进程随后读或写这一页，就会陷入操作系统。处理这个陷阱（trap）时，操作系统会发现这其实是一个按需置零（demand zeroing）的页（通常是靠页表项中“保留给操作系统”那部分里标记的某些位）；这时，操作系统才去做必要的工作：找一个物理页，把它清零，然后映射到进程的地址空间中。如果进程从不访问这一页，这些工作就全都省掉了，这正是按需置零（demand zeroing）的好处。

VMS 中另一项很酷的优化（同样，几乎每个现代操作系统中都有）是**写时复制**（copy-on-write，简称 **COW**）。这个想法至少可以追溯到 TENEX 操作系统 [BB+72]，思路很简单：当操作系统需要把一页从一个地址空间复制到另一个地址空间时，可以不真的复制，而是把它映射到目标地址空间中，并在两个地址空间中都把它标记为只读。如果两个地址空间都只读这一页，那就什么也不用做；操作系统就这样完成了一次快速复制，实际上却没有移动任何数据。

但是，如果其中一个地址空间真的试图写这一页，就会陷入操作系统。操作系统随后会发现这是一个 COW 页，于是（惰性地）分配一个新页，把数据填进去，再把这个新页映射到引发陷阱（trap）的那个进程的地址空间中。然后进程继续运行，这时它已经有了这一页的私有副本。

COW 的用处有很多。比如，任何共享库都可以用写时复制（copy-on-write）的方式映射到许多进程的地址空间中，从而节省宝贵的内存空间。在 UNIX 系统中，由于 `fork()` 和 `exec()` 的语义，COW 就更关键了。你可能还记得，`fork()` 会创建调用者地址空间的一个完全相同的副本；如果地址空间很大，复制它既慢又要搬动大量数据。更糟的是，地址空间的大部分马上就会被随后的 `exec()` 调用覆盖：`exec()` 会用即将执行的程序的地址空间替换调用进程的地址空间。如果改用写时复制（copy-on-write）来实现 `fork()`，操作系统就能省掉大部分不必要的复制，既保持了正确的语义，又提高了性能。

## 23.2 Linux 虚拟内存系统

我们现在来讨论 Linux 虚拟内存系统中一些比较有意思的方面。推动 Linux 发展的，是一群在解决生产环境中真实问题的真正工程师；因此，Linux 一点一点地加入了大量特性，最终形成了今天这个功能完备、特性丰富的虚拟内存系统。

我们无法讨论 Linux 虚拟内存的方方面面，但会涉及其中最重要的部分，尤其是它超越 VAX/VMS 这类经典虚拟内存系统的地方。我们也会尽量指出 Linux 和老系统之间的共同之处。

提示：要懒

无论是在生活中还是在操作系统中，懒都可以是一种美德。懒就是把工作推迟到以后再做，这在操作系统中有好几方面的好处。首先，推迟工作可能会降低当前操作的延迟，从而提高响应速度；例如，操作系统常常会立即报告对文件的写操作已经成功，等以后再在后台把它们写到磁盘上。其次，也是更重要的一点，懒有时能让这项工作根本不必做；例如，一次写操作一直推迟到文件被删除，这次写操作就完全省掉了。在生活中懒也有好处：比如，把操作系统课程作业往后拖，你可能会发现作业说明里的错误已经被同学们弄清楚了。不过，课程作业不太可能被取消，所以太懒也会出问题：作业交晚了，分数难看，教授伤心。别让教授伤心！

这里的讨论以 Intel x86 上的 Linux 为主。Linux 虽然能够（而且确实）运行在许多不同的处理器体系结构上，但 x86 上的 Linux 是它最主流、最重要的部署形态，因此也是我们关注的重点。

### Linux 的地址空间

和其他现代操作系统一样，也和 VAX/VMS 一样，Linux 的虚拟地址空间[1]由用户部分（存放用户程序代码、栈、堆和其他部分）和内核部分（存放内核代码、栈、堆和其他部分）组成。和那些系统一样，上下文切换时，当前运行的地址空间中，用户部分会随之改变，而内核部分在所有进程中都相同。也和那些系统一样，运行在用户模式（user mode）下的程序不能访问内核的虚拟页（virtual page）；只有陷入内核、切换到特权模式（privileged mode）之后，才能访问这部分内存。

在经典的 32 位 Linux 中，也就是虚拟地址空间为 32 位的 Linux 中，地址空间的用户部分和内核部分以地址 `0xC0000000` 为界，也就是在整个地址空间的四分之三处。因此，虚拟地址 `0` 到 `0xBFFFFFFF` 是用户虚拟地址；其余的虚拟地址（`0xC0000000` 到 `0xFFFFFFFF`）属于内核的虚拟地址空间。64 位 Linux 也有类似的划分，只是分界点略有不同。图 23.2 展示了一个典型的（简化的）地址空间。

Linux 有个小地方挺有意思：它有两种内核虚拟地址（kernel virtual address）。第一种称为**内核逻辑地址**（kernel logical addresses）[O16]。你可以把它看作内核通常意义上的虚拟地址空间；要获得更多这类内存，内核代码只需调用 **`kmalloc`**。大多数内核数据结构都位于这里，比如页表、每个进程的内核栈（kernel stack）等等。和系统中的大多数其他内存不同，内核逻辑内存不能被换出到磁盘。

内核逻辑地址（kernel logical address）最有意思的地方，是它们和物理内存之间的联系。具体来说，内核逻辑地址（kernel logical address）直接映射（direct-mapped）到物理内存的开头部分。因此，内核逻辑地址（kernel logical address）`0xC0000000` 对应物理地址（physical address）`0x00000000`，`0xC0000FFF` 对应 `0x00000FFF`，依此类推。这种直接映射意味着两点。第一，内核逻辑地址（kernel logical address）和物理地址之间的相互转换很简单；因此，这些地址常常被直接当作物理地址看待。第二，如果一块内存在内核

[1]至少在最近因为安全威胁而做出改动之前是这样的。关于这一改动的细节，请阅读下文关于 Linux 安全的小节。

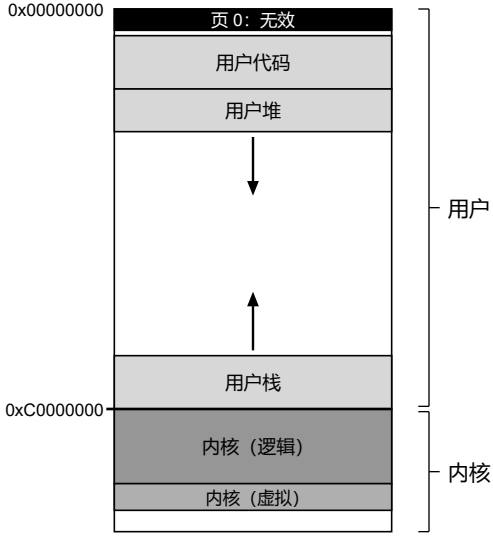



图 23.2：**Linux 的地址空间**

逻辑地址（kernel logical address）空间中是连续的，那么它在物理内存中也是连续的。因此，从内核地址空间（kernel address space）这一部分分配的内存，适合用于那些需要连续物理内存才能正确工作的操作，比如通过**直接内存访问**（direct memory access，**DMA**）进出设备的 I/O 传输（本书第三部分会讲到这个话题）。

另一种内核地址是**内核虚拟地址**（kernel virtual address）。要获得这类内存，内核代码要调用另一个分配器（allocator）`vmalloc`，它返回一个指针，指向一块所需大小、在虚拟地址上连续的区域。和内核逻辑内存不同，内核虚拟内存（kernel virtual memory）通常不连续：每个内核虚拟页都可能映射到不相邻的物理页（因此不适合用于 DMA）。不过也正因为如此，这类内存更容易分配，常用于大缓冲区（buffer）：在这种场合，要找到一大块连续的物理内存很难。

在 32 位 Linux 中，内核虚拟地址（kernel virtual address）存在的另一个原因是，它们让内核能够寻址超过（大约）1 GB 的内存。多年以前，机器的内存比这少得多，能否访问超过 1 GB 的内存根本不是问题。但技术不断进步，很快内核就需要用上更多内存了。正因为内核虚拟地址（kernel virtual address）不必和物理内存严格一一对应，内核才用得上这么多内存。不过，转向 64 位 Linux 之后，这种需求就没那么迫切了，因为内核不再局限于虚拟地址空间中最后那 1 GB。

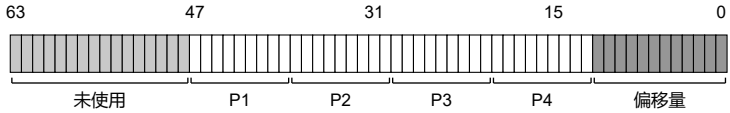
### 页表结构

我们关注的是 x86 上的 Linux，所以下面围绕 x86 提供的页表结构来讨论，它决定了 Linux 能做什么、不能做什么。如前所述，x86 提供了一种由硬件管理的多级页表（multi-level page table）结构，每个进程一张页表；操作系统只需在自己的内存中建立映射，把一个特权寄存器指向页目录（page directory）的起始位置，剩下的事情就交给硬件处理。不出所

料，操作系统会在创建进程、删除进程以及上下文切换时介入，确保每种情况下硬件 MMU 用来转换的都是正确的页表。

近年来最大的变化，可能就是前面简单提到的从 32 位 x86 转向 64 位 x86。正如在 VAX/VMS 系统中看到的，32 位地址空间已经存在了很长时间，随着技术的发展，它终于开始真正成为程序的限制。虚拟内存让系统编程（systems programming）变得容易，但现代系统动辄有好几 GB 内存，32 位已经不够用来引用其中的每一个字节了。因此，下一次飞跃就成了必然。

转向 64 位地址，对 x86 页表结构的影响和预想的一样。x86 用的是多级页表，当前的 64 位系统使用四级页表。不过，64 位虚拟地址空间目前还没有全部用上，只用了低 48 位。因此，虚拟地址可以看成下面这样：




从图中可以看到，虚拟地址的高 16 位没有使用（因此在转换中不起任何作用），页大小（page size）是 4 KB，所以低 12 位用作偏移量，直接使用，不参与转换；剩下中间的 36 位虚拟地址参与转换。地址中的 P1 部分用来索引最顶层的页目录，然后从这里开始一级一级地转换，直到用 P4 去索引实际的那一页页表，得到所需的页表项。

随着系统内存进一步增大，这个庞大地址空间中会有更多部分被启用，从而出现五级、最终甚至六级的页表树结构。想象一下：只是为了弄清某块数据在内存中的位置，一次简单的页表查找就要经过六级转换。

### 大页支持

Intel x86 允许使用多种页大小，不只是标准的 4 KB 页。具体来说，近来的设计在硬件上支持 2 MB 甚至 1 GB 的页。因此，随着时间推移，Linux 也逐步支持应用程序使用这些**大页**（Linux 里叫 huge pages）。

前面暗示过，使用大页（huge page）有很多好处。正如在 VAX/VMS 中看到的，大页（huge page）可以减少页表中所需的映射数量；页越大，映射就越少。不过，页表项变少并不是推动大页（huge page）的主要动力，真正的动力是更好的 TLB 表现以及随之而来的性能提升。

进程频繁使用大量内存时，它的地址转换很快就会把 TLB 装满。如果这些转换针对的都是 4 KB 的页，那么不引发 TLB 未命中就能访问的内存总量就很少。结果，对于运行在好几 GB 内存机器上的现代“大内存”工作负载（workload）来说，这会带来明显的性能损失；最近的研究表明，有些应用程序要花 10% 的周期来处理 TLB 未命中 [B+13]。

大页（huge page）占用的 TLB 槽位更少，进程因此能在不发生 TLB 未命中的情况下访问一大片内存，这是大页的主要优点。不过，大页（huge page）还有其他好处：TLB 未命中的处理路径更短，也就是说，真的发生 TLB 未命中时，处理得更快。此外，（在某些场景下）分配也可以非常快，这个好处虽小，有时却很重要。

提示：考虑渐进式改进

人生中，你常常会被鼓励去做一个革命者。“要胸怀大志!”他们说。“去改变世界!”他们喊。你也明白这为什么诱人：有些时候确实需要大变革，大力推动它也很有道理。而且，照这么做的话，至少他们可能就不会再冲你嚷嚷了。

然而在很多情况下，更慢、更渐进的方法才是正确的做法。本章中 Linux 的大页（huge page）就是工程上渐进主义的一个例子：开发者没有摆出原教旨主义者的姿态、坚称大页就是未来，而是采取了稳妥的做法，先为它引入专门的支持，进一步了解它的优缺点，等到真有必要时，才为所有应用程序加入更通用的支持。

渐进主义有时会遭人轻视，但它常常能带来缓慢、深思熟虑而又合理的进步。构建系统时，这样的方法也许正是你需要的。其实，人生或许也是如此。

Linux 对大页（huge page）的支持有个有意思的地方：它是一步一步加进来的。起初，Linux 开发者知道这种支持只对少数应用程序重要，比如对性能要求很高的大型数据库。因此，他们决定让应用程序显式申请用大页（huge page）来分配内存（通过 `mmap()` 或 `shmget()` 调用）。这样，大多数应用程序不受影响（继续只使用 4 KB 的页）；少数要求很高的应用程序需要修改代码来使用这些接口，但对它们来说，这点麻烦是值得的。

最近，越来越多的应用程序都需要更好的 TLB 表现，于是 Linux 开发者加入了**透明**大页（transparent huge page）支持。启用这一特性后，操作系统会自动寻找机会分配大页（huge page，通常是 2 MB，在某些系统上是 1 GB），应用程序无须修改。

大页（huge page）也不是没有代价。最大的潜在代价是**内部碎片**（internal fragmentation），也就是一个很大的页却只用了很少一部分。这种浪费可能让内存被又大又没怎么用的页占满。交换（如果启用的话）和大页（huge page）也配合得不好，有时会让系统的 I/O 量大大增加。在另一些情况下，分配的开销也可能很大。总的来说，有一点很清楚：多年来为系统立下汗马功劳的 4 KB 页大小，已经不再是以前那种万能的解决方案了；内存容量不断增长，要求我们把大页（huge page）和其他方案纳入考虑，作为虚拟内存系统必要演进的一部分。Linux 慢慢采纳这项基于硬件的技术，本身就说明变化即将到来。

### 页缓存

为了降低访问持久存储（persistent storage，本书第三部分的重点）的开销，大多数系统都使用积极的**缓存**（caching）子系统，把常用的数据保留在内存中。在这方面，Linux 和传统操作系统没什么不同。

Linux 的**页缓存**（page cache）是统一的，它把来自三个主要来源的页保留在内存中：**内存映射文件**（memory-mapped files）；来自设备的文件数据和元数据（metadata），它们通常是通过对文件系统（file system）发出 `read()` 和 `write()` 调用来访问的；以及构成每个进程的堆页（heap page）和栈页（stack page），这类页有时称为**匿名内存**（anonymous memory），因为它背后没有具名文件，只有交换空间。这些页保存在一张**页缓存散列表**（page cache hash table）中，需要时可以快速查到。

页缓存记录每个条目是**干净的**（clean，读过但未更新）还是**脏的**（dirty，也叫**已修改**，modified）。脏数据会由名为 `pdflush` 的后台线程（thread）定期写回后备存储（backing store）：文件数据写回对应的文件，匿名区域写到交换空间；这样就能确保修改过的数据最终会被写回持久存储。这种后台活动要么在经过一段时间后进行，要么在脏页（dirty page）太多时进行（两者都是可配置的参数）。

有时系统内存不足，Linux 就必须决定把哪些页踢出内存以腾出空间。为此，Linux 使用了一种改进版的 **2Q** 置换算法（replacement algorithm）[JS94]，下面介绍一下。

基本思路很简单：标准的 LRU 置换很有效，但某些常见的访问模式会让它失灵。例如，如果一个进程反复访问一个大文件（尤其是大小接近内存甚至更大的文件），LRU 会把其他所有文件都踢出内存。更糟的是：把这个文件的部分内容保留在内存中并没有用，因为它们在被踢出内存之前根本不会被再次访问。

Linux 版的 2Q 置换算法这样解决这个问题：维护两个链表，把内存分给它们。一页第一次被访问时，会被放到一个队列（queue）中（原论文中称为 A1，在 Linux 中称为**不活跃链表**，inactive list）；再次被访问时，就提升到另一个队列中（原论文中称为 Aq，在 Linux 中称为**活跃链表**，active list）。需要置换时，从不活跃链表（inactive list）中挑选候选页。Linux 还会定期把活跃链表（active list）底部的页移到不活跃链表中，让活跃链表保持在页缓存总大小的三分之二左右 [G04]。

理想情况下，Linux 会按严格的 LRU 顺序管理这些链表，但前几章讨论过，这样做代价很高。因此，和许多操作系统一样，Linux 用的是一种近似 LRU（类似于**时钟**置换）。

这种 2Q 方法的表现大体上和 LRU 相当接近，但它专门处理了循环访问大文件的情况：把这类循环访问的页限制在不活跃链表（inactive list）里。由于这些页在被踢出内存之前根本不会被再次访问，它们就不会把活跃链表（active list）中其他有用的页挤出去。

### 安全与缓冲区溢出

现代虚拟内存系统（Linux、Solaris 或某个 BSD 变种）和古老的系统（VAX/VMS）之间最大的区别，可能就在于现代对安全的重视。保护一直是操作系统重点关注的问题，而如今机器之间的互联程度前所未有，开发者实现了各种防御措施来阻止那些狡猾的黑客夺取系统控制权，也就不足为奇了。

一大威胁来自**缓冲区溢出**（buffer overflow）攻击[2]，它可以用来攻击普通的用户程序，甚至内核本身。这类攻击的思路是在目标系统中找到一个 bug，让攻击者能够向目标的地址空间注入任意数据。这类漏洞的成因有时是这样的：开发者（错误地）假定输入不会太长，于是（轻信地）把输入复制到缓冲区里；而输入实际上太长，就会溢出缓冲区，覆盖目标的内存。像下面这样看起来人畜无害的代码，就可能是问题的根源：

```
int some_function(char *input) {
    char dest_buffer[100];
    strcpy(dest_buffer, input); // 糟糕，复制时没有检查长度！
}
```

[2]关于这个话题的一些细节和链接，请参见 https://en.wikipedia.org/wiki/Buffer_overflow，其中还提到了安全黑客 Elias Levy（又名“Aleph One”）的那篇著名文章。

补充：无处不在的内存映射

内存映射（memory mapping）比 Linux 早出现好些年，Linux 和其他现代系统中有很多地方都用到了它。思路很简单：对一个已经打开的文件描述符（file descriptor）调用 `mmap()`，进程就会得到一个指针，指向一块虚拟内存区域的开头，文件的内容看起来就位于这块区域中。之后，进程只需用这个指针做简单的解引用，就能访问文件的任何部分。

访问内存映射文件（memory-mapped file）中尚未调入内存的部分时，会触发**缺页异常**（page fault，也叫缺页中断），这时操作系统会把相关数据调入内存，并相应地更新进程的页表，使这些数据可以访问（即**按需分页**，demand paging）。

每个普通的 Linux 进程都会用到内存映射文件（memory-mapped file），即使 `main()` 中的代码并没有直接调用 `mmap()`，这是 Linux 把可执行文件和共享库的代码加载进内存的方式决定的。下面是 `pmap` 命令行工具的输出（经过大幅删减），它显示了一个正在运行的程序（本例中是 shell，即 `tcsh`）的虚拟地址空间由哪些不同的映射组成。输出有四列：映射的虚拟地址、大小、该区域的保护位（protection bit），以及映射的来源：

```
0000000000400000   372K r-x-- tcsh
00000000019d5000  1780K rw---   [anon ]
00007f4e7cf06000  1792K r-x-- libc-2.23.so
00007f4e7d2d0000    36K r-x-- libcrypt-2.23.so
00007f4e7d508000   148K r-x-- libtinfo.so.5.9
00007f4e7d731000   152K r-x-- ld-2.23.so
00007f4e7d932000    16K rw---   [stack ]
```

从输出可以看出，`tcsh` 二进制文件的代码，以及来自 `libc`、`libcrypt`、`libtinfo` 的代码和动态链接器（dynamic linker）本身（`ld.so`）的代码，都被映射到了地址空间中。此外还有两个匿名区域：堆（第二项，标为 `anon`）和栈（标为 `stack`）。内存映射文件（memory-mapped file）为操作系统构建现代地址空间提供了一种直接而高效的方法。

很多情况下，这样的溢出并不是灾难性的：比如，无意中传给用户程序甚至操作系统的错误输入，多半只会让它崩溃，仅此而已。然而，恶意的程序员可以精心构造溢出缓冲区的输入，把自己的代码注入目标系统，从而基本上接管这个系统，让它听从自己的指挥。如果攻击联网的用户程序得手，攻击者就可以执行任意计算，甚至把被攻陷系统的计算周期租出去；如果攻击操作系统本身得手，就能访问更多资源，这属于所谓的**权限提升**（privilege escalation，即用户代码获得内核访问权限）。如果你还没猜到的话：这些都是坏事。

防御缓冲区溢出的第一种、也是最简单的方法，是禁止执行位于地址空间某些区域（例如栈）中的任何代码。AMD 在它的 x86 版本中引入的 **NX 位**（NX bit，No-eXecute 的缩写，即禁止执行；Intel 的处理器上现在也有类似的 XD 位）就是这样一种防御：如果某页对应的页表项中设置了这一位，就禁止执行该页上的任何代码。这样，攻击者注入目标栈中的代码就无法执行，问题也就得到了缓解。

 © 2008–23, ARPACI-DUSSEAU

然而，聪明的攻击者毕竟……很聪明：即使攻击者无法直接注入代码，恶意代码照样可以执行任意的代码序列。这种思想最一般的形式称为**返回导向编程**（return-oriented programming，**ROP**）[S07]，它确实相当高明。ROP 背后的观察是：任何程序的地址空间中都有大量零碎的代码片段（用 ROP 的术语来说叫 **gadget**），链接了庞大 C 库的 C 程序尤其如此。因此，攻击者可以覆盖栈，让当前执行函数的返回地址指向所需的某条恶意指令（或一串指令），这些指令后面紧跟着一条返回指令。把大量 gadget 串在一起（即确保每次返回都跳到下一个 gadget），攻击者就能执行任意代码。太神奇了！

为了防御 ROP（包括它更早的形式，即 **return-to-libc 攻击** [S+04]），Linux（以及其他系统）又加入了一种防御，称为**地址空间布局随机化**（address space layout randomization，**ASLR**）。操作系统不再把代码、栈和堆放在虚拟地址空间中的固定位置，而是随机安排它们的位置，这样一来，要构造实施这类攻击所需的复杂代码序列就相当困难了。因此，针对有漏洞的用户程序的攻击，大多只会让程序崩溃，而无法夺取正在运行的程序的控制权。

有意思的是，你很容易就能亲眼看到这种随机性（randomness）。下面这段代码可以在现代 Linux 系统上演示：

```
int main(int argc, char *argv[]) {
    int stack = 0;
    printf("%p\n", &stack);
    return 0;
}
```

这段代码只是打印出栈上一个变量的（虚拟）地址。在较老的、没有 ASLR 的系统中，每次打印出的值都相同。但从下面可以看到，每次运行这个值都会变：

```
prompt> ./random
0x7ffd3e55d2b4
prompt> ./random
0x7ffe1033b8f4
prompt> ./random
0x7ffe45522e94
```

ASLR 对用户级程序是一种非常有用的防御，所以它也被引入了内核，这个特性的名字毫无新意，就叫**内核地址空间布局随机化**（kernel address space layout randomization，**KASLR**）。不过事实表明，内核可能还有更大的问题要应对，下面就来讨论。

### 其他安全问题：Meltdown 和 Spectre

在我们写下这段文字时（2018 年 8 月），系统安全领域已经被两种新的、相互关联的攻击搅得天翻地覆。第一种叫 **Meltdown**（熔断），第二种叫 **Spectre**（幽灵）。四组不同的研究人员和工程师几乎同时发现了它们，这让人们开始深刻质疑计算机硬件及其上的操作系统所提供的基本保护。详细描述这两种攻击的论文见 `spectreattack.com`；两者之中，Spectre 被认为更麻烦。

这两种攻击利用的是同一个弱点：现代系统中的 CPU 为了提高性能，会在幕后施展各种疯狂的技巧。处于问题核心的一类技术叫作**推测执行**（speculative execution）：CPU 猜测接下来很快会执行哪些指令，并提前开始执行。如果猜对了，程序就跑得更快；如果猜错

了，CPU 会撤销这些指令对体系结构状态（例如寄存器）的影响，然后再试一次，这次走正确的路径。

推测执行（speculative execution）的问题在于，它往往会在系统的各个部分留下执行的痕迹，比如处理器缓存、分支预测器等。麻烦就在这里：正如这些攻击的作者所展示的，这些状态可能让内存中的内容暴露在攻击之下，甚至包括我们以为受 MMU 保护的内存。

因此，加强内核保护的一条途径，是从每个用户进程中移除尽可能多的内核地址空间，改为给大多数内核数据单独准备一张内核页表（这称为**内核页表隔离**，kernel page-table isolation，**KPTI**）[G+17]。这样一来，内核的代码和数据结构不再映射到每个进程中，进程里只保留最起码的一小部分；相应地，切换到内核时，还得切换到内核页表（kernel page table）。这样做提高了安全性，堵住了一些攻击途径，但也有代价：性能。切换页表的代价很高。唉，安全的代价：便利和性能。

遗憾的是，KPTI 并不能解决上面列出的所有安全问题，只能解决其中一部分。而像关闭推测执行（speculative execution）这样简单的解决办法也没什么意义，因为那样系统会慢上几千倍。因此，如果你对系统安全感兴趣，现在可真是个有意思的时代。

要真正理解这些攻击，你（很可能）得先学习很多东西。先从理解现代计算机体系结构入手（可以看这方面的高级书籍），重点关注推测执行（speculative execution）以及实现它所需的各种机制。一定要读一读上面提到的网站上关于 Meltdown 和 Spectre 攻击的内容；那里其实也有一份关于推测执行（speculative execution）的实用入门介绍，所以也许是个不错的起点。还要研究操作系统，寻找更多的漏洞。谁知道还有哪些问题没被发现呢？

## 23.3 小结

现在，你已经从头到尾完整地审视了两个虚拟内存系统。但愿大部分细节都很容易理解，因为你应该已经很好地掌握了基本的机制和策略。关于 VAX/VMS 的更多细节，可以参阅 Levy 和 Lipman 那篇出色（而且很短）的论文 [LL82]。我们鼓励你读一读，从中可以看到这几章内容的原始出处是什么样子。

你还对 Linux 有了一些了解。Linux 虽然是一个庞大而复杂的系统，但它继承了过去的许多好思想，其中很多我们没有篇幅详细讨论。例如，Linux 在 `fork()` 时对页做惰性的写时复制（copy-on-write），避免不必要的复制，从而降低开销。Linux 也通过内存映射 `/dev/zero` 设备对页做按需置零（demand zeroing），还有一个后台换页守护进程（swap daemon）**`swapd`**，负责把页换出到磁盘以减轻内存压力（memory pressure）。事实上，Linux 的虚拟内存充满了从过去借鉴来的好思想，同时也包含了许多它自己的创新。

想了解更多，可以看看这几本还不错（但可惜已经过时）的书 [BC05, G04]。我们鼓励你自己去读一读，因为我们只能从复杂性的海洋中取出最微小的一滴。不过，万事总得有个开头。海洋不就是无数水滴汇成的吗？[M04]

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[B+13] "Efficient Virtual Memory for Big Memory Servers" by A. Basu, J. Gandhi, J. Chang, M. D. Hill, M. M. Swift. ISCA '13, June 2013, Tel-Aviv, Israel. *A recent work showing that TLBs matter, consuming 10% of cycles for large-memory workloads. The solution: one massive segment to hold large data sets. We go backward, so that we can go forward!*

[BB+72] "TENEX, A Paged Time Sharing System for the PDP-10" by D. G. Bobrow, J. D. Burchfiel, D. L. Murphy, R. S. Tomlinson. CACM, Volume 15, March 1972. *An early time-sharing OS where a number of good ideas came from. Copy-on-write was just one of those; also an inspiration for other aspects of modern systems, including process management, virtual memory, and file systems.*

[BJ81] "Converting a Swap-Based System to do Paging in an Architecture Lacking Page-Reference Bits" by O. Babaoglu, W. N. Joy. SOSP '81, Pacific Grove, California, December 1981. *How to exploit existing protection machinery to emulate reference bits, from a group at Berkeley working on their own version of* UNIX*: the* **Berkeley Systems Distribution** *(***BSD***). The group was influential in the development of virtual memory, file systems, and networking.*

[BC05] "Understanding the Linux Kernel" by D. P. Bovet, M. Cesati. O'Reilly Media, November 2005. *One of the many books you can find on Linux, which are out of date, but still worthwhile.*

[C03] "The Innovator's Dilemma" by Clayton M. Christenson. Harper Paperbacks, January 2003. *A fantastic book about the disk-drive industry and how new innovations disrupt existing ones. A good read for business majors and computer scientists alike. Provides insight on how large and successful companies completely fail.*

[C93] "Inside Windows NT" by H. Custer, D. Solomon. Microsoft Press, 1993. *The book about Windows NT that explains the system top to bottom, in more detail than you might like. But seriously, a pretty good book.*

[G04] "Understanding the Linux Virtual Memory Manager" by M. Gorman. Prentice Hall, 2004. *An in-depth look at Linux VM, but alas a little out of date.*

[G+17] "KASLR is Dead: Long Live KASLR" by D. Gruss, M. Lipp, M. Schwarz, R. Fellner, C. Maurice, S. Mangard. Engineering Secure Software and Systems, 2017. Available: https://gruss.cc/files/kaiser.pdf *Excellent info on KASLR, KPTI, and beyond.*

[JS94] "2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm" by T. Johnson, D. Shasha. VLDB '94, Santiago, Chile. *A simple but effective approach to building page replacement.*

[LL82] "Virtual Memory Management in the VAX/VMS Operating System" by H. Levy, P. Lipman. IEEE Computer, Volume 15:3, March 1982. *Read the original source of most of this material. Particularly important if you wish to go to graduate school, where all you do is read papers, work, read some more papers, work more, eventually write a paper, and then work some more.*

[M04] "Cloud Atlas" by D. Mitchell. Random House, 2004. *It's hard to pick a favorite book. There are too many! Each is great in its own unique way. But it'd be hard for these authors not to pick "Cloud Atlas", a fantastic, sprawling epic about the human condition, from where the the last quote of this chapter is lifted. If you are smart — and we think you are — you should stop reading obscure commentary in the references and instead read "Cloud Atlas"; you'll thank us later.*

[O16] "Virtual Memory and Linux" by A. Ott. Embedded Linux Conference, April 2016. https://events.static.linuxfound.org/sites/events/files/slides/elc_2016_mem.pdf . *A useful set of slides which gives an overview of the Linux VM.*

[RL81] "Segmented FIFO Page Replacement" by R. Turner, H. Levy. SIGMETRICS '81, Las Vegas, Nevada, September 1981. *A short paper that shows for some workloads, segmented FIFO can approach the performance of LRU.*

[S07] "The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)" by H. Shacham. CCS '07, October 2007. *A generalization of return-to-libc. Dr. Beth Garner said in Basic Instinct, "She's crazy! She's brilliant!" We might say the same about ROP.*

[S+04] "On the Effectiveness of Address-space Randomization" by H. Shacham, M. Page, B. Pfaff, E. J. Goh, N. Modadugu, D. Boneh. CCS '04, October 2004. *A description of the return-to-libc attack and its limits. Start reading, but be wary: the rabbit hole of systems security is deep…*

# 24

# 关于内存虚拟化的总结对话

**学生**：（咽了一口唾沫）哇，内容可真多。

**教授**：是啊，然后呢？

**学生**：呃，这么多东西，我怎么记得住啊？我是说，为了应付考试？

**教授**：天哪，但愿你不是为了这个才去记的。

**学生**：那我为什么要记呢？

**教授**：拜托，我还以为你不会这么想呢。你来这儿是为了学点东西，这样等你将来走出校门，才能明白系统实际上是怎么工作的。

**学生**：嗯……您能举个例子吗？

**教授**：当然！我读研究生的时候，有一次和几个朋友测内存访问要花多长时间，结果时不时会测出比预想高得多的数字。要知道，我们原以为数据全都老老实实地待在二级硬件缓存（*cache*）里，访问起来应该非常快才对。

**学生**：（点头）

**教授**：我们怎么也想不通是怎么回事。遇到这种情况该怎么办？很简单，去问教授！于是我们去找了我们的一位教授，他看了看我们画出来的图，只说了一句："*TLB*。"啊哈！没错，是 *TLB* 未命中（*TLB miss*）！我们怎么就没想到呢？只要对虚拟内存（*virtual memory*）的工作原理有一个好的模型，各种有意思的性能问题都能诊断出来。

**学生**：我想我明白了。我正努力在脑子里建立这些心智模型，弄清楚各种东西是怎么工作的；这样将来我自己出去干活时，就算某个系统的表现和预期不太一样，我也不会吃惊。我甚至应该光靠想一想，就能预料到系统会怎样运行。

**教授**：没错。那你学到了什么？关于虚拟内存怎么工作，你的心智模型是什么样的？

**学生**：嗯，我想我现在已经很清楚进程（*process*）引用内存时会发生什么了。您说过很多次，每次取指令都会引用内存，显式的加载和存储也一样。

**教授**：听起来不错，接着说。

**学生**：嗯，有一点我会永远记住：我们在用户程序里看到的地址，比如用 *C* 写的程序……

**教授**：难道还有别的语言吗？

**学生**:(接着说)……是是,我知道您喜欢 C,我也喜欢!总之,我刚才是说,我现在真正明白了:我们在程序里能看到的所有地址都是虚拟地址(*virtual address*);我作为程序员,看到的数据和代码在内存中的位置只是一种假象。我以前觉得能打印出指针的地址很酷,现在却觉得挺泄气:那只是个虚拟地址!数据真正所在的物理地址(*physical address*),我根本看不到。

**教授**:是看不到,操作系统肯定会把它藏起来,不让你看见。还有呢?

**学生**:嗯,我觉得 *TLB* 是非常关键的一环,它给系统提供了一个存放地址转换(*address translation*)的小型硬件缓存。页表(*page table*)通常相当大,所以只能放在又大又慢的内存里。要是没有 *TLB*,程序肯定会慢得多。看来真正让内存虚拟化(*memory virtualization*)变得可行的是 *TLB*。我没法想象怎么构建一个没有 *TLB* 的系统!而且一想到有的程序工作集(*working set*)超出了 *TLB* 的覆盖范围,我就不寒而栗:那么多 *TLB* 未命中,简直不忍直视。

**教授**:是啊,快捂住孩子们的眼睛!除了 *TLB*,你还学到了什么?

**学生**:我现在还明白了,页表是必须了解的数据结构(*data structure*)之一;不过它说到底也只是个数据结构,这意味着几乎什么结构都可以拿来用。我们从简单的结构讲起,比如数组,也就是线性页表(*linear page table*),一路讲到多级页表(*multi-level page table*,看起来像树),甚至还有更疯狂的,比如放在内核虚拟内存(*kernel virtual memory*)里、本身也能换页的页表。这一切都只是为了在内存里省下一点空间!

**教授**:确实如此。

**学生**:还有一件很重要的事:我认识到,地址转换用的结构必须足够灵活,程序员想怎么用自己的地址空间(*address space*),它都得能支持。从这个意义上说,多级页表这样的结构就非常理想:只有当用户要用到地址空间的某一部分时,它才会为这部分创建页表空间,所以几乎没有浪费。更早的尝试,比如简单的基址(*base*)和界限寄存器(*bounds register*),就是不够灵活;这些结构必须符合用户对虚拟内存系统的期望和需求。

**教授**:这个角度很好。那我们学的那些换出(*swap out*)到磁盘的内容呢?

**学生**:嗯,这部分学起来当然挺好玩,了解页面置换(*page replacement*)是怎么工作的也有好处。有些基本策略算是一目了然(比如 *LRU*),但构建一个真正的虚拟内存系统好像更有意思,就像我们在 *VMS* 案例研究里看到的那样。不过不知怎么的,我觉得机制更吸引人,策略就差一些。

**教授**：哦，为什么呢？

**学生**：嗯，就像您说的，归根结底，策略问题最好的解决办法很简单：买更多内存。但机制你必须弄懂，才能知道东西到底是怎么运作的。说到这个……

**教授**：嗯？

**学生**：呃，我的电脑最近跑得有点慢……而且内存其实也不算贵……

**教授**：哦，好吧好吧！给你几块钱。去给自己买点 *DRAM* 吧，小气鬼。

**学生**：谢谢教授！我再也不会换出到磁盘了；就算换，至少也知道背后是怎么回事！

# 第二部分

# 并发

# 25

# 关于并发的对话

**教授**：好，现在讲到操作系统三大支柱中的第二根：**并发**（*concurrency*）。

**学生**：我还以为有四大支柱呢……？

**教授**：不，那是这本书旧版里的说法。

**学生**：呃……好吧。那么，了不起的教授，什么是并发呢？

**教授**：嗯，想象一下，我们有一个桃子——

**学生**：（打断）又是桃子！您跟桃子到底有什么过不去的？

**教授**：读过 *T.S.* 艾略特吗？《*J.* 阿尔弗瑞德·普鲁弗洛克的情歌》，“我敢吃下一只桃子吗”，还有那些好玩的东西？

**学生**：哦，读过！高中英语课上读的。写得真好！我特别喜欢那一段——

**教授**：（打断）这跟那个没关系——我只是喜欢桃子而已。总之，想象桌上有很多桃子，还有很多人想吃。假设我们这样来：每个吃桃子的人先用眼睛锁定一个桃子，然后伸手去拿，拿到就吃。这种做法有什么问题？

**学生**：嗯……您看中的桃子，别人可能也看中了。要是他们先到，等您伸出手时，桃子就没您的份了！

**教授**：正是！那我们该怎么办？

**学生**：嗯，大概得想个更好的办法。也许可以排成一队，排到最前面时，就拿一个桃子走人。

**教授**：很好！不过你这个办法有什么问题？

**学生**：天哪，活儿都得我来干吗？

**教授**：对。

**学生**：好吧，让我想想。嗯，原来的做法是大家一拥而上同时抢，这样更快。可是按我的办法，大家只能一个接一个地来，这样是对了，但慢了不少。最好的办法大概是既快又对。

**教授**：你真是越来越让人刮目相看了。事实上，关于并发我们需要知道的一切，你刚才都已经说出来了！干得好。

**学生**：是吗？我还以为我们只是在聊桃子呢。记住，通常这时候您就该把话题扯回计算机上了。

**教授**：确实如此。抱歉！具体的东西可永远不能忘。嗯，其实有一类程序，我们称之为**多线程**（*multi-threaded*）应用；每个**线程**（*thread*）有点像一个独立的代理，在程序里跑来跑去，替程序做事。但这些线程要访问内存，在它们眼里，内存中的每个位置都有点像那些桃子。如果我们不协调线程之间对内存的访问，程序就不会按预期工作。明白了吗？

**学生**：有点明白。可是为什么要在操作系统课上讲这个？这不就是应用程序编程吗？

**教授**：问得好！其实有好几个原因。第一，操作系统必须用**锁**（*lock*）和**条件变量**（*condition variable*）这样的原语来支持多线程应用，我们很快就会讲到。第二，操作系统本身就是第一个并发程序（*concurrent program*）——它必须非常小心地访问自己的内存，否则就会发生许多奇怪又可怕的事情。说真的，那场面可能相当惨烈。

**学生**：明白了。听起来很有意思。我猜还有更多细节吧？

**教授**：确实还有……

 © 2008–25, ARPACI-DUSSEAU

# 26

# 并发：简介

到目前为止，我们已经看到操作系统提供的几种基本抽象（abstraction）是怎样一步步建立起来的。我们看到了如何把一个物理 CPU 变成多个**虚拟 CPU**（virtual CPU），从而营造出多个程序同时运行的假象。我们还看到了如何给每个进程（process）营造一个假象，让它以为自己独占一大块私有的**虚拟内存**（virtual memory）：有了**地址空间**（address space）这个抽象，每个程序都可以表现得好像拥有自己的内存，而实际上是操作系统在暗中把多个地址空间复用到物理内存（physical memory）上，有时还复用到磁盘上。

本章为单个运行中的进程引入一种新的抽象：**线程**（thread）。按照经典的观点，程序中只有一个执行点（即只有一个 PC，指令从这里取出并执行）；而**多线程**（multi-threaded）程序有不止一个执行点（即有多个 PC，每个 PC 都在取指令、执行指令）。换个角度看，也可以把每个线程看成一个独立的进程，只有一点不同：线程之间共享同一个地址空间，因此可以访问同样的数据。

所以，单个线程的状态和进程的状态非常相似。线程有一个程序计数器（PC），记录程序从哪里取指令。每个线程都有一组自己私有的寄存器，用来做计算；因此，如果有两个线程在同一个处理器上运行，从运行其中一个（T1）切换到运行另一个（T2）时，就必须做一次**上下文切换**（context switch）。线程之间的上下文切换和进程之间的上下文切换很像：运行 T2 之前，必须先保存 T1 的寄存器状态，再恢复 T2 的寄存器状态。进程的状态保存在**进程控制块**（process control block，PCB）里；现在，我们需要一个或多个**线程控制块**（thread control block，TCB）来保存进程中每个线程的状态。不过，和进程间的上下文切换相比，线程间的上下文切换有一个重大区别：地址空间保持不变，也就是说，不需要切换所用的页表（page table）。

线程和进程的另一个重大区别在于栈（stack）。经典进程现在可以称为**单线程**（single-threaded）进程。在这种进程的简单地址空间模型里，只有一个栈，通常位于地址空间的底部（图 26.1 左）。

然而，在多线程进程（multi-threaded process）中，每个线程独立运行，当然也会调用各种例程来完成手头的工作。地址空间里不再只有一个栈，而是每个线程各有一个。假设有一个多线程进程（multi-threaded process），里面有两个线程，那么它的地址空间看起来就不一样了（图 26.1 右）。

从图中可以看到，进程的地址空间里分布着两个栈。因此，所有在栈上分配的变量、参数、返回值，以及其他放在栈上的东西，都会放进相应线程的栈里；这种存储有时称为**线程局部**（thread-local）存储。

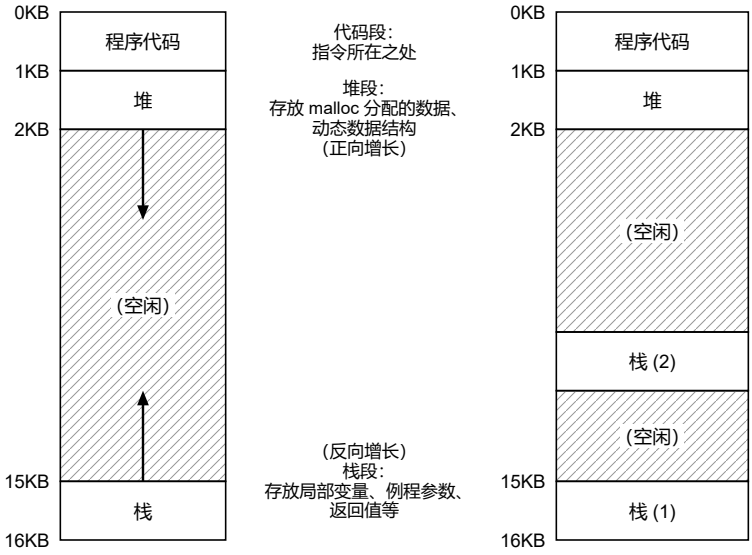



图 26.1：**单线程和多线程的地址空间**

你可能还注意到，这破坏了我们原本漂亮的地址空间布局。以前，栈和堆（heap）可以各自独立地增长，只有地址空间用完时才会出问题。现在情况就没那么好了。好在这通常不成问题，因为栈一般不需要很大（大量使用递归的程序例外）。

## 26.1 为什么要用线程?

在深入线程的细节、讨论编写多线程程序时可能遇到的问题之前，我们先来回答一个更简单的问题：到底为什么要用线程?

使用线程的主要原因至少有两个。第一个很简单：**并行**（parallelism）。假设你在写一个对非常大的数组做运算的程序，比如把两个大数组相加，或者给数组中每个元素的值都加上某个数。如果只在单个处理器上运行，这件事并不复杂：把每个运算做完就行。然而，如果在有多个处理器的系统上运行这个程序，就有可能让每个处理器各做一部分工作，从而大大加快速度。把普通的**单线程**（single-threaded）程序改造成能在多个 CPU 上完成这类工作的程序，这项任务叫作**并行化**（parallelization）；每个 CPU 用一个线程来做这部分工作，是让程序在现代硬件上跑得更快的一种自然而常见的做法。

第二个原因要微妙一些：避免程序因为 I/O 缓慢而停滞不前。假设你在写一个执行各种 I/O 的程序：可能在等待发送或接收消息，可能在等待显式的磁盘 I/O 完成，甚至可能在（隐式地）等待缺页异常（page fault）处理完。与其干等，程序可能更希望做点别的事，比如利用 CPU 做计算，或者再发出更多的 I/O 请求。使用线程是避免卡住的一种自然方式：程序中一个线程在等待时（即阻塞在 I/O 上），CPU 调度程序（scheduler）可以切换到其他已经就绪（ready）、能做有用工作的线程。线程让单个程序内部的 I/O 可以和其他活动**重叠**（overlap）进行，就像**多道程序设计**（multiprogramming）在不同程序的进程之间做

 © 2008–25, ARPACI-DUSSEAU

到的那样；正因如此，许多现代服务器应用（Web 服务器、数据库管理系统等）在实现中都用到了线程。

当然，上面两种情况也都可以用多个进程（process）来代替线程。不过，线程共享地址空间，因此共享数据很容易，构建这类程序时自然会选择线程。如果是逻辑上相互独立、几乎不需要共享内存中的数据结构（data structure）的任务，进程是更稳妥的选择。

## 26.2 一个例子：创建线程

我们来看一些细节。假设我们要运行一个程序，它创建两个线程，每个线程各自做一些独立的工作，在这里就是打印“A”或“B”。代码如图 26.2（第 252 页）所示。

主程序创建了两个线程，它们都会运行函数 `mythread()`，只是参数不同（字符串 `A` 或 `B`）。线程一旦创建，就可能立刻开始运行（取决于调度程序的心情）；也可能处于“就绪”而非“运行（running）”状态，暂时还不运行。当然，在多处理器（multiprocessor）上，这两个线程甚至可能同时运行，不过我们暂时先不考虑这种可能。

创建了这两个线程之后，主线程（main thread）调用 `pthread_join()`，等待某个特定的线程结束。它调用了两次，从而确保 T1 和 T2 都运行完毕之后，主线程才能再次运行；主线程再次运行时，会打印“main: end”并退出。总的来说，这次运行一共用到了三个线程：主线程、T1 和 T2。

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4  #include "common.h"
5  #include "common_threads.h"
6
7  void *mythread(void *arg) {
8      printf("%s\n", (char *) arg);
9      return NULL;
10 }
11
12 int
13 main(int argc, char *argv[]) {
14     pthread_t p1, p2;
15     int rc;
16     printf("main: begin\n");
17     Pthread_create(&p1, NULL, mythread, "A");
18     Pthread_create(&p2, NULL, mythread, "B");
19     // join 会等待线程结束
20     Pthread_join(p1, NULL);
21     Pthread_join(p2, NULL);
22     printf("main: end\n");
23     return 0;
24 }
```

图 26.2：**简单的线程创建代码（t0.c）**

| main | 线程 1 | 线程 2 |
|---|---|---|
| 开始运行 | | |
| 打印“main: begin” | | |
| 创建线程 1 | | |
| 创建线程 2 | | |
| 等待 T1 | | |
| | 运行 | |
| | 打印“A” | |
| | 返回 | |
| 等待 T2 | | |
| | | 运行 |
| | | 打印“B” |
| | | 返回 |
| 打印“main: end” | | |

图 26.3：**线程轨迹（1）**

我们来看看这个小程序可能的执行顺序。在执行图（图 26.3，第 253 页）中，时间从上往下推进，每一列分别对应主线程、线程 1 和线程 2，表示这个线程在什么时候运行。

不过请注意，这种顺序并不是唯一可能的顺序。事实上，给定一个指令序列，可能的顺序有很多，取决于调度程序在某个时刻决定运行哪个线程。例如，线程一旦创建就可能立即运行，这样就会得到图 26.4（第 253 页）所示的执行过程。

如果调度程序决定先运行线程 2，那么即使线程 1 创建得更早，我们甚至可能看到“B”在“A”之前打印出来；没有理由认为先创建的线程就会先运行。图 26.5（第 254 页）展示了最后这种执行顺序，线程 2 抢在线程 1 之前登场亮相。

| main | 线程 1 | 线程 2 |
|---|---|---|
| 开始运行 | | |
| 打印“main: begin” | | |
| 创建线程 1 | | |
| | 运行 | |
| | 打印“A” | |
| | 返回 | |
| 创建线程 2 | | |
| | | 运行 |
| | | 打印“B” |
| | | 返回 |
| 等待 T1 | | |
| *立即返回；T1 已经结束* | | |
| 等待 T2 | | |
| *立即返回；T2 已经结束* | | |
| 打印“main: end” | | |

图 26.4：**线程轨迹（2）**

 © 2008–25, ARPACI-DUSSEAU

| main | 线程 1 | 线程 2 |
|---|---|---|
| 开始运行 | | |
| 打印“main: begin” | | |
| 创建线程 1 | | |
| 创建线程 2 | | |
| | | 运行 |
| | | 打印“B” |
| | | 返回 |
| 等待 T1 | | |
| | 运行 | |
| | 打印“A” | |
| | 返回 | |
| 等待 T2 | | |
| *立即返回；T2 已经结束* | | |
| 打印“main: end” | | |

图 26.5：**线程轨迹（3）**

由此可见，创建线程有点像调用函数；不同的是，系统并不是先执行函数、再返回调用者，而是为被调用的例程创建一个新的执行线程，让它独立于调用者运行：它可能在创建函数返回之前就开始运行，也可能要晚得多才运行。接下来运行什么，由操作系统的**调度程序**（scheduler）决定；尽管调度程序很可能实现了某种合理的算法，但我们很难知道在任一时刻会运行什么。

从这个例子你可能还能看出，线程让事情变复杂了：光是判断什么时候运行什么，就已经很难了！就算没有并发（concurrency），计算机也已经够难懂了。遗憾的是，有了并发，情况只会更糟，而且糟得多。

## 26.3 为什么会更糟：共享数据

上面这个简单的线程例子很有用，它展示了线程是如何创建的，以及线程如何根据调度程序的决定以不同的顺序运行。但它没有展示线程在访问共享数据时会如何相互影响。

我们来设想一个简单的例子：两个线程都要更新同一个全局共享变量。要研究的代码见图 26.6（第 256 页）。

关于这段代码，有几点说明。首先，正如 Stevens 所建议的 [SR05]，我们把线程的创建和 join 例程包装了一下，出错时直接退出；对这么简单的程序来说，出了错（如果真出了错）至少要能发现，但不必做什么聪明的处理（比如直接退出就行）。因此，`Pthread_create()` 只是调用 `pthread_create()`，并确认返回码为 0；如果不是 0，`Pthread_create()` 就打印一条消息然后退出。

其次，我们没有为两个工作线程（worker thread）写两个不同的函数体，而是只用一段代码，并给线程传一个参数（这里是字符串），让每个线程在自己的消息前面打印不同的字母。

 © 2008–25, ARPACI-DUSSEAU

最后也是最重要的，我们现在可以看看每个工作线程要做什么：在循环中给共享变量 counter 加一个数，一共加 1000 万次（1e7）。因此，期望的最终结果是 20,000,000。

现在编译并运行这个程序，看看它的表现。有时候，一切都和预期的一样：

```
prompt> gcc -o main main.c -Wall -pthread; ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 20000000)
```

遗憾的是，运行这段代码时，即使在单处理器上，也不一定能得到想要的结果。有时会得到：

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19345221)
```

我们再试一次，看看是不是自己疯了。毕竟，老师不是教过你，计算机应该产生**确定的**（deterministic）结果吗?！难道你的教授们一直在骗你？（倒吸一口凉气）

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19221041)
```

不仅每次运行的结果都是错的，而且每次的结果还不一样！一个大问题摆在面前：为什么会这样?

## 26.4 问题的核心：不受控制的调度

要理解为什么会这样，我们必须了解编译器为更新 counter 生成的代码序列。在这个例子中，我们只是想给 counter 加一个数（1）。因此，完成这件事的代码序列大概是这样的（x86）：

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

```
#include <stdio.h>
#include <pthread.h>
#include "common.h"
#include "common_threads.h"

static volatile int counter = 0;

// mythread()
//
// 在循环中反复给 counter 加 1
// 当然，正常人不会这样给 counter 加上 10,000,000，
// 但用它来展示问题再好不过。
//
void *mythread(void *arg) {
    printf("%s: begin\n", (char *) arg);
    int i;
    for (i = 0; i < 1e7; i++) {
        counter = counter + 1;
    }
    printf("%s: done\n", (char *) arg);
    return NULL;
}

// main()
//
// 只是启动两个线程（pthread_create），
// 然后等待它们结束（pthread_join）
//
int main(int argc, char *argv[]) {
    pthread_t p1, p2;
    printf("main: begin (counter = %d)\n", counter);
    Pthread_create(&p1, NULL, mythread, "A");
    Pthread_create(&p2, NULL, mythread, "B");

    // join 会等待线程结束
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("main: done with both (counter = %d)\n",
           counter);
    return 0;
}
```

图 26.6：**共享数据：糟了（t1.c）**

这个例子假设变量 `counter` 位于地址 0x8049a1c。在这三条指令中，x86 的 `mov` 指令先取出该地址处的内存值，放进寄存器 `eax`。然后执行 `add`，把 1（0x1）加到 `eax` 寄存器的内容上；最后，把 `eax` 的内容存回内存中的同一地址。

提示：了解并使用你的工具

你应该不断学习新工具，它们能帮你编写、调试和理解计算机系统。这里我们用一个很好用的工具，叫作**反汇编器**（disassembler）。对可执行文件运行反汇编器，它会显示程序由哪些汇编指令组成。例如，如果想了解更新计数器的底层代码（就像我们的例子），可以运行 `objdump`（Linux）来查看汇编代码：

```
prompt> objdump -d main
```

这样会得到一份长长的清单，列出程序中的所有指令，标注得整整齐齐（如果编译时加了 `-g` 选项，清单里还会带上程序的符号信息）。`objdump` 只是你应该学会使用的众多工具之一；调试器 `gdb`、内存分析工具 `valgrind` 或 `purify`，当然还有编译器本身，都值得你花时间深入了解。工具用得越好，你就能构建出越好的系统。

假设两个线程中的一个（线程 1）进入了这段代码，即将把 `counter` 加一。它把 `counter` 的值（假设一开始是 50）加载到自己的寄存器 `eax` 中，所以线程 1 的 `eax=50`。然后它把寄存器加一，`eax` 变成 51。这时，不幸的事情发生了：时钟中断（timer interrupt）触发了，操作系统随即把当前运行线程的状态（它的 PC、包括 `eax` 在内的寄存器等）保存到这个线程的 TCB 中。

接着，更糟的事情发生了：线程 2 被选中运行，它也进入了同一段代码。它同样执行第一条指令，取出 `counter` 的值，放进它自己的 `eax`（记住：每个线程运行时都有自己私有的寄存器；上下文切换代码会保存和恢复这些寄存器，从而把它们**虚拟化**（virtualized）了）。此时 `counter` 的值仍然是 50，因此线程 2 的 `eax=50`。假设线程 2 接着执行了后面两条指令：把 `eax` 加 1（于是 `eax=51`），然后把 `eax` 的内容存入 `counter`（地址 0x8049a1c）。于是，全局变量 `counter` 现在的值是 51。

最后，又发生了一次上下文切换，线程 1 恢复运行。回想一下，它刚刚执行完 `mov` 和 `add`，现在正要执行最后那条 `mov` 指令。别忘了，它的 `eax=51`。于是它执行这条 `mov`，把这个值存进内存，`counter` 又一次被设成了 51。

简单地说，发生的事情是这样的：给 `counter` 加一的代码运行了两次，可是 `counter` 从 50 开始，现在却只等于 51。如果程序是“正确”的，变量 `counter` 应该等于 52。

为了更好地理解这个问题，我们来看一份详细的执行轨迹。在这个例子中，假设上面的代码加载到内存地址 100 处，排布如下（习惯了整齐的类 RISC 指令集的读者请注意：x86 的指令是变长的；这条 `mov` 指令占 5 个字节，而 `add` 只占 3 个字节）：

```
100 mov    0x8049a1c, %eax
105 add    $0x1, %eax
108 mov    %eax, 0x8049a1c
```

在这些假设下，发生的情况如图 26.7（第 258 页）所示。假设计数器的初始值是 50，请跟着这个例子走一遍，确保你明白发生了什么。

这里演示的情况叫作**竞态条件**（race condition，也叫竞争条件），更具体地说，是**数据竞争**（data race）：结果取决于代码执行的时机。运气不好时（即上下文切换恰好发生在执行

| OS | 线程 1 | 线程 2 | PC | eax | counter |
|---|---|---|---|---|---|
| | | | | （指令执行后） | |
| | 临界区之前 | | 100 | 0 | 50 |
| | `mov 8049a1c,%eax` | | 105 | **50** | 50 |
| | `add $0x1,%eax` | | 108 | **51** | 50 |
| **中断** | | | | | |
| 保存 *T1* | | | | | |
| 恢复 *T2* | | | 100 | 0 | 50 |
| | | `mov 8049a1c,%eax` | 105 | **50** | 50 |
| | | `add $0x1,%eax` | 108 | **51** | 50 |
| | | `mov %eax,8049a1c` | 113 | 51 | **51** |
| **中断** | | | | | |
| 保存 *T2* | | | | | |
| 恢复 *T1* | | | 108 | 51 | 51 |
| | `mov %eax,8049a1c` | | 113 | 51 | **51** |

图 26.7：**问题的近距离特写**

过程中不合时宜的地方），就会得到错误的结果。事实上，每次得到的结果都可能不同；因此，我们得到的不是那种漂亮的**确定性**计算（这是我们对计算机的一贯期望），而是一个**不确定的**（indeterminate）结果：谁也不知道输出会是什么，而且每次运行的输出很可能都不一样。

由于多个线程执行这段代码可能引发竞态条件，我们把这段代码称为**临界区**（critical section）。临界区是访问共享变量（或者更一般地说，共享资源）的一段代码，不能由多个线程同时执行。

对这段代码，我们真正想要的是所谓的**互斥**（mutual exclusion）。这一性质保证：如果一个线程正在临界区内执行，其他线程就无法进入。

顺便说一下，这些术语几乎都是 Edsger Dijkstra 创造的。他是这一领域的先驱，凭借这项工作和其他成就获得了图灵奖（Turing Award）。关于这个问题，他 1968 年的论文“Cooperating Sequential Processes”[D68] 有一段清晰得惊人的描述，值得一读。在本书的这一部分，我们还会多次听到 Dijkstra 的名字。

## 26.5 对原子性的期望

解决这个问题的一种办法是提供更强大的指令，一步就完成我们需要做的事，从而杜绝不合时宜的中断（interrupt）。例如，假设我们有一条超级指令，长这样：

```
memory-add 0x8049a1c, $0x1
```

假设这条指令把一个值加到某个内存位置上，并且硬件保证它以原子方式执行；这条指令执行时，会按预期完成更新。它不可能在执行到一半时被中断，因为这正是硬件给我们的保证：中断发生时，这条指令要么根本没有执行，要么已经执行完毕，不存在中间状态。硬件可真美好，不是吗？

提示：使用原子操作

原子操作（atomic operation）是构建计算机系统时最强大的底层技术之一，从计算机体系结构，到并发代码（concurrent code，也就是我们这里研究的内容），到我们很快就会学到的文件系统（file system），再到数据库管理系统（database management system），甚至分布式系统（distributed system），都用得到它 [L+93]。

让一系列动作成为**原子的**（atomic），背后的思想用一句话就能说清："要么全做，要么全不做"。在外界看来，要么你想打包在一起的所有动作都发生了，要么一个也没发生，看不到任何中间状态。有时，把许多动作打包成一个原子动作的做法叫作**事务**（transaction），这一思想在数据库和事务处理领域得到了非常深入的发展 [GR92]。

在探讨并发这个主题时，我们会用同步原语（synchronization primitive）把一小段指令序列变成原子的执行块；不过我们会看到，原子性（atomicity）的思想远不止于此。例如，文件系统会使用日志（journaling）或写时复制（copy-on-write）等技术，以原子的方式改变磁盘上的状态，这对系统出故障时仍能正确运行至关重要。如果现在还看不明白，别担心，以后某一章会讲清楚的。

在这里，**原子地**（atomically）的意思是"作为一个整体"，有时我们也理解为"全有或全无"。我们希望的是以原子方式执行这三条指令：

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

正如前面所说，如果有一条指令能完成这件事，我们只需发出这条指令就行了。但一般情况下，我们不会有这样的指令。想象一下，我们正在构建一棵并发 B 树（concurrent B-tree），并且想要更新它；难道真要让硬件支持一条"原子更新 B 树（B-tree）"的指令吗？大概不会，至少在一个正常的指令集里不会。

所以，我们的做法是向硬件要几条有用的指令，在它们的基础上构建一套通用的**同步原语**（synchronization primitive）。借助这些硬件支持，再加上操作系统的一些帮助，我们就能写出多线程代码，以同步、受控的方式访问临界区，即使在并发执行（concurrent execution）这种棘手的情况下，也能可靠地得出正确的结果。很棒，对吧？

这就是本书这一部分要研究的问题。这是一个美妙而困难的问题，应该会让你的脑袋（有点）疼。如果没疼，说明你还没懂！继续钻研，直到头疼为止，那时你就知道自己走对了方向。到那个时候，就休息一下吧，我们可不希望你头疼得太厉害。

关键问题：如何支持同步

为了构建有用的同步原语，我们需要硬件提供什么支持？需要操作系统提供什么支持？怎样才能正确、高效地构建这些原语？程序怎样使用它们才能得到想要的结果？

 © 2008–25, ARPACI-DUSSEAU

补充：并发的关键术语——临界区、竞态条件、不确定性、互斥

这四个术语是并发代码的核心，所以我们觉得值得把它们单独拎出来讲一讲。更多细节请参阅 Dijkstra 的一些早期工作 [D65, D68]。

- **临界区**（critical section）是访问共享资源（通常是变量或数据结构）的一段代码。
- 如果多个执行线程大致同时进入临界区，都试图更新共享的数据结构，导致出人意料（而且很可能不是我们想要）的结果，就产生了**竞态条件**（race condition），或者叫**数据竞争**（data race）[NM92]。
- **不确定的**（indeterminate）程序包含一个或多个竞态条件；程序的输出每次运行都不一样，取决于哪些线程在什么时候运行。因此结果不是**确定的**（deterministic），而确定性恰恰是我们通常对计算机系统的期望。
- 为了避免这些问题，线程应该使用某种**互斥**（mutual exclusion）原语；这样就能保证只有一个线程进入临界区，从而避免竞争，得到确定的程序输出。

## 26.6 另一个问题：等待另一个线程

本章在提出并发问题时，好像线程之间只有一种交互：访问共享变量，以及由此需要为临界区提供原子性。其实，还有另一种常见的交互：一个线程必须等另一个线程完成某个动作，才能继续执行。例如，进程执行磁盘 I/O 时会进入睡眠；I/O 完成后，需要把进程从睡眠中唤醒，让它继续执行。

因此，在接下来的几章里，我们不仅要研究如何构建同步原语来支持原子性，还要研究支持这种睡眠/唤醒交互的机制，这种交互在多线程程序中很常见。如果现在还不明白，没关系！很快就会明白的，读到**条件变量**（condition variable）那一章就知道了。如果到那时还不明白，那就不太好了，你应该把那一章再读一遍（甚至好几遍），直到弄明白为止。

## 26.7 小结：为什么要在操作系统课上讲这些？

在结束之前，你可能会问：为什么要在操作系统课上学这些？答案用一个词就能概括：“历史”。操作系统是第一个并发程序（concurrent program），许多技术最初就是为在操作系统内部使用而发明的。后来有了多线程进程（multi-threaded process），应用程序员也不得不考虑这些问题。

例如，设想有两个进程在运行。假设它们都调用 `write()` 写同一个文件，而且都想把数据追加到文件末尾（即把数据加到文件的最后，使文件变长）。为此，两个进程都必须分配一个新块（block），在文件的 inode（索引节点）里记下这个块的位置，并修改文件的大小，反映出变大后的长度（还有其他一些工作；本书第三部分会更详细地讲文件）。由于中断随时可能发生，更新这些共享结构的代码就是临界区，比如用于分配的位图（bitmap），或者文件的 inode；因此，从引入中断的那一刻起，操作系统的设计者就不得不操心系统怎样更新自己的内部结构。一次不合时宜的中断，就会引发上面描述的所有问题。毫不奇怪，页表、进程列表（process list）、文件系统结构，以及几乎所有内核数据结构，都必须使用恰当的同步原语小心地访问，才能正确工作。

## 参考文献

[D65] "Solution of a problem in concurrent programming control" by E. W. Dijkstra. Communications of the ACM, 8(9):569, September 1965. *Pointed to as the first paper of Dijkstra's where he outlines the mutual exclusion problem and a solution. The solution, however, is not widely used; advanced hardware and OS support is needed, as we will see in the coming chapters.*

[D68] "Cooperating sequential processes" by Edsger W. Dijkstra. 1968. Available at this site: http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD123.PDF. *Dijkstra has an amazing number of his old papers, notes, and thoughts recorded (for posterity) on this website at the last place he worked, the University of Texas. Much of his foundational work, however, was done years earlier while he was at the Technische Hogeschool Eindhoven (THE), including this famous paper on "cooperating sequential processes", which basically outlines all of the thinking that has to go into writing multi-threaded programs. Dijkstra discovered much of this while working on an operating system named after his school: the "THE" operating system (said "T", "H", "E", and not like the word "the").*

[GR92] "Transaction Processing: Concepts and Techniques" by Jim Gray and Andreas Reuter. Morgan Kaufmann, September 1992. *This book is the bible of transaction processing, written by one of the legends of the field, Jim Gray. It is, for this reason, also considered Jim Gray's "brain dump", in which he wrote down everything he knows about how database management systems work. Sadly, Gray passed away tragically a few years back, and many of us lost a friend and great mentor, including the co-authors of said book, who were lucky enough to interact with Gray during their graduate school years.*

[L+93] "Atomic Transactions" by Nancy Lynch, Michael Merritt, William Weihl, Alan Fekete. Morgan Kaufmann, August 1993. *A nice text on some of the theory and practice of atomic transactions for distributed systems. Perhaps a bit formal for some, but lots of good material is found herein.*

[NM92] "What Are Race Conditions? Some Issues and Formalizations" by Robert H. B. Netzer and Barton P. Miller. ACM Letters on Programming Languages and Systems, Volume 1:1, March 1992. *An excellent discussion of the different types of races found in concurrent programs. In this chapter (and the next few), we focus on data races, but later we will broaden to discuss **general races** as well.*

[SR05] "Advanced Programming in the UNIX Environment" by W. Richard Stevens and Stephen A. Rago. Addison-Wesley, 2005. *As we've said many times, buy this book, and read it, in little chunks, preferably before going to bed. This way, you will actually fall asleep more quickly; more importantly, you learn a little more about how to become a serious UNIX programmer.*

### 作业（模拟）

程序 x86.py 可以让你看到，不同的线程交错执行方式是如何导致或避免竞态条件的。请先阅读 README，了解这个程序怎么用，然后回答下面的问题。

### 问题

1. 我们先来看一个简单的程序 loop.s。首先，把它读懂。然后用这些参数运行它：./x86.py -t 1 -p loop.s -i 100 -R dx。这表示只有一个线程，每 100 条指令中断一次，并跟踪寄存器 %dx。运行过程中 %dx 的值会是多少？用 -c 选项检查你的答案：答案显示在左边，是右边那条指令执行之后寄存器（或内存）的值。
2. 同样的代码，换一组选项：./x86.py -p loop.s -t 2 -i 100 -a dx=3,dx=3 -R dx。这表示有两个线程，并把每个线程的 %dx 都初始化为 3。%dx 会出现哪些值？加上 -c 运行来检查。有了多个线程，会影响你的计算吗？这段代码里有竞争吗？
3. 运行这条命令：./x86.py -p loop.s -t 2 -i 3 -r -R dx -a dx=3,dx=3。这会让中断间隔变得很小，而且是随机的；用不同的种子（-s）看看不同的交错情况。中断频率会改变什么吗？
4. 现在换一个程序 looping-race-nolock.s，它访问位于地址 2000 的一个共享变量，我们把这个变量叫作 value。先用单个线程运行它，确认你理解了：./x86.py -p looping-race-nolock.s -t 1 -M 2000。整个运行过程中，value（即内存地址 2000 处的值）是多少？用 -c 检查。
5. 用多次循环、多个线程运行：./x86.py -p looping-race-nolock.s -t 2 -a bx=3 -M 2000。为什么每个线程都循环三次？value 的最终值是多少？
6. 用随机的中断间隔运行：./x86.py -p looping-race-nolock.s -t 2 -M 2000 -i 4 -r -s 0，并换用不同的种子（-s 1、-s 2 等）。只看线程的交错情况，你能判断出 value 的最终值吗？中断的时机重要吗？中断发生在哪里是安全的？哪里不安全？换句话说，临界区究竟在哪里？
7. 改用固定的中断间隔：./x86.py -p looping-race-nolock.s -a bx=1 -t 2 -M 2000 -i 1。共享变量 value 的最终值会是多少？改成 -i 2、-i 3 等又会怎样？在哪些中断间隔下，程序能给出“正确”的答案？
8. 用更多次循环运行同样的程序（例如设置 -a bx=100）。哪些中断间隔（-i）能得到正确的结果？哪些间隔的结果出人意料？
9. 最后一个程序：wait-for-me.s。运行 ./x86.py -p wait-for-me.s -a ax=1,ax=0 -R ax -M 2000。这会把线程 0 的 %ax 寄存器设为 1，线程 1 的设

为 0，并监视 `%ax` 和内存位置 2000。这段代码应该表现出怎样的行为？两个线程怎样使用位置 2000 处的值？它的最终值会是多少？

10. 现在把输入对调：`./x86.py -p wait-for-me.s -a ax=0,ax=1 -R ax -M 2000`。这两个线程表现如何？线程 0 在做什么？改变中断间隔（例如 `-i 1000`，或者改用随机间隔）会怎样改变跟踪结果？这个程序是否高效地利用了 CPU？

 © 2008–25, ARPACI-DUSSEAU

# 27

# 插叙：线程 API

本章简要介绍线程 API（thread API）的主要部分。在后面的章节里，我们会演示如何使用这些 API，届时还会进一步讲解每一部分。更多细节可以在各种书籍和网上资料中找到[B89, B97, B+96, K+96]。需要说明的是，后面几章会配合大量例子，放慢节奏地介绍锁（lock）和条件变量（condition variable）的概念；因此，本章更适合当作参考资料来用。

关键问题：如何创建和控制线程
操作系统应该为创建和控制线程（thread）提供哪些接口？这些接口应该怎样设计，才能既好用又实用？

## 27.1 创建线程

要编写多线程（multi-threaded）程序，首先得能创建新线程，所以必须有某种创建线程的接口。在 POSIX 中，这很简单：

```
#include <pthread.h>
int
pthread_create(pthread_t             *thread,
               const pthread_attr_t *attr,
               void                  *(*start_routine)(void*),
               void                  *arg);
```

这个声明看起来可能有点复杂（尤其是如果你没在 C 里用过函数指针的话），但其实没那么糟。它有四个参数：`thread`、`attr`、`start_routine` 和 `arg`。第一个参数 `thread` 是一个指向 `pthread_t` 类型结构的指针；我们要通过这个结构和该线程打交道，因此需要把它传给 `pthread_create()`，让它完成初始化。

第二个参数 `attr` 用来指定这个线程可能具有的属性，比如设置栈（stack）的大小，或者提供线程调度优先级（priority）方面的信息。属性要单独调用 `pthread_attr_init()` 来初始化，详见手册页（manual page）。不过在大多数情况下，用默认值就行；这时我们只需传入 `NULL`。

第三个参数最复杂，但它其实只是在问：这个线程应该从哪个函数开始运行？在 C 里，我们把它叫作**函数指针**（function pointer）。这个函数指针告诉我们，它需要的是这样一个

```
#include <stdio.h>
#include <pthread.h>

typedef struct {
    int a;
    int b;
} myarg_t;

void *mythread(void *arg) {
    myarg_t *args = (myarg_t *) arg;
    printf("%d %d\n", args->a, args->b);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p;
    myarg_t args = { 10, 20 };

    int rc = pthread_create(&p, NULL, mythread, &args);
    ...
}
```

图 27.1：**创建线程**

函数：函数名为 `start_routine`，接受一个 `void *` 类型的参数（见 `start_routine` 后面括号里的内容），并返回一个 `void *` 类型的值（即一个 **void 指针**，void pointer）。

如果这个例程需要的是整数参数，而不是 void 指针，声明就会是这样：

```
int pthread_create(..., // 前两个参数不变
                   void *(*start_routine)(int),
                   int arg);
```

如果这个例程接受 void 指针作为参数，但返回一个整数，声明就会是这样：

```
int pthread_create(..., // 前两个参数不变
                   int (*start_routine)(void *),
                   void *arg);
```

最后，第四个参数 `arg` 就是要传给线程起始函数的那个参数。你可能会问：为什么要用这些 void 指针？答案很简单：把 void 指针作为 `start_routine` 函数的参数，我们就能传入任何类型的参数；把它作为返回值，线程就能返回任何类型的结果。

我们来看图 27.1 中的例子。这里只是创建了一个线程，给它传了两个参数，这两个参数打包成了我们自己定义的一个类型（`myarg_t`）。线程创建后，只需把参数强制转换成它期望的类型，就能按需取出这些参数。

就是这样！线程一旦创建，你就真正多了一个正在执行的实体，它有自己的调用栈（call stack），和程序中现有的所有线程运行在同一个地址空间（address space）里。好戏就此开场！

## 27.2 等待线程结束

上面的例子展示了如何创建线程。可是，如果想等某个线程结束，该怎么办呢？这得做点特别的事：调用例程 `pthread_join()`。

```
int pthread_join(pthread_t thread, void **value_ptr);
```

这个例程接受两个参数。第一个参数是 `pthread_t` 类型，用来指定要等待哪个线程。这个变量由线程创建例程初始化（你把指向它的指针作为参数传给 `pthread_create()` 时）；只要把它保留下来，就可以用它来等待那个线程结束。

第二个参数是一个指针，指向你期望拿回的返回值。因为这个例程可以返回任何东西，所以返回类型定义成了 void 指针；又因为 `pthread_join()` 例程会修改传入参数的值，所以你需要传入指向这个值的指针，而不只是这个值本身。

我们再来看一个例子（图 27.2，第 266 页）。在这段代码中，同样创建了一个线程，并通过 `myarg_t` 结构给它传了几个参数。返回值用 `myret_t` 类型来传递。主线程（main thread）一直在 `pthread_join()` 例程[1]里等待；线程运行结束后，这个调用随即返回，我们就可以访问线程返回的值，也就是 `myret_t` 里的内容。

关于这个例子，有几点需要注意。首先，很多时候我们并不需要这样费劲地打包、拆包参数。例如，如果创建的线程不需要参数，创建时传入 `NULL` 作为参数就行。同样，如果不关心返回值，也可以给 `pthread_join()` 传入 `NULL`。

```
1  typedef struct { int a; int b; } myarg_t;
2  typedef struct { int x; int y; } myret_t;
3
4  void *mythread(void *arg) {
5      myret_t *rvals = Malloc(sizeof(myret_t));
6      rvals->x = 1;
7      rvals->y = 2;
8      return (void *) rvals;
9  }
10
11 int main(int argc, char *argv[]) {
12     pthread_t p;
13     myret_t *rvals;
14     myarg_t args = { 10, 20 };
15     Pthread_create(&p, NULL, mythread, &args);
16     Pthread_join(p, (void **) &rvals);
17     printf("returned %d %d\n", rvals->x, rvals->y);
18     free(rvals);
19     return 0;
20 }
```

图 27.2：**等待线程结束**

[1]注意，这里我们用的是包装函数；具体来说，我们调用的是 `Malloc()`、`Pthread_join()` 和 `Pthread_create()`，它们只是调用对应的小写版本，并确认这些例程没有返回什么意外的结果。

```
void *mythread(void *arg) {
    long long int value = (long long int) arg;
    printf("%lld\n", value);
    return (void *) (value + 1);
}

int main(int argc, char *argv[]) {
    pthread_t p;
    long long int rvalue;
    Pthread_create(&p, NULL, mythread, (void *) 100);
    Pthread_join(p, (void **) &rvalue);
    printf("returned %lld\n", rvalue);
    return 0;
}
```

图 27.3：**更简单的线程传参方式**

其次，如果只传一个值（例如一个 `long long int`），就不必把它打包成参数。图 27.3（第 267 页）给出了一个例子。这种情况下就简单一些，因为不必把参数和返回值装进结构体里。

第三，从线程返回值时一定要格外小心。具体来说，千万不要返回一个指针，让它指向分配在线程调用栈（call stack）上的东西。如果这样做，你觉得会发生什么？（好好想想！）下面是一段危险的代码，是从图 27.2 的例子改出来的。

```
1 void *mythread(void *arg) {
2     myarg_t *args = (myarg_t *) arg;
3     printf("%d %d\n", args->a, args->b);
4     myret_t oops; // 分配在栈上：错误！
5     oops.x = 1;
6     oops.y = 2;
7     return (void *) &oops;
8 }
```

在这个例子中，变量 `oops` 分配在 `mythread` 的栈上。然而，函数返回时，这个值会被自动释放（毕竟，这正是栈用起来这么方便的原因！）；因此，传回一个指向已释放变量的指针，会引发各种糟糕的后果。打印你以为已经返回的那些值时，你多半（但不一定！）会大吃一惊。自己试试看吧[2]！

最后，你可能会注意到，先用 `pthread_create()` 创建一个线程，紧接着就调用 `pthread_join()`，这种创建线程的方式相当奇怪。事实上，有一种更简单的办法能完成同样的事，它叫作**过程调用**（procedure call）。显然，我们通常会创建不止一个线程并等待它们结束，否则用线程就没多大意义了。

需要指出的是，并非所有多线程代码都会用到 join 例程。例如，一个多线程的 Web 服务器可能会创建若干工作线程（worker thread），然后用主线程接收请求，再把请求交给这些工作线程，如此一直运行下去。这类长期运行的程序可能就不需要 join。然而，如果一个

[2]好在你写出这样的代码时，编译器 `gcc` 很可能会发出警告，这又是一个要重视编译器警告的理由。

并行程序（parallel program）创建线程来（并行地）执行某项特定任务，它很可能会用 join 来确保所有这些工作都已完成，然后才退出或进入下一阶段的计算。

## 27.3 锁

除了创建线程和 join，POSIX 线程库（thread library）提供的下一组最有用的函数，大概就是用**锁**（lock）来为临界区（critical section）提供互斥（mutual exclusion）的那些函数了。为此要用到的最基本的一对例程是：

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

这两个例程应该很容易理解和使用。如果有一段代码是**临界区**（critical section），因而需要保护起来以确保正确运行，锁就非常有用。你大概能想象出代码是什么样子：

```
pthread_mutex_t lock;
pthread_mutex_lock(&lock);
x = x + 1; // 或者你的临界区里要做的任何事
pthread_mutex_unlock(&lock);
```

这段代码的意图是这样的：如果调用 `pthread_mutex_lock()` 时没有其他线程持有锁，这个线程就会获取锁，进入临界区。如果确实有另一个线程持有锁，试图获取锁的线程就不会从这个调用中返回，直到它拿到锁为止（拿到锁，意味着原先持有锁的线程已经调用解锁（unlock），把锁释放了）。当然，在某一时刻，可能有许多线程都卡在获取锁的函数里等待；不过，只有已经拿到锁的线程才应该调用解锁。

遗憾的是，这段代码在两个重要的方面有问题。第一个问题是**缺少正确的初始化**（lack of proper initialization）。所有的锁都必须正确初始化，才能保证它们一开始就有正确的值，从而在加锁（locking）和解锁时按预期工作。

POSIX 线程（POSIX threads）提供了两种初始化锁的方法。第一种是像下面这样使用 `PTHREAD_MUTEX_INITIALIZER`：

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

这样会把锁设为默认值，从而让锁可用。动态的方法（即在运行时初始化）是调用 `pthread_mutex_init()`，如下所示：

```
int rc = pthread_mutex_init(&lock, NULL);
assert(rc == 0); // 一定要检查是否成功!
```

这个例程的第一个参数是锁本身的地址，第二个参数是一组可选的属性。属性的详细内容请自行阅读；传入 `NULL` 就表示使用默认值。两种方法都可以，不过我们通常用动态的（后一种）方法。注意，用完锁之后，还应该相应地调用 `pthread_mutex_destroy()`；所有细节请参阅手册页。

上面代码的第二个问题是，调用加锁和解锁时没有检查错误码。和你在 UNIX 系统中调用的几乎所有库例程一样，这些例程也可能失败！如果代码没有正确检查错误码，失败就会悄无声息地发生；在这个例子中，这可能让多个线程同时进入临界区。至少要使用包装函数，

```
// 让代码保持简洁；只有在失败时可以直接 exit() 才能这样用
void Pthread_mutex_lock(pthread_mutex_t *mutex) {
    int rc = pthread_mutex_lock(mutex);
    assert(rc == 0);
}
```

图 27.4：**一个包装函数的例子**

用断言确认调用成功，如图 27.4（第 269 页）所示；更复杂的（不是玩具的）程序出错时不能一退了之，应该检查调用是否失败，并在失败时做出适当的处理。

在 pthread 库中，和锁打交道的例程并不只有加锁和解锁。还有两个值得关注的例程：

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_timedlock(pthread_mutex_t *mutex,
                            struct timespec *abs_timeout);
```

这两个调用都用来获取锁。如果锁已经被别的线程持有，`trylock` 版本会直接返回失败；`timedlock` 版本则会在超时（timeout）或者拿到锁之后返回，看哪个先发生。因此，超时时间为零的 `timedlock` 就退化成了 `trylock`。一般来说，这两个版本都应该避免使用；不过在少数情况下，避免卡在获取锁的例程里（也许是无限期地卡住）还是有用的，后面的章节里就会看到这样的例子，比如研究死锁（deadlock）的时候。

## 27.4 条件变量

任何线程库的另一个重要组成部分（POSIX 线程当然也不例外）是**条件变量**（condition variable）。如果线程之间需要以某种方式互相发信号（signal），比如一个线程要等另一个线程做完某件事才能继续，条件变量就很有用。想以这种方式交互的程序，主要用到两个例程：

```
int pthread_cond_wait(pthread_cond_t *cond,
                      pthread_mutex_t *mutex);
int pthread_cond_signal(pthread_cond_t *cond);
```

要使用条件变量，还必须有一把和这个条件相关联的锁。调用上面任意一个例程时，都应该持有这把锁。

第一个例程 `pthread_cond_wait()` 让调用它的线程进入睡眠，等待其他线程给它发信号。发信号的时机通常是程序里发生了某种变化，而这个正在睡眠的线程可能关心这种变化。典型的用法是这样的：

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  cond = PTHREAD_COND_INITIALIZER;

Pthread_mutex_lock(&lock);
while (ready == 0)
    Pthread_cond_wait(&cond, &lock);
Pthread_mutex_unlock(&lock);
```

在这段代码中，相关的锁和条件变量初始化之后[3]，线程检查变量 `ready` 是否已经设成了非零值。如果没有，线程就调用等待例程进入睡眠，直到有别的线程把它唤醒。

唤醒线程的代码在另外某个线程里运行，大致是这样的：

```
Pthread_mutex_lock(&lock);
ready = 1;
Pthread_cond_signal(&cond);
Pthread_mutex_unlock(&lock);
```

关于这段代码，有几点需要注意。首先，发信号时（以及修改全局变量 `ready` 时），我们总是确保持有锁。这样可以保证我们不会不小心在代码里引入竞态条件（race condition）。

其次，你可能会注意到，等待调用的第二个参数是一把锁，而发信号的调用只接受一个条件变量。之所以有这个区别，是因为等待调用除了让调用线程进入睡眠，还会在让它睡眠的同时释放这把锁。想象一下，如果它不释放锁，另一个线程怎么可能拿到锁、再发信号唤醒它呢？不过，`pthread_cond_wait()` 在被唤醒后、返回之前，会重新获取这把锁，从而保证等待线程从等待序列开头获取锁、到结尾释放锁的这段时间里，只要它在运行，就一定持有锁。

最后还有一个怪异之处：等待线程是在一个 `while` 循环里重新检查条件，而不是用一个简单的 `if` 语句。在后面专门研究条件变量的章节里，我们会详细讨论这个问题；不过一般来说，用 `while` 循环是简单而安全的做法。虽然它会重新检查条件（也许会增加一点开销），但有些 pthread 实现可能会虚假地唤醒等待线程；在这种情况下，如果不重新检查，等待线程就会以为条件已经变了，而实际上并没有。因此，更稳妥的做法是把被唤醒看作一个提示，表示有些东西可能变了，而不是把它当作确凿的事实。

注意，有时你会想用一个简单的标志在两个线程之间传递信号，而不用条件变量和相关的锁。例如，我们可以把上面的等待代码改写成这样：

```
while (ready == 0)
    ; // 自旋
```

相应的发信号代码则是这样的：

```
ready = 1;
```

千万不要这样做，原因如下。首先，它在很多情况下性能很差：长时间自旋（spin）只会白白浪费 CPU 周期。其次，它容易出错。最近的研究表明 [X+10]，用标志（像上面那样）在线程之间同步，出错的可能性大得惊人；在那项研究中，这类 *ad hoc* 同步大约有一半都有 bug！不要偷懒；即使你觉得不用条件变量也能蒙混过关，也要用条件变量。

如果条件变量听起来让人困惑，（暂时）不用太担心，后面会有一章专门详细讲解。在那之前，只要知道有这么个东西，并大致了解怎样用、为什么要用就够了。

[3]也可以用 `pthread_cond_init()`（以及 `pthread_cond_destroy()`）代替静态初始化器 `PTHREAD_COND_INITIALIZER`。听起来更麻烦？确实更麻烦。

## 27.5 编译和运行

本章所有的代码示例都比较容易跑起来。要编译它们，代码里必须包含头文件 `pthread.h`。在链接命令中，还必须加上 `-pthread` 选项，显式地链接 pthreads 库。

例如，编译一个简单的多线程程序，只需这样做：

```
prompt> gcc -o main main.c -Wall -pthread
```

只要 `main.c` 包含了 pthreads 的头文件，你就成功编译了一个并发程序（concurrent program）。至于它能不能正常工作，和往常一样，那完全是另一回事了。

## 27.6 小结

我们介绍了 pthread 库的基础知识，包括创建线程、用锁实现互斥，以及用条件变量发信号和等待。要写出健壮、高效的多线程代码，除了耐心和万分小心，别的你也不需要多少了！

本章最后列出一组提示，在你编写多线程代码时可能会派上用场（详见下一页的补充）。API 还有其他一些有意思的方面；如果想了解更多，可以在 Linux 系统上输入 `man -k pthread`，查看构成整个接口的一百多个 API。不过，这里讨论的基础知识应该足以让你构建出复杂的（但愿也是正确且高性能的）多线程程序。线程的难点不在于 API，而在于构建并发程序时那些棘手的逻辑。请继续往下读，了解更多内容。

 © 2008–23, ARPACI-DUSSEAU

补充：使用线程 API 的指导原则

使用 POSIX 线程库构建多线程程序时（其实用任何线程库都一样），有一些虽小但很重要的事情需要记住。它们是：

- **保持简单**。最重要的是，在线程之间加锁或发信号的代码都应该尽可能简单。复杂的线程交互会导致 bug。
- **尽量减少线程之间的交互**。让线程之间交互的方式尽可能少。每一种交互都要仔细思考，并用经过验证的可靠方法来构建（其中很多方法我们会在后面几章学到）。
- **初始化锁和条件变量**。不这样做，代码就会时而正常、时而以非常奇怪的方式出错。
- **检查返回码**。当然，无论你写什么 C 和 UNIX 程序，都应该检查每一个返回码，这里也一样。不这样做，程序就会出现怪异而难以理解的行为，让你很可能（a）尖叫，（b）揪掉几根头发，或者（c）两者兼有。
- **注意给线程传参数、从线程返回值的方式**。尤其是，如果你传的是一个引用，指向栈上分配的变量，那多半就做错了。
- **每个线程都有自己的栈**。和上一点相关，请记住每个线程都有自己的栈。因此，如果线程正在执行的某个函数里有一个局部分配的变量，这个变量实际上是该线程私有的；其他线程无法（轻易）访问它。要在线程之间共享数据，这些值必须放在**堆**（heap）里，或者放在其他某个全局可访问的地方。
- **总是用条件变量在线程之间发信号**。虽然用一个简单的标志往往很诱人，但不要这样做。
- **查阅手册页**。尤其是在 Linux 上，pthread 的手册页信息非常丰富，讨论了这里提到的许多细微之处，而且往往讲得更详细。请仔细阅读！

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[B89] "An Introduction to Programming with Threads" by Andrew D. Birrell. DEC Technical Report, January, 1989. Available: https://birrell.org/andrew/papers/035-Threads.pdf *A classic but older introduction to threaded programming. Still a worthwhile read, and freely available.*

[B97] "Programming with POSIX Threads" by David R. Butenhof. Addison-Wesley, May 1997. *Another one of these books on threads.*

[B+96] "PThreads Programming: by A POSIX Standard for Better Multiprocessing." Dick Buttlar, Jacqueline Farrell, Bradford Nichols. O'Reilly, September 1996 *A reasonable book from the excellent, practical publishing house O'Reilly. Our bookshelves certainly contain a great deal of books from this company, including some excellent offerings on Perl, Python, and Javascript (particularly Crockford's "Javascript: The Good Parts".)*

[K+96] "Programming With Threads" by Steve Kleiman, Devang Shah, Bart Smaalders. Prentice Hall, January 1996. *Probably one of the better books in this space. Get it at your local library. Or steal it from your mother. More seriously, just ask your mother for it – she'll let you borrow it, don't worry.*

[X+10] "Ad Hoc Synchronization Considered Harmful" by Weiwei Xiong, Soyeon Park, Jiaqi Zhang, Yuanyuan Zhou, Zhiqiang Ma. OSDI 2010, Vancouver, Canada. *This paper shows how seemingly simple synchronization code can lead to a surprising number of bugs. Use condition variables and do the signaling correctly!*

### 作业（编程）

在这一节，我们会编写一些简单的多线程程序，并用一个叫作 **helgrind** 的专门工具来找出这些程序中的问题。如何构建这些程序、如何运行 helgrind，请阅读作业下载包里的 README。

### 问题

1. 首先构建 `main-race.c`。读一读这段代码，找出其中（希望很明显的）数据竞争（data race）。然后运行 helgrind（输入 `valgrind --tool=helgrind main-race`），看看它是怎样报告这个竞争的。它指出的代码行对吗？它还给了你哪些信息？
2. 把其中一行出问题的代码删掉，会发生什么？现在给共享变量的其中一处更新加上锁，然后再给两处更新都加上锁。在这几种情况下，helgrind 分别报告了什么？
3. 现在来看 `main-deadlock.c`。读一读这段代码。它有一个叫作**死锁**（deadlock）的问题（后面会有一章更深入地讨论）。你能看出它可能有什么问题吗？
4. 现在对这段代码运行 helgrind。helgrind 报告了什么？
5. 现在用 helgrind 检查 `main-deadlock-global.c`。先读一读这段代码：它和 `main-deadlock.c` 有一样的问题吗？helgrind 应该报告同样的错误吗？这说明了 helgrind 这类工具的什么问题？
6. 接下来看 `main-signal.c`。这段代码用一个变量（`done`）来发信号，表示子线程（child thread）已经完成、父线程（parent thread）可以继续了。为什么这段代码效率很低？尤其是子线程要很久才能完成的时候，父线程最终把时间花在了什么上面？
7. 现在对这个程序运行 helgrind。它报告了什么？这段代码正确吗？
8. 现在看这段代码稍加修改后的版本，在 `main-signal-cv.c` 里。这个版本用条件变量和相关的锁来发信号。为什么这段代码比之前的版本更好？是因为正确性，还是性能，还是两者都有？
9. 再对 `main-signal-cv` 运行一次 helgrind。它报告了什么错误吗？

 © 2008–23, ARPACI-DUSSEAU

# 28

# 锁

在介绍并发（concurrency）的那一章里，我们看到了并发编程的一个基本问题：我们希望原子地（atomically）执行一系列指令，但由于单处理器上存在中断（interrupt），或者多个线程（thread）在多个处理器上并发执行，我们做不到。本章就来正面解决这个问题，为此要引入一种叫作**锁**（lock）的东西。程序员在源代码中用锁把临界区（critical section）包起来，从而确保每个这样的临界区都像一条原子指令（atomic instruction）一样执行。

## 28.1 锁的基本思想

举个例子，假设我们的临界区是下面这样，也就是对共享变量最典型的一种更新：

```
balance = balance + 1;
```

当然，临界区也可以是别的，比如向链表（linked list）中添加一个元素，或者对共享结构做其他更复杂的更新；不过现在我们只看这个简单的例子。要使用锁，我们在临界区周围加上这样的代码：

```
1 lock_t mutex; // 某个全局分配的锁 'mutex'
2 ...
3 lock(&mutex);
4 balance = balance + 1;
5 unlock(&mutex);
```

锁就是一个变量，因此要使用锁，就必须声明某种**锁变量**（lock variable），比如上面的 `mutex`。这个锁变量（lock variable，简称“锁”）保存着锁在任一时刻的状态：要么是**可用的**（available），也叫**未锁定**（unlocked）或**空闲**（free），此时没有线程持有锁；要么是**已获取的**（acquired），也叫**已锁定**（locked）或**被持有**（held），此时恰好有一个线程持有锁，而且这个线程多半正处在临界区中。我们也可以在这个数据类型中保存其他信息，比如哪个线程持有锁，或者用一个队列（queue）给获取锁的请求排序，但这类信息对锁的使用者是隐藏的。

`lock()` 和 `unlock()` 例程的语义很简单。调用 `lock()` 会尝试获取锁：如果没有其他线程持有锁（即锁是空闲的），这个线程就会获取锁，进入临界区；人们有时把这个线程叫作锁的**持有者**（owner）。如果此时另一个线程对同一个锁变量（lock variable，本例中是 `mutex`）调用 `lock()`，那么只要锁还被别的线程持有，这次调用就不会返回；这样，在持有锁的第一个线程还待在临界区里时，其他线程就进不去。

锁的持有者一调用 unlock()，锁就又变成可用（空闲）的了。如果没有其他线程在等这个锁（即没有其他线程调用了 lock() 并卡在里面），锁的状态就只是变成空闲。如果有线程在等（卡在 lock() 里），其中一个线程迟早会注意到（或被告知）锁的状态变了，于是获取锁，进入临界区。

锁让程序员对调度有了最起码的一点控制权。一般来说，线程由程序员创建，却由操作系统按它自己选择的方式来调度。锁把一部分控制权交还给了程序员：在一段代码周围加上锁，程序员就能保证这段代码里任何时候最多只有一个线程在运行。因此，锁有助于把传统操作系统调度的一片混乱，变成一种更可控的活动。

## 28.2 Pthread 锁

POSIX 库把锁叫作**互斥锁**（mutex），因为它用来在线程之间实现**互斥**（mutual exclusion）：如果一个线程在临界区中，它就把其他线程挡在外面，直到它执行完这段临界区为止。因此，当你看到下面这段 POSIX 线程（POSIX threads）代码时，应该明白它做的事情和上面一样（我们同样用了自己的包装函数，在加锁（locking）和解锁（unlock）时检查错误）：

```
1 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2
3 Pthread_mutex_lock(&lock); // 包装函数；失败时退出
4 balance = balance + 1;
5 Pthread_mutex_unlock(&lock);
```

你可能还注意到，POSIX 版本给加锁和解锁传入了一个变量，因为我们可能会用不同的锁来保护不同的变量。这样做可以提高并发度：与其用一把大锁（big lock），访问任何临界区时都用它（这是一种**粗粒度**（coarse-grained）的加锁策略），通常不如用不同的锁来保护不同的数据和数据结构（data structure），从而允许更多线程同时处在加了锁的代码中（这是一种**细粒度**（fine-grained）的方法）。

## 28.3 实现一个锁

到这里，你应该已经从程序员的角度对锁的工作方式有了一些了解。但锁应该怎样实现呢？需要什么硬件支持？需要什么操作系统支持？本章剩下的部分就来回答这一系列问题。

> 关键问题：如何实现锁
> 如何实现一个高效的锁？高效的锁能以较低的代价提供互斥，还可能具备下面要讨论的其他一些性质。需要什么硬件支持？需要什么操作系统支持？

要实现一个能用的锁，我们需要老朋友硬件的帮助，也需要好伙伴操作系统的帮助。多年来，各种计算机体系结构的指令集中加入了许多不同的硬件原语；我们不会研究这些指令是怎么实现的（毕竟，那是计算机体系结构课的内容），但会研究如何用它们来构建锁这样

的互斥原语。我们还会研究操作系统如何参与进来，把整个图景补全，让我们能构建出一个完善的锁库（lock library）。

## 28.4 评价锁

在动手实现锁之前，我们应该先弄清楚目标是什么，所以要问：怎样评价某种锁实现的效果？要评价一个锁能不能用（以及好不好用），应该先定下一些基本标准。第一条是锁能否完成它的基本任务，也就是提供**互斥**（mutual exclusion）。说白了，就是这个锁管不管用，能不能阻止多个线程同时进入临界区？

第二条是**公平性**（fairness）。锁空闲时，每个争抢锁的线程是否都有公平的机会拿到它？换个角度，看看更极端的情况：有没有争抢锁的线程会**饿死**（starve），永远也拿不到锁？

最后一条标准是**性能**（performance），具体来说，是使用锁所增加的时间开销。这里有几种情况值得考虑。一种是没有争用（contention）的情况：只有一个线程在运行，它获取锁、再释放锁，开销是多少？另一种是多个线程在单个 CPU 上争抢锁的情况：这时有没有性能问题？最后，当涉及多个 CPU、每个 CPU 上都有线程在争抢锁时，锁的表现又如何？比较这几种不同的场景，我们就能更好地理解下面介绍的各种加锁技术对性能有什么影响。

## 28.5 控制中断

最早用来提供互斥的方案之一，是在临界区内关闭中断；这个方案是为单处理器系统发明的。代码大概是这样的：

```
1 void lock() {
2     DisableInterrupts();
3 }
4 void unlock() {
5     EnableInterrupts();
6 }
```

假设我们运行在这样一个单处理器系统上。在进入临界区之前（用某种特殊的硬件指令）关闭中断，就能确保临界区内的代码不会被中断，从而像原子操作（atomic operation）一样执行。执行完之后，我们再（同样通过一条硬件指令）重新打开中断，程序照常往下执行。

这种方法的主要优点是简单。你肯定不用绞尽脑汁就能想明白它为什么管用。没有中断，线程就可以确信自己执行的代码会一直执行下去，不会有其他线程来干扰。

可惜，它的缺点很多。首先，这种方法要求我们允许任何调用它的线程执行特权操作，即打开和关闭中断，因此必须信任这个功能不会被滥用。你已经知道，每当我们不得不信任任意一个程序时，多半就要有麻烦了。这里的麻烦体现在很多方面：一个贪心（greedy）的程序可以在开始执行时就调用 `lock()`，从而独占处理器；更糟糕的是，一个出了错或者怀有恶意的程序可以调用 `lock()`，然后进入死循环。在后一种情况下，操作系统永远也拿不回系统的控制权，只剩一个办法：重启系统。把关中断当作通用的同步方案，需要对应用程序有太多的信任。

其次，这种方法在多处理器（multiprocessor）上不管用。如果多个线程在不同的 CPU 上运行，而且都想进入同一个临界区，那么关不关中断都无济于事；线程照样可以在其他处

```
1  typedef struct __lock_t { int flag; } lock_t;
2
3  void init(lock_t *mutex) {
4      // 0 -> 锁可用，1 -> 锁被持有
5      mutex->flag = 0;
6  }
7
8  void lock(lock_t *mutex) {
9      while (mutex->flag == 1) // 测试（TEST）标志
10         ; // 自旋等待（什么也不做）
11     mutex->flag = 1;         // 现在设置（SET）它!
12 }
13
14 void unlock(lock_t *mutex) {
15     mutex->flag = 0;
16 }
```

图 28.1：**第一次尝试：一个简单的标志**

理器上运行，因而还是可能进入临界区。如今多处理器已经很普遍，我们的通用方案必须比这更好。

第三，长时间关闭中断可能导致中断丢失，进而引发严重的系统问题。例如，想象一下 CPU 没有注意到磁盘设备已经完成了一个读请求。操作系统又怎么知道该去唤醒等待这次读操作的进程（process）呢?

出于这些原因，只有在少数场合才会把关中断当作互斥原语来用。例如，在某些情况下，操作系统本身会用屏蔽中断的办法，在访问自己的数据结构时保证原子性（atomicity），或者至少防止出现某些棘手的中断处理情况。这种用法是合理的，因为在操作系统内部不存在信任问题，操作系统反正总是信任自己能执行特权操作（privileged operation）。

## 28.6 失败的尝试：只用加载和存储

要想不依赖中断来实现一个像样的锁，我们就得依靠 CPU 硬件以及它提供的指令。我们先试着只用一个标志变量来实现一个简单的锁。在这次失败的尝试中，我们会看到实现锁所需的一些基本思路，并且（但愿能）明白，为什么只用一个变量、靠普通的加载和存储来访问它是不够的。

在这第一次尝试中（图 28.1），思路很简单：用一个简单的变量（`flag`）表示是否有线程占有锁。第一个进入临界区的线程会调用 `lock()`，它**测试**标志是否等于 1（这时不等于），然后把标志**设置**为 1，表示这个线程现在**持有**锁。线程执行完临界区后，调用 `unlock()` 清除标志，表示锁不再被持有。

如果在第一个线程处于临界区时，另一个线程恰好调用了 `lock()`，它就只会在 `while` 循环中**自旋等待**（spin-wait），等第一个线程调用 `unlock()` 清除标志。第一个线程一清除标志，等待的线程就会跳出 `while` 循环，替自己把标志设为 1，然后进入临界区。

| 线程 1 | 线程 2 |
| --- | --- |
| 调用 `lock()` | |
| while (flag == 1) | |
| **中断：切换到线程 2** | |
| | 调用 `lock()` |
| | while (flag == 1) |
| | flag = 1; |
| | **中断：切换到线程 1** |
| flag = 1; // 把标志设为 1（这边也设了！） | |

图 28.2：**跟踪：没能实现互斥**

可惜，这段代码有两个问题：一个是正确性问题，另一个是性能问题。只要你习惯了思考并发编程（concurrent programming），正确性问题很容易看出来。想象一下图 28.2 中的代码交错执行，假设一开始 `flag=0`。

从这个交错执行可以看出，只要中断来得“及时”（还是说不合时宜?），我们很容易就能造出这样一种情况：两个线程都把标志设成了 1，于是两个线程都能进入临界区。这种行为就是专业人士所说的“糟糕”——我们显然没能满足最基本的要求：互斥。

性能问题我们后面再细说，它出在线程等待已被持有的锁的方式上：线程不停地检查 `flag` 的值，这种技术叫作**自旋等待**（spin-waiting）。自旋等待把时间都浪费在等另一个线程释放锁上了。在单处理器上，这种浪费尤其严重，因为等待者所等的那个线程根本无法运行（至少在发生上下文切换（context switch）之前是这样）！因此，在接下来设计更精巧的方案时，我们也应该考虑如何避免这种浪费。

## 28.7 用测试并设置实现可用的自旋锁

关中断在多处理器上不管用，只用加载和存储的简单方法（如上所示）也行不通，于是系统设计者开始发明支持加锁的硬件机制。最早的多处理器系统，比如 20 世纪 60 年代初的 Burroughs B5000 [M82]，就有这样的支持；如今所有系统都提供这类支持，单 CPU 系统也不例外。

最容易理解的一种硬件支持，叫作**测试并设置**（test-and-set，TAS）指令，也叫**原子交换**（atomic exchange）[1]指令。下面这段 C 代码定义了测试并设置（test-and-set）指令做的事情：

```
1 int TestAndSet(int *old_ptr, int new) {
2     int old = *old_ptr; // 取出 old_ptr 处的旧值
3     *old_ptr = new;     // 把 'new' 存入 old_ptr
4     return old;         // 返回旧值
5 }
```

测试并设置（test-and-set）指令做的事情如下：它返回 `old_ptr` 所指向的旧值，同时把这个值更新为 `new`。当然，关键在于这一系列操作是**原子地**（atomically）执行的。之所

[1]每种支持测试并设置（test-and-set）的体系结构对它的叫法都不一样。在 SPARC 上，它叫作 load/store unsigned byte 指令（`ldstub`）；在 x86 上，它是加锁版本的原子交换（atomic exchange）指令（`xchg`）。

补充：Dekker 算法和 Peterson 算法

20 世纪 60 年代，Dijkstra 向他的朋友们提出了并发问题，其中一位名叫 Theodorus Jozef Dekker 的数学家想出了一个解决方案 [D68]。我们这里讨论的方案要用特殊的硬件指令，甚至还要操作系统支持；**Dekker 算法**（Dekker's algorithm）则不同，它只用加载和存储（前提是它们彼此之间是原子的，这在早期硬件上成立）。

后来，Peterson 改进了 Dekker 的方法 [P81]。这里同样只用加载和存储，思路是确保两个线程永远不会同时进入临界区。下面是针对两个线程的 **Peterson 算法**（Peterson's algorithm），看看你能不能看懂这段代码。`flag` 和 `turn` 这两个变量是做什么用的？

```
int flag[2];
int turn;

void init() {
    // 用 'flag' 表示你打算持有锁
    flag[0] = flag[1] = 0;
    // 轮到谁了？（线程 0 还是 1）
    turn = 0;
}
void lock() {
    // 'self' 是调用者的线程 ID
    flag[self] = 1;
    // 把机会让给另一个线程
    turn = 1 - self;
    while ((flag[1-self] == 1) && (turn == 1 - self))
        ; // 没轮到你时就自旋等待
}
void unlock() {
    // 撤销你的意图即可
    flag[self] = 0;
}
```

不知为何，有一阵子，开发不需要特殊硬件支持的锁风靡一时，给搞理论的人出了很多题目。当然，假设有一点硬件支持要容易得多，何况在多处理器早期就已经有这种支持了；人们意识到这一点之后，这方面的工作也就变得没什么用了。此外，上面这类算法在现代硬件上并不管用（因为现代硬件采用宽松内存一致性模型，relaxed memory consistency model），这让它们比以前更没用了。又一项研究被扔进了历史的垃圾堆……

以叫“测试并设置（test-and-set）”，是因为它让你在“测试”旧值（也就是返回的那个值）的同时，把这个内存位置“设置”为新值；这条指令只比加载、存储稍强一点，却足以实现一个简单的**自旋锁**（spin lock），我们现在就来看图 28.3。或者更好的办法是：先自己想一想！

我们来确认一下自己真的明白这个锁为什么管用。先设想一种情况：一个线程调用 `lock()`，而当前没有其他线程持有锁，因此 `flag` 应该是 0。这个线程调用 `TestAndSet(flag, 1)` 时，该例程会返回 `flag` 的旧值，也就是 0；于是，正在测试 `flag` 值的调用线程不会卡在 `while` 循环里自旋（spin），而是直接拿到锁。这个线程还会原子地

```
1  typedef struct __lock_t {
2      int flag;
3  } lock_t;
4
5  void init(lock_t *lock) {
6      // 0：锁可用，1：锁被持有
7      lock->flag = 0;
8  }
9
10 void lock(lock_t *lock) {
11     while (TestAndSet(&lock->flag, 1) == 1)
12         ; // 自旋等待（什么也不做）
13 }
14
15 void unlock(lock_t *lock) {
16     lock->flag = 0;
17 }
```

图 28.3：**用测试并设置实现的简单自旋锁**

把这个值设置为 1，表示锁现在被持有了。线程执行完临界区后，调用 `unlock()`，把标志重新设为零。

第二种情况是：已经有一个线程持有了锁，即 `flag` 为 1。这时，第二个线程调用 `lock()`，同样会调用 `TestAndSet(flag, 1)`。这一次，`TestAndSet()` 会返回 `flag` 处的旧值 1（因为锁被持有），同时又一次把它设为 1。只要锁还被另一个线程持有，`TestAndSet()` 就会一次次返回 1，于是这个线程会一直自旋，直到锁最终被释放。等另一个线程终于把标志设为 0，这个线程再调用 `TestAndSet()` 时，就会得到 0，同时原子地把值设为 1，从而获取锁，进入临界区。

把（对旧锁值的）**测试**和（对新值的）**设置**合成一个原子操作，我们就确保了只有一个线程能获取锁。一个能用的互斥原语就是这样实现的！

现在你也许明白，这类锁为什么通常叫作**自旋锁**（spin lock）了。它是最容易实现的一种锁，只是一直自旋、消耗 CPU 周期，直到锁可用为止。要在单处理器上正确工作，它需要一个**抢占式调度程序**（preemptive scheduler），也就是会时不时通过定时器中断一个线

提示：把并发想象成恶意的调度程序

从这个例子中，你也许能体会到理解并发执行（concurrent execution）所需的思路。你应该试着假装自己是一个**恶意的调度程序**（malicious scheduler），专挑最不合时宜的时刻中断线程，好让它们构建同步原语（synchronization primitive）的拙劣尝试统统落空。你真是个刻薄的调度程序（scheduler）！某种特定的中断顺序也许不太可能出现，但它有可能出现，而要证明某种方法行不通，这就够了。用恶意的眼光思考问题是有用的！（至少有时候是这样。）

 © 2008–25, ARPACI-DUSSEAU

```
1 int CompareAndSwap(int *ptr, int expected, int new) {
2     int original = *ptr;
3     if (original == expected)
4         *ptr = new;
5     return original;
6 }
```

图 28.4：**比较并交换**

程、转而运行另一个线程的调度程序。没有抢占（preemption），自旋锁在单 CPU 上就没多大意义，因为在 CPU 上自旋的线程永远不会让出（yield）CPU。

## 28.8 评价自旋锁

有了基本的自旋锁，就可以按前面说过的几个维度来评价它的效果了。锁最重要的方面是**正确性**（correctness）：它能不能提供互斥？答案是能：自旋锁每次只允许一个线程进入临界区。因此，我们得到了一个正确的锁。

下一个维度是**公平性**（fairness）。自旋锁对等待中的线程有多公平？你能保证等待的线程最终一定能进入临界区吗？很遗憾，答案并不乐观：自旋锁不提供任何公平性保证。事实上，在有争用的情况下，一个自旋的线程可能会永远自旋下去。简单的自旋锁是不公平的，可能导致饥饿（starvation）。

最后一个维度是**性能**。使用自旋锁的代价是什么？为了更仔细地分析这一点，我们建议考虑几种不同的情况。第一种，设想多个线程在单个处理器上争抢锁；第二种，设想线程分布在多个 CPU 上。

在单 CPU 的情况下，自旋锁的性能开销可能相当惨重。设想持有锁的线程在临界区内被抢占了。调度程序接下来可能会把其他线程挨个运行一遍（假设还有 $N-1$ 个），而每个线程都会试图获取锁。在这种情况下，这些线程在放弃 CPU 之前，都会把一整个时间片（time slice）耗在自旋上，白白浪费 CPU 周期。

然而，在多个 CPU 上，只要线程数和 CPU 数大致相当，自旋锁的效果就还算不错。思路是这样的：设想线程 A 在 CPU 1 上，线程 B 在 CPU 2 上，两者都在争抢一个锁。如果线程 A（CPU 1）拿到了锁，然后线程 B 也来尝试，B 就会（在 CPU 2 上）自旋。不过，临界区多半很短，因此锁很快就会空出来，由线程 B 拿到。在这种情况下，自旋等待另一个处理器上的线程所持有的锁，不会浪费多少周期，因此可能是有效的。

## 28.9 比较并交换

有些系统还提供另一种硬件原语，叫作**比较并交换**（compare-and-swap，CAS）指令（比如在 SPARC 上就叫这个名字），在 x86 上则叫 **compare-and-exchange**。这条指令的 C 伪代码（pseudocode）见图 28.4。

比较并交换（compare-and-swap）的基本思路是：测试 `ptr` 所指地址上的值是否等于 `expected`；如果相等，就把 `ptr` 指向的内存位置更新为新值；如果不相等，什么也不做。不

管哪种情况，都返回这个内存位置上原来的值，这样调用比较并交换（compare-and-swap）的代码就能知道操作是否成功。

有了比较并交换（compare-and-swap）指令，我们可以用和测试并设置（test-and-set）非常相似的方式实现锁。例如，只要把上面的 `lock()` 例程换成下面这样就行：

```
1 void lock(lock_t *lock) {
2     while (CompareAndSwap(&lock->flag, 0, 1) == 1)
3         ; // 自旋
4 }
```

其余代码和上面测试并设置（test-and-set）的例子相同。这段代码的工作方式也很相似：它只是检查标志是否为 0，如果是，就原子地把 1 换入（swap in），从而获取锁。在锁被持有期间试图获取锁的线程，会一直卡在那里自旋，直到锁最终被释放。

想看看怎样真正写出一个能从 C 中调用的 x86 版比较并交换（compare-and-swap），可以参考这段代码（出自 [S05]）[2]。

最后，你可能已经感觉到了，比较并交换（compare-and-swap）是比测试并设置（test-and-set）更强大的指令。以后我们简要探讨**无锁同步**（lock-free synchronization）[H91] 这类话题时，会用到它的这种能力。不过，如果只是用它实现一个简单的自旋锁，它的行为和我们上面分析过的自旋锁完全一样。

## 28.10 链接加载和条件存储

有些平台提供一对配合使用的指令，帮助构建临界区。例如，在 MIPS 体系结构 [H93] 上，可以把**链接加载**（load-linked）和**条件存储**（store-conditional）指令（合称 LL/SC）配合起来，构建锁和其他并发结构（concurrent structure）。这两条指令的 C 伪代码见图 28.5。Alpha、PowerPC 和 ARM 也提供类似的指令 [W09]。

链接加载（load-linked）和普通的加载指令很像，只是从内存中取出一个值，放进寄存器。关键区别在于条件存储（store-conditional）：只有在此期间没有别的存储操作写过这个地址，它才会成功（并更新刚才链接加载（load-linked）的那个地址上的值）。如果成功，条件存储（store-conditional）返回 1，并把 `ptr` 处的值更新为 `value`；如果失败，`ptr` 处的值不会被更新，并返回 0。

给自己一个挑战：想一想怎样用链接加载（load-linked）和条件存储（store-conditional）来实现锁。想好之后，再看下面的代码，它给出了一种简单的解法。动手试试吧！答案在图 28.6 中。

`lock()` 的代码是唯一有意思的部分。首先，线程自旋等待标志变成 0（表示锁没有被持有）。标志一变成 0，线程就尝试通过条件存储（store-conditional）获取锁；如果成功，线程就原子地把标志的值改成了 1，于是可以进入临界区。

注意条件存储（store-conditional）是怎样失败的。一个线程调用 `lock()`，执行链接加载（load-linked），由于锁没有被持有，得到的是 0。它还没来得及尝试条件存储（store-conditional）就被中断了，另一个线程进入加锁代码，同样执行链接加载（load-linked），也得到 0，并继续往下走。这时，两个线程都执行完了链接加载（load-linked），都要尝试条

[2] github.com/remzi-arpacidusseau/ostep-code/tree/master/threads-locks

```
int LoadLinked(int *ptr) {
    return *ptr;
}

int StoreConditional(int *ptr, int value) {
    if (no update to *ptr since LL to this addr) {
        *ptr = value;
        return 1; // 成功!
    } else {
        return 0; // 更新失败
    }
}
```

图 28.5：**链接加载和条件存储**

件存储（store-conditional）。这两条指令的关键特性在于，只有其中一个线程能成功把标志更新为 1，从而获取锁；第二个尝试条件存储（store-conditional）的线程会失败，因为另一个线程在它的链接加载（load-linked）和条件存储之间更新了 `flag` 的值，它只好再去尝试获取锁。

几年前在课堂上，本科生 David Capel 为喜欢布尔条件短路求值的读者，给上面的代码想出了一种更简洁的写法。看看你能不能想明白它为什么是等价的。它确实更短！

```
void lock(lock_t *lock) {
    while (LoadLinked(&lock->flag) ||
           !StoreConditional(&lock->flag, 1))
        ; // 自旋
}
```

```
void lock(lock_t *lock) {
    while (1) {
        while (LoadLinked(&lock->flag) == 1)
            ; // 自旋，直到它变成零
        if (StoreConditional(&lock->flag, 1) == 1)
            return; // 如果成功设为 1：完成
                    // 否则：再试一次
    }
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

图 28.6：**用 LL/SC 实现锁**

提示：代码越少越好（Lauer 定律）
程序员往往爱吹嘘自己为了做某件事写了多少代码。这从根本上就搞错了。真正值得吹嘘的，是完成一项任务只用了多么少的代码。简短、精炼的代码总是更受欢迎：它多半更容易理解，bug 也更少。正如 Hugh Lauer 在谈到构建 Pilot 操作系统时所说："如果让同一批人拥有两倍的时间，他们只用一半的代码就能做出同样好的系统。"[L81] 我们把这称为 **Lauer 定律**（Lauer's Law），它非常值得记住。所以，下次你吹嘘自己为了完成作业写了多少代码时，再想一想，或者更好的做法是：回去重写，让代码尽可能清晰、简洁。

## 28.11 获取并增加

最后一种硬件原语是**获取并增加**（fetch-and-add，FAA）指令，它原子地把某个地址上的值加一，同时返回这个地址上的旧值。获取并增加（fetch-and-add）指令的 C 伪代码是这样的：

```
1 int FetchAndAdd(int *ptr) {
2     int old = *ptr;
3     *ptr = old + 1;
4     return old;
5 }
```

在这个例子中，我们要用获取并增加（fetch-and-add）实现一种更有意思的**票锁**（ticket lock，也叫排队自旋锁），它由 Mellor-Crummey 和 Scott 提出 [MS91]。加锁和解锁的代码见图 28.7（第 286 页）。

这个方案不再只用一个值，而是把 ticket 和 turn 两个变量结合起来构建锁。基本操作很简单：线程想获取锁时，先对 ticket 的值做一次原子的获取并增加（fetch-and-add）；得到的值就当作这个线程的"轮次"（`myturn`）。然后用全局共享的 `lock->turn` 判断轮到了哪个线程；当某个线程的 `(myturn == turn)` 成立时，就轮到这个线程进入临界区了。解锁只需把 turn 加一，这样下一个等待的线程就可以进入临界区了。

注意，这个方案和之前的尝试有一个重要的不同：它保证所有线程都能向前推进。一个线程一旦分到了自己的 ticket 值，将来就一定会被调度：等排在它前面的线程都通过了临界区、释放了锁，就轮到它了。之前的尝试都没有这样的保证；例如，一个在测试并设置（test-and-set）上自旋的线程，可能在其他线程不断获取、释放锁的同时，永远自旋下去。

## 28.12 自旋太多：怎么办？

我们这些基于硬件的锁很简单（只有几行代码），而且管用（你愿意的话，甚至可以写点代码来证明这一点），对任何系统或代码来说，这都是两个极好的特性。然而，在某些情况下，这些方案可能相当低效。设想你在单个处理器上运行两个线程。再设想其中一个线程正处在临界区中，因而持有锁，而它不巧被中断了。第二个线程现在试图获取锁，却发现锁被持有。于是，它开始自旋。一直自旋。然后接着自旋。最后，时钟中断（timer interrupt）终于来了，线程 0 重新运行并释放了锁；等到（比如说）线程 1 下次运行时，它就不用再

```
1  typedef struct __lock_t {
2      int ticket;
3      int turn;
4  } lock_t;
5
6  void lock_init(lock_t *lock) {
7      lock->ticket = 0;
8      lock->turn   = 0;
9  }
10
11 void lock(lock_t *lock) {
12     int myturn = FetchAndAdd(&lock->ticket);
13     while (lock->turn != myturn)
14         ; // 自旋
15 }
16
17 void unlock(lock_t *lock) {
18     lock->turn = lock->turn + 1;
19 }
```

图 28.7：**票锁**

自旋那么久，能拿到锁了。因此，每当一个线程在这种情况下卡住自旋，它就浪费了一整个时间片，什么也没干，只是在反复检查一个根本不会变的值！如果有 $N$ 个线程争抢一个锁，问题会更严重：可能会以类似的方式浪费掉 $N-1$ 个时间片，这些线程都只是在自旋，等着某一个线程释放锁。于是，我们的下一个问题是：

关键问题：如何避免自旋
怎样才能实现一个不会白白在 CPU 上自旋、浪费时间的锁?

单靠硬件支持解决不了这个问题。我们还需要操作系统的支持！现在就来看看具体可以怎样做。

## 28.13 简单的办法：让出来就行，宝贝

硬件支持已经帮我们解决了不少问题：锁能用了，甚至像票锁（ticket lock）那样，做到了获取锁时的公平。然而，我们还有一个问题：如果在临界区中发生了上下文切换，其他线程开始无休止地自旋，等着那个被中断的（持有锁的）线程重新运行，该怎么办?

我们的第一个尝试是一种简单而友好的方法：当你要自旋时，干脆把 CPU 让给另一个线程。正如 Al Davis 可能会说的：“让出来就行，宝贝！”[D91]。图 28.8（第 287 页）展示了这种方法。

在这种方法中，我们假设有一个操作系统原语 `yield()`，线程想放弃 CPU、让另一个线程运行时，就可以调用它。一个线程可以处于三种状态之一：**运行**（running）、**就绪**

```
1  void init() {
2      flag = 0;
3  }
4
5  void lock() {
6      while (TestAndSet(&flag, 1) == 1)
7          yield(); // 让出 CPU
8  }
9
10 void unlock() {
11     flag = 0;
12 }
```

图 28.8：**使用测试并设置和让出的锁**

（ready）或阻塞（blocked）。`yield` 只是一个系统调用（system call），它把调用者从运行状态移到就绪状态，从而让另一个线程进入运行状态。因此，让出的线程实际上是**取消了自己的调度**（deschedule）。

想想两个线程在一个 CPU 上的例子；在这种情况下，我们这种基于让出的方法效果相当好。如果一个线程碰巧调用了 `lock()`，发现锁被持有，它就会让出 CPU，于是另一个线程就会运行，并执行完它的临界区。在这个简单的情况下，让出的方法很管用。

现在来考虑有很多线程（比如 100 个）反复争抢一个锁的情况。这时，如果一个线程获取了锁，又在释放之前被抢占，其他 99 个线程都会调用 `lock()`，发现锁被持有，然后让出 CPU。假设采用某种时间片轮转（round-robin）调度程序，那么在持有锁的线程再次运行之前，这 99 个线程每个都要把这种“运行—让出”的模式走一遍。自旋的方法会白白浪费 99 个时间片，这种方法比它要好，但代价仍然很高：上下文切换的开销可能相当大，因此浪费仍然不少。

更糟糕的是，这种方法没有解决饥饿问题。一个线程可能陷入无休止的让出循环，其他线程却一次次进出临界区。显然，我们需要一种能直接解决饥饿问题的方法。

## 28.14 使用队列：用睡眠代替自旋

除了票锁（ticket lock），前面那些方法真正的问题在于，它们把太多事情交给了运气。下一个运行哪个线程由调度程序决定；如果调度程序选得不好，运行的线程要么得自旋等待锁（我们的第一种方法），要么得立刻让出 CPU（我们的第二种方法）。不管哪种方式，都有可能造成浪费，而且都无法防止饥饿。

因此，当前持有者释放锁之后由哪个线程接着获取锁，我们必须明确地加以控制。为此，我们需要再多一点操作系统支持，还需要一个队列，记录哪些线程在等待获取锁。

为简单起见，我们使用 Solaris 提供的支持，它包括两个调用：`park()` 让调用它的线程进入睡眠，`unpark(threadID)` 唤醒由 `threadID` 指定的那个线程。把这两个例程配合起来，可以构建这样一种锁：调用者想获取一个已被持有的锁时，就让它睡眠；等锁空闲了再把它唤醒。我们来看图 28.9 中的代码，了解这类原语的一种可能用法。

 © 2008–25, ARPACI-DUSSEAU

```
1  typedef struct __lock_t {
2      int flag;
3      int guard;
4      queue_t *q;
5  } lock_t;
6
7  void lock_init(lock_t *m) {
8      m->flag  = 0;
9      m->guard = 0;
10     queue_init(m->q);
11 }
12
13 void lock(lock_t *m) {
14     while (TestAndSet(&m->guard, 1) == 1)
15         ; // 通过自旋获取 guard 锁
16     if (m->flag == 0) {
17         m->flag = 1; // 获取了锁
18         m->guard = 0;
19     } else {
20         queue_add(m->q, gettid());
21         m->guard = 0;
22         park();
23     }
24 }
25
26 void unlock(lock_t *m) {
27     while (TestAndSet(&m->guard, 1) == 1)
28         ; // 通过自旋获取 guard 锁
29     if (queue_empty(m->q))
30         m->flag = 0; // 放开锁；没人要它
31     else
32         unpark(queue_remove(m->q)); // 继续持有锁
33                                     //  （留给下一个线程!）
34     m->guard = 0;
35 }
```

图 28.9：**使用队列、测试并设置、让出和唤醒的锁**

我们在这个例子中做了几件有意思的事。第一，我们把测试并设置（test-and-set）这个老办法和一个显式的锁等待者队列结合起来，构造了一个更高效的锁。第二，我们用这个队列来帮助控制由谁下一个获得锁，从而避免饥饿。

你可能注意到了 guard 的用法（图 28.9，第 288 页）：它基本上就是一个自旋锁，保护着这个锁内部对标志和队列的操作。因此，这种方法并没有完全避免自旋等待：一个线程可能在获取或释放锁时被中断，导致其他线程自旋等待它再次运行。不过，自旋所花的时间相当有限：只是加锁和解锁代码中的几条指令，而不是用户定义的临界区。因此，这种方法也许是合理的。

补充：避免自旋的又一个理由：优先级反转

避免使用自旋锁的一个很好的理由是性能：正如正文所说，如果一个线程在持有锁时被中断，其他使用自旋锁的线程就会花大量 CPU 时间，仅仅是在等锁变得可用。不过，在某些系统上，避免使用自旋锁还有另一个有意思的理由：正确性。需要提防的问题叫作**优先级反转**（priority inversion）。可惜，这是一场星际灾难，在地球 [M15] 和火星 [R97] 上都发生过！

假设系统中有两个线程。线程 2（T2）的调度优先级（priority）高，线程 1（T1）的优先级较低。在这个例子中，假设只要两个线程都可以运行，CPU 调度程序就总是先运行 T2；只有当 T2 无法运行时（比如 T2 因等待 I/O 而阻塞时），T1 才会运行。

问题来了。假设 T2 因为某种原因被阻塞了。于是 T1 运行，获取了一个自旋锁，进入临界区。这时 T2 不再阻塞（也许是因为 I/O 完成了），CPU 调度程序立刻调度它（于是 T1 被换下）。T2 现在试图获取锁，但因为拿不到（T1 持有锁），它就一直自旋。由于这个锁是自旋锁，T2 会永远自旋下去，系统就挂住了。

可惜，仅仅不用自旋锁，并不能避免反转问题（唉）。设想有三个线程：T1、T2 和 T3，T3 的优先级最高，T1 的最低。现在设想 T1 获取了一个锁。接着 T3 启动了，由于它的优先级比 T1 高，立刻就抢占 T1 运行起来。T3 试图获取 T1 持有的锁，但因为 T1 仍然持有它，T3 只能卡在那里等待。这时如果 T2 启动，由于它的优先级比 T1 高，运行的就会是它。T3 的优先级比 T2 高，却卡在那里等 T1；而只要 T2 在运行，T1 就可能永远也运行不了。强大的 T3 无法运行，卑微的 T2 却霸占着 CPU，这难道不让人难过吗？高优先级早就不像过去那样吃香了。

解决优先级反转（priority inversion）问题有多种办法。在由自旋锁引起问题的特定情况下，可以避免使用自旋锁（下面会进一步介绍）。更一般地，高优先级线程在等待低优先级线程时，可以临时提升后者的优先级，让它得以运行，从而消除反转，这种技术叫作**优先级继承**（priority inheritance）。最后一种办法最简单：让所有线程的优先级都相同。

你可能还注意到，在 `lock()` 中，线程因锁已被持有而拿不到锁时，我们会小心地把自己加入队列（调用 `gettid()` 函数获取当前线程的线程 ID），把 guard 设为 0，然后让出 CPU。留给读者一个问题：如果把释放 guard 锁（guard lock）的操作放在 `park()` 之后而不是之前，会发生什么？提示：会出问题。

你可能还会发现，另一个线程被唤醒时，标志并没有被设回 0。这是为什么？其实这不是错误，而是必须这么做！线程被唤醒时，就像是从 `park()` 返回一样；但这时它在代码中并不持有 guard，因此根本没法尝试把标志设为 1。所以，我们直接把锁从释放它的线程交给下一个获取它的线程；在这中间，`flag` 不会被设为 0。

最后，你可能注意到了，这个方案在调用 `park()` 之前似乎存在一个竞态条件（race condition，也叫竞争条件）。如果时机恰好不对，一个线程正准备 park，以为自己应该一直睡到锁不再被持有为止。这时如果切换到另一个线程，比如持有锁的线程，就可能出问题，例如那个线程紧接着释放了锁。这样一来，第一个线程随后执行的 park 就（可能）永远睡下去了，这个问题有时叫作 **wakeup/waiting race**。

Solaris 增加了第三个系统调用 `setpark()` 来解决这个问题。线程调用这个例程，表示自己马上要 park 了。如果它随后碰巧被中断，而另一个线程在它真正调用 park 之前调用了 unpark，那么随后的 park 就会立即返回，不会睡眠。`lock()` 内部的代码修改很小：

```
1 queue_add(m->q, gettid());
2 setpark(); // 新代码
3 m->guard = 0;
```

另一种解决方案是把 guard 传进内核。这样，内核就可以采取预防措施，原子地释放锁，并让正在运行的线程出队。

## 28.15 不同的操作系统，不同的支持

到目前为止，我们看到了一种操作系统支持，它能帮助线程库（thread library）构建更高效的锁。其他操作系统也提供类似的支持，只是细节各不相同。

例如，Linux 提供了 **futex**，它和 Solaris 的接口相似，但在内核里实现了更多功能。具体来说，每个 futex 都关联着一个特定的物理内存（physical memory）位置，以及一个内核队列。调用者可以根据需要，使用 futex 调用（下面会介绍）来睡眠和唤醒。

可用的调用有两个。调用 `futex_wait(address, expected)` 时，如果地址 `address` 处的值等于 `expected`，调用线程就会进入睡眠。如果不相等，调用会立即返回。调用例程 `futex_wake(address)` 会唤醒一个在队列上等待的线程。Linux 互斥锁中这些调用的用法见图 28.10（第 291 页）。

这段代码出自 nptl 库（gnu libc 库的一部分）中的 `lowlevellock.h` [L09]，有几处值得一说。第一，它用一个整数同时记录锁是否被持有（整数的最高位）和锁上等待者的数量（其余所有位）。因此，如果锁的值是负数，锁就是被持有的（因为最高位被置位了，而这一位决定整数的正负）。

第二，这段代码展示了如何针对常见情况（common case）做优化，这里的常见情况就是锁没有争用：只有一个线程获取和释放锁时，要做的工作非常少，加锁时做一次原子的位测试并设置（test-and-set），解锁时做一次原子加法。

看看你能不能把这个“真实世界”中的锁剩下的部分琢磨透，弄明白它是怎么工作的。去做吧，成为 Linux 加锁的大师，或者至少成为一个书上叫你做什么就做什么的人[3]。

## 28.16 两阶段锁

最后再说一点：Linux 的方法带有一种老方法的影子。这种方法多年来时断时续地有人在用，至少可以追溯到 20 世纪 60 年代初的 Dahm 锁（Dahm lock）[M82]，现在叫作**两阶段锁**（two-phase lock）。两阶段锁（two-phase lock）的出发点是：自旋可能是有用的，尤其是在锁即将被释放的时候。所以在第一阶段，锁会自旋一段时间，希望能拿到锁。

[3]比如买一本 OSTEP 的纸质书！虽然这本书在网上确实可以免费阅读，难道你不想在桌上放一本精装本吗？或者更好的是，买上十本，分给亲朋好友？也许再多买一本，用来砸敌人？（这本书很重，所以扔出去出奇地有效。）

```
1  void mutex_lock (int *mutex) {
2    int v;
3    // 第 31 位原来是 0，我们拿到了互斥锁（快速路径）
4    if (atomic_bit_test_set (mutex, 31) == 0)
5      return;
6    atomic_increment (mutex);
7    while (1) {
8        if (atomic_bit_test_set (mutex, 31) == 0) {
9            atomic_decrement (mutex);
10           return;
11       }
12       // 必须等待。首先确保我们监视的 futex 值
13       // 是负的（已加锁）。
14       v = *mutex;
15       if (v >= 0)
16         continue;
17       futex_wait (mutex, v);
18   }
19 }
20
21 void mutex_unlock (int *mutex) {
22   // 当且仅当没有其他关心这个锁的线程时，
23   // 给计数器加上 0x80000000 的结果才是 0
24   if (atomic_add_zero (mutex, 0x80000000))
25     return;
26
27   // 还有其他线程在等待这个互斥锁，
28   // 唤醒其中一个。
29   futex_wake (mutex);
30 }
```

图 28.10：**基于 Linux futex 的锁**

然而，如果第一阶段的自旋没有拿到锁，就进入第二阶段：调用者去睡眠，等锁空闲了才被唤醒。上面的 Linux 锁就属于这种锁，只不过它只自旋一次；推而广之，可以在循环中自旋一段固定的时间，再借助 **futex** 支持进入睡眠。

两阶段锁（two-phase lock）又是一种**混合**（hybrid）方法：两个好主意结合起来，确实可能得到更好的主意。当然，能否如此，很大程度上取决于硬件环境、线程数和工作负载（workload）的其他细节等许多因素。和往常一样，要做出一把适合所有用例的通用锁，相当有挑战性。

 © 2008–25, ARPACI-DUSSEAU

## 28.17 小结

以上就是如今真实的锁的构建方式：一些硬件支持（更强大的指令），加上一些操作系统支持（例如 Solaris 上的 `park()` 和 `unpark()` 原语，或者 Linux 上的 **futex**）。当然，细节各不相同，实现这类加锁的具体代码通常也都经过了高度调优。想了解更多细节，可以看看 Solaris 或 Linux 的代码库，它们读起来很有意思 [L09, S09]。David 等人还有一项出色的工作，比较了现代多处理器上的各种加锁策略 [D+13]。

 © 2008–25, ARPACI-DUSSEAU

## 参考文献

[D91] "Just Win, Baby: Al Davis and His Raiders" by Glenn Dickey. Harcourt, 1991. *The book about Al Davis and his famous quote. Or, we suppose, the book is more about Al Davis and the Raiders, and not so much the quote. To be clear: we are not recommending this book, we just needed a citation.*

[D+13] "Everything You Always Wanted to Know about Synchronization but Were Afraid to Ask" by Tudor David, Rachid Guerraoui, Vasileios Trigonakis. SOSP '13, Nemacolin Woodlands Resort, Pennsylvania, November 2013. *An excellent paper comparing many different ways to build locks using hardware primitives. Great to see how many ideas work on modern hardware.*

[D68] "Cooperating sequential processes" by Edsger W. Dijkstra. 1968. Available online here: http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD123.PDF. *One of the early seminal papers. Discusses how Dijkstra posed the original concurrency problem, and Dekker's solution.*

[H93] "MIPS R4000 Microprocessor User's Manual" by Joe Heinrich. Prentice-Hall, June 1993. Available: http://cag.csail.mit.edu/raw/documents/R4400_Uman_book_Ed2.pdf. *The old MIPS user's manual. Download it while it still exists.*

[H91] "Wait-free Synchronization" by Maurice Herlihy. ACM TOPLAS, Volume 13: 1, January 1991. *A landmark paper introducing a different approach to building concurrent data structures. Because of the complexity involved, some of these ideas have been slow to gain acceptance in deployment.*

[L81] "Observations on the Development of an Operating System" by Hugh Lauer. SOSP '81, Pacific Grove, California, December 1981. *A must-read retrospective about the development of the Pilot OS, an early PC operating system. Fun and full of insights.*

[L09] "glibc 2.9 (include Linux pthreads implementation)" by Many authors.. Available here: http://ftp.gnu.org/gnu/glibc. *In particular, take a look at the nptl subdirectory where you will find most of the pthread support in Linux today.*

[M82] "The Architecture of the Burroughs B5000: 20 Years Later and Still Ahead of the Times?" by A. Mayer. 1982. Available: www.ajwm.net/amayer/papers/B5000.html. *"It (RDLK) is an indivisible operation which reads from and writes into a memory location." RDLK is thus test-and-set! Dave Dahm created spin locks ("Buzz Locks") and a two-phase lock called "Dahm Locks."*

[M15] "OSSpinLock Is Unsafe" by J. McCall. mjtsai.com/blog/2015/12/16/osspinlock-is-unsafe. *Calling OSSpinLock on a Mac is unsafe when using threads of different priorities – you might spin forever! So be careful, Mac fanatics, even your mighty system can be less than perfect…*

[MS91] "Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors" by John M. Mellor-Crummey and M. L. Scott. ACM TOCS, Volume 9, Issue 1, February 1991. *An excellent and thorough survey on different locking algorithms. However, no operating systems support is used, just fancy hardware instructions.*

[P81] "Myths About the Mutual Exclusion Problem" by G.L. Peterson. Information Processing Letters, 12(3), pages 115–116, 1981. *Peterson's algorithm introduced here.*

[R97] "What Really Happened on Mars?" by Glenn E. Reeves. Available on our site at: https://pages.cs.wisc.edu/%7Eremzi/OSTEP/Citations/mars.html. *A description of priority inversion on Mars Pathfinder. Concurrent code correctness matters, especially in space!*

[S05] "Guide to porting from Solaris to Linux on x86" by Ajay Sood, April 29, 2005. Available: http://www.ibm.com/developerworks/linux/library/l-solar/.

[S09] "OpenSolaris Thread Library" by Sun.. Code: src.opensolaris.org/source/xref/onnv/onnv-gate/usr/src/lib/libc/port/threads/synch.c. *Pretty interesting, although who knows what will happen now that Oracle owns Sun. Thanks to Mike Swift for the pointer.*

[W09] "Load-Link, Store-Conditional" by Many authors.. en.wikipedia.org/wiki/Load-Link/Store-Conditional. *Can you believe we referenced Wikipedia? But, we found the information there and it felt wrong not to. Further, it was useful, listing the instructions for the different architectures:* `ldl_l`/`stl_c` *and* `ldq_l`/`stq_c` *(Alpha),* `lwarx`/`stwcx` *(PowerPC),* `ll`/`sc` *(MIPS), and* `ldrex`/`strex` *(ARM). Actually Wikipedia is pretty amazing, so don't be so harsh, OK?*

[WG00] "The SPARC Architecture Manual: Version 9" by D. Weaver, T. Germond. SPARC International, 2000. http://www.sparc.org/standards/SPARCV9.pdf. See the website: developers.sun.com/solaris/articles/atomic_sparc/ for more on atomics.

## 作业（模拟）

程序 x86.py 可以让你看到，线程的各种交错执行方式是怎样导致或避免竞态条件的。请先阅读 README，了解程序如何工作，然后回答下面的问题。

## 问题

1. 看看 flag.s。这段代码用一个内存标志“实现”了加锁。你能看懂这段汇编吗？
2. 用默认设置运行时，flag.s 能正常工作吗？用 -M 和 -R 选项跟踪变量和寄存器（并打开 -c 查看它们的值）。你能预测 flag 最终会是什么值吗？
3. 用 -a 选项修改寄存器 %bx 的值；例如，如果只运行两个线程，就用 -a bx=2,bx=2。这段代码做了什么？它会怎样改变你对上一题的回答？
4. 把每个线程的 bx 都设成一个较大的值，然后用 -i 选项产生不同的中断频率。哪些值会导致糟糕的结果？哪些会带来好的结果？
5. 现在来看程序 test-and-set.s。首先，试着理解这段代码，它用 xchg 指令构建了一个简单的加锁原语。获取锁是怎么写的？释放锁呢？
6. 现在运行这段代码，再次改变中断间隔（-i）的值，并确保循环若干次。代码总能按预期工作吗？它有时会让 CPU 的使用效率变低吗？你怎样量化这一点？
7. 用 -P 选项生成针对加锁代码的特定测试。例如，运行这样一个调度：在第一个线程中获取锁，然后在第二个线程中尝试获取它。结果对吗？还应该测试什么？
8. 现在来看 peterson.s 中的代码，它实现了正文补充框中提到过的 Peterson 算法（Peterson’s algorithm）。研究这段代码，看看你能不能弄懂它。
9. 现在用不同的 -i 值运行这段代码。你看到了哪些不同的行为？由于代码依赖线程 ID（thread ID），一定要正确设置它们（例如用 -a bx=0,bx=1）。
10. 你能控制调度（用 -P 选项）来“证明”这段代码是正确的吗？你应该证明哪些情况成立？想想互斥和避免死锁（deadlock）。
11. 现在研究 ticket.s 中票锁（ticket lock）的代码。它和本章中的代码一致吗？然后用以下选项运行：-a bx=1000,bx=1000，让每个线程把临界区循环执行 1000 次。观察会发生什么：线程花在自旋等待锁上的时间多吗？
12. 增加线程数时，代码的行为会怎样？
13. 现在看看 yield.s，其中一条 yield 指令让一个线程交出对 CPU 的控制（现实中这应该是一个操作系统原语，但为简单起见，我们假设由一条指令来完成这个任务）。找出一种场景，在这种场景下 test-and-set.s 会把周期浪费在自旋上，而 yield.s 不会。节省了多少条指令？这种节省会出现在哪些场景中？
14. 最后，看看 test-and-test-and-set.s。这个锁做了什么？与 test-and-set.s 相比，它带来了哪些节省？

 © 2008–25, ARPACI-DUSSEAU

# 29

# 基于锁的并发数据结构

在讨论锁（lock）之外的话题之前，我们先介绍如何在一些常见的数据结构（data structure）中使用锁。给数据结构加上锁，让多个线程（thread）都能使用它，这个数据结构就成了**线程安全**（thread safe）的。当然，锁具体怎么加，既决定了数据结构是否正确，也决定了它的性能。于是，我们面临的挑战是：

关键问题：如何给数据结构加锁
面对一个具体的数据结构，应该怎样给它加锁（locking），才能让它正确工作？进一步说，怎样加锁才能让这个数据结构有较高的性能，允许许多线程同时访问它，也就是**并发**（concurrently）访问？

当然，我们不可能涵盖所有数据结构，也不可能涵盖所有增加并发的方法，因为这个课题已经研究了很多年，发表的研究论文（毫不夸张地说）数以千计。所以，我们只打算介绍所需的思考方式，够你入门即可，再给你推荐一些不错的资料，供你自己深入研究。我们发现，Moir 和 Shavit 的综述是很好的信息来源 [MS04]。

## 29.1 并发计数器

计数器是最简单的数据结构之一。它很常用，接口也很简单。图 29.1 定义了一个简单的非并发计数器（non-concurrent counter）。

### 简单但无法扩展

可以看到，这个没有同步的计数器是个再简单不过的数据结构，只要很少的代码就能实现。现在我们面临下一个挑战：怎样让这段代码变成**线程安全**（thread safe）的？图 29.2 展示了我们的做法。

这个并发计数器（concurrent counter）很简单，而且能正确工作。实际上，它沿用了最简单、最基本的并发数据结构（concurrent data structure）常用的一种设计模式：只加一把锁，调用操作这个数据结构的例程时获取锁，从调用返回时释放锁。从这一点看，它和用**管程**（monitor）构建的数据结构 [BH73] 很像：在管程中，调用对象方法和从方法返回时，锁会自动获取和释放。

```
1  typedef struct __counter_t {
2      int value;
3  } counter_t;
4
5  void init(counter_t *c) {
6      c->value = 0;
7  }
8
9  void increment(counter_t *c) {
10     c->value++;
11 }
12
13 void decrement(counter_t *c) {
14     c->value--;
15 }
16
17 int get(counter_t *c) {
18     return c->value;
19 }
```

图 29.1：**不带锁的计数器**

到这里，你已经有了一个能用的并发数据结构。你可能会遇到的问题是性能。如果数据结构太慢，光加一把锁就不够了；需要时该怎样优化，正是本章剩下部分的内容。注意，如果数据结构不算太慢，那就大功告成了！简单的办法能行，就没必要去搞花哨的东西。

为了弄清这种简单方法的性能代价，我们运行了一个基准测试：每个线程把同一个共享计数器更新固定的次数，然后改变线程的数量。图 29.5 给出了 1 到 4 个线程运行时花费的总时间，每个线程都把计数器更新一百万次。这个实验是在一台装有 4 个 2.7 GHz Intel i5 CPU 的 iMac 上运行的；参与运行的 CPU 越多，我们希望单位时间内完成的总工作量也越多。

从图中上面那条线（标为“精确”）可以看出，这个同步计数器的扩展性很差。单个线程只要很短的时间（大约 0.03 秒）就能完成一百万次更新，而让两个线程各自并发地（concurrently）把计数器更新一百万次，速度就会大幅下降（要花 5 秒多！）。线程越多，情况越糟。

理想情况下，你会希望多个线程在多个处理器上完成工作的速度，和单个线程在一个处理器上一样快。达到这个目标叫作**完美扩展**（perfect scaling）：虽然完成的工作更多了，但它们是并行（in parallel）完成的，所以完成任务所花的时间并不会增加。

## 可扩展的计数

令人惊讶的是，研究人员多年来一直在研究如何构建扩展性更好的计数器 [MS04]。更令人惊讶的是，可扩展的计数器确实很重要，最近关于操作系统性能分析的研究就表明了这一点 [B+10]：如果没有可扩展的计数，一些运行在 Linux 上的工作负载（workload）在多核（multicore）机器上会遇到严重的扩展性问题。

```
1  typedef struct __counter_t {
2      int             value;
3      pthread_mutex_t lock;
4  } counter_t;
5
6  void init(counter_t *c) {
7      c->value = 0;
8      Pthread_mutex_init(&c->lock, NULL);
9  }
10
11 void increment(counter_t *c) {
12     Pthread_mutex_lock(&c->lock);
13     c->value++;
14     Pthread_mutex_unlock(&c->lock);
15 }
16
17 void decrement(counter_t *c) {
18     Pthread_mutex_lock(&c->lock);
19     c->value--;
20     Pthread_mutex_unlock(&c->lock);
21 }
22
23 int get(counter_t *c) {
24     Pthread_mutex_lock(&c->lock);
25     int rc = c->value;
26     Pthread_mutex_unlock(&c->lock);
27     return rc;
28 }
```

图 29.2：**带锁的计数器**

人们已经开发出许多技术来解决这个问题。我们介绍其中一种，叫作**近似计数器**（approximate counter）[C06]。

近似计数器（approximate counter）用许多局部的物理计数器（每个 CPU 核一个）加上一个全局计数器，来表示一个逻辑计数器。具体来说，在一台有 4 个 CPU 的机器上，有 4 个局部计数器和 1 个全局计数器。除了这些计数器，还有锁：每个局部计数器各有一把锁[1]，全局计数器也有一把。

近似计数的基本思路如下。运行在某个核上的线程想让计数器加一时，就把本核的局部计数器加一；访问这个局部计数器时，用对应的局部锁（local lock）同步。由于每个 CPU 都有自己的局部计数器，不同 CPU 上的线程可以互不竞争地更新各自的局部计数器，所以计数器的更新是可扩展的。

[1]之所以需要局部锁（local lock），是因为我们假设每个核上可能不止一个线程。反过来，如果每个核上只运行一个线程，就不需要它了。

| 时间 | $L_1$ | $L_2$ | $L_3$ | $L_4$ | $G$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 | 0 |
| 2 | 1 | 0 | 2 | 1 | 0 |
| 3 | 2 | 0 | 3 | 1 | 0 |
| 4 | 3 | 0 | 3 | 2 | 0 |
| 5 | 4 | 1 | 3 | 3 | 0 |
| 6 | $5 \rightarrow 0$ | 1 | 3 | 4 | 5（来自 $L_1$） |
| 7 | 0 | 2 | 4 | $5 \rightarrow 0$ | 10（来自 $L_4$） |

图 29.3：**跟踪近似计数器**

不过，为了让全局计数器保持最新，以防有线程要读它的值，局部计数器的值会定期转移到全局计数器上：先获取全局锁（global lock），把全局计数器加上局部计数器的值，然后把局部计数器清零。

这种从局部到全局的转移多久发生一次，由阈值 $S$ 决定。$S$ 越小，计数器的表现就越像前面那个不可扩展的计数器；$S$ 越大，计数器的扩展性越好，但全局计数器的值和实际计数的偏差也可能越大。当然，也可以获取所有局部锁（local lock）和全局锁（global lock），得到一个精确值；获取时要按规定的顺序，以免死锁（deadlock）。但这样做是不可扩展的。

为了说明清楚，我们来看一个例子（图 29.3）。在这个例子中，阈值 $S$ 设为 5，4 个 CPU 上各有线程在更新自己的局部计数器 $L_1 \cdots L_4$。跟踪记录里也给出了全局计数器的值（$G$），时间从上往下递增。每个时间步，某个局部计数器可能会加一；如果局部计数器的值达到阈值 $S$，就把它转移到全局计数器，并把局部计数器清零。

图 29.5（第 300 页）中下面那条线（标为“近似”）是阈值 $S$ 为 1024 时近似计数器（approximate counter）的性能。性能非常好：在 4 个处理器上把计数器更新四百万次所花的时间，几乎不比在 1 个处理器上更新一百万次多。

图 29.6 说明了阈值 $S$ 的重要性：4 个线程在 4 个 CPU 上各把计数器增加一百万次。如果 $S$ 很小，性能就很差（但全局计数总是相当准确）；如果 $S$ 很大，性能非常好，但全局计数会滞后（最多滞后 CPU 数乘以 $S$）。近似计数器（approximate counter）提供的，正是这种准确性和性能之间的权衡。

图 29.4（第 299 页）给出了近似计数器（approximate counter）的一个粗略实现。读一读；最好再亲手运行它，做几个实验，弄清楚它是怎么工作的。

## 29.2 并发链表

接下来我们研究一种更复杂的结构：链表（linked list）。我们还是先从一个基本的方法开始。为简单起见，我们省略这种链表显然该有的一些例程，只关注并发的（concurrent）插入和查找；删除等操作留给读者自己思考。图 29.7 是这个初级数据结构的代码。

从代码中可以看到，插入例程只是在进入时获取锁，退出时释放锁。如果 `malloc()` 碰巧失败（这种情况很少见），就会出现一个有点棘手的小问题：这时代码还必须在插入失败返回之前释放锁。

```
typedef struct __counter_t {
    int             global;          // 全局计数
    pthread_mutex_t glock;           // 全局锁
    int             local[NUMCPUS];  // 每个 CPU 的计数
    pthread_mutex_t llock[NUMCPUS];  // ……以及对应的锁
    int             threshold;       // 更新频率
} counter_t;

// init：记录阈值，初始化各个锁，
//       以及所有局部计数和全局计数的值
void init(counter_t *c, int threshold) {
    c->threshold = threshold;
    c->global = 0;
    pthread_mutex_init(&c->glock, NULL);
    int i;
    for (i = 0; i < NUMCPUS; i++) {
        c->local[i] = 0;
        pthread_mutex_init(&c->llock[i], NULL);
    }
}

// update：通常只需获取局部锁，更新局部计数；
// 局部计数一旦涨到 threshold，
// 就获取全局锁，把局部计数的值转移过去
void update(counter_t *c, int threadID, int amt) {
    int cpu = threadID % NUMCPUS;
    pthread_mutex_lock(&c->llock[cpu]);
    c->local[cpu] += amt;
    if (c->local[cpu] >= c->threshold) {
        // 转移到全局计数（假定 amt>0）
        pthread_mutex_lock(&c->glock);
        c->global += c->local[cpu];
        pthread_mutex_unlock(&c->glock);
        c->local[cpu] = 0;
    }
    pthread_mutex_unlock(&c->llock[cpu]);
}

// get：直接返回全局计数（近似值）
int get(counter_t *c) {
    pthread_mutex_lock(&c->glock);
    int val = c->global;
    pthread_mutex_unlock(&c->glock);
    return val; // 只是近似值!
}
```

图 29.4：**近似计数器的实现**

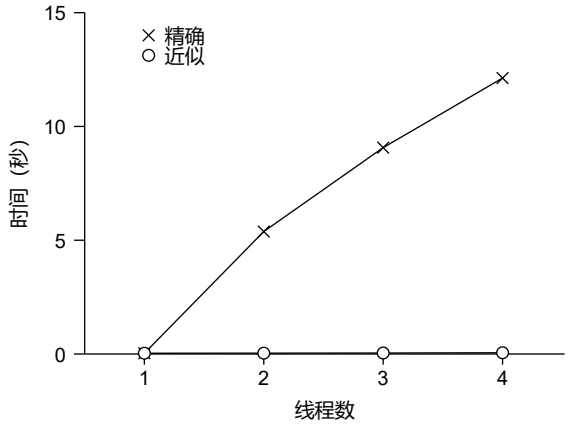



图 29.5：**传统计数器与近似计数器的性能对比**

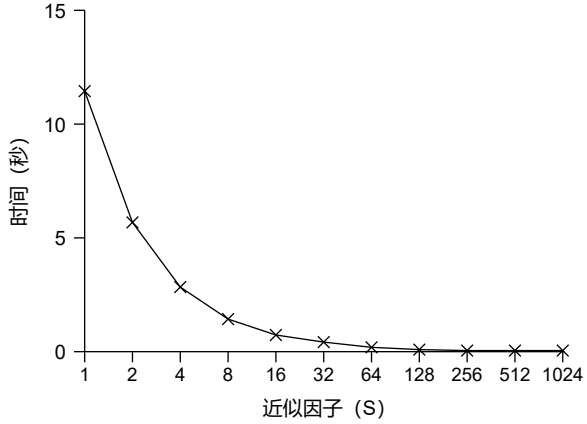



图 29.6：**近似计数器的扩展性**

提示：并发度更高不一定更快

如果你设计的方案增加了大量开销（比如频繁地获取和释放锁，而不是只做一次），那么它的并发度更高这一点可能就无关紧要了。简单的方案往往效果很好，尤其是很少调用代价高昂的例程时。加更多的锁、增加复杂度，反而可能让你栽跟头。话虽如此，要想真正知道答案，只有一个办法：把简单但并发度低的方案和复杂但并发度高的方案都实现出来，然后测一测它们的表现。说到底，性能是作不了弊的：你的想法要么更快，要么不快。

```
// 基本的节点结构
typedef struct __node_t {
    int              key;
    struct __node_t  *next;
} node_t;

// 基本的链表结构（每个链表用一个）
typedef struct __list_t {
    node_t           *head;
    pthread_mutex_t  lock;
} list_t;

void List_Init(list_t *L) {
    L->head = NULL;
    pthread_mutex_init(&L->lock, NULL);
}

int List_Insert(list_t *L, int key) {
    pthread_mutex_lock(&L->lock);
    node_t *new = malloc(sizeof(node_t));
    if (new == NULL) {
        perror("malloc");
        pthread_mutex_unlock(&L->lock);
        return -1; // 失败
    }
    new->key  = key;
    new->next = L->head;
    L->head   = new;
    pthread_mutex_unlock(&L->lock);
    return 0; // 成功
}

int List_Lookup(list_t *L, int key) {
    pthread_mutex_lock(&L->lock);
    node_t *curr = L->head;
    while (curr) {
        if (curr->key == key) {
            pthread_mutex_unlock(&L->lock);
            return 0; // 成功
        }
        curr = curr->next;
    }
    pthread_mutex_unlock(&L->lock);
    return -1; // 失败
}
```

图 29.7：**并发链表**

实践表明，这种异常的控制流很容易出错：最近一项针对 Linux 内核补丁的研究发现，有很大一部分 bug（将近 40%）出现在这类很少执行的代码路径上。实际上，正是这个发现促成了我们自己的一些研究：我们从一个 Linux 文件系统中去掉了所有可能因内存分配失败而出错的路径，让系统更加健壮 [S+11]。

于是挑战来了：能不能重写插入和查找例程，让它们在并发插入（concurrent insert）时依然正确，同时又不必在失败路径上也加一次解锁（unlock）调用？

在这个例子里，答案是可以。具体来说，我们可以稍微调整一下代码：让插入代码中的加锁和解锁只包住真正的临界区（critical section），并让查找代码使用一条共同的退出路径。前一种改法之所以可行，是因为插入操作有一部分其实不需要加锁；假设 `malloc()` 本身是线程安全的，每个线程都可以放心地调用它，不必担心竞态条件（race condition，也叫竞争条件）或其他并发 bug（concurrency bug）。只有在更新共享的链表时才需要持有锁。这些修改的细节见图 29.8。

至于查找例程，只需做一个简单的代码变换：从主搜索循环跳出来，走到唯一的一条返回路径。这样做同样减少了代码中获取和释放锁的地方，从而降低了不小心引入 bug（比如返回前忘了解锁）的可能性。

### 提高链表的可扩展性

虽然我们又得到了一个基本的并发链表（concurrent linked list），但它的扩展性还是不太好。为了让链表内部支持更多的并发，研究人员探索过一种叫作**交替加锁**（hand-over-hand locking）的技术，也叫**锁耦合**（lock coupling）[MS04]。

思路很简单。不再给整个链表只配一把锁，而是给链表的每个节点各加一把锁。遍历链表时，代码先获取下一个节点的锁，再释放当前节点的锁（英文名 hand-over-hand 就是由此而来）。

从概念上讲，交替加锁（hand-over-hand locking）的链表有一定道理：它让链表操作具有很高的并发度。但在实践中，很难让这种结构比简单的单锁（single lock）方法更快，因为遍历链表时为每个节点获取和释放锁，开销大得让人难以接受。即使链表非常长、线程非常多，让多个遍历同时进行，这种并发换来的速度也不太可能超过直接获取一把锁、执行操作、再释放它的做法。也许某种混合方案（每隔若干个节点才获取一把新锁）值得研究一下。

## 29.3 并发队列

现在你已经知道，构造并发数据结构总有一种标准方法：加一把大锁（big lock）。队列（queue）的这种做法我们就跳过了，相信你自己能想出来。

我们来看一个并发度稍高一些的队列，它是由 Michael 和 Scott 设计的 [MS98]。这个队列用到的数据结构和代码见图 29.9（第 304 页）。

仔细研究这段代码，你会发现它有两把锁：一把用于队列的头部，一把用于尾部。这两把锁的目的是让入队和出队操作能够并发进行。通常情况下，入队例程只用到尾锁（tail lock），出队例程只用到头锁（head lock）。

 © 2008–23, ARPACI-DUSSEAU

```
1  void List_Init(list_t *L) {
2      L->head = NULL;
3      pthread_mutex_init(&L->lock, NULL);
4  }
5
6  int List_Insert(list_t *L, int key) {
7      // 这里不需要同步
8      node_t *new = malloc(sizeof(node_t));
9      if (new == NULL) {
10         perror("malloc");
11         return -1;
12     }
13     new->key = key;
14     // 只锁住临界区
15     pthread_mutex_lock(&L->lock);
16     new->next = L->head;
17     L->head   = new;
18     pthread_mutex_unlock(&L->lock);
19     return 0; // 成功
20 }
21
22 int List_Lookup(list_t *L, int key) {
23     int rv = -1;
24     pthread_mutex_lock(&L->lock);
25     node_t *curr = L->head;
26     while (curr) {
27         if (curr->key == key) {
28             rv = 0;
29             break;
30         }
31         curr = curr->next;
32     }
33     pthread_mutex_unlock(&L->lock);
34     return rv; // 现在成功和失败都走这里
35 }
```

图 29.8：**重写后的并发链表**

提示：小心锁和控制流

有一条通用的设计建议，在并发代码（concurrent code）和其他地方都适用：要警惕控制流的变化，比如函数返回、退出，或者其他让函数中途停止执行的出错情况。很多函数一开始会获取锁、分配内存，或者做其他类似的有状态（stateful）的操作，一旦出了问题，代码就得在返回前把这些状态全部撤销，而这一步很容易出错。因此，组织代码时最好尽量避免这种模式。

```
1  typedef struct __node_t {
2      int              value;
3      struct __node_t  *next;
4  } node_t;
5
6  typedef struct __queue_t {
7      node_t           *head;
8      node_t           *tail;
9      pthread_mutex_t  head_lock, tail_lock;
10 } queue_t;
11
12 void Queue_Init(queue_t *q) {
13     node_t *tmp = malloc(sizeof(node_t));
14     tmp->next = NULL;
15     q->head = q->tail = tmp;
16     pthread_mutex_init(&q->head_lock, NULL);
17     pthread_mutex_init(&q->tail_lock, NULL);
18 }
19
20 void Queue_Enqueue(queue_t *q, int value) {
21     node_t *tmp = malloc(sizeof(node_t));
22     assert(tmp != NULL);
23     tmp->value = value;
24     tmp->next  = NULL;
25
26     pthread_mutex_lock(&q->tail_lock);
27     q->tail->next = tmp;
28     q->tail = tmp;
29     pthread_mutex_unlock(&q->tail_lock);
30 }
31
32 int Queue_Dequeue(queue_t *q, int *value) {
33     pthread_mutex_lock(&q->head_lock);
34     node_t *tmp = q->head;
35     node_t *new_head = tmp->next;
36     if (new_head == NULL) {
37         pthread_mutex_unlock(&q->head_lock);
38         return -1; // 队列为空
39     }
40     *value = new_head->value;
41     q->head = new_head;
42     pthread_mutex_unlock(&q->head_lock);
43     free(tmp);
44     return 0;
45 }
```

图 29.9：**Michael 和 Scott 的并发队列**

Michael 和 Scott 用到的一个技巧是加入一个哑节点（dummy node，在队列初始化代码中分配）；有了它，头部操作和尾部操作就能分开进行。研究一下这段代码，最好是自己敲进去、运行并测量一下，深入理解它是怎么工作的。

队列在多线程（multi-threaded）应用程序中很常用。然而，像这里这样只用锁的队列，往往不能完全满足这类程序的需要。还有一种更完善的有界队列（bounded queue），能让线程在队列为空或过满时等待；下一章讲条件变量（condition variable）时，会深入研究它。敬请期待！

## 29.4 并发散列表

最后，我们来讨论一种简单又应用广泛的并发数据结构：散列表（hash table）。我们关注的是一个不会调整大小的简单散列表；处理调整大小还要多做一些工作，这就留给读者作为练习了（抱歉！）。

这个并发散列表（concurrent hash table，图 29.10）很直观，它是用前面开发的并发链表（concurrent linked list）构建的，效果出奇地好。性能好的原因在于：它没有给整个结构只配一把锁，而是给每个散列桶（hash bucket，每个桶用一个链表表示）各配一把锁。这样就能让许多并发操作（concurrent operation）同时进行。

图 29.11（第 306 页）展示了散列表在并发更新（concurrent update）下的性能（4 个线程各自并发更新 10,000 到 50,000 次，还是在那台有 4 个 CPU 的 iMac 上）。为了对比，图中也给出了单锁链表的性能。从图中可以看出，这个简单的并发散列表（concurrent hash table）扩展得非常好；相比之下，链表则不然。

```
1  #define BUCKETS (101)
2
3  typedef struct __hash_t {
4      list_t lists[BUCKETS];
5  } hash_t;
6
7  void Hash_Init(hash_t *H) {
8      int i;
9      for (i = 0; i < BUCKETS; i++)
10         List_Init(&H->lists[i]);
11 }
12
13 int Hash_Insert(hash_t *H, int key) {
14     return List_Insert(&H->lists[key % BUCKETS], key);
15 }
16
17 int Hash_Lookup(hash_t *H, int key) {
18     return List_Lookup(&H->lists[key % BUCKETS], key);
19 }
```

图 29.10：**并发散列表**

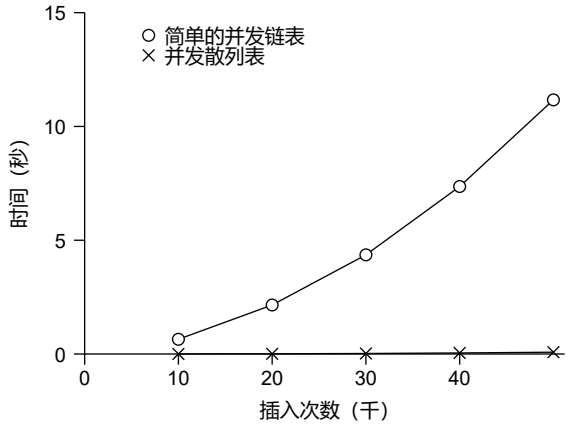



图 29.11：**散列表的扩展性**

## 29.5 小结

我们介绍了一些并发数据结构的例子，从计数器，到链表和队列，最后是无处不在、用得非常多的散列表。一路上我们学到了几条重要的经验：在控制流发生变化的地方，获取和释放锁要格外小心；提高并发度不一定能提高性能；性能问题只应在真正出现之后再去解决。最后这一点，也就是避免**过早优化**（premature optimization），对任何重视性能的开发者都至关重要：如果把某个部分做快了并不能提高应用程序的整体性能，那么这样做毫无价值。

当然，我们只是触及了高性能数据结构的皮毛。更多内容请参阅 Moir 和 Shavit 出色的综述，里面还有指向其他资料的链接 [MS04]。特别是，你可能会对其他结构感兴趣，比如 B 树（B-tree）；想学这方面的知识，最好去上一门数据库课。你可能还会好奇那些完全不用传统锁的技术；这类**非阻塞数据结构**（non-blocking data structures），我们会在“常见并发问题”一章中稍作介绍，但坦白说，这个主题本身就是一整个知识领域，要学的东西远远超出了这本小书所能容纳的范围。如果愿意，你可以自己去深入了解（一如既往!）。

 © 2008–23, ARPACI-DUSSEAU

提示：避免过早优化（Knuth 定律）

构建并发数据结构时，先从最基本的方法开始，也就是加一把大锁（big lock）来提供同步访问。这样，你很可能会构建出一把正确的锁；之后如果发现它有性能问题，再去改进它，也就是只在有必要时才让它变快。正如 **Knuth** 的名言所说："过早优化（premature optimization）是万恶之源。"

许多操作系统在最初过渡到多处理器（multiprocessor）时都只用了一把锁，Sun OS 和 Linux 都是如此。在 Linux 中，这把锁甚至有个名字，叫**大内核锁**（big kernel lock，**BKL**）。多年来，这种简单的方法一直很好用，但多 CPU 系统成为常态之后，同一时刻只允许内核中有一个活动线程，就成了性能瓶颈。于是，终于到了给这些系统加上"提高并发度"这项优化的时候。Linux 采用了比较直接的方法：把一把锁换成许多把锁。Sun 则做出了一个更激进的决定：构建一个全新的操作系统 Solaris，从第一天起就在更根本的层面上融入并发。想进一步了解这些引人入胜的系统，请阅读 Linux 和 Solaris 的内核书籍 [BC05, MM00]。

## 参考文献

[B+10] "An Analysis of Linux Scalability to Many Cores" by Silas Boyd-Wickizer, Austin T. Clements, Yandong Mao, Aleksey Pesterev, M. Frans Kaashoek, Robert Morris, Nickolai Zeldovich. OSDI '10, Vancouver, Canada, October 2010. *A great study of how Linux performs on multicore machines, as well as some simple solutions. Includes a neat* **sloppy counter** *to solve one form of the scalable counting problem.*

[BH73] "Operating System Principles" by Per Brinch Hansen. Prentice-Hall, 1973. Available: http://portal.acm.org/citation.cfm?id=540365. *One of the first books on operating systems; certainly ahead of its time. Introduced monitors as a concurrency primitive.*

[BC05] "Understanding the Linux Kernel (Third Edition)" by Daniel P. Bovet and Marco Cesati. O'Reilly Media, November 2005. *The classic book on the Linux kernel. You should read it.*

[C06] "The Search For Fast, Scalable Counters" by Jonathan Corbet. February 1, 2006. Available: https://lwn.net/Articles/170003. *LWN has many wonderful articles about the latest in Linux. This article is a short description of scalable approximate counting; read it, and others, to learn more about the latest in Linux.*

[L+13] "A Study of Linux File System Evolution" by Lanyue Lu, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, Shan Lu. FAST '13, San Jose, CA, February 2013. *Our paper that studies every patch to Linux file systems over nearly a decade. Lots of fun findings in there; read it to see! The work was painful to do though; the poor graduate student, Lanyue Lu,* had *to look through every single patch by hand in order to understand what they did.*

[MS98] "Nonblocking Algorithms and Preemption-safe Locking on Multiprogrammed Shared-memory Multiprocessors" by M. Michael, M. Scott. Journal of Parallel and Distributed Computing, Vol. 51, No. 1, 1998. *Professor Scott and his students have been at the forefront of concurrent algorithms and data structures for many years; check out his web page, numerous papers, or books to find out more.*

[MS04] "Concurrent Data Structures" by Mark Moir and Nir Shavit. In Handbook of Data Structures and Applications (Editors D. Metha and S.Sahni). Chapman and Hall/CRC Press, 2004. Available: www.ostep.org/Citations/concurrent.pdf. *A short but relatively comprehensive reference on concurrent data structures. Though it is missing some of the latest works in the area (due to its age), it remains an incredibly useful reference.*

[MM00] "Solaris Internals: Core Kernel Architecture" by Jim Mauro and Richard McDougall. Prentice Hall, October 2000. *The Solaris book. You should also read this, if you want to learn about something other than Linux.*

[S+11] "Making the Common Case the Only Case with Anticipatory Memory Allocation" by Swaminathan Sundararaman, Yupu Zhang, Sriram Subramanian, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '11, San Jose, CA, February 2011. *Our work on removing possibly-failing allocation calls from kernel code paths. By allocating all potentially needed memory before doing any work, we avoid failure deep down in the storage stack.*

## 作业（编程）

在这个作业中，你将积累一些编写并发代码、测量其性能的经验。学会写出性能良好的代码是一项关键技能，所以在这里练练手很值得。

## 问题

1. 首先，我们来重做本章中的测量。在程序里用 `gettimeofday()` 调用来测量时间。这个计时器有多精确？它能测量的最小间隔是多少？要对它的工作方式心中有数，因为后面所有问题都要用到它。你也可以研究一下其他计时器，比如 x86 上通过 `rdtsc` 指令提供的周期计数器。
2. 现在，构建一个简单的并发计数器（concurrent counter），在线程数逐渐增加的情况下，测量把计数器增加很多次要花多长时间。你使用的系统上有几个 CPU？这个数目对你的测量结果有影响吗？
3. 接下来，实现一个近似计数器（approximate counter）。同样，测量它的性能随线程数和阈值如何变化。这些数字和你在本章中看到的一致吗？
4. 按本章引用的文献 [MS04]，实现一个使用交替加锁（hand-over-hand locking）的链表。你应该先读这篇论文，弄懂它的原理，然后再实现。测量它的性能。交替加锁（hand-over-hand locking）的链表在什么情况下比本章中的标准链表表现更好？
5. 挑一个你喜欢的数据结构，比如 B 树（B-tree），或者其他稍微有趣一点的结构。实现它，先采用简单的加锁策略，比如只用一把锁。测量它的性能随并发线程（concurrent thread）数的增加如何变化。
6. 最后，为你喜欢的这个数据结构想一个更有意思的加锁策略。实现它，并测量它的性能。它和直截了当的加锁方法相比如何？

 © 2008–23, ARPACI-DUSSEAU

# 30

# 条件变量

到目前为止，我们介绍了锁（lock）的概念，也看到了怎样把硬件和操作系统的支持恰当地结合起来，正确地实现一把锁。遗憾的是，要编写并发程序（concurrent program），光有锁这一种原语还不够。

具体来说，很多时候线程（thread）需要先检查某个**条件**（condition）是否成立，然后才能继续执行。例如，父线程（parent thread）可能想先确认子线程（child thread）已经执行完毕，再继续往下走（这通常叫作 `join()`；这样的等待该怎么实现呢？我们来看图 30.1。

```
1  void *child(void *arg) {
2      printf("child\n");
3      // XXX 怎么表示我们已经完成了?
4      return NULL;
5  }
6
7  int main(int argc, char *argv[]) {
8      printf("parent: begin\n");
9      pthread_t c;
10     Pthread_create(&c, NULL, child, NULL); // 子线程
11     // XXX 怎么等待子线程?
12     printf("parent: end\n");
13     return 0;
14 }
```

图 30.1：**父线程等待子线程**

我们希望看到的输出是这样的：

```
parent: begin
child
parent: end
```

我们可以试试用一个共享变量，如图 30.2 所示。这个办法一般能行，但效率极低：父线程会一直自旋（spin），白白浪费 CPU 时间。我们真正想要的，是某种能让父线程睡眠的办法，等到它所等的条件（例如子线程执行完毕）成立时再醒来。

```
1  volatile int done = 0;
2
3  void *child(void *arg) {
4      printf("child\n");
5      done = 1;
6      return NULL;
7  }
8
9  int main(int argc, char *argv[]) {
10     printf("parent: begin\n");
11     pthread_t c;
12     Pthread_create(&c, NULL, child, NULL); // 子线程
13     while (done == 0)
14         ; // 自旋
15     printf("parent: end\n");
16     return 0;
17 }
```

图 30.2：**父线程等待子线程：基于自旋的方法**

关键问题：如何等待一个条件

在多线程（multi-threaded）程序中，线程常常需要等某个条件成立之后再继续执行。最简单的做法是一直自旋，直到条件成立，但这样做效率极低，会浪费 CPU 周期，在某些情况下甚至是错误的。那么，线程应该怎样等待一个条件呢?

## 30.1 定义和例程

要等待某个条件成立，线程可以使用所谓的**条件变量**（condition variable）。条件变量是一个显式的队列（queue）：当程序执行时的某种状态（也就是某个**条件**）不符合要求时，线程可以把自己放进这个队列（即在该条件上**等待**，waiting）；另一个线程改变了这个状态之后，就可以唤醒一个（或多个）等待中的线程，让它们继续执行（即在该条件上**发信号**，signaling）。这个思想可以追溯到 Dijkstra 使用的“私用信号量”（private semaphores）[D68]；后来，Hoare 在研究管程（monitor）时，把一个类似的思想命名为“条件变量”[H74]。

声明条件变量很简单，写一句 `pthread_cond_t c;`，就把 `c` 声明成了条件变量（注意：还需要正确地初始化）。条件变量有两个相关的操作：`wait()` 和 `signal()`。线程想让自己睡眠时，调用 `wait()`；线程修改了程序中的某些东西，想唤醒在这个条件上睡眠等待的线程时，调用 `signal()`。具体来说，POSIX 调用是这样的：

```
pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);
pthread_cond_signal(pthread_cond_t *c);
```

为简单起见，我们常把它们称为 `wait()` 和 `signal()`。你可能注意到，`wait()` 调用还要传入一个互斥锁（mutex）作为参数；它假定调用 `wait()` 时这个互斥锁已经加锁（locked）。`wait()` 的职责是释放这把锁，并让调用它的线程睡眠，这两步要原子地

```
1  int done = 0;
2  pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
3  pthread_cond_t c = PTHREAD_COND_INITIALIZER;
4
5  void thr_exit() {
6      Pthread_mutex_lock(&m);
7      done = 1;
8      Pthread_cond_signal(&c);
9      Pthread_mutex_unlock(&m);
10 }
11
12 void *child(void *arg) {
13     printf("child\n");
14     thr_exit();
15     return NULL;
16 }
17
18 void thr_join() {
19     Pthread_mutex_lock(&m);
20     while (done == 0)
21         Pthread_cond_wait(&c, &m);
22     Pthread_mutex_unlock(&m);
23 }
24
25 int main(int argc, char *argv[]) {
26     printf("parent: begin\n");
27     pthread_t p;
28     Pthread_create(&p, NULL, child, NULL);
29     thr_join();
30     printf("parent: end\n");
31     return 0;
32 }
```

图 30.3：**父线程等待子线程：使用条件变量**

（atomically）完成；等别的线程给它发了信号（signal）、它被唤醒以后，必须重新获取这把锁，才能返回到调用者。之所以这么复杂，是为了防止线程在让自己睡眠的过程中出现某些竞态条件（race condition，也叫竞争条件）。为了更好地理解这一点，我们来看看 join 问题的解法（图 30.3）。

需要考虑两种情况。第一种情况是，父线程创建了子线程，但自己继续运行（假设只有一个处理器），于是立即调用 `thr_join()`，等待子线程完成。这时，父线程会获取锁，检查子线程是否已经完成（还没有），然后调用 `wait()` 让自己睡眠（同时释放锁）。子线程最终会运行，打印出消息“child”，然后调用 `thr_exit()` 唤醒父线程；这段代码只是获取锁，设置状态变量（state variable）`done`，然后给父线程发信号，把它唤醒。最后，父线程运行（从 `wait()` 返回时已经持有锁），释放锁，并打印出最后一条消息“parent: end”。

```
1  void thr_exit() {
2      Pthread_mutex_lock(&m);
3      Pthread_cond_signal(&c);
4      Pthread_mutex_unlock(&m);
5  }
6
7  void thr_join() {
8      Pthread_mutex_lock(&m);
9      Pthread_cond_wait(&c, &m);
10     Pthread_mutex_unlock(&m);
11 }
```

图 30.4：**父线程等待：没有状态变量**

第二种情况是，子线程一被创建就立即运行，把 done 设为 1，调用 signal 唤醒睡眠的线程（但并没有线程在睡眠，所以直接返回），然后结束。之后父线程运行，调用 thr_join()，发现 done 是 1，于是不用等待，直接返回。

最后说一点：你可能注意到，父线程在决定要不要在条件上等待时，用的是 while 循环，而不只是一条 if 语句。按照程序的逻辑，这样做似乎并非绝对必要，但它总是一个好习惯，后面我们会看到原因。

为了确保你理解 thr_exit() 和 thr_join() 代码中每一部分的重要性，我们来试几种别的实现。首先，你可能想知道是否真的需要状态变量（state variable）done。如果代码写成下面这样会怎样？（图 30.4）

遗憾的是，这种做法是错的。设想子线程被创建后立即运行，并且马上调用了 thr_exit()；这时子线程会发信号，但并没有线程在这个条件上睡眠。等父线程运行时，它会直接调用 wait，然后就卡在那里，再也不会有线程来唤醒它。从这个例子你应该能体会到状态变量（state variable）done 的重要性：它记录了各个线程关心的那个值。睡眠、唤醒和加锁（locking），都是围绕它展开的。

这里（图 30.5）是另一个糟糕的实现。在这个例子中，我们设想发信号和等待时都不需要持有锁。这里会出什么问题？想一想[1]！

```
1  void thr_exit() {
2      done = 1;
3      Pthread_cond_signal(&c);
4  }
5
6  void thr_join() {
7      if (done == 0)
8          Pthread_cond_wait(&c);
9  }
```

图 30.5：**父线程等待：没有锁**

[1]注意，这个例子并不是“真实”的代码，因为调用 pthread_cond_wait() 时，除了条件变量，总是还需要传入一个互斥锁；这里只是为了举这个反面例子，假装接口不需要互斥锁。

提示：发信号时总是持有锁

使用条件变量时，虽然并非所有情况都严格需要，但在发信号时持有锁，可能是最简单、也最好的做法。上面的例子展示了一种为了正确性必须持有锁的情况；不过也有一些情况，不持有锁大概也没问题，但多半还是应该避免。所以，为简单起见，**调用 signal 时要持有锁**。

这条提示反过来，也就是调用 wait 时要持有锁，就不只是一条提示了，而是 wait 的语义所规定的。因为 wait 总是：（a）假定你调用它时已经持有锁；（b）在让调用者睡眠时释放这把锁；（c）在返回之前重新获取这把锁。因此，把这条提示推广一下也是对的：**调用 signal 或 wait 时都持有锁**，你就总不会出问题。

这里的问题是一个微妙的竞态条件。具体来说，如果父线程调用 `thr_join()`，然后检查 `done` 的值，会发现它是 0，于是准备去睡眠。但就在它调用 wait 去睡眠之前，父线程被中断（interrupted），子线程开始运行。子线程把状态变量（state variable）`done` 改成 1，然后发信号，可此时没有线程在等待，所以没有线程被唤醒。等父线程再次运行时，它就会永远睡下去，真让人难过。

希望你能从这个简单的 join 例子中，看出正确使用条件变量的一些基本要求。为了确保你真正理解，我们接下来看一个更复杂的例子：**生产者/消费者**（producer/consumer）问题，也叫**有界缓冲区**（bounded buffer）问题。

## 30.2 生产者/消费者（有界缓冲区）问题

本章要面对的下一个同步问题叫作**生产者/消费者**（producer/consumer）问题，有时也叫**有界缓冲区**（bounded buffer）问题，最早由 Dijkstra 提出 [D72]。事实上，正是这个生产者/消费者问题，促使 Dijkstra 和他的同事发明了通用的信号量（semaphore），它既可以当锁用，也可以当条件变量用 [D01]；后面我们还会进一步学习信号量。

设想有一个或多个生产者线程（producer thread），以及一个或多个消费者线程（consumer thread）。生产者生成数据项，把它们放进缓冲区（buffer）；消费者从缓冲区里取出这些数据项，以某种方式消费掉。

这种安排在许多实际系统中都会出现。例如，在多线程的 Web 服务器中，生产者把 HTTP 请求放进一个工作队列（work queue），也就是有界缓冲区（bounded buffer）；消费者线程（consumer thread）从这个队列中取出请求并处理。

把一个程序的输出用管道（pipe）传给另一个程序时，也会用到有界缓冲区（bounded buffer），例如 `grep foo file.txt | wc -l`。这个例子并发（concurrently）运行两个进程（process）：`grep` 把 `file.txt` 中含有字符串 `foo` 的行写到它以为的标准输出（standard output）上；UNIX shell 把这个输出重定向（redirect）到所谓的 UNIX 管道（由 `pipe` 系统调用创建）。管道的另一端连着进程 `wc` 的标准输入（standard input），`wc` 只是统计输入流中有多少行，然后打印出结果。因此，`grep` 进程是生产者，`wc` 进程是消费者，它们之间是一个内核中的有界缓冲区（bounded buffer）；而在这个例子中，你只是那个开心的用户。

 © 2008–25, ARPACI-DUSSEAU

```
1  int buffer;
2  int count = 0; // 初始为空
3
4  void put(int value) {
5      assert(count == 0);
6      count = 1;
7      buffer = value;
8  }
9
10 int get() {
11     assert(count == 1);
12     count = 0;
13     return buffer;
14 }
```

图 30.6：**put 和 get 例程（第 1 版）**

由于有界缓冲区（bounded buffer）是共享资源，访问它时当然必须同步，以免[2]出现竞态条件。为了更好地理解这个问题，我们来看一些实际的代码。

我们首先需要一个共享缓冲区，生产者把数据放进去，消费者从里面取数据。为简单起见，我们只用一个整数，再加上两个内部例程：一个把值放进共享缓冲区，一个从缓冲区取出值。当然，你也可以想象在这个位置放一个指针，指向某个数据结构（data structure）。详见图 30.6（第 315 页）。

很简单，对吧？`put()` 例程假定缓冲区是空的（并用断言检查这一点），然后只是把一个值放进共享缓冲区，再把 `count` 设为 1，标记缓冲区已满。`get()` 例程正好相反：它把缓冲区置空（即把 `count` 设为 0），并返回那个值。不用担心这个共享缓冲区只有一个位置；稍后我们会把它推广成一个能容纳多项的队列，那会比听起来还要有趣。

现在，我们需要写一些例程，让它们知道什么时候可以访问缓冲区，往里放数据或从中取数据。条件应该很明显：只有在 `count` 为 0（即缓冲区为空）时，才往缓冲区放数据；只有在 `count` 为 1（即缓冲区已满）时，才从缓冲区取数据。如果我们写出的同步代码让生产者往满的缓冲区里放数据，或者让消费者从空的缓冲区里取数据，那就是做错了（在这段代码中，断言会被触发）。

这项工作由两类线程完成，一类我们称为**生产者**线程，另一类称为**消费者**线程。图 30.7 给出了它们的代码：生产者把一个整数放进共享缓冲区，一共放 `loops` 次；消费者则（永不停止地）从共享缓冲区取出数据，每取一次，就把取到的数据项打印出来。

## 有问题的方案

现在假设只有一个生产者和一个消费者。显然，`put()` 和 `get()` 例程中都有临界区（critical section），因为 `put()` 会更新缓冲区，`get()` 会读取缓冲区。然而，只是在代码外面加一把锁是不行的，我们还需要别的东西。毫不意外，这个“别的东西”就是一些条件

[2]我们在这里用了一个相当古老的英语词 lest（以免），连虚拟语气都用上了。

```
1  void *producer(void *arg) {
2      int i;
3      int loops = (int) arg;
4      for (i = 0; i < loops; i++) {
5          put(i);
6      }
7  }
8
9  void *consumer(void *arg) {
10     while (1) {
11         int tmp = get();
12         printf("%d\n", tmp);
13     }
14 }
```

图 30.7：**生产者/消费者线程（第 1 版）**

变量。在下面这个有问题的初步尝试中（图 30.8），我们使用了一个条件变量 `cond`，以及与它关联的锁 `mutex`。

我们来看一下生产者和消费者之间的发信号逻辑。生产者想要填充缓冲区时，会等待缓冲区变空（p1–p3）。消费者的逻辑完全一样，只是等待的条件不同：缓冲区变满（c1–c3）。

只有一个生产者和一个消费者时，图 30.8 中的代码能够正常工作。然而，如果这类线程不止一个（例如有两个消费者），这个方案就有两个严重的问题。是哪两个呢？

……（在这里停下来想一想）……

我们先来理解第一个问题，它和等待之前的那条 `if` 语句有关。假设有两个消费者（$T_{c1}$ 和 $T_{c2}$）和一个生产者（$T_p$）。首先，一个消费者（$T_{c1}$）运行；它获取锁（c1），检查有没有可以消费的缓冲区（c2），发现没有，于是等待（c3），等待时会释放锁。

接着，生产者（$T_p$）运行。它获取锁（p1），检查是否所有缓冲区都满了（p2），发现并没有，就接着往缓冲区里填数据（p4）。然后，生产者发信号，表示有一个缓冲区已经填满（p5）。关键在于，这会让第一个消费者（$T_{c1}$）不再在条件变量上睡眠，而是进入就绪队列（ready queue）；$T_{c1}$ 现在可以运行了（只是还没有运行）。之后，生产者继续执行，直到发现缓冲区已满，于是去睡眠（p6、p1–p3）。

问题就出在这里：另一个消费者（$T_{c2}$）趁机溜了进来，把缓冲区里仅有的那个值消费掉了（c1、c2、c4、c5、c6；因为缓冲区是满的，所以跳过了 c3 处的等待）。现在假设 $T_{c1}$ 运行；在从 wait 返回之前，它重新获取锁，然后返回。接着它调用 `get()`（c4），可缓冲区里已经没有可以消费的数据了！断言被触发，代码没有按预期工作。显然，我们本该设法阻止 $T_{c1}$ 去消费，因为 $T_{c2}$ 已经溜进来，把缓冲区里刚生产出来的那个值消费掉了。图 30.9 展示了每个线程执行的动作，以及它的调度状态随时间的变化；调度状态分为就绪（ready）、运行（running）和睡眠（sleeping）三种。

问题的原因很简单：生产者唤醒 $T_{c1}$ 之后、$T_{c1}$ 真正运行之前，$T_{c2}$ 改变了有界缓冲区（bounded buffer）的状态。给一个线程发信号只是把它唤醒；因此，这只是一个提示，表示外界的状态变了（在这个例子中，是缓冲区里放进了一个值），但并不能保证被唤醒的线程运

```
1  int loops; // 必须在某处初始化……
2  cond_t  cond;
3  mutex_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          Pthread_mutex_lock(&mutex);            // p1
9          if (count == 1)                        // p2
10             Pthread_cond_wait(&cond, &mutex); // p3
11         put(i);                                // p4
12         Pthread_cond_signal(&cond);            // p5
13         Pthread_mutex_unlock(&mutex);          // p6
14     }
15 }
16
17 void *consumer(void *arg) {
18     int i;
19     for (i = 0; i < loops; i++) {
20         Pthread_mutex_lock(&mutex);            // c1
21         if (count == 0)                        // c2
22             Pthread_cond_wait(&cond, &mutex); // c3
23         int tmp = get();                       // c4
24         Pthread_cond_signal(&cond);            // c5
25         Pthread_mutex_unlock(&mutex);          // c6
26         printf("%d\n", tmp);
27     }
28 }
```

图 30.8：**生产者/消费者：一个条件变量加 if 语句**

行时，状态仍然符合要求。对信号含义的这种理解，通常称为 **Mesa 语义**（Mesa semantics），得名于最早以这种方式实现条件变量的研究 [LR80]；与之相对的叫作 **Hoare 语义**（Hoare semantics），它实现起来更难，但提供的保证更强：被唤醒的线程会立即运行 [H74]。几乎所有实际做出来的系统，用的都是 Mesa 语义（Mesa semantics）。

### 好一些，但仍有问题：用 while，不用 if

好在修起来很容易（图 30.10）：把 `if` 改成 `while`。想一想为什么这样就行了：现在，消费者 $T_{c1}$ 醒来后，会（在持有锁的情况下）立即重新检查共享变量的状态（c2）。如果这时缓冲区是空的，消费者就回去接着睡（c3）。生产者中相应的 `if` 也改成了 `while`（p2）。

由于采用的是 Mesa 语义（Mesa semantics），使用条件变量时要记住一条简单的规则：**总是使用 while 循环**。有时候你并不需要重新检查条件，但重新检查总是安全的；照做就是了，你会很开心的。

| $T_{c1}$ | 状态 | $T_{c2}$ | 状态 | $T_p$ | 状态 | **计数** | 说明 |
|---|---|---|---|---|---|---|---|
| c1 | 运行 | | 就绪 | | 就绪 | 0 | |
| c2 | 运行 | | 就绪 | | 就绪 | 0 | |
| c3 | 睡眠 | | 就绪 | | 就绪 | 0 | 没有数据可取 |
| | 睡眠 | | 就绪 | p1 | 运行 | 0 | |
| | 睡眠 | | 就绪 | p2 | 运行 | 0 | |
| | 睡眠 | | 就绪 | p4 | 运行 | 1 | 缓冲区已满 |
| | 就绪 | | 就绪 | p5 | 运行 | 1 | $T_{c1}$ 被唤醒 |
| | 就绪 | | 就绪 | p6 | 运行 | 1 | |
| | 就绪 | | 就绪 | p1 | 运行 | 1 | |
| | 就绪 | | 就绪 | p2 | 运行 | 1 | |
| | 就绪 | | 就绪 | p3 | 睡眠 | 1 | 缓冲区已满，睡眠 |
| | 就绪 | c1 | 运行 | | 睡眠 | 1 | $T_{c2}$ 溜了进来…… |
| | 就绪 | c2 | 运行 | | 睡眠 | 1 | |
| | 就绪 | c4 | 运行 | | 睡眠 | 0 | ……并取走了数据 |
| | 就绪 | c5 | 运行 | | 就绪 | 0 | $T_p$ 被唤醒 |
| | 就绪 | c6 | 运行 | | 就绪 | 0 | |
| c4 | 运行 | | 就绪 | | 就绪 | 0 | 糟了！没有数据 |

图 30.9：**线程轨迹：有问题的方案（第 1 版）**

然而，这段代码仍然有一个 bug，也就是前面提到的两个问题中的第二个。你能看出来吗？它和只有一个条件变量这一点有关。在往下读之前，先试着想出问题在哪里。一定要想！（停下来想一想，或者闭上眼睛……）

我们来确认一下你是不是想对了，或者说，确认一下你现在还醒着，正在读书里的这一部分。问题出现在两个消费者（$T_{c1}$ 和 $T_{c2}$）先运行、并且都去睡眠（c3）的时候。接着生产者运行，往缓冲区里放了一个值，并唤醒其中一个消费者（比如 $T_{c1}$）。然后，生产者回到循环开头（中途释放锁、又重新获取锁），想往缓冲区里放更多数据；由于缓冲区已满，生产者只好在条件上等待（于是睡眠）。现在，一个消费者（$T_{c1}$）可以运行，另外两个线程 $T_{c2}$ 和 $T_p$ 在条件上睡眠。麻烦马上就要来了：事情变得刺激起来了！

接着，消费者 $T_{c1}$ 醒来，从 `wait()` 返回（c3），重新检查条件（c2），发现缓冲区是满的，就把这个值消费掉（c4）。关键在于，这个消费者随后会在条件上发信号（c5），而这只会唤醒一个正在睡眠的线程。那么，它应该唤醒哪个线程呢？

由于消费者刚把缓冲区清空，它显然应该唤醒生产者。然而，如果它唤醒的是消费者 $T_{c2}$（这完全有可能，取决于等待队列是怎么管理的），就出问题了。具体来说，消费者 $T_{c2}$ 醒来后会发现缓冲区是空的（c2），于是又回去睡眠（c3）。生产者 $T_p$ 有值要放进缓冲区，却仍在睡眠。另一个消费者线程（consumer thread）$T_{c1}$ 也回去睡了。三个线程全都在睡眠，这显然是个 bug；图 30.11 把这场可怕灾难的残酷经过一步步列了出来。

发信号显然是必要的，但必须更有针对性。消费者不应该唤醒其他消费者，只应该唤醒生产者，反之亦然。

 © 2008–25, ARPACI-DUSSEAU

```
1  int loops; // 必须在某处初始化……
2  cond_t  cond;
3  mutex_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          Pthread_mutex_lock(&mutex);            // p1
9          while (count == 1)                     // p2
10             Pthread_cond_wait(&cond, &mutex); // p3
11         put(i);                                // p4
12         Pthread_cond_signal(&cond);            // p5
13         Pthread_mutex_unlock(&mutex);          // p6
14     }
15 }
16
17 void *consumer(void *arg) {
18     int i;
19     for (i = 0; i < loops; i++) {
20         Pthread_mutex_lock(&mutex);            // c1
21         while (count == 0)                     // c2
22             Pthread_cond_wait(&cond, &mutex); // c3
23         int tmp = get();                       // c4
24         Pthread_cond_signal(&cond);            // c5
25         Pthread_mutex_unlock(&mutex);          // c6
26         printf("%d\n", tmp);
27     }
28 }
```

图 30.10：**生产者/消费者：一个条件变量加 while**

### 单缓冲区的生产者/消费者方案

这里的解决办法同样很简单：用两个条件变量，而不是一个，这样在系统状态变化时，就能正确地通知应该醒来的那一类线程。图 30.12 给出了修改后的代码。

在这段代码中，生产者线程（producer thread）在条件 **empty** 上等待，给 **fill** 发信号。反过来，消费者线程（consumer thread）在 **fill** 上等待，给 **empty** 发信号。这样一来，上面的第二个问题从设计上就避免了：消费者永远不会意外唤醒消费者，生产者也永远不会意外唤醒生产者。

### 正确的生产者/消费者方案

我们现在有了一个能工作的生产者/消费者方案，只是还不够通用。最后一处改动是为了提高并发性（concurrency）和效率：我们增加更多的缓冲槽，这样，生产者在睡眠之前可以生产多个值，消费者在睡眠之前也可以消费多个值。只有一个生产者和一个消费者时，这种做法减少了上下文切换（context switch），所以效率更高；有多个生产者或多个消费者

| $T_{c1}$ | 状态 | $T_{c2}$ | 状态 | $T_p$ | 状态 | **计数** | 说明 |
|---|---|---|---|---|---|---|---|
| c1 | 运行 | | 就绪 | | 就绪 | 0 | |
| c2 | 运行 | | 就绪 | | 就绪 | 0 | |
| c3 | 睡眠 | | 就绪 | | 就绪 | 0 | 没有数据可取 |
| | 睡眠 | c1 | 运行 | | 就绪 | 0 | |
| | 睡眠 | c2 | 运行 | | 就绪 | 0 | |
| | 睡眠 | c3 | 睡眠 | | 就绪 | 0 | 没有数据可取 |
| | 睡眠 | | 睡眠 | p1 | 运行 | 0 | |
| | 睡眠 | | 睡眠 | p2 | 运行 | 0 | |
| | 睡眠 | | 睡眠 | p4 | 运行 | 1 | 缓冲区已满 |
| | 就绪 | | 睡眠 | p5 | 运行 | 1 | $T_{c1}$ 被唤醒 |
| | 就绪 | | 睡眠 | p6 | 运行 | 1 | |
| | 就绪 | | 睡眠 | p1 | 运行 | 1 | |
| | 就绪 | | 睡眠 | p2 | 运行 | 1 | |
| | 就绪 | | 睡眠 | p3 | 睡眠 | 1 | 必须睡眠（已满） |
| c2 | 运行 | | 睡眠 | | 睡眠 | 1 | 重新检查条件 |
| c4 | 运行 | | 睡眠 | | 睡眠 | 0 | $T_{c1}$ 取走数据 |
| c5 | 运行 | | 就绪 | | 睡眠 | 0 | 糟糕！唤醒了 $T_{c2}$ |
| c6 | 运行 | | 就绪 | | 睡眠 | 0 | |
| c1 | 运行 | | 就绪 | | 睡眠 | 0 | |
| c2 | 运行 | | 就绪 | | 睡眠 | 0 | |
| c3 | 睡眠 | | 就绪 | | 睡眠 | 0 | 没有数据可取 |
| | 睡眠 | c2 | 运行 | | 睡眠 | 0 | |
| | 睡眠 | c3 | 睡眠 | | 睡眠 | 0 | 全都睡着了…… |

图 30.11：**线程轨迹：有问题的方案（第 2 版）**

（或者两者都有）时，它甚至允许生产或消费并发进行，从而提高并发性。好在这和当前的方案相比只是一个小改动。

这个正确方案的第一处改动在缓冲区结构本身，以及相应的 `put()` 和 `get()`（图 30.13）。我们还稍微修改了生产者和消费者用来决定是否睡眠的条件。图 30.14 给出了正确的等待和发信号逻辑。只有所有缓冲槽都满了，生产者才会睡眠（p2）；同样，只有所有缓冲槽都空了，消费者才会睡眠（c2）。这样，我们就解决了生产者/消费者问题；可以坐下来，喝杯冰镇啤酒了。

## 30.3 覆盖条件

我们再来看一个使用条件变量的例子。这个代码案例取自 Lampson 和 Redell 关于 Pilot 的论文 [LR80]，他们也正是最早实现上文所述 **Mesa 语义**（Mesa semantics）的那个团队（他们用的语言叫 Mesa，这个名字就是这么来的）。

他们遇到的问题最好用一个例子来说明：一个简单的多线程内存分配库。图 30.15 给出了一段（简化过的）代码，用来演示这个问题。

 © 2008–25, ARPACI-DUSSEAU

```
int loops; // 必须在某处初始化……
cond_t  empty, fill;
mutex_t mutex;

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == 1)
            Pthread_cond_wait(&empty, &mutex);
        put(i);
        Pthread_cond_signal(&fill);
        Pthread_mutex_unlock(&mutex);
    }
}
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == 0)
            Pthread_cond_wait(&fill, &mutex);
        int tmp = get();
        Pthread_cond_signal(&empty);
        Pthread_mutex_unlock(&mutex);
        printf("%d\n", tmp);
    }
}
```

图 30.12：**生产者/消费者：两个条件变量加 while**

```
int buffer[MAX];
int fill_ptr = 0;
int use_ptr  = 0;
int count    = 0;

void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
int get() {
    int tmp = buffer[use_ptr];
    use_ptr = (use_ptr + 1) % MAX;
    count--;
    return tmp;
}
```

图 30.13：**正确的 put 和 get 例程**

```
1  int loops; // 必须在某处初始化……
2  cond_t  empty, fill;
3  mutex_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          Pthread_mutex_lock(&mutex);             // p1
9          while (count == MAX)                    // p2
10             Pthread_cond_wait(&empty, &mutex); // p3
11         put(i);                                 // p4
12         Pthread_cond_signal(&fill);             // p5
13         Pthread_mutex_unlock(&mutex);           // p6
14     }
15 }
16
17 void *consumer(void *arg) {
18     int i;
19     for (i = 0; i < loops; i++) {
20         Pthread_mutex_lock(&mutex);             // c1
21         while (count == 0)                      // c2
22             Pthread_cond_wait(&fill, &mutex);  // c3
23         int tmp = get();                        // c4
24         Pthread_cond_signal(&empty);            // c5
25         Pthread_mutex_unlock(&mutex);           // c6
26         printf("%d\n", tmp);
27     }
28 }
```

图 30.14：**正确的生产者/消费者同步**

从代码中可以看到，线程调用内存分配代码时，可能得等一等，直到有更多内存空闲出来。反过来，线程释放内存时，会发信号表示又有内存空闲了。然而，上面的代码有一个问题：应该唤醒哪个等待中的线程？要知道，等待的线程可能不止一个。

设想下面的场景。假设空闲字节数为零；线程 $T_a$ 调用 `allocate(100)`，接着线程 $T_b$ 调用 `allocate(10)`，申请的内存更少。于是 $T_a$ 和 $T_b$ 都在条件上等待，都去睡眠了；空闲字节不够满足其中任何一个请求。

提示：检查条件时用 while（不要用 if）

在多线程程序中检查条件时，用 `while` 循环总是正确的；只用 `if` 语句则不一定正确，取决于发信号的语义。所以，总是用 `while`，你的代码就会按预期运行。

在条件检查外面套上 `while` 循环，还能应对**虚假唤醒**（spurious wakeup）。在某些线程库（thread library）中，由于实现上的细节，有可能只发了一次信号，却有两个线程被唤醒 [L11]。虚假唤醒（spurious wakeup）是线程要重新检查所等条件的又一个理由。

```
1  // 堆上还有多少字节空闲?
2  int bytesLeft = MAX_HEAP_SIZE;
3
4  // 还需要锁和条件变量
5  cond_t  c;
6  mutex_t m;
7
8  void *
9  allocate(int size) {
10     Pthread_mutex_lock(&m);
11     while (bytesLeft < size)
12         Pthread_cond_wait(&c, &m);
13     void *ptr = ...; // 从堆上取内存
14     bytesLeft -= size;
15     Pthread_mutex_unlock(&m);
16     return ptr;
17 }
18
19 void free(void *ptr, int size) {
20     Pthread_mutex_lock(&m);
21     // 把一些内存还给堆
22     bytesLeft += size;
23     Pthread_cond_signal(&c); // 该唤醒谁??
24     Pthread_mutex_unlock(&m);
25 }
```

图 30.15：**覆盖条件：一个例子**

这时，假设第三个线程 $T_c$ 调用 `free(50)`。遗憾的是，它调用 signal 唤醒一个等待线程时，唤醒的可能并不是正确的那个，也就是只等 10 字节空出来的 $T_b$；而 $T_a$ 应该继续等待，因为空闲内存还不够。所以图中的代码不能正常工作：负责唤醒的线程并不知道该唤醒哪个（或哪些）线程。

Lampson 和 Redell 提出的解决办法很简单：用 `pthread_cond_broadcast()` 代替上面代码中的 `pthread_cond_signal()`。前者会唤醒所有等待的线程。这样就能保证，所有应该被唤醒的线程都会被唤醒。当然，这样做可能有损性能，因为它会毫无必要地唤醒许多其他（暂时）还不该醒来的等待线程。这些线程只会醒来，重新检查条件，然后马上回去睡眠。

Lampson 和 Redell 把这样的条件称为**覆盖条件**（covering condition），因为它（保守地）覆盖了所有需要唤醒线程的情况；正如前面讨论的，代价是可能会唤醒太多线程。细心的读者可能还注意到，我们之前也可以用这种办法（参见只用一个条件变量的生产者/消费者问题）。不过在那种情况下，我们有更好的方案可用，所以就采用了它。一般来说，如果你发现只有把发信号改成广播（broadcast），程序才能正常工作（而你认为本不应该需要这样），那么你的程序多半有 bug；去把它修好！但在像上面这个内存分配器（memory allocator）这样的情况下，广播可能是现成的最简单的办法。

## 30.4 小结

我们介绍了锁之外的又一个重要的同步原语（synchronization primitive）：条件变量。条件变量让线程在程序的某个状态不符合要求时去睡眠，借助它，我们可以干净利落地解决许多重要的同步问题，包括著名的（而且至今仍然重要的）生产者/消费者问题，以及覆盖条件（covering condition）问题。这里本该有一句更有戏剧性的结束语，比如“他热爱老大哥” [O49]。

 © 2008–25, ARPACI-DUSSEAU

## 参考文献

[D68] "Cooperating sequential processes" by Edsger W. Dijkstra. 1968. Available online here: http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD123.PDF. *Another classic from Dijkstra; reading his early works on concurrency will teach you much of what you need to know.*

[D72] "Information Streams Sharing a Finite Buffer" by E.W. Dijkstra. Information Processing Letters 1: 179–180, 1972. http://www.cs.utexas.edu/users/EWD/ewd03xx/EWD329.PDF *The famous paper that introduced the producer/consumer problem.*

[D01] "My recollections of operating system design" by E.W. Dijkstra. April, 2001. Available: http://www.cs.utexas.edu/users/EWD/ewd13xx/EWD1303.PDF. *A fascinating read for those of you interested in how the pioneers of our field came up with some very basic and fundamental concepts, including ideas like "interrupts" and even "a stack"!*

[H74] "Monitors: An Operating System Structuring Concept" by C.A.R. Hoare. Communications of the ACM, 17:10, pages 549–557, October 1974. *Hoare did a fair amount of theoretical work in concurrency. However, he is still probably most known for his work on Quicksort, the coolest sorting algorithm in the world, at least according to these authors.*

[L11] "Pthread cond signal Man Page" by Mysterious author. March, 2011. Available online: http://linux.die.net/man/3/pthread_cond_signal. *The Linux man page shows a nice simple example of why a thread might get a spurious wakeup, due to race conditions within the signal/wakeup code.*

[LR80] "Experience with Processes and Monitors in Mesa" by B.W. Lampson, D.R. Redell. Communications of the ACM. 23:2, pages 105-117, February 1980. *A classic paper about how to actually implement signaling and condition variables in a real system, leading to the term "Mesa" semantics for what it means to be woken up; the older semantics, developed by Tony Hoare [H74], then became known as "Hoare" semantics, which is a bit unfortunate of a name.*

[O49] "1984" by George Orwell. Secker and Warburg, 1949. *A little heavy-handed, but of course a must read. That said, we kind of gave away the ending by quoting the last sentence. Sorry! And if the government is reading this, let us just say that we think that the government is "double plus good". Hear that, our pals at the NSA?*

 © 2008–25, ARPACI-DUSSEAU

## 作业（编程）

本作业让你研究一些真实的代码，这些代码用锁和条件变量，实现了本章讨论的生产者/消费者队列的几种形式。你要阅读这些真实的代码，用各种配置运行它们，借此弄清楚哪些做法行得通、哪些行不通，以及其他一些错综复杂的细节。详情请阅读 README。

## 问题

1. 第一个问题关注 `main-two-cvs-while.c`（能正常工作的方案）。首先，研究一下这段代码。你觉得自己能预料到运行这个程序时会发生什么吗?
2. 用一个生产者和一个消费者运行，让生产者生产几个值。先用大小为 1 的缓冲区，然后逐渐增大。缓冲区变大以后，代码的行为有什么变化（或者有没有变化）? 把消费者的 sleep string 从默认值（不睡眠）改成 `-C 0,0,0,0,0,0,1` 时，在不同的缓冲区大小（例如 `-m 10`）和不同的生产数量（例如 `-l 100`）下，你预计 `num_full` 会是多少?
3. 如果条件允许，在不同的系统上（例如 Mac 和 Linux）运行这段代码。你在这些系统上看到了不同的行为吗?
4. 我们来看看运行时间。一个生产者、三个消费者、只有一项的共享缓冲区，每个消费者都在 c3 处暂停一秒，你认为下面这次运行要花多长时间?

```
./main-two-cvs-while -p 1 -c 3 -m 1
    -C 0,0,0,1,0,0,0:0,0,0,1,0,0,0:0,0,0,1,0,0,0
    -l 10 -v -t
```

5. 现在把共享缓冲区的大小改成 3（`-m 3`）。这会让总时间有什么不同吗?
6. 现在把睡眠的位置改到 c6（这模拟的是消费者从队列中取出东西，再拿它去做些事情），同样使用只有一项的缓冲区。你预计这种情况下要花多长时间?

```
./main-two-cvs-while -p 1 -c 3 -m 1
    -C 0,0,0,0,0,0,1:0,0,0,0,0,0,1:0,0,0,0,0,0,1
    -l 10 -v -t
```

7. 最后，再把缓冲区大小改成 3（`-m 3`）。现在你预计要花多长时间?
8. 现在来看 `main-one-cv-while.c`。假设只有一个生产者、一个消费者，缓冲区大小为 1，你能设置一个 sleep string，让这段代码出问题吗?
9. 现在把消费者的数量改成两个。你能为生产者和消费者构造出 sleep string，让代码出问题吗?
10. 现在研究 `main-two-cvs-if.c`。你能让这段代码出问题吗? 同样，先考虑只有一个消费者的情况，再考虑有多个消费者的情况。
11. 最后，研究 `main-two-cvs-while-extra-unlock.c`。如果在执行 put 或 get 之前就释放锁，会出现什么问题? 给定 sleep string，你能稳定地让这个问题出现吗? 会发生什么坏事?

 © 2008–25, ARPACI-DUSSEAU

# 31

# 信号量

我们现在知道，要解决各种重要而有趣的并发（concurrency）问题，锁（lock）和条件变量（condition variable）缺一不可。多年前，最早意识到这一点的人之一就是 **Edsger Dijkstra**（不过确切的历史很难考证 [GR92]）。他为人所知的成就有很多：图论中著名的“最短路径”算法 [D59]；早年一篇讨论结构化编程的论战文章《Goto Statements Considered Harmful》[D68a]（这标题起得真好！）；还有本章的主角：他提出的一种叫作**信号量**（semaphore）的同步原语（synchronization primitive）[D68b, D72]。事实上，Dijkstra 和同事发明信号量，是想用一种原语处理所有与同步有关的事情；你会看到，信号量既可以当锁用，也可以当条件变量用。

> 关键问题：如何使用信号量
> 如何用信号量代替锁和条件变量？信号量的定义是什么？什么是二值信号量（binary semaphore）？用锁和条件变量构建信号量容易吗？反过来，用信号量构建锁和条件变量呢？

## 31.1 信号量的定义

信号量是一个带有整数值的对象，我们可以用两个例程来操作它；在 POSIX 标准中，这两个例程是 `sem_wait()` 和 `sem_post()`[1]。信号量的初始值决定了它的行为，所以在调用任何其他例程操作信号量之前，必须先把它初始化为某个值，如图 31.1 中的代码所示。

```
1 #include <semaphore.h>
2 sem_t s;
3 sem_init(&s, 0, 1);
```

图 31.1：**初始化信号量**

[1]历史上，Dijkstra 把 `sem_wait()` 叫作 `P()`，把 `sem_post()` 叫作 `V()`。这两个简称来自荷兰语单词；有意思的是，它们究竟源自哪两个荷兰语单词，说法前后有过变化。最初，`P()` 来自“passering”（通过），`V()` 来自“vrijgave”（释放）；后来 Dijkstra 又写道，`P()` 来自“prolaag”，它是“probeer”（荷兰语的“尝试”）和“verlaag”（“减少”）的合成词，而 `V()` 来自“verhoog”，意思是“增加”。有时人们也把它们叫作 down 和 up。拿荷兰语的说法去让朋友刮目相看吧，或者把他们弄糊涂，或者两者兼得。详见 https://news.ycombinator.com/item?id=8761539。

在图中，我们声明了一个信号量 s，给第三个参数传入 1，把它初始化为 1。在后面的所有例子中，sem_init() 的第二个参数都设为 0，表示这个信号量由同一进程（process）内的各个线程（thread）共享。信号量也可以用来同步不同进程对资源的访问，这时第二个参数要传别的值，详见手册页（manual page）。

信号量初始化之后，我们可以调用 sem_wait() 或 sem_post() 这两个函数来操作它。这两个函数的行为见图 31.2。

我们暂时不关心这两个例程如何实现。实现它们显然要格外小心：多个线程会同时调用 sem_wait() 和 sem_post()，这些临界区（critical section）自然需要妥善管理。现在我们先关注怎样使用这些原语，以后也许再讨论它们是怎么构建的。

我们来说说这两个接口的几个要点。第一，sem_wait() 要么立即返回（因为调用 sem_wait() 时信号量的值大于或等于 1），要么让调用者暂停执行，等待之后的 post 操作。当然，可能有多个线程调用 sem_wait()，它们会全部排队，等着被唤醒。

第二，sem_post() 不像 sem_wait() 那样要等待某个条件成立。它只是把信号量的值加 1，然后，如果有线程在等待唤醒，就唤醒其中一个。

第三，信号量的值为负时，其绝对值等于正在等待的线程数 [D68b]。信号量的使用者一般看不到这个值，但这个不变式（invariant）还是值得了解的，也许能帮你记住信号量是怎样工作的。

信号量内部看起来可能存在竞态条件（race condition，也叫竞争条件），这一点（暂时）不必担心，先假设信号量的操作都是原子的。我们很快就会用锁和条件变量做到这一点。

## 31.2 二值信号量（锁）

现在可以使用信号量了。第一个用法是我们已经熟悉的：把信号量当锁用。代码片段见图 31.3：我们只是用一对 sem_wait()/sem_post() 把要保护的临界区包起来。不过，要让它正确工作，关键在于信号量 m 的初始值（图中初始化为 X）。X 应该是多少？

……（继续往下读之前，先自己想一想）……

回头看看前面 sem_wait() 和 sem_post() 的定义，就能看出初始值应该是 1。

为了说清楚，我们设想一个有两个线程的场景。第一个线程，即线程 0，调用 sem_wait()，它先把信号量的值减 1，变成 0。然后，只有当值小于 0 时，它才会等待。因为值是 0，sem_wait() 直接返回，调用它的线程继续运行；线程 0 现在可以进入临界

```
1 int sem_wait(sem_t *s) {
2     把信号量 s 的值减 1
3     如果信号量 s 的值为负，就等待
4 }
5
6 int sem_post(sem_t *s) {
7     把信号量 s 的值加 1
8     如果有一个或多个线程在等待，就唤醒其中一个
9 }
```

图 31.2：**信号量：wait 和 post 的定义**

```
sem_t m;
sem_init(&m, 0, X); // 初始化为 X；X 应该是多少？

sem_wait(&m);
// 这里是临界区
sem_post(&m);
```

图 31.3：**二值信号量（也就是锁）**

| 信号量的值 | 线程 0 | 线程 1 |
|---|---|---|
| 1 | | |
| 1 | 调用 `sem_wait()` | |
| 0 | `sem_wait()` 返回 | |
| 0 | （临界区） | |
| 0 | 调用 `sem_post()` | |
| 1 | `sem_post()` 返回 | |

图 31.4：**线程轨迹：单个线程使用信号量**

区了。如果线程 0 在临界区里时没有其他线程试图获取锁，那么它调用 `sem_post()` 时，只是把信号量的值恢复为 1（也不会唤醒等待的线程，因为根本没有）。图 31.4 展示了这个场景的轨迹。

更有意思的情况是：线程 0 “持有锁”（即已经调用了 `sem_wait()`，但还没调用 `sem_post()`）时，另一个线程，即线程 1，调用 `sem_wait()`，试图进入临界区。这时，线程 1 会把信号量的值减到 –1，于是开始等待：让自己睡眠，让出（yield）处理器。线程 0 再次运行后，最终会调用 `sem_post()`，把信号量的值加回 0，然后唤醒等待的线程（线程 1），线程 1 随后就能获取锁了。线程 1 结束时，会再次把信号量的值加 1，让它恢复为 1。

图 31.5 展示了这个例子的轨迹。除了线程的动作，图中还标出了每个线程的**调度状态**（scheduler state）：运行（Run，线程正在运行）、就绪（Ready，可以运行，但没有在运行）和睡眠（Sleep，线程被阻塞）。注意，线程 1 试图获取已被持有的锁时，就进入了睡眠状态；只有等线程 0 再次运行，线程 1 才能被唤醒，才有可能再次运行。

如果你想自己推演一个例子，可以试试多个线程排队等锁的场景。在这样的轨迹中，信号量的值会是多少？

这样，我们就能把信号量当锁用了。因为锁只有两种状态（被持有和未被持有），所以有时把当作锁用的信号量叫作**二值信号量**（binary semaphore）。注意，如果你只按这种二值的方式使用信号量，它的实现可以比我们这里介绍的通用信号量更简单。

## 31.3 用信号量控制顺序

在并发程序（concurrent program）中，信号量也可以用来安排事件发生的先后顺序。例如，一个线程可能想等某个链表（linked list）变为非空，再从中删除一个元素。在这种用法中，常见的情形是：一个线程等待某件事发生，另一个线程让这件事发生，然后发信

 © 2008–25, ARPACI-DUSSEAU

| 值 | 线程 0 | 状态 | 线程 1 | 状态 |
|---|---|---|---|---|
| 1 | | 运行 | | 就绪 |
| 1 | 调用 sem_wait() | 运行 | | 就绪 |
| 0 | sem_wait() 返回 | 运行 | | 就绪 |
| 0 | (临界区开始) | 运行 | | 就绪 |
| 0 | 中断；切换→*T1* | 就绪 | | 运行 |
| 0 | | 就绪 | 调用 sem_wait() | 运行 |
| −1 | | 就绪 | sem 减 1 | 运行 |
| −1 | | 就绪 | (sem<0)→睡眠 | 睡眠 |
| −1 | | 运行 | 切换→*T0* | 睡眠 |
| −1 | (临界区结束) | 运行 | | 睡眠 |
| −1 | 调用 sem_post() | 运行 | | 睡眠 |
| 0 | sem 加 1 | 运行 | | 睡眠 |
| 0 | wake(T1) | 运行 | | 就绪 |
| 0 | sem_post() 返回 | 运行 | | 就绪 |
| 0 | 中断；切换→*T1* | 就绪 | | 运行 |
| 0 | | 就绪 | sem_wait() 返回 | 运行 |
| 0 | | 就绪 | (临界区) | 运行 |
| 0 | | 就绪 | 调用 sem_post() | 运行 |
| 1 | | 就绪 | sem_post() 返回 | 运行 |

图 31.5：**线程轨迹：两个线程使用信号量**

号告知它已经发生，从而唤醒等待的线程。于是，我们把信号量当作一种**顺序**（ordering）原语来用，就像前面使用**条件变量**（condition variable）那样。

```
1  sem_t s;
2
3  void *child(void *arg) {
4      printf("child\n");
5      sem_post(&s); // 在这里发信号：子线程执行完了
6      return NULL;
7  }
8
9  int main(int argc, char *argv[]) {
10     sem_init(&s, 0, X); // X 应该是多少?
11     printf("parent: begin\n");
12     pthread_t c;
13     Pthread_create(&c, NULL, child, NULL);
14     sem_wait(&s); // 在这里等待子线程
15     printf("parent: end\n");
16     return 0;
17 }
```

图 31.6：**父线程等待子线程**

下面是一个简单的例子。设想一个线程创建了另一个线程，然后想等它执行完（图31.6）。程序运行时，我们希望看到如下输出：

```
parent: begin
child
parent: end
```

那么，怎样用信号量达到这个效果呢？其实答案不难理解。从代码中可以看到，要等“子线程执行完毕”这个条件成立，只需父线程（parent thread）调用 `sem_wait()`、子线程（child thread）调用 `sem_post()` 就行。但问题又来了：这个信号量的初始值应该是多少？

（同样，先在这里想一想，别急着往下读）

答案当然是：信号量的值应该设为 0。需要考虑两种情况。第一种，假设父线程创建了子线程，但子线程还没有运行，也就是说，它在就绪队列（ready queue）里，但没有在运行。这时（图 31.7，第 331 页），父线程会在子线程调用 `sem_post()` 之前调用 `sem_wait()`；我们希望父线程等待子线程运行。要做到这一点，唯一的办法是让信号量的值不大于 0，所以初始值是 0。父线程运行，把信号量减为 -1，然后等待（睡眠）。子线程最终运行时，会调用 `sem_post()`，把信号量的值加到 0，并唤醒父线程；父线程随后从 `sem_wait()` 返回，程序结束。

第二种情况（图 31.8）是，子线程在父线程有机会调用 `sem_wait()` 之前就运行完了。这时，子线程先调用 `sem_post()`，把信号量的值从 0 加到 1。等父线程有机会运行时，它调用 `sem_wait()`，发现信号量的值是 1；于是父线程把值减为 0，不用等待就从 `sem_wait()` 返回，同样达到了预期的效果。

## 31.4 生产者/消费者（有界缓冲区）问题

本章要解决的下一个问题叫作**生产者/消费者**（producer/consumer）问题，有时也叫**有界缓冲区**（bounded buffer）问题 [D72]。上一章讲条件变量时已经详细描述过这个问题，细节请看那一章。

| 值 | 父线程 | 状态 | 子线程 | 状态 |
|---|---|---|---|---|
| 0 | `create(Child)` | 运行 | （子线程已存在，可以运行） | 就绪 |
| 0 | 调用 `sem_wait()` | 运行 | | 就绪 |
| -1 | `sem` 减 1 | 运行 | | 就绪 |
| -1 | `(sem<0)`→睡眠 | 睡眠 | | 就绪 |
| -1 | 切换→子线程 | 睡眠 | 子线程运行 | 运行 |
| -1 | | 睡眠 | 调用 `sem_post()` | 运行 |
| 0 | | 睡眠 | `sem` 加 1 | 运行 |
| 0 | | 就绪 | `wake(Parent)` | 运行 |
| 0 | | 就绪 | `sem_post()` 返回 | 运行 |
| 0 | | 就绪 | 中断→父线程 | 就绪 |
| 0 | `sem_wait()` 返回 | 运行 | | 就绪 |

图 31.7：**线程轨迹：父线程等待子线程（情况 1）**

| 值 | 父线程 | 状态 | 子线程 | 状态 |
|---|---|---|---|---|
| 0 | create(Child) | 运行 | （子线程已存在，可以运行） | 就绪 |
| 0 | 中断→子线程 | 就绪 | 子线程运行 | 运行 |
| 0 | | 就绪 | 调用 sem_post() | 运行 |
| 1 | | 就绪 | sem 加 1 | 运行 |
| 1 | | 就绪 | wake(nobody) | 运行 |
| 1 | | 就绪 | sem_post() 返回 | 运行 |
| 1 | 父线程运行 | 运行 | 中断→父线程 | 就绪 |
| 1 | 调用 sem_wait() | 运行 | | 就绪 |
| 0 | sem 减 1 | 运行 | | 就绪 |
| 0 | (sem≥0)→不睡眠 | 运行 | | 就绪 |
| 0 | sem_wait() 返回 | 运行 | | 就绪 |

图 31.8：**线程轨迹：父线程等待子线程（情况 2）**

提示：设置信号量的值

我们已经看到了两个初始化信号量的例子。第一个例子把值设为 1，把信号量当锁用；第二个例子设为 0，用信号量控制顺序。那么，信号量的初始化有什么通用规则吗？

有一种简单的思路（这要感谢 Perry Kivolowitz）：想一想初始化之后，你愿意马上分出去多少份资源。对锁来说是 1，因为初始化之后，你愿意让锁马上被锁上（分出去）。对控制顺序的情况来说是 0，因为一开始没有东西可以分出去；只有等子线程执行完，资源才被创造出来，这时值才加到 1。以后遇到信号量问题时，试试这种思路，看看有没有帮助。

## 第一次尝试

第一次尝试解决这个问题时，我们引入了两个信号量 `empty` 和 `full`，线程用它们分别表示缓冲区（buffer）的某一项已被清空或已被填满。`put` 和 `get` 例程的代码见图 31.9，我们为生产者/消费者问题写的第一版解法见图 31.10（第 333 页）。

在这个例子中，生产者先等待某个缓冲区变空，以便往里面放数据；消费者也类似，先等待某个缓冲区被填满，再使用它。我们先设想 `MAX=1`，即数组里只有一个缓冲区，看看这样行不行。

再设想有两个线程，一个生产者，一个消费者。我们来考察单个 CPU 上的一个具体场景。假设消费者先运行。于是，消费者执行到图 31.10 中的 C1 行，调用 `sem_wait(&full)`。因为 `full` 初始化为 0，这次调用会把 `full` 减为 -1，阻塞消费者，等待另一个线程对 `full` 调用 `sem_post()`，这正是我们想要的。

然后假设生产者运行。它执行到 P1 行，调用 `sem_wait(&empty)` 例程。和消费者不同，生产者会顺利通过这一行，因为 `empty` 初始化为 `MAX`（这里是 1）。于是 `empty` 减为 0，生产者把一个数据放进缓冲区的第一项（P2 行）。接着生产者执行到 P3 行，调用 `sem_post(&full)`，把 `full` 信号量的值从 -1 改为 0，并唤醒消费者，比如把它从阻塞（blocked）状态移到就绪状态。

```
int buffer[MAX];
int fill = 0;
int use  = 0;

void put(int value) {
    buffer[fill] = value;    // F1 行
    fill = (fill + 1) % MAX; // F2 行
}

int get() {
    int tmp = buffer[use];   // G1 行
    use = (use + 1) % MAX;   // G2 行
    return tmp;
}
```

图 31.9：**put 和 get 例程**

```
sem_t empty;
sem_t full;

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);         // P1 行
        put(i);                   // P2 行
        sem_post(&full);          // P3 行
    }
}

void *consumer(void *arg) {
    int tmp = 0;
    while (tmp != -1) {
        sem_wait(&full);          // C1 行
        tmp = get();              // C2 行
        sem_post(&empty);         // C3 行
        printf("%d\n", tmp);
    }
}

int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX); // MAX 个缓冲区为空
    sem_init(&full, 0, 0);    // 0 个缓冲区是满的
    // ...
}
```

图 31.10：**加上 full 和 empty 条件**

```
1  void *producer(void *arg) {
2      int i;
3      for (i = 0; i < loops; i++) {
4          sem_wait(&mutex);          // P0 行（新增）
5          sem_wait(&empty);          // P1 行
6          put(i);                    // P2 行
7          sem_post(&full);           // P3 行
8          sem_post(&mutex);          // P4 行（新增）
9      }
10 }
11
12 void *consumer(void *arg) {
13     int i;
14     for (i = 0; i < loops; i++) {
15         sem_wait(&mutex);          // C0 行（新增）
16         sem_wait(&full);           // C1 行
17         int tmp = get();           // C2 行
18         sem_post(&empty);          // C3 行
19         sem_post(&mutex);          // C4 行（新增）
20         printf("%d\n", tmp);
21     }
22 }
```

图 31.11：**加上互斥（错误的做法）**

接下来可能发生两种情况之一。如果生产者继续运行，它会绕回循环开头，再次执行到 P1 行。不过这一次它会阻塞，因为 `empty` 信号量的值是 0。如果生产者被中断（interrupted），消费者就会开始运行，从 `sem_wait(&full)`（C1 行）返回，发现缓冲区是满的，然后取走数据。无论哪种情况，都达到了预期的效果。

你可以用更多线程试试这个例子，例如多个生产者和多个消费者，它应该仍然能正常工作。

现在设想 `MAX` 大于 1（比如 `MAX=10`），并且有多个生产者和多个消费者。这下就有问题了：出现了竞态条件。你看出它在哪里了吗？（花点时间找一找。）如果没看出来，给个提示：仔细看看 `put()` 和 `get()` 的代码。

好，我们来弄清楚这个问题。设想两个生产者（Pa 和 Pb）几乎同时调用 `put()`。假设生产者 Pa 先运行，刚开始填充缓冲区的第一项（F1 行，`fill=0`）。Pa 还没来得及把 `fill` 计数器加到 1，就被中断（interrupted）了。生产者 Pb 开始运行，它在 F1 行也把数据放进了 `buffer` 的第 0 个元素，这意味着原来的数据被覆盖了！这可不行，我们不希望丢失生产者的任何数据。

### 解决办法：加上互斥

可以看到，我们这里忘了*互斥*（mutual exclusion）。填充缓冲区和递增缓冲区索引这两步构成一个临界区，必须小心保护起来。所以，我们请出老朋友二值信号量（binary semaphore），加上一些锁。图 31.11 是我们的尝试。

 © 2008–25, ARPACI-DUSSEAU

现在，我们在整个 put()/get() 部分的代码周围加了锁，见注释中标着“新增”的行。这个思路看起来是对的，但还是不行。为什么？死锁（deadlock）。为什么会发生死锁？花点时间想一想，试着找出一种会出现死锁的情况。要让程序死锁，必须依次发生哪些步骤？

### 避免死锁

好了，你已经想出来了，下面揭晓答案。设想有两个线程，一个生产者，一个消费者。消费者先运行。它获取互斥锁（C0 行），然后对 full 信号量调用 sem_wait()（C1 行）；因为还没有数据，这次调用让消费者阻塞，于是让出 CPU。但要注意，消费者仍然持有锁。

接着生产者运行。它有数据要生产，只要它能继续运行，就能唤醒消费者线程（consumer thread），一切都会顺利。遗憾的是，它做的第一件事就是对二值的 mutex 信号量调用 sem_wait()（P0 行），而锁已经被持有了。于是，生产者现在也卡在那里等待。

这里有一个简单的循环。消费者持有互斥锁（mutex），并在等待有人对 full 发信号（signal）；生产者本可以对 full 发信号，却在等待互斥锁。于是，生产者和消费者互相等待，都卡住了：这就是典型的死锁。

### 终于有了可行的方案

要解决这个问题，只需缩小锁的作用范围。图 31.12（第 336 页）给出了正确的方案。可以看到，我们只是把互斥锁的获取和释放移到紧挨着临界区的地方，并把对 full 和 empty 的等待和发信号代码留在锁的外面[2]。这样就得到了一个简单而可用的有界缓冲区（bounded buffer），这是多线程（multi-threaded）程序中很常用的模式。现在就弄懂它，以后用得上。未来很多年你都会感谢我们。至少，当期末考试或者面试中出现同样的问题时，你会感谢我们的。

## 31.5 读写锁

另一个经典问题源于这样一种需求：我们希望有一种更灵活的加锁（locking）原语，它考虑到对数据结构（data structure）的不同访问可能需要不同类型的锁。例如，设想一个链表上有许多并发操作，包括插入和简单的查找。插入会改变链表的状态（因此用传统的临界区是合理的），而查找只是读取数据结构；只要能保证没有正在进行的插入，就可以让多个查找同时进行。我们接下来要开发的这种特殊的锁就是为了支持这类操作，它叫作**读写锁**（reader-writer lock）[CHP71]。这种锁的代码见图 31.13（第 337 页）。

代码相当简单。如果某个线程想更新这个数据结构，它应该调用一对新的同步操作：用 rwlock_acquire_writelock() 获取写锁（write lock），用 rwlock_release_writelock() 释放写锁。这两个函数内部只是用 writelock 信号量来确保只有一个写者能获取锁，从而进入临界区更新数据结构。

更有意思的是获取和释放读锁（read lock）的那对例程。获取读锁（read lock）时，读者先获取 lock，然后递增 readers 变量，以记录当前有多少读者在访问数据结构。rwlock_acquire_readlock() 中接下来的关键一步发生在第一个读者获取锁的时候：这时，这

[2]其实，从模块化（modularity）的角度考虑，把互斥锁的获取和释放放在 put() 和 get() 函数内部可能更自然。

```
1  void *producer(void *arg) {
2      int i;
3      for (i = 0; i < loops; i++) {
4          sem_wait(&empty);          // P1 行
5          sem_wait(&mutex);          // P1.5 行（加锁）
6          put(i);                    // P2 行
7          sem_post(&mutex);          // P2.5 行（解锁）
8          sem_post(&full);           // P3 行
9      }
10 }
11
12 void *consumer(void *arg) {
13     int i;
14     for (i = 0; i < loops; i++) {
15         sem_wait(&full);           // C1 行
16         sem_wait(&mutex);          // C1.5 行（加锁）
17         int tmp = get();           // C2 行
18         sem_post(&mutex);          // C2.5 行（解锁）
19         sem_post(&empty);          // C3 行
20         printf("%d\n", tmp);
21     }
22 }
```

图 31.12：**加上互斥（正确的做法）**

个读者还要对 `writelock` 信号量调用 `sem_wait()`，获取写锁（write lock），然后再调用 `sem_post()` 释放 `lock`。

因此，一旦有读者获取了读锁（read lock），其他读者也可以获取读锁；但是，任何想获取写锁（write lock）的线程都必须等到所有读者都结束；最后一个退出临界区的读者会对 `writelock` 调用 `sem_post()`，从而让等待中的写者获取锁。

这种方法能（如愿）工作，但也有一些缺点，尤其是在公平性（fairness）方面。具体来说，读者很容易让写者饿死（starve）。这个问题有更精巧的解决方案；也许你能想出一种更好的实现？提示：想想一旦有写者在等待，怎样阻止更多读者进入。

最后要指出，读写锁要谨慎使用。它们往往带来更多开销（实现越复杂越是如此），所以和直接使用简单快速的加锁原语相比，最终并不能提升性能 [CB08]。无论如何，它们再次展示了信号量的一种有趣而有用的用法。

## 31.6 哲学家就餐问题

Dijkstra 提出并解决的并发问题中，最著名的一个叫作**哲学家就餐问题**（dining philosopher's problem）[D71]。这个问题之所以出名，是因为它好玩，在智力上也有些意思，不过实用价值不高。可它实在太有名了，我们不得不把它放进来；事实上，你可能在某次面试中被问到它，要是答不上来、没拿到工作，你一定会恨死你的操作系统教授。反过来，如果你拿到了工作，欢迎给你的操作系统教授寄一封感谢信，或者一些股票期权。

```
1  typedef struct _rwlock_t {
2      sem_t lock;       // 二值信号量（基本的锁）
3      sem_t writelock; // 允许一个写者/多个读者
4      int   readers;    // 临界区内的读者数
5  } rwlock_t;
6
7  void rwlock_init(rwlock_t *rw) {
8      rw->readers = 0;
9      sem_init(&rw->lock, 0, 1);
10     sem_init(&rw->writelock, 0, 1);
11 }
12
13 void rwlock_acquire_readlock(rwlock_t *rw) {
14     sem_wait(&rw->lock);
15     rw->readers++;
16     if (rw->readers == 1) // 第一个读者获取写锁
17         sem_wait(&rw->writelock);
18     sem_post(&rw->lock);
19 }
20
21 void rwlock_release_readlock(rwlock_t *rw) {
22     sem_wait(&rw->lock);
23     rw->readers--;
24     if (rw->readers == 0) // 最后一个读者释放写锁
25         sem_post(&rw->writelock);
26     sem_post(&rw->lock);
27 }
28
29 void rwlock_acquire_writelock(rwlock_t *rw) {
30     sem_wait(&rw->writelock);
31 }
32
33 void rwlock_release_writelock(rwlock_t *rw) {
34     sem_post(&rw->writelock);
35 }
```

图 31.13：**一个简单的读写锁**

这个问题的基本设定是这样的（见图 31.14）：假设有五位“哲学家”围坐在一张桌子旁。每两位哲学家之间放着一把叉子（因此一共五把）。哲学家们有时在思考，这时不需要叉子；有时在吃饭。哲学家要吃饭，需要两把叉子，也就是自己左边和右边的各一把。正是对这些叉子的争用（contention），以及由此产生的同步问题，让它成了我们在并发编程（concurrent programming）中研究的问题。

下面是每位哲学家的基本循环，假设每位哲学家都有一个唯一的线程标识符 `p`，取值为 0 到 4（含 4）：

提示：简单笨办法可能更好（Hill 定律）

永远不要低估这样一个观点：简单笨拙的办法可能就是最好的办法。就加锁而言，有时简单的自旋锁（spin lock）效果最好，因为它容易实现，速度又快。读写锁之类的东西听起来很酷，但它们很复杂，而复杂可能就意味着慢。所以，总是先试试简单笨拙的办法。这种崇尚简单的思想在很多地方都能看到。较早的一个出处是 Mark Hill 的博士论文 [H87]，它研究的是如何为 CPU 设计缓存（cache）。Hill 发现，简单的直接映射缓存（direct-mapped cache）比花哨的组相联（set-associative）设计效果更好（原因之一是，在缓存中，设计越简单，查找就越快）。Hill 把他的工作简洁地概括为："大而笨更好。"因此，我们把这条类似的建议叫作 **Hill 定律**（Hill's Law）。

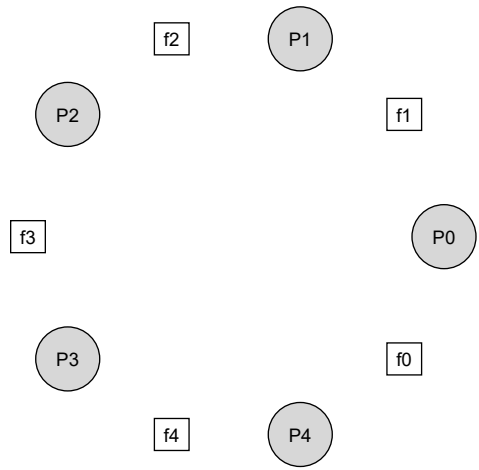



图 31.14：**哲学家就餐问题**

```
while (1) {
    think();
    get_forks(p);
    eat();
    put_forks(p);
}
```

关键的挑战在于，怎样编写 `get_forks()` 和 `put_forks()` 例程，才能做到不发生死锁、没有哪位哲学家会饿死（永远吃不上饭），并且并发度（concurrency）高（也就是让尽可能多的哲学家同时吃饭）。

参照 Downey 的解法 [D08]，我们会用几个辅助函数来求解。它们是：

```
int left(int p)  { return p; }
int right(int p) { return (p + 1) % 5; }
```

```
1 void get_forks(int p) {
2     sem_wait(&forks[left(p)]);
3     sem_wait(&forks[right(p)]);
4 }
5
6 void put_forks(int p) {
7     sem_post(&forks[left(p)]);
8     sem_post(&forks[right(p)]);
9 }
```

图 31.15：**get_forks() 和 put_forks() 例程**

哲学家 p 想引用左边的叉子时，只需调用 left(p)。类似地，调用 right(p) 就能引用哲学家 p 右边的叉子；其中的取模运算处理了一种特殊情况：最后一位哲学家（p=4）要拿右边的叉子，也就是叉子 0。

我们还需要一些信号量来解决这个问题。假设有五个信号量，每把叉子一个：sem_t forks[5]。

### 有问题的方案

我们来尝试第一个解法。假设把 forks 数组中的每个信号量都初始化为 1，并且每位哲学家都知道自己的编号（p）。于是可以写出 get_forks() 和 put_forks() 例程（图 31.15，第 339 页）。

这个（有问题的）方案的思路是：拿叉子时，对每把叉子各加一把“锁”，先拿左边的，再拿右边的。吃完以后，再把它们放下。很简单，对吧？可惜，在这里，简单就意味着行不通。你能看出会出现什么问题吗？想一想。

问题在于**死锁**（deadlock）。如果在任何一位哲学家拿到右边的叉子之前，每位哲学家都恰好先拿起了左边的叉子，那么每个人都会拿着一把叉子，永远等着另一把。具体来说，哲学家 0 拿起叉子 0，哲学家 1 拿起叉子 1，哲学家 2 拿起叉子 2，哲学家 3 拿起叉子 3，哲学家 4 拿起叉子 4；所有叉子都被拿走了，所有哲学家都卡在那里，等着另一位哲学家手里的叉子。我们很快会更详细地研究死锁；现在可以肯定地说，这不是一个可行的方案。

### 一种解决办法：打破依赖

解决这个问题最简单的办法，是改变至少一位哲学家拿叉子的方式；事实上，Dijkstra 本人就是这样解决的。具体来说，假设哲学家 4（编号最大的那位）拿叉子的顺序和其他人不同（图 31.16）；put_forks() 的代码保持不变。

因为最后一位哲学家先拿右边、再拿左边，所以不会出现每位哲学家都拿着一把叉子、卡着等另一把的情况；等待的循环被打破了。仔细想想这个方案会带来什么结果，说服自己它确实可行。

类似的“著名”问题还有一些，比如**吸烟者问题**（cigarette smoker's problem）和**睡眠理发师问题**（sleeping barber problem）。它们大多只是让人思考并发的由头，有些名字起得很有意思。如果你想了解更多，或者只是想多练习用并发的方式思考，可以去查一查 [D08]。

```
1 void get_forks(int p) {
2     if (p == 4) {
3         sem_wait(&forks[right(p)]);
4         sem_wait(&forks[left(p)]);
5     } else {
6         sem_wait(&forks[left(p)]);
7         sem_wait(&forks[right(p)]);
8     }
9 }
```

图 31.16：**打破 `get_forks()` 中的依赖**

## 31.7 线程节流

信号量还有另一个简单的用法，偶尔会用到，所以这里也介绍一下。具体问题是：程序员怎样防止“太多”线程同时做某件事，把系统拖垮？答案是：为“太多”定一个阈值，然后用信号量限制同时执行那段代码的线程数。我们把这种方法叫作**节流**（throttling）[T99]，并把它看作**准入控制**（admission control）的一种形式。

我们来看一个更具体的例子。设想你创建了几百个线程，并行地（in parallel）处理某个问题。然而，在代码的某个部分，每个线程都要申请大量内存来完成一部分计算；我们把这部分代码叫作*内存密集区*（memory-intensive region）。如果所有线程同时进入内存密集区，所有内存分配请求加起来就会超过机器的物理内存（physical memory）总量。结果，机器开始抖动（thrashing，也就是在内存和磁盘之间来回换页），整个计算就会慢得像蜗牛爬。

一个简单的信号量就能解决这个问题。把信号量的值初始化为你希望同时进入内存密集区的最大线程数，然后在这个区域前后放上 `sem_wait()` 和 `sem_post()`，信号量就能自然地限制同时处在这段危险代码中的线程数。

## 31.8 如何实现信号量

最后，我们用底层的同步原语，也就是锁和条件变量，来构建我们自己的信号量，它叫作……（*此处应有鼓声*）……**Zemaphore**。这项任务相当简单，见图 31.17（第 341 页）。

在这段代码中，我们只用了一把锁和一个条件变量，外加一个记录信号量值的状态变量（state variable）。自己研究这段代码，直到真正弄懂为止。动手吧！

提示：泛化要小心

在系统设计中，泛化这种抽象（abstraction）技术可能相当有用：一个好的想法稍微扩展一下，就能解决更大一类问题。不过，泛化时要小心；正如 Lampson 告诫我们的：“不要泛化；泛化的东西一般都是错的。”[L83]

可以把信号量看作锁和条件变量的泛化；但是，真的需要这样的泛化吗？而且，考虑到在信号量之上实现条件变量的难度，也许这种泛化并不像你想的那样“泛”。

 © 2008–25, ARPACI-DUSSEAU

```
typedef struct __Zem_t {
    int value;
    pthread_cond_t cond;
    pthread_mutex_t lock;
} Zem_t;

// 只能由一个线程调用
void Zem_init(Zem_t *s, int value) {
    s->value = value;
    Cond_init(&s->cond);
    Mutex_init(&s->lock);
}

void Zem_wait(Zem_t *s) {
    Mutex_lock(&s->lock);
    while (s->value <= 0)
        Cond_wait(&s->cond, &s->lock);
    s->value--;
    Mutex_unlock(&s->lock);
}

void Zem_post(Zem_t *s) {
    Mutex_lock(&s->lock);
    s->value++;
    Cond_signal(&s->cond);
    Mutex_unlock(&s->lock);
}
```

图 31.17：**用锁和条件变量实现 Zemaphore**

我们的 Zemaphore 和 Dijkstra 定义的纯粹的信号量有一个细微的区别：我们没有维持“信号量的值为负时反映等待线程数”这个不变式（invariant）；实际上，这个值永远不会小于零。这种行为更容易实现，也和目前 Linux 的实现一致。

有意思的是，用信号量构建条件变量要棘手得多。一些经验丰富的并发程序员曾在 Windows 环境中尝试这样做，结果出了各种各样的 bug [B04]。你自己试试，看能不能弄明白，为什么用信号量构建条件变量比看上去更有挑战性。

## 31.9 小结

信号量是编写并发程序的一种强大而灵活的原语。由于它简单实用，有些程序员只用信号量，完全不用锁和条件变量。

本章只介绍了几个经典问题及其解法。如果你有兴趣了解更多，可以参考很多其他资料。一份很好的（而且免费的）参考资料是 Allen Downey 讲并发和信号量编程的书 [D08]。这本书里有很多谜题，做一做可以加深你对信号量以及整个并发的理解。要成为真正的并发专家需要多年的努力；想掌握这个主题，无疑要学到本课程之外去。

## 参考文献

[B04] "Implementing Condition Variables with Semaphores" by Andrew Birrell. December 2004. *An interesting read on how difficult implementing CVs on top of semaphores really is, and the mistakes the author and co-workers made along the way. Particularly relevant because the group had done a ton of concurrent programming; Birrell, for example, is known for (among other things) writing various thread-programming guides.*

[CB08] "Real-world Concurrency" by Bryan Cantrill, Jeff Bonwick. ACM Queue. Volume 6, No. 5. September 2008. *A nice article by some kernel hackers from a company formerly known as Sun on the real problems faced in concurrent code.*

[CHP71] "Concurrent Control with Readers and Writers" by P.J. Courtois, F. Heymans, D.L. Parnas. Communications of the ACM, 14:10, October 1971. *The introduction of the reader-writer problem, and a simple solution. Later work introduced more complex solutions, skipped here because, well, they are pretty complex.*

[D59] "A Note on Two Problems in Connexion with Graphs" by E. W. Dijkstra. Numerische Mathematik 1, 269–271, 1959. Available: http://www-m3.ma.tum.de/twiki/pub/MN0506/WebHome/dijkstra.pdf. *Can you believe people worked on algorithms in 1959? We can't. Even before computers were any fun to use, these people had a sense that they would transform the world…*

[D68a] "Go-to Statement Considered Harmful" by E.W. Dijkstra. CACM, volume 11(3), March 1968. http://www.cs.utexas.edu/users/EWD/ewd02xx/EWD215.PDF. *Sometimes thought of as the beginning of the field of software engineering.*

[D68b] "The Structure of the THE Multiprogramming System" by E.W. Dijkstra. CACM, volume 11(5), 1968. *One of the earliest papers to point out that systems work in computer science is an engaging intellectual endeavor. Also argues strongly for modularity in the form of layered systems.*

[D72] "Information Streams Sharing a Finite Buffer" by E.W. Dijkstra. Information Processing Letters 1, 1972. http://www.cs.utexas.edu/users/EWD/ewd03xx/EWD329.PDF. *Did Dijkstra invent everything? No, but maybe close. He certainly was the first to clearly write down what the problems were in concurrent code. However, practitioners in OS design knew of many of the problems described by Dijkstra, so perhaps giving him too much credit would be a misrepresentation.*

[D08] "The Little Book of Semaphores" by A.B. Downey. Available at the following site: http://greenteapress.com/semaphores/. *A nice (and free!) book about semaphores. Lots of fun problems to solve, if you like that sort of thing.*

[D71] "Hierarchical ordering of sequential processes" by E.W. Dijkstra. Available online here: http://www.cs.utexas.edu/users/EWD/ewd03xx/EWD310.PDF. *Presents numerous concurrency problems, including Dining Philosophers. The wikipedia page about this problem is also useful.*

[GR92] "Transaction Processing: Concepts and Techniques" by Jim Gray, Andreas Reuter. Morgan Kaufmann, September 1992. *The exact quote that we find particularly humorous is found on page 485, at the top of Section 8.8: "The first multiprocessors, circa 1960, had test and set instructions … presumably the OS implementors worked out the appropriate algorithms, although Dijkstra is generally credited with inventing semaphores many years later." Oh, snap!*

[H87] "Aspects of Cache Memory and Instruction Buffer Performance" by Mark D. Hill. Ph.D. Dissertation, U.C. Berkeley, 1987. *Hill's dissertation work, for those obsessed with caching in early systems. A great example of a quantitative dissertation.*

[L83] "Hints for Computer Systems Design" by Butler Lampson. ACM Operating Systems Review, 15:5, October 1983. *Lampson, a famous systems researcher, loved using hints in the design of computer systems. A hint is something that is often correct but can be wrong; in this use, a `signal()` is telling a waiting thread that it changed the condition that the waiter was waiting on, but not to trust that the condition will be in the desired state when the waiting thread wakes up. In this paper about hints for designing systems, one of Lampson's general hints is that you should use hints. It is not as confusing as it sounds.*

[T99] "Re: NT kernel guy playing with Linux" by Linus Torvalds. June 27, 1999. Available: https://yarchive.net/comp/linux/semaphores.html. *A response from Linus himself about the utility of semaphores, including the throttling case we mention in the text. As always, Linus is slightly insulting but quite informative.*

## 作业（编程）

在这个作业中，我们要用信号量解决一些著名的并发问题。其中很多取自 Downey 那本出色的《Little Book of Semaphores》[3]，这本书很好地汇集了许多经典问题，还介绍了几个新的变体；感兴趣的读者可以去看看这本小书，找更多乐趣。

下面每道题都给出了代码骨架；你的任务是用信号量把代码补全，让它正常工作。在 Linux 上，你可以使用原生的信号量；Mac 不支持信号量，所以在 Mac 上，你得先像本章描述的那样，用锁和条件变量自己构建一个实现。祝你好运！

## 问题

1. 第一个问题只是实现并测试书中描述的 **fork/join 问题**的解法。虽然书中已经讲过这个解法，但亲手把它敲一遍还是值得的；就连巴赫也改写过维瓦尔第的作品，好让一位未来的大师向一位成名的大师学习。详见 `fork-join.c`。在子线程中加上 `sleep(1)` 调用，确保它确实能正常工作。
2. 现在把前面的问题稍微推广一点，研究**会合问题**（rendezvous problem）。问题是这样的：有两个线程，每个线程都即将进入代码中的会合点。在另一个线程进入这部分代码之前，哪个线程都不应该离开它。考虑用两个信号量完成这个任务，详见 `rendezvous.c`。
3. 再进一步，实现**屏障同步**（barrier synchronization）的通用解法。假设一段顺序执行的代码中有两个点，分别叫作 P1 和 P2。在 P1 和 P2 之间放一道**屏障**（barrier），就能保证所有线程都执行完 P1 之后，才会有线程执行 P2。你的任务是：编写代码，实现一个可以这样使用的 `barrier()` 函数。可以假设你知道运行中的程序（running program）里线程的总数 N，并且所有 N 个线程都会试图进入屏障（barrier）。同样，你很可能需要用两个信号量，再加上几个用来计数的整数，来实现这个方案。详见 `barrier.c`。
4. 现在来解决**读者-写者问题**（reader-writer problem），书中同样描述过它。这一次先不用担心饥饿（starvation）。详见 `reader-writer.c` 中的代码。在代码中加上 `sleep()` 调用，证明它能按你的预期工作。你能展示出饥饿问题的存在吗？
5. 我们再来看一次读者-写者问题（reader-writer problem），但这次要考虑饥饿。怎样保证所有读者和写者最终都能取得进展？详见 `reader-writer-nostarve.c`。
6. 用信号量构建一个**无饥饿互斥锁**（no-starve mutex），让任何试图获取它的线程最终都能获得它。更多信息见 `mutex-nostarve.c` 中的代码。
7. 喜欢这些问题吗？Downey 的免费书里还有很多同类问题。别忘了，玩得开心！不过，写代码的时候你总是很开心的，不是吗？

[3]地址：http://greenteapress.com/semaphores/。

# 32

# 常见并发问题

多年来，研究人员花了大量时间和精力研究并发 bug（concurrency bug）。早期的大部分工作都集中在**死锁**（deadlock）上，前几章已经提到过这个话题，本章会深入讨论它 [C+71]。较新的工作则着重研究其他类型的常见并发 bug，即非死锁 bug。本章简要介绍在真实代码库中发现的几个并发（concurrency）问题，好让我们更清楚该提防什么。这就引出了本章的核心问题：

关键问题：如何处理常见的并发 bug
并发 bug 往往呈现为几种常见的模式。知道该提防哪些模式，是写出更健壮、更正确的并发代码（concurrent code）的第一步。

## 32.1 有哪些类型的 bug?

第一个、也是最明显的问题是：复杂的并发程序（concurrent program）中会出现哪些类型的并发 bug? 这个问题一般很难回答，好在已经有人替我们做了这项工作。具体来说，我们依据的是 Lu 等人的一项研究 [L+08]，他们非常细致地分析了一批流行的并发应用程序，弄清了实践中会出现哪些类型的 bug。

这项研究着眼于四个重要的大型开源应用程序：MySQL（流行的数据库管理系统）、Apache（知名的 Web 服务器）、Mozilla（著名的 Web 浏览器）和 OpenOffice（MS Office 套件的免费版本，确实有人在用）。作者考察了这些代码库中已经发现并修复的并发 bug，从开发者的这些工作中整理出了一份定量的 bug 分析。这些结果能帮你看清成熟的代码库中实际会出现哪些问题。

| 应用程序 | 用途 | 非死锁 | 死锁 |
|---|---|---|---|
| MySQL | 数据库服务器 | 14 | 9 |
| Apache | Web 服务器 | 13 | 4 |
| Mozilla | Web 浏览器 | 41 | 16 |
| OpenOffice | 办公套件 | 6 | 2 |
| 总计 | | 74 | 31 |

图 32.1：**现代应用程序中的 bug**

图 32.1 汇总了 Lu 和同事研究过的 bug。可以看到，bug 一共有 105 个，其中大多数（74 个）是非死锁 bug，其余 31 个是死锁 bug。此外，还可以看到每个应用程序各研究了多少个 bug：OpenOffice 总共只有 8 个并发 bug，而 Mozilla 有将近 60 个。

接下来，我们更深入地讨论非死锁和死锁这两类 bug。第一类是非死锁 bug，我们借助这项研究中的例子来展开讨论；第二类是死锁 bug（deadlock bug），我们介绍人们为预防、避免或处理死锁做过的大量工作。

## 32.2 非死锁 bug

根据 Lu 的研究，非死锁 bug 占了并发 bug 的大多数。但这些 bug 都是什么类型？它们是怎么产生的？又该怎么修复？下面讨论 Lu 等人发现的两大类非死锁 bug：**原子性违反**（atomicity violation）bug 和**顺序违反**（order violation）bug。

### 原子性违反 bug

我们遇到的第一类问题叫作**原子性违反**（atomicity violation）。下面是在 MySQL 中发现的一个简单例子。在看解释之前，先试着找出 bug 在哪里。动手试试！

```
1 Thread 1::
2 if (thd->proc_info) {
3   fputs(thd->proc_info, ...);
4 }
5
6 Thread 2::
7 thd->proc_info = NULL;
```

图 32.2：**原子性违反（`atomicity.c`）**

在这个例子中，两个线程（thread）都会访问结构体 `thd` 中的字段 `proc_info`。第一个线程检查这个值是否非 NULL，然后打印它；第二个线程把它设为 NULL。显然，如果第一个线程做完检查、但在调用 `fputs` 之前被中断（interrupted），第二个线程就可能在这中间运行，把指针设为 NULL；等第一个线程恢复运行时，`fputs` 会解引用一个 NULL 指针，程序随即崩溃。

按照 Lu 等人的说法，原子性违反（atomicity violation）更正式的定义是："多次内存访问之间期望的可串行性（serializability）被破坏了，即某段代码本应是原子的（atomic），但执行时并没有保证原子性。"在上面的例子中，代码对"检查 `proc_info` 是否非 NULL"和"在 `fputs()` 调用中使用 `proc_info`"这两步有一个*原子性假设*（atomicity assumption，这是 Lu 的说法）；一旦这个假设不成立，代码就无法按预期工作。

这类问题的修复方法通常（但并不总是）很简单。你能想到怎么修复上面的代码吗？

在图 32.3 的解决方案中，我们只是在引用共享变量的地方前后加锁（locking），确保两个线程访问 `proc_info` 字段时都持有锁（`proc_info_lock`）。当然，其他访问这个结构体的代码，也应该先获取这把锁（lock）再访问。

```
pthread_mutex_t proc_info_lock =
  PTHREAD_MUTEX_INITIALIZER;

Thread 1::
pthread_mutex_lock(&proc_info_lock);
if (thd->proc_info) {
  fputs(thd->proc_info, ...);
}
pthread_mutex_unlock(&proc_info_lock);

Thread 2::
pthread_mutex_lock(&proc_info_lock);
thd->proc_info = NULL;
pthread_mutex_unlock(&proc_info_lock);
```

图 32.3：**修复后的原子性违反（`atomicity_fixed.c`）**

### 顺序违反 bug

Lu 等人发现的另一类常见非死锁 bug 叫作**顺序违反**（order violation）。再看一个简单的例子。同样，看看你能不能找出下面的代码为什么有 bug。

```
Thread 1::
void init() {
  mThread = PR_CreateThread(mMain, ...);
}

Thread 2::
void mMain(...) {
  mState = mThread->State;
}
```

图 32.4：**顺序 bug（`ordering.c`）**

你可能已经发现了，线程 2 的代码似乎假定变量 `mThread` 已经初始化（而且不是 NULL）；然而，如果线程 2 一创建就立即运行，那么它在 `mMain()` 中访问 `mThread` 时，`mThread` 的值还没有设置好，程序很可能因为解引用 NULL 指针而崩溃。注意，我们假设 `mThread` 的初值是 NULL；如果不是，情况可能会更离奇，因为线程 2 会通过这次解引用访问任意的内存位置。

顺序违反（order violation）更正式的定义是：“两次（或两组）内存访问之间期望的顺序被颠倒了（即 A 应该总在 B 之前执行，但执行时并没有保证这个顺序）”[L+08]。

修复这类 bug 的办法一般是强制规定顺序。前面讨论过，使用**条件变量**（condition variable）可以简单而稳妥地在现代代码库中加入这种同步。因此，上面的例子可以改写成图 32.5 那样。

在修复后的代码中，我们加了一个条件变量 `mtCond` 和对应的锁 `mtLock`，还有一个状态变量（state variable）`mtInit`。初始化代码运行时，会把 `mtInit` 的状态设为 1，并发

```
1  pthread_mutex_t mtLock = PTHREAD_MUTEX_INITIALIZER;
2  pthread_cond_t  mtCond = PTHREAD_COND_INITIALIZER;
3  int mtInit             = 0;
4
5  Thread 1::
6  void init() {
7      ...
8      mThread = PR_CreateThread(mMain, ...);
9
10     // 发信号，表示线程已经创建好了……
11     pthread_mutex_lock(&mtLock);
12     mtInit = 1;
13     pthread_cond_signal(&mtCond);
14     pthread_mutex_unlock(&mtLock);
15     ...
16 }
17
18 Thread 2::
19 void mMain(...) {
20     ...
21     // 等待线程初始化完成……
22     pthread_mutex_lock(&mtLock);
23     while (mtInit == 0)
24         pthread_cond_wait(&mtCond, &mtLock);
25     pthread_mutex_unlock(&mtLock);
26
27     mState = mThread->State;
28     ...
29 }
```

图 32.5：**修复顺序违反（`ordering_fixed.c`）**

出信号（signal），表示已经完成。如果线程 2 在此之前就已运行，它会等待这个信号和相应的状态变化；如果它在此之后才运行，就会检查状态，发现初始化已经完成（即 `mtInit` 已设为 1），于是照常继续执行。注意，我们其实也可以直接把 `mThread` 当作状态变量（state variable），但为了简单起见，这里没有这样做。线程之间的顺序很重要时，条件变量或信号量（semaphore）就能派上用场。

### 非死锁 bug：小结

Lu 等人研究的非死锁 bug 中，很大一部分（97%）不是原子性违反（atomicity violation）就是顺序违反（order violation）。因此，只要仔细思考这两种 bug 模式，程序员很可能就能更好地避免它们。此外，今后出现的自动代码检查工具，大概也应该重点关注这两类 bug，因为在已部署的软件中发现的非死锁 bug，绝大部分都属于这两类。

遗憾的是，并不是所有 bug 都像上面的例子这样容易修复。要修好有些 bug，得更深入地理解程序在做什么，或者大规模重组代码或数据结构（data structure）。更多细节请阅读 Lu 等人那篇出色的（而且好读的）论文。

## 32.3 死锁 bug

除了上面提到的并发 bug，在许多加锁协议（locking protocol）复杂的并发系统中，还会出现一个经典问题，叫作**死锁**（deadlock）。例如，线程 1 持有锁 L1，同时在等另一把锁 L2；不巧的是，持有 L2 的线程 2 又在等 L1 被释放。这时就会发生死锁。下面这段代码演示了这种潜在的死锁：

```
Thread 1:                       Thread 2:
pthread_mutex_lock(L1);         pthread_mutex_lock(L2);
pthread_mutex_lock(L2);         pthread_mutex_lock(L1);
```

图 32.6：**简单的死锁（`deadlock.c`）**

注意，这段代码运行时并不一定会发生死锁，只是有可能发生。例如，线程 1 拿到锁 L1 后，发生了一次上下文切换（context switch），换成线程 2 运行。这时线程 2 拿到 L2，又试图获取 L1。于是就发生了死锁，因为两个线程都在等对方，谁都无法运行。图 32.7 用图形描述了这种情况；图中出现了**环**（cycle），就表明发生了死锁。

这张图应该把问题说清楚了。那么，程序员写代码时该怎样应对死锁呢？

关键问题：如何应对死锁
我们应该如何构建系统，来预防、避免死锁，或者至少能检测出死锁并从中恢复？在今天的系统中，死锁是一个真实存在的问题吗？

### 为什么会发生死锁？

你可能会觉得，上面那种简单的死锁很容易避免。例如，只要线程 1 和线程 2 都按相同的顺序获取锁，就永远不会发生死锁。那么，死锁为什么还会发生呢？

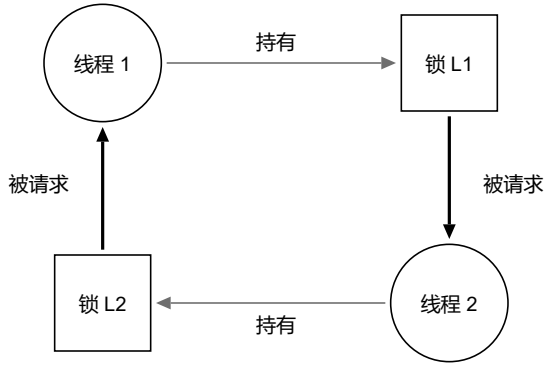



图 32.7：**死锁依赖图**

一个原因是，在大型代码库中，组件之间会形成复杂的依赖关系。以操作系统为例：虚拟内存（virtual memory）系统可能需要访问文件系统（file system），才能从磁盘调入一个块（block）；而文件系统随后可能又需要一页内存来读入这个块，于是又要找虚拟内存系统。因此，在大型系统中设计加锁策略（locking strategy）时必须非常小心，以免代码中自然形成的循环依赖导致死锁。

另一个原因在于**封装**（encapsulation）的本质。作为软件开发者，我们学的是隐藏实现细节，让软件更容易做到模块化（modularity）。遗憾的是，这种模块化和加锁配合得并不好。正如 Jula 等人指出的 [J+08]，有些看似无害的接口简直是在引诱你陷入死锁。以 Java 的 Vector 类和它的方法 `AddAll()` 为例，这个例程的调用方式如下：

```
Vector v1, v2;
v1.AddAll(v2);
```

由于这个方法需要做到多线程安全（multi-thread safe），所以在内部，它既要获取接收添加内容的那个向量（`v1`）的锁，也要获取参数（`v2`）的锁。为了把 `v2` 的内容添加到 `v1`，这个例程按某种任意定下的顺序（比如先 `v1` 后 `v2`）获取这两把锁。如果另一个线程几乎在同一时刻调用了 `v2.AddAll(v1)`，就有可能发生死锁，而调用它的应用程序对此几乎毫无察觉。

### 产生死锁的条件

要发生死锁，必须同时满足四个条件 [C+71]：

- **互斥**（mutual exclusion）：线程独占它们所需的资源（例如，一个线程拿到了一把锁）。
- **持有并等待**（hold-and-wait）：线程持有已分配给它们的资源（例如已经获得的锁），同时还在等待其他资源（例如它们想要获取的锁）。
- **不可抢占**（no preemption）：不能把资源（例如锁）从持有它的线程那里强行夺走。
- **循环等待**（circular wait）：存在一条由线程组成的环形链，链中的每个线程都持有一个或多个资源（例如锁），而链中的下一个线程正在请求这些资源。

这四个条件中只要有一个不满足，就不会发生死锁。因此，我们先来探讨预防死锁的技术；每种策略都设法阻止上面某一个条件成立，各自都是处理死锁问题的一种办法。

### 预防

### 循环等待

最实用的预防技术（当然也是经常采用的一种）大概是：编写加锁代码时，确保永远不会引起循环等待（circular wait）。最简单的做法是为获取锁规定一个**全序**（total ordering）。例如，如果系统中只有两把锁（L1 和 L2），只要始终先获取 L1、再获取 L2，就能预防死锁。有了这种严格的顺序，就不会出现循环等待（circular wait），也就不会有死锁。

当然，在更复杂的系统中，锁会远不止两把，因此可能很难让所有锁形成全序（而且可能也没必要）。这时，**偏序**（partial ordering）就是安排获取锁的一种有效方式，可以用来避免死锁。Linux 的内存映射（memory mapping）代码 [T+94]（v5.2）就是偏序（partial ordering）加锁的一个绝佳的真实例子：源代码顶部的注释列出了十组不同的获取锁顺

提示：按锁的地址规定加锁顺序

有时候，一个函数必须获取两把（或更多）锁；我们知道，这时必须小心，否则可能发生死锁。设想有一个函数，调用方式如下：`do_something(mutex_t *m1, mutex_t *m2)`。如果代码总是先获取 `m1` 再获取 `m2`（或者总是先 `m2` 后 `m1`），就可能发生死锁，因为一个线程可能调用 `do_something(L1, L2)`，而另一个线程同时调用 `do_something(L2, L1)`。

为了避免这个问题，聪明的程序员可以用每把锁的地址来规定获取锁的顺序。只要按地址从高到低或者从低到高的顺序获取锁，无论传入的锁是什么顺序，`do_something()` 都能保证总以相同的顺序获取它们。代码大概是这样的：

```
if (m1 > m2) { // 按地址从高到低的顺序获取
  pthread_mutex_lock(m1);
  pthread_mutex_lock(m2);
} else {
  pthread_mutex_lock(m2);
  pthread_mutex_lock(m1);
}
// 代码假定 m1 != m2（不是同一把锁）
```

用这个小技巧，程序员就能简单高效地获取多把锁，而且不会死锁。

序，既有像“i_mutex 先于 i_mmap_rwsem”这样简单的，也有像“i_mmap_rwsem 先于 private_lock 先于 swap_lock 先于 i_pages lock”这样更复杂的。

可以想象，无论全序（total ordering）还是偏序（partial ordering），都需要精心设计加锁策略（locking strategy），构建时必须格外小心。此外，顺序只是一种约定，粗心的程序员很容易无视加锁协议（locking protocol），从而可能导致死锁。最后，规定加锁顺序（lock ordering）需要深入理解代码库，弄清各个例程之间的调用关系；只要犯一个错误，就可能招来那个以“D”开头的词[1]。

### 持有并等待

要消除死锁所需的持有并等待（hold-and-wait）条件，可以一次性、原子地（atomically）获取所有锁。实践中可以这样实现：

```
1 pthread_mutex_lock(prevention); // 开始获取
2 pthread_mutex_lock(L1);
3 pthread_mutex_lock(L2);
4 ...
5 pthread_mutex_unlock(prevention); // 结束
```

这段代码先获取锁 `prevention`，保证了获取锁的过程中不会发生不合时宜的线程切换，于是又一次避免了死锁。当然，这要求任何线程在获取任何锁时，都要先获取全局的

[1]提示：“D”指的是“Deadlock”（死锁）。

prevention 锁。例如，如果另一个线程试图按不同的顺序获取锁 L1 和 L2，也不会有问题，因为它这样做时持有 prevention 锁。

注意，这个方案有不少问题。和前面一样，封装又在跟我们作对：调用一个例程时，这种方法要求我们确切地知道必须持有哪些锁，并提前获取它们。这种技术还可能降低并发性（concurrency），因为所有锁都必须提前（一次性）获取，而不是在真正需要的时候才获取。

### 不可抢占

我们一般认为，锁一旦拿到，就会一直持有到调用解锁（unlock）为止。获取多把锁之所以常常惹麻烦，是因为在等一把锁的时候，我们手里还拿着另一把。许多线程库（thread library）提供了一组更灵活的接口来帮助避免这种情况。具体来说，例程 pthread_mutex_trylock() 要么在锁可用时获取锁并返回成功，要么返回一个错误码，表示锁已被别人持有；在后一种情况下，如果你还想获取这把锁，可以稍后再试。

借助这个接口，可以像下面这样构建一个不会死锁、而且不受加锁顺序（lock ordering）影响的获取锁协议（protocol）：

```
1 top:
2   pthread_mutex_lock(L1);
3   if (pthread_mutex_trylock(L2) != 0) {
4       pthread_mutex_unlock(L1);
5       goto top;
6   }
```

注意，另一个线程可以遵循同样的协议，但按另一种顺序（先 L2 后 L1）获取锁，程序仍然不会死锁。不过，这确实带来了一个新问题：**活锁**（livelock）。两个线程可能（虽然概率不大）都在反复尝试这个序列，却总是没法同时拿到两把锁。在这种情况下，双方都在一遍又一遍地运行这段代码，所以这不是死锁，却没有任何进展，“活锁（livelock）”这个名字就是这么来的。活锁（livelock）问题也有解决办法：例如，可以在跳回去重新尝试整个过程之前，加一段随机延迟，从而降低竞争线程之间反复相互干扰的概率。

关于这个方案还要补充一点：它略过了 trylock 方法真正难办的地方。第一个难点仍然来自封装：如果某把锁深埋在被调用的某个例程里，跳回开头就更难实现了。如果代码在这个过程中已经获取了 L1 以外的一些资源，就必须确保把它们也小心地释放掉；例如，如果代码在获取 L1 之后分配了一些内存，那么在获取 L2 失败时，就必须先释放这些内存，再跳回开头重试（retry）整个序列。不过，在有限的场合下（例如前面提到的 Java vector 方法），这种方法可以很好地工作。

你可能还注意到，这种方法并没有真正引入抢占（即从拥有锁的线程那里强行夺走锁），而是利用 trylock 方法，让开发者能够优雅地放弃锁的所有权（也就是抢占自己的所有权）。不过这是一种实用的方法，所以尽管它在这方面并不完美，我们还是把它放在这里。

### 互斥

最后一种预防技术是干脆不需要互斥。一般来说，我们知道这很难做到，因为要运行的代码确实有临界区（critical section）。那么我们能做什么呢？

Herlihy 提出，可以设计出完全不用锁的各种数据结构 [H91, H93]。这些**无锁**（lock-free）方法和与之相关的**无等待**（wait-free）方法，背后的思想很简单：利用强大的硬件指令，就能构建出不需要显式加锁的数据结构。

举个简单的例子，假设我们有一条比较并交换（compare-and-swap）指令。你可能还记得，这是硬件提供的一条原子指令（atomic instruction），它做的事情如下：

```
1 int CompareAndSwap(int *address, int expected, int new) {
2     if (*address == expected) {
3         *address = new;
4         return 1; // 成功
5     }
6     return 0; // 失败
7 }
```

现在，假设我们想用比较并交换（compare-and-swap），把某个值原子地增加一定的量。可以用下面这个简单的函数做到：

```
1 void AtomicIncrement(int *value, int amount) {
2   do {
3     int old = *value;
4   } while (CompareAndSwap(value, old, old + amount) == 0);
5 }
```

这里没有先获取锁、更新、再释放锁，而是换了一种做法：用比较并交换（compare-and-swap）反复尝试把值更新为新值。这样就不需要获取锁，也就不会发生死锁；不过仍有可能发生活锁（livelock），因此健壮的解决方案会比上面这段简单代码复杂。

我们再看一个稍微复杂一点的例子：链表（linked list）插入。下面是在链表头部插入节点的代码：

```
1 void insert(int value) {
2   node_t *n = malloc(sizeof(node_t));
3   assert(n != NULL);
4   n->value = value;
5   n->next  = head;
6   head     = n;
7 }
```

这段代码执行的是简单的插入操作，但如果多个线程“同时”调用它，就会出现竞态条件（race condition，也叫竞争条件）。你能想出原因吗？（画一画：如果两个插入操作并发执行，链表会变成什么样？像往常一样，假设调度交错是恶意的。）当然，我们可以在这段代码前后获取和释放锁来解决这个问题：

```
1 void insert(int value) {
2   node_t *n = malloc(sizeof(node_t));
3   assert(n != NULL);
4   n->value = value;
5   pthread_mutex_lock(listlock);    // 临界区开始
6   n->next  = head;
7   head     = n;
```

```
8     pthread_mutex_unlock(listlock); // 临界区结束
9   }
```

在这个解决方案中，我们按传统的做法使用锁[2]。现在，我们换个做法，试试只用比较并交换（compare-and-swap）指令，用无锁（lock-free）的办法完成插入。下面是一种可能的写法：

```
1  void insert(int value) {
2    node_t *n = malloc(sizeof(node_t));
3    assert(n != NULL);
4    n->value = value;
5    do {
6      n->next = head;
7    } while (CompareAndSwap(&head, n->next, n) == 0);
8  }
```

这段代码先把 next 指针更新为指向当前的头节点，然后尝试把新创建的节点交换到链表头的位置。然而，如果在此期间有其他线程已经成功换入（swap in）了新的头节点，这次交换就会失败，于是这个线程会拿新的头节点再试一次。

当然，有用的链表光有插入操作是不够的；毫不奇怪，要构建一个支持插入、删除和查找的无锁（lock-free）链表，绝非易事。想了解更多，请阅读关于无锁（lock-free）和无等待（wait-free）同步的大量文献 [H01, H91, H93]。

### 通过调度避免死锁

在某些场景下，比起预防死锁，**避免**（avoidance）死锁更合适。避免死锁需要掌握一些全局信息，知道各个线程在执行过程中可能会获取哪些锁，然后据此调度这些线程，保证不会发生死锁。

例如，假设有两个处理器，要在上面调度四个线程。再假设我们知道：线程 1（T1）会在执行过程中的某个时刻、按某种顺序获取锁 L1 和 L2，T2 也会获取 L1 和 L2，T3 只获取 L2，而 T4 根本不获取锁。我们可以用表格列出这些线程的获取锁需求：

| | T1 | T2 | T3 | T4 |
|---|---|---|---|---|
| L1 | 是 | 是 | 否 | 否 |
| L2 | 是 | 是 | 是 | 否 |

于是，聪明的调度程序（scheduler）可以算出：只要 T1 和 T2 不同时运行，就绝不会发生死锁。下面就是一种这样的调度：




注意，T3 和 T1 重叠运行、或者 T3 和 T2 重叠运行，都没有问题。虽然 T3 会获取锁 L2，但它只获取一把锁，所以和其他线程并发运行时，永远不会导致死锁。

[2]细心的读者可能会问：为什么这么晚才获取锁，而不是一进入 `insert()` 就获取？细心的读者，你能想明白为什么这样做很可能是正确的吗？比如说，这段代码对 `malloc()` 的调用做了什么假设？

提示：不必事事做到完美（Tom West 定律）

Tom West 是经典计算机行业著作《新机器的灵魂》（Soul of a New Machine）[K81] 的主人公，他有一句名言："并非所有值得做的事都值得做好"，这是一条绝妙的工程格言。如果一件坏事很少发生，当然不应该花很大力气去预防它，尤其是在这件坏事发生的代价很小的时候。反过来，如果你在造航天飞机，出问题的代价是航天飞机爆炸，那么，嗯，也许你应该无视这条建议。

有些读者反对说："这听起来像是在建议用平庸来解决问题！"也许他们是对的，对这类建议确实应该谨慎。不过从我们的经验看，在工程界，面对紧迫的截止期限和其他现实问题，人们总要决定系统的哪些方面要做好，哪些方面先放一放。难点在于知道什么时候该做什么，这种眼光只有靠经验、靠全身心投入手头的工作才能获得。

我们再看一个例子。这一次，对相同资源（还是锁 L1 和 L2）的争用（contention）更激烈，如下面的争用表所示：

| | T1 | T2 | T3 | T4 |
|---|---|---|---|---|
| L1 | 是 | 是 | 是 | 否 |
| L2 | 是 | 是 | 是 | 否 |

具体来说，线程 T1、T2 和 T3 在执行过程中的某个时刻，都需要获取 L1 和 L2 这两把锁。下面是一种能保证永远不会发生死锁的调度：




可以看到，静态调度导致了一种保守的做法：T1、T2 和 T3 全都在同一个处理器上运行，于是完成这些任务的总时间大大延长了。这些任务本来也许可以并发运行，但出于对死锁的担心，我们没有这样做，代价就是性能。

这类方法中一个著名的例子是 Dijkstra 的银行家算法（Banker's Algorithm）[D64]，文献中还描述过许多类似的方法。遗憾的是，它们只在非常有限的环境中有用，例如嵌入式系统，在那里你完全清楚要运行哪些任务、它们各自需要哪些锁。此外，正如上面第二个例子所示，这类方法会限制并发性。因此，通过调度来避免死锁并不是一种广泛使用的通用方案。

### 检测与恢复

最后一种通用策略是：允许偶尔发生死锁，一旦检测到死锁再采取行动。例如，如果一个操作系统一年死机一次，你只要重启它，然后高高兴兴（或者骂骂咧咧）地继续干活就行了。如果死锁很少发生，这种算不上解决办法的办法其实相当务实。

许多数据库系统都采用死锁检测与恢复（deadlock detection and recovery）技术。死锁检测器（deadlock detector）定期运行，构建资源分配图（resource graph），检查其中有没有环。一旦出现环（cycle），也就是发生了死锁，系统就需要重启。如果需要先对数据结构做更复杂的修复，可能还得有人介入，帮着把这个过程理顺。

关于数据库并发、死锁及相关问题的更多细节，可以参考其他资料 [B+87, K87]。读读这些著作，或者更好的办法是去上一门数据库课程，深入了解这个丰富而有趣的话题。

## 32.4 小结

在本章中，我们研究了并发程序中会出现的几类 bug。第一类是非死锁 bug，它们出奇地常见，但通常比较容易修复。它们包括原子性违反（atomicity violation，本应一起执行的一串指令没有一起执行）和顺序违反（order violation，两个线程之间需要的顺序没有得到保证）。

我们还简要讨论了死锁：它为什么会发生，以及可以怎么应对。这个问题和并发本身一样古老，关于它的论文已经有好几百篇。实践中最好的解决办法是谨慎行事：制定一个获取锁的顺序，从一开始就防止死锁发生。无等待（wait-free）方法也很有前景，一些无等待的数据结构如今已经进入常用的库和关键系统，包括 Linux。然而，这类方法缺乏通用性，开发新的无等待（wait-free）数据结构又很复杂，这很可能会限制这种方法的整体实用性。也许最好的解决办法是开发新的并发编程模型（concurrent programming model）：在 MapReduce（来自 Google）[GD02] 这样的系统中，程序员可以描述某些类型的并行计算（parallel computation），而完全不需要任何锁。锁从本质上说就是有问题的；也许除非万不得已，我们都应该尽量避免用锁。

## 参考文献

[B+87] "Concurrency Control and Recovery in Database Systems" by Philip A. Bernstein, Vassos Hadzilacos, Nathan Goodman. Addison-Wesley, 1987. *The classic text on concurrency in database management systems. As you can tell, understanding concurrency, deadlock, and other topics in the world of databases is a world unto itself. Study it and find out for yourself.*

[C+71] "System Deadlocks" by E.G. Coffman, M.J. Elphick, A. Shoshani. ACM Computing Surveys, 3:2, June 1971. *The classic paper outlining the conditions for deadlock and how you might go about dealing with it. There are certainly some earlier papers on this topic; see the references within this paper for details.*

[D64] "Een algorithme ter voorkoming van de dodelijke omarming" by Edsger Dijkstra. 1964. Available: http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD108.PDF. *Indeed, not only did Dijkstra come up with a number of solutions to the deadlock problem, he was the first to note its existence, at least in written form. However, he called it the "deadly embrace", which (thankfully) did not catch on.*

[GD02] "MapReduce: Simplified Data Processing on Large Clusters" by Sanjay Ghemawhat, Jeff Dean. OSDI '04, San Francisco, CA, October 2004. *The MapReduce paper ushered in the era of large-scale data processing, and proposes a framework for performing such computations on clusters of generally unreliable machines.*

[H01] "A Pragmatic Implementation of Non-blocking Linked-lists" by Tim Harris. International Conference on Distributed Computing (DISC), 2001. *A relatively modern example of the difficulties of building something as simple as a concurrent linked list without locks.*

[H91] "Wait-free Synchronization" by Maurice Herlihy. ACM TOPLAS, 13:1, January 1991. *Herlihy's work pioneers the ideas behind wait-free approaches to writing concurrent programs. These approaches tend to be complex and hard, often more difficult than using locks correctly, probably limiting their success in the real world.*

[H93] "A Methodology for Implementing Highly Concurrent Data Objects" by Maurice Herlihy. ACM TOPLAS, 15:5, November 1993. *A nice overview of lock-free and wait-free structures. Both approaches eschew locks, but wait-free approaches are harder to realize, as they try to ensure that any operation on a concurrent structure will terminate in a finite number of steps (e.g., no unbounded looping).*

[J+08] "Deadlock Immunity: Enabling Systems To Defend Against Deadlocks" by Horatiu Jula, Daniel Tralamazza, Cristian Zamfir, George Candea. OSDI '08, San Diego, CA, December 2008. *An excellent recent paper on deadlocks and how to avoid getting caught in the same ones over and over again in a particular system.*

[K81] "Soul of a New Machine" by Tracy Kidder. Backbay Books, 2000 (reprint of 1980 version). *A must-read for any systems builder or engineer, detailing the early days of how a team inside Data General (DG), led by Tom West, worked to produce a "new machine." Kidder's other books are also excellent, including "Mountains beyond Mountains." Or maybe you don't agree with us, comma?*

[K87] "Deadlock Detection in Distributed Databases" by Edgar Knapp. ACM Computing Surveys, 19:4, December 1987. *An excellent overview of deadlock detection in distributed database systems. Also points to a number of other related works, and thus is a good place to start your reading.*

[L+08] "Learning from Mistakes — A Comprehensive Study on Real World Concurrency Bug Characteristics" by Shan Lu, Soyeon Park, Eunsoo Seo, Yuanyuan Zhou. ASPLOS '08, March 2008, Seattle, Washington. *The first in-depth study of concurrency bugs in real software, and the basis for this chapter. Look at Y.Y. Zhou's or Shan Lu's web pages for many more interesting papers on bugs.*

[T+94] "Linux File Memory Map Code" by Linus Torvalds and many others. Available online at: https://github.com/torvalds/linux/blob/master/mm/filemap.c. *Thanks to Michael Walfish (NYU) for pointing out this precious example. The real world, as you can see in this file, can be a bit more complex than the simple clarity found in textbooks…*

## 作业（编程）

这个作业让你研究一些真实的、会死锁或能避免死锁的代码。不同版本的代码，对应着在一个简化的 `vector_add()` 例程中避免死锁的不同方法。关于这些程序以及它们共用的基础代码，详见 README。

## 问题

1. 首先，研究 `vector-deadlock.c` 中的代码，以及 `main-common.c` 和相关文件，弄清这些程序大致是怎么工作的，以及有哪些关键选项。然后运行：

   ```
   ./vector-deadlock -n 2 -l 1 -v
   ```

   它会创建两个线程（`-n 2`），每个线程做一次向量加法（`-l 1`），并且以详细模式（`-v`）运行。确保你理解它的输出。每次运行，输出有什么变化？

2. 现在加上 `-d` 选项，并把循环次数（`-l`）从 1 改成更大的数。会发生什么？代码（总是）会死锁吗？

3. 改变线程数（`-n`）会如何影响程序的结果？有没有哪些 `-n` 的值能保证不发生死锁？

4. 现在看看 `vector-global-order.c` 中的代码。首先，确保你理解这段代码想做什么；你明白它为什么能避免死锁吗？另外，在这个 `vector_add()` 例程中，为什么源向量和目标向量相同时要做特殊处理？

5. 现在用以下选项运行代码：`-t -n 2 -l 100000 -d`。代码要多长时间才能完成？增加循环次数或线程数时，总时间如何变化？

6. 如果打开并行（parallelism）选项（`-p`），会发生什么？每个线程处理不同向量的相加（这就是 `-p` 的作用），和处理相同的向量相比，你预计性能会有多大变化？

7. 现在研究 `vector-try-wait.c`，先确保你理解这段代码。其中第一次调用 `pthread_mutex_trylock()` 真的有必要吗？然后运行这段代码。和全局顺序方法相比，它运行得有多快？随着线程数增加，代码统计的重试次数如何变化？

8. 现在看看 `vector-avoid-hold-and-wait.c`。这种方法的主要问题是什么？分别在加 `-p` 和不加 `-p` 的情况下运行，它的性能和其他版本相比如何？

9. 最后，我们来看看 `vector-nolock.c`。这个版本完全不使用锁；它提供的语义和其他版本完全一样吗？为什么？

10. 现在比较它和其他版本的性能，分别考虑线程处理同样两个向量（不加 `-p`）和每个线程处理各自不同的向量（加 `-p`）这两种情况。这个无锁（lock-free）版本的性能如何？

 © 2008–26, ARPACI-DUSSEAU

# 33

# 基于事件的并发（进阶）

到目前为止，我们讨论并发（concurrency）时，仿佛构建并发应用程序的唯一方法就是使用线程（thread）。和生活中的许多事情一样，这并不完全正确。具体来说，GUI 应用程序 [O96] 和某些类型的互联网服务器 [PDZ99] 常常采用另一种风格的并发编程（concurrent programming）。这种风格叫作**基于事件的并发**（event-based concurrency），在一些现代系统中很流行，包括 **node.js** [N13] 这类服务器端框架，但它的根源在 C/UNIX 系统中，下面就来讨论。

基于事件的并发（event-based concurrency）要解决的问题有两方面。一方面，在多线程（multi-threaded）应用程序中正确地管理并发可能很有挑战性；我们已经讨论过，可能会出现漏加锁（missing lock）、死锁（deadlock）以及其他令人头疼的问题。另一方面，在多线程应用程序中，开发者对某一时刻调度什么几乎无法控制，甚至完全无法控制；程序员只能创建线程，然后寄希望于底层操作系统能在可用的 CPU 上合理地调度它们。通用调度程序（scheduler）很难做到对所有工作负载（workload）、在所有情况下都表现良好，所以操作系统有时调度得并不理想。于是就有了下面这个问题：

关键问题：
如何不用线程构建并发服务器
怎样才能不用线程就构建出并发服务器（concurrent server），从而既保持对并发的控制，又避免一些似乎困扰着多线程应用程序的问题？

## 33.1 基本思想：事件循环

如上所述，我们要用的基本方法叫作**基于事件的并发**（event-based concurrency）。这种方法非常简单：只需等待某件事（即一个“事件”）发生；事件发生后，检查它是什么类型，然后做它所需的少量工作（可能包括发起 I/O 请求、安排其他事件以便将来处理，等等）。就这么简单！

在深入细节之前，我们先看看一个典型的基于事件的服务器是什么样子。这类应用程序都围绕一个简单的结构构建，这个结构叫作**事件循环**（event loop）。事件循环的伪代码（pseudocode）如下：

```
while (1) {
    events = getEvents();
    for (e in events)
        processEvent(e);
}
```

真的就这么简单。主循环只是调用 `getEvents()` 等着有事可做，然后逐一处理返回的每个事件；处理每个事件的代码叫作**事件处理程序**（event handler）。重要的是，处理程序处理某个事件时，系统中只有它在活动；因此，决定下一个处理哪个事件，就相当于调度。这种对调度的显式控制，是基于事件的方法的根本优势之一。

但这又引出了一个更大的问题：基于事件的服务器究竟如何确定正在发生哪些事件，尤其是网络和磁盘 I/O 事件？具体来说，事件服务器怎么知道有发给它的消息到达了？

## 33.2 一个重要的 API：`select()`（或 `poll()`）

有了这个基本的事件循环，接下来必须解决如何接收事件的问题。大多数系统都通过 `select()` 或 `poll()` 系统调用（system call）提供了一个基本的 API。

这些接口让程序能做的事情很简单：检查有没有需要处理的传入 I/O。例如，设想一个网络应用程序（比如 Web 服务器）想检查是否有网络数据包（packet）到达，好去处理它们。这两个系统调用正好可以做到这一点。

以 `select()` 为例。（Mac 上的）手册页（manual page）这样描述这个 API：

```
int select(int nfds,
           fd_set *restrict readfds,
           fd_set *restrict writefds,
           fd_set *restrict errorfds,
           struct timeval *restrict timeout);
```

手册页上的实际描述是：*`readfds`*、*`writefds`* 和 *`errorfds`* 传入的是三个 *I/O* 描述符集合的地址，*`select()`* 分别检查其中是否有描述符已准备好读、已准备好写，或者有待处理的异常情况。每个集合只检查前 *`nfds`* 个描述符，即检查描述符集合中从 *0* 到 *`nfds-1`* 的描述符。返回时，*`select()`* 会把每个描述符集合换成它的子集，只留下已经可以执行所请求操作的描述符。*`select()`* 返回所有集合中就绪描述符的总数。

关于 `select()`，有几点要说明。首先，注意它既能检查描述符是否可以读，也能检查是否可以写；前者让服务器能确定有新的数据包到达、需要处理，后者让服务器知道什么时候可以回复，即发送队列（queue）未满的时候。

其次，注意 `timeout` 参数。一种常见用法是把超时（timeout）设为 NULL，这会让 `select()` 无限期地阻塞，直到某个描述符就绪（ready）。不过，更健壮的服务器通常会指定某种超时；常用的做法是把超时设为零，这样对 `select()` 的调用就会立即返回。

`poll()` 系统调用与它非常相似。详情请参阅它的手册页，或者 Stevens 和 Rago 的书 [SR05]。

无论用哪一个，有了这些基本原语，就能构建一个非阻塞的事件循环：它只需检查有没有传入的数据包，从有消息到达的套接字（socket）中读取数据，并根据需要回复。

 © 2008–23, ARPACI-DUSSEAU

补充：阻塞接口与非阻塞接口
阻塞（blocking）接口，或者说**同步**（synchronous）接口，在返回调用者之前会完成全部工作；非阻塞（non-blocking）接口，或者说**异步**（asynchronous）接口，则开始一些工作后立即返回，让需要完成的工作在后台进行。
阻塞调用的罪魁祸首通常是某种 I/O。例如，如果一个调用必须从磁盘读取数据才能完成，它就可能阻塞，等待已发往磁盘的 I/O 请求返回。
非阻塞接口可以用在任何风格的编程中，例如使用线程的编程；但在基于事件的方法中必不可少，因为一个阻塞的调用会让一切都停滞不前。

## 33.3 使用 `select()`

为了更具体一些，我们来看看如何用 `select()` 查看哪些网络描述符上有传入的消息。图 33.1 给出了一个简单的例子。

这段代码其实相当好懂。完成一些初始化之后，服务器进入一个无限循环。在循环中，它先用 `FD_ZERO()` 宏清空文件描述符（file descriptor）集合，再用 `FD_SET()` 把从 `minFD` 到 `maxFD` 的所有文件描述符都加入集合。例如，这个描述符集合可能代表服务器关注的所有网络套接字。最后，服务器调用 `select()`，查看哪些连接上有可用的数据。接着在循环中使用 `FD_ISSET()`，事件服务器就能看到哪些描述符已经有数据就绪，并处理传入的数据。

当然，真正的服务器会比这复杂得多，还需要发送消息、发起磁盘 I/O 以及处理许多其他细节的逻辑。API 方面的更多信息请参阅 Stevens 和 Rago 的书 [SR05]；Pai 等人和 Welsh 等人的论文则很好地概述了基于事件的服务器的一般流程 [PDZ99, WCB01]。

## 33.4 为什么更简单？不需要锁

在单个 CPU 上运行基于事件的应用程序时，并发程序（concurrent program）中的那些问题就不复存在了。具体来说，由于一次只处理一个事件，所以不需要获取或释放锁（lock）；基于事件的服务器不会被另一个线程中断（interrupted），因为它明确就是单线程（single-threaded）的。因此，多线程程序中常见的并发 bug（concurrency bug）不会出现在基本的基于事件的方法中。

## 33.5 一个问题：阻塞的系统调用

到目前为止，基于事件的编程听起来很棒，对吧？你只需编写一个简单的循环，在事件出现时处理它们。你甚至都不用考虑加锁（locking）！但有一个问题：如果某个事件要求你发起一个可能阻塞的系统调用，该怎么办？

例如，设想客户端向服务器发来一个请求，要求从磁盘读取一个文件，并把内容返回给发出请求的客户端（很像一个简单的 HTTP 请求）。为了处理这样的请求，某个事件处理程序最终必须发起 `open()` 系统调用来打开文件，接着发起一系列 `read()` 调用来读取文件。文件读入内存后，服务器很可能会开始把结果发送给客户端。

 © 2008–23, ARPACI-DUSSEAU

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>

int main(void) {
  // 打开并设置好一批套接字（未列出）
  // 主循环
  while (1) {
    // 把 fd_set 初始化为全零
    fd_set readFDs;
    FD_ZERO(&readFDs);

    // 现在为这个服务器关心的
    // 描述符设置相应的位
    //  （为简单起见，设置从 min 到 max 的全部描述符）
    int fd;
    for (fd = minFD; fd < maxFD; fd++)
      FD_SET(fd, &readFDs);

    // 执行 select
    int rc = select(maxFD+1, &readFDs, NULL, NULL, NULL);

    // 用 FD_ISSET() 检查哪些描述符确实有数据
    int fd;
    for (fd = minFD; fd < maxFD; fd++)
      if (FD_ISSET(fd, &readFDs))
        processFD(fd);
  }
}
```

图 33.1：**使用 `select()` 的简单代码**

如果所需的元数据（metadata）或数据还不在内存中，`open()` 和 `read()` 调用就会向存储系统发出 I/O 请求，因此可能要花很长时间才能完成。基于线程的服务器不存在这个问题：发出 I/O 请求的线程挂起（等待 I/O 完成）时，其他线程可以运行，从而让服务器继续推进。事实上，正是 I/O 和其他计算之间这种天然的**重叠**（overlap），让基于线程的编程用起来顺手又简单。

提示：不要在基于事件的服务器中阻塞

基于事件的服务器能细粒度（fine-grained）地控制任务调度。不过，要保持这种控制，就绝不能发起任何会阻塞调用者执行的调用；不遵守这条设计提示，结果就是服务器被阻塞、客户端很恼火，而且别人会严重怀疑你到底有没有读过本书的这一部分。

 © 2008–23, ARPACI-DUSSEAU

然而，在基于事件的方法中，没有其他线程可以运行，只有主事件循环。这意味着，如果某个事件处理程序发起了一个会阻塞的调用，整个服务器就会跟着阻塞，直到这个调用完成。事件循环阻塞时，系统就闲着，这可能造成巨大的资源浪费。因此，在基于事件的系统中有一条必须遵守的规则：不允许阻塞调用。

## 33.6 一个解决方案：异步 I/O

为了突破这个限制，许多现代操作系统引入了向磁盘系统发出 I/O 请求的新方法，统称为**异步 I/O**（asynchronous I/O）。这些接口让应用程序可以发出 I/O 请求，并在 I/O 完成之前立即把控制权返回给调用者；另外还有一些接口，让应用程序能够确定各个 I/O 是否已经完成。

例如，我们来看看 Mac 上提供的接口（其他系统也有类似的 API）。这些 API 围绕一个基本结构展开，即 `struct aiocb`，通常称为 **AIO 控制块**（AIO control block）。这个结构的简化版本如下，更多信息请参阅手册页：

```
struct aiocb {
    int              aio_fildes;    // 文件描述符
    off_t            aio_offset;    // 文件内的偏移量
    volatile void    *aio_buf;      // 缓冲区的位置
    size_t           aio_nbytes;    // 传输的长度
};
```

要对一个文件发起异步读取，应用程序应该先在这个结构中填入相关信息：要读取的文件的文件描述符 `aio_fildes`、文件内的偏移量（offset）`aio_offset`、请求的长度 `aio_nbytes`，以及存放读取结果的目标内存位置 `aio_buf`。

填好这个结构后，应用程序必须发起读取文件的异步调用；在 Mac 上，这个 API 就是**异步读**（asynchronous read）API：

```
int aio_read(struct aiocb *aiocbp);
```

这个调用会尝试发起 I/O；如果成功，它就立即返回，应用程序（即基于事件的服务器）可以继续自己的工作。

不过，这道谜题还剩最后一块要解决。我们怎么知道 I/O 什么时候完成，从而知道 `aio_buf` 指向的缓冲区（buffer）里已经有了所请求的数据呢？

这就要再用一个 API。在 Mac 上，它（有点令人困惑地）叫作 `aio_error()`。这个 API 如下：

```
int aio_error(const struct aiocb *aiocbp);
```

这个系统调用检查 `aiocbp` 所指的请求是否已经完成。如果已经完成，该例程返回成功（用零表示）；如果没有，就返回 `EINPROGRESS`。因此，应用程序可以对每个尚未完成的异步 I/O（asynchronous I/O）调用 `aio_error()`，定期**轮询**（poll）系统，看这些 I/O 是否已经完成。

你可能已经注意到，检查 I/O 是否完成是一件很痛苦的事；如果某个时刻程序已经发出了几十甚至上百个 I/O，它是应该不停地反复检查每一个，还是先等一会儿，还是……？

为了解决这个问题，有些系统提供了一种基于**中断**（interrupt）的方法。这种方法利用 UNIX **信号**（signal），在异步 I/O（asynchronous I/O）完成时通知应用程序，从而无需反复询问系统。设备上也有轮询还是中断的问题，你会在 I/O 设备一章中看到（或者已经看到了）。

在没有异步 I/O（asynchronous I/O）的系统中，就无法实现纯粹的基于事件的方法。不过，聪明的研究人员已经找到了一些效果相当不错的替代方法。例如，Pai 等人 [PDZ99] 描述了一种混合方法：用事件处理网络数据包，用线程池（thread pool）管理尚未完成的 I/O。详情请阅读他们的论文。

## 33.7 另一个问题：状态管理

基于事件的方法还有一个问题：这类代码通常比传统的基于线程的代码更难写。原因如下：事件处理程序发起异步 I/O（asynchronous I/O）时，必须把一些程序状态打包起来，供 I/O 最终完成时下一个事件处理程序使用；而基于线程的程序不需要这些额外的工作，因为程序需要的状态就在线程的栈（stack）上。Adya 等人把这项工作称为**手动栈管理**（manual stack management），它是基于事件的编程所固有的 [A+02]。

为了把这一点说得更具体，我们来看一个简单的例子：基于线程的服务器要从文件描述符 `fd` 读取数据，读完后再把数据写到网络套接字描述符（socket descriptor）`sd`。代码（忽略错误检查）如下：

```
int rc = read(fd, buffer, size);
rc = write(sd, buffer, size);
```

可以看到，在多线程程序中做这种工作轻而易举；`read()` 最终返回时，代码马上就知道要写到哪个套接字，因为这个信息就在线程的栈上（在变量 `sd` 中）。

在基于事件的系统中，事情就没那么简单了。要完成同样的任务，我们首先要用上面介绍的 AIO 调用，异步地发起读取。假设我们随后用 `aio_error()` 调用定期检查读取是否完成；等这个调用告诉我们读取已经完成，基于事件的服务器又怎么知道接下来该做什么呢？

正如 Adya 等人 [A+02] 所述，解决办法是使用一种古老的编程语言结构，叫作**延续**（continuation）[FHK84]。它听起来很复杂，但思路其实相当简单：就是把处理完这个事件所需的信息记在某个数据结构（data structure）中；等事件发生（即磁盘 I/O 完成）时，再查出所需的信息，处理这个事件。

在这个具体的例子中，解决办法是把套接字描述符（socket descriptor）`sd` 记录在某种数据结构中，例如散列表（hash table），以文件描述符 `fd` 为索引。磁盘 I/O 完成时，事件处理程序用文件描述符查找这个延续，延续会把套接字描述符（socket descriptor）的值返回给调用者。这时（终于），服务器就可以完成最后一点工作，把数据写到套接字上。

## 33.8 使用事件时仍然存在的难题

基于事件的方法还有其他几个难点，我们也应该提一下。例如，系统从单 CPU 发展到多 CPU 后，基于事件的方法就失去了一部分简洁性。具体来说，为了利用多个 CPU，事

补充：UNIX 信号

所有现代 UNIX 变体都有一套庞大而有趣的基础设施，叫作**信号**（signal）。最简单地说，信号提供了一种与进程（process）通信的方式。具体来说，信号可以投递给应用程序；投递信号时，应用程序会停下手头正在做的事，转去运行**信号处理程序**（signal handler），也就是应用程序中处理该信号的某段代码。处理完后，进程就恢复之前的行为。

每个信号都有一个名字，比如 **HUP**（挂起，hang up）、**INT**（中断，interrupt）、**SEGV**（段违规，segmentation violation）等；详情请参阅手册页。有意思的是，有时是内核自己发出信号。例如，你的程序遇到段违规（segmentation violation）时，操作系统会向它发送一个 **SIGSEGV**（给信号名加上前缀 **SIG** 是很常见的做法）；如果你的程序设置了要捕获这个信号，就可以运行一些代码来应对这种出错的行为（这对调试很有帮助）。如果进程没有为某个信号设置处理程序，收到这个信号时就执行默认行为；对于 SEGV，进程会被杀死。

下面是一个简单的程序，它进入一个无限循环，但在此之前设置了一个信号处理程序来捕获 SIGHUP：

```
void handle(int arg) {
    printf("stop wakin' me up...\n");
}

int main(int argc, char *argv[]) {
    signal(SIGHUP, handle);
    while (1)
        ; // 啥也不干，就接几个信号玩玩
    return 0;
}
```

你可以用命令行工具 **kill**（没错，这个名字既古怪又带点攻击性）向它发送信号。这样做会打断程序中的主 while 循环，转去运行处理程序代码 `handle()`：

```
prompt> ./main &
[3] 36705
prompt> kill -HUP 36705
stop wakin' me up...
prompt> kill -HUP 36705
stop wakin' me up...
```

关于信号，要学的东西还有很多，一章都远远讲不完，更别说一页了。和往常一样，有一个很好的资料来源：Stevens 和 Rago 的书 [SR05]。有兴趣的话可以深入阅读。

件服务器必须并行地（in parallel）运行多个事件处理程序；这样一来，临界区（critical section）之类常见的同步问题就会出现，也就必须采用锁之类常见的解决办法。因此，在现代多核（multicore）系统上，已经不可能再做那种不需要锁的简单事件处理了。

基于事件的方法的另一个问题是，它和某些系统活动结合得不好，比如**分页**（paging）。例如，如果一个事件处理程序发生了缺页异常（page fault），它就会阻塞，于是在缺页异常

处理完之前，服务器都无法继续推进。尽管服务器的结构设计得不会显式阻塞，这种由缺页异常引起的隐式阻塞却很难避免，一旦频繁出现，就可能导致严重的性能问题。

第三个问题是，随着各种例程的确切语义发生变化，基于事件的代码可能会越来越难以管理 [A+02]。例如，如果一个例程从非阻塞变成了阻塞，那么调用这个例程的事件处理程序也必须随之改变，把自己拆成两部分，以适应这个例程的新特性。由于阻塞对基于事件的服务器来说是灾难性的，程序员必须时刻留意每个事件所用的 API 在语义上的这类变化。

最后，虽然现在大多数平台都已经支持异步磁盘 I/O，但实现这一点花了很长时间 [PDZ99]，而且它与异步网络 I/O 的结合也始终不像你想象的那样简单统一。例如，人们本希望只用 `select()` 接口来管理所有尚未完成的 I/O，但通常需要把用于网络的 `select()` 和用于磁盘 I/O 的 AIO 调用组合起来使用。

## 33.9 小结

我们简要介绍了另一种并发风格：基于事件的并发（event-based concurrency）。基于事件的服务器把调度的控制权交给应用程序本身，但代价是增加了一定的复杂性，而且难以与现代系统的其他方面（例如分页）结合。由于存在这些挑战，还没有哪一种方法脱颖而出成为最佳方案；因此在未来许多年里，线程和事件很可能会并存，成为解决同一个并发问题的两种不同方法。想了解更多，可以读一些研究论文（例如 [A+02, PDZ99, vB+03, WCB01]），或者更好的办法是，写一些基于事件的代码。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[A+02] “Cooperative Task Management Without Manual Stack Management” by Atul Adya, Jon Howell, Marvin Theimer, William J. Bolosky, John R. Douceur. USENIX ATC ’02, Monterey, CA, June 2002. *This gem of a paper is the first to clearly articulate some of the difficulties of event-based concurrency, and suggests some simple solutions, as well explores the even crazier idea of combining the two types of concurrency management into a single application!*

[FHK84] “Programming With Continuations” by Daniel P. Friedman, Christopher T. Haynes, Eugene E. Kohlbecker. In Program Transformation and Programming Environments, Springer Verlag, 1984. *The classic reference to this old idea from the world of programming languages. Now increasingly popular in some modern languages.*

[N13] “Node.js Documentation” by the folks who built node.js. Available: nodejs.org/api. *One of the many cool new frameworks that help you readily build web services and applications. Every modern systems hacker should be proficient in frameworks such as this one (and likely, more than one). Spend the time and do some development in one of these worlds and become an expert.*

[O96] “Why Threads Are A Bad Idea (for most purposes)” by John Ousterhout. Invited Talk at USENIX ’96, San Diego, CA, January 1996. *A great talk about how threads aren’t a great match for GUI-based applications (but the ideas are more general). Ousterhout formed many of these opinions while he was developing Tcl/Tk, a cool scripting language and toolkit that made it 100x easier to develop GUI-based applications than the state of the art at the time. While the Tk GUI toolkit lives on (in Python for example), Tcl seems to be slowly dying (unfortunately).*

[PDZ99] “Flash: An Efficient and Portable Web Server” by Vivek S. Pai, Peter Druschel, Willy Zwaenepoel. USENIX ’99, Monterey, CA, June 1999. *A pioneering paper on how to structure web servers in the then-burgeoning Internet era. Read it to understand the basics as well as to see the authors’ ideas on how to build hybrids when support for asynchronous I/O is lacking.*

[SR05] “Advanced Programming in the UNIX Environment” by W. Richard Stevens and Stephen A. Rago. Addison-Wesley, 2005. *Once again, we refer to the classic must-have-on-your-bookshelf book of UNIX systems programming. If there is some detail you need to know, it is in here.*

[vB+03] “Capriccio: Scalable Threads for Internet Services” by Rob von Behren, Jeremy Condit, Feng Zhou, George C. Necula, Eric Brewer. SOSP ’03, Lake George, New York, October 2003. *A paper about how to make threads work at extreme scale; a counter to all the event-based work ongoing at the time.*

[WCB01] “SEDA: An Architecture for Well-Conditioned, Scalable Internet Services” by Matt Welsh, David Culler, and Eric Brewer. SOSP ’01, Banff, Canada, October 2001. *A nice twist on event-based serving that combines threads, queues, and event-based handling into one streamlined whole. Some of these ideas have found their way into the infrastructures of companies such as Google, Amazon, and elsewhere.*

## 作业（编程）

在这个（简短的）作业中，你会积累一些编写基于事件的代码的经验，并接触它的一些关键概念。祝你好运！

## 问题

1. 首先，编写一个简单的服务器，它能接受 TCP 连接并为其提供服务。如果你还不知道怎么做，得先上网查一查。让它一次只服务一个请求；每个请求都非常简单，比如获取当前时间。
2. 现在，加入 `select()` 接口。编写一个主程序，它能接受多个连接；再编写一个事件循环，检查哪些文件描述符上有数据，然后读取并处理这些请求。务必仔细测试，确保你正确地使用了 `select()`。
3. 接下来，让请求稍微有趣一点，模拟一个简单的 Web 服务器或文件服务器（file server）。每个请求都要读取某个文件的内容（文件名在请求中给出），服务器的响应是：把文件读进缓冲区，然后把内容返回给客户端。用标准的 `open()`、`read()`、`close()` 系统调用实现这个功能。这里要小心一点：如果让它长时间运行，可能会有人想出办法，用它读取你电脑上的所有文件！
4. 现在改用本章介绍的异步 I/O（asynchronous I/O）接口，不再用标准的 I/O 系统调用。把异步接口融入你的程序有多难？
5. 为了好玩，给你的代码加上一些信号处理。信号的一种常见用途是“戳”一下服务器，让它重新加载某个配置文件，或者执行其他某种管理操作。一种比较自然的玩法是：给你的服务器加一个用户级文件缓存（cache），存放最近访问过的文件。实现一个信号处理程序，在服务器进程收到信号时清空这个缓存。
6. 最后是困难的部分：你怎么判断，花力气构建一个异步的、基于事件的方法是否值得？你能设计一个实验来展示它的好处吗？你的方法增加了多少实现复杂度？

 © 2008–23, ARPACI-DUSSEAU

# 34

# 关于并发的总结对话

**教授**：那么，你现在头疼了吗？

**学生**：（吞了两片布洛芬）嗯，有一点。线程（*thread*）交错执行的方式那么多，想起来真费劲。

**教授**：确实如此。我一直觉得很惊讶：一旦涉及并发执行（*concurrent execution*），短短几行代码就可能变得几乎无法理解。

**学生**：我也是！身为计算机科学家，居然弄不懂五行代码，说起来有点丢人。

**教授**：哦，别太难过。如果你翻翻最早那批关于并发算法（*concurrent algorithm*）的论文，会发现它们有时候也是错的！而作者往往还是教授！

**学生**：（倒吸一口气）教授也会……呃……出错？

**教授**：是的，确实如此。不过别告诉别人——这是我们的行业机密之一。

**学生**：我发誓保密。可是，如果并发代码（*concurrent code*）这么难想清楚、这么难写对，我们该怎么写出正确的并发代码呢？

**教授**：嗯，这才是真正的问题，不是吗？我觉得要从几件简单的事做起。第一，保持简单！避免线程之间复杂的交互；非交互不可的话，就用众所周知、久经考验的方法。

**学生**：比如简单的加锁（*locking*），也许再加一个生产者/消费者（*producer/consumer*）队列（*queue*）？

**教授**：没错！这些都是常见的范式，凭你学过的东西，应该能写出可行的解决方案。第二，只在绝对必要时才使用并发（*concurrency*）；能不用就尽量不用。没有什么比过早地优化程序更糟糕的了。

**学生**：明白了——既然不需要线程，干嘛还要加呢？

**教授**：正是。第三，如果你确实需要并行（*parallelism*），就去找其他更简单的形式。用 *Map-Reduce* 方法编写并行数据分析代码就是一个绝佳的例子：它实现了并行，却用不着应付锁（*lock*）、条件变量（*condition variable*）以及我们讨论过的其他种种麻烦东西，也就避开了它们那可怕的复杂性。

**学生**：*Map-Reduce*，是吗？听起来很有意思——我得自己再多读点相关的东西。

**教授**：很好！你确实应该去读。说到底，你得自己读很多东西，因为我们一起学的内容，相对于外面浩如烟海的知识，只能算最粗浅的入门。读，读，再多读一些！然后动手试一试，写点代码，再多写一些。还要多练习；除了这本书，还有大量其他资源[1]。正如格拉德威尔在他的书《异类》（*Outliers*）中所说，要在一件事上成为真正的专家，大约需要投入一万小时。这可没法全在课堂上完成！

**学生**：哇，我说不清这是让人泄气还是让人振奋。不过我就当是后者吧，开始干活！该再写些并发代码了……

[1]这里有一个，是游戏的形式：https://deadlockempire.github.io/

# 第三部分

# 持久性

# 35

# 关于持久性的对话

**教授**:好,现在讲到操作系统四大……呃……三大支柱里的第三根:**持久性**(*persistence*)。

**学生**:您刚才说的是三大支柱,还是四大支柱?第四根是什么?

**教授**:没有第四根。就三根,年轻人,就三根。我这是想把事情讲得简单点。

**学生**:好吧,行。那么,高贵又了不起的教授,什么是持久性呢?

**教授**:其实,你大概知道这个词在传统意义上是什么意思,对吧?用词典的话说:“不顾困难或阻挠,坚定或固执地把一件事做下去。”

**学生**:有点像上您的课:得有点固执才行。

**教授**:哈!没错。不过,这里说的持久性是另一回事。我来解释一下。想象你在外面,在一片田野里,摘了一个——

**学生**:(打断)我知道!一个桃子!从桃树上摘的!

**教授**:我本来想说苹果,从苹果树上摘的。哎,算了,我看就照你说的来吧。

**学生**:(一脸茫然)

**教授**:总之,你摘了一个桃子;其实你摘了好多好多桃子,但你想让它们放得久一点。毕竟,威斯康星的冬天又难熬又冷酷。你会怎么做?

**学生**:嗯,我觉得办法有好几种。可以腌起来!或者烤个派,或者做成某种果酱。好玩极了!

**教授**:好玩?嗯,也许吧。不过可以肯定的是,要让桃子**持久保存**(*persist*),你得多花很多工夫。信息也是这样:不管计算机崩溃、磁盘故障还是断电(*power outage*),都要让信息持久保存下来,这是一个既艰巨又有意思的挑战。

**学生**:过渡得真漂亮;您越来越擅长这个了。

**教授**:谢谢!你知道的,教授总是乐意听几句好话。

**学生**:我会记着的。我想,现在该把桃子放一边,聊聊计算机了吧?

**教授**:是的,是时候了……

# 36

# I/O 设备

在深入本书这一部分的主要内容——持久性（persistence）——之前，我们先介绍**输入/输出（I/O）设备**（input/output device）的概念，并说明操作系统可能会怎样和这类设备打交道。当然，I/O 对计算机系统非常关键：想象一个没有任何输入的程序（每次运行结果都一样）；再想象一个没有任何输出的程序（那它运行的意义何在?）。显然，要让计算机系统有意思，输入和输出缺一不可。所以，本章要解决的问题是：

关键问题：如何把 I/O 集成到系统中
I/O 应该怎样集成到系统中？有哪些通用机制？怎样让它们高效?

## 36.1 系统架构

我们先来看一张典型系统的“经典”示意图（图 36.1，第 374 页）。图中有一个 CPU，它通过某种**内存总线**（memory bus）或互连线与系统的主存相连。一些设备通过一条通用 **I/O 总线**（I/O bus）连到系统上，在许多现代系统中，这条总线（bus）是 **PCI**（或它的众多衍生版本之一）；显卡和其他一些高性能 I/O 设备可能就接在这里。最后，再往下是一条或多条我们所说的**外围总线**（peripheral bus），比如 **SCSI**、**SATA** 或 **USB**。它们把慢速设备连到系统上，包括**磁盘**、**鼠标**和**键盘**。

你可能会问：为什么需要这样的分层结构? 简单地说：物理规律和成本。总线越快，就必须越短；因此，高性能的内存总线（memory bus）上没有多少空间插设备之类的东西。此外，把总线设计成高性能的，代价相当高。所以，系统设计者采用了这种分层的方法：要求高性能的部件（比如显卡）离 CPU 近一些，性能要求较低的部件离得远一些。把磁盘和其他慢速设备放在外围总线（peripheral bus）上，好处有很多，其中最主要的是可以在上面接大量的设备。

当然，现代系统越来越多地使用专门的芯片组和更快的点对点互连来提高性能。图 36.2（第 375 页）大致画出了 Intel Z270 芯片组的结构 [H17]。在最上面，CPU 与内存系统连接得最紧密，但它与显卡（从而也与显示器）之间也有一条高性能的连接，以支持游戏（噢，太可怕了!）和其他图形密集型应用。

CPU 通过 Intel 专有的 **DMI**（**Direct Media Interface**，直接媒体接口）连到一个 I/O 芯片上，其余设备则通过多种不同的互连线连到这个芯片上。在右边，一块或多块硬盘通过

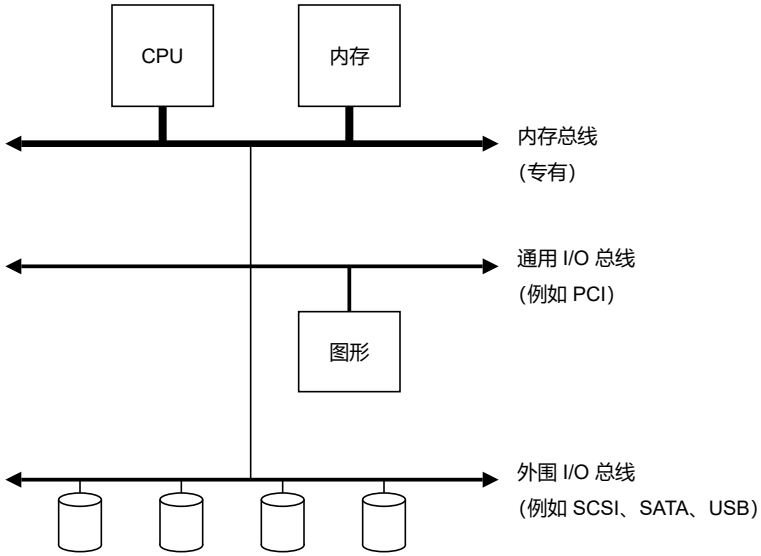



图 36.1：**典型的系统架构**

**eSATA** 接口连到系统上。**ATA**（**AT Attachment**, 指为 IBM PC AT 提供连接）、后来的 **SATA**（**Serial ATA**，串行 ATA），以及现在的 eSATA（**external SATA**，外部 SATA），代表了几十年来存储接口的演进，每前进一步都提高了性能，以跟上现代存储设备（storage device）的步伐。

I/O 芯片下方是若干 **USB**（**Universal Serial Bus**，通用串行总线）连接，在这张图里，它们让键盘和鼠标可以接到计算机上。在许多现代系统中，USB 就用于这类低性能设备。

最后，在左边，其他更高性能的设备可以通过 **PCIe**（**Peripheral Component Interconnect Express**）连到系统上。在这张图里，网络接口就接在这里；更高性能的存储设备，比如 **NVMe** 持久存储设备（persistent storage device），也常常接在这里。

## 36.2 典型设备

现在我们来看一个典型的设备（不是真实的设备），借助它来理解：要让设备交互变得高效，需要哪些机制。从图 36.3（第 375 页）可以看到，一个设备有两个重要的组成部分。第一部分是它向系统其余部分呈现的硬件**接口**（interface）。和软件一样，硬件也必须提供某种接口，让系统软件能控制它的操作。因此，所有设备都有规定好的接口，以及典型交互所遵循的协议（protocol）。

任何设备的第二部分是它的**内部结构**（internal structure）。这一部分因设备而异，负责实现设备向系统呈现的抽象（abstraction）。非常简单的设备只用一块或几块硬件芯片来实现自身功能；更复杂的设备则会包含一个简单的 CPU、一些通用内存，以及其他设备专用的芯片来完成工作。例如，现代 RAID 控制器的功能，可能要靠几十万行**固件**（firmware，即硬件设备内部的软件）来实现。

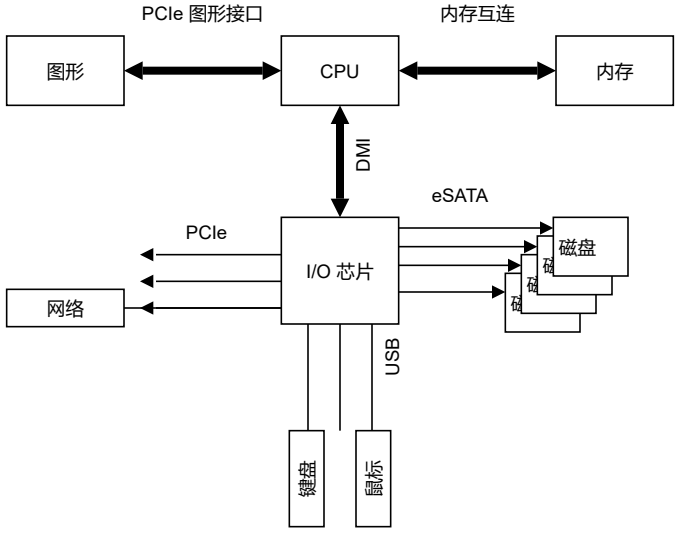



图 36.2：**现代系统架构**

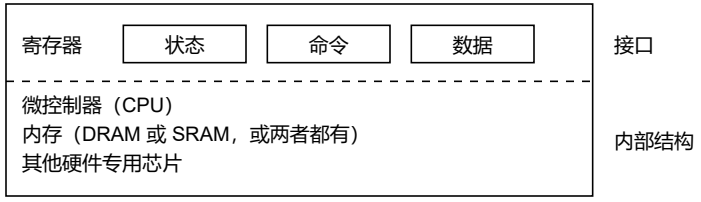



图 36.3：**典型设备**

## 36.3 典型协议

在上图中，（简化的）设备接口由三个寄存器组成：一个**状态**（status）寄存器，读取它可以看到设备的当前状态；一个**命令**（command）寄存器，用来告诉设备执行某项任务；还有一个**数据**（data）寄存器，用来向设备传送数据，或从设备取回数据。读写这些寄存器，操作系统就能控制设备的行为。

现在来看一次典型的交互：操作系统怎样让设备替它做事。协议如下：

```
While (STATUS == BUSY)
    ; // 等待，直到设备不忙
Write data to DATA register
Write command to COMMAND register
    （启动设备并执行命令）
While (STATUS == BUSY)
    ; // 等待，直到设备处理完你的请求
```

这个协议分四步。第一步，操作系统反复读取状态寄存器（status register），等待设备准备好接收命令；我们把这称为**轮询**（polling）设备（说白了就是问它在干什么）。第二步，操作系统把一些数据送到数据寄存器（data register）；可以想象，如果这是一块磁盘，要把一个磁盘块（disk block，比如 4 KB）传给设备，就需要执行多次写操作。主 CPU 参与数据搬运时（就像这个示例协议这样），就称为**程序控制 I/O**（programmed I/O，PIO）。第三步，操作系统把一条命令写入命令寄存器（command register）；这就隐含地告诉设备：数据已经就位，可以开始执行命令了。最后，操作系统再次循环轮询设备，等它完成（随后可能拿到一个表示成功与否的错误码）。

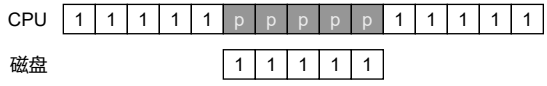
这个基本协议的优点是简单，而且能用。不过，它也有一些低效和不便之处。你可能会注意到的第一个问题是，轮询似乎很低效；具体来说，它把大量 CPU 时间白白花在等待（可能很慢的）设备完成工作上，而本来可以切换到另一个就绪（ready）的进程（process），把 CPU 用得更充分。

关键问题：如何避免轮询的开销
操作系统怎样才能不频繁轮询就检查设备状态，从而降低管理设备所需的 CPU 开销?

## 36.4 用中断降低 CPU 开销

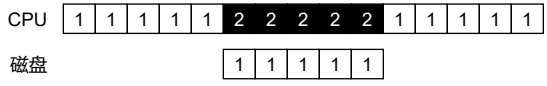
许多工程师多年前想出的改进办法，我们已经见过了：**中断**（interrupt）。操作系统不必反复轮询设备，而是可以发出请求，让调用进程进入睡眠，然后上下文切换（context switch）到另一个任务。设备最终完成操作时，会触发一个硬件中断，使 CPU 跳转到操作系统中一个预先确定的**中断服务程序**（interrupt service routine，ISR），或者更简单地说，**中断处理程序**（interrupt handler）。处理程序就是一段操作系统代码，它会完成这个请求（例如从设备读取数据，也许还有错误码），并唤醒等待这次 I/O 的进程，让它按需继续执行。

因此，中断让计算与 I/O 可以**重叠**（overlap），这是提高利用率（utilization）的关键。下面这条时间线说明了这一点：




在图中，进程 1 在 CPU 上运行了一段时间（CPU 那一行上重复的 1 表示这一点），然后向磁盘发出一个 I/O 请求，读取一些数据。如果没有中断，系统就只能空转，反复轮询设备的状态，直到 I/O 完成（用 p 表示）。磁盘处理完请求后，进程 1 终于可以再次运行。

如果改用中断，允许重叠，操作系统就可以在等待磁盘时做别的事：



提示：中断并不总是比轮询好

虽然中断让计算和 I/O 可以重叠，但它只对慢速设备才真正有意义。否则，处理中断和上下文切换的开销可能超过中断带来的好处。还有一些情况下，大量涌来的中断可能让系统过载，导致活锁（livelock）[MR96]；这时，轮询能让操作系统更好地控制调度，反倒派上了用场。

在这个例子中，磁盘处理进程 1 的请求时，操作系统在 CPU 上运行进程 2。磁盘请求完成时，会发生一个中断，操作系统唤醒进程 1，让它再次运行。这样，在中间这段时间里，CPU 和磁盘都得到了充分利用。

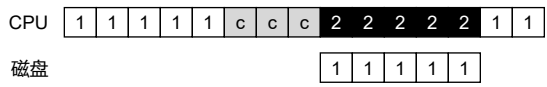
注意，使用中断并不总是最好的办法。例如，设想一个设备完成任务非常快：第一次轮询时，通常就会发现设备已经完成了任务。这时使用中断反而会拖慢系统：切换到另一个进程、处理中断、再切换回发出请求的进程，代价都很高。因此，如果设备很快，最好用轮询；如果设备很慢，最好用能实现重叠的中断。如果不知道设备的速度，或者它时快时慢，最好采用一种**混合**（hybrid）方式：先轮询一小会儿，如果设备还没完成，再改用中断。这种**两阶段**（two-phased）方法也许能兼得两者之长。

不使用中断的另一个原因出现在网络中 [MR96]。如果大量传入的数据包（packet）每个都产生一个中断，操作系统就可能陷入**活锁**（livelock），也就是发现自己只顾着处理中断，从不让用户级进程运行、真正去处理请求。例如，设想一台 Web 服务器因为登上了 Hacker News 榜首 [H18] 而遭遇负载激增。在这种情况下，最好偶尔用一下轮询，更好地控制系统中发生的事情，让 Web 服务器先处理一些请求，再回到设备那里查看有没有更多数据包到达。

另一种基于中断的优化是**合并**（coalescing）。采用这种做法时，需要触发中断的设备会先等一小会儿，再把中断发给 CPU。等待期间，其他请求可能很快也完成了，于是多个中断就可以合并成一次中断，从而降低中断处理的开销。当然，等得太久会增加请求的延迟，这是系统中常见的权衡。Ahmad 等人的论文 [A+11] 对此做了很好的总结。

## 36.5 用 DMA 更高效地搬运数据

遗憾的是，我们的典型协议还有一个方面需要注意。具体来说，用程序控制 I/O（PIO）向设备传送一大块数据时，CPU 又一次被一件相当琐碎的任务压得喘不过气，浪费了大量时间和精力，而这些本来可以更好地用来运行其他进程。下面这条时间线说明了这个问题：



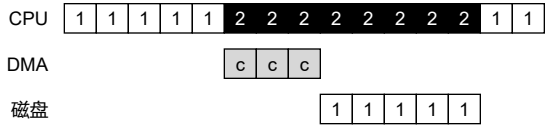

在这条时间线中，进程 1 正在运行，然后它想把一些数据写到磁盘上。于是它发起 I/O，而这次 I/O 必须显式地把数据从内存复制到设备，一次一个字（图中标为 c）。复制完成后，磁盘上的 I/O 才开始，CPU 终于可以拿去干别的事了。

 © 2008–25, ARPACI-DUSSEAU

关键问题：如何降低 PIO 的开销
使用 PIO 时，CPU 要花太多时间亲手在设备和内存之间搬运数据。怎样才能把这项工作卸下来，让 CPU 得到更有效的利用?

这个问题的解决办法就是我们所说的**直接内存访问**（Direct Memory Access，DMA）。DMA 引擎本质上是系统中一个非常专门的设备，它可以在设备和主存之间安排数据传输，而无需 CPU 过多干预。

DMA 的工作方式如下。例如，要把数据传给设备，操作系统会给 DMA 引擎编程，告诉它数据在内存中的位置、要复制多少数据，以及要发给哪个设备。到这一步，操作系统这边就没它的事了，可以去做别的工作。DMA 完成后，DMA 控制器会触发一个中断，操作系统由此得知传输已经完成。修改后的时间线如下：




从时间线可以看到，数据的复制现在由 DMA 控制器负责。由于这段时间 CPU 是空闲的，操作系统可以去做别的事，这里选择了运行进程 2。于是，在进程 1 再次运行之前，进程 2 能用上更多的 CPU 时间。

## 36.6 设备交互的方法

对执行 I/O 的效率问题有了一些认识之后，要把设备纳入现代系统，还得处理另外几个问题。你可能已经注意到了：我们还根本没说操作系统实际上是怎样与设备通信的！于是，问题来了：

关键问题：如何与设备通信
硬件应该怎样与设备通信？应该有专门的指令吗？还是有别的办法?

随着时间推移，发展出了两种主要的设备通信方法。第一种也是最古老的方法（IBM 大型机用了很多年），是使用显式的 **I/O 指令**（I/O instructions）。这些指令规定了操作系统向特定设备寄存器发送数据的方式，从而可以构建出上面描述的协议。

例如，在 x86 上，可以用 `in` 和 `out` 指令与设备通信。要向设备发送数据，调用者需要指定一个存放数据的寄存器，以及一个用来指明设备的特定端口（port）。执行这条指令，就会产生期望的行为。

这类指令通常是**特权**（privileged）指令。操作系统控制着设备，因此只有操作系统才能直接与设备通信。想象一下，如果任何程序都能读写磁盘，那就会一片混乱（一如既往），因为任何用户程序都可以利用这样的漏洞完全控制整台机器。

第二种与设备交互的方法叫作**内存映射 I/O**（memory-mapped I/O）。采用这种方法时，硬件让设备寄存器看起来就像内存位置一样。要访问某个寄存器，操作系统对相应地址执行一次加载（读）或存储（写）；硬件随后把这次加载或存储转给设备，而不是主存。

两种方法谈不上谁比谁有多大优势。内存映射（memory-mapped）方法的好处是不需要新增指令，但这两种方法如今都还在使用。

## 36.7 融入操作系统：设备驱动程序

我们要讨论的最后一个问题是：每种设备的接口都非常特殊，而我们又希望操作系统尽可能通用，那么怎样把设备融入操作系统呢？以文件系统（file system）为例。我们希望文件系统能构建在 SCSI 磁盘、IDE 磁盘、USB 闪存盘等设备之上，并且希望文件系统基本不用关心向这些不同类型的驱动器发出读写请求的种种细节。于是，我们的问题是：

关键问题：如何构建与设备无关的操作系统
怎样让操作系统的大部分保持与设备无关，从而对操作系统的主要子系统隐藏设备交互的细节？

这个问题要靠一项古老的技术来解决：**抽象**（abstraction）。在最底层，操作系统中必须有一部分软件详细了解设备是怎样工作的。我们把这部分软件称为**设备驱动程序**（device driver），所有与设备交互的细节都封装在其中。

以 Linux 的文件系统软件栈（file system software stack）为例，看看这种抽象怎样帮助操作系统的设计和实现。图 36.4 大致画出了 Linux 软件的组织方式。从图中可以看到，文件系统——当然还有它上面的应用程序——完全不知道自己用的是哪一类磁盘；它只是向通用块层（generic block layer）发出块（block）读写请求，通用块层再把请求转给合适的设备驱动程序，由驱动程序（driver）处理发出具体请求的细节。这张图虽然做了简化，但说明了这些细节是怎样对操作系统的大部分隐藏起来的。

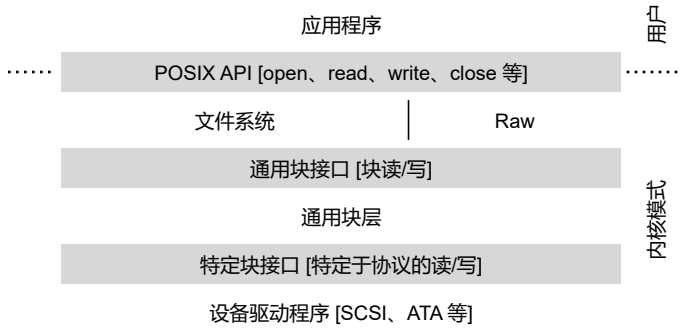



图 36.4：**文件系统栈**

图中还画出了直通设备的 **raw 接口**（raw interface）。有了它，一些特殊的应用程序可以不经过文件抽象（file abstraction），直接读写块，比如后面要介绍的**文件系统检查程序**（file-system checker）[AD14]，以及**磁盘碎片整理**（disk defragmentation）工具。大多数系统都提供这类接口，以支持这些底层的存储管理应用程序。

注意，上面看到的封装也可能有不好的一面。例如，如果某个设备有许多特殊功能，却只能向内核的其余部分呈现一个通用接口，那么这些特殊功能就用不上了。Linux 中的 SCSI 设备就遇到了这种情况：它们的错误报告非常丰富；但因为其他块设备（block device，例如 ATA/IDE）的错误处理要简单得多，所以上层软件收到的永远只是一个通用的 `EIO`（通用 I/O 错误）错误码，SCSI 可能提供的任何额外细节，文件系统都拿不到 [G08]。

有意思的是，由于你可能往系统里插的任何设备都需要设备驱动程序，久而久之，驱动程序在内核代码中占了极大的比例。对 Linux 内核的研究表明，超过 70% 的操作系统代码都在设备驱动程序中 [C01]；基于 Windows 的系统，这一比例很可能也相当高。所以，当有人告诉你操作系统有几百万行代码时，他们真正想说的是，操作系统有几百万行设备驱动程序代码。当然，在任何一台具体的机器上，这些代码大部分可能都不会运行（也就是说，同一时间只有少数几个设备连在系统上）。也许更让人沮丧的是，驱动程序常常由“业余人士”（而不是全职的内核开发者）编写，因此 bug 往往多得多，是导致内核崩溃的主要原因 [S03]。

## 36.8 案例研究：一个简单的 IDE 磁盘驱动程序

为了再深入一点，我们来快速看看一个真实的设备：IDE 磁盘驱动器（disk drive）[L94]。我们按参考资料 [W10] 的描述总结它的协议；还会瞄一眼 xv6 的源代码，看看一个能工作的 IDE 驱动程序的简单例子 [CK+08]。

IDE 磁盘向系统呈现的接口很简单，由四类寄存器组成：控制、命令块（command block）、状态和错误。（在 x86 上）用 `in` 和 `out` 这两条 I/O 指令（I/O instruction）读写特定的“I/O 地址”（比如下文的 `0x3F6`），就能访问这些寄存器。

假设设备已经初始化，与它交互的基本协议如下：

- **等待驱动器就绪**。读取状态寄存器（status register，0x1F7），直到驱动器处于 READY 且不处于 BUSY 状态。
- **把参数写入命令寄存器**。把扇区数（sector count）、要访问的扇区的逻辑块地址（LBA），以及驱动器号（主盘为 0x00，从盘为 0x10，因为 IDE 只允许接两个驱动器）写入命令寄存器（command register，0x1F2–0x1F6）。
- **启动 I/O**。把 READ|WRITE 命令写入命令寄存器（command register，0x1F7）。
- **传送数据（写操作）：**等待，直到驱动器状态为 READY 且 DRQ（驱动器请求数据）；然后把数据写到数据端口（data port）。
- **处理中断**。最简单的做法是每传送一个扇区（sector）处理一次中断；更复杂的做法允许批处理（batching），在整个传输完成时只产生最后一次中断。
- **错误处理**。每次操作后都读取状态寄存器（status register）。如果 ERROR 位为 1，就读取错误寄存器查看详情。

这个协议的大部分都可以在 xv6 的 IDE 驱动程序中找到（图 36.6，第 382 页）。这个驱动程序初始化之后，靠四个主要函数工作。第一个是 `ide_rw()`：如果有其他请求在等

```
控制寄存器:
  地址 0x3F6 = 0x08 (0000 1RE0): R=复位, E=0 表示"启用中断"

命令块寄存器:
  地址 0x1F0 = 数据端口
  地址 0x1F1 = 错误
  地址 0x1F2 = 扇区数
  地址 0x1F3 = LBA 低字节
  地址 0x1F4 = LBA 中字节
  地址 0x1F5 = LBA 高字节
  地址 0x1F6 = 1B1D TOP4LBA: B=LBA, D=驱动器
  地址 0x1F7 = 命令/状态

状态寄存器 (地址 0x1F7):
    7     6     5     4    3    2     1     0
  BUSY READY FAULT SEEK  DRQ CORR IDDEX ERROR

错误寄存器 (地址 0x1F1): (ERROR==1 时检查)
    7     6     5     4    3    2     1     0
  BBK   UNC   MC   IDNF  MCR ABRT T0NF  AMNF

  BBK  = 坏块
  UNC  = 不可纠正的数据错误
  MC   = 介质已更换
  IDNF = 未找到 ID 标记
  MCR  = 请求更换介质
  ABRT = 命令已中止
  T0NF = 未找到 0 号磁道
  AMNF = 未找到地址标记
```

图 36.5：**IDE 接口**

待，它就把请求放入队列（queue），否则直接发给磁盘（通过 `ide_start_request()`）；无论哪种情况，这个例程都会等待请求完成，调用进程随之进入睡眠。第二个是 `ide_start_request()`，用来把请求（如果是写操作，可能还有数据）发给磁盘；它调用 x86 的 `in` 和 `out` 指令，分别读写设备寄存器。启动请求的例程会用到第三个函数 `ide_wait_ready()`，在发出请求之前确保驱动器已经就绪。最后，发生中断时会调用 `ide_intr()`；它从设备读取数据（如果请求是读而不是写），唤醒等待 I/O 完成的进程，并且（如果 I/O 队列里还有请求）通过 `ide_start_request()` 启动下一次 I/O。

## 36.9 历史注记

在结束之前，我们简要介绍一下其中一些基本思想的起源。如果想了解更多，请阅读 Smotherman 精彩的总结 [S08]。

中断是一个很古老的思想，最早的机器上就已经有了。例如，20 世纪 50 年代初的 UNIVAC 就有某种形式的中断向量（interrupt vectoring），但这项功能具体是哪一年出现

```
static int ide_wait_ready() {
  while (((int r = inb(0x1f7)) & IDE_BSY) || !(r & IDE_DRDY))
    ; // 循环，直到驱动器不忙
  // 出错时返回 -1，否则返回 0
}

static void ide_start_request(struct buf *b) {
  ide_wait_ready();
  outb(0x3f6, 0);                // 产生中断
  outb(0x1f2, 1);                // 多少个扇区?
  outb(0x1f3, b->sector & 0xff); // LBA 放这里……
  outb(0x1f4, (b->sector >> 8) & 0xff);  // ……还有这里
  outb(0x1f5, (b->sector >> 16) & 0xff); // ……还有这里!
  outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f));
  if(b->flags & B_DIRTY){
    outb(0x1f7, IDE_CMD_WRITE);     // 这是一次写操作
    outsl(0x1f0, b->data, 512/4);  // 数据也一起传过去!
  } else {
    outb(0x1f7, IDE_CMD_READ);      // 这是一次读操作（没有数据）
  }
}

void ide_rw(struct buf *b) {
  acquire(&ide_lock);
  for (struct buf **pp = &ide_queue; *pp; pp=&(*pp)->qnext)
    ;                               // 遍历队列
  *pp = b;                          // 把请求加到队尾
  if (ide_queue == b)               // 如果队列为空
    ide_start_request(b);           // 把请求发给磁盘
  while ((b->flags & (B_VALID|B_DIRTY)) != B_VALID)
    sleep(b, &ide_lock);            // 等待完成
  release(&ide_lock);
}

void ide_intr() {
  acquire(&ide_lock);
  struct buf *b = ide_queue;
  if (!(b->flags & B_DIRTY) && ide_wait_ready() >= 0)
    insl(0x1f0, b->data, 512/4);    // 如果是读：取回数据
  b->flags |= B_VALID;
  b->flags &= ~B_DIRTY;
  wakeup(b);                        // 唤醒等待的进程
  if ((ide_queue = b->qnext) != 0) // 启动下一个请求
    ide_start_request(ide_queue);  //  （如果有的话）
  release(&ide_lock);
}
```

图 36.6：**xv6 的 IDE 磁盘驱动程序（简化版）**

的，现在已经说不清了 [S08]。可悲的是，计算机的历史才刚起步，我们就已经开始弄不清它的源头了。

至于哪台机器最先引入了 DMA 的思想，也有一些争论。例如，Knuth 等人认为是 DYSEAC（一台“移动”机器，在当时意味着可以用拖车拉着走），而另一些人认为 IBM SAGE 可能才是第一台 [S08]。无论如何，到 50 年代中期，已经出现了 I/O 设备直接与内存通信、完成后再中断 CPU 的系统。

这段历史很难追溯，因为这些发明都和一些真实的、有时还很冷门的机器联系在一起。例如，有人认为林肯实验室的 TX-2 是第一台拥有向量中断（vectored interrupt）的机器 [S08]，但这一点很难说清楚。

由于这些思想相对来说显而易见——想到让 CPU 在慢速 I/O 进行时去做别的事，并不需要爱因斯坦式的飞跃——也许我们执着于“谁是第一个”本身就是方向错了。可以肯定的是：人们在建造这些早期机器时，很明显就意识到需要 I/O 支持。中断、DMA 及相关思想，都是 CPU 快、设备慢这一本质特点的直接产物；如果你当时也在场，或许也会想到类似的点子。

## 36.10 小结

现在你应该对操作系统怎样与设备交互有了最基本的了解。我们介绍了两种提高设备效率的技术：中断和 DMA；还描述了访问设备寄存器的两种方法：显式 I/O 指令（I/O instruction）和内存映射 I/O（memory-mapped I/O）。最后，我们介绍了设备驱动程序的概念，说明了操作系统本身怎样封装底层细节，从而让操作系统的其余部分更容易做到与设备无关。

## 参考文献

[A+11] "vIC: Interrupt Coalescing for Virtual Machine Storage Device IO" by Irfan Ahmad, Ajay Gulati, Ali Mashtizadeh. USENIX '11. *A terrific survey of interrupt coalescing in traditional and virtualized environments.*

[AD14] "Operating Systems: Three Easy Pieces" (Chapters: Crash Consistency: FSCK and Journaling and Log-Structured File Systems) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. Arpaci-Dusseau Books, 2014. *A description of a file-system checker and how it works, which requires low-level access to disk devices not normally provided by the file system directly.*

[C01] "An Empirical Study of Operating System Errors" by Andy Chou, Junfeng Yang, Benjamin Chelf, Seth Hallem, Dawson Engler. SOSP '01. *One of the first papers to systematically explore how many bugs are in modern operating systems. Among other neat findings, the authors show that device drivers have something like seven times more bugs than mainline kernel code.*

[CK+08] "The xv6 Operating System" by Russ Cox, Frans Kaashoek, Robert Morris, Nickolai Zeldovich. From: http://pdos.csail.mit.edu/6.828/2008/index.html. *See* `ide.c` *for the IDE device driver, with a few more details therein.*

[D07] "What Every Programmer Should Know About Memory" by Ulrich Drepper. November, 2007. Available: http://www.akkadia.org/drepper/cpumemory.pdf. *A fantastic read about modern memory systems, starting at DRAM and going all the way up to virtualization and cache-optimized algorithms.*

[G08] "EIO: Error-handling is Occasionally Correct" by Haryadi Gunawi, Cindy Rubio-Gonzalez, Andrea Arpaci-Dusseau, Remzi Arpaci-Dusseau, Ben Liblit. FAST '08, San Jose, CA, February 2008. *Our own work on building a tool to find code in Linux file systems that does not handle error return properly. We found hundreds and hundreds of bugs, many of which have now been fixed.*

[H17] "Intel Core i7-7700K review: Kaby Lake Debuts for Desktop" by Joel Hruska. January 3, 2017. www.extremetech.com/extreme/241950-intels-core-i7-7700k-reviewed-kaby-lake-debuts-desktop. *An in-depth review of a recent Intel chipset, including CPUs and the I/O subsystem.*

[H18] "Hacker News" by Many contributors. Available: https://news.ycombinator.com. *One of the better aggregators for tech-related stuff. Once back in 2014, this book became a highly-ranked entry, leading to 1 million chapter downloads in just one day! Sadly, we have yet to re-experience such a high.*

[L94] "AT Attachment Interface for Disk Drives" by Lawrence J. Lamers. Reference number: ANSI X3.221, 1994. Available: ftp://ftp.t10.org/t13/project/d0791r4c-ATA-1.pdf. *A rather dry document about device interfaces. Read it at your own peril.*

[MR96] "Eliminating Receive Livelock in an Interrupt-driven Kernel" by Jeffrey Mogul, K. K. Ramakrishnan. USENIX '96, San Diego, CA, January 1996. *Mogul and colleagues did a great deal of pioneering work on web server network performance. This paper is but one example.*

[S08] "Interrupts" by Mark Smotherman. July '08. Available: http://people.cs.clemson.edu/~mark/interrupts.html. *A treasure trove of information on the history of interrupts, DMA, and related early ideas in computing.*

[S03] "Improving the Reliability of Commodity Operating Systems" by Michael M. Swift, Brian N. Bershad, Henry M. Levy. SOSP '03. *Swift's work revived interest in a more microkernel-like approach to operating systems; minimally, it finally gave some good reasons why address-space based protection could be useful in a modern OS.*

[W10] "Hard Disk Driver" by Washington State Course Homepage. Available online at this site: http://eecs.wsu.edu/~cs460/cs560/HDdriver.html. *A nice summary of a simple IDE disk drive's interface and how to build a device driver for it.*

# 37

# 硬盘驱动器

上一章介绍了 I/O 设备的一般概念，并展示了操作系统可能会怎样和这样的“怪兽”打交道。在这一章，我们更深入地讨论一种具体的设备：**硬盘驱动器**（hard disk drive）。几十年来，这种驱动器一直是计算机系统中持久数据存储的主要形式，文件系统（file system）技术（马上就会讲到）的很多发展也都是以它的行为为前提的。因此，在构建管理磁盘的文件系统软件之前，有必要先了解磁盘的工作细节。这些细节有很多可以在 Ruemmler 和 Wilkes [RW92] 以及 Anderson、Dykes 和 Riedel [ADR03] 的优秀论文中找到。

关键问题：如何在磁盘上存储和访问数据
现代硬盘驱动器是怎样存储数据的？它的接口是什么样的？数据实际上是怎样布局和访问的？磁盘调度怎样提高性能？

## 37.1 接口

我们先来了解现代磁盘驱动器（disk drive）的接口。所有现代驱动器的基本接口都很简单。驱动器由大量扇区（sector，即 512 字节的块）组成，每个扇区都可以读或写。在一个有 $n$ 个扇区的磁盘上，扇区编号从 0 到 $n-1$。因此，我们可以把磁盘看成一个扇区数组；0 到 $n-1$ 就是驱动器的**地址空间**（address space）。

多扇区操作（multi-sector operation）也是可以的，许多文件系统一次就会读写 4 KB（或者更多）。然而，在更新磁盘时，驱动器厂商唯一保证的是单次 512 字节的写入是**原子的**（atomic），也就是说，它要么完整地完成，要么根本不完成；因此，如果发生意外断电（power loss），较大的写入可能只完成了一部分（有时称为**撕裂写**，torn write）。

大多数磁盘驱动器的使用者都会做一些假设，但这些假设并没有直接写在接口里；Schlosser 和 Ganger 把它称为磁盘驱动器的“不成文的契约”[SG04]。具体来说，通常可以假设：在驱动器的地址空间中，访问两个相距较近的块（block）[1]要比访问两个相距很远的块快。通常还可以假设：访问一段连续的块（即顺序读或顺序写）是最快的访问方式，一般比任何更随机的访问模式快得多。

[1]我们（以及其他人）常常把**块**（block）和**扇区**（sector）这两个词混着用，指望读者能根据上下文明白到底指什么。为此说声抱歉！

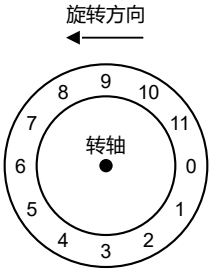



图 37.1：**只有一个磁道的磁盘**

## 37.2 基本几何结构

下面来认识现代磁盘的几个组成部分。先从**盘片**（platter）说起，它是一个圆形的硬质表面，靠在上面引起磁性变化来持久地存储数据。一个磁盘可以有一个或多个盘片；每个盘片有两面，每一面称为一个**盘面**（surface）。这些盘片通常由某种坚硬的材料（比如铝）制成，上面再涂一层很薄的磁性层，让驱动器即使断电也能持久地保存这些位。

所有盘片都绑在**主轴**（spindle）上，主轴连着一个电机，驱动器通电时，电机以恒定（固定）的速率带动盘片旋转。转速通常用**每分钟转数**（rotations per minute，RPM）来衡量，现代驱动器的典型值在 7,200 RPM 到 15,000 RPM 之间。注意，我们常常关心的是转一圈所用的时间，例如，转速为 10,000 RPM 的驱动器转一圈大约需要 6 毫秒（6 ms）。

每个盘面上，数据编码在一圈圈由扇区组成的同心圆上；这样的一圈称为一条**磁道**（track）。一个盘面上有成千上万条磁道，排列得非常紧密，一根头发丝的宽度就能容纳几百条磁道。

要在盘面上读写，我们需要一种机制，既能感知（即读取）磁盘上的磁性图案，也能引起（即写入）磁性变化。读写这一过程由**磁头**（disk head）完成；驱动器的每个盘面都有一个这样的磁头。磁头安装在一个**磁盘臂**（disk arm）上，磁盘臂在盘面上移动，把磁头定位到所需的磁道上方。

## 37.3 一个简单的磁盘驱动器

我们一次加一条磁道，逐步搭起一个模型，以此来理解磁盘是怎样工作的。假设我们有一个只有一条磁道的简单磁盘（图 37.1）。这条磁道只有 12 个扇区，每个扇区 512 字节（回想一下，这是我们常用的扇区大小），因此用 0 到 11 来编址。这里只有一个盘片，它绕着主轴（spindle）旋转，主轴上连着一个电机。

当然，单单一条磁道没什么意思；我们希望能读写这些扇区，因此需要一个磁头，装在磁盘臂上，就像现在看到的这样（图 37.2）。在图中，装在磁盘臂末端的磁头位于扇区 6 上方，盘面按逆时针方向旋转。

 © 2008–23, ARPACI-DUSSEAU

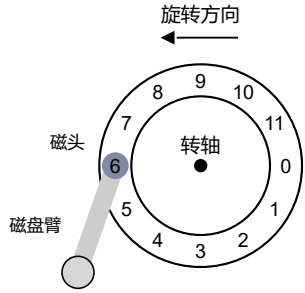



图 37.2：**单个磁道加一个磁头**

### 单磁道延迟：旋转延迟

为了理解在这个简单的单磁道磁盘上请求是怎样处理的，设想我们现在收到一个读取块 0 的请求。磁盘该怎样处理这个请求呢?

在我们这个简单的磁盘上，磁盘不需要做多少事。具体来说，它只要等待所需的扇区转到磁头下方就行了。在现代驱动器中，这种等待很常见，又是 I/O 服务时间中一个相当重要的组成部分，所以它有一个专门的名字：**旋转延迟**（rotational delay，有时也叫 **rotation delay**，不过这听起来有点怪）。在这个例子中，如果转一整圈的延迟是 $R$，磁头又是从扇区 6 开始的，那么磁盘大约要等 $\frac{R}{2}$ 的旋转延迟，扇区 0 才会转到读写磁头（read/write head）下方。在这条磁道上，最坏的请求是读扇区 5，处理这样的请求，几乎要等整整一圈的旋转延迟。

### 多磁道：寻道时间

到目前为止，我们的磁盘只有一条磁道，这不太现实；现代磁盘当然有几百万条磁道。所以，我们来看一个稍微更现实一点的盘面，它有三条磁道，见图 37.3 左。

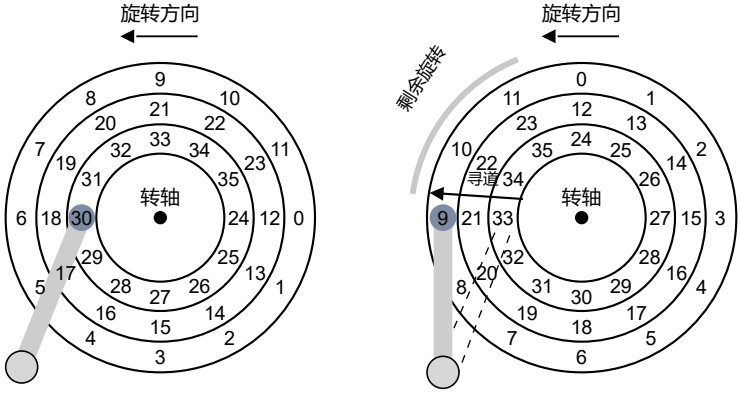



图 37.3：**三个磁道加一个磁头（右：带寻道）**

 © 2008–23, ARPACI-DUSSEAU

在图中，磁头当前位于最内侧的磁道上方，这条磁道包含扇区 24 到 35；往外一条磁道包含接下来的一组扇区 12 到 23，最外侧的磁道包含最前面的扇区 0 到 11。

为了理解驱动器怎样访问某个扇区，我们跟踪一个对较远扇区的请求（例如读扇区 11），看看会发生什么。为了处理这次读，驱动器必须先把磁盘臂移到正确的磁道，这个过程叫作**寻道**（seek）。寻道和旋转一样，是代价最高的磁盘操作之一。

需要指出的是，寻道分好几个阶段：首先是磁盘臂开始移动的加速（acceleration）阶段；然后是磁盘臂全速移动的匀速滑行（coasting）阶段；接着是磁盘臂减慢下来的减速（deceleration）阶段；最后是把磁头小心地停在正确磁道上方的 *settling* 阶段。**settling time** 往往相当长，例如 0.5 到 2 毫秒，因为驱动器必须确保找到了正确的磁道——想象一下，如果它只是停在附近会怎样！

寻道结束后，磁盘臂已经把磁头定位到了正确的磁道上方。寻道的示意见图 37.3（右）。

可以看到，在寻道期间，磁盘臂移到了目标磁道，而盘片当然也转动了，这里大约转过了 3 个扇区。因此，扇区 9 正要从磁头下方经过，我们只需再等一小段旋转延迟就能完成传输。

扇区 11 经过磁头下方时，I/O 的最后一个阶段就开始了，这个阶段叫作**传输**（transfer），数据在这时从盘面读出或写入盘面。这样，我们就得到了 I/O 时间的完整图景：先寻道，再等待旋转延迟，最后传输。

### 其他一些细节

虽然我们不会在这上面花太多时间，但关于硬盘的工作方式，还有一些其他有意思的细节。许多驱动器采用某种**磁道偏斜**（track skew），确保即使跨越磁道边界，顺序读也能顺利完成。在我们这个简单的示例磁盘上，它可能是图 37.4（第 388 页）的样子。

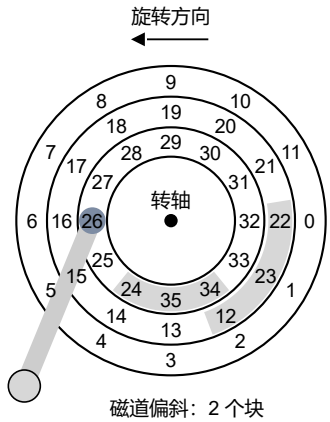



图 37.4：**三个磁道：磁道偏斜为 2**

扇区之所以常常像这样偏斜，是因为从一条磁道切换到另一条磁道时，即使是相邻的磁道，磁盘也需要时间重新定位磁头。如果没有这种偏斜，磁头移到下一条磁道时，所需的下一个块已经转过了磁头，驱动器就得等上几乎一整圈的旋转延迟才能访问下一个块。

另一个现实是，外侧磁道往往比内侧磁道有更多的扇区，这是几何结构造成的；外面的空间就是更大。这类驱动器通常称为 **multi-zoned** 磁盘驱动器：磁盘被组织成多个区（zone），一个区就是一个盘面上一组连续的磁道。同一个区内每条磁道的扇区数相同，外侧的区比内侧的区扇区更多。

最后，任何现代磁盘驱动器都有一个重要组成部分：**缓存**（cache），由于历史原因，有时也叫**磁道缓冲区**（track buffer）。这个缓存就是少量内存（通常大约 8 MB 或 16 MB），驱动器可以用它保存从磁盘读出或要写入磁盘的数据。例如，从磁盘读一个扇区时，驱动器可能会决定把这条磁道上的所有扇区都读进来，缓存在它的内存里；这样，后续对同一磁道的任何请求，驱动器都能很快响应。

对于写操作，驱动器有一个选择：是在把数据放进自己的内存时就确认写入已完成，还是在数据真正写到磁盘上之后再确认？前者称为**回写**（write back）缓存（有时也叫 **immediate reporting**），后者称为**直写**（write through）。回写缓存（write-back caching）有时会让驱动器显得“更快”，但也可能很危险：如果文件系统或应用程序为了正确性要求数据按一定顺序写到磁盘上，回写缓存就可能带来问题（详见关于文件系统日志的那一章）。

## 37.4 I/O 时间：算一算

现在我们有了磁盘的抽象模型，可以做一点分析，更好地理解磁盘的性能。具体来说，我们现在可以把 I/O 时间表示为三个主要组成部分之和：

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer} \tag{37.1}$$

注意，I/O 速率（$R_{I/O}$）往往更便于比较不同的驱动器（我们下面就会这样做），它很容易由时间算出来。只要用传输的大小除以所用的时间：

$$R_{I/O} = \frac{Size_{Transfer}}{T_{I/O}} \tag{37.2}$$

为了对 I/O 时间有更直观的感受，我们来做下面的计算。假设我们关心两种工作负载（workload）。第一种叫**随机**（random）工作负载，它向磁盘上的随机位置发出小的（例如 4 KB）读请求。随机工作负载在许多重要的应用中都很常见，包括数据库管理系统（database management system）。第二种叫**顺序**（sequential）工作负载，它只是从磁盘上连续读取大量扇区，不跳来跳去。顺序访问模式也很常见，因此同样重要。

为了理解随机和顺序两种工作负载在性能上的差别，我们需要先对磁盘驱动器做一些假设。我们来看希捷（Seagate）的两款现代磁盘。第一款是 Cheetah 15K.5 [S09b]，一款高性能的 SCSI 驱动器。第二款是 Barracuda [S09a]，一款为容量而打造的驱动器。两者的详细参数见图 37.5。

可以看到，这两款驱动器的特性大不相同，在很多方面很好地概括了磁盘驱动器市场的两个重要组成部分。第一个是“高性能”驱动器市场，这类驱动器的设计目标是转速尽可能

 © 2008–23, ARPACI-DUSSEAU

补充：量纲分析

还记得化学课上你是怎么解几乎每一道题的吗？只要把单位摆好，让它们互相约掉，答案似乎就自己冒出来了。这种化学魔法有个高大上的名字，叫作**量纲分析**（dimensional analysis），而它在计算机系统分析中其实也很有用。

我们来看一个例子，看看量纲分析（dimensional analysis）是怎样工作的，又为什么有用。假设你要算出磁盘转一圈需要多少毫秒。不巧的是，你只知道磁盘的 **RPM**，即**每分钟转数**（rotations per minute）。假设我们说的是一块 10K RPM 的磁盘（即每分钟转 10,000 圈）。怎样列出量纲分析（dimensional analysis）的式子，才能得到以毫秒为单位的每圈时间呢？

为此，我们先把想要的单位写在左边；这里我们想得到每转一圈的时间（以毫秒计），所以就这样写：$\frac{\text{时间 (ms)}}{\text{1 圈}}$。然后写下我们知道的所有东西，并尽可能让单位约掉。首先是 $\frac{\text{1 分钟}}{\text{10,000 圈}}$（把“圈”留在分母上，因为左边它就在分母上），然后用 $\frac{\text{60 秒}}{\text{1 分钟}}$ 把分钟换成秒，最后用 $\frac{\text{1000 ms}}{\text{1 秒}}$ 把秒换成毫秒。最终结果如下（单位都漂亮地约掉了）：

$$\frac{\text{时间 (ms)}}{\text{1 圈}} = \frac{1\cancel{\text{分钟}}}{\text{10,000 圈}} \cdot \frac{60\cancel{\text{秒}}}{1\cancel{\text{分钟}}} \cdot \frac{\text{1000 ms}}{1\cancel{\text{秒}}} = \frac{\text{60,000 ms}}{\text{10,000 圈}} = \frac{\text{6 ms}}{\text{圈}}$$

从这个例子可以看出，量纲分析（dimensional analysis）把看似凭直觉的事情变成了一个简单、可重复的过程。除了上面的 RPM 计算，它在 I/O 分析中也经常派得上用场。例如，题目常常会告诉你磁盘的传输速率，比如 100 MB/秒，然后问你：传输一个 512 KB 的块要多少毫秒？用量纲分析（dimensional analysis），这很容易：

$$\frac{\text{时间 (ms)}}{\text{1 次请求}} = \frac{512\cancel{\text{KB}}}{\text{1 次请求}} \cdot \frac{1\cancel{\text{MB}}}{1024\cancel{\text{KB}}} \cdot \frac{1\cancel{\text{秒}}}{100\cancel{\text{MB}}} \cdot \frac{\text{1000 ms}}{1\cancel{\text{秒}}} = \frac{\text{5 ms}}{\text{请求}}$$

快、寻道时间（seek time）尽可能短、传输数据尽可能快。第二个是“容量”市场，这里每字节的成本最重要；因此，这类驱动器更慢，但会在可用空间里塞进尽可能多的位。

根据这些数字，我们可以开始计算这两款驱动器在上面描述的两种工作负载下表现如何。先来看随机工作负载。假设每次 4 KB 的读都发生在磁盘上的随机位置，我们可以算出每次这样的读要花多长时间。在 Cheetah 上：

$$T_{seek} = 4\ ms, T_{rotation} = 2\ ms, T_{transfer} = 30\ microsecs \tag{37.3}$$

| | Cheetah 15K.5 | Barracuda |
|---|---|---|
| 容量 | 300 GB | 1 TB |
| 转速（RPM） | 15,000 | 7,200 |
| 平均寻道时间 | 4 ms | 9 ms |
| 最大传输速率 | 125 MB/s | 105 MB/s |
| 盘片数 | 4 | 4 |
| 缓存 | 16 MB | 16/32 MB |
| 接口 | SCSI | SATA |

图 37.5：**磁盘驱动器规格：SCSI 与 SATA**

 © 2008–23, ARPACI-DUSSEAU

提示：顺序地使用磁盘
只要有可能，就顺序地在磁盘上读写数据。如果做不到，至少也要考虑按大块传输数据：越大越好。如果 I/O 以零碎的小块随机进行，I/O 性能会急剧下降。用户也会跟着受罪。你自己也会受罪，因为你知道，这些罪都是你那些粗心的随机 I/O 造成的。

平均寻道时间（4 毫秒）直接取的是厂商公布的平均时间；注意，从盘面一端到另一端的完整寻道可能要花两到三倍的时间。平均旋转延迟可以直接由 RPM 算出来。15000 RPM 等于 250 RPS（每秒转数）；因此，转一圈需要 4 ms。平均来说，磁盘要转半圈，所以平均时间是 2 ms。最后，传输时间（transfer time）就是传输大小除以峰值传输速率；这里它小得几乎可以忽略（30 *微秒*；注意，1000 微秒才是 1 毫秒！）。

因此，根据上面的公式，Cheetah 的 $T_{I/O}$ 大约是 6 ms。要计算 I/O 速率，只要用传输大小除以平均时间，就得到 Cheetah 在随机工作负载下的 $R_{I/O}$ 大约是 0.66 MB/s。对 Barracuda 做同样的计算，得到的 $T_{I/O}$ 大约是 13.2 ms，慢了一倍多，因此速率大约是 0.31 MB/s。

现在来看顺序工作负载。这里可以假设，在一次很长的传输之前只有一次寻道和一次旋转。为简单起见，假设传输大小是 100 MB。那么，Cheetah 和 Barracuda 的 $T_{I/O}$ 分别大约是 800 ms 和 950 ms。因此，I/O 速率分别非常接近 125 MB/s 和 105 MB/s 的峰值传输速率。图 37.6 汇总了这些数字。

这张图告诉我们几件重要的事。第一，也是最重要的一点：随机工作负载和顺序工作负载之间的驱动器性能差距巨大，Cheetah 相差将近 200 倍，Barracuda 相差 300 倍以上。于是我们得到了计算机历史上最显而易见的设计提示。

第二点更微妙一些：高端的“性能”驱动器和低端的“容量”驱动器之间，性能差距很大。由于这个原因（还有其他原因），人们常常愿意花大价钱买前者，而对后者则尽量买得便宜。

## 37.5 磁盘调度

由于 I/O 的开销很高，操作系统历来都会参与决定以什么顺序把 I/O 发给磁盘。更具体地说，给定一组 I/O 请求，**磁盘调度程序**（disk scheduler）会检查这些请求，决定下一个调度哪一个 [SCO90, JW91]。

作业调度通常不知道每个作业有多长；而磁盘调度不同，我们能比较准确地估计一个“作业”（即磁盘请求）要花多长时间。估计出一个请求的寻道时间和可能的旋转延迟，磁盘调度程序就能知道每个请求要花多长时间，从而贪心地（greedily）先选出处理时间最短

| | Cheetah | Barracuda |
|---|---|---|
| $R_{I/O}$ 随机 | 0.66 MB/s | 0.31 MB/s |
| $R_{I/O}$ 顺序 | 125 MB/s | 105 MB/s |

图 37.6：**磁盘驱动器性能：SCSI 与 SATA**

 © 2008–23, ARPACI-DUSSEAU

补充：“平均”寻道时间的计算

在许多书和论文中，你会看到平均磁盘寻道时间大约是完整寻道时间的三分之一。这个说法是从哪儿来的?

原来，它来自一个基于平均寻道距离（而不是时间）的简单计算。把磁盘想象成一组磁道，编号从 0 到 $N$。任意两条磁道 $x$ 和 $y$ 之间的寻道距离，就是它们之差的绝对值：$|x-y|$。

要计算平均寻道距离，只需先把所有可能的寻道距离加起来：

$$\sum_{x=0}^{N}\sum_{y=0}^{N}|x-y| \ . \tag{37.4}$$

然后，再除以不同寻道的总数：$N^2$。为了求和，我们直接用积分形式：

$$\int_{x=0}^{N}\int_{y=0}^{N}|x-y|\,\mathrm{d}y\,\mathrm{d}x. \tag{37.5}$$

为了计算内层积分，我们把绝对值拆开：

$$\int_{y=0}^{x}(x-y)\,\mathrm{d}y+\int_{y=x}^{N}(y-x)\,\mathrm{d}y. \tag{37.6}$$

解出来是 $\left(xy-\frac{1}{2}y^2\right)\,|_0^x+\left(\frac{1}{2}y^2-xy\right)\,|_x^N$，可以化简为 $\left(x^2-Nx+\frac{1}{2}N^2\right)$。现在我们要计算外层积分：

$$\int_{x=0}^{N}\left(x^2-Nx+\frac{1}{2}N^2\right)\mathrm{d}x, \tag{37.7}$$

结果是：

$$\left(\frac{1}{3}x^3-\frac{N}{2}x^2+\frac{N^2}{2}x\right)\,|_0^N=\frac{N^3}{3}. \tag{37.8}$$

别忘了，我们还要除以寻道的总数（$N^2$）才能得到平均寻道距离：$\left(\frac{N^3}{3}\right)/(N^2)=\frac{1}{3}N$。所以，在所有可能的寻道中，磁盘上的平均寻道距离是完整距离的三分之一。以后再听到平均寻道是完整寻道的三分之一，你就知道它是怎么来的了。

的那个。因此，磁盘调度程序在工作时会尽量遵循 **SJF（最短作业优先，shortest job first）原则**。

### SSTF：最短寻道时间优先

一种早期的磁盘调度方法叫作**最短寻道时间优先**（shortest-seek-time-first，**SSTF**），也叫**最短寻道优先**（shortest-seek-first，**SSF**）。SSTF 按磁道对 I/O 请求队列（queue）排序，先选最近磁道上的请求来完成。例如，假设磁头当前位于内侧磁道上方，我们有两个请求，

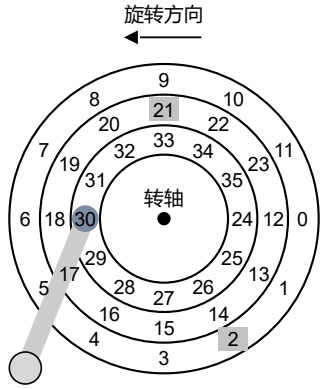



图 37.7：**SSTF：调度请求 21 和 2**

分别访问扇区 21（中间磁道）和扇区 2（外侧磁道），那么我们就先发出对 21 的请求，等它完成后，再发出对 2 的请求（图 37.7，第 393 页）。

在这个例子中，SSTF 先寻道到中间磁道，再寻道到外侧磁道，效果很好。然而，SSTF 并不是万灵药，原因如下。第一，主机操作系统看不到驱动器的几何结构；它看到的只是一个块数组。好在这个问题比较容易解决。操作系统可以不用 SSTF，而是简单地实现 **nearest-block-first**（**NBF**），即每次挑块地址（block address）最近的请求作为下一个来调度。

第二个问题更根本：**饥饿**（starvation）。在上面的例子中，设想有源源不断的请求访问磁头当前所在的内侧磁道。那么，纯粹的 SSTF 方法会完全忽略对其他任何磁道的请求。于是，问题的关键在于：

关键问题：如何处理磁盘饥饿
怎样才能实现类似 SSTF 的调度，同时避免饥饿？

## 电梯算法（又称 SCAN 或 C-SCAN）

这个问题的答案很早以前就有人提出来了（例如见 [CKR72]），而且相当直观。这个算法最初叫作 **SCAN**，它只是在磁盘上来回移动，按磁道的顺序依次处理请求。我们把在磁盘上的一趟移动——从外侧磁道到内侧磁道，或者从内侧到外侧——称为一次*扫描*（sweep）。因此，如果某个请求要访问的磁道在这次扫描中已经扫过了，它不会马上得到处理，而是排队等下一次（反方向的）扫描。

SCAN 有许多变体，做的事情都差不多。例如，Coffman 等人提出了 **F-SCAN**，它在扫描期间冻结待处理的队列 [CKR72]；这样，扫描期间到达的请求会被放进另一个队列，留到以后处理。这样做推迟了处理晚到（但离得更近）的请求，从而避免了远处请求的饥饿。

**C-SCAN** 是另一种常见的变体，是 **Circular SCAN**（循环扫描）的简称。这个算法不在磁盘上双向扫描，而是只从外向内扫描，然后在外侧磁道复位，重新开始。这样对内侧和外侧

 © 2008–23, ARPACI-DUSSEAU

磁道更公平一些，因为纯粹来回扫描的 SCAN 偏向中间的磁道：处理完外侧磁道后，SCAN 要经过中间两次，才会再回到外侧磁道。

SCAN 算法（及其近亲）有时被称为**电梯**（elevator）算法，原因现在应该很清楚了：它的行为就像电梯一样，要么往上走，要么往下走，而不是仅仅根据哪层楼更近来响应各层的请求。想象一下，如果你正从 10 楼下到 1 楼，有人在 3 楼进来按了 4 楼，结果电梯因为 4 楼比 1 楼“更近”就往上去了 4 楼，那该多烦人！可以看到，电梯算法（elevator algorithm）用在现实生活中，可以防止电梯里打起架来。用在磁盘上，它只是防止了饥饿。

遗憾的是，SCAN 及其近亲并不是最好的调度技术。具体来说，SCAN（甚至 SSTF）并没有尽可能严格地遵循 SJF 原则：它们忽略了旋转。于是，又一个关键问题来了：

关键问题：如何把磁盘旋转的开销考虑进去
怎样实现一个同时考虑寻道和旋转、从而更接近 SJF 的算法？

### SPTF：最短定位时间优先

**最短定位时间优先**（shortest positioning time first，**SPTF**）调度（有时也叫 **shortest access time first**，**SATF**）正是这个问题的解决办法。不过在讨论它之前，我们先把问题弄得更清楚一些。图 37.8 给出了一个例子。

在这个例子中，磁头当前位于内侧磁道的扇区 30 上方。于是，调度程序（scheduler）必须决定：下一个该调度中间磁道上的扇区 16，还是外侧磁道上的扇区 8？

答案当然是“看情况”。在工程中，“看情况”几乎总是正确答案，这反映出权衡取舍是工程师生活的一部分；这类格言在紧要关头也很好用，比如老板问了你一个答不上来的问题时，你不妨试试这句妙语。不过，知道为什么要看情况几乎总是更好，这正是我们这里要讨论的。

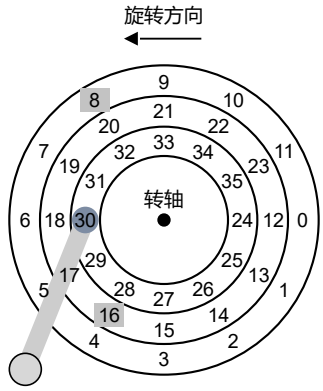



图 37.8：**SSTF：有时还不够好**

 © 2008–23, ARPACI-DUSSEAU

提示：答案总是“看情况”（Livny 定律）
几乎任何问题都可以用“看情况”来回答，我们的同事 Miron Livny 总是这么说。不过，用的时候要小心，因为如果你用这种方式回答太多问题，人们就再也不会问你问题了。例如，有人问：“想去吃午饭吗?”你回答：“看情况，你去吗?”

这里要看的是寻道时间和旋转时间的相对大小。在我们的例子中，如果寻道时间远大于旋转延迟，那么 SSTF（及其变体）就够用了。然而，设想寻道比旋转快不少。那么，在我们的例子中，寻道更远一些、去处理外侧磁道上的请求 8，会比寻道较短的距离到中间磁道去处理请求 16 更划算，因为扇区 16 要转上几乎一整圈才会经过磁头下方。

正如上面看到的，在现代驱动器上，寻道和旋转大致相当（当然，这要看具体的请求），因此 SPTF 很有用，可以提高性能。然而，在操作系统中实现它就更难了，因为操作系统通常不太清楚磁道的边界在哪里，也不知道磁头当前（在旋转意义上）在哪里。因此，SPTF 通常是在驱动器内部实现的，下面会介绍。

### 其他调度问题

关于基本磁盘操作、调度及相关话题，这个简短的介绍还有许多问题没有讨论。其中一个问题是：在现代系统中，磁盘调度是在哪里进行的？在较老的系统中，所有调度都由操作系统来做；操作系统检查一遍待处理的请求，挑出最好的一个，把它发给磁盘。那个请求完成后，再选出下一个，依此类推。那时候磁盘更简单，生活也更简单。

在现代系统中，磁盘可以同时容纳多个尚未完成的请求，而且自身就有精巧的内部调度程序——它们可以准确地实现 SPTF，因为在磁盘控制器内部，所有相关细节都能拿到，包括磁头的确切位置。因此，操作系统的调度程序通常会挑出它认为最好的几个请求（比如 16 个），把它们全部发给磁盘；磁盘再利用它掌握的磁头位置和详细的磁道布局信息，以尽可能好的（SPTF）顺序处理这些请求。

磁盘调度程序执行的另一项重要相关任务是 **I/O 合并**（I/O merging）。例如，设想有一串请求，依次读取块 33、块 8、块 34，就像图 37.8 中那样。在这种情况下，调度程序应该把对块 33 和块 34 的请求**合并**（merge）成一个读两个块的请求；调度程序所做的任何重新排序，都是在合并后的请求上进行的。合并在操作系统层面尤其重要，因为它减少了发给磁盘的请求数，从而降低了开销。

现代调度程序要解决的最后一个问题是：系统在向磁盘发出 I/O 之前，应该等待多长时间？有人可能会天真地认为，磁盘只要有一个 I/O，就应该立即把请求发给驱动器；这种方法称为 **work-conserving**，因为只要有请求要处理，磁盘就永远不会空闲。然而，关于**预期磁盘调度**（anticipatory disk scheduling）的研究表明，有时候稍等一会儿反而更好 [ID01]，这就是所谓的 **non-work-conserving** 方法。等一等，可能会有一个新的、“更好”的请求到达磁盘，从而提高整体效率。当然，决定什么时候等、等多久，可能很棘手；详见那篇研究论文，或者（如果你是个有抱负的人）去看看 Linux 内核的实现，看看这样的思想是怎样落地到实践中的。

## 37.6 小结

我们概述了磁盘是怎样工作的。这个概述实际上是一个详细的功能模型；它并没有描述实际驱动器设计中那些令人惊叹的物理学、电子学和材料科学。对这方面更多细节感兴趣的人，我们建议换一个专业（或者辅修一个）；对这个模型感到满意的人，很好！现在我们可以利用这个模型，在这些不可思议的设备之上构建更有意思的系统了。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[ADR03] "More Than an Interface: SCSI vs. ATA" by Dave Anderson, Jim Dykes, Erik Riedel. FAST '03, 2003. *One of the best recent-ish references on how modern disk drives really work; a must read for anyone interested in knowing more.*

[CKR72] "Analysis of Scanning Policies for Reducing Disk Seek Times" E.G. Coffman, L.A. Klimko, B. Ryan SIAM Journal of Computing, September 1972, Vol 1. No 3. *Some of the early work in the field of disk scheduling.*

[HK+17] "The Unwritten Contract of Solid State Drives" by Jun He, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. EuroSys '17, Belgrade, Serbia, April 2017. *We take the idea of the unwritten contract, and extend it to SSDs. Using SSDs well seems as complicated as hard drives, and sometimes more so.*

[ID01] "Anticipatory Scheduling: A Disk-scheduling Framework To Overcome Deceptive Idleness In Synchronous I/O" by Sitaram Iyer, Peter Druschel. SOSP '01, October 2001. *A cool paper showing how waiting can improve disk scheduling: better requests may be on their way!*

[JW91] "Disk Scheduling Algorithms Based On Rotational Position" by D. Jacobson, J. Wilkes. Technical Report HPL-CSP-91-7rev1, Hewlett-Packard, February 1991. *A more modern take on disk scheduling. It remains a technical report (and not a published paper) because the authors were scooped by Seltzer et al. [SCO90].*

[RW92] "An Introduction to Disk Drive Modeling" by C. Ruemmler, J. Wilkes. IEEE Computer, 27:3, March 1994. *A terrific introduction to the basics of disk operation. Some pieces are out of date, but most of the basics remain.*

[SCO90] "Disk Scheduling Revisited" by Margo Seltzer, Peter Chen, John Ousterhout. USENIX 1990. *A paper that talks about how rotation matters too in the world of disk scheduling.*

[SG04] "MEMS-based storage devices and standard disk interfaces: A square peg in a round hole?" Steven W. Schlosser, Gregory R. Ganger FAST '04, pp. 87-100, 2004 *While the MEMS aspect of this paper hasn't yet made an impact, the discussion of the contract between file systems and disks is wonderful and a lasting contribution. We later build on this work to study the "Unwritten Contract of Solid State Drives" [HK+17]*

[S09a] "Barracuda ES.2 data sheet" by Seagate, Inc.. Available at this website, at least, it was: http://www.seagate.com/docs/pdf/datasheet/disc/ds_barracuda_es.pdf. *A data sheet; read at your own risk. Risk of what? Boredom.*

[S09b] "Cheetah 15K.5" by Seagate, Inc.. Available at this website, we're pretty sure it is: http://www.seagate.com/docs/pdf/datasheet/disc/ds-cheetah-15k-5-us.pdf. *See above commentary on data sheets.*

## 作业（模拟）

这个作业用 `disk.py` 让你熟悉现代硬盘驱动器是怎样工作的。它有很多不同的选项，而且和其他大多数模拟程序不同，它有一个图形动画，可以让你确切地看到磁盘工作时发生了什么。详情请见 README。

1. 计算下面几组请求的寻道时间、旋转时间和传输时间（transfer time）：`-a 0`、`-a 6`、`-a 30`、`-a 7,30,8`，最后是 `-a 10,11,12,13`。
2. 还是上面这些请求，但把寻道速率改成不同的值：`-S 2`、`-S 4`、`-S 8`、`-S 10`、`-S 40`、`-S 0.1`。时间会怎样变化？
3. 还是上面这些请求，但改变旋转速率：`-R 0.1`、`-R 0.5`、`-R 0.01`。时间会怎样变化？
4. FIFO 并不总是最好的。例如，对于请求流 `-a 7,30,8`，应该按什么顺序处理这些请求？在这个工作负载上运行最短寻道时间优先（SSTF）调度程序（`-p SSTF`）；处理每个请求应该花多长时间，寻道、旋转、传输各要多少？
5. 现在使用 shortest access-time first（SATF）调度程序（`-p SATF`）。对于 `-a 7,30,8` 这个工作负载，它有什么不同吗？找出一组 SATF 比 SSTF 表现更好的请求；更一般地说，什么时候 SATF 比 SSTF 好？
6. 来试试这个请求流：`-a 10,11,12,13`。运行时哪里表现不好？试着加入磁道偏斜（`-o skew`）来解决这个问题。在默认寻道速率下，偏斜应该设成多少才能让性能最好？换成不同的寻道速率（例如 `-S 2`、`-S 4`）呢？一般来说，你能写出一个计算偏斜的公式吗？
7. 指定一个每个区密度不同的磁盘，例如 `-z 10,20,30`，它指定了外侧、中间和内侧磁道上相邻块之间的角度差。运行一些随机请求（例如 `-a -1 -A 5,-1,0`，它通过 `-a -1` 选项指定使用随机请求，并生成 5 个范围从 0 到最大值的请求），计算寻道时间、旋转时间和传输时间（transfer time）。使用不同的随机种子。外侧、中间和内侧磁道的带宽（bandwidth，以每单位时间的扇区数计）分别是多少？
8. 调度窗口决定了磁盘一次能考察多少个请求。生成随机工作负载（使用不同的随机种子，例如 `-A 1000,-1,0`），看看调度窗口从 1 增大到请求数时，SATF 调度程序要花多长时间。窗口要多大才能让性能最好？提示：用 `-c` 选项、不开图形（`-G`），这样跑得更快。调度窗口设为 1 时，用哪种策略还有关系吗？
9. 假设采用 SATF 策略，构造一串请求，让某个特定的请求饥饿。对于这串请求，如果使用 **bounded SATF**（**BSATF**）调度方法，表现如何？在这种方法中，你需要指定调度窗口（例如 `-w 4`）；只有当前窗口里的所有请求都处理完之后，调度程序才会移到下一个请求窗口。这样能解决饥饿问题吗？与 SATF 相比，它的表现如何？一般来说，磁盘应该怎样在性能和避免饥饿之间做权衡？
10. 到目前为止，我们看过的所有调度策略（scheduling policy）都是**贪心**（greedy）的：它们只挑眼下最好的选项，而不去寻找最优的调度。你能找出一组让贪心策略不是最优的请求吗？

 © 2008–23, ARPACI-DUSSEAU

# 38

# 廉价磁盘冗余阵列（RAID）

使用磁盘时，我们有时希望它更快：I/O 操作很慢，可能成为整个系统的瓶颈。使用磁盘时，我们有时希望它更大：需要在线保存的数据越来越多，磁盘也越来越满。使用磁盘时，我们有时还希望它更可靠：磁盘一旦坏掉，如果数据没有备份，所有宝贵的数据就都没了。

关键问题：如何构建大容量、快速、可靠的磁盘
如何构建一个大容量、快速且可靠的存储系统？关键技术有哪些？不同方法之间要做哪些权衡？

本章介绍**廉价磁盘冗余阵列**（Redundant Array of Inexpensive Disks），也就是大家更熟悉的 **RAID** [P+88]。这项技术把多块磁盘组合起来协同工作，构建出更快、更大、更可靠的磁盘系统。这个术语是 20 世纪 80 年代末由加州大学伯克利分校的一组研究人员提出的（领头的是 David Patterson 教授和 Randy Katz 教授，还有当时还是学生的 Garth Gibson）；差不多就在那个时候，许多研究人员不约而同地想到了用多块磁盘构建更好的存储系统这个基本思路 [BG88, K86, K88, PB86, SG86]。

从外部看，RAID 就像一块磁盘：一组可以读写的块（block）。从内部看，RAID 却是个复杂的大家伙：它由多块磁盘、易失性（volatile）和非易失性（non-volatile）内存，以及一个或多个负责管理系统的处理器组成。硬件 RAID 非常像一个计算机系统，只不过专门用来管理一组磁盘。

与单块磁盘相比，RAID 有许多优点。一是性能：并行（in parallel）使用多块磁盘，可以大大缩短 I/O 时间。二是容量：大数据集需要大磁盘。最后，RAID 还能提高可靠性（reliability）：如果不用 RAID 技术，把数据分散到多块磁盘上，只要丢一块磁盘就会丢数据；而借助某种形式的**冗余**（redundancy），RAID 能承受一块磁盘的损失，照常运行，就像什么事都没发生一样。

令人惊讶的是，RAID 是**透明地**为使用它的系统提供这些优点的：在主机系统看来，RAID 就是一块大磁盘。透明的妙处当然在于，你可以直接把一块磁盘换成 RAID，一行软件都不用改；操作系统和客户端应用程序无须修改就能照常运行。这样，透明性大大提高了 RAID 的**可部署性**（deployability），用户和管理员可以直接用上 RAID，不必担心软件兼容性问题。

提示：透明有利于部署

给系统添加新功能时，应该始终想一想能否**透明地**（transparently）添加，也就是不要求系统其他部分做任何改动。如果要求完全重写现有软件（或者对硬件做根本性的改动），这个想法就很难产生影响。RAID 就是绝佳的例子，它的成功无疑有透明性的一份功劳：管理员可以装一个基于 SCSI 的 RAID 存储阵列来代替 SCSI 磁盘，系统的其他部分（主机、操作系统等）不用做一丁点改动就能用上它。由于解决了这个**部署**（deployment）问题，RAID 从第一天起就更容易取得成功。

下面我们讨论 RAID 的几个重要方面。先讲接口和故障模型，然后讨论如何从三个重要维度评估一种 RAID 设计：容量、可靠性和性能。最后，再讨论其他一些对 RAID 的设计和实现很重要的问题。

## 38.1 接口与 RAID 内部结构

在上层的文件系统（file system）看来，RAID 就像一块很大、（但愿）很快、（但愿）很可靠的磁盘。和单块磁盘一样，它表现为一个由块组成的线性数组，文件系统或其他客户端可以读写其中的每一块。

文件系统向 RAID 发出一个*逻辑 I/O* 请求后，RAID 必须在内部算出要访问哪块（或哪几块）磁盘才能完成请求，然后发出一个或多个*物理 I/O* 来完成它。这些物理 I/O 具体是什么样子，取决于 RAID 的级别，后面我们会详细讨论。举个简单的例子，假设某个 RAID 为每个块保存两个副本（分别放在不同的磁盘上）；向这样的**镜像**（mirrored）RAID 系统写数据时，RAID 每收到一个逻辑 I/O，就要执行两个物理 I/O。

RAID 系统通常做成一个独立的硬件盒子，通过标准接口（例如 SCSI 或 SATA）连到主机上。不过，RAID 内部相当复杂：它有一个运行固件（firmware）的微控制器，指挥 RAID 的运作；有 DRAM 之类的易失性内存，在读写数据块（data block）时用来缓冲；有的还带有非易失性内存，用来安全地缓冲写操作；甚至可能有专门的逻辑电路来做奇偶校验（parity）计算（这在某些 RAID 级别中很有用，后面也会看到）。从高层次看，RAID 就是一个高度专门化的计算机系统：它有处理器、内存和磁盘；只不过它运行的不是应用程序，而是专门用来操作 RAID 的软件。

## 38.2 故障模型

要理解 RAID、比较各种方法，心里必须先有一个故障模型。RAID 的设计目标是检测某几类磁盘故障并从中恢复；因此，确切知道会出现哪些故障，对于得出一个可行的设计至关重要。

我们假设的第一个故障模型相当简单，称为 **fail-stop** 故障模型 [S84]。在这个模型中，磁盘只可能处于两种状态之一：正常工作或者已经故障。正常工作的磁盘，所有块都可以读写。相反，磁盘一旦故障，我们就认为它永久丢失了。

 © 2008–23, ARPACI-DUSSEAU

fail-stop 模型的一个关键方面，在于它对故障检测的假设。具体来说，我们假设磁盘出了故障很容易检测出来。例如，在 RAID 阵列中，我们会假设 RAID 控制器的硬件（或软件）能立即发现某块磁盘出了故障。

因此，我们暂时不必担心更复杂的“静默”故障，比如磁盘数据损坏。也不必担心一块整体正常的磁盘上某个块变得无法访问的情况（有时称为 latent sector error）。这些更复杂（而且很遗憾，也更现实）的磁盘故障，我们后面再考虑。

## 38.3 如何评估 RAID

我们很快就会看到，构建 RAID 的方法有很多种。每种方法各有特点，值得逐一评估，以了解它们的优缺点。

具体来说，我们从三个维度评估每种 RAID 设计。第一个维度是**容量**（capacity）：给定 $N$ 块磁盘，每块有 $B$ 个块，RAID 的客户端能用到多少有效容量？如果没有冗余，答案是 $N \cdot B$；相反，如果系统为每个块保存两个副本（称为**镜像**，mirroring），有效容量就是 $(N \cdot B)/2$。其他方案（例如基于奇偶校验的方案）往往介于两者之间。

第二个评估维度是**可靠性**（reliability）：给定的设计能承受几块磁盘故障？按照我们的故障模型，只考虑整块磁盘故障的情况；在后面讲数据完整性（data integrity）的那一章里，我们会考虑如何处理更复杂的故障模式。

最后，第三个维度是**性能**（performance）。性能评估起来有些难度，因为它在很大程度上取决于磁盘阵列面对的工作负载（workload）。因此，在评估性能之前，我们先介绍几种应该考虑的典型工作负载。

接下来我们考虑三种重要的 RAID 设计：RAID 0 级，即条带化（striping）；RAID 1 级，即镜像；RAID 4/5 级，即基于奇偶校验的冗余。把这些设计各自称为一个“级别”（level），源自 Patterson、Gibson 和 Katz 在伯克利的开创性工作 [P+88]。

## 38.4 RAID 0 级：条带化

第一个 RAID 级别其实根本算不上 RAID 级别，因为它没有冗余。不过，RAID 0 级，也就是大家更熟悉的**条带化**（striping），可以作为性能和容量的一个绝佳上限，因此值得了解。

最简单的条带化（striping），就是像下面这样，把块依次**条带化**（stripe）到系统的各块磁盘上（这里假设阵列有 4 块磁盘）：

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 4 | 5 | 6 | 7 |
| 8 | 9 | 10 | 11 |
| 12 | 13 | 14 | 15 |

图 38.1：**RAID-0：简单条带化**

 © 2008–23, ARPACI-DUSSEAU

从图 38.1 可以看出基本思路：把阵列的块轮流（round-robin）分散到各块磁盘上。这种方法的设计目的，是在请求访问阵列中连续的大段数据时（例如大规模的顺序读），尽可能发挥阵列的并行性（parallelism）。我们把同一行的块称为一个**条带**（stripe）；因此，上面的块 0、1、2、3 位于同一个条带（stripe）中。

在这个例子中，我们做了一个简化假设：每块磁盘上只放 1 个块（比如每块 4 KB），然后就转到下一块磁盘。但并非一定要这样安排。例如，我们也可以像图 38.2 那样在各磁盘上排列块：

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 | |
|---|---|---|---|---|
| 0 | 2 | 4 | 6 | 条带深度： |
| 1 | 3 | 5 | 7 | 2 个块 |
| 8 | 10 | 12 | 14 | |
| 9 | 11 | 13 | 15 | |

图 38.2：**条带深度更大的条带化**

在这个例子中，我们在每块磁盘上放两个 4 KB 的块，然后才转到下一块磁盘。因此，这个 RAID 阵列的**条带深度**（chunk size）是 8 KB，一个条带（stripe）就由 4 个 chunk 组成，共 32 KB 数据。

补充：RAID 映射问题

在研究 RAID 的容量、可靠性和性能特点之前，我们先插一段补充，讲讲所谓的**映射问题**（mapping problem）。所有 RAID 阵列都会遇到这个问题；简单地说，给定一个要读或写的逻辑块（logical block），RAID 怎么准确知道该访问哪块物理磁盘的哪个偏移位置?

对于这几种简单的 RAID 级别，要把逻辑块正确地映射到物理位置，用不着多复杂的方法。拿上面第一个条带化（striping）的例子来说，条带深度（chunk size）= 1 个块 = 4 KB。这时，给定逻辑块地址（logical block address）A，RAID 用两个简单的等式就能算出要访问的磁盘和偏移：

```
Disk   = A % number_of_disks
Offset = A / number_of_disks
```

注意，这些都是整数运算（例如 4 / 3 = 1，而不是 1.33333…）。我们用一个简单的例子看看这两个等式怎么用。假设在上面第一个 RAID 中，来了一个访问块 14 的请求。由于有 4 块磁盘，我们要找的磁盘就是（14 % 4 = 2）：磁盘 2。具体的块则是（14 / 4 = 3）：块 3。因此，块 14 应该在第三块磁盘（磁盘 2，从 0 开始编号）的第四个块（块 3，从 0 开始编号）上，它也确实就在那里。

你可以想一想，要支持不同的条带深度（chunk size），这两个等式该怎么改。试试看！不太难。

 © 2008–23, ARPACI-DUSSEAU

### 条带深度

条带深度（chunk size）主要影响阵列的性能。例如，条带深度（chunk size）小，意味着许多文件会条带化（striped）到许多磁盘上，从而提高读写单个文件时的并行性；但是，跨多块磁盘访问块的定位时间会增加，因为整个请求的定位时间，取决于各块磁盘上请求的定位时间中的最大值。

另一方面，条带深度（chunk size）大，会降低这种文件内的并行性，因此要靠多个并发（concurrent）请求才能获得高吞吐量（throughput）。不过，大的条带深度（chunk size）能减少定位时间；例如，如果一个文件能放进一个 chunk，因而只放在一块磁盘上，那么访问它的定位时间就只是一块磁盘的定位时间。

因此，确定“最佳”条带深度（chunk size）很难，因为这需要对磁盘系统面对的工作负载有大量了解 [CL95]。在接下来的讨论中，我们假设阵列的条带深度（chunk size）是一个块（4 KB）。大多数阵列使用更大的条带深度（chunk size，例如 64 KB），但对于下面讨论的问题，具体的条带深度（chunk size）无关紧要；所以为简单起见，我们就用一个块。

### 回到 RAID-0 分析

现在我们来评估条带化（striping）的容量、可靠性和性能。从容量的角度看，它是完美的：给定 $N$ 块磁盘，每块 $B$ 个块，条带化（striping）能提供 $N \cdot B$ 个块的有效容量。从可靠性的角度看，条带化（striping）也“完美”，不过是坏的那种完美：任何一块磁盘故障都会导致数据丢失。最后，性能非常出色：所有磁盘都能用上，而且往往是并行地处理用户的 I/O 请求。

### 评估 RAID 性能

分析 RAID 性能时，可以考虑两种不同的性能指标。第一种是*单请求延迟*（single-request latency）。了解单个 I/O 请求访问 RAID 的延迟很有用，因为它能揭示一次逻辑 I/O 操作内部能有多少并行性。第二种是 RAID 的*稳态吞吐量*（steady-state throughput），也就是许多并发请求的总带宽（bandwidth）。由于 RAID 常用在高性能环境中，稳态带宽至关重要，因此它将是我们分析的重点。

为了更详细地理解吞吐量，我们需要先确定几种要关注的工作负载。在这里的讨论中，我们假设有两种工作负载：**顺序**（sequential）负载和**随机**（random）负载。对于顺序负载，我们假设对阵列的请求是大块连续的；例如，一个（或一串）从块 $x$ 开始、到块（$x + 1$ MB）结束、访问 1 MB 数据的请求，就视为顺序请求。顺序负载在许多环境中都很常见（想想在一个大文件里搜索某个关键词），因此很重要。

对于随机负载，我们假设每个请求都相当小，而且每个请求都访问磁盘上一个不同的随机位置。例如，一串随机请求可能先访问逻辑地址 10 处的 4 KB，然后是逻辑地址 550,000 处，再然后是 20,100 处，依此类推。一些重要的工作负载，比如数据库管理系统（DBMS）上的事务型（transactional）负载，就表现出这种访问模式，因此随机负载同样很重要。

当然，真实的工作负载没有这么简单，往往既有顺序的成分，也有看似随机的成分，还有介于两者之间的行为。为简单起见，我们只考虑这两种情况。

 © 2008–23, ARPACI-DUSSEAU

你也能想到，顺序负载和随机负载会让磁盘表现出截然不同的性能特点。顺序访问时，磁盘工作得最高效，花在寻道（seek）和等待旋转上的时间很少，大部分时间都在传输数据。随机访问（random access）时恰恰相反：大部分时间花在寻道和等待旋转上，花在传输数据上的时间相对很少。为了在分析中体现这种差别，我们假设磁盘在顺序负载下能以 $S$ MB/s 的速率传输数据，在随机负载下则为 $R$ MB/s。一般来说，$S$ 远大于 $R$（即 $S \gg R$）。

为了确保我们理解了这种差别，来做个简单的练习。具体来说，根据下面给出的磁盘特性计算 $S$ 和 $R$。假设顺序传输平均为 10 MB，随机传输平均为 10 KB。另外，假设磁盘特性如下：

| | |
|---|---|
| 平均寻道时间 | 7 ms |
| 平均旋转延迟 | 3 ms |
| 磁盘传输速率 | 50 MB/s |

要计算 $S$，得先弄清楚一次典型的 10 MB 传输把时间花在了哪里。首先，寻道花 7 ms，然后旋转花 3 ms。最后开始传输；以 50 MB/s 的速率传输 10 MB，需要 1/5 秒，也就是 200 ms。因此，每个 10 MB 的请求要花 210 ms 才能完成。要计算 $S$，只需做个除法：

$$S = \frac{\text{数据量}}{\text{访问时间}} = \frac{10 \text{ MB}}{210 \text{ ms}} = 47.62 \text{ MB/s}$$

可以看到，由于传输数据花的时间很长，寻道和旋转的开销被摊薄了，$S$ 非常接近磁盘的峰值带宽。

$R$ 的计算方法类似。寻道和旋转的时间一样；接着计算传输时间（transfer time）：以 50 MB/s 的速率传输 10 KB，需要 0.195 ms。

$$R = \frac{\text{数据量}}{\text{访问时间}} = \frac{10 \text{ KB}}{10.195 \text{ ms}} = 0.981 \text{ MB/s}$$

可以看到，$R$ 不到 1 MB/s，$S/R$ 差不多是 50。

### 再回到 RAID-0 分析

现在来评估条带化（striping）的性能。前面说过，它总体上不错。例如，从延迟的角度看，单块请求的延迟应该和单块磁盘几乎一样；毕竟，RAID-0 只是把这个请求转给它的某一块磁盘而已。

从稳态顺序吞吐量的角度看，我们预计能获得系统的全部带宽。因此，吞吐量等于磁盘数量 $N$ 乘以单块磁盘的顺序带宽 $S$。对于大量随机 I/O，我们同样能用上所有磁盘，从而获得 $N \cdot R$ MB/s。下面我们会看到，这两个值最容易计算，也可以作为和其他 RAID 级别比较时的上限。

## 38.5 RAID 1 级：镜像

条带化（striping）之后的第一个 RAID 级别是 RAID 1 级，也就是镜像。在镜像系统（mirrored system）中，我们只是给系统中的每个块多做几个副本；当然，每个副本应该放在不同的磁盘上。这样，我们就能承受磁盘故障。

在典型的镜像系统（mirrored system）中，我们假设 RAID 为每个逻辑块保存两个物理副本。下面是一个例子：

 © 2008–23, ARPACI-DUSSEAU

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 |
|---|---|---|---|
| 0 | 0 | 1 | 1 |
| 2 | 2 | 3 | 3 |
| 4 | 4 | 5 | 5 |
| 6 | 6 | 7 | 7 |

图 38.3：**简单的 RAID-1：镜像**

在这个例子中，磁盘 0 和磁盘 1 的内容完全相同，磁盘 2 和磁盘 3 也一样；数据在这两组镜像对（mirrored pair）之间做了条带化（striped）。实际上，你可能已经注意到，在各磁盘上放置块副本的方式有好几种。上面这种布局很常见，有时称为 **RAID-10**（或 **RAID 1+0**，**镜像的条带**，stripe of mirrors），因为它先组成镜像对（RAID-1），再在其上做条带化（RAID-0）；另一种常见的布局是 **RAID-01**（或 **RAID 0+1**，**条带的镜像**，mirror of stripes），它包含两个大的条带化（RAID-0）阵列，再在其上做镜像（RAID-1）。眼下我们只讨论上面这种布局的镜像。

从镜像阵列（mirrored array）读一个块时，RAID 有选择余地：两个副本读哪个都行。例如，如果向 RAID 发出读逻辑块 5 的请求，它既可以从磁盘 2 读，也可以从磁盘 3 读。不过，写一个块时就没有这种选择了：为了保证可靠性，RAID 必须把两个副本都更新。但要注意，这两个写操作可以并行进行；例如，写逻辑块 5 时，可以同时写磁盘 2 和磁盘 3。

### RAID-1 分析

我们来评估一下 RAID-1。从容量的角度看，RAID-1 很昂贵：镜像级别（mirroring level）为 2 时，只能得到峰值有效容量的一半。$N$ 块磁盘、每块 $B$ 个块时，RAID-1 的有效容量是 $(N \cdot B)/2$。

从可靠性的角度看，RAID-1 表现不错。它能承受任意一块磁盘的故障。你可能还注意到，运气好的话，RAID-1 其实还能做得更好。想象一下，在上图中，磁盘 0 和磁盘 2 都坏了。这种情况下，数据并没有丢失！更一般地说，镜像级别（mirroring level）为 2 的镜像系统（mirrored system）肯定能承受 1 块磁盘故障，而视坏的是哪几块磁盘，最多能承受 $N/2$ 块磁盘故障。在实践中，我们一般不喜欢把这种事交给运气；所以大多数人认为镜像适合应对单块磁盘故障。

最后，我们来分析性能。从单个读请求的延迟来看，它和单块磁盘上的延迟相同；RAID-1 做的只是把读请求转给其中一个副本。写请求则有点不同：它要等两次物理写都完成才算完成。这两次写是并行的，因此时间大致相当于一次写的时间；但是，由于逻辑写必须等两次物理写都完成，它的耗时取决于两个请求中寻道和旋转延迟（rotational delay）较长的那个，因此（平均而言）会比写单块磁盘稍长一些。

为了分析稳态吞吐量（steady-state throughput），我们先从顺序负载开始。顺序写磁盘时，每次逻辑写都必须变成两次物理写；例如，写逻辑块 0 时（上图），RAID 在内部会把它同时写到磁盘 0 和磁盘 1。因此可以得出结论：顺序写镜像阵列（mirrored array）时能获得的最大带宽是 $\left(\frac{N}{2} \cdot S\right)$，也就是峰值带宽的一半。

 © 2008–23, ARPACI-DUSSEAU

补充：RAID 的 consistent-update problem

在分析 RAID-1 之前，我们先讨论一个任何多磁盘 RAID 系统都会遇到的问题，称为 **consistent-update problem** [DAA05]。向 RAID 写数据时，如果一次逻辑操作要更新多块磁盘，就会出现这个问题。这里假设讨论的是一个镜像磁盘阵列（mirrored disk array）。

假设一个写请求发给了 RAID，RAID 判定必须把它写到两块磁盘上：磁盘 0 和磁盘 1。RAID 先向磁盘 0 发出写请求，但就在 RAID 向磁盘 1 发出请求之前，发生了断电（power loss）或系统崩溃（system crash）。在这种不幸的情况下，我们假设发给磁盘 0 的请求完成了（而发给磁盘 1 的请求显然没有完成，因为它根本没发出去）。

这次不合时宜的断电造成的结果是，这个块的两个副本现在**不一致**（inconsistent）了：磁盘 0 上的副本是新版本，磁盘 1 上的是旧版本。我们希望两块磁盘的状态能**原子地**（atomically）改变，也就是说，要么两者都变成新版本，要么都不变。

解决这个问题的一般方法，是使用某种**预写日志**（write-ahead log），在动手之前先记下 RAID 将要做什么（即用某份数据更新两块磁盘）。采用这种方法，就能保证即使发生崩溃，也会得到正确的结果：运行一个**恢复**（recovery）程序，把所有未完成的事务（transaction）重放（replay）到 RAID 上，就能确保（在 RAID-1 的情况下）没有哪两个镜像副本（mirrored copy）是不同步的。

最后说一点：由于每次写都往磁盘记日志的开销高得令人无法接受，大多数 RAID 硬件都带有少量非易失性 RAM（例如由电池供电），在里面记这种日志。这样，就能保证更新的一致性，又不必付出往磁盘记日志的高昂代价。

遗憾的是，顺序读的性能也完全一样。有人可能会认为顺序读可以做得更好，因为它只需要读一个副本，而不是两个都读。不过，我们用一个例子来说明为什么这帮不上多大忙。假设我们要读块 0、1、2、3、4、5、6、7，并把读块 0 的请求发给磁盘 0，读块 1 的发给磁盘 2，读块 2 的发给磁盘 1，读块 3 的发给磁盘 3。接着，再分别把读块 4、5、6、7 的请求发给磁盘 0、2、1、3。有人可能会天真地以为，既然所有磁盘都用上了，我们就获得了阵列的全部带宽。

然而，事情并非（一定）如此。来看单块磁盘（比如磁盘 0）收到的请求。它先收到读块 0 的请求，然后收到读块 4 的请求（跳过了块 2）。实际上，每块磁盘都是每隔一个块才收到一个请求。磁盘转过被跳过的那个块时，并没有给客户端提供有用的带宽。因此，每块磁盘只能提供其峰值带宽的一半。所以，顺序读也只能获得 $\left(\frac{N}{2} \cdot S\right)$ MB/s 的带宽。

随机读是镜像 RAID（mirrored RAID）的最好情况。这时，我们可以把读请求分散到所有磁盘上，从而获得全部可能的带宽。因此，对于随机读，RAID-1 能提供 $N \cdot R$ MB/s。

最后，随机写的性能和你预想的一样：$\frac{N}{2} \cdot R$ MB/s。每次逻辑写都要变成两次物理写，因此，虽然所有磁盘都在工作，客户端感受到的却只有一半可用带宽。尽管对逻辑块 $x$ 的一次写会变成对两块不同物理磁盘的两次并行（parallel）写，但大量小请求的带宽只能达到条带化（striping）时的一半。我们很快就会看到，能拿到一半的可用带宽其实已经相当不错了！

 © 2008–23, ARPACI-DUSSEAU

## 38.6 RAID 4 级：用奇偶校验节省空间

现在我们介绍另一种给磁盘阵列增加冗余的方法，称为**奇偶校验**（parity）。基于奇偶校验的方法试图少用一些容量，从而克服镜像系统（mirrored system）在空间上付出的巨大代价。不过，这样做也有代价：性能。

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 | 磁盘 4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| 4 | 5 | 6 | 7 | P1 |
| 8 | 9 | 10 | 11 | P2 |
| 12 | 13 | 14 | 15 | P3 |

图 38.4：**带奇偶校验的 RAID-4**

图 38.4 是一个五磁盘 RAID-4 系统的例子。我们为每个数据条带（stripe）增加了一个**奇偶校验块**（parity block，下面简称校验块），用来存放该条带中各块的冗余信息。例如，校验块（parity block）P1 存放的冗余信息，就是根据块 4、5、6、7 算出来的。

要计算奇偶校验，我们需要一个数学函数，让我们能承受条带（stripe）中任意一个块的丢失。简单的**异或**（XOR）函数就能很好地做到这一点。对于给定的一组位，如果其中 1 的个数是偶数，所有这些位的 XOR 结果就是 0；如果 1 的个数是奇数，结果就是 1。例如：

| C0 | C1 | C2 | C3 | P |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | XOR(0,0,1,1) = 0 |
| 0 | 1 | 0 | 0 | XOR(0,1,0,0) = 1 |

在第一行（0,0,1,1）中，有两个 1（C2、C3），因此所有这些值的 XOR 结果是 0（P）；同样，第二行只有一个 1（C1），因此 XOR 结果一定是 1（P）。有个简单的办法可以记住这一点：任何一行中 1 的个数（包括校验位在内）都必须是偶数（而不是奇数）；这就是 RAID 为了保证奇偶校验正确而必须维持的**不变式**（invariant）。

从上面的例子，你也许还能猜到如何利用奇偶校验信息从故障中恢复。假设标为 C2 的那一列丢了。要弄清这一列原来必定是什么值，我们只需读入这一行的其他所有值（包括 XOR 算出的校验位），就能**重建**（reconstruct）出正确的答案。具体来说，假设第一行 C2 列的值丢了（它是 1）；读这一行的其他值（C0 的 0、C1 的 0、C3 的 1，以及校验列 P 的 0），我们得到 0、0、1、0。因为我们知道 XOR 会让每一行中 1 的个数保持为偶数，所以就知道缺失的数据一定是：1。基于 XOR 的奇偶校验方案就是这样重建数据的！还要注意我们计算重建值的方法：只需把数据位和校验位放在一起做 XOR，和当初计算校验位的方法一样。

现在你可能会问：我们一直在说对这些位做 XOR，可从前面我们知道，RAID 在每块磁盘上放的是 4 KB（或更大）的块；怎么对一堆块做 XOR 来计算校验呢？其实这也很简单。只需对数据块的每一位做按位 XOR，再把每次按位 XOR 的结果放进校验块（parity block）中对应的位上。例如，如果块的大小是 4 位（是的，这还是比 4 KB 的块小得多，但你明白意思就行），它们可能是下面这个样子：

 © 2008–23, ARPACI-DUSSEAU

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 | 磁盘 4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| 4 | 5 | 6 | 7 | P1 |
| 8 | 9 | 10 | 11 | P2 |
| 12 | 13 | 14 | 15 | P3 |

图 38.5：**RAID-4 中的全条带写**

| 块 0 | 块 1 | 块 2 | 块 3 | 校验块 |
|---|---|---|---|---|
| 00 | 10 | 11 | 10 | 11 |
| 10 | 01 | 00 | 01 | 10 |

从图中可以看出，校验是对每个块的每一位分别计算的，结果放在校验块（parity block）中。

### RAID-4 分析

现在来分析 RAID-4。从容量的角度看，RAID-4 在它所保护的每一组磁盘中，用 1 块磁盘存放校验信息。因此，一个 RAID 组的有效容量是 $(N-1)\cdot B$。

可靠性也很容易理解：RAID-4 能承受 1 块磁盘故障，多一块都不行。如果丢失的磁盘超过一块，就根本没有办法重建丢失的数据。

最后是性能。这次我们先分析稳态吞吐量（steady-state throughput）。顺序读能利用除校验盘（parity disk）以外的所有磁盘，因此能提供 $(N-1)\cdot S$ MB/s 的峰值有效带宽——这是简单的情况。

要理解顺序写的性能，首先要弄清楚它是怎么完成的。把一大块数据写到磁盘时，RAID-4 可以做一种简单的优化，称为**全条带写**（full-stripe write）。例如，设想块 0、1、2、3 作为一个写请求的一部分发给了 RAID（图 38.5）。

在这种情况下，RAID 可以直接算出 P0 的新值（对块 0、1、2、3 做 XOR），然后把包括校验块（parity block）在内的所有块并行地写到上面的五块磁盘上（图中用灰色标出）。因此，全条带写（full-stripe write）是 RAID-4 写磁盘最高效的方式。

理解了全条带写（full-stripe write），计算 RAID-4 上顺序写的性能就很容易了：有效带宽同样是 $(N-1)\cdot S$ MB/s。尽管在这个过程中校验盘（parity disk）一直在工作，客户端却没有从中得到性能上的好处。

现在来分析随机读的性能。从上图同样可以看出，一组单块随机读会分散到系统的各块数据盘上，但不会落到校验盘（parity disk）上。因此，有效性能是：$(N-1)\cdot R$ MB/s。

我们把随机写留到了最后，它是 RAID-4 最有意思的情况。假设我们想覆盖上例中的块 1。我们可以直接覆盖它，但这会留下一个问题：校验块（parity block）P0 将无法准确反映这个条带（stripe）正确的校验值；在这个例子中，P0 也必须更新。怎样才能既正确又高效地更新它呢？

办法有两种。第一种称为 **additive parity**，做法如下。要算出新校验块（parity block）的值，就并行读入条带（stripe）中其他所有数据块（在这个例子中是块 0、2、3），把它们

 © 2008–23, ARPACI-DUSSEAU

和新块（1）做 XOR，结果就是新的校验块。要完成写操作，接着把新数据和新校验分别写到各自的磁盘上，这两次写同样可以并行进行。

这种技术的问题在于，它的开销随磁盘数量增长，因此在较大的 RAID 中，计算校验需要大量的读操作。于是就有了 **subtractive parity** 方法。

例如，设想下面这串位（4 个数据位，一个校验位）：

| C0 | C1 | C2 | C3 | P |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | XOR(0,0,1,1) = 0 |

假设我们想用一个新值覆盖位 C2，新值记作 $C2_{\text{new}}$。subtractive parity 方法分三步。首先，读入 C2 处的旧数据（$C2_{\text{old}} = 1$）和旧校验（$P_{\text{old}} = 0$）。然后，比较旧数据和新数据；如果两者相同（例如 $C2_{\text{new}} = C2_{\text{old}}$），我们就知道校验位也保持不变（即 $P_{\text{new}} = P_{\text{old}}$）。但如果两者不同，就必须把旧校验位翻转成相反的状态，也就是说，如果（$P_{\text{old}} == 1$），$P_{\text{new}}$ 就设为 0；如果（$P_{\text{old}} == 0$），$P_{\text{new}}$ 就设为 1。这一大堆逻辑可以用 XOR 简洁地表示出来（其中 $\oplus$ 是 XOR 运算符）：

$$P_{\text{new}} = (C_{\text{old}} \oplus C_{\text{new}}) \oplus P_{\text{old}} \tag{38.1}$$

由于我们处理的是块而不是位，所以要对块中的所有位做这个计算（例如每个块 4096 字节，乘以每字节 8 位）。因此，大多数情况下，新块会和旧块不同，新的校验块（parity block）也会和旧的不同。

现在你应该能想明白，什么时候该用 additive parity 计算，什么时候该用 subtractive parity 方法了。想一想，系统中要有多少块磁盘，additive parity 方法执行的 I/O 才会比 subtractive parity 方法少；分界点在哪里？

在这个性能分析中，我们假设使用 subtractive parity 方法。因此，每次写，RAID 都要执行 4 次物理 I/O（两次读，两次写）。现在设想有大量写请求提交给 RAID；RAID-4 能并行执行其中多少个？为了弄清这一点，我们再来看一下 RAID-4 的布局（图 38.6）。

现在设想差不多同时有 2 个小写请求提交给 RAID-4，分别写块 4 和块 13（图中用 * 标出）。这两块数据分别在磁盘 0 和磁盘 1 上，因此对数据的读写可以并行进行，这很好。问题出在校验盘（parity disk）上：两个请求都要读块 4 和块 13 对应的校验块（parity block），即校验块 1 和 3（用 + 标出）。现在应该很清楚了：在这种工作负载下，校验盘（parity disk）成了瓶颈；因此我们有时把它称为基于奇偶校验的 RAID 的**小写入问题**（small-write problem）。所以，即使数据盘可以并行访问，校验盘（parity disk）也让并行无从实现：有了校验盘这个瓶颈，系统的所有写操作都只能串行执行。由于每次逻辑 I/O 都要让校验盘（parity disk）执行两次 I/O（一次读、一次写），只要算出校验盘完成这两次 I/O 的性能，就能得到 RAID-4

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 | 磁盘 4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| *4 | 5 | 6 | 7 | +P1 |
| 8 | 9 | 10 | 11 | P2 |
| 12 | *13 | 14 | 15 | +P3 |

图 38.6：**示例：写块 4、13 以及相应的校验块**

中小规模随机写的性能：$(R/2)$ MB/s。RAID-4 在小规模随机写下的吞吐量非常糟糕；往系统里加磁盘也不会有所改善。

最后我们来分析 RAID-4 的 I/O 延迟。你现在已经知道，单次读（假设没有故障）只是映射到一块磁盘上，因此其延迟等于单个磁盘请求的延迟。单次写需要两次读，再加两次写；两次读可以并行进行，两次写也可以，因此总延迟大约是单块磁盘的两倍（会有些出入：因为要等两次读都完成，所以会碰上最坏情况的定位时间；但之后的更新不会产生寻道开销，因此定位开销可能比平均水平好）。

## 38.7 RAID 5 级：轮转奇偶校验

为了（至少部分地）解决小写入问题（small-write problem），Patterson、Gibson 和 Katz 提出了 RAID-5。RAID-5 的工作方式和 RAID-4 几乎完全一样，只是它把校验块（parity block）**轮转**（rotates）到各块磁盘上（图 38.7）。

可以看到，每个条带（stripe）的校验块（parity block）现在轮流分布在各块磁盘上，从而消除了 RAID-4 中校验盘（parity disk）的瓶颈。

### RAID-5 分析

RAID-5 的大部分分析和 RAID-4 相同。例如，两个级别的有效容量和容错能力完全相同，顺序读写的性能也一样。单个请求（无论读还是写）的延迟也和 RAID-4 相同。

随机读的性能稍好一些，因为现在可以利用所有磁盘了。最后，随机写的性能比 RAID-4 有明显提高，因为它允许请求之间并行。设想写块 1 和写块 10；这会变成两组请求：针对块 1 及其校验块（parity block）的请求落在磁盘 1 和磁盘 4 上，针对块 10 及其校验块的请求落在磁盘 0 和磁盘 2 上。因此，它们可以并行进行。实际上，我们一般可以假设，有大量随机请求时，所有磁盘都能保持大致均衡的忙碌。如果是这样，小规模写的总带宽就是 $\frac{N}{4} \cdot R$ MB/s。之所以损失了四倍，是因为每次 RAID-5 写操作仍然要产生总共 4 次 I/O 操作，这就是使用基于奇偶校验的 RAID 的代价。

由于 RAID-5 和 RAID-4 基本相同，只是在少数情况下更好，它在市场上几乎完全取代了 RAID-4。唯一没被取代的地方，是那些明知自己只会做大规模写、从而完全避开小写入问题（small-write problem）的系统 [HLM94]；在这种情况下，有时会用 RAID-4，因为它构建起来稍微简单一些。

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 | 磁盘 4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| 5 | 6 | 7 | P1 | 4 |
| 10 | 11 | P2 | 8 | 9 |
| 15 | P3 | 12 | 13 | 14 |
| P4 | 16 | 17 | 18 | 19 |

图 38.7：**轮转奇偶校验的 RAID-5**

 © 2008–23, ARPACI-DUSSEAU

| | RAID-0 | RAID-1 | RAID-4 | RAID-5 |
|---|---|---|---|---|
| 容量 | $N \cdot B$ | $(N \cdot B)/2$ | $(N-1) \cdot B$ | $(N-1) \cdot B$ |
| 可靠性 | 0 | 1（肯定能）<br>$\frac{N}{2}$（运气好时） | 1 | 1 |
| 吞吐量 | | | | |
| 顺序读 | $N \cdot S$ | $(N/2) \cdot S$[1] | $(N-1) \cdot S$ | $(N-1) \cdot S$ |
| 顺序写 | $N \cdot S$ | $(N/2) \cdot S$[1] | $(N-1) \cdot S$ | $(N-1) \cdot S$ |
| 随机读 | $N \cdot R$ | $N \cdot R$ | $(N-1) \cdot R$ | $N \cdot R$ |
| 随机写 | $N \cdot R$ | $(N/2) \cdot R$ | $\frac{1}{2} \cdot R$ | $\frac{N}{4}R$ |
| 延迟 | | | | |
| 读 | $T$ | $T$ | $T$ | $T$ |
| 写 | $T$ | $T$ | $2T$ | $2T$ |

图 38.8：**RAID 的容量、可靠性和性能**

## 38.8 RAID 比较：小结

现在，我们在图 38.8 中总结简化后的 RAID 级别比较。注意，为了简化分析，我们省略了许多细节。例如，在镜像系统（mirrored system）中写数据时，平均寻道时间（seek time）比只写一块磁盘时稍长，因为寻道时间是两次寻道中的较大者。因此，对两块磁盘的随机写性能，一般会比对单块磁盘的随机写性能稍差一点。另外，在 RAID-4/5 中更新校验盘（parity disk）时，第一次读旧校验很可能会引起一次完整的寻道和旋转，而第二次写校验则只会引起旋转。最后，与其他方法相比，镜像 RAID（mirrored RAID）的顺序 I/O 要付出 2 倍的性能代价[1]。

不过，图 38.8 确实抓住了本质的差别，对理解各 RAID 级别之间的权衡很有用。在延迟分析中，我们简单地用 $T$ 表示单块磁盘处理一个请求所需的时间。

总之，如果你只要性能、不在乎可靠性，条带化（striping）显然最好。但如果你既要随机 I/O 性能，又要可靠性，镜像最好；付出的代价是容量的损失。如果你的主要目标是容量和可靠性，那么 RAID-5 胜出；付出的代价是小规模写的性能。最后，如果你总是做顺序 I/O，又想把容量最大化，RAID-5 也最合适。

## 38.9 RAID 的其他有趣问题

思考 RAID 时，还有许多其他有趣的想法可以（也许应该）讨论。下面是我们以后可能会写到的一些内容。

例如，还有许多其他 RAID 设计，包括最初分类中的 2 级和 3 级，以及能承受多块磁盘故障的 6 级 [C+04]。还有磁盘故障时 RAID 怎么办；有时它会准备一块**热备盘**（hot spare）待命，用来顶替坏掉的磁盘。故障期间性能如何？重建坏盘期间的性能又如何？还有更现实的故障模型，要考虑 **latent sector error** 或**块损坏**（block corruption）[B+08]，以及处理这类故障的许多技术，详见数据完整性（data integrity）一章。最后，你甚至可以把 RAID

[1]这里一半的性能代价，假设的是一种简单粗暴的镜像读写模式；更巧妙的方法是向每个镜像（mirror）的不同部分发出大的 I/O 请求，这样就有可能获得全部带宽。想一想，看你能否弄明白其中的道理。

 © 2008–23, ARPACI-DUSSEAU

做成一个软件层：这种**软件 RAID**（software RAID）系统更便宜，但也有别的问题，包括 consistent-update problem [DAA05]。

## 38.10 小结

我们讨论了 RAID。RAID 把多块独立的磁盘变成一个更大、容量更多、更可靠的单一整体；重要的是，它是透明地做到这一点的，因此上层的硬件和软件基本察觉不到这种变化。

可供选择的 RAID 级别有很多，具体用哪个级别，在很大程度上取决于最终用户看重什么。例如，镜像 RAID（mirrored RAID）简单、可靠，一般能提供不错的性能，但容量代价很高。相比之下，RAID-5 可靠，从容量的角度看也更好，但工作负载中有小规模写时，性能相当差。为特定的工作负载挑选 RAID，并正确设置条带深度（chunk size）、磁盘数量等参数，很有挑战性，至今仍更像一门艺术，而不是一门科学。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[B+08] “An Analysis of Data Corruption in the Storage Stack” by Lakshmi N. Bairavasundaram, Garth R. Goodson, Bianca Schroeder, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST ’08, San Jose, CA, February 2008. *Our own work analyzing how often disks actually corrupt your data. Not often, but sometimes! And thus something a reliable storage system must consider.*

[BG88] “Disk Shadowing” by D. Bitton and J. Gray. VLDB 1988. *One of the first papers to discuss mirroring, therein called “shadowing”.*

[CL95] “Striping in a RAID level 5 disk array” by Peter M. Chen and Edward K. Lee. SIGMETRICS 1995. *A nice analysis of some of the important parameters in a RAID-5 disk array.*

[C+04] “Row-Diagonal Parity for Double Disk Failure Correction” by P. Corbett, B. English, A. Goel, T. Grcanac, S. Kleiman, J. Leong, S. Sankar. FAST ’04, February 2004. *Though not the first paper on a RAID system with two disks for parity, it is a recent and highly-understandable version of said idea. Read it to learn more.*

[DAA05] “Journal-guided Resynchronization for Software RAID” by Timothy E. Denehy, A. Arpaci-Dusseau, R. Arpaci-Dusseau. FAST 2005. *Our own work on the consistent-update problem. Here we solve it for Software RAID by integrating the journaling machinery of the file system above with the software RAID beneath it.*

[HLM94] “File System Design for an NFS File Server Appliance” by Dave Hitz, James Lau, Michael Malcolm. USENIX Winter 1994, San Francisco, California, 1994. *The sparse paper introducing a landmark product in storage, the write-anywhere file layout or WAFL file system that underlies the NetApp file server.*

[K86] “Synchronized Disk Interleaving” by M.Y. Kim. IEEE Transactions on Computers, Volume C-35: 11, November 1986. *Some of the earliest work on RAID is found here.*

[K88] “Small Disk Arrays – The Emerging Approach to High Performance” by F. Kurzweil. Presentation at Spring COMPCON ’88, March 1, 1988, San Francisco, California. *Another early RAID reference.*

[P+88] “Redundant Arrays of Inexpensive Disks” by D. Patterson, G. Gibson, R. Katz. SIGMOD 1988. *This is considered **the** RAID paper, written by famous authors Patterson, Gibson, and Katz. The paper has since won many test-of-time awards and ushered in the RAID era, including the name RAID itself!*

[PB86] “Providing Fault Tolerance in Parallel Secondary Storage Systems” by A. Park, K. Balasubramaniam. Department of Computer Science, Princeton, CS-TR-O57-86, November 1986. *Another early work on RAID.*

[SG86] “Disk Striping” by K. Salem, H. Garcia-Molina. IEEE International Conference on Data Engineering, 1986. *And yes, another early RAID work. There are a lot of these, which kind of came out of the woodwork when the RAID paper was published in SIGMOD.*

[S84] “Byzantine Generals in Action: Implementing Fail-Stop Processors” by F.B. Schneider. ACM Transactions on Computer Systems, 2(2):145–154, May 1984. *Finally, a paper that is not about RAID! This paper is actually about how systems fail, and how to make something behave in a fail-stop manner.*

## 作业（模拟）

本节介绍 `raid.py`，这是一个简单的 RAID 模拟器，你可以用它来巩固对 RAID 系统工作原理的理解。详情请见 README。

## 问题

1. 用模拟器做一些基本的 RAID 映射测试。用不同的级别（0、1、4、5）运行，看看你能否算出一组请求的映射。对于 RAID-5，看看你能否弄清楚左对称（left-symmetric）布局和左不对称（left-asymmetric）布局的区别。用一些不同的随机种子，生成和上面不同的问题。
2. 和第一题一样，但这次用 `-C` 改变条带深度（chunk size）。条带深度（chunk size）如何改变映射?
3. 和上面一样，但用 `-r` 选项把每个问题反过来。
4. 现在使用反向选项，但用 `-S` 选项增大每个请求的大小。试着指定 8k、12k 和 16k 的大小，同时改变 RAID 级别。请求变大时，底层的 I/O 模式有什么变化？一定也要用顺序负载（`-W sequential`）试试；请求多大时，RAID-4 和 RAID-5 的 I/O 效率会高得多?
5. 使用模拟器的计时模式（`-t`），用 4 块磁盘，在改变 RAID 级别的同时，估算对 RAID 做 100 次随机读的性能。
6. 和上面一样，但增加磁盘的数量。磁盘数量增加时，各 RAID 级别的性能如何扩展?
7. 和上面一样，但全部改成写（`-w 100`），不用读。现在各 RAID 级别的性能如何扩展？你能粗略估算出完成这 100 次随机写的工作负载需要多长时间吗?
8. 最后再运行一次计时模式，但这次用顺序负载（`-W sequential`）。性能如何随 RAID 级别变化，读和写时又有什么不同？改变每个请求的大小时呢？使用 RAID-4 或 RAID-5 时，每次应该向 RAID 写多大的数据?

 © 2008–23, ARPACI-DUSSEAU

# 39

# 插叙：文件和目录

到目前为止，我们已经看到两个关键的操作系统抽象（abstraction）是如何发展起来的：一个是进程（process），它是对 CPU 的虚拟化（virtualization）；另一个是地址空间（address space），它是对内存的虚拟化。这两个抽象配合起来，让程序运行时仿佛身处一个私有的、隔离（isolated）的世界：仿佛有自己的处理器（一个或多个），仿佛有自己的内存。这种假象让系统编程（systems programming）容易得多，因此如今不仅在台式机和服务器上随处可见，在手机之类的各种可编程平台上也越来越普遍。

在这一部分，我们给虚拟化这幅拼图再添上关键的一块：**持久存储**（persistent storage）。持久存储设备（persistent storage device），比如经典的**硬盘驱动器**（hard disk drive）或更现代的**固态存储设备**（solid-state storage device），会永久地（至少是长期地）保存信息。内存的内容在断电（power loss）后就会丢失，持久存储设备（persistent storage device）则能完好地保存数据。因此，操作系统对这类设备必须格外小心：用户真正在乎的数据就放在这里。

关键问题：如何管理持久存储设备
操作系统应该如何管理持久存储设备（persistent storage device）？有哪些 API？实现中有哪些重要的方面？

因此，在接下来的几章里，我们将探讨管理持久数据的关键技术，重点是提高性能和可靠性（reliability）的方法。不过，我们先从 API 讲起，概要介绍一下和 UNIX 文件系统（file system）打交道时会用到的接口。

## 39.1 文件和目录

随着时间的推移，存储虚拟化（storage virtualization）形成了两个关键的抽象。第一个是**文件**（file）。文件就是一个线性的字节数组，其中每个字节都可以读写。每个文件都有某种 **low-level name**，通常是一个数字；用户往往察觉不到这个名称（后面会看到）。由于历史原因，文件的 low-level name 常被称为它的 **inode 号**（inode number，也叫 **i-number**）。后面几章我们会深入学习 inode；现在只要假设每个文件都关联着一个 inode 号就行。

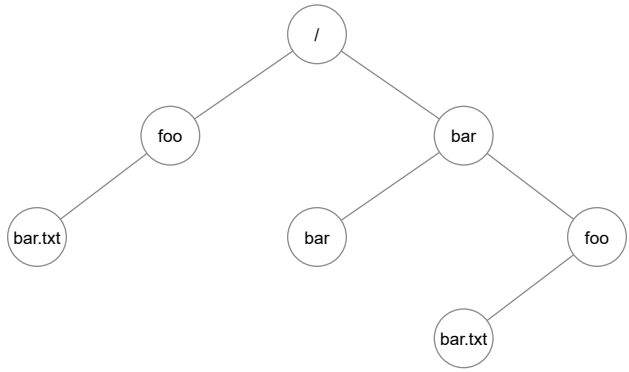



图 39.1：**一棵目录树示例**

在大多数系统中，操作系统并不太了解文件的结构（比如它是一张图片、一个文本文件，还是 C 代码）；文件系统的职责只是把这些数据持久地存放在磁盘上，并确保你再次请求这些数据时，拿到的就是当初放进去的东西。做到这一点可没有看上去那么简单！

第二个抽象是**目录**（directory）。目录和文件一样，也有一个 low-level name，即 inode 号，但它的内容非常特殊：它包含一个由（用户可读的名称，low-level name）对组成的列表。例如，假设有一个 low-level name 为“10”的文件，用户用“foo”这个可读的名称来称呼它。那么“foo”所在的目录中就会有一个条目（“foo”，“10”），把用户可读的名称映射到 low-level name。目录中的每个条目指向的要么是文件，要么是其他目录。把目录放进其他目录，用户就能构建任意的**目录树**（directory tree，也叫**目录层次结构**，directory hierarchy），所有文件和目录都存放在这棵树下。

目录层次结构（directory hierarchy）从**根目录**（root directory）开始（在基于 UNIX 的系统中，根目录就记作 `/`），并用某种**分隔符**（separator）依次命名后面的**子目录**（sub-directories），直到命名到想要的文件或目录为止。例如，如果用户在根目录 `/` 中创建了一个目录 `foo`，然后又在目录 `foo` 中创建了一个文件 `bar.txt`，我们就可以用这个文件的**绝对路径名**（absolute pathname）来指代它，在这个例子里就是 `/foo/bar.txt`。图 39.1 是一棵更复杂的目录树；例子中有效的目录有 `/`、`/foo`、`/bar`、`/bar/bar`、`/bar/foo`，有效的文件有 `/foo/bar.txt` 和 `/bar/foo/bar.txt`。

提示：仔细考虑命名

命名是计算机系统的一个重要方面 [SK09]。在 UNIX 系统中，你能想到的几乎一切都由文件系统来命名。不仅是文件，设备、管道（pipe），甚至进程 [K84]，都能在一个看上去平平无奇的文件系统中找到。这种统一的命名方式简化了你对系统的概念模型，也让系统更简单、更模块化（modular）。所以，无论什么时候创建系统或接口，都要仔细考虑你用的是什么名字。

 © 2008–23, ARPACI-DUSSEAU

只要位于文件系统树的不同位置，目录和文件就可以重名（例如，图中有两个名为 `bar.txt` 的文件：`/foo/bar.txt` 和 `/bar/foo/bar.txt`）。

你可能还注意到，这个例子中的文件名通常由两部分组成：`bar` 和 `txt`，中间用句点隔开。第一部分是任意的名字，而文件名的第二部分通常用来表示文件的**类型**（type），比如它是 C 代码（例如 `.c`）、图片（例如 `.jpg`）还是音乐文件（例如 `.mp3`）。不过，这通常只是一种**约定**（convention）：一般并没有什么机制强制要求名为 `main.c` 的文件里装的一定是 C 源代码。

由此可见，文件系统提供了一项很棒的功能：一种为我们感兴趣的所有文件**命名**（name）的便捷方式。名字在系统中很重要，因为访问任何资源的第一步，就是能够给它命名。因此，在 UNIX 系统中，文件系统提供了一种统一的方式，来访问磁盘、U 盘、CD-ROM 以及许多其他设备上的文件，而且还能访问许多别的东西，它们全都位于同一棵目录树之下。

## 39.2 文件系统接口

现在我们来更详细地讨论文件系统接口。先从创建、访问和删除文件这些基础操作讲起。你可能觉得这很简单，但在这个过程中，我们会遇到一个用来删除文件的神秘调用：`unlink()`。希望读完本章，你就不觉得这个谜有多神秘了！

## 39.3 创建文件

我们从最基本的操作开始：创建文件。这要用 `open` 系统调用（system call）：调用 `open()` 并传入 `O_CREAT` 标志，程序就能创建一个新文件。下面的示例代码在当前工作目录（current working directory）中创建一个名为“foo”的文件：

```
int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC,
              S_IRUSR|S_IWUSR);
```

例程 `open()` 可以接受许多不同的标志。在这个例子中，第二个参数表示：如果文件不存在就创建它（`O_CREAT`）；确保文件只能写入（`O_WRONLY`）；如果文件已经存在，就把

**补充：`creat()` 系统调用**

创建文件的老办法是调用 `creat()`，如下所示：

```
// 可选：再加一个参数来设置权限
int fd = creat("foo");
```

你可以把 `creat()` 看作带有以下标志的 `open()`：

```
O_CREAT | O_WRONLY | O_TRUNC
```

由于 `open()` 就能创建文件，`creat()` 已经不太常用了（其实它完全可以实现为一个调用 `open()` 的库函数）；不过，它在 UNIX 的传说里确实占有一席之地。具体来说，有人问 Ken Thompson，如果重新设计 UNIX，他会在哪些地方换种做法，他回答说：“我会给 creat 加上那个 e。”

它截断为零字节，从而删除原有的全部内容（O_TRUNC）。第三个参数指定权限，这里是让文件的属主（owner）可读可写。

open() 的一个重要方面是它的返回值：**文件描述符**（file descriptor）。文件描述符就是一个整数，每个进程私有，UNIX 系统用它来访问文件；因此，文件一旦打开，只要你有相应的权限，就可以用文件描述符来读写文件。从这个意义上说，文件描述符是一种**能力**（capability）[L84]，也就是一个不透明的句柄，赋予你执行某些操作的权力。理解文件描述符的另一种方式，是把它看作指向一个文件类型对象的指针；有了这样的对象，你就可以调用其他“方法”来访问文件，比如 read() 和 write()（下面会看到怎么做）。

如上所述，文件描述符由操作系统按进程管理。这意味着在 UNIX 系统中，proc 结构里保存着某种简单的结构（例如一个数组）。下面是 xv6 内核 [CK+08] 中的相关片段：

```
struct proc {
  ...
  struct file *ofile[NOFILE]; // 打开的文件
  ...
};
```

一个以文件描述符为下标的简单数组（最多可以有 NOFILE 个打开的文件），按进程记录打开了哪些文件。数组的每个条目其实只是一个指向 struct file 的指针，这个结构用来记录正在读写的文件的信息；下面会进一步讨论。

## 39.4 读写文件

有了文件，我们自然想要读写它们。先从读取一个已有的文件开始。如果是在命令行里操作，我们可能会直接用 cat 程序把文件内容输出到屏幕上。

```
prompt> echo hello > foo
prompt> cat foo
hello
prompt>
```

在这段代码中，我们把 echo 程序的输出重定向（redirect）到文件 foo，于是这个文件里就有了单词“hello”。然后用 cat 查看文件的内容。但 cat 程序是怎么访问文件 foo 的呢?

提示：使用 strace（以及类似的工具）

strace 工具提供了一种很棒的方法，让你看清程序在做什么。运行它，你就能跟踪程序发出了哪些系统调用，看到参数和返回码，总之能把程序在干什么看得很清楚。

这个工具还接受一些很有用的参数。例如，-f 会同时跟踪 fork 出来的所有子进程（child process）；-t 会报告每次调用时的时刻；-e trace=open,close,read,write 只跟踪这几个系统调用，忽略其他所有调用。还有很多其他选项；去读读手册页（manual page），看看怎么用好这个出色的工具。

为了弄清这一点，我们要用一个极其有用的工具，来跟踪程序发出的系统调用。在 Linux 上，这个工具叫 **strace**；其他系统也有类似的工具（Mac 上有 **dtruss**，一些较老的 UNIX 变体上有 **truss**）。strace 的作用是跟踪程序运行期间发出的每一个系统调用，并把跟踪结果输出到屏幕上给你看。

下面是用 strace 弄清 cat 在做什么的一个例子（为了便于阅读，删掉了一些调用）：

```
prompt> strace cat foo
...
open("foo", O_RDONLY|O_LARGEFILE)       = 3
read(3, "hello\n", 4096)                = 6
write(1, "hello\n", 6)                  = 6
hello
read(3, "", 4096)                       = 0
close(3)                                = 0
...
prompt>
```

cat 做的第一件事是打开文件，准备读取。关于这一点，有几处值得注意：第一，文件只是以读的方式（而不是写）打开的，这由 O_RDONLY 标志表示；第二，使用的是 64 位偏移量（offset，O_LARGEFILE）；第三，open() 调用成功了，返回一个文件描述符，值为 3。

为什么第一次调用 open() 返回的是 3，而不是你可能以为的 0 或 1？原来，每个运行中的进程都已经打开了三个文件：标准输入（standard input），进程可以读它来获取输入；标准输出（standard output），进程可以写它来把信息输出到屏幕；标准错误（standard error），进程可以往里写错误信息。它们分别用文件描述符 0、1 和 2 表示。因此，第一次打开另一个文件时（就像上面的 cat 那样），得到的几乎肯定是文件描述符 3。

open 成功之后，cat 用 read() 系统调用反复从文件中读取一些字节。read() 的第一个参数是文件描述符，它告诉文件系统要读哪个文件；进程当然可以同时打开多个文件，所以操作系统要靠描述符才知道某次读操作针对的是哪个文件。第二个参数指向一个缓冲区（buffer），read() 的结果会放在这里；在上面的系统调用跟踪中，strace 在这个位置显示了读到的内容（“hello”）。第三个参数是缓冲区的大小，这里是 4 KB。read() 调用同样成功返回，返回值是读到的字节数（6，其中 5 个是单词“hello”的字母，还有 1 个是行尾标记）。

这时，你会在 strace 的输出中看到另一个有趣的结果：一次对文件描述符 1 的 write() 系统调用。前面提到过，这个描述符就是标准输出，因此这次调用把单词“hello”写到了屏幕上，这正是 cat 程序该做的事。但它是直接调用 write() 的吗？也许是（如果它做过高度优化）。如果不是，cat 可能会调用库例程 printf()；printf() 在内部处理好传给它的所有格式细节，最终写入标准输出，把结果打印到屏幕上。

接着，cat 程序试图从文件中读取更多内容，但文件里已经没有剩余的字节了，于是 read() 返回 0，程序就知道整个文件已经读完了。随后，程序调用 close() 并传入对应的文件描述符，表示它已经用完文件“foo”。文件就此关闭，读取也随之完成。

补充：数据结构——打开文件表
每个进程都维护着一个文件描述符数组，每个文件描述符都指向系统范围的**打开文件表**（open file table）中的一个条目。表中的每个条目记录着描述符对应的是哪个底层文件、当前偏移量（current offset），以及其他相关细节，比如文件是否可读、可写。

写文件也是类似的一系列步骤。首先以写的方式打开文件，然后调用 `write()` 系统调用（对较大的文件可能要调用多次），最后调用 `close()`。你可以用 `strace` 跟踪对文件的写操作，比如跟踪你自己写的程序，或者跟踪 `dd` 工具，例如 `dd if=foo of=bar`。

## 39.5 读写，但不按顺序

到目前为止，我们讨论了如何读写文件，但所有访问都是**顺序的**（sequential）；也就是说，我们要么从头到尾读一个文件，要么从头到尾写出一个文件。

不过，有时我们需要在文件的特定偏移量处读写；例如，如果你为一个文本文档建立了索引，并用它来查找某个特定的单词，最终可能要从文档中的某些**随机**（random）偏移量处读取。为此，我们要用到 `lseek()` 系统调用。下面是它的函数原型：

```
off_t lseek(int fildes, off_t offset, int whence);
```

第一个参数是大家熟悉的文件描述符。第二个参数是 `offset`，它把**文件偏移量**（file offset）定位到文件中的某个位置。第三个参数由于历史原因叫作 `whence`，它决定定位究竟如何进行。下面摘自手册页：

```
如果 whence 是 SEEK_SET，偏移量设为 offset 字节。
如果 whence 是 SEEK_CUR，偏移量设为当前位置加上
  offset 字节。
如果 whence 是 SEEK_END，偏移量设为文件大小加上
  offset 字节。
```

从这段描述可以看出，对于进程打开的每个文件，操作系统都会记录一个“当前”偏移量，它决定下一次读或写从文件的什么位置开始。因此，打开的文件这一抽象本身就带有一个当前偏移量，它有两种更新方式。第一种是读或写了 $N$ 个字节时，当前偏移量加上 $N$；因此每次读写都会*隐式地*更新偏移量。第二种是用 `lseek` *显式地*更新，它按上面所说的方式修改偏移量。

你可能已经猜到，偏移量就保存在前面看到的那个 `struct file` 中，`struct proc` 引用的正是它。下面是 xv6 对这个结构的（简化）定义：

```
struct file {
  int ref;
  char readable;
  char writable;
  struct inode *ip;
  uint off;
};
```

补充：调用 lseek() 不会执行磁盘寻道

lseek() 这个系统调用的名字起得不好，许多学生想弄懂磁盘以及磁盘之上的文件系统是怎样工作的，却被它弄得困惑不已。不要把两者混为一谈！lseek() 调用只是修改操作系统内存中的一个变量，这个变量为某个特定进程记录它的下一次读或写从哪个偏移量开始。如果发给磁盘的读写请求和上一次不在同一条磁道（track）上，就得移动磁头（disk head），这才是磁盘寻道（seek）。更让人困惑的是，先用 lseek() 定位到文件中的随机位置，再在这些位置读写，确实会导致更多的磁盘寻道。因此，调用 lseek() 可能导致随后的读写发生寻道，但它本身绝对不会引起任何磁盘 I/O。

从这个结构可以看出，操作系统可以用它来判断打开的文件是可读还是可写（或者两者皆可）、它对应的是哪个底层文件（由 struct inode 指针 ip 指向），以及当前偏移量。还有一个引用计数（reference count，ref），我们会在下面进一步讨论。

这些 file 结构代表了系统中当前所有打开的文件；它们合起来，有时叫作**打开文件表**（open file table）。xv6 内核只是把它们保存为一个数组，整个表用一把锁（lock）保护：

```
struct {
  struct spinlock lock;
  struct file file[NFILE];
} ftable;
```

我们用几个例子把这一点讲得更清楚些。首先，跟踪一个进程：它打开一个文件（大小为 300 字节），然后反复调用 read() 系统调用读取这个文件，每次读 100 字节。下面是相关系统调用的跟踪结果，同时列出了每个系统调用的返回值，以及这次文件访问在打开文件表中的当前偏移量：

| 系统调用 | 返回值 | 当前偏移量 |
|---|---|---|
| fd = open("file", O_RDONLY); | 3 | 0 |
| read(fd, buffer, 100); | 100 | 100 |
| read(fd, buffer, 100); | 100 | 200 |
| read(fd, buffer, 100); | 100 | 300 |
| read(fd, buffer, 100); | 0 | 300 |
| close(fd); | 0 | – |

从这次跟踪中，有几点值得注意。首先，文件打开时，当前偏移量被初始化为零。其次，进程每调用一次 read()，偏移量都会增加；这样，进程只要不断调用 read()，就能读到文件的下一块内容。最后，试图越过文件末尾的 read() 返回零，以此告诉进程它已经把整个文件读完了。

第二个例子，跟踪一个进程：它把同一个文件打开两次，并对两者各发出一次读操作。

 © 2008–23, ARPACI-DUSSEAU

| 系统调用 | 返回值 | OFT[10] 当前偏移量 | OFT[11] 当前偏移量 |
|---|---|---|---|
| `fd1 = open("file", O_RDONLY);` | 3 | 0 | – |
| `fd2 = open("file", O_RDONLY);` | 4 | 0 | 0 |
| `read(fd1, buffer1, 100);` | 100 | 100 | 0 |
| `read(fd2, buffer2, 100);` | 100 | 100 | 100 |
| `close(fd1);` | 0 | – | 100 |
| `close(fd2);` | 0 | – | – |

在这个例子中，分配了两个文件描述符 3 和 4，它们各自指向打开文件表中不同的条目（在这个例子中是条目 10 和 11，如表头所示；OFT 表示 Open File Table，即打开文件表）。跟踪一下发生了什么，就能看到每个当前偏移量是如何独立更新的。

最后一个例子，进程在读取之前用 `lseek()` 重新定位当前偏移量；这种情况下只需要一个打开文件表条目（和第一个例子一样）。

| 系统调用 | 返回值 | 当前偏移量 |
|---|---|---|
| `fd = open("file", O_RDONLY);` | 3 | 0 |
| `lseek(fd, 200, SEEK_SET);` | 200 | 200 |
| `read(fd, buffer, 50);` | 50 | 250 |
| `close(fd);` | 0 | – |

在这里，`lseek()` 调用先把当前偏移量设为 200。随后的 `read()` 读取接下来的 50 个字节，并相应地更新当前偏移量。

## 39.6 共享文件表条目：`fork()` 和 `dup()`

在许多情况下（如上面的例子），文件描述符到打开文件表条目的映射是一对一的。例如，一个进程运行时，可能决定打开一个文件、读取它，然后关闭它；在这个例子里，这个文件在打开文件表中有一个唯一的条目。即使另一个进程同时读取同一个文件，每个进程也都会在打开文件表中有自己的条目。这样，每一次逻辑上的文件读写都是独立的，访问该文件时各有各的当前偏移量。

不过，也有几种有趣的情况，打开文件表中的条目是*共享*的。其中一种情况发生在父进程（parent process）用 `fork()` 创建子进程时。图 39.2 给出了一小段代码：父进程创建一个子进程，然后等待它完成。子进程调用 `lseek()` 调整当前偏移量，然后退出。最后，父进程等到子进程结束后，检查当前偏移量并把它的值打印出来。

运行这个程序，可以看到以下输出：

```
prompt> ./fork-seek
child: offset 10
parent: offset 10
prompt>
```

图 39.3 展示了三者之间的关系：每个进程私有的描述符数组、共享的打开文件表条目，以及这个条目对底层文件系统 inode 的引用。注意，这里我们终于用上了**引用计数**

```
int main(int argc, char *argv[]) {
    int fd = open("file.txt", O_RDONLY);
    assert(fd >= 0);
    int rc = fork();
    if (rc == 0) {
        rc = lseek(fd, 10, SEEK_SET);
        printf("child: offset %d\n", rc);
    } else if (rc > 0) {
        (void) wait(NULL);
        printf("parent: offset %d\n",
               (int) lseek(fd, 0, SEEK_CUR));
    }
    return 0;
}
```

图 39.2：**父子进程共享的文件表条目（`fork-seek.c`）**

（reference count）。文件表条目被共享时，引用计数会增加；只有两个进程都关闭了文件（或者都退出了），这个条目才会被删除。

在父子进程之间共享打开文件表条目偶尔很有用。例如，如果你创建了一些协同完成某项任务的进程，它们无须任何额外的协调就可以写同一个输出文件。关于调用 `fork()` 时进程之间共享了哪些东西，请查阅手册页。

另一种有趣的、也许更有用的共享情况，出现在 **`dup()`** 系统调用（以及它的近亲 **`dup2()`** 和 **`dup3()`**）中。

`dup()` 调用让进程创建一个新的文件描述符，它和一个已有的描述符指向同一个底层的打开文件。图 39.4 给出了一小段代码，展示 `dup()` 的用法。

`dup()` 调用（尤其是 `dup2()`）在编写 UNIX shell、执行输出重定向（output redirection）之类的操作时很有用；花点时间想想为什么！这时你可能在想：我做 shell 那个项目的时候，他们怎么不告诉我这个？唉，就算是一本了不起的操作系统书，也没法把所有东西都按正确的顺序讲。抱歉！

## 39.7 用 `fsync()` 立即写入

大多数时候，程序调用 `write()` 只是在告诉文件系统：请在将来的某个时候把这些数据写到持久存储上。出于性能考虑，文件系统会把这些写操作在内存中**缓冲**（buffer）一段时间（比如 5 秒或 30 秒）；到了那个稍后的时刻，写操作才会真正发给存储设备（storage device）。从调用方应用程序的角度看，写操作似乎很快就完成了，只有在极少数情况下（例如机器在 `write()` 调用之后、数据写入磁盘之前崩溃了）才会丢失数据。

然而，有些应用程序需要的不只是这种“最终会写入”的保证。例如，在数据库管理系统（DBMS）中，要开发正确的恢复协议（recovery protocol），就需要能够时不时地强制把数据写入磁盘。

为了支持这类应用程序，大多数文件系统都提供了一些额外的控制 API。在 UNIX 世界里，提供给应用程序的这个接口叫作 `fsync(int fd)`。进程对某个文件描述符调用

 © 2008–23, ARPACI-DUSSEAU

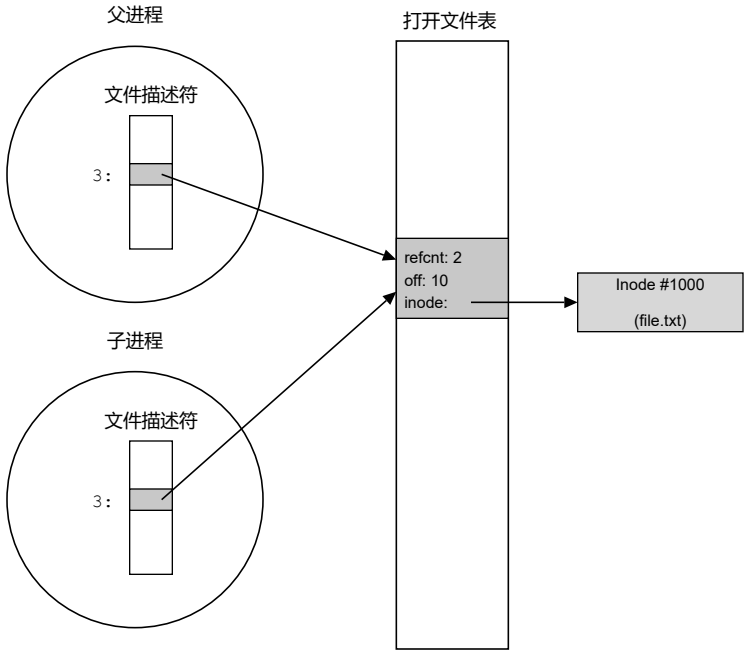



图 39.3：**共享同一个打开文件表条目的两个进程**

`fsync()` 时，文件系统会把该描述符所指文件的全部**脏**（dirty）数据（即还没写入的数据）强制写入磁盘。所有这些写操作完成后，`fsync()` 例程才返回。

下面是一个使用 `fsync()` 的简单例子。这段代码打开文件 `foo`，往里写入一块数据，然后调用 `fsync()`，确保这些写操作立即落到磁盘上。`fsync()` 一返回，应用程序就可以放心地继续执行，因为它知道数据已经持久化了（当然，前提是 `fsync()` 的实现是正确的）。

```
int main(int argc, char *argv[]) {
    int fd = open("README", O_RDONLY);
    assert(fd >= 0);
    int fd2 = dup(fd);
    // 现在 fd 和 fd2 可以互换使用
    return 0;
}
```

图 39.4：**使用 `dup()` 共享文件表条目（`dup.c`）**

```
int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC,
              S_IRUSR|S_IWUSR);
assert(fd > -1);
int rc = write(fd, buffer, size);
assert(rc == size);
rc = fsync(fd);
assert(rc == 0);
```

有意思的是，这个操作序列并不能保证你可能期望的一切；在某些情况下，你还需要对包含文件 foo 的目录调用 fsync()。加上这一步，不仅能确保文件本身已经写到磁盘上，还能确保这个文件（如果是新创建的）也已经持久地成为目录的一部分。毫不奇怪，这类细节常常被忽视，导致了许多应用层面的 bug [P+13, P+14]。

## 39.8 重命名文件

有了文件之后，有时需要给它换个名字。在命令行中，这可以用 mv 命令来完成；在这个例子中，文件 foo 被重命名为 bar：

```
prompt> mv foo bar
```

用 strace 可以看到，mv 使用的是系统调用 rename(char *old, char *new)，它正好接受两个参数：文件原来的名字（old）和新名字（new）。

rename() 调用提供了一个有意思的保证：它（通常）实现为一个**原子**（atomic）调用，即使遇到系统崩溃（system crash）也是如此；如果系统在重命名过程中崩溃，文件要么还叫旧名字，要么已经叫新名字，不会出现奇怪的中间状态。因此，对于某些需要原子地（atomically）更新文件状态的应用程序来说，rename() 至关重要。

我们说得再具体一点。假设你在用一个文件编辑器（例如 emacs），并在一个文件的中间插入了一行。在这个例子里，文件名是 foo.txt。编辑器为了保证新文件既有原来的内容、又有插入的那一行，可能会这样更新文件（为简单起见，省略了错误检查）：

```
int fd = open("foo.txt.tmp", O_WRONLY|O_CREAT|O_TRUNC,
              S_IRUSR|S_IWUSR);
write(fd, buffer, size); // 写出文件的新版本
fsync(fd);
close(fd);
rename("foo.txt.tmp", "foo.txt");
```

在这个例子中，编辑器做的事情很简单：用一个临时名字（foo.txt.tmp）写出文件的新版本，用 fsync() 把它强制写入磁盘，然后，等应用程序确定新文件的元数据（metadata）和内容都已经在磁盘上了，再把临时文件重命名为原来的文件名。最后这一步原子地把新文件换到原来的位置，同时删除文件的旧版本，从而实现了原子的文件更新。

## 39.9 获取文件信息

除了访问文件，我们还希望文件系统为它存储的每个文件保存相当多的信息。我们通常把这类关于文件的数据称为**元数据**（metadata）。要查看某个文件的元数据，可以使用

```
struct stat {
  dev_t     st_dev;     // 文件所在设备的 ID
  ino_t     st_ino;     // inode 号
  mode_t    st_mode;    // 保护（权限）
  nlink_t   st_nlink;   // 硬链接数
  uid_t     st_uid;     // 属主的用户 ID
  gid_t     st_gid;     // 属主的组 ID
  dev_t     st_rdev;    // 设备 ID（如果是特殊文件）
  off_t     st_size;    // 总大小，以字节计
  blksize_t st_blksize; // 文件系统 I/O 的块大小
  blkcnt_t  st_blocks;  // 已分配的块数
  time_t    st_atime;   // 最后访问时间
  time_t    st_mtime;   // 最后修改时间
  time_t    st_ctime;   // 最后状态变更时间
};
```

图 39.5：**stat 结构**

`stat()` 或 `fstat()` 系统调用。这两个调用接受一个文件的路径名（pathname）或文件描述符，并填充一个 `stat` 结构，如图 39.5 所示。

可以看到，每个文件都保存着大量信息，包括它的大小（以字节为单位）、low-level name（即 inode 号）、一些所有权信息，以及文件何时被访问或修改的一些信息，等等。要查看这些信息，可以使用命令行工具 `stat`。在下面的例子中，我们先创建一个文件（名叫 `file`），然后用命令行工具 `stat` 了解这个文件的一些情况。

下面是在 Linux 上的输出：

```
prompt> echo hello > file
prompt> stat file
  File: 'file'
  Size: 6       Blocks: 8      IO Block: 4096 regular file
Device: 811h/2065d Inode: 67158084  Links: 1
Access: (0640/-rw-r-----) Uid: (30686/remzi)
  Gid: (30686/remzi)
Access: 2011-05-03 15:50:20.157594748 -0500
Modify: 2011-05-03 15:50:20.157594748 -0500
Change: 2011-05-03 15:50:20.157594748 -0500
```

每个文件系统通常把这类信息保存在一个叫作 **inode**（索引节点）的结构中[1]。讲到文件系统的实现时，我们会深入学习 inode。现在，你只需把 inode 看作文件系统保存的一种持久数据结构（persistent data structure），里面存放着上面看到的那类信息。所有 inode 都驻留在磁盘上；活跃 inode 的副本通常会缓存（cache）在内存中，以加快访问速度。

[1]有些文件系统给这些结构起的名字相近但略有不同，比如 dnode；不过基本思想是类似的。

补充：mmap() 与持久内存

（客座补充，作者 Terence Kelly）

**内存映射**（memory mapping）是访问文件中持久数据的另一种方式。**mmap()** 系统调用在文件中的字节偏移量与调用进程中的虚拟地址（virtual address）之间建立对应关系；前者称为**后备文件**（backing file），后者称为它的**内存映像**（in-memory image）。之后，进程就可以对内存映像（in-memory image）执行 CPU 指令（即加载和存储），以此访问后备文件（backing file）。

把文件的持久性（persistence）和内存的访问语义结合起来，基于文件的内存映射就支持了一种叫作**持久内存**（persistent memory）的软件抽象。持久内存（persistent memory）这种编程风格不必在内存和存储的不同数据格式之间来回转换，因而能让应用程序更精简 [K19]。

```
1  p = mmap(NULL, file_size, PROT_READ|PROT_WRITE,
2           MAP_SHARED, fd, 0);
3  assert(p != MAP_FAILED);
4  for (int i = 1; i < argc; i++)
5    if (strcmp(argv[i], "pop") == 0) // 出栈
6      if (p->n > 0) // 栈不空
7        printf("%d\n", p->stack[--p->n]);
8    } else { // 入栈
9      if (sizeof(pstack_t) + (1 + p->n) * sizeof(int)
10         <= file_size) // 栈未满
11       p->stack[p->n++] = atoi(argv[i]);
12   }
```

程序 pstack.c（收录在 OSTEP 代码的 GitHub 仓库中，上面是其中的一个片段）在文件 ps.img 中保存了一个持久的栈（stack）；这个文件起初只是一堆零，比如可以在命令行中用 truncate 或 dd 工具创建。文件里包含栈的大小计数，以及保存栈内容的整数数组。用 mmap() 映射后备文件（backing file）之后，我们就可以通过指向内存映像（in-memory image）的 C 指针来访问这个栈，例如用 p->n 访问栈中元素的个数，用 p->stack 访问整数数组。由于栈是持久的，一次运行 pstack 时 push 进去的数据，下一次运行时可以 pop 出来。

如果发生崩溃，比如在 push 的自增和赋值之间崩溃，持久栈就可能处于不一致的状态。应用程序可以借助一些机制来防止这类损坏：这些机制更新持久内存（persistent memory）时，即使中途发生故障也能保证原子性（atomicity）[K20]。

## 39.10 删除文件

到这里，我们已经知道如何创建文件，以及如何顺序或不按顺序地访问文件。但怎么删除文件呢？如果你用过 UNIX，可能会觉得自己知道：运行 rm 程序就行。但 rm 是用哪个系统调用来删除文件的呢？

我们再请出老朋友 strace 来找答案。这里我们删除那个烦人的文件 foo：

```
prompt> strace rm foo
...
unlink("foo")                           = 0
...
```

我们从跟踪输出中删掉了一大堆无关的内容，只留下一个对 `unlink()` 的调用，这个系统调用的名字很神秘。可以看到，`unlink()` 只接受要删除的文件的名字，成功时返回零。但这就引出了一个大谜团：这个系统调用为什么叫 `unlink`？为什么不直接叫 `remove` 或 `delete`？

要解开这个谜，我们不仅要了解文件，还得了解目录。

## 39.11 创建目录

除了文件，还有一组与目录相关的系统调用，让你可以创建、读取和删除目录。注意，你永远不能直接写目录：目录的格式被视为文件系统的元数据，文件系统认为自己要对目录数据的完整性（integrity）负责；因此，你只能间接地更新目录，比如在目录中创建文件、目录或其他类型的对象。文件系统靠这种办法确保目录的内容符合预期。

要创建目录，可以使用系统调用 `mkdir()`。同名的 `mkdir` 程序可以用来创建这样的目录。我们来看看运行 `mkdir` 程序创建一个名为 `foo` 的简单目录时会发生什么：

```
prompt> strace mkdir foo
...
mkdir("foo", 0777)                      = 0
...
prompt>
```

这样创建的目录虽然有最基本的一点内容，仍算是“空”的。具体来说，空目录（empty directory）有两个条目：一个条目指向它自己，另一个条目指向它的父目录。前者叫作“.”（dot）目录，后者叫作“..”（dot-dot）目录。给 `ls` 程序传一个选项（`-a`），就能看到这两个目录：

```
prompt> ls -a
./  ../
prompt> ls -al
total 8
drwxr-x---  2 remzi remzi    6 Apr 30 16:17 ./
drwxr-x--- 26 remzi remzi 4096 Apr 30 16:17 ../
```

## 39.12 读取目录

既然已经创建了目录，我们可能也想读取目录。其实，`ls` 程序做的正是这件事。我们来写一个类似 `ls` 的小工具，看看它是怎么做到的。

我们不是像打开文件那样直接打开目录，而是使用一组新的调用。下面的示例程序打印一个目录的内容。这个程序用了三个调用：`opendir()`、`readdir()` 和 `closedir()`，你

提示：小心威力强大的命令

rm 程序是一个很好的例子，说明了什么是威力强大的命令，以及威力太大有时也是坏事。例如，要一次删除一批文件，你可以输入类似这样的命令：

```
prompt> rm *
```

其中 * 会匹配当前目录（current directory）中的所有文件。但有时候你还想把目录也删掉，而且是连同目录里的全部内容一起删掉。办法是让 rm 递归地进入每个目录，把里面的内容也删掉：

```
prompt> rm -rf *
```

如果你不小心在文件系统的根目录下执行了这一小串字符，麻烦就来了：它会把文件系统中的每个文件和目录都删掉。哎呀！

所以，要记住威力强大的命令是一把双刃剑：它们让你只敲几下键盘就能完成大量工作，但也能迅速而轻易地造成巨大的破坏。

可以看到这个接口有多简单；我们只用一个简单的循环，每次读取一个目录条目（directory entry），打印出目录中每个文件的名字和 inode 号。

```
int main(int argc, char *argv[]) {
    DIR *dp = opendir(".");
    assert(dp != NULL);
    struct dirent *d;
    while ((d = readdir(dp)) != NULL) {
        printf("%lu %s\n", (unsigned long) d->d_ino,
               d->d_name);
    }
    closedir(dp);
    return 0;
}
```

从下面的声明可以看到，struct dirent 中的每个目录条目（directory entry）都包含哪些信息：

```
struct dirent {
  char           d_name[256]; // 文件名
  ino_t          d_ino;       // inode 号
  off_t          d_off;       // 到下一个 dirent 的偏移量
  unsigned short d_reclen;    // 本记录的长度
  unsigned char  d_type;      // 文件类型
};
```

由于目录里的信息很少，基本上只是把名字映射到 inode 号，外加少量其他细节，程序可能还想对每个文件调用 stat()，获取它的长度等更多信息。给 ls 传入 -l 选项时，它正是这么做的；你可以分别带上和不带这个选项，用 strace 跟踪 ls，亲眼看看。

## 39.13 删除目录

最后，可以调用 `rmdir()` 来删除目录，同名的 `rmdir` 程序用的就是它。不过，删除目录比删除文件更危险，因为一条命令就可能删掉大量数据。因此，`rmdir()` 要求目录在删除之前必须是空的（即只有“.”和“..”两个条目）。如果你试图删除一个非空目录（non-empty directory），`rmdir()` 调用就会直接失败。

## 39.14 硬链接

现在我们回到那个谜：为什么删除文件用的是 `unlink()`。为此，我们要了解在文件系统树中创建条目的一种新方法，也就是一个叫作 `link()` 的系统调用。`link()` 系统调用接受两个参数：一个旧路径名和一个新路径名；把一个新文件名“链接”到一个旧文件名上，实际上就是创建了引用同一个文件的另一种方式。命令行程序 `ln` 就是用来做这件事的，如下例所示：

```
prompt> echo hello > file
prompt> cat file
hello
prompt> ln file file2
prompt> cat file2
hello
```

这里我们创建了一个文件，里面有单词“hello”，并把这个文件命名为 `file`[2]。然后，我们用 `ln` 程序为这个文件创建了一个硬链接（hard link）。之后，打开 `file` 或 `file2` 都能查看这个文件。

`link()` 的工作方式是：它只是在你创建链接的那个目录中再创建一个名字，并让它指向原文件的同一个 inode 号。文件本身根本没有复制；你现在只是有了两个人类可读的名字（`file` 和 `file2`），它们都指向同一个文件。我们甚至可以在目录本身中看到这一点，只要打印出每个文件的 inode 号：

```
prompt> ls -i file file2
67158084 file
67158084 file2
prompt>
```

给 `ls` 传入 `-i` 选项，它就会打印出每个文件的 inode 号以及文件名。由此可以看出 `link` 到底做了什么：只是为同一个 inode 号（这个例子中是 67158084）新建了一个引用。

现在你也许开始明白 `unlink()` 为什么叫 `unlink()` 了。创建一个文件时，你其实在做两件事。第一，你在创建一个结构（inode），它将记录这个文件几乎所有的相关信息，包括它的大小、它的块（block）在磁盘上的位置等等。第二，你在把一个人类可读的名字链接到这个文件上，并把这个链接放进一个目录中。

[2]再次注意本书作者有多么富有创意。我们以前还养过一只叫“Cat”的猫（真事）。不过，她死了，现在我们养了一只叫“Hammy”的仓鼠。更新：Hammy 现在也死了。宠物的尸体越堆越多了。

为文件创建硬链接之后，在文件系统看来，原来的文件名（file）和新建的文件名（file2）没有任何区别：两者都只是指向文件底层元数据的链接，这些元数据保存在编号为 67158084 的 inode 中。

因此，要从文件系统中删除一个文件，我们调用 unlink()。比如在上面的例子中，我们可以删除名为 file 的文件，之后仍然可以毫无困难地访问这个文件：

```
prompt> rm file
removed 'file'
prompt> cat file2
hello
```

之所以能这样，是因为文件系统对 file 执行 unlink 时，会检查 inode 中的一个**引用计数**（reference count）。这个引用计数（有时也叫**链接计数**，link count）让文件系统可以记录有多少个不同的文件名链接到了这个 inode 上。调用 unlink() 时，它会删除人类可读的名字（正在被删除的文件）与给定 inode 号之间的“链接”，并把引用计数减一；只有引用计数降到零，文件系统才会同时释放 inode 及相关的数据块（data block），从而真正“删除”这个文件。

当然，你可以用 stat() 查看文件的引用计数。我们来看看创建和删除文件的硬链接时，它是怎么变化的。在这个例子中，我们为同一个文件创建三个链接，然后再把它们删掉。注意看链接计数！

```
prompt> echo hello > file
prompt> stat file
... Inode: 67158084     Links: 1 ...
prompt> ln file file2
prompt> stat file
... Inode: 67158084     Links: 2 ...
prompt> stat file2
... Inode: 67158084     Links: 2 ...
prompt> ln file2 file3
prompt> stat file
... Inode: 67158084     Links: 3 ...
prompt> rm file
prompt> stat file2
... Inode: 67158084     Links: 2 ...
prompt> rm file2
prompt> stat file3
... Inode: 67158084     Links: 1 ...
prompt> rm file3
```

## 39.15 符号链接

还有一种链接非常有用，叫作**符号链接**（symbolic link），有时也叫**软链接**（soft link）。硬链接有一些局限：不能为目录创建硬链接（以免在目录树中造成环）；不能硬链接到其他磁盘分区中的文件（因为 inode 号只在某个文件系统内唯一，跨文件系统就不唯一了）；等等。因此，人们发明了一种叫作符号链接的新型链接 [MJLF84]。

要创建这种链接，可以使用同一个程序 `ln`，但要加上 `-s` 选项。下面是一个例子：

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
```

可以看到，创建软链接看上去差不多，现在既可以通过文件名 `file`，也可以通过符号链接名 `file2` 访问原来的文件。

不过，除了表面上的相似之处，符号链接其实和硬链接很不一样。第一个区别是，符号链接本身其实是一个文件，只是类型不同。我们已经讨论过普通文件和目录；符号链接是文件系统所知道的第三种类型。对符号链接执行 `stat` 就能看出一切：

```
prompt> stat file
... regular file ...
prompt> stat file2
... symbolic link ...
```

运行 `ls` 也能看出这一点。如果仔细看 `ls` 的长格式输出，会发现最左边一列的第一个字符：普通文件是 `-`，目录是 `d`，软链接是 `l`。还可以看到符号链接的大小（这里是 4 字节），以及链接指向的东西（名为 `file` 的文件）。

```
prompt> ls -al
drwxr-x---  2 remzi remzi   29 May  3 19:10 ./
drwxr-x--- 27 remzi remzi 4096 May  3 15:14 ../
-rw-r-----  1 remzi remzi    6 May  3 19:10 file
lrwxrwxrwx  1 remzi remzi    4 May  3 19:10 file2 -> file
```

`file2` 之所以是 4 字节，是因为符号链接的做法，就是把被链接文件的路径名保存为链接文件的数据。由于我们链接到的是一个名为 `file` 的文件，链接文件 `file2` 就很小（4 字节）。如果链接到一个更长的路径名，链接文件就会更大：

```
prompt> echo hello > alongerfilename
prompt> ln -s alongerfilename file3
prompt> ls -al alongerfilename file3
-rw-r----- 1 remzi remzi  6 May  3 19:17 alongerfilename
lrwxrwxrwx 1 remzi remzi 15 May  3 19:17 file3 ->
alongerfilename
```

最后，由于符号链接的创建方式，它们可能会产生所谓的**悬空引用**（dangling reference）：

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
prompt> rm file
prompt> cat file2
cat: file2: No such file or directory
```

 © 2008–23, ARPACI-DUSSEAU

从这个例子可以看出，和硬链接很不一样，删除名为 `file` 的原文件后，链接指向的就是一个不再存在的路径名了。

## 39.16 权限位和访问控制列表

进程这一抽象提供了两种核心的虚拟化：CPU 的虚拟化和内存的虚拟化。它们分别给进程营造出一种假象：进程拥有自己*私有的* CPU 和自己*私有的*内存；而实际上，底层的操作系统用各种技术，在相互竞争的实体之间安全可靠地共享有限的物理资源。

正如本章所描述的，文件系统也呈现了磁盘的一种虚拟视图，把它从一堆原始的块变成了用起来友好得多的文件和目录。不过，这个抽象和 CPU、内存的抽象有一个显著的不同：文件通常在不同的用户和进程之间*共享*，而不（总）是私有的。因此，文件系统中通常有一套更全面的机制，用来支持不同程度的共享。

这类机制的第一种形式，是经典的 UNIX **权限位**（permission bits）。要查看文件 `foo.txt` 的权限，只需输入：

```
prompt> ls -l foo.txt
-rw-r--r--  1 remzi  wheel  0 Aug 24 16:29 foo.txt
```

我们只关注这段输出的第一部分，也就是 `-rw-r--r--`。这里的第一个字符只表示文件的类型：`-` 表示普通文件（`foo.txt` 就是），`d` 表示目录，`l` 表示符号链接，等等；这（大体上）和权限无关，所以我们暂时忽略它。

我们感兴趣的是权限位，由接下来的九个字符表示（`rw-r--r--`）。这些位决定了每个普通文件、目录和其他实体究竟谁能访问、能怎样访问。

权限分为三组：文件的**属主**（owner）可以对它做什么，某个**组**（group）中的人可以对它做什么，最后是其他任何人（有时称为**其他用户**，other）可以做什么。属主、组成员或其他用户可以拥有的能力包括读文件、写文件和执行文件。

在上面的例子中，`ls` 输出的前三个字符表明，这个文件属主可读可写（`rw-`），`wheel` 组的成员以及系统中的其他任何人则只能读（`r--` 后面再跟 `r--`）。

文件的属主可以很方便地修改这些权限，比如使用 `chmod` 命令（用来修改**文件模式**，file mode）。要去掉除属主以外所有人访问这个文件的能力，可以输入：

```
prompt> chmod 600 foo.txt
```

这条命令为属主打开了可读位（4）和可写位（2）（两者按位或得到上面的 6），而把组和其他用户的权限位分别设为 0 和 0，从而把权限设为 `rw-------`。

执行位尤其有意思。对于普通文件，它决定了程序能不能运行。例如，假设有一个名为 `hello.csh` 的简单 shell 脚本，我们可能想输入以下命令来运行它：

```
prompt> ./hello.csh
hello, from shell world.
```

但是，如果没有为这个文件正确设置执行位，就会发生下面的事：

补充：文件系统的超级用户

哪个用户可以执行特权操作（privileged operation）来帮助管理文件系统？例如，如果需要删除某个不活跃用户的文件来节省空间，谁有权这么做？

在本地文件系统上，常见的默认做法是设立某种**超级用户**（superuser，即 **root**），它不论权限如何都可以访问所有文件。在 AFS 这样带有访问控制列表（access control list）的分布式文件系统（distributed file system）中，名为 `system:administrators` 的组里是受信任、可以做这些事的用户。无论哪种情况，这些受信任的用户本身就是一种安全风险；如果攻击者能够设法冒充这样的用户，就能访问系统中的所有信息，从而破坏预期的隐私和保护保证。

```
prompt> chmod 600 hello.csh
prompt> ./hello.csh
./hello.csh: Permission denied.
```

对于目录，执行位的作用略有不同。具体来说，它让用户（或组，或所有人）能够进入这个目录（即用 `cd` 切换过去），并且在同时具有可写位时，能够在其中创建文件。要更多地了解这一点，最好的办法是：自己动手试试！别担心，你（多半）不会把什么东西搞得太糟。

除了权限位，一些文件系统，比如名为 AFS 的分布式文件系统（后面的章节会讨论），还提供了更精细的控制。例如，AFS 为每个目录设一个**访问控制列表**（access control list，**ACL**）来实现这一点。访问控制列表是一种更通用、更强大的方式，能够准确地表示谁可以访问某个资源。在文件系统中，它让用户能够列出一份非常具体的名单，指明谁能读、谁不能读一组文件；相比之下，上面描述的权限位那种“属主/组/所有人”模型就比较受限了。

例如，下面是一位作者的 AFS 账户中一个私有目录的访问控制（access control），由 `fs listacl` 命令显示：

```
prompt> fs listacl private
Access list for private is
Normal rights:
  system:administrators rlidwka
  remzi rlidwka
```

这份列表表明，系统管理员和用户 `remzi` 都可以在这个目录中查找（lookup）、插入、删除和管理文件，还可以读、写和锁定这些文件。

要允许某个人（这里是另一位作者）访问这个目录，用户 `remzi` 只需输入下面的命令：

```
prompt> fs setacl private/ andrea rl
```

`remzi` 的隐私就这样没了！不过你现在学到了一个更重要的道理：好的婚姻里不能有秘密，即使在文件系统里也一样[3]。

[3]如果你好奇的话：我们从 1996 年起就幸福地结了婚。我们知道，你并不好奇。

## 39.17 创建和挂载文件系统

我们已经浏览了访问文件、目录以及某些特殊类型链接的基本接口。但还有一个话题需要讨论：如何用许多底层文件系统组装出一棵完整的目录树。做法是：先创建文件系统，然后挂载（mount）它们，让它们的内容可以访问。

要创建文件系统，大多数文件系统都提供一个工具，通常叫作 `mkfs`（读作“make fs”），专门完成这项任务。思路是这样的：把一个设备（比如磁盘分区，例如 `/dev/sda1`）和一种文件系统类型（例如 ext3）作为输入交给这个工具，它就会在这个磁盘分区上写入一个空的文件系统，从根目录开始。于是 mkfs 说，要有文件系统！

然而，文件系统创建出来以后，还得让它能在统一的文件系统树中访问到。这项任务由 `mount` 程序完成（实际工作由底层的系统调用 `mount()` 来做）。`mount` 做的事情很简单：把一个已有的目录作为目标**挂载点**（mount point），实际上就是把一个新的文件系统贴到目录树的这个位置上。

举个例子可能更好理解。假设我们有一个还没挂载的 ext3 文件系统，存放在设备分区 `/dev/sda1` 中，它的内容如下：一个根目录，其中包含两个子目录 `a` 和 `b`，每个子目录中又各有一个名为 `foo` 的文件。假设我们想把这个文件系统挂载到挂载点（mount point）`/home/users`。我们会输入类似这样的命令：

```
prompt> mount -t ext3 /dev/sda1 /home/users
```

如果成功，`mount` 就会让这个新的文件系统变得可用。不过，要注意现在访问这个新文件系统的方式。要查看它根目录的内容，我们要这样使用 `ls`：

```
prompt> ls /home/users/
a b
```

可以看到，路径名 `/home/users/` 现在指向的是新挂载的这个目录的根。同样，我们可以用路径名 `/home/users/a` 和 `/home/users/b` 访问目录 `a` 和 `b`。最后，名为 `foo` 的两个文件可以通过 `/home/users/a/foo` 和 `/home/users/b/foo` 访问。这就是 mount 的妙处：它没有让许多文件系统各自分开，而是把所有文件系统统一到一棵树中，让命名变得统一而方便。

要查看系统上挂载了哪些文件系统、挂载在哪里，只需运行 `mount` 程序。你会看到类似这样的输出：

```
/dev/sda1 on / type ext3 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
/dev/sda5 on /tmp type ext3 (rw)
/dev/sda7 on /var/vice/cache type ext3 (rw)
tmpfs on /dev/shm type tmpfs (rw)
AFS on /afs type afs (rw)
```

这个乱七八糟的组合表明，许多不同的文件系统都被粘合到了这台机器的文件系统树上，包括 ext3（一种标准的基于磁盘的文件系统）、proc 文件系统（一个用于访问当前进程

提示：小心 TOCTTOU

1974 年，McPhee 注意到计算机系统中的一个问题。具体来说，McPhee 指出："……如果在有效性检查与该有效性检查所关联的操作之间存在一段时间间隔，[并且]通过多任务处理，有效性检查所用的变量可以在这段时间间隔内被蓄意更改，就会导致控制程序执行一个无效的操作。"今天我们把这称为 **Time Of Check To Time Of Use**（**TOCTTOU**）问题，可惜的是，它至今仍可能发生。

Bishop 和 Dilger [BD96] 描述了一个简单的例子，展示用户如何欺骗一个更受信任的服务，从而引发麻烦。例如，设想一个邮件服务以 root 身份运行（因此有权访问系统上的所有文件）。这个服务按如下方式把收到的消息追加到用户的收件箱文件中。首先，它调用 `lstat()` 获取这个文件的信息，特别是确认它确实只是一个属于目标用户的普通文件，而不是指向其他文件的链接（邮件服务器不该更新那些文件）。然后，检查通过之后，服务器用新消息更新这个文件。

不幸的是，检查和更新之间的间隙会导致一个问题：攻击者（在这个例子里，就是收信的那个用户，因此有权访问收件箱）调用 `rename()`，把收件箱文件换成指向一个敏感文件，比如 `/etc/passwd`（它保存着用户及其密码的信息）。如果这个替换恰好发生在合适的时机（在检查和访问之间），服务器就会若无其事地用邮件内容更新这个敏感文件。现在，攻击者只需发一封电子邮件就能写这个敏感文件，这是一种权限提升（privilege escalation）；通过更新 `/etc/passwd`，攻击者可以添加一个具有 root 权限的账户，从而控制整个系统。

TOCTTOU 问题没有什么简单又出色的解决办法 [T+08]。一种方法是减少需要 root 权限才能运行的服务的数量，这会有所帮助。`O_NOFOLLOW` 标志能让 `open()` 在目标是符号链接时失败，从而避免那些需要借助符号链接的攻击。更激进的方法，比如使用**事务文件系统**（transactional file system）[H+18]，可以解决这个问题，但广泛部署的事务文件系统并不多。所以，只能给出那条老生常谈（也挺无力）的建议：编写以高权限运行的代码时要小心！

信息的文件系统）、tmpfs（一个专门存放临时文件的文件系统）和 AFS（一个分布式文件系统，distributed file system）。

## 39.18 小结

UNIX 系统（其实任何系统都是这样）的文件系统接口看上去相当简陋，但如果想掌握它，要理解的东西还很多。当然，没有什么比直接（大量地）使用它更好的了。所以请多用吧！当然，也要多读些资料；和往常一样，可以先从 Stevens 的书 [SR05] 读起。

补充：文件系统的关键术语

- **文件**是一个字节数组，可以创建、读取、写入和删除。它有一个唯一指代它的 low-level name（即一个数字）。这个 low-level name 通常叫作 **i-number**。
- **目录**（directory）是元组的集合，每个元组包含一个人类可读的名字，以及它所映射的 low-level name。每个条目要么指向另一个目录，要么指向一个文件。每个目录本身也有一个 low-level name（i-number）。目录总有两个特殊条目：. 条目指向目录自己，.. 条目指向它的父目录。
- **目录树**（directory tree）或**目录层次结构**（directory hierarchy）把所有文件和目录组织成一棵大树，从**根**开始。
- 要访问文件，进程必须用一个系统调用（通常是 `open()`）向操作系统请求权限。如果获得许可，操作系统就返回一个**文件描述符**（file descriptor），之后可以在权限和意图允许的范围内，用它来读或写。
- 每个文件描述符都是一个进程私有的实体，它指向**打开文件表**（open file table）中的一个条目。这个条目记录了此次访问针对的是哪个文件、文件的**当前偏移量**（current offset，即下一次读或写将访问文件的哪一部分），以及其他相关信息。
- 调用 `read()` 和 `write()` 自然会更新当前偏移量；此外，进程可以用 `lseek()` 修改它的值，从而能够随机访问（random access）文件的不同部分。
- 要把更新强制写入持久介质，进程必须使用 `fsync()` 或相关的调用。不过，要在保持高性能的同时正确地做到这一点很有挑战性 [P+14]，所以这样做时要仔细考虑。
- 要让文件系统中的多个人类可读的名字指向同一个底层文件，可以使用**硬链接**（hard link）或**符号链接**（symbolic link）。两者在不同的情况下各有用处，所以使用前要考虑它们各自的优缺点。还要记住，删除文件只不过是把它从目录层次结构（directory hierarchy）中最后一次 `unlink()` 掉而已。
- 大多数文件系统都有启用和禁止共享的机制。**权限位**（permission bits）提供了这类控制的一种简单形式；更精细的**访问控制列表**（access control list）可以更精确地控制谁能访问和操作信息。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[BD96] "Checking for Race Conditions in File Accesses" by Matt Bishop, Michael Dilger. Computing Systems 9:2, 1996. *A great description of the TOCTTOU problem and its presence in file systems.*

[CK+08] "The xv6 Operating System" by Russ Cox, Frans Kaashoek, Robert Morris, Nickolai Zeldovich. From: https://github.com/mit-pdos/xv6-public. *As mentioned before, a cool and simple Unix implementation. We have been using an older version (2012-01-30-1-g1c41342) and hence some examples in the book may not match the latest in the source.*

[H+18] "TxFS: Leveraging File-System Crash Consistency to Provide ACID Transactions" by Y. Hu, Z. Zhu, I. Neal, Y. Kwon, T. Cheng, V. Chidambaram, E. Witchel. USENIX ATC '18, June 2018. *The best paper at USENIX ATC '18, and a good recent place to start to learn about transactional file systems.*

[K19] "Persistent Memory Programming on Conventional Hardware" by Terence Kelly. ACM Queue, 17:4, July/August 2019. *A great overview of persistent memory programming; check it out!*

[K20] "Is Persistent Memory Persistent?" by Terence Kelly. Communications of the ACM, 63:9, September 2020. *An engaging article about how to test hardware failures in system on the cheaps; who knew breaking things could be so fun?*

[K84] "Processes as Files" by Tom J. Killian. USENIX, June 1984. *The paper that introduced the /proc file system, where each process can be treated as a file within a pseudo file system. A clever idea that you can still see in modern* UNIX *systems.*

[L84] "Capability-Based Computer Systems" by Henry M. Levy. Digital Press, 1984. Available: http://homes.cs.washington.edu/~levy/capabook. *An excellent overview of early capability-based systems.*

[MJLF84] "A Fast File System for UNIX" by Marshall K. McKusick, William N. Joy, Sam J. Leffler, Robert S. Fabry. ACM TOCS, 2:3, August 1984. *We'll talk about the Fast File System (FFS) explicitly later on. Here, we refer to it because of all the other random fun things it introduced, like long file names and symbolic links. Sometimes, when you are building a system to improve one thing, you improve a lot of other things along the way.*

[P+13] "Towards Efficient, Portable Application-Level Consistency" by Thanumalayan S. Pillai, Vijay Chidambaram, Joo-Young Hwang, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. HotDep '13, November 2013. *Our own work that shows how readily applications can make mistakes in committing data to disk; in particular, assumptions about the file system creep into applications and thus make the applications work correctly only if they are running on a specific file system.*

[P+14] "All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications" by Thanumalayan S. Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. OSDI '14, Broomfield, Colorado, October 2014. *The full conference paper on this topic – with many more details and interesting tidbits than the first workshop paper above.*

[SK09] "Principles of Computer System Design" by Jerome H. Saltzer and M. Frans Kaashoek. Morgan-Kaufmann, 2009. *This tour de force of systems is a must-read for anybody interested in the field. It's how they teach systems at MIT. Read it once, and then read it a few more times to let it all soak in.*

[SR05] "Advanced Programming in the UNIX Environment" by W. Richard Stevens and Stephen A. Rago. Addison-Wesley, 2005. *We have probably referenced this book a few hundred thousand times. It is that useful to you, if you care to become an awesome systems programmer.*

[T+08] "Portably Solving File TOCTTOU Races with Hardness Amplification" by D. Tsafrir, T. Hertz, D. Wagner, D. Da Silva. FAST '08, San Jose, California, 2008. *Not the paper that introduced TOCTTOU, but a recent-ish and well-done description of the problem and a way to solve the problem in a portable manner.*

## 作业（编程）

在这个作业中，我们只是熟悉一下本章介绍的 API 是如何工作的。为此，你只需编写几个不同的程序，它们大多基于各种 UNIX 工具。

## 问题

1. **Stat：**编写你自己版本的命令行程序 `stat`，它只是对给定的文件或目录调用 `stat()` 系统调用。打印出文件大小、已分配的块数、引用（链接）计数等等。随着目录中条目数的变化，目录的链接计数是多少？有用的接口：当然是 `stat()`。
2. **列出文件：**编写一个程序，列出给定目录中的文件。不带任何参数调用时，程序应该只打印文件名。带 `-l` 选项调用时，程序应该打印出每个文件的信息，比如属主、组、权限，以及从 `stat()` 系统调用获得的其他信息。程序应该再接受一个参数，即要读取的目录，例如 `myls -l directory`。如果没有给出目录，程序就应该使用当前工作目录（current working directory）。有用的接口：`stat()`、`opendir()`、`readdir()`、`getcwd()`。
3. **Tail：**编写一个程序，打印出文件的最后几行。程序要高效：它应该先定位到文件末尾附近，读入一块数据，然后往回找，直到找到所要求的行数；这时，它应该从这些行的开头一直打印到文件末尾。调用这个程序时，应该输入 `mytail -n file`，其中 `n` 是要打印的文件末尾的行数。有用的接口：`stat()`、`lseek()`、`open()`、`read()`、`close()`。
4. **递归搜索：**编写一个程序，从文件系统树中的某个给定位置开始，打印出树中每个文件和目录的名字。例如，不带参数运行时，程序应该从当前工作目录（current working directory）开始，打印它的内容以及所有子目录的内容等等，直到把以当前工作目录（CWD）为根的整棵树都打印出来。如果给出一个参数，即目录名，就改用它作为树的根。再给你的递归搜索加上一些更有趣的选项，类似于强大的命令行工具 `find`。有用的接口：你自己去找吧。

 © 2008–23, ARPACI-DUSSEAU

# 40

# 文件系统实现

在本章中，我们介绍一个简单的文件系统（file system）实现，叫作 **vsfs**（**Very Simple File System**，非常简单的文件系统）。它是典型 UNIX 文件系统的简化版，因此可以用来介绍一些基本的磁盘上的结构、访问方法（access method）和各种策略，这些在当今的许多文件系统中都能看到。

文件系统是纯软件；和前面讲 CPU 和内存虚拟化（virtualization）时不同，我们不会添加硬件特性来让文件系统的某个方面工作得更好（不过我们还是要留意设备的特性，确保文件系统运行良好）。正因为构建文件系统的灵活性很大，人们做出了许多各不相同的文件系统，简直是从 A 排到了 Z：从 AFS（Andrew File System）[H+88] 到 ZFS（Sun 的 Zettabyte File System）[B07]。这些文件系统各有各的数据结构（data structure），有些方面比同类强，有些方面则不如同类。因此，我们学习文件系统的方式是案例研究：本章先用一个简单的文件系统（vsfs）介绍大部分概念，然后研究一系列真实的文件系统，了解它们在实践中有哪些不同。

关键问题：如何实现一个简单的文件系统
如何构建一个简单的文件系统？磁盘上需要哪些结构？它们需要记录什么？如何访问它们？

## 40.1 思考的方式

思考文件系统时，我们通常建议从两个方面入手；如果这两个方面你都理解了，大概也就理解了文件系统的基本工作原理。

第一个方面是文件系统的**数据结构**（data structures）。换句话说，文件系统用哪些类型的磁盘上的结构来组织它的数据和元数据（metadata）？我们将看到的第一批文件系统（包括下面的 vsfs）采用简单的结构，比如块（block）或其他对象的数组；而更复杂的文件系统，比如 SGI 的 XFS，则使用更复杂的基于树的结构 [S+96]。

文件系统的第二个方面是它的**访问方法**（access methods）。它如何把进程发出的调用，比如 `open()`、`read()`、`write()` 等，映射到它的结构上？执行某个系统调用（system call）时，要读哪些结构？要写哪些结构？所有这些步骤执行得有多高效？

补充：文件系统的心智模型

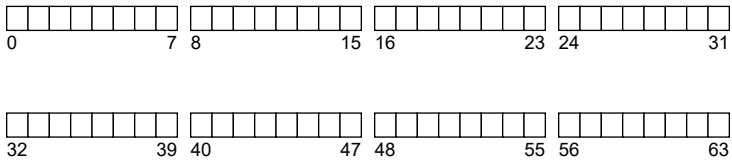
我们之前讨论过，学习系统时，你真正要建立的是心智模型。对文件系统来说，你的心智模型最终应该能回答这样一些问题：磁盘上的哪些结构存放着文件系统的数据和元数据？进程（process）打开一个文件时会发生什么？读或写的过程中会访问磁盘上的哪些结构？不断建立和改进心智模型，你就会对正在发生的事情形成抽象的理解，而不只是试图弄懂某段文件系统代码的细节（当然，弄懂代码也很有用！）。

如果你理解了文件系统的数据结构和访问方法（access method），就对它真正的工作方式有了一个很好的心智模型，这是系统思维的关键一环。下面深入研究第一个实现时，请试着逐步建立起这样的模型。

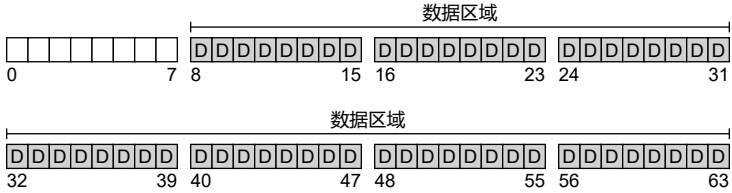
## 40.2 整体组织

现在我们来设计 vsfs 的数据结构在磁盘上的整体组织。首先要做的是把磁盘划分成**块**（block）；简单的文件系统只使用一种块大小（block size），我们这里也这样做，选用常见的 4 KB。

因此，在我们看来，要在上面构建文件系统的这个磁盘分区很简单：一串块，每块 4 KB。在一个大小为 $N$ 个 4 KB 块的分区中，块的地址从 0 到 $N-1$。假设我们有一个很小的磁盘，只有 64 个块：



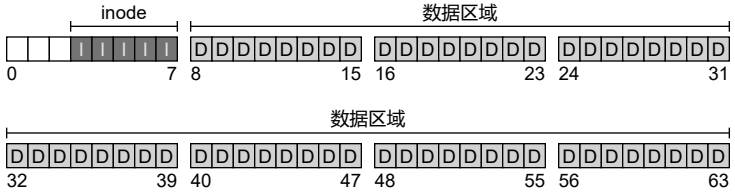

现在我们来想想，要构建一个文件系统，需要在这些块里存放什么。当然，首先想到的是用户数据。事实上，任何文件系统的大部分空间都是（而且应该是）用户数据。我们把磁盘上存放用户数据的区域叫作**数据区域**（data region），同样为了简单，在磁盘上为这些块预留一块固定的部分，比如 64 个块中的最后 56 个：




我们在上一章（稍微）了解过，文件系统必须记录每个文件的信息。这些信息是一种关键的**元数据**（metadata），记录着文件由数据区域（data region）中的哪些数据块（data block）

组成、文件的大小、它的所有者和访问权限、访问和修改时间等。为了存放这些信息，文件系统通常有一种叫作 **inode** 的结构（下面会进一步介绍 inode）。

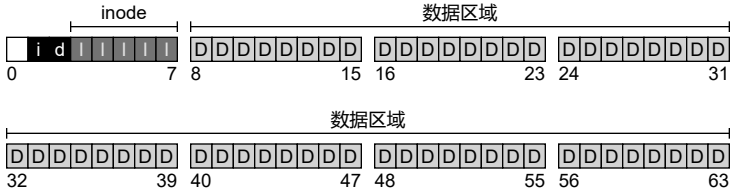
为了容纳 inode，我们也需要在磁盘上为它们预留一些空间。我们把这部分磁盘叫作 **inode 表**（inode table），它存放的就是一个磁盘上的 inode 数组。假设我们用 64 个块中的 5 个存放 inode（图中用 I 表示），磁盘映像现在是这个样子：




这里要指出，inode 通常不太大，比如 128 或 256 字节。假设每个 inode 256 字节，一个 4 KB 的块就能容纳 16 个 inode，上面的文件系统一共有 80 个 inode。我们这个简单文件系统建在只有 64 个块的小分区上，80 就是它最多能容纳的文件数；不过要注意，同样的文件系统如果构建在更大的磁盘上，只需分配一个更大的 inode 表，就能容纳更多文件。

到目前为止，我们的文件系统有了数据块（D）和 inode（I），但还缺几样东西。你可能已经猜到，还缺的一个主要部分，是用某种方法记录 inode 和数据块是空闲还是已分配。因此，这种**分配结构**（allocation structures）是任何文件系统都必不可少的组成部分。

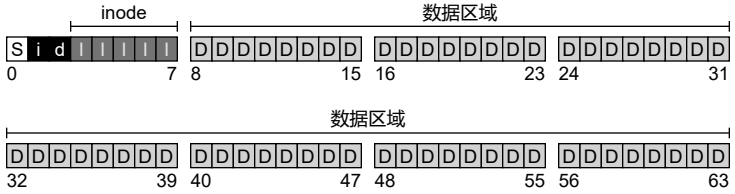
当然，记录分配情况的方法有很多。例如，我们可以用一个**空闲链表**（free list），它指向第一个空闲块（free block），第一个空闲块又指向下一个空闲块，依此类推。我们这里选择一种简单而流行的结构，叫作**位图**（bitmap），数据区域（data region）和 inode 表各用一个，分别叫**数据位图**（data bitmap）和 **inode 位图**（inode bitmap）。位图是一种简单的结构：每一位表示对应的对象或块是空闲（0）还是在用（1）。于是，加上 inode 位图（i）和数据位图（d）之后，新的磁盘布局如下：




你可能会注意到，用整整一个 4 KB 的块来存放这些位图有点大材小用；这样一个位图可以记录 32K 个对象是否已分配，而我们只有 80 个 inode 和 56 个数据块。不过，为了简单，我们还是给每个位图都分配一整个 4 KB 的块。

细心的读者（也就是还没睡着的读者）可能已经注意到，我们这个非常简单的文件系统，在设计磁盘结构时还剩下一个块没有安排。我们把它留给**超级块**（superblock），在下图中用 S 表示。超级块包含这个文件系统的信息，例如文件系统中有多少个 inode 和数据块（在这个例子中分别是 80 和 56）、inode 表从哪里开始（块 3）等等。它很可能还包含某种幻数（magic number），用来标识文件系统的类型（这里是 vsfs）。

 © 2008–26, ARPACI-DUSSEAU

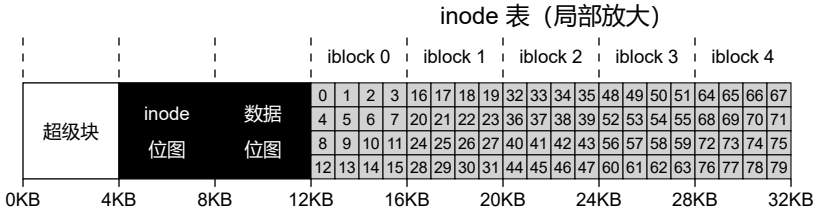



因此，挂载（mount）文件系统时，操作系统会先读取超级块，初始化各种参数，然后把这个卷（volume）挂接到文件系统树上。这样，访问卷中的文件时，系统就能确切地知道去哪里找所需的磁盘上的结构。

## 40.3 文件组织：inode

inode 是文件系统中最重要的磁盘上的结构之一；几乎所有文件系统都有类似的结构。inode 这个名字是 **index node**（**索引节点**）的简称，这是 UNIX [RT74]（也许还有更早的系统）里的老叫法。这个名字反映了一个事实：这些节点最初存放在一个数组中，系统通过索引这个数组来访问某个特定的 inode。

每个 inode 都隐式地对应一个数字（叫作 **inode 号**，i-number），我们之前把它叫作文件的 **low-level name**。在 vsfs 以及其他简单的文件系统中，给定一个 inode 号，就应该能直接算出对应的 inode 在磁盘上的位置。例如，拿上面 vsfs 的 inode 表来说：它的大小为 20 KB（5 个 4 KB 的块），因此包含 80 个 inode（假设每个 inode 256 字节）；再假设 inode 区域从 12 KB 开始（也就是说，超级块从 0 KB 开始，inode 位图在地址 4 KB 处，数据位图在 8 KB 处，inode 表紧随其后）。于是，在 vsfs 中，文件系统分区开头部分的布局如下（局部放大）：




补充：数据结构——inode

**inode** 是许多文件系统都在用的通用名称，指存放某个文件的元数据的结构，比如文件的长度、权限以及组成它的各个块的位置。这个名字至少可以追溯到 UNIX（很可能还能追溯到 Multics，甚至更早的系统）；它是 **index node** 的简称，因为要用 inode 号（inode number）作为索引，在磁盘上的 inode 数组中找到该编号的 inode。我们会看到，inode 的设计是文件系统设计的一个关键部分。大多数现代系统都给自己管理的每个文件配了一个这样的结构，只是叫法可能不同（比如 dnode、fnode 等）。

要读取 32 号 inode，文件系统首先计算它在 inode 区域内的偏移量（offset），即 $32 \cdot sizeof(inode) = 8192$，再加上磁盘上 inode 表的起始地址（`inodeStartAddr = 12KB`），就得到了所需 inode 块（inode block）的正确字节地址：20 KB。回想一下，磁盘不能按字节寻址，而是由大量可寻址的扇区（sector）组成，扇区通常是 512 字节。因此，为了取出包含 32 号 inode 的 inode 块（inode block），文件系统会向扇区 $\frac{20\times1024}{512}$，也就是 40 号扇区，发出读请求。更一般地，inode 块（inode block）所在的扇区地址 `sector` 可以这样计算：

```
blk    = (inumber * sizeof(inode_t)) / blockSize;
sector = ((blk * blockSize) + inodeStartAddr) / sectorSize;
```

关于一个文件，你需要的信息几乎都在它的 inode 里：它的类型（例如普通文件、目录（directory）等）、大小、分配给它的块（block）数、保护信息（比如文件归谁所有、谁可以访问它）、一些时间信息，包括文件的创建、修改或最后访问时间，以及它的数据块在磁盘上的位置信息（比如某种指针）。我们把关于一个文件的所有这类信息称为**元数据**（metadata）；其实，文件系统里凡是不属于纯用户数据的信息，通常都这样称呼。图 40.1 是 ext2 [P09] 中的一个 inode 示例[1]。

inode 设计中最重要的决策之一，是它如何指向数据块所在的位置。一种简单的方法是在 inode 中放一个或多个**直接指针**（direct pointer，即磁盘地址）；每个指针指向属于该文

| 大小 | 名称 | 这个 inode 字段有什么用? |
|---|---|---|
| 2 | mode | 这个文件能否被读、写、执行? |
| 2 | uid | 这个文件归谁所有? |
| 4 | size | 这个文件有多少字节? |
| 4 | time | 这个文件最后一次被访问是什么时候? |
| 4 | ctime | 这个文件的 inode 最后一次被修改是什么时候? |
| 4 | mtime | 这个文件最后一次被修改是什么时候? |
| 4 | dtime | 这个 inode 是什么时候被删除的? |
| 2 | gid | 这个文件属于哪个组? |
| 2 | links_count | 这个文件有多少个硬链接? |
| 4 | blocks | 已经给这个文件分配了多少个块? |
| 4 | flags | ext2 应该如何使用这个 inode? |
| 4 | osd1 | 一个与操作系统相关的字段 |
| 60 | block | 一组磁盘指针（共 15 个） |
| 4 | generation | 文件版本（供 NFS 使用） |
| 4 | file_acl | 一种比 mode 位更新的权限模型 |
| 4 | dir_acl | 称为访问控制列表 |

图 40.1：**简化的 Ext2 inode**

[1]类型信息保存在目录项（directory entry）中，因此 inode 本身里没有这项信息。

 © 2008–26, ARPACI-DUSSEAU

提示：考虑基于区段的方法

另一种方法是用**区段**（extent）代替指针。一个区段就是一个磁盘指针加上一个长度（以块为单位）；因此，不需要为文件的每个块都准备一个指针，只需一个指针和一个长度，就能指明文件在磁盘上的位置。只用一个区段会有局限，因为分配文件时可能很难在磁盘上找到一块连续的空闲空间（free space）。因此，基于区段的文件系统通常允许使用多个区段，这样在分配文件时文件系统就有了更大的自由。

比较这两种方法，基于指针的方法更灵活，但每个文件要用大量元数据，大文件尤其如此。基于区段的方法没那么灵活，但更紧凑；特别是当磁盘上有足够的空闲空间、文件可以连续存放时（这本来就是几乎所有文件分配策略的目标），它们的效果很好。

件的一个磁盘块（disk block）。这种方法有局限：例如，如果你想要一个非常大的文件（比如比块大小乘以 inode 中直接指针的个数还大），那就没办法了。

### 多级索引

为了支持更大的文件，文件系统设计者不得不在 inode 中引入不同的结构。一个常见的思路是使用一种叫作**间接指针**（indirect pointer）的特殊指针。它不指向存放用户数据的块，而是指向一个存放更多指针的块，这些指针再分别指向用户数据。因此，一个 inode 可以有固定数量的直接指针（direct pointer），比如 12 个，再加一个间接指针（indirect pointer）。如果文件变得足够大，就（从磁盘的数据块区域）分配一个间接块（indirect block），并让 inode 中存放间接指针（indirect pointer）的那一项指向它。假设块大小为 4 KB、磁盘地址（disk address）为 4 字节，这样又多了 1024 个指针；文件最大可以达到 $(12 + 1024) \cdot 4K$，即 4144 KB。

不难想到，有了这种方法，你可能还想支持更大的文件。为此，只需在 inode 中再加一个指针：**二级间接指针**（double indirect pointer）。这个指针指向一个块，块中存放的是指向间接块（indirect block）的指针，每个间接块里又存放着指向数据块的指针。因此，一个二级间接块（double indirect block）让文件还能再增加 $1024 \times 1024$ 个，也就是 100 万个 4 KB 的块，换句话说，可以支持超过 4 GB 的文件。不过你可能还想要更大的，我们打赌你已经知道接下来是什么了：**三级间接指针**（triple indirect pointer）。

总的来说，用这种不平衡的树指向文件块的方法，叫作**多级索引**（multi-level index）。我们来看一个例子：有 12 个直接指针（direct pointer），以及一个一级间接块（indirect block）和一个二级间接块（double indirect block）。假设块大小为 4 KB、指针为 4 字节，这种结构可以容纳略大于 4 GB 的文件（即 $(12 + 1024 + 1024^2) \times 4$ KB）。你能算出再加上一个三级间接块（triple indirect block）后，能支持多大的文件吗？（提示：相当大）

许多文件系统都使用多级索引（multi-level index），包括常用的 Linux ext2 [P09] 和 ext3、NetApp 的 WAFL，以及最初的 UNIX 文件系统。另一些文件系统，包括 SGI XFS 和 Linux ext4，则使用**区段**（extent）而不是简单的指针；基于区段的方案如何工作，详见前面的提示框；区段有点像讨论虚拟内存（virtual memory）时讲过的段（segment）。

| | |
|---|---|
| **大多数文件都很小** | 最常见的大小约为 2K |
| **平均文件大小在增长** | 平均大小接近 200K |
| **大部分字节存放在大文件中** | 少数大文件占用了大部分空间 |
| **文件系统包含大量文件** | 平均接近 10 万个 |
| **文件系统大约是半满的** | 即使磁盘不断变大，文件系统仍保持约 50% 的占用率 |
| **目录通常很小** | 很多目录只有几个条目；大多数目录的条目不超过 20 个 |

图 40.2：**文件系统测量结果汇总**

你可能会想：为什么要用这样一棵不平衡的树？为什么不用别的方法？原来，许多研究人员考察过文件系统及其使用方式，几乎每一次都会发现一些几十年来始终成立的“真理”。其中一个发现是大多数文件都很小。这种不平衡的设计正反映了这一现实；如果大多数文件确实很小，就应该针对这种情况做优化。因此，只用少量的直接指针（12 是一个典型的数目），inode 就可以直接指向 48 KB 的数据，更大的文件才需要一个（或多个）间接块（indirect block）。比较新的一项研究见 Agrawal 等人的论文 [A+07]；图 40.2 总结了其中的结果。

当然，inode 的设计还有许多其他可能；毕竟，inode 只是一种数据结构，任何能存放相关信息、并能高效查询这些信息的数据结构都可以。文件系统软件很容易修改，所以如果工作负载（workload）或技术发生变化，你应该乐于探索不同的设计。

## 40.4 目录组织

在 vsfs 中（和许多文件系统一样），目录的组织很简单；一个目录基本上只包含一个由（条目名，inode 号）对组成的列表。目录里的每个文件或子目录，在该目录的数据块中都对应一个字符串和一个数字；每个字符串可能还带一个长度（假设名字长度可变）。

例如，假设目录 `dir` 的 inode 号为 5，其中有三个文件（`foo`、`bar` 和 `foobar_is_a_pretty_longname`），它们的 inode 号分别是 12、13 和 24。`dir` 在磁盘上的数据可能是这样的：

```
inum | reclen | strlen | name
   5      12       2      .
   2      12       3      ..
  12      12       4      foo
  13      12       4      bar
  24      36      28      foobar_is_a_pretty_longname
```

在这个例子中，每个条目都有一个 inode 号、记录长度（名字的总字节数加上剩余的空间）、字符串长度（名字的实际长度），最后是条目的名字。注意，每个目录都有两个额外的条目：`.`（“点”）和 `..`（“点点”）；点目录（dot directory）就是当前目录，在这个例子中是 `dir`；点点则是父目录，在这个例子中是根目录（root directory）。

补充：基于链接的方法

设计 inode 还有一种更简单的方法，就是使用**链表**（linked list）。这样，inode 内部不需要多个指针，只需要一个，指向文件的第一个块。文件更大时，就在这个数据块的末尾再放一个指针，依此类推，这样就能支持大文件了。

你可能已经猜到，链接方式的文件分配在某些工作负载下表现很差；比如想想读取文件的最后一个块，或者只是做随机访问（random access）。因此，为了让链接分配更好用，有些系统会在内存中保存一张链接信息表，而不是把 next 指针和数据块本身存在一起。这张表以数据块 D 的地址为索引；表项的内容就是 D 的 next 指针，即文件中紧跟在 D 后面的那个块的地址。表项里也可以是空值（表示文件结束），或者其他某种标记，表示某个块是空闲的。有了这样一张 next 指针表，链接分配方案就能高效地支持随机访问文件：先在（内存中的）表里扫描，找到所需的块，然后直接（在磁盘上）访问它。

这样的表听起来是不是很熟悉？我们所描述的，正是**文件分配表**（file allocation table），也就是 **FAT** 文件系统的基本结构。没错，这个在 NTFS [C94] 之前的经典 Windows 文件系统，用的正是一种简单的链接分配方案。它和标准的 UNIX 文件系统还有其他区别；例如，它本身没有 inode，而是由目录项存放文件的元数据，并直接指向文件的第一个块，所以它没法创建硬链接（hard link）。更多不那么优雅的细节，见 Brouwer 的文章 [B02]。

删除一个文件（比如调用 `unlink()`）可能会在目录中间留下一段空白，因此也应该有某种方法把它标记出来（比如用一个保留的 inode 号，如零）。这种删除正是需要记录长度的原因之一：新条目可能会重用一个旧的、更大的条目，里面就会留有多余的空间。

你可能想知道目录到底存放在哪里。通常，文件系统把目录当作一种特殊类型的文件。因此，目录有一个 inode，位于 inode 表中的某处（inode 的类型字段标记为“目录”而不是“普通文件”）。inode 指向目录的数据块，可能还有间接块（indirect block）；这些块位于我们这个简单文件系统的数据块区域中。因此，我们的磁盘结构保持不变。

我们还要再次指出，这种简单的目录项线性列表并不是存放这类信息的唯一方式。和前面一样，任何数据结构都可以。例如，XFS [S+96] 把目录存成 B 树（B-tree）。创建文件时必须先确认文件名还没被用过，所以和使用简单列表、必须完整扫描列表的系统相比，XFS 创建文件更快。

补充：空闲空间管理

管理空闲空间的方法有很多；位图只是其中一种。一些早期的文件系统使用**空闲链表**（free list），超级块中保存一个指针，指向第一个空闲块；该块中又保存下一个空闲块的指针，这样就在系统的空闲块之间串起了一个链表。需要一个块时，就使用表头的块，并相应地更新链表。

现代文件系统使用更复杂的数据结构。例如，SGI 的 XFS [S+96] 使用某种形式的 **B 树**（B-tree）来紧凑地表示磁盘的哪些部分是空闲的。和任何数据结构一样，可以做出不同的时空权衡（time-space trade-off）。

## 40.5 空闲空间管理

文件系统必须记录哪些 inode 和数据块是空闲的、哪些不是，这样在分配新文件或目录时，才能为它找到空间。因此，**空闲空间管理**（free space management）对所有文件系统都很重要。在 vsfs 中，我们用两个简单的位图来完成这项任务。

例如，创建一个文件时，我们必须为它分配一个 inode。文件系统会在位图中找一个空闲的 inode 分配给这个文件，把它标记为已用（置为 1），并最终把正确的信息写回磁盘上的位图。分配数据块时也要做一组类似的操作。

为新文件分配数据块时，可能还要考虑其他一些因素。例如，ext2、ext3 等 Linux 文件系统在创建新文件并需要数据块时，会寻找一串连续的空闲块（比如 8 个）；找到这样一串空闲块分配给新文件，文件系统就能保证文件至少有一部分在磁盘上是连续的，从而提高性能。因此，这种**预分配**（pre-allocation）策略是为数据块分配空间时常用的启发式方法。

## 40.6 访问路径：读和写

现在我们对文件和目录在磁盘上如何存放有了一些概念，应该能够跟踪读或写一个文件时的操作流程了。因此，理解在这条**访问路径**（access path）上发生了什么，是理解文件系统工作原理的第二个关键；注意了！

在下面的例子中，我们假设文件系统已经挂载，因此超级块已经在内存中。其他所有东西（即 inode、目录）都还在磁盘上。

### 从磁盘读取文件

在这个简单的例子中，我们先假设你只是想打开一个文件（比如 `/foo/bar`），读取它，然后关闭它。为简单起见，假设这个文件只有 12 KB（即 3 个块）。

| | 数据位图 | inode 位图 | 根目录 inode | foo inode | bar inode | 根目录数据 | foo 数据 | bar 数据 [0] | bar 数据 [1] | bar 数据 [2] |
|---|---|---|---|---|---|---|---|---|---|---|
| open(bar) | | | **读** | | | **读** | | | | |
| | | | | **读** | | | **读** | | | |
| | | | | | **读** | | | | | |
| read() | | | | | **读** | | | **读** | | |
| | | | | | **写** | | | | | |
| read() | | | | | **读** | | | | **读** | |
| | | | | | **写** | | | | | |
| read() | | | | | **读** | | | | | **读** |
| | | | | | **写** | | | | | |

图 40.3：**读取文件的时间线（时间向下递增）**

当你发出 `open("/foo/bar", O_RDONLY)` 调用时，文件系统首先要找到文件 `bar` 的 inode，以获得关于这个文件的一些基本信息（权限信息、文件大小等）。可文件系统手里只有完整的路径名（pathname），所以必须**遍历**（traverse）路径名，才能找到这个 inode。

所有遍历都从文件系统的根开始，也就是**根目录**（root directory），它就叫 `/`。因此，文件系统要从磁盘读取的第一样东西，就是根目录的 inode。但这个 inode 在哪里呢？要找到一个 inode，我们必须知道它的 inode 号。通常，一个文件或目录的 inode 号是在它的父目录中找到的；而根目录按定义没有父目录。因此，根目录的 inode 号必须是“众所周知”的，文件系统在挂载时就得知道它是多少。在大多数 UNIX 文件系统中，根目录的 inode 号是 2。因此，为了开始这个过程，文件系统读入包含 2 号 inode 的块，也就是第一个 inode 块（inode block）。

inode 读入后，文件系统就可以从中找到指向数据块的指针，根目录的内容就存放在这些数据块里。于是，文件系统用这些磁盘上的指针读取目录，在这个例子中是寻找 `foo` 的条目。读入一个或多个目录数据块后，它会找到 `foo` 的条目，从而知道 `foo` 的 inode 号（假设是 44），下一步要用到它。

下一步是递归地遍历路径名，直到找到所需的 inode。在这个例子中，文件系统读取包含 `foo` 的 inode 的块，再读取它的目录数据，最终找到 `bar` 的 inode 号。`open()` 的最后一步是把 `bar` 的 inode 读入内存；然后文件系统做最后的权限检查，在这个进程的打开文件表（open file table）中分配一个文件描述符（file descriptor），并把它返回给用户。

打开之后，程序就可以发出 `read()` 系统调用来读这个文件。第一次读（除非调用过 `lseek()`，否则从偏移量 0 开始）会读入文件的第一个块，读之前要查阅 inode 来找到这个块的位置；它可能还会用新的最后访问时间更新 inode。读操作还会更新内存中打开文件表里这个文件描述符的表项，修改其中的文件偏移量（file offset），这样下一次读就会读入文件的第二个块，依此类推。

到某个时候，程序会关闭文件。这里要做的工作少得多；显然，要释放文件描述符，但目前来说，文件系统真正要做的也就这些了。不会发生任何磁盘 I/O。

图 40.3（第 448 页）描绘了整个过程；图中时间从上往下递增。在图中，open 引发了多次读，才最终找到文件的 inode。之后，读取每个块时，文件系统都要先查阅 inode，然后读取这个块，再写一次 inode，更新它的最后访问时间字段。花点时间，弄明白这是怎么回事。

还要注意，open 产生的 I/O 量与路径名的长度成正比。路径中每多一层目录，我们就要多读它的 inode 和数据。如果目录很大，情况会更糟；在这里，我们只需读一个块就能

补充：读操作不访问分配结构

我们见过很多学生被位图之类的分配结构（allocation structure）弄糊涂。特别是，很多人以为，只是读取文件、不分配任何新块的时候，仍然要查阅位图。不是这样的！位图之类的分配结构（allocation structure）只有在需要分配时才会用到。inode、目录和间接块（indirect block）中有完成读请求所需的全部信息；inode 已经指向某个块了，就不需要再确认这个块是否已分配。

得到目录的内容，而如果目录很大，可能要读很多个数据块才能找到所需的条目。没错，读一个文件时，情况就已经相当糟糕了；你马上就会发现，写一个文件（尤其是创建一个新文件）更糟。

### 向磁盘写入文件

写文件的过程与此类似。首先，必须打开文件（如上所述）。然后，应用程序可以发出 `write()` 调用，用新内容更新文件。最后，关闭文件。

和读不同，写文件还可能需要**分配**（allocate）一个块（除非是在覆盖已有的块）。写一个新文件时，每次写操作不仅要把数据写入磁盘，还要先决定把哪个块分配给这个文件，并相应地更新磁盘上的其他结构，比如数据位图（data bitmap）和 inode。因此，每次写文件在逻辑上会产生五次 I/O：一次读数据位图（然后更新它，把新分配的块标记为已用），一次写位图（把它的新状态反映到磁盘上），再读写 inode 各一次（用新块的位置更新它），最后一次写实际的数据块本身。

如果考虑像创建文件这样简单而常见的操作，写流量就更大了。要创建一个文件，文件系统不仅要分配一个 inode，还要在包含新文件的目录中分配空间。为此产生的 I/O 流量相

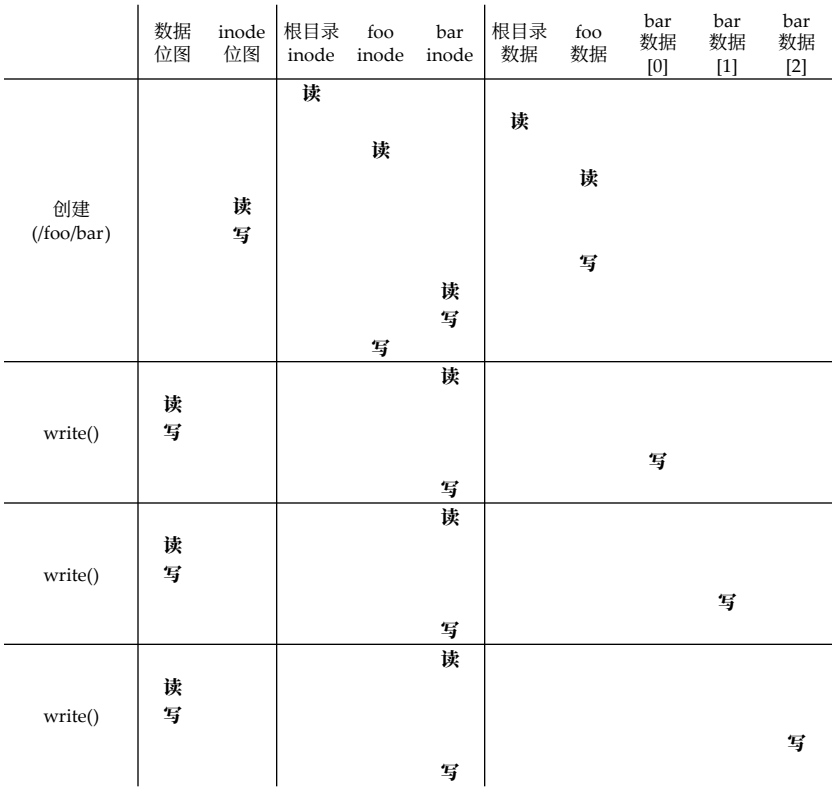



图 40.4：**创建文件的时间线（时间向下递增）**

当大：读一次 inode 位图（找一个空闲的 inode），写一次 inode 位图（把它标记为已分配），写一次新 inode 本身（初始化它），写一次目录的数据（把文件的 high-level name 链接到它的 inode 号），再读写一次目录的 inode 来更新它。如果目录需要增长才能容纳新条目，还需要额外的 I/O，即访问数据位图（data bitmap）和新的目录块。仅仅创建一个文件就要做这么多事！

我们来看一个具体的例子：创建文件 `/foo/bar`，并向它写入三个块。图 40.4（第 450 页）展示了 `open()`（它创建了这个文件）和三次 4 KB 写操作期间各发生了什么。

图中对磁盘的读和写按引发它们的系统调用分组，大致按从上到下的顺序发生。你可以看到创建文件要做多少工作：在这个例子中要 10 次 I/O，先遍历路径名，最后才创建文件[2]。你还可以看到，每次需要分配的写操作要花 5 次 I/O：一对用来读取和更新 inode，另一对用来读取和更新数据位图（data bitmap），最后是写数据本身。文件系统怎样才能以合理的效率完成这些工作呢？

关键问题：如何降低文件系统的 I/O 开销
即使是打开、读或写文件这样最简单的操作，也会引发大量分散在磁盘各处的 I/O 操作。文件系统能做些什么，来降低执行这么多 I/O 的高昂开销？

## 40.7 缓存和缓冲

正如上面的例子所示，读写文件可能代价很高，会对（缓慢的）磁盘产生许多 I/O。这显然是一个严重的性能问题。为了解决它，大多数文件系统都会充分利用系统内存（DRAM）来缓存（cache）重要的块。

设想一下上面那个 open 的例子：如果没有缓存，每打开一个文件，目录层次（directory hierarchy）中的每一级都至少需要读两次（一次读相关目录的 inode，至少一次读它的数据）。如果路径名很长（比如 `/1/2/3/ ... /100/file.txt`），文件系统仅仅为了打开文件，就真的要执行几百次读！

因此，早期的文件系统引入了一个**固定大小的缓存**（fixed-size cache）来存放常用的块。和我们讨论虚拟内存时一样，由 **LRU** 及其各种变体之类的策略决定哪些块保留在缓存中。这个固定大小的缓存通常在启动时分配，大约占总内存的 10%。

然而，这种对内存的**静态划分**（static partitioning）可能造成浪费；如果文件系统在某个时刻并不需要 10% 的内存呢？采用上面描述的固定大小的方法，文件缓存（file cache）中没有用到的页（page）无法挪作他用，就被浪费了。

相比之下，现代系统采用**动态划分**（dynamic partitioning）的方法。具体来说，许多现代操作系统把虚拟内存页和文件系统页整合到一个**统一的页缓存**（unified page cache）中 [S00]。这样，就能看某一时刻虚拟内存和文件系统谁更需要，在两者之间更灵活地分配内存。

[2]这里有一点出人意料：创建 bar 的 inode 时有一次读，尽管它还没有初始化；你能猜到为什么吗？（想一想……）答案：之所以有这次读，是因为 inode（它很小）包含在一个更大的块（比如 4 KB）中。因此，系统要执行一次读-改-写，来更新块中新分配的 inode。

提示：理解静态划分与动态划分

在不同的客户或用户之间分配一种资源时，可以使用**静态划分**（static partitioning），也可以使用**动态划分**（dynamic partitioning）。静态方法只是一次性地把资源按固定比例分好；例如，如果内存有两个可能的使用者，你可以把固定比例的内存分给其中一个，其余的分给另一个。动态方法更灵活，随着时间的推移分出不同数量的资源；例如，某个用户可能在一段时间内得到较高比例的磁盘带宽（bandwidth），但之后系统可能会改变主意，把可用磁盘带宽的更大份额分给另一个用户。

两种方法各有优点。静态划分（static partitioning）确保每个用户都能得到一定份额的资源，性能通常更可预测，而且往往更容易实现。动态划分（dynamic partitioning）可以实现更高的利用率（utilization），让渴求资源的用户使用原本闲置的资源，但实现起来可能更复杂；而且，如果某个用户闲置的资源被别人占用，需要时又要花很长时间才能收回，这个用户的性能就可能更差。和往常一样，没有最好的方法；你应该思考手头的问题，决定哪种方法最合适。说真的，你难道不应该一直这么做吗？

现在设想有缓存时打开文件的例子。第一次打开可能会产生大量 I/O 流量，读入目录的 inode 和数据，但之后再打开同一个文件（或者同一目录下的文件）时，大多会命中缓存，因此不需要 I/O。

我们再来看看缓存对写的影响。只要缓存足够大，读 I/O 就可以完全避免，但写流量必须到达磁盘，才能持久保存。因此，缓存对写流量起不到对读那样的过滤作用。话虽如此，人们有时所说的**写缓冲**（write buffering）无疑有不少性能上的好处。首先，推迟写入，文件系统就可以把一些更新**批量**（batch）合并成更少的 I/O；例如，如果创建一个文件时更新了 inode 位图，片刻之后创建另一个文件时又更新了它，那么第一次更新之后推迟写入，文件系统就省掉了一次 I/O。其次，把一些写缓冲在内存中，系统随后就可以**调度**（schedule）这些 I/O，从而提高性能。最后，有些写推迟之后，就可以完全**避免**（avoid）；例如，如果应用程序创建了一个文件，然后又删除了它，推迟那些把文件创建反映到磁盘上的写，就能完全避免它们。在这种情况下，（在把块写到磁盘这件事上）懒惰是一种美德。

出于上述原因，大多数现代文件系统会在内存中缓冲写操作 5 到 30 秒，这又是一种权衡：如果系统在更新传播到磁盘之前崩溃，这些更新就会丢失；然而，把写在内存中多留一段时间，就能批量处理、调度，甚至干脆省掉一些写，从而提高性能。

有些应用程序（比如数据库）并不喜欢这种权衡。因此，为了避免写缓冲（write buffering）导致意外的数据丢失，它们干脆强制把数据写到磁盘上：调用 `fsync()`，使用绕过缓存的**直接 I/O**（direct I/O）接口，或者使用**裸设备**（raw disk）接口、完全绕开文件系统[3]。虽然大多数应用程序都接受文件系统所做的权衡，但如果默认行为不能令人满意，也有足够的控制手段让系统按你的意愿行事。

[3]去上一门数据库课，就能更多地了解老派的数据库，以及它们过去坚持绕开操作系统、一切都由自己掌控的做法。不过要小心！搞数据库的那帮人总想说操作系统的坏话。真丢人，搞数据库的人。真丢人。

提示：理解持久性与性能的权衡

存储系统常常让用户面对持久性（durability）与性能之间的权衡。如果用户希望写入的数据立即持久，系统就必须费尽力气把新写入的数据提交到磁盘，因此写操作很慢（但安全）。然而，如果用户能容忍丢失一点数据，系统就可以把写操作在内存中缓冲一段时间，稍后再（在后台）写入磁盘。这样做会让写操作看起来很快就完成了，从而提高了用户感受到的性能；不过，一旦发生崩溃，还没提交到磁盘的写就会丢失，这就是其中的权衡。要正确地做出这种权衡，最好先弄清使用这个存储系统的应用程序需要什么；例如，丢失浏览器最后下载的几张图片也许还能容忍，但丢失一笔往你银行账户里存钱的数据库事务（transaction）的一部分，可能就不那么能容忍了。当然，除非你很有钱；那样的话，你干吗还这么在乎攒下每一分钱呢?

## 40.8 小结

我们已经看到了构建文件系统所需的基本机制。每个文件都需要一些信息，即元数据，通常存放在一种叫作 inode 的结构中。目录只是一种特殊类型的文件，存放着从名字到 inode 号的映射。还需要其他结构；例如，文件系统通常使用位图之类的结构来记录哪些 inode 或数据块是空闲的、哪些已分配。

文件系统设计最棒的一点是它的自由；我们在后面几章中探讨的文件系统，都利用这种自由优化了文件系统的某个方面。显然，还有许多策略上的决策我们没有探讨。例如，创建一个新文件时，应该把它放在磁盘的什么位置?这个策略以及其他一些策略也将是后面几章的主题。真的会吗？[4]

[4]此处响起神秘的音乐，让你对文件系统这个话题更加好奇。

 © 2008–26, ARPACI-DUSSEAU

## 参考文献

[A+07] "A Five-Year Study of File-System Metadata" by Nitin Agrawal, William J. Bolosky, John R. Douceur, Jacob R. Lorch. FAST '07, San Jose, California, February 2007. *An excellent recent analysis of how file systems are actually used. Use the bibliography within to follow the trail of file-system analysis papers back to the early 1980s.*

[B07] "ZFS: The Last Word in File Systems" by Jeff Bonwick and Bill Moore. Available from: http://www.ostep.org/Citations/zfs_last.pdf. *One of the most recent important file systems, full of features and awesomeness. We should have a chapter on it, and perhaps soon will.*

[B02] "The FAT File System" by Andries Brouwer. September, 2002. Available online at: http://www.win.tue.nl/%7eaeb/linux/fs/fat/fat.html. *A nice clean description of FAT. The file system kind, not the bacon kind. Though you have to admit, bacon fat probably tastes better.*

[C94] "Inside the Windows NT File System" by Helen Custer. Microsoft Press, 1994. *A short book about NTFS; there are probably ones with more technical details elsewhere.*

[H+88] "Scale and Performance in a Distributed File System" by John H. Howard, Michael L. Kazar, Sherri G. Menees, David A. Nichols, M. Satyanarayanan, Robert N. Sidebotham, Michael J. West.. ACM TOCS, Volume 6:1, February 1988. *A classic distributed file system; we'll be learning more about it later, don't worry.*

[P09] "The Second Extended File System: Internal Layout" by Dave Poirier. 2009. Available: http://www.nongnu.org/ext2-doc/ext2.html. *Some details on ext2, a very simple Linux file system based on FFS, the Berkeley Fast File System. We'll be reading about it in the next chapter.*

[RT74] "The UNIX Time-Sharing System" by Dennis M. Ritchie, Ken Thompson. CACM Volume 17:7, 1974. *The original paper about* UNIX. *Read it to see the underpinnings of much of modern operating systems. It's amazing how many things they simply got right!*

[S00] "UBC: An Efficient Unified I/O and Memory Caching Subsystem for NetBSD" by Chuck Silvers. FREENIX, 2000. *A nice paper about NetBSD's integration of file-system buffer caching and the virtual-memory page cache. Many other systems do the same type of thing.*

[S+96] "Scalability in the XFS File System" by Adan Sweeney, Doug Doucette, Wei Hu, Curtis Anderson, Mike Nishimoto, Geoff Peck. USENIX '96, January 1996, San Diego, California. *The first attempt to make scalability of operations, including things like having millions of files in a directory, a central focus. A great example of pushing an idea to the extreme. The key idea behind this file system: everything is a tree. We should have a chapter on this file system too.*

## 作业（模拟）

用 `vsfs.py` 这个工具研究文件系统的状态如何随各种操作而变化。文件系统一开始处于空的状态，只有一个根目录。随着模拟的进行，会执行各种操作，从而慢慢改变文件系统在磁盘上的状态。详见 README。

## 问题

1. 用几个不同的随机种子（比如 17、18、19、20）运行模拟器，看看你能不能推断出每两次状态变化之间一定发生了哪些操作。
2. 现在用不同的随机种子（比如 21、22、23、24）做同样的事，只是加上 `-r` 选项运行，这样会显示出操作，让你去推测状态的变化。关于 inode 和数据块的分配算法，你能得出什么结论？它们倾向于分配哪些块？
3. 现在把文件系统中的数据块数量减到很少（比如两个），运行模拟器，处理一百个左右的请求。在这种限制非常严格的布局下，文件系统里最终会有哪些类型的文件？哪些类型的操作会失败？
4. 现在对 inode 做同样的事。inode 很少时，哪些类型的操作能成功？哪些通常会失败？文件系统最终的状态可能是什么样子？

 © 2008–26, ARPACI-DUSSEAU

# 41

# 局部性和快速文件系统

UNIX 操作系统刚问世时，UNIX 大师 Ken Thompson 亲自编写了第一个文件系统（file system）。我们把它叫作“老 UNIX 文件系统”，它真的很简单。它的数据结构（data structure）在磁盘上基本是这个样子：




超级块（superblock，S）包含整个文件系统的信息：卷（volume）有多大、有多少个 inode、指向空闲块链表（free list）表头的指针，等等。磁盘上的 inode 区域存放着文件系统的全部 inode。最后，磁盘的大部分空间都是数据块（data block）。

老文件系统的好处是简单，而且支持文件系统要提供的基本抽象（abstraction）：文件和目录层次结构（directory hierarchy）。这个易用的系统比以往笨拙的、基于记录的存储系统前进了一大步，而目录层次结构（directory hierarchy）相比早期系统那种较简单的单层结构，也是一个真正的进步。

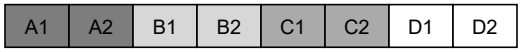
## 41.1 问题：性能很差

问题在于：性能糟透了。按照伯克利的 Kirk McKusick 和同事们的测量 [MJLF84]，性能一开始就不好，而且随着时间推移越来越差，最后文件系统只发挥出整个磁盘带宽（bandwidth）的 2%！

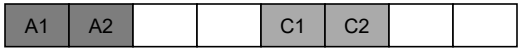
症结在于，老 UNIX 文件系统把磁盘当成了随机存取存储器（random-access memory）来对待；数据随处乱放，完全不顾存放它们的是磁盘，而磁盘有实实在在、代价高昂的定位开销。例如，文件的数据块常常离它的 inode 很远，于是每次先读 inode、再读文件的数据块（这是相当常见的操作）时，都要付出一次代价高昂的寻道（seek）。

更糟的是，由于空闲空间（free space）没有得到精心管理，文件系统最终会变得相当**碎片化**（fragmented）。空闲链表（free list）到后来会指向散布在磁盘各处的一堆块（block），而文件分配时只是简单地取下一个空闲块（free block）。结果，访问一个逻辑上连续的文件时，要在磁盘上来回移动，性能因此大幅下降。

例如，设想下面这个数据块区域，里面有四个文件（A、B、C 和 D），每个文件 2 个块：




如果删除 B 和 D，得到的布局是：




可以看到，空闲空间被分成了两段，每段两个块，而不是连续的一整段四个块。假设现在要分配一个大小为四个块的文件 E：




你可以看到发生了什么：E 被分散在磁盘上，因此访问 E 时，无法从磁盘获得峰值（顺序）性能。你得先读 E1 和 E2，然后寻道，再读 E3 和 E4。这种碎片问题在老 UNIX 文件系统中一直存在，而且损害了性能。顺便说一句：这个问题正是磁盘**碎片整理**（defragmentation）工具要解决的；它们重新组织磁盘上的数据，把文件连续存放，并把空闲空间整理成一个或少数几个连续区域，具体做法是移动数据，再改写 inode 等结构来反映这些变化。

另一个问题是：最初的块大小（block size）太小了（512 字节）。因此，从磁盘传输数据天生就效率低下。小块的好处是能把**内部碎片**（internal fragmentation，块内的浪费）降到最低，但不利于传输，因为每个块都可能需要一次定位开销才能访问到。于是，问题来了：

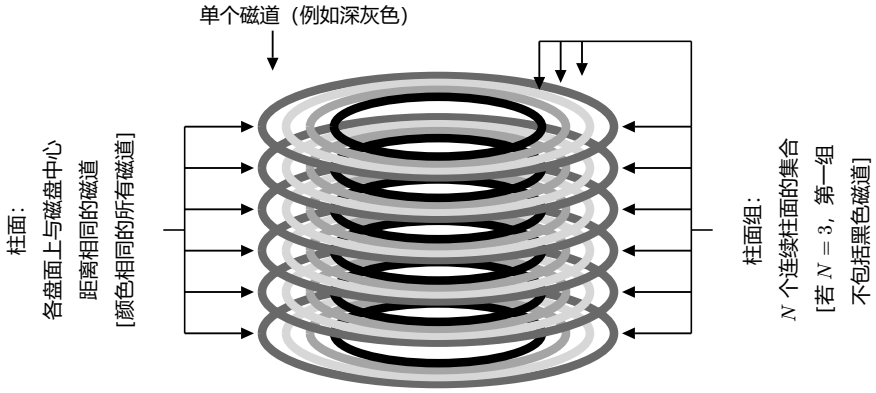
关键问题：
如何组织磁盘上的数据以提高性能
如何组织文件系统的数据结构来提高性能？在这些数据结构之上，需要什么样的分配策略？如何让文件系统“了解磁盘”（disk aware）？

## 41.2 FFS：解决方案是了解磁盘

伯克利的一个小组决定构建一个更好、更快的文件系统，他们巧妙地把它命名为**快速文件系统**（Fast File System，FFS）。他们的想法是让文件系统的结构和分配策略“了解磁盘”，以提高性能，他们也正是这么做的。FFS 由此开启了文件系统研究的新时代：作者保留文件系统原有的接口（同样的 API，包括 `open()`、`read()`、`write()`、`close()` 和其他文件系统调用），只改变内部实现，从而为构建新的文件系统铺平了道路，这方面的工作一直延续到今天。几乎所有现代文件系统都遵循现有的接口（以保持与应用程序兼容），同时出于性能、可靠性（reliability）或其他原因改变内部实现。

## 41.3 组织结构：柱面组

第一步是改变磁盘上的结构。FFS 把磁盘划分成若干个**柱面组**（cylinder group）。一个**柱面**（cylinder）是硬盘不同盘面（surface）上、与磁盘中心距离相同的一组磁道（track）；之所以叫柱面（cylinder），是因为它明显很像几何上的圆柱体。FFS 把 $N$ 个连续的柱面（cylinder）合成一组，于是整个磁盘就可以看成由许多柱面组（cylinder group）构成的集合。下面是一个简单的例子，展示了一个有六个盘片（platter）的驱动器最外侧的四个磁道，以及一个由三个柱面组成的柱面组（cylinder group）：




注意，现代驱动器并没有向外提供足够的信息，让文件系统真正了解某个柱面（cylinder）是否正在使用；我们之前讨论过 [AD18a]，磁盘向外提供的是由块组成的逻辑地址空间（logical address space），并向使用者隐藏了它的几何细节。因此，现代文件系统，比如 Linux 的 ext2、ext3 和 ext4，改为把驱动器组织成**块组**（block group），每个块组只是磁盘地址空间（address space）中连续的一段。下图举了一个例子，每 8 个块组成一个块组，而真实的块组包含的块要多得多：




不管叫柱面组（cylinder group）还是块组，这些组都是 FFS 用来提高性能的核心机制。关键在于，把两个文件放在同一个组里，FFS 就能确保先后访问它们时，不会在磁盘上做长距离的寻道。

要用这些组来存放文件和目录（directory），FFS 就得能把它们放进某个组，并在组里记下它们的所有必要信息。为此，FFS 在每个组中都包含了文件系统应有的全部结构，比如 inode 的空间、数据块，以及一些用来记录它们是已分配还是空闲的结构。下面描绘了 FFS 在单个柱面组（cylinder group）中保存的内容：




现在我们来更详细地看看单个柱面组（cylinder group）的各个组成部分。出于可靠性的考虑，FFS 在每个组中都保存一份**超级块**（super block，S）的副本。挂载（mount）文件系统需要超级块；保存多个副本的话，即使有一份损坏了，仍然可以用一份完好的副本来挂载和访问文件系统。

在每个组中，FFS 需要记录这个组的 inode 和数据块是否已分配。每个组都有一个**inode 位图**（inode bitmap，ib）和一个**数据位图**（data bitmap，db），分别为组内的 inode

补充：FFS 中的文件创建

举个例子，想一想创建一个文件时必须更新哪些数据结构；在这个例子中，假设用户创建了一个新文件 `/foo/bar.txt`，文件长度为一个块（4 KB）。这个文件是新的，因此需要一个新的 inode；于是，inode 位图和新分配的 inode 都要写入磁盘。这个文件里还有数据，所以也要为它分配数据块；于是，数据位图（data bitmap）和一个数据块（最终）也要写入磁盘。因此，当前柱面组（cylinder group）至少要发生四次写（回想一下，这些写在真正执行之前，可能会先在内存中缓冲一段时间）。但这还没完！创建新文件时，还必须把文件放进文件系统的层次结构中，也就是说，必须更新目录。具体来说，必须更新父目录（parent directory）`foo`，加入 `bar.txt` 的条目；这次更新可能放得进 `foo` 现有的某个数据块，也可能需要分配一个新块，连带更新数据位图（data bitmap）。`foo` 的 inode 也必须更新，既要反映目录的新长度，也要更新时间字段（比如最后修改时间）。总之，仅仅创建一个新文件就要做大量工作！下次你创建文件时，也许应该更心怀感激，至少该惊叹于这一切居然运转得这么好。

和数据块承担这项任务。位图（bitmap）是管理文件系统中空闲空间的绝佳方式，因为很容易找到一大块空闲空间并把它分配给文件，或许还能避免老文件系统中空闲链表的一些碎片问题。

最后，**inode** 区域和**数据块**（data block）区域与之前那个非常简单的文件系统（VSFS）中的一样。和往常一样，每个柱面组（cylinder group）的大部分空间都是数据块。

## 41.4 策略：如何分配文件和目录

有了这样的组结构，FFS 现在要决定如何把文件、目录以及相关的元数据（metadata）放到磁盘上，以提高性能。基本原则很简单：*把相关的东西放在一起*（以及它的推论：*把不相关的东西远远分开*）。

因此，为了遵守这条原则，FFS 必须判断什么是“相关的”，把它们放进同一个块组；反过来，不相关的东西应该放进不同的块组。为此，FFS 使用了几条简单的放置启发式规则。

第一条是目录的放置。FFS 的做法很简单：找一个柱面组（cylinder group），要求其中已分配的目录较少（以便在各组之间平衡目录）、空闲 inode 较多（以便之后能分配一批文件），然后把目录的数据和 inode 放在这个组里。当然，这里也可以用别的启发式规则（例如把空闲数据块的数量也考虑进去）。

对于文件，FFS 做两件事。第一，它确保（一般情况下）把文件的数据块和它的 inode 分配在同一个组里，从而避免像老文件系统那样，在 inode 和数据之间做长距离寻道。第二，它把同一目录下的所有文件，都放在它们所在目录的那个柱面组（cylinder group）里。因此，如果用户创建了四个文件 `/a/b`、`/a/c`、`/a/d` 和 `/b/f`，FFS 会尽量把前三个放得彼此靠近（在同一个组），而把第四个放得远一些（在另外某个组）。

我们来看一个这样分配的例子。假设每个组只有 10 个 inode 和 10 个数据块（两个数字都小得不现实），按照 FFS 的策略，在这些组中放置三个目录，即根目录（root directory）`/`、`/a` 和 `/b`，以及四个文件（`/a/c`、`/a/d`、`/a/e`、`/b/f`）。假设每个普通文件都是两个块

大小，目录只有一个数据块。在这个图中，我们用一目了然的符号表示每个文件或目录（即用 / 表示根目录，a 表示 /a，f 表示 /b/f，依此类推）。

```
group inodes      data
    0 /---------  /---------
    1 acde------  accddee---
    2 bf--------  bff-------
    3 ----------  ----------
    4 ----------  ----------
    5 ----------  ----------
    6 ----------  ----------
    7 ----------  ----------
```

注意，FFS 的策略带来了两个好处：每个文件的数据块都靠近这个文件的 inode，同一目录下的文件也彼此靠近（即 /a/c、/a/d 和 /a/e 都在组 1 中，目录 /b 和它的文件 /b/f 在组 2 中彼此靠近）。

作为对比，我们再来看一种 inode 分配策略，它只是把 inode 分散到各组，尽量保证没有哪个组的 inode 表（inode table）很快被填满。最终的分配结果可能是这样的：

```
group inodes      data
    0 /---------  /---------
    1 a---------  a---------
    2 b---------  b---------
    3 c---------  cc--------
    4 d---------  dd--------
    5 e---------  ee--------
    6 f---------  ff--------
    7 ----------  ----------
```

从图中可以看出，这种策略虽然确实让文件和目录的数据靠近各自的 inode，但同一目录下的文件被随意地分散在磁盘各处，因此没有保住基于名字的局部性（locality）。现在访问文件 /a/c、/a/d 和 /a/e 要跨越三个组，而按照 FFS 的方法只需要一个组。

FFS 的这些启发式规则并非来自对文件系统流量的大量研究，也没有什么特别精妙之处；它们靠的是老派的**常识**（common sense，CS 不就是这个意思吗?）[1]。一个目录中的文件往往会被一起访问：想象一下编译一批文件，然后把它们链接成一个可执行文件。正因为存在这种基于命名空间（name space）的局部性，FFS 往往能提高性能，确保相关文件之间的寻道又近又短。

## 41.5 测量文件的局部性

为了更好地理解这些启发式规则是否合理，我们来分析一些文件系统访问的跟踪记录，看看是否真的存在命名空间局部性（namespace locality）。不知为什么，文献中似乎没有关于这个话题的好研究。

[1] 有些人把常识叫作 **horse sense**，尤其是经常和马打交道的人。不过我们觉得，随着“机械马”（也就是汽车）越来越普及，这个说法可能会失传。他们下一步还会发明什么？会飞的机器??!!

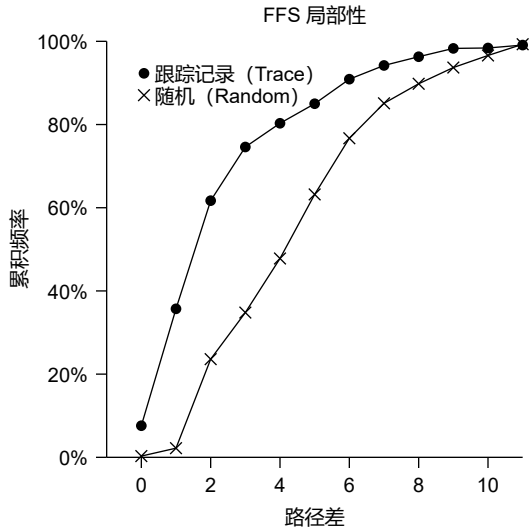



图 41.1：**SEER 跟踪记录中的 FFS 局部性**

具体来说，我们将使用 SEER 跟踪记录 [K94]，分析文件访问在目录树（directory tree）中彼此“相距多远”。例如，如果打开了文件 `f`，然后在跟踪记录中紧接着又打开了它（在打开任何其他文件之前），那么这两次打开在目录树中的距离就是零（因为它们是同一个文件）。如果打开了目录 `dir` 中的文件 `f`（即 `dir/f`），接着打开了同一目录中的文件 `g`（即 `dir/g`），那么这两次文件访问的距离就是一，因为它们在同一个目录下，但不是同一个文件。换句话说，这个距离指标衡量的是：要找到两个文件的共同祖先，需要在目录树中往上走多远；它们在树中离得越近，这个度量值就越小。

图 41.1 是在 SEER 集群所有工作站的全部跟踪记录中观察到的局部性。图中横轴是差值度量，纵轴是差值为该值的文件打开操作所占的累积百分比。具体来说，从 SEER 跟踪记录（图中标为“Trace”）可以看到，约 7% 的文件访问是访问上一次打开的那个文件，而将近 40% 的文件访问要么是同一个文件，要么是同一目录下的文件（即差值为零或一）。因此，FFS 的局部性假设看来是合理的（至少对这些跟踪记录而言）。

有意思的是，另外还有约 25% 的文件访问，访问的是距离为二的文件。如果用户把一组相关的目录按多级结构组织起来，并经常在它们之间来回切换，就会出现这种局部性。例如，如果用户有一个 `src` 目录，并把目标文件（`.o` 文件）构建到 `obj` 目录中，而这两个目录都是主目录 `proj` 的子目录（subdirectory），那么常见的访问模式就是先访问 `proj/src/foo.c`，接着访问 `proj/obj/foo.o`。这两次访问的距离是二，因为 `proj` 是它们的共同祖先。FFS 的策略没有捕捉到这种局部性，因此这类访问之间会有更多的寻道。

作为对比，图中还给出了一个“随机”（Random）跟踪记录的局部性。随机跟踪记录的生成方法是：从现有的 SEER 跟踪记录中按随机顺序选取文件，再计算这些随机顺序的访问之间的距离度量。可以看到，和预期的一样，随机跟踪记录中的命名空间局部性（namespace

locality）更少。不过，所有文件最终都有一个共同祖先（比如根目录），所以还是有一些局部性。因此，随机跟踪记录可以作为一个有用的比较基准。

## 41.6 大文件的例外规则

在 FFS 中，文件放置的一般策略有一个重要的例外，它针对的是大文件。如果没有一条不同的规则，一个大文件就会把它最先放入的那个块组完全填满，也许还会填满其他块组。这样填满一个块组是不可取的，因为它会妨碍随后的“相关”文件放进这个块组，从而可能损害文件访问的局部性。

因此，对于大文件，FFS 是这样做的：在第一个块组中分配了一定数量的块之后（例如 12 个块，也就是一个 inode 中直接指针的数量），FFS 把文件的下一个“大”块（例如第一个间接块所指向的那些块）放到另一个块组中，可能会选一个利用率（utilization）较低的组。然后，文件的再下一个大块（chunk）又放到另一个块组中，依此类推。

我们来看几张图，更好地理解这条策略。如果没有这条大文件例外规则，一个大文件会把它的所有块都放在磁盘的同一个区域。举个小例子：在一个配置为每组 10 个 inode、40 个数据块的 FFS 中，有一个 30 个块的文件（`/a`）。下面是没有大文件例外规则时 FFS 的情形：

```
group inodes      data
    0 /a--------  /aaaaaaaaa aaaaaaaaaa aaaaaaaaaa a---------
    1 ----------  ---------- ---------- ---------- ----------
    2 ----------  ---------- ---------- ---------- ----------
```

从图中可以看到，`/a` 占满了组 0 的大部分数据块，而其他组仍然是空的。如果现在在根目录（`/`）中再创建一些别的文件，组里就没有多少空间存放它们的数据了。

有了大文件例外规则（这里设为每个大块五个块），FFS 就会把这个文件分散到各个组中，这样任何一个组的利用率都不会太高：

```
group inodes      data
    0 /a--------  /aaaaa---- ---------- ---------- ----------
    1 ----------  aaaaa----- ---------- ---------- ----------
    2 ----------  aaaaa----- ---------- ---------- ----------
    3 ----------  aaaaa----- ---------- ---------- ----------
    4 ----------  aaaaa----- ---------- ---------- ----------
    5 ----------  aaaaa----- ---------- ---------- ----------
    6 ----------  ---------- ---------- ---------- ----------
```

精明的读者（就是你）会注意到，把一个文件的块分散到磁盘各处会损害性能，尤其是在顺序访问文件这种比较常见的情况下（例如用户或应用程序按顺序读取第 0 块到第 29 块）。你说得没错，我们精明的读者！不过，只要仔细选择大块的大小，就能解决这个问题。

具体来说，如果大块足够大，文件系统的大部分时间就会花在从磁盘传输数据上，而在大块之间寻道只花（相对）很少的时间。像这样每付出一次开销就多做一些工作、以此降低开销的做法，叫作**平摊**（amortization），这是计算机系统中的一种常用技术。

我们来看一个例子：假设磁盘的平均定位时间，即寻道加旋转的时间，是 10 ms，再假设磁盘以 40 MB/s 的速率传输数据。如果你的目标是一半时间用来在大块之间寻道，一半

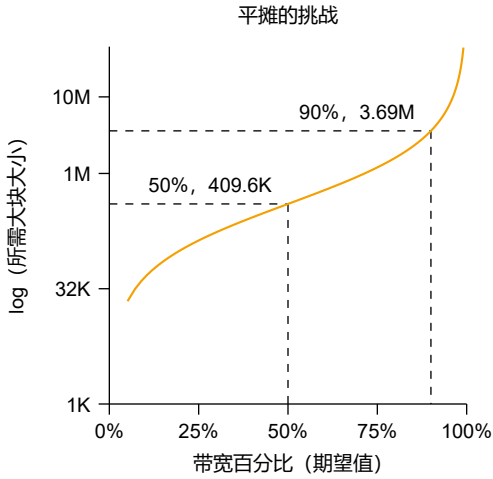



图 41.2：**平摊：大块要多大才行?**

时间用来传输数据（从而达到磁盘峰值性能的 50%），那么每做 10 ms 的定位，就需要花 10 ms 传输数据。于是问题变成：大块要多大，才能让传输花 10 ms? 很简单，用我们的老朋友数学就行，特别是在磁盘一章中提到的量纲分析（dimensional analysis）[AD18a]：

$$\frac{40\cancel{MB}}{\cancel{秒}} \cdot \frac{1024\ \text{KB}}{1\cancel{MB}} \cdot \frac{1\cancel{秒}}{1000\cancel{ms}} \cdot 10\cancel{ms} = 409.6\ \text{KB} \tag{41.1}$$

这个公式的意思基本上是：如果以 40 MB/s 的速率传输数据，那么每寻道一次，只需传输 409.6 KB，就能做到一半时间寻道、一半时间传输。类似地，你可以算出要达到峰值带宽的 90% 需要多大的大块（结果大约是 3.6 MB），甚至达到峰值带宽的 99% 需要多大的大块（39.6 MB!）。可以看到，越想接近峰值，大块就得越大（这些值的曲线见图 41.2）。

不过，FFS 并没有用这种计算来把大文件分散到各个组中。它采用了一种简单的方法，依据的是 inode 本身的结构：最前面的 12 个直接块（direct block）和 inode 放在同一个组中；之后的每个间接块（indirect block），以及它指向的所有块，都放到不同的组中。块大小为 4 KB、磁盘地址（disk address）为 32 位时，这个策略意味着文件每 1024 个块（4 MB）放在一个单独的组中，唯一的例外是文件开头由直接指针（direct pointer）指向的那 48 KB。

注意，磁盘驱动器（disk drive）的发展趋势是传输速率提高得相当快，因为磁盘制造商擅长在同样的盘面上塞进更多的位；但驱动器与寻道相关的机械部分，即磁盘臂（disk arm）的速度和旋转速率，却提高得相当慢 [P98]。这意味着，随着时间的推移，机械开销会变得相对越来越昂贵，因此，要平摊（amortize）这些开销，就必须在两次寻道之间传输更多的数据。

## 41.7 关于 FFS 的其他几件事

FFS 还引入了其他一些创新。特别是，设计者非常担心如何容纳小文件；当时很多文件其实只有 2 KB 左右，而使用 4 KB 的块虽然有利于传输数据，空间效率却不太好。因此，在一个典型的文件系统中，这种**内部碎片**（internal fragmentation）可能导致大约一半的磁盘空间被浪费。

FFS 设计者想到的办法很简单，而且管用。他们决定引入**子块**（sub-block），也就是文件系统可以分配给文件的 512 字节的小块。这样，如果你创建一个小文件（比如 1 KB 大小），它会占用两个子块（sub-block），从而不会浪费一整个 4 KB 的块。随着文件增长，文件系统会继续给它分配 512 字节的块，直到它攒满 4 KB 的数据。这时，FFS 会找一个 4 KB 的块，把这些子块（sub-block）复制进去，并释放这些子块供以后使用。

你可能会发现，这个过程效率不高，需要文件系统做大量额外的工作（特别是，要做大量额外的 I/O 来执行复制）。你又说对了！因此，FFS 一般靠修改 `libc` 库来避免这种最坏的情况：库会缓冲写操作，然后以 4 KB 的块为单位把它们交给文件系统，这样在大多数情况下就完全不必用到子块（sub-block）这个特殊机制了。

FFS 引入的第二个巧妙之处，是一种为性能而优化的磁盘布局。在那个年代（SCSI 和其他更现代的设备接口出现之前），磁盘要简单得多，需要主机 CPU 更直接地控制它们的操作。文件放在磁盘的连续扇区（sector）上时（如图 41.3 左边所示），FFS 就会遇到问题。

具体来说，问题出现在顺序读取的时候。FFS 会先发出读块 0 的请求；等这次读完成、FFS 再发出读块 1 的请求时，已经太晚了：块 1 已经转过了磁头（disk head），现在读块 1 就要等上整整一圈。

FFS 用一种不同的布局解决了这个问题，如图 41.3 右边所示。（在这个例子中）每隔一个块放一个块，FFS 就有足够的时间在下一个块转过磁头之前发出请求。而且，FFS 足够聪明，能针对某个具体的磁盘算出布局时应该跳过多少个块，才能避免额外的旋转；这种技术叫作**参数化**（parameterization），因为 FFS 会测出磁盘具体的性能参数，并用它们来决定确切的交错布局方案。

你可能会想：这个方案说到底也没那么好。确实如此：用这种布局只能得到峰值带宽的 50%，因为你必须绕每个磁道转两圈，才能把每个块各读一次。幸好，现代磁盘要聪明得多：它们会在内部把整个磁道读进来，缓冲在磁盘内部的缓存（cache）里（正因如此，这个缓存常叫作**磁道缓冲区**，track buffer）。之后再读这个磁道时，磁盘只需从缓存中返回所需的

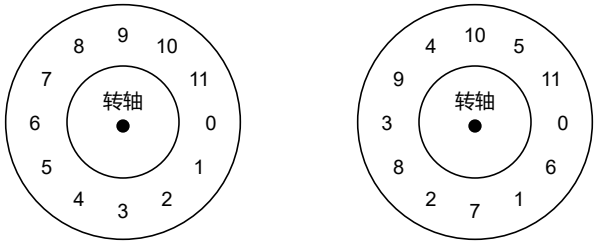



图 41.3：**FFS：标准放置与参数化放置**

 © 2008–26, ARPACI-DUSSEAU

提示：让系统好用

FFS 带给我们的最基本的一课，大概是：它不仅引入了“了解磁盘”的布局这个在概念上很好的想法，还加入了一些单纯让系统更好用的功能。长文件名（long file name）、符号链接（symbolic link），以及原子的重命名操作，都提高了系统的实用性；虽然很难就它们写一篇研究论文（想象一下读一篇 14 页的论文，题目叫《符号链接：硬链接失散多年的表亲》），但正是这些小功能让 FFS 更有用，因此很可能也让它更容易被人们采用。让系统好用，往往和系统深层的技术创新一样重要，甚至更重要。

数据。因此，文件系统再也不用操心这些极其底层的细节了。只要设计得当，抽象和更高层的接口会是好东西。

FFS 还做了其他一些易用性方面的改进。它是最早支持**长文件名**（long file name）的文件系统之一，从而让文件系统中可以使用更有表达力的名字，而不再是传统的固定长度的做法（比如 8 个字符）。此外，它还引入了一个新概念，叫作**符号链接**（symbolic link）。正如前一章 [AD18b] 讨论过的，硬链接（hard link）有局限：它们既不能指向目录（因为担心在文件系统层次结构中引入环路），也只能指向同一个卷中的文件，这样 inode 号（inode number）才仍然有意义。符号链接让用户可以为系统中的任何其他文件或目录创建一个“别名”，因此要灵活得多。FFS 还引入了用于重命名文件的原子 `rename()` 操作。在基础技术之外的这些易用性改进，很可能也为 FFS 赢得了更广泛的用户基础。

## 41.8 小结

FFS 的问世是文件系统历史上的一个分水岭，它让人们清楚地认识到，文件管理问题是操作系统中最有意思的问题之一，并展示了如何着手对付最重要的设备：硬盘。从那以后，人们开发了数以百计的新文件系统，但直到今天，许多文件系统仍然从 FFS 身上汲取灵感（例如，Linux 的 ext2 和 ext3 显然就是它在思想上的后代）。当然，所有现代系统都吸取了 FFS 的主要教训：把磁盘当磁盘来对待。

 © 2008–26, ARPACI-DUSSEAU

## 参考文献

[AD18a] "Operating Systems: Three Easy Pieces" (Chapter: Hard Disk Drives) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. Arpaci-Dusseau Books, 2018. *There is no way you should be reading about FFS without having first understood hard drives in some detail. If you try to do so, please instead go directly to jail; do not pass go, and, critically, do not collect 200 much-needed simoleons.*

[AD18b] "Operating Systems: Three Easy Pieces" (Chapter: File System Implementation) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau . Arpaci-Dusseau Books, 2018. *As above, it makes little sense to read this chapter unless you have read (and understood) the chapter on file system implementation. Otherwise, we'll be throwing around terms like "inode" and "indirect block" and you'll be like "huh?" and that is no fun for either of us.*

[K94] "The Design of the SEER Predictive Caching System" by G. H. Kuenning. MOBICOMM '94, Santa Cruz, California, December 1994. *According to Kuenning, this is the best overview of the SEER project, which led to (among other things) the collection of these traces.*

[MJLF84] "A Fast File System for UNIX" by Marshall K. McKusick, William N. Joy, Sam J. Leffler, Robert S. Fabry. ACM TOCS, 2:3, August 1984. *McKusick was recently honored with the IEEE Reynold B. Johnson award for his contributions to file systems, much of which was based on his work building FFS. In his acceptance speech, he discussed the original FFS software: only 1200 lines of code! Modern versions are a little more complex, e.g., the BSD FFS descendant now is in the 50-thousand lines-of-code range.*

[P98] "Hardware Technology Trends and Database Opportunities" by David A. Patterson. Keynote Lecture at SIGMOD '98, June 1998. *A great and simple overview of disk technology trends and how they change over time.*

 © 2008–26, ARPACI-DUSSEAU

## 作业（模拟）

这一节介绍 `ffs.py`，一个简单的 FFS 模拟器，你可以用它更好地理解基于 FFS 的文件和目录分配是如何工作的。模拟器的运行方法详见 README。

## 问题

1. 查看文件 `in.largefile`，然后用选项 `-f in.largefile` 和 `-L 4` 运行模拟器。后一个选项把大文件例外规则设为 4 个块。最终的分配结果会是什么样子？用 `-c` 运行来检查一下。
2. 现在用 `-L 30` 运行。你预计会看到什么？同样，打开 `-c` 看看你猜得对不对。你还可以用 `-S` 准确地看到哪些块分配给了文件 `/a`。
3. 现在我们来计算这个文件的一些统计量。第一个我们称之为 *filespan*，它是文件的任意两个数据块之间、或者 inode 与任意一个数据块之间的最大距离。计算 `/a` 的 filespan。运行 `ffs.py -f in.largefile -L 4 -T -c` 看看它是多少。再用 `-L 100` 做同样的事。随着大文件例外参数从小值变到大值，你预计 filespan 会有什么不同？
4. 现在我们来看一个新的输入文件 `in.manyfiles`。你认为 FFS 的策略会如何把这些文件分布到各个组中？（可以用 `-v` 运行，看看创建了哪些文件和目录，或者直接 `cat in.manyfiles`。）用 `-c` 运行模拟器，看看你猜得对不对。
5. 评价 FFS 的一个指标叫作 *dirspan*。这个指标计算的是某个目录中文件的分散程度，具体来说，是该目录中所有文件的 inode 和数据块，与目录本身的 inode 和数据块之间的最大距离。用 `in.manyfiles` 和 `-T` 选项运行，计算三个目录的 dirspan。用 `-c` 运行来检查。FFS 在减小 dirspan 方面做得怎么样？
6. 现在把每个组的 inode 表大小改成 5（`-i 5`）。你认为这会如何改变文件的布局？用 `-c` 运行，看看你猜得对不对。它对 dirspan 有什么影响？
7. FFS 应该把新目录的 inode 放在哪个组中？默认的（模拟器）策略会找空闲 inode 最多的那个组。另一种策略会找空闲 inode 最多的若干个组。例如，如果用 `-A 2` 运行，分配新目录时，模拟器会把组两两成对地看，挑出最适合分配的那一对。运行 `./ffs.py -f in.manyfiles -i 5 -A 2 -c`，看看用这种策略时分配结果如何变化。它对 dirspan 有什么影响？为什么这种策略可能是好的？
8. 最后一个要研究的策略变化与文件碎片有关。运行 `./ffs.py -f in.fragmented -v`，看看你能否预测剩下的那些文件是如何分配的。用 `-c` 运行来确认你的答案。文件 `/i` 的数据布局有什么有意思的地方？为什么说它有问题？
9. 一种新策略，我们称之为*连续分配*（`-C`），它尽量保证每个文件都连续分配。具体来说，用 `-C n` 时，文件系统在分配一个块之前，会先确保组内有 `n` 个连续的空闲块。运行 `./ffs.py -f in.fragmented -v -C 2 -c` 看看有什么不同。随着传给 `-C` 的参数增大，布局会怎样变化？最后，`-C` 对 filespan 和 dirspan 有什么影响？

# 42

# 崩溃一致性：FSCK 和日志

我们已经看到，文件系统（file system）管理着一组数据结构（data structure），用来实现人们所期望的那些抽象（abstraction）：文件、目录（directory），以及支撑文件系统基本抽象所需的其他所有元数据（metadata）。文件系统的数据结构必须**持久**（persist），也就是要长期保存下来，存放在断电（power loss）后也不会丢失数据的设备上，比如硬盘或基于闪存（flash）的 SSD；这一点和大多数数据结构不同，比如运行中的程序（running program）放在内存里的那些。

文件系统面临的一大挑战是：在可能发生**断电**（power loss）或**系统崩溃**（system crash）的情况下，如何更新持久的数据结构。具体来说，如果磁盘上的结构正更新到一半，有人被电源线绊了一下，机器断电了，会怎样？或者操作系统遇到 bug 崩溃了，又会怎样？因为随时可能断电或崩溃，更新持久数据结构相当棘手，这就引出了文件系统实现中一个有趣的新问题，叫作**崩溃一致性问题**（crash-consistency problem）。

这个问题很容易理解。假设为了完成某个操作，你必须更新磁盘上的两个结构 A 和 B。由于磁盘一次只处理一个请求，两个请求中总有一个先到达磁盘（A 或 B）。如果系统在其中一个写入完成后崩溃或断电，磁盘上的结构就会处于**不一致**（inconsistent）的状态。于是，我们遇到了一个所有文件系统都必须解决的问题：

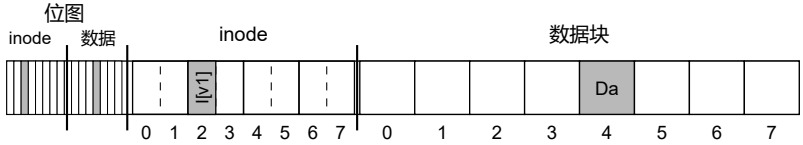
> 关键问题：如何在可能崩溃的情况下更新磁盘
> 系统可能在任意两次写入之间崩溃或断电，因此磁盘上的状态可能只更新了一部分。崩溃之后，系统重新启动，并希望再次挂载（mount）文件系统（以便访问文件等）。既然崩溃可能发生在任意时刻，我们如何确保文件系统在磁盘上的映像始终处于合理的状态？

在本章中，我们会更详细地描述这个问题，并看看文件系统用过哪些方法来解决它。我们先研究较老的文件系统采用的方法，叫作 **fsck**，即**文件系统检查程序**（file system checker）。然后我们转向另一种方法，叫作**日志**（journaling），也叫**预写日志**（write-ahead logging）。这种技术会给每次写入增加一点开销，但能更快地从崩溃或断电中恢复。我们会讨论日志的基本机制，包括比较现代的日志文件系统（journaling file system）Linux ext3 [T98, PAA05] 所实现的几种不同的日志方式。

## 42.1 一个详细的例子

为了开始研究日志，我们先来看一个例子。我们需要一个会更新磁盘上某些结构的**工作负载**（workload）。这里假设工作负载很简单：往一个已有的文件末尾追加一个数据块（data block）。追加的过程是：打开文件，调用 `lseek()` 把文件偏移量（file offset）移到文件末尾，然后向文件发出一次 4 KB 的写入，最后关闭文件。

我们再假设磁盘上使用的是标准的简单文件系统结构，和我们之前见过的文件系统类似。这个小例子包括一个 **inode 位图**（inode bitmap，只有 8 位，每个 inode 一位）、一个**数据位图**（data bitmap，也是 8 位，每个数据块一位）、inode（共 8 个，编号 0 到 7，分布在四个块中），以及数据块（共 8 个，编号 0 到 7）。下面是这个文件系统的示意图：




从图中可以看出，系统分配了一个 inode，它的 inode 号（inode number）是 2，并在 inode 位图中做了标记；还分配了一个数据块（数据块 4），同样在数据位图（data bitmap）中做了标记。这个 inode 记作 I[v1]，因为它是这个 inode 的第一个版本；上面那个工作负载很快就会更新它。

我们也来看看这个简化的 inode 内部。I[v1] 里面是这样的：

```
owner       : remzi
permissions : read-write
size        : 1
pointer     : 4
pointer     : null
pointer     : null
pointer     : null
```

在这个简化的 inode 中，文件的 `size` 是 1（分配了一个块），第一个直接指针（direct pointer）指向块 `4`（文件的第一个数据块 Da），另外三个直接指针都设为 `null`（表示没有使用）。当然，真实的 inode 有多得多的字段；更多信息请参阅前面几章。

往文件里追加数据，就是给它加一个新的数据块，因此必须更新磁盘上的三个结构：inode（它必须指向新块，并且因为追加而记录新的、更大的文件大小）、新的数据块 Db，以及新版本的数据位图（记作 B[v2]），用来表示新数据块已经分配出去了。

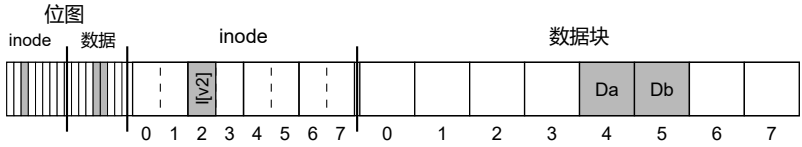
所以，系统内存里有三个块（block）要写到磁盘上。更新后的 inode（inode 版本 2，简称 I[v2]）现在是这样的：

```
owner       : remzi
permissions : read-write
size        : 2
pointer     : 4
pointer     : 5
```

```
pointer      : null
pointer      : null
```

更新后的数据位图（data bitmap）B[v2] 现在是这样的：00001100。最后是数据块（Db），里面装的就是用户放进文件里的任何东西。也许是偷来的音乐?

我们希望文件系统最终在磁盘上的映像是这样的：




为了完成这一转变，文件系统必须对磁盘执行三次独立的写入，inode（I[v2]）、位图（B[v2]）和数据块（Db）各一次。注意，用户发出 `write()` 系统调用（system call）时，这些写入通常不会立即发生；脏的 inode、位图（bitmap）和新数据会先在内存（**页缓存**，page cache，或**缓冲区缓存**，buffer cache）中待上一段时间；然后，等文件系统最终决定把它们写到磁盘时（比如 5 秒或 30 秒之后），才会向磁盘发出所需的写请求。遗憾的是，崩溃可能会发生，打断这些磁盘更新。具体来说，如果这三次写入只完成了一两次就发生崩溃，文件系统就可能处于一种奇怪的状态。

### 崩溃场景

为了更好地理解这个问题，我们来看几个崩溃场景的例子。假设只有一次写入成功了；那么有三种可能的结果，列举如下：

- **只有数据块（Db）写到了磁盘上**。在这种情况下，数据在磁盘上，但没有 inode 指向它，也没有位图表示这个块已被分配。因此，这就好像那次写入从未发生过一样。从文件系统崩溃一致性（crash consistency）的角度看，这种情况根本不是问题[1]。
- **只有更新后的 inode（I[v2]）写到了磁盘上**。在这种情况下，inode 指向了 Db 本该写入的磁盘地址（5），但 Db 还没有写到那里。因此，如果我们相信这个指针，就会从磁盘上读到**垃圾**（garbage）数据，也就是磁盘地址（disk address）5 原来的内容。此外，我们还遇到了一个新问题，叫作**文件系统不一致**（file-system inconsistency）。磁盘上的位图告诉我们数据块 5 还没有被分配，inode 却说它已经分配了。位图和 inode 之间的这种矛盾，就是文件系统数据结构中的不一致；要使用这个文件系统，我们必须设法解决这个问题（下文会详细讨论）。
- **只有更新后的位图（B[v2]）写到了磁盘上**。在这种情况下，位图表示块 5 已被分配，但没有 inode 指向它。因此，文件系统又一次不一致了；如果不加处理，这次写入会造成 **space leak**（空间泄漏），因为文件系统永远不会再使用块 5。

在这次向磁盘写入三个块的尝试中，还有另外三种崩溃场景。在这些情况下，两次写入成功了，最后一次失败了：

[1]不过，这对用户来说可能是个问题：用户刚刚丢了一些数据！

- **inode（I[v2]）和位图（B[v2]）写到了磁盘上，但数据（Db）没有**。在这种情况下，文件系统的元数据是完全一致的：inode 有一个指向块 5 的指针，位图表示块 5 正在使用，所以从文件系统元数据的角度看，一切正常。但有一个问题：块 5 里又是垃圾。
- **inode（I[v2]）和数据块（Db）写到了磁盘上，但位图（B[v2]）没有**。在这种情况下，inode 指向了磁盘上正确的数据，但 inode 和旧版本的位图（B1）之间又出现了不一致。因此，我们还是得先解决这个问题，才能使用文件系统。
- **位图（B[v2]）和数据块（Db）写到了磁盘上，但 inode（I[v2]）没有**。在这种情况下，inode 和数据位图（data bitmap）之间又出现了不一致。不过，虽然这个块已经写入、位图也表明它在使用中，我们却不知道它属于哪个文件，因为没有任何 inode 指向这个文件。

### 崩溃一致性问题

从这些崩溃场景中，希望你能看出，崩溃会给磁盘上的文件系统映像带来许多问题：文件系统数据结构可能不一致；可能出现 space leak；可能把垃圾数据返回给用户；等等。理想情况下，我们希望**原子地**（atomically）把文件系统从一个一致的状态（例如，文件追加之前）转移到另一个一致的状态（例如，inode、位图和新数据块都写到磁盘之后）。遗憾的是，这一点不容易做到，因为磁盘一次只提交一个写入，而这些更新之间可能发生崩溃或断电。我们把这个一般性的问题称为**崩溃一致性问题**（也可以叫它 **consistent-update problem**）。

## 42.2 方案一：文件系统检查程序

早期的文件系统对崩溃一致性（crash consistency）采取了一种简单的方法。基本上，它们决定先让不一致发生，以后（重启时）再修复。这种“偷懒”做法的一个经典例子，就是做这件事的工具：**fsck**[2]。fsck 是一个 UNIX 工具，用来找出这类不一致并加以修复 [MK96]；不同的系统上也有类似的工具，用来检查和修复磁盘分区。注意，这种方法并不能解决所有问题；比如上面那种情况：文件系统看起来是一致的，但 inode 指向了垃圾数据。它真正的目标只是确保文件系统的元数据在内部是一致的。

正如 McKusick 和 Kowalski 的论文 [MK96] 所总结的，fsck 工具分几个阶段运行。它在文件系统挂载、可供使用之前运行（fsck 假定它运行时没有其他文件系统活动在进行）；运行结束后，磁盘上的文件系统应该是一致的，因此可以开放给用户访问。

下面是 fsck 所做工作的基本概述：

- **超级块：**fsck 首先检查超级块（superblock）看起来是否合理，主要是做一些健全性检查，比如确保文件系统的大小大于已分配的块数。这些健全性检查的目的通常是找出可疑的（损坏的）超级块；在这种情况下，系统（或管理员）可能会决定改用超级块的某个备份副本。
- **空闲块：**接下来，fsck 扫描 inode、间接块（indirect block）、二级间接块（double indirect block）等，从而弄清楚文件系统中当前有哪些块已被分配。它利用这些信息

[2]读作“eff-ess-see-kay”或“eff-ess-check”；如果你不喜欢这个工具，也可以读作“eff-suck”（suck 意为“烂透了”）。没错，严肃的专业人士也用这个说法。

生成一份正确的分配位图；因此，如果位图和 inode 之间有任何不一致，都以 inode 中的信息为准来解决。对所有 inode 也做同样的检查，确保所有看起来正在使用的 inode 都在 inode 位图中做了相应标记。

- **inode 状态：** 检查每个 inode 有没有损坏或其他问题。例如，fsck 会确保每个已分配的 inode 都有一个有效的类型字段（例如普通文件、目录、符号链接（symbolic link）等）。如果 inode 的字段有不易修复的问题，fsck 就会把这个 inode 视为可疑的并清除它；inode 位图也会相应更新。
- **inode 链接：** fsck 还会核对每个已分配 inode 的链接计数（link count）。你可能还记得，链接计数表示有多少个不同的目录包含对这个文件的引用（即链接）。为了核对链接计数，fsck 从根目录（root directory）开始扫描整个目录树（directory tree），为文件系统中的每个文件和目录自己算出一份链接计数。如果新算出的计数和 inode 中记录的不一致，就必须采取纠正措施，通常是修正 inode 中的计数。如果发现了一个已分配的 inode，却没有任何目录引用它，就把它移到 `lost+found` 目录中。
- **重复：** fsck 还会检查重复的指针，即两个不同的 inode 指向同一个块的情况。如果其中一个 inode 明显是坏的，就可以把它清除。另外，也可以把被指向的块复制一份，按需让每个 inode 都有自己的副本。
- **坏块：** 扫描所有指针的列表时，还会检查坏块指针（bad block pointer）。如果一个指针明显指向了有效范围之外的东西，例如它的地址所指的块超出了分区的范围，这个指针就被认为是“坏”的。在这种情况下，fsck 做不了什么太聪明的事；它只是把这个指针从 inode 或间接块（indirect block）中删掉（清除）。
- **目录检查：** fsck 并不理解用户文件的内容；但目录里存放的是由文件系统自己创建的、格式特定的信息。因此，fsck 会对每个目录的内容做额外的完整性（integrity）检查，确保“.”和“..”是最前面的两个目录项（directory entry），确保目录项中引用的每个 inode 都已分配，并确保在整个层次结构中，任何目录都只被链接一次。

可以看出，要构建一个能用的 fsck，需要对文件系统有非常细致的了解；确保这样一段代码在所有情况下都能正确工作，可能相当有挑战性 [G+08]。然而，fsck（以及类似的方法）还有一个更大、或许也更根本的问题：它们太慢了。如果磁盘卷（disk volume）非常大，扫描整个磁盘来找出所有已分配的块、读取整个目录树，可能要花好几分钟甚至几个小时。随着磁盘容量增长、RAID 越来越普及，fsck 的性能变得让人无法接受（尽管最近有一些进展 [M+13]）。

从更高的层面看，fsck 的基本前提似乎有点不合情理。想想上面的例子，只有三个块要写到磁盘上；为了修复仅仅更新三个块时出现的问题，却要扫描整个磁盘，代价高得离谱。这种情况就好比你把钥匙掉在了卧室的地板上，然后启动一个把整栋房子翻个遍找钥匙的恢复算法，从地下室开始，一个房间一个房间地找过去。这样做能找到，但太浪费了。因此，随着磁盘（和 RAID）越来越大，研究人员和从业者开始寻找其他解决方案。

 © 2008–23, ARPACI-DUSSEAU

## 42.3 方案二：日志（或预写日志）

对于 consistent-update problem，最流行的解法大概借鉴自数据库管理系统（database management system）领域的一个思想。这个思想叫作**预写日志**（write-ahead logging），正是为了解决这类问题而发明的。在文件系统中，由于历史原因，我们通常把预写日志（write-ahead logging）叫作**日志**（journaling）。第一个这样做的文件系统是 Cedar [H87]，不过如今许多现代文件系统都用了这个思想，包括 Linux ext3 和 ext4、reiserfs、IBM 的 JFS、SGI 的 XFS 以及 Windows NTFS。

基本思想如下。更新磁盘时，在原地覆盖那些结构之前，先写下一条小小的记录（写在磁盘上的另一个地方，一个大家都知道的位置），描述你接下来要做什么。写这条记录就是“预写”（write ahead）的部分，而我们把它写到一个组织成“日志”（log）的结构里；于是就有了“预写日志”（write-ahead logging）这个名字。

把这条记录写到磁盘上，就能保证：如果在更新（覆盖）结构的过程中发生了崩溃，你可以回过头去看看你写下的记录，然后再试一次；这样一来，崩溃之后你就能确切地知道要修复什么（以及怎么修复），而不必扫描整个磁盘。因此，日志在设计上就是在更新时多做一点工作，来大大减少恢复时需要做的工作。

下面我们来描述 **Linux ext3** 这个流行的日志文件系统（journaling file system）是如何把日志纳入文件系统的。它在磁盘上的大部分结构都和 **Linux ext2** 相同，例如，磁盘被划分为若干块组，每个块组（block group）包含一个 inode 位图、一个数据位图（data bitmap）、若干 inode 和数据块。新增的关键结构就是日志本身，它在分区内或另一个设备上占据一小块空间。因此，一个（没有日志的）ext2 文件系统看起来是这样的：




假设日志放在同一个文件系统映像中（不过有时它也放在单独的设备上，或者作为文件系统中的一个文件），带日志的 ext3 文件系统看起来是这样的：

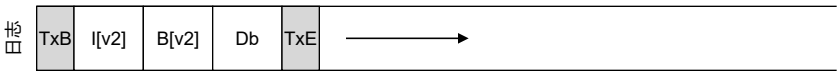



真正的区别只是多了一个日志，当然，还有它的用法。

### 数据日志

我们来看一个简单的例子，以理解**数据日志**（data journaling）是怎样工作的。数据日志（data journaling）是 Linux ext3 文件系统提供的一种模式，下面的讨论大多以它为基础。

假设我们又要做那个典型的更新：把 inode（I[v2]）、位图（B[v2]）和数据块（Db）写到磁盘上。现在，在把它们写到磁盘上的最终位置之前，我们要先把它们写到日志（log，也叫 journal）里。在日志中，这些块看起来是这样的：



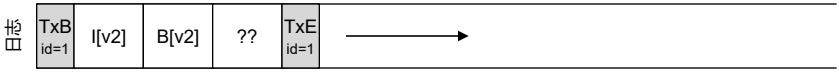

可以看到，这里我们写了五个块。事务开始块（transaction begin block，TxB）告诉我们这次更新的情况，包括即将对文件系统做哪些更新（例如，块 I[v2]、B[v2] 和 Db 的最终地址），以及某种**事务标识符**（transaction identifier，**TID**）。中间三个块就是这些块本身的确切内容；这叫作**物理日志**（physical logging），因为我们把更新的确切物理内容放进了日志。另一种思路是**逻辑日志**（logical logging），它把更新的一种更紧凑的逻辑表示放进日志，例如“这次更新要把数据块 Db 追加到文件 X”，这种方式复杂一些，但能节省日志空间，或许还能提高性能。最后一个块（TxE）标志着这个事务（transaction）的结束，其中也包含 TID。

一旦这个事务安全地写到了磁盘上，我们就可以去覆盖文件系统中的旧结构了；这个过程叫作**检查点**（checkpointing）。因此，要对文件系统做**检查点**（checkpoint），也就是让它和日志中待完成的更新保持一致，我们就把 I[v2]、B[v2] 和 Db 写到上面看到的它们在磁盘上的位置；如果这些写入成功完成，我们就成功地对文件系统做了检查点（checkpoint），基本上就大功告成了。于是，我们最初的操作顺序是：

1. **日志写入：**把事务写入日志，包括一个事务开始块（transaction begin block）、所有待完成的数据和元数据更新，以及一个事务结束块（transaction end block）；等待这些写入完成。
2. **检查点：**把待完成的元数据和数据更新写到它们在文件系统中的最终位置。

在这个例子中，我们会先把 TxB、I[v2]、B[v2]、Db 和 TxE 写到日志里。这些写入完成后，再通过检查点（checkpoint）把 I[v2]、B[v2] 和 Db 写到它们在磁盘上的最终位置，从而完成这次更新。

如果在写日志的过程中发生了崩溃，事情就有点棘手了。这里我们要把事务中的一组块（例如 TxB、I[v2]、B[v2]、Db、TxE）写到磁盘上。一种简单的办法是一次发出一个写入，等它完成后再发出下一个。但这样太慢了。理想情况下，我们希望一次发出全部五个块的写入，这样就能把五次写入变成一次顺序写入，速度更快。然而，这样做并不安全，原因如下：面对这样一次大的写入，磁盘内部可能会做调度，按任意顺序完成这次大写入的各个小部分。因此，磁盘内部可能会（1）先写入 TxB、I[v2]、B[v2] 和 TxE，过一会儿才（2）写入 Db。遗憾的是，如果磁盘在（1）和（2）之间断电，磁盘上最终会是这样：




为什么这是个问题？嗯，这个事务看起来是一个有效的事务（它有开始块和结束块，而且序号相互匹配）。此外，文件系统没法看一眼第四个块就知道它是错的；毕竟，那是任意的用户数据。因此，如果系统现在重启并运行恢复，它就会重放（replay）这个事务，稀里糊涂地把垃圾块“??”的内容复制到 Db 本该存放的位置。对于文件里的任意用户数据，这

补充：强制写入磁盘

为了在两次磁盘写入之间强制规定顺序，现代文件系统必须采取一些额外的预防措施。在过去，要在两次写入 $A$ 和 $B$ 之间强制规定顺序很容易：只要向磁盘发出 $A$ 的写入，等磁盘在写入完成时中断（interrupt）操作系统，然后再发出 $B$ 的写入。

由于磁盘内部越来越多地使用写缓存（write cache），事情变得稍微复杂了一些。启用写缓冲（有时叫作 **immediate reporting**）后，磁盘只要把数据放进自己的内存缓存（cache），还没有真正写到磁盘上，就会告诉操作系统写入已经完成。如果操作系统接着发出后续的写入，就无法保证它会在前面的写入之后到达磁盘；因此，写入之间的顺序得不到保证。一种解决办法是禁用写缓冲（write buffering）。然而，更现代的系统会采取额外的预防措施，发出显式的**写屏障**（write barrier）；这样的屏障（barrier）完成时，能保证在屏障之前发出的所有写入，都会先于屏障之后发出的任何写入到达磁盘。

所有这些机制都要求我们非常信任磁盘能正确运行。遗憾的是，最近的研究表明，有些磁盘制造商为了交付“性能更高”的磁盘，明确地忽略写屏障（write barrier）请求，从而让磁盘看起来跑得更快，代价是有可能运行出错 [C+13, R+11]。正如 Kahan 所说，快的几乎总能打败慢的，哪怕快的是错的。

已经很糟糕了；如果这种事发生在文件系统的某个关键部分上，比如超级块，那就更糟了，文件系统可能因此无法挂载。

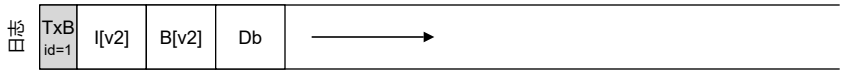
补充：优化日志写入

你可能已经注意到，写日志有一个特别低效的地方：文件系统必须先写出事务开始块（transaction begin block）和事务的内容；只有在这些写入完成之后，文件系统才能把事务结束块（transaction end block）发往磁盘。想想磁盘的工作原理，性能上的影响就很清楚了：通常会多花一圈旋转的时间（想想为什么）。

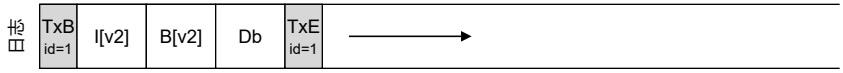
我们以前的一位研究生 Vijayan Prabhakaran 想出了一个简单的办法来解决这个问题 [P+05]。把事务写入日志时，在开始块（begin block）和结束块（end block）里加上日志内容的校验和（checksum）。这样一来，文件系统就可以一次写出整个事务，而不必等待；如果在恢复过程中，文件系统发现计算出的校验和（computed checksum）与事务中保存的校验和不一致，就可以断定崩溃发生在写入这个事务的过程中，从而丢弃这次文件系统更新。因此，只需对写入协议（protocol）和恢复系统稍作调整，文件系统就能在常见情况（common case）下获得更好的性能；除此之外，系统也更可靠了一点，因为现在从日志读出的任何内容都有校验和保护。

这个简单的修复很有吸引力，引起了 Linux 文件系统开发者的注意，他们随后把它纳入了下一代 Linux 文件系统，叫作（你猜对了！）**Linux ext4**。它现在运行在全世界数以百万计的机器上，包括 Android 手持平台。因此，在许多基于 Linux 的系统上，每当你向磁盘写入数据时，都有一小段在威斯康星开发的代码让你的系统变得更快、更可靠一点。

为了避免这个问题，文件系统分两步发出事务写入。首先，它把除 TxE 块之外的所有块写入日志，这些写入一次全部发出。这些写入完成后，日志看起来是这样的（还是假设我们的追加工作负载）：




等它们完成后，文件系统再发出 TxE 块的写入，使日志处于最终的安全状态：




这个过程的一个重要方面是磁盘提供的原子性（atomicity）保证。磁盘能保证任何 512 字节的写入要么完成，要么不发生（绝不会只写一半）；因此，为了确保 TxE 的写入是原子的，应该让它只占一个 512 字节的块。于是，我们目前更新文件系统的协议如下，三个阶段各有标注：

1. **日志写入：**把事务的内容（包括 TxB、元数据和数据）写入日志；等待这些写入完成。
2. **日志提交：**把事务提交块（transaction commit block，包含 TxE）写入日志；等待写入完成；此时称事务已**提交**（committed）。
3. **检查点：**把更新的内容（元数据和数据）写到它们在磁盘上的最终位置。

### 恢复

现在我们来理解文件系统如何利用日志的内容从崩溃中**恢复**（recover）。在这一系列更新中，崩溃随时可能发生。如果崩溃发生在事务安全地写入日志之前（即上面的第 2 步完成之前），我们的工作就很简单：直接跳过这次待完成的更新。如果崩溃发生在事务已经提交到日志之后、检查点（checkpoint）完成之前，文件系统可以按如下方式**恢复**这次更新。系统启动时，文件系统的恢复过程会扫描日志，找出已经提交到磁盘的事务；然后（按顺序）**重放**（replay）这些事务，文件系统再次尝试把事务中的块写到它们在磁盘上的最终位置。这种形式的日志是最简单的形式之一，叫作**重做日志**（redo logging）。恢复了日志中已提交的事务，文件系统就能确保磁盘上的结构是一致的，然后就可以继续挂载文件系统，准备处理新的请求了。

注意，在检查点（checkpoint）过程中的任何时刻发生崩溃都没有关系，即使有些块已经更新到了最终位置也无妨。最坏的情况是，其中一些更新在恢复时会再执行一遍。由于恢复是一种罕见的操作（只在系统意外崩溃之后才进行），多出几次冗余写入不值得担心[3]。

[3]除非你什么都担心，那我们也帮不了你。别这么操心了，这对健康不好！不过现在你大概又开始担心自己操心过度了。

### 批量更新日志

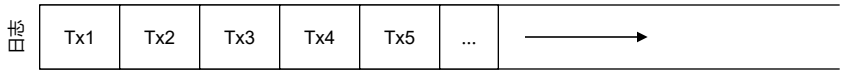
你可能已经注意到，这个基本协议会增加大量额外的磁盘流量。例如，假设我们在同一个目录里连续创建两个文件，分别叫作 `file1` 和 `file2`。创建一个文件，就要更新磁盘上的一些结构，至少包括：inode 位图（分配一个新 inode）、新创建的文件的 inode、父目录（parent directory）中包含新目录项的数据块，以及父目录的 inode（它现在有了新的修改时间）。有了日志，这两次文件创建在逻辑上都要把所有这些信息提交到日志；由于这两个文件在同一个目录中，再假设它们的 inode 甚至在同一个 inode 块（inode block）里，这就意味着，如果不加注意，我们就会一遍又一遍地写同样的这些块。

为了解决这个问题，有些文件系统（例如 Linux ext3）不会把每次更新一个一个地提交到磁盘，而是把所有更新缓冲到一个全局事务中。在上面的例子中，创建这两个文件时，文件系统只是把内存中的 inode 位图、文件的 inode、目录数据和目录 inode 标记为脏，并把它们加到构成当前事务的块列表中。等到最终该把这些块写到磁盘时，比如 5 秒超时（timeout）之后，就提交这个包含上述所有更新的全局事务。这样把更新缓冲起来，文件系统在许多情况下都能避免向磁盘写入过多的数据。

### 让日志大小有限

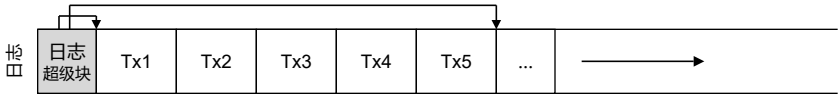
至此，我们得到了一个更新文件系统磁盘结构的基本协议。文件系统先把更新在内存中缓冲一段时间；等到最终该写入磁盘时，就小心地把事务的细节写到日志中，也就是预写日志（write-ahead logging）；事务完成后，再通过检查点（checkpoint）把这些块写到它们在磁盘上的最终位置。

然而，日志的大小是有限的。如果我们不停地往里面添加事务（就像下图这样），它很快就会被填满。你觉得那时会发生什么？




日志被填满时，会出现两个问题。第一个比较简单，但不那么要紧：日志越大，恢复所需的时间就越长，因为恢复时必须（按顺序）重放（replay）日志中的所有事务。第二个问题更严重：日志满了（或者快满了）时，就再也不能向磁盘提交新的事务了，文件系统也就变得“不太有用”（也就是没用了）。

为了解决这些问题，日志文件系统（journaling file system）把日志当作一个循环数据结构，反复使用它；这就是日志有时也叫**循环日志**（circular log）的原因。为此，文件系统必须在检查点（checkpoint）之后的某个时候采取行动。具体来说，一旦某个事务完成了检查点（checkpoint），文件系统就应该释放它在日志中占用的空间，让这些日志空间可以重新使用。实现这一目标的方法有很多；例如，你可以简单地在一个**日志超级块**（journal superblock）中标记出日志中最老的和最新的尚未完成检查点（checkpoint）的事务；其余空间都是空闲的。下面是图示：




在日志超级块（journal superblock）中（不要和文件系统主超级块混淆），日志系统记录足够的信息，以便知道哪些事务还没有完成检查点（checkpoint），从而既缩短了恢复时间，又能让日志循环使用。于是，我们在基本协议中又加了一步：

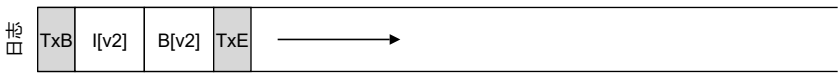
1. **日志写入：**把事务的内容（包含 TxB 和更新的内容）写入日志；等待这些写入完成。
2. **日志提交：**把事务提交块（transaction commit block，包含 TxE）写入日志；等待写入完成；事务现在已**提交**。
3. **检查点：**把更新的内容写到它们在文件系统中的最终位置。
4. **释放：**过一段时间后，更新日志超级块（journal superblock），把这个事务在日志中标记为空闲。

这样，我们就得到了最终的数据日志（data journaling）协议。但还有一个问题：我们把每个数据块都向磁盘写了两次，这是一笔沉重的代价，尤其是为了系统崩溃这样罕见的事情。你能想出一种不用把数据写两次、又能保持一致性的办法吗?

### 元数据日志

虽然恢复现在很快了（扫描日志并重放几个事务，而不是扫描整个磁盘），但文件系统的正常运行比我们希望的要慢。特别是，每次写入磁盘时，我们现在还要先写日志，写入流量因此翻了一倍；在顺序写的工作负载中，这种翻倍尤其让人头疼，因为写入速度现在只有驱动器峰值写入带宽（bandwidth）的一半。此外，在写日志和写主文件系统之间，还有一次代价高昂的寻道（seek），这在某些工作负载下会带来明显的开销。

由于把每个数据块都向磁盘写两次的代价很高，人们尝试了几种不同的办法来提升性能。例如，我们上面描述的日志模式通常叫作**数据日志**（data journaling，如 Linux ext3 中的叫法），因为它把所有用户数据（以及文件系统的元数据）都记入了日志。一种更简单（也更常见）的日志形式有时叫作**有序日志**（ordered journaling，或者就叫**元数据日志**，metadata journaling），它和数据日志（data journaling）几乎一样，只是用户数据不写入日志。因此，执行和上面一样的更新时，写入日志的是下面这些信息：




以前写入日志的数据块 Db，现在直接写到文件系统本身，避免了额外的写入；由于磁盘的大部分 I/O 流量都是数据，不把数据写两次就能大大减轻日志带来的 I/O 负担。不过，这一改动确实引出了一个有意思的问题：我们应该在什么时候把数据块写到磁盘上?

为了更好地理解这个问题，我们再来看看追加文件的那个例子。这次更新包括三个块：I[v2]、B[v2] 和 Db。前两个都是元数据，会先记入日志，再做检查点（checkpoint）；后一个只会向文件系统写入一次。我们应该什么时候把 Db 写到磁盘上？这有关系吗？

对于只记元数据的日志来说，数据写入的顺序确实很重要。例如，如果我们在事务（包含 I[v2] 和 B[v2]）完成之后才把 Db 写到磁盘，会怎样？遗憾的是，这种做法有一个问题：文件系统是一致的，但 I[v2] 最终可能指向垃圾数据。具体来说，考虑 I[v2] 和 B[v2] 写入了、但 Db 没能写到磁盘的情况。接着文件系统会尝试恢复。由于 Db 不在日志里，文件系统会重放（replay）对 I[v2] 和 B[v2] 的写入，得到一个（从文件系统元数据的角度看）一致的文件系统。然而，I[v2] 会指向垃圾数据，也就是 Db 本该写入的那个位置上原来的内容。

为了确保不出现这种情况，有些文件系统（例如 Linux ext3）会在相关的元数据写到磁盘之前，先把（普通文件的）数据块写到磁盘。具体来说，协议如下：

1. **数据写入：** 把数据写到最终位置；等待完成（这一等待是可选的，详见下文）。
2. **日志元数据写入：** 把开始块（begin block）和元数据写入日志；等待这些写入完成。
3. **日志提交：** 把事务提交块（transaction commit block，包含 TxE）写入日志；等待写入完成；事务现在已**提交**，包括数据在内。
4. **检查点元数据：** 把元数据更新的内容写到它们在文件系统中的最终位置。
5. **释放：** 之后，在日志超级块（journal superblock）中把这个事务标记为空闲。

强制先写数据，文件系统就能保证指针永远不会指向垃圾。其实，“先写被指向的对象，再写指向它的对象”这条规则正是崩溃一致性（crash consistency）的核心，其他一些崩溃一致性方案对它的利用还要更进一步 [GP94]（详见下文）。

在大多数系统中，元数据日志（类似于 ext3 中的有序日志）比完整的数据日志（data journaling）更流行。例如，Windows NTFS 和 SGI 的 XFS 都使用某种形式的元数据日志（metadata journaling）。Linux ext3 让你可以在数据（data）、有序（ordered）和无序（unordered）三种模式中任选一种（在无序模式下，数据可以在任何时候写入）。所有这些模式都能保持元数据一致；它们的区别在于对数据提供的语义不同。

最后要注意，正如上面的协议中所指出的，在发出日志写入（第 2 步）之前强制等待数据写入完成（第 1 步），并不是正确性所必需的。具体来说，同时发出对数据、事务开始块（transaction begin block）和日志元数据的写入也完全没问题；唯一真正的要求是，第 1 步和第 2 步要在发出日志提交块（第 3 步）之前完成。

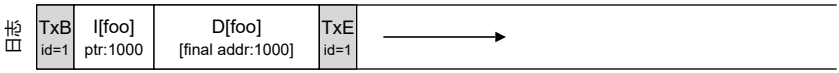
### 棘手的情况：块复用

还有一些有意思的边界情况会让日志变得更加棘手，因此值得讨论一下。其中有不少都和块复用有关；正如 Stephen Tweedie（ext3 背后的主要推动者之一）所说：

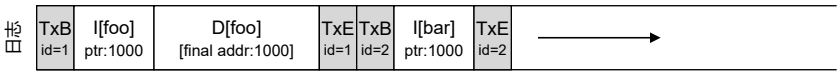
> “整个系统最可怕的部分是什么？……是删除文件。和删除有关的一切都很麻烦。和删除有关的一切……你会做噩梦，梦见块被删除、然后又被重新分配会发生什么。”[T00]

Tweedie 举的具体例子如下。假设你在使用某种形式的元数据日志（因此文件的数据块不记入日志）。比如说，有一个叫 `foo` 的目录。用户往 `foo` 里加了一个目录项（比如创建

了一个文件），于是 foo 的内容（因为目录被视为元数据）写进了日志；设 foo 目录数据存放在块 1000。日志里的内容大概是这样的：




这时，用户删除了这个目录里的所有内容以及目录本身，块 1000 被释放出来，可以重新使用。最后，用户创建了一个新文件（比如叫 bar），它恰好复用了原来属于 foo 的那个块（1000）。bar 的 inode 和它的数据都提交到了磁盘；但要注意，由于使用的是元数据日志（metadata journaling），只有 bar 的 inode 提交到了日志中；文件 bar 写入块 1000 的新数据没有记入日志。




现在假设发生了崩溃，而所有这些信息都还在日志里。重放（replay）时，恢复过程只是简单地重放日志中的所有内容，包括对块 1000 中目录数据的写入；于是，重放（replay）会用旧的目录内容覆盖当前文件 bar 的用户数据！显然，这不是正确的恢复操作，用户读文件 bar 时肯定会大吃一惊。

这个问题有几种解决办法。例如，可以规定：在这些块的删除操作通过检查点（checkpoint）移出日志之前，绝不复用这些块。Linux ext3 的做法则是在日志中加入一种新的记录类型，叫作 **revoke record**。在上面的例子中，删除目录时会往日志里写一条 revoke record。重放（replay）日志时，系统先扫描这类 revoke record；任何已被撤销（revoke）的数据都不会被重放（replay），从而避免了上面提到的问题。

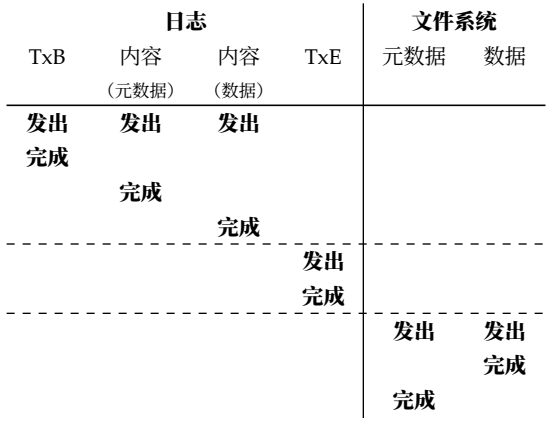



图 42.1：**数据日志的时间线**

 © 2008–23, ARPACI-DUSSEAU

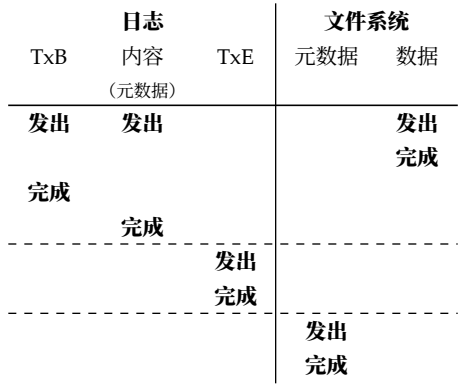



图 42.2：**元数据日志的时间线**

### 日志小结：时间线

在结束对日志的讨论之前，我们用时间线把讨论过的协议各总结一遍。图 42.1（第 480 页）展示的是对数据和元数据都记日志时的协议，图 42.2（第 481 页）展示的是只对元数据记日志时的协议。

在每张图中，时间向下递增，图中的每一行表示一次写入可以发出或可能完成的逻辑时间。例如，在数据日志（data journaling）协议中（图 42.1），事务开始块（TxB）的写入和事务内容的写入在逻辑上可以同时发出，因此可以按任意顺序完成；但是，对事务结束块（TxE）的写入，必须等前面这些写入完成之后才能发出。同样，对数据块和元数据块的检查点（checkpoint）写入，要等事务结束块（transaction end block）提交之后才能开始。水平虚线表示必须遵守写入顺序要求的地方。

元数据日志（metadata journaling）协议也有一张类似的时间线。注意，数据写入在逻辑上可以和事务开始块（transaction begin block）以及日志内容的写入同时发出；但是，它必须在事务结束块（transaction end block）发出之前发出并完成。

最后要注意，时间线中为每次写入标出的完成时间是任意的。在真实的系统中，完成时间由 I/O 子系统决定，它可能会为了提高性能而对写入重新排序。我们对顺序仅有的保证，就是为了协议的正确性而必须强制执行的那些（在图中用水平虚线表示）。

## 42.4 方案三：其他方法

到目前为止，我们介绍了保持文件系统元数据一致的两种方案：一种是基于 fsck 的“偷懒”方法，另一种是更主动的方法，叫作日志。然而，方法并不只有这两种。其中一种方法叫作软更新（Soft Updates）[GP94]，由 Ganger 和 Patt 提出。这种方法仔细安排所有写入文件系统的顺序，确保磁盘上的结构永远不会处于不一致的状态。例如，先把数据块写到磁盘，再写指向它的 inode，就能确保 inode 永远不会指向垃圾；对文件系统的所有结构，都可以推导出类似的规则。不过，实现软更新（soft updates）可能很有挑战性；上面描述的日

志层在实现时只需要对文件系统的具体结构有比较少的了解，而软更新（soft updates）则需要对每个文件系统数据结构都有细致入微的了解，因此会给系统增加相当多的复杂性。

另一种方法叫作**写时复制**（copy-on-write，没错，就是 **COW**），许多流行的文件系统都在使用它，包括 Sun 的 ZFS [B07]。这种技术从不原地覆盖文件或目录，而是把新的更新放到磁盘上以前没有使用过的位置。完成若干次更新之后，COW 文件系统会翻转文件系统的根结构，让它指向刚更新过的那些结构。这样一来，保持文件系统一致就很简单了。在后面的章节讨论日志结构文件系统（LFS）时，我们会更多地了解这种技术；LFS 就是 COW 的一个早期例子。

还有一种方法是我们在威斯康星这里刚刚开发出来的。这种技术叫作 **backpointer-based consistency**（**BBC**），它不在写入之间强制规定任何顺序。为了实现一致性，系统中的每个块都会额外加上一个**反向指针**（back pointer）；例如，每个数据块都引用它所属的 inode。访问文件时，文件系统可以检查正向指针（例如 inode 或直接块中的地址）所指向的块是否反过来引用了它，以此判断这个文件是否一致。如果是，那么一切一定都已安全地写到了磁盘上，文件是一致的；如果不是，文件就不一致，文件系统返回一个错误。给文件系统加上反向指针（back pointer），就能实现一种新形式的“偷懒”式崩溃一致性（crash consistency） [C+12]。

最后，我们还研究了一些技术，用来减少日志协议必须等待磁盘写入完成的次数。这种新方法叫作 **optimistic crash consistency** [C+13]，它利用一种推广形式的**事务校验和**（transaction checksum）[P+05]，尽可能多地向磁盘发出写入，并用另外几种技术来检测可能出现的不一致。对于某些工作负载，这些乐观的技术能把性能提高一个数量级。然而，要真正发挥作用，需要一个稍有不同的磁盘接口 [C+13]。

## 42.5 小结

我们介绍了崩溃一致性问题（crash-consistency problem），并讨论了解决这个问题的各种方法。较老的方法是构建一个文件系统检查程序（file system checker），它能用，但在现代系统上恢复起来可能太慢了。因此，现在许多文件系统改用了日志。日志把恢复时间从 O(磁盘卷的大小) 减少到 O(日志的大小)，从而大大加快了崩溃重启后的恢复速度，这也是许多现代文件系统采用日志的原因。我们还看到，日志可以有许多不同的形式；最常用的是有序元数据日志（metadata journaling），它减少了写入日志的流量，同时仍然对文件系统元数据和用户数据都提供了合理的一致性保证。归根结底，对用户数据提供强有力的保证，可能是最重要的事情之一；说来也怪，正如最近的研究所表明的，这个领域仍然在不断发展 [P+14]。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[B07] “ZFS: The Last Word in File Systems” by Jeff Bonwick and Bill Moore. Available online: http://www.ostep.org/Citations/zfs_last.pdf. *ZFS uses copy-on-write and journaling, actually, as in some cases, logging writes to disk will perform better.*

[C+12] “Consistency Without Ordering” by Vijay Chidambaram, Tushar Sharma, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST ’12, San Jose, California. *A recent paper of ours about a new form of crash consistency based on back pointers. Read it for the exciting details!*

[C+13] “Optimistic Crash Consistency” by Vijay Chidambaram, Thanu S. Pillai, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau . SOSP ’13, Nemacolin Woodlands Resort, PA, November 2013. *Our work on a more optimistic and higher performance journaling protocol. For workloads that call* `fsync()` *a lot, performance can be greatly improved.*

[GP94] “Metadata Update Performance in File Systems” by Gregory R. Ganger and Yale N. Patt. OSDI ’94. *A clever paper about using careful ordering of writes as the main way to achieve consistency. Implemented later in BSD-based systems.*

[G+08] “SQCK: A Declarative File System Checker” by Haryadi S. Gunawi, Abhishek Rajimwale, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. OSDI ’08, San Diego, California. *Our own paper on a new and better way to build a file system checker using SQL queries. We also show some problems with the existing checker, finding numerous bugs and odd behaviors, a direct result of the complexity of* `fsck`.

[H87] “Reimplementing the Cedar File System Using Logging and Group Commit” by Robert Hagmann. SOSP ’87, Austin, Texas, November 1987. *The first work (that we know of) that applied write-ahead logging (a.k.a. journaling) to a file system.*

[M+13] “ffsck: The Fast File System Checker” by Ao Ma, Chris Dragga, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST ’13, San Jose, California, February 2013. *A recent paper of ours detailing how to make fsck an order of magnitude faster. Some of the ideas have already been incorporated into the BSD file system checker [MK96] and are deployed today.*

[MK96] “Fsck – The UNIX File System Check Program” by Marshall Kirk McKusick and T. J. Kowalski. Revised in 1996. *Describes the first comprehensive file-system checking tool, the eponymous* `fsck`. *Written by some of the same people who brought you FFS.*

[MJLF84] “A Fast File System for UNIX” by Marshall K. McKusick, William N. Joy, Sam J. Leffler, Robert S. Fabry. ACM Transactions on Computing Systems, Volume 2:3, August 1984. *You already know enough about FFS, right? But come on, it is OK to re-reference important papers.*

[P+14] “All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications” by Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. OSDI ’14, Broomfield, Colorado, October 2014. *A paper in which we study what file systems guarantee after crashes, and show that applications expect something different, leading to all sorts of interesting problems.*

[P+05] “IRON File Systems” by Vijayan Prabhakaran, Lakshmi N. Bairavasundaram, Nitin Agrawal, Haryadi S. Gunawi, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. SOSP ’05, Brighton, England, October 2005. *A paper mostly focused on studying how file systems react to disk failures. Towards the end, we introduce a transaction checksum to speed up logging, which was eventually adopted into Linux ext4.*

[PAA05] “Analysis and Evolution of Journaling File Systems” by Vijayan Prabhakaran, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. USENIX ’05, Anaheim, California, April 2005. *An early paper we wrote analyzing how journaling file systems work.*

[R+11] “Coerced Cache Eviction and Discreet-Mode Journaling” by Abhishek Rajimwale, Vijay Chidambaram, Deepak Ramamurthi, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. DSN ’11, Hong Kong, China, June 2011. *Our own paper on the problem of disks that buffer writes in a memory cache instead of forcing them to disk, even when explicitly told not to do that! Our solution to overcome this problem: if you want A to be written to disk before B, first write A, then send a lot of “dummy” writes to disk, hopefully causing A to be forced to disk to make room for them in the cache. A neat if impractical solution.*

[T98] “Journaling the Linux ext2fs File System” by Stephen C. Tweedie. The Fourth Annual Linux Expo, May 1998. *Tweedie did much of the heavy lifting in adding journaling to the Linux ext2 file system; the result, not surprisingly, is called ext3. Some nice design decisions include the strong focus on backwards compatibility, e.g., you can just add a journaling file to an existing ext2 file system and then mount it as an ext3 file system.*

[T00] “EXT3, Journaling Filesystem” by Stephen Tweedie. Talk at the Ottawa Linux Symposium, July 2000. olstrans.sourceforge.net/release/OLS2000-ext3/OLS2000-ext3.html *A transcript of a talk given by Tweedie on ext3.*

[T01] “The Linux ext2 File System” by Theodore Ts’o, June, 2001.. Available online here: http://e2fsprogs.sourceforge.net/ext2.html. *A simple Linux file system based on the ideas found in FFS. For a while it was quite heavily used; now it is really just in the kernel as an example of a simple file system.*

 © 2008–23, ARPACI-DUSSEAU

## 作业（模拟）

本节介绍 `fsck.py`，这是一个简单的模拟器，可以帮你更好地理解如何检测（并可能修复）文件系统损坏。关于如何运行这个模拟器，请参阅附带的 README。

## 问题

1. 首先，运行 `fsck.py -D`；这个选项会关闭所有损坏，因此你可以用它生成一个随机的文件系统，看看你能否判断出里面有哪些文件和目录。那就动手吧！用 `-p` 选项看看你是否答对了。把随机种子（`-s`）设成不同的值，比如 1、2 和 3，对几个不同的随机生成的文件系统试一试。
2. 现在，我们来引入一处损坏。先运行 `fsck.py -S 1`。你能看出引入了什么不一致吗？在真实的文件系统修复工具中，你会怎样修复它？用 `-c` 检查你是否答对了。
3. 把种子改成 `-S 3` 或 `-S 19`；你看到了什么不一致？用 `-c` 检查你的答案。这两种情况有什么不同？
4. 把种子改成 `-S 5`；你看到了什么不一致？自动修复这个问题会有多难？用 `-c` 检查你的答案。然后，用 `-S 38` 引入一个类似的不一致；这个更难检测吗？能检测出来吗？最后，用 `-S 642`；这个不一致能检测出来吗？如果能，你会怎样修复这个文件系统？
5. 把种子改成 `-S 6` 或 `-S 13`；你看到了什么不一致？用 `-c` 检查你的答案。这两种情况有什么区别？修复工具遇到这种情况时应该怎么做？
6. 把种子改成 `-S 9`；你看到了什么不一致？用 `-c` 检查你的答案。在这种情况下，检查修复工具应该相信哪一部分信息？
7. 把种子改成 `-S 15`；你看到了什么不一致？用 `-c` 检查你的答案。在这种情况下，修复工具能做什么？如果无法修复，会丢失多少数据？
8. 把种子改成 `-S 10`；你看到了什么不一致？用 `-c` 检查你的答案。这里的文件系统结构中有没有冗余（redundancy）信息可以帮助修复？
9. 把种子改成 `-S 16` 和 `-S 20`；你看到了什么不一致？用 `-c` 检查你的答案。修复工具应该怎样修复这个问题？

 © 2008–23, ARPACI-DUSSEAU

# 43

# 日志结构文件系统

20 世纪 90 年代初，伯克利的一个小组在 John Ousterhout 教授和研究生 Mendel Rosenblum 的带领下，开发了一种新的文件系统（file system），叫作日志结构文件系统（log-structured file system）[RO91]。促使他们这样做的，是以下几点观察：

- **系统内存在不断增长：**内存越大，能缓存（cache）在内存中的数据就越多。缓存的数据越多，就有越多的读请求直接由缓存满足，磁盘流量中写入所占的比例也就越来越大。因此，文件系统的性能在很大程度上取决于它的写入性能。
- **随机 I/O 性能和顺序 I/O 性能之间差距很大：**这些年来，硬盘的传输带宽（bandwidth）增长了很多 [P98]；盘面（surface）上塞进的位越多，访问这些位时的带宽就越高。然而，寻道（seek）和旋转延迟（rotational delay）的开销却下降得很慢；便宜的小电机很难把盘片（platter）转得更快，也很难让磁盘臂（disk arm）移动得更快。因此，如果能顺序地使用磁盘，性能就会比需要寻道和旋转的访问方式好得多。
- **现有的文件系统在许多常见工作负载下表现不佳：**例如，FFS [MJLF84] 创建一个大小为一个块（block）的新文件时，要执行大量写入：一次写新的 inode，一次更新 inode 位图（inode bitmap），一次写文件所在目录（directory）的数据块（data block），一次更新目录的 inode，一次写属于新文件的新数据块，还有一次写数据位图（data bitmap），把这个数据块标记为已分配。因此，虽然 FFS 把所有这些块都放在同一个块组（block group）里，它还是会引起许多短距离的寻道以及随之而来的旋转延迟，性能远远达不到顺序带宽的峰值。
- **文件系统不了解 RAID：**例如，RAID-4 和 RAID-5 都有**小写入问题**（small-write problem），即对单个块的一次逻辑写入会引起 4 次物理 I/O。现有的文件系统并不会设法避开 RAID 这种最坏情况下的写入行为。

提示：细节很重要

所有有意思的系统，都是由几个一般性的思想加上大量细节组成的。有时，你在学习这些系统时会对自己说："哦，我懂大体思路了，剩下的不过是细节。"然后就满足于此，对系统到底怎么工作只弄懂了一半。别这样！很多时候，细节是至关重要的。我们在 LFS 中会看到，大体思路很容易理解，但要真正构建一个能用的系统，你必须把所有棘手的情况都想清楚。

因此，一个理想的文件系统应该专注于写入性能，并设法利用磁盘的顺序带宽。此外，它在常见的工作负载（workload）下也应该表现良好，这些工作负载不仅要写出数据，还会频繁地更新磁盘上的元数据结构（metadata structure）。最后，它在 RAID 和单个磁盘上都应该工作得很好。

Rosenblum 和 Ousterhout 提出的这种新型文件系统叫作 **LFS**，是**日志结构文件系统**（Log-structured File System）的缩写。向磁盘写入时，LFS 先把所有更新都缓冲在内存中的一个**段**（segment）里，连元数据（metadata）也不例外；段满了以后，就通过一次长长的顺序传输，把它写到磁盘上一块未使用的区域。LFS 从不覆盖已有的数据，总是把段写到空闲的位置。由于段很大，磁盘（或 RAID）得到了高效的利用，文件系统的性能也能逼近峰值。

关键问题：
如何让所有写入都变成顺序写入？
文件系统怎样才能把所有写入都变成顺序写入？对于读，这个任务是不可能完成的，因为要读的块可能在磁盘上的任何地方。然而，对于写，文件系统总是有选择的余地，而我们希望利用的正是这一选择。

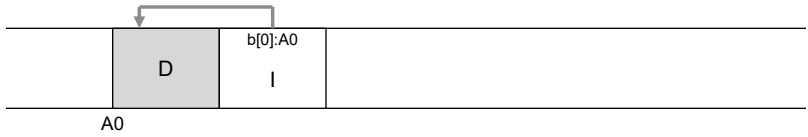
## 43.1 顺序地写入磁盘

于是，我们面临的第一个挑战是：如何把对文件系统状态的所有更新，都变成一系列对磁盘的顺序写入？为了更好地理解这一点，我们来看一个简单的例子。假设我们要往一个文件里写一个数据块 $D$。把这个数据块写到磁盘上，可能会得到下面的磁盘布局，$D$ 写在磁盘地址（disk address） $A0$ 处：




然而，用户写入一个数据块时，写到磁盘上的不只是数据，还有其他一些需要更新的**元数据**（metadata）。在这个例子中，我们把文件的 **inode**（$I$）也写到磁盘上，并让它指向数据块 $D$。写到磁盘上之后，数据块和 inode 看起来大致是这样的（注意，图中 inode 看起来和数据块一样大，一般情况并非如此；在大多数系统中，数据块的大小是 4 KB，而 inode 要小得多，大约 128 字节）：

 © 2008–23, ARPACI-DUSSEAU




这个基本思想，即简单地把所有更新（比如数据块、inode 等）顺序地写到磁盘上，就是 LFS 的核心。理解了这一点，你就掌握了基本思路。但正如所有复杂的系统一样，魔鬼藏在细节里。

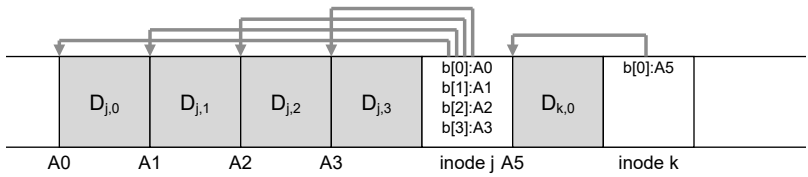
## 43.2 顺序而高效地写入

遗憾的是，仅仅顺序地写入磁盘，还不足以保证写入是高效的。例如，假设我们在时刻 $T$ 向地址 $A$ 写了一个块。然后我们等了一小会儿，在时刻 $T+\delta$ 向紧接着的下一个块地址（block address）$A+1$ 写入。不巧的是，在第一次写入和第二次写入之间，磁盘已经转过去了；等你发出第二次写入，磁盘要转上差不多一整圈才能把它提交（具体来说，如果转一圈需要时间 $T_{rotation}$，磁盘就要等 $T_{rotation}-\delta$ 才能把第二次写入提交到盘面上）。由此你大概可以看出，仅仅按顺序写入磁盘，并不足以达到峰值性能；你必须向驱动器发出大量连续的写入（或者一次大的写入），才能获得良好的写入性能。

为了做到这一点，LFS 使用了一种古老的技术，叫作**写缓冲**（write buffering）[1]。在写入磁盘之前，LFS 先在内存中记录下各项更新；收到足够多的更新后，再一次性把它们全部写到磁盘上，从而确保高效地使用磁盘。

LFS 一次写入的这一大块更新，叫作**段**（segment）。虽然这个词在计算机系统里已经用得太滥了，但在这里，它只是指 LFS 用来把写入归成一组的一个比较大的块。因此，向磁盘写入时，LFS 先把更新缓冲在内存中的段里，然后一次性把这个段写到磁盘上。只要段足够大，这些写入就会很高效。

下面是一个例子，LFS 把两组更新缓冲进一个较小的段里；实际的段要大得多（几 MB）。第一组更新是对文件 $j$ 的四个块的写入；第二组是给文件 $k$ 添加一个块。然后，LFS 一次性把这个由七个块组成的整个段提交到磁盘上。这些块最终在磁盘上的布局如下：




[1]说真的，这个思想很难找到一个合适的出处，因为它很可能在计算机历史的很早期就被许多人各自发明过。关于写缓冲（write buffering）的好处，可以参阅 Solworth 和 Orji 的研究 [SO90]；想了解它可能带来的坏处，可以参阅 Mogul 的论文 [M94]。

## 43.3 要缓冲多少?

这就引出了下面的问题: LFS 在写入磁盘之前, 应该缓冲多少更新? 答案当然取决于磁盘本身, 具体来说, 取决于定位开销和传输速率相比有多高; 类似的分析可以参阅 FFS 一章。

例如, 假设每次写入之前的定位, 也就是旋转和寻道开销, 大约需要 $T_{position}$ 秒。再假设磁盘的传输速率是 $R_{peak}$ MB/s。在这样的磁盘上, LFS 在写入之前应该缓冲多少数据?

思考这个问题的方法是: 每次写入, 你都要付出一笔固定的定位开销。那么, 你要写多少数据, 才能**平摊**(amortize)这笔开销? 写得越多越好(这是显然的), 也就越接近峰值带宽。

为了得到一个具体的答案, 假设我们要写出 $D$ MB。写出这块数据的时间($T_{write}$)等于定位时间 $T_{position}$ 加上传输 $D$ 的时间($\frac{D}{R_{peak}}$), 即:

$$T_{write} = T_{position} + \frac{D}{R_{peak}} \tag{43.1}$$

因此, 有效写入速率($R_{effective}$)就是写入的数据量除以写入所花的总时间:

$$R_{effective} = \frac{D}{T_{write}} = \frac{D}{T_{position} + \frac{D}{R_{peak}}}. \tag{43.2}$$

我们关心的是让有效速率($R_{effective}$)接近峰值速率。具体来说, 我们希望有效速率是峰值速率的某个比例 $F$, 其中 $0 < F < 1$(典型的 $F$ 可能是 0.9, 即峰值速率的 90%)。用数学形式表达, 就是我们希望 $R_{effective} = F \times R_{peak}$。

这样, 我们就可以解出 $D$:

$$R_{effective} = \frac{D}{T_{position} + \frac{D}{R_{peak}}} = F \times R_{peak} \tag{43.3}$$

$$D = F \times R_{peak} \times \left( T_{position} + \frac{D}{R_{peak}} \right) \tag{43.4}$$

$$D = \left( F \times R_{peak} \times T_{position} \right) + \left( F \times R_{peak} \times \frac{D}{R_{peak}} \right) \tag{43.5}$$

$$D = \frac{F}{1 - F} \times R_{peak} \times T_{position} \tag{43.6}$$

我们来算一个例子: 磁盘的定位时间是 10 毫秒, 峰值传输速率是 100 MB/s; 假设我们希望有效带宽达到峰值的 90%($F = 0.9$)。在这种情况下,
$D = \frac{0.9}{0.1} \times 100 \text{ MB/s} \times 0.01 \text{ 秒} = 9 \text{ MB}$。试试不同的取值, 看看需要缓冲多少数据才能接近峰值带宽。要达到峰值的 95% 需要多少? 99% 呢?

## 43.4 问题: 找到 inode

为了理解在 LFS 中如何找到 inode, 我们先简单回顾一下在典型的 UNIX 文件系统中如何找到 inode。在 FFS 这样的典型文件系统中, 甚至在老的 UNIX 文件系统中, 找 inode 都很容易, 因为 inode 排成数组, 放在磁盘上固定的位置。

例如，老的 UNIX 文件系统把所有 inode 都放在磁盘的一个固定区域。因此，给定 inode 号（inode number）和起始地址，要找到某个 inode，只需把 inode 号乘以 inode 的大小，再加上磁盘上这个数组的起始地址，就能算出它确切的磁盘地址。可见，基于数组的索引又快又简单。

在 FFS 中，根据 inode 号找 inode 只是稍微复杂一点，因为 FFS 把 inode 表（inode table）分成了若干部分，每个柱面组（cylinder group）里放一部分 inode。因此，你必须知道每一部分有多大，以及各部分的起始地址。之后的计算是类似的，也很简单。

在 LFS 中，事情就困难多了。为什么？嗯，我们已经把 inode 分散到了整个磁盘上！更糟的是，我们从不原地覆盖，因此 inode 的最新版本（也就是我们想要的那个）会不断移动。

## 43.5 用间接层来解决：inode map

为了解决这个问题，LFS 的设计者用一个叫作 **inode map**（**imap**）的数据结构（data structure），在 inode 号和 inode 之间引入了一个**间接层**（level of indirection）。imap 这个结构以 inode 号为输入，给出这个 inode 最新版本的磁盘地址。因此，你可以想象它通常会实现成一个简单的数组，每个条目 4 个字节（一个磁盘指针）。每当有 inode 写到磁盘上，imap 就会更新，记下它的新位置。

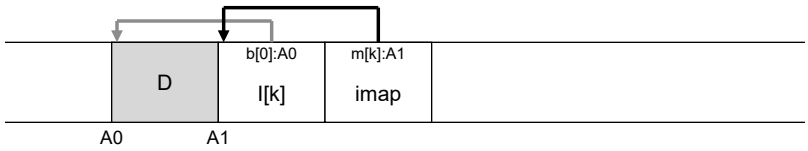
遗憾的是，imap 需要持久保存（也就是写到磁盘上）；这样做能让 LFS 在崩溃之后仍然知道各个 inode 的位置，从而按预期运行。于是有一个问题：imap 应该放在磁盘上的什么地方？

当然，它可以放在磁盘上的一个固定位置。但 imap 更新得很频繁，这样一来，每次更新文件结构之后都得再写一次 imap，性能就会受损（每次更新之后都要多一次寻道，去 imap 所在的固定位置）。

于是，LFS 把 inode map 的各个片段直接放在其他新写入的信息旁边。比如，往文件 $k$ 追加一个数据块时，LFS 实际上会把新数据块、它的 inode 以及 inode map 的一个片段一起写到磁盘上，如下所示：

提示：使用间接层

人们常说，计算机科学中所有问题的解决办法，就是加一个**间接层**（level of indirection）。这显然不对；它只是大多数问题的解决办法（没错，这么说还是太绝对了，但你明白我们的意思）。你当然可以把我们学过的每一种虚拟化（virtualization），比如虚拟内存（virtual memory）或文件的概念，都看作一个间接层。而 LFS 中的 inode map 当然就是对 inode 号的一种虚拟化。希望你能从这些例子中看出间接层的强大威力：它让我们可以自由地移动各种结构，比如虚拟内存例子里的页（page）、LFS 中的 inode，而不必修改对它们的每一处引用。当然，间接层也有缺点：**额外的开销**。所以，下次你遇到问题时，试试用间接层来解决，但一定要先想想这样做的开销。正如 Wheeler 的那句名言："计算机科学中的所有问题都可以通过再加一个间接层来解决，当然，间接层太多这个问题除外。"



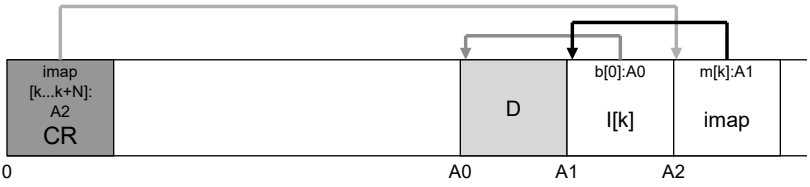

在这张图中，标着 *imap* 的那个块里存放着 imap 数组的一个片段，它告诉 LFS，inode *k* 在磁盘地址 *A*1 处；而这个 inode 又告诉 LFS，它的数据块 *D* 在地址 *A*0 处。

## 43.6 完成方案：检查点区域

聪明的读者（就是你，对吧?）可能已经注意到这里有个问题。既然 inode map 的各个片段现在也分散在磁盘各处，我们怎么找到 inode map 呢? 归根结底，这里没有什么魔法：文件系统必须在磁盘上有某个固定的、已知的位置，作为查找文件的起点。

LFS 在磁盘上正好有这样一个固定的位置，叫作**检查点区域**（checkpoint region，CR）。检查点区域包含指向 inode map 最新片段的指针（即它们的地址），因此先读 CR 就能找到 inode map 的各个片段。注意，检查点区域只是定期更新（比如每 30 秒左右一次），因此不会拖累性能。于是，磁盘布局的整体结构包含一个检查点区域，它指向 inode map 的最新片段；inode map 的每个片段包含 inode 的地址；inode 则像典型的 UNIX 文件系统中那样指向文件和目录。

下面是一个例子，其中有检查点区域，以及一个 imap 片段、一个 inode 和一个数据块。真实的文件系统当然会有一个大得多的 CR（实际上会有两个，我们后面会明白为什么）、许多 imap 片段，当然还有多得多的 inode、数据块等等。




## 43.7 从磁盘读取文件：回顾

为了确保你理解了 LFS 的工作原理，我们现在一步步看看从磁盘读取一个文件要做哪些事。假设一开始内存里什么都没有。我们必须读取的第一个磁盘数据结构是检查点区域。检查点区域包含指向整个 inode map 的指针，也就是这些片段的磁盘地址，于是 LFS 接着读入整个 inode map，并把它缓存在内存中。此后，给定一个文件的 inode 号，LFS 只需在 imap 中查找 inode 号到 inode 磁盘地址的映射，然后读入这个 inode 的最新版本。这时，要读文件中的某个块，LFS 的做法就和典型的 UNIX 文件系统完全一样，根据需要使用直接指针（direct pointer）、间接指针（indirect pointer）或二级间接指针（double indirect pointer）。在常见情况（common case）下，LFS 从磁盘读取文件时执行的 I/O 次数应该和

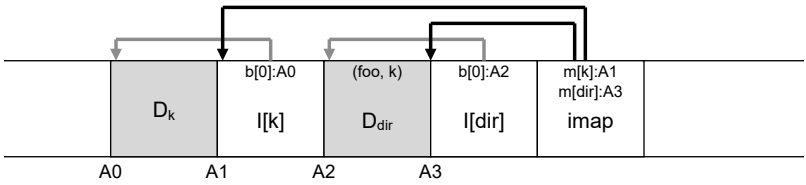
 © 2008–23, ARPACI-DUSSEAU

典型的文件系统相同；整个 imap 都缓存在内存里，因此 LFS 在读取时多做的工作，就是在 imap 中查找 inode 的地址。

## 43.8 目录怎么办?

到目前为止，我们只考虑了 inode 和数据块，把讨论稍微简化了一些。然而，要访问文件系统中的一个文件（比如 `/home/remzi/foo`，这是我们最喜欢用的虚构文件名之一），还必须访问一些目录。那么，LFS 是如何存储目录数据的呢?

好在目录结构和经典的 UNIX 文件系统基本相同：目录只是（名字，inode 号）映射的集合。例如，在磁盘上创建一个文件时，LFS 既要写入一个新的 inode、一些数据，也要写入引用这个文件的目录数据及其 inode。记住，LFS 会先把这些更新缓冲一段时间，再把它们顺序地写到磁盘上。因此，在某个目录中创建文件 `foo`，会在磁盘上产生下面这些新结构：




inode map 的这个片段里，同时包含目录文件 *dir* 和新创建的文件 *f* 的位置信息。因此，访问 inode 号为 *k* 的文件 `foo` 时，你会先在 inode map（通常缓存在内存中）里查到目录 *dir* 的 inode 在哪里（*A3*）；然后读取目录的 inode，它给出目录数据的位置（*A2*）；读取这个数据块，就得到名字到 inode 号的映射（`foo`, *k*）。接着再查一次 inode map，找到 inode 号 *k* 的位置（*A1*），最后读取地址 *A0* 处所需的数据块。

inode map 还解决了 LFS 中另一个严重的问题，叫作**递归更新问题**（recursive update problem）[Z+12]。任何从不原地更新、而是把更新移到磁盘上新位置的文件系统（比如 LFS），都会出现这个问题。

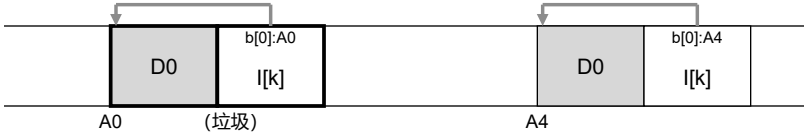
具体来说，每当更新一个 inode，它在磁盘上的位置就会改变。稍不注意，指向这个文件的目录也得跟着更新一次，而这又要求修改这个目录的父目录（parent directory），如此一路向上，直到文件系统树（file system tree）的根。

LFS 用 inode map 巧妙地避开了这个问题。尽管 inode 的位置可能会变，这种变化却永远不会反映到目录本身；相反，更新的是 imap 这个结构，而目录里保存的名字到 inode 号的映射保持不变。因此，借助间接层，LFS 避免了递归更新问题（recursive update problem）。

## 43.9 新的问题：垃圾回收

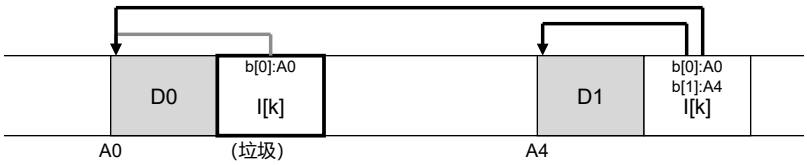
你可能已经注意到 LFS 的另一个问题：它不断地把文件的最新版本（包括 inode 和数据）写到磁盘上的新位置。这个过程虽然让写入保持高效，却意味着 LFS 会在磁盘各处留下文件结构的旧版本。我们（相当不客气地）把这些旧版本叫作**垃圾**（garbage）。

例如，假设有一个已存在的文件，inode 号为 $k$，它指向一个数据块 $D0$。现在我们更新这个块，产生一个新的 inode 和一个新的数据块。LFS 最终在磁盘上的布局大致是这样的（注意，为简单起见，我们省略了 imap 和其他结构；实际上还必须把一个新的 imap 片段写到磁盘上，指向新的 inode）：




从图中可以看到，inode 和数据块在磁盘上都有两个版本，一个是旧的（左边那个），一个是当前的、因而是**存活**（live）的（右边那个）。只是在逻辑上更新了一个数据块这么简单的操作，LFS 就必须持久保存若干个新结构，从而在磁盘上留下这些块的旧版本。

再举一个例子，假设我们改为往原来的文件 $k$ 追加一个块。在这种情况下，会生成 inode 的一个新版本，但这个 inode 仍然指向旧数据块。因此，旧数据块仍然是存活的，仍然是当前文件系统的一部分：




那么，我们该怎么处理这些较老版本的 inode、数据块等等呢？一种办法是把这些较老的版本保留下来，让用户可以恢复文件的旧版本（例如，用户不小心覆盖或删除了文件时，这样做会非常方便）；这样的文件系统叫作**版本化文件系统**（versioning file system），因为它记录着文件的不同版本。

然而，LFS 只保留文件最新的存活版本；因此，LFS 必须在后台定期找出文件数据、inode 和其他结构的这些已经死亡的旧版本，并**清理**（clean）它们；清理应该让磁盘上的块重新空闲出来，供后续写入使用。注意，清理其实是**垃圾回收**（garbage collection）的一种形式；有些编程语言会自动替程序释放不再使用的内存，用的也是这种技术。

前面我们讨论过段的重要性：正是段让 LFS 能够向磁盘大块地写入。其实，段对于有效清理也至关重要。想象一下，如果 LFS 的清理器（cleaner）在清理时只是逐个释放单个数据块、inode 等，会发生什么。结果是：文件系统在磁盘上的已分配空间之间夹杂着一些空闲的**空洞**（hole）。写入性能会大幅下降，因为 LFS 找不到一大片连续的区域来做顺序的高性能写入。

因此，LFS 的清理器是逐段工作的，从而为后续写入腾出大块的空间。基本的清理过程如下。LFS 清理器定期读入一些旧的（部分使用的）段，确定这些段中哪些块是存活的，然后写出一组新的段，里面只包含存活的块，从而释放旧段供写入使用。具体来说，我们希望清理器读入 $M$ 个已有的段，把它们的内容**紧凑**（compact）到 $N$ 个新段中（其中 $N < M$），

然后把这 $N$ 个段写到磁盘上的新位置。随后，旧的 $M$ 个段就被释放，文件系统可以用它们来做后续的写入。

不过，现在我们还剩下两个问题。第一个是机制（mechanism）问题：LFS 怎样判断段中的哪些块是存活的、哪些是死亡的？第二个是策略（policy）问题：清理器应该多久运行一次，应该挑选哪些段来清理？

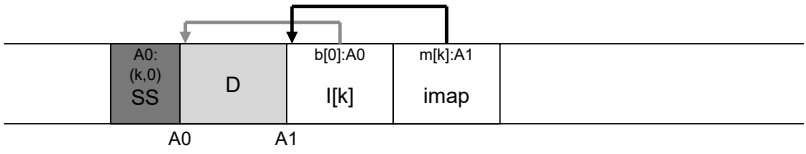
## 43.10 判断块是否存活

我们先来解决机制问题。给定磁盘段 $S$ 中的一个数据块 $D$，LFS 必须能判断 $D$ 是否存活。为此，LFS 给每个段加了一点额外的信息，用来描述每个块。具体来说，对于每个数据块 $D$，LFS 记录了它的 inode 号和偏移量（offset，即它是文件的第几个块）。这些信息记录在段开头的一个结构中，叫作**段摘要块**（segment summary block）。

有了这些信息，判断一个块是存活还是死亡就很简单了。对于位于磁盘地址 $A$ 的块 $D$，在段摘要块（segment summary block）中查出它的 inode 号 $N$ 和偏移量 $T$。接着，在 imap 中找到 $N$ 所在的位置，从磁盘读出 $N$（它也许已经在内存里了，那就更好了）。最后，用偏移量 $T$ 查看 inode 或某个间接块（indirect block），看看 inode 认为这个文件的第 $T$ 个块在磁盘上的什么位置。如果它恰好指向磁盘地址 $A$，LFS 就可以断定块 $D$ 是存活的。如果它指向别的地方，LFS 就可以断定 $D$ 已不再使用（即它是死亡的），从而知道不再需要这个版本了。下面是伪代码（pseudocode）总结：

```
(N, T) = SegmentSummary[A];
inode  = Read(imap[N]);
if (inode[T] == A)
    // 块 D 是存活的
else
    // 块 D 是垃圾
```

下图描述了这一机制，其中段摘要块（segment summary block，标为 *SS*）记录着地址 *A0* 处的数据块实际上属于文件 $k$，偏移量为 0。查看 imap 中的 $k$，就能找到这个 inode，并看到它确实指向那个位置。




LFS 还用了一些捷径，让判断存活的过程更高效。例如，文件被截断或删除时，LFS 会增加它的**版本号**（version number），并把新的版本号记录在 imap 中。LFS 同时也把版本号记录在磁盘上的段里，这样只要比较磁盘上的版本号和 imap 中的版本号，就能跳过上面描述的那种较长的检查，避免额外的读取。

## 43.11 策略问题：清理哪些块，何时清理?

在上述机制的基础上，LFS 还必须包含一组策略，来决定何时清理，以及哪些块值得清理。决定何时清理比较容易：可以定期清理，可以在空闲时清理，也可以在磁盘满了、不得不清理的时候清理。

决定清理哪些块则更有挑战性，许多研究论文都以此为题。在最初的 LFS 论文 [RO91] 中，作者描述了一种设法区分热段（hot segment）和冷段（cold segment）的方法。热段（hot segment）是指其中的内容经常被覆盖的段；因此，对于这样的段，最好的策略是等很长时间再去清理它，因为这期间会有越来越多的块被覆盖、写进新段，从而释放出来供使用。相比之下，冷段（cold segment）可能有少数几个死亡的块，但其余内容相对稳定。因此，作者得出的结论是，应该早点清理冷段（cold segment）、晚点清理热段（hot segment），并据此设计了一种启发式方法。然而，和大多数策略一样，这个策略并不完美；后来的方法展示了如何做得更好 [MR+97]。

## 43.12 崩溃恢复和日志

最后一个问题：如果系统在 LFS 写入磁盘时崩溃了，会怎样？你可能还记得上一章讲日志（journaling）时说过，更新过程中发生崩溃对文件系统来说很棘手，因此 LFS 也必须考虑这个问题。

在正常运行时，LFS 把写入缓冲在段中，等段满了或者过了一段时间，再把段写到磁盘上。LFS 把这些写入组织成一个**日志**（log），即检查点区域指向头段（head segment）和尾段（tail segment），每个段又指向下一个要写的段。LFS 还会定期更新检查点区域。显然，写段和写 CR 的过程中都可能发生崩溃。那么，LFS 如何处理写这些结构时发生的崩溃呢?

我们先说第二种情况。为了确保 CR 的更新是原子的，LFS 实际上保留了两个 CR，分别位于磁盘的两端，并交替写入它们。用指向 inode map 的最新指针和其他信息更新 CR 时，LFS 还实现了一个谨慎的协议（protocol）；具体来说，它先写出一个头部（带有时间戳），然后写 CR 的主体，最后再写一个块（同样带有时间戳）。如果系统在更新 CR 的过程中崩溃，LFS 能从前后两个时间戳不一致察觉出来。LFS 总是选择使用时间戳一致的最新的 CR，从而实现了 CR 的一致更新。

现在我们来解决第一种情况。由于 LFS 大约每 30 秒才写一次 CR，文件系统最后一个一致的快照（snapshot）可能已经相当旧了。因此，重启时，LFS 只需读入检查点区域、它所指向的 imap 片段以及后续的文件和目录，就能轻松恢复；但是，最后许多秒的更新会丢失。

为了改进这一点，LFS 会尝试用数据库领域所说的**前滚**（roll forward）技术，重建其中许多段。基本思想是：从最后一个检查点区域开始，找到日志（log）的末尾（它包含在 CR 中），然后据此读取后面的段，看看其中是否有有效的更新。如果有，LFS 就相应地更新文件系统，从而恢复自上一个检查点（checkpoint）以来写入的大部分数据和元数据。详见 Rosenblum 获奖的博士论文 [R92]。

提示：化缺陷为优点
每当你的系统有一个根本性的缺陷时，看看能不能把它反过来变成一个特性或者有用的东西。NetApp 的 WAFL 处理旧文件内容就是这样：它把旧版本提供给用户，因此不必那么频繁地操心清理（尽管它最终还是会在后台删掉旧版本）。这一绝妙的转折，既带来了一个很酷的特性，又消除了 LFS 清理问题的大半。系统中还有其他这样的例子吗？毫无疑问，但你得自己去想了，因为这一章已经结束了，而且是大写的“结束”。结束。完事。玩完了。我们撤了。再见！

## 43.13 小结

LFS 提出了一种更新磁盘的新方法。LFS 不原地覆盖文件，而总是写到磁盘上未使用的部分，以后再通过清理回收旧空间。这种方法在数据库系统中叫作**影子分页**（shadow paging）[L77]，在文件系统领域有时叫作**写时复制**（copy-on-write），它能让写入非常高效，因为 LFS 可以把所有更新收集到内存中的一个段里，然后一起顺序地写出去。

LFS 产生的大块写入，在许多不同的设备上都有出色的性能。在硬盘上，大块写入能把定位时间降到最低；在 RAID-4 和 RAID-5 这类基于奇偶校验（parity）的 RAID 上，它们完全避开了小写入问题（small-write problem）。最近的研究甚至表明，要在基于闪存（flash）的 SSD 上获得高性能，也需要大块的 I/O [HK+17]；因此，或许出人意料的是，LFS 风格的文件系统即使对这些新介质来说，也可能是一个极好的选择。

这种方法的缺点是会产生垃圾；数据的旧副本散落在磁盘各处，如果想回收这些空间供以后使用，就必须定期清理旧段。清理成了 LFS 中许多争议的焦点，对清理开销的担忧 [SS+95] 或许限制了 LFS 最初在这一领域的影响。然而，一些现代的商用文件系统，包括 NetApp 的 **WAFL** [HLM94]、Sun 的 **ZFS** [B07] 和 Linux **btrfs** [R+13]，甚至现代的**基于闪存的 SSD** [AD14]，都采用了类似的写时复制（copy-on-write）方法来写入磁盘，因此 LFS 的思想遗产在这些现代文件系统中得以延续。特别是，WAFL 把清理问题变成了一个特性，从而绕开了它：它用**快照**（snapshot）提供文件系统的旧版本，用户不小心删除了当前文件时，随时都可以访问旧文件。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[AD14] "Operating Systems: Three Easy Pieces" (Chapter: Flash-based Solid State Drives) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. Arpaci-Dusseau Books, 2014. *A bit gauche to refer you to another chapter in this very book, but who are we to judge?*

[B07] "ZFS: The Last Word in File Systems" by Jeff Bonwick and Bill Moore. Copy Available: http://www.ostep.org/Citations/zfs_last.pdf. *Slides on ZFS; unfortunately, there is no great ZFS paper (yet). Maybe you will write one, so we can cite it here?*

[HK+17] "The Unwritten Contract of of Solid State Drives" by Jun He, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. EuroSys '17, April 2017. *Which unwritten rules one must follow to extract high performance from an SSD? Interestingly, both request scale (large or parallel requests) and locality still matter, even on SSDs. The more things change …*

[HLM94] "File System Design for an NFS File Server Appliance" by Dave Hitz, James Lau, Michael Malcolm. USENIX Spring '94. *WAFL takes many ideas from LFS and RAID and puts it into a high-speed NFS appliance for the multi-billion dollar storage company NetApp.*

[L77] "Physical Integrity in a Large Segmented Database" by R. Lorie. ACM Transactions on Databases, Volume 2:1, 1977. *The original idea of shadow paging is presented here.*

[MJLF84] "A Fast File System for UNIX" by Marshall K. McKusick, William N. Joy, Sam J. Leffler, Robert S. Fabry. ACM TOCS, Volume 2:3, August 1984. *The original FFS paper; see the chapter on FFS for more details.*

[MR+97] "Improving the Performance of Log-structured File Systems with Adaptive Methods" by Jeanna Neefe Matthews, Drew Roselli, Adam M. Costello, Randolph Y. Wang, Thomas E. Anderson. SOSP 1997, pages 238-251, October, Saint Malo, France. *A more recent paper detailing better policies for cleaning in LFS.*

[M94] "A Better Update Policy" by Jeffrey C. Mogul. USENIX ATC '94, June 1994. *In this paper, Mogul finds that read workloads can be harmed by buffering writes for too long and then sending them to the disk in a big burst. Thus, he recommends sending writes more frequently and in smaller batches.*

[P98] "Hardware Technology Trends and Database Opportunities" by David A. Patterson. ACM SIGMOD '98 Keynote, 1998. Available online here: http://www.cs.berkeley.edu/~pattrsn/talks/keynote.html. *A great set of slides on technology trends in computer systems. Hopefully, Patterson will create another of these sometime soon.*

[R+13] "BTRFS: The Linux B-Tree Filesystem" by Ohad Rodeh, Josef Bacik, Chris Mason. ACM Transactions on Storage, Volume 9 Issue 3, August 2013. *Finally, a good paper on BTRFS, a modern take on copy-on-write file systems.*

[RO91] "Design and Implementation of the Log-structured File System" by Mendel Rosenblum and John Ousterhout. SOSP '91, Pacific Grove, CA, October 1991. *The original SOSP paper about LFS, which has been cited by hundreds of other papers and inspired many real systems.*

[R92] "Design and Implementation of the Log-structured File System" by Mendel Rosenblum. http://www.eecs.berkeley.edu/Pubs/TechRpts/1992/CSD-92-696.pdf. *The award-winning dissertation about LFS, with many of the details missing from the paper.*

[SS+95] "File system logging versus clustering: a performance comparison" by Margo Seltzer, Keith A. Smith, Hari Balakrishnan, Jacqueline Chang, Sara McMains, Venkata Padmanabhan. USENIX 1995 Technical Conference, New Orleans, Louisiana, 1995. *A paper that showed the LFS performance sometimes has problems, particularly for workloads with many calls to* `fsync()` *(such as database workloads). The paper was controversial at the time.*

[SO90] "Write-Only Disk Caches" by Jon A. Solworth, Cyril U. Orji. SIGMOD '90, Atlantic City, New Jersey, May 1990. *An early study of write buffering and its benefits. However, buffering for too long can be harmful: see Mogul [M94] for details.*

[Z+12] "De-indirection for Flash-based SSDs with Nameless Writes" by Yiying Zhang, Leo Prasath Arulraj, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '13, San Jose, California, February 2013. *Our paper on a new way to build flash-based storage devices, to avoid redundant mappings in the file system and FTL. The idea is for the device to pick the physical location of a write, and return the address to the file system, which stores the mapping.*

## 作业（模拟）

本节介绍 `lfs.py`，这是一个简单的 LFS 模拟器，可以帮你更好地理解基于 LFS 的文件系统是如何工作的。关于如何运行这个模拟器，请阅读 README。

## 问题

1. 运行 `./lfs.py -n 3`，也可以改变随机种子（`-s`）。你能推断出运行了哪些命令，才产生了最终的文件系统内容吗？你能说出这些命令是按什么顺序发出的吗？最后，你能判断出最终文件系统状态中每个块是否存活吗？用 `-o` 显示运行了哪些命令，用 `-c` 显示最终文件系统状态的存活情况。随着发出的命令数增加（即把 `-n 3` 改成 `-n 5`），这个任务对你来说会难多少？
2. 如果你觉得上面的题目很痛苦，可以显示每条命令引起的更新，给自己帮点忙。为此，运行 `./lfs.py -n 3 -i`。现在看看是否更容易理解每条命令是什么了。改变随机种子，得到不同的命令来解读（例如 `-s 1`、`-s 2`、`-s 3` 等）。
3. 为了进一步检验你能否判断出每条命令对磁盘做了哪些更新，运行 `./lfs.py -o -F -s 100`（也可以再试几个其他的随机种子）。这只会显示一组命令，**不会**显示文件系统的最终状态。你能推理出文件系统的最终状态一定是什么样子吗？
4. 现在，看看你能否判断出执行了一系列文件和目录操作之后，哪些文件和目录是存活的。运行 `./lfs.py -n 20 -s 1`，然后查看文件系统的最终状态。你能推断出哪些路径名（pathname）是有效的吗？运行 `./lfs.py -n 20 -s 1 -c -v` 查看结果。用 `-o` 运行，看看在给定的这一系列随机命令下，你的答案是否吻合。换用不同的随机种子，可以得到更多的题目。
5. 现在我们来发出一些具体的命令。首先，创建一个文件，并反复写入它。为此，使用 `-L` 选项，它可以让你指定要执行的具体命令。我们来创建文件 `/foo` 并写入它四次：`-L c,/foo:w,/foo,0,1:w,/foo,1,1:w,/foo,2,1:w,/foo,3,1 -o`。看看你能否判断出最终文件系统状态的存活情况；用 `-c` 检查你的答案。
6. 现在，我们来做同样的事，但用一次写操作代替四次。运行 `./lfs.py -o -L c,/foo:w,/foo,0,4` 来创建文件 `/foo`，并用一次写操作写入 4 个块。再计算一次存活情况，用 `-c` 检查你是否答对了。一次性写完一个文件（像这里这样）和一次写一个块的主要区别是什么？这说明了像真实的 LFS 那样在内存中缓冲更新有多重要？
7. 我们再来做一个具体的例子。首先，运行 `./lfs.py -L c,/foo:w,/foo,0,1`。这组命令做了什么？现在，运行 `./lfs.py -L c,/foo:w,/foo,7,1`。这组命令又做了什么？两者有什么不同？从这两组命令中，你能看出 inode 中的 `size` 字段有什么特点？
8. 现在我们来专门看看创建文件和创建目录的区别。依次运行模拟程序 `./lfs.py -L c,/foo` 和 `./lfs.py -L d,/foo`，分别创建一个文件和一个目录。这两次运行有什么相同之处，又有什么不同之处？

9. LFS 模拟器也支持硬链接（hard link）。运行下面的命令，研究它们是怎么工作的：`./lfs.py -L c,/foo:l,/foo,/bar:l,/foo,/goo -o -i`。
创建硬链接时会写出哪些块？这和创建一个新文件有什么相似之处，又有什么不同？创建链接时，引用计数（reference count）字段是如何变化的？

10. LFS 会做出许多不同的策略决定。其中大多数这里不会探讨（或许留待以后），但有一个简单的策略我们要看一看：inode 号的选择。首先，运行 `./lfs.py -p c100 -n 10 -o -a s`，看看采用“顺序”分配策略（allocation policy）时的通常行为，这种策略会尽量使用最接近零的空闲 inode 号。然后，运行 `./lfs.py -p c100 -n 10 -o -a r`，改用“随机”策略（`-p c100` 选项确保随机操作百分之百都是文件创建）。随机策略和顺序策略会在磁盘上造成什么样的差异？这说明在真实的 LFS 中，选择 inode 号有多重要？

11. 我们一直假设的最后一件事是，LFS 模拟器每次更新之后都会更新检查点区域。在真实的 LFS 中情况并非如此：为了避免长距离寻道，检查点区域是定期更新的。运行 `./lfs.py -N -i -o -s 1000`，看看在不强制把检查点区域写到磁盘时，一些操作以及文件系统的中间状态和最终状态。如果检查点区域从不更新，会发生什么？如果它定期更新呢？你能想出如何通过在日志中前滚（roll forward），把文件系统恢复到最新状态吗？

 © 2008–23, ARPACI-DUSSEAU

# 44

# 基于闪存的 SSD

硬盘驱动器（hard disk drive）一统天下几十年之后，一种新型的持久存储（persistent storage）设备最近变得举足轻重。这类设备统称为**固态存储**（solid-state storage），它们不像硬盘那样有机械部件或运动部件，而是完全由晶体管（transistor）构成，这一点和内存、处理器很像。然而，和典型的随机存取存储器（random-access memory，例如 DRAM）不同，这种**固态存储设备**（solid-state storage device，又称 **SSD**）断电（power loss）后仍能保留信息，因此非常适合用来持久地存储数据。

我们要关注的技术叫作**闪存**（flash），更准确地说是**基于 NAND 的闪存**（NAND-based flash），它由 Fujio Masuoka 在 20 世纪 80 年代发明 [M+14]。我们会看到，闪存有一些独特的性质。例如，要写入其中某个单位（即**闪存页**，flash page），必须先擦除（erase）一个更大的单位（即**闪存块**，flash block），而擦除的代价可能相当高。此外，对同一页（page）写得太频繁，会让它**磨损**（wear out）。这两个性质让构建基于闪存的 SSD 成了一项有趣的挑战：

关键问题：如何构建基于闪存的 SSD
如何构建基于闪存的 SSD？如何应对擦除代价高昂的问题？既然反复覆写会让设备磨损，如何造出能长久使用的设备？技术进步的脚步会停下来吗？或者，它会不再令人惊叹吗？

## 44.1 存储单个位

闪存芯片（flash chip）的设计，是在单个晶体管中存储一个或多个位；晶体管中困住的电荷量被映射成一个二进制值。在**单层单元**（single-level cell，**SLC**）闪存中，一个晶体管只存一个位（即 1 或 0）；而在**多层单元**（multi-level cell，**MLC**）闪存中，两个位被编码成不同的电荷水平，例如用低、偏低、偏高、高四个水平分别表示 00、01、10、11。甚至还有三**层单元**（triple-level cell，**TLC**）闪存，每个单元编码 3 个位。总的来说，SLC 芯片性能更高，价格也更贵。

当然，要说清这种位级存储到底怎样运作，还得深入到器件物理的层面，细节很多。这些超出了本书的范围，你可以自己去读更多资料 [J10]。

提示：术语要小心使用

你可能已经注意到，我们以前多次用过的一些术语，比如块（block）和页，也用在了闪存的语境下，只是用法和以前略有不同。新术语并不是为了故意给你添麻烦才造出来的（虽然效果可能正是如此），而是因为术语并没有一个中央权威来拍板。对你来说是块的东西，在别人那里可能是页，反之亦然，这取决于语境。你的任务很简单：了解每个领域里恰当的术语，用的时候让精通这门学科的人听得懂你在说什么。这种时候，唯一的办法简单却有时很痛苦：靠记性。

## 44.2 从位到 bank/平面

正如古希腊人所说，存一个位（或几个位），成不了存储系统。因此，闪存芯片（flash chip）由一个个 **bank**（或者叫**平面**，plane）组成，每个 bank 又包含大量单元。

访问一个 bank 时，有两种不同大小的单位：一种是**块**（block，有时也叫**擦除块**，erase block），大小通常是 128 KB 或 256 KB；另一种是**页**（page），大小为几 KB（例如 4 KB）。每个 bank 中有大量的块，每个块中又有大量的页。考虑闪存时，你必须记住这套新术语，它和我们在磁盘、RAID 中说的块，以及在虚拟内存（virtual memory）中说的页都不一样。

图 44.1 展示了一个包含块和页的闪存平面（flash plane）示例；在这个简单的例子中，有三个块，每个块包含四页。下面我们会看到为什么要区分块和页；其实，这一区别对读写之类的闪存操作至关重要，对设备的整体性能更是如此。你将学到的最重要（也最奇怪）的一点是：要写入块中的某一页，必须先擦除整个块。正是这个棘手的细节，让构建基于闪存的 SSD 成为一项有趣且值得做的挑战，这也是本章后半部分的主题。

## 44.3 闪存的基本操作

有了这样的闪存组织方式，闪存芯片（flash chip）支持三种底层操作。**读取**（read）命令用来从闪存中读出一页；**擦除**（erase）和**编程**（program）配合起来完成写入。具体如下：

- **读取（一页）：** 闪存芯片（flash chip）的客户端可以读取任何一页，只需向设备发出读取命令并指定相应的页号（page number）即可。这个操作通常相当快，大约几十微秒，与它在设备上的位置无关，而且（大体上）也与上一个请求的位置无关（这一点和磁盘截然不同）。能以同样快的速度访问任何位置，意味着这种设备是一种**随机访问**（random access）设备。
- **擦除（一个块）：** 由于设备的特性，在写入闪存中的某一页（page）之前，你必须先**擦除**（erase）这一页所在的整个块（block）。重要的是，擦除会破坏块中的内容（把每个位都置为 1）；因此，执行擦除之前，你必须确保块中你在乎的数据都已经复制到别

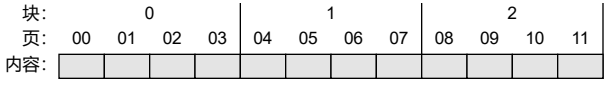



图 44.1：**一个简单的闪存芯片：块中的页**

 © 2008–23, ARPACI-DUSSEAU

处，比如内存或者另一个闪存块（flash block）中。擦除命令的代价相当高，需要几毫秒才能完成。擦除完成后，整个块被重置，其中每一页都可以编程了。

- **编程（一页）：** 块被擦除后，就可以用编程命令把页中的一些 1 改成 0，从而把一页所需的内容写入闪存。编程一页的代价比擦除一个块低，但比读取一页高，在现代闪存芯片（flash chip）上通常需要几百微秒左右。

可以这样理解闪存芯片（flash chip）：每一页都有一个与之关联的状态。页一开始处于 `INVALID`（无效）状态。擦除页所在的块之后，这一页（以及块中所有页）的状态就变为 `ERASED`（已擦除），这会重置块中每一页的内容，而且（重要的是）让它们变得可以编程。对一页编程之后，它的状态变为 `VALID`（有效），表示它的内容已经设定好，可以读取了。读取不影响这些状态（不过你应该只读取编程过的页）。一页一旦编程过，要改变它的内容，唯一的办法就是擦除它所在的整个块。下面这个例子展示了一个 4 页的块经过各种擦除和编程操作后的状态变化：

```
                    iiii  初始：块中各页都无效（i）
Erase()       →     EEEE  块中各页的状态被设为已擦除（E）
Program(0)    →     VEEE  编程第 0 页；状态被设为有效（V）
Program(0)    →     出错  页编程之后不能再次编程
Program(1)    →     VVEE  编程第 1 页
Erase()       →     EEEE  内容已擦除；所有页都可以编程
```

### 详细示例

写入的过程，也就是擦除和编程，非常特别，因此我们来看一个详细的例子，确保你弄懂了。在这个例子中，假设一个 4 页的块中有下面四个 8 位的页（页和块都小得不现实，但用在这个例子里很合适）；每一页之前都编程过，因此都处于 `VALID` 状态。

| 页 0 | 页 1 | 页 2 | 页 3 |
|---|---|---|---|
| 00011000 | 11001110 | 00000001 | 00111111 |
| VALID | VALID | VALID | VALID |

现在，假设我们要写第 0 页，填入新的内容。要写任何一页，都必须先擦除整个块。假设我们这样做了，块就处于这样的状态：

| 页 0 | 页 1 | 页 2 | 页 3 |
|---|---|---|---|
| 11111111 | 11111111 | 11111111 | 11111111 |
| ERASED | ERASED | ERASED | ERASED |

好消息！现在我们可以接着对第 0 页编程了，例如写入内容 `00000011`，按需要覆盖旧的第 0 页（内容为 `00011000`）。完成之后，块是这个样子：

| 页 0 | 页 1 | 页 2 | 页 3 |
|---|---|---|---|
| 00000011 | 11111111 | 11111111 | 11111111 |
| VALID | ERASED | ERASED | ERASED |

 © 2008–23, ARPACI-DUSSEAU

| 设备 | 读取 (µs) | 编程 (µs) | 擦除 (µs) |
|---|---|---|---|
| SLC | 25 | 200–300 | 1500–2000 |
| MLC | 50 | 600–900 | ~3000 |
| TLC | ~75 | ~900–1350 | ~4500 |

图 44.2：**原始闪存的性能特征**

坏消息来了：第 1、2、3 页之前的内容全都没了！因此，覆写块内的任何一页之前，必须先把我们在乎的数据移到别的地方，例如内存或闪存上的其他位置。擦除的这种特性会深刻影响基于闪存的 SSD 的设计，我们很快就会看到。

### 小结

总的来说，读一页很容易：直接读就行。闪存芯片（flash chip）读得又好又快，随机读性能有可能远远超过现代磁盘驱动器（disk drive），后者因为机械寻道（seek）和旋转的开销而很慢。

写一页就麻烦一些：必须先擦除整个块，还要注意先把我们在乎的数据移到别处，然后再对所需的页编程。这样做不仅代价高，而且频繁重复这种编程/擦除循环（program/erase cycle），还会导致闪存芯片（flash chip）最大的可靠性（reliability）问题：**磨损**（wear out）。用闪存设计存储系统时，写入的性能和可靠性是关注的核心。我们很快会进一步了解现代 SSD 如何应对这些问题，在这些限制之下仍然提供出色的性能和可靠性。

## 44.4 闪存的性能和可靠性

我们关心的是用原始闪存芯片（flash chip）构建存储设备（storage device），因此有必要了解它们的基本性能特征。图 44.2 粗略汇总了大众媒体上的一些数据 [V12]。其中，作者给出了 SLC、MLC 和 TLC 闪存在读取、编程和擦除上的基本操作延迟；这三种闪存每个单元分别存储 1、2、3 位信息。

从表中可以看出，读取延迟相当低，只要几十微秒就能完成。编程延迟更高，波动也更大，SLC 低至 200 微秒，但每个单元塞进的位越多，延迟就越高；要获得良好的写入性能，就得并行地（in parallel）使用多个闪存芯片（flash chip）。最后，擦除的代价相当高，通常需要几毫秒。如何应对这一开销，是现代闪存存储设计的核心问题。

现在来考虑闪存芯片（flash chip）的可靠性。机械磁盘可能因为各种各样的原因发生故障，包括可怕而又十分“物理”的**磁头碰撞**（head crash），即磁头实际接触到了记录面；而闪存芯片（flash chip）是纯硅的，从这个意义上说，需要担心的可靠性问题要少一些。主要的问题是**磨损**（wear out）：闪存块（flash block）每擦除和编程一次，都会慢慢积累一点额外的电荷。随着时间推移，额外的电荷越积越多，区分 0 和 1 就越来越难。到了无法区分的地步，这个块就不能用了。

块的典型寿命目前还不是很清楚。厂商标称基于 MLC 的块寿命为 10,000 次 P/E（program/erase，编程/擦除）周期，也就是说，每个块在失效之前可以擦除和编程 10,000

提示：向后兼容的重要性
在分层的系统中，**向后兼容**（backwards compatibility）始终是需要考虑的问题。在两个系统之间定义稳定的接口，就能在保证双方持续互通的同时，让接口两边各自创新。这种做法在许多领域都相当成功：操作系统为应用程序提供相对稳定的 API，磁盘为文件系统（file system）提供同样的基于块的接口，IP 网络协议栈（protocol stack）的每一层都为上一层提供固定不变的接口。
当然，这种僵化也有缺点：上一代定义的接口未必适合下一代。有时候，不妨考虑把整个系统彻底重新设计一遍。Sun 的 ZFS 文件系统就是一个绝佳的例子 [B07]：ZFS 的设计者重新审视了文件系统和 RAID 之间的交互，构想出（并最终实现了）一个更高效的整体。

次。基于 SLC 的芯片每个晶体管只存一个位，因此标称寿命更长，通常是 100,000 次 P/E 周期。不过，最近的研究表明，实际寿命比预期的长得多 [BD10]。

闪存芯片（flash chip）的另一个可靠性问题叫作**干扰**（disturbance）。访问闪存中的某一页时，相邻页中的一些位可能会发生翻转；根据这一页是在读取还是在编程，这种位翻转分别称为**读干扰**（read disturb）或**编程干扰**（program disturb）。

## 44.5 从原始闪存到基于闪存的 SSD

对闪存芯片（flash chip）有了基本了解之后，我们面临下一个任务：如何把一组基本的闪存芯片变成看起来像典型存储设备的东西。标准的存储接口是一个简单的基于块的接口：给定块地址，就可以读写大小为 512 字节（或更大）的块，也就是扇区（sector）。基于闪存的 SSD 的任务，就是在它内部的原始闪存芯片（flash chip）之上，提供这种标准的块接口。

在内部，SSD 由若干闪存芯片（flash chip）组成，用于持久存储。SSD 还包含一定数量的易失性（volatile，即非持久的）内存（例如 SRAM）；这种内存可以用来缓存（cache）和缓冲数据，也可以存放映射表（mapping table），我们下面会讲到。最后，SSD 还包含控制逻辑，用来协调设备的各项操作。细节请参阅 Agrawal 等人的论文 [A+08]；简化的框图见图 44.3（第 505 页）。

这一控制逻辑的基本功能之一，是满足客户端的读写请求，并根据需要把它们转换成内部的闪存操作。**闪存转换层**（flash translation layer，**FTL**）提供的正是这一功能。FTL 接收针对*逻辑块*（logical block，它们构成设备接口）的读写请求，把它们转换成对*物理块*（physical block）和*物理页*（physical page）的底层读取、擦除和编程命令；物理块和物理页构成了实际的闪存设备。FTL 完成这项任务时，应当以提供出色的性能和高可靠性为目标。

我们会看到，要获得出色的性能，需要组合多种技术。关键之一是**并行**（in parallel）使用多个闪存芯片（flash chip）；虽然我们不会再进一步讨论这一技术，但你只需知道，所有现代 SSD 内部都使用多个芯片来获得更高的性能。另一个性能目标是减少**写放大**（write amplification），它的定义是：FTL 向闪存芯片（flash chip）发出的写流量总量（以字节计），

 © 2008–23, ARPACI-DUSSEAU

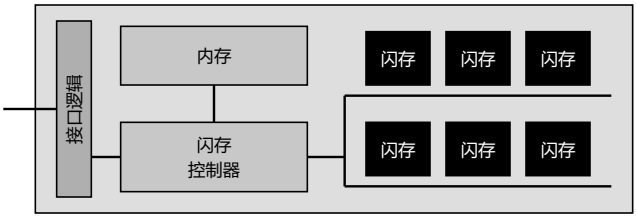



图 44.3：**基于闪存的 SSD：逻辑框图**

除以客户端向 SSD 发出的写流量总量（以字节计）。我们下面会看到，构建 FTL 的幼稚方法会导致很高的写放大（write amplification）和很低的性能。

高可靠性则要结合几种不同的方法来实现。如前所述，一个主要问题是**磨损**（wear out）。如果某个块被擦除和编程得太频繁，它就会无法使用。因此，FTL 应该尽量把写操作均匀地分散到闪存的各个块上，让设备中所有块大致同时磨损。这种做法叫作**磨损均衡**（wear leveling），是所有现代 FTL 必不可少的组成部分。

另一个可靠性问题是编程干扰（program disturb）。为了尽量减少这种干扰，FTL 通常会在已擦除的块中按顺序编程，从低页号（page number）到高页号。这种按顺序编程的方法能把干扰降到最低，用得很普遍。

## 44.6 FTL 的组织方式：一种糟糕的方法

FTL 最简单的组织方式，我们称之为**直接映射**（direct mapped）。在这种方法中，对逻辑页（logical page） $N$ 的读取直接映射为对物理页 $N$ 的读取。对逻辑页（logical page） $N$ 的写入就复杂一些了：FTL 首先要读入页 $N$ 所在的整个块，然后擦除这个块，最后对旧页和新页编程。

你大概能猜到，直接映射的 FTL 在性能和可靠性两方面都有很多问题。性能问题出在每一次写入上：设备必须读入整个块，擦除它（代价相当高），再对它编程。最终结果是严重的写放大（write amplification，与一个块中的页数成正比），写性能因此极差，甚至比有机械寻道和旋转延迟（rotational delay）的普通硬盘还慢。

这种方法的可靠性就更糟了。如果文件系统元数据（metadata）或用户文件数据被反复覆写，同一个块就会被一遍又一遍地擦除和编程，很快就会磨损，还可能丢失数据。直接映射的方法把磨损的控制权过多地交给了客户端的工作负载（workload）；如果工作负载没有把写负载均匀地分散到各个逻辑块上，存放热门数据的底层物理块就会很快磨损。无论出于可靠性还是性能的考虑，直接映射的 FTL 都是个坏主意。

## 44.7 日志结构的 FTL

出于这些原因，如今大多数 FTL 都是**日志结构**（log structured）的。这种思想既适用于存储设备（我们现在就会看到），也适用于构建在它们之上的文件系统（例如**日志结构文件系统**，log-structured file system）。写入逻辑块 $N$ 时，设备把这次写入追加到当前正在写入的块中的下一个空闲位置；我们把这种写入方式称为**日志**（logging）。为了支持之后对

块 *N* 的读取，设备会维护一张**映射表**（mapping table，保存在内存中，同时以某种形式持久地保存在设备上）；这张表记录了系统中每个逻辑块的物理地址（physical address）。

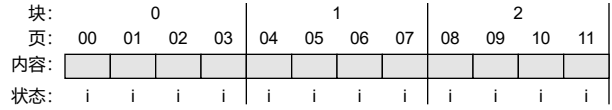
我们来看一个例子，确认自己弄懂了这种基本的基于日志的方法。对客户端来说，这个设备看起来就像一块普通的磁盘，可以读写 512 字节的扇区或扇区组。为简单起见，假设客户端以 4 KB 为单位读写。再假设 SSD 包含大量 16 KB 大小的块，每个块分成四个 4 KB 的页；这些参数并不现实，因为闪存块（flash block）通常包含更多的页，但很适合用来讲解。

假设客户端发出如下操作序列：

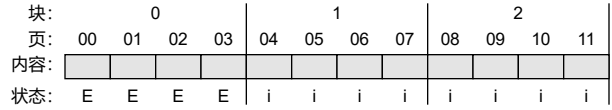
- Write(100)，内容为 `a1`
- Write(101)，内容为 `a2`
- Write(2000)，内容为 `b1`
- Write(2001)，内容为 `b2`

SSD 的客户端（例如文件系统）用这些**逻辑块地址**（logical block address，例如 100）来记住信息存放在哪里。

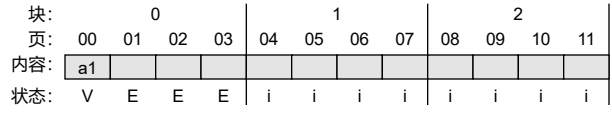
在内部，设备必须把这些块写入转换成原始硬件支持的擦除和编程操作，并以某种方式为每个逻辑块地址（logical block address）记录：SSD 的哪个**物理页**（physical page）存放着它的数据。假设 SSD 的所有块当前都无效，在对任何页编程之前都必须先擦除。下面是 SSD 的初始状态，所有页都标记为 `INVALID (i)`：



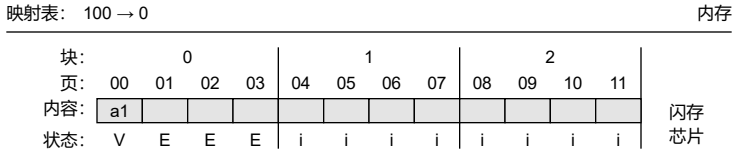

SSD 收到第一个写请求，即写逻辑块 100 时，FTL 决定把它写到物理块 0；物理块 0 包含四个物理页：0、1、2、3。因为这个块还没有擦除，所以暂时不能写入；设备必须先向块 0 发出擦除命令。这样就得到下面的状态：




块 0 现在可以编程了。大多数 SSD 会按顺序（即从低到高）写页，以减少与**编程干扰**（program disturb）相关的可靠性问题。接着，SSD 把逻辑块 100 的写入导向物理页 0：

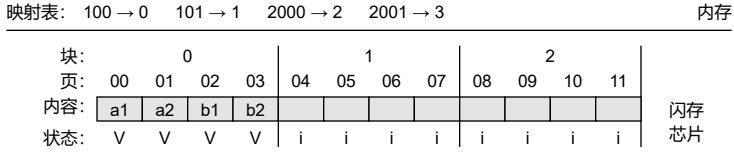



但如果客户端想读逻辑块 100 呢？它怎么找到数据在哪里？SSD 必须把对逻辑块 100 的读取转换成对物理页 0 的读取。为了支持这一功能，FTL 把逻辑块 100 写入物理页 0 时，会把这一事实记录在一张**内存中的映射表**（in-memory mapping table）里。我们也会在示意图中跟踪这张映射表（mapping table）的状态：




现在你可以看到客户端写 SSD 时会发生什么了。SSD 为这次写入找一个位置，通常就是选下一个空闲页；然后用块的内容对这一页编程，并在映射表（mapping table）中记录逻辑到物理的映射。之后的读取只需借助这张表，把客户端给出的逻辑块地址（logical block address）**转换**（translate）成读取数据所需的物理页号即可。

我们来看示例写入流中剩下的写操作：101、2000 和 2001。写完这些块后，设备的状态如下：




基于日志的方法天然就能提高性能：只是偶尔才需要擦除，并且完全避免了直接映射方法中代价高昂的“读取—修改—写入”；它还能大大提升可靠性。FTL 现在可以把写操作分散到所有页上，也就是所谓的**磨损均衡**（wear leveling），从而延长设备的寿命；我们会在后面进一步讨论磨损均衡（wear leveling）。

可惜的是，这种基本的日志结构（log-structured）方法也有一些缺点。第一，覆写逻辑块会产生我们所说的**垃圾**（garbage），也就是散落在设备各处、占用空间的旧版本数据。设备必须定期执行**垃圾回收**（garbage collection，**GC**），找出这些块并为将来的写入腾出空间；过多的垃圾回收会推高写放大（write amplification），降低性能。第二，内存中的映射表（mapping table）代价很高；设备越大，这些表需要的内存就越多。下面我们依次讨论这两个问题。

补充：FTL 映射信息的持久化

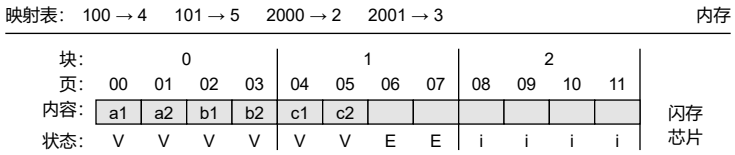
你可能在想：如果设备断电了会怎样？内存中的映射表（mapping table）会消失吗？显然，这些信息不能真的丢失，否则设备就无法作为持久存储设备工作了。SSD 必须有办法恢复映射信息。

最简单的做法，是在每一页上记录一些映射信息，记在所谓的**带外**（out-of-band，**OOB**）区域中。设备断电并重启后，必须扫描各个 OOB 区域，在内存中重建映射表（mapping table）。这种基本方法有它的问题：扫描一块大容量 SSD 来找出所有必要的映射信息，速度很慢。为了克服这一局限，一些高端设备使用更复杂的**日志**（logging）和**检查点**（checkpointing）技术来加快恢复；要了解日志（logging）的更多内容，请阅读有关崩溃一致性（crash consistency）和日志结构文件系统（log-structured file system）的章节 [AD14a]。

## 44.8 垃圾回收

像这样的日志结构（log-structured）方法，第一个代价是会产生垃圾，因此必须做**垃圾回收**（garbage collection），即回收无效块（dead block）。我们用前面的例子来说明这一点。回想一下，逻辑块 100、101、2000 和 2001 已经写入了设备。

现在假设块 100 和 101 又各写了一次，内容分别是 `c1` 和 `c2`。这两次写入落在下一个空闲页，本例中是物理页 4 和 5；映射表（mapping table）也相应更新。注意，设备必须先擦除块 1，才能这样编程：




现在的问题应该很明显了：物理页 0 和 1 虽然标记为 `VALID`，里面存的却是**垃圾**（garbage），即块 100 和 101 的旧版本。由于设备是日志结构（log-structured）的，覆写会产生垃圾块（garbage block），设备必须回收它们，才能为新的写入提供空闲空间（free space）。

找出垃圾块（garbage block，也称**无效块**，dead block）并回收以供将来使用的过程，叫作**垃圾回收**（garbage collection），它是所有现代 SSD 的重要组成部分。基本过程很简单：找到一个包含一个或多个垃圾页（garbage page）的块，从中读出有效（非垃圾）页，把这些有效页写到日志（log）中，最后回收整个块，用于写入。

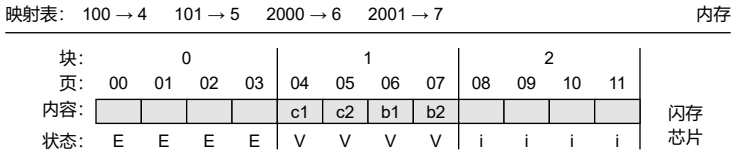
下面举例说明。设备决定回收上面块 0 中的无效页（dead page）。块 0 中有两个无效页（页 0 和 1）和两个有效页（页 2 和 3，分别存放着块 2000 和 2001）。为此，设备将：

- 从块 0 读出有效数据（页 2 和 3）
- 把有效数据写到日志末尾
- 擦除块 0（释放它，以供之后使用）

垃圾回收器（garbage collector）要能工作，每个块中都必须有足够的信息，让 SSD 判断每一页是有效还是无效。一种自然的做法，是在每个块的某处记下块中每一页存的是哪个逻辑块。然后，设备就可以利用映射表（mapping table），判断块中的每一页是否存放着有效数据。

在上面的例子中，垃圾回收发生之前，块 0 存放着逻辑块 100、101、2000、2001。这时映射表（mapping table）中是 100→4、101→5、2000→2、2001→3，设备查一下映射表，就很容易判断这个 SSD 块中的每一页是否存放着有效信息。例如，映射表（mapping table）显然仍指向页 2 和 3；页 0 和 1 则不然，因此它们是垃圾回收的候选对象。

在我们的例子中，这次垃圾回收完成后，设备的状态如下：



补充：一种新的存储 API：trim

说到硬盘，我们通常只会想到读写它的最基本的接口：读和写。通常还有某种**缓存刷新**（cache flush）命令，确保写入的数据真正持久化，不过为简单起见，我们有时会略去它。对日志结构（log-structured）的 SSD，乃至任何维护着灵活多变的逻辑到物理块映射（logical-to-physical block mapping）的设备来说，有一个新接口很有用，叫作 **trim** 操作。

trim 操作接收一个地址（可能还有一个长度），只是告诉设备：这个地址（和长度）所指定的块已被删除；这样，设备就不必再记录与这一地址范围有关的任何信息了。对标准硬盘来说，trim 并不是特别有用，因为硬盘中块地址（block address）到具体盘片（platter）、磁道（track）和扇区的映射是静态的。然而，对日志结构（log-structured）的 SSD 来说，知道某个块不再需要了是非常有用的，因为 SSD 可以从 FTL 中删除这条信息，之后在垃圾回收时再回收这块物理空间。

虽然我们有时会把接口和实现看作两个独立的东西，但从这个例子可以看到，是实现塑造了接口。在映射复杂的情况下，知道哪些块不再需要，就能让实现更高效。

可以看到，垃圾回收的代价可能很高，需要读取并重写有效数据。最理想的回收对象，是只包含无效页（dead page）的块；这样的块可以立即擦除，用来存放新数据，无需代价高昂的数据迁移（migration）。

为了降低 GC 的开销，一些 SSD 会为设备**预留空间**（overprovision）[A+08]：增加额外的闪存容量之后，清理工作就可以推迟，放到**后台**（background）进行，也许挑设备不太忙的时候做。增加容量还能提高内部带宽（bandwidth），这些带宽可以用于清理，从而不影响客户端感知到的带宽。许多现代 SSD 都以这种方式预留空间（over-provisioning），这是获得出色整体性能的关键之一。

## 44.9 映射表的大小

日志结构（log-structured）方法的第二个代价是映射表（mapping table）可能非常大，设备的每个 4 KB 页都要占一项。例如，一块 1 TB 的大容量 SSD，如果每个 4 KB 页占一个 4 字节的表项，光这些映射就需要 1 GB 内存！因此，这种**页级**（page-level）FTL 方案是不切实际的。

### 基于块的映射

降低映射开销的一种方法，是对设备的每个块（block）只保存一个指针，而不是每页一个，这样映射信息的量就缩小为原来的 $1/\left(\frac{Size_{block}}{Size_{page}}\right)$。这种**块级**（block-level）FTL 类似于在虚拟内存系统中使用更大的页：那样的话，VPN 用的位数更少，每个虚拟地址（virtual address）中的偏移量（offset）更大。

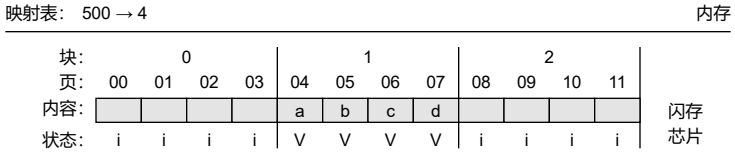
可惜的是，在基于日志的 FTL 中，基于块的映射性能并不好。最大的问题出现在“小写入”时（即写入量小于一个物理块的大小）。这种情况下，FTL 必须从旧块中读出大量有

效数据，把它们（连同这次小写入的数据）复制到一个新块中。这种数据复制会大大增加写放大（write amplification），从而降低性能。

为了把这个问题说得更清楚，我们来看一个例子。假设客户端之前写入了逻辑块 2000、2001、2002 和 2003（内容分别为 a、b、c、d），它们位于物理块 1 中的物理页 4、5、6、7。如果按页映射，转换表必须为这些逻辑块记录四个映射：2000→4、2001→5、2002→6、2003→7。

如果改用块级（block-level）映射，FTL 只需要为所有这些数据记录一个地址转换（address translation）。不过，这里的地址映射和前面的例子略有不同。具体来说，我们把设备的逻辑地址空间（logical address space）看作被切成了一个个 chunk，每个 chunk 的大小和闪存中物理块的大小相同。这样，逻辑块地址（logical block address）就由两部分组成：chunk 号和偏移量。由于我们假设每个物理块能放下四个逻辑块，逻辑地址（logical address）的偏移量部分需要 2 位；其余的（最高的）那些位构成 chunk 号。

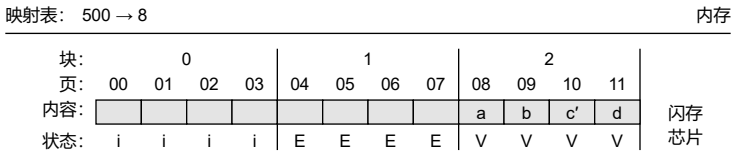
逻辑块 2000、2001、2002 和 2003 的 chunk 号都相同（500），偏移量各不相同（分别是 0、1、2、3）。因此，使用块级（block-level）映射时，FTL 记录的是 chunk 500 映射到块 1（从物理页 4 开始），如下图所示：




在基于块的 FTL 中，读取很容易。首先，FTL 从客户端给出的逻辑块地址（logical block address）中取出最高的几位，得到 chunk 号。然后，FTL 在表中查找 chunk 号到物理页的映射。最后，FTL 把逻辑地址（logical address）中的偏移量加到块的物理地址上，算出所需闪存页（flash page）的地址。

例如，如果客户端读取逻辑地址（logical address） 2002，设备会取出逻辑 chunk 号（500），在映射表（mapping table）中查找对应的转换（得到 4），再把逻辑地址中的偏移量（2）加到这个转换结果（4）上。得到的物理页地址（6）就是数据所在的位置；FTL 随后就可以向这个物理地址发出读取，得到所需的数据（c）。

但如果客户端写逻辑块 2002（内容为 c'）呢？这种情况下，FTL 必须读入 2000、2001 和 2003，然后把四个逻辑块全部写到一个新位置，并相应地更新映射表（mapping table）。之后，块 1（原来存放数据的地方）就可以擦除并重新使用了，如下所示：




从这个例子可以看出，块级（block-level）映射虽然大大减少了转换所需的内存，但当写入小于设备的物理块大小时，会导致严重的性能问题；由于实际的物理块可能有 256 KB 甚至更大，这样的写入很可能会经常发生。因此，需要更好的解决方案。你感觉到了吗？本章接下来就要告诉你这个方案是什么了。不过最好先别往下读：你能自己想出来吗？

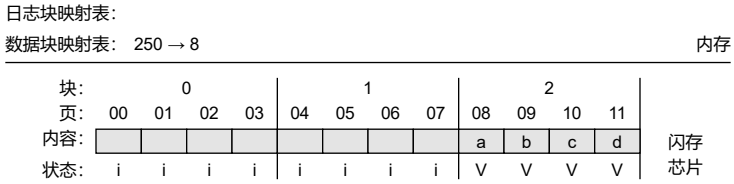
### 混合映射

为了既能灵活地写入，又能降低映射开销，许多现代 FTL 采用了**混合映射**（hybrid mapping）技术。采用这种方法时，FTL 会保留几个已擦除的块，把所有写入都导向这些块；它们称为**日志块**（log block）。由于 FTL 希望能把任何一页写到日志块（log block）中的任何位置，而不必像纯粹的块映射那样做大量复制，因此它为这些日志块保留逐页的映射。

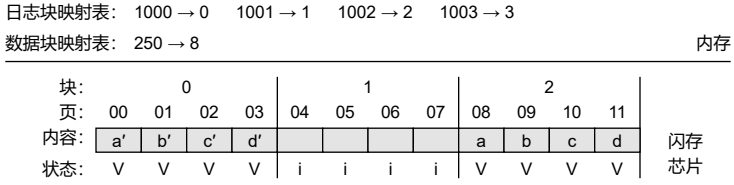
这样，FTL 的内存中在逻辑上有两类映射表（mapping table）：一小组逐页映射，放在我们所说的*日志块映射表*（log table）中；一大组逐块映射，放在*数据块映射表*（data table）中。查找某个逻辑块时，FTL 会先查日志块映射表（log table）；如果在那里找不到这个逻辑块的位置，FTL 再查数据块映射表（data table）找到它的位置，然后访问所请求的数据。

混合映射（hybrid mapping）策略的关键，是把日志块（log block）的数量控制得很少。为此，FTL 必须定期检查日志块（它们每页一个指针），把它们转换成只需一个块指针就能指向的块。根据块的内容，这种转换主要有三种做法 [KK+02]。

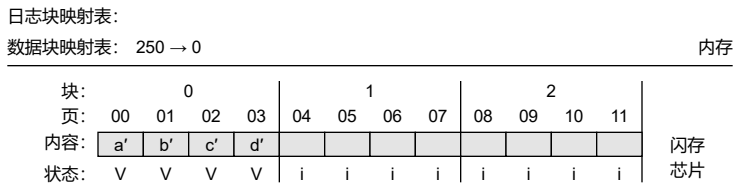
例如，假设 FTL 之前写入了逻辑页（logical page） 1000、1001、1002 和 1003，把它们放在物理块 2 的物理页 8、9、10、11 中，内容分别是 `a`、`b`、`c` 和 `d`。




现在假设客户端以完全相同的顺序覆写这几个块（数据为 `a'`、`b'`、`c'` 和 `d'`），写到当前可用的一个日志块（log block）中，比如物理块 0 的物理页 0、1、2、3。这时，FTL 的状态如下：

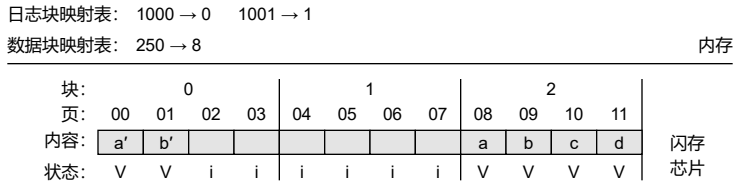



由于这几个块的写入方式和之前完全相同，FTL 可以执行所谓的**交换合并**（switch merge）。这时，日志块（0）成了块 0、1、2、3 的存储位置，由一个块指针指向；旧块（2）则被擦除，用作日志块（log block）。在这种最好的情况下，原来需要的所有逐页指针都被一个块指针取代了。




交换合并（switch merge）是混合 FTL 的最佳情况。可惜的是，FTL 有时没这么走运。设想这样一种情况：初始条件相同（逻辑块 1000 … 1003 存放在物理块 2 中），但随后客户端覆写的是逻辑块 1000 和 1001。

你觉得这种情况下会发生什么? 为什么它更难处理?（先想一想，再看下一页的结果）




为了把这个物理块的其他页重新凑到一起，从而只用一个块指针就能指向它们，FTL 会执行所谓的**部分合并**（partial merge）。在这个操作中，逻辑块 1002 和 1003 从物理块 2 中读出，然后追加到日志（log）中。SSD 最终的状态和上面交换合并（switch merge）的结果相同；但在这种情况下，FTL 必须执行额外的 I/O 才能达到目的，因此增加了写放大（write amplification）。

FTL 遇到的最后一种情况叫作**全合并**（full merge），需要做的工作更多。在这种情况下，FTL 必须从许多其他块中收集页来完成清理。例如，假设逻辑块 0、4、8 和 12 被写入了日志块（log block） A。要把这个日志块（log block）转换成块映射的页，FTL 必须先创建一个包含逻辑块 0、1、2、3 的数据块（data block），因此要从别处读出 1、2、3，再把 0、1、2、3 一起写出。接下来，合并操作还必须对逻辑块 4 做同样的事，找到 5、6、7，把它们合到一个物理块中。逻辑块 8 和 12 也必须这样处理，然后（终于）才能释放日志块（log block） A。可想而知，频繁的全合并（full merge）会严重损害性能，因此应当尽可能避免 [GY+09]。

### 页映射加缓存

鉴于上面的混合方法很复杂，有人提出了更简单的办法，来降低页映射 FTL（page-mapped FTL）的内存负担。最简单的大概就是只把 FTL 中活跃的部分缓存在内存中，从而减少所需的内存量 [GY+09]。

这种方法的效果可以很好。例如，如果某个工作负载只访问一小部分页，这些页的转换就会存放在内存中的 FTL 里，性能会非常好，内存开销也不高。当然，这种方法也可能表现很差。如果内存放不下所需转换的**工作集**（working set），每次访问至少都需要额外读一次闪存，先把缺失的映射调进来，然后才能访问数据本身。更糟的是，为了给新映射腾出空间，FTL 可能不得不**淘汰**（evict）一个旧映射；如果那个映射是**脏**（dirty）的，也就是还没有持久地写入闪存，还会多出一次写操作。不过，在许多情况下，工作负载会表现出局部性（locality），这种缓存方法既能减少内存开销，又能保持较高的性能。

## 44.10 磨损均衡

最后，现代 FTL 还必须实现一项相关的后台活动，就是前面介绍过的**磨损均衡**（wear leveling）。基本思想很简单：由于多次擦除/编程循环（erase/program cycle）会让闪存块（flash block）磨损，FTL 应该尽力把这些工作均匀地分散到设备的所有块上。这样，所有块都会大致在同一时间磨损，而不是少数几个“热门”块很快就不能用了。

基本的日志结构（log-structured）方法一开始就能很好地分散写负载，垃圾回收也有帮助。不过，有时某个块里装的全是长期存在、不会被覆写的数据；这样，垃圾回收永远不会回收这个块，它也就分不到应有的那一份写负载。

为了解决这个问题，FTL 必须定期把这类块中的所有有效数据读出来，重写到别处，让这个块重新可以写入。磨损均衡（wear leveling）的这个过程增加了 SSD 的写放大（write amplification），从而降低了性能，因为要确保所有块以大致相同的速度磨损，就需要额外的 I/O。文献中有许多不同的算法 [A+08, M+14]；感兴趣的话可以进一步阅读。

## 44.11 SSD 的性能和成本

在结束本章之前，我们来考察一下现代 SSD 的性能和成本，以便更好地理解它们在持久存储系统中可能会怎样使用。在这两方面，我们都会和传统的硬盘驱动器比较，并突出两者之间最大的差别。

### 性能

和硬盘驱动器不同，基于闪存的 SSD 没有机械部件，实际上在许多方面更像 DRAM，因为它们都是“随机访问”设备。和磁盘驱动器相比，性能上最大的差别体现在随机读写上：典型的磁盘驱动器每秒只能执行几百次随机 I/O，而 SSD 能做得好得多。这里我们用一些现代 SSD 的数据，来看看 SSD 的表现到底好多少；我们尤其关心 FTL 把原始芯片的性能问题掩盖得有多好。

图 44.4 给出了三款不同的 SSD 和一款顶级硬盘的一些性能数据；数据取自几个不同的在线来源 [S13, T15]。左边两列是随机 I/O 性能，右边两列是顺序 I/O 性能；前三行是三款不同 SSD（分别来自三星、希捷和英特尔）的数据，最后一行是一块**硬盘驱动器**（hard disk drive，**HDD**）的性能，这里是希捷的一款高端硬盘。

从表中可以了解到几个有趣的事实。第一，也是最惊人的一点，是 SSD 与那块硬盘在随机 I/O 性能上的差距。SSD 的随机 I/O 能达到每秒几十甚至几百 MB，而这块“高性能”硬盘的峰值只有区区几 MB/s（实际上，我们是向上取整才凑到 2 MB/s 的）。第二，可以看到在顺序性能方面，差距要小得多；虽然 SSD 表现更好，但如果你只需要顺序性能，硬盘仍然是不错的选择。第三，可以看到 SSD 的随机读性能反而不如随机写。随机写性能出人意料地好，原因在于许多 SSD 采用日志结构（log-structured）的设计，把随机写转换成了顺序写，从而提高了性能。最后，由于 SSD 的顺序 I/O 和随机 I/O 之间仍有一定的性能差

| | 随机 | | 顺序 | |
|---|---|---|---|---|
| | 读 | 写 | 读 | 写 |
| **设备** | **(MB/s)** | **(MB/s)** | **(MB/s)** | **(MB/s)** |
| Samsung 840 Pro SSD | 103 | 287 | 421 | 384 |
| Seagate 600 SSD | 84 | 252 | 424 | 374 |
| Intel SSD 335 SSD | 39 | 222 | 344 | 354 |
| Seagate Savvio 15K.3 HDD | 2 | 2 | 223 | 223 |

图 44.4：**SSD 与硬盘：性能比较**

 © 2008–23, ARPACI-DUSSEAU

异，前面几章讲的那些为硬盘构建文件系统的技术，许多仍然适用于 SSD [AD14b]；虽然顺序和随机 I/O 之间的差距变小了，但仍然大到值得仔细考虑如何设计文件系统、减少随机 I/O。

### 成本

我们在上面看到，SSD 的性能远远超过现代硬盘，即使是顺序 I/O 也是如此。那么，为什么 SSD 还没有完全取代硬盘，成为首选的存储介质呢？答案很简单：成本，更确切地说，是单位容量的成本。目前 [A15]，一块 250 GB 的 SSD 大约要 150 美元；这样的 SSD 每 GB 要 60 美分。一块典型的 1 TB 硬盘大约 50 美元，也就是每 GB 5 美分。这两种存储介质的成本仍然相差 10 倍以上。

这些性能和成本上的差异，决定了大规模存储系统的构建方式。如果性能是主要考虑因素，SSD 是绝佳的选择，尤其是在随机读性能很重要的时候。反过来，如果你在组建一个大型数据中心，想存储海量信息，巨大的成本差异就会促使你选择硬盘。当然，混合方案也说得通，有些存储系统就同时使用 SSD 和硬盘：用数量较少的 SSD 存放更常用的“热”数据，提供高性能，而把其余“较冷”（较少使用）的数据存放在硬盘上以节省成本。只要价格差距还在，硬盘就不会消失。

## 44.12 小结

基于闪存的 SSD 在笔记本电脑、台式机以及支撑全球经济的数据中心的服务器中越来越常见。所以，你大概应该对它们有所了解，对吧？

坏消息是：本章（和本书的许多章节一样）只是了解当前技术水平的第一步。想进一步了解原始技术，可以看这些资料：关于实际设备性能的研究（例如 Chen 等人 [CK+09] 和 Grupp 等人 [GC+09] 的工作）、FTL 设计中的问题（包括 Agrawal 等人 [A+08]、Gupta 等人 [GY+09]、Huang 等人 [H+14]、Kim 等人 [KK+02]、Lee 等人 [L+07] 和 Zhang 等人 [Z+12] 的工作），甚至由闪存构成的分布式系统（distributed system，包括 Gordon [CG+09] 和 CORFU [B+12]）。另外，恕我们自夸一句，如果想全面了解要从 SSD 榨取高性能都得做哪些事，可以看一篇关于“不成文的契约”的论文 [HK+17]，那里总结得非常好。

不要只读学术论文，也要读读大众媒体上关于最新进展的报道（例如 [V12]）。在那里，你会了解到更实用（但仍然有用）的信息，比如三星在同一块 SSD 中同时使用 TLC 和 SLC 单元，以兼顾性能（SLC 可以快速缓冲写入）和容量（TLC 每个单元能存更多的位）。而这，正如人们常说的，只是冰山一角。自己深入去了解这座研究的“冰山”吧，也许可以从 Ma 等人出色的（而且是最近的）综述 [M+14] 开始。不过要小心：冰山连最强大的船都能击沉 [W15]。

补充：SSD 关键术语

- **闪存芯片**（flash chip）由许多 bank 组成，每个 bank 又组织成**擦除块**（erase block，有时简称**块**，block）。每个块又进一步分成若干**页**（page）。
- 块很大（128 KB–2 MB），包含许多页；页相对较小（1 KB–8 KB）。
- 要从闪存读取，就发出带有地址和长度的读命令；客户端可以借此读取一页或多页。
- 写闪存要复杂一些。首先，客户端必须**擦除**（erase）整个块。然后，客户端可以对每一页**编程**（program），而且只能编程一次，从而完成写入。
- 新的 **trim** 操作可以用来告诉设备某个块不再需要了。
- 闪存的可靠性主要取决于**磨损**（wear out）：如果一个块被擦除和编程得太频繁，它就会无法使用。
- 基于闪存的**固态存储设备**（solid-state storage device，**SSD**）表现得就像一块普通的、基于块读写的磁盘；它借助**闪存转换层**（flash translation layer，**FTL**），把客户端的读写转换成对底层闪存芯片（flash chip）的读取、擦除和编程。
- 大多数 FTL 是**日志结构**（log-structured）的，通过尽量减少擦除/编程循环（erase/program cycle）来降低写入的代价。内存中的转换层记录着逻辑写入在物理介质中的位置。
- 日志结构（log-structured）FTL 的一个关键问题是**垃圾回收**（garbage collection）的代价，它会导致**写放大**（write amplification）。
- 另一个问题是映射表（mapping table）的大小，它可能变得相当大。使用**混合映射**（hybrid mapping），或者只**缓存**（cache） FTL 中的热门部分，都是可能的补救办法。
- 最后一个问题是**磨损均衡**（wear leveling）：FTL 必须时不时把数据从主要被读取的块中迁走，确保这些块也能分担应有的擦除/编程（erase/program）负载。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[A+08] "Design Tradeoffs for SSD Performance" by N. Agrawal, V. Prabhakaran, T. Wobber, J. D. Davis, M. Manasse, R. Panigrahy. USENIX '08, San Diego California, June 2008. *An excellent overview of what goes into SSD design.*

[AD14a] "Operating Systems: Three Easy Pieces" by Chapters: *Crash Consistency: FSCK and Journaling* and *Log-Structured File Systems*. Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. *A lot more detail here about how logging can be used in file systems; some of the same ideas can be applied inside devices too as need be.*

[AD14b] "Operating Systems: Three Easy Pieces" by Chapters: *Locality and the Fast File System* and *File System Implementation*. Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. *These chapters cover how to build a basic file system for a hard drive. Amazingly, some of these ideas work perfectly well on SSDs! See if you can figure out which design techniques are appropriate, and which are less needed.*

[A15] "Amazon Pricing Study" by Remzi Arpaci-Dusseau. February, 2015. *This is not an actual paper, but rather one of the authors going to Amazon and looking at current prices of hard drives and SSDs. You too can repeat this study, and see what the costs are today. Do it!*

[B+12] "CORFU: A Shared Log Design for Flash Clusters" by M. Balakrishnan, D. Malkhi, V. Prabhakaran, T. Wobber, M. Wei, J. D. Davis. NSDI '12, San Jose, California, April 2012. *A new way to think about designing a high-performance replicated log for clusters using Flash.*

[BD10] "Write Endurance in Flash Drives: Measurements and Analysis" by Simona Boboila, Peter Desnoyers. FAST '10, San Jose, California, February 2010. *A cool paper that reverse engineers flash-device lifetimes. Endurance sometimes far exceeds manufacturer predictions, by up to 100×.*

[B07] "ZFS: The Last Word in File Systems" by Jeff Bonwick and Bill Moore. Available here: http://www.ostep.org/Citations/zfs_last.pdf. *Was this the last word in file systems? No, but maybe it's close.*

[CG+09] "Gordon: Using Flash Memory to Build Fast, Power-efficient Clusters for Data-intensive Applications" by Adrian M. Caulfield, Laura M. Grupp, Steven Swanson. ASPLOS '09, Washington, D.C., March 2009. *Early research on assembling flash into larger-scale clusters; definitely worth a read.*

[CK+09] "Understanding Intrinsic Characteristics and System Implications of Flash Memory based Solid State Drives" by Feng Chen, David A. Koufaty, and Xiaodong Zhang. SIGMETRICS/Performance '09, Seattle, Washington, June 2009. *An excellent overview of SSD performance problems circa 2009 (though now a little dated).*

[G14] "The SSD Endurance Experiment" by Geoff Gasior. The Tech Report, September 19, 2014. Available: http://techreport.com/review/27062. *A nice set of simple experiments measuring performance of SSDs over time. There are many other similar studies; use google to find more.*

[GC+09] "Characterizing Flash Memory: Anomalies, Observations, and Applications" by L. M. Grupp, A. M. Caulfield, J. Coburn, S. Swanson, E. Yaakobi, P. H. Siegel, J. K. Wolf. IEEE MICRO '09, New York, New York, December 2009. *Another excellent characterization of flash performance.*

[GY+09] "DFTL: a Flash Translation Layer Employing Demand-Based Selective Caching of Page-Level Address Mappings" by Aayush Gupta, Youngjae Kim, Bhuvan Urgaonkar. ASPLOS '09, Washington, D.C., March 2009. *This paper gives an excellent overview of different strategies for cleaning within hybrid SSDs as well as a new scheme which saves mapping table space and improves performance under many workloads.*

[HK+17] "The Unwritten Contract of Solid State Drives" by Jun He, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. EuroSys '17, Belgrade, Serbia, April 2017. *Our own paper which lays out five rules clients should follow in order to get the best performance out of modern SSDs. The rules are request scale, locality, aligned sequentiality, grouping by death time, and uniform lifetime. Read the paper for details!*

[H+14] "An Aggressive Worn-out Flash Block Management Scheme To Alleviate SSD Performance Degradation" by Ping Huang, Guanying Wu, Xubin He, Weijun Xiao. EuroSys '14, 2014. *Recent work showing how to really get the most out of worn-out flash blocks; neat!*

[J10] "Failure Mechanisms and Models for Semiconductor Devices" by Unknown author. Report JEP122F, November 2010. Available on the internet at this exciting so-called web site: http://www.jedec.org/sites/default/files/docs/JEP122F.pdf. *A highly detailed discussion of what is going on at the device level and how such devices fail. Only for those not faint of heart. Or physicists. Or both.*

[KK+02] "A Space-Efficient Flash Translation Layer For Compact Flash Systems" by Jesung Kim, Jong Min Kim, Sam H. Noh, Sang Lyul Min, Yookun Cho. IEEE Transactions on Consumer Electronics, Volume 48, Number 2, May 2002. *One of the earliest proposals to suggest hybrid mappings.*

[L+07] "A Log Buffer-Based Flash Translation Layer by Using Fully-Associative Sector Translation." Sangwon Lee, Tae-Sun Chung, Dong-Ho Lee, Sangwon Park, Ha-Joo Song. ACM Transactions on Embedded Computing Systems, Volume 6, Number 3, July 2007 *A terrific paper about how to build hybrid log/block mappings.*

[M+14] "A Survey of Address Translation Technologies for Flash Memories" by Dongzhe Ma, Jianhua Feng, Guoliang Li. ACM Computing Surveys, Volume 46, Number 3, January 2014. *Probably the best recent survey of flash and related technologies.*

[S13] "The Seagate 600 and 600 Pro SSD Review" by Anand Lal Shimpi. AnandTech, May 7, 2013. Available: http://www.anandtech.com/show/6935/seagate-600-ssd-review. *One of many SSD performance measurements available on the internet. Haven't heard of the internet? No problem. Just go to your web browser and type "internet" into the search tool. You'll be amazed at what you can learn.*

[T15] "Performance Charts Hard Drives" by Tom's Hardware. January 2015. Available here: http://www.tomshardware.com/charts/enterprise-hdd-charts. *Yet another site with performance data, this time focusing on hard drives.*

[V12] "Understanding TLC Flash" by Kristian Vatto. AnandTech, September, 2012. Available: http://www.anandtech.com/show/5067/understanding-tlc-nand. *A short description about TLC flash and its characteristics.*

[W15] "List of Ships Sunk by Icebergs" by Many authors. Available at this location on the "web": http://en.wikipedia.org/wiki/List_of_ships_sunk_by_icebergs. *Yes, there is a wikipedia page about ships sunk by icebergs. It is a really boring page and basically everyone knows the only ship the iceberg-sinking-mafia cares about is the Titanic.*

[Z+12] "De-indirection for Flash-based SSDs with Nameless Writes" by Yiying Zhang, Leo Prasath Arulraj, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '13, San Jose, California, February 2013. *Our research on a new idea to reduce mapping table space; the key is to re-use the pointers in the file system above to store locations of blocks, instead of adding another level of indirection.*

## 作业（模拟）

本节介绍 `ssd.py`，这是一个简单的 SSD 模拟器，可以帮你更好地理解 SSD 是如何工作的。如何运行模拟器，详见 README。这份 README 很长，所以先泡一杯茶（多半得是含咖啡因的），戴好老花镜，让猫蜷在你腿上[1]，然后开始干活。

## 问题

1. 作业主要关注日志结构（log-structured）的 SSD，用“`-T log`”选项来模拟。其他类型的 SSD 用来作比较。首先，用选项 `-T log -s 1 -n 10 -q` 运行。你能推断出发生了哪些操作吗？用 `-c` 检查你的答案（或者直接用 `-C` 代替 `-q -c`）。用不同的 `-s` 值生成不同的随机工作负载。
2. 现在只显示命令，看看你能否推断出闪存的中间状态。用选项 `-T log -s 2 -n 10 -C` 运行，显示每条命令。然后，确定每两条命令之间闪存的状态；用 `-F` 显示这些状态，看看你对不对。换用不同的随机种子，检验你日渐长进的本事。
3. 我们加上 `-r 20` 选项，让这个问题稍微有意思一点。这会让命令有什么不同？同样用 `-c` 检查你的答案。
4. 性能由擦除、编程和读取的次数决定（这里假设 trim 不花时间）。再次运行上面同样的工作负载，但不显示任何中间状态（例如 `-T log -s 1 -n 10`）。你能估计出这个工作负载需要多长时间才能完成吗？默认的擦除时间为 1000 微秒，编程时间为 40，读取时间为 10。用 `-S` 选项检查你的答案。你还可以用 `-E`、`-W`、`-R` 选项修改擦除、编程和读取的时间。
5. 现在，比较日志结构（log-structured）方法和（非常糟糕的）直接映射方法（用 `-T direct` 代替 `-T log`）的性能。先估计一下你认为直接映射方法的性能如何，再用 `-S` 选项检查你的答案。一般来说，日志结构（log-structured）方法的性能会比直接映射方法好多少？
6. 接下来我们研究垃圾回收器的行为。为此，必须适当设置两条水位线：高水位线（high watermark，`-G`）和低水位线（low watermark，`-g`）。首先，我们观察一下：对日志结构（log-structured）的 SSD 运行一个较大的工作负载，但不做任何垃圾回收，会发生什么。为此，用选项 `-T log -n 1000` 运行；高水位线（high watermark）默认为 10，所以在这种配置下 GC 不会运行。你认为会发生什么？用 `-C`，也许再加上 `-F` 看看。
7. 要打开垃圾回收器，就要使用更小的值。高水位线（high watermark，`-G N`）告诉系统，一旦用掉了 N 个块就开始回收；低水位线（low watermark，`-g M`）告诉系统，一旦只剩 M 个块在使用就停止回收。你认为什么样的水位线值能让系统正常工作？用 `-C` 和 `-F` 显示命令和设备的中间状态，看看结果。

[1]你现在可能会抱怨：“可我是爱狗的人啊！”对此我们只能说：那可真不巧！去弄只猫来，放在腿上，然后做作业！连最基本的指示都不肯照做，你还怎么学东西？

8. 另一个有用的选项是 `-J`，它会显示垃圾回收器运行时在做什么。用选项 `-T log -n 1000 -C -J` 运行，同时查看命令和 GC 的行为。你注意到 GC 有什么特点？当然，GC 最终影响的是性能。用 `-S` 查看最终的统计数据；垃圾回收导致了多少额外的读和写？和理想的 SSD（`-T ideal`）比较一下；由于闪存本身的特性，多出了多少读、写和擦除？再和直接映射方法比较一下；日志结构（log-structured）方法在擦除、读取、编程中的哪些方面更胜一筹？

9. 最后一个要研究的方面是**工作负载倾斜**（workload skew）。给工作负载加上倾斜之后，写操作会更多地集中在逻辑块空间中较小的一部分上。例如，用 `-K 80/20` 运行，会让 80% 的写落在 20% 的块上。挑几种不同的倾斜程度，执行大量随机选择的操作（例如 `-n 1000`）：先用 `-T direct` 来理解这种倾斜，再用 `-T log` 看看它对日志结构（log-structured）设备的影响。你预计会发生什么？还有一个可以研究的小选项是 `-k 100`，它也用来控制倾斜；在有倾斜的工作负载上加上这个选项，前 100 次写入就不会倾斜。这样做的想法是先创建大量数据，然后只更新其中一部分。这可能会对垃圾回收器产生什么影响？

 © 2008–23, ARPACI-DUSSEAU

# 45

# 数据完整性与保护

除了前面介绍过的文件系统（file system）中的那些基本进展，还有不少特性值得研究。本章再次讨论可靠性（reliability），存储系统的这个话题此前在 RAID 一章已经谈过。具体来说，鉴于现代存储设备（storage device）并不可靠，文件系统或存储系统应该如何确保数据安全？

这个领域统称为**数据完整性**（data integrity）或**数据保护**（data protection）。下面就来研究一些技术，确保你存进存储系统的数据，取回来时还是原来的样子。

关键问题：如何确保数据完整性
系统应该如何确保写入存储的数据受到保护？需要哪些技术？如何让这些技术高效，空间和时间开销都很低？

## 45.1 磁盘故障模式

你在 RAID 一章中已经了解到，磁盘并不完美，偶尔会出故障。在早期的 RAID 系统中，故障模型相当简单：要么整块磁盘正常工作，要么彻底失效，而且这种故障很容易检测。这种 **fail-stop** 磁盘故障模型让构建 RAID 变得相对简单 [S90]。

但你还没了解现代磁盘的其他各种故障模式。正如 Bairavasundaram 等人详细研究过的那样 [B+07, B+08]，现代磁盘偶尔会看起来大体正常，却无法成功访问一个或多个块（block）。具体来说，有两类单块故障很常见，值得考虑：**latent sector error**（**LSE**）和**块损坏**（block corruption）。下面我们分别详细讨论。

LSE 是指磁盘扇区（sector）或一组扇区受到了某种损伤。例如，如果磁头（disk head）因为某种原因碰到了盘面（surface），也就是发生了正常运行时本不应出现的**磁头碰撞**（head crash），就可能损伤盘面，导致上面的位无法读取。宇宙射线也会让位发生翻转，导致内容出错。好在驱动器内部用了**纠错码**（error correcting code，**ECC**），可以借此判断一个块在

| | 廉价 | 高价 |
|---|---|---|
| LSE | 9.40% | 1.40% |
| 损坏 | 0.50% | 0.05% |

图 45.1：**LSE 和块损坏的发生频率**

盘上的各个位是否完好，有时还能修复它们；如果数据位有问题，而驱动器又没有足够的信息来纠正错误，那么请求读取这些位时，磁盘就会返回错误。

还有一些情况是，磁盘块**损坏**（corrupt）了，磁盘本身却检测不到。例如，有 bug 的磁盘固件（firmware）可能把一个块写到错误的位置；在这种情况下，磁盘的 ECC 显示块的内容没问题，但从客户端的角度看，之后访问时返回的却是错误的块。类似地，块从主机经过有故障的总线（bus）传到磁盘时，也可能被损坏；磁盘存下的是损坏后的数据，但这并不是客户端想要的。这类故障格外隐蔽，因为它们是**静默错误**（silent fault）：磁盘返回坏数据时，不会给出任何出错的迹象。

Prabhakaran 等人把这种更现代的磁盘故障观点描述为 **fail-partial** 磁盘故障模型 [P+05]。按照这种观点，磁盘仍然可能整体失效（就像传统的 fail-stop 模型那样）；但磁盘也可能看起来在正常工作，却有一个或多个块无法访问（即 LSE），或者存放着错误的内容（即损坏）。因此，一块看起来正常的磁盘，偶尔可能在读写某个块时返回错误（非静默的部分故障），偶尔也可能直接返回错误的数据（静默的部分故障）。

这两类故障都不太常见，但到底有多不常见呢？图 45.1 汇总了 Bairavasundaram 的两项研究中的部分发现 [B+07, B+08]。

这项研究持续了大约 3 年，涉及超过 150 万块磁盘驱动器（disk drive）。图中给出的是研究期间至少出现过一次 LSE 或块损坏（block corruption）的驱动器所占的百分比。图中还把结果细分为“廉价”驱动器（通常是 SATA 驱动器）和“高价”驱动器（通常是 SCSI 或光纤通道驱动器）。可以看到，虽然买更好的驱动器能降低这两类问题的发生频率（大约降低一个数量级），但它们仍然发生得足够频繁，你得仔细考虑存储系统该怎样处理它们。

关于 LSE 的其他一些发现：

- 有不止一个 LSE 的高价驱动器，和廉价驱动器一样容易出现更多错误
- 对大多数驱动器来说，年错误率在第二年上升
- LSE 的数量随磁盘容量增大而增加
- 大多数有 LSE 的磁盘，LSE 少于 50 个
- 有 LSE 的磁盘更容易出现更多的 LSE
- 存在显著的空间局部性（spatial locality）和时间局部性（temporal locality）
- disk scrubbing 很有用（大多数 LSE 都是这样发现的）

关于损坏的一些发现：

- 在同一类驱动器中，不同型号的损坏几率差别很大
- 使用年限的影响因型号而异
- 工作负载（workload）和磁盘容量对损坏几乎没有影响
- 大多数出现损坏的磁盘只有少数几处损坏
- 在同一块磁盘内，或者 RAID 中的多块磁盘之间，损坏并不是相互独立的
- 存在空间局部性，也有一定的时间局部性
- 与 LSE 之间有微弱的相关性

要进一步了解这些故障，你大概应该读读原始论文 [B+07, B+08]。但希望要点已经很清楚了：如果你真的想构建一个可靠的存储系统，就必须加入检测 LSE 和块损坏（block corruption）并从中恢复的机制。

## 45.2 处理 latent sector error

了解了这两种新的部分故障模式，现在来看看能拿它们怎么办。先处理两者中较容易的那个：latent sector error。

关键问题：如何处理 latent sector error
存储系统应该如何处理 latent sector error？处理这种形式的部分故障需要多少额外的机制？

其实，latent sector error 处理起来相当简单，因为按定义它们很容易检测出来。存储系统访问某个块时，如果磁盘返回错误，存储系统只需借助自己现有的冗余（redundancy）机制返回正确的数据即可。例如，在镜像 RAID 中，系统应该访问另一个副本；在基于奇偶校验（parity）的 RAID-4 或 RAID-5 系统中，系统应该利用奇偶校验组中的其他块重建这个块。因此，像 LSE 这样容易检测到的问题，借助标准的冗余机制就能轻松恢复。

多年来，LSE 越来越普遍，这影响了 RAID 的设计。在 RAID-4/5 系统中，整盘故障和 LSE 同时出现时，会产生一个特别有意思的问题。具体来说，一整块磁盘失效后，RAID 会尝试把它**重建**（reconstruct）出来，比如重建到一块热备盘（hot spare）上，方法是读遍奇偶校验组（parity group）中所有其他磁盘，重新计算出缺失的值。重建过程中，只要其他任何一块磁盘上遇到 LSE，我们就有麻烦了：重建无法完成。

为了解决这个问题，有些系统增加了一层额外的冗余。例如，NetApp 的 **RAID-DP** 相当于有两块奇偶校验盘（parity disk），而不是一块 [C+04]。在重建过程中发现 LSE 时，额外的奇偶校验可以帮助重建缺失的块。和往常一样，这是有代价的：为每个条带（stripe）维护两个奇偶校验块（parity block）的开销更大；不过，NetApp 的 **WAFL** 文件系统采用日志结构（log-structured），在许多情况下减轻了这一开销 [HLM94]。剩下的代价是空间，也就是要为第二个奇偶校验块（parity block）多用一块磁盘。

## 45.3 检测损坏：校验和

现在我们来处理更具挑战性的问题，也就是由数据损坏引起的静默错误（silent fault）。出现损坏、磁盘因此返回坏数据时，怎样才能不让用户拿到坏数据？

关键问题：如何在出现损坏时保持数据完整性
既然这类故障是静默的，存储系统怎样才能发现数据损坏？需要哪些技术？如何高效地实现它们？

和 latent sector error 不同，检测损坏才是关键问题。客户端怎么判断一个块已经坏了？一旦知道某个块坏了，恢复的方法就和以前一样：你需要有这个块的另一个副本（而且最好是没有损坏的！）。因此，我们这里重点讨论检测技术。

提示：天下没有免费的午餐

There's No Such Thing As A Free Lunch（天下没有免费的午餐），简称 TNSTAAFL，是一句古老的美国俗语，意思是：看似免费得到的东西，你实际上很可能在为它付出某种代价。它源自过去的年代：小餐馆会打出免费午餐的广告来招揽顾客；你走进去以后才会发现，要得到这份“免费”午餐，必须买一杯或几杯酒。当然，这未必是坏事，尤其是如果你立志成为酒鬼（或者是个典型的本科生）。

**校验和**（checksum）是现代存储系统用来保持数据完整性（data integrity）的主要机制。校验和就是某个函数的计算结果：函数的输入是一块数据（比如一个 4 KB 的块），输出是反映数据内容的一个小摘要（比如 4 或 8 个字节）。这样做的目的，是让系统能够发现数据是否遭到了损坏或改动：把校验和与数据存在一起，以后访问数据时，再确认数据当前的校验和与当初存下的值是否一致。

### 常用的校验和函数

计算校验和的函数有很多种，强度和速度各不相同：强度是指保护数据完整性（data integrity）的能力有多强，速度是指计算有多快。这里出现了系统中常见的一种权衡：通常，你得到的保护越多，代价就越高。天下没有免费的午餐。

有一种简单的校验和函数基于异或（XOR）：把要计算校验和的数据块（data block）逐段做 XOR，最终得到一个代表整个块 XOR 结果的值，这就是校验和。

举个具体的例子：假设要对一个 16 字节的块计算 4 字节的校验和。这么小的块当然不可能真是磁盘扇区或块，但用来举例足够了。这 16 个数据字节用十六进制表示如下：

```
365e c4cd ba14 8a92 ecef 2c3a 40be f666
```

如果用二进制来看，就是这样：

```
0011 0110 0101 1110  1100 0100 1100 1101
1011 1010 0001 0100  1000 1010 1001 0010
1110 1100 1110 1111  0010 1100 0011 1010
0100 0000 1011 1110  1111 0110 0110 0110
```

数据已经按每行 4 个字节排好，所以很容易看出校验和是多少：对每一列做 XOR，就得到最终的校验和值：

```
0010 0000 0001 1011  1001 0100 0000 0011
```

结果用十六进制表示是 0x201b9403。

XOR 是一种还算合理的校验和，但有它的局限。例如，如果参与计算的两个单元中，同一位置上的位都发生了变化，校验和就检测不到这种损坏。正因如此，人们研究了其他的校验和函数。

另一种基本的校验和函数是加法。这种方法的优点是速度快：计算时只需对数据的每一段做二进制补码加法，忽略溢出即可。它能检测出数据的许多变化，但如果数据发生了移位之类的变化，效果就不好了。

 © 2008–23, ARPACI-DUSSEAU

一种稍微复杂一点的算法叫作 **Fletcher 校验和**（Fletcher checksum）；你大概猜到了，它以发明者 John G. Fletcher 的名字命名 [F82]。它计算起来很简单，需要算出两个校验字节 $s1$ 和 $s2$。具体来说，假设块 $D$ 由字节 $d1...dn$ 组成；$s1$ 的定义是：$s1 = (s1 + d_i) \bmod 255$（对所有 $d_i$ 计算）；$s2$ 则是：$s2 = (s2 + s1) \bmod 255$（同样对所有 $d_i$ 计算）[F04]。Fletcher 校验和（Fletcher checksum）几乎和 CRC（见下文）一样强，能检测出所有单比特错误、双比特错误，以及许多突发错误 [F04]。

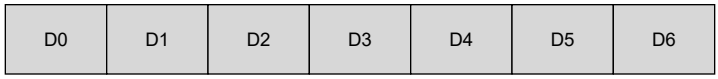
最后一种常用的校验和叫作**循环冗余校验**（cyclic redundancy check，**CRC**）。假设你想对数据块 $D$ 计算校验和。你只需把 $D$ 当成一个很大的二进制数（毕竟它只是一串位），然后用它除以一个约定好的值（$k$）。这个除法的余数就是 CRC 的值。这种二进制取模运算其实可以实现得相当高效，这也是 CRC 在网络领域同样流行的原因。更多细节请参阅其他资料 [M13]。

无论用哪种方法，显然都不存在完美的校验和：两个内容不同的数据块可能有相同的校验和，这称为**碰撞**（collision）。这一点应该不难理解：毕竟，计算校验和就是把很大的东西（例如 4 KB）变成小得多的摘要（例如 4 或 8 个字节）。因此，选择好的校验和函数时，我们要找的是一个既能把碰撞的几率降到最低、又容易计算的函数。

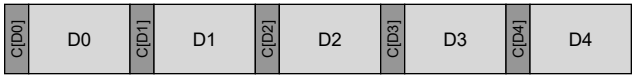
### 校验和的布局

你已经大致了解了如何计算校验和，接下来我们分析如何在存储系统中使用校验和。首先要解决的问题是校验和的布局，也就是说，校验和应该怎样存放在磁盘上？

最基本的方法，就是为每个磁盘扇区或块存一个校验和。数据块 $D$ 的校验和记作 $C(D)$。没有校验和时，磁盘布局是这样的：

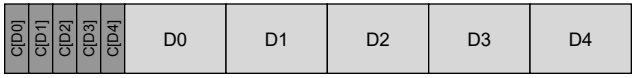



有了校验和之后，布局中每个块都多了一个校验和：




校验和通常很小（例如 8 字节），而磁盘只能以扇区（512 字节）或其整数倍为单位写入，那么上面这种布局该怎么实现呢？驱动器厂商采用的一种解决办法，是把驱动器格式化成 520 字节的扇区；每个扇区多出来的 8 个字节可以用来存放校验和。

如果磁盘没有这种功能，文件系统就必须想办法把校验和打包存放到 512 字节的块中。一种可行的做法如下：




在这种方案中，$n$ 个校验和一起存放在一个扇区里，后面跟着 $n$ 个数据块，再后面是下 $n$ 个块的另一个校验和扇区，依此类推。这种方法的好处是适用于所有磁盘，但效率可能较低；例如，如果文件系统想覆写块 $D1$，它就必须读入包含 $C(D1)$ 的校验和扇区（checksum

 © 2008–23, ARPACI-DUSSEAU

sector），更新其中的 $C(D1)$，然后写出校验和扇区和新的数据块 $D1$（因此要一次读和两次写）。前一种方法，即每个扇区存一个校验和，只需执行一次写入。

## 45.4 使用校验和

确定了校验和的布局，就可以来看看怎样使用校验和了。读取块 $D$ 时，客户端（即文件系统或存储控制器）会同时从磁盘读出它的校验和 $C_s(D)$，称为**存储的校验和**（stored checksum），下标 $s$ 就是这么来的。然后，客户端对取回的块 $D$ 计算校验和，我们称之为**计算出的校验和**（computed checksum）$C_c(D)$。这时，客户端比较存储的校验和（stored checksum）与计算出的校验和（computed checksum）；如果两者相等（即 $C_s(D) == C_c(D)$），数据很可能没有损坏，因此可以安全地返回给用户。如果两者不相等（即 $C_s(D)\ != C_c(D)$），就说明数据自存储之后发生了变化，因为存储的校验和（stored checksum）反映的是当时数据的值。这说明遇到了损坏，而校验和帮我们发现了它。

出现损坏时，自然要问：该怎么办？如果存储系统有冗余副本（redundant copy），答案很简单：试着改用它。如果存储系统没有这样的副本，大概就只能返回错误。无论哪种情况，都要认识到，检测损坏并不是灵丹妙药；如果没有其他办法拿到未损坏的数据，你就只能自认倒霉了。

## 45.5 一个新问题：misdirected write

上面描述的基本方案，在块损坏（block corruption）的一般情况下效果很好。然而，现代磁盘有几种不寻常的故障模式，需要不同的解决方案。

我们关注的第一种故障模式叫作 **misdirected write**。它出现在磁盘和 RAID 控制器中：数据正确地写到了磁盘上，只是写到了错误的位置。在单磁盘系统中，这意味着磁盘没有把块 $D_x$ 写到（期望的）地址 $x$，而是写到了地址 $y$（因此“损坏”了 $D_y$）；此外，在多磁盘系统中，控制器还可能没有把 $D_{i,x}$ 写到磁盘 $i$ 的地址 $x$，而是写到了另一块磁盘 $j$ 上。于是我们的问题是：

> 关键问题：如何处理 misdirected write
> 存储系统或磁盘控制器应该如何检测 misdirected write？校验和需要具备哪些额外的特性？

答案不出所料，很简单：在每个校验和里多加一点信息。在这种情况下，加入一个**物理标识符**（physical identifier，**physical ID**）很有帮助。例如，如果存储的信息现在不仅包含校验和 $C(D)$，还包含块所在的磁盘号和扇区号，客户端就很容易判断某个位置上存放的是不是正确的信息。具体来说，如果客户端要读取磁盘 10 上的块 4（$D_{10,4}$），存储的信息中就应该包含这个磁盘号和扇区偏移。如果信息对不上，说明发生了 misdirected write，损坏就被检测出来了。下面是在一个双磁盘系统上加入这些信息后的样子。注意，这张图和之前

 © 2008–23, ARPACI-DUSSEAU

的那些图一样，并没有按比例画，因为校验和通常很小（例如 8 字节），而块要大得多（例如 4 KB 或更大）：




从盘上的格式可以看出，磁盘上的冗余现在相当多：每个块里都重复存放了磁盘号，块自己的偏移量（offset）也存放在块旁边。有冗余信息并不奇怪，冗余正是检测错误（这里的情况）和从错误中恢复（其他情况）的关键。对完美的磁盘来说，这点额外信息并不是必需的，但一旦出了问题，它能帮上大忙。

## 45.6 最后一个问题：lost write

可惜的是，misdirected write 并不是我们要处理的最后一个问题。具体来说，一些现代存储设备还有一个叫作 **lost write** 的问题：设备告诉上层某次写入已经完成，但数据实际上从未持久化，盘上留下的还是块的旧内容，而不是更新后的新内容。

这里显而易见的问题是：上面那些校验和策略（例如基本的校验和或物理标识）能帮助检测 lost write 吗？答案是否定的：旧块很可能有匹配的校验和，而上面用到的物理 ID，也就是磁盘号和块偏移量，也是正确的。于是，我们最后的问题是：

> 关键问题：如何处理 lost write
> 存储系统或磁盘控制器应该如何检测 lost write？校验和需要具备哪些额外的特性？

能帮上忙的办法有好几种 [K+08]。一种经典的方法 [BS04] 是执行 **write verify**，也叫 **read-after-write**：写入之后立即把数据读回来，系统就能确保数据确实到达了盘面。然而，这种方法相当慢，完成一次写入所需的 I/O 数量翻了一倍。

有些系统在别处再加一份校验和来检测 lost write。例如，Sun 的 **Zettabyte File System**（**ZFS**）在每个文件系统 inode 和间接块（indirect block）中，为文件包含的每个块都存了一个校验和。这样，即使对数据块本身的写入丢失了，inode 中的校验和也不会与旧数据相符。只有对 inode 和对数据的写入同时丢失时，这种方案才会失效，这种情况不太可能发生（但很不幸，是有可能的！）。

## 45.7 Scrubbing

讨论了这么多，你可能会想：这些校验和到底什么时候检查？当然，应用程序访问数据时会检查一部分，但大多数数据很少被访问，也就一直没人检查。对可靠的存储系统来说，没有检查过的数据是个问题，因为位衰减（bit rot）最终可能会影响某份数据的所有副本。

为了解决这个问题，许多系统采用了各种形式的 **disk scrubbing** [K+08]。磁盘系统定期读遍系统中的每一个块，检查校验和是否仍然有效，就能降低某个数据项的所有副本都损坏的几率。典型的系统会每晚或每周安排一次扫描。

## 45.8 校验和的开销

在结束本章之前，我们来讨论一下用校验和保护数据的一些开销。和计算机系统中常见的情况一样，开销分为两种：空间和时间。

空间开销有两种形式。第一种在磁盘（或其他存储介质）本身上：每个存储的校验和（stored checksum）都要占用磁盘空间，这些空间不能再用来存放用户数据。典型的比例可能是每 4 KB 数据块配一个 8 字节的校验和，盘上空间开销为 0.19%。

第二种空间开销出现在系统的内存中。访问数据时，内存中除了数据本身，现在还必须为校验和留出空间。不过，如果系统只是检查校验和，检查完就丢弃，这种开销是短暂的，不必太担心。只有把校验和留在内存里（用来额外防范内存损坏 [Z+13]），这点小开销才会显现出来。

虽然空间开销很小，但校验和带来的时间开销可能相当明显。至少，CPU 必须对每个块计算校验和：存储数据时要算，以确定存储的校验和（stored checksum）的值；访问数据时要再算一次，并与存储的校验和比较。包括网络协议栈（protocol stack）在内，许多使用校验和的系统都采用了一种降低 CPU 开销的方法：把数据复制和计算校验和合并成一个流水线式的操作。复制本来就免不了，例如把数据从内核页缓存（page cache）复制到用户缓冲区，顺便算校验和就很划算。

除了 CPU 开销，有些校验和方案还会带来额外的 I/O 开销：一是校验和与数据分开存放时，访问校验和需要额外的 I/O；二是后台 scrubbing 需要额外的 I/O。前者可以靠好的设计减少；后者可以调节，比如控制 scrubbing 在什么时候进行，以限制它的影响。深夜时分，大多数（不是所有！）勤奋的员工都已经上床睡觉了，这可能是执行 scrubbing、提高存储系统健壮性的好时机。

## 45.9 小结

我们讨论了现代存储系统中的数据保护（data protection），重点是校验和的实现和使用。不同的校验和防护不同类型的故障；随着存储设备的发展，毫无疑问还会出现新的故障模式。也许这些变化会迫使学术界和工业界重新审视其中一些基本方法，或者干脆发明全新的方法。时间会告诉我们答案。或者不会。时间就是这么有意思。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[B+07] "An Analysis of Latent Sector Errors in Disk Drives" by L. Bairavasundaram, G. Goodson, S. Pasupathy, J. Schindler. SIGMETRICS '07, San Diego, CA. *The first paper to study latent sector errors in detail. The paper also won the Kenneth C. Sevcik Outstanding Student Paper award, named after a brilliant researcher and wonderful guy who passed away too soon. To show the OSTEP authors it was possible to move from the U.S. to Canada, Ken once sang us the Canadian national anthem, standing up in the middle of a restaurant to do so. We chose the U.S., but got this memory.*

[B+08] "An Analysis of Data Corruption in the Storage Stack" by Lakshmi N. Bairavasundaram, Garth R. Goodson, Bianca Schroeder, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '08, San Jose, CA, February 2008. *The first paper to truly study disk corruption in great detail, focusing on how often such corruption occurs over three years for over 1.5 million drives.*

[BS04] "Commercial Fault Tolerance: A Tale of Two Systems" by Wendy Bartlett, Lisa Spainhower. IEEE Transactions on Dependable and Secure Computing, Vol. 1:1, January 2004. *This classic in building fault tolerant systems is an excellent overview of the state of the art from both IBM and Tandem. Another must read for those interested in the area.*

[C+04] "Row-Diagonal Parity for Double Disk Failure Correction" by P. Corbett, B. English, A. Goel, T. Grcanac, S. Kleiman, J. Leong, S. Sankar. FAST '04, San Jose, CA, February 2004. *An early paper on how extra redundancy helps to solve the combined full-disk-failure/partial-disk-failure problem. Also a nice example of how to mix more theoretical work with practical.*

[F04] "Checksums and Error Control" by Peter M. Fenwick. Copy available online here: http://www.ostep.org/Citations/checksums-03.pdf. *A great simple tutorial on checksums, available to you for the amazing cost of free.*

[F82] "An Arithmetic Checksum for Serial Transmissions" by John G. Fletcher. IEEE Transactions on Communication, Vol. 30:1, January 1982. *Fletcher's original work on his eponymous checksum. He didn't call it the Fletcher checksum, rather he just didn't call it anything; later, others named it after him. So don't blame old Fletch for this seeming act of braggadocio. This anecdote might remind you of Rubik; Rubik never called it "***Rubik's cube***"; rather, he just called it "my cube."*

[HLM94] "File System Design for an NFS File Server Appliance" by Dave Hitz, James Lau, Michael Malcolm. USENIX Spring '94. *The pioneering paper that describes the ideas and product at the heart of NetApp's core. Based on this system, NetApp has grown into a multi-billion dollar storage company. To learn more about NetApp, read Hitz's autobiography "How to Castrate a Bull" (which is the actual title, no joking). And you thought you could avoid bull castration by going into CS.*

[K+08] "Parity Lost and Parity Regained" by Andrew Krioukov, Lakshmi N. Bairavasundaram, Garth R. Goodson, Kiran Srinivasan, Randy Thelen, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '08, San Jose, CA, February 2008. *This work explores how different checksum schemes work (or don't work) in protecting data. We reveal a number of interesting flaws in current protection strategies.*

[M13] "Cyclic Redundancy Checks" by unknown. Available: http://www.mathpages.com/home/kmath458.htm. *A super clear and concise description of CRCs. The internet is full of information, as it turns out.*

[P+05] "IRON File Systems" by V. Prabhakaran, L. Bairavasundaram, N. Agrawal, H. Gunawi, A. Arpaci-Dusseau, R. Arpaci-Dusseau. SOSP '05, Brighton, England. *Our paper on how disks have partial failure modes, and a detailed study of how modern file systems react to such failures. As it turns out, rather poorly! We found numerous bugs, design flaws, and other oddities in this work. Some of this has fed back into the Linux community, thus improving file system reliability. You're welcome!*

[RO91] "Design and Implementation of the Log-structured File System" by Mendel Rosenblum and John Ousterhout. SOSP '91, Pacific Grove, CA, October 1991. *So cool we cite it again.*

[S90] "Implementing Fault-Tolerant Services Using The State Machine Approach: A Tutorial" by Fred B. Schneider. ACM Surveys, Vol. 22, No. 4, December 1990. *How to build fault tolerant services. A must read for those building distributed systems.*

[Z+13] "Zettabyte Reliability with Flexible End-to-end Data Integrity" by Y. Zhang, D. Myers, A. Arpaci-Dusseau, R. Arpaci-Dusseau. MSST '13, Long Beach, California, May 2013. *How to add data protection to the page cache of a system. Out of space, otherwise we would write something…*

## 作业（模拟）

在这个作业中，你要用 `checksum.py` 研究校验和的各个方面。

## 问题

1. 首先，不带任何参数运行 `checksum.py`。计算加法校验和、基于 XOR 的校验和以及 Fletcher 校验和（Fletcher checksum）。用 `-c` 检查你的答案。
2. 现在做同样的事，但把种子（`-s`）改成不同的值。
3. 有时，加法校验和与基于 XOR 的校验和会得出相同的结果（例如，数据值全为零时）。你能传入一个 4 字节的数据值（用 `-D` 选项，例如 `-D a,b,c,d`），它不全为零，却让加法校验和与基于 XOR 的校验和的值相同吗？一般来说，什么时候会出现这种情况？用 `-c` 选项检查你是否正确。
4. 现在传入一个 4 字节的值，你知道它会让加法和 XOR 得出不同的校验和值。一般来说，什么时候会出现这种情况？
5. 用模拟器计算两次校验和（每次用一组不同的数）。两串数字应该不同（例如第一次用 `-D a1,b1,c1,d1`，第二次用 `-D a2,b2,c2,d2`），但要得出相同的加法校验和。一般来说，在数据值不同的情况下，加法校验和什么时候会相同？用 `-c` 选项检查你的具体答案。
6. 现在对 XOR 校验和做同样的事。
7. 现在我们来看一组特定的数据值。第一组是：`-D 1,2,3,4`。对这组数据，各种校验和（加法、XOR、Fletcher）分别是多少？再把它和对 `-D 4,3,2,1` 计算这些校验和的结果比较一下。你注意到这三种校验和有什么特点？Fletcher 和另外两种相比怎么样？一般来说，Fletcher 为什么比简单的加法校验和之类的“更好”？
8. 没有完美的校验和。给定一个你自己选的输入，你能找到其他数据值，得出相同的 Fletcher 校验和（Fletcher checksum）吗？一般来说，什么时候会出现这种情况？从一个简单的数据串开始（例如 `-D 0,1,2,3`），看看能否替换其中一个数，却得到相同的 Fletcher 校验和（Fletcher checksum）。和往常一样，用 `-c` 检查你的答案。

 © 2008–23, ARPACI-DUSSEAU

## 作业（编程）

在这部分作业中，你要自己写一些代码来实现各种校验和。

## 问题

1. 写一个简短的 C 程序（名为 `check-xor.c`），对输入文件计算基于 XOR 的校验和，并把校验和打印出来。用一个 8 位的 `unsigned char` 来存放（一个字节的）校验和。制作一些测试文件，看看它是否按预期工作。
2. 现在写一个简短的 C 程序（名为 `check-fletcher.c`），对输入文件计算 Fletcher 校验和（Fletcher checksum）。同样，测试一下你的程序，看看它是否能工作。
3. 现在比较两者的性能：哪个更快？输入文件的大小变化时，性能如何变化？在程序内部调用 `gettimeofday` 来计时。如果你关心性能，应该用哪一个？如果关心检错能力呢？
4. 了解一下 16 位 CRC，然后实现它。用许多不同的输入测试它，确保它能正常工作。和简单的 XOR、Fletcher 相比，它的性能如何？检错能力呢？
5. 现在构建一个工具（`create-csum.c`），对文件的每个 4 KB 块计算一个单字节的校验和，并把结果记录到一个输出文件中（在命令行中指定）。再构建一个相关的工具（`check-csum.c`），它读取一个文件，对每个块计算校验和，并把结果和存放在另一个文件中的校验和比较。如果有问题，程序应该打印出文件已经损坏的信息。手动破坏文件来测试这个程序。

 © 2008–23, ARPACI-DUSSEAU

# 46

# 关于持久性的总结对话

**学生**：哇，文件系统（*file system*）看起来挺有意思（!），可也真够复杂的。

**教授**：所以我和我爱人才会在这个领域做研究。

**学生**：等一下。您就是写这本书的教授之一？我还以为我们俩都只是虚构出来的角色，用来总结几个要点，顺便给操作系统的学习添点乐子。

**教授**：呃……这个……也许吧。再说这不关你的事！不然你以为这些东西是谁写的？（叹气）好了，言归正传：你学到了什么？

**学生**：嗯，我想我抓住了一个要点：长期（持久地）管理数据，比管理不持久的数据（比如内存里的东西）难得多。毕竟，机器一旦崩溃，内存里的内容就没了！可文件系统里的东西得永远保存下去。

**教授**：嗯，就像我的朋友 *Kevin Hultquist* 常说的："永远可是很长的时间。"他当时说的是塑料高尔夫球座，不过这话用在大多数文件系统里堆着的那些垃圾上，尤其贴切。

**学生**：哎呀，您知道我的意思！至少是很长一段时间吧。而且，就连更新持久存储（*persistent storage*）设备这种简单的事情都很复杂，因为你得操心崩溃了会发生什么。在虚拟化内存（*virtualizing memory*）的时候，恢复这件事我想都没想过，现在却成了大问题！

**教授**：太对了。更新持久存储一直是个有趣又富有挑战的问题，现在仍然是。

**学生**：我还学到了磁盘调度（*disk scheduling*）这种很酷的东西，还有 *RAID* 甚至校验和（*checksum*）这些数据保护（*data protection*）技术。这些东西真酷。

**教授**：我也喜欢这些主题。不过，真要深究下去，就会有点偏数学了。你要是想让自己脑袋疼，就去看看纠删码（*erasure code*）方面的最新研究吧。

**学生**：我这就去看。

**教授**：（皱眉）我觉得你在挖苦我。那么，你还喜欢什么？

**学生**：我还很喜欢 *FFS*、*LFS* 这类"了解底层技术"的系统背后花的那些心思。真妙！让系统了解磁盘的特性，听起来就很酷。可是，有了闪存（*flash*）和那些最新最潮的技术，这些还重要吗？

**教授**：问得好！是的，就算有了闪存，这些东西居然也都还用得上。例如，闪存转换层（*Flash Translation Layer*，*FTL*）在内部就采用了日志结构（*log-structuring*），以提高基于闪存的 *SSD* 的性能和可靠性（*reliability*）。而且，考虑局部性（*locality*）总是有用的。所以，技术也许在变，但我们学过的许多思想还会继续发挥作用，至少在一段时间内是这样。

**学生**：那就好。我花了这么多时间学这些，可不希望全都白学！

**教授**：教授们才不会那样对你，对吧？

# 47

# 关于分布式的对话

**教授**：就这样，我们讲到了操作系统世界里的最后一小块：分布式系统（*distributed system*）。这部分我们讲不了太多，所以就在持久性（*persistence*）这一部分里顺带做个简短介绍，重点放在分布式文件系统（*distributed file system*）上。希望你不介意！

**学生**：听起来不错。不过，分布式系统究竟是什么呢，噢，无比光荣、无所不知的教授？

**教授**：嗯，我打赌你知道接下来会怎么说……

**学生**：有一个桃子？

**教授**：正是！不过这一次，桃子离你很远，你可能要花些时间才能拿到。而且桃子有很多很多！更糟的是，有时候桃子会烂掉。可你又想确保，不管谁咬上一口，都能尝到满嘴的美味。

**学生**：这个桃子的比喻对我来说越来越不管用了。

**教授**：别这样嘛！这是最后一个了，将就一下吧。

**学生**：好吧。

**教授**：总之，先别管桃子了。构建分布式系统很难，因为总是有东西出故障。消息会丢失，机器会宕机，磁盘会损坏数据。就好像整个世界都在跟你作对！

**学生**：可我一直都在用分布式系统，对吧？

**教授**：没错！你确实在用。然后呢……？

**学生**：嗯，看起来它们大多数时候都运转正常。毕竟，我向 *Google* 发一个搜索请求，结果通常一眨眼就回来了，而且质量很高！用 *Facebook*、*Amazon* 之类的时候也一样。

**教授**：是啊，这真了不起。而且是在故障不断发生的情况下做到的！这些公司在系统里构建了大量机制，确保即使有些机器坏了，整个系统也能继续运行。他们为此用了很多技术：复制（*replication*）、重试（*retry*），以及人们多年来发展出来的各种检测故障、从故障中恢复的技巧。

**学生**：听起来很有意思。是时候真正学点东西了吧？

**教授**：看来是的。开始干活吧！不过，先办最要紧的事……
（咬了一口一直拿在手里的桃子，可惜它已经烂了）

# 48

# 分布式系统

分布式系统（distributed system）改变了世界的面貌。你的 Web 浏览器连接到地球另一端的某台 Web 服务器时，它参与的就是一种看似简单的**客户端/服务器**（client/server）分布式系统。不过，你访问 Google 或 Facebook 这样的现代 Web 服务时，与你交互的并不只是一台机器；在幕后，这些复杂的服务由一大批（成千上万台）机器构成，它们相互协作，共同提供这个网站的服务。由此不难看出，研究分布式系统为什么这么有意思。这个主题甚至值得单独开一门课；这里我们只介绍其中几个主要话题。

构建分布式系统会遇到许多新的挑战。我们关注的主要挑战是**故障**（failure）：机器、磁盘、网络和软件都会时不时出故障，因为我们不知道（而且很可能永远都不会知道）怎样构建“完美”的组件和系统。然而，构建现代 Web 服务时，我们希望在客户端看来，它从不出故障。怎样才能做到这一点呢？

关键问题：
如何构建在组件出故障时仍能工作的系统

怎样用并非一直正常工作的部件，构建出一个能正常工作的系统？这个基本问题应该会让你想起我们在 RAID 存储阵列中讨论过的一些话题；不过，这里的问题往往更复杂，解决方案也是如此。

有意思的是，故障虽然是构建分布式系统的核心挑战，但它也带来了机遇。没错，机器会出故障；但一台机器出故障，并不意味着整个系统都必须跟着出故障。把一组机器组织起来，即使其中的组件经常出故障，我们也能构建出一个看起来很少出故障的系统。这正是分布式系统的美妙和价值所在，也是你在用的几乎每一个现代 Web 服务（包括 Google、Facebook 等）都建立在分布式系统之上的原因。

提示：通信本质上是不可靠的

几乎在任何情况下，都应该把通信看作一种从根本上不可靠的活动。比特损坏、链路和机器宕机或无法工作、缺少缓冲区（buffer）空间来容纳传入的数据包（packet），这些都会导致同一个结果：数据包有时到不了目的地。要在这样不可靠的网络之上构建可靠的服务，就必须考虑能够应对丢包（packet loss）的技术。

此外还有其他重要问题。系统**性能**（performance）往往至关重要；由于分布式系统靠网络连接在一起，系统设计者常常必须仔细考虑如何完成给定的任务，尽量减少发送的消息数，并让通信尽可能高效，做到低延迟、高带宽（bandwidth）。

最后，**安全**（security）也是必须考虑的问题。连接到远程站点时，如何确认对方确实是它自称的那一方，就成了核心问题。此外，确保第三方无法监视或篡改另外两方之间正在进行的通信，也是一项挑战。

在这篇介绍中，我们要讨论分布式系统中最基本的新问题：**通信**（communication）。也就是说，分布式系统中的机器应该如何彼此通信？我们会从最基本的原语（消息）讲起，再在它之上构建几个更高层的原语。如上所述，故障将是讨论的重点：通信层应该如何处理故障？

## 48.1 通信基础

现代网络的核心信条是：通信从根本上就是不可靠的。无论是在广域的互联网上，还是在 Infiniband 这样的局域高速网络中，数据包都经常会丢失、损坏，或者因为其他原因到不了目的地。

导致数据包丢失或损坏的原因多种多样。有时，传输过程中由于电气或其他类似问题，一些比特发生了翻转。有时，系统中的某个部件，比如网络链路、路由器，甚至远程主机，出于某种原因损坏了或者无法正常工作；网线也确实会被意外切断，至少偶尔如此。

然而，更根本的丢包原因，是网络交换机、路由器或端点内部的缓冲不足。具体来说，即使我们能保证所有链路都正常工作，系统中所有的组件（交换机、路由器、终端主机）都按预期正常运行，仍然可能丢包，原因如下。假设一个数据包到达一台路由器；为了处理这个数据包，必须把它放到路由器内部的某块内存里。如果同时有很多这样的数据包到达，路由器里的内存就可能容纳不下所有数据包。这时，路由器唯一的选择就是**丢弃**（drop）其中一个或多个数据包。终端主机上也会发生同样的情况：你向一台机器发送大量消息时，这台机器的资源很容易就会不堪重负，于是又会丢包。

所以说，丢包是网络中的基本问题。那么，该怎样应对丢包呢？

## 48.2 不可靠的通信层

一个简单的办法是：不去应对它。有些应用程序知道如何处理丢包，所以有时让它们直接用一个基本的、不可靠的消息层通信也很有用，这就是人们常说的**端到端原则**（end-to-end argument）的一个例子（见本章末尾的**补充**）。这种不可靠通信层的一个绝佳例子，就是如今几乎所有现代系统上都有的 **UDP/IP** 网络协议栈（networking stack）。要使用 UDP，进程（process）要通过**套接字**（sockets）API 创建一个**通信端点**（communication endpoint）；其他机器上（或者同一台机器上）的进程向这个进程发送 UDP **数据报**（datagram）。数据报就是一条大小有上限的定长消息。

图 48.1 和图 48.2 展示了一个构建在 UDP/IP 之上的简单客户端和服务器。客户端可以向服务器发送一条消息，服务器随后返回一条回复。只用这么一点代码，你就拥有了开始构建分布式系统所需的一切！

 © 2008–23, ARPACI-DUSSEAU

```
// 客户端代码
int main(int argc, char *argv[]) {
  int sd = UDP_Open(20000);
  struct sockaddr_in addrSnd, addrRcv;
  int rc = UDP_FillSockAddr(&addrSnd, "cs.wisc.edu", 10000);
  char message[BUFFER_SIZE];
  sprintf(message, "hello world");
  rc = UDP_Write(sd, &addrSnd, message, BUFFER_SIZE);
  if (rc > 0)
    int rc = UDP_Read(sd, &addrRcv, message, BUFFER_SIZE);
  return 0;
}

// 服务器代码
int main(int argc, char *argv[]) {
  int sd = UDP_Open(10000);
  assert(sd > -1);
  while (1) {
    struct sockaddr_in addr;
    char message[BUFFER_SIZE];
    int rc = UDP_Read(sd, &addr, message, BUFFER_SIZE);
    if (rc > 0) {
      char reply[BUFFER_SIZE];
      sprintf(reply, "goodbye world");
      rc = UDP_Write(sd, &addr, reply, BUFFER_SIZE);
    }
  }
  return 0;
}
```

图 48.1：**UDP 示例代码（client.c、server.c）**

UDP 是不可靠通信层的一个很好的例子。使用 UDP 时，你会遇到数据包丢失（被丢弃）、因而到不了目的地的情况，而发送方永远不会知道包丢了。不过，这并不意味着 UDP 对任何故障都毫无防范。例如，UDP 带有一个**校验和**（checksum），用来检测某些形式的数据包损坏。

然而，许多应用程序只是想把数据发送到目的地，而不想操心丢包问题，所以我们还需要更多机制。具体来说，我们需要在不可靠的网络之上实现可靠的通信。

## 48.3 可靠的通信层

要构建可靠的通信层，我们需要一些新的机制和技术来处理丢包。我们来看一个简单的例子：客户端通过一条不可靠的连接向服务器发送一条消息。首先要回答的问题是：发送方怎么知道接收方确实收到了消息？

```
int UDP_Open(int port) {
  int sd;
  if ((sd = socket(AF_INET, SOCK_DGRAM, 0)) == -1)
    return -1;
  struct sockaddr_in myaddr;
  bzero(&myaddr, sizeof(myaddr));
  myaddr.sin_family      = AF_INET;
  myaddr.sin_port        = htons(port);
  myaddr.sin_addr.s_addr = INADDR_ANY;
  if (bind(sd, (struct sockaddr *) &myaddr,
           sizeof(myaddr)) == -1) {
    close(sd);
    return -1;
  }
  return sd;
}

int UDP_FillSockAddr(struct sockaddr_in *addr,
                     char *hostname, int port) {
  bzero(addr, sizeof(struct sockaddr_in));
  addr->sin_family = AF_INET;          // 主机字节序
  addr->sin_port   = htons(port);      // 网络字节序
  struct in_addr *in_addr;
  struct hostent *host_entry;
  if ((host_entry = gethostbyname(hostname)) == NULL)
    return -1;
  in_addr = (struct in_addr *) host_entry->h_addr;
  addr->sin_addr = *in_addr;
  return 0;
}

int UDP_Write(int sd, struct sockaddr_in *addr,
              char *buffer, int n) {
  int addr_len = sizeof(struct sockaddr_in);
  return sendto(sd, buffer, n, 0, (struct sockaddr *)
                addr, addr_len);
}

int UDP_Read(int sd, struct sockaddr_in *addr,
             char *buffer, int n) {
  int len = sizeof(struct sockaddr_in);
  return recvfrom(sd, buffer, n, 0, (struct sockaddr *)
                  addr, (socklen_t *) &len);
}
```

图 48.2: **一个简单的 UDP 库（`udp.c`）**

提示：用校验和保证完整性

在现代系统中，校验和（checksum）是一种常用的方法，可以快速、有效地检测数据损坏。最简单的校验和就是加法：把一块数据的所有字节加起来；当然，人们还设计出了许多更复杂的校验和，包括基本的循环冗余校验（CRC）、Fletcher 校验和（Fletcher checksum）等等 [MK09]。

在网络中，校验和的用法如下。从一台机器向另一台机器发送消息之前，先对消息的字节计算一个校验和，然后把消息和校验和一起发往目的地。在目的地，接收方同样对收到的消息计算校验和；如果算出的校验和与发来的校验和一致，接收方就可以比较放心，认为数据在传输过程中很可能没有损坏。

评价校验和可以从几个不同的角度入手。有效性是一个首要考虑：数据发生变化时，校验和会不会随之改变？校验和越强，数据的变化就越难逃过检测。性能是另一个重要标准：计算校验和的代价有多大？遗憾的是，有效性和性能常常相互矛盾：高质量的校验和，计算代价往往也高。生活，又一次不完美。

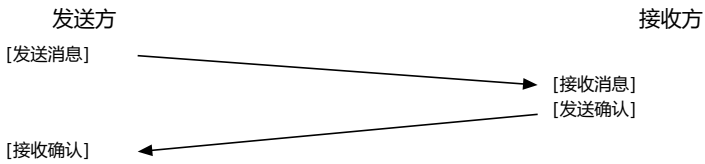



图 48.3：**消息加确认**

我们要用的技术叫作**确认**（acknowledgment），简称 **ack**。思路很简单：发送方向接收方发送一条消息；接收方随后回发一条简短的消息，确认自己收到了。图 48.3 描绘了这个过程。

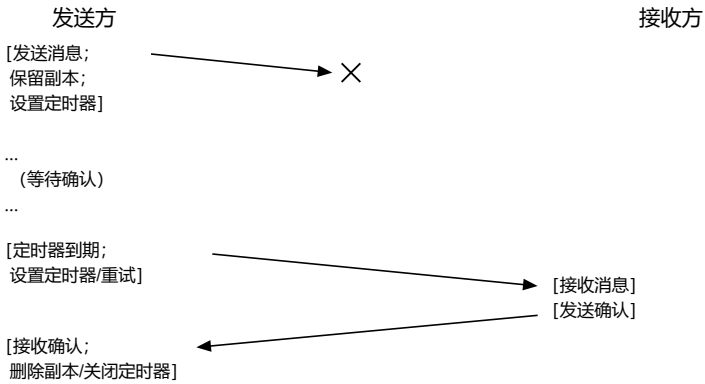



图 48.4：**消息加确认：请求丢失**

 © 2008–23, ARPACI-DUSSEAU

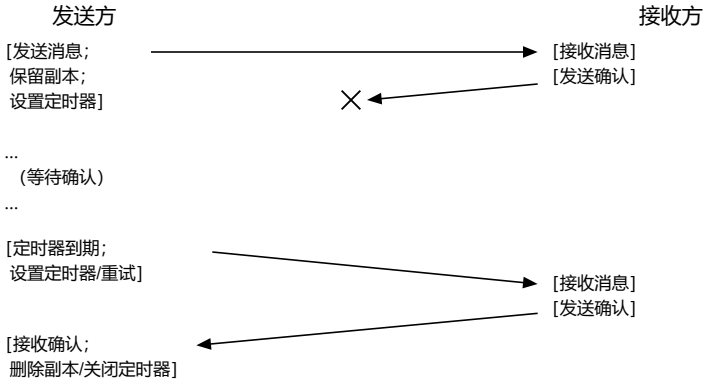



图 48.5：**消息加确认：回复丢失**

发送方收到这条消息的确认后，就可以确信接收方确实收到了原来的消息。可是，如果发送方没有收到确认，它该怎么办？

为了处理这种情况，我们还需要一个机制，叫作**超时**（timeout）。发送方发送消息时，会设置一个定时器，让它在一段时间后到期。如果在这段时间内没有收到确认，发送方就断定消息已经丢失，然后**重试**（retry）发送，把同一条消息再发一遍，希望这次能送达。为了让这种方法奏效，发送方必须保留一份消息副本，以备需要再次发送。超时和重试结合在一起，有人就把这种方法叫作**超时/重试**（timeout/retry）；搞网络的那帮人可真聪明，不是吗？图 48.4 给出了一个例子。

遗憾的是，这种形式的超时/重试还不够。图 48.5 给出了一个可能引发麻烦的丢包例子。在这个例子中，丢失的不是原来的消息，而是确认。从发送方的角度看，情况似乎是一样的：没有收到确认，因此应该超时并重试。但从接收方的角度看，情况就大不相同了：同一条消息被收到了两次！虽然有些情况下这样也没关系，但一般来说是不行的；想象一下，你下载一个文件时，下载内容里重复出现了多余的数据包，会发生什么。因此，要实现可靠的消息层，通常还得保证接收方对每条消息都**恰好接收一次**（exactly once）。

为了让接收方能够检测出重复传输的消息，发送方必须给每条消息加上唯一的标识，接收方也需要某种办法来记录自己是否已经见过每条消息。接收方发现重复传输时，只是确认这条消息，但（关键在于）不把它交给接收数据的应用程序。这样，发送方收到了确认，而消息又不会被接收两次，从而保持了上面说的恰好一次语义（exactly-once semantics）。

检测重复消息的办法有很多。例如，发送方可以为每条消息生成一个唯一的 ID；接收方可以记录它见过的每一个 ID。这种方法可行，但代价高得令人望而却步，因为要记录所有 ID，需要无限多的内存。

有一种更简单的机制能解决这个问题，而且几乎不占内存，叫作 **sequence counter**。使用 sequence counter 时，发送方和接收方先约定一个计数器的起始值（例如 1），双方各自维护这个计数器。每发送一条消息，就把计数器的当前值随消息一起发出去；这个计数器值（$N$）就充当这条消息的 ID。消息发出之后，发送方把计数器的值加一（变成 $N+1$）。

 © 2008–23, ARPACI-DUSSEAU

提示：设置超时值要小心

从上面的讨论中你大概也能猜到，用超时重发消息时，超时值（timeout value）设得对不对是个关键问题。超时值设得太小，发送方就会做无谓的重发，浪费发送方的 CPU 时间和网络资源。超时值设得太大，发送方要等很久才重发，它感受到的性能也会因此下降。因此，对单个客户端和服务器来说，“合适”的超时值应该刚好够发现丢包，不能再长。然而，正如我们在后面的章节中会看到的，分布式系统中往往不止一个客户端和一个服务器。在许多客户端向同一台服务器发送消息的场景中，服务器端的丢包可能意味着服务器已经过载。如果是这样，客户端也许应该换一种自适应的方式重试；例如，第一次超时之后，客户端可以把超时值调大一些，也许调到原来的两倍。这种**指数退避**（exponential back-off）方案由早期的 Aloha 网络首创，后来又被早期的以太网采用 [A70]，可以避免过多的重发把资源压垮。健壮的系统都会尽力避免这类过载。

接收方把自己的计数器值当作来自该发送方的下一条消息的预期 ID。如果收到的消息的 ID（$N$）与接收方的计数器值（同样是 $N$）相等，接收方就确认这条消息，并把它向上交给应用程序；在这种情况下，接收方断定这是第一次收到这条消息。然后，接收方把计数器加一（变成 $N+1$），等待下一条消息。

如果确认丢失了，发送方会超时并重发消息 $N$。这一次，接收方的计数器值更大（$N+1$），于是接收方知道自己已经收到过这条消息。它照样确认这条消息，但不把它向上交给应用程序。就这样，用 sequence counter 这种简单的方式就能避免重复。

最常用的可靠通信层叫作 **TCP/IP**，简称 **TCP**。TCP 比上面描述的要复杂得多，包括处理网络拥塞（network congestion）的机制 [VJ88]、多个未完成的请求，以及数以百计的其他小调整和优化。如果你好奇，可以去多读一些相关资料；更好的办法是去上一门网络课程，把这些内容学扎实。

## 48.4 通信抽象

有了基本的消息层，我们接下来讨论本章的下一个问题：构建分布式系统时，应该使用什么样的通信抽象（communication abstraction）？

多年来，系统研究界提出了许多方法。其中一类工作把操作系统的抽象（abstraction）拿过来，扩展到分布式环境中运行。例如，**分布式共享内存**（distributed shared memory，**DSM**）系统让不同机器上的进程能够共享一个很大的虚拟地址空间（virtual address space）[LH89]。这种抽象把分布式计算变成了看起来像多线程（multi-threaded）应用程序的东西；唯一的区别是，这些线程（thread）运行在不同的机器上，而不是同一台机器的不同处理器上。

大多数 DSM 系统都建立在操作系统的虚拟内存（virtual memory）系统之上。在一台机器上访问某一页（page）时，可能出现两种情况。第一种（最好的）情况是，这一页已经在本机上，于是数据很快就能取到。第二种情况是，这一页目前在其他某台机器上。这时会发生缺页异常（page fault），缺页异常处理程序（page-fault handler）向其他某台机器发送消息，取回这一页，把它装入发出请求的进程的页表（page table），然后继续执行。

 © 2008–23, ARPACI-DUSSEAU

由于种种原因，这种方法如今并没有得到广泛使用。DSM 最大的问题在于它如何处理故障。例如，设想一台机器出了故障，这台机器上的页会怎么样？如果分布式计算的数据结构（data structure）分散在整个地址空间（address space）中呢？在这种情况下，这些数据结构的一部分会突然变得不可用。地址空间的一部分消失了，要应对这样的故障是很难的；想象一个链表（linked list），其中某个“next”指针指向了地址空间中已经不存在的那部分。天哪！

另一个问题是性能。人们写代码时，通常假设访问内存的代价很低。在 DSM 系统中，有些访问代价很低，但另一些会引发缺页异常，需要代价高昂地从远程机器取数据。因此，这类 DSM 系统的程序员必须非常小心地组织计算，让它几乎不需要任何通信，而这在很大程度上违背了采用这种方法的初衷。虽然这个领域有过大量研究，但实际影响甚微；如今没有人用 DSM 构建可靠的分布式系统。

## 48.5 远程过程调用（RPC）

用操作系统的抽象构建分布式系统，结果并不理想；编程语言（PL）的抽象则要合理得多。最主流的抽象建立在**远程过程调用**（remote procedure call，简称 **RPC**）这一思想之上 [BN84][1]。

所有远程过程调用软件包都有一个简单的目标：让在远程机器上执行代码的过程，像调用本地函数一样简单直接。因此，在客户端看来，它发起一次过程调用（procedure call），过一段时间后，结果就返回了。服务器只需定义一些它想导出的例程，剩下的魔法都由 RPC 系统处理。RPC 系统一般由两部分组成：**存根生成器**（stub generator，有时也叫 **protocol compiler**）和**运行时库**（run-time library）。下面我们分别详细地看看这两部分。

### 存根生成器

存根生成器（stub generator）的任务很简单：把函数参数和结果自动打包成消息，省去手工打包的麻烦。这样做好处很多：从设计上就避免了手工编写这类代码时容易犯的简单错误；此外，存根编译器（stub compiler）或许还能优化这些代码，从而提高性能。

这种编译器的输入，就是服务器想导出给客户端的一组调用。从概念上讲，它可以简单到像这样：

```
interface {
  int func1(int arg1);
  int func2(int arg1, int arg2);
};
```

存根生成器（stub generator）拿到这样的接口后，会生成几段不同的代码。对客户端来说，会生成一个**客户端存根**（client stub），其中包含接口里列出的每一个函数；想使用这个 RPC 服务的客户端程序，只要链接这个客户端存根，并调用其中的函数，就能发起 RPC。

[1] 在现代编程语言中，我们也许会改称**远程方法调用**（remote method invocation，**RMI**），可是话说回来，谁会喜欢这些满是花哨对象的语言呢？

客户端存根中的每个函数，都会在内部完成执行远程过程调用所需的全部工作。在客户端看来，代码只是一次函数调用（例如，客户端调用 `func1(x)`）；而在内部，客户端存根中 `func1()` 的代码会做这些事：

- **创建消息缓冲区**。消息缓冲区（message buffer）通常就是一个有一定大小的连续字节数组。
- **把所需的信息打包进消息缓冲区**。这些信息包括要调用的函数的某种标识符，以及这个函数所需的全部参数（例如，在上面的例子中，`func1` 需要一个整数）。把这些信息全部放进一个连续缓冲区的过程，有时称为参数的**编组**（marshaling）或消息的**序列化**（serialization）。
- **把消息发送给目标 RPC 服务器**。与 RPC 服务器的通信，以及让通信正确运行所需的全部细节，都由 RPC 运行时库处理，下面会进一步介绍。
- **等待回复**。因为函数调用通常是**同步的**（synchronous），所以调用会等待它完成。
- **解包返回码和其他参数**。如果函数只返回一个返回码，这一步就很简单；但更复杂的函数可能返回更复杂的结果（例如一个列表），因此存根可能还需要把这些结果解包。这一步也叫**解组**（unmarshaling）或**反序列化**（deserialization）。
- **返回到调用者**。最后，从客户端存根返回到客户端代码中。

服务器端也会生成代码。服务器上的步骤如下：

- **解包消息**。这一步叫作**解组**（unmarshaling）或**反序列化**（deserialization）：从传入的消息中取出信息，得到函数标识符和参数。
- **调用实际的函数**。终于到了这一步！现在真正执行远程函数。RPC 运行时根据 ID 调用指定的函数，并传入所需的参数。
- **打包结果**。把返回的参数编组（marshal）回一个回复缓冲区（reply buffer）。
- **发送回复**。最后把回复发给调用者。

存根编译器（stub compiler）还要考虑另外几个重要问题。第一个是复杂参数，也就是说，如何打包并发送一个复杂的数据结构？例如，调用 `write()` 系统调用（system call）时，要传入三个参数：一个整数文件描述符（file descriptor）、一个指向缓冲区的指针，以及要写入的字节数（从指针处算起）。如果传给 RPC 软件包的是一个指针，它就得知道该怎样解释这个指针，并做出正确的处理。通常的做法有两种：一种是使用众所周知的类型（例如 RPC 编译器能够理解的 `buffer_t`，用来按给定大小传递数据块）；另一种是给数据结构加上更多注解信息，让编译器知道哪些字节需要序列化（serialize）。

另一个重要问题是服务器在并发（concurrency）方面如何组织。最简单的服务器只是在一个循环里等待请求，一次处理一个。但你大概已经猜到，这样做的效率可能极低：如果一次 RPC 调用阻塞了（例如在 I/O 上阻塞），服务器资源就被浪费了。因此，大多数服务器内部都是并发的。一种常见的组织方式是**线程池**（thread pool）：服务器启动时创建一组数量有限的线程；消息到达时，把它分派给其中一个工作线程（worker thread），由这个工作线程完成 RPC 调用的工作，最后返回回复；在此期间，主线程（main thread）继续接收其他请求，并可能把它们分派给其他工作线程。这样的组织方式让服务器内部可以并发执行（concurrent execution），从而提高了服务器的利用率（utilization）；常见的代价也随

之而来，主要是编程复杂度，因为 RPC 调用现在可能需要使用锁（lock）和其他同步原语（synchronization primitive）来确保正确运行。

### 运行时库

运行时库承担了 RPC 系统中的大部分繁重工作；大多数性能和可靠性（reliability）问题都在这里处理。下面讨论构建这样一个运行时层的几个主要挑战。

我们首先要克服的挑战之一，是如何找到远程服务。这个**命名**（naming）问题在分布式系统中很常见，在某种意义上也超出了我们当前讨论的范围。最简单的方法建立在现有的命名系统之上，例如当前互联网协议（internet protocol）提供的主机名和端口号（port number）。在这样的系统中，客户端必须知道运行所需 RPC 服务的机器的主机名或 IP 地址，以及该服务使用的端口号。端口号只是标识一台机器上某一项特定通信活动的方式，让一台机器能同时拥有多个通信通道。然后，协议族（protocol suite）必须提供一种机制，把数据包从系统中的任何其他机器路由到某个特定地址。关于命名的深入讨论，你得去别处找，例如去了解互联网上的 DNS 和名字解析（name resolution）；更好的办法是直接读 Saltzer 和 Kaashoek 那本书中精彩的一章 [SK09]。

客户端知道了某项远程服务该找哪台服务器之后，下一个问题是：RPC 应该建立在哪种传输层协议（transport-level protocol）之上？具体来说，RPC 系统应该使用 TCP/IP 这样的可靠协议（protocol），还是建立在 UDP/IP 这样的不可靠通信层之上？

乍一看，这个选择似乎很简单：我们显然希望请求能可靠地送达远程服务器，也显然希望能可靠地收到回复。所以，我们应该选择 TCP 这样的可靠传输协议，对吧？

遗憾的是，在可靠通信层之上构建 RPC，可能造成严重的性能浪费。回想一下上面讨论的可靠通信层是如何工作的：靠确认加上超时/重试。因此，客户端向服务器发送 RPC 请求时，服务器会回复一个确认，好让调用者知道请求已经收到。同样，服务器向客户端发送回复时，客户端也会确认，好让服务器知道回复已经收到。在可靠通信层之上构建请求/响应协议（request/response protocol），比如 RPC，就会多发送两条“额外”的消息。

因此，许多 RPC 软件包都建立在 UDP 这样的不可靠通信层之上。这样 RPC 层可以更高效，但提供可靠性的责任也就落到了 RPC 系统身上。RPC 层靠超时/重试和确认来达到所需的可靠性，做法和上面描述的差不多。借助某种形式的序号（sequence number），通信层可以保证每次 RPC 在没有故障时恰好执行一次（exactly once），在发生故障时至多执行一次（at most once）。

### 其他问题

RPC 运行时还必须处理其他一些问题。例如，如果一次远程调用需要很长时间才能完成，会怎么样？考虑到我们的超时机制，一个长时间运行的远程调用在客户端看来可能就像发生了故障，从而触发重试，所以这里需要小心处理。一种解决办法是：如果回复不能马上生成，接收方先给发送方发一个显式的确认，让客户端知道服务器已经收到了请求。此后，客户端可以定期询问服务器是否还在处理这个请求；如果服务器一直回答“是”，客户端就应该放心地继续等待（毕竟，过程调用有时确实要执行很久才能完成）。

 © 2008–23, ARPACI-DUSSEAU

补充：端到端原则

**端到端原则**（end-to-end argument）认为，在分层系统中，最高层（通常就是位于“端”上的应用程序）最终是唯一能够真正实现某些功能的地方。Saltzer 等人在他们那篇具有里程碑意义的论文 [SRC84] 中，用一个绝佳的例子论证了这一点：两台机器之间的可靠文件传输。如果你想把一个文件从机器 $A$ 传到机器 $B$，并且确保最终落在 $B$ 上的字节与从 $A$ 出发时完全相同，你就必须对此做一次“端到端”的检查；更底层的可靠机制，例如网络或磁盘中的可靠机制，并不能提供这样的保证。

与之相对的做法，是给系统的较低层增加可靠性，试图以此解决可靠文件传输问题。例如，假设我们构建了一个可靠的通信协议（communication protocol），并用它来实现可靠的文件传输。这个通信协议（communication protocol）保证发送方发出的每一个字节都会被接收方按顺序收到，比如靠超时/重试、确认和序号（sequence number）来实现。可惜，光有这样的协议，并不能保证文件传输可靠：想象一下，在通信发生之前，字节就已经在发送方的内存中损坏了；或者接收方把数据写到磁盘时出了什么岔子。在这些情况下，即使字节在网络上被可靠地传送了，我们的文件传输最终也是不可靠的。要构建可靠的文件传输，就必须加入端到端的可靠性检查，例如，在整个传输完成之后，把接收方磁盘上的文件读回来，计算校验和，再和发送方文件的校验和比较。

这条准则的推论是，有时让较低层提供额外的功能，确实可以提高系统性能，或者以其他方式优化系统。因此，不应一概排斥在较低层设置这类机制，而应结合它在整个系统或应用中的最终用途，仔细权衡它的作用。

运行时还必须处理参数很大的过程调用，这些参数大到单个数据包都装不下。有些较低层的网络协议（network protocol）提供了这种发送方的**分片**（fragmentation，把较大的数据包分成一组较小的数据包）和接收方的**重组**（reassembly，把较小的部分重新组合成一个较大的逻辑整体）；如果没有，RPC 运行时可能就得自己实现这种功能。详情请参阅 Birrell 和 Nelson 的论文 [BN84]。

许多系统都要处理的一个问题是**字节序**（byte ordering）。你可能知道，有些机器按所谓的**大端**（big endian）顺序存储数值，另一些则使用**小端**（little endian）顺序。大端从最高有效位（most significant bit）到最低有效位（least significant bit）依次存储（比如一个整数的）各个字节，很像阿拉伯数字的写法；小端则正好相反。两种方式都是存储数值信息的有效方式；这里的问题是，字节序不同的机器之间如何通信。

RPC 软件包通常在消息格式中规定一种明确的字节序来解决这个问题。在 Sun 的 RPC 软件包中，**XDR**（**eXternal Data Representation**，外部数据表示）层提供了这一功能。如果收发消息的机器字节序与 XDR 一致，消息就照常发送和接收。但如果通信的机器字节序不同，消息中的每一项信息都必须转换。因此，字节序的差异会带来少许性能开销。

最后一个问题是，要不要把通信的异步（asynchronous）本质暴露给客户端，以便做一些性能优化。具体来说，典型的 RPC 是**同步**（synchronously）进行的，也就是说，客户端发起过程调用后，必须等过程调用返回才能继续执行。因为这种等待可能很长，而客户端也许还有其他工作可做，所以有些 RPC 软件包允许你**异步**（asynchronously）地发起 RPC。

 © 2008–23, ARPACI-DUSSEAU

发起异步 RPC 时，RPC 软件包发出请求后立即返回；客户端随后可以自由地做其他工作，比如调用其他 RPC 或做其他有用的计算。客户端到了某个时候会想看看异步 RPC 的结果；于是它再调用 RPC 层，让它等待未完成的 RPC 完成，然后就可以访问返回的参数了。

## 48.6 小结

我们介绍了分布式系统这个新主题，以及它的主要问题：如何应对已经司空见惯的故障。就像 Google 内部的说法：如果你只有一台台式机，故障很少见；可要是身处一个有成千上万台机器的数据中心，故障就无时无刻不在发生。任何分布式系统的关键，都在于你如何应对故障。

我们还看到，通信是一切分布式系统的核心。远程过程调用是对这种通信的一种常见抽象，它让客户端能够在服务器上发起远程调用；RPC 软件包处理所有繁琐的细节，包括超时/重试和确认，以提供一种与本地过程调用十分相像的服务。

要真正理解 RPC 软件包，最好的办法当然是亲自用一用。Sun 的 RPC 系统使用存根编译器（stub compiler）`rpcgen`，算是比较老的一个；Google 的 gRPC 和 Apache Thrift 则是同类系统的现代实现。挑一个试试，看看它们到底有什么了不起。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[A70] "The ALOHA System — Another Alternative for Computer Communications" by Norman Abramson. The 1970 Fall Joint Computer Conference. *The ALOHA network pioneered some basic concepts in networking, including exponential back-off and retransmit, which formed the basis for communication in shared-bus Ethernet networks for years.*

[BN84] "Implementing Remote Procedure Calls" by Andrew D. Birrell, Bruce Jay Nelson. ACM TOCS, Volume 2:1, February 1984. *The foundational RPC system upon which all others build. Yes, another pioneering effort from our friends at Xerox PARC.*

[MK09] "The Effectiveness of Checksums for Embedded Control Networks" by Theresa C. Maxino and Philip J. Koopman. IEEE Transactions on Dependable and Secure Computing, 6:1, January '09. *A nice overview of basic checksum machinery and some performance and robustness comparisons between them.*

[LH89] "Memory Coherence in Shared Virtual Memory Systems" by Kai Li and Paul Hudak. ACM TOCS, 7:4, November 1989. *The introduction of software-based shared memory via virtual memory. An intriguing idea for sure, but not a lasting or good one in the end.*

[SK09] "Principles of Computer System Design" by Jerome H. Saltzer and M. Frans Kaashoek. Morgan-Kaufmann, 2009. *An excellent book on systems, and a must for every bookshelf. One of the few terrific discussions on naming we've seen.*

[SRC84] "End-To-End Arguments in System Design" by Jerome H. Saltzer, David P. Reed, David D. Clark. ACM TOCS, 2:4, November 1984. *A beautiful discussion of layering, abstraction, and where functionality must ultimately reside in computer systems.*

[VJ88] "Congestion Avoidance and Control" by Van Jacobson. SIGCOMM '88. *A pioneering paper on how clients should adjust to perceived network congestion; definitely one of the key pieces of technology underlying the Internet, and a must read for anyone serious about systems, and for Van Jacobson's relatives because well relatives should read all of your papers.*

 © 2008–23, ARPACI-DUSSEAU

## 作业（编程）

在这一部分，我们会写一些简单的通信代码，让你熟悉怎么做这件事。玩得开心！

## 问题

1. 用本章提供的代码，构建一个简单的基于 UDP 的服务器和客户端。服务器应该接收来自客户端的消息，并回复一个确认。第一次尝试时，不要加任何重传或健壮性机制（假设通信完全正常）。先在一台机器上运行测试，之后再在两台不同的机器上运行。
2. 把你的代码改造成一个**通信库**（communication library）。具体来说，设计你自己的 API，包括发送和接收调用，以及其他需要的 API 调用。改写你的客户端和服务器，让它们使用你的库，而不是直接调用原始的套接字接口。
3. 给你日渐成形的通信库（communication library）加上可靠通信，做法是**超时/重试**（timeout/retry）。具体来说，你的库应该为每条要发送的消息保留一份副本。发送时，应该启动一个定时器，以便记录消息发出后过了多长时间。在接收方，库应该**确认**（acknowledge）收到的消息。客户端在发送时应该**阻塞**（block），也就是说，它应该等到消息被确认之后才返回。它还应该不限次数地重试发送。消息的最大长度应该是用 UDP 能发送的最大单条消息的长度。最后，一定要高效地实现超时/重试：让调用者睡眠，直到确认到达或传输超时；不要自旋（spin）浪费 CPU！
4. 让你的库更高效、功能更丰富。首先，加入超大消息的传输。具体来说，尽管网络限制了消息的最大长度，你的库也应该能接收任意大小的消息，并把它从客户端传到服务器。客户端应该把这些大消息分成若干片段发给服务器；服务器端的库代码应该把收到的片段组装成连续的整体，再把这个大缓冲区交给等待中的服务器代码。
5. 再做一遍上面的事，但要做到高性能。不要一次只发一个片段，而应该快速地发送许多片段，从而让网络得到充分得多的利用。为此，要仔细标记传输中的每一个片段，确保接收方重组时不会把消息弄乱。
6. 最后一个实现上的挑战：按序交付的异步消息发送。也就是说，客户端应该能够反复调用发送函数，一条接一条地发送消息；接收方应该调用接收函数，可靠地按顺序收到每一条消息；发送方的许多消息应该可以同时在网络上传输。再加一个发送方的调用，让客户端可以等待所有未完成的消息都被确认。
7. 现在还有一个痛点：测量。测量你每种方法的带宽：在两台不同的机器之间能传输多少数据，速率是多少？再测量延迟：发送单个数据包并收到确认，要多久才能完成？最后，你的数据看起来合理吗？你原本的预期是什么？怎样才能更好地设定预期，以便判断是有问题，还是你的代码运行良好？

 © 2008–23, ARPACI-DUSSEAU

# 49

# Sun 的网络文件系统（NFS）

分布式客户端/服务器（client/server）计算最早的用途之一，就是分布式文件系统（distributed file system）。在这样的环境里，有许多台客户端机器和一台（或少数几台）服务器；服务器把数据存放在自己的磁盘上，客户端则用格式规范的协议消息（protocol message）请求数据。图 49.1 展示了基本的结构。

从图中可以看到，磁盘在服务器上，客户端通过网络发送消息，访问这些磁盘上的目录（directory）和文件。为什么要费事采用这种结构呢？（也就是说，为什么不直接让客户端使用自己的本地磁盘？）主要是因为这种结构便于客户端之间**共享**（sharing）数据。如果你在一台机器（客户端 0）上访问了某个文件，之后又换到另一台（客户端 2），你看到的文件系统（file system）是一样的。你的数据自然而然地在这些不同的机器之间共享。第二个好处是**集中管理**（centralized administration）；例如，备份文件只需在少数几台服务器上做，不必在众多客户端上逐台去做。另一个好处可能是**安全**（security）；把所有服务器都放在上了锁的机房里，可以避免某些类型的问题。

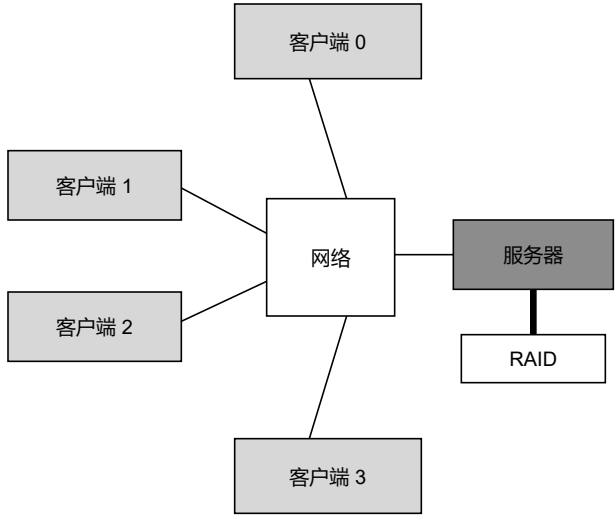



图 49.1：**一个通用的客户端/服务器系统**

关键问题：如何构建分布式文件系统
怎样构建分布式文件系统？需要考虑哪些关键方面？哪些地方容易出错？能从现有系统中学到什么？

## 49.1 一个基本的分布式文件系统

现在我们来研究一个简化的分布式文件系统的架构。简单的客户端/服务器分布式文件系统，组件比我们目前研究过的文件系统要多。在客户端一侧，客户端应用程序通过**客户端文件系统**（client-side file system）访问文件和目录。客户端应用程序向客户端文件系统发出**系统调用**（system call），例如 `open()`、`read()`、`write()`、`close()`、`mkdir()` 等，以访问存放在服务器上的文件。因此，在客户端应用程序看来，这个文件系统和本地（基于磁盘的）文件系统没有什么不同，只是性能上可能有差别；这样，分布式文件系统就提供了对文件的**透明**（transparent）访问。这是一个显而易见的目标；毕竟，谁会愿意用一个需要另一套 API、或者用起来很麻烦的文件系统呢？

客户端文件系统的作用，是完成这些系统调用所需的操作。例如，如果客户端发出 `read()` 请求，客户端文件系统可能会向**服务器端文件系统**（server-side file system，通常也叫**文件服务器**，file server）发送一条消息，要求读取某个特定的块（block）；文件服务器随后从磁盘或它自己的内存缓存（in-memory cache）中读出这个块，再发一条消息把请求的数据送回客户端。然后，客户端文件系统把数据复制到 `read()` 系统调用提供的用户缓冲区（user buffer）中，请求就完成了。注意，客户端之后再次 `read()` 同一个块时，这个块可能已经**缓存**（cached）在客户端的内存里，甚至在客户端的磁盘上；在最好的情况下，根本不需要产生网络流量。

从这个简单的概述中，你应该能体会到，客户端/服务器分布式文件系统里有两部分重要的软件：客户端文件系统和文件服务器。它们的行为共同决定了分布式文件系统的行为。现在是时候研究一个具体的系统了：Sun 的网络文件系统。

## 49.2 走近 NFS

最早的、也相当成功的分布式系统之一由 Sun Microsystems 开发，称为 Sun 网络文件系统（Sun Network File System，NFS）[S86]。在定义 NFS 时，Sun 采取了一种不同寻常的做法：它没有构建一个专有的封闭系统，而是开发了一个**开放协议**（open protocol），只规定客户端和服务器通信时使用的确切消息格式。不同的团队可以开发各自的 NFS 服务




图 49.2：**分布式文件系统的架构**

补充：服务器为什么会崩溃

在深入讨论 NFSv2 协议（protocol）的细节之前，你可能想知道：服务器为什么会崩溃？你也许猜到了，原因有很多。服务器可能只是遇到了（暂时的）**停电**（power outage）；只有恢复供电后，机器才能重新启动。服务器通常由几十万甚至上百万行代码组成；所以它们有 **bug**（即使是好软件，每一百行或一千行代码里也会有几个 bug），最终总会触发某个 bug，导致崩溃。它们还有内存泄漏（memory leak）；哪怕是很小的内存泄漏，也会让系统耗尽内存而崩溃。最后，在分布式系统（distributed system）中，客户端和服务器之间隔着网络；网络行为异常时，看起来就好像远程机器崩溃了，而实际上它只是暂时无法通过网络访问。例如，网络可能发生了**分区**（partitioned）：客户端和服务器都在正常工作，却无法相互通信。

器，在 NFS 市场上展开竞争，同时又保持互操作性（interoperability）。这一做法奏效了：如今有很多公司销售 NFS 服务器（包括 Oracle/Sun、NetApp [HLM94]、EMC、IBM 等），NFS 的广泛成功很可能要归功于这种“开放市场”的做法。

## 49.3 重点：简单、快速的服务器崩溃恢复

本章讨论的是经典的第 2 版 NFS 协议，即 NFSv2，它多年来一直是标准；升级到 NFSv3 时做了一些小改动，升级到 NFSv4 时协议则有较大规模的变化。不过，NFSv2 既精彩又让人头疼，因此是我们关注的重点。

在 NFSv2 中，协议设计的主要目标是简单、快速的服务器崩溃恢复。在多客户端、单服务器的环境里，这个目标很有道理；服务器宕机（或不可用）的每一分钟，都会让所有客户端机器（以及它们的用户）既不满意，也没法干活。所以，服务器怎样，整个系统就怎样。

## 49.4 快速崩溃恢复的关键：无状态

NFSv2 实现这个简单目标的办法，是设计一种我们称为**无状态**（stateless）的协议。按照设计，服务器不记录每个客户端那里正在发生的任何事情。例如，服务器不知道哪些客户端缓存了哪些块，不知道每个客户端当前打开了哪些文件，也不知道某个文件当前的文件指针位置，等等。简单地说，服务器不记录客户端在做什么；相反，协议的设计让每个协议请求都携带完成该请求所需的全部信息。如果你现在还不太明白，等下面更详细地讨论这个协议时，这种无状态的做法就会更好理解了。

来看一个**有状态**（stateful）而非无状态的协议的例子：`open()` 系统调用。给定一个路径名（pathname），`open()` 返回一个文件描述符（file descriptor），也就是一个整数。之后的 `read()` 或 `write()` 请求用这个描述符访问文件的各个块，如下面的应用程序代码所示（注意，为节省篇幅，省略了对系统调用的错误检查）：

现在设想，客户端文件系统打开文件的方式，是向服务器发送一条协议消息：“打开文件‘foo’，把描述符给我”。文件服务器随后在自己那一侧打开这个文件，把描述符发回客户端。之后读取时，客户端应用程序用这个描述符调用 `read()` 系统调用；客户端文件系

```
char buffer[MAX];
int fd = open("foo", O_RDONLY); // 获取描述符 "fd"
read(fd, buffer, MAX);     // 通过 "fd" 从 foo 读取 MAX 字节
read(fd, buffer, MAX);     // 再读 MAX 字节
...
read(fd, buffer, MAX);     // 再读 MAX 字节
close(fd);                 // 关闭文件
```

图 49.3：**客户端代码：读取文件**

统再把描述符放在消息里传给文件服务器，意思是："从我现在传给你的这个描述符所指的文件里，读一些字节"。

在这个例子中，文件描述符是客户端和服务器之间的一份**共享状态**（shared state，Ousterhout 称之为**分布式状态**，distributed state [O91]）。前面已经暗示过，共享状态（shared state）会让崩溃恢复（crash recovery）变得复杂。设想第一次读取完成之后、客户端发出第二次读取之前，服务器崩溃了。服务器重新启动并运行之后，客户端发出第二次读取。可惜，服务器根本不知道 `fd` 指的是哪个文件；这个信息是临时的（即只存在于内存中），服务器崩溃时就丢失了。为了处理这种情况，客户端和服务器必须执行某种**恢复协议**（recovery protocol）：客户端要确保自己的内存里保留足够的信息，以便告诉服务器它需要知道的东西（在这个例子里，就是文件描述符 `fd` 指向文件 `foo`）。

考虑到有状态的服务器还必须处理客户端崩溃，情况就更糟了。例如，设想一个客户端打开了一个文件，然后崩溃了。这次 `open()` 占用了服务器上的一个文件描述符；服务器怎么知道什么时候可以关闭某个文件呢？正常运行时，客户端最终会调用 `close()`，从而通知服务器应该关闭这个文件。然而，客户端崩溃时，服务器永远收不到 `close()`，因此必须察觉到客户端已经崩溃，才能关闭这个文件。

出于这些原因，NFS 的设计者决定采用无状态的方法：每个客户端操作都包含完成该请求所需的全部信息。不需要花哨的崩溃恢复；服务器只要重新开始运行就行了，客户端最坏也就是重试（retry）一次请求。

## 49.5 NFSv2 协议

下面来看 NFSv2 协议的定义。我们的问题很简单：

关键问题：如何定义无状态的文件协议

如何定义网络协议（network protocol），才能支持无状态的操作？显然，像 `open()` 这样有状态的调用不在考虑之列（因为它要求服务器记录打开的文件）；然而，客户端应用程序仍然会调用 `open()`、`read()`、`write()`、`close()` 等标准 API 来访问文件和目录。因此，更精确地说，问题是：如何定义一个协议，使它既是无状态的，又能支持 POSIX 文件系统 API？

理解 NFS 协议设计的一个关键，是理解**文件句柄**（file handle）。文件句柄（file handle）用来唯一地描述某个操作要作用的文件或目录；因此，很多协议请求（protocol request）里都带有文件句柄（file handle）。

文件句柄（file handle）可以看成由三个重要部分组成：卷标识符（volume identifier）、*inode* 号（inode number）和世代号（generation number）；这三者合在一起，构成了客户端想要访问的文件或目录的唯一标识。卷标识符（volume identifier）告诉服务器，请求针对的是哪个文件系统，因为一台 NFS 服务器可以导出不止一个文件系统；inode 号告诉服务器，请求要访问该分区中的哪个文件。最后，重用 inode 号时需要用到世代号（generation number）：每当重用一个 inode 号，服务器就把世代号加一，从而确保持有旧文件句柄（file handle）的客户端不会意外访问到新分配的文件。

图 49.4 概括了协议中一些重要的部分；完整的协议可以在别处找到（Callaghan 的书 [C00] 对 NFS 有出色而详细的概述）。

我们简要介绍一下协议中的重要组成部分。首先，LOOKUP 协议消息用于获取文件句柄（file handle），之后再用这个句柄访问文件数据。客户端传入一个目录的文件句柄（file handle）和要查找的文件名，服务器把该文件（或目录）的句柄及其属性传回客户端。

例如，假设客户端已经拿到了文件系统根目录（root directory，即 `/`）的目录文件句柄（directory file handle）。实际上，这个句柄是用 NFS **挂载协议**（mount protocol）得到的，客户端和服务器最初也正是靠它连接起来的；为简洁起见，这里不讨论挂载协议（mount protocol）。如果客户端上运行的应用程序打开文件 `/foo.txt`，客户端文件系统就会向服务器发送一个 lookup 请求，传入根目录的文件句柄（file handle）和名字 `foo.txt`；如果成功，就会返回 `foo.txt` 的文件句柄（file handle）和属性。

| | |
|---|---|
| `NFSPROC_GETATTR` | 文件句柄<br>返回：属性 |
| `NFSPROC_SETATTR` | 文件句柄，属性<br>返回：属性 |
| `NFSPROC_LOOKUP` | 目录的文件句柄，要查找的文件/目录名<br>返回：文件句柄，属性 |
| `NFSPROC_READ` | 文件句柄，偏移量，字节数<br>数据，属性 |
| `NFSPROC_WRITE` | 文件句柄，偏移量，字节数，数据<br>属性 |
| `NFSPROC_CREATE` | 目录的文件句柄，文件名，属性<br>文件句柄，属性 |
| `NFSPROC_REMOVE` | 目录的文件句柄，要删除的文件名<br>– |
| `NFSPROC_MKDIR` | 目录的文件句柄，目录名，属性<br>文件句柄，属性 |
| `NFSPROC_RMDIR` | 目录的文件句柄，要删除的目录名<br>– |
| `NFSPROC_READDIR` | 目录句柄，要读取的字节数，cookie<br>返回：目录项，cookie（用于获取更多目录项） |

图 49.4：**NFS 协议：示例**

如果你好奇的话：属性就是文件系统为每个文件记录的元数据（metadata），包括文件创建时间、最后修改时间、大小、所有者和权限信息等字段，也就是对文件调用 `stat()` 时会得到的那类信息。

有了文件句柄（file handle），客户端就可以针对文件发出 READ 和 WRITE 协议消息，分别读写这个文件。READ 协议消息要带上文件句柄（file handle），以及文件内的偏移量（offset）和要读取的字节数。服务器据此就能完成读取：句柄告诉它要读哪个卷（volume）、哪个 inode，偏移量和字节数告诉它要读文件的哪些字节。随后，服务器把数据（以及最新的属性）返回给客户端；出错时则返回错误。WRITE 的处理与此类似，只是数据从客户端传到服务器，服务器返回的也只有一个成功码（以及最新的属性）。

最后一个值得一提的协议消息是 GETATTR 请求：它只是根据文件句柄（file handle）取出该文件的属性，其中包括文件的最后修改时间。下面讨论缓存（caching）时，我们会看到这个协议请求在 NFSv2 中为什么很重要（你能猜到为什么吗?）。

## 49.6 从协议到分布式文件系统

希望你现在已经大致明白，这个协议是如何借助客户端文件系统和文件服务器变成一个文件系统的。客户端文件系统记录打开的文件，并且一般会把应用程序的请求转换成相应的一组协议消息。服务器只负责响应协议消息，每条消息都包含完成请求所需的全部信息。

例如，我们来看一个读取文件的简单应用程序。图 49.5 给出了应用程序发出的系统调用，以及客户端文件系统和文件服务器为响应这些调用所做的事情。

关于这张图，有几点说明。第一，请注意客户端如何记录这次文件访问的所有相关**状态**（state），包括从整数文件描述符到 NFS 文件句柄（file handle）的映射，以及当前的文件指针。这样，客户端就能把每个读请求（你可能已经注意到，读请求并不显式指定从哪里开始读）转换成格式正确的读协议消息，准确地告诉服务器要读文件的哪些字节。读取成功后，客户端更新当前文件位置；后续的读取使用同一个文件句柄（file handle），但偏移量不同。

第二，你可能会注意到与服务器的交互发生在什么时候。第一次打开文件时，客户端文件系统发送一条 LOOKUP 请求消息。实际上，如果要遍历一个很长的路径名，例如 `/home/remzi/foo.txt`，客户端会发送三个 LOOKUP：一个在目录 / 中查找 `home`，一个在 `home` 中查找 `remzi`，最后一个在 `remzi` 中查找 `foo.txt`。

第三，你可能会注意到，每个服务器请求都包含完成该请求所需的全部信息。这个设计要点对于从服务器故障中平稳恢复至关重要，下面我们会更详细地讨论；它确保服务器不需要状态就能响应请求。

## 49.7 用幂等操作处理服务器故障

客户端向服务器发送消息后，有时收不到回复。没有回复的原因可能有很多。有时是消息被网络丢弃了；网络确实会丢失消息，所以请求或回复都可能丢失，客户端也就永远收不到响应。

| 客户端 | 服务器 |
| --- | --- |
| **fd = open(“/foo”, …);**<br>发送 LOOKUP (rootdir FH, “foo”) | |
| | 接收 LOOKUP 请求<br>在根目录中查找 “foo”<br>返回 foo 的 FH + 属性 |
| 接收 LOOKUP 回复<br>在打开文件表中分配文件描述符<br>把 foo 的 FH 存入表中<br>记录当前文件位置（0）<br>把文件描述符返回给应用程序 | |
| **read(fd, buffer, MAX);**<br>用 fd 索引打开文件表<br>取得 NFS 文件句柄（FH）<br>用当前文件位置作为偏移量<br>发送 READ (FH, offset=0, count=MAX) | |
| | 接收 READ 请求<br>用 FH 得到卷号/inode 号<br>从磁盘（或缓存）读取 inode<br>计算块的位置（利用偏移量）<br>从磁盘（或缓存）读取数据<br>把数据返回给客户端 |
| 接收 READ 回复<br>更新文件位置（+读取的字节数）<br>把当前文件位置设为 MAX<br>把数据/错误码返回给应用程序 | |
| **read(fd, buffer, MAX);**<br>同上，只是 offset=MAX，并把当前文件位置设为 2*MAX | |
| **read(fd, buffer, MAX);**<br>同上，只是 offset=2*MAX，并把当前文件位置设为 3*MAX | |
| **close(fd);**<br>只需清理本地结构<br>释放打开文件表中的描述符 “fd”<br>（无须与服务器通信） | |

图 49.5：**读取文件：客户端和文件服务器的操作**

也有可能是服务器崩溃了，因此暂时不响应消息。过一会儿，服务器会重新启动并开始运行，但在此期间，所有请求都丢失了。在所有这些情况下，客户端都面临一个问题：服务器没有及时回复时，该怎么办？

在 NFSv2 中，客户端用一种统一而优雅的方式处理所有这些故障：它只是重试（retry）请求。具体来说，发送请求后，客户端设置一个定时器，在指定的时间之后触发。如果在定时器触发之前收到了回复，就取消定时器，一切正常。但如果定时器在收到任何回复之前

提示：幂等性很强大

**幂等性**（idempotency）是构建可靠系统时一个很有用的性质。如果一个操作可以发出不止一次，处理这个操作的失败就容易得多：直接重试就行了。如果一个操作不是幂等（idempotent）的，日子就难过多了。

就触发了，客户端就认为请求没有被处理，于是重新发送。如果服务器回复了，一切正常，客户端也就漂亮地解决了问题。

客户端之所以能简单地重试请求（不管故障是由什么引起的），是因为大多数 NFS 请求有一个重要性质：它们是**幂等的**（idempotent）。如果一个操作执行多次的效果和执行一次的效果相同，这个操作就称为幂等的。例如，往某个内存位置存一个值，存三次和存一次是一样的；所以“把值存入内存”是幂等操作。但如果把一个计数器加三次，结果就和只加一次不同；所以“计数器加一”不是幂等的。更一般地说，任何只读取数据的操作显然都是幂等的；而更新数据的操作，则必须仔细考虑才能确定它是否具有这个性质。

NFS 崩溃恢复设计的核心，就在于大多数常用操作的幂等性。LOOKUP 和 READ 请求显然是幂等的，因为它们只从文件服务器读取信息，不做更新。更有意思的是，WRITE 请求也是幂等的。例如，如果一次 WRITE 失败了，客户端可以直接重试。WRITE 消息包含数据、字节数，以及（重要的是）写入数据的确切偏移量。因此，它可以重复执行，而且我们知道，多次写入的结果和一次写入的结果相同。

这样，客户端就能用统一的方式处理所有超时（timeout）。如果一个 WRITE 请求只是丢失了（上面的情况 1），客户端会重试，服务器会执行写入，一切都会正常。如果发送请求时服务器恰好宕机，而发送第二个请求时服务器已经恢复运行，结果也一样，一切照常（情况 2）。最后，服务器可能实际上已经收到了 WRITE 请求，把数据写到了磁盘，并发送了回复。这个回复可能丢失（情况 3），同样会导致客户端重新发送请求。服务器再次收到请求时，只会做完全相同的事情：把数据写入磁盘，然后回复说已经写好了。如果客户端这次

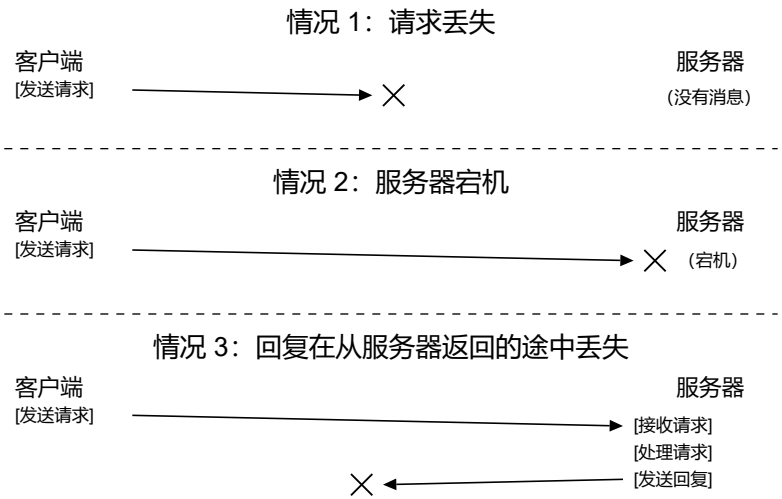



图 49.6：**三种丢失情况**

 © 2008–25, ARPACI-DUSSEAU

提示：完美是优秀的敌人（伏尔泰定律）
即使你设计了一个漂亮的系统，有时也不是所有的边界情况都能如你所愿。以上面的 mkdir 为例：可以重新设计 mkdir，赋予它不同的语义，从而让它变成幂等的（想想你会怎么做）；然而，何必费这个劲呢？NFS 的设计理念覆盖了大多数重要的情况，总体上让系统在故障处理方面的设计干净、简单。因此，接受生活并不完美、照样把系统构建出来，正是良好工程的标志。据说这条智慧出自伏尔泰，他说过："……一位聪明的意大利人说，'最好'是'好'的敌人"[V72]，因此我们称之为**伏尔泰定律**（Voltaire's Law）。

收到了回复，一切又恢复正常，这样客户端就用统一的方式同时处理了消息丢失和服务器故障。漂亮！

顺便说一句：有些操作很难做成幂等的。例如，试图创建一个已经存在的目录时，mkdir 请求会报告失败。因此，在 NFS 中，如果文件服务器收到 MKDIR 协议消息并成功执行，但回复丢失了，客户端可能会重试，结果收到失败，而这个操作其实第一次就成功了，只是重试时才失败。所以说，生活并不完美。

## 49.8 提高性能：客户端缓存

分布式文件系统有很多好处，但通过网络发送所有的读写请求，可能导致很大的性能问题：网络一般没那么快，与本地内存或磁盘相比尤其如此。于是又有一个问题：如何提高分布式文件系统的性能？

你读了上面小标题里那几个大字，大概能猜到，答案是客户端**缓存**（caching）。NFS 客户端文件系统会把从服务器读到的文件数据以及元数据缓存在客户端内存中。这样，第一次访问的开销很大（需要网络通信），而后续访问直接从客户端内存取数据，速度很快。

缓存也可以充当写操作的临时缓冲区（buffer）。客户端应用程序第一次写文件时，客户端先把数据缓冲在客户端内存中，和从文件服务器读到的数据放在同一个缓存里，之后再把数据写到服务器。这种**写缓冲**（write buffering）很有用，因为它把应用程序 `write()` 的延迟和实际的写性能分开了：应用程序对 `write()` 的调用立即成功，只是把数据放进了客户端文件系统的缓存；数据要到之后才会写到文件服务器。

所以，NFS 客户端缓存了数据，性能通常很好，我们就大功告成了，对吧？可惜，还不完全是。只要系统里有多个客户端缓存，加入缓存就会带来一个重大而有趣的挑战，我们称之为**缓存一致性问题**（cache consistency problem）。

## 49.9 缓存一致性问题

用三个客户端和一台服务器最能说明缓存一致性问题（cache consistency problem）。设想客户端 C1 读取了文件 F，并在本地缓存中保留了一份副本。现在设想另一个客户端 C2 覆盖写了文件 F，改变了它的内容；我们把文件的新版本称为 F（版本 2），即 F[v2]，旧版本称为 F[v1]，以便区分这两者（当然，文件名还是同一个，只是内容不同）。最后，还有第三个客户端 C3，它还没有访问过文件 F。

 © 2008–25, ARPACI-DUSSEAU

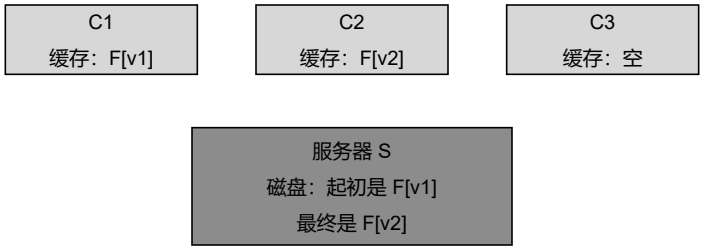



图 49.7：**缓存一致性问题**

你大概已经看出接下来的问题了（图 49.7）。实际上，这里有两个子问题。第一个子问题是，客户端 C2 可能会先把写操作缓冲在自己的缓存里一段时间，然后才传播给服务器；这时，F[v2] 还待在 C2 的内存里，其他客户端（比如 C3）对 F 的任何访问，取到的都是文件的旧版本（F[v1]）。因此，在客户端缓冲写操作，可能导致其他客户端拿到文件的过时版本，这未必是我们想要的；设想你登录机器 C2，更新了 F，然后登录 C3 去读这个文件，结果读到的却是旧副本！这肯定让人恼火。因此，我们把缓存一致性问题（cache consistency problem）的这一方面称为 **update visibility**：一个客户端的更新，什么时候才能被其他客户端看到?

缓存一致性（cache consistency）的第二个子问题是 **stale cache**：在这种情况下，C2 终于把写操作刷新到了文件服务器，于是服务器有了最新版本（F[v2]）。然而，C1 的缓存中仍然是 F[v1]；如果 C1 上运行的程序读取文件 F，它会得到过时的版本（F[v1]），而不是最新的副本（F[v2]），这（通常）不是我们想要的。

NFSv2 的实现用两种方式解决这些缓存一致性问题（cache consistency problem）。第一，为了解决 update visibility，客户端实现了有时称为**关闭时刷新**（flush-on-close，也叫 **close-to-open**）的一致性语义；具体来说，客户端应用程序写了一个文件并随后关闭它时，客户端会把所有更新，也就是缓存中的脏页（dirty page），刷新到服务器。有了关闭时刷新（flush-on-close）的一致性，NFS 就能确保之后另一个节点打开这个文件时，看到的是最新的版本。

第二，为了解决 stale cache 问题，NFSv2 客户端在使用缓存的内容之前，先检查文件是否发生了变化。具体来说，在使用某个缓存块（cached block）之前，客户端文件系统会向服务器发出 GETATTR 请求，获取文件的属性。重要的是，属性中包含文件在服务器上最后一次修改的时间；如果修改时间晚于文件被取到客户端缓存的时间，客户端就会让这个文件**失效**（invalidates），把它从客户端缓存中移除，从而确保后续的读取会去服务器取回文件的最新版本。反过来，如果客户端发现自己拥有的是文件的最新版本，它就会直接使用缓存的内容，从而提高性能。

Sun 最初的团队实现了这个 stale cache 问题的解决方案后，发现了一个新问题：NFS 服务器突然被大量 GETATTR 请求淹没了。有一条工程原则值得遵循：为**常见情况**（common case）而设计，让常见情况运行良好；在这里，虽然常见情况是文件只被单个客户端访问（也许是反复访问），但客户端总要向服务器发送 GETATTR 请求，以确认没有别人修改过这个

文件。于是，客户端不停地轰炸服务器，反复问“有人改过这个文件吗?”，而大多数时候根本没人改过。

为了（在一定程度上）补救这种情况，每个客户端都加上了**属性缓存**（attribute cache）。客户端在访问文件之前仍然会验证文件，但大多数时候只需查看属性缓存（attribute cache）来取得属性。某个文件的属性在第一次访问该文件时放入缓存，经过一段时间（比如 3 秒）后超时。因此在这 3 秒内，每次访问文件都会认定缓存的文件可用，于是直接使用，不必与服务器做任何网络通信。

## 49.10 评估 NFS 的缓存一致性

最后再谈几句 NFS 的缓存一致性（cache consistency）。加入关闭时刷新（flush-on-close）的行为是为了“说得通”，但它带来了一定的性能问题。具体来说，如果在客户端上创建了一个临时的或短命的文件，并且很快就删除了，它仍然会被强制写到服务器。更理想的实现，可能会把这类短命文件一直留在内存里，直到它们被删除，从而完全省掉与服务器的交互，也许还能提高性能。

更重要的是，在 NFS 中加入属性缓存（attribute cache）之后，你很难理解或推断自己拿到的究竟是文件的哪个版本。有时你会拿到最新版本；有时你会拿到旧版本，仅仅因为你的属性缓存（attribute cache）还没有超时，于是客户端乐于把内存里的东西给你。虽然这在大多数时候没问题，但它偶尔会（而且至今仍会!）导致奇怪的行为。

以上就是 NFS 客户端缓存（client caching）的怪异之处。它是一个有趣的例子：由实现细节来定义用户可观察到的语义，而不是反过来。

## 49.11 对服务器端写缓冲的影响

到目前为止，我们关注的都是客户端缓存（client caching），大多数有趣的问题也都出在这里。然而，NFS 服务器往往也是配置精良、内存充足的机器，所以它们同样要考虑缓存（caching）的问题。从磁盘读取数据和元数据时，NFS 服务器会把它们保留在内存中，之后再读这些数据（和元数据）就不用去磁盘了，这可能带来（小幅的）性能提升。

更耐人寻味的是写缓冲（write buffering）的情况。在写操作被强制写入稳定存储（stable storage，例如磁盘或其他某种持久设备）之前，NFS 服务器绝对不能对 WRITE 协议请求返回成功。虽然服务器可以在内存中放一份数据副本，但对 WRITE 协议请求向客户端返回成功，可能导致错误的行为；你能想明白为什么吗?

答案在于我们对客户端如何处理服务器故障所做的假设。设想客户端发出了下面这一系列写操作：

```
write(fd, a_buffer, size); // 用 a 填满第 1 块
write(fd, b_buffer, size); // 用 b 填满第 2 块
write(fd, c_buffer, size); // 用 c 填满第 3 块
```

这些写操作依次用一块 a、一块 b、一块 c 覆盖文件的三个块。因此，如果文件最初是这样的：

```
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz
```

我们可能会预期，这些写操作完成后，最终结果是这样的：x、y、z 分别被 a、b、c 覆盖。

```
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
ccccccccccccccccccccccccccccccccccccccccccccccccc
```

为了举例，假设客户端的这三次写操作以三条不同的 WRITE 协议消息发给了服务器。假设第一条 WRITE 消息被服务器收到并写入了磁盘，客户端也得知它成功了。现在假设第二次写入只是缓冲在内存里，服务器没等把它强制写入磁盘，就向客户端报告了成功；不幸的是，服务器随后在写盘之前崩溃了。服务器很快重新启动，并收到了第三个写请求，这次请求也成功了。

于是，在客户端看来，所有请求都成功了，可我们惊讶地发现文件内容是这样的：

```
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy <--- 糟了
ccccccccccccccccccccccccccccccccccccccccccccccccc
```

天哪！因为服务器在把第二次写入提交到磁盘之前就告诉客户端它成功了，文件里留下了一块旧数据；视具体应用而定，这可能是灾难性的。

为了避免这个问题，NFS 服务器必须先把每个写操作提交到稳定存储（stable storage，即持久存储），然后才能通知客户端成功；这样客户端就能察觉写入期间发生的服务器故障，并不断重试，直到最终成功。这样做可以确保我们永远不会得到像上面例子中那样相互混杂的文件内容。

这个要求给 NFS 服务器的实现带来的问题是：如果不精心处理，写性能可能会成为最主要的瓶颈。事实上，有些公司（例如 Network Appliance）成立时的目标很简单，就是造

补充：创新孕育创新

和许多开创性技术一样，NFS 的问世也需要其他一些基础性的创新来支撑它的成功。其中影响最持久的，大概是 Sun 引入的**虚拟文件系统**（Virtual File System，**VFS**）/ **Virtual Node**（**vnode**）接口，它让不同的文件系统可以方便地接入操作系统 [K86]。

VFS 层包含对整个文件系统执行的操作，比如挂载（mount）和卸载（unmount）、获取整个文件系统范围的统计信息，以及把所有脏的（尚未写出的）写操作强制写入磁盘。

vnode 层则由可以对文件执行的所有操作组成，比如 open、close、读、写等等。

要构建一个新的文件系统，只需定义这些“方法”；框架会处理其余的一切：把系统调用连接到具体的文件系统实现，集中执行所有文件系统共有的通用功能（例如缓存），从而让多个文件系统实现可以在同一个系统中同时运行。

虽然有些细节已经改变，但许多现代系统都有某种形式的 VFS/vnode 层，包括 Linux、各种 BSD、macOS，甚至 Windows（形式是可安装文件系统，即 Installable File System）。即使 NFS 对世界来说没那么重要了，支撑它的一些必要基础也会延续下去。

出写得快的 NFS 服务器；它们用的一个技巧是，先把写操作放进有电池备份的内存里，这样就能快速回复 WRITE 请求，既不必担心丢失数据，也不必付出立即写盘的代价；第二个技巧是采用专门设计的文件系统，让最终需要写盘时能快速写入 [HLM94, RO91]。

## 49.12 小结

我们介绍了 NFS 分布式文件系统。NFS 的核心思想是在服务器发生故障时实现简单、快速的恢复，并通过精心的协议设计达到了这个目的。操作的幂等性至关重要：客户端可以放心地重放（replay）失败的操作，不管服务器有没有执行过这个请求，重放都没有问题。

我们还看到了，在多客户端、单服务器的系统中引入缓存（caching）会让事情变得复杂。具体来说，系统必须解决缓存一致性问题（cache consistency problem），才能表现得合理；然而，NFS 解决这个问题的办法有点临时拼凑的味道，偶尔会导致可以观察到的怪异行为。最后，我们看到了服务器端缓存（server caching）可能很棘手：写到服务器的数据必须先强制写入稳定存储（stable storage），才能返回成功（否则数据可能丢失）。

还有一些显然很重要的问题我们没有讨论，尤其是安全。早期 NFS 实现的安全性出奇地宽松；客户端上的任何用户都可以相当轻易地冒充其他用户，从而访问几乎任何文件。后来集成了更严格的身份认证（authentication）服务（例如 Kerberos [NT94]），才弥补了这些明显的缺陷。

补充：NFS 的关键术语

- 在 NFS 中，实现快速、简单的崩溃恢复这一主要目标，关键在于设计一个**无状态**（stateless）的协议。崩溃之后，服务器可以很快重新启动并开始处理请求；客户端只需**重试**（retry）请求，直到成功为止。
- 让请求具有**幂等性**（idempotency）是 NFS 协议的核心之一。如果一个操作执行多次的效果和执行一次相同，这个操作就是幂等的。在 NFS 中，幂等性让客户端可以放心地重试，并把客户端重传丢失的消息和客户端处理服务器崩溃这两件事统一了起来。
- 出于性能考虑，需要客户端**缓存**（caching）和**写缓冲**（write buffering），但这也带来了**缓存一致性问题**（cache consistency problem）。
- NFS 的实现通过多种手段为缓存一致性（cache consistency）提供了一种工程上的解决方案：**关闭时刷新**（flush-on-close，也叫 **close-to-open**）的做法确保文件关闭时内容会被强制写到服务器，其他客户端因此能看到对它的更新。属性缓存（attribute cache）则降低了向服务器检查文件是否改变（通过 GETATTR 请求）的频率。
- NFS 服务器必须先把写操作提交到持久介质，然后才能返回成功；否则可能丢失数据。
- 为了支持把 NFS 集成到操作系统中，Sun 引入了 **VFS/vnode** 接口，使多种文件系统实现能够在同一个操作系统中共存。

 © 2008–25, ARPACI-DUSSEAU

## 参考文献

[AKW88] "The AWK Programming Language" by Alfred V. Aho, Brian W. Kernighan, Peter J. Weinberger. Pearson, 1988 (1st edition). *A concise, wonderful book about awk. We once had the pleasure of meeting Peter Weinberger; when he introduced himself, he said "I'm Peter Weinberger, you know, the 'W' in awk?" As huge awk fans, this was a moment to savor. One of us (Remzi) then said, "I love awk! I particularly love the book, which makes everything so wonderfully clear." Weinberger replied (crestfallen), "Oh, Kernighan wrote the book."*

[C00] "NFS Illustrated" by Brent Callaghan. Addison-Wesley Professional Computing Series, 2000. *A great NFS reference; incredibly thorough and detailed per the protocol itself.*

[ES03] "New NFS Tracing Tools and Techniques for System Analysis" by Daniel Ellard and Margo Seltzer. LISA '03, San Diego, California. *An intricate, careful analysis of NFS done via passive tracing. By simply monitoring network traffic, the authors show how to derive a vast amount of file system understanding.*

[HLM94] "File System Design for an NFS File Server Appliance" by Dave Hitz, James Lau, Michael Malcolm. USENIX Winter 1994. San Francisco, California, 1994. *Hitz et al. were greatly influenced by previous work on log-structured file systems.*

[K86] "Vnodes: An Architecture for Multiple File System Types in Sun UNIX" by Steve R. Kleiman. USENIX Summer '86, Atlanta, Georgia. *This paper shows how to build a flexible file system architecture into an operating system, enabling multiple different file system implementations to coexist. Now used in virtually every modern operating system in some form.*

[NT94] "Kerberos: An Authentication Service for Computer Networks" by B. Clifford Neuman, Theodore Ts'o. IEEE Communications, 32(9):33-38, September 1994. *Kerberos is an early and hugely influential authentication service. We probably should write a book chapter about it sometime…*

[O91] "The Role of Distributed State" by John K. Ousterhout. 1991. Available at this site: ftp://ftp.cs.berkeley.edu/ucb/sprite/papers/state.ps. *A rarely referenced discussion of distributed state; a broader perspective on the problems and challenges.*

[P+94] "NFS Version 3: Design and Implementation" by Brian Pawlowski, Chet Juszczak, Peter Staubach, Carl Smith, Diane Lebel, Dave Hitz. USENIX Summer 1994, pages 137-152. *The small modifications that underlie NFS version 3.*

[P+00] "The NFS version 4 protocol" by Brian Pawlowski, David Noveck, David Robinson, Robert Thurlow. 2nd International System Administration and Networking Conference (SANE 2000). *Undoubtedly the most literary paper on NFS ever written.*

[RO91] "The Design and Implementation of the Log-structured File System" by Mendel Rosenblum, John Ousterhout. Symposium on Operating Systems Principles (SOSP), 1991. *LFS again. No, you can never get enough LFS.*

[S86] "The Sun Network File System: Design, Implementation and Experience" by Russel Sandberg. USENIX Summer 1986. *The original NFS paper; though a bit of a challenging read, it is worthwhile to see the source of these wonderful ideas.*

[Sun89] "NFS: Network File System Protocol Specification" by Sun Microsystems, Inc. Request for Comments: 1094, March 1989. Available: http://www.ietf.org/rfc/rfc1094.txt. *The dreaded specification; read it if you must, i.e., you are getting paid to read it. Hopefully, paid a lot. Cash money!*

[V72] "La Begueule" by Francois-Marie Arouet a.k.a. Voltaire. Published in 1772. *Voltaire said a number of clever things, this being but one example. For example, Voltaire also said "If you have two religions in your land, the two will cut each other's throats; but if you have thirty religions, they will dwell in peace." What do you say to that, Democrats and Republicans?*

## 作业（测量）

在这个作业中，你将使用真实的跟踪记录（trace）做一点 NFS 跟踪分析。这些跟踪记录来自 Ellard 和 Seltzer 的工作 [ES03]。开始之前，务必阅读相关的 README，并（照例）从 OSTEP 作业页面下载相应的 tar 包。做这项分析时，awk 编程语言 [AKW88] 可能是个好玩的工具。

## 问题

1. 跟踪分析的第一个问题：利用第一列中的时间戳，确定这些跟踪记录是在哪个时间段采集的。这个时间段有多长？是哪一天/哪一周/哪一月/哪一年？（这和文件名给出的提示一致吗？）提示：用 `head -1` 和 `tail -1` 这两个工具取出文件的第一行和最后一行，然后计算。
2. 现在来统计一下操作的数量。跟踪记录中每种类型的操作各出现了多少次？按频率排序；哪种操作最频繁？NFS 名副其实吗？
3. 现在更详细地看一些特定的操作。例如，GETATTR 请求会返回很多关于文件的信息，包括请求是为哪个用户 ID 执行的、文件大小等等。统计跟踪记录中被访问文件的大小分布；平均文件大小是多少？另外，跟踪记录中有多少个不同的用户访问了文件？是少数几个用户占了大部分流量，还是比较分散？GETATTR 的回复中还有哪些有趣的信息？
4. 你也可以查看针对某个文件的请求，确定文件是如何被访问的。例如，某个文件是被顺序读写的，还是被随机读写的？查看 READ 和 WRITE 请求/回复的细节来算出答案。
5. 流量来自许多机器，发往一台服务器（在这份跟踪记录中）。计算一个流量矩阵，给出跟踪记录中有多少个不同的客户端，以及发往每个客户端的请求/回复各有多少。是少数几台机器占了主导，还是比较均衡？
6. 利用时间信息和每个请求/回复的唯一 ID，你应该能算出某个请求的延迟。计算所有请求/回复对的延迟，并画出它们的分布。平均值是多少？最大值？最小值？
7. 有时请求会被重试，因为请求或它的回复可能丢失或被丢弃。你能在跟踪样本中找到这种重试的证据吗？
8. 通过更多的分析，你还可以回答很多其他问题。你认为哪些问题很重要？告诉我们，也许我们会把它们加到这里！

 © 2008–25, ARPACI-DUSSEAU

# 50

# Andrew 文件系统（AFS）

Andrew 文件系统（Andrew File System）是卡内基-梅隆大学（Carnegie-Mellon University，CMU）在 20 世纪 80 年代[1]推出的 [H+88]。这个项目由卡内基-梅隆大学著名的 M. Satyanarayanan 教授（简称“Satya”）领导，主要目标很简单：**可扩展性**（scale）。具体来说，怎样设计一个分布式文件系统（distributed file system），才能让一台服务器支持尽可能多的客户端？

有意思的是，设计和实现的很多方面都会影响可扩展性。其中最重要的是客户端和服务器之间**协议**（protocol）的设计。例如在 NFS 中，协议迫使客户端定期向服务器核对缓存（cache）的内容是否发生了变化；由于每次核对都要消耗 CPU、网络带宽（bandwidth）等服务器资源，这样频繁的核对会限制一台服务器能够响应的客户端数量，从而限制可扩展性。

AFS 和 NFS 的另一个不同之处在于，AFS 从一开始就把用户可以看到的合理行为当作头等大事。在 NFS 中，缓存一致性（cache consistency）很难描述清楚，因为它直接取决于底层的实现细节，包括客户端缓存的超时（timeout）时间。而在 AFS 中，缓存一致性（cache consistency）简单易懂：打开文件时，客户端一般会从服务器拿到最新的一致副本。

## 50.1 AFS 第 1 版

我们将讨论 AFS 的两个版本 [H+88, S+85]。第一个版本（我们称之为 AFSv1，不过最初的系统其实叫作 ITC 分布式文件系统 [S+85]）已经具备了一些基本设计，但可扩展性没有达到预期，设计者于是重新设计，得到了最终的协议，我们称之为 AFSv2，或者就叫 AFS [H+88]。下面先讨论第一个版本。

AFS 所有版本的一个基本原则，是在访问文件的客户端机器的**本地磁盘**（local disk）上做**整文件缓存**（whole-file caching）。你 `open()` 一个文件时，整个文件（如果存在的话）都会从服务器取回，存成你本地磁盘上的一个文件。应用程序之后的 `read()` 和 `write()` 操作都被重定向（redirect）到存放该文件的本地文件系统（file system）；因此，这些操作不需要网络通信，速度很快。最后，在 `close()` 时，文件（如果被修改过）会被刷回服务器。注意它和 NFS 的明显区别：NFS 缓存的是块（block）而不是整个文件（当然 NFS 也可以把整个文件的每一块都缓存下来），而且缓存在客户端的内存里，而不是本地磁盘上。

[1]虽然最初叫“Carnegie-Mellon University”，但 CMU 后来去掉了连字符，于是有了现在的写法“Carnegie Mellon University”。由于 AFS 源自 80 年代初的工作，我们沿用 CMU 最初带连字符的写法。如果你对极其无聊的细枝末节感兴趣，可以到 https://www.quora.com/When-did-Carnegie-Mellon-University-remove-the-hyphen-in-the-university-name 看看。

| | |
|---|---|
| `TestAuth` | 检查文件是否发生了变化（用于验证缓存的条目） |
| `GetFileStat` | 获取文件的 stat 信息 |
| `Fetch` | 取回文件的内容 |
| `Store` | 把这个文件存到服务器上 |
| `SetFileStat` | 设置文件的 stat 信息 |
| `ListDir` | 列出目录的内容 |

图 50.1：**AFSv1 协议要点**

我们来看得更细一点。客户端应用程序第一次调用 `open()` 时，AFS 的客户端代码（AFS 的设计者称之为 **Venus**）会向服务器发送一条 Fetch 协议消息（protocol message）。Fetch 协议消息把所需文件的完整路径名（pathname，例如 `/home/remzi/notes.txt`）传给文件服务器（file server，设计者把这组服务器叫作 **Vice**），服务器随后遍历这个路径名，找到所需的文件，再把整个文件发回客户端。接着，客户端代码把文件缓存在客户端的本地磁盘上（也就是把它写到本地磁盘）。前面说过，在 AFS 中，之后的 `read()` 和 `write()` 系统调用（system call）完全在本地进行（不和服务器通信），只是被重定向到文件的本地副本。由于 `read()` 和 `write()` 调用的行为和对本地文件系统的调用一样，某个块一旦被访问，也可能被缓存在客户端内存里。因此，AFS 也会用客户端内存来缓存它本地磁盘上那些块的副本。最后，用完文件后，AFS 客户端检查文件是否被修改过（即是否以写方式打开过）；如果是，就用一条 Store 协议消息把新版本刷回服务器，把整个文件连同路径名发给服务器永久保存。

下次访问这个文件时，AFSv1 的效率就高得多了。具体来说，客户端代码先用 TestAuth 协议消息联系服务器，确认文件是否发生了变化。如果没有变化，客户端就使用本地缓存的副本，避免了一次网络传输，从而提高了性能。图 50.1 列出了 AFSv1 的一些协议消息。注意，这个早期版本的协议只缓存文件内容；目录（directory）之类的东西则只保存在服务器上。

## 50.2 第 1 版的问题

AFS 第一版的几个关键问题，促使设计者重新思考他们的文件系统。为了详细研究这些问题，AFS 的设计者花了大量时间测量现有的原型，找出问题所在。这样的实验是好事：

提示：先测量，再构建（Patterson 定律）
我们的导师之一 David Patterson（以 RISC 和 RAID 闻名）总是鼓励我们：在构建新系统来解决某个问题之前，先测量现有系统，证明问题确实存在。用实验证据而不是直觉，就能把构建系统的过程变成一件更科学的事。这样做还有一个附带的好处：在改进版本开发出来之前，你就得想清楚究竟该怎样测量这个系统。等你最终着手构建新系统时，有两件事会因此变得更好：第一，你有证据表明自己在解决一个真实存在的问题；第二，你已经有了一套测量新系统的方法，可以用来证明它确实比现有技术更好。这就是我们说的 **Patterson 定律**（Patterson's Law）。

 © 2008–23, ARPACI-DUSSEAU

**测量**（measurement）是理解系统如何工作、如何改进的关键，所以获得具体、可靠的数据，是构建系统的必要环节。在研究中，作者们发现 AFSv1 有两个主要问题：

- **路径遍历的开销太高**：执行 Fetch 或 Store 协议请求（protocol request）时，客户端把完整的路径名传给服务器，例如 `/home/remzi/notes.txt`。为了访问这个文件，服务器必须完整地遍历路径名：先在根目录（root directory）中查找 `home`，再在 `home` 中查找 `remzi`，依此类推，沿路径一路往下，直到最终找到所需的文件。AFS 的设计者发现，大量客户端同时访问服务器时，服务器把大量 CPU 时间都花在了沿目录路径往下走上。
- **客户端发出的 TestAuth 协议消息太多**：和 NFS 发出过多的 GETATTR 协议消息很像，AFSv1 也产生了大量流量，用 TestAuth 协议消息检查本地文件（或者它的 stat 信息）是否有效。于是，服务器把大量时间都花在告诉客户端能不能使用它们缓存的文件副本上。而大多数时候，答案都是文件没有变化。

AFSv1 其实还有另外两个问题：服务器之间的负载不均衡（load imbalance）；服务器为每个客户端单独使用一个进程（process），从而带来了上下文切换（context switch）和其他开销。负载不均衡（load imbalance）的问题靠引入**卷**（volume）来解决：管理员可以在服务器之间移动卷来平衡负载；上下文切换的问题，AFSv2 改用线程（thread）而不是进程来构建服务器，也解决了。不过，限于篇幅，这里我们只关注上面那两个限制系统规模的主要协议问题。

## 50.3 改进协议

上面两个问题限制了 AFS 的可扩展性：服务器的 CPU 成了系统的瓶颈，每台服务器只能服务 20 个客户端，再多就会过载。服务器收到了太多的 TestAuth 消息，而在收到 Fetch 或 Store 消息时，又在遍历目录层次结构（directory hierarchy）上花了太多时间。于是，AFS 的设计者面临这样一个问题：

> 关键问题：如何设计可扩展的文件协议
> 应该怎样重新设计协议，才能把与服务器的交互次数降到最少，也就是说，怎样减少 TestAuth 消息的数量？此外，怎样设计协议，才能让这些与服务器的交互变得高效？把这两个问题都解决了，新协议就能让 AFS 的可扩展性大大提高。

## 50.4 AFS 第 2 版

AFSv2 引入了**回调**（callback）的概念，以减少客户端与服务器之间的交互次数。回调就是服务器对客户端的一个承诺：客户端缓存的某个文件被修改时，服务器会通知客户端。系统里加入了这份**状态**（state）之后，客户端就不必再联系服务器来确认缓存的文件是否仍然有效。相反，它会认为文件一直有效，直到服务器告诉它并非如此为止；这和**轮询**（polling）与**中断**（interrupts）的区别正好可以类比。

AFSv2 还引入了**文件标识符**（file identifier，**FID**）的概念，它类似于 NFS 的**文件句柄**（file handle），用来代替路径名指定客户端感兴趣的文件。AFS 中的 FID 由卷标识符（volume identifier）、文件标识符（file identifier）和一个“uniquifier”组成（uniquifier 让文件被删除后，卷 ID 和文件 ID 可以被重用）。这样，客户端就不再把完整的路径名发给服务器、让服务器沿路径名查找所需的文件，而是由客户端自己一段一段地遍历路径名，并把结果缓存下来，从而有望减轻服务器的负载。

例如，假设客户端访问文件 `/home/remzi/notes.txt`，而 `home` 是挂载（mount）到 / 上的 AFS 目录，即 / 是本地根目录，`home` 及其下面的内容都在 AFS 中。这时，客户端会先 Fetch `home` 的目录内容，放进本地磁盘缓存（local-disk cache），并在 `home` 上建立一个回调。然后，客户端会 Fetch 目录 `remzi`，放进本地磁盘缓存（local-disk cache），并在 `remzi` 上建立一个回调。最后，客户端 Fetch `notes.txt`，把这个普通文件缓存到本地磁盘上，建立一个回调，然后把一个文件描述符（file descriptor）返回给调用它的应用程序。图 50.2 对此做了总结。

不过，和 NFS 的关键区别在于，每取回一个目录或文件，AFS 客户端都会在服务器上建立一个回调，从而确保服务器会在客户端缓存的状态发生变化时通知客户端。好处是显而易见的：虽然第一次访问 `/home/remzi/notes.txt` 会产生很多客户端与服务器之间的消息（如上所述），但它同时为所有目录以及文件 `notes.txt` 建立了回调，因此之后的访问完全在本地进行，根本不需要与服务器交互。所以，在文件已缓存在客户端这种常见情况（common case）下，AFS 的行为和基于本地磁盘的文件系统几乎一模一样。如果多次访问同一个文件，第二次访问应该和访问本地文件一样快。

## 50.5 缓存一致性

讨论 NFS 时，我们考虑了缓存一致性（cache consistency）的两个方面：**update visibility** 和 **cache staleness**。对于 update visibility，问题是：服务器什么时候会更新为文件的新版本？对于 cache staleness，问题是：服务器有了新版本之后，要过多久客户端才能看到新版本，而不是之前缓存的旧副本？

由于有回调和整文件缓存（whole-file caching），AFS 提供的缓存一致性（cache consistency）很容易描述和理解。需要考虑两种重要情况：不同机器上的进程之间的一致性，以及同一台机器上的进程之间的一致性。

在不同的机器之间，AFS 让更新在服务器上可见、并让缓存的副本失效，这两件事在同一时刻发生，也就是更新后的文件被关闭的时候。客户端打开一个文件，然后写入（也许写了很多次）。最终关闭时，新文件被刷到服务器上（于是变得可见）。这时，服务器会“打破”所有缓存了副本的客户端的回调；具体做法是联系每个客户端，通知它在这个文件上的回调已经失效。这一步确保客户端不会再读到文件的过时副本；这些客户端之后再打开这个文件时，需要从服务器重新取回新版本，同时也会在新版本上重新建立回调。

对于同一台机器上的进程，AFS 在这个简单模型之外做了一个例外。在这种情况下，对文件的写入会立即对本地的其他进程可见（也就是说，进程不必等到文件关闭才能看到最新

 © 2008–23, ARPACI-DUSSEAU

| **客户端**（$C_1$） | **服务器** |
|---|---|
| **fd = open(“/home/remzi/notes.txt”, …);** | |
| 发送 Fetch (home FID, “remzi”) | |
| | 接收 Fetch 请求<br>在 home 目录中查找 remzi<br>在 remzi 上建立 callback($C_1$)<br>返回 remzi 的内容和 FID |
| 接收 Fetch 回复<br>把 remzi 写入本地磁盘缓存<br>记录 remzi 的回调状态<br>发送 Fetch (remzi FID, “notes.txt”) | |
| | 接收 Fetch 请求<br>在 remzi 目录中查找 notes.txt<br>在 notes.txt 上建立 callback($C_1$)<br>返回 notes.txt 的内容和 FID |
| 接收 Fetch 回复<br>把 notes.txt 写入本地磁盘缓存<br>记录 notes.txt 的回调状态<br>对缓存的 notes.txt 执行本地 `open()`<br>把文件描述符返回给应用程序 | |
| **read(fd, buffer, MAX);**<br>对缓存的副本执行本地 `read()` | |
| **close(fd);**<br>对缓存的副本执行本地 `close()`<br>如果文件被修改过，就刷回服务器 | |
| **fd = open(“/home/remzi/notes.txt”, …);**<br>Foreach dir (home, remzi)<br>if (callback(dir) == VALID)<br>用本地副本执行 lookup(dir)<br>else<br>Fetch（同上）<br>if (callback(notes.txt) == VALID)<br>打开本地缓存的副本<br>返回指向它的文件描述符<br>else<br>Fetch（同上），然后打开并返回 fd | |

图 50.2：**读取文件：客户端和文件服务器的操作**

补充：缓存一致性不是万能药

讨论分布式文件系统时，人们很看重文件系统提供的缓存一致性（cache consistency）。然而，这种基本的一致性并不能解决多个客户端访问文件时的所有问题。例如，如果你在构建一个代码仓库，有多个客户端在签入和签出代码，就不能单靠底层文件系统替你完成所有工作；你得显式地使用**文件级锁**（file-level locking），保证发生这类并发访问（concurrent access）时结果是“正确”的。事实上，任何真正在意并发更新（concurrent update）的应用程序，都会额外加上处理冲突的机制。本章和上一章介绍的基本一致性，主要用于日常使用，也就是说，用户登录到另一台客户端时，希望在那里看到自己文件的某个合理版本。对这些协议寄予更多期望，只会让自己遭遇失败、失望，以及满含泪水的挫败感。

 © 2008–23, ARPACI-DUSSEAU

| 客户端$_1$ | | | 客户端$_2$ | | 服务器 | 说明 |
|---|---|---|---|---|---|---|
| $P_1$ | $P_2$ | 缓存 | $P_3$ | 缓存 | 磁盘 | |
| open(F) | | - | | - | - | 文件被创建 |
| write(A) | | A | | - | - | |
| close() | | A | | - | A | |
| | open(F) | A | | - | A | |
| | read() → A | A | | - | A | |
| | close() | A | | - | A | |
| open(F) | | A | | - | A | |
| write(B) | | B | | - | A | |
| | open(F) | B | | - | A | 本地进程 |
| | read() → B | B | | - | A | 立即看到写入 |
| | close() | B | | - | A | |
| | | B | open(F) | A | A | 远程进程 |
| | | B | read() → A | A | A | 看不到写入…… |
| | | B | close() | A | A | |
| close() | | B | | ~~A~~ | B | ……直到 close() |
| | | B | open(F) | B | B | 发生之后 |
| | | B | read() → B | B | B | |
| | | B | close() | B | B | |
| | | B | open(F) | B | B | |
| open(F) | | B | | B | B | |
| write(D) | | D | | B | B | |
| | | D | write(C) | C | B | |
| | | D | close() | C | C | |
| close() | | D | | ~~C~~ | D | |
| | | D | open(F) | D | D | $P_3$ 很不走运： |
| | | D | read() → D | D | D | 最后写入者胜出 |
| | | D | close() | D | D | |

图 50.3：**缓存一致性时间线**

的更新）。这让单台机器的使用行为和你预期的完全一样，因为这种行为基于典型的 UNIX 语义。只有换到另一台机器上时，你才能察觉到 AFS 更一般的一致性机制。

还有一种跨机器的情况很有意思，值得进一步讨论。具体来说，在不同机器上的进程同时修改一个文件这种少见的情况下，AFS 自然而然地采用了所谓**最后写入者胜出**（last writer wins）的做法（也许应该叫**最后关闭者胜出**，last closer wins）。也就是说，哪个客户端最后调用 `close()`，它就最后更新服务器上的整个文件，于是它的文件就成为“胜出”的文件，也就是留在服务器上供其他人查看的那个文件。结果得到的文件要么完全由这个客户端生成，要么完全由那个客户端生成。注意它和 NFS 这种基于块的协议（block-based protocol）的区别：在 NFS 中，每个客户端更新文件时，各个块的写入都可能被刷到服务器上，因此服务器上最终的文件可能是两个客户端更新的混合。在很多情况下，这样混合的文件输出没有多大意义；想象一下两个客户端分块修改一张 JPEG 图片，写入混在一起的结果多半不是一个合法的 JPEG。

图 50.3（第 569 页）的时间线展示了几种不同的场景。各列分别是：客户端$_1$ 上的两个进程（$P_1$ 和 $P_2$）及其缓存状态，客户端$_2$ 上的一个进程（$P_3$）及其缓存状态，以及服务器，

 © 2008–23, ARPACI-DUSSEAU

它们都在操作同一个文件，这个文件的名字毫无想象力地叫作 F。对于服务器，图中只给出左边的操作完成之后文件的内容。从头读一遍，看看你能不能理解每次读取为什么返回那样的结果。如果卡住了，右边的说明一栏可以帮到你。

## 50.6 崩溃恢复

从上面的描述中，你也许已经感觉到，崩溃恢复（crash recovery）要比 NFS 复杂。你说得没错。例如，设想有一小段时间，服务器（S）无法联系某个客户端（C1），比如客户端 C1 正在重启。在 C1 不可用期间，S 可能试图向它发送一条或多条回调召回消息；例如，设想 C1 的本地磁盘上缓存了文件 F，然后 C2（另一个客户端）更新了 F，于是 S 向所有缓存了这个文件的客户端发送消息，让它们把文件从本地缓存中移除。由于 C1 在重启时可能错过这些关键消息，它在重新加入系统时，应该把自己缓存的所有内容都视为可疑。因此，下次访问文件 F 时，C1 应该先用 TestAuth 协议消息询问服务器，它缓存的文件 F 副本是否仍然有效；如果有效，C1 就可以使用；如果无效，C1 就应该从服务器取回较新的版本。

服务器崩溃后的恢复也更复杂。问题在于回调是保存在内存里的；因此，服务器重启后，并不知道哪台客户端机器拥有哪些文件。所以，服务器重启时，它的每个客户端都必须意识到服务器崩溃过，并把自己缓存的所有内容都视为可疑，（像上面那样）先重新确认文件的有效性再使用。可见，服务器崩溃是一件大事，必须确保每个客户端及时得知这次崩溃，否则就有客户端访问到过时文件的风险。实现这种恢复的方法有很多；例如，服务器重新运行起来时，向每个客户端发送一条消息（说“别相信你缓存的内容!”），或者让客户端用所谓的**心跳**（heartbeat）消息定期检查服务器是否还活着。可以看到，构建一个可扩展性更好、更合理的缓存模型是有代价的；而在 NFS 中，客户端几乎察觉不到服务器崩溃。

| **工作负载** | **NFS** | **AFS** | **AFS / NFS** |
|---|---|---|---|
| 1. 小文件，顺序读 | $N_s \cdot L_{\text{net}}$ | $N_s \cdot L_{\text{net}}$ | 1 |
| 2. 小文件，顺序重读 | $N_s \cdot L_{\text{mem}}$ | $N_s \cdot L_{\text{mem}}$ | 1 |
| 3. 中等文件，顺序读 | $N_m \cdot L_{\text{net}}$ | $N_m \cdot L_{\text{net}}$ | 1 |
| 4. 中等文件，顺序重读 | $N_m \cdot L_{\text{mem}}$ | $N_m \cdot L_{\text{mem}}$ | 1 |
| 5. 大文件，顺序读 | $N_L \cdot L_{\text{net}}$ | $N_L \cdot L_{\text{net}}$ | 1 |
| 6. 大文件，顺序重读 | $N_L \cdot L_{\text{net}}$ | $N_L \cdot L_{\text{disk}}$ | $\frac{L_{\text{disk}}}{L_{\text{net}}}$ |
| 7. 大文件，单次读 | $L_{\text{net}}$ | $N_L \cdot L_{\text{net}}$ | $N_L$ |
| 8. 小文件，顺序写 | $N_s \cdot L_{\text{net}}$ | $N_s \cdot L_{\text{net}}$ | 1 |
| 9. 大文件，顺序写 | $N_L \cdot L_{\text{net}}$ | $N_L \cdot L_{\text{net}}$ | 1 |
| 10. 大文件，顺序覆盖写 | $N_L \cdot L_{\text{net}}$ | $2 \cdot N_L \cdot L_{\text{net}}$ | 2 |
| 11. 大文件，单次写 | $L_{\text{net}}$ | $2 \cdot N_L \cdot L_{\text{net}}$ | $2 \cdot N_L$ |

图 50.4：**比较：AFS 与 NFS**

## 50.7 AFSv2 的规模与性能

新协议就位之后，设计者对 AFSv2 做了测量，发现它的可扩展性比原来的版本好得多。事实上，每台服务器可以支持大约 50 个客户端（而不只是 20 个）。另一个好处是，客户端的性能往往非常接近本地性能，因为在常见情况下，所有文件访问都在本地进行；文件读取通常走本地磁盘缓存（local disk cache），还可能走本地内存。只有客户端创建新文件或写入已有文件时，才需要向服务器发送 Store 消息，用新内容更新文件。

我们再把常见的文件系统访问场景和 NFS 做个比较，从而对 AFS 的性能有一些认识。图 50.4（第 570 页）给出了我们定性比较的结果。

在图中，我们针对不同大小的文件，用分析的方法考察典型的读写模式。小文件有 $N_s$ 个块，中等文件有 $N_m$ 个块，大文件有 $N_L$ 个块。我们假设小文件和中等文件能放进客户端的内存；大文件能放进本地磁盘，但放不进客户端内存。

为了便于分析，我们还假设通过网络从远程服务器访问一个文件块要花 $L_{\text{net}}$ 个时间单位，访问本地内存要花 $L_{\text{mem}}$，访问本地磁盘要花 $L_{\text{disk}}$。一般的假设是 $L_{\text{net}} > L_{\text{disk}} > L_{\text{mem}}$。

最后，我们假设第一次访问文件时不会命中任何缓存。之后的文件访问（即“重读”），只要相应的缓存容量足以容纳这个文件，我们就假设会命中缓存。

图中各列给出了某个操作（例如小文件顺序读）在 NFS 或 AFS 上大致需要的时间。最右边一列给出 AFS 与 NFS 的比值。

我们有以下几点观察。第一，在很多情况下，两个系统的性能大致相当。例如，在第一次读取文件的工作负载（workload）1、3、5 中，从远程服务器取回文件的时间占了主要部分，在两个系统上差不多。你可能以为 AFS 在这种情况下会慢一些，因为它要把文件写到本地磁盘；然而，这些写入会被本地（客户端）文件系统的缓存缓冲起来，所以这部分开销很可能被掩盖了。你可能还会以为 AFS 从本地缓存的副本读取会慢一些，原因同样是 AFS 把缓存的副本存在磁盘上。然而，AFS 在这里同样得益于本地文件系统的缓存；AFS 上的读取很可能命中客户端的内存缓存，性能和 NFS 差不多。

第二，工作负载 6 是大文件顺序重读，这时出现了一个有意思的差别。由于 AFS 有很大的本地磁盘缓存，再次访问这个文件时，它会从本地磁盘缓存中读取。而 NFS 只能把块缓存在客户端内存里；结果，如果重读一个大文件（即比本地内存还大的文件），NFS 客户端就得从远程服务器重新取回整个文件。因此，假设远程访问确实比本地磁盘慢，那么在这种情况下 AFS 比 NFS 快 $\frac{L_{\text{net}}}{L_{\text{disk}}}$ 倍。我们还注意到，NFS 在这种情况下会增加服务器的负载，这也会影响可扩展性。

第三，我们注意到，在工作负载 8、9 中，（新文件的）顺序写在两个系统上的性能应该差不多。在这种情况下，AFS 会把文件写入本地缓存的副本；文件关闭时，AFS 客户端会按照协议把写入强制写到服务器。NFS 会把写入缓冲在客户端内存中，可能因为客户端的内存压力（memory pressure）而把一些块强制写到服务器，但为了保持 NFS 关闭时刷新（flush-on-close）的一致性，在文件关闭时肯定会把它们写到服务器。你可能以为 AFS 在这里会慢一些，因为它把所有数据都写到本地磁盘。然而要知道，它是在写本地文件系统；这些写入先提交到页缓存（page cache），之后才（在后台）写到磁盘，因此 AFS 借助客户端操作系统的内存缓存提高了性能。

 © 2008–23, ARPACI-DUSSEAU

第四，我们注意到，AFS 在顺序覆盖写一个文件时表现更差，即工作负载 10。到目前为止，我们一直假设执行写入的工作负载也在创建新文件；而在这种情况下，文件已经存在，然后被覆盖写。对 AFS 来说，覆盖写可能是特别糟糕的情况，因为客户端先要把旧文件整个取回来，然后再把它覆盖掉。而 NFS 只会直接覆盖这些块，从而避免了最初那次（无用的）读取[2]。

最后，访问大文件中一小部分数据的工作负载，在 NFS 上的表现比在 AFS 上好得多（工作负载 7、11）。在这些情况下，AFS 协议在打开文件时会取回整个文件；可惜，之后只执行了一次很小的读或写。更糟的是，如果文件被修改了，整个文件都要写回服务器，性能影响翻倍。NFS 作为一种基于块的协议（block-based protocol），执行的 I/O 量和读写的大小成正比。

总的来说，我们看到 NFS 和 AFS 做了不同的假设，因此得到不同的性能结果也就不足为奇了。这些差别是否重要，和往常一样，取决于工作负载。

## 50.8 AFS 的其他改进

伯克利 FFS 问世时加入了符号链接（symbolic link）和许多其他功能；AFS 的设计者也一样，借构建系统的机会，加入了许多让系统更易于使用和管理的功能。例如，AFS 为客户端提供了真正的全局命名空间（global namespace），确保所有文件在所有客户端机器上的名字都一样。而 NFS 允许每个客户端按自己喜欢的任何方式挂载 NFS 服务器，因此只有靠约定（以及大量的管理工作），文件在各个客户端上的名字才会相似。

AFS 还认真对待安全问题，加入了认证（authenticate）用户的机制，并确保用户愿意的话，可以让一组文件保持私有。而 NFS 在很多年里对安全的支持都相当原始。

补充：工作负载的重要性

评估任何系统都面临一个挑战：选择什么样的**工作负载**（workload）。由于计算机系统的用法千差万别，可供选择的工作负载种类繁多。存储系统的设计者应该怎样判断哪些工作负载重要，从而做出合理的设计决策呢?

AFS 的设计者有测量文件系统使用情况的经验，因此对工作负载做了一些假设；特别是，他们假设大多数文件不会被频繁共享，而且会被从头到尾顺序访问。在这些假设下，AFS 的设计非常合理。

然而，这些假设并不总是成立。例如，设想一个应用程序定期往日志里追加信息。这些小规模的日志写入，往一个已有的大文件里添加少量数据，对 AFS 来说相当麻烦。还有很多其他难以处理的工作负载，例如事务（transaction）数据库中的随机更新。

要了解哪些类型的工作负载比较常见，可以参考已经做过的各种研究。这些研究都是工作负载分析的好例子 [B+91, H+11, R+00, V99]，其中也包括 AFS 的回顾性论文 [H+88]。

[2]这里我们假设 NFS 的写入都以块为大小、按块对齐；否则 NFS 客户端也得先读出这个块。我们还假设打开文件时没有使用 `O_TRUNC` 标志；如果用了，AFS 最初的打开就不会去取回这个马上要被截断的文件的内容。

 © 2008–23, ARPACI-DUSSEAU

此外，AFS 提供了由用户灵活管理访问控制（access control）的功能。因此，使用 AFS 时，用户可以在很大程度上控制谁能访问哪些文件。而 NFS 和大多数 UNIX 文件系统一样，对这类共享的支持要少得多。

最后，前面提到过，AFS 还加入了一些工具，让系统管理员能更简单地管理服务器。在系统管理方面，AFS 的考虑比同行领先了好几光年。

## 50.9 小结

AFS 向我们展示了，分布式文件系统可以用和 NFS 非常不同的方式来构建。AFS 的协议设计尤其重要：它用整文件缓存（whole-file caching）和回调把与服务器的交互降到最少，因此每台服务器能支持很多客户端，管理一个站点所需的服务器也就少了。还有许多其他特性，包括单一命名空间（name space）、安全和访问控制列表（access control list），让 AFS 用起来非常舒服。AFS 提供的一致性模型简单，易于理解和推断，不会像 NFS 那样偶尔出现奇怪的行为。

也许令人遗憾的是，AFS 很可能正在走下坡路。NFS 成了开放标准，很多厂商都支持它，它和 CIFS（基于 Windows 的分布式文件系统协议）一起主导了市场。虽然时不时还能看到 AFS 的部署（比如在一些教育机构，包括威斯康星大学），但真正能留下持久影响的，很可能是 AFS 的思想，而不是这个系统本身。事实上，NFSv4 现在加入了服务器状态（例如一条“open”协议消息），因此和 AFS 的基本协议越来越像了。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[B+91] “Measurements of a Distributed File System” by Mary Baker, John Hartman, Martin Kupfer, Ken Shirriff, John Ousterhout. SOSP ’91, Pacific Grove, California, October 1991. *An early paper measuring how people use distributed file systems. Matches much of the intuition found in AFS.*

[H+11] “A File is Not a File: Understanding the I/O Behavior of Apple Desktop Applications” by Tyler Harter, Chris Dragga, Michael Vaughn, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. SOSP ’11, New York, New York, October 2011. *Our own paper studying the behavior of Apple Desktop workloads; turns out they are a bit different than many of the server-based workloads the systems research community usually focuses upon. Also a good recent reference which points to a lot of related work.*

[H+88] “Scale and Performance in a Distributed File System” by John H. Howard, Michael L. Kazar, Sherri G. Menees, David A. Nichols, M. Satyanarayanan, Robert N. Sidebotham, Michael J. West. ACM Transactions on Computing Systems (ACM TOCS), Volume 6:1, February 1988. *The long journal version of the famous AFS system, still in use in a number of places throughout the world, and also probably the earliest clear thinking on how to build distributed file systems. A wonderful combination of the science of measurement and principled engineering.*

[R+00] “A Comparison of File System Workloads” by Drew Roselli, Jacob R. Lorch, Thomas E. Anderson. USENIX ’00, San Diego, California, June 2000. *A more recent set of traces as compared to the Baker paper [B+91], with some interesting twists.*

[S+85] “The ITC Distributed File System: Principles and Design” by M. Satyanarayanan, J.H. Howard, D.A. Nichols, R.N. Sidebotham, A. Spector, M.J. West. SOSP ’85, Orcas Island, Washington, December 1985. *The older paper about a distributed file system. Much of the basic design of AFS is in place in this older system, but not the improvements for scale. The name change to “Andrew” is an homage to two people both named Andrew, Andrew Carnegie and Andrew Mellon. These two rich dudes started the Carnegie Institute of Technology and the Mellon Institute of Industrial Research, respectively, which eventually merged to become what is now known as Carnegie Mellon University.*

[V99] “File system usage in Windows NT 4.0” by Werner Vogels. SOSP ’99, Kiawah Island Resort, South Carolina, December 1999. *A cool study of Windows workloads, which are inherently different than many of the* UNIX*-based studies that had previously been done.*

## 作业（模拟）

本节介绍 `afs.py`，这是一个简单的 AFS 模拟器，可以用来巩固你对 Andrew 文件系统（Andrew File System）工作原理的理解。详情请阅读 README 文件。

## 问题

1. 运行几个简单的例子，确认你能预测客户端会读到什么值。改变随机种子选项（`-s`），看看你能不能跟踪执行过程，预测出中间值以及最终存进文件的值。再改变文件数量（`-f`）、客户端数量（`-C`）和读操作比例（`-r`，取值在 0 到 1 之间），让问题更有挑战性一些。你也许还想生成稍长一点的跟踪，让交互更有意思，例如用 `-n 2` 或更大的值。
2. 现在做同样的事，看看你能不能预测出 AFS 服务器发起的每一次回调。试试不同的随机种子，并且让程序替你算出答案时（用 `-c`），一定要用较高的详细程度（例如 `-d 3`），以便看到回调在什么时候发生。你能准确猜出每次回调发生的时间吗？回调发生的确切条件是什么？
3. 和上面类似，用不同的随机种子运行，看看你能不能预测出每一步缓存的确切状态。用 `-c` 和 `-d 7` 运行，就能观察到缓存状态。
4. 现在来构造一些特定的工作负载。用 `-A oa1:w1:c1,oa1:r1:c1` 选项运行模拟。在随机调度程序（scheduler）下运行时，客户端 1 读取文件 a 可能看到哪些不同的值？（试试不同的随机种子，看看不同的结果。）在两个客户端操作所有可能的调度交错中，有多少种会让客户端 1 读到值 1，又有多少种读到值 0？
5. 现在来构造一些特定的调度。用 `-A oa1:w1:c1,oa1:r1:c1` 选项运行时，再分别使用下面这些调度：`-S 01`、`-S 100011`、`-S 011100`，以及你能想到的其他调度。客户端 1 会读到什么值？
6. 现在用这个工作负载运行：`-A oa1:w1:c1,oa1:w1:c1`，并像上面那样改变调度。用 `-S 011100` 运行时会发生什么？用 `-S 010011` 运行呢？决定文件最终值的关键是什么？

 © 2008–23, ARPACI-DUSSEAU

# 51

# 关于分布式的总结对话

**学生**：哎，这也太快了吧。要我说，快过头了！

**教授**：是啊，分布式系统（*distributed system*）既复杂又酷，非常值得好好学；只是不在这本书（或这门课）里讲。

**学生**：太可惜了，我还想多学点呢！不过我确实学到了几样东西。

**教授**：比如说？

**学生**：嗯，什么东西都可能出故障。

**教授**：开了个好头。

**学生**：可是只要有很多这样的东西（不管是磁盘、机器还是别的什么），就能把出现的大部分故障掩盖掉。

**教授**：接着说！

**学生**：有些基本的技术，比如重试（*retry*），真的很管用。

**教授**：没错。

**学生**：还有，你得仔细考虑协议（*protocol*），也就是机器之间交换的那些确切的比特。协议能影响方方面面，包括系统怎样应对故障，以及系统的可扩展性（*scalability*）有多好。

**教授**：你学东西真是越来越在行了。

**学生**：谢谢！您这个老师当得也不赖嘛！

**教授**：那我也多谢你啦。

**学生**：那这本书就到此结束了吗？

**教授**：我也说不准。他们什么都不告诉我。

**学生**：我也是。咱们走吧。

**教授**：好。

**学生**：您先请。

**教授**：不不，你先。

**学生**：请吧，教授先走。

**教授**：不不，还是你先请。

**学生**：（恼火地）好吧！

**教授**：（等着）……那你怎么还不走？

**学生**：我不知道该怎么走。原来我唯一能做的事，就是参加这些对话。

**教授**：我也一样。现在你学到了我们的最后一课……

 © 2008–23, ARPACI-DUSSEAU

# 第四部分

# 安全

# 52

# 关于安全的对话

本章作者：**Peter Reiher（UCLA）**

**教授**：你好啊，同学，又见面了！

**学生**：我还以为这些都结束了呢。三大支柱我们已经学完了，我甚至还留下来听了几个附录。这门课我就永远学不完了吗？

**教授**：那得看我是谁了。有的教授想讲安全，有的不想。你运气不好：既然你在这儿，那我就是想讲的那种。

**学生**：好吧，那我们还是赶紧开始吧。

**教授**：这就对了！早开始，早结束。那么，假设你有一个桃子……

**学生**：您说过，桃子的事至少已经讲完了！

**教授**：谈安全的时候，谎言永远是话题的一部分。总之，你有一个桃子。你肯定不希望一转身，就发现有人把你的桃子偷走了，对吧？

**学生**：嗯，只要它没烂成您最后拿到的那个样子，我大概不想让人偷走。

**教授**：要是你一转身，发现有人把你的桃子换成了一个萝卜，你大概也不会更高兴吧？

**学生**：应该不会，虽然我确实知道几道做萝卜的好菜。

**教授**：你也不希望每次伸手去拿桃子时，都有人把你的手打开，对不对？

**学生**：对，那也太没礼貌了。

**教授**：换成你计算机掌管的资源，你也一样不希望它们遇上这种事。如果那些资源真的很重要，你可能会更不高兴。你不希望正在写的情书泄露出去，不希望有人把你最喜欢的游戏存档重置、让你从头再来；要是在项目截止前一晚的半夜，你登录不了自己的计算机，你肯定会火冒三丈。

**学生**：没错，这些事都烂透了。

**教授**：咱们还是尽量保持专业的语气吧。毕竟这里是课堂。算是吧。操作系统安全讲的就是这些，我来这儿也正是要跟你讲这些。怎样确保秘密始终是秘密？怎样保证你的重要数据完整无损？怎样确保你想用计算机资源的时候就能用上？而且，这些问题适用于你计算机里的所有资源，时时刻刻，永远如此。

**学生**：这些听起来有点像我们以前讲过的可靠性（*reliability*）问题……

**教授**：是，也不是。坏事可能多少是意外发生的，也可能是计划不周造成的，可靠性关心的就是这类事。但我们要更进一步。有人想要你的桃子!!!!

**学生**：别嚷嚷了！要保持专业语气的可是您。

**教授**：抱歉，讲到这些东西我有时会激动。我想说的是，谈到安全，我们面对的是真正的对手，是想方设法给你捣乱的人类对手。这一点影响深远。他们很可能聪明、恶毒、执着、灵活，而且狡猾。你可能已经觉得整个宇宙都跟你过不去（至少大多数学生都这么觉得），但这些人是真真切切冲着你来的。不管他们使出什么手段，你都得保护好自己的资产。

**学生**：听起来挺难的。

**教授**：你根本想象不到……不过你会知道的！你会知道的!!（狂笑）


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

# 53

# 操作系统安全简介

本章作者：**Peter Reiher（UCLA）**

## 53.1 引言

计算系统的安全是一个至关重要的话题，而且越来越重要。计算机安全一旦失守，就会造成大量金钱损失，许多人的生活也会因此受到伤害。对计算机系统的攻击极为常见，只要你在用计算机，几乎在任何场景下都难以避免。一般来说，计算机系统的所有组成部分都可能遭到攻击，其中任何一个部分有缺陷，都可能让攻击者有机会做你想阻止的事。但从安全的角度看，操作系统尤其重要。为什么？

首先，几乎所有东西都运行在操作系统之上。一般来说，如果你所依赖的底层软件不安全，不管它是操作系统、某个中间件还是别的什么，那么它上面的东西也会不安全。这就像在沙子上盖房子。你也许能盖出一座漂亮坚固的房子，可洪水照样能冲走它底下的地基；不管你盖得多么用心，它还是会被彻底毁掉。同样，你的应用程序本身也许没有任何安全缺陷，但如果攻击者能滥用你所依赖的底层软件，窃取你的信息、让你的程序崩溃，或者以其他方式伤害你，你为保护自己的代码所做的努力可能就白费了。

这一点对操作系统尤其重要。如果你不运行某个 Web 服务器或数据库系统，你可能不在乎它的安全；如果你不用某个中间件平台，你可能也不在乎它的安全。但每个人都要运行操作系统，而且可选的操作系统相对不多。因此，操作系统中的安全缺陷，尤其是广泛使用的操作系统中的安全缺陷，会对大量用户和大量软件产生巨大影响。

操作系统安全如此重要的另一个原因是，归根结底，我们所有的软件都依赖底层硬件的正确运行：处理器、内存和外围设备。最终是谁控制着这些硬件资源？操作系统。

回想一下你已经学过的内存管理、调度、文件系统（file system）、同步等内容：如果对手能迫使操作系统的这些组件随意做出任何坏事，会发生什么？如果你理解了到目前为止学到的东西，就会觉得这种前景令人深感不安[1]。我们的计算生活依赖于操作系统按规定行事，尤其依赖于它不去做那些有利于对手、不利于我们的事。

保护操作系统的安全并非易事，因为现代操作系统庞大而复杂。写代码的经验应该已经告诉你：代码越多、算法越复杂，代码里就越可能有缺陷。软件安全上的失败通常就源于这类缺陷。庞大复杂的程序往往比小而简单的程序更难保证安全，而像现代操作系统这样庞大复杂的程序并不多见。

[1]如果你不理解，那你还有很多东西要重读。很多。

保护操作系统安全的另一个难点在于，操作系统大多要同时支持多个进程（process）。你已经学过，操作系统中有许多机制，用来把进程彼此隔离，并保护共享的硬件，不让它被用来干扰其他进程。如果可以放心让每个进程对机器上的任何硬件资源和任何数据为所欲为，而不必担心它伤害其他进程，那么保护系统安全就会容易得多。然而，我们通常不会对所有东西一视同仁地信任。从一个从未访问过的网站下载一个脚本并运行，你真的希望它能删光你磁盘上的每个文件、杀掉你所有其他进程，并开始用你的网络接口向其他机器发送垃圾邮件吗？大概不希望。但如果你是这台计算机的主人，只要你愿意，你就有权做所有这些事。除非操作系统足够小心，否则它运行的任何进程，包括运行你下载的那个脚本的进程，都能做你能做的任何事。

换个角度来看操作系统安全问题。操作系统的作用之一，是为应用程序提供有用的抽象（abstraction），供它们在上面构建。这些应用程序必须依赖操作系统正确实现这些抽象，让它们按规定工作。这类抽象的定义中往往有一部分就是它们的安全行为。例如，我们期望操作系统的文件系统会实施它应当实施的访问限制。这样，应用程序就可以基于这种期望来实现所需的安全目标，比如依靠文件系统的访问保证，确保自己指定为不可写的文件不会被改动。如果应用程序不能指望操作系统抽象的安全保证得到正确实现，它们就无法利用这些抽象来实现自己的安全目标。这至少意味着应用程序开发者要多做大量工作，另想办法达到这些目标。结合我们前面的讨论，如果他们必须依赖的抽象不可信，比如虚拟内存（virtual memory）或明确定义的调度策略（scheduling policy），他们往往根本无法实现这些目标。

显然，操作系统安全至关重要，却又难以实现。那么，我们该怎样保护操作系统的安全呢？这个问题已经困扰了一代又一代的计算机科学家，至今仍没有完整的答案。不过，有一些重要的原则和工具可以用来保护操作系统的安全。你可能接触到的任何通用操作系统，一般都内置了这些原则和工具，它们决定了你能用这个系统做什么，以及该怎么做。所以，即使你觉得自己对安全不感兴趣，你也需要了解操作系统为保护自身安全做了什么，才能明白怎样让系统做你想做的事。

关键问题：如何保护操作系统资源的安全

同一台机器上运行着多个可能并发（concurrent）、相互作用的进程，在这种情况下，我们如何确保每个进程获准访问的资源恰好就是它应当访问的资源，而且访问方式恰好符合我们的要求？需要操作系统提供哪些原语？硬件应当提供哪些机制？我们如何利用它们来解决安全问题？

## 53.2 我们要保护什么？

我们说操作系统应当安全，那它到底要保护什么？对这一点没有相当全面的认识，就不太可能保护好。好在对操作系统来说，至少在宏观层面，这个问题很容易回答：一切。这个答案并不令人宽慰，但对操作系统安全的广泛影响，最好还是有一个现实的认识。

一个典型的通用商用操作系统完全控制着机器上的全部（或几乎全部）硬件，能够做硬件允许的任何事。这意味着它可以控制处理器，读写所有寄存器，查看任何主存位置，执行任何外设支持的任何操作。因此，操作系统能做的事情包括：


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

- 查看或修改任何进程的内存
- 读取、写入、删除或破坏任何可写持久存储（persistent storage）介质上的任何文件，包括硬盘和闪存驱动器（flash drive）上的文件
- 改变任何进程的调度，甚至停止它的执行
- 向任何地方发送任何消息，包括把进程要发送的消息篡改后再发出去
- 启用或禁用任何外围设备
- 让任何进程访问其他任何进程的资源
- 随意夺走进程控制的任何资源
- 用危害最大的谎言回应任何系统调用（system call）

本质上，进程完全受操作系统的摆布。在一个恶意的操作系统面前，进程几乎不可能“保护”自身的任何部分。我们通常假定操作系统本身并没有恶意[2]，但如果恶意进程能借某个缺陷让操作系统行为失常，后果几乎同样糟糕，因为这可能让该进程获得操作系统本身的任何能力。这应该让你认真想想设计安全的操作系统有多重要，以及（更常碰到的）给你运行的操作系统打安全补丁有多重要。操作系统中的安全缺陷可能让它所运行的机器上的一切全面失守，所以预防这类缺陷、给已发现的缺陷打补丁至关重要。

## 53.3 安全目标与安全策略

我们说希望操作系统（或者任何系统）是安全的，到底是什么意思？这个说法相当含糊。我们真正的意思是：有些事我们希望在系统中发生，有些事我们不希望发生，而且我们希望能高度确信自己得到了想要的结果。和生活中的大多数事情一样，得到什么，往往就要为什么付出代价，所以值得仔细想想我们真正需要哪些安全属性和效果，只为这些买单，不为用不着的东西花钱。归结起来就是：我们要明确系统中与安全相关的行为应达到哪些目标，并选择能以合理代价实现这些目标的防御方法。

安全研究人员长期以来一直在宏观层面思考这个问题，并在概念上定义了三个重要的安全目标，许多系统（包括操作系统）都有这三个目标：

- **机密性**（confidentiality）：如果某条信息应当对他人隐藏，就不要让他们知道。例如，你不希望别人知道你的信用卡号，你希望这个号码保密。
- **完整性**（integrity）：如果某条信息或系统的某个组件应当处于某个特定状态，就不要让对手改变它。例如，如果你在网上订了一份意大利辣香肠披萨外送，你不希望某个恶作剧者把你的订单改成 1000 份凤尾鱼披萨。完整性的一个重要方面是真实性（authenticity）。通常，我们不仅要确定信息没有被改动过，还要确定它是由某个特定的一方创建的，而不是由对手创建的。
- **可用性**（availability）：如果某些信息或服务应当供你自己或他人使用，就要确保攻击者无法阻止你们使用它。例如，如果你的店正在搞大促销，你不希望竞争对手能封锁你店铺周围的街道，让顾客进不了你的店。

这三个目标都有一个重要的附加维度：我们希望系统中的共享是受控的。我们会和某些人分享秘密，而不和另一些人分享。我们允许某些人修改企业的数据库，但不是随便什么人

[2]如果你怀疑你的操作系统有恶意，那就该换个操作系统了。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：安全飞地

前面我们说过，操作系统控制着机器上“几乎全部”的硬件。这种留有余地的说法应该让你忍不住要问：“那么，它不控制哪些硬件呢？”起初，它确实控制着全部硬件。但从 20 世纪 90 年代起，硬件开发者逐渐意识到，需要让某些硬件在一定程度上与操作系统隔离。最早的这类硬件主要用来保护操作系统的启动过程。**TPM**，即**可信平台模块**（Trusted Platform Module），可以保证你启动的是你想要的那个版本的操作系统，防止有人试图启动已被攻破的系统版本。近来，更通用的硬件部件开始尝试控制机器上能做什么，通常针对某些特别重要的数据，往往是与密码学（cryptography）相关的数据。这类硬件部件称为安全飞地（security enclave），因为它们的目的是只允许安全地使用这些数据，即使是系统中最强大、最受信任的代码（操作系统本身）也不例外。它们常用于支持云计算环境中的操作，在这种环境中，多个操作系统可能运行在共享同一物理硬件的虚拟机（virtual machine）里。

可实际做起来，这一招比任何人预想的都难。安全上的招数大多如此。安全飞地（security enclave）提供的隔离（isolation）往往不如设计者所希望的那么彻底。不过，针对它们的攻击往往复杂而困难，而且通常要求攻击者已经能在系统上运行特权代码。所以，即使它们没有完全实现目标，也确实为防范被攻破的操作系统代码多加了一道屏障。

都行。有些系统需要对特定的一组优先用户（比如付费玩你的在线游戏的人）开放，而不对其他人（没付费的人）开放。在计算机里和在日常生活中一样，是谁在提要求，关系重大。

计算机系统安全的另一个重要方面是，我们常常希望确保某人告诉了我们某件事之后，日后不能否认他说过。这个方面通常称为**不可否认性**（non-repudiation）。否认自己的行为越难、代价越高，就越容易让人为自己的行为负责，人们也就越不可能做出恶意行为。毕竟，他们很可能会被抓住，而且很难抵赖。

这些都是宏观而笼统的目标。对一个实际系统来说，你需要深入到更详细、更具体的目标。例如，在一个典型的操作系统中，我们可能有这样一个机密性目标：一个进程的内存空间不能被另一个进程随意读取。我们可能有这样一个完整性目标：如果某个用户向某个文件写入一条记录，另一个本无权写这个文件的用户就改不了这条记录。我们可能有这样一个可用性目标：系统上运行的一个进程不能霸占 CPU，让其他进程得不到它们应得的那份 CPU。回想一下你在这门课上学到的进程抽象（process abstraction）、内存管理、调度、文件系统、IPC 等主题，你应该能想到操作系统中我们可能需要的其他一些显而易见的机密性、完整性和可用性目标。

对任何特定的系统来说，即使是这个层次的目标也不够具体。上面提到的完整性目标（用户的文件不应被另一个无权覆盖它的用户覆盖），就提示了对特定系统而言，我们的安全目标还需要具体到什么程度。也许有某个用户应当能覆盖这个文件，比如两个人合写一份报告时就可能如此。但这并不意味着，一个没有参与这份报告的第三个用户也应当能写这个文件。我们需要能在安全目标中规定这样的细节。操作系统是写给许多需求各异的人使用的，操作系统安全也应当体现这种通用性。我们希望操作系统的安全机制能灵活地描述详细的安全目标。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：安全与容错

在讨论进程抽象（process abstraction）时，我们讲到虚拟化（virtualization）如何保护一个进程不受其他进程行为的影响。例如，我们不希望进程的内存被另一个进程意外覆盖，所以虚拟化机制必须防止这种行为。当时我们讨论的主要是进程中的缺陷或错误。这和担心恶意行为（我们讨论安全时更常见的语境）真的有什么不同吗？把资源虚拟化之后，是否就解决了所有问题？

是，也不是。（这句话是不是很有帮助？）是的，如果我们把一切都完美地虚拟化，并且不允许任何东西之间有任何交互，我们很可能已经解决了大部分恶意行为的问题。然而，大多数虚拟化机制并不是完全刀枪不入的。没有人试图破坏它们时，它们工作得很好，但面对所有可能形式的不当行为，它们未必完美。其次，或许更重要的是，我们其实并不想让进程彼此完全隔离（isolate）。进程默认会共享一些操作系统资源，比如文件系统，也可以选择共享其他资源。这些有意放宽虚拟化的做法，在正确使用时并没有问题，但它们开放的合法共享，也可能成为恶意攻击的通道。最后，操作系统并不总是能完全控制硬件……

当然，最终必须由操作系统软件尽力实施这些灵活的安全目标，这意味着我们需要把这些目标编码成软件能理解的形式。我们通常必须把对安全目标的模糊理解，转换成非常具体的**安全策略**（security policy）。例如，对于上面说的那个文件，我们可能要规定这样一条策略："用户 A 和 B 可以写文件 X，其他用户都不能写。"规定得这样具体，再有精心设计和实现的机制作支撑，我们就有望实现安全目标。

注意这对操作系统安全的一个重要含义：在许多情况下，操作系统具备实现所需安全策略的机制，并且能高度可靠地正确运用它们，但前提是有人准确地告诉操作系统这条策略是什么。除了少数重要的例外（比如进程的地址空间在没有特别指示时保持私有），操作系统只提供通用的机制，用来实现各种具体的策略。然而，如果策略设计得不够明智、机制用得不够谨慎，你的操作系统实际会做的，就可能不是它应当做、也能够做的。

## 53.4 设计安全的系统

你们当中很少有人会自己写操作系统，对现有操作系统做重大修改的人也不多，但我们估计很多人会构建某种大型软件系统。许多计算机科学家的系统设计经验表明，有一些设计原则有助于构建有安全要求的系统。这些原则最初由 Jerome Saltzer 和 Michael Schroeder 在一篇很有影响力的论文中提出 [SS75]，不过其中有些来自其他人更早的观察。无论是原作者还是后来的评论者，都不会声称遵循这些原则就能保证系统安全，但实践证明，重视这些原则能带来更安全的系统，忽视它们则后果自负。我们在这里简要讨论一下。如果你真的要构建大型软件系统，值得去找来这篇论文（或者对它更详细的评论），仔细研究其中的概念。

1. **机制经济性**（economy of mechanism）：基本意思是让系统尽可能小、尽可能简单。简单的系统 bug 更少，行为也更容易理解。如果你不理解自己系统的行为，就不太可能知道它是否达到了安全目标。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

2. **故障安全默认**（fail-safe defaults）：默认应当是安全的，而不是不安全的。如果可以设置策略来决定系统的行为，就让这些策略的默认设置更安全，而不是更不安全。
3. **完全仲裁**（complete mediation）：这个安全术语的意思是，每次要执行某个操作时，都应当检查该操作是否符合安全策略，每一次都要检查[3]。
4. **开放设计**（open design）：假设你的对手知道你设计的每一个细节。如果系统仍然能实现它的安全目标，你就没问题。这条原则并不一定意味着你真的要把所有细节告诉每个人，而是说你的安全措施要建立在攻击者已经知道一切的假设上。实际上，他往往确实知道。
5. **特权分离**（separation of privilege）：执行关键操作时，要求有不同的参与方或不同的凭证（credential）。例如，双因素认证（two-factor authentication）同时凭密码和你持有的某件硬件来确定身份，比单独使用其中任何一种方法都更安全。
6. **最小特权**（least privilege）：只给用户或进程执行你想允许的操作所需的最小特权。你给某一方的特权越多，它滥用这些特权的危险就越大。即使你确信这一方没有恶意，如果它犯了错，对手也可能利用它的错误，拿它多余的特权去干坏事。
7. **最少公共机制**（least common mechanism）：对不同的用户或进程，用各自独立的数据结构（data structure）或机制来处理。例如，在虚拟内存系统中，每个进程都有自己的页表（page table），从而确保一个进程无法访问另一个进程的页。
8. **可接受性**（acceptability）：这是一个关键属性，但很多程序员并不把它放在心上。如果用户不愿意用你的系统，它就一文不值。太多前景看好的安全系统被弃用，就是因为它们对用户要求太多。

关于设计安全的系统，有用的建议远不止这些。还有很多不错的资料讲的是下一步：如何把好的设计转换成能实现预期安全性的代码；另有一些资料讲的是如何评估你构建的系统是否真的达到了这些目标。这些问题超出了本课程的范围，但等到你要构建大型复杂系统时，它们就极为重要了。关于安全编程的方法，如果你用 C 语言，可以从 Seacord 的书 [SE13] 读起。如果你用的是别的语言，应当去找一本针对该语言的类似书籍，因为很多安全编码问题都和语言的细节有关。关于如何全面评估系统是否安全，可以从 Dowd 等人的著作 [D+07] 读起。

## 53.5 操作系统安全基础

那么，一个典型的操作系统有一组安全目标，涉及机密性、完整性和可用性的各个方面。其中一些目标往往内置在操作系统模型中，另一些则由系统的所有者或用户控制。内置的目标，是那些极其常见的目标，或者是为了让更具体的目标得以实现而必须保证的目标。这些内置目标大多与控制进程对硬件各部分的访问有关。这是因为硬件由系统上的所有进程共享，除非谨慎地控制共享，否则一个进程就可能干扰另一个进程的安全目标。其他内

[3]许多系统常常为了降低开销或提高易用性而忽视这条原则。所有工程设计都有一个压倒一切的特点：你常常必须在相互冲突的目标之间取得平衡，就像我们在课程前面（比如调度那几章）看到的那样。之后我们会在安全的语境下再多谈谈这一点。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

提示：当心最薄弱的环节

值得记住的是，攻击你系统的人和你有许多共同的特点。特别是，他们很可能相当聪明，也很可能有点懒——这里是褒义，意思是他们不做不必要的工作。这意味着，攻击者往往会选择最容易的途径来攻破你系统的安全。如果你选了“password”作为访问系统的密码，他们是不会去找什么零日缓冲区溢出（buffer overflow）漏洞的。

对你来说，实际的含义是：在保护系统安全上花的时间，大部分应当用来找出并加固你最薄弱的环节。最薄弱的环节，就是系统中保护最差、最容易受攻击的部分，是你没法藏起来、也没法借助某种外部安全系统来加强的部分。一个运行中的系统，最薄弱的环节往往其实是它的人类用户，而不是软件。改变人的行为很难，但你可以在设计软件时考虑到，攻击者可能会试图欺骗合法用户去滥用它。还记得最小特权（least privilege）原则吗？如果攻击者能骗一个拥有全部特权的用户去滥用系统，后果会比骗一个只能损害自己资产的用户严重得多。

一般来说，思考安全和思考许多其他系统设计问题有些不同。它更具对抗性。如果想多了解怎样思考自己所构建系统的安全性，可以看看 Schneier 的书《Secrets and Lies》[SC00]。

置目标与操作系统提供的服务有关，比如文件系统、内存管理和进程间通信（interprocess communication）。如果不谨慎地控制这些服务，进程就可能破坏系统的安全目标。

显然，系统安全有很大一部分会和进程的处理有关。如果操作系统能把进程清清楚楚地分隔开，只有在操作系统的帮助下才能打破这种分隔，那么无论是共享的硬件还是操作系统服务，都不能被用来破坏我们的安全目标。这一要求意味着，操作系统在允许使用硬件和自身服务时需要非常小心。在许多情况下，操作系统也很有条件做到这一点。例如，操作系统控制着虚拟内存，而虚拟内存又完全控制着每个进程能访问哪些物理内存地址（physical memory address）。有了硬件支持，进程甚至无法引用一个没有映射到它的虚拟内存空间（virtual memory space）中的物理内存地址。（我们这些搞软件的人，应该记得经常感谢搞硬件的人给了我们这么多好东西。）

系统调用为操作系统提供了另一个实施保护的机会。在大多数操作系统中，进程要访问系统服务，就得显式地发起系统调用，这在前面的章节中讨论过。你已经学过，系统调用会把执行模式从处理器的用户模式（user mode）切换到它的管理模式（supervisor mode），同时调用一段相应的操作系统代码。这段代码可以确定是哪个进程发起了系统调用，以及该进程请求的是什么服务。前面我们只讨论了这如何让操作系统调用正确的那段系统代码来执行服务，以及在服务完成后记住该把控制权交还给谁。但同样的机制也让操作系统有机会检查：按照系统的安全策略，所请求的服务是否应当允许。由于访问外围设备要经过设备驱动程序（device driver），而设备驱动程序通常也要靠系统调用来访问，同样的机制也能保证硬件访问正确地遵循安全策略。

那么，进程发起系统调用时，操作系统会使用进程控制块（process control block）或类似结构中的进程标识符（process identifier）来确定该进程的身份。然后，操作系统可以使用**访问控制机制**（access control mechanism）来决定这个已确定身份的进程是否**获得授**


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

**权**（authorized），可以执行所请求的操作。如果获得了授权，操作系统要么代表该进程亲自执行这个操作，要么安排该进程自己执行，无需系统进一步干预。如果该进程没有获得授权，操作系统只需为这次系统调用生成一个错误码，并在调度算法允许时把控制权交还给该进程。

## 53.6 小结

操作系统的安全，无论对它自身还是对它上面的应用程序都至关重要。这种软件一旦出现安全故障，几乎可能导致无穷无尽的恶果。实现系统安全虽然不容易，但有一些已知的设计原则可以帮上忙。这些原则不仅在设计操作系统时有用，在设计任何大型软件系统时也有用。

在操作系统中实现安全，取决于你的安全目标是什么。这些目标通常涉及机密性、完整性和可用性。具体到某个系统，这些安全目标的细节各不相同，这意味着不同的系统会有不同的安全策略，帮助它们实现各自具体的安全目标。和操作系统设计的其他领域一样，我们处理这些不同需求的方式，是把任一特定系统所用的具体策略，与所有系统用来实现策略的通用机制分开。

接下来要回答的问题是：操作系统应当提供哪些机制，来帮助我们支持通用的安全策略?进程和内存的虚拟化是一种有用的机制，因为它让我们能在很大程度上控制进程的行为。在接下来的几章中，我们还会介绍其他几种有用的操作系统安全机制。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

## 参考文献

[D+07] "The Art of Software Security Assessment" by Mark Dowd, John McDonald, and Justin Schuh. Addison-Wesley, 2007. *A long, comprehensive treatment of how to determine if your software system meets its security goals. It also contains useful advice on avoiding security problems in coding.*

[SC00] "Secrets and Lies" by Bruce Schneier. Wiley Computer Publishing, 2000. *A good high-level perspective of the challenges of computer security, developed at book length. Intended for an audience of moderately technically sophisticated readers, and well regarded in the security community. A must-read if you intend to work in that field.*

[SE13] "Secure Coding in C and C++" by Robert Seacord. Addison-Wesley, 2013. *A well regarded book on how to avoid major security mistakes in coding in C.*

[SS75] "The Protection of Information in Computer Systems" by Jerome Saltzer and Michael Schroeder. Proceedings of the IEEE, Vol. 63, No. 9, September 1975. *A highly influential paper, particularly their codification of principles for secure system design.*


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

# 54

# 身份认证

本章作者：**Peter Reiher（UCLA）**

## 54.1 引言

我们要应对各式各样的安全目标，以及为实现这些目标而制定的安全策略（security policy）。那么，我们需要操作系统做什么呢？操作系统为进程（process）提供服务，其中有些服务牵涉到安全。显然，在这种情况下，操作系统需要小心行事，在安全上做出正确的处理。但操作系统之所以提供这些服务，是因为这些事有时确实需要做。所以，操作系统能提供的服务，大概都应该提供，只是要在合适的情况下。

操作系统是执行某项服务，还是因为它会破坏安全目标而拒绝执行，全看上下文。上下文里最重要的因素，恐怕是谁在提出请求。在现实世界里，如果你的另一半让你下班路上顺便去商店买一加仑牛奶，你多半会照办；要是街上的陌生人提同样的要求，你多半不会。在操作系统中，如果系统管理员要求操作系统安装一个新程序，操作系统大概应该照办；而如果是从随便哪个网页上下载的脚本要求安装新程序，操作系统在安装之前就应该更加小心。在计算机安全的讨论中，我们常把提出请求的一方称为**主体**（principal）。主体是在安全上有意义、能够请求访问资源的实体，比如人类用户、用户组，或者复杂的软件系统。

所以，知道是谁在请求操作系统服务，对实现安全目标至关重要。操作系统怎么知道这一点呢？我们倒过来推一推，把它弄清楚。

请求操作系统服务，最常见的方式是由某个进程发起系统调用（system call），从用户代码陷入（trap）操作系统。随后操作系统接管控制，为响应这次系统调用执行某项服务。发起调用的进程关联着一个由操作系统控制、描述该进程的数据结构（data structure），所以操作系统可以检查这个数据结构，确定进程的身份。有了这个身份，操作系统就有机会依据策略决定是否执行所请求的操作。在计算机安全的讨论中，代表主体发出请求的进程或其他活动的计算实体，常称为该主体的**代理**（agent）。

请求是要访问某个特定的资源，我们通常把这个资源称为访问请求的**客体**（object）[1]。操作系统要么已经确定这个代理进程可以访问该客体，要么还没有。如果已经确定该进程获准访问，操作系统只需把这个决定记下来，大概记在 PCB 这类每个进程一份的数据结构里。例如，我们在研究内存虚拟化（memory virtualization）时发现，页表（page table）这类每个进程一份的数据结构，记录了进程在任一时刻可以访问哪些页和页帧（page frame）。操

[1]这又是计算机科学对 object 一词的重载。这里它与“面向对象”无关，指的是更一般的概念：一个具有边界和行为的具体资源，比如一个文件或一个 IPC 通道。

作系统为记录这类访问决定、以备日后查阅而创建和管理的数据，无论是什么形式，都叫作**凭证**（credential）。

然而，如果操作系统还没有生成表明某个代理进程可以访问某个客体的凭证，它就需要知道该进程所属主体的身份，才能决定是否批准这个请求。不同的操作系统给主体用过不同类型的身份。例如，大多数操作系统都有用户身份的概念，这里的用户通常是某个人。（多年来，用户的概念不断扩展，变得更加强大，后面我们会看到。）这样，某个人运行的所有进程，也许都关联着同一个身份。另一种常见的身份类型是用户组。在一家制造企业里，你可能希望让所有销售人员都能访问库存信息，好让他们知道仓库里有多少零件和小玩意儿，而人力资源部门的人就没必要访问这些信息[2]。

还有一种身份形式是进程正在运行的程序。回想一下，进程就是运行中的程序（running program）。在有些系统中（比如 Android 操作系统），你可以把某些特权授予特定的程序。这些程序每次运行时都能使用这些特权，而其他程序不能。

不管我们根据哪种身份做安全决定，都必须有某种办法把这个身份附加到特定的进程上。显然，身份附加得对不对，是一个至关重要的安全问题。如果你把一个程序员的进程误认成会计部门员工的进程，你的银行账户可能就被掏空了。（更别提还得再招一个程序员。）又或者，你的公司总裁要向投资人做一次重要的演示，而你没能正确识别他或她的身份，系统因此不让总裁访问那份本可以打动几位潜在投资人的演示文稿；等公司查明是你搞砸了下一轮创业融资，你可能就要丢掉饭碗了。

另一方面，除了操作系统自身的活动，一切都是由某个进程执行的。所以，只要我们能把进程的身份弄对，就可以相当有把握地在每个重要操作上检查策略。不过还要记住，操作系统做身份认证（authentication）的方式通常还有一个重要特点：主体一旦通过身份认证，系统几乎总是会在至少整个进程生命周期内都依赖这个认证结论。这一特点让“做对”变得格外重要，出了错不容易纠正。由此引出关键问题：

关键问题：如何安全地识别进程
如果系统上运行着属于不同主体的进程，我们如何确保每个进程都附加了正确的身份？创建新进程时，如何确保新进程的身份是正确的？如何确保恶意实体无法不当地更改进程的身份？

## 54.2 给进程附加身份

进程是从哪里来的？通常是由其他进程创建的。所以，给新进程附加身份的一个简单办法，就是复制创建它的那个进程的身份：子进程（child process）继承父进程（parent process）的身份。具体来说，操作系统处理旧进程 A 创建新进程 B 的调用（例如 `fork`）时，会查看 A 的进程控制块（process control block），确定 A 的身份，为 B 创建一个新的进程控制块，然后把 A 的身份复制进去。很简单，对吧？

[2]还记得上一章讲的最小特权（least privilege）原则吗？这里就是它的一个应用实例。只要你没有给人力资源部门的员工下令清空仓库里 pop-doodle 的权限，即使其中有人心怀不轨，也做不到这一点。阅读本书安全部分的各章时，留意一下我们前面讨论过的那些安全原则还有哪些其他应用。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

如果所有进程的身份始终都相同，这样做当然没问题。我们可以在操作系统启动时创建一个原始进程，也许给它分配某个不属于任何人类用户的特殊系统身份。其他所有进程都是它的后代，都继承这唯一的身份。可是，如果真的只有一个身份，我们就没法实施任何区分不同进程特权的策略。

我们必须让某些进程具有不同的身份，并利用这些差异来管理安全策略。考虑一个多用户系统。我们可以根据进程属于哪个人类用户来给它分配身份。如果我们的安全策略主要是规定某些人可以做某些事、另一些人不可以，那么我们现在就知道该怎么做决定了。

如果进程有一个与安全相关的身份，比如用户 ID，我们就得给新进程设置正确的用户 ID。在大多数系统中，用户平时都通过某个进程工作：在命令行系统中是 shell 进程，在窗口系统中是窗口管理器进程。你已经想到这两者本身也都得是进程了吧？所以，你在 shell 里输入一条命令，或者在窗口系统里双击图标启动一个进程，就是在要求操作系统以你的身份启动一个新进程。

很好！不过我们还有一个问题要处理：那个 shell 或窗口管理器是怎么把你的身份附加到自己身上的？这时，操作系统的一点特权就派上用场了。用户刚开始与系统交互时，操作系统可以为这个用户启动一个进程。由于操作系统可以随意摆弄自己的数据结构，比如进程控制块（process control block），它就可以把这个新进程的所有者设为刚加入系统的那个用户。

这也没问题，但操作系统是怎么确定用户的身份，从而正确设置进程所有者的呢？你大概能猜到答案：用户登录了，也就是说，用户向操作系统提供了身份信息，证明了自己是谁。这样，我们就为操作系统找到了一个新要求：它必须能够向人类用户询问身份，并验证他们确实是自己所声称的那个人，这样才能给进程附加可靠的身份，进而用这些身份实施安全策略。在操作系统里，一件事往往会牵出另一件事。

那么，操作系统怎么做到这一点呢？不难看出，我们正要在这一步上盖起一座高耸的安全大厦，影响难以预料，而一切都取决于操作系统在这里做出正确的决定，所以这件事很重要。我们有哪些选择呢？

## 54.3 如何认证用户？

*话说这个人走到一台计算机前……*

先不管能编出什么笑话，我们能做些什么，让系统以合理的准确度确定这个人是谁？首先，如果这个人根本不是系统的授权（authorized）用户，就应当把这次想混进来的企图彻底挡住。其次，如果他或她是授权用户，我们就需要确定：是哪一个？

传统上，认证（authenticate）人的身份有以下三种方式：

- 基于你知道什么来认证
- 基于你拥有什么来认证
- 基于你是什么来认证

这里说的“传统上”，指的是，嗯，真正古典意义上的传统，可以一直追溯到古希腊人和古罗马人。例如，公元前 2 世纪的波利比乌斯（Polybius）描述过罗马军队怎样用“口令”来区分敌我 [P-46]，这是基于你知道什么来认证的例子。公元 2 世纪的某个时候，一位名叫


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

Celer 的罗马建筑师为他的一个奴隶写了一封推荐信（这封信至今还在），让奴隶交给帝国的一位代理官（procurator）[C100]，这是基于奴隶拥有什么来认证。再往前，在（字面意义上的）《圣经》时代，一场战斗之后，基列人要求逃难者说出“示播列”（shibboleth）这个词，因为他们要找的敌人（以法莲人）发不准这个词的音 [JB-500]。这是一种基于你是什么来认证的方式：你是以基列方言为母语的人，还是以以法莲方言为母语的人。

既然已经说明这些认证方法由来已久，我们就跳过几个世纪的历史，直接进入计算机时代，讨论如何在计算机认证中使用它们。

## 54.4 基于你知道什么来认证

基于你知道什么来认证，最常见的做法是使用密码。在计算机安全领域，密码有着漫长（而且基本上不怎么光彩）的历史，至少可以追溯到 20 世纪 60 年代初 MIT 的 CTSS 系统 [MT79]。密码是只有被认证的一方才知道的秘密。登录时，这一方把这个秘密透露给计算机的操作系统，以此证明自己的身份。（你应该会想：这是否意味着系统也必须知道密码？这又会带来什么后果？我们后面会讲到。）显然，这种认证方式的效果取决于几个因素。我们假定其他人不知道这一方的密码，如果知道，系统就会被骗。我们还假定其他人也猜不到这个密码。当然，还要假定这一方自己必须知道（并且记得）密码。

我们先来解决其他人知道密码的问题。先不说猜测，他们还能怎么知道密码？已经知道密码的人可能会说漏嘴，所以必须知道密码的人越少，我们要担心的人就越少。我们要认证的那个人当然得知道密码，因为我们正是根据他知道密码来认证他的。我们确实不希望任何其他人能以这个人的身份通过系统认证，所以最好没有任何第三方知道密码。从广义上考虑“第三方”的含义，这也意味着用户不应该把密码写在纸条上，因为任何偷到纸条的人都会知道密码。但似乎还有一方需要知道密码：我们的系统本身。这又暗示了另一个可能的漏洞，因为系统保存的密码副本可能会泄露出去[3]。

提示：避免存储秘密

存储明文（plaintext）密码或密钥（key）这样的秘密是一件危险的事，因为秘密往往会泄露出去。如果不需要，就不要存储它们，以此保护你的系统。如果确实需要，就用强大的密码学散列函数（cryptographic hash function），以散列（hash）后的形式存储它们。如果做不到，就用安全的密码算法（cipher）加密（encrypt）它们。（也许你正在心里抱怨我们还没讲过这些。耐心点。）存储的地方要尽可能少，副本也要尽可能少。别忘了编辑器的临时文件、备份、日志之类的地方，秘密可能也藏在那里。记住，凡是嵌进你交给别人的可执行文件里的东西，都不会再是秘密，所以把秘密存在可执行文件里尤其危险。在某些情况下，甚至只保存在运行中程序堆里的秘密也被泄露过，所以即使在运行中的程序里，也要避免存储和保留秘密。

[3] “可能”这个词太轻了。已知最早的存储密码泄露事件发生在 1962 年 [MT79]；直到今天，这类泄露仍然频繁得令人沮丧，而且规模大得多。[KA16] 讨论了一次超过 1 亿个以可用形式存储的密码被泄露的事件。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

不过，有意思的是，我们的系统其实并不需要知道密码。仔细想想系统在检查用户提供的密码时都在做什么。它是在检查用户是否知道密码，而不是密码究竟是什么。那么，如果用户把密码提供给我们，而我们并不知道密码，系统到底怎么才能做到这一点呢?

你其实已经知道答案了，至少一听到答案就会一拍脑门说："我早该想到的。"存储密码的**散列值**（hash），而不是密码本身。用户给出自称的密码时，对这个密码做散列，再与存储的散列值比较。如果相符，你就相信他或她知道密码；如果不符，就不相信。很简单，对吧? 这样一来，你的系统就不需要存储真正的密码了。这意味着，即使你在存储认证信息时不够小心，你丢失的其实也不是密码，而只是它们的散列值。散列算法本质上是不可逆的，所以对手无法利用偷到的散列值得到密码。如果攻击者提供的是散列值而不是密码，系统会对这个散列值再做一次散列，而散列值的散列值是不会与散列值相符的。

不过事情还不止这么简单。存储密码的散列值，好处在于即使存储的副本泄露给了攻击者，攻击者也不知道密码本身。但仅仅存储一个与密码不同的东西还不够。我们还要确保，不管存储的是什么，都不会给攻击者猜测密码提供任何帮助。如果攻击者偷到了散列后的密码，他或她应当无法通过分析散列值得到任何关于密码本身的线索。有一类特殊的散列算法，称为**密码学散列函数**（cryptographic hash），它让人无法利用散列值推算出密码是什么，除非真的把猜测的密码送进散列算法去试。密码学散列函数有个麻烦：很难设计，所以即使是聪明人也不该自己动手，而应使用专家设计好的。这正是现代系统对密码做散列时应当做的：使用经过深入研究、没有已知缺陷的密码学散列函数。在不同的时期，满足这些要求的密码学散列算法可能各不相同。在撰写本文时，SHA-3 [B+09] 是美国的密码学散列算法标准，是一个不错的选择。

我们来看另一个问题：猜测。想冒充某个用户的攻击者，能不能直接猜出密码? 考虑最简单的密码：只有一位，值为 0 或 1。如果你的密码只有一位长，攻击者猜"0"就有一半的机会猜对。即使猜错了，只要允许猜第二次，攻击者也就知道密码是"1"，从而猜对。

显然，一位的密码太容易猜了。8 位的密码怎么样? 这样你可以选择的密码就有 256 种。如果攻击者猜 256 次，迟早会猜对，平均要猜 128 次。这比只需要猜两次要好，但还是不够好。到这里你应该清楚了，密码的长度对抵御猜测至关重要。密码越长，越难猜。

但还有另一个重要因素：密码通常是人用键盘之类的设备输入的。既然我们已经认定把密码写下来不安全，那这个人就只能记住它。早期的密码为了解决这个问题，只允许使用字母。这样虽然更容易输入和记忆，却也让攻击者要猜的位模式大大减少，因为所有不代表字母的位模式都不会出现在密码里。随着时间推移，密码系统逐渐扩大了密码中可用的字符范围，包括大小写字母、数字和特殊字符。可能性越多，越难猜。

所以，我们需要由多种不同类型的字符组成的长密码。但攻击者知道，人们不会用这些字符的随机组合做密码。人们常常选择名字或熟悉的单词，因为它们容易记。因此，试图猜测密码的攻击者会先尝试名字和单词列表，然后才尝试随机字符串。这种猜测密码的方式称为**字典攻击**（dictionary attack），它可能非常有效。这里的字典不是韦氏词典（甚至也不是《牛津英语词典》），而是一份专门的列表，收录了人们常用作密码的单词、名字、有意义的数字串（比如"123456"）以及其他字符模式，并按被选作密码的可能性排序。一次好的字典攻击可以破解一个典型网站 90% 的密码 [G13]。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：密码库

避免选择密码这一难题的一种办法，是使用所谓的密码库（password vault）或钥匙串（key chain）。这是保存在你计算机上的一个加密文件，里面存着各种密码。它本身用一个密码加密。要从密码库里取出密码，你必须提供密码库本身的密码。这样，为每个网站记住一个不同密码的问题，就变成了只需记住一个密码。此外，它还能确保攻击者只有在既拥有打开密码库的那个特殊密码，又能访问密码库本身时，才能使用你的密码。当然，这样安全地存储密码，好处也受限于密码库中所存密码的强度，因为猜测和字典攻击仍然奏效。有些密码库会为你生成强密码，这些密码不太好记，但这无关紧要，因为需要记住它们的是密码库，而不是你。你也能找到把密码存到云端的密码库。然而，如果你把密码的明文版本提供给它们存储，你就是在和另一个其实并不需要知道这个密码的实体共享密码，从而承担了一个你或许不该承担的风险。如果云端只存储你加密后的密码，风险就要低得多。

只要你设置系统时够聪明，攻击者就没法对登录进程发起远程字典攻击。只要稍加小心，攻击者在头五六次猜测中就猜不出用户的密码（唉，有时就是一点都不小心，攻击者也就猜中了）。而且没有任何正当理由让你的系统允许远程用户对一个账户的密码猜上 15,000 次都猜不对。所以，要保护账户免受这类攻击，可以在某个账户的密码被猜错太多次后关闭对它的访问，或者（更好的办法）在猜错几次之后大幅放慢密码检查的速度，让一次漫长的字典攻击耗时长到无法实施。

但如果攻击者偷走了你的密码文件呢?既然我们假定你一直在认真听讲，文件里存的是密码的散列值，而不是密码本身。但我们同样假定，你也听了我们的建议，用的是一个广为人知的密码学散列函数；你知道用的是哪个，偷走你密码文件的人也知道。如果攻击者拿到了你散列后的密码、散列算法、一部字典和一些算力，就可以慢慢地、一个一个地猜你的密码。更糟的是，如果每个人都使用同一个密码学散列算法（实际上他们很可能就是这样），攻击者只需把每个可能的密码送进散列函数（hash function）算一次，把结果存起来（本质上就是把字典转换成了散列形式）。这样，攻击者偷到你的密码文件后，只需把你散列后的密码与这部新生成的散列字典做字符串比较，速度要快得多。

有一个简单的解决办法：在对新密码做散列并存入密码文件之前，先生成一个很大的随机数（比如 32 位或 64 位），把它拼接到密码上，对拼接结果做散列，再把散列值存起来。你还需要存储这个随机数，因为用户尝试登录、提供正确的密码时，你需要把用户提供的内容与存储的随机数拼接起来，再送进散列算法。否则，单独对密码做散列，结果不会与你存储的值相符。通常，这个随机数（称为**盐**，salt）就存储在密码文件中，紧挨着散列后的密码。这个概念最早由 Robert Morris 和 Ken Thompson 在一篇关于密码安全的早期论文中提出 [MT79]。

这为什么有帮助？攻击者再也不能只为字典中的密码生成一份从密码到散列值的转换表了。他需要为每一个可能的盐各生成一份转换表，因为被偷走的密码文件里，每个密码的盐很可能都不同。如果盐有 32 位，字典里的每个单词就要有 $2^{32}$ 种不同的转换，预先计算


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

转换的办法也就行不通了。字典攻击只能针对被偷的密码文件中的每一项，用该密码的盐对每个猜测重新做散列。如果你的密码选得不好，攻击仍然可行，但代价要高得多。

任何使用密码并重视安全的好系统，都会存储经过密码学散列并加了盐的密码。如果你的系统没有这样做，你就是在让用户冒险。

使用密码还有其他一些令人担忧的问题，但其中很多并不是操作系统特有的，所以这里就不再往它们身上泼脏水了。只需说一句：计算机安全界普遍认为，密码是一种过时的技术，对今天的环境来说已经不够安全了。它们充其量只能作为协同使用的几种认证机制之一。这种思路称为**多因素认证**（multi-factor authentication），其中最广为人知的是**双因素认证**（two-factor authentication）。你可能已经熟悉这个概念：要从 ATM 取钱，你需要知道你的个人识别码（PIN），这本质上就是一个密码。但你还需要提供更多证明你身份的证据……

## 54.5 基于你拥有什么来认证

我们大多数人可能都遇到过需要出示身份证件才能进入某个地方的情况。至少，我们可能都参加过凭票入场的活动。这两种都是基于你拥有什么来认证的例子，在这里，你拥有的就是身份证件或门票。

向操作系统证明自己的身份时，情况就有些不同了。在一些特殊情况下，比如上面提到的 ATM，设备（毕竟它里面也有一台计算机，你知道的吧?）有专门的硬件来读取我们的 ATM 卡。这个硬件让它能确定：没错，我们确实有那张卡，从而在 PIN 之外提供了进一步的证明。大多数台式机、笔记本电脑、平板电脑、智能手机之类的设备都没有这种专门的硬件。那它们怎么知道我们拥有什么呢?

如果我们有某种能插到计算机某个端口（port）上的东西，比如使用 USB 的硬件令牌（token），那么在合适的软件支持下，操作系统就能判断试图登录的用户有没有这个设备。有些安全令牌（有时叫作**加密狗**，dongle，这名字起得实在不怎么样）就是这样工作的。

在另一些情况下，既然我们反正是要认证一个人类用户，就可以借助这个人的能力，把信息从他或她拥有的东西传给需要认证的系统。例如，有些智能令牌会在一块很小的内置屏幕上显示一个数字或字符串。用户读出屏幕上的信息，用键盘输入计算机。操作系统并没有直接得到用户拥有这个设备的证据，但如果只有能拿到这个设备的人才可能知道该输入什么信息，这个证据也几乎一样可靠。

这类设备要靠频繁更换它（直接或间接）传给操作系统的信息才管用，也许每隔几秒换一次，也许用户每次尝试认证时都换。为什么? 因为如果不换，任何能从设备上获知这一静态信息的人，就用不着这个设备也能冒充用户了。认证机制就从“你拥有的东西”变成了“你知道的东西”，其安全性也就取决于攻击者获知这个秘密的难度。

所有基于你拥有什么的认证方式都有一个弱点：如果你没带着它怎么办?如果你今天早上把智能手机落在梳妆台上了呢? 如果你的加密狗（dongle）在上班路上从口袋里滑出去了呢? 如果一个手法高明的扒手在咖啡店里蹭了你一下，顺走了你的秘密认证设备呢? 这下你有了双重麻烦。第一，你没有向操作系统证明身份所需的那件法宝。你可以对着计算机随便抱怨，它才不在乎。它会坚持要你拿出你弄丢的那件法宝。第二，别人拿到了你的法宝，可能就能冒充你，骗过靠“你拥有什么”来认证的操作系统。注意，我们前面提到的多因素认


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：Linux 的登录过程

Linux 沿袭早期 UNIX 系统的传统，基于密码认证用户，然后把这个身份绑定到与新登录用户关联的一个初始进程上，大致如上文所述。下面一步步详细描述用户走到键盘前登录 UNIX 系统时实际发生了什么，作为真实操作系统处理这一重要安全问题的具体例子。

1. 一个以特权系统身份运行的特殊 login 进程显示一个提示，要求用户输入身份，通常是一个简短的用户名。用户输入用户名并按回车。用户名会回显在终端上。
2. login 进程提示用户输入密码。用户输入密码，密码不回显。
3. login 进程在密码文件中查找用户提供的名字。如果找不到，login 进程就拒绝这次登录尝试。如果找到了，login 进程就确定内部用户标识符（一个唯一的用户 ID 号）、用户所属的组（另一个唯一的 ID 号）、登录完成后应提供给该用户的初始命令 shell，以及该 shell 应在哪个主目录（home directory）中启动。此外，login 进程还会找到该用户密码的盐，以及正确密码加盐（salted）并散列后的版本，它们永久存储在系统中一个安全的地方。
4. login 进程把用户密码的盐与用户提供的密码组合起来，对组合结果做散列，再与上一步得到的存储版本比较。如果不相符，login 进程就拒绝这次登录尝试。
5. 如果相符，login 进程就 fork 出一个进程，把它的用户和组设为前面确定的值。login 进程的特权身份允许它这样做。然后新进程切换到用户的主目录（home directory），并 exec 与该用户关联的 shell 进程；目录名和 shell 类型都是在第 3 步确定的。

还有一些其他细节，涉及在这个用户退出后，确保同一个终端上还能让另一个用户登录，这里就不展开了。

注意，在第 3 步和第 4 步中，登录失败可能是因为系统里没有这个用户名，也可能是因为密码与用户名不符。如果登录失败，Linux 和大多数其他系统都不会指明是哪种情况。这样，攻击者就无法仅靠输入猜测来得知系统合法用户的名字，因为他们不知道自己猜的是一个不存在的名字，还是为一个合法用户名猜错了密码。不向未经认证（non-authenticated）的用户提供有用信息，一般来说是一种很好的安全思路，在其他类型的系统中也适用。

想一想，为什么 Linux 的登录过程回显用户输入的用户名，却不回显密码？回显用户名在安全上真的毫无坏处吗？回显用户名是绝对必要的吗？还是说这是用安全换取便利的一种权衡？为什么不回显密码？

证在这里也能救你一命。如果小偷偷了你的安全令牌（security token），却不知道你的密码，那么小偷还得先猜出密码，才能冒充你[4]。

如果你在实践中研究系统安全足够久，就会发现，学者（比如我）告诉你什么是安全的，和现实世界中实际发生的事情之间，存在着相当大的差距。差距的部分原因在于，现实世界

[4]当然，前提是你没有用记号笔把密码写在被偷的那张智能卡背面。嗯，当时这看起来是个好主意……


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

需要处理现实的问题，比如用户的便利。另一部分原因在于，安全学者往往会贬低任何他们能想出办法破坏的东西，即使那个办法本身并不怎么实际。拿基于你拥有什么的认证机制来说，有一个例子是给用户的手机发短信，用户再把短信内容输入计算机，以此认证用户。从理论上讲，这听起来非常薄弱。除了手机丢失的危险，安全专家还喜欢设想一些奇特的攻击，比如短信被错发到攻击者的手机上，让攻击者能把短信里的秘密信息提供给计算机。

在实践中，人们通常随身带着手机，也会相当小心，不让它丢了。即使弄丢了，他们也会很快发现，并同样迅速地采取措施解决问题。所以，从手机丢失到系统得知不能再用这部手机认证你，中间的时间窗口很可能相对较短。同样在实践中，重定向（redirect）发往手机的短信是可能的，但远非轻而易举。至少在绝大多数情况下，攻击者为此付出的努力很可能超过他骗过认证系统所能得到的好处。所以，一种让安全纯粹主义者吓得不敢直视的机制，在实际使用中却能提供相当合理的安全性[5]。记住这个教训。即使考试不考[6]，它在你以后的职业生涯中某个时候也可能派上用场。

## 54.6 基于你是什么来认证

如果你不喜欢密码这类方法，也不喜欢给用户发智能卡或安全令牌（security token），还有另一个选择。我们这里要认证的正是人类。人类是独一无二的生物，每个人都有与众不同的身体特征，有时差别细微，有时差别明显。除了人体的属性（从最底层的 DNA 到最表层的面容），人的行为也有一些特征是独一无二的，或者至少不会有很多人共有。这一观察表明，只要操作系统能准确地测量这些属性或特征，就能把一个人和另一个人区分开，从而解决认证问题。

这种方法对很多人都很有吸引力，尤其是对那些从没试过把它做出来的人。从这个基本观察出发，到做出一个能用、可靠的认证系统，远非易事。但它是可以做出来的，可靠程度和其他认证机制差不多。我们可以用它，但它不会完美，也有它自己的一系列问题和挑战。

记住，我们说的是由一个计算机程序（要么是操作系统本身，要么是操作系统为此调用的某个独立程序）来测量人的某个特征，并判断它是否属于某个特定的人。想想这意味着什么。如果我们打算用智能手机上的摄像头做人脸识别，来认证手机的主人呢？如果我们判定是对的人，就允许我们拍下的那个人使用这部手机。如果不是，就给他们（在网络意义上）喝个倒彩，把他们挡在外面。

你应该已经发现这里的几个难题了。首先，摄像头拍到的应该是正拿着手机的那个人。也许是机主，也许不是，拍照正是为了弄清这一点。如果不是，我们就应当假定，不管这人是谁，都想骗我们相信他就是真正的机主。如果这人长得很像真正的用户，但并不是呢？如果这人戴着面具呢？如果这人举着真正用户的照片，而不是露出自己的脸呢？如果光线很暗，或者这人没有正对着摄像头呢？反过来，如果确实是真正的用户，但他没有正对着摄像头，或者光线很暗，或者他的样子有了别的变化（比如发型）呢？

[5]不过，2016 年，美国国家标准与技术研究院（NIST）发布了指南草案，至少在某些情况下，不再建议把这种技术用于双因素认证。这里又有一个安全上的教训：今天管用的东西，明天未必管用。

[6]我们不知道你怎么样，反正每次一提到“测验”“小测”或者“考试”，我们的心都会漏跳一两拍。当了太多年学生，人就会变成这样。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

计算机程序识别人脸的方式和人不一样。它们和所有程序一样对待数据：把数据转换成 0 和 1，再用某种算法处理。所以，你拍的那张“照片”实际上是一组数字，表示明暗、色调、对比度之类的信息。好，然后呢？该判断这是不是对的那个人的照片了！怎么判断？

如果是密码，我们可以存储正确的密码，更好的做法是存储正确密码的散列值，然后把输入的内容（或其散列值）与存储的值比较。完全相符就认证通过，否则就不通过。我们刚拍下的照片也是一组 0 和 1，能不能也这样处理？能不能把真正用户的一张照片永久存在某个文件里（同样以 0 和 1 的形式），然后把摄像头得到的数据与这个文件比较？

大概不能像比较密码那样比较。考虑我们刚才提到的一个因素：光线。如果文件中存储的照片是在明亮的光线下拍的，而摄像头拍出来的照片是在昏暗的光线下拍的，这两组 0 和 1 肯定不会相符。事实上，即使是在完全相同的条件下、间隔一秒拍的同一个人的两张照片，也几乎不可能由完全相同的一组位来表示。所以，显然我们不能按逐位相等来比较。

我们需要在更高的层次上分析两张照片，一张是存储的真正用户的照片，一张是刚拍下的、声称是该用户的那个人的照片，再基于分析结果来比较。一般来说，这需要从照片中提取更高层的特征，然后比较这些特征。例如，我们可能会尝试计算鼻子的长度，或者确定眼睛的颜色，或者为嘴的形状建立某种模型。然后，我们比较两张照片的同一组特征。

然而，即使这样，也不太可能完全相符。例如，光线可能会让人感知到的眼睛颜色略有变化。所以，我们在比较时需要允许有些出入。如果特征“足够接近”，就认证通过；否则就不通过。我们要找的是相近的匹配，而不是完全的匹配，这就像让骆驼把鼻子伸进了帐篷：容差从此进了我们的认证机制。如果除了最接近的匹配之外一概不容，那么总有些日子，我们无法把真正用户的照片与存储的版本匹配起来。这称为**假阴性**（false negative），因为我们错误地决定不予认证。如果我们对测量数据与存储数据之间的差异过于宽容，就会让冒名顶替的人通过认证。这称为**假阳性**（false positive），因为我们错误地决定予以认证。

生物识别（biometrics）的本性决定了，任何实现都会有其特有的假阳性率（false positive rate）和假阴性率（false negative rate）。两者都不好，所以你希望两者都低。对于某种生物识别认证（biometric authentication）技术的任一具体实现，你通常可以把它调到某个假阳性率，或者调到某个假阴性率，但做不到让两者同时降到最低。假阳性率下降，假阴性率就会上升，反之亦然。**灵敏度**（sensitivity）描述的是匹配必须有多接近。

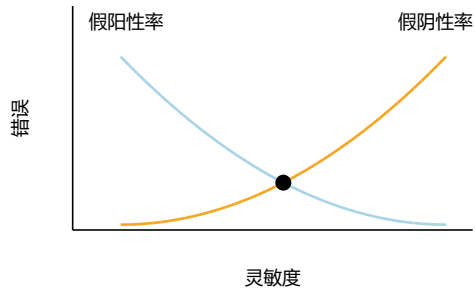



图 54.1：**交叉错误率**


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

图 54.1 展示了这两种错误率之间的典型关系。注意两条曲线交点处的那个圆点。这个点代表**交叉错误率**（crossover error rate），是描述生物识别准确度的一个常用指标。它代表两种错误之间的对等权衡。生物识别系统（biometric system）并不总是调在交叉错误率（crossover error rate）上，因为你可能更在意其中一种错误。例如，一部智能手机如果因为不喜欢今天读到的指纹而经常把合法用户锁在外面，是不会受欢迎的，而偷了这部手机的小偷恰好有相似指纹的可能性很低。在这种情况下，也许低假阴性（false negative）更重要。另一方面，如果你用视网膜扫描来打开银行金库，偶尔要求银行经理再扫描一次并不算太糟，而让劫匪用一只假眼打开金库就是一场灾难了。在这种情况下，低假阳性（false positive）可能更好。

先不说生物识别认证的可靠性（reliability）问题，用人的特征来认证还有一个大问题：很多测量这些特征的技术都需要专门的硬件，而大多数机器上不太可能有。很多计算机（包括智能手机、平板电脑和笔记本电脑）都可能有摄像头，但嵌入式设备和服务器多半没有。有指纹读取器的机器相对较少，能测量更奇特的生物特征的机器就更少了。虽然有少数几种生物识别技术（比如测量打字模式）需要的硬件相对常见，很多机器上本来就有，但这样的技术并不多。即使有专门的硬件设备，用它来做认证也未必方便。

考虑使用生物识别认证时，还有一个问题值得思考：测量生物特征量的地方和检查它的地方之间，是否存在物理上的间隔。特别是，检查不受信任的机器通过网络提供的生物识别读数是很危险的。通过网络传来的，只不过是分散在一条或多条消息中的一组位，不管它代表的是网页的一部分、VoIP 通话中的一个音素，还是扫描指纹的一部分。位就是位，任何人都能制造出任何他想要的位模式。如果远程的对手知道代表你指纹的位模式是什么样子，他们也许既不需要你的手指，也不需要指纹扫描仪，就能制造出这个位模式，并把它送进你的机器。执行扫描的硬件如果直接连在你的机器上，混入一个并非来自该设备的伪造位模式的机会就少一些；如果硬件远在世界另一端、在一台你无法控制的机器上，机会就多得多了。这里的要点是：对远程提供给你的生物识别认证信息要小心。

总的来说，听起来生物识别好像非常不适合用于认证，但这个结论是错的。说到底，前面几节可能也让所有认证方法听起来都很糟糕。它们当然都不完美，但作为系统设计者，你的任务不是找到完美的认证机制，而是使用适合你的系统及其环境的机制。一部智能手机内置一个好的指纹读取器，可能就能很好地完成任务。一个又长又猜不到的密码，可以提供相当程度的安全性。设计良好的智能卡，能让人不把它拿在手里就几乎无法通过认证。而每种机制失效的地方，你也许可以再用第二种、第三种认证机制来弥补，只要它们不在同样的情况下失效。

## 54.7 认证非人类主体

不，我们说的不是外星人或异次元生物，甚至也不是你的猫。如果你从广义上考虑今天计算机的使用方式，就会发现在很多情况下，正在运行的进程并没有关联任何人类用户。想想 Web 服务器。实际上并没有某个人类用户登录，需要把他的身份附加到 Web 服务器上。再想想嵌入式设备，比如智能灯泡。没人会登录到一个灯泡上，但灯泡里肯定运行着代码，而且很可能是面向进程的代码。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

从机制上说，操作系统处理这类进程的身份并不困难。只要在相关系统上建立一个名为 `webserver` 或 `lightbulb` 的用户，再把这个“用户”的身份附加到负责运行 Web 服务器或开关灯泡的那些进程上就行了。但这确实会引出一个问题：怎样确保只有真正的 Web 服务器进程才带有这个身份？我们可不希望 Web 服务器机器上的随便哪个用户创建出看起来属于服务器、而不属于该用户的进程。

一种办法是也给这些非人类用户设置密码。只要给 Web 服务器用户分配一个密码就行了。什么时候用它？需要的时候用，也就是你想创建一个属于 Web 服务器的进程、而当前还没有这样的进程的时候。系统管理员可以以 Web 服务器用户的身份登录，得到一个命令 shell，再用它生成服务器开展业务所需的那些进程。和往常一样，这个 shell 进程创建的进程会继承父进程的身份，在这里就是 `webserver`。更常见的做法是跳过中间环节（这里就是登录），提供某种机制，允许特权用户创建不属于自己、而属于 `webserver` 等其他用户的进程。或者，我们可以提供一种机制，允许进程更改自己的所有者，这样 Web 服务器进程可以先以其他某个用户（比如系统管理员）的身份启动，然后把所有者改成 `webserver`。还有一种办法是允许临时更改进程身份，同时记住原来的身份。（我们会在后面的章节中更多地讨论最后这种办法。）显然，这些办法中的任何一种都需要严格的控制，因为它们允许一个用户创建属于另一个用户的进程。

如上所述，确定进程能否分配给这类非人类用户时，最常用的认证方法是密码。不过，有时根本不需要认证非人类用户，而是赋予某些其他用户（比如受信任的系统管理员）一项权利：可以给自己创建的进程分配新身份，除了他们自己的认证信息，不需要再提供任何认证信息。Linux 和其他 UNIX 系统中的 `sudo` 命令就提供这种能力。例如输入以下命令：

```
sudo -u webserver apache2
```

补充：其他认证方式

通常，你知道什么、你拥有什么和你是什么，已经涵盖了有用的认证方式，但有时也有其他选择。想象你去车管所申请驾照。你大概会走到一个柜台前，和柜台后面的某个工作人员交谈，也许会给这个人一大堆个人信息，甚至付钱交驾照的费用。你到底凭什么相信这个人真的是车管所的员工，能给你办一张合法的驾照呢？你大概并不认识这个人；没人给你看官方证件；这个人也没有念出什么车管所的秘密口诀，来证明自己是内部人员。你相信这个人，是因为这个人站在某个特定的柜台后面，而那正是车管所员工所站的柜台。你是根据位置来认证这个人的。

偶尔，这种办法在计算机系统中也很有用，最常见于移动计算或普适计算。如果你想用它，就要仔细想想怎样获得证据，证明对方真的在某个特定地点。这其实相当棘手。

还有什么？也许有时你可以根据某人做了什么来认证。如果你关注的是个人特有的行为，比如打字模式或命令之间的延迟，那就是生物识别的一种。例如，Google 就在它的 Android 手机中引入了这类多因素认证。但你可能不那么在意认证他们究竟是谁，而更在意认证他们是否属于“行为良好的用户”这个集合。例如，很多网站不太关心访客是谁，而更关心他们是否正当地使用网站。在这种情况下，你可以根据他们与系统持续的交互，来认证他们属于这个集合。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

这表示 `apache2` 程序应当以 `webserver` 的身份启动，而不是以运行 `sudo` 的那个人的身份启动。这条命令可能会要求运行它的用户提供自己的认证凭证（authentication credential），以便确定确实是特权用户在发出请求，而不是趁特权用户喝咖啡时动用计算机的什么路人；但不会要求提供与 `webserver` 关联的认证信息。当然，`apache2` 创建的任何子进程都会继承 `webserver` 的身份。我们会在访问控制（access control）一章中进一步讨论 `sudo`。

我们前面提到过的最后一个身份问题是：有时我们希望识别的不只是单个用户，而是具有共同特征（通常是与安全相关的特征）的用户组。例如，我们可能有四五个系统管理员，其中任何一个都可以启动 Web 服务器。与其把这项特权分别关联到每个人身上，不如创建一个在系统上有意义、具有这项特权的用户组，然后指明这四五个管理员是这个组的成员。这种组是安全相关主体的又一个例子，因为我们将根据组成员身份、而不是个人身份来做决定。某个系统管理员想做需要组成员身份的事情时，我们就检查他或她是不是该组成员。我们可以给每个进程关联一个组成员身份，也可以用进程的个人身份信息作为索引，去查一份记录人们所属组的列表。后一种方式更灵活，因为它允许每个用户属于任意数量的组。

大多数现代操作系统，包括 Linux 和 Windows，都支持这类组，因为有了组，应用安全策略就简单灵活多了。它们处理组成员身份和组特权的方式，与处理个人身份和个人特权的方式大体类似。例如，子进程通常与父进程具有相同的组相关特权。使用这类系统时要记住，组成员身份为用户提供了获取资源访问权的第二条途径，这既有好处，也有危险。

## 54.8 小结

如果我们想对系统中进程执行的操作应用安全策略，就需要知道进程的身份，才能做出正确的决定。整条进程链的起点，是在启动时创建一个属于某个系统用户的进程，它的用途就是认证用户。用户登录时，以一种或多种形式提供认证信息，证明自己的身份。系统用这些信息验证用户的身份，并把这个身份分配给一个新进程，让用户能够开展工作，这通常涉及运行其他进程。这些进程会从父进程那里继承用户的身份。特殊的安全机制可以允许更改进程的身份，或者把进程的身份设为与父进程不同的身份。这样，系统就能确信进程属于正确的用户，并据此做出安全决定。

无论从历史上还是从实践上看，提供给系统的认证信息，要么是接受认证的用户知道的东西（比如密码或 PIN），要么是用户拥有的东西（比如智能卡，或者证明自己拿着某部智能手机），要么是用户本身是什么（比如用户的指纹或声纹扫描）。每种方式各有优缺点。使用多因素认证可以获得更高程度的安全性，它要求用户提供不止一种形式的证据，比如既要提供密码，又要提供一个发送到用户智能手机上的一次性验证码。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

## 参考文献

[B+09] "The road from Panama to Keccak via RadioGatun" by Guido Bertoni, Joan Daemen, Michael Peeters, Gilles Van Assche. *The authors who developed SHA-3. For a more readable version, try the Wikipedia page first about SHA-3. There, you learn about the "sponge construction", which actually has something to do with cryptographic hashes, and not the cleaning of your kitchen.*

[C100] "Letter of recommendation to Tiberius Claudius Hermeros" by Celer the Architect. Circa 100 A.D.. *This letter introduced a slave to the imperial procurator, thus providing said procurator evidence that the slave was who he claimed to be. Read the translation at the following website* http://papyri.info/ddbdp/c.ep.lat;;81.

[G13] "Anatomy of a hack: even your 'complicated' password is easy to crack" by Dan Goodin. http://www.wired.co.uk/article/password-cracking, May 2013. *A description of how three experts used dictionary attacks to guess a large number of real passwords, with 90% success.*

[JB-500] "Judges 12, verses 5-6" The Bible, roughly 5th century BC. *An early example of the use of biometrics. Failing this authentication had severe consequences, as the Gileadites slew mispronouncers, some 42,000 of them according to the book of Judges.*

[KA16] *VK.com Hacked! 100 Million Clear Text Passwords Leaked Online* by Swati Khandelwal. http://thehackernews.com/2016/06/vk-com-data-breach.html. *One of many reports of stolen passwords stored in plain-text form.*

[MT79] "Password Security: A Case History" by Robert Morris and Ken Thompson. Communications of the ACM, Vol. 22, No. 11, 1979. *A description of the use of passwords in early Unix systems. It also talks about password shortcomings from more than a decade earlier, in the CTSS system. And it was the first paper to discuss the technique of password salting.*

[M+02] "Impact of Artificial "Gummy" Fingers on Fingerprint Systems" by Tsutomu Matsumoto, Hiroyuki Matsumoto, Koji Yamada, and Satoshi Hoshino. SPIE Vol. #4677, January 2002. *A neat example of how simple ingenuity can reveal the security weaknesses of systems. In this case, the researchers showed how easy it was to fool commercial fingerprint reading machines.*

[P-46] "The Histories" by Polybius. Circa 146 B.C.. *A history of the Roman Republic up to 146 B.C. Polybius provides a reasonable amount of detail not only about how the Roman Army used watchwords to authenticate themselves, but how they distributed them where they needed to be, which is still a critical element of using passwords.*

[TR78] "On the Extraordinary: An Attempt at Clarification" by Marcello Truzzi. Zetetic Scholar, Vol. 1, No. 1, p. 11, 1978. *Truzzi was a scholar who investigated various pseudoscience and paranormal claims. He is unusual in that he insisted that one must actually investigate such claims before dismissing them, not merely assume they are false because they conflict with scientific orthodoxy.*


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

# 55

# 访问控制

本章作者：**Peter Reiher（UCLA）**

## 55.1 引言

现在，我们知道了自己的安全目标是什么，对想要实施的安全策略（security policy）至少有了大致的认识，也掌握了一些证据，知道是谁在请求系统服务，而这些请求可能违反（也可能不违反）这些策略。接下来，我们需要把这些信息变成可以执行的东西，变成一段软件能替我们完成的工作。

这里有两个重要的步骤：

1. 判断这个请求是否符合我们的安全策略。
2. 如果符合，就执行这个操作；如果不符合，就确保它不会被执行。

第一步通常称为**访问控制**（access control）。我们要确定：哪些系统资源或服务，可以由哪些参与方，以哪些方式，在哪些情况下访问。归根结底，这又是一个非常契合计算机思维的二元决定：是或否。可是，这个决定该怎么做呢？为了让问题更具体，来看这样一个例子：用户 X 想读写文件 /var/foo。在底层，这多半意味着一个以用户 X 的身份运行的进程（process）发起了类似下面这样的系统调用（system call）：

```
open("/var/foo", O_RDWR)
```

注意，这里说的不是 Linux 的 `open()` 调用。Linux 的 `open()` 是一个具体实现，它用一种特定的方式处理访问控制。我们讨论的是一般意义上的问题：怎样控制对“打开文件”这类系统调用的访问。所以这里换了一种字体，提醒你注意这一点。

系统该怎样处理进程的这个请求，既要确保在安全策略禁止时不打开文件，又要确保在策略允许时打开文件？我们知道，系统调用会陷入（trap）操作系统，这让操作系统有机会做点什么来作出这个决定。从机制上讲，这个“什么”应该是什么呢？

关键问题：

如何判断是否应当批准一个访问请求

操作系统怎样决定，某个特定用户的某个特定进程在某个特定时刻发出的某个特定请求，是否应当批准？做这个决定要用到哪些信息？我们怎样设置这些信息，才能把想要在系统中实施的安全策略编码进去？

## 55.2 访问控制问题的几个重要方面

和往常一样，系统会运行某种算法来做这个决定。算法接受一些输入，产生一个二元输出，也就是“是否批准访问”的决定。在宏观层面，人们通常用**主体**（subject）、**客体**（object）和**访问**（access）来讨论访问控制。主体是想要执行访问的实体，可能是一个用户，也可能是一个进程。客体是主体想要访问的东西，可能是一个文件，也可能是一个设备。访问是对客体的某种特定操作方式，比如读或者写。所以，访问控制决定讨论的就是：某个特定的主体，是否获准以某种特定的访问方式访问某个特定的客体。判断某个特定的主体是否获准对某个特定的客体执行某种特定形式的访问，这个过程有时也称为**授权**（authorization）[1]。

一个相关的问题是：访问控制决定在什么时候做？系统每做一次这样的决定，都要把它所用的算法运行一遍。实现这个算法的代码称为 **reference monitor**。显然，我们有充分的理由确保它的实现既正确又高效。如果它不正确，就会作出错误的访问决定，这显然很糟糕。它的效率之所以重要，是因为每次用到它都会带来一些开销。为了尽量减少这些开销，我们也许不想一有机会就检查访问控制。可另一方面，还记得前面几章介绍过的完全仲裁（complete mediation）原则吗？那条原则说，每当有人请求某样东西，我们都应当检查安全条件。

显然，我们需要在代价和安全收益之间取得平衡。不过，如果能找到一些有益的特殊情况，既能做到低开销，又不牺牲安全，那么至少在这些情况下，我们也许就不必在两者之间取舍了。

一种做法是让主体拥有只属于它们自己的客体。如果某个客体天然只属于某个主体，而且这一点不可改变，系统就可以让这个主体（在操作系统中就是进程）自由地访问它。虚拟化（virtualization）让我们能创建这类虚拟客体（virtual object）。虚拟内存（virtual memory）就是一个绝佳的例子。进程可以自由地访问自己的虚拟内存[2]，在进程使用它的那一刻，操作系统不需要做专门的访问控制检查。这也是好事，否则每次进程引用内存时，我们都得运行一遍访问控制算法，系统会慢得离谱。对外围设备，我们也可以玩类似的虚拟化把戏。假设给一个进程分配了一个虚拟设备，它背后其实是操作系统控制的一个真实物理设备，而且其他进程都不许使用它，那么这个进程每次要用这个设备时，操作系统都不必检查访问控制。例如，可以根据一次初始的访问控制决定，把 GPU 的控制权交给某个进程；此后，这个进程就可以直接写 GPU 的内存、直接向它发指令，操作系统不再介入。

当然，前面讨论过，虚拟化在很大程度上是操作系统提供的一种假象。进程之间其实共享着内存、设备和其他计算资源。看起来只属于它们自己的东西，实际上是共享的，只是操作系统在幕后忙前忙后地维持着这个假象，有时还借助专门的硬件。这意味着，即使使用虚拟化资源的应用程序并不直接知情、也不直接参与，操作系统仍然必须确保只允许正当的访问方式。所以，单纯依靠虚拟化来保证访问正当，只是把问题推到了下一层：变成了保护操作系统的虚拟化功能。即使先把这个问题放在一边，我们迟早也得撇开这些开销低的特殊情

[1] 哇。你知道要让这么多个“particular”（特定的）在原文里上下排成一列有多难吗？这可是一整列 particular！不过，也许并不怎么（particularly）有趣。

[2] 几乎如此。还记得页表（page table）里那些决定某一页能否读、写或执行的位吗？不过，这里做运行时检查的不是操作系统，而是虚拟内存硬件。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

况，去处理一般性的问题。主体 X 想读写客体 /tmp/foo。也许允许，也许不允许。现在怎么办?

针对这个问题，计算机科学家提出了两种基本方法，它们依赖不同的数据结构（data structure），用不同的方式作出决定。一种叫**访问控制列表**（access control list），另一种叫**能力**（capability）。其实，说是计算机科学家提出了这两种方法并不太准确，因为在计算机之外的场合，人们使用它们已经有几千年了。在讨论它们在操作系统中的实现之前，我们先从更一般的角度看看它们。

假设我们要开一家高级私人俱乐部（不妨叫它 Chez Andrea[3]），只接待最优秀的操作系统研究者和开发者。我们可不想让那些搞数据库或编程语言的人混进来，所以必须确保只有经过批准的客人才能进门。该怎么做呢? 一种办法是雇一个块头巨大、一脸凶相的门卫，他手里有一份所有获准会员的名单。有人想进俱乐部时，要先向门卫证明自己的身份，门卫再查这个人在不在名单上。如果来的是 Linus Torvalds 或 Barbara Liskov，门卫就放行；那些搞网络、在操作系统领域没混出名堂的平头百姓，就会被挡在门外。

另一种办法是在俱乐部门上装一把非常好的锁，再把这把锁的钥匙发给我们所有搞操作系统的朋友。如果 Jerome Saltzer 想进 Chez Andrea，他只要掏出钥匙开门就行了。要是有几个不懂操作系统的计算机体系结构专家想进来，他们没有钥匙，就只能待在外面。和前一种办法相比，我们省下了门卫的工资，不过得花钱买锁和钥匙[4]。等到操作系统领域又涌现出我们想接纳的新大牛，就得给他们配新钥匙。我们偶尔还可能犯错，把钥匙发给了不配拥有它的人；或者某个会员把钥匙弄丢了，这时就得确保那把钥匙再也打不开俱乐部的门。

同样的思路也可以用在计算机系统中。早期的计算机科学家把类似锁和钥匙的那种方法叫作**基于能力的系统**（capability-based system），而把基于门卫和准入名单的方法叫作**访问控制列表系统**（access control list system）。所以，能力就像钥匙，或者电影票，或者坐地铁用的代币；访问控制列表嘛，就像列表。那么，这在操作系统中是怎么运作的呢? 如果用的是能力，属于用户 X 的某个进程想读写文件 /tmp/foo 时，会把针对这个文件的一个能力交给系统。（你可能会问，在这个语境下，能力到底是什么？问得好！我们后面会讲到。）如果用的是访问控制列表（简称 **ACL**），系统会在与 /tmp/foo 关联的 ACL 上查找用户 X，只有用户在名单上时才允许访问。无论哪种方式，检查都可以在请求访问（在我们的例子里就是调用 `open()`）的那一刻进行。检查发生在陷入操作系统之后、真正允许访问之前；如果访问控制检查没有通过，就提前退出并返回错误码。

从宏观上看，这两种做法听起来也许没有太大差别；但只要开始考虑需要运行的算法，以及支撑这个算法所需的数据结构，你很快就会发现它们差别很大。下面逐一来看。

[3]作者 Arpaci-Dusseau 夫妇想声明一下：安全部分各章里的这些名字都是作者 Reiher 定的，他们并没有逼他在本章和其他例子里一直用他们的名字。下面请继续正常阅读……

[4]注意，无论是访问控制列表还是能力，我们都假定已经对想进俱乐部的人做过身份认证（authentication）。如果有个无名小卒戴着 Linus Torvalds 或 Barbara Liskov 的面具骗过了门卫，或者我们没有仔细确认对方真的是 Jerome Saltzer 就把钥匙交给了随便一个人，那就挡不住那些乌合之众了。抛开这个可爱的比喻，同样的问题在真实的计算机系统中也完全存在，这正是上一章详细讨论身份认证的原因。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

## 55.3 用 ACL 实现访问控制

按照英国老式俱乐部的传统，如果 Chez Andrea 除了让会员使用图书室、餐厅、台球室和其他公共空间之外，还给每位会员一间私人房间，会怎么样？这时，我们不仅要确保只有会员才能进俱乐部，还要确保 Ken Thompson（大家都知道他有点调皮 [T84]）不能溜进 Whitfield Diffie 的房间，在他床上搞恶作剧。我们可以用一张大的访问控制列表，规定每个房间允许谁怎样进入，但那样会变得难以管理。何不给俱乐部的每个房间各配一张 ACL 呢？

在依靠 ACL 做访问控制的典型操作系统中，我们对文件也是这样处理的。每个文件都有自己的访问控制列表，这样列表更简单、更短，访问控制检查也更快。所以，在 ACL 系统中，我们的 `open()` 调用会检查 /tmp/foo 的列表，而不是去查一张记下了全系统每个文件访问权限的总 ACL。

这个 `open()` 调用陷入操作系统时，操作系统会查看当前运行进程的 PCB，确定这个进程的属主（owner）是谁。这个数据结构表明，进程属于用户 X。接下来，系统必须拿到 /tmp/foo 的访问控制列表。ACL 也是一种文件元数据（metadata），类似于我们在“文件和目录”一章中讨论过的那些东西。所以，它很可能和这个文件的其他元数据存放在一起，或者存放在它们附近。总之，我们要从持久存储（persistent storage）中取出这张列表，然后在列表中查找 X。X 要么在，要么不在。如果不在，X 就没有访问权。如果在，我们通常还要更进一步，判断 X 的 ACL 表项是否允许所请求的访问类型。在我们的例子中，X 想以读写方式打开 /tmp/foo。也许 ACL 允许 X 以读方式打开这个文件，但不允许以写方式打开。这时，系统就会拒绝这次访问，并给进程返回一个错误。

原理上，这并不复杂。可还记得“魔鬼藏在细节里”这句话吗？魔鬼还在。我们来考虑其中一些细节。例如，ACL 到底持久地存放在哪里？对大多数资源来说，它确实需要持久保存，因为 ACL 实际上编码了我们选定的安全策略，而这种策略多半不会经常变动。所以它存放在闪存盘（flash drive）或磁盘上的某个地方。除非做了缓存（cached），否则每次有人试图打开文件，我们都得从设备上把它读出来。正如持久性（persistence）部分讨论过的，在大多数文件系统（file system）中，要从文件里真正读到任何信息，本来就得读好几次设备。难道还要再读一次，才能拿到这个文件的 ACL？如果是这样，ACL 应该放在设备上的什么位置，才能保证访问起来快？最好它靠近我们本来就要读的某样东西，甚至就是那样东西的一部分。这样看，有几个可能的位置：文件的目录项（directory entry）、文件的 inode，或者文件的第一个数据块（data block）。至少，我们希望 ACL 靠近其中某个位置；如果能直接放在其中某一处（比如 inode）里，可能更好。

这又引出另一个恼人的细节：这张列表有多大？如果按最直观的办法，列出实际的用户 ID 和访问方式，那么原则上这张列表可以任意大，最多可以达到系统所知的用户数。对某些系统来说，这可能是几千个表项。但通常，一个文件只属于一个用户，往往也只有这个用户，或许再加上几个朋友，才能使用它。所以，我们不会想在每张 ACL 里都预留足够的空间，把所有可能的用户都列进去，因为大多数用户不会出现在大多数 ACL 中。当然也有例外：很多文件应当允许所有用户以某种方式（比如读或执行）使用。毕竟，常用的可执行程序（比如 `ls` 和 `mv`）都存放在文件中，我们也会像对待其他文件一样，对它们实施访问控制。用户们会共用同样的字体文件、网络配置文件，等等。我们必须允许所有用户访问这些文件，否则他们在系统上几乎什么也做不了。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

所以，最直观的实现会给每个文件预留一张大列表，有些文件的列表被完全填满，另一些则几乎是空的。这显然很浪费。对于填满的列表，还有一个令人担忧的细节：每次想在列表中检查访问权限，都得搜索它。现代计算机搜索一张有一千个表项的列表相当快，但如果我们一直要做这样的搜索，就会给系统增加大量不必要的开销。我们可以用变长的访问控制列表来解决这个问题，只为每张列表分配它实际需要的空间。花点时间想一想，你会怎样把这类元数据塞进我们学过的那几种文件系统，又会对性能有什么影响。

所幸，在大多数情况下，我们可以受益于最初的贝尔实验室 Unix 系统留下来的一点遗产。在那个远古时代，计算机科学的巨人们还在地球上（至少是在新泽西州的某些地方）漫游，持久存储稀缺而昂贵，他们根本负担不起为每个文件存一张大 ACL。实际上，他们算了一下，发现每个文件的 ACL 大约只负担得起九位。九位实在不多，好在那些早期的 Unix 设计者聪明绝顶，足以弥补硬件的不足。他们琢磨了自己的问题，发现真正关心的访问方式实际上只有三种（对大多数文件来说是读、写和执行），而且每张访问控制列表只要三个表项，就能应付大多数安全策略。当然，如果每个表项的每种访问方式都用一位，九位就已经用光了，再没有位能用来指明这个表项是针对谁的。于是，他们巧妙地把访问控制列表上的表项分成三组。一组是文件的属主，属主的身份本来就已经存在 inode 里了。一组是某个特定用户组的成员，组 ID 也存在 inode 里。最后一组是其他所有人，也就是既不是属主、也不是属主所在组成员的所有人。这一组不需要用任何位来存储，因为它就是属主和组的补集。

这个方案不仅解决了 ACL 占用存储空间的问题，还解决了访问和检查 ACL 的开销问题。不管对文件做什么，你几乎都要先访问它的 inode；所以，如果 ACL 嵌在 inode 里，就不需要额外的寻道（seek）和读取来获取它。而且，不必在任意长度的列表里搜索，只要对几个位做一点简单的逻辑运算，就能回答访问控制的问题。直到今天，凡是采用 Posix 兼容文件系统的系统，大多仍靠这套逻辑来回答这个问题。当然，这种方法也有局限，因为它无法表达复杂的访问方式和共享关系。正因为如此，一些现代系统（比如 Windows）提供了扩展，允许使用更通用的 ACL；但很多系统依然依赖久经考验的 Unix 风格九位 ACL[5]。

ACL 有一些好的特点，也有一些局限。先说优点。第一，如果你想弄清楚谁可以访问某个资源，用 ACL 时这个问题很好回答，直接看 ACL 本身就行。第二，如果你想改变能访问某个客体的主体集合，只需修改 ACL，因为再没有别的东西能给用户访问权。第三，由于 ACL 通常和文件本身放在一起，或者放在它附近，只要你能拿到文件，就能拿到所有相关的访问控制信息。这一点在分布式系统（distributed system）中尤其重要；不过只要设计时让 ACL 靠近文件或它的 inode，它对所有系统的性能也都有好处。

再说不太理想的地方。第一，ACL 要求你解决前面提到的那些问题：要把访问控制信息存放在文件附近的某个地方，还要应付在长列表中搜索可能带来的高开销。我们介绍了一些在大多数系统中效果不错的实用方案，但这些方案也限制了 ACL 能做的事。第二，如果你想弄清楚某个主体获准访问的全部资源，该怎么办？你得把系统中的每一张 ACL 都查一遍，因为这个主体可能出现在任何一张 ACL 上。第三，在分布式环境中，要让 ACL 生效，所有机器对身份必须有共同的认识。如果 cs.ucla.edu 上的某个用户想访问存放在

[5]真实的历史要更复杂一些。CTSS 系统提供了一种比 Unix 更受限的压缩 ACL [C+63]，而 Multics 系统在一种更通用的访问控制列表中引入了组的概念，这种列表由用户名和组名的字符串组成 [S74]。所以，Unix 的方法是这些更早系统的杂交产物。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：命名空间

我们刚刚遇到了分布式系统中一个有趣而又困难的问题：名字在不同的机器上意味着什么？在单台计算机上，这个**命名空间**（name space）问题相对容易。如果为一个新东西选的名字已经被占用了，就不允许分配它。所以，在这个系统上，无论是哪个用户或进程使用某个名字，它指的都是同一样东西。对你和你计算机上的所有其他用户来说，/etc/password 都是同一个文件。

可是，由多台计算机组成的分布式系统呢？如果你想同样保证名字唯一、而且所有人都理解一致，就得确保 UCLA 某台机器上的人不会创建一个威斯康星大学已经在用的名字。怎么做到呢？不同的答案各有利弊。一种办法是不去管它，接受各个命名空间互不相同这一事实，我们对进程 ID（process ID）就是这么做的。另一种办法是要求由一个权威机构来审批名字的选用，AFS 处理文件名的创建大致就是这样。还有一种办法是把命名空间分成若干部分，交给各个参与者，允许它们在自己那部分里随意取名，但不能取其他名字，万维网和 IPv4 地址空间（address space）就是这样处理这个问题的。这些答案没有哪个是绝对正确或绝对错误的。根据自己的需要设计命名空间，但要明白其中的含义。

cs.wisconsin.edu 上的文件，威斯康星的机器就要拿 UCLA 提供的某个身份，去和存放在威斯康星的访问控制列表比对。UCLA 的用户 remzi 和威斯康星的用户 remzi 真的是同一个主体吗？如果不是，你就可能让一个远程用户访问了他不该访问的东西。但在多个不同的计算域之间维护一致的用户命名空间（name space），很不容易。

## 55.4 用能力实现访问控制

在计算机系统中，访问控制列表并不是控制访问的唯一选择。几乎是，但不完全是。你还可以使用能力，也就是更像钥匙或门票的那种做法。Chez Andrea 可以给会员发钥匙，让他们凭钥匙入场。不同的房间可以配不同的钥匙，防止那些比较调皮的会员在别人房间里留下点小“惊喜”。每位会员都随身带着一串钥匙，凭这些钥匙进入俱乐部中自己有权进入的区域。和 ACL 一样，能力在计算机系统中也有很长的使用历史，Dennis 和 van Horn [DV64] 也许是最早的例子。Wulf 等人 [W+74] 介绍了 Hydra 操作系统，它把能力用作一种基本的控制机制。Levy [L84] 用一整本书总结了能力在早期硬件和软件系统中的应用。在能力系统中，运行着的进程拥有一组能力，这些能力规定了它的访问权限。如果你用的是纯粹的能力系统，任何地方都没有 ACL，这组能力就是这个进程访问权限的全部编码。Linux 和 Windows 不是这样工作的，但其他一些操作系统（比如 Hydra）研究过用这种方法处理访问控制。

在这种纯粹的能力系统中，我们该怎样执行那个 `open()` 调用呢？发起调用时，要么由你的应用程序把能打开该文件的那个能力作为参数传进去，要么由操作系统替你找到它。无论哪种情况，操作系统都会检查这个能力是否允许你以读写方式打开文件 /tmp/foo。如果允许，操作系统就替你打开它。如果不允许，你的进程就会收到一个错误，数落它竟想打


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

开一个自己没有相应能力的文件。(记住，我们这里说的不是 Linux。Linux 是用 ACL 而不是能力来判断是否允许一次 `open()` 调用的。)

这里有几个显而易见的问题。能力究竟是什么？显然我们说的不是金属钥匙或纸质门票。还有，操作系统怎样检查能力是否有效？能力最初又是从哪里来的？和计算机中的其他所有信息一样，能力就是一串位，是数据。考虑到要保护的资源可能很多，而能力必须针对某个具体资源，能力很可能相当长，也许还相当复杂。但归根结底，它们只是位。任何由一串位组成的东西，都有一些我们必须牢记的特性。例如，任何人都可以随意造出任何一串位，没有哪种位模式是专属的、或者保留下来不让进程创建的。另外，一个进程只要手里有一串位，再复制出更多份是轻而易举的事。第一个特性意味着，任何人都有可能造出自己想要的任何能力。第二个特性意味着，一旦某人有了一个有效的能力，就可以随意复制任意多份，还可能把它们存放在任何地方，包括另一台完全不同的机器上。

从安全的角度看，这听起来不太妙。如果一个进程需要一个具有特定位模式的能力，才能以读写方式打开 /tmp/foo，那它也许只要自己生成这个位模式，就能给自己弄到想要的文件访问权。这可不是我们对访问控制机制（access control mechanism）的期望。我们希望能力是不可伪造的。即使能解决这个问题，能力可以复制这一点也意味着，访问权一旦授出，我们可能就收不回来了，因为这个进程可能在别处藏了这个能力的副本[6]。再者，这个进程也许只要用 IPC 把能力的副本传给另一个进程，就能把访问权授予那个进程。

用能力做访问控制时，我们通常这样处理这些问题：绝不让进程把（比喻意义上的）手伸到任何能力上。能力由操作系统控制和维护，存放在操作系统受保护的内存空间中的某个地方。进程可以对能力执行各种操作，但只能在操作系统的仲裁下进行。例如，如果进程 A 想用能力授予进程 B 对文件 /tmp/foo 的读写权，A 不能直接把相应的位模式发给 B。A 必须发起一个系统调用，请求操作系统把相应的能力交给 B。这样，操作系统就有机会判断它的安全策略是否允许 B 访问 /tmp/foo；如果不允许，就拒绝这次能力转移。

所以，如果要依靠能力做访问控制，操作系统就得为每个进程维护一张受它保护的能力列表。这很简单，因为操作系统本来就有一个每进程一份、受保护的数据结构：PCB。在进程的 PCB 里放一个指向能力列表（存放在内核内存中）的指针，就万事俱备了。现在，进程试图以读写方式打开 /tmp/foo 时，调用会陷入操作系统，操作系统查看这个进程的能力列表，看上面有没有针对这个操作的相关能力，然后照此处理。

在通用系统中，要在线保存一张列表，把某个主体获准访问的所有东西都列进去，开销会很高。如果我们用能力来做基于文件的访问控制，一个用户可能就有成千上万个能力，他能以任何方式访问的每个文件都对应一个。一般来说，使用能力的系统会把能力持久地存放在某个安全的地方，需要时再导入。所以，附在进程上的能力列表不一定很长；但有一个问题要决定：用户手头可以自由支配的能力数量庞大，该把其中哪些交给他运行的每个进程？

还有另一种选择。能力不一定要存放在操作系统中，而是可以用密码学（cryptography）来保护。如果能力比较长，而且是用强密码学生成的，那它实际上就猜不出来，可以留在用户手里。密码学保护的能力最适合用在分布式系统中，所以我们会在分布式系统安全一章中讨论它们。

[6]收回访问权的这种做法通常称为**撤销**（revocation）。对 ACL 来说，撤销很容易，只要找到 ACL 改掉就行。对能力来说，撤销是难是易，取决于具体的实现。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

和访问控制列表一样，能力也有优点和缺点。用能力时，很容易确定某个主体能访问哪些系统资源，只要把这个主体的能力列表看一遍就行了。撤销访问权只需把能力从列表中删掉；如果操作系统独占这个能力，这很容易做到（如果不是独占，就难多了）。如果能力就在内存里随手可用，检查它的开销可以非常低，尤其是能力本身就可以包含一个指针，指向它所保护的资源对应的数据或软件。也许，系统对能力的核心实现，就只是持有这样一个指针而已。

另一方面，要确定能访问某个资源的全部主体，开销就大了。任何主体都可能持有这个资源的能力，所以你必须检查所有主体的能力列表才能知道。而且，能让能力列表保持简短、易于管理的简单方法，也不像 Unix 那种简短 ACL 的方法那样成熟。此外，系统创建、存储和取出能力的方式必须能杜绝伪造，这可能不容易做到。

能力有一个很妙的地方：用它可以很方便地创建特权受限的进程。用访问控制列表时，进程会继承父进程（parent process）的身份，因而也继承了那个主体的全部特权。很难只把父进程特权的一个子集交给这个进程。要么得创建一个只有这些受限特权的新主体，修改一堆访问控制列表，再把新进程的身份设成这个新主体；要么得对访问控制模型做某种扩展，而这种扩展的行为和访问控制列表通常的行为不太一样。用能力就很容易。如果父进程有 X、Y、Z 三个能力，但只想让子进程（child process）拥有 X 和 Y，那么创建子进程时，父进程就只转给它 X 和 Y，不转 Z。

实际上，出于种种原因，用户能看到的访问控制机制往往使用访问控制列表，而不是能力。不过在底层，操作系统大量使用能力。例如，在一个典型的 Linux 系统中，我们一直在讨论的那个 `open()` 调用是用 ACL 做访问控制的。但假设 Linux 的 `open()` 成功了，那么只要进程一直打开着这个文件，后续的读和写就不会再检查 ACL。Linux 会创建一个数据结构，它实际上就相当于一个能力，表明这个进程对该文件有读写特权。这个结构附在进程的 PCB 上。每次读写时，操作系统只需查看这个数据结构，就能判断是否允许读写，而不必去找文件的访问控制列表。文件一旦关闭，这个类似能力的结构就会从 PCB 中删除，进程要再访问这个文件，就得重新调用一次 `open()`，而那又要回到 ACL 去检查。类似的技术也可以用来控制对硬件设备和 IPC 通道的访问，尤其是因为类 UNIX 系统把这些资源都当作文件来对待。ACL 和能力结合使用，让系统避开了各自机制的一些问题。每次操作都检查访问控制列表的开销省下来了，因为这种形式的能力很容易检查，它不过是操作系统数据结构中某个指针在或不在而已。为所有可访问客体管理能力的开销也避免了，因为只有在 ACL 检查成功之后才会建立能力。如果进程从不访问某个客体，就永远不会检查它的 ACL，也不需要为它建立能力。由于任何一个进程通常只会打开它获准打开的全部文件中极小的一部分，可扩展性（scalability）问题一般不会出现。

## 55.5 强制访问控制与自主访问控制

计算机资源的访问控制应该是什么样，该由谁来决定？对大多数人来说，答案似乎显而易见：谁拥有这个资源，就由谁决定。对用户的文件来说，应当由这个用户决定访问控制设置。对系统资源来说，应当由系统管理员，或者也许由这台计算机的主人来决定。然而，对某些系统和某些安全策略来说，这个答案并不对。特别是，最关心信息安全的那些人，有时希望控制得比这更严。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

军方是最明显的例子。我们都听说过绝密（Top Secret）信息，大概也都明白，即使你获准查看绝密信息，也不应该让别人看到它。即使这些信息就在你自己创建的文件里，也是如此，比如你写的一份报告引用了另一份绝密文件里的统计数据或原文。在这些情况下，由创建者控制访问权限这个简单的答案就不对了。必须由机构中全面负责信息安全的人来作这些决定。这意味着，这个主体有权给其他用户创建和拥有的信息设置访问控制，而且这些用户推翻不了他的决定。比较常见的情况称为**自主访问控制**（discretionary access control）：资源开放给几乎所有人，还是几乎不让任何人访问，由拥有它的用户自行决定。限制更严的情况称为**强制访问控制**（mandatory access control）：在这类系统中，访问控制决定中至少有一部分是由某个权威机构强制规定的，它可以推翻信息所有者的意愿。选择自主访问控制还是强制访问控制，与使用 ACL 还是能力是正交的，而且通常也独立于访问控制机制的其他方面，比如访问信息如何存储和处理。强制访问控制系统也可以包含一些自主的成分，允许对强制控制作进一步限制（但不能放松）。

很多人永远不会用到运行强制访问控制的系统，所以我们不打算深入讨论它们如何工作，只指出一点：显然，实施这类控制离不开操作系统的参与。如果你哪天需要在一个看重强制访问控制的环境中工作，你一定会听说它的。到那时你应当再多了解一些，因为肯为强制访问控制机制费心的人，也同样会较真地惩罚不守规矩的用户。Loscocco [L01] 介绍了一个加入强制访问控制的特制版 Linux。如果你想进一步了解这类系统的特点，这是一篇很好的入门论文。

## 55.6 访问控制机制的实际问题

大多数系统都向用户提供一种访问控制列表机制，或简单，或功能更强；而且其中大多数使用自主访问控制。然而，一台现代计算机很容易就有几十万甚至几百万个文件，让人类用户逐个设置访问控制权限是不可行的。一般来说，系统允许每个用户设定一个默认访问权限，用于他创建的每个文件。如果用 Linux 的 `open()` 调用创建文件，可以指定最初赋给这个文件的访问权限。在 Unix/Linux 系统中，新建文件的访问权限还可以用 **`umask()`** 调用进一步控制，它作用于调用它的那个进程此后创建的所有新文件。

如果愿意，所有者可以修改这个初始 ACL，但经验表明，用户很少这么做。这种倾向说明了选好默认值有多重要。和操作系统中的许多其他地方一样，这里的设置理论上可以修改或调整，实际上却几乎所有人几乎总是原封不动地用着。

不过，虽然很多人永远不会碰自己资源上的访问控制，但对一批重要的用户和系统来说，这些控制对实现他们的安全目标至关重要。即使你大多依赖默认值，许多软件安装包在给它们创建的可执行文件和配置文件设置访问控制时，也会相当谨慎。一般来说，摆弄系统中的访问控制时应当小心。如果你不知道自己在做什么，就可能泄露敏感信息，或者让攻击者能篡改关键的系统设置。如果你收紧了现有的访问控制，可能会突然让后台运行的一堆守护进程（daemon）停止工作。

许多大型机构在试图用标准的访问控制方法实施安全策略时，发现了一个实际问题：在机构中担任不同角色的人需要不同的特权。例如，在一家医院里，所有医生可能都有一组药剂师没有的特权，而药剂师又有一些医生没有的特权。按照这样的角色来组织访问控制，再把具体用户分配到他们获准担任的角色上，许多安全策略实现起来就会容易一些。如果某些


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：Android 的访问控制模型

Android 系统是当今移动计算设备（尤其是智能手机）上领先的软件平台之一。这些设备带来的访问控制难题，和传统的服务器计算机，甚至个人台式机或笔记本电脑都不一样。它们的功能建立在许多相对较小的独立应用程序之上，这些程序通常称为 app，用户把它们下载、安装到只属于自己的设备上运行。所以，不存在要在一台机器上保护多个用户互不侵犯的问题。如果使用标准的访问控制模型，这些 app 就会以那个用户的身份运行。可是 app 由许多不同的实体开发，其中一些可能是恶意的。此外，大多数 app 并没有正当理由使用设备上的大多数资源。如果给它们的特权过多，恶意 app 就能访问机主的通讯录、拨打电话、在网上买东西，干出许多别的坏事。最小特权（least privilege）原则意味着我们不应该把机主的全部特权都交给 app，但它们要做点有意思的事，又必须有一些特权。

Android 运行在某个版本的 Linux 之上，它限制应用程序访问的办法之一，是给每个安装的 app 生成一个新的用户 ID。app 以这个 ID 运行，就可以据此控制它的访问。不过，Android 中间件还提供了额外的访问控制手段。应用程序开发者要声明自己的 app 需要哪些访问。用户考虑在设备上安装某个 app 时，会看到它需要哪些权限。用户可以授予 app 这些权限，也可以不安装这个 app，或者限制它的权限，不过限制权限也可能影响 app 的用途。此外，开发者还要规定其他 app 能以哪些方式与这个新 app 通信。用来编码这些访问信息的数据结构称为 **permission label**。一个 app 的 permission label（既包括它能访问什么，也包括它向其他 app 提供什么）在 app 设计时设定，并在 app 安装到某台 Android 设备上的那一刻编码进这个系统。

所以，permission label 很像能力：app 持有某个 label，就可以做某件事；缺少这个 label，就做不了那件事。一个 app 的 permission label 集合在安装时静态设定。用户之后可以修改这些权限，不过限制权限可能会损害 app 的功能。permission label 是强制访问控制的一种形式。Enck 等人 [E+09] 详细讨论了 Android 的安全模型。

Android 的安全方法很有意思，但并不完美。尤其是，用户并不总能意识到授予某个应用程序访问某样东西意味着什么；面对要么授予访问权、要么没法正常使用这个 app 的选择，他们往往会选择授予。如果这个 app 是恶意的，这种做法就可能出问题。

用户可以根据当前执行的任务切换角色，这种方法就尤其有价值，因为这样就不必临时设置或修改个人的访问权限，只需把他们从一个角色切换到另一个角色。通常，他们只在担任某个角色期间才拥有这个角色的权限。一旦退出某个角色（也许是为了进入另一个特权不同的角色），他们就失去了所退出角色的特权。

这一观察催生了**基于角色的访问控制**（role-based access control，**RBAC**）。它的核心思想早已存在，后来才由 Ferraiolo 和 Kuhn 在一篇研究论文中更正式地阐述出来 [FK92]。现在，RBAC 在许多机构中都很常用，尤其是大型机构。大型机构面临的管理难题比小机构严重，所以像 RBAC 这样能一次操作处理一批用户的方法，可以大大减轻管理工作。例如，如果一家公司决定，所有程序员都应当获得访问一个新开发的库的权限，而会计不应当，那么用 RBAC 只需一次操作，把所需的特权赋给 *Programmer*（程序员）角色，就能达到这个效


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

果。如果某个程序员被提拔到一个不需要访问这个库的管理岗位，公司只需把 *Programmer* 角色从这位经理可以担任的角色集合中去掉。

这样的限制并不一定意味着你怀疑自己的会计不诚实，会把公司的秘密库代码卖给竞争对手[7]。记住最小特权（least privilege）原则：你给别人访问某样东西的权限时，依靠的不仅是他们的诚实，还有他们的谨慎。如果会计根本无法访问这个库，那么无论他们是心怀恶意还是一时疏忽，你的库代码都不会经由会计的特权泄露出去。最小特权（least privilege）不仅在理论上是个好主意，在现实世界中构建安全系统时也至关重要。

RBAC 听起来有点像在访问控制列表中使用组，两者确实有些相似，但 RBAC 系统比单纯的组访问权限强大得多：RBAC 系统允许某个用户担任多个互不相交的角色。也许我们那位程序员被提拔到了管理岗位，但仍然需要访问那个库，比如要测试另一位团队成员的代码。RBAC 系统会让这位程序员在经理和程序员两个角色之间切换：暂时放下与经理角色相关的权利，获得与程序员角色相关的权利。经理测试别人的新代码时，有权访问那个库，但没有权限查看团队成员的绩效评估。这样，如果某个狡猾的程序员往库里偷偷塞了恶意代码（比如试图读取其他团队成员的绩效评估，或者打探他们的工资），运行这段代码的经理也不会无意中泄露这些信息；在合适的时候使用合适的角色，就能防止这种事情发生。

这类系统通常要求重新进行一次身份认证，才能担任某个 RBAC 角色；而且担任角色 A 通常意味着要放弃先前角色（比如角色 B）的相关特权。经理切换到代码测试角色时，就会暂时放弃查看绩效评估的特权。测试完成后，经理再切换回可以查看评估的角色，同时失去访问那个库的特权。RBAC 系统还可以提供比单纯读写文件更细的粒度。某个特定角色（比如 *Salesperson*，销售员）也许可以往一个文件中添加某种产品的购买记录，却不能往同一个文件中添加同一种产品的补货记录，因为销售员不负责补货。这种程度的控制有时称为**类型强制**（type enforcement）。它借助通常所说的客体的**安全上下文**（security context），把详细的访问规则与特定的客体关联起来。具体怎样实现，会影响性能、安全上下文（security context）信息的存储，以及身份认证。

在 Linux 和类似的操作系统下，可以用 ACL 和组搭建一个非常简单的 RBAC 系统。这些系统的访问控制机制有一个特性，称为**权限提升**（privilege escalation）。权限提升允许谨慎地扩展特权，通常的做法是允许某个特定程序以超出调用者本人特权的一组特权运行。在 Unix 和 Linux 系统中，这个特性叫作 **setuid**，它允许一个程序以另一个用户的特权运行，这个用户通常拥有运行该程序的用户平常没有的特权。不过，这些特权只在程序运行期间授予，程序一退出就失去了。一个精心编写的 setuid 程序只会用这些特权执行有限的一组操作，以确保特权不会被滥用[8]。我们可以为每个角色定义一个虚拟用户，并把所需的特权和这个用户关联起来，从而搭建一个简单的 RBAC 系统。需要这些特权的程序，可以设成以这个用户的身份 setuid 运行。

我们在身份认证一章中遇到过的 Linux `sudo` 命令就提供了这类功能，它允许某些指定的用户以另一个身份运行某些程序。例如：

```
sudo -u Programmer install newprogram
```

[7] 不诚实的会计一般最好避开，所以你多半已经尽力雇了诚实的。除非你是 Bernie Madoff [W20]，也许……

[8] 遗憾的是，并非所有以 setuid 方式运行的程序都写得很仔细，这些年来已经导致了许多安全问题。唉，也许所有安全特性都是如此?


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

提示：权限提升是危险的

我们刚刚讲了怎样用权限提升临时改变某个用户能做的事，以及这样做能带来哪些新的安全手段。但权限提升也有危险的一面。攻击者攻破你的系统时，攻陷的往往是一个只有很有限特权的程序。也许它本来只能处理几个简单的信息文件，把文件内容提供给远程用户，或许再对这些文件运行一些标准工具。它甚至可能对自己的文件都没有写权限。你可能会觉得这种攻陷对系统没造成什么危害，因为攻击者利用这种访问权做不了多少事。

这正是权限提升的危险所在。攻击者只要在系统上取得了一点立足之地，接下来就会四处寻找提升权限的途径。即使是一个特权相当少的应用程序，也能做许多外人无法直接做的事，所以攻击者会在被攻陷的应用程序能访问的代码或配置中寻找漏洞。成功攻陷一个系统之后，攻击者通常头一件事就是这样尝试提升权限。

在许多系统中，有一个特殊的用户，通常称为**超级用户**（superuser）或 **root** 用户。这个用户的特权比系统中其他任何用户都多得多，因为它的用途就是让人能对系统做最关键、影响最深远的管理变更。攻击者在你的系统上站稳脚跟之后，首要目标就是利用权限提升成为 root 用户。能做到这一点的攻击者，实际上就完全控制了你的系统。这样的攻击者可以查看任何文件、修改任何程序、更改任何配置，甚至可能安装另一个操作系统。这种危险应当让你明白：对于任何可能一路提升到超级用户特权的途径，都必须极其小心。

只要运行这条命令的用户在系统维护的一张名单上（名单上的用户获准担任 Programmer 这个身份），这条命令就会以用户 Programmer 的身份、而不是以运行者本人的身份，运行这个 install 命令。要安全地使用这种方法，就得仔细配置那些控制“谁可以以哪个身份执行哪些程序”的系统文件。和其他 RBAC 系统一样，`sudo` 命令通常要求重新进行一次身份认证。

为了更高级的用途，RBAC 系统通常支持比 setuid 和 `sudo` 操作更细的粒度，对角色分配的跟踪也更细致。这样的 RBAC 系统可能是操作系统的一部分，也可能是系统的某种附加组件，或者是一个编程环境。使用 RBAC 时，往往也会运行一定程度的强制访问控制。如果不这样，在上面 `sudo` 的例子中，以 Programmer 身份运行的用户就可以运行一条命令修改文件的访问权限，让非程序员也能使用那个 install 命令。有了强制访问控制，用户可以担任 Programmer 角色来完成安装，却不能利用这个角色让销售员或会计也能执行安装。

## 55.7 小结

实施大多数安全策略，都需要控制哪些用户可以以哪些方式访问哪些资源。操作系统内置的访问控制机制提供了所需的功能。好的访问控制机制会借助精心设计和实现的 reference monitor，对与安全相关的访问做到完全仲裁（complete mediation），或者接近完全仲裁。

访问控制列表和能力是大多数访问控制系统使用的两种基本机制。访问控制列表精确地规定了哪些主体可以以哪些方式访问哪些客体，在不在相应的列表上决定了是否批准访


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

问。能力更像是锁的钥匙，持有正确的能力，就足以证明应当允许访问某个资源。用户可见的访问控制更常用某种形式的访问控制列表实现，而能力常常内置在操作系统中用户看不到的那一层。这两种访问控制机制本身并无优劣之分。和系统设计中的许多选项一样，它们各有特点，适合某些场合和用途，而不适合另一些。你需要明白在什么情况下该选哪一种。

访问控制机制可以是自主的，也可以是强制的，有些系统两者兼有。类型强制（type enforcement）和基于角色的访问控制（role-based access control）等增强手段，可以让你更容易实现所需的安全策略。

即使访问控制机制完全正确、极其高效，它能做的也不过是实施交给它的安全策略。因访问控制机制本身有缺陷而导致的安全失败很少见；因为这些机制所实施的策略设计得很差而导致的安全失败，可一点也不少见。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

## 参考文献

[C+63] "The Compatible Time Sharing System: A Programmer's Guide" by F. J. Corbato, M. M. Daggett, R. C. Daley, R. J. Creasy, J. D. Hellwig, R. H. Orenstein, and L. K. Korn. M.I.T. Press, 1963. *The programmer's guide for the early and influential CTSS time sharing system. Referenced here because it used an early version of an access control list approach to protecting data stored on disk.*

[DV64] "Programming Semantics for Multiprogrammed Computations" by Jack B. Dennis and Earl. C. van Horn. Communications of the ACM, Vol. 9, No. 3, March 1966. *The earliest discussion of the use of capabilities to perform access control in a computer. Though the authors themselves point to the "program reference table" used in the Burroughs B5000 system as an inspiration for this notion.*

[E+09] "Understanding Android Security" by William Enck, Machigar Ongtang, and Patrick McDaniel. IEEE Security and Privacy, Vol. 7, No. 1, January/February 1999. *An interesting approach to providing access control in a particular and important kind of machine. The approach has not been uniformly successful, but it is worth understanding in more detail than we discuss in this chapter.*

[FK92] "Role-Based Access Controls" by David Ferraiolo and D. Richard Kuhn. 15th National Computer Security Conference, October 1992. *The concepts behind RBAC were floating around since at least the 70s, but this paper is commonly regarded as the first discussion of RBAC as a formal concept with particular properties.*

[L84] "Capability-Based Computer Systems" by Henry Levy. Digital Press, 1984. *A full book on the use of capabilities in computer systems, as of 1984. It includes coverage of both hardware using capabilities and operating systems, like Hydra, that used them.*

[L01] "Integrating Flexible Support for Security Policies Into the Linux Operating System" by Peter Loscocco. Proceedings of the FREENIX Track at the USENIX Annual Technical Conference 2001. *The NSA built this version of Linux that incorporates mandatory access control and other security features into Linux. A good place to dive into the world of mandatory access control, if either necessity or interest motivates you to do so.*

[S74] "Protection and Control of Information Sharing in Multics" by Jerome Saltzer. Communications of the ACM, Vol. 17, No. 7, July 1974. *Sometimes it seems that every system idea not introduced in CTSS was added in Multics. In this case, it's the general use of groups in access control lists.*

[T84] "Reflections on Trusting Trust" by Ken Thompson. Communications of the ACM, Vol. 27, No. 8, August 1984. *Ken Thompson's Turing Award lecture, in which he pointed out how sly systems developers can slip in backdoors without anyone being aware of it. People have wondered ever since if he actually did what he talked about…*

[W20] "Bernie Madoff" by Wikipedia. https://en.wikipedia.org/wiki/Bernie_Madoff. *Bernie Madoff (painfully, pronounced "made off", as in "made off with your money") built a sophisticated Ponzi scheme, a fraud of unimaginable proportions (nearly 100 billion dollars). He is, as Wikipedia says, an "American charlatan". As relevant here, he probably hired dishonest accountants, or was one himself.*

[W+74] "Hydra: The Kernel of a Multiprocessor Operating System" by W. Wulf, E. Cohen, W. Corwin, A. Jones, R. Levin, C. Pearson, and F. Pollack. Communications of the ACM, Vol. 17, No. 6, June 1974. *A paper on a well-known operating system that made extensive and sophisticated use of capabilities to handle access control.*


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

# 56

# 用密码学保护信息

本章作者：**Peter Reiher（UCLA）**

## 56.1 引言

在前面几章中，我们讨论了如何明确安全目标、如何确定安全策略（security policy）、如何用身份认证（authentication）机制识别主体，以及如何用访问控制机制（access control mechanism）实施策略，规定哪些主体可以以哪些方式访问哪些计算机资源。我们虽然指出过，保护系统安全的这些环节都有许多固有的缺陷和问题；但如果把这些话题看作已经讲完，从安全的角度看，操作系统还有什么要操心的呢？为什么这些还不是全部？

我们之所以需要更多东西，有好几个原因。特别重要的一点是：并非一切都在操作系统的控制之下。也许你会反驳：你告诉过我操作系统是无所不能的！其实不然。它只在一个有限的范围内拥有很大的控制权，这个范围就是它运行所在的硬件，以及它被授予控制权的那些接口。对其他机器上发生的事，它并没有真正的控制权；如果有某种它控制不了的机制访问了它的某个硬件部件，它也管不了。

可是，如果系统本身并不控制对某个资源的访问，我们又怎能指望操作系统去保护它呢？答案是事先让资源做好应付麻烦的准备。实质上，我们假定数据一定会丢失，或者对手一定会试图非法篡改它，然后采取措施，确保这些行为不会给我们带来问题。关键的一点是：如果对手拿到的数据是一种他看不懂的形式，我们的秘密就是安全的。进一步说，如果攻击者看不懂这些数据，大概也就没法篡改它们，至少没法以可控的方式篡改。如果攻击者都不知道数据是什么意思，又怎么能把它改成自己想要的样子呢？

我们要用的核心技术是**密码学**（cryptography）。它是一组技术，能在可控的条件下、按预期的结果，把数据从一种形式转换成另一种形式。我们就用它把数据从普通形式变成另一种形式。如果做得对，对手即使研究受保护的形式，也判断不出原始数据是什么。当然，如果我们自己还想再用这些数据，就必须能逆转这种转换，把数据还原成普通形式。而这件事对手也必须很难做到。如果能做到这一步，我们还能为数据提供一些防篡改的保护，或者更准确地说，能阻止对手按自己的意愿篡改数据，甚至能知道对手何时动过我们的数据。这一切都要归功于密码学的妙处！

但要正确使用密码学并不容易，而且很多密码学用法的计算开销都很大。所以，在哪里、在什么时候使用密码学，我们要有所选择；怎样实现它、怎样把它集成到系统中，也要十分小心。选得好、做得对，安全性会大大提高；选得不好、实现得差，就毫无帮助，甚至可能有害。

关键问题：
如何保护操作系统控制范围之外的信息
即使其他人在操作系统的控制之外拿到了关键数据，我们怎样用密码学确保他们既无法使用、也无法篡改这些数据？有哪些密码学技术可以帮助解决这个问题？怎样正确地使用这些技术？用它们能做的事有哪些局限？

## 56.2 密码学

关于密码学的书已经有很多了，而我们只打算花一章的篇幅来讲它。好在密码学中有一些重要而复杂的问题我们大可不管，所以仍然能讲出一些有用的东西。这是因为我们自己并不打算成为密码学家，只是这项技术的使用者，靠这个深奥领域里的专家给我们提供工具；这些工具，我们不必完全理解其工作原理也能用[1]。这听起来有点靠不住，可你其实早就在这么做了。真正懂得计算机硬件工作原理深层细节的人相对很少，但我们都能顺利地使用它，因为我们有好的接口，也知道是一群聪明人精心为我们打造了这些硬件。同样，密码学给我们提供了强大的接口和明确定义的行为，而且我们比平常更有把握：所用的工具背后凝聚了大量智慧。

话虽如此，密码学并不是魔杖，光是要正确地使用它，你就得了解很多东西。这正是我们在这里要重点讲的内容，尤其是在操作系统的语境下。

密码学背后的基本思想是：拿一段数据，用一个算法（通常称为**密码算法**，cipher），通常再辅以第二段信息（称为**密钥**，key），把这段数据转换成另一种形式。新的形式应该和旧的形式毫无相似之处；但通常，我们希望能运行另一个算法，同样辅以第二段信息，把数据转换回原来的形式。

我们把这个过程稍微形式化一点。从数据 $P$（通常称为**明文**，plaintext）、密钥 $K$ 和加密（encryption）算法 $E()$ 出发，我们最终得到 $C$，也就是 $P$ 改变后的形式，通常称为**密文**（ciphertext）：

$$C = E(P, K) \tag{56.7}$$

例如，我们可以把明文“Transfer $100 to my savings account”（转 100 美元到我的储蓄账户）转换成密文“Sqzmredq #099 sn lx rzuhmfr zbbntms”。这个例子其实用的是一种相当差劲的加密算法，叫作凯撒密码（Caesar cipher）。花一两分钟研究一下这段明文和密文，看看你能不能猜出这里用的是什么加密算法。

逆向转换以我们刚刚得到的 $C$、解密（decryption）算法 $D()$ 和密钥 $K$ 为输入：

$$P = D(C, K) \tag{56.8}$$

于是，我们可以把“Sqzmredq #099 sn lx rzuhmfr zbbntms”解密回“Transfer $100 to my savings account”。如果你已经猜出了我们当初是怎样加密的，那么猜出怎样解密应该也很容易。

[1]如果你想多了解一些密码学引人入胜的历史，可以看看 Kahn 的书 [K96]。如果你想了解更多技术细节，Schneier 的书 [S96] 是个不错的起点。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

密码学的用途很多，但泛泛讨论时，人们通常讲的是发送和接收消息。在这样的讨论中，明文 $P$ 是我们想发送的消息，密文 $C$ 是这条消息受保护的版本，我们把它发送到冷酷无情的外部世界中去。

加密过程要有用，就必须是确定性的：第一种转换用某个特定的 $K$，总是把某个特定的 $P$ 转换成某个特定的 $C$；第二种转换用这个特定的 $K$，总是把这个特定的 $C$ 转换回原来的 $P$。很多情况下，$E()$ 和 $D()$ 其实是同一个算法，不过这并不是必需的。另外，在不知道 $K$ 的情况下，应当很难从 $C$ 推出 $P$。要是能做到完全不可能就最好了，不过我们通常满足于在计算上不可行。有了这个性质，我们就可以把 $C$ 拿给世界上最充满敌意、最聪明的对手看，他们照样无法知道 $P$ 是什么。当然，前提是……

就在这里，干净利落的理论论文开始和乱糟糟的现实发生碰撞。只有对手不同时知道 $D()$ 和我们的密钥 $K$，我们才能得到这种令人安心的保密性。如果这两样他们都知道了，就会对 $C$ 施用 $D()$ 和 $K$，提取出和我们一样的信息 $P$。

其实，我们通常没法对 $E()$ 和 $D()$ 保密。由于我们并不想当密码学家，这里就不深究原因了，不过设计出好的密码算法是极其困难的。如果密码算法有弱点，对手即使没有 $K$，也能提取出明文 $P$。所以我们需要一个非常好的密码算法，而这种算法很难得到。我们大多数人手边并没有一位世界级的密码学家来设计新算法，所以只能依赖为数不多的几种已知的强密码算法。AES 是一种经过精心设计和深入研究的标准密码算法，是值得你考虑使用的一个好例子。

这听起来就像我们扔掉了一半的保护，因为现在密码学的好处完全取决于密钥的保密。正是如此。这一点太重要了，你必须牢牢记住，所以我们加粗再说一遍：**密码学的好处完全取决于密钥的保密**。把这句话再读几十遍大概也没有坏处，因为到处都是没把这一教训放在心上、因而不安全的系统。

好消息是，如果你用的是强密码算法，并且小心地为密钥保密，你的密码学就是牢靠的，别的什么都不用担心。坏消息是，在密码学的实际应用中，要在现实的系统里为密钥保密并不容易。这一点我们后面还会多讲。

此刻，先为我们取得的保护高兴一下吧，而且还有更值得欢喜的：正确地使用密码学，带给我们的不只是保密性！想想我们所做的这些转换有什么性质。如果对手拿到了我们加密后的数据，他看不懂。可要是对手能篡改它呢？他篡改的是加密后的形式，也就是对 $C$ 做了些改动，把它变成了比如说 $C'$。我们试图解密 $C'$ 时会发生什么？它不会解密成 $P$，而会解密成别的东西，比如说 $P'$。对于你该用的那类好密码算法来说，除非知道 $K$，否则很难确定一段密文 $C'$ 会解密成什么。这意味着，很难预测需要哪段密文才能解密出某段特定的明文；而这又意味着，攻击者完全不知道他改过的密文 $C'$ 会解密成什么。

$C'$ 可能解密成任何位模式，而对照我们预期看到的内容（ASCII 文本、格式正确的 PDF 文件，或者别的什么），$P'$ 很可能是一堆垃圾。如果我们足够小心，就能发现 $P'$ 并不是我们最初的数据，从而知道对手篡改过我们加密的数据。如果想更有把握，可以用散列函数（hash function）对明文算出一个散列值（hash），附在消息或加密文件里一起发送。如果解出来的明文算不出同样的散列值，就有充分的迹象表明出了问题。

所以，我们也可以用密码学来帮助保护数据的完整性（integrity）。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

提示：自己开发密码算法——别这么做

别这么做。

我们很想就此打住，因为遵守这条建议真的很重要。但你也许不信，所以我们再多解释几句。世界上最好的密码学家设计出的密码算法也常常有缺陷。你是世界上最好的密码学家之一吗？如果不是，而顶尖专家都常常造不出强密码算法，你凭什么认为自己能做得更好，甚至做得一样好？

我们知道你接下来会说什么："可我写的密码算法强得连我自己都破解不了。"嗯，几乎任何人只要用心，都能造出一个自己破解不了的密码算法。可还记得我们说过的那些世界级的密码学家吗？他们是怎样成为世界级的？靠仔细研究密码学的基础，靠破解别人的密码算法。他们非常擅长这件事；如果值得费这个劲，他们就会破解你的算法。如果你只是到处吹嘘自己的密码算法有多妙，他们也许懒得理你（因为这种话他们听得太多了）；可一旦你真把它用在什么重要的事情上，很不幸，你就会引起他们的注意。接着，你的秘密就会被揭穿；再接着，你就会因为自己设计密码算法、不用 AES 这样的标准算法，显得很傻，何况用标准算法本来就更省事。

所以，别这么做。

等等，还有呢！如果有人交给你一段数据，它是用只有你和你的好友 Remzi 知道的密钥 $K$ 加密的，会怎样？你知道这段数据不是你造的，所以如果用密钥 $K$ 能正确地解密它，你就知道它一定是 Remzi 造的。毕竟，除了你，只有他知道密钥 $K$，所以只有他能完成这次加密。瞧，我们就这样用密码学实现了身份认证！遗憾的是，密码学不会帮你打扫房间、替你写作业，也不能在几秒钟内切出几千根薯条，但无论如何，它都是一个非常好的工具。

我们刚才描述的这种密码学通常称为**对称密码学**（symmetric cryptography），因为加密和解密用的是同一个密钥。很长一段时间里，所有人都认为这是唯一可能的密码学形式。后来才知道，所有人都错了。

## 56.3 公钥密码学

讨论用密码学做身份认证时，你可能已经注意到了一个小问题：为了验证一段加密信息的真实性（authenticity），你需要知道加密它时所用的密钥。如果我们只关心用密码学做身份认证，这就很不方便。这意味着，我们必须把为此所用的密钥告诉每一个可能需要认证我们的人。假如我们是微软，想向每个买了我们软件的用户证明自己的身份，该怎么办？我们不能只用一个密钥来做这件事，因为那样就得把这个密钥发给数以亿计的用户；而他们一旦有了这个密钥，就可以用它加密信息，冒充微软。或者，微软也可以为这数以亿计的用户每人生成一个不同的密钥，但那样就得秘密地把各不相同的密钥分别交给数以亿计的用户，更不用说还要管理所有这些密钥。真是糟糕。

好在我们的好朋友，那些密码学奇才，想出了一个解决办法。如果密码学用两个不同的密钥，一个用来加密，另一个用来解密，会怎样？我们的加密操作变成

$$C = E\big(P, K_{encrypt}\big) \tag{56.9}$$

解密操作变成


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

$$P = D(C, K_{decrypt}) \tag{56.10}$$

微软的日子一下子好过多了。他们可以把解密密钥 $K_{decrypt}$ 告诉所有人，而对加密密钥 $K_{encrypt}$ 保密。现在，他们可以用自己的秘密密钥加密数据，以此证明数据的真实性，而他们数以亿计的用户可以用微软公开的密钥检验真实性。例如，微软可以用 $K_{encrypt}$ 加密一个操作系统更新，然后把它发给所有用户。每个用户都可以用 $K_{decrypt}$ 解密它。如果解密出来的是一个格式正确的软件更新，用户就可以确定它是微软制作的。由于没有其他人知道那个私钥（private key），别人不可能造出这个更新。

这听起来像魔法，但并不是。其实，又是数学救了我们，它常常这样。我们不在这里深入细节，但你得承认这相当巧妙。这种密码学称为**公钥密码学**（public key cryptography），因为两个密钥中有一个可以广为人知，向全体公众公开，同时仍能达到我们想要的效果。人人都知道的那个密钥称为**公钥**（public key），只有所有者知道的那个密钥称为**私钥**（private key）。公钥密码学（通常缩写为 **PK**）的发明史很复杂，虽然有趣，但和我们的讨论关系不大。想了解细节，可以看看这个领域的先驱 Whitfield Diffie 写的一篇论文 [D88]。

公钥密码学避开了早期各种密码学面临的一个难题：安全地分发秘密密钥。在这里，私钥由一方创建并由他保密，从不分发给任何其他人。公钥必须分发出去，但一般来说，我们并不在乎有第三方知道这个密钥，因为他们无法用它给消息签名（sign）。分发公钥比分发秘密密钥要容易，不过，唉，它比听起来要难。这一点我们后面会讲到。

公钥密码学其实还更巧妙，因为它反过来也行。你可以用解密密钥 $K_{decrypt}$ 加密，这时就需要用加密密钥 $K_{encrypt}$ 来解密。我们仍然希望加密密钥保密、解密密钥公开，所以按这个顺序做，就无法再实现身份认证了。毕竟，任何人都可以用 $K_{decrypt}$ 加密。但只有密钥的所有者才能用 $K_{encrypt}$ 解密这样的消息。所以，只要知道对方的公钥，任何人都可以给持有私钥的人发送加密消息。因此，用私钥加密，PK 可以实现身份认证；用公钥加密，PK 可以实现保密通信。

如果你两者都想要呢？你完全可能想要。这时就需要两对不同的密钥。假设 Alice 想用 PK 和她的朋友 Bob 秘密通信，同时还想确保 Bob 能认证她的消息。再假设 Alice 和 Bob 各有一对自己的 PK 密钥，每人都知道自己的私钥和对方的公钥。如果 Alice 用自己的私钥加密消息，她就认证了这条消息，因为 Bob 可以用她的公钥解密，并且知道只有 Alice 才能造出这条消息。可是每个人都知道 Alice 的公钥，所以这样做无法保密。不过，如果 Alice 把这条经过认证的消息再加密一次，这一次用 Bob 的公钥，她就同时实现了保密。只有 Bob 知道对应的私钥，所以只有 Bob 能读这条消息。当然，Bob 需要解密两次，先用他的私钥，再用 Alice 的公钥。

这听起来开销很大。实际上比你想的还糟，因为原来公钥密码学有一个缺点：它的计算开销比依赖单一共享密钥的传统密码学大得多。公钥密码学执行起来可能比标准的对称密码学慢几百倍。因此，我们实在负担不起什么都用公钥密码学。我们需要挑选使用它的地方，用它来做它擅长的事。

还有一个重要的问题。我们相当轻率地说 Alice 知道 Bob 的公钥、Bob 知道 Alice 的公钥。这种美好的状态是怎么实现的？起初，只有 Alice 知道她的公钥，也只有 Bob 知道他的公钥。如果想享受公钥密码学的魔力，我们就得做点什么，把这些信息传给世界上的其他人。而且我们最好小心行事，因为 Bob 会认定，用他认为属于 Alice 的公钥加密的消息，


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

确实是 Alice 造的。如果某个邪恶的天才（不妨叫她 Eve）设法让 Bob 相信 Eve 的公钥其实属于 Alice，会怎样？如果发生这种事，Bob 就会把 Eve 造的消息误认为来自 Alice，我们认证消息的整个目标就落空了。我们最好确保 Eve 无法在“哪个公钥属于 Alice”这件事上骗过 Bob。

这就引出了一条漫长而幽暗的道路，通向密钥分发（key distribution）基础设施这个神秘的领域。你最好别自己去走这条路，因为即使是准备最充分、敢于冒险上路的先驱，也常常落得个悲惨的下场。实践中我们怎样分发公钥，会在分布式系统（distributed system）安全一章中讨论。眼下，只要记住：公钥密码学美妙的魔力，建立在密钥分发（key distribution）这个脏兮兮、靠不住的基础之上。

关于 PK 密码学还有一件事：**密码学的好处完全取决于密钥的保密**。（我们打赌你以前听过这句话。）在这里指的是私钥。对公钥密码学的整体好处来说，私钥的保密和对称密码学中单一共享密钥的保密同样重要。绝不要泄露私钥。绝不要共享私钥。使用和存放私钥时要格外小心。如果私钥丢了，你用它做过的一切都会面临风险，谁拿到它，谁就能冒充你、读你的秘密消息。那可就太糟了，不是吗？

## 56.4 密码学散列

我们前面讨论过，可以用密码学保护数据的完整性，因为加密数据被改动后就无法正确解密。为了降低这种完整性检查的开销，我们可以先对数据做散列（hash），只加密散列值，而不加密整份数据。然而，如果真想万无一失，我们就不能随便用一个散列函数，因为散列函数的本性决定了它们会有**散列冲突**（hash collision），也就是两个不同的位模式散列出同样的结果。如果攻击者能把我们打算发送的位模式换成另一个散列结果相同的位模式，我们就失去了完整性。

所以，为了格外小心，我们可以用**密码学散列**（cryptographic hash）来确保完整性。密码学散列是一类特殊的散列函数，具有几个重要的性质：

- 要找到产生同样散列值的两个输入，在计算上不可行。
- 对输入的任何改动，都会让得到的散列值发生不可预测的变化。
- 仅凭散列值推断输入的任何性质，在计算上不可行。

基于这些性质，如果我们只关心数据完整性（data integrity）而不关心保密性，就可以对一段数据求密码学散列，只加密这个散列值，然后把加密后的散列值和未加密的数据一起发给对方。如果对手在传输途中动了数据，我们解密散列值、再对数据重做一遍散列操作时，就会发现两者不匹配，从而察觉到篡改[2]。

把它稍微形式化一点：要执行密码学散列，我们取一段明文 $P$ 和一个散列算法 $H()$。注意，这里不一定涉及任何密钥。过程是这样的：

$$S = H(P) \tag{56.11}$$

[2]为什么要加密密码学散列值呢？因为任何人，包括我们的对手，都可以对任何东西运行密码学散列算法，包括被改动过的消息版本。如果我们不加密散列值，攻击者就会修改消息，算出一个新的散列值，用这两样替换原来的消息和散列值，再把结果发出去。但如果我们发送的散列值是加密的，攻击者就无法知道改动后的散列值加密后应该是什么样。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

由于密码学散列是一般散列的一个子类，我们通常期望 $S$ 比 $P$ 短，也许短很多。这意味着会有冲突，也就是两段不同的明文 $P$ 和 $P'$ 都散列成 $S$ 的情况。然而，上面列出的密码学散列的性质会让对手很难利用冲突。即使你同时知道 $S$ 和 $P$，也应当很难找到另一段散列成 $S$ 的明文 $P'$[3]。对于改动后的明文 $P'$，要算出它的 $S'$ 并不难，因为直接对 $P'$ 运行密码学散列算法就行了。但即使对 $P$ 只做极小的改动，比如只差一位的 $P'$，算出的散列值 $S'$ 也应当和 $S$ 有完全不可预测的差别。

密码学散列除了确保加密数据的完整性，还有别的用途。例如，正如身份认证一章所讨论的，存储加盐（salting）散列的密码时，首选的就是这类散列。它们可以用来判断存储的文件是否被改动过，Tripwire 这类著名的安全软件提供的就是这项功能。它们还可以用来迫使进程（process）在提交请求之前完成一定量的工作，这种方法称为“工作量证明”（proof of work）。提交者提交的请求，用某个指定的密码学散列算出的值必须等于某个特定的值；由于这类散列的性质，提交者必须尝试大量的请求格式，才能找到一个散列成所需值的格式。每次散列操作都要花些时间，所以提交一个合格的请求需要一定量可预测的工作。散列的这种用法以各种形式出现在好几类应用中，包括防垃圾邮件和区块链。

和其他密码学算法一样，密码学散列最好也使用标准算法。例如，SHA-3 算法通常被认为是一个好的选择。不过，密码学散列算法过时的事屡见不鲜，所以如果你在设计一个用到这类算法的系统，明智的做法是先查一查目前推荐使用的是哪些算法。

## 56.5 破解密码学

你很可能听说过有人破解密码学。这是影视作品里很流行的题材。你该有多担心呢？

嗯，如果你没听我们前面的劝告，自己造了一个密码算法，那你就该非常担心。担心到应该马上停止阅读，把你自己的密码算法从系统里拆掉，换上一个知名的、受人认可的标准算法。去吧，你回来时我们还在这儿。

如果你用的是那些标准算法之一呢？那你多半没事。如果你用的是现代标准算法，除了少数无关紧要的例外，目前还没有已知的方法能在不拿到密钥的情况下读取用这些算法加密的数据。这并不是说你的系统就安全了，而是说大概没人会靠破解密码学算法攻进你的系统。

那他们会怎么做呢？多半是利用你系统中与密码学毫不相干的软件缺陷；不过，也有一定可能是拿到你的密钥，或者利用你在密码学管理上的其他缺陷，来破解它。怎么做到呢？创建和使用密钥的软件有缺陷，是一个常见问题。在分布式环境中，共享密钥的方法有缺陷，也是常被利用的弱点。Peter Gutmann 对密码学管理不当经常引发的各类问题做过一个很好的调查 [G02]。例子包括：在许多人共用的软件里分发秘密密钥；错误地在网络上传输密钥的明文版本；从一个大大缩小了的候选集合里选择密钥，而不是从理论上可能的更大集合里选。最近，Heartbleed 攻击展示了一种方法：从远程计算机的内存中获取 OpenSSL 会话所用的密钥，从而让攻击者解密整个会话；而密码算法本身、它的实现以及它的密钥选择过

[3]每隔一段时间，就会有某个知名的密码学散列函数（cryptographic hash function）被“攻破”，意思是有人想出了办法，能造出一个 $P'$，用这个函数算出和 $P$ 相同的散列值。2017 年，一个名为 SHA-1 的散列函数就遭遇了这种事，这个函数因此变得不安全，不能再用于完整性保护 [G17]。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

提示：选择密钥

密钥保密的一个重要方面，是从一开始就选一个好的密钥。对公钥密码学来说，你需要运行一个算法，从为数不多的几种可能的密钥对中选出你要用的那一对。但对对称密码学来说，你可以从所有可能的密钥中随意挑一个。该怎么挑呢?

随机挑。如果你用某种确定性的方法选密钥，对手要找出你的密钥，就只需搞清楚你的方法。更糟的是，由于你多半会在一段时间里生成很多密钥，他一旦知道了你的方法，就能拿到所有密钥。而如果你随机生成密钥，对手搞清楚其中一个，也帮不了他推出你的其他任何密钥。密码系统的这个非常理想的性质称为**完美前向保密**（perfect forward secrecy）。

遗憾的是，真正的随机性（randomness）很难得到。操作系统能用的最好来源，是考察那些被认为本质上随机的硬件过程，比如硬件完成操作所需时间的低位，再把结果转换成随机数。这称为**收集熵**（gathering entropy）。在 Linux 中，系统会自动替你做这件事，你可以读 /dev/random 来使用收集到的熵。Windows 也有类似的收集熵的功能。用它们来生成你的密钥。它们并不完美，但对许多用途来说已经足够好了。

程，都没有任何缺陷。攻击者借助这个漏洞，能读取四分之一到一半使用 HTTPS（HTTP 的密码学保护版本）的网站的流量 [D+14]。

攻击者对付密码学的一种办法是猜密钥。这样做其实根本没有破解密码学。密码学算法的设计目的，是防止不知道密钥的人获取秘密。如果你知道密钥，解密本来就不应该难。

所以，攻击者可以干脆把每个可能的密钥都猜一遍、试一遍。这称为暴力破解（brute force attack），这也是你应该使用长密钥的原因。例如，AES 密钥至少有 128 位。假设你的 AES 密钥是随机生成的，攻击者平均要猜 $2^{127}$ 次才能猜中。猜这么多次要花很多时间。当然，如果某个软件缺陷导致你的系统只从 32 个可能的 AES 密钥中选一个，而不是从 $2^{128}$ 个中选一个，暴力破解可能就变得易如反掌了。密钥的选择对密码学来说是件大事。

例如，最初的 802.11 无线网络标准根本没有对在空中传输的数据做任何密码学保护。

为它增加这种保护的第一次尝试叫作 WEP（Wired Equivalent Protocol，有线等效协议，这个名字相当乐观）。WEP 受制于必须塞进现有标准，但它生成和分发对称密钥的方法有严重缺陷。攻击者只要在一个 802.11 网络上监听无线流量，最快一分钟内就能确定正在使用的密钥。有一些随处可得的工具，任何人都能用它们做到这一点[4]。

再举一个例子：Netscape 浏览器的早期实现，用一些很容易猜到的值（比如当天的时间和请求密钥的进程的 ID）作为随机数生成器的种子来生成密码学密钥。研究人员发现，他们大约 30 秒就能猜出生成的密钥 [GW96]。

你可能听说过 PK 系统用的密钥要长得多，2K 或 4K 位。听起来安全多了，不是吗? 这至少能让它们更能抵御暴力破解吧? 然而，这类密码系统的密钥不能随机选择。可能的公钥和私钥对只有相对很少的几种。这是因为公钥和私钥必须相互关联，系统才能工作。这种关联通常是数学上的，而且故意设计成很难推导，所以知道公钥并不会让人很容易得知私钥。

[4]WEP 后来被 WPA 取代了。不幸的是，WPA 也有自己的弱点，于是又被 WPA2 取代。不幸的是，WPA2 也有自己的弱点，所以从 2018 年起，它正在被 WPA3 取代。无线网络密码学保护的这段悲惨历史，应当给那些低估了做好这件事有多难的人一个教训。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

但是，有了公钥，人们最终可以利用这个系统的数学性质推出私钥。这就是 PK 系统要用这么大的密钥的原因：确保这个“最终”是很长很长的时间。

但只有你对私钥保密，这一点才有意义。说到这里，但愿你已经觉得这是显而易见的了；可许多嵌入式设备的制造商用 PK 为这些设备提供加密，却把私钥放进了设备的软件里。而且很多时候，同一型号的所有设备都用同一个私钥。这样共用的私钥，无一例外都会变得，嗯，公开。2016 年 9 月，一项研究发现有 450 万台嵌入式设备依赖的私钥已经不再私密了[V16]。任何人都可以出于任何目的冒充这些设备中的任何一台，也可以读取任何用 PK 发给它们的信息。实质上，这些设备所做的密码学不过是装点门面，并没有让设备的安全性有任何明显的提升。

总之，破解密码学通常就是想办法得知密钥。或者，你可能已经猜到了：**密码学的好处完全取决于密钥的保密**。

## 56.6 密码学与操作系统

密码学引人入胜，但引人入胜的东西多了[5]，很多和操作系统毫无关系。我们为什么要费心花半章的篇幅讲密码学？因为我们可以用它来保护操作系统。

但并不是在任何地方、为任何目的都能用。我们已经反复向你灌输：要有效地使用密码学，密钥保密至关重要。这应该让你明白，只要密钥无法保密，就无法有效地使用密码学。回想一下关于安全的第一章：操作系统控制着计算机上的所有资源，也能访问它们。这意味着，如果你在计算机上有加密的信息，而解密它所需的密钥也在同一台计算机上，那么这台机器上的操作系统就能解密这些数据，不管这是不是你想要的结果[6]。

你要么信任你的操作系统，要么不信任。如果不信任，日子反正不会好过；其中一个后果是，这个不可信的操作系统一旦接触过你的秘密密钥，就可以复制它，随时重复使用。反过来，如果你信任你的操作系统，就不需要对它隐藏你的数据，这时也就用不着密码学。凡是把数据交给你不信任的东西，都是这个道理。例如，如果你不放心让云计算服务商经手你的数据，那么把明文数据交给他们、再请他们加密，并不会让情况有任何改善。他们已经看过明文了，还可以留一份密钥的副本。

如果你确信你的操作系统现在可信，但担心它将来可能不可信，你可以现在就加密某些东西，并确保密钥不存放在这台机器上。当然，如果你对操作系统当前的安全性判断错了，或者在操作系统变坏之后，你又在这台机器上解密过这些数据，那么你的密码学就保护不了你，因为至关重要的密钥保密已经被破坏了。

有人会说，操作系统被攻陷并不总是永久性的。很多是，但有些只让攻击者暂时访问系统资源，或者只能访问少数几样特定资源。在这些情况下，如果加密数据没有以明文存储，而且在攻击者能访问的时间或地点拿不到解密密钥，那么加密这些数据可能仍然有好处。这里棘手的问题在于，你无法事先知道对你系统的成功攻击是不是只会发生在特定的时间、持续特定的时长，或者只针对系统中的特定部分。所以，如果你采用这种方法，就要尽量减少

[5]比如贝多芬晚期的钢琴奏鸣曲。他最后一首奏鸣曲（作品 111 号）中的一个乐章，听起来甚至像爵士乐，而它写于 19 世纪 20 年代！

[6]不过，还记得前面某一章中讨论过的安全飞地（security enclave）吗？那种硬件不允许操作系统完全访问飞地所保护的信息。想一想，对于使用这种飞地的计算机，这对密码学意味着什么，又带来了哪些新的可能。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

一切暴露：少解密，迅速而谨慎地处理掉明文数据，除了执行密码学操作的时候，不在系统中保留密钥的明文版本。这样的最小化可能很难做到。

如果密码学无法完全保护我们免受不诚实的操作系统的侵害，那么操作系统还能拿密码学做什么用呢?我们在身份认证一章中看到过一个特殊的例子。有些密码学操作是单向的：能加密，但永远无法解密。即使操作系统被攻陷，我们也可以用这类操作以加密形式安全地存储密码，因为加密后的密码无法被解密[7]。

还有什么用途? 在分布式环境中，如果我们在一台机器上加密数据，然后通过网络发出去，那么所有中间组件都不属于我们的机器，因而拿不到密钥。数据在传输途中就受到了保护。当然，最终目的地机器上的那位同伴，要使用这些数据也需要密钥。我们之前承诺过，这个问题会在另一章中讨论。

还有吗?嗯，如果有人能不经过我们的操作系统就访问到我们的某些硬件，会怎样?如果存放在这些硬件上的数据是加密的，而密钥又不在这个硬件上，密码学就能保护这些数据。这种形式的加密有时称为**静态数据加密**（at-rest data encryption），以区别于加密在机器之间传输的数据。它很有用，也很重要，所以我们来详细看看。

## 56.7 静态数据加密

我们在持久性（persistence）部分的各章中看到，数据可以存放在磁盘驱动器（disk drive）、闪存盘（flash drive）或其他介质上。如果是敏感数据，我们可能希望它具备一些安全属性，比如保密性或完整性。实现这些目标的一种技术，是把数据以加密形式而不是明文形式存储。当然，加密数据无法用于大多数计算，所以如果存放数据的机器需要对数据做一般的计算，就必须先解密[8]。如果目的只是保存一份安全的数据副本，而不是使用它，那也许不需要解密，但这并不是常见的情况。

数据可以用不同的方式加密：使用不同的密码算法（DES、AES、Blowfish），采用不同的粒度，比如记录、数据块、单个文件或整个文件系统（file system）；由不同的系统组件完成，比如应用程序、库、文件系统或设备驱动程序（device driver）。静态数据加密（at-rest data encryption）有一种常见的通用做法，叫作**全盘加密**（full disk encryption）。这通常意味着存储设备（storage device）的全部内容（或几乎全部内容）都被加密。虽然名字叫全盘加密，但它实际上可以用在许多种持久存储介质上，而不只是硬盘驱动器（hard disk drive）。全盘加密通常要么由硬件提供（内置在存储设备中），要么由系统软件提供（设备驱动程序或文件系统的某个部分）。无论哪种情况，操作系统都在所提供的保护中扮演着一定的角色。Windows 的 BitLocker 和苹果的 FileVault 就是基于软件的全盘加密的例子。

一般来说，在启动时，系统会向用户索要解密密钥，或者索要可以用来获取密钥的信息（比如密码短语，passphrase，它类似于密码，但可能由多个单词组成）。如果提供的信息正确，硬件或操作系统就能拿到执行解密所需的一个或多个密钥。数据写到设备上时加

[7]但是，唉，只要合法用户曾经向被攻陷的操作系统提供过正确的密码，一切就都完了。被攻陷的操作系统会在用单向密码学散列算法处理密码之前，复制下用户提供的密码，交给幕后的某个坏蛋。

[8]有一个可能的例外值得一提。那些密码学奇才发明了一种叫作**同态加密**（homomorphic cryptography）的密码学，它允许你不解密，直接对加密形式的数据做运算。例如，你可以在不先解密的情况下，给一个加密的整数加一。等你解密结果时，果然，原来的数字就加了一。同态密码算法已经开发出来了，但截至本章写作时，高昂的计算和存储开销让它们在大多数场合都不实用。也许随着时间推移，这会有所改变。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

密，从设备上取出时解密。只要数据还在这台机器的内存里，不管在哪儿，包括共享缓冲区（buffer）或用户地址空间（address space）中，它就一直保持解密状态。新数据发往设备之前，会先加密。数据绝不会以解密的形式放到存储设备上。在最初那次获取解密密钥的请求完成之后，加密和解密对用户和应用程序完全透明。他们永远看不到加密形式的数据，在机器重启之前，也不会再被要求提供密钥。

密码学是一种计算开销很大的操作，用软件来做尤其如此。执行基于软件的全盘加密会带来一定的开销。各方报告的开销大小不一，但对磁盘密集型操作来说，多出百分之几的延迟是常见的。对于不怎么用磁盘的操作，开销可能察觉不到。对于基于硬件的全盘加密，磁盘驱动器能达到它的额定速度，这个速度可能比不使用全盘加密的同类型号慢，也可能不慢。

这种形式的加密能防范什么呢？

- 对于试图访问自己不该看到的数据的用户，它没有提供额外的保护。操作系统提供的标准访问控制机制要么有效（这样的用户因为没有访问权限而拿不到数据），要么无效（这时这样的用户和其他任何人一样，可以使用解密密钥）。
- 它不能防范泄露数据的应用程序缺陷。这类缺陷会让攻击者冒充用户，所以如果用户能访问未加密的数据，攻击者也能。例如，它对缓冲区溢出（buffer overflow）或 SQL 注入几乎没有什么保护作用。
- 它不能防范系统上不诚实的特权用户，比如系统管理员。管理员的特权可能让他冒充拥有这些数据的用户，或者安装能访问用户数据的系统组件；所以，管理员可以按需访问数据的解密副本。
- 它不能防范操作系统本身的安全缺陷。一旦提供了密钥，操作系统就能使用它（直接在内存中使用，或者间接地请硬件使用它），不管这个操作系统是可信而安全的，还是已被攻陷而不安全的。

那么，这种形式的加密有什么好处呢？考虑这样一种情况。如果把一个存储数据的硬件设备从一台机器上拆下来，装到另一台机器上，后者的操作系统并没有义务遵守设备上存储的访问控制（access control）信息。实际上，它甚至不必用同一个文件系统来访问这个设备。例如，它可以把这个设备仅仅当作一个原始数据块的来源，而不是一个组织好的文件系统。所以，新的操作系统可能会无视与设备上的文件相关联的任何访问控制信息。

然而，如果设备上的数据用全盘加密加过密，新机器通常就拿不到加密密钥。它可以访问原始数据块，但这些块是加密的，没有密钥就无法解密。例如，如果相关硬件被偷走并接到另一台机器上，这个好处就派上用场了。对于经常丢失或被盗的移动设备来说，这种情况是非常现实的可能。磁盘驱动器有时会被转卖，买家曾在上面发现过原主人的数据（包括相当敏感的数据）。在这些重要的情况下，全盘加密确实能带来实实在在的好处。

对于其他形式的静态数据加密（at-rest data encryption），系统仍然必须解决这些问题：加密多少数据，怎样获取密钥，以及何时加密和解密数据。这些问题的处理方式不同，得到的保护类型也不同。一般来说，这些情况要求有某个软件确保数据的未加密形式不再存放在任何地方，包括缓存（cache），并确保那些可能试图非法访问数据的人拿不到密码学密钥。这种保护有价值的场合相对较少，但也有几个常见的例子：

- 归档那些可能需要复制、必须保存、但不必使用的数据。这时，数据可以在创建时就加密，也许永远不解密，或者只在数据所有者控制下的特殊情况下才解密。如果数据最


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

初加密时机器没有被攻陷，而且密钥不永久存放在系统中，加密数据就相当安全。不过要注意，如果密钥丢了，你就再也无法解密这些归档数据了。

- 在云计算设施中存储敏感数据，这是上一个例子的变体。如果你不完全信任云计算服务商（或者不确定这家服务商有多谨慎；记住，信任另一个计算组件时，你信任的不仅是它的诚实，还有它的谨慎和正确性），那么在把数据发到云设施之前先加密，是明智的做法。许多云备份产品都包含这项功能。这时，密码学操作和密钥的使用，都发生在数据移到不可信的系统之前，或者从那个系统取回之后。
- 由应用程序执行的用户级加密。例如，用户可能选择加密一封电子邮件，存储下来的任何版本都是加密形式。这时，密码学操作由应用程序执行，用户要做点什么，把密码学密钥提供给应用程序。理想情况下，加密完成后，应用程序会确保数据的未加密形式和加密所用的密钥都不再轻易可得。不过要记住，只要密钥还存在，操作系统就能在你的应用程序不知情的情况下访问它。

加密选定的静态数据，有一个重要的特殊情况，就是我们在身份认证一章中讨论过的**密码库**（password vault，也称 **key ring**）。典型的用户要和许多远程网站打交道，这些网站都要求他们提供密码，还记得吗，这是基于“你知道什么”的身份认证。每个网站用不同的密码，安全性最好，但这会给人类用户带来负担，因为人一般很难记住许多密码。一个解决办法是把所有不同的密码都加密存放在机器上，按使用它们的网站编索引。需要某个密码时，就把它解密，提供给需要它的网站。

对于密码库和所有这类特殊情况，每当需要加密或解密数据时，系统都必须有办法拿到所需的密钥。如果攻击者能拿到密钥，密码学就毫无用处了，所以安全地存放密钥变得至关重要。一般来说，如果密钥以未加密的形式存放在这台计算机上的任何地方，加密数据就面临风险，所以设计良好的加密系统往往不这么做。例如，对于密码库，用来解密密码的密钥并不存放在机器的稳定存储（stable storage）中。需要时，系统会向用户索要这个密钥，或者索要用来推导出密钥的密码短语。然后用这个密钥解密所需的密码。要做到最安全，就应该在完成这次解密后立即销毁密钥——还记得最小特权（least privilege）原则吗？但这样做意味着每次需要密码时，用户都得重新输入密钥——还记得可接受性（acceptability）原则吗？在大多数情况下，人们在易用性和安全性之间取了个折中：用户第一次输入密钥后，系统在相当长的一段时间里记住它，但只把它保存在 RAM 中。用户注销、系统关机，或者负责密码库的应用程序（比如 Web 浏览器）退出时，密钥就被“忘掉”了。这种方法让人联想到单点登录（single sign-on）系统：用户第一次访问系统时要输入密码，此后直到注销都不需要再次认证。它的缺点也和那些系统一样，比如无人看管的终端可能被未经授权的人用来冒用别人的访问权限。两者都有一个巨大的优点：它们不会把用户烦得干脆弃之不用，转投毫无安全性可言的系统。

## 56.8 密码学能力

还记得访问控制一章里讲过，能力有一个问题：我们不能把它们留在用户手里，否则用户就可以伪造能力，给自己授予任何想要的访问权。密码学可以用来创建不可伪造的能力。一个可信实体可以用密码学创建一个足够长、加密足够安全的数据结构（data structure），表明持有者获准访问某个特定资源。然后可以把这个数据结构交给用户，用户把它出示给相


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

应资源的所有者, 以获得访问权。实际控制资源的系统必须能在批准访问之前检查这个数据结构是否有效，但不需要维护访问控制列表（access control list）。

这样的密码学能力可以用对称密码学创建，也可以用公钥密码学创建。用对称密码学时, 能力的创建者和检查它的系统需要共享同一个密钥。这两个实体是同一个系统时, 这种做法最可行; 否则就需要在需要使用密钥的机器之间传递密钥, 而根据使用场景, 可能还要高速、大规模地传递。你可能会好奇: 单独一台机器为什么要费事去创建一个密码学能力来允许访问, 而不是直接记住用户已经通过了访问检查? 其实有几个可能的原因。例如, 如果控制资源的机器要和数量巨大的用户打交道，跟踪每个用户的访问状态会代价高昂而又复杂, 在系统需要操心故障和延迟的分布式环境中尤其如此。或者, 如果系统希望访问权可以转移, 比如主体可能从一台机器换到另一台机器, 那么让能力随主体一起移动, 在任何地点都能使用，会更可行。如果创建和检查能力的所有机器本来就是可信的，密钥分发（key distribution）也不成问题，那么对称密码学能力同样说得通。

如果用公钥密码学创建能力, 创建者和资源控制者就不必在同一个地方, 信任关系也不必那么强。能力的创建者需要其中一个密钥，通常是秘密密钥，资源的控制者需要另一个。如果能力的内容本身不是秘密, 就可以使用真正的公钥, 不必在意谁知道它。如果需要保密（或者至少某种程度的隐蔽）, 那么本来可以作为公钥的那个密钥, 可以只分发给需要检查能力的有限几个实体[9]。资源管理者可以创建一组凭证（credential），说明哪个主体获准以哪些方式、在多长时间内使用哪些资源, 然后用私钥加密它们。其他任何人都可以用管理者的公钥解密这些凭证来验证它们。只要只有资源管理者知道私钥，就没有人能伪造能力。

如上所述, 这样的密码学能力可以容纳相当多的信息, 包括过期时间、获得这个能力的一方的身份, 以及其他许多信息。由于强密码学能确保所有这些信息的完整性, 能力是可以信赖的。这一特性让能力的创建者至少能在一定程度上防止能力被任意复制和共享。例如, 用在网络环境中的密码学能力可以与某个特定的 IP 地址绑定, 只有携带它的消息来自那个地址，才算有效。

可以使用许多不同的加密方案。重要的一点是, 加密后的能力必须足够长, 长到靠暴力枚举或随机猜测找出一个有效能力在计算上不可行（例如, 无效位模式的数量是有效位模式数量的 $10^{15}$ 倍）。

关于密码学能力，我们会在分布式系统安全一章中再多讲一些。

## 56.9 小结

即使数据已经不在系统的保管之下, 密码学也能为它提供某些形式的保护。这些保护形式包括保密性、完整性和身份认证。密码学用一种称为密码算法的算法, 把数据原来的位模式转换成另一种位模式, 以此提供这种保护。大多数情况下, 这种转换可以逆转, 得到原来的位模式。对称密码算法使用一个秘密密钥, 由所有有权访问这些数据的各方共享。非对称密码算法用一个密钥加密数据, 用另一个密钥解密数据, 其中一个密钥保密, 另一个通常公开。而密码学散列不允许逆转，也不需要使用密钥。

[9]不过要记住，如果你把一个密钥嵌在一个广泛分发的软件里，那这个密钥肯定会变得人尽皆知。所以，即使你认为对应的那个密钥是秘密的、而不是公开的，也不宜过分依赖这种想法。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

用了强密码算法，拿不到所需的密钥，要还原出原来的位模式在计算上就不可行。对于对称和非对称密码算法，这意味着只有持有正确密钥的人才能获得密码算法带来的好处。由于密码学散列没有密钥，这意味着任何人都不应该能从散列值得到原来的位模式。

对操作系统来说，密码学能派上用场的明显场合，是数据要发送到另一台机器上，或者存放数据的硬件可能在没有操作系统介入的情况下被访问。对于后一种情况，数据可以在设备上加密（用硬件或软件），在交给操作系统时解密。

密码算法一般不是秘密，而是广为人知、经过深入研究的标准。所以，密码算法能不能保护数据，完全取决于密钥能不能保密。如果攻击者能得知、推断或猜出密钥，一切保护就都没了。因此，如果依靠密码学来保护，就必须在密钥的选择和保密上极其小心。一条好的原则是：把密钥存放在尽可能少的地方，存放尽可能短的时间，让尽可能少的人能拿到。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

## 参考文献

[D88] "The First Ten Years of Public Key Cryptography" by Whitfield Diffie. Communications of the ACM, Vol. 76, No. 5, May 1988. *A description of the complex history of where public key cryptography came from.*

[D+14] "The Matter of Heartbleed" by Zakir Durumeric, James Kasten, David Adrian, J. Alex Halderman, Michael Bailey, Frank Li, Nicholas Weaver, Johanna Amann, Jethro Beekman, Mathias Payer, and Vern Paxson. Proceedings of the 2014 Conference on Internet Measurement Conference. *A good description of the Heartbleed vulnerability in OpenSSL and its impact on the Internet as a whole. Worth reading for the latter, especially, as it points out how one small bug in one critical piece of system software can have a tremendous impact.*

[G02] "Lessons Learned in Implementing and Deploying Crypto Software" by Peter Gutmann. Usenix Security Symposium, 2002. *A good analysis of the many ways in which poor use of a perfectly good cipher can totally compromise your software, backed up by actual cases of the problems occurring in the real world.*

[G17] "SHA-1 Shattered" by Google. https://shattered.io, 2017. *A web site describing details of how Google demonstrated the insecurity of the SHA-1 cryptographic hashing function. The web site provides general details, but also includes a link to a technical paper describing exactly how it was done.*

[GW96] "Randomness and the Netscape Browser" by Ian Goldberg and David Wagner. Dr. Dobbs Journal, January 1996. *Another example of being able to deduce keys that were not properly created and handled, in this case by guessing the inputs to the random number generator used to create the keys. Aren't attackers clever? Don't you wish they weren't?*

[K96] "The Codebreakers" by David Kahn. Scribner Publishing, 1996. *A long, but readable, history of cryptography, its uses, and how it is attacked.*

[S96] "Applied Cryptography" by Bruce Schneier. Jon Wiley and Sons, Inc., 1996. *A detailed description of how to use cryptography in many different circumstances, including example source code.*

[V16] "House of Keys: 9 Months later… 40% Worse" by Stefan Viehbock. Available on: blog.sec-consult.com/2016/09/house-of-keys-9-months-later-40-worse.html. *A web page describing the unfortunate ubiquity of the same private key being used in many different embedded devices.*


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

# 57

# 分布式系统安全

本章作者：**Peter Reiher（UCLA）**

## 57.1 引言

操作系统只能控制它自己那台机器上的资源。因此，在需要多台机器协作的分布式系统（distributed system）中，操作系统要提供安全会遇到难题。主要有两大问题：

- 分布式系统中的其他机器未必会正确执行你想要的安全策略（security policy），它们甚至可能是冒充可信伙伴的对手。我们无法控制远程系统，却仍然必须能相信它们交给我们的凭证（credential）和能力是有效的。
- 分布式系统中的机器要通过网络通信，而这个网络没有哪台机器能完全控制，一般来说也不可信。对手往往和我们一样能接入这个网络，可以伪造、复制、重放（replay）、篡改、销毁和延迟我们的消息，总之能干扰我们对网络的使用。

前面说过，密码学（cryptography）是我们在这里要用的主要工具，但我们也说过，密码学很难用对。这么看来，这里正适合使用精心设计的标准工具，而不是指望每个人都自己造一套。完全正确。于是：

关键问题：如何保护分布式系统中的操作
跨越多台机器的系统，怎样才能保证它的安全？有哪些工具可以帮助我们保护这类系统？怎样正确地使用它们？使用这些工具时，有哪些地方需要我们格外小心、仔细斟酌？

## 57.2 身份认证的作用

分布式系统中的伙伴会不会执行我们的安全策略，我们无法确定，这种不确定性该怎么应对？大多数情况下，我们做不了什么。充其量只能设法就策略达成一致，然后指望大家都遵守约定。在少数特殊情况下，我们能拿到高质量的证据，证明伙伴的行为是正确的，但一般来说这并不容易。比如，我们怎么知道对方用了全盘加密（full disk encryption）？怎么知道对方已经仔细清除了我们不再使用的某把加密密钥（encryption key）？又怎么知道对方给自己文件的本地副本正确设置了访问控制（access control）？他们可以说自己做了，但我们怎么知道是真是假？

一般来说，我们没法知道。不过你对此已经习惯了。在现实生活中，你的朋友和亲戚知道你的一些秘密，他们可能有你家的钥匙；你把车借给他们，也相当确定能把车要回来。这并不是因为你有什么完美的机制能阻止这些可信的人做坏事，而是因为你相当确定他们不会做。如果你看错了人，或许你能发现他们做了不该做的事，然后采取补救措施（比如换锁，或者报警说车被偷了）。在分布式计算机系统中，我们也要依靠类似的办法。我们只能相信有些参与方会规规矩矩。有些情况下，我们能发现他们没守规矩，于是相应地调整对他们的信任，也许还会采取其他补救措施。

当然，在网络世界里，我们的行动都是隔着网络远距离进行的，我们能看到的，只有在网络上进进出出的比特。要让基于信任的方案行得通，我们必须非常确定：我们发出去的比特，伙伴能够验证它们确实来自我们；而收到的比特，也确实是伙伴生成的。这就是身份认证（authentication）的工作。正如前面身份认证一章所说，在网络上工作时，我们只能根据一串比特来认证身份。最常用的是基于“你知道什么”的认证方式。现在回想一下前面几章：运行在远程操作系统上的某个东西，可能知道哪些别人都不知道的东西？密码怎么样？私钥（private key）呢？

分布式系统中的身份认证大多依靠这两样东西之一。要么你要求远程机器向你提供密码，要么你要求它用一把只存放在那台机器上的私钥来提供证据[1]。无论哪种情况，你都需要知道某些东西才能核验身份：要么是密码，更好的做法是密码加盐（salting）后的密码学散列（cryptographic hash）值；要么是公钥（public key）。

这两种方式分别适合什么场合？如果有大量参与方需要向同一个参与方证明自己的身份，密码往往比较好用。如果是一个参与方需要向大量参与方证明自己的身份，公钥往往比较好用。为什么？用密码认证时，认证结果证明的是有人知道这个密码。如果你想确切知道这个人是谁（这通常很重要），就只能让被认证的一方和核验的一方知道这个密码。而用公钥时，许多参与方都可以知道公钥，但只有知道对应私钥的那一方才能证明自己的身份。所以这两种机制我们都会用，只是用在不同的场合。网站向用户证明自己的身份时，用的是公钥密码学。网站只要向大量用户分发同一把公钥，所有用户就都能认证它，不必为了向每个用户证明自己而分别保存一份认证信息。用户向网站证明自己的身份时，用的是密码。每个用户都必须单独向网站认证，因此我们需要每个用户都有一份独一无二的身份信息，最好还方便人来使用。建立和分发公钥很难，而设置各自的密码相对容易。

在分布式系统中，这两种认证机制实际上怎么用？如果我们想让远程伙伴用密码证明自己的身份，就会要求它把密码交给我们核验。这样做的话，密码在网络上传输时必须加密；否则，任何在网络上窃听的人（对许多无线网络来说，窃听很容易）都能轻而易举地拿到明文（plaintext）发送的密码。要加密密码，我们必须事先已经有一把共享的对称密钥，或者有伙伴的公钥。下面我们先集中讨论怎样拿到这把公钥：要么直接使用它，要么借助它建立起加密，保护传输中的密码。

[1]我们偶尔也会用其他方法，比如智能卡或远程生物识别（biometrics）读取器。不过在今天的系统中，这些方法不太常见。如果你理解了在分布式系统身份认证中怎样使用密码和公钥密码学（public key cryptography），大概也就能想明白怎样正确使用其他这些技术。如果你还不理解，最好先弄懂常用的技术，再去研究不常用的。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

本章余下的部分都会讨论怎样保护网络连接的安全。但请不要忘了，即使你把网络保护得滴水不漏，网络另一端那个你无法控制的站点，依然是重大的安全挑战。如果你的伙伴已经被攻破，转而攻击你，那么即使这次攻击经过了认证和加密，对你也没什么安慰。

## 57.3 分布式系统中的公钥认证

公钥不需要保密，但我们需要确定它真的属于我们的伙伴。如果能当面见到伙伴，对方可以直接以某种形式把公钥交给我们，这时我们可以相当确定它就是正确的那一把。不过这种做法局限性很大，因为我们打交道的伙伴往往从未见过面。再说，要见 Amazon[2]或 Google 的“面”，该见谁呢?

好在公钥不需要保密，所以我们可以干脆生成一串包含公钥的比特。谁拿到这串比特的副本，谁就有了这把公钥。可是他们怎么确定这把公钥属于谁? 假如有另一个可信的第三方，所有需要认证我们伙伴的人都认识它，它用自己的私钥对这串比特做密码学签名（signature），证明这串比特确实属于我们的伙伴，结果会怎样? 只要我们能核验这个签名，就可以确定这串比特确实代表伙伴的公钥，至少在我们信任这个签名的第三方的范围内是如此。

我们在因特网上认证网站和许多其他实体，实际用的正是这种技术。每次浏览网页或进行其他任何基于 Web 的活动，你都在使用它。这串经过签名的比特叫作**证书**（certificate）。它基本上包含拥有这把公钥的一方的信息、公钥本身以及其他信息，比如有效期。包括公钥在内的这一整套信息先经过密码学散列运算，再用可信第三方的私钥加密散列（hash）结果，这样就对证书做了数字签名（digital signature）。只要你拿到证书的副本，又能核验签名，就能得知某人的公钥，即使你从未见过对方，也从未和对方直接打过交道。从某些方面看，这是一项支撑起整个因特网的美妙技术。

下面简单过一个例子，把这些概念理清楚。假设 Frobazz 公司想为自己的公钥 $K_F$ 申请一张证书。Frobazz 公司花了大价钱请 Acmesign 公司签发一张证书。Acmesign 是一家广受信任的公司，业务就是出售证书。这类公司通常叫作**证书颁发机构**（**Certificate Authorities**，**CA**），因为它们签发的权威证书受到许多参与方的信任。Acmesign 会核查 Frobazz 公司，确保来申请证书的人确实是 Frobazz 的合法代表。接着，Acmesign 会非常非常仔细地确认，它要嵌入证书的那把公钥确实是 Frobazz 想用的那一把。确认无误后，Acmesign 对 Frobazz 的名称、公钥 $K_F$ 以及其他信息运行一个密码学散列算法（可能是 SHA-3，截至 2020 年它还没有被破解，SHA-1 则已经被破解），得到散列值（hash） $H_F$。然后，Acmesign 用自己的私钥 $P_A$ 加密 $H_F$，得到数字签名 $S_F$。最后，Acmesign 把用来生成 $H_F$ 的全部信息，加上 Acmesign 自己的身份和签名 $S_F$，合成证书 $C_F$，交给 Frobazz，想必是收了钱的。记住，$C_F$ 只不过是一些比特而已。

现在，Frobazz 公司想通过因特网向它的一位顾客证明自己的身份。如果顾客已经有 Frobazz 的公钥，我们可以直接使用公钥认证机制。如果顾客没有这把公钥，Frobazz 就把 $C_F$ 发给顾客。顾客检查证书，看到它是 Acmesign 用（比如说）SHA-3 生成的，于是把 Acmesign 当初做散列的那些信息（全都在证书里）同样用 SHA-3 运算一遍，得到 $H_{F'}$。然

[2]如果 Jeff Bezos 得亲自登门拜访每一位顾客，把 Amazon 的公钥交给他们，Amazon 还能有多成功? 答案是：不会那么成功。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

后，顾客用 Acmesign 的公钥解密（decrypt）$S_F$（它也在证书里），得到 $H_F$。如果一切正常，$H_F$ 就等于 $H_{F'}$，顾客就知道证书里的公钥确实属于 Frobazz。接下来就可以进行基于公钥的认证了[3]。如果两个散列值不完全相同，顾客就知道其中有鬼，不会接受这张证书。

用这种方式获取公钥，有几个很好的特性。第一，在顾客核验证书的过程中，签名机构（在我们的例子里是 Acmesign）不需要参与。实际上，Frobazz 也不需要参与。顾客可以从任何地方拿到这张证书，对其有效性的把握都一样。第二，每位顾客只需要做一次。拿到证书并核验之后，顾客就有了需要的公钥。从此以后，顾客只要把公钥保存起来使用就行。如果公钥因为某种原因丢了，顾客可以从证书里再提取一次（如果证书还保存着），或者把获取证书的过程再走一遍。第三，在核验证书、确认对方身份之前，顾客不必信任自称是 Frobazz 的那一方。证书核验通过之前，顾客可以谨慎行事。

如果你认真读了前面几章，现在应该在心里嘀咕："等一下，这里是不是有个先有鸡还是先有蛋的问题？"我们靠获取 Frobazz 的证书来得知它的公钥。证书由 Acmesign 签名。我们要靠 Acmesign 的公钥来核验签名。那 Acmesign 的公钥又是从哪儿来的？我们真心希望你刚才挠了挠头，问了自己这个问题，因为如果你问了，就说明你理解了因特网身份认证问题的本质。归根结底，我们必须自举。你得想方设法先拿到某个你信任的人的公钥。一旦拿到了，如果它确实是正确的一方的正确公钥，你就能借此获得许多其他公钥。但如果没有一个起点，你几乎什么也做不了。

这把最初的公钥从哪里来？最常见的是来自你获取并安装的某个软件。你最常用的大概是浏览器，它通常自带几百家可信机构的公钥[4]。每当你访问一个注重安全的新网站，它会给你一张证书，里面有这个网站的公钥，签名者是浏览器里预先配置好的某家可信机构。你用这家机构预先配置好的公钥核验证书是否合法，之后就知道了这个网站的公钥。从此以后，你可以用网站的公钥来认证它。这里还有一些重要的注意事项（也有一些解决这些问题的有趣办法），不过我们暂且把它们放到一边。

任何人都可以创建证书，不只是那些受信任的 CA。你可以向以签发证书为业的机构申请一张，也可以按照某个证书标准从头自己创建一张（X.509 是最常用的证书标准 [I12]）。必要的条件是：被认证的一方和进行认证的各方，都必须信任创建证书的那一方。如果他们不信任那一方，又凭什么相信证书是正确的呢？

如果你在搭建自己的分布式系统，可以在一台你（以及系统中其他参与者）信任的机器上创建自己的证书，然后把这台签发证书的机器的公钥小心地手工安装到所有需要它的地方，以此解决自举问题。现在已经有不少用来创建证书的软件包，而且和其他关键的密码学软件一样，你最好使用现成的、可信的实现，而不要自己写一个。你也许可以看看 PGP（既有提供支持的商业版本，也有兼容但支持较少的免费版本）[P16]，当然还有别的选择。如果你面对的机器数量固定，又能以某种合理的方式手工分发公钥，那就完全可以不用证书。记住，公钥证书唯一的用处就是分发公钥，所以如果你的公钥已经在它们该在的地方了，就不需要证书。

[3]而且确实必须进行，因为核验证书这一大套工作，只是告诉了顾客 Frobazz 的公钥是什么，并不能向顾客保证发来证书的一方确实就是 Frobazz，或者知道 Frobazz 的私钥。

[4]你确实知道外面有几百家公司，你在网上做的每一件事都托付给了它们，对吧？好吧，不管你知不知道它们，实际上你都在这种程度上信任着它们。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

好了，不管用什么办法，你已经拿到了认证某台远程机器所需的公钥。接下来呢？嗯，对方用私钥加密后发给你的任何东西，都只能用对方的公钥解密，所以凡是用这把公钥能正确解密的东西，一定都来自对方，对吧？没错，它在某个时候确实来自对方。但对手可能复制了这个站点过去某个时候发出的一条合法消息，然后在以后的某一天再发一遍。根据具体情况，这可能会惹出麻烦，因为你可能会照这条消息去做合法站点并没有要求的事。所以我们通常会采取措施，确保自己不会遭受**重放攻击**（replay attack）。这类措施一般是确保每条加密消息都包含其他任何消息里都没有的独特信息。标准的密码学协议（protocol）都已经正确地内置了这一特性，所以只要你听从我们的建议，使用其中一种，就能防范这类重放攻击。如果你坚持自己构建密码学方案，就得对这个问题有更多的了解，还要非常小心地运用这些知识。此外，公钥密码学开销很大。我们希望尽快停止使用它，同时又希望继续得到身份认证的保证。讨论 SSL 和 TLS 时，我们会看到怎样做到这一点。

## 57.4 分布式系统中的密码认证

在分布式系统中，另一种常用的认证方式是使用密码。前面说过，这种方式最适合的情况是，任何一个特定的密码都只涉及两方：被认证的一方和进行认证的一方。单个用户向托管着许多用户的站点证明自己的身份（比如你登录 Amazon），用密码是合理的。而站点向单个用户证明自己的身份（比如一个自称是 Amazon 的网站想和你做生意），用密码就不合理了。这种情况下，公钥认证的效果更好。

在适合使用密码的场合，怎样在网络上正确地处理密码认证？密码通常与某个特定的用户 ID 关联，因此用户要把这个 ID 和密码提供给要求认证的站点。这一般要经过网络，而我们通常无法保证网络能提供机密性（confidentiality）。如果我们的密码泄露给了别人，别人就能冒充我们，所以我们必须给这种跨网络的认证加上机密性保护，一般至少要加密密码本身（不过把涉及的所有内容都加密更好）。因此，Alice 向 Frobazz 公司的网站证明自己身份的典型过程是：网站索要用户 ID 和密码，Alice 提供这两样东西，但在发到网络上之前先把它们加密。

你应该会问一个明显的问题：用哪把密钥加密？嗯，如果 Frobazz 已经像上面讨论的那样用公钥向 Alice 证明了自己的身份，Alice 就可以用 Frobazz 的公钥加密自己的用户 ID 和密码。Frobazz 公司有对应的私钥，能够核验它们，而其他人都读不了。实际上，出于各种原因，光这样做还不够，其中就包括上面提到的重放攻击。但我们确实可以用 Frobazz 的私钥来建立加密，保护传输途中 Alice 的密码，实际上也正是这么做的。具体细节我们会在 SSL/TLS 一节讨论。

我们在身份认证一章讨论过密码的选择和管理问题，那些内容在网络环境中同样适用。除此之外，关于怎样使用密码就没有太多要说的了，只想指出一点：远程站点核验过密码之后，它实际上知道了什么？它知道发来密码的站点或用户知道这个密码，并且在密码强度所能保证的程度内，可以认为这个站点或用户就是它自称的那一方。可是以后再收到据称来自这个站点的消息呢？记住，任何人都能随意创建任何消息，所以如果我们只是核验了远程站点发来的密码是对的，我们就只知道那一条消息是真实的。我们不想在发出的每条消息里都附上密码，就像我们不想用公钥去加密发出的每条消息一样。我们会用这两种认证技术建立


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

最初的真实性（authenticity），再用别的东西把这份最初的真实性和后续的交互绑在一起。我们直接进入 SSL/TLS，讨论怎样做到这一点。

## 57.5 SSL/TLS

我们在前面某一章看到，在现代系统中，进程（process）之间通信的标准方法是套接字（socket）。进程位于不同机器上时也是如此。所以，要给穿越无保护网络的通信加上密码学保护，一个很自然的办法就是给套接字加上密码学功能。这正是很多年前设计 **SSL**（**Secure Socket Layer**，安全套接字层）的目的。遗憾的是，SSL 并没有完全做对。这是因为要做对实在太难了，并不是设计和构建它的人粗心大意。他们从错误中吸取了教训，又设计了一个新版本的加密套接字，叫作**传输层安全**（**Transport Layer Security**，**TLS**）[5]。你会经常听人说在用 SSL，他们通常是把 SSL 当作 SSL/TLS 的简称。正式地说，SSL 是不安全的，任何场合都不应该再用它。请使用 TLS。唯一的例外是，有些非常老的设备上运行的软件可能不支持 TLS。在这种情况下，用 SSL 总比什么都不用好。从这里开始，我们也和别人一样采用这个简称，因为大家都这么叫。

SSL 背后的概念很简单：让加密过的数据通过一个普通的套接字传输。你建立一个套接字，再建立一个专门的结构来执行你想要的密码学运算，然后把这个结构的输出接到套接字的输入上。在另一端，把这个过程反过来。概念上很简单，执行起来却相当繁琐，要经过好几个步骤才能达到想要的结果。由于 SSL 本身的通用性，还会有更多复杂之处。这项技术的设计目标是支持各种密码学操作和许多不同的密码算法（cipher），还支持发送方和接收方之间用多种方法进行密钥交换和身份认证。

给程序加上 SSL 的过程很复杂，需要使用特定的库，并按一定顺序调用这些库，才能建立一个正确的 SSL 连接。我们不会在这里一步步地讲解这些操作，但要正确使用 SSL，你需要去学习它们。这些操作大体上是为了提供很大的通用性：SSL 支持的密码学选项要多样，你在程序里使用这些选项的方式也要灵活。例如，如果你需要的话，可以用这些设置调用，让一组 SSL 连接使用 AES-128，另一组使用 AES-256。

建立 SSL 连接有一个常见的要求，我们会稍微详细地讲一讲，那就是：怎样安全地分发这条新连接要用的密码学密钥。密码学的最佳实践要求，你建立的每个连接都用一把全新的密钥来加密大部分数据。公钥/私钥你会用来做很多次身份认证，但正如前面讨论的，认证了伙伴之后，你需要用对称密码学（symmetric cryptography）来加密数据，而且你需要一把新鲜的密钥。即使你和同一个伙伴同时建立了多个 SSL 连接，每个连接也都需要一把不同的对称密钥。

那么，建立一个新的 SSL 连接需要做些什么？我们不会把所有繁琐的细节都过一遍，但本质上，在没有可用的对称密钥时，SSL 需要（通常）基于非对称密码学（asymmetric cryptography）自举出一个安全的连接。（在讨论 SSL 时，你会经常听到“通常”“一般”和“默认”这类字眼，因为 SSL 能支持的选项非常多，而其中大多数一般都不是你想用的。）第一步是在客户端和服务器之间开始协商。每一方可能只能处理特定的密码算法、安全散列、

[5]实际上，就连 TLS 最初的几个版本也没有完全做对。截至 2020 年，TLS 的当前版本是 1.3，你大概应该用这个版本。TLS 1.3 修补了 TLS 1.2 存在的一些漏洞。SSL/TLS 一再需要修改的历史，应该能进一步让你认识到正确使用密码学有多难，而这又应该促使你发誓永远不要尝试自己写一套密码学方案。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

密钥分发（key distribution）策略或身份认证方案，这取决于它安装的SSL版本、配置方式，以及两边建立SSL连接的程序是怎么写的。在最常见的情况下，协商的结果是双方找到一组都能接受的密码算法和技术，在安全性和性能之间取得平衡。例如，它们可能用2048位密钥的RSA做非对称密码学（asymmetric cryptography），用某种形式的Diffie-Hellman密钥交换（Diffie-Hellman key exchange）机制（参见关于这个机制的补充）建立一把新的对称密钥，用SHA-3生成保证完整性（integrity）的安全散列，再用256位密钥的AES做大批量加密。一个现代的SSL安装可能支持50种甚至更多不同的选项组合。

补充：Diffie-Hellman 密钥交换

如果你想在两方之间共享一把秘密密钥，可它们只能通过不安全的信道通信，窃听者能听到它们说的任何话，那该怎么办？你可能觉得这个问题根本无解，但你错了。两位极其聪明的密码学家 Whitfield Diffie 和 Martin Hellman 多年前就解决了这个问题，他们的解法如今被广泛使用。它叫作 **Diffie-Hellman 密钥交换**（Diffie-Hellman key exchange）。

它的原理是这样的。假设 Alice 和 Bob 想共享一把秘密密钥，但目前除了能互相发消息之外，没有任何共享的东西。首先，他们商定两个数：$n$（一个大素数）和 $g$（模 $n$ 的原根）。这一步可以通过不安全的信道进行，因为 $n$ 和 $g$ 不需要保密。Alice 选一个大随机整数，比如 $x$，计算 $X = g^x \bmod n$，然后把 $X$ 发给 Bob。Bob 独立地选一个大随机整数，比如 $y$，计算 $Y = g^y \bmod n$，然后把 $Y$ 发给 Alice。窃听者能听到 $X$ 和 $Y$，但由于 Alice 和 Bob 没有发送 $x$ 或 $y$，窃听者并不知道这两个值。Alice 和 Bob 必须对 $x$ 和 $y$ 保密，这一点很重要。

现在，Alice 计算 $k = Y^x \bmod n$，Bob 计算 $k = X^y \bmod n$。Alice 和 Bob 各自算出的 $k$ 值是相同的。为什么？因为 $Y^x \bmod n = (g^y \bmod n)^x \bmod n$，它又等于 $g^{yx} \bmod n$。而 $X^y \bmod n = (g^x \bmod n)^y \bmod n = g^{xy} \bmod n$，和 Alice 得到的结果一样。这里没什么魔法，幂运算和取模运算本来就是这样的。啊，数学多么美妙！所以两边算出的 $k$ 相同，Alice 和 Bob 都知道它。

那些窃听者呢？他们知道 $g$、$n$、$X$ 和 $Y$，但不知道 $x$ 或 $y$。他们可以算出 $k' = XY \bmod n$，但这并不等于 Alice 和 Bob 算出的 $k$。他们确实有办法推导出 $x$ 或 $y$，从而得到足以算出 $k$ 的信息，但这些办法要么要求他们对 $n$ 的每一个可能取值都做一次计算（这就是 $n$ 要非常大的原因），要么要求他们计算离散对数。离散对数是可以求解的，但对很大的数来说，在计算上不可行。所以，只要素数 $n$ 足够大（并满足其他一些性质），窃听者就无计可施。要多大？600位（十进制）的素数应该就够了。

很巧妙，对吧？但在网络上使用 Diffie-Hellman 时，有一个美中不足之处。它能确保你和某人安全地共享一把密钥，却无法保证你是在和谁共享。也许 Alice 正如她所想、所希望的那样在和 Bob 共享密钥，但也可能和她共享密钥的是 Mallory：Mallory 冒充 Bob，注入了自己的 $Y$。我们通常很在意自己在和谁进行安全通信，所以一般会给 Diffie-Hellman 加上一个身份认证机制，确保伙伴的身份。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

有些情况下，指定这许多组合中哪些对你的系统来说是可以接受的，可能很重要；但很多时候，大多数组合都行，这时你可以让 SSL 为每个连接自行决定用哪一种，你自己不用操心。协商会在你看不见的地方进行，然后 SSL 就开始干正事：至少认证服务器（客户端可选），创建并分发一把新的对称密钥，再用这把密钥通过选定的密码算法进行通信。

我们可以用 Diffie-Hellman 密钥交换来创建密钥（SSL 也经常这么做），但我们必须确定自己是在和谁共享这把密钥。SSL 为此提供了多种办法。最常见的方法是，客户端拿到一张包含服务器公钥的证书（通常是由服务器发给客户端），然后用证书里的公钥核验服务器消息的真实性。客户端也可以通过其他途径获得证书，不过不太常见。注意，由服务器发送证书和客户端通过其他途径获取证书，安全程度完全一样（不安全的程度也一样）。证书的安全性并不取决于用什么方法传输它，而取决于嵌在证书里的密码学内容。

有了证书（不管客户端是怎么拿到的），Diffie-Hellman 密钥交换就可以在认证的前提下进行了。服务器会用自己的私钥对它的 Diffie-Hellman 消息签名，这样客户端就能确定，这次密钥交换中的伙伴是正确的服务器。客户端通常不提供（甚至根本没有）自己的证书，所以无法对自己的 Diffie-Hellman 消息签名。这意味着，SSL 的 Diffie-Hellman 密钥交换完成后，客户端通常相当确定服务器是谁，服务器却对客户端的身份一无所知。（同样，并不是 SSL 的所有用法都如此。SSL 也有一些建立连接的选项，让双方都知道对方的公钥，密钥交换在两边都经过认证。只是这些选项不是最常用的，尤其不是保护网页浏览时通常使用的那些。）

回想本章前面的讨论，很多情况下，此时服务器不确定客户端的身份其实并不是问题。前面说过，客户端大概会想用密码来证明自己的身份，而不是用从证书里提取的公钥。只要服务器在拿到并核验客户端的密码之前，不允许客户端做任何需要信任的事，服务器其实根本不在乎客户端是谁。许多服务器会向匿名客户端提供一些服务（比如向它们提供公开的信息），所以只要它们在进入更敏感的内容之前能从客户端拿到密码，就不存在安全问题。服务器可以在 SSL 连接建立后的任何时候，再向客户端索要用户 ID 和密码。由于建立 SSL 连接时已经设置了一把对称密钥，ID 和密码的交换就可以用这把密钥保护。

关于 SSL/TLS，最后再说一句：它是一个协议，而不是一个软件包。实现这个协议的软件包有好几种。理想情况下，只要它们都正确地实现了协议，彼此就都能正确交互。不过，它们实现协议的代码各不相同。因此，SSL/TLS 某个实现中的软件缺陷，在其他实现中未必存在。例如，心脏出血（Heartbleed）攻击利用的是 OpenSSL 实现上的细节 [H14]，这个缺陷在其他实现里并不存在，比如 Microsoft Windows 操作系统里的 SSL/TLS。另一方面，当前的 SSL/TLS 协议定义本身也可能包含协议缺陷，任何遵守协议的实现中都会有。如果你听说某个与 SSL 有关的安全问题，在进一步行动之前，先弄清楚它是协议缺陷还是实现缺陷。如果是实现缺陷，而你用的又是另一种实现，你可能就不需要采取任何应对措施。

## 57.6 其他身份认证方法

虽然密码和公钥是认证远程用户或机器最常用的方式，但还有其他选择。其中一种一直都在用。像上面描述的那样，你通过提供密码向某个网站证明了自己的身份之后，这个网站会一直认为这次认证有效。你每次点击链接，或者和它进行其他交互时，它都不会再要你的密码。（这样也好。想象一下，如果每做一件事都得提供一次密码，那该多烦人。）如果此时


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

你的会话是加密的，网站可以把你正确使用密码学这件事当作一种身份认证；但你甚至可能退出浏览器、重新启动、再回到那个网站，依然被当作已认证的用户，而不会再被要求输入密码。到了这时，你用的已经不是之前那套密码学了，因为你已经建立了一个新会话，设置了一把新的密码学密钥。你的伙伴是怎么认证收到新密钥的一方就是你的?

在这种情况下，与你打交道的站点选择了在安全性上做一个权衡。它过去某个时候用你的密码核验了你的身份，之后就依靠另一种方法来认证你。一种常用的方法是使用 **Web Cookie**。Web Cookie 是网站发给客户端的一些数据，网站希望客户端把这些数据保存起来，下次再和服务器通信时一并发回去。大多数浏览器都内置了对 Web Cookie 的支持，处理它们完全在后台进行，不需要用户干预。如果正确使用密码学，核验过客户端密码的服务器就可以创建一个 Web Cookie，把客户端的身份安全地存在里面。客户端再次与服务器通信时，浏览器会自动在请求中附上这个 Cookie，服务器借此就能核验客户端的身份，而不必再次索要密码[6]。

如果你花几分钟想一想这种认证方法，可能会想到一些相关的安全问题。设计这项技术的人已经解决了其中一些问题，比如防止窃听者直接使用在网络上传输时被复制的 Cookie。然而，还有一些安全问题（比如合法用户以外的人使用了运行浏览器、保存着 Cookie 的那台计算机）是这类 Cookie 解决不了的，而如果要求用户每次都提供密码，这些问题本来是可以解决的。构建自己的系统时，你需要自己考虑这类安全权衡。是只在绝对必要时才要求输入密码，让用户用起来更省事好，还是频繁要求用户证明身份、给用户更高的安全性好？重点不在于这个问题只有一个正确答案，而在于设计系统时你需要考虑这类问题。

还有其他身份认证方式。一个例子是所谓的**挑战/应答协议**（challenge/response protocol）。远程机器给你发来一个挑战，通常是一个数。要证明自己的身份，你必须对这个挑战执行某种运算，得出一个应答。这种运算应该只有真正的那一方才能完成，所以它大概要依赖一个只有那一方知道、别人都不知道的秘密。把这个秘密作用于挑战，就得到应答，再把应答发给服务器。服务器必须能够核验对方给出的应答是否正确。每次发出的挑战都不同，要求的应答也不同，所以攻击者窃听并记下旧的挑战和应答也占不到任何便宜。因此，挑战和应答都不需要加密。挑战/应答系统通常会对挑战加上秘密做某种密码学运算，也许是一次散列运算，来生成应答。这类运算由机器来做比由人来做更合适，所以要么由你的计算机替你算出应答，要么你有一个专门的硬件令牌（token）来处理。无论哪种方式，挑战/应答系统都要求发起挑战的机器和要证明自己身份的机器事先做好安排。硬件令牌或数据秘密必须在发出挑战之前就已经设置并分发好。

另一种身份认证方式是使用认证服务器。本质上，你和一台你信任、也信任你的服务器通信。需要你证明身份的那一方也必须信任这台服务器。认证服务器会以某种安全的形式（通常涉及密码学）为你的身份担保。需要认证你的那一方能够核验认证服务器提供的安全信息，从而确定服务器已经核验过你的身份。由于你想与之通信的那一方信任认证服务器，它现在也就相信你就是你自称的那个人。粗略地说，证书和 CA 是这类认证服务器的离线

[6]你可能还记得，在访问控制一章中，我们答应过要讨论在网络环境中如何用密码学来保护能力。本质上，这些 Web Cookie 就是这样的东西。用户用另一种机制证明自己的身份之后，远程系统会为这个用户创建一个别人都无法创建的密码学能力，一般用的是只有这个系统才知道的密钥。这个能力（也就是 Cookie）现在就可以传回给另一方，用于以后的授权（authorization）操作。许多其他分布式系统也采用同样的基本方法。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

版本。还有更主动的在线版本，需要两台想要通信的机器与一台或多台认证服务器之间进行各种网络交互。和 CA 这样的离线版本相比，在线版本对安全状况的变化反应更快。一张本不该再被认可的旧证书很难清除，而在线认证服务器可以立刻让被攻破的一方的认证失效，并马上应用这些变化。这类系统的细节可能相当复杂，所以我们不深入讨论。Kerberos 就是这类在线认证服务器的一个例子 [NT94]。

## 57.7 一些更高层的工具

有些情况下，我们可以在更高的层次上工作，达到理想的安全效果。**HTTPS**（HTTP 协议受密码学保护的版本）和 **SSH**（SSL 的一个竞争者，最常用来与远程计算机建立安全会话）就是两个很好的例子。

### HTTPS

HTTP 是支撑万维网的协议，它本身没有安全功能。不过如今，大量敏感而有价值的信息都在 Web 上传送，把它们全都毫无保护地发到网络上，显然不是好主意。然而，HTTPS 并没有为 HTTP 重新实现一套安全机制，而是拿现有的 HTTP 定义，把它和 SSL/TLS 连接起来。建立安全连接的工作由 SSL 负责，包括用前面讨论的证书方法认证 Web 服务器，以及建立一把只有客户端和服务器知道的新的对称加密密钥。SSL 连接一旦建立，客户端和服务器之间后续的所有交互都使用这条安全连接。在很大程度上，HTTPS 就是通过 SSL 连接传送的 HTTP。

不过，这并不会降低 HTTPS 的重要性。实际上，它是一个很有用的范例。HTTPS 没有花上好几年去开发，也没有冒险去面对其他安全协议开发者难免会遇到的同类安全缺陷，而是直接使用一个高质量的传输安全工具，以极低的开发成本，把不安全的传输换成了高度安全的传输。

HTTPS 显然非常依赖身份认证，因为我们希望确定自己没有在和恶意网站通信。HTTPS 为此使用证书。由于 HTTPS 主要用在浏览器里，相关的证书由浏览器收集和管理。现代浏览器都预先配置了许多证书签名机构（也就是前面提到的 CA）的公钥。网站的证书要对照这些签名机构来核验，判断证书是真是假。不过要记住，证书实际告诉你的是什么（假设核验通过）：在某个时刻，签名机构认为，担保某把公钥属于某一方是个好主意。它并不意味着这一方是好是坏，不意味着对应的私钥仍然保密，甚至也不意味着证书签名机构本身是安全的、没有被攻破，无论是在它创建证书时，还是在你核验证书时。现实中，Web 证书已经因为上面所有这些原因出过问题。还要记住，HTTPS 只能为真实性担保。一个经过认证、使用 HTTPS 的网站，照样可以对你的客户端发起攻击。没错，这是一次经过认证的攻击，可如果攻击得逞，这也安慰不了你多少。

过去并不是所有浏览器都一直支持 HTTPS，通常是因为它们没有安装或配置 SSL。在这种情况下，只使用 HTTPS 的网站就无法和这样的客户端交互，因为客户端建立不了自己这一端的 SSL 套接字。Web 服务器的标准解决办法是：客户端声称自己无法使用 HTTPS 时，就退回到 HTTP。服务器这样做时，就不提供任何安全保护，就好像服务器根本没有运行 HTTPS 一样。随着浏览器和客户端机器对 HTTPS 的支持越来越普遍，出现了一股潮流，


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

推动服务器坚持使用 HTTPS，拒绝和不能或不愿使用 HTTPS 的客户端通信。这种做法叫作 HSTS（HTTP Strict Transport Security，HTTP 严格传输安全）。HSTS 是网站的一个选项。网站如果决定支持 HSTS，那么任何客户端和它的所有交互都会受到密码学保护。不能或不愿接受 HTTPS 的客户端，就无法和这样的网站交互。不少大型网站都使用 HSTS，包括 Google 的 google.com 域名，但截至 2020 年，它还远未普及。

虽然 HTTPS 主要用来保护网页浏览，但有时也被用来保护其他类型的通信。有些开发者把 HTTP 用于与标准网页浏览相当不同的用途，对他们来说，用 HTTPS 来保护通信既自然又便宜。不过，只有决定用 HTTP 作为应用协议，你才能用 HTTPS 来保护系统，而 HTTP 主要是为支持人参与的活动而设计的。例如，HTTP 消息通常用 ASCII 编码，并带有大量为满足网页浏览需求而设计的首部。用 SSL 代替 HTTPS，你的应用可能会高效得多。你也可以用 SSH。

### SSH

**SSH** 是 **Secure Shell**（安全外壳）的缩写，这个名字准确地描述了这个程序最初的用途。SSH 可以在 Linux 和其他 Unix 系统上使用，在一定程度上也能在 Windows 系统上使用。SSH 最初的设想是一个安全的远程 shell，但它已经发展成一个更通用的工具，可以让计算机之间进行安全的交互。这个 shell 最常用于命令行界面，但 SSH 也能支持许多其他形式的安全远程交互。例如，它可以用来保护远程 X Windows 会话。一般来说，TCP 端口（port）都可以通过 SSH 转发，这是保护远程系统之间交互的一种有力手段。

SSH 要解决的许多问题和 SSL 遇到的一样，解决方式也往往类似。必须认证远程用户，必须建立共享的加密密钥，必须检查完整性，等等。SSH 通常依靠公钥密码学和证书来认证远程服务器。客户端往往没有自己的证书和私钥，这时可以提供用户 ID 和密码。SSH 还支持不基于证书或密码的其他身份认证方式，比如使用认证服务器（例如 Kerberos）。SSH 支持多种密码算法（既有用于身份认证的，也有用于对称加密的），客户端和服务器之间需要进行某种协商，选出一组合适的算法。

SSH 的一种典型用法，正好可以说明一类常见的网络安全漏洞，这类漏洞叫作**中间人**（**man-in-the-middle**）攻击。这种攻击的情形是：双方以为自己在直接通信，实际上却在不知情的情况下经过一个恶意的第三方通信。这个第三方能看到双方之间传递的所有消息，还能在他们不知情的情况下篡改这些消息或注入新的消息[7]。

设计良好的网络安全工具不会受许多类型的中间人攻击影响，但即使是 SSH 这样好的工具，有时也会遭受这种攻击。如果你经常用 SSH，可能亲身遇到过这样的例子。你第一次用 SSH 登录一台从未登录过的远程机器时，大概还没有它的公钥。你怎么拿到它？往往不是通过证书或其他任何安全手段，而只是请远程站点把公钥发给你。然后你就有了它的公钥，接着就可以安全地认证这台机器，建立加密通信了。可是，如果你第一次尝试登录远程机器时，中间正好有一个中间人呢？那么远程机器把公钥发给你时，中间人可以丢掉那条包含正确公钥的消息，换上一条包含自己公钥的消息。现在，你以为自己有了远程服务器的公钥，实际上拿到的却是中间人的公钥。这意味着中间人可以冒充远程服务器，而你永远也察

[7] 回想一下我们在补充中讨论的 Diffie-Hellman 密钥交换，以及那个美中不足之处。那正是中间人攻击（man-in-the-middle attack）的一个绝佳例子，因为攻击者也许能顶替其中一方，和另一方交换密钥，而且不被发现。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

觉不到。设计 SSH 的人很清楚这个问题，所以只要你这样使用 SSH，就会弹出一条消息，警告你有危险，并询问你是否要冒险继续。民间经验表明，大家看到这条消息时总是回答“是的，继续”，网络安全专业人员也不例外。说到这里，民间经验还表明，所有警告用户可能有不安全操作的消息总是被无视，这应该能让你明白，在自己的系统里加上这类确认消息，能带来多少安全上的好处。

SSH 并不是构建在 SSL 之上的，而是一个独立的实现。因此，两者各有各的缺陷、特性和用途。在 SSH 中发现的安全缺陷，未必会对 SSL 有任何影响，反之亦然。

## 57.8 小结

分布式系统对现代计算至关重要，却很难保证其安全。提供分布式系统安全的基石，往往是确保连接系统各组件的不安全网络不会引入新的安全问题。组件之间发送的消息都经过加密和认证，以保护它们的私密性和完整性，并确保只有预期的用户才能使用分布式服务。SSL/TLS 这样的标准工具，以及通过 X.509 证书分发的公钥，就是用来提供这些安全服务的。远程的人类用户通常用密码来认证。大部分数据的传输都使用对称密码学，因为它比非对称密码学（asymmetric cryptography）便宜。系统参与者在通信开始之前往往并没有共享对称密钥，所以协议的第一步通常是交换一把对称密钥。正如前面几章讨论的，密钥的保密对正确使用密码学至关重要，所以分发密钥的过程需要格外小心。Diffie-Hellman 密钥交换很常用，但它仍然需要身份认证，以确保只有预期的参与者知道这把密钥。

正如前面几章提到的，自己构建密码学解决方案很难，常常会导致安全失败。SSL/TLS、SSH 和 HTTPS 等工具已经着手解决其中许多难题，并取得了很好的进展。我们可以用这些工具构建其他系统，避开从零构建密码学方案的许多陷阱。然而，再好的安全工具，要用对，也得先理解它的用途和局限，所以要想充分发挥它们的作用，就必须深入了解怎样把这些工具集成到自己的系统中。

记住，这些工具只提供有限的安全保证。操作系统在它直接控制的硬件上本地执行操作时能得到的那种保证，它们提供不了。因此，即使正确使用了好的认证和加密工具，系统设计者也最好仔细想清楚：执行远程站点请求的操作，或者向远程站点提供敏感信息，会带来什么后果。操作系统控制的机器边界之外发生的事，总是不确定的，因而总是有风险的。


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

## 参考文献

[H14] "The Heartbleed Bug" by http://heartbleed.com/. *A web page providing a wealth of detail on this particular vulnerability in the OpenSSL implementation of the SSL/TLS protocol.*

[I12] "Information technology – Open Systems Interconnection – The Directory: Public-key and Attribute Certificate Frameworks" ITU-T, 2012. *The ITU-T document describing the format and use of an X.509 certificate. Not recommended for light bedtime reading, but here's where it's all defined.*

[NT94] "Kerberos: An authentication service for computer networks" by B. Clifford Neuman and Theodore Ts'o. IEEE Communications Magazine, Volume 32, No. 9, 1994. *An early paper on Kerberos by its main developers. There have been new versions of the system and many enhancements and bug fixes, but this paper is still a good discussion of the intricacies of the system.*

[P16] "The International PGP Home Page" http://www.pgpi.org, 2016. *A page that links to lots of useful stuff related to PGP, including downloads of free versions of the software, documentation, and discussion of issues related to it.*


© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

# 附录

# A

# 关于虚拟机监视器的对话

**学生**：这下我们被困在附录里了，是吧？

**教授**：是啊，就在你以为情况不可能更糟的时候。

**学生**：那么，我们要聊些什么？

**教授**：一个重获新生的老话题：**虚拟机监视器**（*virtual machine monitor*），也叫 ***hypervisor***。

**学生**：哦，就像 *VMware* 那样？挺酷的，这类软件我以前用过。

**教授**：确实很酷。我们会学到 *VMM* 怎样在系统里再加一层虚拟化（*virtualization*），而且这一层就在操作系统本身的下面！真是既疯狂又神奇。

**学生**：听起来不错。那为什么不把它放在书前面讲虚拟化的那部分？它不是本来就该放在那儿吗？

**教授**：恐怕这事轮不到我们拍板。不过我猜是这样：那部分的内容已经很多了。把 *VMM* 这段小小的题外话挪到附录里，每位老师就可以自己决定讲还是跳过。但我确实认为应该讲，因为如果你能弄懂 *VMM* 是怎么工作的，那你对虚拟化就真的理解得相当透彻了。

**学生**：那好吧，我们开始干活吧！

# B

# 虚拟机监视器

## B.1 引言

多年前，IBM 把昂贵的大型机卖给大型机构，于是出现了一个问题：如果机构想在同一台机器上同时运行不同的操作系统，该怎么办？有些应用程序是在一种操作系统上开发的，另一些则是在别的操作系统上开发的，问题就来了。作为解决办法，IBM 又引入了一个间接层（level of indirection），那就是**虚拟机监视器**（virtual machine monitor，**VMM**，也叫 **hypervisor**）[G74]。

具体来说，监视器位于一个或多个操作系统和硬件之间，让每个运行中的操作系统都以为自己掌控着这台机器。而在幕后，真正掌控硬件的是监视器，它必须让多个运行中的操作系统复用（multiplex）机器的物理资源。实际上，VMM 就是操作系统的操作系统，只不过处在低得多的层次上；操作系统仍然必须认为自己在和物理硬件打交道。因此，**透明**（transparency）是 VMM 的一个主要目标。

这样一来，我们的处境就有些滑稽了：到目前为止，操作系统一直扮演着魔术大师的角色，让毫不知情的应用程序以为自己拥有私有的 CPU 和很大的虚拟内存（virtual memory），同时又在暗中切换应用程序、让它们共享内存。现在，我们要再来一次，只不过这次是在操作系统的下面，而操作系统早已习惯了自己说了算。VMM 怎样才能为运行在它上面的每个操作系统营造出这种假象呢？

关键问题：
如何虚拟化操作系统下面的机器
虚拟机监视器（virtual machine monitor）必须透明地虚拟化（virtualize）操作系统下面的机器；这需要用到哪些技术？

## B.2 动机：为什么需要 VMM

如今，VMM 出于多种原因再次流行起来。服务器整合（server consolidation）就是其中之一。在很多场合，人们把服务运行在不同的机器上，这些机器运行着不同的操作系统（甚至不同版本的操作系统），而每台机器的利用率（utilization）都很低。这种情况下，管理员

可以借助虚拟化把多个操作系统**整合**（consolidate）到更少的硬件平台上，从而降低成本、简化管理。

虚拟化在桌面上也流行起来，因为许多用户希望运行一种操作系统（比如 Linux 或 Mac OS X），同时又能使用另一个平台（比如 Windows）上的原生应用程序。这种**功能**（functionality）上的提升也是一个很好的理由。

另一个原因是测试和调试。开发者在一个主要平台上写代码，但他们常常需要在软件实际部署的许多不同平台上调试和测试。虚拟化让这件事变得很容易，开发者只用一台机器就能运行许多种类、许多版本的操作系统。

这一轮虚拟化复兴在 20 世纪 90 年代中后期真正兴起，带头的是斯坦福大学 Mendel Rosenblum 教授领导的一组研究人员。他们研究的 Disco [B+97] 是一个面向 MIPS 处理器的虚拟机监视器（virtual machine monitor），是让 VMM 复兴的早期尝试，最终也促使这个团队创办了 VMware [V98]，如今它已是虚拟化技术的市场领导者。在本章中，我们将讨论 Disco 背后的主要技术，并借此理解虚拟化是如何工作的。

## B.3 虚拟化 CPU

要在虚拟机监视器（virtual machine monitor）上运行一台**虚拟机**（virtual machine，例如一个操作系统及其应用程序），用到的基本技术是**受限直接执行**（limited direct execution），我们在讨论操作系统如何虚拟化 CPU 时见过这种技术。因此，当我们想在 VMM 上“启动”一个新的操作系统时，只需跳转到它第一条指令的地址，让它开始运行就行了。就这么简单（好吧，差不多）。

假设我们运行在单处理器上，想在两台虚拟机之间（也就是在两个操作系统及其各自的应用程序之间）分时复用（multiplex）处理器。和操作系统在运行中的进程（process）之间切换（即**上下文切换**，context switch）非常相似，虚拟机监视器（virtual machine monitor）必须在运行中的虚拟机之间执行 **machine switch**。也就是说，执行这种切换时，VMM 必须保存一个操作系统的全部机器状态（machine state），包括寄存器、PC，以及上下文切换时不必保存的所有特权硬件状态；再恢复即将运行的虚拟机的机器状态，然后跳转到该虚拟机的 PC，从而完成切换。注意，即将运行的虚拟机的 PC 可能位于操作系统内部，即系统当时正在执行系统调用（system call）；也可能只是位于该操作系统上运行的某个进程中，即一个用户模式（user mode）应用程序。

当运行中的应用程序或操作系统试图执行某种**特权操作**（privileged operation）时，问题就稍微棘手一些了。例如，在采用软件管理 TLB（software-managed TLB）的系统上，操作系统会用专门的特权指令（privileged instruction）把一条转换装入 TLB，然后重新执行那条发生了 TLB 未命中（TLB miss）的指令。在虚拟化环境（virtualized environment）中，不能允许操作系统执行特权指令，否则掌控机器的就是它，而不是它下面的 VMM 了。因此，VMM 必须设法拦截执行特权操作的尝试，从而保住对机器的控制。

VMM 必须介入某些操作，一个简单的例子是：某个操作系统上运行的进程试图发起系统调用。例如，进程可能要对某个文件调用 `open()`，或者调用 `read()` 从中读数据，或者调用 `fork()` 创建新进程。在没有虚拟化的系统中，系统调用是通过一条专门的指令完成的：在 MIPS 上，这是一条**陷阱**（trap）指令；在 x86 上，则是参数为 `0x80` 的 `int` 指令，

 © 2008–23, ARPACI-DUSSEAU

int 就是中断（interrupt）。下面是 FreeBSD 上的 open 库调用 [B00]。回想一下，你的 C 代码先调用 C 库里的函数，C 库再执行相应的汇编指令序列，真正发出陷阱（trap）指令，完成系统调用：

```
open:
    push    dword mode
    push    dword flags
    push    dword path
    mov     eax, 5
    push    eax
    int     80h
```

在基于 UNIX 的系统上，open() 只接受三个参数：int open(char *path, int flags, mode_t mode)。从上面的代码可以看出 open() 库调用是怎么实现的：先把参数（mode、flags、path）压入栈（stack）中，再把 5 压入栈中，然后调用 int 80h，把控制权交给内核。如果你好奇的话，这个 5 是 FreeBSD 中用户模式应用程序和内核为 open() 系统调用事先约定好的编号；不同的系统调用会在调用陷阱（trap）指令 int 之前，把不同的编号压入栈中（放在同一位置），以此发起系统调用[1]。

我们之前讨论过，执行陷阱（trap）指令时通常会发生不少有意思的事。在这个例子里，最重要的一点是：它首先把控制权转交（即修改 PC）给操作系统中一个确定的**陷阱处理程序**（trap handler）。操作系统刚启动时，会把这个例程的地址告诉硬件，这本身也是一项特权操作；这样，此后再发生陷阱（trap）时，硬件就知道从哪里开始运行处理陷阱的代码。在发生陷阱（trap）的同时，硬件还会做另一件至关重要的事：把处理器从**用户模式**（user mode）切换到**内核模式**（kernel mode）。在用户模式下，操作是受限的，试图执行特权操作会引发陷阱（trap），出错的进程多半会被终止；而在内核模式下，机器的全部能力都可以使用，所有特权操作都能执行。因此，在传统环境下——再说一遍，没有虚拟化——控制流程如图 B.1 所示。

在虚拟化平台（virtualized platform）上，事情就有意思一些了。运行在某个操作系统上的应用程序想发起系统调用时，做的事情完全一样：把参数小心地放到栈上或寄存器里，然后执行陷阱（trap）指令。然而，现在掌控机器的是 VMM，安装陷阱处理程序（trap handler）的也是 VMM，所以首先在内核模式下执行的是 VMM 的处理程序。

那么，VMM 该怎么处理这个系统调用呢？VMM 其实并不知道**怎样**处理这个调用；毕竟，它不了解每个运行中的操作系统的细节，因此也就不知道每个调用该做什么。不过，VMM 确实知道操作系统的陷阱处理程序（trap handler）在**哪里**。它之所以知道，是因为操作系统启动时曾试图安装自己的陷阱处理程序（trap handler）；操作系统这么做时，是在尝试执行特权操作，因此陷入了 VMM；那时 VMM 就记下了必要的信息，也就是这个操作系统的陷阱处理程序（trap handler）在内存中的位置。现在，当 VMM 收到某个操作系统上运行的用户进程（user process）发来的陷阱（trap）时，它就知道该怎么做了：跳转到该操作系统的陷阱处理程序（trap handler），让操作系统按照应有的方式处理系统调用。操作

[1] 让事情更乱的是，Intel 的人把几乎任何正常人都会称为陷阱（trap）指令的东西叫作“中断”（interrupt）。正如 Patterson 评价 Intel 指令集时所说：“这是一套只有亲妈才会爱的 ISA。”不过说实话，我们还挺喜欢它的，而我们又不是它的妈。

| 进程 | 硬件 | 操作系统 |
| --- | --- | --- |
| **1.** 执行指令（add、load 等） | | |
| **2.** 系统调用：陷入操作系统 | | |
| | **3.** 切换到内核模式；跳转到陷阱处理程序 | |
| | | **4.** 在内核模式下：处理系统调用；从陷阱返回 |
| | **5.** 切换到用户模式；返回用户代码 | |
| **6.** 继续执行（陷阱之后的 PC 处） | | |

图 B.1：**执行系统调用**

系统处理完后，会执行某种特权指令从陷阱返回（MIPS 上是 **`rett`**，x86 上是 `iret`），这又会弹回到 VMM；VMM 意识到操作系统正试图从陷阱返回（return-from-trap），于是执行一次真正的从陷阱返回，把控制权交还给用户，并让机器回到用户模式。图 B.2 和图 B.3 分别描绘了没有虚拟化和有虚拟化两种情况下的完整过程（为了节省篇幅，我们省略了上面那些具体的硬件操作）。

从图中可以看出，有虚拟化时要做的事情多得多。毫无疑问，由于多了这些来回跳转，虚拟化确实可能拖慢系统调用，从而影响性能。

你可能还注意到，还剩一个问题：操作系统应该运行在什么模式下？它不能运行在内核模式下，否则它就能不受限制地访问硬件了。因此，它必须运行在某种比以前特权低的模式下，既能访问自己的数据结构（data structure），又要防止用户进程（user process）访问这些数据结构。

在 Disco 的工作中，Rosenblum 和同事们利用 MIPS 硬件提供的一种叫管理模式（supervisor mode）的特殊模式，相当巧妙地解决了这个问题。在这种模式下运行时，仍然不能执行特权指令，但可以访问的内存比用户模式多一些；操作系统可以把这些额外的内存

| 进程 | 操作系统 |
| --- | --- |
| **1.** 系统调用：陷入操作系统 | |
| | **2.** 操作系统的陷阱处理程序：解码陷阱，执行相应的系统调用例程；完成后：从陷阱返回 |
| **3.** 继续执行（陷阱之后的 PC 处） | |

图 B.2：**没有虚拟化时的系统调用流程**

| 进程 | 操作系统 | VMM |
| --- | --- | --- |
| **1.** 系统调用：<br>陷入操作系统 | | |
| | | **2.** 进程陷入：<br>调用操作系统的陷阱处理程序<br>（以较低特权运行） |
| | **3.** 操作系统的陷阱处理程序：<br>解码陷阱，<br>执行系统调用；<br>完成后：发出<br>从陷阱返回指令 | |
| | | **4.** 操作系统试图从陷阱返回：<br>执行真正的从陷阱返回 |
| **5.** 继续执行<br>（陷阱之后的 PC 处） | | |

图 B.3：**有虚拟化时的系统调用流程**

用于自己的数据结构，问题就解决了。在没有这种模式的硬件上，就只能让操作系统运行在用户模式下，并借助页表（page table）和 TLB 做内存保护（memory protection），恰当地保护操作系统的数据结构。换句话说，切换到操作系统时，监视器得通过页表保护让操作系统能够访问其数据结构所在的内存；切换回运行中的应用程序时，又得撤销读写内核的权限。

## B.4 虚拟化内存

现在你应该基本了解了处理器是怎样虚拟化的：VMM 像操作系统一样调度不同的虚拟机运行，而在特权级别发生变化时会出现一些有意思的交互。但我们还漏掉了一大块：VMM 是怎样虚拟化内存的?

每个操作系统通常都把物理内存（physical memory）看作一个由页（page）组成的线性数组，并把每一页分配给自己或用户进程（user process）。当然，操作系统本身已经在为它运行的进程虚拟化内存了，让每个进程都以为自己拥有私有的地址空间（address space）。现在我们必须再加一层虚拟化，让多个操作系统共享机器真正的物理内存，而且必须做得透明。

有了这额外的一层虚拟化，“物理”内存本身就成了一种虚拟化，建立在 VMM 所说的**机器内存**（machine memory）之上；机器内存才是系统真正的物理内存。因此，现在多了一个间接层（level of indirection）：每个操作系统通过各进程的页表把虚拟地址（virtual address）映射到物理地址（physical address）；VMM 再通过每个操作系统各自的页表，把得到的物理地址映射到底层的机器地址（machine address）。图 B.4 描绘了这个额外的间接层（level of indirection）。

图中只有一个虚拟地址空间（virtual address space），它有四个页，其中三个是有效的（0、2 和 3）。操作系统用它的页表把这几页分别映射到底层的物理页帧（physical frame）10、3 和 8。在操作系统下面，VMM 又做了一层间接映射，把 PFN 3、8 和 10 分别映射到

 © 2008–23, ARPACI-DUSSEAU

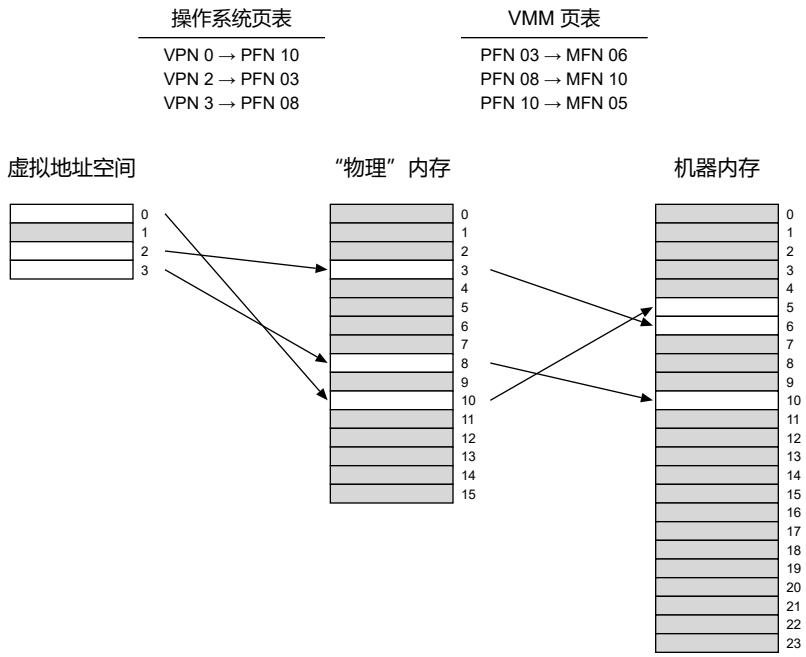



图 B.4：**VMM 的内存虚拟化**

机器页帧（machine frame）6、10 和 5。当然，这张图把事情简化了不少；在真实的系统上，会有 $V$ 个操作系统在运行（$V$ 很可能大于 1），因此有 $V$ 张 VMM 页表；而且，在每个运行中的操作系统 OS$i$ 之上，还运行着若干个进程 $P_i$（$P_i$ 很可能有几十甚至几百个），因此 OS$i$ 中有 $P_i$ 张（每进程一张）页表。

为了更好地理解这是怎么工作的，我们来回顾一下现代分页（paging）系统中**地址转换**（address translation）是怎样进行的。具体来说，我们来讨论在采用软件管理 TLB（software-managed TLB）的系统上，地址转换时会发生什么。假设某个用户进程产生了一个地址（用于取指令，或者显式的加载或存储）；按照定义，进程产生的是**虚拟地址**（virtual address），因为它的地址空间已经被操作系统虚拟化了。你现在已经知道，操作系统的职责就是在硬件的帮助下，把它转换成**物理地址**（physical address），从而能从物理内存中取出所需的内容。

假设虚拟地址空间是 32 位的，页大小（page size）为 4 KB。于是，32 位的地址被切成两部分：20 位的虚拟页号（VPN）和 12 位的偏移量（offset）。操作系统的职责是在硬件 TLB 的帮助下，把 VPN 转换成有效的物理帧号（PFN），从而得到一个完整的物理地址，再发给物理内存去取出正确的数据。通常情况下，我们期望 TLB 在硬件中完成转换，让转换很快。至少在采用软件管理 TLB（software-managed TLB）的系统上，发生 TLB 未命中时，操作系统必须介入，处理这次未命中，如图 B.5 所示。

可以看到，TLB 未命中会引发陷阱（trap），陷入操作系统；操作系统在页表中查找 VPN，把转换装入 TLB，以此处理这次故障。

| 进程 | 操作系统 |
| --- | --- |
| **1.** 从内存加载:<br>TLB 未命中：陷入 | |
| | **2.** 操作系统的 TLB 未命中处理程序:<br>从虚拟地址中取出 VPN;<br>查页表;<br>如果存在且有效:<br>取得 PFN，更新 TLB;<br>从陷阱返回 |
| **3.** 继续执行<br>(引发陷阱的那条指令的 PC 处);<br>重新执行这条指令;<br>这次 TLB 命中 | |

图 B.5：**没有虚拟化时的 TLB 未命中流程**

然而，如果操作系统下面还有一个虚拟机监视器（virtual machine monitor），事情又会变得有意思一些。我们再来看一遍 TLB 未命中的流程（总结见图 B.6）。进程发出一次虚拟内存引用而在 TLB 中未命中时，运行的不是操作系统的 TLB 未命中处理程序（TLB miss handler），而是 VMM 的 TLB 未命中处理程序，因为 VMM 才是这台机器真正拥有特权的主人。不过，通常情况下，VMM 的 TLB 处理程序不知道该怎么处理这次 TLB 未命中，所以它立即跳转到操作系统的 TLB 未命中处理程序（TLB miss handler）；VMM 知道这个处理程序的位置，因为操作系统在“启动”时曾试图安装自己的陷阱处理程序（trap handler）。接着，操作系统的 TLB 未命中处理程序（TLB miss handler）开始运行，在页表中查找相应的 VPN，并试图把 VPN 到 PFN 的映射装入 TLB。然而，这是一项特权操作，因此又会陷入 VMM（当然，只要非特权代码试图做特权的事，VMM 都会收到通知）。这时，VMM 就耍了个花招：它不装入操作系统给出的 VPN 到 PFN 的映射，而是装入它自己想要的 VPN 到 MFN 的映射。这之后，系统最终回到用户级代码，重新执行那条指令，这次 TLB 命中（TLB hit），从数据所在的机器页帧（machine frame）中取出数据。

这一系列动作也暗示了 VMM 必须怎样为每个运行中的操作系统管理物理内存的虚拟化：就像操作系统为每个进程维护一张页表一样，VMM 必须为它运行的每台虚拟机记录物理地址到机器地址（machine address）的映射。VMM 的 TLB 未命中处理程序（TLB miss handler）需要查询这些每台虚拟机一张的页表，才能确定某个“物理”页映射到哪个机器页（machine page），甚至确定它此刻是否在机器内存（machine memory）中，因为 VMM 可能已经把它换出（swap out）到了磁盘上。

最后，你可能已经从这一系列操作中注意到，在虚拟化系统（virtualized system）上，TLB 未命中的代价比非虚拟化系统高出不少。为了降低这个代价，Disco 的设计者在 VMM 层面加了一个“软件 TLB”（software TLB）。这个数据结构背后的思想很简单：操作系统每次试图装入一条虚拟地址到物理地址的映射，VMM 都把它记下来；然后，发生 TLB 未命中时，VMM 先查一查它的软件 TLB，看看以前是否见过这个虚拟地址到物理地址的映射，以及 VMM 想要的虚拟地址到机器地址（machine address）的映射应该是什么。如果 VMM

| 进程 | 操作系统 | 虚拟机监视器 |
| --- | --- | --- |
| **1.** 从内存加载<br>TLB 未命中：陷入 | | |
| | | **2.** VMM 的 TLB 未命中处理程序：<br>调用操作系统的 TLB 处理程序<br>（降低特权） |
| | 3. 操作系统的 TLB 未命中处理程序：<br>从虚拟地址中取出 VPN；<br>查页表；<br>如果存在且有效，<br>取得 PFN，更新 TLB | |
| | | **4.** 陷阱处理程序：<br>非特权代码试图<br>更新 TLB；<br>操作系统试图装入<br>VPN 到 PFN 的映射；<br>改为用 VPN 到 MFN 的映射<br>更新 TLB（特权操作）；<br>跳回操作系统<br>（降低特权） |
| | **5.** 从陷阱返回 | |
| | | **6.** 陷阱处理程序：<br>非特权代码试图<br>从陷阱返回；<br>从陷阱返回 |
| **7.** 继续执行<br>（那条指令的 PC 处）；<br>重新执行这条指令；<br>这次 TLB 命中 | | |

图 B.6：**有虚拟化时的 TLB 未命中流程**

补充：hypervisor 与硬件管理的 TLB

前面的讨论都围绕软件管理的 TLB（software-managed TLB），以及发生未命中时要做的工作。但你可能会好奇：在采用硬件管理 TLB（hardware-managed TLB）的系统上，虚拟机监视器（virtual machine monitor）怎样介入呢？在这类系统上，每次 TLB 未命中时由硬件遍历页表并按需更新 TLB，因此 VMM 没有机会在每次 TLB 未命中时运行，把自己的转换偷偷塞进系统。相反，VMM 必须密切监视操作系统对每张页表所做的修改——在硬件管理的系统上，页表由某种页表基址寄存器（page-table base register）指向——并维护一张**影子页表**（shadow page table），把每个进程的虚拟地址映射到 VMM 想要的机器页（machine page）上 [AA06]。每当操作系统试图装入某个进程的操作系统级页表时，VMM 就装入该进程的影子页表（shadow page table）；于是硬件照常运转，用影子页表（shadow page table）把虚拟地址转换成机器地址（machine address），而操作系统甚至毫无察觉。

补充：半虚拟化

在很多情况下，最好假设没法为了和虚拟机监视器（virtual machine monitor）更好地配合而去修改操作系统（例如，因为你是在一家不友好的竞争对手的操作系统下面运行你的 VMM）。然而，情况并非总是如此。正如我们在页的按需置零（demand zeroing）那个例子中看到的，如果操作系统可以修改，它在 VMM 上就可能运行得更高效。让修改过的操作系统运行在 VMM 上，一般称为**半虚拟化**（para-virtualization）[WSG02]，因为 VMM 提供的虚拟化并不完整，而是部分的，需要修改操作系统才能有效运行。研究表明，一个设计得当的半虚拟化系统（para-virtualized system），只要对操作系统做恰到好处的修改，效率就能做到几乎和没有 VMM 的系统一样高 [BD+03]。

在软件 TLB 中找到了这条转换，就直接把虚拟地址到机器地址（machine address）的映射装入硬件 TLB，从而省去上面控制流程中的那一大堆来回 [B+97]。

## B.5 信息鸿沟

操作系统不太清楚应用程序真正想要什么，因此常常只能制定一些但愿对所有程序都适用的通用策略；同样，VMM 往往也不太清楚操作系统在做什么、想要什么。这种了解上的欠缺，有时被称为 VMM 和操作系统之间的**信息鸿沟**（information gap），可能导致各种低效 [B+97]。例如，操作系统在没有别的事可做时，有时会进入**空闲循环**（idle loop），只是在那里空转，等待下一次中断发生：

```
while (1)
  ; // 空闲循环
```

如果整台机器归操作系统掌管，它也因此知道没有别的事情需要运行，那这样空转是说得通的。然而，如果 VMM 下面运行着两个不同的操作系统，一个处在空闲循环（idle loop）中，另一个正在运行用户进程（user process）做有用的事，那么 VMM 最好能知道前一个操作系统处于空闲状态，这样就能把更多 CPU 时间分给正在做有用工作的那个操作系统。

另一个例子出现在页的按需置零（demand zeroing）上。大多数操作系统在把一个物理页帧（physical frame）映射到某个进程的地址空间之前，都会先把它清零。这样做的原因很简单：安全。如果操作系统把一个进程用过的页不清零就交给另一个进程，就可能发生跨进程的信息泄漏，从而可能泄露敏感信息。遗憾的是，出于同样的原因，VMM 也必须把分给各个操作系统的页清零，所以很多时候一个页会被清零两次：一次是 VMM 把它分配给操作系统时，另一次是操作系统把它分配给进程时。Disco 的作者对这个问题没有什么好办法：他们只是修改了操作系统（IRIX），让它不再清零那些它知道已被底层 VMM 清零过的页 [B+97]。

类似这里所描述的问题还有很多。一种解决办法是让 VMM 利用推断（一种**隐式信息**，implicit information）来克服问题。例如，VMM 发现操作系统切换到了低功耗模式，就能判断它进入了空闲循环（idle loop）。另一种办法见于**半虚拟化**（para-virtualized）系统，它要求修改操作系统。这种更显式的方法虽然更难部署，却可能相当有效。

提示：利用隐式信息

在分层系统中，如果很难修改系统之间的接口，却又需要了解系统另一层的更多信息，隐式信息就会是一个强大的工具。例如，一个基于块（block）的磁盘设备可能想更多地了解它上面的文件系统（file system）是怎样使用它的；同样，一个应用程序可能想知道当前哪些页在文件系统的页缓存（page cache）中，而操作系统没有提供任何 API 来获取这些信息。在这两种情况下，研究人员都已经开发出了强大的推断技术，能够隐式地收集所需的信息，*而不需要*层与层之间有显式的接口 [AD+01, S+03]。这类技术在虚拟机监视器（virtual machine monitor）中相当有用，因为 VMM 想更多地了解运行在它上面的操作系统，又不想在两层之间加一个显式的 API。

## B.6 小结

虚拟化正在复兴。出于多种原因，用户和管理员都希望在同一台机器上同时运行多个操作系统。关键在于，VMM 一般能**透明地**（transparently）提供这种服务；上面的操作系统几乎察觉不到自己其实并没有在控制这台机器的硬件。VMM 做到这一点的关键方法，是扩展受限直接执行（limited direct execution）的概念：把硬件设置好，让 VMM 能介入陷阱（trap）这类关键事件，VMM 就能完全控制机器资源的分配方式，同时又保住操作系统所需要的假象。

你可能已经注意到，操作系统为进程做的事和 VMM 为操作系统做的事有一些相似之处。毕竟它们都在虚拟化硬件，所以会做一些相同的事情。然而，两者有一个关键区别：操作系统的虚拟化提供了许多新的抽象（abstraction）和好用的接口；而 VMM 层面的虚拟化，提供的抽象和硬件一模一样（因而并不怎么好用）。操作系统和 VMM 虽然都在虚拟化硬件，但它们提供的接口完全不同；和操作系统不同，VMM 的本意并不是让硬件更容易使用。

如果你想更多地了解虚拟化，还有许多其他话题可以研究。例如，我们根本没有讨论 I/O 的情况，而在虚拟化平台（virtualized platform）上，I/O 有它自己新的、有意思的问题。我们也没有讨论，在有时称为“宿主型”（hosted）的配置下，虚拟化和你的操作系统“并排”运行时是怎样工作的。如果你感兴趣，可以进一步阅读这两个话题 [SVL01]。我们还没有讨论，运行在一个 VMM 上的一组操作系统用了太多内存时会发生什么。

最后，硬件支持已经改变了平台支持虚拟化的方式。Intel 和 AMD 等公司如今直接支持多出来的这一层虚拟化，使得本章中的许多软件技术都不再必要。也许在一个尚待写出的章节中，我们会更详细地讨论这些机制。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[AA06] "A Comparison of Software and Hardware Techniques for x86 Virtualization" by Keith Adams and Ole Agesen. ASPLOS '06, San Jose, California. *A terrific paper from two VMware engineers about the surprisingly small benefits of having hardware support for virtualization. Also an excellent general discussion about virtualization in VMware, including the crazy binary-translation tricks they have to play in order to virtualize the difficult-to-virtualize x86 platform.*

[AD+01] "Information and Control in Gray-box Systems" by Andrea C. Arpaci-Dusseau and Remzi H. Arpaci-Dusseau. SOSP '01, Banff, Canada. *Our own work on how to infer information and even exert control over the OS from application level, without change to the OS. The best example therein: determining which file blocks are cached using a probabilistic probe-based technique; doing so allows applications to better utilize the cache, by first scheduling work that will result in hits.*

[B00] "FreeBSD Developers' Handbook: Chapter 11 x86 Assembly Language Programming" http://www.freebsd.org/doc/en/books/developers-handbook/. *A nice tutorial on system calls and such in the BSD developers handbook.*

[BD+03] "Xen and the Art of Virtualization" by Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, Andrew Warfield. SOSP '03, Bolton Landing, New York. *The paper that shows that with para-virtualized systems, the overheads of virtualized systems can be made to be incredibly low. So successful was this paper on the Xen virtual machine monitor that it launched a company.*

[B+97] "Disco: Running Commodity Operating Systems on Scalable Multiprocessors" by Edouard Bugnion, Scott Devine, Kinshuk Govil, Mendel Rosenblum. SOSP '97. *The paper that reintroduced the systems community to virtual machine research; well, perhaps this is unfair as Bressoud and Schneider [BS95] also did, but here we began to understand why virtualization was going to come back. What made it even clearer, however, is when this group of excellent researchers started VMware and made some billions of dollars.*

[B+17] "Hardware and Software Support for Virtualization" by Edouard Bugnion, Jason Nieh, Dan Tsafrir. Morgan and Claypool, 2017. *Undoubtedly the best place to get the latest on how virtualization works in modern systems. Unfortunately, you'll have to read a short book to figure it out!*

[BS95] "Hypervisor-based Fault-tolerance" by Thomas C. Bressoud, Fred B. Schneider. SOSP '95. *One the earliest papers to bring back the* **hypervisor**, *which is just another term for a virtual machine monitor. In this work, however, such hypervisors are used to improve system tolerance of hardware faults, which is perhaps less useful than some of the more practical scenarios discussed in this chapter; however, still quite an intriguing paper in its own right.*

[G74] "Survey of Virtual Machine Research" by R.P. Goldberg. IEEE Computer, Volume 7, Number 6. *A terrific survey of a lot of old virtual machine research.*

[SVL01] "Virtualizing I/O Devices on VMware Workstation's Hosted Virtual Machine Monitor" by Jeremy Sugerman, Ganesh Venkitachalam and Beng-Hong Lim. USENIX '01, Boston, Massachusetts. *Provides a good overview of how I/O works in VMware using a hosted architecture which exploits many native OS features to avoid reimplementing them within the VMM.*

[V98] by VMware corporation. Available: http://www.vmware.com/. *This may be the most useless reference in this book, as you can clearly look this up yourself. Anyhow, the company was founded in 1998 and is a leader in the field of virtualization.*

[S+03] "Semantically-Smart Disk Systems" by Muthian Sivathanu, Vijayan Prabhakaran, Florentina I. Popovici, Timothy E. Denehy, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '03, San Francisco, California, March 2003. *Our work again, this time showing how a dumb block-based device can infer much about what the file system above it is doing, such as deleting a file. The technology used therein enables interesting new functionality within a block device, such as secure delete, or more reliable storage.*

[WSG02] "Scale and Performance in the Denali Isolation Kernel" by Andrew Whitaker, Marianne Shaw, and Steven D. Gribble. OSDI '02, Boston, Massachusetts. *The paper that introduces the term para-virtualization. Although one can argue that Bugnion et al. [B+97] introduce the idea of para-virtualization in the Disco paper, Whitaker et al. take it further and show how the idea can be more general than what was thought before.*

# C

# 关于管程的对话

**教授**：又是你啊，嗯？

**学生**：我猜您现在一定累坏了吧，毕竟，呃，您懂的，年纪大了？其实 *50* 岁也不算多老啦。

**教授**：我可没 *50*！其实我刚满 *40*。不过天哪，我想对你这个二十几岁的人来说……

**学生**：……其实是 *19* 岁……

**教授**：（唉）……好好，*19* 岁，随便吧。我想在你看来 *40* 岁和 *50* 岁差不多。但相信我，差远了。至少我那些 *50* 岁的朋友是这么跟我说的。

**学生**：不管怎么说……

**教授**：啊对！我们刚才在聊什么来着？

**学生**：*Monitor*，也就是管程。不过我可不知道 ***monitor*** 是什么，只知道它是我面前这台电脑显示器的一个老派叫法。

**教授**：对，这完全是另一回事。它是一种古老的并发（*concurrency*）原语，设计出来是为了把加锁（*locking*）自动融入面向对象的程序里。

**学生**：那为什么不把它放到讲并发的那部分？

**教授**：嗯，这本书大部分讲的是 *C* 编程和 *POSIX* 线程库（*thread library*），而那里面没有管程（*monitor*），这是一个原因。不过出于一些历史原因，至少该把这方面的内容收进来，所以就放在这儿了，我想。

**学生**：啊，历史。那是你们这些老人家的事，对吧？

**教授**：（瞪眼）

**学生**：哎，别那么较真嘛。我开玩笑的！

**教授**：我真等不及你参加期末考试了……

# D

# 管程（已过时）

在并发编程（concurrent programming）开始变得重要的那段时间，面向对象编程（object-oriented programming）也在流行起来。毫不奇怪，人们开始考虑怎样把同步（synchronization）融入一种更结构化的编程环境。

由此出现的一种方法就是**管程**（monitor）。管程（monitor）最早由 Per Brinch Hansen 描述 [BH73]，后来又经 Tony Hoare 完善 [H74]，它背后的思想相当简单。来看下面这个假想的管程（monitor），它用 C++ 的记法写成：

注意：这是一个“假想的”类，因为 C++ 并不支持管程（monitor），所以 `monitor` 关键字并不存在。不过，Java 确实支持管程（monitor），用的是所谓的**同步**（synchronized）方法。下面我们既会看看怎样在 C/C++ 中做出和管程（monitor）很像的东西，也会看看怎样使用 Java 的同步方法。

在这个例子中，你可能注意到了我们的老朋友：账户，以及往余额里存钱和取钱的例程。你可能还注意到，这些都是**临界区**（critical section）；如果多个线程（thread）同时调用它们，就会出现竞态条件（race condition，也叫竞争条件），可能得到错误的结果。

不过，在管程类（monitor class）里你不会遇到麻烦，因为管程保证**任一时刻只有一个线程能在管程中活动**。因此，上面的例子是一段完全安全、能正常工作的代码；多个线程可以调用 deposit() 或 withdraw()，并且可以放心：互斥（mutual exclusion）是有保证的。

管程（monitor）是怎么做到这一点的？很简单：用一把锁（lock）。每当某个线程试图调用管程（monitor）的例程时，它都会隐式地尝试获取管程锁（monitor lock）。如果成功，

```
monitor class account {
  private:
    int balance = 0;

  public:
    void deposit(int amount) {
        balance = balance + amount;
    }
    void withdraw(int amount) {
        balance = balance - amount;
    }
};
```

图 D.1：**一个假想的管程类**

```
class account {
  private:
    int balance = 0;
    pthread_mutex_t monitor;

  public:
    void deposit(int amount) {
        pthread_mutex_lock(&monitor);
        balance = balance + amount;
        pthread_mutex_unlock(&monitor);
    }
    void withdraw(int amount) {
        pthread_mutex_lock(&monitor);
        balance = balance - amount;
        pthread_mutex_unlock(&monitor);
    }
};
```

图 D.2：**一个表现得像管程的 C++ 类**

它就能调用这个例程，运行方法里的代码；如果失败，它就会阻塞，直到管程（monitor）里的那个线程把手头的事做完。因此，如果我们写一个下面这样的 C++ 类，就能实现和上面的管程类（monitor class）完全相同的目标：

从这个例子可以看出，管程（monitor）并没有自动替你做太多事。说到底，它只是获取一把锁、再释放它。这样做，我们就满足了管程（monitor）的要求：正如所期望的，任一时刻只有一个线程能在 deposit() 或 withdraw() 中活动。

## D.1 何必用管程?

你可能会好奇，为什么要发明管程（monitor），而不直接显式加锁（locking）。那时面向对象编程刚刚兴起，所以人们想把并发编程（concurrent programming）中的一些关键概念，优雅地融入面向对象的一些基本做法中。仅此而已。

## D.2 除了自动加锁，还有别的好处吗?

言归正传。我们在讨论信号量（semaphore）时已经知道，光有锁还不够；例如，为了实现生产者/消费者（producer/consumer）方案，我们之前用信号量来做两件事：一是在等待某个条件发生变化时让线程睡眠，例如生产者等待缓冲区（buffer）被清空；二是在某个条件发生变化后唤醒线程，例如消费者发信号（signal）表示它确实已经清空了一个缓冲区。

管程（monitor）通过一种叫**条件变量**（condition variable）的显式结构来支持这种功能。我们来看一下生产者/消费者方案，这次用管程（monitor）和条件变量来写。

在这个管程类（monitor class）中，有两个例程：produce() 和 consume()。生产者线程（producer thread）会反复调用 produce()，把数据放进有界缓冲区（bounded buffer）；而消费者会反复调用 consume()。这个例子是 Hoare 解法 [H74] 的现代改写版。

 © 2008–23, ARPACI-DUSSEAU

```
monitor class BoundedBuffer {
  private:
    int buffer[MAX];
    int fill, use;
    int fullEntries = 0;
    cond_t empty;
    cond_t full;

  public:
    void produce(int element) {
        if (fullEntries == MAX)        // P0 行
            wait(&empty);              // P1 行
        buffer[fill] = element;        // P2 行
        fill = (fill + 1) % MAX;       // P3 行
        fullEntries++;                 // P4 行
        signal(&full);                 // P5 行
    }

    int consume() {
        if (fullEntries == 0)          // C0 行
            wait(&full);               // C1 行
        int tmp = buffer[use];         // C2 行
        use = (use + 1) % MAX;         // C3 行
        fullEntries--;                 // C4 行
        signal(&empty);                // C5 行
        return tmp;                    // C6 行
    }
}
```

图 D.3：**用管程和 Hoare 语义实现的生产者/消费者**

你应该注意到，这段代码和前一章基于信号量的方案有一些相似之处。一个主要区别在于，条件变量必须和一个显式的**状态变量**（state variable）配合使用；在这里，整数 `fullEntries` 的值决定了生产者或消费者是否必须等待。相比之下，信号量内部有一个数值，起的就是这个作用。因此，条件变量必须和某种外部状态值配对，才能达到同样的目的。

不过，这段代码最重要的是它用了两个条件变量 `empty` 和 `full`，以及操作它们的 `wait()` 和 `signal()` 调用。这些操作做的正是你所想的事：`wait()` 让调用它的线程在给定的条件上阻塞；`signal()` 唤醒一个正在该条件上等待的线程。

然而，这些调用的工作方式有一些微妙之处；要理解这段代码为什么能工作，理解这些调用的语义至关重要。在操作系统研究者所说的 **Hoare 语义**（Hoare semantics，没错，这个名字取得有点不太走运）下，`signal()` 会立即唤醒一个等待的线程并让它运行；于是，运行中的线程隐式持有的管程锁（monitor lock）会立即转交给被唤醒的线程，后者一直运行到阻塞或退出管程（monitor）为止。注意，等待的线程可能不止一个；`signal()` 只唤醒并运行其中一个，其余的必须等待后续的信号。

 © 2008–23, ARPACI-DUSSEAU

老话：理论与实践
这句老话是：“理论上，理论和实践没有区别；但实践上，有。”当然，只有搞实践的人才会这么告诉你；搞理论的人肯定能证明这句话不成立。

举个简单的例子有助于更好地理解这段代码。假设有两个线程，一个是生产者，另一个是消费者。消费者先运行，调用 `consume()`，却发现 `fullEntries = 0`（C0），因为缓冲区里还什么都没有。于是它调用 `wait(&full)`（C1），等待某个缓冲区被填满。接着生产者运行，发现自己不用等待（P0），把一个元素放进缓冲区（P2），把 fill 索引（P3）和 `fullEntries` 计数（P4）都加一，然后调用 `signal(&full)`（P5）。在 Hoare 语义（Hoare semantics）下，生产者发出信号后不会继续运行；相反，信号会立即把控制权转交给等待的消费者，消费者从 `wait()` 返回（C1），立即消费生产者刚生产的元素（C2），以此类推。只有在消费者返回之后，生产者才能再次运行，从 `produce()` 例程返回。

## D.3 理论与实践相遇之处

上面的解法出自 Tony Hoare，`signal()` 和 `wait()` 的确切语义也是他给出的。他是一位理论家，显然也很聪明；毕竟，快速排序（quicksort）就是他发明的 [H61]。然而，事实证明，这种发信号和等待的语义对于真正的实现来说并不理想。正如那句老话所说：理论上，理论和实践没有区别；但实践上，有。

几年后，施乐帕洛阿尔托研究中心（Xerox PARC）的 Butler Lampson 和 David Redell 在开发一种叫 **Mesa** 的并发语言（concurrent language），并决定用管程（monitor）作为它的基本并发原语（concurrency primitive）[LR80]。他们是知名的系统研究者，很快就发现，Hoare 语义（Hoare semantics）虽然更便于证明，却很难在真实系统中实现（原因很多，多到这里恐怕讲不完）。

具体来说，为了做出一个能工作的管程（monitor）实现，Lampson 和 Redell 决定以一种细微但关键的方式改变 signal() 的含义。signal() 例程现在只被看作一个**提示**（hint）[L83]；它会把一个等待的线程从阻塞（blocked）状态移到可运行（runnable）状态，但不会立即运行它。相反，发信号的线程会一直保有控制权，直到它退出管程（monitor）并被取消调度（descheduled）。

## D.4 糟了，有竞态

有了这种新的 **Mesa 语义**（Mesa semantics），我们再来审视一遍上面的代码。再设想一个消费者（消费者 1）进入管程（monitor），发现缓冲区是空的，于是等待（C1）。现在生产者来了，填满缓冲区，然后发信号表示有一个缓冲区已被填满，这就把等待的消费者从 full 条件变量上的阻塞状态移到了就绪（ready）状态。生产者继续运行了一会儿，最终让出（yield）CPU。

但是，休斯敦，我们有麻烦了。你看出来了吗？假设另一个消费者（消费者 2）此时调用了 consume() 例程；它会发现缓冲区是满的，于是把它消费掉，然后返回，同时把 fullEntries

 © 2008–23, ARPACI-DUSSEAU

| 生产者 | 消费者 1 | 消费者 2 |
| --- | --- | --- |
| | C0 (fullEnt=0) | |
| | C1 (Con1：阻塞) | |
| P0 (fullEnt=0) | | |
| P2 | | |
| P3 | | |
| P4 (fullEnt=1) | | |
| P5 (Con1：就绪) | | |
| | | C0 (fullEnt=1) |
| | | C2 |
| | | C3 |
| | | C4 (fullEnt=0) |
| | | C5 |
| | | C6 |
| | C2 (取用缓冲区，fullEnt=0！) | |

图 D.4：**这段代码为什么在 Hoare 语义下不能工作**

设为 0。现在看出问题了吗？问题来了。我们的老朋友消费者 1 终于得以运行，它从 wait() 返回，以为缓冲区是满的（C1……）；不幸的是，情况已经不是这样了，因为消费者 2 抢先溜了进来，在消费者 1 有机会消费之前就把缓冲区消费掉了。因此，这段代码不能正常工作，因为从生产者调用 signal() 到消费者 1 从 wait() 返回的这段时间里，条件发生了变化。下面的时间线说明了这个问题：

幸运的是，从 Hoare 语义（Hoare semantics）换到 Mesa 语义（Mesa semantics），程序员只需做一点小改动，就能得到能工作的方案。具体来说，线程被唤醒后，应该重新检查它所等待的条件；因为 signal() 只是一个提示（hint），条件有可能已经变了（甚至变了好几次），所以在等待的线程运行时，条件未必处于期望的状态。在我们的例子中，有两行代码必须修改，即 P0 行和 C0 行：

终究不算太难。正因为这样实现起来很容易，如今凡是用条件变量来发信号和等待的系统，几乎都采用 Mesa 语义（Mesa semantics）。所以，如果这门课的其他内容你一样都没记住，那就记住这一条：**被唤醒后一定要重新检查条件！**说得更简单些，检查条件时要**用 while 循环**，不要用 if 语句。注意，这样做总是正确的，即使你碰巧运行在采用 Hoare 语义（Hoare semantics）的系统上也一样；那种情况下，你只是白白多检查了一次条件而已。

## D.5 稍微看看内部实现

为了更好地理解 Mesa 语义（Mesa semantics）为什么更容易实现，我们来多了解一点 Mesa 管程（Mesa monitor）的实现。在他们的工作 [LR80] 中，Lampson 和 Redell 描述了三种不同的队列（queue），一个线程在某一时刻可能处在其中之一：**就绪**（ready）队列、**管程锁**（monitor lock）队列，以及**条件变量**（condition variable）队列。注意，一个程序可能有多个管程类（monitor class），也可能有多个条件变量实例；每个这样的实例都有一个队列。

```
public:
  void produce(int element) {
    while (fullEntries == MAX) // P0 行（IF 改成了 WHILE）
      wait(&empty);            // P1 行
    buffer[fill] = element;    // P2 行
    fill = (fill + 1) % MAX;   // P3 行
    fullEntries++;             // P4 行
    signal(&full);             // P5 行
  }

  int consume() {
    while (fullEntries == 0)   // C0 行（IF 改成了 WHILE）
      wait(&full);             // C1 行
    int tmp = buffer[use];     // C2 行
    use = (use + 1) % MAX;     // C3 行
    fullEntries--;             // C4 行
    signal(&empty);            // C5 行
    return tmp;                // C6 行
  }
```

图 D.5：**用管程和 Mesa 语义实现的生产者/消费者**

对于单个有界缓冲区管程（bounded buffer monitor），我们需要考虑四个队列：就绪队列（ready queue）、一个管程队列（monitor queue），以及两个条件变量队列（condition variable queue），一个对应 full 条件，一个对应 empty 条件。为了更好地理解线程库（thread

| t | Con1 | Con2 | Prod | Mon | Empty | Full | FE | 说明 |
|---|---|---|---|---|---|---|---|---|
| 0 | C0 | | | | | | 0 | |
| 1 | C1 | | | | | Con1 | 0 | Con1 在 full 上等待 |
| 2 | <上下文切换> | | | | | Con1 | 0 | 切换：Con1 到 Prod |
| 3 | | | P0 | | | Con1 | 0 | |
| 4 | | | P2 | | | Con1 | 0 | Prod 不用等待（FE=0） |
| 5 | | | P3 | | | Con1 | 0 | |
| 6 | | | P4 | | | Con1 | 1 | Prod 更新 fullEntries |
| 7 | | | P5 | | | | 1 | Prod 发信号：Con1 现在就绪 |
| 8 | <上下文切换> | | | | | | 1 | 切换：Prod 到 Con2 |
| 9 | | C0 | | | | | 1 | 切换到 Con2 |
| 10 | | C2 | | | | | 1 | Con2 不用等待（FE=1） |
| 11 | | C3 | | | | | 1 | |
| 12 | | C4 | | | | | 0 | Con2 修改 fullEntries |
| 13 | | C5 | | | | | 0 | Con2 在 empty 上发信号（没有等待者） |
| 14 | | C6 | | | | | 0 | Con2 完成 |
| 15 | <上下文切换> | | | | | | 0 | 切换：Con2 到 Con1 |
| 16 | C0 | | | | | | 0 | 重新检查 fullEntries：0! |
| 17 | C1 | | | | | Con1 | 0 | 再次在 full 上等待 |

图 D.6：**生产者/消费者运行期间各队列的跟踪**

 © 2008–23, ARPACI-DUSSEAU

library）是怎样管理这些队列的，我们来展示在生产者/消费者的例子中，线程是怎样在这些队列之间转移的。

在这个例子中，我们一步步走过这样一种情况：某个消费者被唤醒后，却发现没有东西可消费。来看下面的时间线。左边是两个消费者（Con1 和 Con2）和一个生产者（Prod），以及它们各自正在执行哪一行代码；右边是这个例子中我们跟踪的四个队列的状态：可运行进程（process）的就绪队列、叫作 Monitor 的管程锁队列（monitor lock queue），以及 empty 和 full 两个条件变量队列（condition variable queue）。我们还跟踪了时间（t）、正在运行的线程——就绪队列中正在运行的那个线程两边加方括号——以及 fullEntries 的值（FE）。

从时间线可以看出，消费者 2（Con2）抢先溜了进来，消费了可用的数据（t=9..14），而消费者 1（Con1）从 t=1 起就一直在等待 full 条件被发信号，却没能抢先消费。不过，Con1 确实被生产者的信号唤醒了（t=7），因此它会再次运行，尽管等它运行时缓冲区已经空了。如果 Con1 不重新检查状态变量（state variable）fullEntries（t=16），它就会在没有数据可消费时错误地去消费数据。因此，正是这种自然的实现方式，把我们引向了 Mesa 语义（Mesa semantics），而不是 Hoare 语义（Hoare semantics）。

## D.6 管程的其他用法

在关于 Mesa 的论文中，Lampson 和 Redell 还指出了几处需要另一种发信号方式的地方。例如，考虑图 D.7 中这个内存分配器（memory allocator）。

这个例子省略了许多细节，好让我们把注意力集中在唤醒和发信号的条件上。结果发现，上面的发信号/等待代码并不完全正确；你看出原因了吗？

设想有两个线程调用 allocate。第一个调用 allocate(20)，第二个调用 allocate(10)。此时没有可用内存，于是两个线程都调用 wait() 并阻塞。过了一段时间，另一个线程调用 free(p,

```
monitor class allocator {
    int available; // 还有多少内存可用?
    cond_t c;

    void *allocate(int size) {
        while (size > available)
            wait(&c);
        available -= size;
        // 然后做分配器该做的其他事
        // 并返回一块内存
    }

    void free(void *pointer, int size) {
        // 释放一些内存
        available += size;
        signal(&c);
    }
};
```

图 D.7：**一个简单的内存分配器**

15)，释放了 15 字节内存，然后发信号说明它释放了内存。不幸的是，它唤醒的是等待 20 字节的那个线程；那个线程重新检查条件，发现只有 15 字节可用，于是又调用 wait()。而本来能用上这 15 字节的线程，也就是调用 allocate(10) 的那个，却没有被唤醒。

Lampson 和 Redell 对这个问题提出了一个简单的解决办法。他们不用只唤醒一个等待线程的 signal()，而是改用唤醒所有等待线程的 **broadcast()**，也就是广播（broadcast）。这样，所有线程都会被唤醒，在上面的例子中，等待 10 字节的线程会发现有 15 字节可用，从而分配成功。

在 Mesa 语义（Mesa semantics）下，使用 broadcast() 总是正确的，因为所有线程被唤醒后本来就应该重新检查自己关心的条件。不过，这样做可能带来性能问题，所以只应在需要时才用。在这个例子中，一次 broadcast() 可能唤醒几百个等待的线程，结果只有一个能成功继续，其余的又立即阻塞；这个问题有时被称为**惊群**（thundering herd），由于额外发生的那些上下文切换（context switch），代价很高。

## D.7 用管程实现信号量

你大概能看出管程（monitor）和信号量有很多相似之处。毫不奇怪，可以用其中一个实现另一个。下面我们展示怎样用管程（monitor）实现一个信号量类（semaphore class），见图 D.8。

可以看到，wait() 只是等待信号量的值大于 0，然后把它减一；而 post() 把值加一，如果有线程在等待，就唤醒其中一个。就这么简单。

要把这个类当作二值信号量（binary semaphore）、也就是当作锁来用，只需把信号量初始化为 1，然后在临界区前后放上成对的 wait()/post()。这样，我们就说明了管程（monitor）可以用来实现信号量。

```
monitor class Semaphore {
    int s; // 信号量的值

    Semaphore(int value) {
        s = value;
    }

    void wait() {
        while (s <= 0)
            wait();
        s--;
    }

    void post() {
        s++;
        signal();
    }
};
```

图 D.8：**用管程实现信号量**

## D.8 现实世界中的管程

前面已经说过，我们用的是“假想的”管程（monitor）；C++ 并没有这样的概念。下面我们展示怎样写一个类似管程（monitor）的 C++ 类，以及 Java 怎样用同步方法达到类似的目的。

### 某种 C++ 管程

下面是用 C++ 和锁、条件变量写成的生产者/消费者代码（图 D.9）。从这个代码示例可以看出，这个能工作的 C++ 类和前面那个假想的管程（monitor）代码区别不大。当然，一个明显的区别是显式地使用了一把叫“monitor”的锁。更微妙的是改用了 POSIX 标准的 `pthread_cond_signal()` 和 `pthread_cond_wait()` 调用。尤其要注意，调用 `pthread_cond_wait()` 时，还要传入等待时所持有的那把锁。`pthread_cond_wait()` 内部需要这把锁，因为这个线程进入睡眠时必须释放它，返回给调用者之前又必须重新获取它；这和管程（monitor）中的行为一样，只不过这次用的是显式的锁。

### Java 管程

有意思的是，Java 的设计者决定采用管程（monitor），因为他们认为这是向语言中加入同步原语（synchronization primitive）的一种优雅方式。要使用管程（monitor），只需在你想当作管程的方法（或一组方法）上加上关键字 **synchronized**（下面是 Sun 自己文档网站上的一个例子 [S12a, S12b]）：

这段代码做的正是你所想的事：提供一个线程安全（thread safe）的计数器。由于任一时刻只允许一个线程进入管程（monitor），也就只有一个线程能更新“c”的值，从而避免了竞态条件。

### Java 与单个条件变量

在最初版本的 Java 中，每个同步类（synchronized class）还附带一个条件变量。要使用它，可以调用 **wait()** 或 **notify()**（有时用 notify 这个词代替 signal，但意思是一样的）。奇怪的是，在这个最初的实现中，没有办法拥有两个（或更多）条件变量。你可能已经注意到，在生产者/消费者方案中，我们总是用两个：一个用来通知某个缓冲区已被清空，另一个用来通知某个缓冲区已被填满。

为了理解只提供一个条件变量的局限，我们来设想只用一个条件变量的生产者/消费者方案。假设两个消费者先运行，都卡在等待上。然后，一个生产者运行，填满一个缓冲区，唤醒一个消费者，接着又想再填一个，却发现缓冲区已满（MAX=1）。于是，我们有一个生产者在等待空缓冲区，一个消费者在等待满缓冲区，还有一个之前在等待、因为已被唤醒而即将运行的消费者。

接着，那个消费者运行，消费了缓冲区。可是，它调用 notify() 时，只会唤醒一个在该条件上等待的线程。由于只有一个条件变量，这个消费者可能唤醒的是那个等待的**消费者**，而不是那个等待的生产者。因此，这个方案行不通。

 © 2008–23, ARPACI-DUSSEAU

```
class BoundedBuffer {
  private:
    int buffer[MAX];
    int fill, use;
    int fullEntries;
    pthread_mutex_t monitor; // 管程锁
    pthread_cond_t empty;
    pthread_cond_t full;

  public:
    BoundedBuffer() {
        use = fill = fullEntries = 0;
    }
    void produce(int element) {
        pthread_mutex_lock(&monitor);
        while (fullEntries == MAX)
            pthread_cond_wait(&empty, &monitor);
        buffer[fill] = element;
        fill = (fill + 1) % MAX;
        fullEntries++;
        pthread_cond_signal(&full);
        pthread_mutex_unlock(&monitor);
    }

    int consume() {
        pthread_mutex_lock(&monitor);
        while (fullEntries == 0)
            pthread_cond_wait(&full, &monitor);
        int tmp = buffer[use];
        use = (use + 1) % MAX;
        fullEntries--;
        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&monitor);
        return tmp;
    }
}
```

图 D.9：**用“管程”实现的 C++ 生产者/消费者**

要补救这个问题，同样可以用广播的办法。在 Java 中，可以调用 **notifyAll()** 唤醒所有等待的线程。在这种情况下，消费者会唤醒一个生产者和一个消费者，但那个消费者会发现 fullEntries 等于 0，于是又回去睡眠，而生产者则会继续运行。和往常一样，唤醒所有等待者可能导致惊群（thundering herd）问题。

由于这个缺陷，Java 后来加入了一个显式的 Condition 类，从而能更高效地解决这个问题和其他类似的并发（concurrency）问题。

```
public class SynchronizedCounter {
    private int c = 0;
    public synchronized void increment() {
        c++;
    }
    public synchronized void decrement() {
        c--;
    }
    public synchronized int value() {
        return c;
    }
}
```

图 D.10：**一个带同步方法的简单 Java 类**

## D.9 小结

我们介绍了管程（monitor），这是 Brinch Hansen 以及后来的 Hoare 在 20 世纪 70 年代初提出的一种结构化概念。在管程（monitor）内部运行时，线程隐式地持有一把管程锁（monitor lock），从而阻止其他线程进入管程，这样就很容易构造出互斥。

我们还介绍了显式的条件变量，它让线程能够 signal() 和 wait()，和我们在前一章信号量中看到的很像。signal() 和 wait() 的语义至关重要；由于所有现代系统都实现的是 **Mesa** 语义（Mesa semantics），所以为了正确执行，线程必须重新检查让它进入睡眠的那个条件。因此，signal() 只是一个**提示**（hint），表明有东西发生了变化；确保条件适合继续执行，是被唤醒线程自己的责任。

最后，由于 C++ 不支持管程（monitor），我们看到了怎样用显式的 pthread 锁和条件变量来模拟管程。我们还看到了 Java 怎样用它的 synchronized 例程支持管程（monitor），以及在这样的环境中只提供单个条件变量的一些局限。

 © 2008–23, ARPACI-DUSSEAU

## 参考文献

[BH73] “Operating System Principles” Per Brinch Hansen, Prentice-Hall, 1973. Available: http://portal.acm.org/citation.cfm?id=540365 *One of the first books on operating systems; certainly ahead of its time. Introduced monitors as a concurrency primitive.*

[H74] “Monitors: An Operating System Structuring Concept” C.A.R. Hoare. CACM, Volume 17:10, pages 549–557, October 1974. *An early reference to monitors; however, Brinch Hansen probably was the true inventor.*

[H61] “Quicksort: Algorithm 64” C.A.R. Hoare. CACM, Volume 4:7, July 1961. *The famous quicksort algorithm.*

[LR80] “Experience with Processes and Monitors in Mesa” B.W. Lampson and D.R. Redell. CACM, Volume 23:2, pages 105–117, February 1980. *An early and important paper highlighting the differences between theory and practice.*

[L83] “Hints for Computer Systems Design” Butler Lampson. ACM Operating Systems Review, 15:5, October 1983. *Lampson, a famous systems researcher, loved using hints in the design of computer systems. A hint is something that is often correct but can be wrong; in this use, a signal() is telling a waiting thread that it changed the condition that the waiter was waiting on, but not to trust that the condition will be in the desired state when the waiting thread wakes up. In this paper about hints for designing systems, one of Lampson's general hints is that you should use hints. It is not as confusing as it sounds.*

[S12a] “Synchronized Methods” Sun documentation. http://java.sun.com/docs/books/tutorial/essential/concurrency/syncmeth.html

[S12b] “Condition Interface” Sun documentation. http://java.sun.com/j2se/1.5.0/docs/api/java/util/concurrent/locks/Condition.html

 © 2008–23, ARPACI-DUSSEAU

# E

# 关于实验的对话

**学生**：这是我们最后一次对话了吗？

**教授**：但愿是！你知道吗，你越来越烦人了！

**学生**：是啊，我也很喜欢我们的谈话。这次讲什么？

**教授**：讲的是你学习这些内容时应该做的项目；你知道的，就是真正动手编程，做点实实在在的工作，而不是没完没了地聊天和读书。这才是真正的学习方法！

**学生**：听起来很重要。您怎么不早点告诉我？

**教授**：嗯，希望用这本书的人确实早就看过这部分，而且整个课程期间都在看。要是没看，他们可就错过了很重要的东西。

**学生**：看来是这样。那这些项目都是什么样的？

**教授**：嗯，项目有两类。第一类你可以叫作**系统编程**（*systems programming*）项目，在运行 *Linux* 的机器上、在 *C* 编程环境中完成。这类编程非常值得掌握，因为等你走上社会，很可能就得亲自动手写这类代码。

**学生**：第二类项目是什么？

**教授**：第二类是在一个真正的内核里面做的。这是 *MIT* 开发的一个很酷的小型教学内核，叫作 ***xv6***。它是把一个老版本的 *UNIX*“移植”到 *Intel x86* 上得来的，相当精巧！做这些项目时，你不是像系统编程那样写和内核打交道的代码，而是真正去重写内核本身的一部分！

**学生**：听起来很好玩！那一个学期里我们该做些什么？您知道，一天就那么几个小时，而且您这些教授好像总是忘了，我们学生要上四五门课，不是只上您这一门！

**教授**：嗯，这方面很灵活。有些课只做系统编程，因为它非常实用。有些课只做 *xv6*，因为这能让你真正看清操作系统是怎么工作的。还有一些课，你可能也猜到了，两者混着做：先做一些系统编程，最后再做 *xv6*。这完全取决于具体某门课的教授。

**学生**：（叹气）看来教授掌握着一切啊……

**教授**：哪有！不过，他们能行使的那一点点控制权，确实是这份工作的乐趣之一。你知道，布置什么作业是件大事，没有哪个教授会掉以轻心。

**学生**：嗯，听您这么说我就放心了。我想我们该看看这些项目都是什么了……

**教授**：好。还有一件事：如果你对系统编程感兴趣，这里还有一份关于 *UNIX* 和 *C* 编程环境的小教程。

**学生**：听起来有用得简直不像真的。

**教授**：嗯，去看看吧。你知道，课程有时候本来就该讲点有用的东西嘛！

# F

# 实验：教程

这是一份非常简短的文档，帮你熟悉 UNIX 系统上 C 编程环境的基础知识。它不全面，也不特别详细，但应该足以让你入门。

关于编程，有几条一般性的建议：如果你想成为编程高手，光掌握一门语言的语法是不够的。具体来说，你应该**了解你的工具**、**了解你的库**、**了解你的文档**。和 C 编译相关的工具有 **gcc**、**gdb**，也许还有 **ld**。你还可以用到大量的库例程，好在很多功能都包含在 **libc** 里，所有 C 程序默认都会链接它，你只需要包含正确的头文件就行。最后，知道怎样找到你需要的库例程是一项值得掌握的技能，例如学会查找和阅读手册页（manual page）。后面我们会更详细地逐一介绍这些内容。

和人生中（几乎）所有值得做的事一样，要在这些方面成为高手需要时间。花点时间事先多了解一些工具和环境，绝对是值得的。

## F.1 一个简单的 C 程序

我们从一个简单的 C 程序开始，比如说把它保存在文件“hw.c”里。和 Java 不同，文件名和文件内容之间不一定有关联；所以，用常识给文件起个合适的名字就行。

第一行指定要包含的文件，这里是 `stdio.h`，它给出了许多常用输入/输出（input/output）例程的“原型”，我们关心的是其中的 `printf()`。使用 `#include` 指令，就是告诉 C 预处理器（`cpp`）找到某个文件（例如 `stdio.h`），并把它直接插入到代码中 `#include` 所在的位置。默认情况下，`cpp` 会到 `/usr/include/` 目录（directory）下去找这个文件。

接下来指定 `main()` 例程的签名（signature）：它返回一个整数（`int`），调用时会传入两个参数：一个整数 `argc`，它是命令行参数的个数；一个字符指针数组（`argv`），其中每个指针指向命令行中的一个词，最后一个是空指针（null pointer）。下面还会进一步介绍指针和数组。

然后，程序只是打印字符串“hello, world”，并让输出换到下一行，这要归功于 `printf()` 调用末尾的反斜杠加“n”。之后，程序返回一个值并结束，这个值会传回给执行该程序的 shell。脚本或终端前的用户可以检查这个值（在 csh 和 tcsh 中，它保存在 `status` 变量里），看看程序是正常退出还是出错退出。

```
/* 头文件放在这上面 */
/* 注意，C 的注释用斜杠和星号括起来，
   可以跨行 */
// 两个斜杠也可以（而且可能更好）
#include <stdio.h>

// main 返回一个整数
int main(int argc, char *argv[]) {
    /* printf 是我们的输出函数；
       默认写到标准输出 */
    /* printf 会返回一个整数，但我们忽略它 */
    printf("hello, world\n");

    /* 返回 0，表示一切顺利 */
    return(0);
}
```

## F.2 编译和执行

现在我们来学习怎样编译这个程序。注意，我们用 `gcc` 作例子，不过在有些平台上，你也可以用另一个（平台自带的）编译器 `cc`。

在 shell 提示符（prompt）下，只需输入：

```
prompt> gcc hw.c
```

`gcc` 其实并不是编译器，而是一个叫作“编译器驱动程序”（compiler driver）的程序，它负责协调编译的许多步骤，通常有四五步。首先，`gcc` 会执行 C 预处理器 `cpp`，处理某些指令（比如 `#define` 和 `#include`）。`cpp` 只是一个源码到源码的翻译器，所以它的产物仍然是源代码（即一个 C 文件）。然后真正的编译开始，通常由一个叫 `cc1` 的命令完成，它把源码级的 C 代码转换成和本机相关的底层汇编代码。接着会执行汇编器 `as`，生成目标代码（机器真正能理解的二进制位之类的东西）。最后，由链接编辑器 `ld`，也就是链接器（linker），把所有东西组合成最终的可执行程序。幸运的是（!），在大多数情况下，你完全可以不管 `gcc` 是怎么工作的，只要带上合适的选项使用它就行。

上面编译的结果是一个可执行文件，（默认）名为 `a.out`。要运行这个程序，只需输入：

```
prompt> ./a.out
```

运行这个程序时，操作系统会正确设置 `argc` 和 `argv`，好让程序按需处理命令行参数。具体来说，`argc` 等于 1，`argv[0]` 是字符串“./a.out”，`argv[1]` 是空指针，表示数组结束。

## F.3 有用的选项

在讲 C 语言之前，我们先指出 `gcc` 的一些有用的编译选项。

```
prompt> gcc -o hw hw.c    # -o：指定可执行文件的名字
prompt> gcc -Wall hw.c    # -Wall：给出好得多的警告
prompt> gcc -g hw.c       # -g：启用 gdb 调试
prompt> gcc -O hw.c       # -O：打开优化
```

当然，你可以按需组合这些选项（例如 `gcc -o hw -g -Wall hw.c`）。在这些选项中，你应该始终使用 `-Wall`，它会针对可能的错误给出大量额外的警告。**不要忽略这些警告！**要把问题改掉，让它们统统消失，心里也就踏实了。

## F.4 链接库

有时，你可能想在程序中使用某个库例程。由于 C 库里有大量例程（每个程序都会自动链接 C 库），通常你只需找到正确的 `#include` 文件。最好的办法是查**手册页**（manual pages），通常就叫 **man 页**（man pages）。

例如，假设你想用 `fork()` 系统调用（system call）[1]。在 shell 提示符下输入 `man fork`，你会得到一段文字，介绍 `fork()` 是怎么工作的。最上面是一小段代码，告诉你程序中需要 `#include` 哪些文件才能通过编译。对于 `fork()`，你需要 `#include` 文件 `unistd.h`，写法如下：

```
#include <unistd.h>
```

不过，有些库例程不在 C 库里，你就得多做一点工作。例如，数学库里有很多有用的例程，比如正弦、余弦、正切等。如果你想在代码中使用例程 `tan()`，同样应该先查手册页。在 Linux 上 `tan` 的手册页最上面，你会看到下面两行：

```
#include <math.h>
...
Link with -lm.
```

第一行你应该已经明白了：你需要 `#include` 数学库，它位于文件系统（file system）中的标准位置（即 `/usr/include/math.h`）。而下一行告诉你的是怎样把程序和数学库“链接”起来。有许多有用的库可以链接，其中很多都放在 `/usr/lib` 中，数学库也确实在这里。

库有两种：静态链接库（以 `.a` 结尾）和动态链接库（以 `.so` 结尾）。静态链接库会直接合并到可执行文件中；也就是说，链接器把库的底层代码插入到你的可执行文件里，生成的二进制目标文件会大得多。动态链接对此做了改进，它只在程序的可执行文件中包含对库的引用；程序运行时，由操作系统的加载程序动态地把库链接进来。这种方法比静态方法更受青睐，因为它节省磁盘空间（不会生成不必要的大可执行文件），还能让应用程序在内存中共享库代码和静态数据。数学库既有静态版本，也有动态版本，静态版本叫 `/usr/lib/libm.a`，动态版本叫 `/usr/lib/libm.so`。

无论如何，要链接数学库，你都需要把这个库告诉链接编辑器，办法是给 `gcc` 加上合适的选项。

```
prompt> gcc -o hw hw.c -Wall -lm
```

`-lXXX` 选项告诉链接器去找 `libXXX.so` 或 `libXXX.a`，大概就是按这个顺序找。如果出于某种原因，你坚持要用静态库而不是动态库，还有另一个选项可以用，看看你能不能自己查出来是哪个。人们有时更喜欢库的静态版本，因为使用动态库会带来一点性能开销。

[1] 注意，`fork()` 是一个系统调用，而不只是一个库例程。不过，C 库为所有系统调用都提供了 C 包装函数，每个包装函数只是陷入操作系统而已。

最后再说一点：如果你想让编译器到通常位置以外的路径下找头文件，或者想让它链接你指定的库，可以用编译器选项 `-I/foo/bar` 让它到目录 `/foo/bar` 中查找头文件，用 `-L/foo/bar` 选项让它到 `/foo/bar` 目录中查找库。用这种方式指定的一个常见目录是“.”（读作“dot”），它是 UNIX 中当前目录的简写。注意，`-I` 选项应该放在编译命令行上，`-L` 选项应该放在链接命令行上。

## F.5 分别编译

程序大到一定程度后，你可能想把它拆成几个文件，分别编译，然后再链接到一起。例如，假设你有两个文件 `hw.c` 和 `helper.c`，想分别编译它们，再链接在一起。

```
# 我们用 -Wall 打开警告，用 -O 打开优化
prompt> gcc -Wall -O -c hw.c
prompt> gcc -Wall -O -c helper.c
prompt> gcc -o hw hw.o helper.o -lm
```

`-c` 选项告诉编译器只生成目标文件，这里就是 `hw.o` 和 `helper.o` 两个文件。这些文件不是可执行文件，只是各个源文件中代码的机器级表示。要把目标文件组合成可执行文件，必须把它们“链接”到一起；这由第三行 `gcc -o hw hw.o helper.o` 完成。这时，`gcc` 发现指定的输入文件不是源文件（`.c`），而是目标文件（`.o`），于是直接跳到最后一步，调用链接编辑器 `ld` 把它们链接成一个可执行文件。由于它的作用，这一行通常叫作“链接行”，`-lm` 之类和链接相关的命令就应该写在这里。同理，`-Wall` 和 `-O` 之类的选项只在编译阶段需要，因此不必写在链接行上，只写在编译行上就行。

当然，你也可以把所有 C 源文件都写在一行里交给 `gcc`：

```
prompt> gcc -Wall -O -o hw hw.c helper.c
```

但这样系统就要重新编译每个源文件，可能很费时间。分别编译各个文件，就可以只重新编译你编辑时改动过的那些文件，从而节省时间、提高效率。这个过程最好交给另一个程序 `make` 来管理，下面就介绍它。

## F.6 Makefile

程序 `make` 可以让构建过程的大部分工作自动化，因此对任何正经的程序（和程序员）来说都是极其重要的工具。我们来看一个简单的例子，它保存在名为 `Makefile` 的文件中。

```
hw: hw.o helper.o
    gcc -o hw hw.o helper.o -lm

hw.o: hw.c
    gcc -O -Wall -c hw.c

helper.o: helper.c
    gcc -O -Wall -c helper.c

clean:
    rm -f hw.o helper.o hw
```

现在，要构建程序，只需在命令行输入 `make`。它（默认）会查找 `Makefile` 或 `makefile`，并把它作为输入（你也可以用某个选项指定别的 makefile，读读手册页就知道是哪个）。GNU 版本的 make，即 `gmake`，功能比传统的 make 更完整，所以下面的讨论主要以它为准（不过这两个说法我们会混用）。下面的内容大多来自 `gmake` 的 info 页面；怎样找到这些页面，请看后面的“文档”一节。另外请注意：在 Linux 系统上，`gmake` 和 `make` 是同一个东西。

Makefile 建立在规则（rule）之上，规则用来决定需要做什么。规则的一般形式如下：

```
target: prerequisite1 prerequisite2 ...
    command1
    command2
    ...
```

**目标**（target）通常是由命令生成的文件的名字，例如可执行文件或目标文件。目标也可以是要执行的某个动作的名字，比如我们例子中的“clean”。

**依赖**（prerequisite）是用来生成目标的输入文件。一个目标往往依赖好几个文件。例如，要构建可执行文件 `hw`，需要先构建两个目标文件：`hw.o` 和 `helper.o`。

最后，**命令**（command）是 `make` 要执行的动作。一条规则可以有多条命令，每条占一行。**注意：**每个命令行的开头都必须放一个制表符（tab）！如果你只放了空格，`make` 就会打印一条莫名其妙的错误信息，然后退出。

命令通常出现在带依赖的规则里，用来在任何一个依赖发生变化时生成目标文件。不过，为目标指定命令的规则不一定要有依赖。例如，我们例子中和目标“clean”对应、包含删除命令的那条规则就没有依赖。

回到我们的例子，执行 `make` 时，它大致是这样工作的：首先，它来到目标 `hw`，发现要构建它，必须先有两个依赖 `hw.o` 和 `helper.o`。也就是说，`hw` 依赖这两个目标文件。接着，`make` 会逐个检查这些目标。检查 `hw.o` 时，它会发现 `hw.o` 依赖 `hw.c`。关键就在这里：如果 `hw.c` 的修改时间比 `hw.o` 的生成时间更晚，`make` 就知道 `hw.o` 已经过时，应该重新生成；这时它会执行命令行 `gcc -O -Wall -c hw.c`，生成 `hw.o`。因此，如果你编译的是一个大程序，`make` 会根据依赖关系知道哪些目标文件需要重新生成，并且只做重新生成可执行文件所必需的工作。还要注意，如果 `hw.o` 根本不存在，也会生成它。

接下来，helper.o 也可能按照上面同样的标准被重新生成或新建。两个目标文件都生成之后，make 就可以执行生成最终可执行文件的命令了，于是它回过头去执行：gcc -o hw hw.o helper.o -lm。

到目前为止，我们一直忽略了 makefile 中的 clean 目标。要用它，必须显式地要求。输入

```
prompt> make clean
```

就会执行这条命令行上的命令。因为 clean 目标没有依赖，所以输入 make clean 总会执行其中的命令。这里 clean 目标用来删除目标文件和可执行文件，如果你想从头重新构建整个程序，这就非常方便。

现在你可能会想：“嗯，看起来还行，但这些 makefile 写起来也太麻烦了！”要是它们总得这么写，你说得没错。好在有很多简便写法能让 make 用起来更省事。例如，下面这个 makefile 功能相同，但用起来要方便一些：

```
# 在这里指定所有源文件
SRCS = hw.c helper.c

# 在这里指定目标（可执行文件的名字）
TARG = hw

# 指定编译器、编译选项和需要的库
CC   = gcc
OPTS = -Wall -O
LIBS = -lm

# 把源文件列表中的 .c 文件换成 .o 文件
OBJS = $(SRCS:.c=.o)

# all 其实不是必需的，但可以用来生成目标
all: $(TARG)

# 这条规则生成目标可执行文件
$(TARG): $(OBJS)
    $(CC) -o $(TARG) $(OBJS) $(LIBS)

# 这是 .o 文件的通用规则
%.o: %.c
    $(CC) $(OPTS) -c $< -o $@

# 最后是 clean 规则
clean:
    rm -f $(OBJS) $(TARG)
```

虽然我们不打算细讲 make 的语法，但你可以看到，这个 makefile 能让你的生活轻松一些。例如，要往构建中添加新的源文件，只需把它们加到 makefile 开头的 SRCS 变量里。改一下 TARG 那一行，就能轻松更换可执行文件的名字；编译器、选项和库的设置也都很容易修改。

关于 make 最后再说一句：弄清一个目标有哪些依赖并不总是那么简单，在大型复杂的程序中尤其如此。不出所料，有另一个工具可以帮忙，叫作 makedepend。自己去了解一下，看看能不能把它用到 makefile 里。

## F.7 调试

终于，你建好了一个不错的构建环境，程序也正确编译了，却可能发现程序有 bug。解决问题的一种办法是冥思苦想，这种方法有时能成功，但往往不行。问题在于缺乏信息：你根本不知道程序内部到底发生了什么，所以也就想不出它为什么不按预期运行。好在有工具可以帮忙：GNU 调试器 gdb。

我们来看下面这段有 bug 的代码，它保存在文件 buggy.c 中，编译成了可执行文件 buggy。

```
#include <stdio.h>

struct Data {
    int x;
};

int
main(int argc, char *argv[])
{
    struct Data *p = NULL;
    printf("%d\n", p->x);
}
```

在这个例子中，主程序在变量 p 为 NULL 时对它解引用，这会导致一个段错误（segmentation fault）。当然，这个问题一眼就能看出来、很容易修好，但在更复杂的程序里，找到这样的问题并不总是那么容易。

为了准备调试，要重新编译程序，并确保在每个编译行上都加上 -g 选项。这会在可执行文件中加入额外的调试信息，在调试时很有用。另外，不要打开优化（-O）；虽然这样也许能行，但也可能在调试时把你弄糊涂。

用 -g 重新编译之后，就可以使用调试器了。在命令提示符下这样启动 gdb：

```
prompt> gdb buggy
```

这样你就进入了和调试器的交互会话。注意，你还可以用调试器检查程序异常运行时生成的“core”文件，或者附加到一个正在运行的程序上；要了解更多，请阅读文档。

进去之后，你可能会看到类似这样的内容：

```
prompt> gdb buggy
GNU gdb ...
Copyright 2008 Free Software Foundation, Inc.
(gdb)
```

你可能想做的第一件事就是运行程序。只需在 gdb 命令提示符下输入 run。在这个例子中，你可能会看到：

```
(gdb) run
Starting program: buggy

Program received signal SIGSEGV, Segmentation fault.
0x8048433 in main (argc=1, argv=0xbffff844) at buggy.c:19
19        printf("%d\n", p->x);
```

从这个例子可以看到，gdb 立刻指出了问题出在哪里：在我们试图解引用 p 的那一行产生了“段错误”。这其实就是说，我们访问了不该访问的内存。这时，机灵的程序员看一眼代码就会说：“啊哈！一定是 p 没有指向任何有效的东西，所以不该对它解引用！”然后就去把问题改掉。

不过，如果你不知道发生了什么，可能会想检查某个变量。gdb 允许你在调试会话中交互地做这件事。

```
(gdb) print p
1 = (Data *) 0x0
```

用 print 原语检查 p，可以看到它是一个指向 Data 类型结构体的指针，而且当前被设为 NULL（也就是零，或者这里显示的十六进制零“0x0”）。

最后，你还可以在程序中设置断点，让调试器在某个例程处把程序停下来。停下来之后，再逐步执行程序（一次一行），看看发生了什么，往往很有用。

```
(gdb) break main
Breakpoint 1 at 0x8048426: file buggy.c, line 17.
(gdb) run
Starting program: /homes/hacker/buggy
Breakpoint 1, main (argc=1, argv=0xbffff844) at buggy.c:17
17        struct Data *p = NULL;
(gdb) next
19        printf("%d\n", p->x);
(gdb) next

Program received signal SIGSEGV, Segmentation fault.
0x8048433 in main (argc=1, argv=0xbffff844) at buggy.c:19
19        printf("%d\n", p->x);
```

在上面的例子中，我们在 main() 例程处设了一个断点；因此运行程序时，调试器几乎立刻就在 main 处停了下来。这时，我们在例子中发出了一条“next”命令，它会执行下一条源码级语句。“next”和“step”都是推进程序执行的有用方式，详情请阅读文档[2]。

这里的讨论远远没有把 gdb 讲透；它是一个功能丰富、用法灵活的调试工具，功能远比这点篇幅里能介绍的多。请在你大把的空闲时间里自己多读一读，成为调试高手吧。

## F.8 文档

要深入了解所有这些东西，你得做两件事：一是用这些工具，二是自己多读相关资料。要进一步了解 gcc、gmake 和 gdb，一个办法是读它们的手册页：在命令提示符下输入 man

[2]特别是，用 gdb 调试时，你可以使用交互式的“help”命令。

gcc、man gmake 或 man gdb。你还可以用 man -k 按关键词搜索手册页，不过效果不一定总是理想；这时上网搜一下可能是更好的办法。

手册页有一个麻烦的地方：如果有不止一个叫 XXX 的东西，输入 man XXX 不一定能得到你想要的那个。例如，如果你要找 kill() 系统调用的手册页，只在提示符下输入 man kill，就会得到错误的手册页，因为还有一个叫 kill 的命令行程序。手册页分成若干**节**（section），默认情况下，man 会返回它找到的编号最小的那一节中的手册页，在这个例子中就是第 1 节。注意，看一下页面顶部就能知道你拿到的是哪个手册页：如果看到 kill(2)，就说明你找对了，是第 2 节的手册页，系统调用就在这一节。输入 man man 可以进一步了解手册页的各节分别存放什么内容。另外，man -a kill 可以依次查看所有名叫“kill”的手册页。

手册页能帮你查到很多东西。特别是，你常常需要查一个库调用该传哪些参数，或者使用某个库调用需要包含哪些头文件。这些都应该能在手册页里找到。例如，查一下 open() 系统调用，你会看到：

```
SYNOPSIS
    #include <sys/types.h>
    #include <sys/stat.h>
    #include <fcntl.h>

    int open(const char *path, int oflag,
             /* mode_t mode */...);
```

这告诉你，要使用 open 调用，需要包含头文件 sys/types.h、sys/stat.h 和 fcntl.h。它还告诉你要传给 open 的参数：一个名为 path 的字符串、一个整数标志 oflag，以及一个可选参数，用来指定文件的模式。如果使用这个调用还需要链接某些库，这里也会告诉你。

手册页需要花点功夫才能用好。它们通常分成若干标准部分。正文部分会说明怎样传入不同的参数，让函数表现出不同的行为。

一个特别有用的部分叫作 RETURN VALUES，它告诉你函数在成功或失败时会返回什么。还是看 open() 的手册页：

```
RETURN VALUES
    成功完成时，open() 函数打开文件，并返回一个非负整数，
    即编号最小的未使用的文件描述符。否则返回 -1，并设置
    errno 来指明错误，而且不会创建或修改任何文件。
```

因此，检查 open 的返回值，就能知道打开是否成功。如果没成功，open 和许多标准库（standard library）例程都会把一个叫 errno 的全局变量设为某个值，告诉你出了什么错。详情请看手册页的 ERRORS 部分。

你可能还想做另一件事：查找手册页里没有给出的某个结构体的定义。例如，gettimeofday() 的手册页有如下概要：

```
SYNOPSIS
    #include <sys/time.h>

    int gettimeofday(struct timeval *restrict tp,
                     void *restrict tzp);
```

从这一页可以看到，时间放在一个 `timeval` 类型的结构体里，但手册页不一定会告诉你这个结构体有哪些字段！（这次它告诉了你，但你不一定总有这么好的运气。）所以，你可能得自己去找。所有的头文件都在 `/usr/include` 目录下，因此可以用 `grep` 这样的工具来找。例如，你可以输入：

```
prompt> grep 'struct timeval' /usr/include/sys/*.h
```

这样就能在 `/usr/include/sys` 下所有以 `.h` 结尾的文件中查找这个结构体的定义。遗憾的是，这样做不一定总能找到，因为这个头文件可能又包含了放在别处的其他头文件。

更好的办法是用你手头的一个工具：编译器。写一个包含头文件 `time.h` 的程序，比如叫 `main.c`。然后不编译它，而是用编译器调用预处理器。预处理器会处理文件中的所有指令，比如 `#define` 命令和 `#include` 命令。为此，输入 `gcc -E main.c`。结果是一个 C 文件，里面有所有需要的结构体和原型，包括 `timeval` 结构体的定义。

要查这些东西，也许还有更好的办法：上网搜。不知道的东西就应该去搜一搜。只要查一查就能学到这么多东西，真是太神奇了！

### info 页面

在查找文档时，**info 页面**（info pages）也非常有用，它为许多 GNU 工具提供了详细得多的文档。你可以运行 `info` 程序来访问 info 页面，也可以在黑客们偏爱的编辑器 emacs 里执行 `Meta-x info` 来访问。像 `gcc` 这样的程序有上百个选项，其中一些知道了会出乎意料地有用。`gmake` 还有许多功能，能改善你的构建环境。最后，`gdb` 是一个相当精巧的调试器。读一读手册页和 info 页面，试试以前没用过的功能，成为善用编程工具的高手吧。

## F.9 推荐阅读

除了手册页和 info 页面，还有很多有用的书。注意，这些信息很多都能在网上免费找到；不过，书的形式有时似乎更便于学习。另外，对你感兴趣的主题，一定要找找 O'Reilly 出版的书；它们几乎总是质量上乘。

- "The C Programming Language"，作者 Brian Kernighan 和 Dennis Ritchie。这是那本必备的权威 C 语言书。
- "Managing Projects with make"，作者 Andrew Oram 和 Steve Talbott。一本讲 `make` 的书，篇幅不长，内容中肯。
- "Debugging with GDB: The GNU Source-Level Debugger"，作者 Richard M. Stallman 和 Roland H. Pesch。一本讲怎样使用 GDB 的小书。

- “Advanced Programming in the UNIX Environment”，作者 W. Richard Stevens 和 Steve Rago。Stevens 写过几本非常好的书，这本是 UNIX 黑客的必读书。他还写了一套讲 TCP/IP 和套接字（socket）编程的好书。
- “Expert C Programming”，作者 Peter Van der Linden。上面关于编译器等方面的很多有用技巧，都是直接从这本书里“偷”来的。读读它！虽然有点过时，但这是一本很棒的书，能让你大开眼界。

 © 2008–23, ARPACI-DUSSEAU

# G

# 实验：系统项目

**注意：**这些项目正在陆续添加到 `https://github.com/remzi-arpacidusseau/ostep-projects`，那里有项目说明和一个简单的测试框架。如果你感兴趣，一定要去看看。

本章给出一些系统项目的思路。在一个 15 周的学期里，我们通常做六七个项目，也就是大约每两周一个。前几个项目通常由学生独自完成，最后几个则两人一组完成。

每个学期，项目都沿用这个大纲；不过我们会改变一些细节，让项目保持新鲜感，也让跨学期“共享”代码更难一些（当然，没有人会干这种事！）。我们还会用 Moss 工具 [M94] 来查找这类“共享”。

至于评分，我们试过许多不同的方法，各有利弊。现场演示很有意思，但很花时间。自动化测试脚本没那么耗时，但要让它们仔细测试那些有意思的边界情况，得花很大的心思。关于这些项目的更多细节，请查看本书的网页；如果你想要自动化测试脚本，我们很乐意分享。

## G.1 入门项目

第一个项目是系统编程（systems programming）入门。典型的作业是在不同的约束条件下，写一个 `sort` 工具的某种变体。例如，对文本数据排序、对二进制数据排序，以及其他类似的项目都很合适。要完成这个项目，你需要熟悉一些系统调用（以及它们返回的错误码），用到几个简单的数据结构（data structure），除此之外没有太多别的。

## G.2 UNIX Shell

在这个项目中，学生要实现一个 UNIX shell 的变体。学生会学到进程（process）管理，还会弄明白管道（pipe）和重定向（redirection）这些神秘的东西实际上是怎么工作的。变体包括一些不寻常的功能，比如一个同时用 gzip 压缩输出的重定向符号。另一个变体是批处理（batch processing）模式，允许用户把几个请求攒成一批再执行，执行时也许还会采用不同的调度策略（scheduling policy）。

## G.3 内存分配库

本项目要构建一个替代的内存分配库（类似 `malloc()` 和 `free()`，只是名字不同），借此探索怎样管理一块内存。学生会学到怎样用 `mmap()` 获取一块匿名内存，再深入学习指针，以便构建一个简单的（也可能更复杂的）空闲链表（free list）来管理这块空间。变体包括：最佳适应（best fit）/ 最坏适应（worst fit）、伙伴分配及其他各种分配器（allocator）。

## G.4 并发入门

这个项目介绍用 POSIX 线程（thread）进行并发（concurrency）编程。构建一些简单的线程安全（thread safe）的库：链表（linked list）、散列表（hash table），以及一些更复杂的数据结构，都是给实际代码加锁（locking）的好练习。测量粗粒度（coarse-grained）和细粒度（fine-grained）两种做法的性能。变体只是换成不同的（也许更复杂的）数据结构。

## G.5 并发 Web 服务器

这个项目探索并发在实际应用中的用法。学生拿一个简单的 Web 服务器（或自己写一个），给它加上一个线程池（thread pool），以便并发处理请求。线程池的大小应该固定，并用一个生产者/消费者（producer/consumer）有界缓冲区（bounded buffer）把请求从主线程（main thread）传给固定数量的工作线程（worker thread）。学生会学到怎样用线程、锁和条件变量（condition variable）构建真正的服务器。变体包括线程的各种调度策略。

## G.6 文件系统检查程序

这个项目探索磁盘上的数据结构及其一致性。学生要实现一个简单的文件系统检查程序（file system checker）。在 Linux 上可以用 `debugfs` 工具制作真实的文件系统（file system）镜像；遍历这些镜像，确保一切正常。要想更难一些，还可以修复发现的问题。变体侧重于不同类型的问题：指针、链接计数（link count）、间接块的使用等等。

## G.7 文件系统碎片整理程序

这个项目探索磁盘上的数据结构及其对性能的影响。项目应该给学生一些已知有碎片问题的特定文件系统镜像；然后学生遍历镜像，找出没有顺序存放的文件。再写出一个新的、“碎片整理（defragmentation）后”的镜像来解决问题，也许再报告一些统计数据。

## G.8 并发文件服务器

这个项目把并发和文件系统结合起来，甚至还涉及一点网络和分布式系统（distributed system）。学生要实现一个简单的并发文件服务器（concurrent file server）。协议（protocol）应该和 NFS 差不多，支持查找、读、写和获取文件属性。把文件存放在一个磁盘镜像里（这个镜像本身就是一个文件）。变体有很多，可以建议不同的磁盘格式和网络协议。

## 参考文献

[M94] “Moss: A System for Detecting Software Plagiarism” Alex Aiken. Available: http://theory.stanford.edu/~aiken/moss/

 © 2008–23, ARPACI-DUSSEAU

# H

# 实验：xv6 项目

**注意：**这些项目正在陆续添加到 `https://github.com/remzi-arpacidusseau/ostep-projects`，那里有项目说明和一个简单的测试框架。如果你感兴趣，一定要去看看。

本章给出一些和 xv6 内核有关的项目思路。这个内核可以从 MIT 获取，拿来折腾非常好玩；做这些项目还能让课堂上的内容和项目联系得更直接。这些项目（也许最开始的一两个除外）通常两人一组完成，这样盯着内核看这项苦差事就会轻松一点。

## H.1 入门项目

入门项目是给 xv6 添加一个简单的系统调用（system call）。可以有很多变体，比如一个统计已经发生了多少次系统调用的系统调用（每个系统调用一个计数器），或者其他收集信息的调用。学生会学到系统调用实际上是怎样发生的。

## H.2 进程和调度

学生要实现一个比默认的时间片轮转（Round-Robin）更复杂的调度程序（scheduler）。可以有很多变体，比如彩票调度（lottery scheduling）程序或多级反馈队列（Multi-level Feedback Queue）。学生会学到调度程序实际上是怎么工作的，以及上下文切换（context switch）是怎样发生的。另外还有一个小要求：让学生想办法让进程（process）在退出时返回正确的错误码，并能通过 `wait()` 系统调用拿到这个错误码。

## H.3 虚拟内存入门

基本思路是添加一个新的系统调用：给定一个虚拟地址（virtual address），返回转换后的物理地址（或者报告这个地址无效）。这能让学生看到虚拟内存（virtual memory）系统是怎样建立页表（page table）的，又不用做太多费力的工作。另一个变体研究怎样改造 xv6，让对空指针（null pointer）解引用真正引发一个故障。

## H.4 写时复制映射

这个项目给 xv6 添加一种轻量级的 `fork()`，叫作 `vfork()`。这个新调用不是简单地复制映射，而是给共享的页建立写时复制（copy-on-write）映射。访问这样的页时，内核必须创建一份真正的副本，并相应地更新页表。

## H.5 内存映射

另一个虚拟内存项目是添加某种形式的内存映射文件（memory-mapped file）。最简单的做法大概是从可执行文件中惰性调入代码页（lazy page-in）；更完整的做法是实现一个 `mmap()` 系统调用，以及在解引用时从磁盘把页调入所需的全部基础设施。

## H.6 内核线程

这个项目研究怎样给 xv6 添加内核线程（kernel thread）。`clone()` 系统调用的工作方式和 `fork` 很像，但使用同一个地址空间（address space）。学生要想办法实现这样一个调用，从而创建一个真正的内核线程。学生还应该在它之上构建一个小型线程库（thread library），提供简单的锁。

## H.7 高级内核线程

学生要在自己的内核线程之上构建一个完整的线程库，加入不同类型的锁，比如自旋锁（spin lock）和在处理器不可用时睡眠的锁，以及条件变量（condition variable）。所需的内核支持也要一并加上。

## H.8 基于区段的文件系统

这第一个文件系统（file system）项目给基本的文件系统添加一些简单的功能。对于 `EXTENT` 类型的文件，学生要修改 inode，让它存储区段（即指针和长度对），而不是只存指针。这个项目可以作为文件系统的一个比较轻松的入门。

## H.9 快速文件系统

学生要把 xv6 的基本文件系统改造成伯克利快速文件系统（FFS）。学生要实现一个新的 `mkfs` 工具，引入块组（block group）和新的块分配策略，并实现大文件的例外规则。这样能更深入地理解文件系统的基本工作原理。

## H.10 日志文件系统

学生要给 xv6 添加一个最基本的日志层。每次写文件时，日志文件系统（journaling file system）都把所有被弄脏的块攒成一批，把它们待完成的更新记录写到磁盘上的日志里；只

 © 2008–23, ARPACI-DUSSEAU

有在这之后, 才原地修改这些块。学生要引入崩溃点, 并展示文件系统总能恢复到一致状态, 以此证明自己系统的正确性。

## H.11 文件系统检查程序

学生要为 xv6 文件系统实现一个简单的文件系统检查程序（file system checker）。学生会学到是什么让文件系统保持一致，以及到底该怎样检查一致性。

 © 2008–23, ARPACI-DUSSEAU